

Intel® 64 and IA-32 Architectures Software Developer's Manual

Volume 1: Basic Architecture

NOTE: The *Intel® 64 and IA-32 Architectures Software Developer's Manual* consists of ten volumes: *Basic Architecture*, Order Number 253665; *Instruction Set Reference, A-L*, Order Number 253666; *Instruction Set Reference, M-U*, Order Number 253667; *Instruction Set Reference, V*, Order Number 326018; *Instruction Set Reference*, W-Z, Order Number 334569; *System Programming Guide, Part 1*, Order Number 253668; *System Programming Guide, Part 2*, Order Number 253669; *System Programming Guide, Part 3*, Order Number 326019; *System Programming Guide, Part 4*, Order Number 332831; *Model-Specific Registers,* Order Number 335592. Refer to all ten volumes when evaluating your design needs.

Order Number: 253665-093US
September 2026

Notices & Disclaimers

Intel technologies may require enabled hardware, software or service activation.

No product or component can be absolutely secure.

Your costs and results may vary.

You may not use or facilitate the use of this document in connection with any infringement or other legal analysis concerning Intel products described herein. You agree to grant Intel a non-exclusive, royalty-free license to any patent claim thereafter drafted which includes subject matter disclosed herein.

All product plans and roadmaps are subject to change without notice.

The products described may contain design defects or errors known as errata which may cause the product to deviate from published specifications. Current characterized errata are available on request.

Intel disclaims all express and implied warranties, including without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement, as well as any warranty arising from course of performance, course of dealing, or usage in trade.

Code names are used by Intel to identify products, technologies, or services that are in development and not publicly available. These are not "commercial" names and not intended to function as trademarks.

No license (express or implied, by estoppel or otherwise) to any intellectual property rights is granted by this document, with the sole exception that a) you may publish an unmodified copy and b) code, identified as Sample Code in this document is licensed subject to the Zero-Clause BSD open source license (0BSD), https://opensource.org/licenses/0BSD. You may create software implementations based on this document and in compliance with the foregoing that are intended to execute on the Intel product(s) referenced in this document. No rights are granted to create modifications or derivatives of this document.

© Intel Corporation. Intel, the Intel logo, and other Intel marks are trademarks of Intel Corporation or its subsidiaries. Other names and brands may be claimed as the property of others.

CONTENTS

PAGE

CHAPTER 1
ABOUT THIS MANUAL

1.1 INTEL® 64 AND IA-32 PROCESSORS COVERED IN THIS MANUAL 1-1
1.2 OVERVIEW OF VOLUME 1: BASIC ARCHITECTURE 1-5
1.3 NOTATIONAL CONVENTIONS 1-6
1.3.1 Bit and Byte Order 1-6
1.3.2 Reserved Bits and Software Compatibility 1-7
1.3.2.1 Instruction Operands 1-7
1.3.3 Hexadecimal and Binary Numbers 1-8
1.3.4 Segmented Addressing 1-8
1.3.5 A Syntax for CPUID, CR, and MSR Values 1-8
1.3.6 Exceptions 1-9
1.4 RELATED LITERATURE 1-9

CHAPTER 2
INTEL® 64 AND IA-32 ARCHITECTURES

2.1 BRIEF HISTORY OF INTEL® 64 AND IA-32 ARCHITECTURES 2-1
2.1.1 16-Bit Processors and Segmentation (1978) 2-1
2.1.2 The Intel® 286 Processor (1982) 2-1
2.1.3 The Intel386™ Processor (1985) 2-1
2.1.4 The Intel486™ Processor (1989) 2-1
2.1.5 The Intel® Pentium® Processor (1993) 2-2
2.1.6 The P6 Family of Processors (1995—1999) 2-2
2.1.7 The Intel® Pentium® 4 Processor Family (2000—2006) 2-3
2.1.8 The Intel® Xeon® Processor (2001—2007) 2-3
2.1.9 The Intel® Pentium® M Processor (2003—2006) 2-3
2.1.10 The Intel® Pentium® Processor Extreme Edition (2005) 2-4
2.1.11 The Intel® Core™ Duo and Intel® Core™ Solo Processors (2006—2007) 2-4
2.1.12 The Intel® Xeon® Processor 5100, 5300 Series, and Intel® Core™ 2 Processor Family (2006) 2-4
2.1.13 The Intel® Xeon® Processor 5200, 5400, 7400 Series, and Intel® Core™ 2 Processor Family (2007) 2-4
2.1.14 The Intel Atom® Processor Family (2008) 2-5
2.1.15 The Intel Atom® Processor Family Based on Silvermont Microarchitecture (2013) 2-5
2.1.16 The Intel® Core™ i7 Processor Family (2008) 2-5
2.1.17 The Intel® Xeon® Processor 7500 Series (2010) 2-5
2.1.18 2010 Intel® Core™ Processor Family (2010) 2-6
2.1.19 The Intel® Xeon® Processor 5600 Series (2010) 2-6
2.1.20 The Second Generation Intel® Core™ Processor Family (2011) 2-6
2.1.21 The Third Generation Intel® Core™ Processor Family (2012) 2-6
2.1.22 The Fourth Generation Intel® Core™ Processor Family (2013) 2-7
2.2 MORE ON SPECIFIC ADVANCES 2-7
2.2.1 P6 Family Microarchitecture 2-7
2.2.2 Intel NetBurst® Microarchitecture 2-8
2.2.2.1 The Front End Pipeline 2-9
2.2.2.2 Out-Of-Order Execution Core 2-10
2.2.2.3 Retirement Unit 2-10
2.2.3 Intel® Core™ Microarchitecture 2-10
2.2.3.1 The Front End 2-11
2.2.3.2 Execution Core 2-12
2.2.4 Intel Atom® Microarchitecture 2-12
2.2.5 Nehalem Microarchitecture 2-13
2.2.6 Sandy Bridge Microarchitecture 2-13
2.2.7 SIMD Instructions 2-14
2.2.8 Intel® Hyper-Threading Technology 2-16

PAGE

2.2.8.1 Some Implementation Notes ... 2-17
2.2.9 Multi-Core Technology ... 2-18
2.2.10 Intel® 64 Architecture ... 2-20
2.2.11 Intel® Virtualization Technology (Intel® VT) ... 2-20
2.3 INTEL® 64 AND IA-32 PROCESSOR GENERATIONS ... 2-20
2.4 PLANNED REMOVAL OF INTEL® INSTRUCTION SET ARCHITECTURE AND FEATURES FROM UPCOMING PRODUCTS ... 2-28
2.5 INTEL® INSTRUCTION SET ARCHITECTURE AND FEATURES REMOVED ... 2-28

CHAPTER 3
BASIC EXECUTION ENVIRONMENT

3.1 MODES OF OPERATION ... 3-1
3.1.1 Intel® 64 Architecture ... 3-1
3.2 OVERVIEW OF THE BASIC EXECUTION ENVIRONMENT ... 3-2
3.2.1 64-Bit Mode Execution Environment ... 3-5
3.3 MEMORY ORGANIZATION ... 3-6
3.3.1 IA-32 Memory Models ... 3-7
3.3.2 Paging and Virtual Memory ... 3-8
3.3.3 Memory Organization in 64-Bit Mode ... 3-8
3.3.4 Modes of Operation vs. Memory Model ... 3-9
3.3.5 32-Bit and 16-Bit Address and Operand Sizes ... 3-9
3.3.6 Extended Physical Addressing in Protected Mode ... 3-9
3.3.7 Address Calculations in 64-Bit Mode ... 3-10
3.3.7.1 Canonical Addressing ... 3-10
3.4 BASIC PROGRAM EXECUTION REGISTERS ... 3-10
3.4.1 General-Purpose Registers ... 3-11
3.4.1.1 General-Purpose Registers in 64-Bit Mode ... 3-12
3.4.2 Segment Registers ... 3-13
3.4.2.1 Segment Registers in 64-Bit Mode ... 3-15
3.4.3 EFLAGS Register ... 3-15
3.4.3.1 Status Flags ... 3-16
3.4.3.2 DF Flag ... 3-17
3.4.3.3 System Flags and IOPL Field ... 3-17
3.4.3.4 RFLAGS Register in 64-Bit Mode ... 3-18
3.5 INSTRUCTION POINTER ... 3-18
3.5.1 Instruction Pointer in 64-Bit Mode ... 3-18
3.6 OPERAND-SIZE AND ADDRESS-SIZE ATTRIBUTES ... 3-18
3.6.1 Operand Size and Address Size in 64-Bit Mode ... 3-19
3.7 OPERAND ADDRESSING ... 3-19
3.7.1 Immediate Operands ... 3-20
3.7.2 Register Operands ... 3-20
3.7.2.1 Register Operands in 64-Bit Mode ... 3-20
3.7.3 Memory Operands ... 3-21
3.7.3.1 Memory Operands in 64-Bit Mode ... 3-21
3.7.4 Specifying a Segment Selector ... 3-21
3.7.4.1 Segmentation in 64-Bit Mode ... 3-22
3.7.5 Specifying an Offset ... 3-22
3.7.5.1 Specifying an Offset in 64-Bit Mode ... 3-23
3.7.6 Assembler and Compiler Addressing Modes ... 3-24
3.7.7 I/O Port Addressing ... 3-24

CHAPTER 4
DATA TYPES

4.1 FUNDAMENTAL DATA TYPES ... 4-1
4.1.1 Alignment of Words, Doublewords, Quadwords, and Double Quadwords ... 4-2
4.2 NUMERIC DATA TYPES ... 4-2
4.2.1 Integers ... 4-3
4.2.1.1 Unsigned Integers ... 4-3
4.2.1.2 Signed Integers ... 4-4
4.2.2 Floating-Point Data Types ... 4-4

PAGE

4.2.3 Brain Float16 4-6
4.2.3.1 Numeric Definition 4-6
4.2.3.2 Rounding, Denormal Handling, and FP Exceptions 4-7
4.3 POINTER DATA TYPES 4-7
4.3.1 Pointer Data Types in 64-Bit Mode 4-8
4.4 BIT FIELD DATA TYPE 4-8
4.5 STRING DATA TYPES 4-9
4.6 PACKED SIMD DATA TYPES 4-9
4.6.1 64-Bit SIMD Packed Data Types 4-9
4.6.2 128-Bit Packed SIMD Data Types 4-9
4.7 BCD AND PACKED BCD INTEGERS 4-10
4.8 REAL NUMBERS AND FLOATING-POINT FORMATS 4-12
4.8.1 Real Number System 4-12
4.8.2 Floating-Point Format 4-12
4.8.2.1 Normalized Numbers 4-14
4.8.2.2 Biased Exponent 4-14
4.8.3 Real Number and Non-number Encodings 4-14
4.8.3.1 Signed Zeros 4-15
4.8.3.2 Normalized and Denormalized Finite Numbers 4-15
4.8.3.3 Signed Infinities 4-16
4.8.3.4 NaNs 4-16
4.8.3.5 Operating on SNaNs and QNaNs 4-17
4.8.3.6 Using SNaNs and QNaNs in Applications 4-17
4.8.3.7 QNaN Floating-Point Indefinite 4-18
4.8.3.8 Half Precision Floating-Point Operation 4-18
4.8.4 Rounding 4-18
4.8.4.1 Rounding Control (RC) Fields 4-19
4.8.4.2 Truncation with Intel® SSE, SSE2, and AVX Conversion Instructions 4-19
4.9 OVERVIEW OF FLOATING-POINT EXCEPTIONS 4-20
4.9.1 Floating-Point Exception Conditions 4-21
4.9.1.1 Invalid Operation Exception (#I) 4-21
4.9.1.2 Denormal Operand Exception (#D) 4-21
4.9.1.3 Divide-By-Zero Exception (#Z) 4-22
4.9.1.4 Numeric Overflow Exception (#O) 4-22
4.9.1.5 Numeric Underflow Exception (#U) 4-23
4.9.1.6 Inexact-Result (Precision) Exception (#P) 4-23
4.9.2 Floating-Point Exception Priority 4-24
4.9.3 Typical Actions of a Floating-Point Exception Handler 4-25

CHAPTER 5
INSTRUCTION SET SUMMARY

5.1 GENERAL-PURPOSE INSTRUCTIONS 5-6
5.1.1 Data Transfer Instructions 5-7
5.1.2 Binary Arithmetic Instructions 5-7
5.1.3 Decimal Arithmetic Instructions 5-8
5.1.4 Logical Instructions 5-8
5.1.5 Shift and Rotate Instructions 5-8
5.1.6 Bit and Byte Instructions 5-8
5.1.7 Control Transfer Instructions 5-9
5.1.8 String Instructions 5-10
5.1.9 I/O Instructions 5-10
5.1.10 Enter and Leave Instructions 5-11
5.1.11 Flag Control (EFLAG) Instructions 5-11
5.1.12 Segment Register Instructions 5-11
5.1.13 Miscellaneous Instructions 5-11
5.1.14 User Mode Extended State Save/Restore Instructions 5-12
5.1.15 Random Number Generator Instructions 5-12
5.1.16 BMI1 and BMI2 Instructions 5-12
5.1.16.1 Detection of VEX-Encoded GPR Instructions, LZCNT, TZCNT, and PREFETCHW 5-12

PAGE

5.2 X87 FPU INSTRUCTIONS . . . 5-13
5.2.1 X87 FPU Data Transfer Instructions . . . 5-13
5.2.2 X87 FPU Basic Arithmetic Instructions . . . 5-13
5.2.3 X87 FPU Comparison Instructions . . . 5-14
5.2.4 X87 FPU Transcendental Instructions . . . 5-14
5.2.5 X87 FPU Load Constants Instructions . . . 5-15
5.2.6 X87 FPU Control Instructions . . . 5-15
5.3 X87 FPU AND SIMD STATE MANAGEMENT INSTRUCTIONS . . . 5-15
5.4 MMX INSTRUCTIONS . . . 5-16
5.4.1 MMX Data Transfer Instructions . . . 5-16
5.4.2 MMX Conversion Instructions . . . 5-16
5.4.3 MMX Packed Arithmetic Instructions . . . 5-16
5.4.4 MMX Comparison Instructions . . . 5-17
5.4.5 MMX Logical Instructions . . . 5-17
5.4.6 MMX Shift and Rotate Instructions . . . 5-17
5.4.7 MMX State Management Instructions . . . 5-17
5.5 INTEL® SSE INSTRUCTIONS . . . 5-18
5.5.1 Intel® SSE SIMD Single Precision Floating-Point Instructions . . . 5-18
5.5.1.1 Intel® SSE Data Transfer Instructions . . . 5-18
5.5.1.2 Intel® SSE Packed Arithmetic Instructions . . . 5-18
5.5.1.3 Intel® SSE Comparison Instructions . . . 5-19
5.5.1.4 Intel® SSE Logical Instructions . . . 5-19
5.5.1.5 Intel® SSE Shuffle and Unpack Instructions . . . 5-19
5.5.1.6 Intel® SSE Conversion Instructions . . . 5-19
5.5.2 Intel® SSE MXCSR State Management Instructions . . . 5-20
5.5.3 Intel® SSE 64-Bit SIMD Integer Instructions . . . 5-20
5.5.4 Intel® SSE Cacheability Control, Prefetch, and Instruction Ordering Instructions . . . 5-20
5.6 INTEL® SSE2 INSTRUCTIONS . . . 5-20
5.6.1 Intel® SSE2 Packed and Scalar Double Precision Floating-Point Instructions . . . 5-21
5.6.1.1 Intel® SSE2 Data Movement Instructions . . . 5-21
5.6.1.2 Intel® SSE2 Packed Arithmetic Instructions . . . 5-21
5.6.1.3 Intel® SSE2 Logical Instructions . . . 5-22
5.6.1.4 Intel® SSE2 Compare Instructions . . . 5-22
5.6.1.5 Intel® SSE2 Shuffle and Unpack Instructions . . . 5-22
5.6.1.6 Intel® SSE2 Conversion Instructions . . . 5-22
5.6.2 Intel® SSE2 Packed Single Precision Floating-Point Instructions . . . 5-23
5.6.3 Intel® SSE2 128-Bit SIMD Integer Instructions . . . 5-23
5.6.4 Intel® SSE2 Cacheability Control and Ordering Instructions . . . 5-23
5.7 INTEL® SSE3 INSTRUCTIONS . . . 5-24
5.7.1 Intel® SSE3 x87-FP Integer Conversion Instruction . . . 5-24
5.7.2 Intel® SSE3 Specialized 128-Bit Unaligned Data Load Instruction . . . 5-24
5.7.3 Intel® SSE3 SIMD Floating-Point Packed ADD/SUB Instructions . . . 5-24
5.7.4 Intel® SSE3 SIMD Floating-Point Horizontal ADD/SUB Instructions . . . 5-24
5.7.5 Intel® SSE3 SIMD Floating-Point LOAD/MOVE/DUPLICATE Instructions . . . 5-25
5.7.6 Intel® SSE3 Agent Synchronization Instructions . . . 5-25
5.8 SUPPLEMENTAL STREAMING SIMD EXTENSIONS 3 (SSSE3) INSTRUCTIONS . . . 5-25
5.8.1 Horizontal Addition/Subtraction . . . 5-25
5.8.2 Packed Absolute Values . . . 5-26
5.8.3 Multiply and Add Packed Signed and Unsigned Bytes . . . 5-26
5.8.4 Packed Multiply High with Round and Scale . . . 5-26
5.8.5 Packed Shuffle Bytes . . . 5-26
5.8.6 Packed Sign . . . 5-26
5.8.7 Packed Align Right . . . 5-27
5.9 INTEL® SSE4 INSTRUCTIONS . . . 5-27
5.10 INTEL® SSE4.1 INSTRUCTIONS . . . 5-27
5.10.1 Dword Multiply Instructions . . . 5-27
5.10.2 Floating-Point Dot Product Instructions . . . 5-28
5.10.3 Streaming Load Hint Instruction . . . 5-28
5.10.4 Packed Blending Instructions . . . 5-28

PAGE

5.10.5 Packed Integer MIN/MAX Instructions 5-28
5.10.6 Floating-Point Round Instructions With Selectable Rounding Mode 5-28
5.10.7 Insertion and Extractions from XMM Registers 5-29
5.10.8 Packed Integer Format Conversions 5-29
5.10.9 Improved Sums of Absolute Differences (SAD) for 4-Byte Blocks 5-29
5.10.10 Horizontal Search 5-30
5.10.11 Packed Test 5-30
5.10.12 Packed Qword Equality Comparisons 5-30
5.10.13 Dword Packing With Unsigned Saturation 5-30
5.11 INTEL® SSE4.2 INSTRUCTION SET 5-30
5.11.1 String and Text Processing Instructions 5-30
5.11.2 Packed Comparison SIMD Integer Instruction 5-30
5.12 INTEL® AES-NI AND PCLMULQDQ 5-30
5.13 INTEL® ADVANCED VECTOR EXTENSIONS (INTEL® AVX) 5-31
5.14 16-BIT FLOATING-POINT CONVERSION 5-31
5.15 FUSED-MULTIPLY-ADD (FMA) 5-31
5.16 INTEL® ADVANCED VECTOR EXTENSIONS 2 (INTEL® AVX2) 5-32
5.17 INTEL® TRANSACTIONAL SYNCHRONIZATION EXTENSIONS (INTEL® TSX) 5-32
5.18 INTEL® SHA EXTENSIONS 5-32
5.19 INTEL® ADVANCED VECTOR EXTENSIONS 512 (INTEL® AVX-512) 5-32
5.20 SYSTEM INSTRUCTIONS 5-37
5.21 64-BIT MODE INSTRUCTIONS 5-38
5.22 VIRTUAL-MACHINE EXTENSIONS 5-39
5.23 SAFER MODE EXTENSIONS 5-39
5.24 INTEL® MEMORY PROTECTION EXTENSIONS 5-40
5.25 INTEL® SOFTWARE GUARD EXTENSIONS 5-40
5.26 SHADOW STACK MANAGEMENT INSTRUCTIONS 5-41
5.27 CONTROL TRANSFER TERMINATING INSTRUCTIONS 5-41
5.28 INTEL® AMX INSTRUCTIONS 5-41
5.29 USER INTERRUPT INSTRUCTIONS 5-41
5.30 ENQUEUE STORE INSTRUCTIONS 5-42
5.31 INTEL® ADVANCED VECTOR EXTENSIONS 10 VERSION 1 INSTRUCTIONS 5-42

CHAPTER 6
PROCEDURE CALLS, INTERRUPTS, AND EXCEPTIONS

6.1 PROCEDURE CALL TYPES 6-1
6.2 STACKS 6-1
6.2.1 Setting Up a Stack 6-2
6.2.2 Stack Alignment 6-2
6.2.3 Address-Size Attributes for Stack Accesses 6-3
6.2.4 Procedure Linking Information 6-3
6.2.4.1 Stack-Frame Base Pointer 6-3
6.2.4.2 Return Instruction Pointer 6-3
6.2.5 Stack Behavior in 64-Bit Mode 6-4
6.3 SHADOW STACKS 6-4
6.4 CALLING PROCEDURES USING CALL AND RET 6-4
6.4.1 Near CALL and RET Operation 6-4
6.4.2 Far CALL and RET Operation 6-5
6.4.3 Parameter Passing 6-7
6.4.3.1 Passing Parameters Through the General-Purpose Registers 6-7
6.4.3.2 Passing Parameters on the Stack 6-7
6.4.3.3 Passing Parameters in an Argument List 6-7
6.4.4 Saving Procedure State Information 6-7
6.4.5 Calls to Other Privilege Levels 6-7
6.4.6 CALL and RET Operation Between Privilege Levels 6-9
6.4.7 Branch Functions in 64-Bit Mode 6-12
6.5 INTERRUPTS AND EXCEPTIONS 6-12
6.5.1 Call and Return Operation for Interrupt or Exception Handling Procedures 6-13
6.5.2 Calls to Interrupt or Exception Handler Tasks 6-18

PAGE

6.5.3 Interrupt and Exception Handling in Real-Address Mode ... 6-18
6.5.4 INT n, INTO, INT3, INT1, and BOUND Instructions ... 6-18
6.5.5 Handling Floating-Point Exceptions ... 6-19
6.5.6 Interrupt and Exception Behavior in 64-Bit Mode ... 6-19
6.6 PROCEDURE CALLS FOR BLOCK-STRUCTURED LANGUAGES ... 6-20
6.6.1 ENTER Instruction ... 6-20
6.6.2 LEAVE Instruction ... 6-24

CHAPTER 7
PROGRAMMING WITH
GENERAL-PURPOSE INSTRUCTIONS

7.1 PROGRAMMING ENVIRONMENT FOR GP INSTRUCTIONS ... 7-1
7.2 PROGRAMMING ENVIRONMENT FOR GP INSTRUCTIONS IN 64-BIT MODE ... 7-1
7.3 SUMMARY OF GP INSTRUCTIONS ... 7-2
7.3.1 Data Transfer Instructions ... 7-2
7.3.1.1 General Data Movement Instructions ... 7-3
7.3.1.2 Exchange Instructions ... 7-4
7.3.1.3 Exchange Instructions in 64-Bit Mode ... 7-5
7.3.1.4 Stack Manipulation Instructions ... 7-5
7.3.1.5 Stack Manipulation Instructions in 64-Bit Mode ... 7-7
7.3.1.6 Type Conversion Instructions ... 7-7
7.3.1.7 Type Conversion Instructions in 64-Bit Mode ... 7-8
7.3.2 Binary Arithmetic Instructions ... 7-8
7.3.2.1 Addition and Subtraction Instructions ... 7-8
7.3.2.2 Increment and Decrement Instructions ... 7-8
7.3.2.3 Increment and Decrement Instructions in 64-Bit Mode ... 7-8
7.3.2.4 Comparison and Sign Change Instructions ... 7-8
7.3.2.5 Multiplication and Division Instructions ... 7-9
7.3.3 Decimal Arithmetic Instructions ... 7-9
7.3.3.1 Packed BCD Adjustment Instructions ... 7-9
7.3.3.2 Unpacked BCD Adjustment Instructions ... 7-9
7.3.4 Decimal Arithmetic Instructions in 64-Bit Mode ... 7-10
7.3.5 Logical Instructions ... 7-10
7.3.6 Shift and Rotate Instructions ... 7-10
7.3.6.1 Shift Instructions ... 7-10
7.3.6.2 Double-Shift Instructions ... 7-12
7.3.6.3 Rotate Instructions ... 7-13
7.3.7 Bit and Byte Instructions ... 7-13
7.3.7.1 Bit Test and Modify Instructions ... 7-14
7.3.7.2 Bit Scan Instructions ... 7-14
7.3.7.3 Byte Set on Condition Instructions ... 7-14
7.3.7.4 Test Instruction ... 7-14
7.3.8 Control Transfer Instructions ... 7-14
7.3.8.1 Unconditional Transfer Instructions ... 7-14
7.3.8.2 Conditional Transfer Instructions ... 7-15
7.3.8.3 Control Transfer Instructions in 64-Bit Mode ... 7-17
7.3.8.4 Software Interrupt Instructions ... 7-17
7.3.8.5 Software Interrupt Instructions in 64-Bit Mode and Compatibility Mode ... 7-18
7.3.9 String Operations ... 7-18
7.3.9.1 String Instructions ... 7-18
7.3.9.2 Repeated String Operations ... 7-19
7.3.9.3 Fast-String Operation ... 7-19
7.3.9.4 String Operations in 64-Bit Mode ... 7-20
7.3.10 I/O Instructions ... 7-20
7.3.11 I/O Instructions in 64-Bit Mode ... 7-20
7.3.12 Enter and Leave Instructions ... 7-21
7.3.13 Flag Control (EFLAG) Instructions ... 7-21
7.3.13.1 Carry and Direction Flag Instructions ... 7-21
7.3.13.2 EFLAGS Transfer Instructions ... 7-21

PAGE

7.3.13.3 Interrupt Flag Instructions . . . 7-22
7.3.14 Flag Control (RFLAG) Instructions in 64-Bit Mode . . . 7-22
7.3.15 Segment Register Instructions . . . 7-22
7.3.15.1 Segment-Register Load and Store Instructions . . . 7-22
7.3.15.2 Far Control Transfer Instructions . . . 7-22
7.3.15.3 Software Interrupt Instructions . . . 7-23
7.3.15.4 Load Far Pointer Instructions . . . 7-23
7.3.16 Miscellaneous Instructions . . . 7-23
7.3.16.1 Address Computation Instruction . . . 7-23
7.3.16.2 Table Lookup Instructions . . . 7-23
7.3.16.3 Processor Identification Instruction . . . 7-23
7.3.16.4 No-Operation and Undefined Instructions . . . 7-23
7.3.17 Random Number Generator Instructions . . . 7-24
7.3.17.1 RDRAND . . . 7-24
7.3.17.2 RDSEED . . . 7-24

CHAPTER 8
PROGRAMMING WITH THE X87 FPU

8.1 X87 FPU EXECUTION ENVIRONMENT . . . 8-1
8.1.1 x87 FPU in 64-Bit Mode and Compatibility Mode . . . 8-1
8.1.2 x87 FPU Data Registers . . . 8-1
8.1.2.1 Parameter Passing With the x87 FPU Register Stack . . . 8-3
8.1.3 x87 FPU Status Register . . . 8-4
8.1.3.1 Top of Stack (TOP) Pointer . . . 8-4
8.1.3.2 Condition Code Flags . . . 8-4
8.1.3.3 x87 FPU Floating-Point Exception Flags . . . 8-5
8.1.3.4 Stack Fault Flag . . . 8-6
8.1.4 Branching and Conditional Moves on Condition Codes . . . 8-6
8.1.5 x87 FPU Control Word . . . 8-7
8.1.5.1 x87 FPU Floating-Point Exception Mask Bits . . . 8-7
8.1.5.2 Precision Control Field . . . 8-7
8.1.5.3 Rounding Control Field . . . 8-8
8.1.6 Infinity Control Flag . . . 8-8
8.1.7 x87 FPU Tag Word . . . 8-8
8.1.8 x87 FPU Instruction and Data (Operand) Pointers . . . 8-9
8.1.9 Last Instruction Opcode . . . 8-10
8.1.9.1 Fopcode Compatibility Sub-mode . . . 8-10
8.1.10 Saving the x87 FPU State with FSTENV/FNSTENV and FSAVE/FNSAVE . . . 8-11
8.1.11 Saving the x87 FPU State with FXSAVE . . . 8-12
8.2 X87 FPU DATA TYPES . . . 8-13
8.2.1 Indefinites . . . 8-14
8.2.2 Unsupported Double Extended Precision Floating-Point Encodings and Pseudo-Denormals . . . 8-14
8.3 X87 FPU INSTRUCTION SET . . . 8-15
8.3.1 Escape (ESC) Instructions . . . 8-15
8.3.2 x87 FPU Instruction Operands . . . 8-15
8.3.3 Data Transfer Instructions . . . 8-15
8.3.4 Load Constant Instructions . . . 8-17
8.3.5 Basic Arithmetic Instructions . . . 8-17
8.3.6 Comparison and Classification Instructions . . . 8-18
8.3.6.1 Branching on the x87 FPU Condition Codes . . . 8-20
8.3.7 Trigonometric Instructions . . . 8-20
8.3.8 Approximation of Pi . . . 8-21
8.3.9 Logarithmic, Exponential, and Scale . . . 8-21
8.3.10 Transcendental Instruction Accuracy . . . 8-21
8.3.11 x87 FPU Control Instructions . . . 8-23
8.3.12 Waiting vs. Non-waiting Instructions . . . 8-24
8.3.13 Unsupported x87 FPU Instructions . . . 8-24
8.4 X87 FPU FLOATING-POINT EXCEPTION HANDLING . . . 8-24
8.4.1 Arithmetic vs. Non-arithmetic Instructions . . . 8-25

PAGE

8.5 X87 FPU FLOATING-POINT EXCEPTION CONDITIONS. . . . 8-26
8.5.1 Invalid Operation Exception 8-26
8.5.1.1 Stack Overflow or Underflow Exception (#IS) 8-26
8.5.1.2 Invalid Arithmetic Operand Exception (#IA). . . . 8-27
8.5.2 Denormal Operand Exception (#D). . . . 8-28
8.5.3 Divide-By-Zero Exception (#Z) 8-28
8.5.4 Numeric Overflow Exception (#O) 8-28
8.5.5 Numeric Underflow Exception (#U) 8-29
8.5.6 Inexact-Result (Precision) Exception (#P) 8-30
8.6 X87 FPU EXCEPTION SYNCHRONIZATION. . . . 8-31
8.7 HANDLING X87 FPU EXCEPTIONS IN SOFTWARE 8-32
8.7.1 Native Mode. . . . 8-32
8.7.2 MS-DOS* Compatibility Sub-mode. . . . 8-32
8.7.3 Handling x87 FPU Exceptions in Software 8-33

CHAPTER 9
PROGRAMMING WITH INTEL® MMX™ TECHNOLOGY

9.1 OVERVIEW OF MMX TECHNOLOGY. . . . 9-1
9.2 THE MMX TECHNOLOGY PROGRAMMING ENVIRONMENT 9-1
9.2.1 MMX Technology in 64-Bit Mode and Compatibility Mode 9-2
9.2.2 MMX Registers 9-2
9.2.3 MMX Data Types 9-3
9.2.4 Memory Data Formats 9-3
9.2.5 Single Instruction, Multiple Data (SIMD) Execution Model 9-4
9.3 SATURATION AND WRAPAROUND MODES 9-4
9.4 MMX INSTRUCTIONS. . . . 9-5
9.4.1 Data Transfer Instructions 9-6
9.4.2 Arithmetic Instructions. . . . 9-6
9.4.3 Comparison Instructions. . . . 9-7
9.4.4 Conversion Instructions 9-7
9.4.5 Unpack Instructions. . . . 9-7
9.4.6 Logical Instructions 9-7
9.4.7 Shift Instructions 9-8
9.4.8 EMMS Instruction 9-8
9.5 COMPATIBILITY WITH X87 FPU ARCHITECTURE 9-8
9.5.1 MMX Instructions and the x87 FPU Tag Word 9-8
9.6 WRITING APPLICATIONS WITH MMX CODE. . . . 9-8
9.6.1 Checking for MMX Technology Support 9-8
9.6.2 Transitions Between x87 FPU and MMX Code 9-9
9.6.3 Using the EMMS Instruction 9-9
9.6.4 Mixing MMX and x87 FPU Instructions 9-10
9.6.5 Interfacing with MMX Code. . . . 9-10
9.6.6 Using MMX Code in a Multitasking Operating System Environment. . . . 9-10
9.6.7 Exception Handling in MMX Code 9-11
9.6.8 Register Mapping 9-11
9.6.9 Effect of Instruction Prefixes on MMX Instructions 9-11

CHAPTER 10
PROGRAMMING WITH INTEL®
STREAMING SIMD EXTENSIONS (INTEL® SSE)

10.1 OVERVIEW OF INTEL® SSE. . . . 10-1
10.2 INTEL® SSE PROGRAMMING ENVIRONMENT 10-2
10.2.1 Intel® SSE in 64-Bit Mode and Compatibility Mode. . . . 10-3
10.2.2 XMM Registers 10-3
10.2.3 MXCSR Control and Status Register. . . . 10-3
10.2.3.1 SIMD Floating-Point Mask and Flag Bits 10-4
10.2.3.2 SIMD Floating-Point Rounding Control Field 10-4
10.2.3.3 Flush-To-Zero 10-4
10.2.3.4 Denormals-Are-Zeros 10-5

PAGE

10.2.4 Compatibility of Intel® SSE with Intel® SSE2 and SSE3, MMX, and the x87 FPU. . . . 10-5
10.3 INTEL® SSE DATA TYPES . . . 10-5
10.4 INTEL® SSE INSTRUCTION SET . . . 10-6
10.4.1 Intel® SSE Packed and Scalar Floating-Point Instructions . . . 10-6
10.4.1.1 Intel® SSE Data Movement Instructions . . . 10-7
10.4.1.2 Intel® SSE Arithmetic Instructions . . . 10-8
10.4.2 Intel® SSE Logical Instructions . . . 10-9
10.4.2.1 Intel® SSE Comparison Instructions . . . 10-9
10.4.2.2 Intel® SSE Shuffle and Unpack Instructions . . . 10-9
10.4.3 Intel® SSE Conversion Instructions . . . 10-11
10.4.4 Intel® SSE 64-Bit SIMD Integer Instructions. . . . 10-11
10.4.5 MXCSR State Management Instructions. . . . 10-12
10.4.6 Cacheability Control, Prefetch, and Memory Ordering Instructions . . . 10-12
10.4.6.1 Cacheability Control Instructions . . . 10-12
10.4.6.2 Caching of Temporal vs. Non-Temporal Data . . . 10-12
10.4.6.3 PREFETCHh Instructions . . . 10-13
10.4.6.4 SFENCE Instruction . . . 10-14
10.5 FXSAVE AND FXRSTOR INSTRUCTIONS. . . . 10-14
10.5.1 FXSAVE Area . . . 10-14
10.5.1.1 x87 State. . . . 10-15
10.5.1.2 SSE State . . . 10-16
10.5.2 Operation of FXSAVE . . . 10-16
10.5.3 Operation of FXRSTOR . . . 10-17
10.6 HANDLING INTEL® SSE INSTRUCTION EXCEPTIONS . . . 10-17
10.7 WRITING APPLICATIONS WITH INTEL® SSE . . . 10-17

CHAPTER 11
PROGRAMMING WITH INTEL®
STREAMING SIMD EXTENSIONS 2 (INTEL® SSE2)
11.1 OVERVIEW OF INTEL® SSE2 . . . 11-1
11.2 INTEL® SSE2 PROGRAMMING ENVIRONMENT . . . 11-2
11.2.1 Intel® SSE2 in 64-Bit Mode and Compatibility Mode . . . 11-3
11.2.2 Compatibility of Intel® SSE2 with Intel® SSE, MMX Technology, and x87 FPU Programming Environment . . . 11-3
11.2.3 Denormals-Are-Zeros Flag . . . 11-3
11.3 INTEL® SSE2 DATA TYPES. . . . 11-3
11.4 INTEL® SSE2 INSTRUCTIONS . . . 11-4
11.4.1 Packed and Scalar Double Precision Floating-Point Instructions . . . 11-4
11.4.1.1 Data Movement Instructions . . . 11-5
11.4.1.2 Intel® SSE2 Arithmetic Instructions . . . 11-6
11.4.1.3 Intel® SSE2 Logical Instructions. . . . 11-7
11.4.1.4 Intel® SSE2 Comparison Instructions . . . 11-7
11.4.1.5 Intel® SSE2 Shuffle and Unpack Instructions. . . . 11-7
11.4.1.6 Intel® SSE2 Conversion Instructions. . . . 11-9
11.4.2 Intel® SSE2 64-Bit and 128-Bit SIMD Integer Instructions . . . 11-11
11.4.3 128-Bit SIMD Integer Instruction Extensions . . . 11-11
11.4.4 Cacheability Control and Memory Ordering Instructions . . . 11-12
11.4.4.1 FLUSH Cache Line . . . 11-12
11.4.4.2 Cacheability Control Instructions . . . 11-12
11.4.4.3 Memory Ordering Instructions. . . . 11-12
11.4.4.4 Pause. . . . 11-12
11.4.5 Branch Hints . . . 11-13
11.5 INTEL® SSE, SSE2, AND SSE3 EXCEPTIONS . . . 11-13
11.5.1 SIMD Floating-Point Exceptions . . . 11-13
11.5.2 SIMD Floating-Point Exception Conditions . . . 11-14
11.5.2.1 Invalid Operation Exception (#I) . . . 11-14
11.5.2.2 Denormal-Operand Exception (#D) . . . 11-15
11.5.2.3 Divide-By-Zero Exception (#Z) . . . 11-15
11.5.2.4 Numeric Overflow Exception (#O) . . . 11-15
11.5.2.5 Numeric Underflow Exception (#U) . . . 11-16

PAGE

11.5.2.6 Inexact-Result (Precision) Exception (#P) 11-16
11.5.3 Generating SIMD Floating-Point Exceptions 11-16
11.5.3.1 Handling Masked Exceptions 11-17
11.5.3.2 Handling Unmasked Exceptions 11-18
11.5.3.3 Handling Combinations of Masked and Unmasked Exceptions 11-18
11.5.4 Handling SIMD Floating-Point Exceptions in Software 11-18
11.5.5 Interaction of SIMD and x87 FPU Floating-Point Exceptions 11-18
11.6 WRITING APPLICATIONS WITH INTEL® SSE AND SSE2 11-19
11.6.1 General Guidelines for Using Intel® SSE and SSE2 11-19
11.6.2 Checking for Intel® SSE and SSE2 Support 11-19
11.6.3 Checking for the DAZ Flag in the MXCSR Register 11-20
11.6.4 Initialization of Intel® SSE and SSE2 11-20
11.6.5 Saving and Restoring the SSE/SSE2 State 11-21
11.6.6 Guidelines for Writing to the MXCSR Register 11-21
11.6.7 Interaction of Intel® SSE and SSE2 Instructions with x87 FPU and MMX Instructions 11-22
11.6.8 Compatibility of SIMD and x87 FPU Floating-Point Data Types 11-22
11.6.9 Mixing Packed and Scalar Floating-Point and 128-Bit SIMD Integer Instructions and Data 11-22
11.6.10 Interfacing with Intel® SSE and SSE2 Procedures and Functions 11-23
11.6.10.1 Passing Parameters in XMM Registers 11-23
11.6.10.2 Saving XMM Register State on a Procedure or Function Call 11-23
11.6.10.3 Caller-Save Recommendation for Procedure and Function Calls 11-24
11.6.11 Updating Existing MMX Technology Routines Using 128-Bit SIMD Integer Instructions 11-24
11.6.12 Branching on Arithmetic Operations 11-24
11.6.13 Cacheability Hint Instructions 11-25
11.6.14 Effect of Instruction Prefixes on Intel® SSE and SSE2 Instructions 11-25

CHAPTER 12
PROGRAMMING WITH INTEL® SSE3, SSSE3,
INTEL® SSE4, AND INTEL® AES-NI

12.1 PROGRAMMING ENVIRONMENT AND DATA TYPES 12-1
12.1.1 Intel® SSE3, SSSE3, and Intel® SSE4 in 64-Bit Mode and Compatibility Mode 12-1
12.1.2 Compatibility of Intel® SSE3 and SSSE3 with MMX Technology, the x87 FPU Environment, and Intel® SSE and SSE2 12-1
12.1.3 Horizontal and Asymmetric Processing 12-1
12.2 OVERVIEW OF INTEL® SSE3 INSTRUCTIONS 12-2
12.3 INTEL® SSE3 INSTRUCTIONS 12-2
12.3.1 x87 FPU Instruction for Integer Conversion 12-3
12.3.2 SIMD Integer Instruction for Specialized 128-Bit Unaligned Data Load 12-3
12.3.3 SIMD Floating-Point Instructions That Enhance LOAD/MOVE/DUPLICATE Performance 12-3
12.3.4 SIMD Floating-Point Instructions Provide Packed Addition/Subtraction 12-4
12.3.5 SIMD Floating-Point Instructions Provide Horizontal Addition/Subtraction 12-4
12.3.6 Two Thread Synchronization Instructions 12-5
12.4 WRITING APPLICATIONS WITH INTEL® SSE3 12-5
12.4.1 Guidelines for Using Intel® SSE3 12-5
12.4.2 Checking for Intel® SSE3 Support 12-5
12.4.3 Enable FTZ and DAZ for SIMD Floating-Point Computation 12-6
12.4.4 Programming Intel® SSE3 with Intel® SSE and SSE2 12-6
12.5 OVERVIEW OF SSSE3 INSTRUCTIONS 12-6
12.6 SSSE3 INSTRUCTIONS 12-6
12.6.1 Horizontal Addition/Subtraction 12-7
12.6.2 Packed Absolute Values 12-7
12.6.3 Multiply and Add Packed Signed and Unsigned Bytes 12-8
12.6.4 Packed Multiply High with Round and Scale 12-8
12.6.5 Packed Shuffle Bytes 12-8
12.6.6 Packed Sign 12-8
12.6.7 Packed Align Right 12-8
12.7 WRITING APPLICATIONS WITH SSSE3 EXTENSIONS 12-9
12.7.1 Guidelines for Using SSSE3 12-9
12.7.2 Checking for SSSE3 Support 12-9
12.8 INTEL® SSE3, SSSE3, AND INTEL® SSE4 EXCEPTIONS 12-9

PAGE

12.8.1 Device Not Available (DNA) Exceptions 12-9
12.8.2 Numeric Error Flag and IGNNE# 12-9
12.8.3 Emulation 12-10
12.8.4 IEEE 754 Compliance of Intel® SSE4.1 Floating-Point Instructions 12-10
12.9 INTEL® SSE4 OVERVIEW 12-10
12.10 INTEL® SSE4.1 INSTRUCTION SET 12-11
12.10.1 Dword Multiply Instructions 12-11
12.10.2 Floating-Point Dot Product Instructions 12-11
12.10.3 Streaming Load Hint Instruction 12-12
12.10.4 Packed Blending Instructions 12-14
12.10.5 Packed Integer MIN/MAX Instructions 12-14
12.10.6 Floating-Point Round Instructions with Selectable Rounding Mode 12-14
12.10.7 Insertion and Extractions from XMM Registers 12-15
12.10.8 Packed Integer Format Conversions 12-15
12.10.9 Improved Sums of Absolute Differences (SAD) for 4-Byte Blocks 12-16
12.10.10 Horizontal Search 12-16
12.10.11 Packed Test 12-17
12.10.12 Packed Qword Equality Comparisons 12-17
12.10.13 Dword Packing With Unsigned Saturation 12-17
12.11 INTEL® SSE4.2 INSTRUCTION SET 12-17
12.11.1 String and Text Processing Instructions 12-17
12.11.1.1 Memory Operand Alignment 12-18
12.11.2 Packed Comparison SIMD Integer Instruction 12-18
12.12 WRITING APPLICATIONS WITH INTEL® SSE4 EXTENSIONS 12-18
12.12.1 Guidelines for Using Intel® SSE4 Extensions 12-18
12.12.2 Checking for Intel® SSE4.1 Support 12-19
12.12.3 Checking for Intel® SSE4.2 Support 12-19
12.13 INTEL® AES-NI OVERVIEW 12-19
12.13.1 Little-Endian Architecture and Big-Endian Specification (FIPS 197) 12-19
12.13.1.1 AES Data Structure in Intel® 64 Architecture 12-20
12.13.2 AES Transformations and Functions 12-21
12.13.3 PCLMULQDQ 12-24
12.13.4 Checking for Intel® AES-NI Support 12-24

CHAPTER 13
MANAGING STATE USING THE XSAVE FEATURE SET
13.1 XSAVE-SUPPORTED FEATURES AND STATE-COMPONENT BITMAPS 13-1
13.2 ENUMERATION OF CPU SUPPORT FOR XSAVE INSTRUCTIONS AND XSAVE-SUPPORTED FEATURES 13-3
13.3 ENABLING THE XSAVE FEATURE SET AND XSAVE-ENABLED FEATURES 13-5
13.4 XSAVE AREA 13-7
13.4.1 Legacy Region of an XSAVE Area 13-8
13.4.2 XSAVE Header 13-9
13.4.3 Extended Region of an XSAVE Area 13-9
13.5 XSAVE-MANAGED STATE 13-10
13.5.1 x87 State 13-10
13.5.2 SSE State 13-11
13.5.3 AVX State 13-11
13.5.4 MPX State 13-11
13.5.5 AVX-512 State 13-12
13.5.6 PT State 13-13
13.5.7 PKRU State 13-13
13.5.8 PASID State 13-14
13.5.9 CET State 13-14
13.5.10 HDC State 13-14
13.5.11 UINTR State 13-14
13.5.12 LBR State 13-16
13.5.13 HWP State 13-16
13.5.14 AMX State 13-17
13.6 PROCESSOR TRACKING OF XSAVE-MANAGED STATE 13-17

PAGE

13.7 OPERATION OF XSAVE ... 13-19
13.8 OPERATION OF XRSTOR ... 13-19
13.8.1 Standard Form of XRSTOR ... 13-20
13.8.2 Compacted Form of XRSTOR ... 13-20
13.8.3 XRSTOR and the Init and Modified Optimizations ... 13-21
13.9 OPERATION OF XSAVEOPT ... 13-21
13.10 OPERATION OF XSAVEC ... 13-23
13.11 OPERATION OF XSAVES ... 13-24
13.12 OPERATION OF XRSTORS ... 13-25
13.13 MEMORY ACCESSES BY THE XSAVE FEATURE SET ... 13-26
13.14 EXTENDED FEATURE DISABLE (XFD) ... 13-27

CHAPTER 14
PROGRAMMING WITH INTEL® AVX, FMA, AND INTEL® AVX2

14.1 INTEL® AVX OVERVIEW ... 14-1
14.1.1 256-Bit Wide SIMD Register Support ... 14-1
14.1.2 Instruction Syntax Enhancements ... 14-2
14.1.3 VEX Prefix Instruction Encoding Support ... 14-2
14.2 FUNCTIONAL OVERVIEW ... 14-3
14.2.1 256-Bit Floating-Point Arithmetic Processing Enhancements ... 14-9
14.2.2 256-Bit Non-Arithmetic Instruction Enhancements ... 14-9
14.2.3 Arithmetic Primitives for 128-Bit Vector and Scalar processing ... 14-11
14.2.4 Non-Arithmetic Primitives for 128-Bit Vector and Scalar Processing ... 14-13
14.3 DETECTION OF INTEL® AVX INSTRUCTIONS ... 14-15
14.3.1 Detection of VEX-Encoded AES and VPCLMULQDQ ... 14-17
14.4 HALF PRECISION FLOATING-POINT CONVERSION ... 14-18
14.4.1 Detection of F16C Instructions ... 14-20
14.5 FUSED-MULTIPLY-ADD (FMA) EXTENSIONS ... 14-21
14.5.1 FMA Instruction Operand Order and Arithmetic Behavior ... 14-22
14.5.2 Fused-Multiply-ADD (FMA) Numeric Behavior ... 14-22
14.5.3 Detection of FMA ... 14-24
14.6 OVERVIEW OF INTEL® ADVANCED VECTOR EXTENSIONS 2 (INTEL® AVX2) ... 14-25
14.6.1 Intel® AVX2 and 256-Bit Vector Integer Processing ... 14-25
14.7 PROMOTED VECTOR INTEGER INSTRUCTIONS IN INTEL® AVX2 ... 14-26
14.7.1 Detection of Intel® AVX2 ... 14-31
14.8 ACCESSING YMM REGISTERS ... 14-32
14.9 MEMORY ALIGNMENT ... 14-32
14.10 SIMD FLOATING-POINT EXCEPTIONS ... 14-34
14.11 EMULATION ... 14-34
14.12 WRITING INTEL® AVX FLOATING-POINT EXCEPTION HANDLERS ... 14-34
14.13 GENERAL PURPOSE INSTRUCTION SET ENHANCEMENTS ... 14-35

CHAPTER 15
PROGRAMMING WITH INTEL® AVX-512

15.1 OVERVIEW ... 15-1
15.1.1 512-Bit Wide SIMD Register Support ... 15-1
15.1.2 32 SIMD Register Support ... 15-1
15.1.3 Eight Opmask Register Support ... 15-1
15.1.4 Instruction Syntax Enhancement ... 15-2
15.1.5 EVEX Instruction Encoding Support ... 15-3
15.2 DETECTION OF AVX-512 FOUNDATION INSTRUCTIONS ... 15-3
15.2.1 Additional 512-Bit Instruction Extensions of the Intel® AVX-512 Family ... 15-4
15.2.2 Detection of AVX512-FP16 Instructions ... 15-5
15.3 DETECTION OF 512-BIT INSTRUCTION GROUPS OF THE INTEL® AVX-512 FAMILY ... 15-6
15.4 DETECTION OF INTEL® AVX-512 INSTRUCTION GROUPS OPERATING AT 256 AND 128-BIT VECTOR LENGTHS ... 15-7
15.5 ACCESSING XMM, YMM, AND ZMM REGISTERS ... 15-8
15.6 ENHANCED VECTOR PROGRAMMING ENVIRONMENT USING EVEX ENCODING ... 15-8
15.6.1 OPMASK Register to Predicate Vector Data Processing ... 15-9
15.6.1.1 Opmask Register K0 ... 15-9

15.6.1.2 Example of Opmask Usages . . . 15-10
15.6.2 OpMask Instructions . . . 15-11
15.6.3 Broadcast . . . 15-11
15.6.4 Static Rounding Mode and Suppress All Exceptions . . . 15-12
15.6.5 Compressed Disp8*N Encoding . . . 15-13
15.7 MEMORY ALIGNMENT . . . 15-13
15.8 SIMD FLOATING-POINT EXCEPTIONS . . . 15-14
15.9 INSTRUCTION EXCEPTION SPECIFICATION . . . 15-15
15.10 EMULATION . . . 15-15
15.11 WRITING FLOATING-POINT EXCEPTION HANDLERS . . . 15-15

CHAPTER 16
PROGRAMMING WITH INTEL® AVX10
16.1 INTRODUCTION . . . 16-1
16.2 FEATURE VERSIONING AND ENUMERATION . . . 16-1

CHAPTER 17
PROGRAMMING WITH INTEL® TRANSACTIONAL SYNCHRONIZATION EXTENSIONS
17.1 OVERVIEW . . . 17-1
17.2 INTEL® TRANSACTIONAL SYNCHRONIZATION EXTENSIONS . . . 17-1
17.2.1 HLE Software Interface . . . 17-2
17.2.2 RTM Software Interface . . . 17-3
17.3 INTEL® TSX APPLICATION PROGRAMMING MODEL . . . 17-3
17.3.1 Detection of Transactional Synchronization Support . . . 17-3
17.3.1.1 Detection of HLE Support . . . 17-3
17.3.1.2 Detection of RTM Support . . . 17-3
17.3.1.3 Detection of XTEST Instruction . . . 17-4
17.3.1.4 Detection of Intel® TSX Suspend Load Address Tracking . . . 17-4
17.3.2 Querying Transactional Execution Status . . . 17-4
17.3.3 Requirements for HLE Locks . . . 17-4
17.3.4 Transactional Nesting . . . 17-4
17.3.4.1 HLE Nesting and Elision . . . 17-4
17.3.4.2 RTM Nesting . . . 17-5
17.3.4.3 Nesting HLE and RTM . . . 17-5
17.3.5 RTM Abort Status Definition . . . 17-5
17.3.6 RTM Memory Ordering . . . 17-6
17.3.7 RTM-Enabled Debugger Support . . . 17-6
17.3.8 Intel® TSX Suspend/Resume Load Address Tracking Support . . . 17-6
17.3.9 Programming Considerations . . . 17-6
17.3.9.1 Instruction Based Considerations . . . 17-7
17.3.9.2 Runtime Considerations . . . 17-8

CHAPTER 18
CONTROL-FLOW ENFORCEMENT TECHNOLOGY (CET)
18.1 INTRODUCTION . . . 18-1
18.1.1 Shadow Stack . . . 18-1
18.1.2 Indirect Branch Tracking . . . 18-1
18.1.3 Speculative Behavior when CET is Enabled . . . 18-2
18.2 SHADOW STACKS . . . 18-2
18.2.1 Shadow Stack Pointer and its Operand and Address Size Attributes . . . 18-2
18.2.2 Terminology . . . 18-2
18.2.3 Supervisor Shadow Stack Token . . . 18-3
18.2.4 Shadow Stack Usage on Task Switch . . . 18-5
18.2.5 Switching Shadow Stacks . . . 18-5
18.2.6 Constraining Execution at Targets of RET . . . 18-7
18.3 INDIRECT BRANCH TRACKING . . . 18-7
18.3.1 No-track Prefix for Near Indirect CALL/JMP . . . 18-8
18.3.2 Terminology . . . 18-9
18.3.3 Indirect Branch Tracking . . . 18-10

PAGE

18.3.3.1 Control Transfers between CPL 3 and CPL < 3 18-10
18.3.3.2 Control Transfers within CPL 3 or CPL < 3 18-10
18.3.4 Indirect Branch Tracking State Machine 18-11
18.3.5 INT3 Treatment 18-12
18.3.6 Legacy Compatibility Treatment 18-12
18.3.6.1 Legacy Code Page Bitmap Format 18-13
18.3.7 Other Considerations 18-13
18.3.7.1 Intel® Transactional Synchronization Extensions (Intel® TSX) Interactions 18-13
18.3.7.2 #CP(ENDBRANCH) Priority w.r.t #NM and #UD 18-13
18.3.7.3 #CP(ENDBRANCH) Priority w.r.t #BP and #DB 18-13
18.3.8 Constraining Speculation after Missing ENDBRANCH 18-14
18.4 INTEL® TRUSTED EXECUTION TECHNOLOGY (INTEL® TXT) INTERACTIONS 18-14

CHAPTER 19
PROGRAMMING WITH INTEL® ADVANCED MATRIX EXTENSIONS
19.1 INTRODUCTION 19-1
19.1.1 Tile Architecture Details 19-3
19.1.2 TMUL Architecture Details 19-4
19.1.3 Handling of Tile Row and Column Limits 19-4
19.1.4 Exceptions and Interrupts 19-5
19.2 RECOMMENDATIONS FOR SYSTEM SOFTWARE 19-5
19.3 IMPLEMENTATION PARAMETERS 19-5
19.4 HELPER FUNCTIONS 19-6

CHAPTER 20
INPUT/OUTPUT
20.1 I/O PORT ADDRESSING 20-1
20.2 I/O PORT HARDWARE 20-1
20.3 I/O ADDRESS SPACE 20-1
20.3.1 Memory-Mapped I/O 20-2
20.4 I/O INSTRUCTIONS 20-3
20.5 PROTECTED-MODE I/O 20-3
20.5.1 I/O Privilege Level 20-3
20.5.2 I/O Permission Bit Map 20-4
20.6 ORDERING I/O 20-5

CHAPTER 21
PROCESSOR IDENTIFICATION AND FEATURE DETERMINATION
21.1 IMPORTANT CONSIDERATIONS WHEN USING THE CPUID INSTRUCTION 21-1
21.1.1 Guidelines for Using the CPUID Instruction 21-1
21.1.2 Identification of Earlier Processors 21-1
21.1.3 CPUID Basic and Extended Range 21-2
21.1.4 CPUID Domains 21-2
21.1.5 CPUID Runtime Mutable Fields 21-3
21.1.6 CPUID Reserved Fields 21-4
21.1.7 CPUID Instruction for Serialization 21-4
21.1.8 IA32_BIOS_SIGN_ID Returns Microcode Update Signature 21-4
21.2 METHODS FOR RETURNING BRANDING INFORMATION USING CPUID 21-4
21.2.1 The Processor Brand String Method 21-4
21.2.2 The Processor Brand Index Method 21-5
21.3 CPUID LEAVES 21-6
CPUID.00H -- Maximum Input for Basic CPUID and Vendor ID 21-7
CPUID.01H -- Version and Features 21-8
CPUID.02H -- TLB/Cache/Prefetch Information 21-15
CPUID.03H -- Processor Serial Number 21-20
CPUID.04H -- Deterministic Cache Parameters 21-21
CPUID.05H -- MONITOR and MWAIT Features 21-23
CPUID.06H -- Thermal and Power Management Features 21-24
CPUID.07H -- Structured Extended Feature Flags 21-26

PAGE

CPUID.08H -- Reserved 21-35
CPUID.09H -- Direct Cache Access Information 21-36
CPUID.0AH -- Architectural Performance Monitoring 21-37
CPUID.0BH -- Extended Topology 21-39
CPUID.0CH -- Reserved 21-41
CPUID.0DH -- Processor Extended State 21-42
CPUID.0EH -- Reserved 21-45
CPUID.0FH -- Intel® Resource Director Technology (Intel® RDT) Monitoring 21-46
CPUID.10H -- Intel® Resource Director Technology (Intel® RDT) Allocation 21-48
CPUID.11H -- Reserved 21-51
CPUID.12H -- Intel® Software Guard Extensions (Intel® SGX) Capability 21-52
CPUID.13H -- Reserved 21-55
CPUID.14H -- Intel® Processor Trace (Intel® PT) 21-56
CPUID.15H -- Time Stamp Counter and Nominal Core Crystal Clock 21-59
CPUID.16H -- Processor Frequency Information 21-60
CPUID.17H -- System-on-Chip Vendor Attribute 21-61
CPUID.18H -- Deterministic Address Translation Parameters 21-63
CPUID.19H -- Key Locker 21-65
CPUID.1AH -- Native Model ID Enumeration 21-66
CPUID.1BH -- PCONFIG Information 21-67
CPUID.1CH -- Last Branch Records (LBR) Information 21-68
CPUID.1DH -- Tile Information 21-69
CPUID.1EH -- TMUL Information 21-70
CPUID.1FH -- V2 Extended Topology 21-71
CPUID.20H -- Processor History Reset Information 21-73
CPUID.21H -- Unimplemented 21-74
CPUID.22H -- Reserved 21-75
CPUID.23H -- Architectural Performance Monitoring Extended 21-76
CPUID.24H -- Converged Vector ISA 21-80
CPUID.27H -- Intel® Resource Director Technology (Intel® RDT) Asymmetric Monitoring 21-81
CPUID.28H -- Intel® Resource Director Technology (Intel® RDT) Asymmetric Allocation 21-83
CPUID.80000000H -- Maximum Input Value for Extended Function CPUID Information 21-87
CPUID.80000001H -- Extended Processor Signature and Feature Bits 21-88
CPUID.80000002H -- Processor Brand String (Bytes 0 to 15) 21-89
CPUID.80000003H -- Processor brand string (Bytes 16 to 31) 21-90
CPUID.80000004H -- Processor brand string (Bytes 32 to 47) 21-91
CPUID.80000005H -- Reserved 21-92
CPUID.80000006H -- Extended Function CPUID Information 21-93
CPUID.80000007H -- Extended Function CPUID Information 1 21-94
CPUID.80000008H -- Extended Function CPUID Information 2 21-95

APPENDIX A
EFLAGS CROSS-REFERENCE
A.1 EFLAGS AND INSTRUCTIONS A-1

APPENDIX B
EFLAGS CONDITION CODES
B.1 CONDITION CODES B-1

APPENDIX C
FLOATING-POINT EXCEPTIONS SUMMARY
C.1 OVERVIEW C-1
C.2 X87 FPU INSTRUCTIONS C-1
C.3 INTEL® SSE INSTRUCTIONS C-3
C.4 INTEL® SSE2 INSTRUCTIONS C-5
C.5 INTEL® SSE3 INSTRUCTIONS C-7
C.6 SSSE3 INSTRUCTIONS C-7
C.7 INTEL® SSE4 INSTRUCTIONS C-7

PAGE

APPENDIX D
GUIDELINES FOR WRITING SIMD FLOATING-POINT EXCEPTION HANDLERS
D.1 TWO OPTIONS FOR HANDLING FLOATING-POINT EXCEPTIONS D-1
D.2 SOFTWARE EXCEPTION HANDLING D-1
D.3 EXCEPTION SYNCHRONIZATION D-3
D.4 SIMD FLOATING-POINT EXCEPTIONS AND THE IEEE STANDARD 754 ..... D-3
D.4.1 Floating-Point Emulation D-3
D.4.2 Intel® SSE, SSE2, and SSE3 Response To Floating-Point Exceptions D-4
D.4.2.1 Numeric Exceptions D-5
D.4.2.2 Results of Operations with NaN Operands or a NaN Result for Intel® SSE, SSE2, and SSE3 Numeric Instructions D-5
D.4.2.3 Condition Codes, Exception Flags, and Response for Masked and Unmasked Numeric Exceptions D-9
D.4.3 Example SIMD Floating-Point Emulation Implementation D-15

APPENDIX E
INTEL® MEMORY PROTECTION EXTENSIONS
E.1 INTEL® MEMORY PROTECTION EXTENSIONS (INTEL® MPX) E-1
E.2 INTRODUCTION E-1
E.3 INTEL MPX PROGRAMMING ENVIRONMENT E-2
E.3.1 Detection and Enumeration of Intel MPX Interfaces E-2
E.3.2 Bounds Registers E-2
E.3.3 Configuration and Status Registers E-3
E.3.4 Read and Write of IA32_BNDCFGS E-4
E.4 INTEL MPX INSTRUCTION SUMMARY E-4
E.4.1 Instruction Encoding E-5
E.4.2 Usage and Examples E-5
E.4.3 Loading and Storing Bounds in Memory E-6
E.4.3.1 BNDLDX and BNDSTX in 64-Bit Mode E-7
E.4.3.2 BNDLDX and BNDSTX Outside 64-Bit Mode E-9
E.5 INTERACTIONS WITH INTEL MPX E-10
E.5.1 Intel® MPX and Operating Modes E-10
E.5.2 Intel® MPX Support for Pointer Operations with Branching E-10
E.5.3 CALL, RET, JMP, and All Jcc E-11
E.5.4 BOUND Instruction and Intel MPX E-11
E.5.5 Programming Considerations E-12
E.5.6 Intel MPX and System Management Mode E-12
E.5.7 Support of Intel MPX in VMCS E-12
E.5.8 Support of Intel MPX in Intel TSX E-12

FIGURES

Figure 1-1. Bit and Byte Order 1-7
Figure 1-2. Syntax for CPUID, CR, and MSR Data Presentation. 1-9
Figure 2-1. The P6 Processor Microarchitecture with Advanced Transfer Cache Enhancement 2-7
Figure 2-2. The Intel NetBurst® Microarchitecture 2-9
Figure 2-3. The Intel® Core™ Microarchitecture Pipeline Functionality 2-11
Figure 2-4. SIMD Extensions, Register Layouts, and Data Types 2-16
Figure 2-5. Comparison of an IA-32 Processor Supporting Intel® Hyper-Threading Technology and a Traditional Dual Processor System. 2-17
Figure 2-6. Intel® 64 and IA-32 Processors that Support Dual-Core 2-18
Figure 2-7. Intel® 64 Processors that Support Quad-Core 2-19
Figure 2-8. Intel® Core™ i7 Processor 2-19
Figure 3-1. IA-32 Basic Execution Environment for Non-64-Bit Modes 3-3
Figure 3-2. 64-Bit Mode Execution Environment 3-6
Figure 3-3. Three Memory Management Models 3-8
Figure 3-4. General System and Application Programming Registers 3-11
Figure 3-5. Alternate General-Purpose Register Names 3-12
Figure 3-6. Use of Segment Registers for Flat Memory Model 3-14
Figure 3-7. Use of Segment Registers in Segmented Memory Model 3-14
Figure 3-8. EFLAGS Register 3-16
Figure 3-9. Memory Operand Address 3-21
Figure 3-10. Memory Operand Address in 64-Bit Mode 3-21
Figure 3-11. Offset (or Effective Address) Computation. 3-23
Figure 4-1. Fundamental Data Types 4-1
Figure 4-2. Bytes, Words, Doublewords, Quadwords, and Double Quadwords in Memory 4-2
Figure 4-3. Numeric Data Types 4-3
Figure 4-4. Comparison of BF16 to FP16 and FP32 4-6
Figure 4-5. Pointer Data Types 4-7
Figure 4-6. Pointers in 64-Bit Mode. 4-8
Figure 4-7. Bit Field Data Type 4-8
Figure 4-8. 64-Bit Packed SIMD Data Types 4-9
Figure 4-9. 128-Bit Packed SIMD Data Types. 4-10
Figure 4-10. BCD Data Types 4-11
Figure 4-11. Binary Real Number System 4-13
Figure 4-12. Binary Floating-Point Format. 4-13
Figure 4-13. Real Numbers and NaNs 4-15
Figure 6-1. Stack Structure 6-2
Figure 6-2. Stack on Near and Far Calls 6-6
Figure 6-3. Shadow Stack on Near and Far Calls 6-6
Figure 6-4. Protection Rings 6-8
Figure 6-5. Stack Switch on a Call to a Different Privilege Level 6-9
Figure 6-6. Shadow Stack Switch on a Call to a Different Privilege Level 6-10
Figure 6-7. Stack Usage on Transfers to Interrupt and Exception Handling Routines 6-15
Figure 6-8. Shadow Stack Usage on Transfers to Interrupt and Exception Handling Routines 6-16
Figure 6-9. Nested Procedures 6-21
Figure 6-10. Stack Frame After Entering the MAIN Procedure 6-22
Figure 6-11. Stack Frame After Entering Procedure A. 6-22
Figure 6-12. Stack Frame After Entering Procedure B. 6-23
Figure 6-13. Stack Frame After Entering Procedure C. 6-24
Figure 7-1. Operation of the PUSH Instruction 7-5
Figure 7-2. Operation of the PUSHA Instruction 7-6
Figure 7-3. Operation of the POP Instruction 7-6
Figure 7-4. Operation of the POPA Instruction 7-7
Figure 7-5. Sign Extension 7-7
Figure 7-6. SHL/SAL Instruction Operation 7-11
Figure 7-7. SHR Instruction Operation 7-11
Figure 7-8. SAR Instruction Operation 7-12
Figure 7-9. SHLD and SHRD Instruction Operations 7-12

PAGE

Figure 7-10. ROL, ROR, RCL, and RCR Instruction Operations. . . . 7-13
Figure 7-11. Flags Affected by the PUSHF, POPF, PUSHFD, and POPFD Instructions 7-21
Figure 8-1. x87 FPU Execution Environment 8-2
Figure 8-2. x87 FPU Data Register Stack. . . . 8-2
Figure 8-3. Example x87 FPU Dot Product Computation 8-3
Figure 8-4. x87 FPU Status Word. . . . 8-4
Figure 8-5. Moving the Condition Codes to the EFLAGS Register. . . . 8-6
Figure 8-6. x87 FPU Control Word 8-7
Figure 8-7. x87 FPU Tag Word 8-8
Figure 8-8. Contents of x87 FPU Opcode Registers 8-10
Figure 8-9. Protected Mode x87 FPU State Image in Memory, 32-Bit Format. . . . 8-11
Figure 8-10. Real Mode x87 FPU State Image in Memory, 32-Bit Format 8-12
Figure 8-11. Protected Mode x87 FPU State Image in Memory, 16-Bit Format. . . . 8-12
Figure 8-12. Real Mode x87 FPU State Image in Memory, 16-Bit Format 8-12
Figure 8-13. x87 FPU Data Type Formats 8-13
Figure 9-1. MMX Technology Execution Environment 9-2
Figure 9-2. MMX Register Set 9-3
Figure 9-3. Data Types Introduced with the MMX Technology 9-3
Figure 9-4. SIMD Execution Model 9-4
Figure 10-1. Intel® SSE Execution Environment 10-2
Figure 10-2. XMM Registers 10-3
Figure 10-3. MXCSR Control/Status Register 10-4
Figure 10-4. 128-Bit Packed Single Precision Floating-Point Data Type 10-6
Figure 10-5. Packed Single Precision Floating-Point Operation 10-7
Figure 10-6. Scalar Single Precision Floating-Point Operation 10-7
Figure 10-7. SHUFPS Instruction, Packed Shuffle Operation 10-10
Figure 10-8. UNPCKHPS Instruction, High Unpack and Interleave Operation 10-10
Figure 10-9. UNPCKLPS Instruction, Low Unpack and Interleave Operation 10-10
Figure 11-1. Intel® Streaming SIMD Extensions 2 Execution Environment. . . . 11-2
Figure 11-2. Data Types Introduced with Intel® SSE2. . . . 11-4
Figure 11-3. Packed Double Precision Floating-Point Operations 11-5
Figure 11-4. Scalar Double Precision Floating-Point Operations 11-5
Figure 11-5. SHUFPD Instruction, Packed Shuffle Operation 11-8
Figure 11-6. UNPCKHPD Instruction, High Unpack, and Interleave Operation. . . . 11-8
Figure 11-7. UNPCKLPD Instruction, Low Unpack, and Interleave Operation 11-8
Figure 11-8. Intel® SSE and SSE2 Conversion Instructions 11-9
Figure 11-9. Example Masked Response for Packed Operations. . . . 11-17
Figure 12-1. Asymmetric Processing in ADDSUBPD 12-2
Figure 12-2. Horizontal Data Movement in HADDPD. . . . 12-2
Figure 12-3. Horizontal Data Movement in PHADDD. . . . 12-7
Figure 12-4. MPSADBW Operation 12-16
Figure 12-5. AES State Flow 12-19
Figure 14-1. 256-Bit Wide SIMD Register 14-2
Figure 14-2. General Procedural Flow of Application Detection of Intel® AVX 14-15
Figure 14-3. General Procedural Flow of Application Detection of Float-16 14-20
Figure 15-1. 512-Bit Wide Vectors and SIMD Register Set. . . . 15-2
Figure 15-2. Procedural Flow for Application Detection of AVX-512 Foundation Instructions. . . . 15-4
Figure 15-3. Procedural Flow for Application Detection of 512-Bit Instructions. . . . 15-5
Figure 15-4. Procedural Flow for Application Detection of 512-Bit Instruction Groups 15-6
Figure 15-5. Procedural Flow for Detection of Intel® AVX-512 Instructions Operating at Vector Lengths < 512 15-7
Figure 18-1. Supervisor Shadow Stack with a Supervisor Shadow Stack Token 18-4
Figure 18-2. RSTORSSP to Switch to New Shadow Stack. . . . 18-6
Figure 18-3. SAVEPREVSSP to Save a Restore Point 18-6
Figure 18-4. Priority of Control Protection Exception on Missing ENDBRANCH. . . . 18-8
Figure 19-1. Intel® AMX Architecture 19-1
Figure 19-2. The TMUL Unit 19-3
Figure 19-3. Matrix Multiply C+= A*B. . . . 19-4
Figure 20-1. Memory-Mapped I/O 20-2
Figure 20-2. I/O Permission Bit Map 20-4

PAGE

Figure 21-1. Determination of Support for the Processor Brand String. 21-5
Figure D-1. Control Flow for Handling Unmasked Floating-Point Exceptions D-4
Figure E-1. Layout of the Bounds Registers BND0-BND3 . . . . . E-3
Figure E-2. Common Layout of the Bound Configuration Registers BNDCFGU and BNDCFGS E-3
Figure E-3. Layout of the Bound Status Register BNDSTATUS E-4
Figure E-4. Bound Paging Structure and Address Translation in 64-Bit Mode. E-8
Figure E-5. Bound Paging Structure and Address Translation Outside 64-Bit Mode E-9

PAGE

TABLES

Table 2-1. Key Features of Most Recent IA-32 Processors 2-21
Table 2-2. Key Features of Most Recent Intel® 64 Processors 2-21
Table 2-3. Key Features of Previous Generations of IA-32 Processors 2-27
Table 2-4. Planned Intel® ISA and Features Removal List 2-28
Table 2-5. Intel® ISA and Features Removal List. 2-28
Table 3-1. Instruction Pointer Sizes 3-10
Table 3-2. Addressable General Purpose Registers. 3-13
Table 3-3. Effective Operand- and Address-Size Attributes. 3-19
Table 3-4. Effective Operand- and Address-Size Attributes in 64-Bit Mode 3-19
Table 3-5. Default Segment Selection Rules 3-22
Table 4-1. Signed Integer Encodings 4-4
Table 4-2. Length, Precision, and Range of Floating-Point Data Types. 4-5
Table 4-3. Floating-Point Number and NaN Encodings 4-5
Table 4-4. BF16 Format Numeric Definitions 4-7
Table 4-5. Packed Decimal Integer Encodings 4-11
Table 4-6. Real and Floating-Point Number Notation 4-13
Table 4-7. Denormalization Process 4-16
Table 4-8. Rules for Handling NaNs 4-17
Table 4-9. Rounding Modes and Encoding of Rounding Control (RC) Field 4-19
Table 4-10. Numeric Overflow Thresholds 4-22
Table 4-11. Masked Responses to Numeric Overflow. 4-22
Table 4-12. Numeric Underflow (Normalized) Thresholds 4-23
Table 5-2. Instruction Set Extensions Introduction in Intel® 64 and IA-32 Processors 5-2
Table 5-1. Instruction Groups in Intel® 64 and IA-32 Processors 5-2
Table 5-3. Supervisor and User Mode Enclave Instruction Leaf Functions in Long-Form of SGX1 5-40
Table 6-1. Exceptions and Interrupts. 6-13
Table 7-1. Move Instruction Operations 7-3
Table 7-2. Conditional Move Instructions 7-4
Table 7-3. Bit Test and Modify Instructions 7-14
Table 7-4. Conditional Jump Instructions. 7-16
Table 8-1. Condition Code Interpretation 8-5
Table 8-2. Precision Control Field (PC) 8-8
Table 8-3. Unsupported Double Extended Precision Floating-Point Encodings and Pseudo-Denormals. 8-14
Table 8-4. Data Transfer Instructions 8-16
Table 8-5. Floating-Point Conditional Move Instructions 8-16
Table 8-6. Setting of x87 FPU Condition Code Flags for Floating-Point Number Comparisons 8-19
Table 8-7. Setting of EFLAGS Status Flags for Floating-Point Number Comparisons 8-19
Table 8-8. TEST Instruction Constants for Conditional Branching 8-20
Table 8-9. Arithmetic and Non-arithmetic Instructions 8-25
Table 8-10. Invalid Arithmetic Operations and the Masked Responses to Them 8-27
Table 8-11. Divide-By-Zero Conditions and the Masked Responses to Them 8-28
Table 9-1. Data Range Limits for Saturation 9-5
Table 9-2. MMX Instruction Set Summary 9-6
Table 9-3. Effect of Prefixes on MMX Instructions 9-11
Table 10-1. PREFETCHh Instructions Caching Hints. 10-13
Table 10-2. Format of an FXSAVE Area. 10-15
Table 11-1. Masked Responses of Intel® SSE, SSE2, and SSE3 Instructions to Invalid Arithmetic Operations 11-14
Table 11-2. SSE and SSE2 State Following a Power-up/Reset or INIT 11-20
Table 11-3. Effect of Prefixes on the Intel® SSE, SSE2, and SSE3 Instructions. 11-26
Table 12-1. SIMD Numeric Exceptions Signaled by SSE4.1 12-10
Table 12-2. Enhanced 32-Bit SIMD Multiply Supported by Intel® SSE4.1. 12-11
Table 12-3. Blend Field Size and Control Modes Supported by Intel® SSE4.1. 12-14
Table 12-4. Enhanced SIMD Integer MIN/MAX Instructions Supported by Intel® SSE4.1 12-14
Table 12-5. New SIMD Integer Conversions Supported by Intel® SSE4.1 12-15
Table 12-6. New SIMD Integer Conversions Supported by Intel® SSE4.1 12-16
Table 12-7. Enhanced SIMD Pack Support by Intel® SSE4.1 12-17
Table 12-8. Byte and 32-Bit Word Representation of a 128-Bit State 12-20

PAGE

Table 12-9. Matrix Representation of a 128-Bit State. . . . 12-20
Table 12-10. Little Endian Representation of a 128-Bit State. . . . 12-21
Table 12-11. Little Endian Representation of a 4x4 Byte Matrix 12-21
Table 12-12. The ShiftRows Transformation. . . . 12-22
Table 12-13. Look-up Table Associated with S-Box Transformation 12-22
Table 12-14. The InvShiftRows Transformation. . . . 12-23
Table 12-15. Look-up Table Associated with InvS-Box Transformation 12-24
Table 13-1. Format of the Legacy Region of an XSAVE Area 13-8
Table 14-1. Promoted SSSE3 and Intel® SSE, SSE2, SSE3, and SSE4 Instructions 14-3
Table 14-2. Promoted 256-Bit and 128-Bit Arithmetic Intel® AVX Instructions. . . . 14-9
Table 14-3. Promoted 256-Bit and 128-Bit Data Movement Intel® AVX Instructions 14-9
Table 14-4. 256-Bit Intel® AVX Instruction Enhancements. . . . 14-10
Table 14-5. Promotion of Legacy SIMD ISA to 128-Bit Arithmetic Intel® AVX Instructions 14-11
Table 14-6. 128-Bit Intel® AVX Instruction Enhancement. . . . 14-13
Table 14-7. Promotion of Legacy SIMD ISA to 128-Bit Non-Arithmetic Intel® AVX instruction. . . . 14-14
Table 14-8. Immediate Byte Encoding for 16-Bit Floating-Point Conversion Instructions 14-18
Table 14-9. Non-Numerical Behavior for VCVTPH2PS and VCVTPS2PH 14-18
Table 14-10. Invalid Operation for VCVTPH2PS and VCVTPS2PH 14-18
Table 14-12. Underflow Condition for VCVTPS2PH. . . . 14-19
Table 14-13. Overflow Condition for VCVTPS2PH. . . . 14-19
Table 14-14. Inexact Condition for VCVTPS2PH. . . . 14-19
Table 14-11. Denormal Condition Summary 14-19
Table 14-15. FMA Instructions 14-21
Table 14-16. Rounding Behavior of Zero Result in FMA Operation 14-23
Table 14-17. FMA Numeric Behavior 14-23
Table 14-18. Promoted Vector Integer SIMD Instructions in Intel® AVX2. . . . 14-26
Table 14-19. VEX-Only SIMD Instructions in Intel® AVX and AVX2. . . . 14-29
Table 14-20. New Primitive in Intel® AVX2 Instructions 14-30
Table 14-21. Alignment Faulting Conditions when Memory Access is Not Aligned 14-33
Table 14-22. Instructions Requiring Explicitly Aligned Memory 14-33
Table 14-23. Instructions Not Requiring Explicit Memory Alignment. . . . 14-34
Table 15-1. 512-Bit Instruction Groups in the Intel® AVX-512 Family 15-6
Table 15-2. Feature Flag Collection Required of 256/128 Bit Vector Lengths for Each Instruction Group 15-7
Table 15-3. Instruction Mnemonics That Do Not Support EVEX.128 Encoding. . . . 15-8
Table 15-4. Characteristics of Three Rounding Control Interfaces. . . . 15-12
Table 15-5. Static Rounding Mode 15-12
Table 15-6. SIMD Instructions Requiring Explicitly Aligned Memory. . . . 15-14
Table 15-7. Instructions Not Requiring Explicit Memory Alignment. . . . 15-14
Table 16-1. CPUID Enumeration of Intel® AVX10. . . . 16-1
Table 16-2. Intel® AVX-512 CPUID Feature Flags Included in Intel® AVX10 16-2
Table 17-1. RTM Abort Status Definition 17-5
Table 18-1. Indirect Branch Tracking State Machine. . . . 18-11
Table 20-1. I/O Instruction Serialization 20-6
Table 21-1. Runtime Mutable CPUID Fields Expected to Change During Normal Operation. . . . 21-3
Table 21-2. Runtime Mutable CPUID Fields That Should Remain Consistent. . . . 21-3
Table 21-3. Mapping of Brand Indices; and Intel 64 and IA-32 Processor Brand Strings. . . . 21-5
Table 21-4. Leaf 00H Maximum Input for Basic CPUID and Vendor ID 21-7
Table 21-5. Leaf 01H Output Registers 21-8
Table 21-6. Leaf 01H Version and Features Returned in EAX. . . . 21-8
Table 21-7. Processor Type Field 21-9
Table 21-8. Leaf 01H Version and Features Returned in EBX. . . . 21-9
Table 21-9. Leaf 01H Version and Features Returned in ECX. . . . 21-10
Table 21-10. Leaf 01H Version and Features Returned in EDX. . . . 21-11
Table 21-11. Leaf 02H TLB/Cache/Prefetch Information 21-15
Table 21-12. Encoding of CPUID Leaf 2 Descriptors 21-16
Table 21-13. Leaf 03H Processor Serial Number 21-20
Table 21-14. Leaf 04H Deterministic Cache Parameters 21-21
Table 21-15. Leaf 05H MONITOR and MWAIT Features 21-23
Table 21-16. Leaf 06H Thermal and Power Management Features. . . . 21-24

PAGE

Table 21-17. Leaf 07H Sub-Leaf (ECX=0) Output Registers 21-26
Table 21-18. Leaf 07H.00H Structured Extended Feature Flags Returned in EAX 21-26
Table 21-19. Leaf 07H.00H Structured Extended Feature Flags Returned in EBX 21-26
Table 21-20. Leaf 07H.00H Structured Extended Feature Flags Returned in ECX 21-27
Table 21-21. Leaf 07H.00H Structured Extended Feature Flags Returned in EDX 21-29
Table 21-22. Leaf 07H.01H Structured Extended Feature Flags Returned in EAX 21-31
Table 21-23. Leaf 07H.01H Structured Extended Feature Flags Returned in EBX 21-32
Table 21-24. Leaf 07H.01H Structured Extended Feature Flags Returned in ECX 21-32
Table 21-25. Leaf 07H.01H Structured Extended Feature Flags Returned in EDX 21-32
Table 21-26. Leaf 07H Sub-Leaf (ECX=2) Output Registers 21-33
Table 21-27. CPUID.07H.02H Extended Feature Information Provided in EDX[1] 21-34
Table 21-28. Leaf 08H Reserved 21-35
Table 21-29. Leaf 09H Direct Cache Access Information 21-36
Table 21-30. Leaf 0AH Architectural Performance Monitoring 21-37
Table 21-31. Leaf 0BH Extended Topology 21-39
Table 21-32. Hierarchy of Valid Domain Enumerations in CPUID.0BH:ECX[15:8] 21-40
Table 21-33. Leaf 0CH Reserved 21-41
Table 21-34. Leaf 0DH.00H Processor Extended State 21-42
Table 21-35. Leaf 0DH.01H Processor Extended State 21-43
Table 21-36. Leaf 0DH.SUB-LEAVES Processor Extended State 21-44
Table 21-37. Leaf 0EH Reserved 21-45
Table 21-38. Leaf 0FH.00H Intel® Resource Director Technology (Intel® RDT) Monitoring 21-46
Table 21-39. Leaf 0FH.01H Intel® Resource Director Technology (Intel® RDT) Monitoring 21-46
Table 21-40. Leaf 10H.00H Intel® Resource Director Technology (Intel® RDT) Allocation 21-48
Table 21-41. Leaf 10H.01H Intel® Resource Director Technology (Intel® RDT) Allocation 21-48
Table 21-42. Leaf 10H.02H Intel® Resource Director Technology (Intel® RDT) Allocation 21-49
Table 21-43. Leaf 10H.03H Intel® Resource Director Technology (Intel® RDT) Allocation 21-49
Table 21-44. Leaf 10H.05H Intel® Resource Director Technology (Intel® RDT) Allocation 21-50
Table 21-45. Leaf 10H.06H Intel® Resource Director Technology (Intel® RDT) Allocation 21-50
Table 21-46. Leaf 11H Reserved 21-51
Table 21-47. Leaf 12H.00H Intel® Software Guard Extensions (Intel® SGX) Capability 21-52
Table 21-48. Leaf 12H.01H Intel® Software Guard Extensions (Intel® SGX) Capability 21-52
Table 21-49. Leaf 12H.SUB-LEAF ENCODING TYPE EAX[3:0] = 0001B Intel® Software Guard Extensions (Intel® SGX) Capability 21-53
Table 21-50. Leaf 13H Reserved 21-55
Table 21-51. Leaf 14H.00H Intel® Processor Trace (Intel® PT) 21-56
Table 21-52. Leaf 14H.01H Intel® Processor Trace (Intel® PT) 21-57
Table 21-53. Leaf 15H Time Stamp Counter and Nominal Core Crystal Clock 21-59
Table 21-54. Leaf 16H Processor Frequency Information 21-60
Table 21-55. Leaf 17H.00H System-on-Chip Vendor Attribute 21-61
Table 21-56. Leaf 17H.01H System-on-Chip Vendor Attribute 21-61
Table 21-57. Leaf 17H.02H System-on-Chip Vendor Attribute 21-61
Table 21-58. Leaf 17H.03H System-on-Chip Vendor Attribute 21-62
Table 21-59. Leaf 17H.M>MAXSOCID_INDEX—RESERVED SUB-LEAVES System-on-Chip Vendor Attribute. 21-62
Table 21-60. Leaf 18H.00H Deterministic Address Translation Parameters 21-63
Table 21-61. Leaf 18H.ECX >= 1 Deterministic Address Translation Parameters 21-63
Table 21-62. Leaf 19H Key Locker 21-65
Table 21-63. Leaf 1AH Native Model ID Enumeration 21-66
Table 21-64. Leaf 1BH PCONFIG Information 21-67
Table 21-65. Leaf 1BH.OUTPUT REGISTERS FOR SUB-LEAVE TYPE TARGET IDENTIFIER (1) PCONFIG Information. 21-67
Table 21-66. Leaf 1CH Last Branch Records (LBR) Information 21-68
Table 21-67. Leaf 1DH.00H Tile Information 21-69
Table 21-68. Leaf 1DH.01H Tile Information 21-69
Table 21-69. Leaf 1EH.00H TMUL Information 21-70
Table 21-70. Leaf 1FH V2 Extended Topology 21-71
Table 21-71. Hierarchy of Valid Domain Enumerations in CPUID.1FH:ECX[15:8] 21-72
Table 21-72. Leaf 20H.00H Processor History Reset Information 21-73
Table 21-73. Leaf 22H Reserved 21-75
Table 21-74. Leaf 23H.00H Architectural Performance Monitoring Extended 21-76

PAGE

Table 21-75. Leaf 23H.01H Architectural Performance Monitoring Extended 21-76
Table 21-76. Leaf 23H.02H Architectural Performance Monitoring Extended 21-77
Table 21-77. Leaf 23H.03H Architectural Performance Monitoring Extended 21-77
Table 21-78. Leaf 23H.04H Architectural Performance Monitoring Extended 21-78
Table 21-79. Leaf 23H.05H Architectural Performance Monitoring Extended 21-78
Table 21-80. Leaf 24H.00H Converged Vector ISA 21-80
Table 21-81. Leaf 27H.00H Intel® Resource Director Technology (Intel® RDT) Asymmetric Monitoring 21-81
Table 21-82. Leaf 27H.01H Intel® Resource Director Technology (Intel® RDT) Asymmetric Monitoring 21-81
Table 21-83. Leaf 28H.00H Intel® Resource Director Technology (Intel® RDT) Asymmetric Allocation 21-83
Table 21-84. Leaf 28H.01H Intel® Resource Director Technology (Intel® RDT) Asymmetric Allocation 21-83
Table 21-85. Leaf 28H.02H Intel® Resource Director Technology (Intel® RDT) Asymmetric Allocation 21-84
Table 21-86. Leaf 28H.03H Intel® Resource Director Technology (Intel® RDT) Asymmetric Allocation 21-85
Table 21-87. Leaf 28H.05H Intel® Resource Director Technology (Intel® RDT) Asymmetric Allocation 21-85
Table 21-88. Leaf 28H.06H Intel® Resource Director Technology (Intel® RDT) Asymmetric Allocation 21-85
Table 21-89. Leaf 80000000H Maximum Input Value for Extended Function CPUID Information 21-87
Table 21-90. Leaf 80000001H Extended Processor Signature and Feature Bits 21-88
Table 21-91. Leaf 80000002H Processor Brand String (Bytes 0 to 15) 21-89
Table 21-92. Leaf 80000003H Processor brand string (Bytes 16 to 31) 21-90
Table 21-93. Leaf 80000004H Processor brand string (Bytes 32 to 47) 21-91
Table 21-94. Leaf 80000005H Reserved 21-92
Table 21-95. Leaf 80000006H Extended Function CPUID Information 21-93
Table 21-96. L2 Associativity Field Encodings 21-93
Table 21-97. Leaf 80000007H Extended Function CPUID Information 1 21-94
Table 21-98. Leaf 80000008H Extended Function CPUID Information 2 21-95
Table A-1. Codes Describing Flags A-1
Table A-2. EFLAGS Cross-Reference A-1
Table B-1. EFLAGS Condition Codes B-1
Table C-1. x87 FPU and SIMD Floating-Point Exceptions C-1
Table C-2. Exceptions Generated with x87 FPU Floating-Point Instructions C-1
Table C-3. Exceptions Generated with Intel® SSE Instructions C-3
Table C-4. Exceptions Generated with Intel® SSE2 Instructions C-5
Table C-5. Exceptions Generated with Intel® SSE3 Instructions C-7
Table C-6. Exceptions Generated with Intel® SSE4 Instructions C-8
Table D-1. ADDPS, ADDSS, SUBPS, SUBSS, MULPS, MULSS, DIVPS, DIVSS, ADDPD, ADDSD, SUBPD, SUBSD, MULPD, MULSD, DIVPD, DIVSD, ADDSUBPS, ADDSUBPD, HADDPS, HADDPD, HSUBPS, and HSUBPD D-5
Table D-2. CMPPS.EQ, CMPSS.EQ, CMPPS.ORD, CMPSS.ORD, CMPPD.EQ, CMPSD.EQ, CMPPD.ORD, and CMPSD.ORD D-6
Table D-3. CMPPS.NEQ, CMPSS.NEQ, CMPPS.UNORD, CMPSS.UNORD, CMPPD.NEQ, CMPSD.NEQ, CMPPD.UNORD, and CMPSD.UNORD D-6
Table D-4. CMPPS.LT, CMPSS.LT, CMPPS.LE, CMPSS.LE, CMPPD.LT, CMPSD.LT, CMPPD.LE, and CMPSD.LE D-6
Table D-5. CMPPS.NLT, CMPSS.NLT, CMPPS.NLE, CMPSS.NLE, CMPPD.NLT, CMPSD.NLT, CMPPD.NLE, and CMPSD.NLE D-7
Table D-6. COMISS and COMISD D-7
Table D-7. UCOMISS and UCOMISD D-7
Table D-8. CVTPS2PI, CVTSS2SI, CVTTPS2PI, CVTTSS2SI, CVTPD2PI, CVTSD2SI, CVTTPD2PI, CVTTSD2SI, CVTPS2DQ, CVTTPS2DQ, CVTPD2DQ, and CVTTPD2DQ D-7
Table D-9. MAXPS, MAXSS, MINPS, MINSS, MAXPD, MAXSD, MINPD, and MINSD D-8
Table D-10. SQRTPS, SQRTSS, SQRTPD, and SQRTSD D-8
Table D-11. CVTPS2PD and CVTSS2SD D-8
Table D-12. CVTPD2PS and CVTSD2SS D-8
Table D-13. #I - Invalid Operations D-9
Table D-14. #Z - Divide-by-Zero D-11
Table D-15. #D - Denormal Operand D-12
Table D-16. #O - Numeric Overflow D-13
Table D-17. #U - Numeric Underflow D-14
Table D-18. #P - Inexact Result (Precision) D-15
Table E-1. Error Code Definition of BNDSTATUS E-4
Table E-2. Intel® MPX Instruction Summary E-5
Table E-3. Effective Address Size of Intel® MPX Instructions with 67H Prefix E-10
Table E-4. Bounds Register INIT Behavior Due to BND Prefix with Branch Instructions E-11

CONTENTS

PAGE

CHAPTER 1
ABOUT THIS MANUAL

The Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 1: Basic Architecture (order number 253665) is part of a set that describes the architecture and programming environment of Intel® 64 and IA-32 architecture processors. Other volumes in this set are:

- The Intel® 64 and IA-32 Architectures Software Developer's Manual, Volumes 2A, 2B, 2C & 2D: Instruction Set Reference (order numbers 253666, 253667, 326018, and 334569).
- The Intel® 64 and IA-32 Architectures Software Developer's Manual, Volumes 3A, 3B, 3C & 3D: System Programming Guide (order numbers 253668, 253669, 326019, and 332831).
- The Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 4: Model-Specific Registers (order number 335592).

The Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 1, describes the basic architecture and programming environment of Intel 64 and IA-32 processors. The Intel® 64 and IA-32 Architectures Software Developer's Manual, Volumes 2A, 2B, 2C, & 2D, describe the instruction set of the processor and the opcode structure. These volumes apply to application programmers and to programmers who write operating systems or executives. The Intel® 64 and IA-32 Architectures Software Developer's Manual, Volumes 3A, 3B, 3C, & 3D, describe the operating-system support environment of Intel 64 and IA-32 processors. These volumes target operating-system and BIOS designers. In addition, the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3B, addresses the programming environment for classes of software that host operating systems. The Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 4, describes the model-specific registers of Intel 64 and IA-32 processors.

1.1 INTEL® 64 AND IA-32 PROCESSORS COVERED IN THIS MANUAL

This manual set includes information pertaining primarily to the most recent Intel 64 and IA-32 processors, which include:

- Pentium® processors
- P6 family processors
- Pentium® 4 processors
- Pentium® M processors
- Intel® Xeon® processors
- Pentium® D processors
- Pentium® processor Extreme Editions
- 64-bit Intel® Xeon® processors
- Intel® Core™ Duo processor
- Intel® Core™ Solo processor
- Dual-Core Intel® Xeon® processor LV
- Intel® Core™ 2 Duo processor
- Intel® Core™ 2 Quad processor Q6000 series
- Intel® Xeon® processor 3000, 3200 series
- Intel® Xeon® processor 5000 series
- Intel® Xeon® processor 5100, 5300 series
- Intel® Core™ 2 Extreme processor X7000 and X6800 series
- Intel® Core™ 2 Extreme processor QX6000 series
- Intel® Xeon® processor 7100 series

- Intel® Pentium® Dual-Core processor
- Intel® Xeon® processor 7200, 7300 series
- Intel® Xeon® processor 5200, 5400, 7400 series
- Intel® Core™ 2 Extreme processor QX9000 and X9000 series
- Intel® Core™ 2 Quad processor Q9000 series
- Intel® Core™ 2 Duo processor E8000, T9000 series
- Intel Atom® processor family
- Intel Atom® processors 200, 300, D400, D500, D2000, N200, N400, N2000, E2000, Z500, Z600, Z2000, C1000 series are built from 45 nm and 32 nm processes
- Intel® Core™ i7 processor
- Intel® Core™ i5 processor
- Intel® Xeon® processor E7-8800/4800/2800 product families
- Intel® Core™ i7-3930K processor
- 2nd generation Intel® Core™ i7-2xxx, Intel® Core™ i5-2xxx, Intel® Core™ i3-2xxx processor series
- Intel® Xeon® processor E3-1200 product family
- Intel® Xeon® processor E5-2400/1400 product family
- Intel® Xeon® processor E5-4600/2600/1600 product family
- 3rd generation Intel® Core™ processors
- Intel® Xeon® processor E3-1200 v2 product family
- Intel® Xeon® processor E5-2400/1400 v2 product families
- Intel® Xeon® processor E5-4600/2600/1600 v2 product families
- Intel® Xeon® processor E7-8800/4800/2800 v2 product families
- 4th generation Intel® Core™ processors
- The Intel® Core™ M processor family
- Intel® Core™ i7-59xx Processor Extreme Edition
- Intel® Core™ i7-49xx Processor Extreme Edition
- Intel® Xeon® processor E3-1200 v3 product family
- Intel® Xeon® processor E5-2600/1600 v3 product families
- 5th generation Intel® Core™ processors
- Intel® Xeon® processor D-1500 product family
- Intel® Xeon® processor E5 v4 family
- Intel Atom® processor X7-Z8000 and X5-Z8000 series
- Intel Atom® processor Z3400 series
- Intel Atom® processor Z3500 series
- 6th generation Intel® Core™ processors
- Intel® Xeon® processor E3-1500m v5 product family
- 7th generation Intel® Core™ processors
- Intel® Xeon Phi™ Processor 3200, 5200, 7200 Series
- Intel® Xeon® Scalable Processor Family
- 8th generation Intel® Core™ processors
- Intel® Xeon Phi™ Processor 7215, 7285, 7295 Series
- Intel® Xeon® E processors
- 9th generation Intel® Core™ processors
- 2nd generation Intel® Xeon® Scalable Processor Family

- 10th generation Intel® Core™ processors
- 11th generation Intel® Core™ processors
- 3rd generation Intel® Xeon® Scalable Processor Family
- 12th generation Intel® Core™ processors
- 13th generation Intel® Core™ processors
- 4th generation Intel® Xeon® Scalable Processor Family
- 5th generation Intel® Xeon® Scalable Processor Family
- Intel® Core™ Ultra 7 processors
- Intel® Xeon® 6 E-Core processors
- Intel® Xeon® 6 P-Core processors
- Intel® Series 2 Core™ Ultra processors
- Intel® Xeon® 6+ E-Core processors
- Intel® Series 3 Core™ processors
- Intel® Series 3 Core™ Ultra processors
- Intel® Atom® P6900 Processor

P6 family processors are IA-32 processors based on the P6 family microarchitecture. This includes the Pentium® Pro, Pentium® II, Pentium® III, and Pentium® III Xeon® processors.

The Pentium® 4, Pentium® D, and Pentium® processor Extreme Editions are based on the Intel NetBurst® microarchitecture. Most early Intel® Xeon® processors are based on the Intel NetBurst® microarchitecture. Intel Xeon processor 5000, 7100 series are based on the Intel NetBurst® microarchitecture.

The Intel® Core™ Duo, Intel® Core™ Solo and dual-core Intel® Xeon® processor LV are based on an improved Pentium® M processor microarchitecture.

The Intel® Xeon® processor 3000, 3200, 5100, 5300, 7200, and 7300 series, Intel® Pentium® dual-core, Intel® Core™ 2 Duo, Intel® Core™ 2 Quad, and Intel® Core™ 2 Extreme processors are based on Intel® Core™ microarchitecture.

The Intel® Xeon® processor 5200, 5400, 7400 series, Intel® Core™ 2 Quad processor Q9000 series, and Intel® Core™ 2 Extreme processors QX9000, X9000 series, Intel® Core™ 2 processor E8000 series are based on Enhanced Intel® Core™ microarchitecture.

The Intel Atom® processors 200, 300, D400, D500, D2000, N200, N400, N2000, E2000, Z500, Z600, Z2000, C1000 series are based on the Intel Atom® microarchitecture and supports Intel 64 architecture.

P6 family, Pentium® M, Intel® Core™ Solo, Intel® Core™ Duo processors, dual-core Intel® Xeon® processor LV, and early generations of Pentium 4 and Intel Xeon processors support IA-32 architecture. The Intel® Atom™ processor Z5xx series support IA-32 architecture.

The Intel® Xeon® processor 3000, 3200, 5000, 5100, 5200, 5300, 5400, 7100, 7200, 7300, 7400 series, Intel® Core™ 2 Duo, Intel® Core™ 2 Extreme, Intel® Core™ 2 Quad processors, Pentium® D processors, Pentium® Dual-Core processor, newer generations of Pentium 4 and Intel Xeon processor family support Intel® 64 architecture.

The Intel® Core™ i7 processor and Intel® Xeon® processor 3400, 5500, 7500 series are based on 45 nm Nehalem microarchitecture. Westmere microarchitecture is a 32 nm version of the Nehalem microarchitecture. Intel® Xeon® processor 5600 series, Intel Xeon processor E7 and various Intel Core i7, i5, i3 processors are based on the Westmere microarchitecture. These processors support Intel 64 architecture.

The Intel® Xeon® processor E5 family, Intel® Xeon® processor E3-1200 family, Intel® Xeon® processor E7-8800/4800/2800 product families, Intel® Core™ i7-3930K processor, and 2nd generation Intel® Core™ i7-2xxx, Intel® Core™ i5-2xxx, Intel® Core™ i3-2xxx processor series are based on the Sandy Bridge microarchitecture and support Intel 64 architecture.

The Intel® Xeon® processor E7-8800/4800/2800 v2 product families, Intel® Xeon® processor E3-1200 v2 product family and 3rd generation Intel® Core™ processors are based on the Ivy Bridge microarchitecture and support Intel 64 architecture.

The Intel® Xeon® processor E5-4600/2600/1600 v2 product families, Intel® Xeon® processor E5-2400/1400 v2 product families and Intel® Core™ i7-49xx Processor Extreme Edition are based on the Ivy Bridge-E microarchitecture and support Intel 64 architecture.

The Intel® Xeon® processor E3-1200 v3 product family and 4th Generation Intel® Core™ processors are based on the Haswell microarchitecture and support Intel 64 architecture.

The Intel® Xeon® processor E5-2600/1600 v3 product families and the Intel® Core™ i7-59xx Processor Extreme Edition are based on the Haswell-E microarchitecture and support Intel 64 architecture.

The Intel Atom® processor Z8000 series is based on the Airmont microarchitecture.

The Intel Atom® processor Z3400 series and the Intel Atom® processor Z3500 series are based on the Silvermont microarchitecture.

The Intel® Core™ M processor family, 5th generation Intel® Core™ processors, Intel® Xeon® processor D-1500 product family and the Intel® Xeon® processor E5 v4 family are based on the Broadwell microarchitecture and support Intel 64 architecture.

The Intel® Xeon® Scalable Processor Family, Intel® Xeon® processor E3-1500m v5 product family and 6th generation Intel® Core™ processors are based on the Skylake microarchitecture and support Intel 64 architecture.

The 7th generation Intel® Core™ processors are based on the Kaby Lake microarchitecture and support Intel 64 architecture.

The Intel Atom® processor C series, the Intel Atom® processor X series, the Intel® Pentium® processor J series, the Intel® Celeron® processor J series, and the Intel® Celeron® processor N series are based on the Goldmont microarchitecture.

The Intel® Xeon Phi™ Processor 3200, 5200, 7200 Series is based on the Knights Landing microarchitecture and supports Intel 64 architecture.

The Intel® Pentium® Silver processor series, the Intel® Celeron® processor J series, and the Intel® Celeron® processor N series are based on the Goldmont Plus microarchitecture.

The 8th generation Intel® Core™ processors, 9th generation Intel® Core™ processors, and Intel® Xeon® E processors are based on the Coffee Lake microarchitecture and support Intel 64 architecture.

The Intel® Xeon Phi™ Processor 7215, 7285, 7295 Series is based on the Knights Mill microarchitecture and supports Intel 64 architecture.

The 2nd generation Intel® Xeon® Scalable Processor Family is based on the Cascade Lake product and supports Intel 64 architecture.

Some 10th generation Intel® Core™ processors are based on the Ice Lake microarchitecture, and some are based on the Comet Lake microarchitecture; both support Intel 64 architecture.

Some 11th generation Intel® Core™ processors are based on the Tiger Lake microarchitecture, and some are based on the Rocket Lake microarchitecture; both support Intel 64 architecture.

Some 3rd generation Intel® Xeon® Scalable Processor Family processors are based on the Cooper Lake product, and some are based on the Ice Lake microarchitecture; both support Intel 64 architecture.

The 12th generation Intel® Core™ processors are based on the Alder Lake hybrid architecture and support Intel 64 architecture.

The 13th generation Intel® Core™ processors are based on the Raptor Lake hybrid architecture and support Intel 64 architecture.

The 4th generation Intel® Xeon® Scalable Processor Family is based on Sapphire Rapids microarchitecture and supports Intel 64 architecture.

The 5th generation Intel® Xeon® Scalable Processor Family is based on Emerald Rapids microarchitecture and supports Intel 64 architecture.

The Intel® Core™ Ultra 7 processor is based on Meteor Lake hybrid architecture and supports Intel 64 architecture.

The Intel® Xeon® 6 E-core processor is based on Sierra Forest microarchitecture and supports Intel 64 architecture.

The Intel® Xeon® 6 P-core processor is based on Granite Rapids microarchitecture and supports Intel 64 architecture.

The Intel® Xeon® 6+ E-core processor is based on Clearwater Forest microarchitecture and supports Intel 64 architecture.

The Intel® Series 2 Core™ Ultra processors are based on Lunar Lake or Arrow Lake hybrid architectures and support Intel 64 architecture.

The Intel® Series 3 Core™ processors are based on Wildcat Lake hybrid architecture and support Intel 64 architecture.

The Intel® Series 3 Core™ Ultra processors are based on Panther Lake hybrid architecture and support Intel 64 architecture.

IA-32 architecture is the instruction set architecture and programming environment for Intel's 32-bit microprocessors. Intel® 64 architecture is the instruction set architecture and programming environment which is the superset of Intel’s 32-bit and 64-bit architectures. It is compatible with the IA-32 architecture.

1.2 OVERVIEW OF VOLUME 1: BASIC ARCHITECTURE

A description of this manual’s content follows:

Chapter 1 — About This Manual. Gives an overview of all volumes of the Intel® 64 and IA-32 Architectures Software Developer’s Manual. It also describes the notational conventions in these manuals and lists related Intel manuals and documentation of interest to programmers and hardware designers.

Chapter 2 — Intel® 64 and IA-32 Architectures. Introduces the Intel 64 and IA-32 architectures along with the families of Intel processors that are based on these architectures. It also gives an overview of the common features found in these processors and brief history of the Intel 64 and IA-32 architectures.

Chapter 3 — Basic Execution Environment. Introduces the models of memory organization and describes the register set used by applications.

Chapter 4 — Data Types. Describes the data types and addressing modes recognized by the processor; provides an overview of real numbers and floating-point formats and of floating-point exceptions.

Chapter 5 — Instruction Set Summary. Lists all Intel 64 and IA-32 instructions, divided into technology groups.

Chapter 6 — Procedure Calls, Interrupts, and Exceptions. Describes the procedure stack and mechanisms provided for making procedure calls and for servicing interrupts and exceptions.

Chapter 7 — Programming with General-Purpose Instructions. Describes basic load and store, program control, arithmetic, and string instructions that operate on basic data types, general-purpose and segment registers; also describes system instructions that are executed in protected mode.

Chapter 8 — Programming with the x87 FPU. Describes the x87 floating-point unit (FPU), including floating-point registers and data types; gives an overview of the floating-point instruction set and describes the processor's floating-point exception conditions.

Chapter 9 — Programming with Intel® MMX™ Technology. Describes Intel MMX technology, including MMX registers and data types; also provides an overview of the MMX instruction set.

Chapter 10 — Programming with Intel® Streaming SIMD Extensions (Intel® SSE). Describes SSE extensions, including XMM registers, the MXCSR register, and packed single precision floating-point data types; provides an overview of the SSE instruction set and gives guidelines for writing code that accesses the SSE extensions.

Chapter 11 — Programming with Intel® Streaming SIMD Extensions 2 (Intel® SSE2). Describes SSE2 extensions, including XMM registers and packed double precision floating-point data types; provides an overview of the SSE2 instruction set and gives guidelines for writing code that accesses SSE2 extensions. This chapter also describes SIMD floating-point exceptions that can be generated with SSE and SSE2 instructions. It also provides general guidelines for incorporating support for SSE and SSE2 extensions into operating system and applications code.

Chapter 12 — Programming with Intel® Streaming SIMD Extensions 3 (Intel® SSE3), Supplemental Streaming SIMD Extensions 3 (SSSE3), Intel® Streaming SIMD Extensions 4 (Intel® SSE4) and Intel®

AES New Instructions (Intel® AES-NI). Provides an overview of the SSE3 instruction set, Supplemental SSE3, SSE4, AESNI instructions, and guidelines for writing code that access these extensions.

Chapter 13 — Managing State Using the XSAVE Feature Set. Describes the XSAVE feature set instructions and explains how software can enable the XSAVE feature set and XSAVE-enabled features.

Chapter 14 — Programming with Intel® AVX, FMA, and Intel® AVX2. Provides an overview of the Intel® AVX instruction set, FMA, and Intel® AVX2 extensions and gives guidelines for writing code that access these extensions.

Chapter 15 — Programming with Intel® AVX-512. Provides an overview of the Intel® AVX-512 instruction set extensions and gives guidelines for writing code that access these extensions.

Chapter 16 — Programming with Intel® AVX10. Provides an overview of the Intel® AVX10 instruction set extensions and gives guidelines for writing code that access these extensions.

Chapter 17 — Programming with Intel® Transactional Synchronization Extensions. Describes the instruction extensions that support lock elision techniques to improve the performance of multi-threaded software with contended locks.

Chapter 18 — Control-flow Enforcement Technology. Provides an overview of the Control-flow Enforcement Technology (CET) and gives guidelines for writing code that access these extensions.

Chapter 19 — Programming with Intel® Advanced Matrix Extensions. Provides an overview of the Intel® Advanced Matrix Extensions and gives guidelines for writing code that access these extensions.

Chapter 20 — Input/Output. Describes the processor's I/O mechanism, including I/O port addressing, I/O instructions, and I/O protection mechanisms.

Chapter 21 — Processor Identification and Feature Determination. Describes how to determine the CPU type and features available in the processor.

Appendix A — EFLAGS Cross-Reference. Summarizes how the IA-32 instructions affect the flags in the EFLAGS register.

Appendix B — EFLAGS Condition Codes. Summarizes how conditional jump, move, and 'byte set on condition code' instructions use condition code flags (OF, CF, ZF, SF, and PF) in the EFLAGS register.

Appendix C — Floating-Point Exceptions Summary. Summarizes exceptions raised by the x87 FPU floating-point and SSE/SSE2/SSE3 floating-point instructions.

Appendix D — Guidelines for Writing SIMD Floating-Point Exception Handlers. Gives guidelines for writing exception handlers for exceptions generated by SSE/SSE2/SSE3 floating-point instructions.

Appendix E — Intel® Memory Protection Extensions. Provides an overview of the Intel® Memory Protection Extensions, a feature that has been deprecated and will not be available on future processors.

1.3 NOTATIONAL CONVENTIONS

This manual uses specific notation for data-structure formats, for symbolic representation of instructions, and for hexadecimal and binary numbers. This notation is described below.

1.3.1 Bit and Byte Order

In illustrations of data structures in memory, smaller addresses appear toward the bottom of the figure; addresses increase toward the top. Bit positions are numbered from right to left. The numerical value of a set bit is equal to two raised to the power of the bit position. Intel 64 and IA-32 processors are "little endian" machines; this means the bytes of a word are numbered starting from the least significant byte. See Figure 1-1.

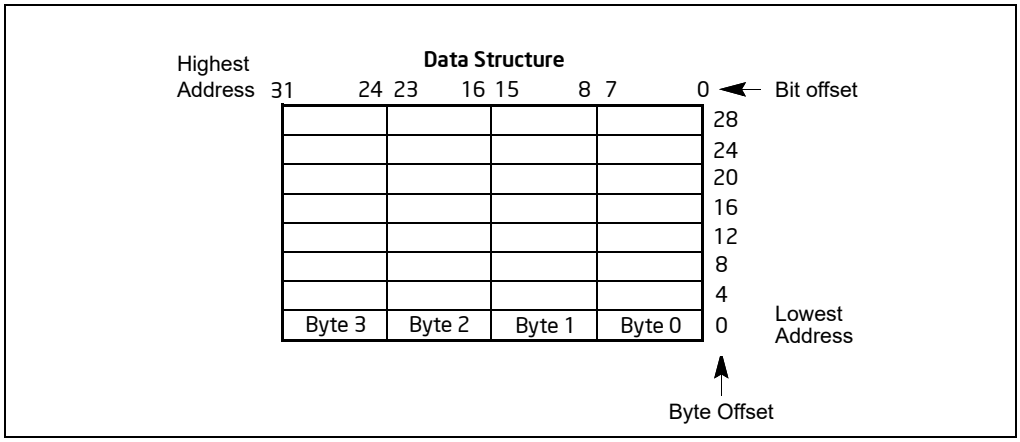


Figure 1-1. Bit and Byte Order

1.3.2 Reserved Bits and Software Compatibility

In many register and memory layout descriptions, certain bits are marked as **reserved**. When bits are marked as reserved, it is essential for compatibility with future processors that software treat these bits as having a future, though unknown, effect. The behavior of reserved bits should be regarded as not only undefined, but unpredictable.

Software should follow these guidelines in dealing with reserved bits:

- Do not depend on the states of any reserved bits when testing the values of registers that contain such bits. Mask out the reserved bits before testing.
- Do not depend on the states of any reserved bits when storing to memory or to a register.
- Do not depend on the ability to retain information written into any reserved bits.
- When loading a register, always load the reserved bits with the values indicated in the documentation, if any, or reload them with values previously read from the same register.

NOTE

Avoid any software dependence upon the state of reserved bits in Intel 64 and IA-32 registers. Depending upon the values of reserved register bits will make software dependent upon the unspecified manner in which the processor handles these bits. Programs that depend upon reserved values risk incompatibility with future processors.

1.3.2.1 Instruction Operands

When instructions are represented symbolically, a subset of the IA-32 assembly language is used. In this subset, an instruction has the following format:

```
label: mnemonic argument1, argument2, argument3
```

where:

- A **label** is an identifier which is followed by a colon.
- A **mnemonic** is a reserved name for a class of instruction opcodes which have the same function.
- The operands **argument1**, **argument2**, and **argument3** are optional. There may be from zero to three operands, depending on the opcode. When present, they take the form of either literals or identifiers for data items. Operand identifiers are either reserved names of registers or are assumed to be assigned to data items declared in another part of the program (which may not be shown in the example).

When two operands are present in an arithmetic or logical instruction, the right operand is the source and the left operand is the destination.

For example:

```
LOADREG: MOV EAX, SUBTOTAL
```

In this example, LOADREG is a label, MOV is the mnemonic identifier of an opcode, EAX is the destination operand, and SUBTOTAL is the source operand. Some assembly languages put the source and destination in reverse order.

1.3.3 Hexadecimal and Binary Numbers

Base 16 (hexadecimal) numbers are represented by a string of hexadecimal digits followed by the character H (for example, 0F82EH). A hexadecimal digit is a character from the following set: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, and F.

Base 2 (binary) numbers are represented by a string of 1s and 0s, sometimes followed by the character B (for example, 1010B). The "B" designation is only used in situations where confusion as to the type of number might arise.

1.3.4 Segmented Addressing

The processor uses byte addressing. This means memory is organized and accessed as a sequence of bytes. Whether one or more bytes are being accessed, a byte address is used to locate the byte or bytes memory. The range of memory that can be addressed is called an **address space**.

The processor also supports segmented addressing. This is a form of addressing where a program may have many independent address spaces, called **segments**. For example, a program can keep its code (instructions) and stack in separate segments. Code addresses would always refer to the code space, and stack addresses would always refer to the stack space. The following notation is used to specify a byte address within a segment:

```
Segment-register:Byte-address
```

For example, the following segment address identifies the byte at address FF79H in the segment pointed by the DS register:

```
DS:FF79H
```

The following segment address identifies an instruction address in the code segment. The CS register points to the code segment and the EIP register contains the address of the instruction.

```
CS:EIP
```

1.3.5 A Syntax for CPUID, CR, and MSR Values

Obtain feature flags, status, and system information by using the CPUID instruction, by checking control register bits, and by reading model-specific registers. See Figure 1-2 for details on the syntax that represents this information.

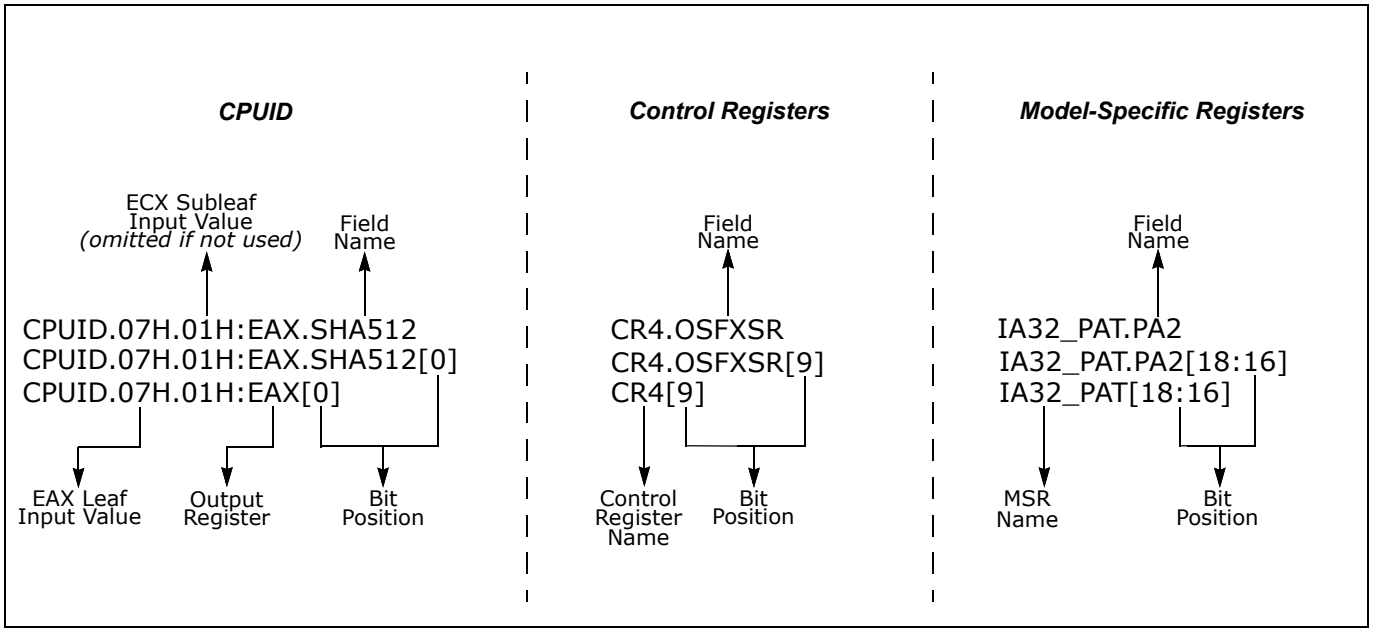


Figure 1-2. Syntax for CPUID, CR, and MSR Data Presentation

1.3.6 Exceptions

An exception is an event that typically occurs when an instruction causes an error. For example, an attempt to divide by zero generates an exception. However, some exceptions, such as breakpoints, occur under other conditions. Some types of exceptions may provide error codes. An error code reports additional information about the error. An example of the notation used to show an exception and error code is shown below:

```
#PF(fault code)
```

This example refers to a page-fault exception under conditions where an error code naming a type of fault is reported. Under some conditions, exceptions that produce error codes may not be able to report an accurate code. In this case, the error code is zero, as shown below for a general-protection exception:

```
#GP(0)
```

1.4 RELATED LITERATURE

Literature related to Intel 64 and IA-32 processors is listed and viewable on-line at:

https://software.intel.com/en-us/articles/intel-sdm

See also:

- The latest security information on Intel® products:
  https://www.intel.com/content/www/us/en/security-center/default.html
- Software developer resources, guidance, and insights for security advisories:
  https://software.intel.com/security-software-guidance/
- The data sheet for a particular Intel 64 or IA-32 processor
- The specification update for a particular Intel 64 or IA-32 processor
- Intel® C++ Compiler documentation and online help:
  http://software.intel.com/en-us/articles/intel-compilers/
- Intel® Fortran Compiler documentation and online help:
  http://software.intel.com/en-us/articles/intel-compilers/

- Intel® Software Development Tools:
  https://software.intel.com/en-us/intel-sdp-home
- Intel® 64 and IA-32 Architectures Software Developer's Manual (in one, four or ten volumes):
  https://software.intel.com/en-us/articles/intel-sdm
- Intel® 64 and IA-32 Architectures Optimization Reference Manual:
  https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm/intel64-and-ia32-architectures-optimization.html
- Intel® Trusted Execution Technology Measured Launched Environment Developer's Guide:

  http://www.intel.com/content/www/us/en/software-developers/intel-txt-software-development-guide.html
- Intel® Software Guard Extensions (Intel® SGX) Information:
  https://software.intel.com/en-us/isa-extensions/intel-sgx
- Developing Multi-threaded Applications: A Platform Consistent Approach:
  https://software.intel.com/sites/default/files/article/147714/51534-developing-multithreaded-applications.pdf
- Using Spin-Loops on Intel® Pentium® 4 Processor and Intel® Xeon® Processor:
  https://software.intel.com/sites/default/files/22/30/25602
- Performance Monitoring Unit Sharing Guide:
  http://software.intel.com/file/30388
- Intel® Trust Domain CPU Architectural Extensions:
  https://www.intel.com/content/www/us/en/developer/tools/trust-domain-extensions/documentation.html

Literature related to select features in future Intel processors are available at:

- Intel® Architecture Instruction Set Extensions Programming Reference:
  https://software.intel.com/en-us/isa-extensions

More relevant links are:

- Intel® Developer Zone:

  https://software.intel.com/en-us
- Developer centers:

  http://www.intel.com/content/www/us/en/hardware-developers/developer-centers.html
- Processor support general link:

  http://www.intel.com/support/processors/
- Intel® Hyper-Threading Technology (Intel® HT Technology):

  http://www.intel.com/technology/platform-technology/hyper-threading/index.htm
- Microarchitectural Data Sampling

  https://www.intel.com/content/www/us/en/developer/articles/technical/software-security-guidance/technical-documentation/intel-analysis-microarchitectural-data-sampling.html
- Hardware Features and Behavior Related to Speculative Execution

  https://www.intel.com/content/www/us/en/developer/articles/technical/software-security-guidance/technical-documentation/hardware-behavior-related-to-speculative-execution.html

CHAPTER 2
INTEL® 64 AND IA-32 ARCHITECTURES

2.1 BRIEF HISTORY OF INTEL® 64 AND IA-32 ARCHITECTURES

The following sections provide a summary of the major technical evolutions from IA-32 to Intel 64 architecture: starting from the Intel 8086 processor to the latest Intel® Core® 2 Duo, Core 2 Quad and Intel Xeon processor 5300 and 7300 series. Object code created for processors released as early as 1978 still executes on the latest processors in the Intel 64 and IA-32 architecture families.

2.1.1 16-Bit Processors and Segmentation (1978)

The IA-32 architecture family was preceded by 16-bit processors, the 8086 and 8088. The 8086 has 16-bit registers and a 16-bit external data bus, with 20-bit addressing giving a 1-MByte address space. The 8088 is similar to the 8086 except it has an 8-bit external data bus.

The 8086/8088 introduced segmentation to the IA-32 architecture. With segmentation, a 16-bit segment register contains a pointer to a memory segment of up to 64 KBytes. Using four segment registers at a time, 8086/8088 processors are able to address up to 256 KBytes without switching between segments. The 20-bit addresses that can be formed using a segment register and an additional 16-bit pointer provide a total address range of 1 MByte.

2.1.2 The Intel® 286 Processor (1982)

The Intel 286 processor introduced protected mode operation into the IA-32 architecture. Protected mode uses the segment register content as selectors or pointers into descriptor tables. Descriptors provide 24-bit base addresses with a physical memory size of up to 16 MBytes, support for virtual memory management on a segment swapping basis, and a number of protection mechanisms. These mechanisms include:

- Segment limit checking.
- Read-only and execute-only segment options.
- Four privilege levels.

2.1.3 The Intel386™ Processor (1985)

The Intel386 processor was the first 32-bit processor in the IA-32 architecture family. It introduced 32-bit registers for use both to hold operands and for addressing. The lower half of each 32-bit Intel386 register retains the properties of the 16-bit registers of earlier generations, permitting backward compatibility. The processor also provides a virtual-8086 mode that allows for even greater efficiency when executing programs created for 8086/8088 processors.

In addition, the Intel386 processor has support for:

- A 32-bit address bus that supports up to 4-GBytes of physical memory.
- A segmented-memory model and a flat memory model.
- Paging, with a fixed 4-KByte page size providing a method for virtual memory management.
- Support for parallel stages.

2.1.4 The Intel486™ Processor (1989)

The Intel486™ processor added more parallel execution capability by expanding the Intel386 processor's instruction decode and execution units into five pipelined stages. Each stage operates in parallel with the others on up to five instructions in different stages of execution.

In addition, the processor added:

- An 8-KByte on-chip first-level cache that increased the percent of instructions that could execute at the scalar rate of one per clock.
- An integrated x87 FPU.
- Power saving and system management capabilities.

2.1.5 The Intel® Pentium® Processor (1993)

The introduction of the Intel Pentium processor added a second execution pipeline to achieve superscalar performance (two pipelines, known as u and v, together can execute two instructions per clock). The on-chip first-level cache doubled, with 8 KBytes devoted to code and another 8 KBytes devoted to data. The data cache uses the MESI protocol to support more efficient write-back cache in addition to the write-through cache previously used by the Intel486 processor. Branch prediction with an on-chip branch table was added to increase performance in looping constructs.

In addition, the processor added:

- Extensions to make the virtual-8086 mode more efficient and allow for 4-MByte as well as 4-KByte pages.
- Internal data paths of 128 and 256 bits add speed to internal data transfers.
- Burstable external data bus was increased to 64 bits.
- An APIC to support systems with multiple processors.
- A dual processor mode to support glueless two processor systems.

A subsequent stepping of the Pentium family introduced Intel MMX technology (the Pentium Processor with MMX technology). Intel MMX technology uses the single-instruction, multiple-data (SIMD) execution model to perform parallel computations on packed integer data contained in 64-bit registers.

See Section 2.2.7, “SIMD Instructions.”

2.1.6 The P6 Family of Processors (1995—1999)

The P6 family of processors was based on a superscalar microarchitecture that set new performance standards; see also Section 2.2.1, “P6 Family Microarchitecture.” One of the goals in the design of the P6 family microarchitecture was to exceed the performance of the Pentium processor significantly while using the same 0.6-micrometer, four-layer, metal BICMOS manufacturing process. Members of this family include the following:

- The **Intel Pentium Pro processor** is three-way superscalar. Using parallel processing techniques, the processor is able on average to decode, dispatch, and complete execution of (retire) three instructions per clock cycle. The Pentium Pro introduced the dynamic execution (micro-data flow analysis, out-of-order execution, superior branch prediction, and speculative execution) in a superscalar implementation. The processor was further enhanced by its caches. It has the same two on-chip 8-KByte 1st-Level caches as the Pentium processor and an additional 256-KByte Level 2 cache in the same package as the processor.
- The **Intel Pentium II processor** added Intel MMX technology to the P6 family processors along with new packaging and several hardware enhancements. The processor core is packaged in the single edge contact cartridge (SECC). The Level l data and instruction caches were enlarged to 16 KBytes each, and Level 2 cache sizes of 256 KBytes, 512 KBytes, and 1 MBytes are supported. A half-frequency backside bus connects the Level 2 cache to the processor. Multiple low-power states such as AutoHALT, Stop-Grant, Sleep, and Deep Sleep are supported to conserve power when idling.
- The **Pentium II Xeon processor** combined the premium characteristics of previous generations of Intel processors. This includes: 4-way, 8-way (and up) scalability and a 2 MBytes 2nd-Level cache running on a full-frequency backside bus.
- The **Intel Celeron processor** family focused on the value PC market segment. Its introduction offers an integrated 128 KBytes of Level 2 cache and a plastic pin grid array (P.P.G.A.) form factor to lower system design cost.
- The **Intel Pentium III processor** introduced the Streaming SIMD Extensions (SSE) to the IA-32 architecture. SSE extensions expand the SIMD execution model introduced with the Intel MMX technology by providing a

new set of 128-bit registers and the ability to perform SIMD operations on packed single precision floating-point values. See Section 2.2.7, "SIMD Instructions."

- The **Pentium III Xeon processor** extended the performance levels of the IA-32 processors with the enhancement of a full-speed, on-die, and Advanced Transfer Cache.

2.1.7 The Intel® Pentium® 4 Processor Family (2000—2006)

The Intel Pentium 4 processor family is based on Intel NetBurst microarchitecture; see Section 2.2.2, "Intel NetBurst® Microarchitecture."

The Intel Pentium 4 processor introduced Streaming SIMD Extensions 2 (SSE2); see Section 2.2.7, "SIMD Instructions." The Intel Pentium 4 processor 3.40 GHz, supporting Hyper-Threading Technology introduced Streaming SIMD Extensions 3 (SSE3); see Section 2.2.7, "SIMD Instructions."

Intel 64 architecture was introduced in the Intel Pentium 4 Processor Extreme Edition supporting Hyper-Threading Technology and in the Intel Pentium 4 Processor 6xx and 5xx sequences.

Intel® Virtualization Technology (Intel® VT) was introduced in the Intel Pentium 4 processor 672 and 662.

2.1.8 The Intel® Xeon® Processor (2001—2007)

Intel Xeon processors (with exception for dual-core Intel Xeon processor LV, Intel Xeon processor 5100 series) are based on the Intel NetBurst microarchitecture; see Section 2.2.2, "Intel NetBurst® Microarchitecture." As a family, this group of IA-32 processors (more recently Intel 64 processors) is designed for use in multi-processor server systems and high-performance workstations.

The Intel Xeon processor MP introduced support for Intel® Hyper-Threading Technology; see Section 2.2.8, "Intel® Hyper-Threading Technology."

The 64-bit Intel Xeon processor 3.60 GHz (with an 800 MHz System Bus) was used to introduce Intel 64 architecture. The Dual-Core Intel Xeon processor includes dual core technology. The Intel Xeon processor 70xx series includes Intel Virtualization Technology.

The Intel Xeon processor 5100 series introduces power-efficient, high performance Intel Core microarchitecture. This processor is based on Intel 64 architecture; it includes Intel Virtualization Technology and dual-core technology. The Intel Xeon processor 3000 series are also based on Intel Core microarchitecture. The Intel Xeon processor 5300 series introduces four processor cores in a physical package, they are also based on Intel Core microarchitecture.

2.1.9 The Intel® Pentium® M Processor (2003—2006)

The Intel Pentium M processor family is a high performance, low power mobile processor family with microarchitectural enhancements over previous generations of IA-32 Intel mobile processors. This family is designed for extending battery life and seamless integration with platform innovations that enable new usage models (such as extended mobility, ultra thin form-factors, and integrated wireless networking).

Its enhanced microarchitecture includes:

- Support for Intel Architecture with Dynamic Execution.
- A high performance, low-power core manufactured using Intel's advanced process technology with copper interconnect.
- On-die, primary 32-KByte instruction cache and 32-KByte write-back data cache.
- On-die, second-level cache (up to 2 MByte) with Advanced Transfer Cache Architecture.
- Advanced Branch Prediction and Data Prefetch Logic.
- Support for MMX technology, Streaming SIMD instructions, and the SSE2 instruction set.
- A 400 or 533 MHz, Source-Synchronous Processor System Bus.
- Advanced power management using Enhanced Intel SpeedStep® technology.

2.1.10 The Intel® Pentium® Processor Extreme Edition (2005)

The Intel Pentium processor Extreme Edition introduced dual-core technology. This technology provides advanced hardware multi-threading support. The processor is based on Intel NetBurst microarchitecture and supports Intel SSE, SSE2, SSE3, Intel Hyper-Threading Technology, and Intel 64 architecture.

See also:

- Section 2.2.2, "Intel NetBurst® Microarchitecture."
- Section 2.2.3, "Intel® Core™ Microarchitecture."
- Section 2.2.7, "SIMD Instructions."
- Section 2.2.8, "Intel® Hyper-Threading Technology."
- Section 2.2.9, "Multi-Core Technology."
- Section 2.2.10, "Intel® 64 Architecture."

2.1.11 The Intel® Core™ Duo and Intel® Core™ Solo Processors (2006—2007)

The Intel Core Duo processor offers power-efficient, dual-core performance with a low-power design that extends battery life. This family and the single-core Intel Core Solo processor offer microarchitectural enhancements over Pentium M processor family.

Its enhanced microarchitecture includes:

- Intel® Smart Cache which allows for efficient data sharing between two processor cores.
- Improved decoding and SIMD execution.
- Intel® Dynamic Power Coordination and Enhanced Intel® Deeper Sleep to reduce power consumption.
- Intel® Advanced Thermal Manager which features digital thermal sensor interfaces.
- Support for power-optimized 667 MHz bus.

The dual-core Intel Xeon processor LV is based on the same microarchitecture as Intel Core Duo processor, and supports IA-32 architecture.

2.1.12 The Intel® Xeon® Processor 5100, 5300 Series, and Intel® Core™ 2 Processor Family (2006)

The Intel Xeon processor 3000, 3200, 5100, 5300, and 7300 series, Intel Pentium Dual-Core, Intel Core 2 Extreme, Intel Core 2 Quad processors, and Intel Core 2 Duo processor family support Intel 64 architecture; they are based on the high-performance, power-efficient Intel® Core microarchitecture built on 65 nm process technology. The Intel Core microarchitecture includes the following innovative features:

- Intel® Wide Dynamic Execution to increase performance and execution throughput.
- Intel® Intelligent Power Capability to reduce power consumption.
- Intel® Advanced Smart Cache which allows for efficient data sharing between two processor cores.
- Intel® Smart Memory Access to increase data bandwidth and hide latency of memory accesses.
- Intel® Advanced Digital Media Boost which improves application performance using multiple generations of Streaming SIMD extensions.

The Intel Xeon processor 5300 series, Intel Core 2 Extreme processor QX6800 series, and Intel Core 2 Quad processors support Intel quad-core technology.

2.1.13 The Intel® Xeon® Processor 5200, 5400, 7400 Series, and Intel® Core™ 2 Processor Family (2007)

The Intel Xeon processor 5200, 5400, and 7400 series, Intel Core 2 Quad processor Q9000 Series, Intel Core 2 Duo processor E8000 series support Intel 64 architecture; they are based on the Enhanced Intel® Core microarchitec-

ture using 45 nm process technology. The Enhanced Intel Core microarchitecture provides the following improved features:

- A radix-16 divider, faster OS primitives further increases the performance of Intel® Wide Dynamic Execution.
- Improves Intel® Advanced Smart Cache with Up to 50% larger level-two cache and up to 50% increase in way-set associativity.
- A 128-bit shuffler engine significantly improves the performance of Intel® Advanced Digital Media Boost and SSE4.

The Intel Xeon processor 5400 series and the Intel Core 2 Quad processor Q9000 Series support Intel quad-core technology. The Intel Xeon processor 7400 series offers up to six processor cores and an L3 cache up to 16 MBytes.

2.1.14 The Intel Atom® Processor Family (2008)

The first generation of Intel Atom® processors are built on 45 nm process technology. They are based on a new microarchitecture, Intel Atom® microarchitecture, which is optimized for ultra low power devices. The Intel Atom® microarchitecture features two in-order execution pipelines that minimize power consumption, increase battery life, and enable ultra-small form factors. The initial Intel Atom Processor family and subsequent generations including Intel Atom processor D2000, N2000, E2000, Z2000, C1000 series provide the following features:

- Enhanced Intel® SpeedStep® Technology.
- Intel® Hyper-Threading Technology.
- Deep Power Down Technology with Dynamic Cache Sizing.
- Support for instruction set extensions up to and including Supplemental Streaming SIMD Extensions 3 (SSSE3).
- Support for Intel® Virtualization Technology.
- Support for Intel® 64 Architecture (excluding Intel Atom processor Z5xx Series).

2.1.15 The Intel Atom® Processor Family Based on Silvermont Microarchitecture (2013)

Intel Atom Processor C2xxx, E3xxx, S1xxx series are based on the Silvermont microarchitecture. Processors based on the Silvermont microarchitecture support instruction set extensions up to and including SSE4.2, AESNI, and PCLMULQDQ.

2.1.16 The Intel® Core™ i7 Processor Family (2008)

The Intel Core i7 processor 900 series supports Intel 64 architecture, and is based on Nehalem microarchitecture using 45 nm process technology. The Intel Core i7 processor and Intel Xeon processor 5500 series include the following features:

- Intel® Turbo Boost Technology converts thermal headroom into higher performance.
- Intel® HyperThreading Technology in conjunction with Quadcore to provide four cores and eight threads.
- Dedicated power control unit to reduce active and idle power consumption.
- Integrated memory controller on the processor supporting three channels of DDR3 memory.
- 8 MB inclusive Intel® Smart Cache.
- Intel® QuickPath interconnect (QPI) providing point-to-point link to chipset.
- Support for SSE4.2 and SSE4.1 instruction sets.
- Second generation Intel Virtualization Technology.

2.1.17 The Intel® Xeon® Processor 7500 Series (2010)

The Intel Xeon processor 7500 and 6500 series are based on Nehalem microarchitecture using 45 nm process technology. These processors support the same features described in Section 2.1.16, plus the following features:

- Up to eight cores per physical processor package.
- Up to 24 MB inclusive Intel® Smart Cache.
- Provides Intel® Scalable Memory Interconnect (Intel® SMI) channels with Intel® 7500 Scalable Memory Buffer to connect to system memory.
- Advanced RAS supporting software recoverable machine check architecture.

2.1.18 2010 Intel® Core™ Processor Family (2010)

The 2010 Intel Core processor family spans Intel Core i7, i5, and i3 processors. These processors are based on Westmere microarchitecture using 32 nm process technology. The features can include:

- Deliver smart performance using Intel Hyper-Threading Technology plus Intel Turbo Boost Technology.
- Enhanced Intel Smart Cache and integrated memory controller.
- Intelligent power gating.
- Repartitioned platform with on-die integration of 45 nm integrated graphics.
- Range of instruction set support up to AESNI, PCLMULQDQ, SSE4.2 and SSE4.1.

2.1.19 The Intel® Xeon® Processor 5600 Series (2010)

The Intel Xeon processor 5600 series are based on Westmere microarchitecture using 32 nm process technology. They support the same features described in Section 2.1.16, plus the following features:

- Up to six cores per physical processor package.
- Up to 12 MB enhanced Intel® Smart Cache.
- Support for AESNI, PCLMULQDQ, SSE4.2 and SSE4.1 instruction sets.
- Flexible Intel Virtualization Technologies across processor and I/O.

2.1.20 The Second Generation Intel® Core™ Processor Family (2011)

The Second Generation Intel Core processor family spans Intel Core i7, i5, and i3 processors based on the Sandy Bridge microarchitecture. These processors are built from 32 nm process technology and have features including:

- Intel Turbo Boost Technology for Intel Core i5 and i7 processors.
- Intel Hyper-Threading Technology.
- Enhanced Intel Smart Cache and integrated memory controller.
- Processor graphics and built-in visual features like Intel® Quick Sync Video, Intel® Insider™, etc.
- Range of instruction set support up to AVX, AESNI, PCLMULQDQ, SSE4.2 and SSE4.1.

The Intel Xeon processor E3-1200 product family is also based on the Sandy Bridge microarchitecture.

The Intel Xeon processor E5-2400/1400 product families are based on the Sandy Bridge-EP microarchitecture.

The Intel Xeon processor E5-4600/2600/1600 product families are based on the Sandy Bridge-EP microarchitecture and provide support for multiple sockets.

2.1.21 The Third Generation Intel® Core™ Processor Family (2012)

The Third Generation Intel Core processor family spans Intel Core i7, i5, and i3 processors based on the Ivy Bridge microarchitecture. The Intel Xeon processor E7-8800/4800/2800 v2 product families and Intel Xeon processor E3-1200 v2 product family are also based on the Ivy Bridge microarchitecture.

The Intel Xeon processor E5-2400/1400 v2 product families are based on the Ivy Bridge-EP microarchitecture.

The Intel Xeon processor E5-4600/2600/1600 v2 product families are based on the Ivy Bridge-EP microarchitecture and provide support for multiple sockets.

2.1.22 The Fourth Generation Intel® Core™ Processor Family (2013)

The Fourth Generation Intel Core processor family spans Intel Core i7, i5, and i3 processors based on the Haswell microarchitecture. Intel Xeon processor E3-1200 v3 product family is also based on the Haswell microarchitecture.

2.2 MORE ON SPECIFIC ADVANCES

The following sections provide more information on major innovations.

2.2.1 P6 Family Microarchitecture

The Pentium Pro processor introduced a new microarchitecture commonly referred to as P6 processor microarchitecture. The P6 processor microarchitecture was later enhanced with an on-die, Level 2 cache, called Advanced Transfer Cache.

The microarchitecture is a three-way superscalar, pipelined architecture. Three-way superscalar means that by using parallel processing techniques, the processor is able on average to decode, dispatch, and complete execution of (retire) three instructions per clock cycle. To handle this level of instruction throughput, the P6 processor family uses a decoupled, 12-stage superpipeline that supports out-of-order instruction execution.

Figure 2-1 shows a conceptual view of the P6 processor microarchitecture pipeline with the Advanced Transfer Cache enhancement.

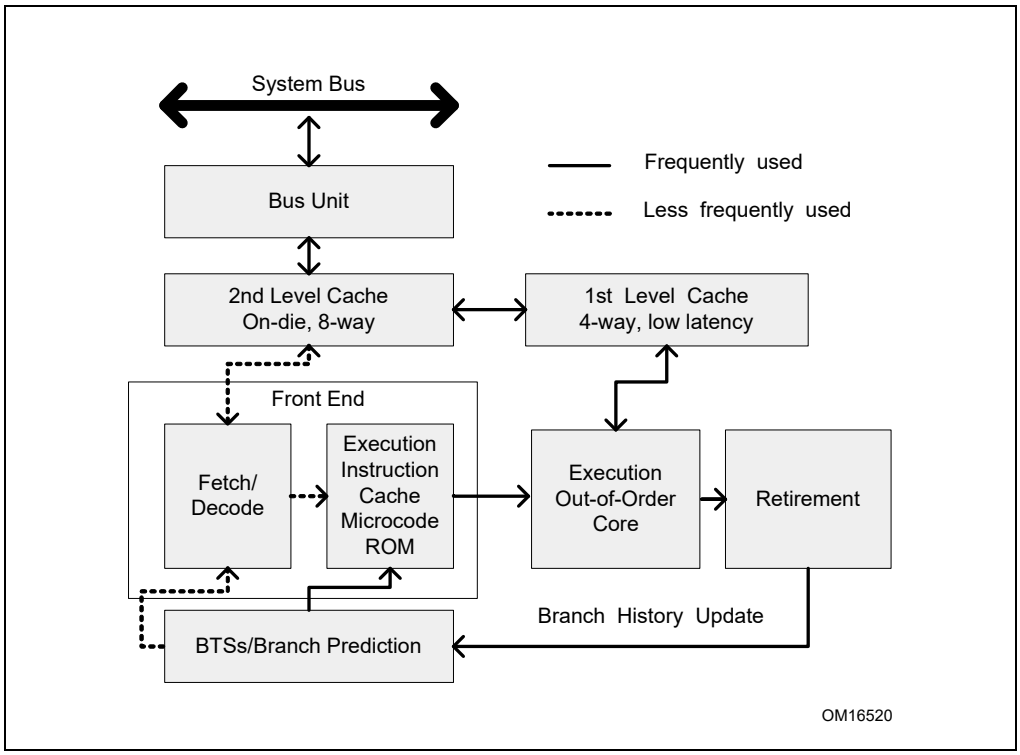


Figure 2-1. The P6 Processor Microarchitecture with Advanced Transfer Cache Enhancement

To ensure a steady supply of instructions and data for the instruction execution pipeline, the P6 processor microarchitecture incorporates two cache levels. The Level 1 cache provides an 8-KByte instruction cache and an 8-KByte data cache, both closely coupled to the pipeline. The Level 2 cache provides 256-KByte, 512-KByte, or 1-MByte static RAM that is coupled to the core processor through a full clock-speed 64-bit cache bus.

The centerpiece of the P6 processor microarchitecture is an out-of-order execution mechanism called dynamic execution. Dynamic execution incorporates three data-processing concepts:

- **Deep branch prediction** allows the processor to decode instructions beyond branches to keep the instruction pipeline full. The P6 processor family implements highly optimized branch prediction algorithms to predict the direction of the instruction.
- **Dynamic data flow analysis** requires real-time analysis of the flow of data through the processor to determine dependencies and to detect opportunities for out-of-order instruction execution. The out-of-order execution core can monitor many instructions and execute these instructions in the order that best optimizes the use of the processor's multiple execution units, while maintaining the data integrity.
- **Speculative execution** refers to the processor's ability to execute instructions that lie beyond a conditional branch that has not yet been resolved, and ultimately to commit the results in the order of the original instruction stream. To make speculative execution possible, the P6 processor microarchitecture decouples the dispatch and execution of instructions from the commitment of results. The processor's out-of-order execution core uses data-flow analysis to execute all available instructions in the instruction pool and store the results in temporary registers. The retirement unit then linearly searches the instruction pool for completed instructions that no longer have data dependencies with other instructions or unresolved branch predictions. When completed instructions are found, the retirement unit commits the results of these instructions to memory and/or the IA-32 registers (the processor's eight general-purpose registers and eight x87 FPU data registers) in the order they were originally issued and retires the instructions from the instruction pool.

2.2.2 Intel NetBurst® Microarchitecture

The Intel NetBurst microarchitecture provides:

- The Rapid Execution Engine.
  - — Arithmetic Logic Units (ALUs) run at twice the processor frequency.
  - — Basic integer operations can dispatch in 1/2 processor clock tick.
- Hyper-Pipelined Technology.
  - — Deep pipeline to enable industry-leading clock rates for desktop PCs and servers.
  - — Frequency headroom and scalability to continue leadership into the future.
- Advanced Dynamic Execution.
  - — Deep, out-of-order, speculative execution engine.
    - Up to 126 instructions in flight.
    - Up to 48 loads and 24 stores in pipeline[1].
  - — Enhanced branch prediction capability.
    - Reduces the misprediction penalty associated with deeper pipelines.
    - Advanced branch prediction algorithm.
    - 4K-entry branch target array.
- New cache subsystem.
  - — First level caches.
    - Advanced Execution Trace Cache stores decoded instructions.
    - Execution Trace Cache removes decoder latency from main execution loops.
    - Execution Trace Cache integrates path of program execution flow into a single line.
    - Low latency data cache.
  - — Second level cache.
    - Full-speed, unified 8-way Level 2 on-die Advance Transfer Cache.
    - Bandwidth and performance increases with processor frequency.

1. Intel 64 and IA-32 processors based on the Intel NetBurst microarchitecture at 90 nm process can handle more than 24 stores in flight.

- High-performance, quad-pumped bus interface to the Intel NetBurst microarchitecture system bus.
  - — Supports quad-pumped, scalable bus clock to achieve up to 4X effective speed.
  - — Capable of delivering up to 8.5 GBytes of bandwidth per second.
- Superscalar issue to enable parallelism.
- Expanded hardware registers with renaming to avoid register name space limitations.
- 64-byte cache line size (transfers data up to two lines per sector).

Figure 2-2 is an overview of the Intel NetBurst microarchitecture. This microarchitecture pipeline is made up of three sections: (1) the front end pipeline, (2) the out-of-order execution core, and (3) the retirement unit.

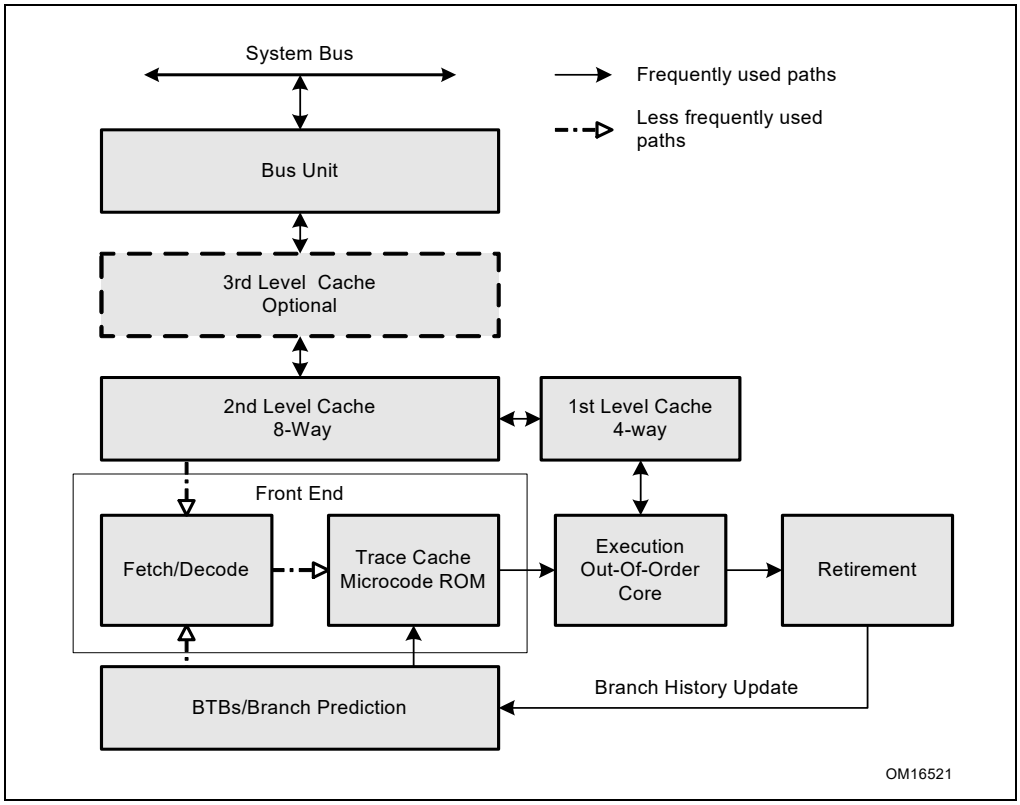


Figure 2-2. The Intel NetBurst® Microarchitecture

2.2.2.1 The Front End Pipeline

The front end supplies instructions in program order to the out-of-order execution core. It performs a number of functions:

- Prefetches instructions that are likely to be executed.
- Fetches instructions that have not already been prefetched.
- Decodes instructions into micro-operations.
- Generates microcode for complex instructions and special-purpose code.
- Delivers decoded instructions from the execution trace cache.
- Predicts branches using highly advanced algorithm.

The pipeline is designed to address common problems in high-speed, pipelined microprocessors. Two of these problems contribute to major sources of delays:

- Time to decode instructions fetched from the target.

- Wasted decode bandwidth due to branches or branch target in the middle of cache lines.

The operation of the pipeline's trace cache addresses these issues. Instructions are constantly being fetched and decoded by the translation engine (part of the fetch/decode logic) and built into sequences of micro-ops called traces. At any time, multiple traces (representing prefetched branches) are being stored in the trace cache. The trace cache is searched for the instruction that follows the active branch. If the instruction also appears as the first instruction in a pre-fetched branch, the fetch and decode of instructions from the memory hierarchy ceases and the pre-fetched branch becomes the new source of instructions (see Figure 2-2).

The trace cache and the translation engine have cooperating branch prediction hardware. Branch targets are predicted based on their linear addresses using branch target buffers (BTBs) and fetched as soon as possible.

2.2.2.2 Out-Of-Order Execution Core

The out-of-order execution core's ability to execute instructions out of order is a key factor in enabling parallelism. This feature enables the processor to reorder instructions so that if one micro-op is delayed, other micro-ops may proceed around it. The processor employs several buffers to smooth the flow of micro-ops.

The core is designed to facilitate parallel execution. It can dispatch up to six micro-ops per cycle (this exceeds trace cache and retirement micro-op bandwidth). Most pipelines can start executing a new micro-op every cycle, so several instructions can be in flight at a time for each pipeline. A number of arithmetic logical unit (ALU) instructions can start at two per cycle; many floating-point instructions can start once every two cycles.

2.2.2.3 Retirement Unit

The retirement unit receives the results of the executed micro-ops from the out-of-order execution core and processes the results so that the architectural state updates according to the original program order.

When a micro-op completes and writes its result, it is retired. Up to three micro-ops may be retired per cycle. The Reorder Buffer (ROB) is the unit in the processor which buffers completed micro-ops, updates the architectural state in order, and manages the ordering of exceptions. The retirement section also keeps track of branches and sends updated branch target information to the BTB. The BTB then purges pre-fetched traces that are no longer needed.

2.2.3 Intel® Core™ Microarchitecture

Intel Core microarchitecture introduces the following features that enable high performance and power-efficient performance for single-threaded as well as multi-threaded workloads:

- **Intel® Wide Dynamic Execution** enable each processor core to fetch, dispatch, execute in high bandwidths to support retirement of up to four instructions per cycle.
  - Fourteen-stage efficient pipeline.
  - Three arithmetic logical units.
  - Four decoders to decode up to five instruction per cycle.
  - Macro-fusion and micro-fusion to improve front-end throughput.
  - Peak issue rate of dispatching up to six micro-ops per cycle.
  - Peak retirement bandwidth of up to 4 micro-ops per cycle.
  - Advanced branch prediction.
  - Stack pointer tracker to improve efficiency of executing function/procedure entries and exits.
- **Intel® Advanced Smart Cache** delivers higher bandwidth from the second level cache to the core, and optimal performance and flexibility for single-threaded and multi-threaded applications.
  - Large second level cache up to 4 MB and 16-way associativity.
  - Optimized for multicore and single-threaded execution environments.
  - 256-bit internal data path to improve bandwidth from L2 to first-level data cache.

- **Intel® Smart Memory Access** prefetches data from memory in response to data access patterns and reduces cache-miss exposure of out-of-order execution.
  - — Hardware prefetchers to reduce effective latency of second-level cache misses.
  - — Hardware prefetchers to reduce effective latency of first-level data cache misses.
  - — Memory disambiguation to improve efficiency of speculative execution engine.
- **Intel® Advanced Digital Media Boost** improves most 128-bit SIMD instructions with single-cycle throughput and floating-point operations.
  - — Single-cycle throughput of most 128-bit SIMD instructions.
  - — Up to eight floating-point operations per cycle.
  - — Three issue ports available to dispatching SIMD instructions for execution.

Intel Core 2 Extreme, Intel Core 2 Duo processors and Intel Xeon processor 5100 series implement two processor cores based on the Intel Core microarchitecture, the functionality of the subsystems in each core are depicted in Figure 2-3.

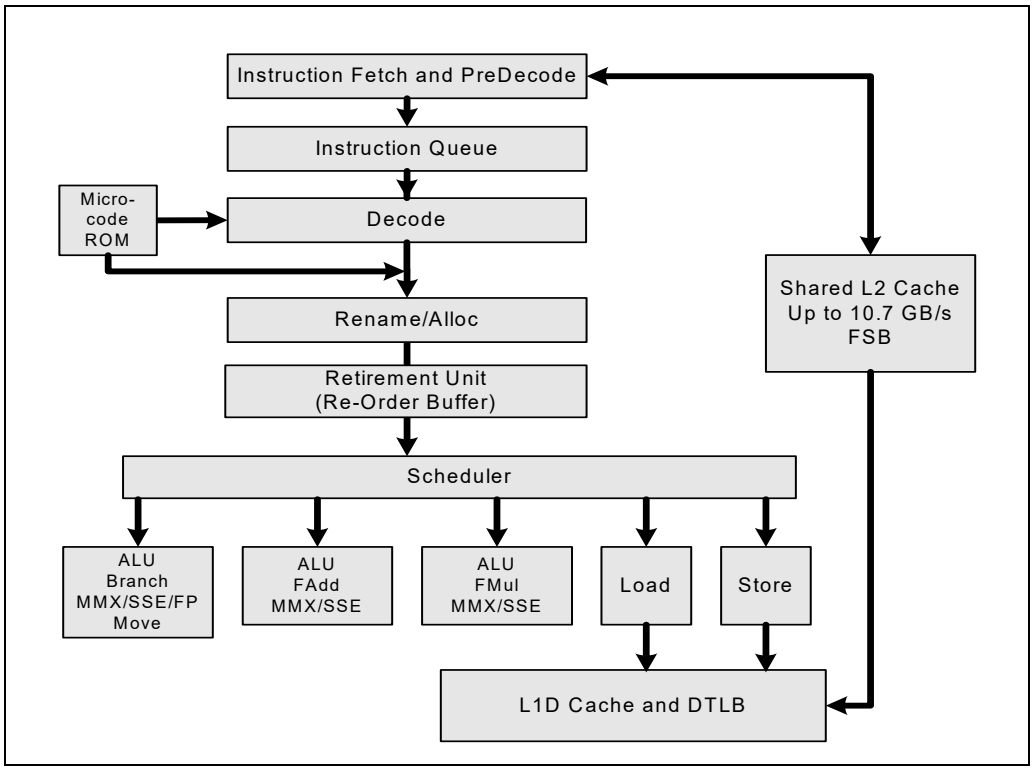


Figure 2-3. The Intel® Core™ Microarchitecture Pipeline Functionality

2.2.3.1 The Front End

The front end of Intel Core microarchitecture provides several enhancements to feed the Intel Wide Dynamic Execution engine:

- Instruction fetch unit prefetches instructions into an instruction queue to maintain steady supply of instruction to the decode units.
- Four-wide decode unit can decode 4 instructions per cycle or 5 instructions per cycle with Macrofusion.
- Macrofusion fuses common sequence of two instructions as one decoded instruction (micro-ops) to increase decoding throughput.
- Microfusion fuses common sequence of two micro-ops as one micro-ops to improve retirement throughput.

- Instruction queue provides caching of short loops to improve efficiency.
- Stack pointer tracker improves efficiency of executing procedure/function entries and exits.
- Branch prediction unit employs dedicated hardware to handle different types of branches for improved branch prediction.
- Advanced branch prediction algorithm directs instruction fetch unit to fetch instructions likely in the architectural code path for decoding.

2.2.3.2 Execution Core

The execution core of the Intel Core microarchitecture is superscalar and can process instructions out of order to increase the overall rate of instructions executed per cycle (IPC). The execution core employs the following feature to improve execution throughput and efficiency:

- Up to six micro-ops can be dispatched to execute per cycle.
- Up to four instructions can be retired per cycle.
- Three full arithmetic logical units.
- SIMD instructions can be dispatched through three issue ports.
- Most SIMD instructions have 1-cycle throughput (including 128-bit SIMD instructions).
- Up to eight floating-point operation per cycle.
- Many long-latency computation operation are pipelined in hardware to increase overall throughput.
- Reduced exposure to data access delays using Intel Smart Memory Access.

2.2.4 Intel Atom® Microarchitecture

Intel Atom microarchitecture maximizes power-efficient performance for single-threaded and multi-threaded workloads by providing:

- **Advanced Micro-Ops Execution**
  - Single-micro-op instruction execution from decode to retirement, including instructions with register-only, load, and store semantics.
  - Sixteen-stage, in-order pipeline optimized for throughput and reduced power consumption.
  - Dual pipelines to enable decode, issue, execution, and retirement of two instructions per cycle.
  - Advanced stack pointer to improve efficiency of executing function entry/returns.
- **Intel® Smart Cache**
  - Second level cache is 512 KB and 8-way associativity.
  - Optimized for multi-threaded and single-threaded execution environments
  - 256-bit internal data path between L2 and L1 data caches improves high bandwidth.
- **Efficient Memory Access**
  - Efficient hardware prefetchers to L1 and L2, speculatively loading data likely to be requested by processor to reduce cache miss impact.
- **Intel® Digital Media Boost**
  - Two issue ports for dispatching SIMD instructions to execution units.
  - Single-cycle throughput for most 128-bit integer SIMD instructions.
  - Up to six floating-point operations per cycle.
  - Up to two 128-bit SIMD integer operations per cycle.
  - Safe Instruction Recognition (SIR) to allow long-latency floating-point operations to retire out of order with respect to integer instructions.

2.2.5 Nehalem Microarchitecture

Nehalem microarchitecture provides the foundation for many features of Intel Core i7 processors. It builds on the success of 45 nm Intel Core microarchitecture and provides the following feature enhancements:

- **Enhanced processor core**
  - Improved branch prediction and recovery from misprediction.
  - Enhanced loop streaming to improve front end performance and reduce power consumption.
  - Deeper buffering in out-of-order engine to extract parallelism.
  - Enhanced execution units to provide acceleration in CRC, string/text processing and data shuffling.
- **Smart Memory Access**
  - Integrated memory controller provides low-latency access to system memory and scalable memory bandwidth.
  - New cache hierarchy organization with shared, inclusive L3 to reduce snoop traffic.
  - Two level TLBs and increased TLB size.
  - Fast unaligned memory access.
- **HyperThreading Technology**
  - Provides two hardware threads (logical processors) per core.
  - Takes advantage of 4-wide execution engine, large L3, and massive memory bandwidth.
- **Dedicated Power management Innovations**
  - Integrated microcontroller with optimized embedded firmware to manage power consumption.
  - Embedded real-time sensors for temperature, current, and power.
  - Integrated power gate to turn off/on per-core power consumption
  - Versatility to reduce power consumption of memory, link subsystems.

2.2.6 Sandy Bridge Microarchitecture

Sandy Bridge microarchitecture builds on the successes of Intel® Core™ microarchitecture and Nehalem microarchitecture. It offers the following features:

- Intel Advanced Vector Extensions (Intel AVX).
  - 256-bit floating-point instruction set extensions to the 128-bit Intel Streaming SIMD Extensions, providing up to 2X performance benefits relative to 128-bit code.
  - Non-destructive destination encoding offers more flexible coding techniques.
  - Supports flexible migration and co-existence between 256-bit AVX code, 128-bit AVX code and legacy 128-bit SSE code.
- Enhanced front-end and execution engine.
  - New decoded Icache component that improves front-end bandwidth and reduces branch misprediction penalty.
  - Advanced branch prediction.
  - Additional macro-fusion support.
  - Larger dynamic execution window.
  - Multi-precision integer arithmetic enhancements (ADC/SBB, MUL/IMUL).
  - LEA bandwidth improvement.
  - Reduction of general execution stalls (read ports, writeback conflicts, bypass latency, partial stalls).
  - Fast floating-point exception handling.

  - XSAVE/XRSTORE performance improvements and XSAVEOPT new instruction.
- Cache hierarchy improvements for wider data path.
  - Doubling of bandwidth enabled by two symmetric ports for memory operation.
  - Simultaneous handling of more in-flight loads and stores enabled by increased buffers.
  - Internal bandwidth of two loads and one store each cycle.
  - Improved prefetching.
  - High bandwidth low latency LLC architecture.
  - High bandwidth ring architecture of on-die interconnect.

For additional information on Intel® Advanced Vector Extensions (AVX), see Section 5.13, "Intel® Advanced Vector Extensions (Intel® AVX)" and Chapter 14, "Programming with Intel® AVX, FMA, and Intel® AVX2" in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 1.

2.2.7 SIMD Instructions

Beginning with the Pentium II and Pentium with Intel MMX technology processor families, six extensions have been introduced into the Intel 64 and IA-32 architectures to perform single-instruction multiple-data (SIMD) operations. These extensions include the MMX technology, SSE extensions, SSE2 extensions, SSE3 extensions, Supplemental Streaming SIMD Extensions 3, and SSE4. Each of these extensions provides a group of instructions that perform SIMD operations on packed integer and/or packed floating-point data elements.

SIMD integer operations can use the 64-bit MMX or the 128-bit XMM registers. SIMD floating-point operations use 128-bit XMM registers. Figure 2-4 shows a summary of the various SIMD extensions (MMX technology, Intel SSE, Intel SSE2, Intel SSE3, SSSE3, and Intel SSE4), the data types they operate on, and how the data types are packed into MMX and XMM registers.

The Intel MMX technology was introduced in the Pentium II and Pentium with MMX technology processor families. MMX instructions perform SIMD operations on packed byte, word, or doubleword integers located in MMX registers. These instructions are useful in applications that operate on integer arrays and streams of integer data that lend themselves to SIMD processing.

Intel SSE was introduced in the Pentium III processor family. Intel SSE instructions operate on packed single precision floating-point values contained in XMM registers and on packed integers contained in MMX registers. Several Intel SSE instructions provide state management, cache control, and memory ordering operations. Other Intel SSE instructions are targeted at applications that operate on arrays of single precision floating-point data elements (3-D geometry, 3-D rendering, and video encoding and decoding applications).

Intel SSE2 was introduced in the Pentium 4 and Intel Xeon processors. Intel SSE2 instructions operate on packed double precision floating-point values contained in XMM registers and on packed integers contained in MMX and XMM registers. Intel SSE2 integer instructions extend IA-32 SIMD operations by adding new 128-bit SIMD integer operations and by expanding existing 64-bit SIMD integer operations to 128-bit XMM capability. Intel SSE2 instructions also provide new cache control and memory ordering operations.

Intel SSE3 was introduced with the Pentium 4 processor supporting Hyper-Threading Technology (built on 90 nm process technology). Intel SSE3 offers 13 instructions that accelerate performance of Streaming SIMD Extensions technology, Streaming SIMD Extensions 2 technology, and x87-FP math capabilities.

SSSE3 was introduced with the Intel Xeon processor 5100 series and Intel Core 2 processor family. SSSE3 offer 32 instructions to accelerate processing of SIMD integer data.

Intel SSE4 offers 54 instructions. 47 of them are referred to as Intel SSE4.1 instructions. Intel SSE4.1 was introduced with the Intel Xeon processor 5400 series and Intel Core 2 Extreme processor QX9650. The other seven Intel SSE4 instructions are referred to as Intel SSE4.2 instructions.

Intel AES-NI and PCLMULQDQ introduced seven new instructions. Six of them are primitives for accelerating algorithms based on AES encryption/decryption standard, and are referred to as Intel AES-NI.

The PCLMULQDQ instruction accelerates general-purpose block encryption, which can perform carry-less multiplication for two binary numbers up to 64-bit wide.

Intel 64 architecture allows four generations of 128-bit SIMD extensions to access up to 16 XMM registers. IA-32 architecture provides eight XMM registers.

Intel® Advanced Vector Extensions offers comprehensive architectural enhancements over previous generations of Streaming SIMD Extensions. Intel AVX introduces the following architectural enhancements:

- Support for 256-bit wide vectors and SIMD register set.
- 256-bit floating-point instruction set enhancement with up to 2X performance gain relative to 128-bit Streaming SIMD extensions.
- Instruction syntax support for generalized three-operand syntax to improve instruction programming flexibility and efficient encoding of new instruction extensions.
- Enhancement of legacy 128-bit SIMD instruction extensions to support three operand syntax and to simplify compiler vectorization of high-level language expressions.
- Support flexible deployment of 256-bit AVX code, 128-bit AVX code, legacy 128-bit code and scalar code.

In addition to performance considerations, programmers should also be cognizant of the implications of VEX-encoded AVX instructions with the expectations of system software components that manage the processor state components enabled by XCR0. For additional information see Section 2.3.10.1, "Vector Length Transition and Programming Considerations" in Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 2A.

See also:

- Section 5.4, "MMX Instructions," and Chapter 9, "Programming with Intel® MMX™ Technology."
- Section 5.5, "Intel® SSE Instructions," and Chapter 10, "Programming with Intel® Streaming SIMD Extensions (Intel® SSE)."
- Section 5.6, "Intel® SSE2 Instructions," and Chapter 11, "Programming with Intel® Streaming SIMD Extensions 2 (Intel® SSE2)."
- Section 5.7, "Intel® SSE3 Instructions," Section 5.8, "Supplemental Streaming SIMD Extensions 3 (SSSE3) Instructions," Section 5.9, "Intel® SSE4 Instructions," and Chapter 12, "Programming with Intel® SSE3, SSSE3, Intel® SSE4, and Intel® AES-NI."

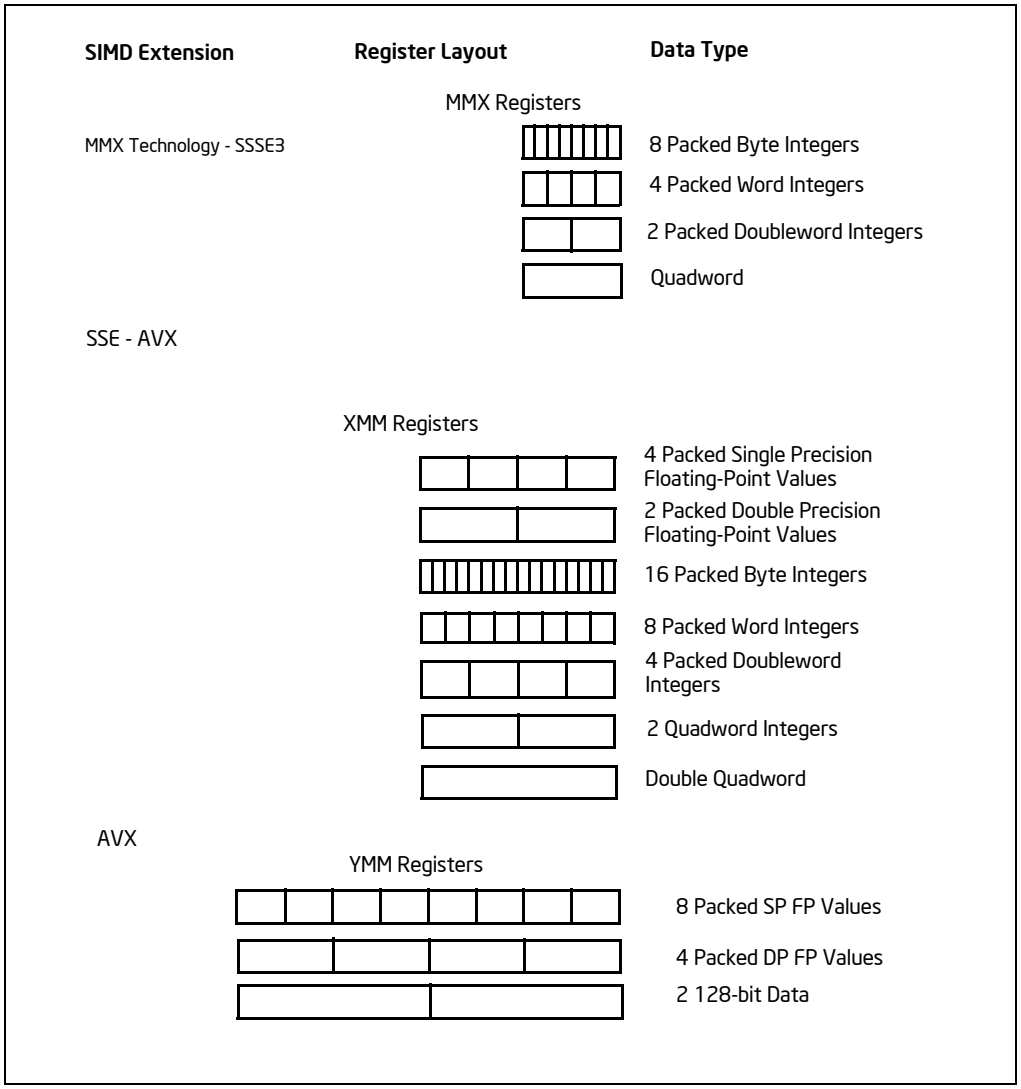


Figure 2-4. SIMD Extensions, Register Layouts, and Data Types

2.2.8 Intel® Hyper-Threading Technology

Intel Hyper-Threading Technology (Intel HT Technology) was developed to improve the performance of IA-32 processors when executing multi-threaded operating system and application code or single-threaded applications under multi-tasking environments. The technology enables a single physical processor to execute two or more separate code streams (threads) concurrently using shared execution resources.

Intel HT Technology is one form of hardware multi-threading capability in IA-32 processor families. It differs from multi-processor capability using separate physically distinct packages with each physical processor package mated with a physical socket. Intel HT Technology provides hardware multi-threading capability with a single physical package by using shared execution resources in a processor core.

Architecturally, an IA-32 processor that supports Intel HT Technology consists of two or more logical processors, each of which has its own IA-32 architectural state. Each logical processor consists of a full set of IA-32 data registers, segment registers, control registers, debug registers, and most of the MSRs. Each also has its own advanced programmable interrupt controller (APIC).

Figure 2-5 shows a comparison of a processor that supports Intel HT Technology (implemented with two logical processors) and a traditional dual processor system.

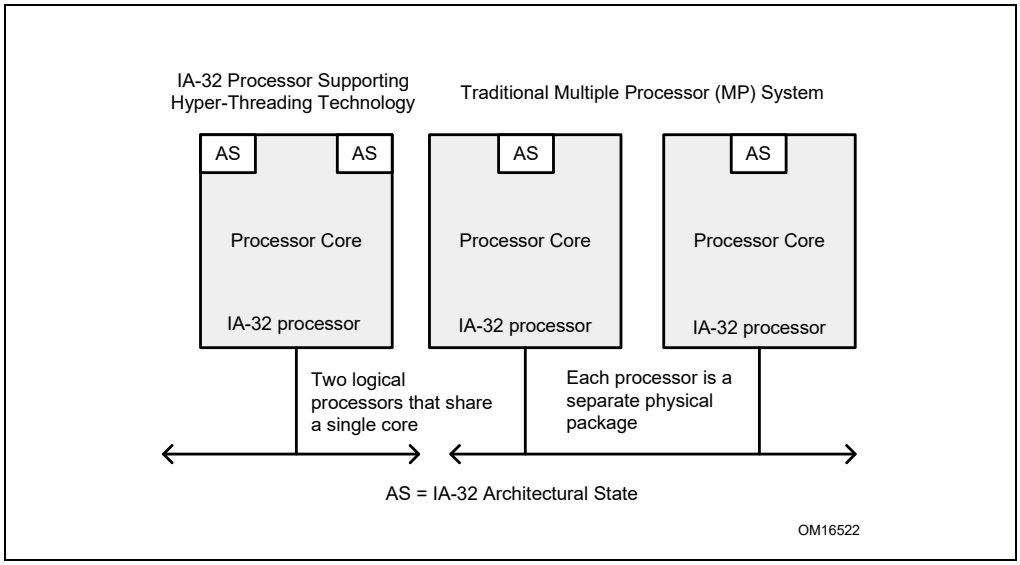


Figure 2-5. Comparison of an IA-32 Processor Supporting Intel® Hyper-Threading Technology and a Traditional Dual Processor System

Unlike a traditional MP system configuration that uses two or more separate physical IA-32 processors, the logical processors in an IA-32 processor supporting Intel HT Technology share the core resources of the physical processor. This includes the execution engine and the system bus interface. After power up and initialization, each logical processor can be independently directed to execute a specified thread, interrupted, or halted.

Intel HT Technology leverages the process and thread-level parallelism found in contemporary operating systems and high-performance applications by providing two or more logical processors on a single chip. This configuration allows two or more threads[1] to be executed simultaneously on each a physical processor. Each logical processor executes instructions from an application thread using the resources in the processor core. The core executes these threads concurrently, using out-of-order instruction scheduling to maximize the use of execution units during each clock cycle.

2.2.8.1 Some Implementation Notes

All Intel HT Technology configurations require:

- A processor that supports Intel HT Technology.
- A chipset and BIOS that utilize the technology.
- Operating system optimizations.

See http://www.intel.com/products/ht/hyperthreading_more.htm for information.

At the firmware (BIOS) level, the basic procedures to initialize the logical processors in a processor supporting Intel HT Technology are the same as those for a traditional DP or MP platform. The mechanisms that are described in the Multiprocessor Specification, Version 1.4, to power-up and initialize physical processors in an MP system also apply to logical processors in a processor that supports Intel HT Technology.

An operating system designed to run on a traditional DP or MP platform may use CPUID to determine the presence of hardware multi-threading support feature and the number of logical processors they provide.

Although existing operating system and application code should run correctly on a processor that supports Intel HT Technology, some code modifications are recommended to get the optimum benefit. These modifications are discussed in Chapter 7, "Multiple-Processor Management," Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3A.

1. In the remainder of this document, the term "thread" will be used as a general term for the terms "process" and "thread."

2.2.9 Multi-Core Technology

Multi-core technology is another form of hardware multi-threading capability in IA-32 processor families. Multi-core technology enhances hardware multi-threading capability by providing two or more execution cores in a physical package.

The Intel Pentium processor Extreme Edition is the first member in the IA-32 processor family to introduce multi-core technology. The processor provides hardware multi-threading support with both two processor cores and Intel Hyper-Threading Technology. This means that the Intel Pentium processor Extreme Edition provides four logical processors in a physical package (two logical processors for each processor core). The Dual-Core Intel Xeon processor features multi-core, Intel Hyper-Threading Technology and supports multi-processor platforms.

The Intel Pentium D processor also features multi-core technology. This processor provides hardware multi-threading support with two processor cores but does not offer Intel Hyper-Threading Technology. This means that the Intel Pentium D processor provides two logical processors in a physical package, with each logical processor owning the complete execution resources of a processor core.

The Intel Core 2 processor family, Intel Xeon processor 3000 series, Intel Xeon processor 5100 series, and Intel Core Duo processor offer power-efficient multi-core technology. The processor contains two cores that share a smart second level cache. The Level 2 cache enables efficient data sharing between two cores to reduce memory traffic to the system bus.

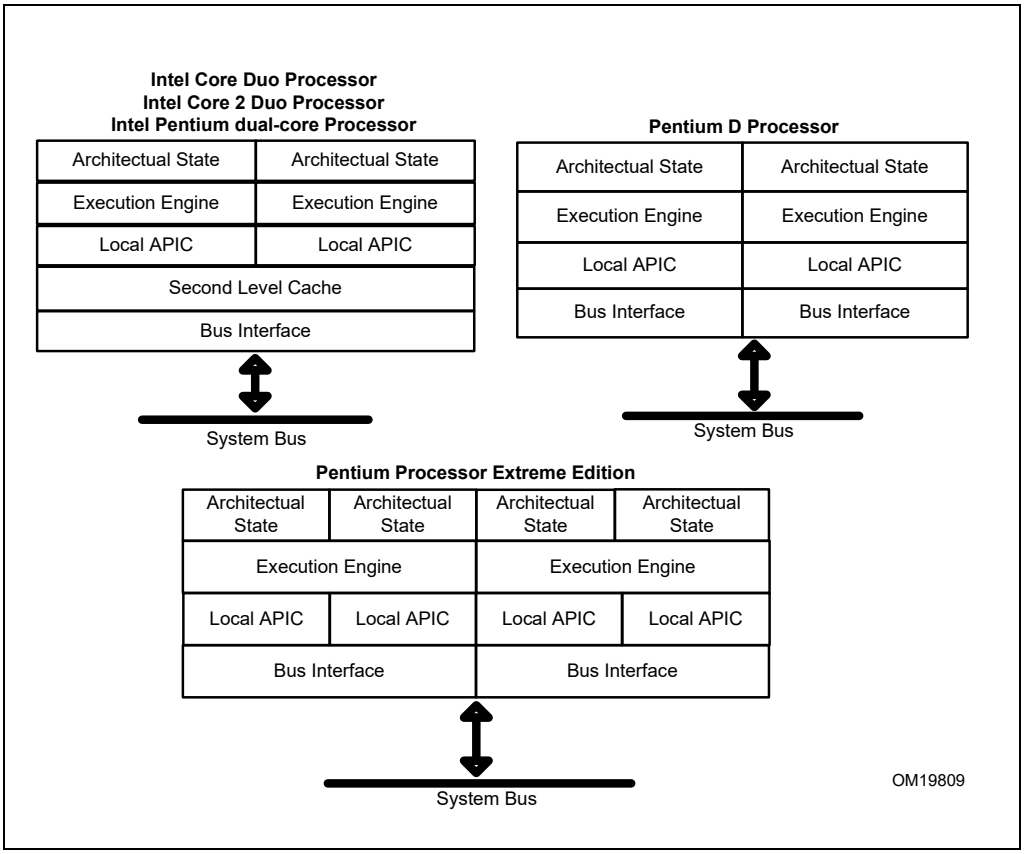


Figure 2-6. Intel® 64 and IA-32 Processors that Support Dual-Core

The Pentium® dual-core processor is based on the same technology as the Intel Core 2 Duo processor family.

The Intel Xeon processor 7300, 5300, and 3200 series, Intel Core 2 Extreme Quad-Core processor, and Intel Core 2 Quad processors support Intel quad-core technology. The Quad-core Intel Xeon processors and the Quad-Core Intel Core 2 processor family are also in Figure 2-7.

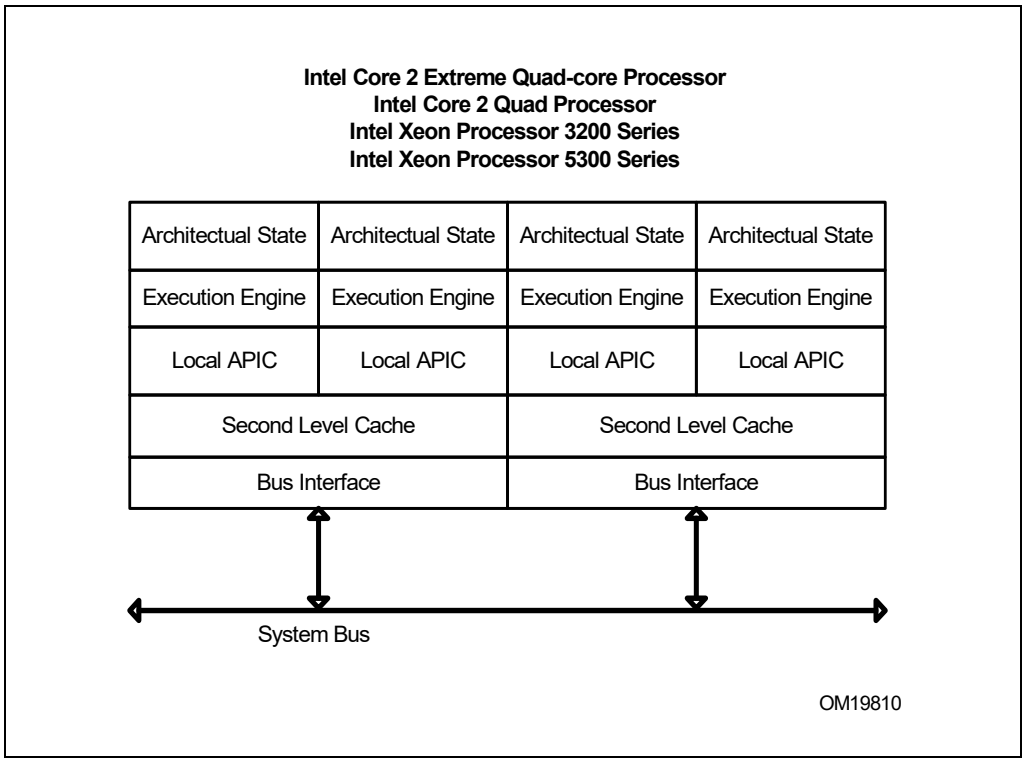


Figure 2-7. Intel® 64 Processors that Support Quad-Core

Intel Core i7 processors support Intel quad-core technology, Intel HyperThreading Technology, provides Intel QuickPath interconnect link to the chipset and have integrated memory controller supporting three channels to DDR3 memory.

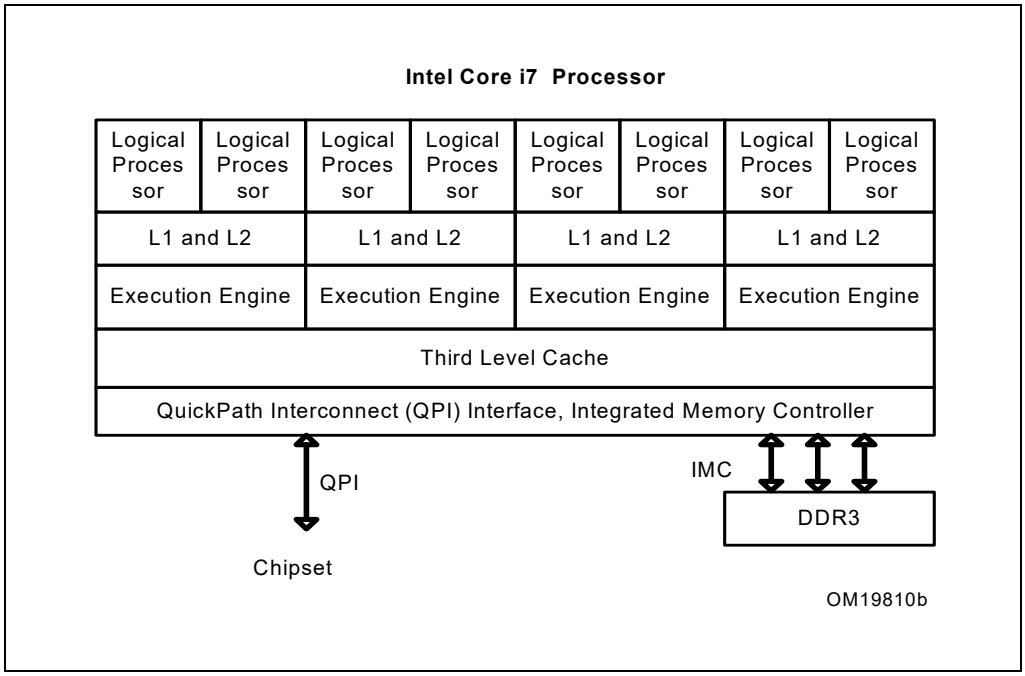


Figure 2-8. Intel® Core™ i7 Processor

2.2.10 Intel® 64 Architecture

Intel 64 architecture increases the linear address space for software to 64 bits and supports physical address space up to 52 bits. The technology also introduces a new operating mode referred to as IA-32e mode.

IA-32e mode operates in one of two sub-modes: (1) compatibility mode enables a 64-bit operating system to run most legacy 32-bit software unmodified, (2) 64-bit mode enables a 64-bit operating system to run applications written to access 64-bit address space.

In the 64-bit mode, applications may access:

- 64-bit flat linear addressing.
- 8 additional general-purpose registers (GPRs).
- 8 additional registers for streaming SIMD extensions (Intel SSE, SSE2, and SSE3, and SSSE3).
- 64-bit-wide GPRs and instruction pointers.
- Uniform byte-register addressing.
- Fast interrupt-prioritization mechanism.
- A new instruction-pointer relative-addressing mode.

An Intel 64 architecture processor supports existing IA-32 software because it is able to run all non-64-bit legacy modes supported by IA-32 architecture. Most existing IA-32 applications also run in compatibility mode.

2.2.11 Intel® Virtualization Technology (Intel® VT)

Intel® Virtualization Technology for Intel 64 and IA-32 architectures provide extensions that support virtualization. The extensions are referred to as Virtual Machine Extensions (VMX). An Intel 64 or IA-32 platform with VMX can function as multiple virtual systems (or virtual machines). Each virtual machine can run operating systems and applications in separate partitions.

VMX also provides programming interface for a new layer of system software (called the Virtual Machine Monitor (VMM)) used to manage the operation of virtual machines. Information on VMX and on the programming of VMMs is in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3C.

Intel Core i7 processor provides the following enhancements to Intel Virtualization Technology:

- Virtual processor ID (VPID) to reduce the cost of VMM managing transitions.
- Extended page table (EPT) to reduce the number of transitions for VMM to manage memory virtualization.
- Reduced latency of VM transitions.

2.3 INTEL® 64 AND IA-32 PROCESSOR GENERATIONS

In the mid-1960s, Intel co-founder and Chairman Emeritus Gordon Moore had this observation: "... the number of transistors that would be incorporated on a silicon die would double every 18 months for the next several years." Over the past three and half decades, this prediction known as "Moore's Law" has continued to hold true.

The computing power and the complexity (or roughly, the number of transistors per processor) of Intel architecture processors has grown in close relation to Moore's law. By taking advantage of new process technology and new microarchitecture designs, each new generation of IA-32 processors has demonstrated frequency-scaling head-room and new performance levels over the previous generation processors.

The key features of the Intel Pentium 4 processor, Intel Xeon processor, Intel Xeon processor MP, Pentium III processor, and Pentium III Xeon processor with advanced transfer cache are shown in Table 2-1. Older generation IA-32 processors, which do not employ on-die Level 2 cache, are shown in Table 2-2.

Table 2-1. Key Features of Most Recent IA-32 Processors

Intel Processor	Date Intro-duced	Microarchitecture	Top-Bin Clock Frequency at Introduction	Tran-sistors	Register Sizes[1]	System Bus Band-width	Max. Extern. Addr. Space	On-Die Caches[2]
Intel Pentium M Processor 755[3]	2004	Intel Pentium M Processor	2.00 GHz	140 M	GP: 32 FPU: 80 MMX: 64 XMM: 128	3.2 GB/s	4 GB	L1: 64 KB L2: 2 MB
Intel Core Duo Processor T2600 [3]	2006	Improved Intel Pentium M Processor Microarchitecture; Dual Core; Intel Smart Cache, Advanced Thermal Manager	2.16 GHz	152 M	GP: 32 FPU: 80 MMX: 64 XMM: 128	5.3 GB/s	4 GB	L1: 64 KB L2: 2 MB (2 MB Total)
Intel Atom Processor Z5xx series	2008	Intel Atom Microarchitecture; Intel Virtualization Technology.	1.86 GHz - 800 MHz	47 M	GP: 32 FPU: 80 MMX: 64 XMM: 128	Up to 4.2 GB/s	4 GB	L1: 56 KB[4] L2: 512 KB

NOTES:

1. The register size and external data bus size are given in bits.
2. First level cache is denoted using the abbreviation L1, 2nd level cache is denoted as L2. The size of L1 includes the first-level data cache and the instruction cache where applicable, but does not include the trace cache.
3. Intel processor numbers are not a measure of performance. Processor numbers differentiate features within each processor family, not across different processor families. See http://www.intel.com/products/processor_number for details.
4. In Intel Atom Processor, the size of L1 instruction cache is 32 KBytes, L1 data cache is 24 KBytes.

Table 2-2. Key Features of Most Recent Intel® 64 Processors

Intel Processor	Date Intro-duced	Micro-architec-ture	Highest Processor Base Fre-quency at Intro-duction	Tran-sistors	Register Sizes	System Bus/QPI Link Speed	Max. Extern. Addr. Space	On-Die Caches
64-bit Intel Xeon Processor with 800 MHz System Bus	2004	Intel NetBurst Microarchitecture; Intel Hyper-Threading Technology; Intel 64 Architecture	3.60 GHz	125 M	GP: 32, 64 FPU: 80 MMX: 64 XMM: 128	6.4 GB/s	64 GB	12K µop Execution Trace Cache; 16 KB L1; 1 MB L2
64-bit Intel Xeon Processor MP with 8MB L3	2005	Intel NetBurst Microarchitecture; Intel Hyper-Threading Technology; Intel 64 Architecture	3.33 GHz	675 M	GP: 32, 64 FPU: 80 MMX: 64 XMM: 128	5.3 GB/s [1]	1024 GB (1 TB)	12K µop Execution Trace Cache; 16 KB L1; 1 MB L2, 8 MB L3

Table 2-2. Key Features of Most Recent Intel® 64 Processors (Contd.)

Intel Processor	Date Intro-duced	Micro-architec-ture	Highest Processor Base Fre-quency at Intro-duction	Tran-sistors	Register Sizes	System Bus/QPI Link Speed	Max. Extern. Addr. Space	On-Die Caches
Intel Pentium 4 Processor Extreme Edition Supporting Hyper-Threading Technology	2005	Intel NetBurst Microarchitecture; Intel Hyper-Threading Technology; Intel 64 Architecture	3.73 GHz	164 M	GP: 32, 64 FPU: 80 MMX: 64 XMM: 128	8.5 GB/s	64 GB	12K μop Execution Trace Cache; 16 KB L1; 2 MB L2
Intel Pentium Processor Extreme Edition 840	2005	Intel NetBurst Microarchitecture; Intel Hyper-Threading Technology; Intel 64 Architecture; Dual-core [2]	3.20 GHz	230 M	GP: 32, 64 FPU: 80 MMX: 64 XMM: 128	6.4 GB/s	64 GB	12K μop Execution Trace Cache; 16 KB L1; 1 MB L2 (2 MB Total)
Dual-Core Intel Xeon Processor 7041	2005	Intel NetBurst Microarchitecture; Intel Hyper-Threading Technology; Intel 64 Architecture; Dual-core [3]	3.00 GHz	321 M	GP: 32, 64 FPU: 80 MMX: 64 XMM: 128	6.4 GB/s	64 GB	12K μop Execution Trace Cache; 16 KB L1; 2 MB L2 (4 MB Total)
Intel Pentium 4 Processor 672	2005	Intel NetBurst Microarchitecture; Intel Hyper-Threading Technology; Intel 64 Architecture; Intel Virtualization Technology.	3.80 GHz	164 M	GP: 32, 64 FPU: 80 MMX: 64 XMM: 128	6.4 GB/s	64 GB	12K μop Execution Trace Cache; 16 KB L1; 2 MB L2
Intel Pentium Processor Extreme Edition 955	2006	Intel NetBurst Microarchitecture; Intel 64 Architecture; Dual Core; Intel Virtualization Technology.	3.46 GHz	376 M	GP: 32, 64 FPU: 80 MMX: 64 XMM: 128	8.5 GB/s	64 GB	12K μop Execution Trace Cache; 16 KB L1; 2 MB L2 (4 MB Total)
Intel Core 2 Extreme Processor X6800	2006	Intel Core Microarchitecture; Dual Core; Intel 64 Architecture; Intel Virtualization Technology.	2.93 GHz	291 M	GP: 32, 64 FPU: 80 MMX: 64 XMM: 128	8.5 GB/s	64 GB	L1: 64 KB L2: 4 MB (4 MB Total)

Table 2-2. Key Features of Most Recent Intel® 64 Processors (Contd.)

Intel Processor	Date Intro-duced	Micro-architec-ture	Highest Processor Base Fre-quency at Intro-duction	Tran-sistors	Register Sizes	System Bus/QPI Link Speed	Max. Extern. Addr. Space	On-Die Caches
Intel Xeon Processor 5160	2006	Intel Core Microarchitecture; Dual Core; Intel 64 Architecture; Intel Virtualization Technology.	3.00 GHz	291 M	GP: 32, 64 FPU: 80 MMX: 64 XMM: 128	10.6 GB/s	64 GB	L1: 64 KB L2: 4 MB (4 MB Total)
Intel Xeon Processor 7140	2006	Intel NetBurst Microarchitecture; Dual Core; Intel 64 Architecture; Intel Virtualization Technology.	3.40 GHz	1.3 B	GP: 32, 64 FPU: 80 MMX: 64 XMM: 128	12.8 GB/s	64 GB	L1: 64 KB L2: 1 MB (2 MB Total) L3: 16 MB (16 MB Total)
Intel Core 2 Extreme Processor QX6700	2006	Intel Core Microarchitecture; Quad Core; Intel 64 Architecture; Intel Virtualization Technology.	2.66 GHz	582 M	GP: 32, 64 FPU: 80 MMX: 64 XMM: 128	8.5 GB/s	64 GB	L1: 64 KB L2: 4 MB (4 MB Total)
Quad-core Intel Xeon Processor 5355	2006	Intel Core Microarchitecture; Quad Core; Intel 64 Architecture; Intel Virtualization Technology.	2.66 GHz	582 M	GP: 32, 64 FPU: 80 MMX: 64 XMM: 128	10.6 GB/s	256 GB	L1: 64 KB L2: 4 MB (8 MB Total)
Intel Core 2 Duo Processor E6850	2007	Intel Core Microarchitecture; Dual Core; Intel 64 Architecture; Intel Virtualization Technology; Intel Trusted Execution Technology	3.00 GHz	291 M	GP: 32, 64 FPU: 80 MMX: 64 XMM: 128	10.6 GB/s	64 GB	L1: 64 KB L2: 4 MB (4 MB Total)
Intel Xeon Processor 7350	2007	Intel Core Microarchitecture; Quad Core; Intel 64 Architecture; Intel Virtualization Technology.	2.93 GHz	582 M	GP: 32, 64 FPU: 80 MMX: 64 XMM: 128	8.5 GB/s	1024 GB	L1: 64 KB L2: 4 MB (8 MB Total)

Table 2-2. Key Features of Most Recent Intel® 64 Processors (Contd.)

Intel Processor	Date Intro-duced	Micro-architec-ture	Highest Processor Base Fre-quency at Intro-duction	Tran-sistors	Register Sizes	System Bus/QPI Link Speed	Max. Extern. Addr. Space	On-Die Caches
Intel Xeon Processor 5472	2007	Enhanced Intel Core Microarchitecture; Quad Core; Intel 64 Architecture; Intel Virtualization Technology.	3.00 GHz	820 M	GP: 32, 64 FPU: 80 MMX: 64 XMM: 128	12.8 GB/s	256 GB	L1: 64 KB L2: 6 MB (12 MB Total)
Intel Atom Processor	2008	Intel Atom Microarchitecture; Intel 64 Architecture; Intel Virtualization Technology.	2.0 - 1.60 GHz	47 M	GP: 32, 64 FPU: 80 MMX: 64 XMM: 128	Up to 4.2 GB/s	Up to 64GB	L1: 56 KB[4] L2: 512 KB
Intel Xeon Processor 7460	2008	Enhanced Intel Core Microarchitecture; Six Cores; Intel 64 Architecture; Intel Virtualization Technology.	2.67 GHz	1.9 B	GP: 32, 64 FPU: 80 MMX: 64 XMM: 128	8.5 GB/s	1024 GB	L1: 64 KB L2: 3 MB (9 MB Total) L3: 16 MB
Intel Atom Processor 330	2008	Intel Atom Microarchitecture; Intel 64 Architecture; Dual core; Intel Virtualization Technology.	1.60 GHz	94 M	GP: 32, 64 FPU: 80 MMX: 64 XMM: 128	Up to 4.2 GB/s	Up to 64GB	L1: 56 KB[5] L2: 512 KB (1 MB Total)
Intel Core i7-965 Processor Extreme Edition	2008	Nehalem microarchitecture; Quadcore; HyperThreading Technology; Intel QPI; Intel 64 Architecture; Intel Virtualization Technology.	3.20 GHz	731 M	GP: 32, 64 FPU: 80 MMX: 64 XMM: 128	QPI: 6.4 GT/s; Memory: 25 GB/s	64 GB	L1: 64 KB L2: 256 KB L3: 8 MB

Table 2-2. Key Features of Most Recent Intel® 64 Processors (Contd.)

Intel Processor	Date Intro-duced	Micro-architec-ture	Highest Processor Base Fre-quency at Intro-duction	Tran-sistors	Register Sizes	System Bus/QPI Link Speed	Max. Extern. Addr. Space	On-Die Caches
Intel Core i7-620M Processor	2010	Intel Turbo Boost Technology, Westmere microarchitecture; Dual-core; HyperThreading Technology; Intel 64 Architecture; Intel Virtualization Technology., Integrated graphics	2.66 GHz	383 M	GP: 32, 64 FPU: 80 MMX: 64 XMM: 128		64 GB	L1: 64 KB L2: 256 KB L3: 4 MB
Intel Xeon-Processor 5680	2010	Intel Turbo Boost Technology, Westmere microarchitecture; Six core; HyperThreading Technology; Intel 64 Architecture; Intel Virtualization Technology.	3.33 GHz	1.1 B	GP: 32, 64 FPU: 80 MMX: 64 XMM: 128	QPI: 6.4 GT/s; 32 GB/s	1 TB	L1: 64 KB L2: 256 KB L3: 12 MB
Intel Xeon-Processor 7560	2010	Intel Turbo Boost Technology, Nehalem microarchitecture; Eight core; HyperThreading Technology; Intel 64 Architecture; Intel Virtualization Technology.	2.26 GHz	2.3 B	GP: 32, 64 FPU: 80 MMX: 64 XMM: 128	QPI: 6.4 GT/s; Memory: 76 GB/s	16 TB	L1: 64 KB L2: 256 KB L3: 24 MB
Intel Core i7-2600K Processor	2011	Intel Turbo Boost Technology, Sandy Bridge microarchitecture; Four core; HyperThreading Technology; Intel 64 Architecture; Intel Virtualization Technology., Processor graphics, Quicksync Video	3.40 GHz	995 M	GP: 32, 64 FPU: 80 MMX: 64 XMM: 128 YMM: 256	DMI: 5 GT/s; Memory: 21 GB/s	64 GB	L1: 64 KB L2: 256 KB L3: 8 MB

Table 2-2. Key Features of Most Recent Intel® 64 Processors (Contd.)

Intel Processor	Date Intro-duced	Micro-architec-ture	Highest Processor Base Fre-quency at Intro-duction	Tran-sistors	Register Sizes	System Bus/QPI Link Speed	Max. Extern. Addr. Space	On-Die Caches
Intel Xeon-Processor E3-1280	2011	Intel Turbo Boost Technology, Sandy Bridge microarchitecture; Four core; HyperThreading Technology; Intel 64 Architecture; Intel Virtualization Technology.	3.50 GHz		GP: 32, 64 FPU: 80 MMX: 64 XMM: 128 YMM: 256	DMI: 5 GT/s; Memory: 21 GB/s	1 TB	L1: 64 KB L2: 256 KB L3: 8 MB
Intel Xeon-Processor E7-8870	2011	Intel Turbo Boost Technology, Westmere microarchitecture; Ten core; HyperThreading Technology; Intel 64 Architecture; Intel Virtualization Technology.	2.40 GHz	2.2 B	GP: 32, 64 FPU: 80 MMX: 64 XMM: 128	QPI: 6.4 GT/s; Memory: 102 GB/s	16 TB	L1: 64 KB L2: 256 KB L3: 30 MB

NOTES:

1. The 64-bit Intel Xeon Processor MP with an 8-MByte L3 supports a multi-processor platform with a dual system bus; this creates a platform bandwidth with 10.6 GBytes.
2. In Intel Pentium Processor Extreme Edition 840, the size of on-die cache is listed for each core. The total size of L2 in the physical package in 2 MBytes.
3. In Dual-Core Intel Xeon Processor 7041, the size of on-die cache is listed for each core. The total size of L2 in the physical package in 4 MBytes.
4. In Intel Atom Processor, the size of L1 instruction cache is 32 KBytes, L1 data cache is 24 KBytes.
5. In Intel Atom Processor, the size of L1 instruction cache is 32 KBytes, L1 data cache is 24 KBytes.

Table 2-3. Key Features of Previous Generations of IA-32 Processors

Intel Processor	Date Intro-duced	Max. Clock Frequency/ Technology at Introduction	Tran-sistors	Register Sizes[1]	Ext. Data Bus Size[2]	Max. Extern. Addr. Space	Caches
8086	1978	8 MHz	29 K	16 GP	16	1 MB	None
Intel 286	1982	12.5 MHz	134 K	16 GP	16	16 MB	Note 3
Intel386 DX Processor	1985	20 MHz	275 K	32 GP	32	4 GB	Note 3
Intel486 DX Processor	1989	25 MHz	1.2 M	32 GP 80 FPU	32	4 GB	L1: 8 KB
Pentium Processor	1993	60 MHz	3.1 M	32 GP 80 FPU	64	4 GB	L1: 16 KB
Pentium Pro Processor	1995	200 MHz	5.5 M	32 GP 80 FPU	64	64 GB	L1: 16 KB L2: 256 KB or 512 KB
Pentium II Processor	1997	266 MHz	7 M	32 GP 80 FPU 64 MMX	64	64 GB	L1: 32 KB L2: 256 KB or 512 KB
Pentium III Processor	1999	500 MHz	8.2 M	32 GP 80 FPU 64 MMX 128 XMM	64	64 GB	L1: 32 KB L2: 512 KB
Pentium III and Pentium III Xeon Processors	1999	700 MHz	28 M	32 GP 80 FPU 64 MMX 128 XMM	64	64 GB	L1: 32 KB L2: 256 KB
Pentium 4 Processor	2000	1.50 GHz, Intel NetBurst Microarchitecture	42 M	32 GP 80 FPU 64 MMX 128 XMM	64	64 GB	12K µop Execution Trace Cache; L1: 8 KB L2: 256 KB
Intel Xeon Processor	2001	1.70 GHz, Intel NetBurst Microarchitecture	42 M	32 GP 80 FPU 64 MMX 128 XMM	64	64 GB	12K µop Execution Trace Cache; L1: 8 KB L2: 512 KB
Intel Xeon Processor	2002	2.20 GHz, Intel NetBurst Microarchitecture, HyperThreading Technology	55 M	32 GP 80 FPU 64 MMX 128 XMM	64	64 GB	12K µop Execution Trace Cache; L1: 8 KB L2: 512 KB
Pentium M Processor	2003	1.60 GHz, Intel NetBurst Microarchitecture	77 M	32 GP 80 FPU 64 MMX 128 XMM	64	4 GB	L1: 64 KB L2: 1 MB

Table 2-3. Key Features of Previous Generations of IA-32 Processors (Contd.)

Intel Pentium 4 Processor Supporting Hyper-Threading Technology at 90 nm process	2004	3.40 GHz, Intel NetBurst Microarchitecture, HyperThreading Technology	125 M	32 GP 80 FPU 64 MMX 128 XMM	64	64 GB	12K µop Execution Trace Cache; L1: 16 KB L2: 1 MB

NOTES:

1. The register size and external data bus size are given in bits. Note also that each 32-bit general-purpose (GP) registers can be addressed as an 8- or a 16-bit data registers in all of the processors.
2. Internal data paths are 2 to 4 times wider than the external data bus for each processor.

2.4 PLANNED REMOVAL OF INTEL® INSTRUCTION SET ARCHITECTURE AND FEATURES FROM UPCOMING PRODUCTS

This section lists Intel Instruction Set Architecture (ISA) and features that Intel plans to remove from select products starting from a specific year.

Table 2-4. Planned Intel® ISA and Features Removal List

Intel ISA/Feature	Expected Year of Removal
Uncore PMI (as enabled by IA32_DEBUGCTL[13])	2027 onwards
xAPIC mode	2027 onwards
User interrupts	To be determined

2.5 INTEL® INSTRUCTION SET ARCHITECTURE AND FEATURES REMOVED

This section lists Intel ISA and features that Intel has already removed for select upcoming products. All sections relevant to the removed features will be identified as such and may be moved to an archived section in future Intel® 64 and IA-32 Architectures Software Developer's Manual releases.

Table 2-5. Intel® ISA and Features Removal List

Intel ISA/Feature	Year of Removal
Intel® Memory Protection Extensions (Intel® MPX)	2019 onwards
Disable assertion of split locks (as enabled by MSR_TEST_CTRL[31])	2019 onwards
Hardware Lock Elision (HLE)	2019 onwards
VP2INTERSECT	2023 onwards
Sub-page write permissions for EPT	2024 onwards
Key Locker	2025 onwards

CHAPTER 3
BASIC EXECUTION ENVIRONMENT

This chapter describes the basic execution environment of an Intel 64 or IA-32 processor as seen by assembly-language programmers. It describes how the processor executes instructions and how it stores and manipulates data. The execution environment described here includes memory (the address space), general-purpose data registers, segment registers, the flag register, and the instruction pointer register.

3.1 MODES OF OPERATION

The IA-32 architecture supports three basic operating modes: protected mode, real-address mode, and system management mode. The operating mode determines which instructions and architectural features are accessible:

- **Protected mode** — This mode is the native state of the processor. Among the capabilities of protected mode is the ability to directly execute "real-address mode" 8086 software in a protected, multi-tasking environment. This feature is called **virtual-8086 mode**, although it is not actually a processor mode. Virtual-8086 mode is actually a protected mode attribute that can be enabled for any task.
- **Real-address mode** — This mode implements the programming environment of the Intel 8086 processor with extensions (such as the ability to switch to protected or system management mode). The processor is placed in real-address mode following power-up or a reset.
- **System management mode (SMM)** — This mode provides an operating system or executive with a transparent mechanism for implementing platform-specific functions such as power management and system security. The processor enters SMM when the external SMM interrupt pin (SMI#) is activated or an SMI is received from the advanced programmable interrupt controller (APIC).

  In SMM, the processor switches to a separate address space while saving the basic context of the currently running program or task. SMM-specific code may then be executed transparently. Upon returning from SMM, the processor is placed back into its state prior to the system management interrupt. SMM was introduced with the Intel386™ SL and Intel486™ SL processors and became a standard IA-32 feature with the Pentium processor family.

3.1.1 Intel® 64 Architecture

Intel 64 architecture adds IA-32e mode. IA-32e mode has two sub-modes.
These are:

- **Compatibility mode (sub-mode of IA-32e mode)** — Compatibility mode permits most legacy 16-bit and 32-bit applications to run without re-compilation under a 64-bit operating system. For brevity, the compatibility sub-mode is referred to as compatibility mode in IA-32 architecture. The execution environment of compatibility mode is the same as described in Section 3.2. Compatibility mode also supports all of the privilege levels that are supported in 64-bit and protected modes. Legacy applications that run in Virtual 8086 mode or use hardware task management will not work in this mode.

  Compatibility mode is enabled by the operating system (OS) on a code segment basis. This means that a single 64-bit OS can support 64-bit applications running in 64-bit mode and support legacy 32-bit applications (not recompiled for 64-bits) running in compatibility mode.

  Compatibility mode is similar to 32-bit protected mode. Applications access only the first 4 GByte of linear-address space. Compatibility mode uses 16-bit and 32-bit address and operand sizes. Like protected mode, this mode allows applications to access physical memory greater than 4 GByte using PAE (Physical Address Extensions).
- **64-bit mode (sub-mode of IA-32e mode)** — This mode enables a 64-bit operating system to run applications written to access 64-bit linear address space. For brevity, the 64-bit sub-mode is referred to as 64-bit mode in IA-32 architecture.

64-bit mode extends the number of general purpose registers and SIMD extension registers from 8 to 16. General purpose registers are widened to 64 bits. The mode also introduces a new opcode prefix (REX) to access the register extensions. See Section 3.2.1 for a detailed description.

64-bit mode is enabled by the operating system on a code-segment basis. Its default address size is 64 bits and its default operand size is 32 bits. The default operand size can be overridden on an instruction-by-instruction basis using a REX opcode prefix in conjunction with an operand size override prefix.

REX prefixes allow a 64-bit operand to be specified when operating in 64-bit mode. By using this mechanism, many existing instructions have been promoted to allow the use of 64-bit registers and 64-bit addresses.

3.2 OVERVIEW OF THE BASIC EXECUTION ENVIRONMENT

Any program or task running on an IA-32 processor is given a set of resources for executing instructions and for storing code, data, and state information. These resources (described briefly in the following paragraphs and shown in Figure 3-1) make up the basic execution environment for an IA-32 processor.

An Intel 64 processor supports the basic execution environment of an IA-32 processor, and a similar environment under IA-32e mode that can execute 64-bit programs (64-bit sub-mode) and 32-bit programs (compatibility sub-mode).

The basic execution environment is used jointly by the application programs and the operating system or executive running on the processor.

- **Address space** — Any task or program running on an IA-32 processor can address a linear address space of up to 4 GBytes (2^{32} bytes) and a physical address space of up to 64 GBytes (2^{36} bytes). See Section 3.3.6, "Extended Physical Addressing in Protected Mode," for more information about addressing an address space greater than 4 GBytes.
- **Basic program execution registers** — The eight general-purpose registers, the six segment registers, the EFLAGS register, and the EIP (instruction pointer) register comprise a basic execution environment in which to execute a set of general-purpose instructions. These instructions perform basic integer arithmetic on byte, word, and doubleword integers, handle program flow control, operate on bit and byte strings, and address memory. See Section 3.4, "Basic Program Execution Registers," for more information about these registers.
- **x87 FPU registers** — The eight x87 FPU data registers, the x87 FPU control register, the status register, the x87 FPU instruction pointer register, the x87 FPU operand (data) pointer register, the x87 FPU tag register, and the x87 FPU opcode register provide an execution environment for operating on single precision, double precision, and double extended precision floating-point values, word integers, doubleword integers, quadword integers, and binary coded decimal (BCD) values. See Section 8.1, "x87 FPU Execution Environment," for more information about these registers.
- **MMX registers** — The eight MMX registers support execution of single-instruction, multiple-data (SIMD) operations on 64-bit packed byte, word, and doubleword integers. See Section 9.2, "The MMX Technology Programming Environment," for more information about these registers.
- **XMM registers** — The eight XMM data registers and the MXCSR register support execution of SIMD operations on 128-bit packed single precision and double precision floating-point values and on 128-bit packed byte, word, doubleword, and quadword integers. See Section 10.2, "Intel® SSE Programming Environment," for more information about these registers.
- **YMM registers** — The YMM data registers support execution of 256-bit SIMD operations on 256-bit packed single precision and double precision floating-point values and on 256-bit packed byte, word, doubleword, and quadword integers.
- **Bounds registers** — Each of the BND0-BND3 register stores the lower and upper **bounds** (64 bits each) associated with the pointer to a memory buffer. They support execution of the Intel MPX instructions.
- **BNDCFGU and BNDSTATUS**— BNDCFGU configures user mode MPX operations on bounds checking. BNDSTATUS provides additional information on the #BR caused by an MPX operation.

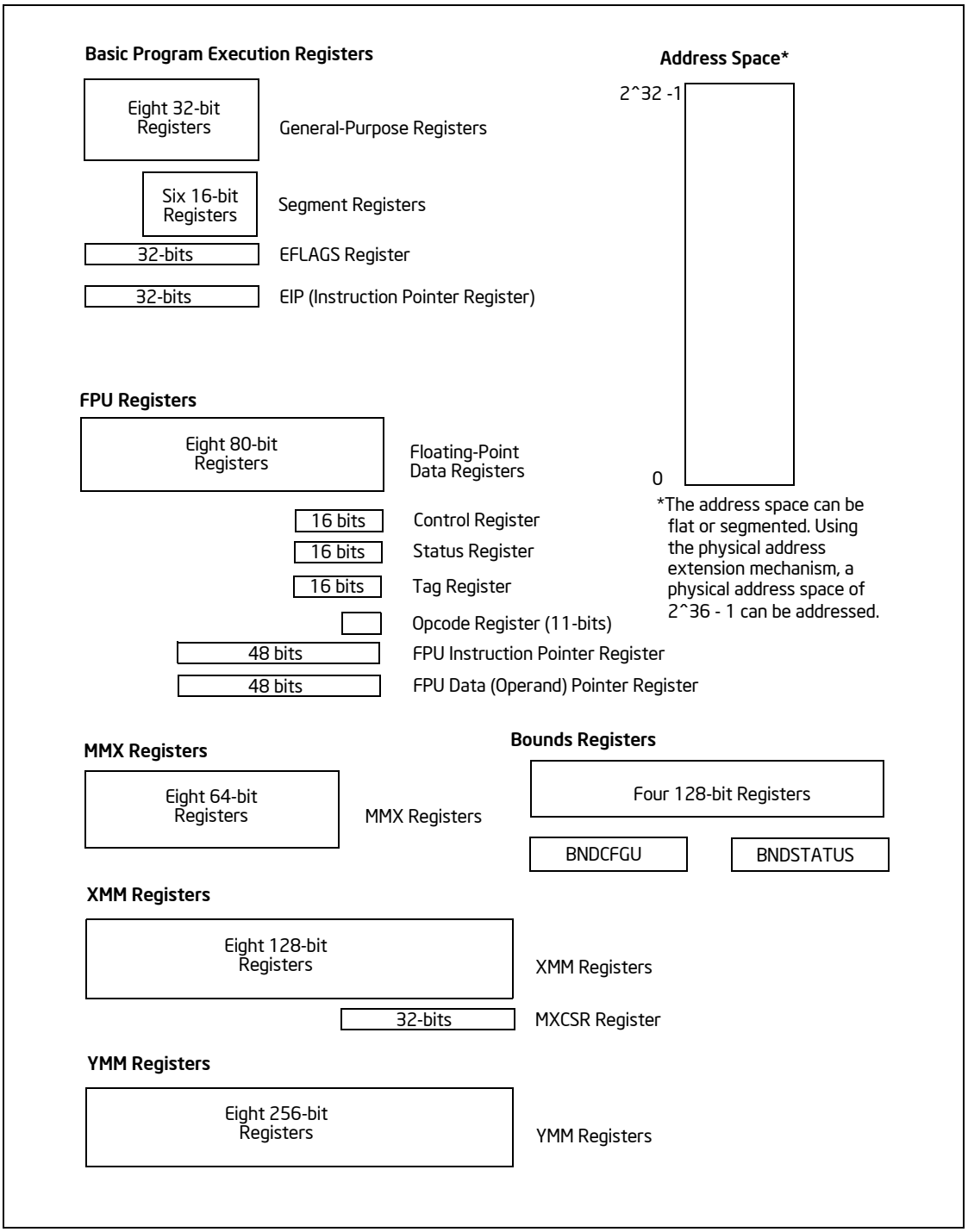


Figure 3-1. IA-32 Basic Execution Environment for Non-64-Bit Modes

- **Stack** — To support procedure or subroutine calls and the passing of parameters between procedures or subroutines, a stack and stack management resources are included in the execution environment. The stack (not shown in Figure 3-1) is located in memory. See Section 6.2, "Stacks," for more information about stack structure.

In addition to the resources provided in the basic execution environment, the IA-32 architecture provides the following resources as part of its system-level architecture. They provide extensive support for operating-system and system-development software. Except for the I/O ports, the system resources are described in detail in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volumes 3A, 3B, 3C & 3D.

- **I/O ports** — The IA-32 architecture supports a transfers of data to and from input/output (I/O) ports. See Chapter 20, "Input/Output," in this volume.
- **Control registers** — The five control registers (CR0 through CR4) determine the operating mode of the processor and the characteristics of the currently executing task. See Chapter 2, "System Architecture Overview," in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3A.
- **Memory management registers** — The GDTR, IDTR, task register, and LDTR specify the locations of data structures used in protected mode memory management. See Chapter 2, "System Architecture Overview," in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3A.
- **Debug registers** — The debug registers (DR0 through DR7) control and allow monitoring of the processor's debugging operations. See in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3B.
- **Memory type range registers (MTRRs)** — The MTRRs are used to assign memory types to regions of memory. See the sections on MTRRs in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volumes 3A, 3B, 3C & 3D.
- **Model-specific registers (MSRs)** — The processor provides a variety of model-specific registers that are used to control and report on processor performance. Virtually all MSRs handle system related functions and are not accessible to an application program. One exception to this rule is the time-stamp counter. The MSRs are described in Chapter 2, "Model-Specific Registers (MSRs)," of the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 4.
- **Machine check registers** — The machine check registers consist of a set of control, status, and error-reporting MSRs that are used to detect and report on hardware (machine) errors. See Chapter 18, "Machine-Check Architecture," of the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3A.
- **Performance monitoring counters** — The performance monitoring counters allow processor performance events to be monitored. See Chapter 22, "Performance Monitoring," in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3B.

The remainder of this chapter describes the organization of memory and the address space, the basic program execution registers, and addressing modes. Refer to the following chapters in this volume for descriptions of the other program execution resources shown in Figure 3-1:

- **X87 FPU registers** — See Chapter 8, "Programming with the x87 FPU."
- **MMX Registers** — See Chapter 9, "Programming with Intel® MMX™ Technology."
- **XMM registers** — See Chapter 10, "Programming with Intel® Streaming SIMD Extensions (Intel® SSE)," Chapter 11, "Programming with Intel® Streaming SIMD Extensions 2 (Intel® SSE2)," and Chapter 12, "Programming with Intel® SSE3, SSSE3, Intel® SSE4, and Intel® AES-NI."
- **YMM registers** — See Chapter 14, "Programming with Intel® AVX, FMA, and Intel® AVX2."
- **BND registers, BNDCFGU, BNDSTATUS** — See Chapter 13, "Managing State Using the XSAVE Feature Set," and Appendix E, "Intel® Memory Protection Extensions."
- **Stack implementation and procedure calls** — See Chapter 6, "Procedure Calls, Interrupts, and Exceptions."

3.2.1 64-Bit Mode Execution Environment

The execution environment for 64-bit mode is similar to that described in Section 3.2. The following paragraphs describe the differences that apply.

- **Address space** — A task or program running in 64-bit mode on an IA-32 processor can address linear address space of up to 2^{64} bytes (subject to the canonical addressing requirement described in Section 3.3.7.1) and physical address space of up to 2^{52} bytes. Software can query CPUID for the physical address size supported by a processor.
- **Basic program execution registers** — The number of general-purpose registers (GPRs) available is 16. GPRs are 64-bits wide and they support operations on byte, word, doubleword, and quadword integers. Accessing byte registers is done uniformly to the lowest 8 bits. The instruction pointer register becomes 64 bits. The EFLAGS register is extended to 64 bits wide, and is referred to as the RFLAGS register. The upper 32 bits of RFLAGS is reserved. The lower 32 bits of RFLAGS is the same as EFLAGS. See Figure 3-2.
- **XMM registers** — There are 16 XMM data registers for SIMD operations. See Section 10.2, "Intel® SSE Programming Environment," for more information about these registers.
- **YMM registers** — There are 16 YMM data registers for SIMD operations. See Chapter 14, "Programming with Intel® AVX, FMA, and Intel® AVX2," for more information about these registers.
- **BND registers, BNDCFGU, BNDSTATUS** — See Chapter 13, "Managing State Using the XSAVE Feature Set," and Appendix E, "Intel® Memory Protection Extensions."
- **Stack** — The stack pointer size is 64 bits. Stack size is not controlled by a bit in the SS descriptor (as it is in non-64-bit modes) nor can the pointer size be overridden by an instruction prefix.
- **Control registers** — Control registers expand to 64 bits. A new control register (the task priority register: CR8 or TPR) has been added. See Chapter 2, "Intel® 64 and IA-32 Architectures," in this volume.
- **Debug registers** — Debug registers expand to 64 bits. See Chapter 20, "Debug, Branch Profile, TSC, and Intel® Resource Director Technology (Intel® RDT) Features," in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3B.

- **Descriptor table registers** — The global descriptor table register (GDTR) and interrupt descriptor table register (IDTR) expand to 10 bytes so that they can hold a full 64-bit base address. The local descriptor table register (LDTR) and the task register (TR) also expand to hold a full 64-bit base address.

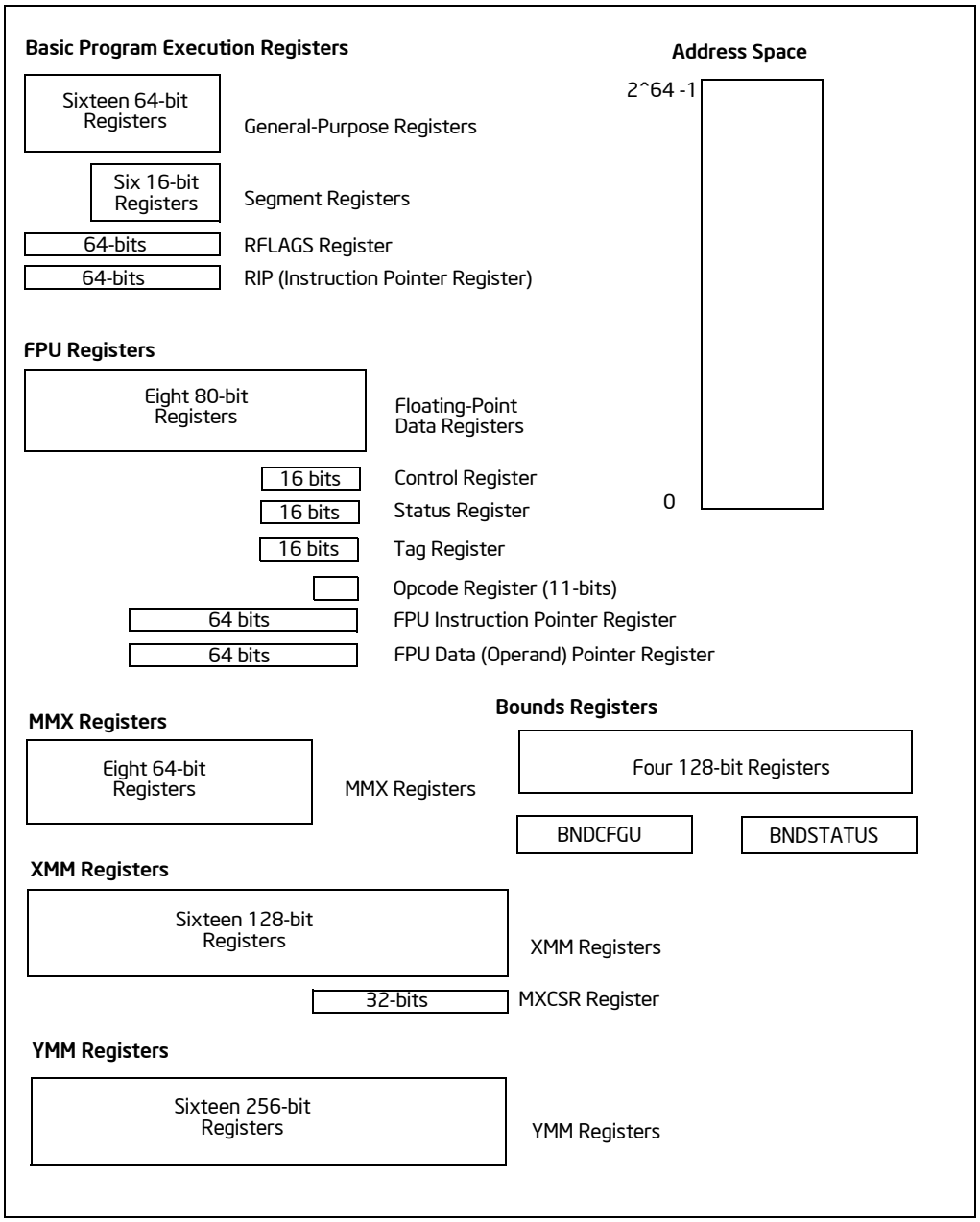


Figure 3-2. 64-Bit Mode Execution Environment

3.3 MEMORY ORGANIZATION

The memory that the processor addresses on its bus is called **physical memory**. Physical memory is organized as a sequence of 8-bit bytes. Each byte is assigned a unique address, called a **physical address**. The **physical address space** ranges from zero to a maximum of $2^{36} - 1$ (64 GBytes) if the processor does not support Intel 64 architecture. Intel 64 architecture introduces a set of changes in physical and linear address space; these are described in Section 3.3.3, Section 3.3.4, and Section 3.3.7.

Virtually any operating system or executive designed to work with an IA-32 or Intel 64 processor will use the processor's memory management facilities to access memory. These facilities provide features such as segmentation and paging, which allow memory to be managed efficiently and reliably. Memory management is described in detail in Chapter 3, "Protected-Mode Memory Management," in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3A. The following paragraphs describe the basic methods of addressing memory when memory management is used.

3.3.1 IA-32 Memory Models

When employing the processor's memory management facilities, programs do not directly address physical memory. Instead, they access memory using one of three memory models: flat, segmented, or real address mode:

- **Flat memory model** — Memory appears to a program as a single, continuous address space (Figure 3-3). This space is called a **linear address space**. Code, data, and stacks are all contained in this address space. Linear address space is byte addressable, with addresses running contiguously from 0 to 2^{32} - 1 (if not in 64-bit mode). An address for any byte in linear address space is called a **linear address**.
- **Segmented memory model** — Memory appears to a program as a group of independent address spaces called segments. Code, data, and stacks are typically contained in separate segments. To address a byte in a segment, a program issues a logical address. This consists of a segment selector and an offset (logical addresses are often referred to as far pointers). The segment selector identifies the segment to be accessed and the offset identifies a byte in the address space of the segment. Programs running on an IA-32 processor can address up to 16,383 segments of different sizes and types, and each segment can be as large as 2^{32} bytes.

  Internally, all the segments that are defined for a system are mapped into the processor's linear address space. To access a memory location, the processor thus translates each logical address into a linear address. This translation is transparent to the application program.

  The primary reason for using segmented memory is to increase the reliability of programs and systems. For example, placing a program's stack in a separate segment prevents the stack from growing into the code or data space and overwriting instructions or data, respectively.
- **Real-address mode memory model** — This is the memory model for the Intel 8086 processor. It is supported to provide compatibility with existing programs written to run on the Intel 8086 processor. The real-address mode uses a specific implementation of segmented memory in which the linear address space for the program and the operating system/executive consists of an array of segments of up to 64 KBytes in size each. The maximum size of the linear address space in real-address mode is 2^{20} bytes.

  See also: Chapter 23, "8086 Emulation," in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3B.

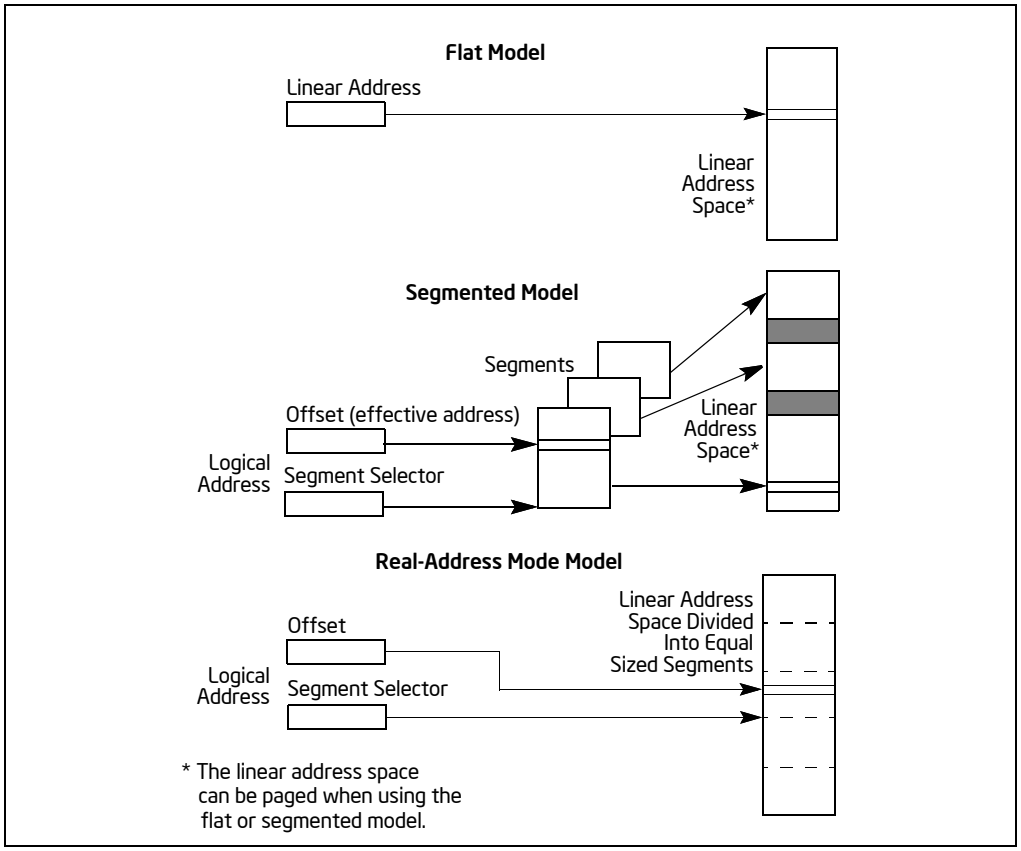


Figure 3-3. Three Memory Management Models

3.3.2 Paging and Virtual Memory

With the flat or the segmented memory model, linear address space is mapped into the processor's physical address space either directly or through paging. When using direct mapping (paging disabled), each linear address has a one-to-one correspondence with a physical address. Linear addresses are sent out on the processor's address lines without translation.

When using the IA-32 architecture's paging mechanism (paging enabled), linear address space is divided into pages which are mapped to virtual memory. The pages of virtual memory are then mapped as needed into physical memory. When an operating system or executive uses paging, the paging mechanism is transparent to an application program. All that the application sees is linear address space.

In addition, IA-32 architecture's paging mechanism includes extensions that support:

- Physical Address Extensions (PAE) to address physical address space greater than 4 GBytes.
- Page Size Extensions (PSE) to map linear address to physical address in 4-MBytes pages.

See also: Chapter 3, "Protected-Mode Memory Management," in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3A.

3.3.3 Memory Organization in 64-Bit Mode

Intel 64 architecture supports physical address space greater than 64 GBytes; the actual physical address size of IA-32 processors is implementation specific. In 64-bit mode, there is architectural support for 64-bit linear address space. However, processors supporting Intel 64 architecture may implement less than 64-bits (see Section 3.3.7.1). The linear address space is mapped into the processor physical address space through the PAE paging mechanism.

3.3.4 Modes of Operation vs. Memory Model

When writing code for an IA-32 or Intel 64 processor, a programmer needs to know the operating mode the processor is going to be in when executing the code and the memory model being used. The relationship between operating modes and memory models is as follows:

- **Protected mode** — When in protected mode, the processor can use any of the memory models described in this section. (The real-addressing mode memory model is ordinarily used only when the processor is in the virtual-8086 mode.) The memory model used depends on the design of the operating system or executive. When multitasking is implemented, individual tasks can use different memory models.
- **Real-address mode** — When in real-address mode, the processor only supports the real-address mode memory model.
- **System management mode** — When in SMM, the processor switches to a separate address space, called the system management RAM (SMRAM). The memory model used to address bytes in this address space is similar to the real-address mode model. See Chapter 34, "System Management Mode," in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3C, for more information on the memory model used in SMM.
- **Compatibility mode** — Software that needs to run in compatibility mode should observe the same memory model as those targeted to run in 32-bit protected mode. The effect of segmentation is the same as it is in 32-bit protected mode semantics.
- **64-bit mode** — Segmentation is generally (but not completely) disabled, creating a flat 64-bit linear-address space. Specifically, the processor treats the segment base of CS, DS, ES, and SS as zero in 64-bit mode (this makes a linear address equal an effective address). Segmented and real address modes are not available in 64-bit mode.

3.3.5 32-Bit and 16-Bit Address and Operand Sizes

IA-32 processors in protected mode can be configured for 32-bit or 16-bit address and operand sizes. With 32-bit address and operand sizes, the maximum linear address or segment offset is FFFFFFFFH (2^{32}-1); operand sizes are typically 8 bits or 32 bits. With 16-bit address and operand sizes, the maximum linear address or segment offset is FFFFH (2^{16}-1); operand sizes are typically 8 bits or 16 bits.

When using 32-bit addressing, a logical address (or far pointer) consists of a 16-bit segment selector and a 32-bit offset; when using 16-bit addressing, an address consists of a 16-bit segment selector and a 16-bit offset.

Instruction prefixes allow temporary overrides of the default address and/or operand sizes from within a program.

When operating in protected mode, the segment descriptor for the currently executing code segment defines the default address and operand size. A segment descriptor is a system data structure not normally visible to application code. Assembler directives allow the default addressing and operand size to be chosen for a program. The assembler and other tools then set up the segment descriptor for the code segment appropriately.

When operating in real-address mode, the default addressing and operand size is 16 bits. An address-size override can be used in real-address mode to enable 32-bit addressing. However, the maximum allowable 32-bit linear address is still 000FFFFFH (2^{20}-1).

3.3.6 Extended Physical Addressing in Protected Mode

Beginning with P6 family processors, the IA-32 architecture supports addressing of up to 64 GBytes (2^{36} bytes) of physical memory. A program or task could not address locations in this address space directly. Instead, it addresses individual linear address spaces of up to 4 GBytes that mapped to 64-GByte physical address space through a virtual memory management mechanism. Using this mechanism, an operating system can enable a program to switch 4-GByte linear address spaces within 64-GByte physical address space.

The use of extended physical addressing requires the processor to operate in protected mode and the operating system to provide a virtual memory management system. See "36-Bit Physical Addressing Using the PAE Paging Mechanism" in Chapter 3, "Protected-Mode Memory Management," in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3A.

3.3.7 Address Calculations in 64-Bit Mode

In most cases, 64-bit mode uses flat address space for code, data, and stacks. In 64-bit mode (if there is no address-size override), the size of effective address calculations is 64 bits. An effective-address calculation uses a 64-bit base and index registers and sign-extend displacements to 64 bits.

In the flat address space of 64-bit mode, linear addresses are equal to effective addresses because the base address is zero. In the event that FS or GS segments are used with a non-zero base, this rule does not hold. In 64-bit mode, the effective address components are added and the effective address is truncated (See for example the instruction LEA) before adding the full 64-bit segment base. The base is never truncated, regardless of addressing mode in 64-bit mode.

The instruction pointer is extended to 64 bits to support 64-bit code offsets. The 64-bit instruction pointer is called the RIP. Table 3-1 shows the relationship between RIP, EIP, and IP.

Table 3-1. Instruction Pointer Sizes

	Bits 63:32	Bits 31:16	Bits 15:0
16-bit instruction pointer	Not Modified		IP
32-bit instruction pointer	Zero Extension	EIP	
64-bit instruction pointer	RIP		

Generally, displacements and immediates in 64-bit mode are not extended to 64 bits. They are still limited to 32 bits and sign-extended during effective-address calculations. In 64-bit mode, however, support is provided for 64-bit displacement and immediate forms of the MOV instruction.

All 16-bit and 32-bit address calculations are zero-extended in IA-32e mode to form 64-bit addresses. Address calculations are first truncated to the effective address size of the current mode (64-bit mode or compatibility mode), as overridden by any address-size prefix. The result is then zero-extended to the full 64-bit address width. Because of this, 16-bit and 32-bit applications running in compatibility mode can access only the low 4 GBytes of the 64-bit mode effective addresses. Likewise, a 32-bit address generated in 64-bit mode can access only the low 4 GBytes of the 64-bit mode effective addresses.

3.3.7.1 Canonical Addressing

In 64-bit mode, an address is considered to be in canonical form if address bits 63 through to the most-significant implemented bit by the microarchitecture are set to either all ones or all zeros.

Intel 64 architecture defines a 64-bit linear address. Implementations can support less. The first implementation of IA-32 processors with Intel 64 architecture supports a 48-bit linear address. This means a canonical address must have bits 63 through 48 set to zeros or ones (depending on whether bit 47 is a zero or one).

Although implementations may not use all 64 bits of the linear address, they should check bits 63 through the most-significant implemented bit to see if the address is in canonical form. If a linear-memory reference is not in canonical form, the implementation should generate an exception. In most cases, a general-protection exception (#GP) is generated. However, in the case of explicit or implied stack references, a stack fault (#SS) is generated.

Instructions that have implied stack references, by default, use the SS segment register. These include PUSH/POP-related instructions and instructions using RSP/RBP as base registers. In these cases, the canonical fault is #SS.

If an instruction uses base registers RSP/RBP and uses a segment override prefix to specify a non-SS segment, a canonical fault generates a #GP (instead of an #SS). In 64-bit mode, only FS and GS segment-overrides are applicable in this situation. Other segment override prefixes (CS, DS, ES, and SS) are ignored. Note that this also means that an SS segment-override applied to a "non-stack" register reference is ignored. Such a sequence still produces a #GP for a canonical fault (and not an #SS).

3.4 BASIC PROGRAM EXECUTION REGISTERS

IA-32 architecture provides 16 basic program execution registers for use in general system and application programing (see Figure 3-4). These registers can be grouped as follows:

- **General-purpose registers**. These eight registers are available for storing operands and pointers.
- **Segment registers**. These registers hold up to six segment selectors.
- **EFLAGS (program status and control) register**. The EFLAGS register report on the status of the program being executed and allows limited (application-program level) control of the processor.
- **EIP (instruction pointer) register**. The EIP register contains a 32-bit pointer to the next instruction to be executed.

3.4.1 General-Purpose Registers

The 32-bit general-purpose registers EAX, EBX, ECX, EDX, ESI, EDI, EBP, and ESP are provided for holding the following items:

- Operands for logical and arithmetic operations.
- Operands for address calculations.
- Memory pointers.

Although all of these registers are available for general storage of operands, results, and pointers, caution should be used when referencing the ESP register. The ESP register holds the stack pointer and as a general rule should not be used for another purpose.

Many instructions assign specific registers to hold operands. For example, string instructions use the contents of the ECX, ESI, and EDI registers as operands. When using a segmented memory model, some instructions assume that pointers in certain registers are relative to specific segments. For instance, some instructions assume that a pointer in the EBX register points to a memory location in the DS segment.

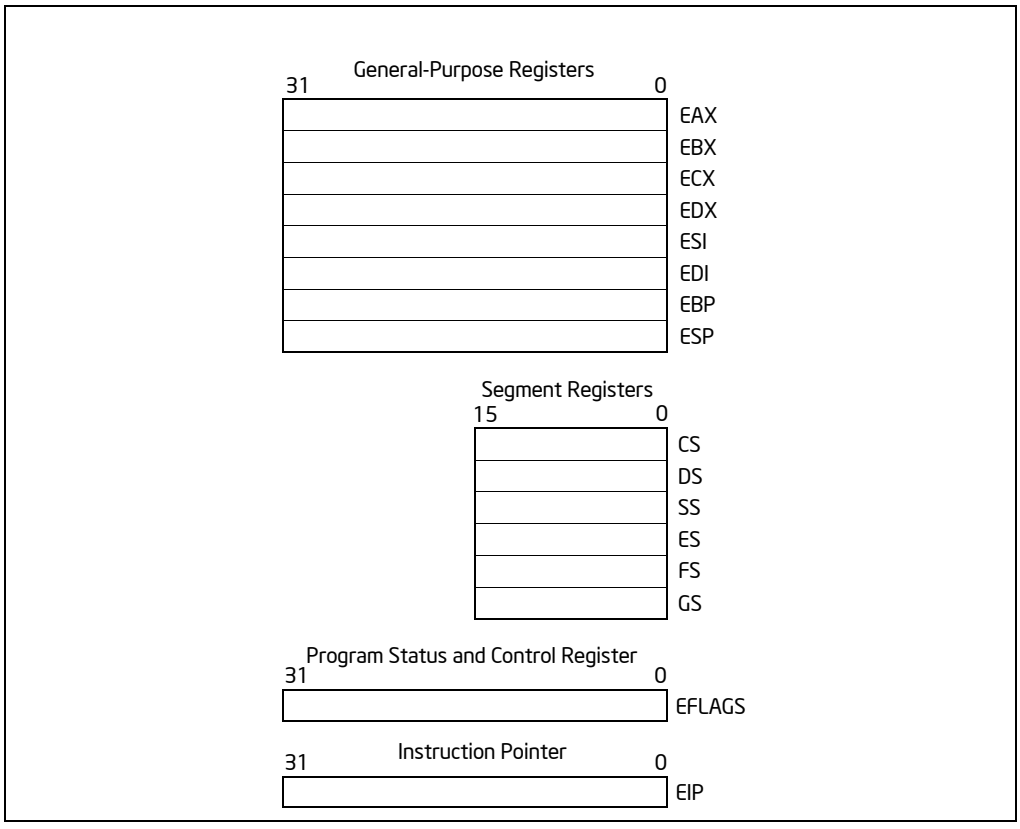


Figure 3-4. General System and Application Programming Registers

The special uses of general-purpose registers by instructions are described in Chapter 5, "Instruction Set Summary," in this volume. See also: Chapter 3, Chapter 4, Chapter 5, and Chapter 6 of the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volumes 2A, 2B, 2C, & 2D. The following is a summary of special uses:

- **EAX** — Accumulator for operands and results data.
- **EBX** — Pointer to data in the DS segment.
- **ECX** — Counter for string and loop operations.
- **EDX** — I/O pointer.
- **ESI** — Pointer to data in the segment pointed to by the DS register; source pointer for string operations.
- **EDI** — Pointer to data (or destination) in the segment pointed to by the ES register; destination pointer for string operations.
- **ESP** — Stack pointer (in the SS segment).
- **EBP** — Pointer to data on the stack (in the SS segment).

As shown in Figure 3-5, the lower 16 bits of the general-purpose registers map directly to the register set found in the 8086 and Intel 286 processors and can be referenced with the names AX, BX, CX, DX, BP, SI, DI, and SP. Each of the lower two bytes of the EAX, EBX, ECX, and EDX registers can be referenced by the names AH, BH, CH, and DH (high bytes) and AL, BL, CL, and DL (low bytes).

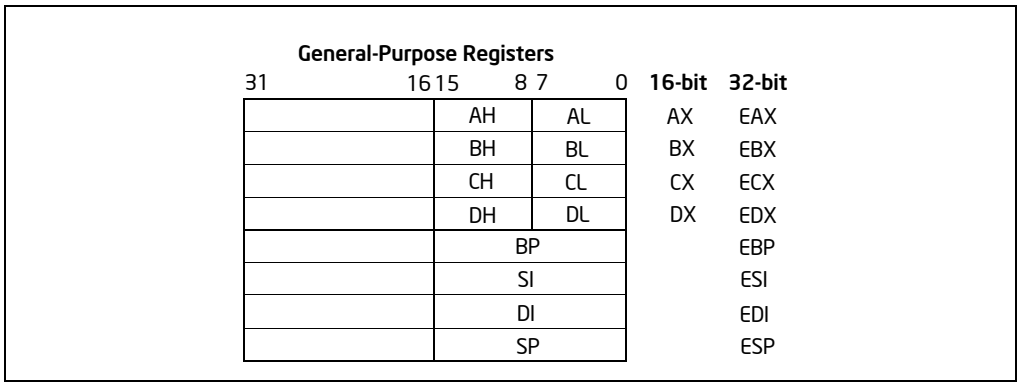


Figure 3-5. Alternate General-Purpose Register Names

3.4.1.1 General-Purpose Registers in 64-Bit Mode

In 64-bit mode, there are 16 general purpose registers and the default operand size is 32 bits. However, general-purpose registers are able to work with either 32-bit or 64-bit operands. If a 32-bit operand size is specified: EAX, EBX, ECX, EDX, EDI, ESI, EBP, ESP, R8D - R15D are available. If a 64-bit operand size is specified: RAX, RBX, RCX, RDX, RDI, RSI, RBP, RSP, R8-R15 are available. R8D-R15D/R8-R15 represent eight new general-purpose registers. All of these registers can be accessed at the byte, word, dword, and qword level. REX prefixes are used to generate 64-bit operand sizes or to reference registers R8-R15.

Registers only available in 64-bit mode (R8-R15 and XMM8-XMM15) are preserved across transitions from 64-bit mode into compatibility mode then back into 64-bit mode. However, values of R8-R15 and XMM8-XMM15 are undefined after transitions from 64-bit mode through compatibility mode to legacy or real mode and then back through compatibility mode to 64-bit mode.

Table 3-2. Addressable General Purpose Registers

Register Type	Without REX	With REX
Byte Registers	AL, BL, CL, DL, AH, BH, CH, DH	AL, BL, CL, DL, DIL, SIL, BPL, SPL, R8B - R15B
Word Registers	AX, BX, CX, DX, DI, SI, BP, SP	AX, BX, CX, DX, DI, SI, BP, SP, R8W - R15W
Doubleword Registers	EAX, EBX, ECX, EDX, EDI, ESI, EBP, ESP	EAX, EBX, ECX, EDX, EDI, ESI, EBP, ESP, R8D - R15D
Quadword Registers	N.A.	RAX, RBX, RCX, RDX, RDI, RSI, RBP, RSP, R8 - R15

In 64-bit mode, there are limitations on accessing byte registers. An instruction cannot reference legacy high-bytes (for example: AH, BH, CH, DH) and one of the new byte registers at the same time (for example: the low byte of the RAX register). However, instructions may reference legacy low-bytes (for example: AL, BL, CL, or DL) and new byte registers at the same time (for example: the low byte of the R8 register, or RBP). The architecture enforces this limitation by changing high-byte references (AH, BH, CH, DH) to low byte references (BPL, SPL, DIL, SIL: the low 8 bits for RBP, RSP, RDI, and RSI) for instructions using a REX prefix.

When in 64-bit mode, operand size determines the number of valid bits in the destination general-purpose register:

- 64-bit operands generate a 64-bit result in the destination general-purpose register.
- 32-bit operands generate a 32-bit result, zero-extended to a 64-bit result in the destination general-purpose register.
- 8-bit and 16-bit operands generate an 8-bit or 16-bit result. The upper 56 bits or 48 bits (respectively) of the destination general-purpose register are not modified by the operation. If the result of an 8-bit or 16-bit operation is intended for 64-bit address calculation, explicitly sign-extend the register to the full 64-bits.

Because the upper 32 bits of 64-bit general-purpose registers are undefined in 32-bit modes, the upper 32 bits of any general-purpose register are not preserved when switching from 64-bit mode to a 32-bit mode (to protected mode or compatibility mode). Software must not depend on these bits to maintain a value after a 64-bit to 32-bit mode switch.

3.4.2 Segment Registers

The segment registers (CS, DS, SS, ES, FS, and GS) hold 16-bit segment selectors. A segment selector is a special pointer that identifies a segment in memory. To access a particular segment in memory, the segment selector for that segment must be present in the appropriate segment register.

When writing application code, programmers generally create segment selectors with assembler directives and symbols. The assembler and other tools then create the actual segment selector values associated with these directives and symbols. If writing system code, programmers may need to create segment selectors directly. See Chapter 3, "Protected-Mode Memory Management," in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3A.

How segment registers are used depends on the type of memory management model that the operating system or executive is using. When using the flat (unsegmented) memory model, segment registers are loaded with segment selectors that point to overlapping segments, each of which begins at address 0 of the linear address space (see Figure 3-6). These overlapping segments then comprise the linear address space for the program. Typically, two overlapping segments are defined: one for code and another for data and stacks. The CS segment register points to the code segment and all the other segment registers point to the data and stack segment.

When using the segmented memory model, each segment register is ordinarily loaded with a different segment selector so that each segment register points to a different segment within the linear address space (see Figure 3-7). At any time, a program can thus access up to six segments in the linear address space. To access a segment not pointed to by one of the segment registers, a program must first load the segment selector for the segment to be accessed into a segment register.

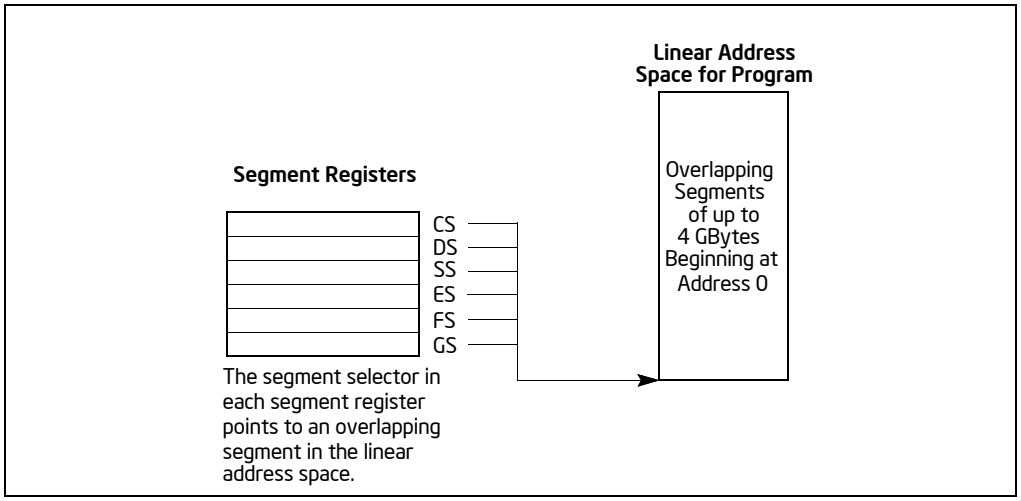


Figure 3-6. Use of Segment Registers for Flat Memory Model

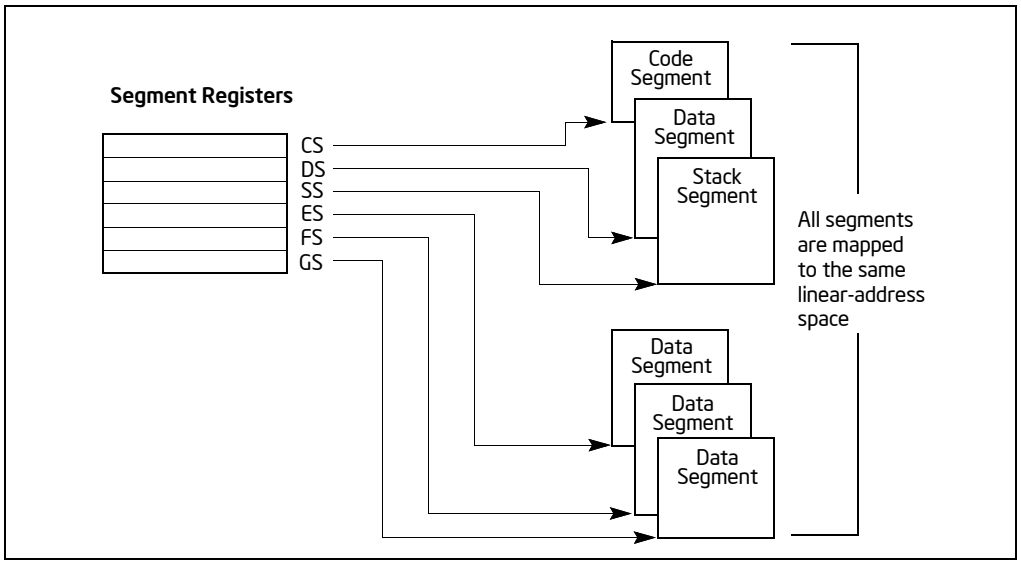


Figure 3-7. Use of Segment Registers in Segmented Memory Model

Each of the segment registers is associated with one of three types of storage: code, data, or stack. For example, the CS register contains the segment selector for the **code segment**, where the instructions being executed are stored. The processor fetches instructions from the code segment, using a logical address that consists of the segment selector in the CS register and the contents of the EIP register. The EIP register contains the offset within the code segment of the next instruction to be executed. The CS register cannot be loaded explicitly by an application program. Instead, it is loaded implicitly by instructions or internal processor operations that change program control (such as procedure calls, interrupt handling, or task switching).

The DS, ES, FS, and GS registers point to four **data segments**. The availability of four data segments permits efficient and secure access to different types of data structures. For example, four separate data segments might be created: one for the data structures of the current module, another for the data exported from a higher-level module, a third for a dynamically created data structure, and a fourth for data shared with another program. To access additional data segments, the application program must load segment selectors for these segments into the DS, ES, FS, and GS registers, as needed.

The SS register contains the segment selector for the **stack segment**, where the procedure stack is stored for the program, task, or handler currently being executed. All stack operations use the SS register to find the stack

segment. Unlike the CS register, the SS register can be loaded explicitly, which permits application programs to set up multiple stacks and switch among them.

See Section 3.3, "Memory Organization," for an overview of how the segment registers are used in real-address mode.

The four segment registers CS, DS, SS, and ES are the same as the segment registers found in the Intel 8086 and Intel 286 processors and the FS and GS registers were introduced into the IA-32 Architecture with the Intel386™ family of processors.

3.4.2.1 Segment Registers in 64-Bit Mode

In 64-bit mode: CS, DS, ES, SS are treated as if each segment base is 0, regardless of the value of the associated segment descriptor base. This creates a flat address space for code, data, and stack. FS and GS are exceptions. Both segment registers may be used as additional base registers in linear address calculations (in the addressing of local data and certain operating system data structures).

Even though segmentation is generally disabled, segment register loads may cause the processor to perform segment access assists. During these activities, enabled processors will still perform most of the legacy checks on loaded values (even if the checks are not applicable in 64-bit mode). Such checks are needed because a segment register loaded in 64-bit mode may be used by an application running in compatibility mode.

Limit checks for CS, DS, ES, SS, FS, and GS are disabled in 64-bit mode.

3.4.3 EFLAGS Register

The 32-bit EFLAGS register contains a group of status flags, a control flag, and a group of system flags. Figure 3-8 defines the flags within this register. Following initialization of the processor (either by asserting the RESET pin or the INIT pin), the state of the EFLAGS register is 00000002H. Bits 1, 3, 5, 15, and 22 through 31 of this register are reserved. Software should not use or depend on the states of any of these bits.

Some of the flags in the EFLAGS register can be modified directly, using special-purpose instructions (described in the following sections). There are no instructions that allow the whole register to be examined or modified directly.

The following instructions can be used to move groups of flags to and from the procedure stack or the EAX register: LAHF, SAHF, PUSHF, PUSHFD, POPF, and POPFD. After the contents of the EFLAGS register have been transferred to the procedure stack or EAX register, the flags can be examined and modified using the processor's bit manipulation instructions (BT, BTS, BTR, and BTC).

When suspending a task (using the processor's multitasking facilities), the processor automatically saves the state of the EFLAGS register in the task state segment (TSS) for the task being suspended. When binding itself to a new task, the processor loads the EFLAGS register with data from the new task's TSS.

When a call is made to an interrupt or exception handler procedure, the processor automatically saves the state of the EFLAGS registers on the procedure stack. When an interrupt or exception is handled with a task switch, the state of the EFLAGS register is saved in the TSS for the task being suspended.

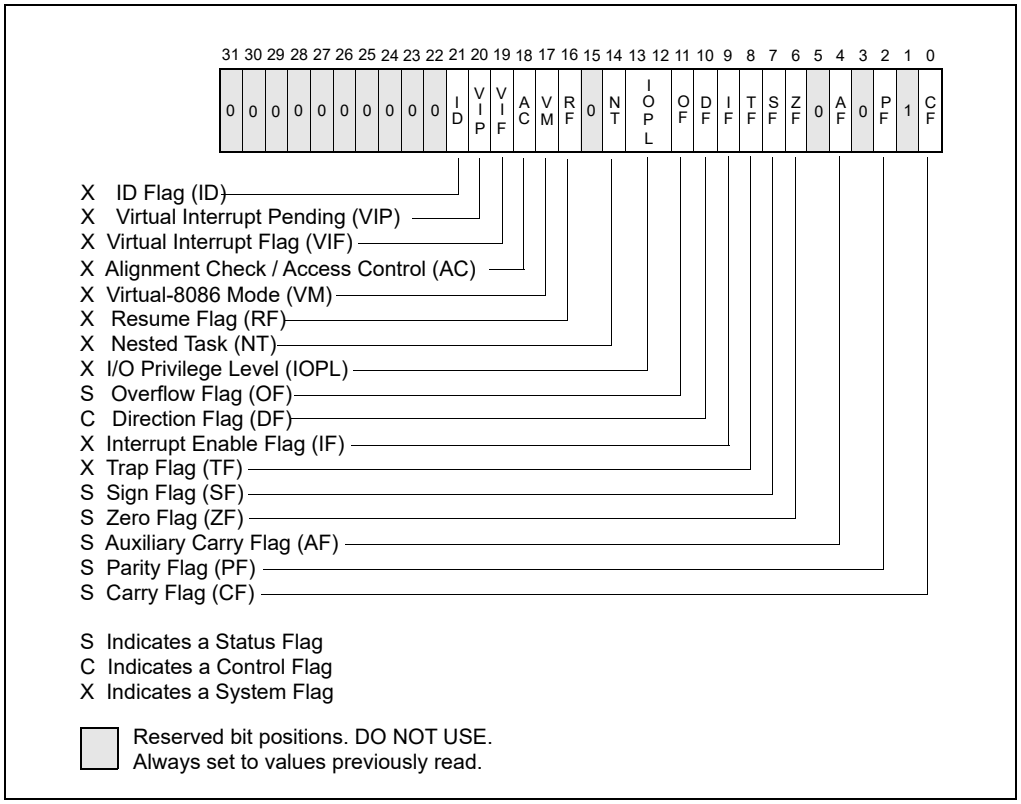


Figure 3-8. EFLAGS Register

As the IA-32 Architecture has evolved, flags have been added to the EFLAGS register, but the function and placement of existing flags have remained the same from one family of the IA-32 processors to the next. As a result, code that accesses or modifies these flags for one family of IA-32 processors works as expected when run on later families of processors.

3.4.3.1 Status Flags

The status flags (bits 0, 2, 4, 6, 7, and 11) of the EFLAGS register indicate the results of arithmetic instructions, such as the ADD, SUB, MUL, and DIV instructions. The status flag functions are:

CF (bit 0) **Carry flag** — Set if an arithmetic operation generates a carry or a borrow out of the most-significant bit of the result; cleared otherwise. This flag indicates an overflow condition for unsigned-integer arithmetic. It is also used in multiple-precision arithmetic.

PF (bit 2) **Parity flag** — Set if the least-significant byte of the result contains an even number of 1 bits; cleared otherwise.

AF (bit 4) **Auxiliary Carry flag** — Set if an arithmetic operation generates a carry or a borrow out of bit 3 of the result; cleared otherwise. This flag is used in binary-coded decimal (BCD) arithmetic.

ZF (bit 6) **Zero flag** — Set if the result is zero; cleared otherwise.

SF (bit 7) **Sign flag** — Set equal to the most-significant bit of the result, which is the sign bit of a signed integer. (0 indicates a positive value and 1 indicates a negative value.)

OF (bit 11) **Overflow flag** — Set if the integer result is too large a positive number or too small a negative number (excluding the sign-bit) to fit in the destination operand; cleared otherwise. This flag indicates an overflow condition for signed-integer (two's complement) arithmetic.

Of these status flags, only the CF flag can be modified directly, using the STC, CLC, and CMC instructions. Also the bit instructions (BT, BTS, BTR, and BTC) copy a specified bit into the CF flag.

The status flags allow a single arithmetic operation to produce results for three different data types: unsigned integers, signed integers, and BCD integers. If the result of an arithmetic operation is treated as an unsigned integer, the CF flag indicates an out-of-range condition (carry or a borrow); if treated as a signed integer (two's complement number), the OF flag indicates a carry or borrow; and if treated as a BCD digit, the AF flag indicates a carry or borrow. The SF flag indicates the sign of a signed integer. The ZF flag indicates either a signed- or an unsigned-integer zero.

When performing multiple-precision arithmetic on integers, the CF flag is used in conjunction with the add with carry (ADC) and subtract with borrow (SBB) instructions to propagate a carry or borrow from one computation to the next.

The condition instructions J*cc* (jump on condition code *cc*), SET*cc* (byte set on condition code *cc*), LOOP*cc*, and CMOV*cc* (conditional move) use one or more of the status flags as condition codes and test them for branch, set-byte, or end-loop conditions.

3.4.3.2 DF Flag

The direction flag (DF, located in bit 10 of the EFLAGS register) controls string instructions (MOVS, CMPS, SCAS, LODS, and STOS). Setting the DF flag causes the string instructions to auto-decrement (to process strings from high addresses to low addresses). Clearing the DF flag causes the string instructions to auto-increment (process strings from low addresses to high addresses).

The STD and CLD instructions set and clear the DF flag, respectively.

3.4.3.3 System Flags and IOPL Field

The system flags and IOPL field in the EFLAGS register control operating-system or executive operations. **They should not be modified by application programs**. The functions of the system flags are as follows:

TF (bit 8) — **Trap flag** — Set to enable single-step mode for debugging; clear to disable single-step mode.

IF (bit 9) — **Interrupt enable flag** — Controls the response of the processor to maskable interrupt requests. Set to respond to maskable interrupts; cleared to inhibit maskable interrupts.

IOPL (bits 12 and 13) — **I/O privilege level field** — Indicates the I/O privilege level of the currently running program or task. The current privilege level (CPL) of the currently running program or task must be less than or equal to the I/O privilege level to access the I/O address space. The POPF and IRET instructions can modify this field only when operating at a CPL of 0.

NT (bit 14) — **Nested task flag** — Controls the chaining of interrupted and called tasks. Set when the current task is linked to the previously executed task; cleared when the current task is not linked to another task.

RF (bit 16) — **Resume flag** — Controls the processor's response to debug exceptions.

VM (bit 17) — **Virtual-8086 mode flag** — Set to enable virtual-8086 mode; clear to return to protected mode without virtual-8086 mode semantics.

AC (bit 18) — **Alignment check (or access control) flag** — If the AM bit is set in the CR0 register, alignment checking of user-mode data accesses is enabled if and only if this flag is 1.

If the SMAP bit is set in the CR4 register, explicit supervisor-mode data accesses to user-mode pages are allowed if and only if this bit is 1. See Section 5.6, "Access Rights," in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3A.

VIF (bit 19) — **Virtual interrupt flag** — Virtual image of the IF flag. Used in conjunction with the VIP flag. (To use this flag and the VIP flag the virtual mode extensions are enabled by setting the VME flag in control register CR4.)

VIP (bit 20) — **Virtual interrupt pending flag** — Set to indicate that an interrupt is pending; clear when no interrupt is pending. (Software sets and clears this flag; the processor only reads it.) Used in conjunction with the VIF flag.

ID (bit 21) — **Identification flag** — The ability of a program to set or clear this flag indicates support for the CPUID instruction.

For a detailed description of these flags: see Chapter 3, "Protected-Mode Memory Management," in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3A.

3.4.3.4 RFLAGS Register in 64-Bit Mode

In 64-bit mode, EFLAGS is extended to 64 bits and called RFLAGS. The upper 32 bits of RFLAGS register is reserved. The lower 32 bits of RFLAGS is the same as EFLAGS.

3.5 INSTRUCTION POINTER

The instruction pointer (EIP) register contains the offset in the current code segment for the next instruction to be executed. It is advanced from one instruction boundary to the next in straight-line code or it is moved ahead or backwards by a number of instructions when executing JMP, J*cc*, CALL, RET, and IRET instructions.

The EIP register cannot be accessed directly by software; it is controlled implicitly by control-transfer instructions (such as JMP, J*cc*, CALL, and RET), interrupts, and exceptions. The only way to read the EIP register is to execute a CALL instruction and then read the value of the return instruction pointer from the procedure stack. The EIP register can be loaded indirectly by modifying the value of a return instruction pointer on the procedure stack and executing a return instruction (RET or IRET). See Section 6.2.4.2, "Return Instruction Pointer."

All IA-32 processors prefetch instructions. Because of instruction prefetching, an instruction address read from the bus during an instruction load does not match the value in the EIP register. Even though different processor generations use different prefetching mechanisms, the function of the EIP register to direct program flow remains fully compatible with all software written to run on IA-32 processors.

3.5.1 Instruction Pointer in 64-Bit Mode

In 64-bit mode, the RIP register becomes the instruction pointer. This register holds the 64-bit offset of the next instruction to be executed. 64-bit mode also supports a technique called RIP-relative addressing. Using this technique, the effective address is determined by adding a displacement to the RIP of the next instruction.

3.6 OPERAND-SIZE AND ADDRESS-SIZE ATTRIBUTES

When the processor is executing in protected mode, every code segment has a default operand-size attribute and address-size attribute. These attributes are selected with the D (default size) flag in the segment descriptor for the code segment (see Chapter 3, "Protected-Mode Memory Management," in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3A). When the D flag is set, the 32-bit operand-size and address-size attributes are selected; when the flag is clear, the 16-bit size attributes are selected. When the processor is executing in real-address mode, virtual-8086 mode, or SMM, the default operand-size and address-size attributes are always 16 bits.

The operand-size attribute selects the size of operands. When the 16-bit operand-size attribute is in force, operands can generally be either 8 bits or 16 bits, and when the 32-bit operand-size attribute is in force, operands can generally be 8 bits or 32 bits.

The address-size attribute selects the sizes of addresses used to address memory: 16 bits or 32 bits. When the 16-bit address-size attribute is in force, segment offsets and displacements are 16 bits. This restriction limits the size of a segment to 64 KBytes. When the 32-bit address-size attribute is in force, segment offsets and displacements are 32 bits, allowing up to 4 GBytes to be addressed.

The default operand-size attribute and/or address-size attribute can be overridden for a particular instruction by adding an operand-size and/or address-size prefix to an instruction. See Chapter 2, "Instruction Format," in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 2A. The effect of this prefix applies only to the targeted instruction.

Table 3-4 shows effective operand size and address size (when executing in protected mode or compatibility mode) depending on the settings of the D flag and the operand-size and address-size prefixes.

Table 3-3. Effective Operand- and Address-Size Attributes

D Flag in Code Segment Descriptor	0	0	0	0	1	1	1	1
Operand-Size Prefix 66H	N	N	Y	Y	N	N	Y	Y
Address-Size Prefix 67H	N	Y	N	Y	N	Y	N	Y
Effective Operand Size	16	16	32	32	32	32	16	16
Effective Address Size	16	32	16	32	32	16	32	16

NOTES:

Y: Yes - this instruction prefix is present.

N: No - this instruction prefix is not present.

3.6.1 Operand Size and Address Size in 64-Bit Mode

In 64-bit mode, the default address size is 64 bits and the default operand size is 32 bits. Defaults can be overridden using prefixes. Address-size and operand-size prefixes allow mixing of 32/64-bit data and 32/64-bit addresses on an instruction-by-instruction basis. Table 3-4 shows valid combinations of the 66H instruction prefix and the REX.W prefix that may be used to specify operand-size overrides in 64-bit mode. Note that 16-bit addresses are not supported in 64-bit mode.

REX prefixes consist of 4-bit fields that form 16 different values. The W-bit field in the REX prefixes is referred to as REX.W. If the REX.W field is properly set, the prefix specifies an operand size override to 64 bits. Note that software can still use the operand-size 66H prefix to toggle to a 16-bit operand size. However, setting REX.W takes precedence over the operand-size prefix (66H) when both are used.

In the case of SSE/SSE2/SSE3/SSSE3 SIMD instructions: the 66H, F2H, and F3H prefixes are mandatory for opcode extensions. In such a case, there is no interaction between a valid REX.W prefix and a 66H opcode extension prefix.

See Chapter 2, "Instruction Format," in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 2A.

Table 3-4. Effective Operand- and Address-Size Attributes in 64-Bit Mode

L Flag in Code Segment Descriptor	1	1	1	1	1	1	1	1
REX.W Prefix	0	0	0	0	1	1	1	1
Operand-Size Prefix 66H	N	N	Y	Y	N	N	Y	Y
Address-Size Prefix 67H	N	Y	N	Y	N	Y	N	Y
Effective Operand Size	32	32	16	16	64	64	64	64
Effective Address Size	64	32	64	32	64	32	64	32

NOTES:

Y: Yes - this instruction prefix is present.

N: No - this instruction prefix is not present.

3.7 OPERAND ADDRESSING

IA-32 machine-instructions act on zero or more operands. Some operands are specified explicitly and others are implicit. The data for a source operand can be located in:

- The instruction itself (an immediate operand).
- A register.
- A memory location.
- An I/O port.

When an instruction returns data to a destination operand, it can be returned to:

- A register.
- A memory location.
- An I/O port.

3.7.1 Immediate Operands

Some instructions use data encoded in the instruction itself as a source operand. These operands are called **immediate** operands (or simply immediates). For example, the following ADD instruction adds an immediate value of 14 to the contents of the EAX register:

```
ADD EAX, 14
```

All arithmetic instructions (except the DIV and IDIV instructions) allow the source operand to be an immediate value. The maximum value allowed for an immediate operand varies among instructions, but can never be greater than the maximum value of an unsigned doubleword integer (2^{32}).

3.7.2 Register Operands

Source and destination operands can be any of the following registers, depending on the instruction being executed:

- 32-bit general-purpose registers (EAX, EBX, ECX, EDX, ESI, EDI, ESP, or EBP).
- 16-bit general-purpose registers (AX, BX, CX, DX, SI, DI, SP, or BP).
- 8-bit general-purpose registers (AH, BH, CH, DH, AL, BL, CL, or DL).
- Segment registers (CS, DS, SS, ES, FS, and GS).
- EFLAGS register.
- X87 FPU registers (ST0 through ST7, status word, control word, tag word, data operand pointer, and instruction pointer).
- MMX registers (MM0 through MM7).
- XMM registers (XMM0 through XMM7) and the MXCSR register.
- Control registers (CR0, CR2, CR3, and CR4) and system table pointer registers (GDTR, LDTR, IDTR, and task register).
- Debug registers (DR0, DR1, DR2, DR3, DR6, and DR7).
- MSR registers.

Some instructions (such as the DIV and MUL instructions) use quadword operands contained in a pair of 32-bit registers. Register pairs are represented with a colon separating them. For example, in the register pair EDX:EAX, EDX contains the high order bits and EAX contains the low order bits of a quadword operand.

Several instructions (such as the PUSHFD and POPFD instructions) are provided to load and store the contents of the EFLAGS register or to set or clear individual flags in this register. Other instructions (such as the J*cc* instructions) use the state of the status flags in the EFLAGS register as condition codes for branching or other decision making operations.

The processor contains a selection of system registers that are used to control memory management, interrupt and exception handling, task management, processor management, and debugging activities. Some of these system registers are accessible by an application program, the operating system, or the executive through a set of system instructions. When accessing a system register with a system instruction, the register is generally an implied operand of the instruction.

3.7.2.1 Register Operands in 64-Bit Mode

Register operands in 64-bit mode can be any of the following:

- 64-bit general-purpose registers (RAX, RBX, RCX, RDX, RSI, RDI, RSP, RBP, or R8-R15).
- 32-bit general-purpose registers (EAX, EBX, ECX, EDX, ESI, EDI, ESP, EBP, or R8D-R15D).
- 16-bit general-purpose registers (AX, BX, CX, DX, SI, DI, SP, BP, or R8W-R15W).
- 8-bit general-purpose registers: AL, BL, CL, DL, SIL, DIL, SPL, BPL, and R8B-R15B are available using REX prefixes; AL, BL, CL, DL, AH, BH, CH, DH are available without using REX prefixes.
- Segment registers (CS, DS, SS, ES, FS, and GS).
- RFLAGS register.
- X87 FPU registers (ST0 through ST7, status word, control word, tag word, data operand pointer, and instruction pointer).
- MMX registers (MM0 through MM7).
- XMM registers (XMM0 through XMM15) and the MXCSR register.
- Control registers (CR0, CR2, CR3, CR4, and CR8) and system table pointer registers (GDTR, LDTR, IDTR, and task register).
- Debug registers (DR0, DR1, DR2, DR3, DR6, and DR7).
- MSR registers.
- RDX:RAX register pair representing a 128-bit operand.

3.7.3 Memory Operands

Source and destination operands in memory are referenced by means of a segment selector and an offset (see Figure 3-9). Segment selectors specify the segment containing the operand. Offsets specify the linear or effective address of the operand. Offsets can be 32 bits (represented by the notation m16:32) or 16 bits (represented by the notation m16:16).

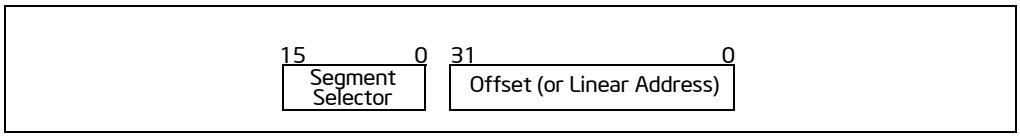


Figure 3-9. Memory Operand Address

3.7.3.1 Memory Operands in 64-Bit Mode

In 64-bit mode, a memory operand can be referenced by a segment selector and an offset. The offset can be 16 bits, 32 bits or 64 bits (see Figure 3-10).

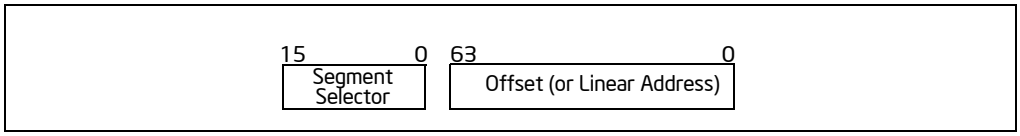


Figure 3-10. Memory Operand Address in 64-Bit Mode

3.7.4 Specifying a Segment Selector

The segment selector can be specified either implicitly or explicitly. The most common method of specifying a segment selector is to load it in a segment register and then allow the processor to select the register implicitly, depending on the type of operation being performed. The processor automatically chooses a segment according to the rules given in Table 3-5.

When storing data in memory or loading data from memory, the DS segment default can be overridden to allow other segments to be accessed. Within an assembler, the segment override is generally handled with a colon ":" operator. For example, the following MOV instruction moves a value from register EAX into the segment pointed to by the ES register. The offset into the segment is contained in the EBX register:

```
MOV ES:[EBX], EAX
```

Table 3-5. Default Segment Selection Rules

Reference Type	Register Used	Segment Used	Default Selection Rule
Instructions	CS	Code Segment	All instruction fetches.
Stack	SS	Stack Segment	All stack pushes and pops. Any memory reference which uses the ESP or EBP register as a base register.
Local Data	DS	Data Segment	All data references, except when relative to stack or string destination.
Destination Strings	ES	Data Segment pointed to with the ES register	Destination of string instructions.

At the machine level, a segment override is specified with a segment-override prefix, which is a byte placed at the beginning of an instruction. The following default segment selections cannot be overridden:

- Instruction fetches must be made from the code segment.
- Destination strings in string instructions must be stored in the data segment pointed to by the ES register.
- Push and pop operations must always reference the SS segment.

Some instructions require a segment selector to be specified explicitly. In these cases, the 16-bit segment selector can be located in a memory location or in a 16-bit register. For example, the following MOV instruction moves a segment selector located in register BX into segment register DS:

```
MOV DS, BX
```

Segment selectors can also be specified explicitly as part of a 48-bit far pointer in memory. Here, the first double-word in memory contains the offset and the next word contains the segment selector.

3.7.4.1 Segmentation in 64-Bit Mode

In IA-32e mode, the effects of segmentation depend on whether the processor is running in compatibility mode or 64-bit mode. In compatibility mode, segmentation functions just as it does in legacy IA-32 mode, using the 16-bit or 32-bit protected mode semantics described above.

In 64-bit mode, segmentation is generally (but not completely) disabled, creating a flat 64-bit linear-address space. The processor treats the segment base of CS, DS, ES, SS as zero, creating a linear address that is equal to the effective address. The exceptions are the FS and GS segments, whose segment registers (which hold the segment base) can be used as additional base registers in some linear address calculations.

3.7.5 Specifying an Offset

The offset part of a memory address can be specified directly as a static value (called a **displacement**) or through an address computation made up of one or more of the following components:

- **Displacement** — An 8-, 16-, or 32-bit value.
- **Base** — The value in a general-purpose register.
- **Index** — The value in a general-purpose register.
- **Scale factor** — A value of 2, 4, or 8 that is multiplied by the index value.

The offset which results from adding these components is called an **effective address**. Each of these components can have either a positive or negative (2s complement) value, with the exception of the scaling factor. Figure 3-11 shows all the possible ways that these components can be combined to create an effective address in the selected segment.

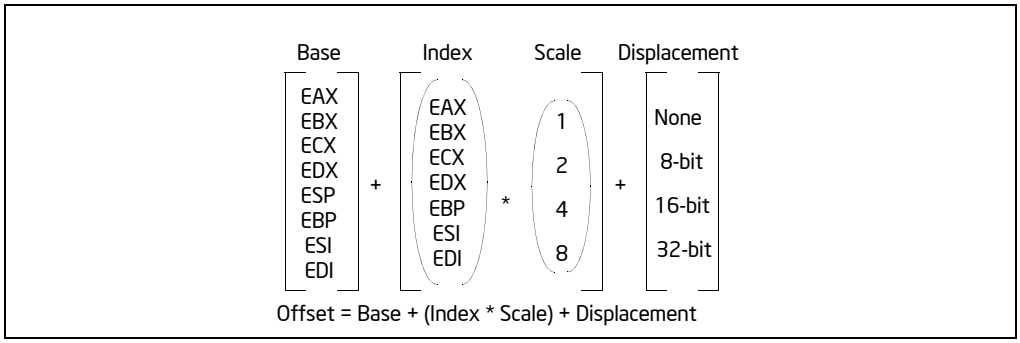


Figure 3-11. Offset (or Effective Address) Computation

The uses of general-purpose registers as base or index components are restricted in the following manner:

- The ESP register cannot be used as an index register.
- When the ESP or EBP register is used as the base, the SS segment is the default segment. In all other cases, the DS segment is the default segment.

The base, index, and displacement components can be used in any combination, and any of these components can be NULL. A scale factor may be used only when an index also is used. Each possible combination is useful for data structures commonly used by programmers in high-level languages and assembly language.

The following addressing modes suggest uses for common combinations of address components.

- **Displacement** — A displacement alone represents a direct (uncomputed) offset to the operand. Because the displacement is encoded in the instruction, this form of an address is sometimes called an absolute or static address. It is commonly used to access a statically allocated scalar operand.
- **Base** — A base alone represents an indirect offset to the operand. Since the value in the base register can change, it can be used for dynamic storage of variables and data structures.
- **Base + Displacement** — A base register and a displacement can be used together for two distinct purposes:
  - — As an index into an array when the element size is not 2, 4, or 8 bytes—The displacement component encodes the static offset to the beginning of the array. The base register holds the results of a calculation to determine the offset to a specific element within the array.
  - — To access a field of a record: the base register holds the address of the beginning of the record, while the displacement is a static offset to the field.

  An important special case of this combination is access to parameters in a procedure activation record. A procedure activation record is the stack frame created when a procedure is entered. Here, the EBP register is the best choice for the base register, because it automatically selects the stack segment. This is a compact encoding for this common function.
- **(Index * Scale) + Displacement** — This address mode offers an efficient way to index into a static array when the element size is 2, 4, or 8 bytes. The displacement locates the beginning of the array, the index register holds the subscript of the desired array element, and the processor automatically converts the subscript into an index by applying the scaling factor.
- **Base + Index + Displacement** — Using two registers together supports either a two-dimensional array (the displacement holds the address of the beginning of the array) or one of several instances of an array of records (the displacement is an offset to a field within the record).
- **Base + (Index * Scale) + Displacement** — Using all the addressing components together allows efficient indexing of a two-dimensional array when the elements of the array are 2, 4, or 8 bytes in size.

3.7.5.1 Specifying an Offset in 64-Bit Mode

The offset part of a memory address in 64-bit mode can be specified directly as a static value or through an address computation made up of one or more of the following components:

- **Displacement —** An 8-bit, 16-bit, or 32-bit value.

- **Base —** The value in a 64-bit general-purpose register.
- **Index —** The value in a 64-bit general-purpose register.
- **Scale factor —** A value of 2, 4, or 8 that is multiplied by the index value.

The base and index value can be specified in one of sixteen available general-purpose registers in most cases. See Chapter 2, "Instruction Format," in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 2A.

The following unique combination of address components is also available.

- **RIP + Displacement** — In 64-bit mode, RIP-relative addressing uses a signed 32-bit displacement to calculate the effective address of the next instruction by sign-extend the 32-bit value and add to the 64-bit value in RIP.

3.7.6 Assembler and Compiler Addressing Modes

At the machine-code level, the selected combination of displacement, base register, index register, and scale factor is encoded in an instruction. All assemblers permit a programmer to use any of the allowable combinations of these addressing components to address operands. High-level language compilers will select an appropriate combination of these components based on the language construct a programmer defines.

3.7.7 I/O Port Addressing

The processor supports an I/O address space that contains up to 65,536 8-bit I/O ports. Ports that are 16-bit and 32-bit may also be defined in the I/O address space. An I/O port can be addressed with either an immediate operand or a value in the DX register. See Chapter 20, "Input/Output," for more information about I/O port addressing.

CHAPTER 4
DATA TYPES

This chapter introduces data types defined for the Intel 64 and IA-32 architectures. A section at the end of this chapter describes the real-number and floating-point concepts used in x87 FPU and Intel SSE, SSE2, SSE3, SSSE3, SSE4, and AVX extensions.

4.1 FUNDAMENTAL DATA TYPES

The fundamental data types are bytes, words, doublewords, quadwords, and double quadwords (see Figure 4-1). A byte is eight bits, a word is 2 bytes (16 bits), a doubleword is 4 bytes (32 bits), a quadword is 8 bytes (64 bits), and a double quadword is 16 bytes (128 bits). A subset of the IA-32 architecture instructions operates on these fundamental data types without any additional operand typing.

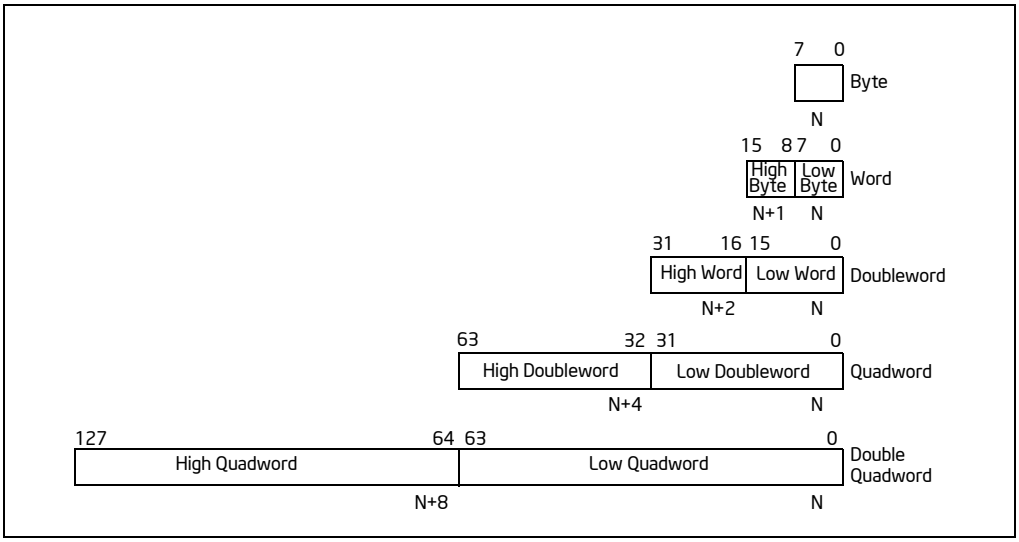


Figure 4-1. Fundamental Data Types

The quadword data type was introduced into the IA-32 architecture in the Intel486 processor; the double quadword data type was introduced in the Pentium III processor with the Intel SSE extensions.

Figure 4-2 shows the byte order of each of the fundamental data types when referenced as operands in memory. The low byte (bits 0 through 7) of each data type occupies the lowest address in memory and that address is also the address of the operand.

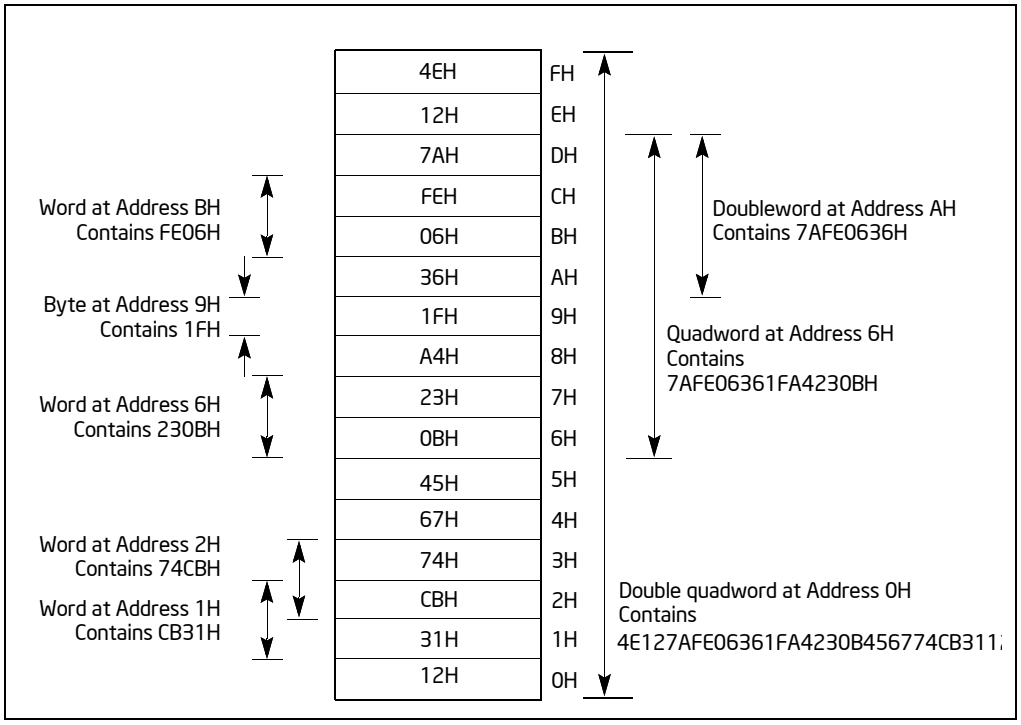


Figure 4-2. Bytes, Words, Doublewords, Quadwords, and Double Quadwords in Memory

4.1.1 Alignment of Words, Doublewords, Quadwords, and Double Quadwords

Words, doublewords, and quadwords do not need to be aligned in memory on natural boundaries. The natural boundaries for words, doublewords, and quadwords are even-numbered addresses, addresses evenly divisible by four, and addresses evenly divisible by eight, respectively. However, to improve the performance of programs, data structures (especially stacks) should be aligned on natural boundaries whenever possible. The reason for this is that the processor requires two memory accesses to make an unaligned memory access; aligned accesses require only one memory access. A word or doubleword operand that crosses a 4-byte boundary or a quadword operand that crosses an 8-byte boundary is considered unaligned and requires two separate memory bus cycles for access.

Some instructions that operate on double quadwords require memory operands to be aligned on a natural boundary. These instructions generate a general-protection exception (#GP) if an unaligned operand is specified. A natural boundary for a double quadword is any address evenly divisible by 16. Other instructions that operate on double quadwords permit unaligned access (without generating a general-protection exception). However, additional memory bus cycles are required to access unaligned data from memory.

4.2 NUMERIC DATA TYPES

Although bytes, words, and doublewords are fundamental data types, some instructions support additional interpretations of these data types to allow operations to be performed on numeric data types (signed and unsigned integers, and floating-point numbers). Single precision (32-bit) floating-point and double precision (64-bit) floating-point data types are supported across all generations of Intel SSE extensions and Intel AVX extensions. The half precision (16-bit) floating-point data type was supported only with F16C extensions (VCVTPH2PS and VCVTPS2PH) beginning with the third generation of Intel® Core™ processors based on Ivy Bridge microarchitecture. Starting with the 4th generation Intel® Xeon® Scalable Processor Family, an Intel® AVX-512 instruction set architecture (ISA) for FP16 was added, supporting a wide range of general-purpose numeric operations for 16-bit half precision floating-point values (binary16 in IEEE Standard 754-2019 for Floating-Point Arithmetic, aka half precision or FP16), which complements the existing 32-bit and 64-bit floating-point instructions already available in the Intel Xeon processor-based products. This ISA also provides complex-valued native hardware support for half precision floating-point. See Figure 4-3.

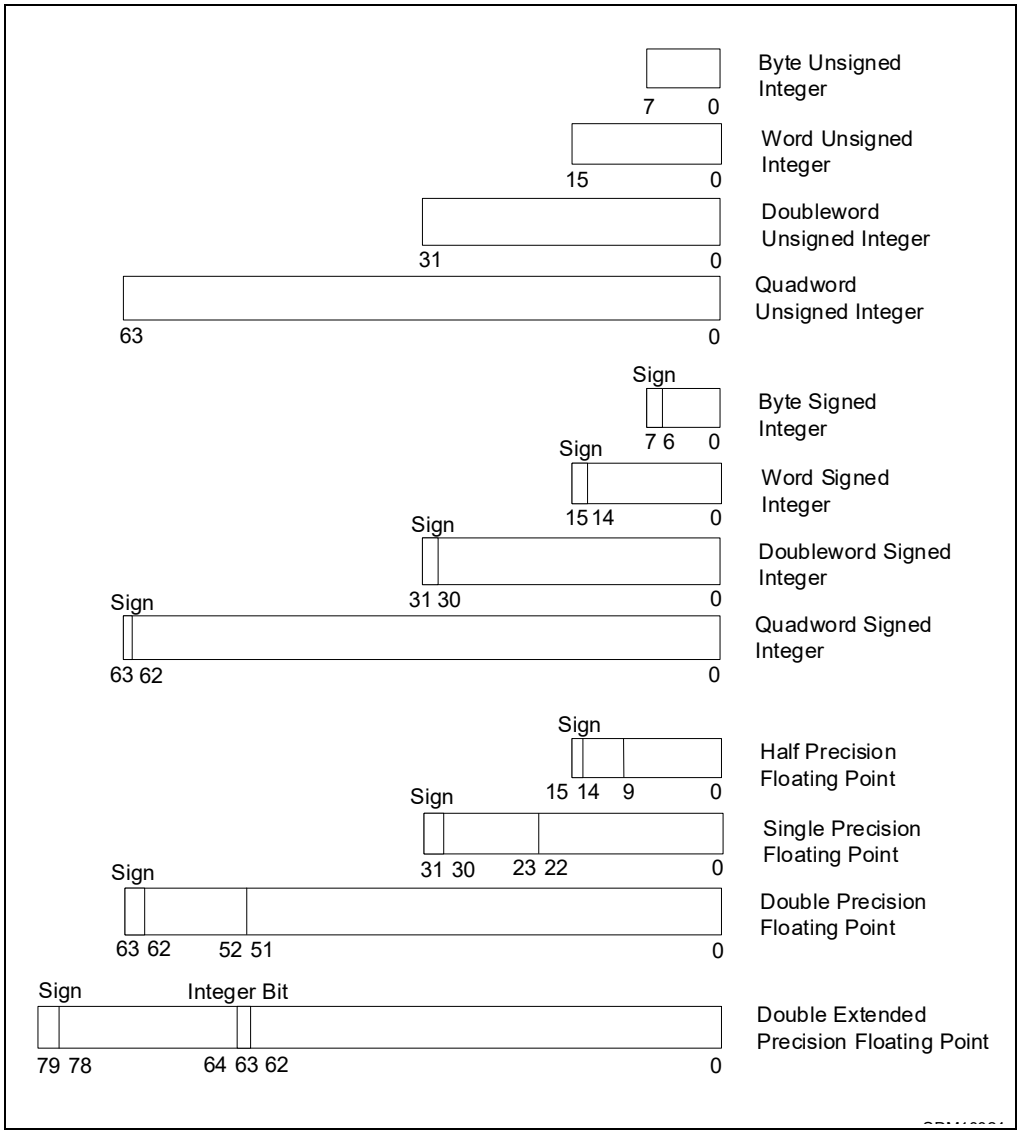


Figure 4-3. Numeric Data Types

4.2.1 Integers

The Intel 64 and IA-32 architectures define two types of integers: unsigned and signed. Unsigned integers are ordinary binary values ranging from 0 to the maximum positive number that can be encoded in the selected operand size. Signed integers are two's complement binary values that can be used to represent both positive and negative integer values.

Some integer instructions (such as the ADD, SUB, PADDB, and PSUBB instructions) operate on either unsigned or signed integer operands. Other integer instructions (such as IMUL, MUL, IDIV, DIV, FIADD, and FISUB) operate on only one integer type.

The following sections describe the encodings and ranges of the two types of integers.

4.2.1.1 Unsigned Integers

Unsigned integers are unsigned binary numbers contained in a byte, word, doubleword, and quadword. Their values range from 0 to 255 for an unsigned byte integer, from 0 to 65,535 for an unsigned word integer, from 0

to $2^{32} - 1$ for an unsigned doubleword integer, and from 0 to $2^{64} - 1$ for an unsigned quadword integer. Unsigned integers are sometimes referred to as **ordinals**.

4.2.1.2 Signed Integers

Signed integers are signed binary numbers held in a byte, word, doubleword, or quadword. All operations on signed integers assume a two's complement representation. The sign bit is located in bit 7 in a byte integer, bit 15 in a word integer, bit 31 in a doubleword integer, and bit 63 in a quadword integer (see the signed integer encodings in Table 4-1).

Table 4-1. Signed Integer Encodings

Class		Two's Complement Encoding	
		Sign	
Positive	Largest	0	11..11
	.	.	.
	.	.	.
	Smallest	0	00..01
Zero		0	00..00
Negative	Smallest	1	11..11
	.	.	.
	.	.	.
	Largest	1	00..00
Integer indefinite		1	00..00
		Signed Byte Integer:	← 7 bits →
		Signed Word Integer:	← 15 bits →
		Signed Doubleword Integer:	← 31 bits →
		Signed Quadword Integer:	← 63 bits →

The sign bit is set for negative integers and cleared for positive integers and zero. Integer values range from −128 to +127 for a byte integer, from −32,768 to +32,767 for a word integer, from -2^{31} to $+2^{31} - 1$ for a doubleword integer, and from -2^{63} to $+2^{63} - 1$ for a quadword integer.

When storing integer values in memory, word integers are stored in 2 consecutive bytes; doubleword integers are stored in 4 consecutive bytes; and quadword integers are stored in 8 consecutive bytes.

The integer indefinite is a special value that is sometimes returned by the x87 FPU when operating on integer values. For more information, see Section 8.2.1, "Indefinites."

4.2.2 Floating-Point Data Types

The IA-32 architecture defines and operates on four floating-point data types: half precision floating-point, single precision floating-point, double precision floating-point, and double-extended precision floating-point (see Figure 4-3). The data formats for these data types correspond directly to formats specified in the IEEE Standard 754 for Floating-Point Arithmetic.

The half precision (16-bit) floating-point data type was supported only with F16C extensions (VCVTPH2PS and VCVTPS2PH) beginning with the third generation of Intel Core processors based on Ivy Bridge microarchitecture. Starting with the 4th generation Intel Xeon Scalable Processor Family, an Intel AVX-512 instruction set architecture (ISA) for FP16 was added, supporting a wide range of general-purpose numeric operations for 16-bit half precision floating-point values (binary16 in the IEEE Standard 754-2019 for Floating-Point Arithmetic, aka half precision or FP16), which complements the existing 32-bit and 64-bit floating-point instructions already available in the Intel Xeon processor-based products.

Table 4-2 gives the length, precision, and approximate normalized range that can be represented by each of these data types. Denormal values are also supported in each of these types.

Table 4-2. Length, Precision, and Range of Floating-Point Data Types

Data Type	Length (Bits)	Precision (Bits)	Approximate Normalized Range	
			Binary	Decimal
Half Precision	16	11	2^{-14} to 2^{16}	6.10×10^{-5} to 6.55×10^{4}
Single Precision	32	24	2^{-126} to 2^{128}	1.18×10^{-38} to 3.40×10^{38}
Double Precision	64	53	2^{-1022} to 2^{1024}	2.23×10^{-308} to 1.80×10^{308}
Double-Extended Precision	80	64	2^{-16382} to 2^{16384}	3.36×10^{-4932} to 1.19×10^{4932}

NOTE

Section 4.8, "Real Numbers and Floating-Point Formats," gives an overview of the IEEE Standard 754 floating-point formats and defines the terms integer bit, QNaN, SNaN, and denormal value.

Table 4-3 shows the floating-point encodings for zeros, denormalized finite numbers, normalized finite numbers, infinites, and NaNs for each of the three floating-point data types. It also gives the format for the QNaN floating-point indefinite value. (See Section 4.8.3.7, "QNaN Floating-Point Indefinite," for a discussion of the use of the QNaN floating-point indefinite value.)

For the half precision, single precision, and double precision formats, only the fraction part of the significand is encoded. The integer is assumed to be 1 for all numbers except 0 and denormalized finite numbers. For the double extended precision format, the integer is contained in bit 63, and the most-significant fraction bit is bit 62. Here, the integer is explicitly set to 1 for normalized numbers, infinities, and NaNs, and to 0 for zero and denormalized numbers.

Table 4-3. Floating-Point Number and NaN Encodings

Class		Sign	Biased Exponent	Significand	
				Integer[1]	Fraction
Positive	+∞	0	11..11	1	00..00
	+Normals	0 . . 0	11..10 . . 00..01	1 . . 1	11..11 . . 00..00
	+Denormals	0 . . 0	00..00 . . 00..00	0 . . 0	11.11 . . 00..01
	+Zero	0	00..00	0	00..00
Negative	–Zero	1	00..00	0	00..00
	–Denormals	1 . . 1	00..00 . . 00..00	0 . . 0	00..01 . . 11..11
	–Normals	1 . . 1	00..01 . . 11..10	1 . . 1	00..00 . . 11..11
	-∞	1	11..11	1	00..00

Table 4-3. Floating-Point Number and NaN Encodings (Contd.)

Class		Sign	Biased Exponent	Significand	
				Integer[1]	Fraction
NaNs	SNaN	X	11..11	1	0X..XX[2]
	QNaN	X	11..11	1	1X..XX
	QNaN Floating-Point Indefinite	1	11..11	1	10..00
	Half Precision Single Precision: Double Precision: Double Extended Precision:		← 5 Bits → ← 8 Bits → ← 11 Bits → ← 15 Bits →		← 10 Bits → ← 23 Bits → ← 52 Bits → ← 63 Bits →

NOTES:

1. Integer bit is implied and not stored for half precision, single precision, and double precision formats.
2. The fraction for SNaN encodings must be non-zero with the most-significant bit 0.

The exponent of each floating-point data type is encoded in biased format; see Section 4.8.2.2, "Biased Exponent." The biasing constant is 15 for the half precision format, 127 for the single precision format, 1023 for the double precision format, and 16,383 for the double extended precision format.

When storing floating-point values in memory, half precision values are stored in 2 consecutive bytes in memory; single precision values are stored in 4 consecutive bytes in memory; double precision values are stored in 8 consecutive bytes; and double extended precision values are stored in 10 consecutive bytes.

The single precision and double precision floating-point data types are operated on by x87 FPU, and Intel SSE/SSE2/SSE3/SSE4.1/AVX instructions. The double extended precision floating-point format is only operated on by the x87 FPU. See Section 11.6.8, "Compatibility of SIMD and x87 FPU Floating-Point Data Types," for a discussion of the compatibility of single precision and double precision floating-point data types between the x87 FPU and Intel SSE/SSE2/SSE3 extensions.

4.2.3 Brain Float16

Brain Float16 (BF16 or bfloat16) is a shortened 16-bit version of the IEEE Standard 754 floating-point 32-bit format. It aims to speed up training and inference for AI workloads. Figure 4-4 illustrates BF16 versus FP16 and FP32.

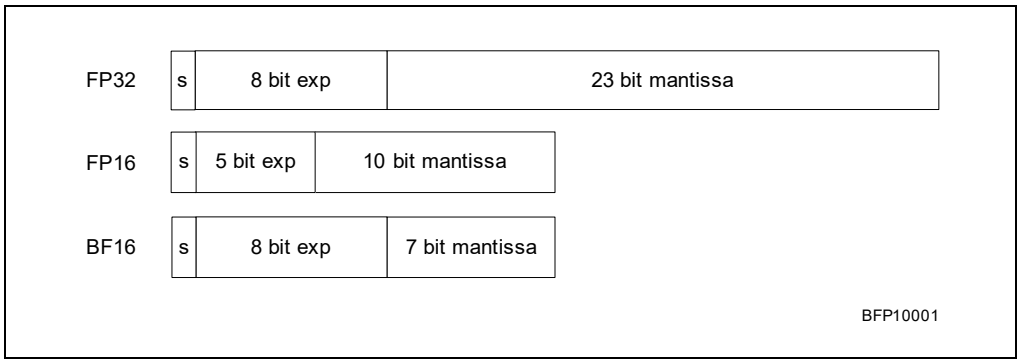


Figure 4-4. Comparison of BF16 to FP16 and FP32

4.2.3.1 Numeric Definition

BF16 has one sign bit, eight exponent bits, and seven mantissa bits.

Table 4-4. BF16 Format Numeric Definitions

Number	BF16 (E8M7)
Exponent Bias	127
Maximum Normal	S 11111110 11111111 = $\pm 2^{127} * 1.99 \sim= \pm 3.39 * 10^{38}$
Minimum Normal	S 00000001 00000000 = $\pm 2^{-126} \sim= \pm 1.18 * 10^{-38}$
Maximum Denormal	S 00000000 1111111 = $\pm 2^{-126} * 0.99 \sim= \pm 1.17 * 10^{-38}$
Minimum Denormal	S 00000000 0000001 = $\pm 2^{-133} \sim= \pm 9.18 * 10^{-41}$
NaNs	S 11111111 {non-zero}
Infinity	S 11111111 0000000
Zeros	S 00000000 0000000

Although denormal values are shown in the table, they are not used in computations (see Section 4.2.3.2).

4.2.3.2 Rounding, Denormal Handling, and FP Exceptions

Intel architecture supports BF16 in AMX dot product instructions, AVX dot product instructions, and AVX convert instructions.

- Rounding: All operations are executed using Round to nearest (even) mode (e.g., for convert instructions).
- Denormal Handling: For BF16, input denormal values are replaced with zeros and FP32 underflow results are flushed to zero. All instructions that accept BF16 inputs have FP32 results.
- Floating-point exceptions:
  - — Instructions operating on BF16 values neither consult nor update the MXCSR.
  - — Instructions operating on BF16 values do not raise exceptions.

4.3 POINTER DATA TYPES

Pointers are addresses of locations in memory.

In non-64-bit modes, the architecture defines two types of pointers: a **near pointer** and a **far pointer**. A near pointer is a 32-bit (or 16-bit) offset (also called an **effective address**) within a segment. Near pointers are used for all memory references in a flat memory model or for references in a segmented model where the identity of the segment being accessed is implied.

A far pointer is a logical address, consisting of a 16-bit segment selector and a 32-bit (or 16-bit) offset. Far pointers are used for memory references in a segmented memory model where the identity of a segment being accessed must be specified explicitly. Near and far pointers with 32-bit offsets are shown in Figure 4-5.

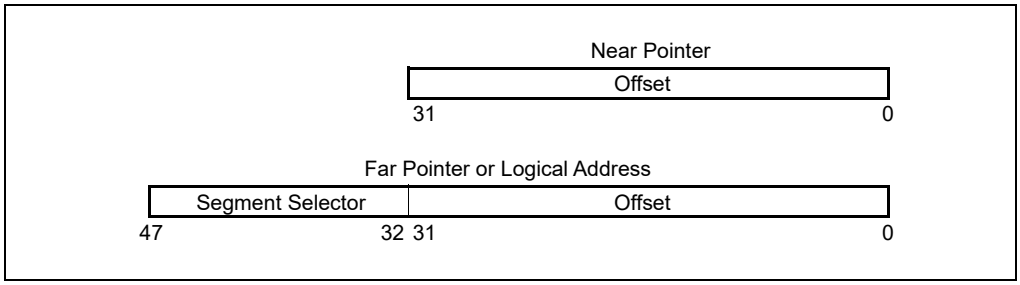


Figure 4-5. Pointer Data Types

4.3.1 Pointer Data Types in 64-Bit Mode

In 64-bit mode (a sub-mode of IA-32e mode), a **near pointer** is 64 bits. This equates to an effective address. **Far pointers** in 64-bit mode can be one of three forms:

- 16-bit segment selector, 16-bit offset if the operand size is 32 bits.
- 16-bit segment selector, 32-bit offset if the operand size is 32 bits.
- 16-bit segment selector, 64-bit offset if the operand size is 64 bits.

See Figure 4-6.

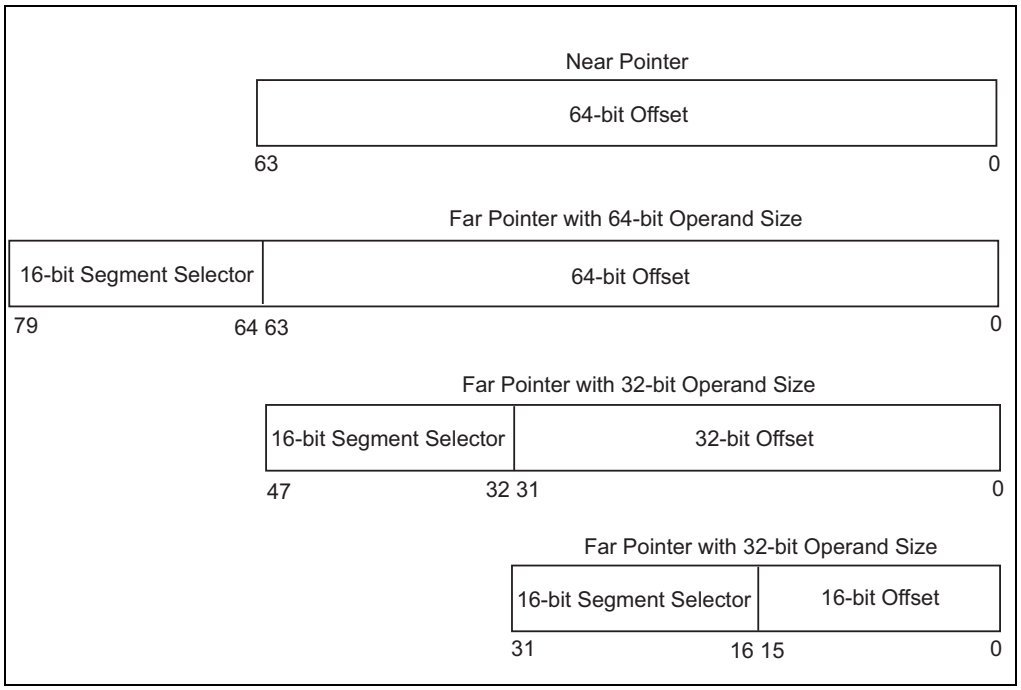


Figure 4-6. Pointers in 64-Bit Mode

4.4 BIT FIELD DATA TYPE

A **bit field** (see Figure 4-7) is a contiguous sequence of bits. It can begin at any bit position of any byte in memory and can contain up to 32 bits.

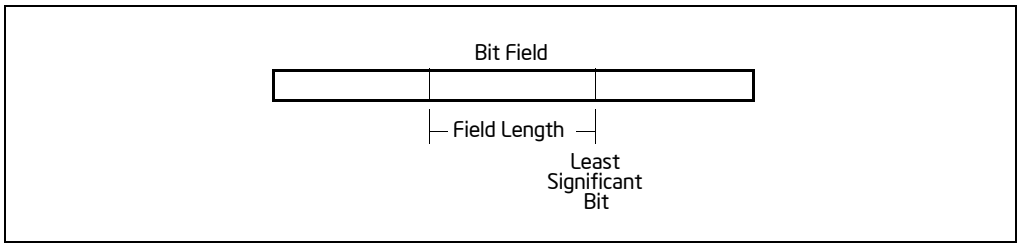


Figure 4-7. Bit Field Data Type

4.5 STRING DATA TYPES

Strings are continuous sequences of bits, bytes, words, or doublewords. A **bit string** can begin at any bit position of any byte and can contain up to $2^{32} - 1$ bits. A **byte string** can contain bytes, words, or doublewords and can range from zero to $2^{32} - 1$ bytes (4 GBytes).

4.6 PACKED SIMD DATA TYPES

Intel 64 and IA-32 architectures define and operate on a set of 64-bit and 128-bit packed data type for use in SIMD operations. These data types consist of fundamental data types (packed bytes, words, doublewords, and quadwords) and numeric interpretations of fundamental types for use in packed integer and packed floating-point operations.

4.6.1 64-Bit SIMD Packed Data Types

The 64-bit packed SIMD data types were introduced into the IA-32 architecture in the Intel MMX technology. They are operated on in MMX registers. The fundamental 64-bit packed data types are packed bytes, packed words, and packed doublewords (see Figure 4-8). When performing numeric SIMD operations on these data types, these data types are interpreted as containing byte, word, or doubleword integer values.

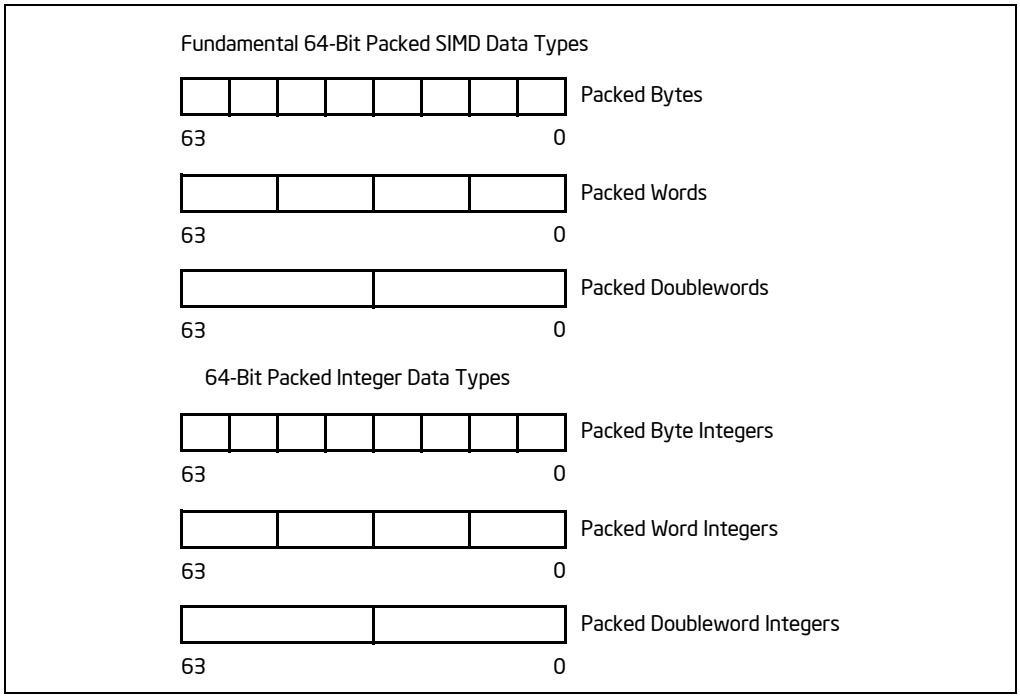


Figure 4-8. 64-Bit Packed SIMD Data Types

4.6.2 128-Bit Packed SIMD Data Types

The 128-bit packed SIMD data types were introduced into the IA-32 architecture in the Intel SSE extensions and used with Intel SSE2, SSE3, SSSE3, SSE4.1, and AVX extensions. They are operated on primarily in the 128-bit XMM registers and memory. The fundamental 128-bit packed data types are packed bytes, packed words, packed doublewords, and packed quadwords (see Figure 4-9). When performing SIMD operations on these fundamental data types in XMM registers, these data types are interpreted as containing packed or scalar half precision floating-point, single precision floating-point or double precision floating-point values, or as containing packed byte, word, doubleword, or quadword integer values.

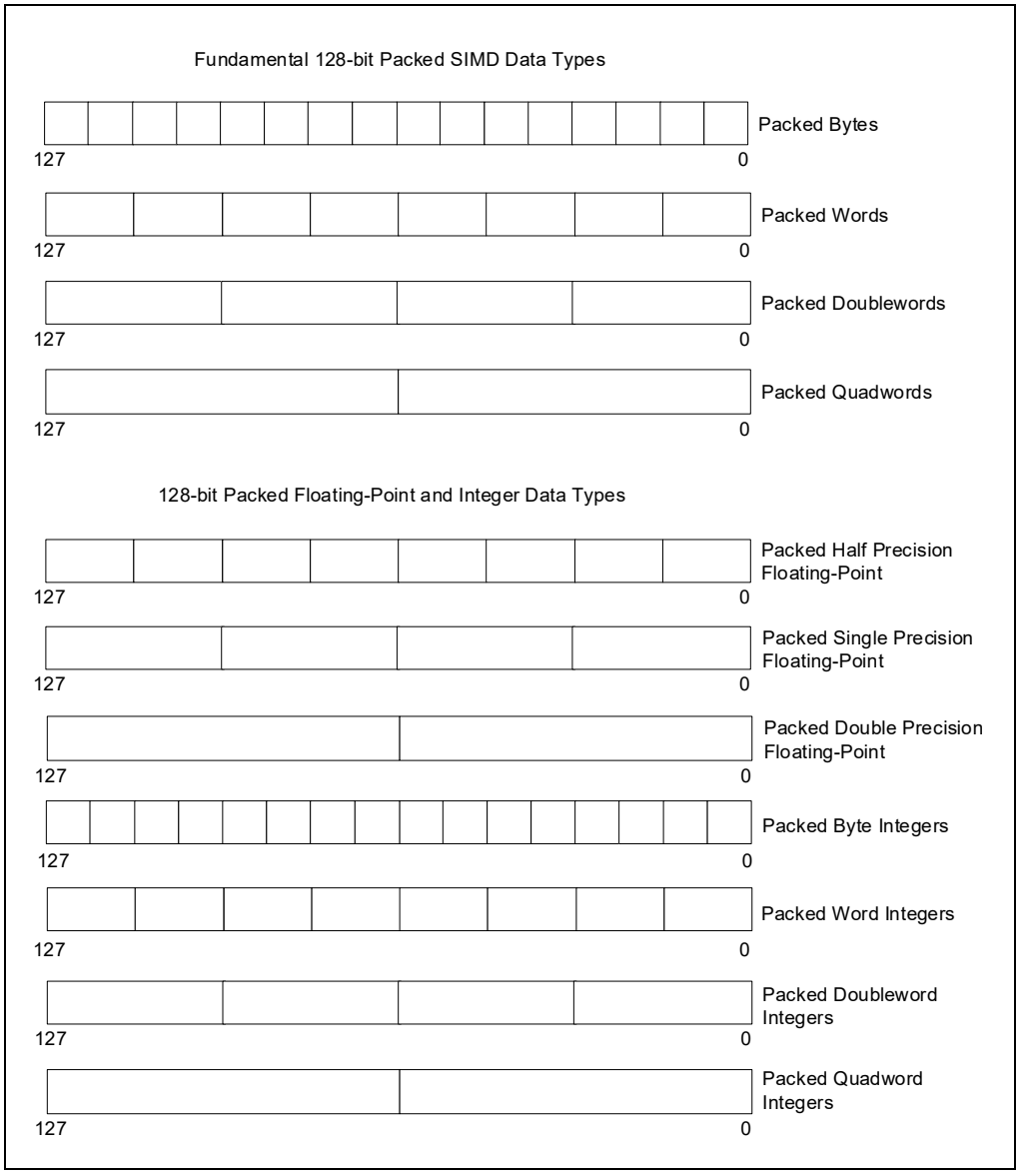


Figure 4-9. 128-Bit Packed SIMD Data Types

4.7 BCD AND PACKED BCD INTEGERS

Binary-coded decimal integers (BCD integers) are unsigned 4-bit integers with valid values ranging from 0 to 9. IA-32 architecture defines operations on BCD integers located in one or more general-purpose registers or in one or more x87 FPU registers (see Figure 4-10).

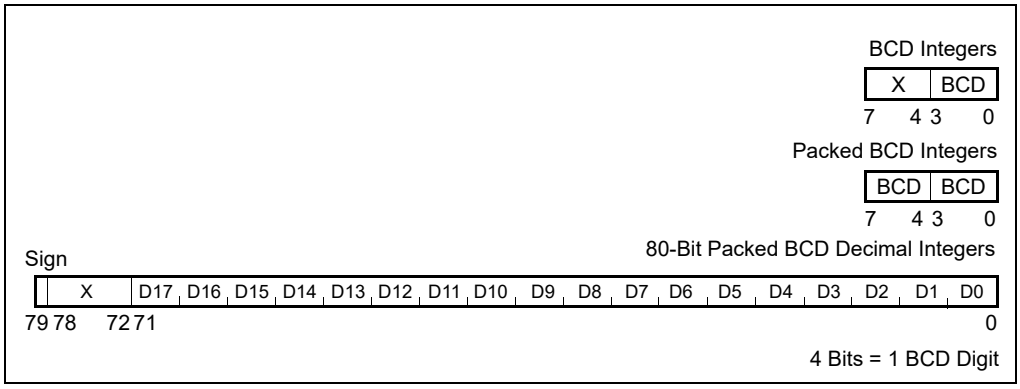


Figure 4-10. BCD Data Types

When operating on BCD integers in general-purpose registers, the BCD values can be unpacked (one BCD digit per byte) or packed (two BCD digits per byte). The value of an unpacked BCD integer is the binary value of the low half-byte (bits 0 through 3). The high half-byte (bits 4 through 7) can be any value during addition and subtraction, but must be zero during multiplication and division. Packed BCD integers allow two BCD digits to be contained in one byte. Here, the digit in the high half-byte is more significant than the digit in the low half-byte.

When operating on BCD integers in x87 FPU data registers, BCD values are packed in an 80-bit format and referred to as decimal integers. In this format, the first 9 bytes hold 18 BCD digits, 2 digits per byte. The least-significant digit is contained in the lower half-byte of byte 0 and the most-significant digit is contained in the upper half-byte of byte 9. The most significant bit of byte 10 contains the sign bit (0 = positive and 1 = negative; bits 0 through 6 of byte 10 are don't care bits). Negative decimal integers are not stored in two's complement form; they are distinguished from positive decimal integers only by the sign bit. The range of decimal integers that can be encoded in this format is $-10^{18}+1$ to $10^{18}-1$.

The decimal integer format exists in memory only. When a decimal integer is loaded in an x87 FPU data register, it is automatically converted to the double extended precision floating-point format. All decimal integers are exactly representable in double extended precision format.

Table 4-5 gives the possible encodings of value in the decimal integer data type.

Table 4-5. Packed Decimal Integer Encodings

Class	Sign		Magnitude					
			digit	digit	digit	digit	...	digit
Positive								
Largest	0	0000000	1001	1001	1001	1001	...	1001
	.	.			.			
	.	.			.			
Smallest	0	0000000	0000	0000	0000	0000	...	0001
Zero	0	0000000	0000	0000	0000	0000	...	0000
Negative								
Zero	1	0000000	0000	0000	0000	0000	...	0000
Smallest	1	0000000	0000	0000	0000	0000	...	0001
	.	.			.			
	.	.			.			
Largest	1	0000000	1001	1001	1001	1001	...	1001

Table 4-5. Packed Decimal Integer Encodings (Contd.)

Class	Sign		Magnitude					
			digit	digit	digit	digit	...	digit
Packed BCD Integer Indefinite	1	1111111	1111	1111	1100	0000	...	0000
	← 1 byte →		← 9 bytes →					

The packed BCD integer indefinite encoding (FFFFC000000000000000H) is stored by the FBSTP instruction in response to a masked floating-point invalid-operation exception. Attempting to load this value with the FBLD instruction produces an undefined result.

4.8 REAL NUMBERS AND FLOATING-POINT FORMATS

This section describes how real numbers are represented in floating-point format in x87 FPU and SSE/SSE2/SSE3/SSE4.1 and Intel AVX floating-point instructions. It also introduces terms such as normalized numbers, denormalized numbers, biased exponents, signed zeros, and NaNs. Readers who are already familiar with floating-point processing techniques and the IEEE Standard 754 for Floating-Point Arithmetic may wish to skip this section.

4.8.1 Real Number System

As shown in Figure 4-11, the real-number system comprises the continuum of real numbers from minus infinity (– ∞) to plus infinity (+ ∞).

Because the size and number of registers that any computer can have is limited, only a subset of the real-number continuum can be used in real-number (floating-point) calculations. As shown at the bottom of Figure 4-11, the subset of real numbers that the IA-32 architecture supports represents an approximation of the real number system. The range and precision of this real-number subset is determined by the IEEE Standard 754 floating-point formats.

4.8.2 Floating-Point Format

To increase the speed and efficiency of real-number computations, computers and microprocessors typically represent real numbers in a binary floating-point format. In this format, a real number has three parts: a sign, a significand, and an exponent (see Figure 4-12).

The sign is a binary value that indicates whether the number is positive (0) or negative (1). The significand has two parts: a 1-bit binary integer (also referred to as the J-bit) and a binary fraction. The integer-bit is often not represented, but instead is an implied value. The exponent is a binary integer that represents the base-2 power by which the significand is multiplied.

Table 4-6 shows how the real number 178.125 (in ordinary decimal format) is stored in IEEE Standard 754 floating-point format. The table lists a progression of real number notations that leads to the single precision, 32-bit floating-point format. In this format, the significand is normalized (see Section 4.8.2.1, "Normalized Numbers") and the exponent is biased (see Section 4.8.2.2, "Biased Exponent"). For the single precision floating-point format, the biasing constant is +127.

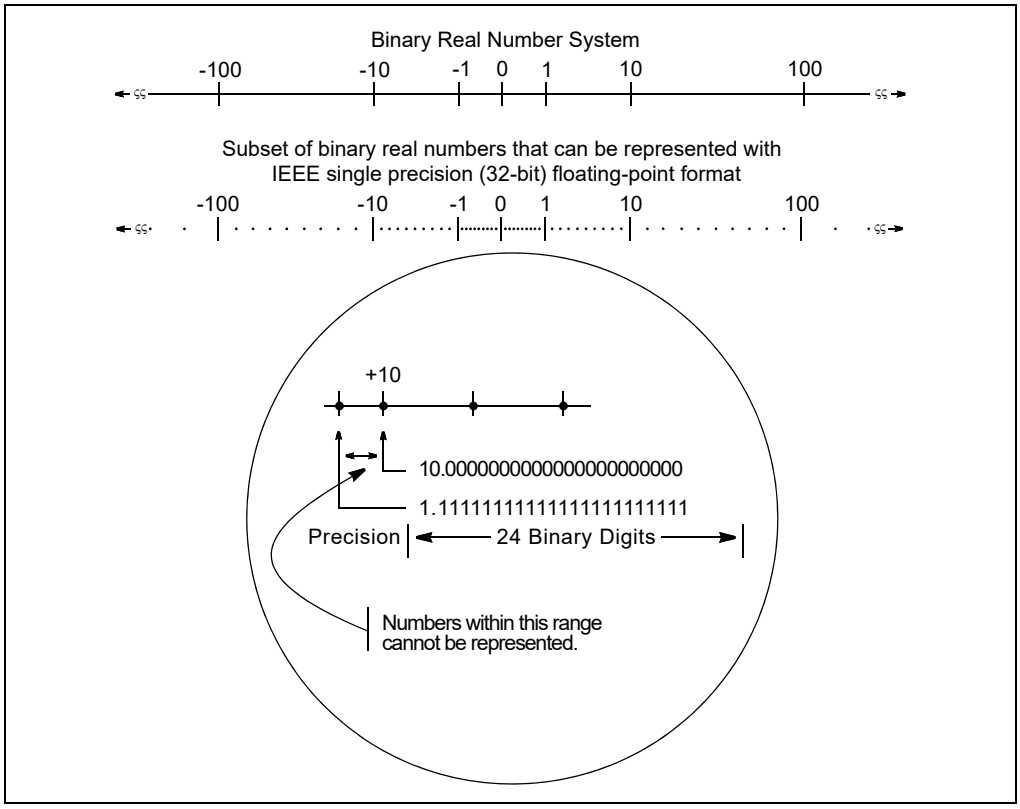


Figure 4-11. Binary Real Number System

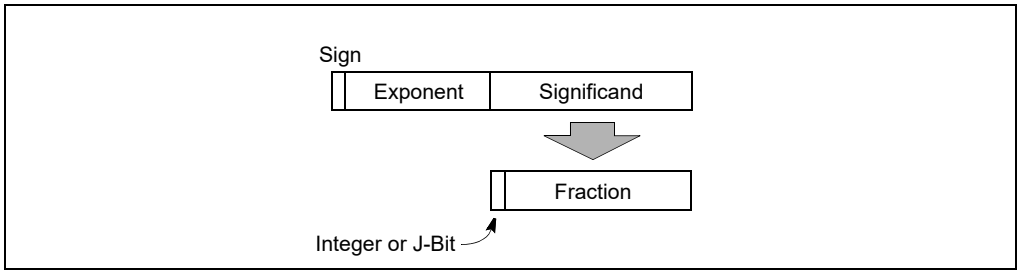


Figure 4-12. Binary Floating-Point Format

Table 4-6. Real and Floating-Point Number Notation

Notation	Value		
Ordinary Decimal	178.125		
Scientific Decimal	$1.78125E_{10}2$		
Scientific Binary	$1.0110010001E_{2}111$		
Scientific Binary (Biased Exponent)	$1.0110010001E_{2}10000110$		
IEEE Single Precision Format	Sign	Biased Exponent	Normalized Significand
	0	10000110	01100100010000000000000 1. (Implied)

4.8.2.1 Normalized Numbers

In most cases, floating-point numbers are encoded in normalized form. This means that except for zero, the significand is always made up of an integer of 1 and the following fraction:

1.fff...ff

For values less than 1, leading zeros are eliminated. (For each leading zero eliminated, the exponent is decremented by one.)

Representing numbers in normalized form maximizes the number of significant digits that can be accommodated in a significand of a given width. To summarize, a normalized real number consists of a normalized significand that represents a real number between 1 and 2 and an exponent that specifies the number's binary point.

4.8.2.2 Biased Exponent

In the IA-32 architecture, the exponents of floating-point numbers are encoded in a biased form. This means that a constant is added to the actual exponent so that the biased exponent is always a positive number. The value of the biasing constant depends on the number of bits available for representing exponents in the floating-point format being used. The biasing constant is chosen so that the smallest normalized number can be reciprocated without overflow.

See Section 4.2.2, "Floating-Point Data Types," for a list of the biasing constants that the IA-32 architecture uses for the various sizes of floating-point data-types.

4.8.3 Real Number and Non-number Encodings

A variety of real numbers and special values can be encoded in the IEEE Standard 754 floating-point format. These numbers and values are generally divided into the following classes:

- Signed zeros
- Denormalized finite numbers
- Normalized finite numbers
- Signed infinities
- NaNs
- Indefinite numbers

(The term NaN stands for "Not a Number.")

Figure 4-13 shows how the encodings for these numbers and non-numbers fit into the real number continuum. The encodings shown here are for the IEEE single precision floating-point format. The term "S" indicates the sign bit, "E" the biased exponent, and "Sig" the significand. The exponent values are given in decimal. The integer bit is shown for the significands, even though the integer bit is implied in single precision floating-point format.

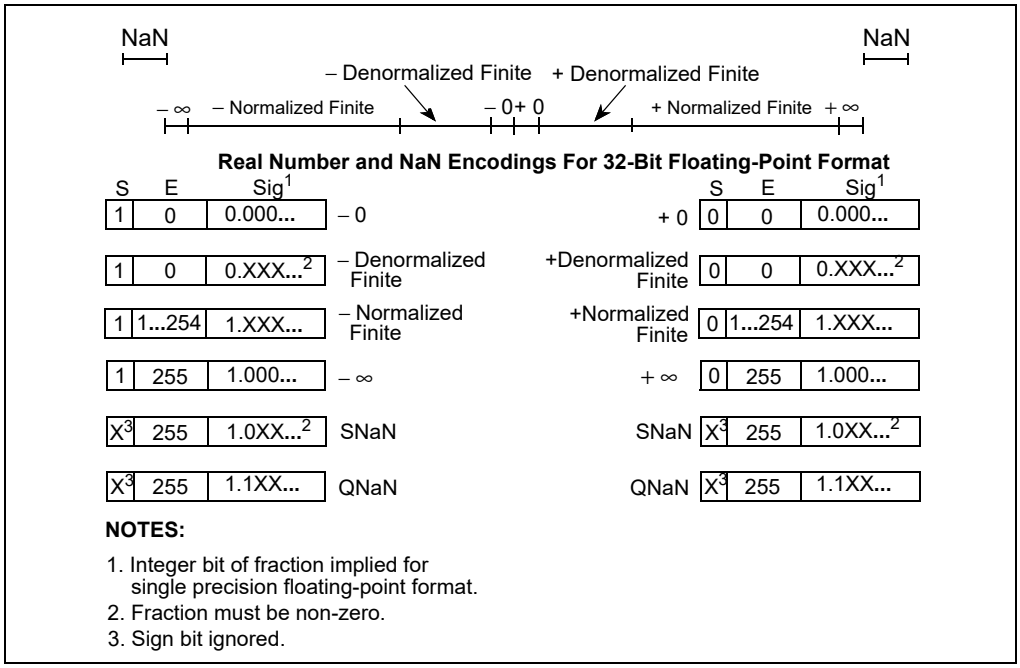


Figure 4-13. Real Numbers and NaNs

An IA-32 processor can operate on and/or return any of these values, depending on the type of computation being performed. The following sections describe these number and non-number classes.

4.8.3.1 Signed Zeros

Zero can be represented as a +0 or a –0 depending on the sign bit. Both encodings are equal in value. The sign of a zero result depends on the operation being performed and the rounding mode being used. Signed zeros have been provided to aid in implementing interval arithmetic. The sign of a zero may indicate the direction from which underflow occurred, or it may indicate the sign of an ∞ that has been reciprocated.

4.8.3.2 Normalized and Denormalized Finite Numbers

Non-zero, finite numbers are divided into two classes: normalized and denormalized. The normalized finite numbers comprise all the non-zero finite values that can be encoded in a normalized real number format between zero and ∞. In the single precision floating-point format shown in Figure 4-13, this group of numbers includes all the numbers with biased exponents ranging from 1 to 254_{10} (unbiased, the exponent range is from -126_{10} to $+127_{10}$).

When floating-point numbers become very close to zero, the normalized-number format can no longer be used to represent the numbers. This is because the range of the exponent is not large enough to compensate for shifting the binary point to the right to eliminate leading zeros.

When the biased exponent is zero, smaller numbers can only be represented by making the integer bit (and perhaps other leading bits) of the significand zero. The numbers in this range are called **denormalized** numbers. The use of leading zeros with denormalized numbers allows smaller numbers to be represented. However, this denormalization may cause a loss of precision (the number of significant bits is reduced by the leading zeros).

When performing normalized floating-point computations, an IA-32 processor normally operates on normalized numbers and produces normalized numbers as results. Denormalized numbers represent an **underflow** condition. The exact conditions are specified in Section 4.9.1.5, “Numeric Underflow Exception (#U).”

A denormalized number is computed through a technique called gradual underflow. Table 4-7 gives an example of gradual underflow in the denormalization process. Here the single precision format is being used, so the minimum exponent (unbiased) is -126_{10}. The true result in this example requires an exponent of -129_{10} in order to have a

normalized number. Since -129_{10} is beyond the allowable exponent range, the result is denormalized by inserting leading zeros until the minimum exponent of -126_{10} is reached.

Table 4-7. Denormalization Process

Operation	Sign	Exponent*	Significand
True Result	0	–129	1.01011100000…00
Denormalize	0	–128	0.10101110000…00
Denormalize	0	–127	0.01010111000…00
Denormalize	0	–126	0.00101011100…00
Denormal Result	0	–126	0.00101011100…00

* Expressed as an unbiased, decimal number.

In the extreme case, all the significant bits are shifted out to the right by leading zeros, creating a zero result.

The Intel 64 and IA-32 architectures deal with denormal values in the following ways:

- It avoids creating denormals by normalizing numbers whenever possible.
- It provides the floating-point underflow exception to permit programmers to detect cases when denormals are created.
- It provides the floating-point denormal-operand exception to permit procedures or programs to detect when denormals are being used as source operands for computations.

4.8.3.3 Signed Infinities

The two infinities, $+\infty$ and $-\infty$, represent the maximum positive and negative real numbers, respectively, that can be represented in the floating-point format. Infinity is always represented by a significand of 1.00…00 (the integer bit may be implied) and the maximum biased exponent allowed in the specified format (for example, 255_{10} for the single precision format).

The signs of infinities are observed, and comparisons are possible. Infinities are always interpreted in the affine sense; that is, $-\infty$ is less than any finite number and $+\infty$ is greater than any finite number. Arithmetic on infinities is always exact. Exceptions are generated only when the use of an infinity as a source operand constitutes an invalid operation.

Whereas denormalized numbers may represent an underflow condition, the two ∞ numbers may represent the result of an overflow condition. Here, the normalized result of a computation has a biased exponent greater than the largest allowable exponent for the selected result format.

4.8.3.4 NaNs

Since NaNs are non-numbers, they are not part of the real number line. In Figure 4-13, the encoding space for NaNs in the floating-point formats is shown above the ends of the real number line. This space includes any value with the maximum allowable biased exponent and a non-zero fraction (the sign bit is ignored for NaNs).

The IA-32 architecture defines two classes of NaNs: quiet NaNs (QNaNs) and signaling NaNs (SNaNs). A QNaN is a NaN with the most significant fraction bit set; an SNaN is a NaN with the most significant fraction bit clear. QNaNs are allowed to propagate through most arithmetic operations without signaling an exception. SNaNs generally signal a floating-point invalid-operation exception whenever they appear as operands in arithmetic operations.

SNaNs are typically used to trap or invoke an exception handler. They must be inserted by software; that is, the processor never generates an SNaN as a result of a floating-point operation.

4.8.3.5 Operating on SNaNs and QNaNs

When a floating-point operation is performed on an SNaN and/or a QNaN, the result of the operation is either a QNaN delivered to the destination operand or the generation of a floating-point invalid operation exception, depending on the following rules:

- If one of the source operands is an SNaN and the floating-point invalid-operation exception is not masked (see Section 4.9.1.1, "Invalid Operation Exception (#I)"), then a floating-point invalid-operation exception is signaled and no result is stored in the destination operand. If one of the source operands is a QNaN and the floating-point invalid-operation exception is not masked and the operation is one that generates an invalid-operation exception for QNaN operands as described in Section 8.5.1.2, "Invalid Arithmetic Operand Exception (#IA)," or Section 11.5.2.1, "Invalid Operation Exception (#I)," then a floating-point invalid-operation exception is signaled and no result is stored in the destination operand.
- If either or both of the source operands are NaNs and floating-point invalid-operation exception is masked, the result is as shown in Table 4-8. When an SNaN is converted to a QNaN, the conversion is handled by setting the most-significant fraction bit of the SNaN to 1. Also, when one of the source operands is an SNaN, or when it is a QNaN and the operation is one that generates an invalid-operation exception for QNaN operands as described in Section 8.5.1.2, "Invalid Arithmetic Operand Exception (#IA)," or Section 11.5.2.1, "Invalid Operation Exception (#I)," then the floating-point invalid-operation exception flag is set. Note that for some combinations of source operands, the result is different for x87 FPU operations and for Intel SSE/SSE2/SSE3/SSE4.1 operations. Intel AVX follows the same behavior as Intel SSE/SSE2/SSE3/SSE4.1 in this respect.
- When neither of the source operands is a NaN, but the operation generates a floating-point invalid-operation exception (see Tables 8-10 and 11-1), the result is commonly a QNaN FP Indefinite (Section 4.8.3.7).

Any exceptions to the behavior described in Table 4-8 are described in Section 8.5.1.2, "Invalid Arithmetic Operand Exception (#IA)," and Section 11.5.2.1, "Invalid Operation Exception (#I)."

Table 4-8. Rules for Handling NaNs

Source Operands	Result[1]
SNaN and QNaN	X87 FPU — QNaN source operand. SSE/SSE2/SSE3/SSE4.1/AVX — First source operand (if this operand is an SNaN, it is converted to a QNaN).
Two SNaNs	X87 FPU — SNaN source operand with the larger significand, converted into a QNaN. SSE/SSE2/SSE3/SSE4.1/AVX — First source operand converted to a QNaN.
Two QNaNs	X87 FPU — QNaN source operand with the larger significand. SSE/SSE2/SSE3/SSE4.1/AVX — First source operand.
SNaN and a floating-point value	SNaN source operand, converted into a QNaN.
QNaN and a floating-point value	QNaN source operand.
SNaN (for instructions that take only one operand)	SNaN source operand, converted into a QNaN.
QNaN (for instructions that take only one operand)	QNaN source operand.

NOTE:

1. For SSE/SSE2/SSE3/SSE4.1 instructions, the first operand is generally a source operand that becomes the destination operand. For AVX instructions, the first source operand is usually the 2nd operand in a non-destructive source syntax. Within the **Result** column, the x87 FPU notation also applies to the FISTTP instruction in SSE3; the SSE3 notation applies to the SIMD floating-point instructions.

4.8.3.6 Using SNaNs and QNaNs in Applications

Except for the rules given at the beginning of Section 4.8.3.4, "NaNs," for encoding SNaNs and QNaNs, software is free to use the bits in the significand of a NaN for any purpose. Both SNaNs and QNaNs can be encoded to carry and store data, such as diagnostic information.

By unmasking the invalid operation exception, the programmer can use signaling NaNs to trap to the exception handler. The generality of this approach and the large number of NaN values that are available provide the sophisticated programmer with a tool that can be applied to a variety of special situations.

For example, a compiler can use signaling NaNs as references to uninitialized (real) array elements. The compiler can preinitialize each array element with a signaling NaN whose significand contains the index (relative position) of the element. Then, if an application program attempts to access an element that it has not initialized, it can use the NaN placed there by the compiler. If the invalid operation exception is unmasked, an interrupt will occur, and the exception handler will be invoked. The exception handler can determine which element has been accessed, since the operand address field of the exception pointer will point to the NaN, and the NaN will contain the index number of the array element.

Quiet NaNs are often used to speed up debugging. In its early testing phase, a program often contains multiple errors. An exception handler can be written to save diagnostic information in memory whenever it is invoked. After storing the diagnostic data, it can supply a quiet NaN as the result of the erroneous instruction, and that NaN can point to its associated diagnostic area in memory. The program will then continue, creating a different NaN for each error. When the program ends, the NaN results can be used to access the diagnostic data saved at the time the errors occurred. Many errors can thus be diagnosed and corrected in one test run.

In embedded applications that use computed results in further computations, an undetected QNaN can invalidate all subsequent results. Such applications should therefore periodically check for QNaNs and provide a recovery mechanism to be used if a QNaN result is detected.

4.8.3.7 QNaN Floating-Point Indefinite

For the floating-point data type encodings (single precision, double precision, and double extended precision), one unique encoding (a QNaN) is reserved for representing the special value QNaN floating-point indefinite. The x87 FPU and the Intel SSE/SSE2/SSE3/SSE4.1/AVX extensions return these indefinite values as responses to some masked floating-point exceptions. Table 4-3 shows the encoding used for the QNaN floating-point indefinite.

4.8.3.8 Half Precision Floating-Point Operation

Two instructions, VCVTPH2PS and VCVTPS2PH, which provide conversion only between half precision and single precision floating-point values, were introduced with the F16C extensions beginning with the third generation of Intel Core processors based on Ivy Bridge microarchitecture. Starting with the 4th generation Intel Xeon Scalable Processor Family, an Intel AVX-512 instruction set architecture (ISA) for FP16 was added, supporting a wide range of general-purpose numeric operations for 16-bit half precision floating-point values (binary16 in the IEEE Standard 754-2019 for Floating-Point Arithmetic, aka half precision or FP16). These additions complement the existing 32-bit and 64-bit floating-point instructions already available in the Intel Xeon processor-based products.

The SIMD floating-point exception behavior of the VCVTPH2PS and VCVTPS2PH instructions, as well as of the other half precision instructions, are described in Section 14.4.1.

4.8.4 Rounding

When performing floating-point operations, the processor produces an infinitely precise floating-point result in the destination format (half precision, single precision, double precision, or double extended precision floating-point) whenever possible. However, because only a subset of the numbers in the real number continuum can be represented in IEEE Standard 754 floating-point formats, it is often the case that an infinitely precise result cannot be encoded exactly in the format of the destination operand.

For example, the following value (*a*) has a 24-bit fraction. The least-significant bit of this fraction (the underlined bit) cannot be encoded exactly in the single precision format (which has only a 23-bit fraction):

(*a*) $1.0001\ 0000\ 1000\ 0011\ 1001\ 011\underline{1}E_2\ 101$

To round this result (*a*), the processor first selects two representable fractions *b* and *c* that most closely bracket *a* in value ($b < a < c$).

(*b*) $1.0001\ 0000\ 1000\ 0011\ 1001\ 011E_2\ 101$

(*c*) $1.0001\ 0000\ 1000\ 0011\ 1001\ 100E_2\ 101$

The processor then sets the result to *b* or to *c* according to the selected rounding mode. Rounding introduces an error in a result that is less than one unit in the last place (the least significant bit position of the floating-point value) to which the result is rounded.

The IEEE Standard 754 defines four rounding modes (see Table 4-9): round to nearest, round up, round down, and round toward zero. The default rounding mode (for the Intel 64 and IA-32 architectures) is round to nearest. This mode provides the most accurate and statistically unbiased estimate of the true result and is suitable for most applications.

Table 4-9. Rounding Modes and Encoding of Rounding Control (RC) Field

Rounding Mode	RC Field Setting	Description
Round to nearest (even)	00B	Rounded result is the closest to the infinitely precise result. If two values are equally close, the result is the even value (that is, the one with the least-significant bit of zero). Default
Round down (toward $-\infty$)	01B	Rounded result is closest to but no greater than the infinitely precise result.
Round up (toward $+\infty$)	10B	Rounded result is closest to but no less than the infinitely precise result.
Round toward zero (Truncate)	11B	Rounded result is closest to but no greater in absolute value than the infinitely precise result.

The round up and round down modes are termed **directed rounding** and can be used to implement interval arithmetic. Interval arithmetic is used to determine upper and lower bounds for the true result of a multistep computation, when the intermediate results of the computation are subject to rounding.

The round toward zero mode (sometimes called the "chop" mode) is commonly used when performing integer arithmetic with the x87 FPU.

The rounded result is called the inexact result. When the processor produces an inexact result, the floating-point precision (inexact) flag (PE) is set (see Section 4.9.1.6, "Inexact-Result (Precision) Exception (#P)").

The rounding modes have no effect on comparison operations, operations that produce exact results, or operations that produce NaN results.

4.8.4.1 Rounding Control (RC) Fields

In the Intel 64 and IA-32 architectures, the rounding mode is controlled by a 2-bit rounding-control (RC) field (Table 4-9 shows the encoding of this field). The RC field is implemented in two different locations:

- X87 FPU control register (bits 10 and 11).
- The MXCSR register (bits 13 and 14).

Although these two RC fields perform the same function, they control rounding for different execution environments within the processor. The RC field in the x87 FPU control register controls rounding for computations performed with the x87 FPU instructions; the RC field in the MXCSR register controls rounding for SIMD floating-point computations performed with the Intel SSE/SSE2/SSE3/SSE4.1/AVX instructions.

4.8.4.2 Truncation with Intel® SSE, SSE2, and AVX Conversion Instructions

The following Intel SSE/SSE2 instructions automatically truncate the results of conversions from floating-point values to integers when the result is inexact: CVTTPD2DQ, CVTTPS2DQ, CVTTPD2PI, CVTTPS2PI, CVTTSD2SI, and CVTTSS2SI. Here, truncation means the round toward zero mode described in Table 4-9. There are also several Intel AVX2 and AVX-512 instructions which use truncation (VCVTT*).

4.9 OVERVIEW OF FLOATING-POINT EXCEPTIONS

The following section provides an overview of floating-point exceptions and their handling in the IA-32 architecture. For information specific to the x87 FPU and to the Intel SSE/SSE2/SSE3/SSE4.1/AVX extensions, refer to the following sections:

- Section 4.9, "Overview of Floating-Point Exceptions."
- Section 11.5, "Intel® SSE, SSE2, and SSE3 Exceptions."
- Section 12.8.4, "IEEE 754 Compliance of Intel® SSE4.1 Floating-Point Instructions."
- Section 14.10, "SIMD Floating-Point Exceptions."

When operating on floating-point operands, the IA-32 architecture recognizes and detects six classes of exception conditions:

- Invalid operation (#I).
- Divide-by-zero (#Z).
- Denormalized operand (#D).
- Numeric overflow (#O).
- Numeric underflow (#U).
- Inexact result (precision) (#P).

The nomenclature of "#" symbol followed by one or two letters (for example, #P) is used in this manual to indicate exception conditions. It is merely a short-hand form and is not related to assembler mnemonics.

NOTE

All of the exceptions listed above except the denormal-operand exception (#D) are defined in IEEE Standard 754.

The invalid-operation, divide-by-zero and denormal-operand exceptions are pre-computation exceptions (that is, they are detected before any arithmetic operation occurs). The numeric-underflow, numeric-overflow and precision exceptions are post-computation exceptions.

Each of the six exception classes has a corresponding flag bit (IE, ZE, OE, UE, DE, or PE) and mask bit (IM, ZM, OM, UM, DM, or PM). When one or more floating-point exception conditions are detected, the processor sets the appropriate flag bits, then takes one of two possible courses of action, depending on the settings of the corresponding mask bits:

- Mask bit set. Handles the exception automatically, producing a predefined (and often times usable) result, while allowing program execution to continue undisturbed.
- Mask bit clear. Invokes a software exception handler to handle the exception.

The masked (default) responses to exceptions have been chosen to deliver a reasonable result for each exception condition and are generally satisfactory for most floating-point applications. By masking or unmasking specific floating-point exceptions, programmers can delegate responsibility for most exceptions to the processor and reserve the most severe exception conditions for software exception handlers.

Because the exception flags are "sticky," they provide a cumulative record of the exceptions that have occurred since they were last cleared. A programmer can thus mask all exceptions, run a calculation, and then inspect the exception flags to see if any exceptions were detected during the calculation.

In the IA-32 architecture, floating-point exception flag and mask bits are implemented in two different locations:

- X87 FPU status word and control word. The flag bits are located at bits 0 through 5 of the x87 FPU status word and the mask bits are located at bits 0 through 5 of the x87 FPU control word (see Figures 8-4 and 8-6).
- MXCSR register. The flag bits are located at bits 0 through 5 of the MXCSR register and the mask bits are located at bits 7 through 12 of the register (see Figure 10-3).

Although these two sets of flag and mask bits perform the same function, they report on and control exceptions for different execution environments within the processor. The flag and mask bits in the x87 FPU status and control words control exception reporting and masking for computations performed with the x87 FPU instructions; the

companion bits in the MXCSR register control exception reporting and masking for SIMD floating-point computations performed with the Intel SSE/SSE2/SSE3/SSE4.1/AVX instructions.

Note that when exceptions are masked, the processor may detect multiple exceptions in a single instruction, because it continues executing the instruction after performing its masked response. For example, the processor can detect a denormalized operand, perform its masked response to this exception, and then detect numeric underflow.

See Section 4.9.2, "Floating-Point Exception Priority," for a description of the rules for exception precedence when more than one floating-point exception condition is detected for an instruction.

4.9.1 Floating-Point Exception Conditions

The following sections describe the various conditions that cause a floating-point exception to be generated and the masked response of the processor when these conditions are detected. The Intel® 64 and IA-32 Architectures Software Developer's Manual, Volumes 3A, 3B, 3C, & 3D, lists the floating-point exceptions that can be signaled for each floating-point instruction.

4.9.1.1 Invalid Operation Exception (#I)

The processor reports an invalid operation exception in response to one or more invalid arithmetic operands. If the invalid operation exception is masked, the processor sets the IE flag and returns an indefinite value or a QNaN. This value overwrites the destination register specified by the instruction. If the invalid operation exception is not masked, the IE flag is set, a software exception handler is invoked, and the operands remain unaltered.

See Section 4.8.3.6, "Using SNaNs and QNaNs in Applications," for information about the result returned when an exception is caused by an SNaN.

The processor can detect a variety of invalid arithmetic operations that can be coded in a program. These operations generally indicate a programming error, such as dividing ∞ by ∞. See the following sections for information regarding the invalid-operation exception when detected while executing x87 FPU or Intel SSE/SSE2/SSE3/SSE4.1/AVX instructions:

- X87 FPU; Section 8.5.1, "Invalid Operation Exception."
- SIMD floating-point exceptions; Section 11.5.2.1, "Invalid Operation Exception (#I)."
- Section 12.8.4, "IEEE 754 Compliance of Intel® SSE4.1 Floating-Point Instructions."
- Section 14.10, "SIMD Floating-Point Exceptions."

4.9.1.2 Denormal Operand Exception (#D)

The processor reports the denormal-operand exception if an arithmetic instruction attempts to operate on a denormal operand (see Section 4.8.3.2, "Normalized and Denormalized Finite Numbers"). When the exception is masked, the processor sets the DE flag and proceeds with the instruction. Operating on denormal numbers will produce results at least as good as, and often better than, what can be obtained when denormal numbers are flushed to zero. Programmers can mask this exception so that a computation may proceed, then analyze any loss of accuracy when the final result is delivered.

When a denormal-operand exception is not masked, the DE flag is set, a software exception handler is invoked, and the operands remain unaltered. When denormal operands have reduced significance due to loss of low-order bits, it may be advisable to not operate on them. Precluding denormal operands from computations can be accomplished by an exception handler that responds to unmasked denormal-operand exceptions.

See the following sections for information regarding the denormal-operand exception when detected while executing x87 FPU or Intel SSE/SSE2/SSE3/SSE4.1/AVX instructions:

- X87 FPU; Section 8.5.2, "Denormal Operand Exception (#D)."
- SIMD floating-point exceptions; Section 11.5.2.2, "Denormal-Operand Exception (#D)."
- Section 12.8.4, "IEEE 754 Compliance of Intel® SSE4.1 Floating-Point Instructions."
- Section 14.10, "SIMD Floating-Point Exceptions."

4.9.1.3 Divide-By-Zero Exception (#Z)

The processor reports the floating-point divide-by-zero exception whenever an instruction attempts to divide a finite non-zero operand by 0. The masked response for the divide-by-zero exception is to set the ZE flag and return an infinity signed with the exclusive OR of the sign of the operands. If the divide-by-zero exception is not masked, the ZE flag is set, a software exception handler is invoked, and the operands remain unaltered.

See the following sections for information regarding the divide-by-zero exception when detected while executing x87 FPU or Intel SSE/SSE2/AVX instructions:

- X87 FPU; Section 8.5.3, "Divide-By-Zero Exception (#Z)."
- SIMD floating-point exceptions; Section 11.5.2.3, "Divide-By-Zero Exception (#Z)."
- Section 12.8.4, "IEEE 754 Compliance of Intel® SSE4.1 Floating-Point Instructions."
- Section 14.10, "SIMD Floating-Point Exceptions."

4.9.1.4 Numeric Overflow Exception (#O)

The processor reports a floating-point numeric overflow exception whenever the rounded result of an instruction exceeds the largest allowable finite value that will fit into the destination operand. Table 4-10 shows the threshold range for numeric overflow for each of the floating-point formats; overflow occurs when a rounded result falls at or outside this threshold range.

Table 4-10. Numeric Overflow Thresholds

Floating-Point Format	Overflow Thresholds
Half Precision	$\lvert x \rvert \geq 1.0 * 2^{16}$
Single Precision	$\lvert x \rvert \geq 1.0 * 2^{128}$
Double Precision	$\lvert x \rvert \geq 1.0 * 2^{1024}$
Double Extended Precision	$\lvert x \rvert \geq 1.0 * 2^{16384}$

When a numeric-overflow exception occurs and the exception is masked, the processor sets the OE flag and returns one of the values shown in Table 4-11, according to the current rounding mode. See Section 4.8.4, "Rounding."

When numeric overflow occurs and the numeric-overflow exception is not masked, the OE flag is set, a software exception handler is invoked, and the source and destination operands either remain unchanged or a biased result is stored in the destination operand (depending whether the overflow exception was generated during an Intel SSE/SSE2/SSE3/SSE4.1/AVX floating-point operation or an x87 FPU operation).

Table 4-11. Masked Responses to Numeric Overflow

Rounding Mode	Sign of True Result	Result
To nearest	+	+∞
	–	–∞
Toward –∞	+	Largest finite positive number
	–	–∞
Toward +∞	+	+∞
	–	Largest finite negative number
Toward zero	+	Largest finite positive number
	–	Largest finite negative number

See the following sections for information regarding the numeric overflow exception when detected while executing x87 FPU instructions or while executing Intel SSE/SSE2/SSE3/SSE4.1/AVX instructions:

- X87 FPU; Section 8.5.4, "Numeric Overflow Exception (#O)."
- SIMD floating-point exceptions; Section 11.5.2.4, "Numeric Overflow Exception (#O)."

- Section 12.8.4, "IEEE 754 Compliance of Intel® SSE4.1 Floating-Point Instructions."
- Section 14.10, "SIMD Floating-Point Exceptions."

4.9.1.5 Numeric Underflow Exception (#U)

The processor detects a potential floating-point numeric underflow condition whenever the result of rounding with unbounded exponent (taking into account precision control for x87) is non-zero and tiny; that is, non-zero and less than the smallest possible normalized, finite value that will fit into the destination operand. Table 4-12 shows the threshold range for numeric underflow for each of the floating-point formats (assuming normalized results); underflow occurs when a rounded result falls strictly within the threshold range. The ability to detect and handle underflow is provided to prevent a very small result from propagating through a computation and causing another exception (such as overflow during division) to be generated at a later time. Results which trigger underflow are also potentially less accurate.

Table 4-12. Numeric Underflow (Normalized) Thresholds

Floating-Point Format	Underflow Thresholds[1]
Half Precision	$\|x\| < 1.0 * 2^{-14}$
Single Precision	$\|x\| < 1.0 * 2^{-126}$
Double Precision	$\|x\| < 1.0 * 2^{-1022}$
Double Extended Precision	$\|x\| < 1.0 * 2^{-16382}$

NOTES:

1. Where 'x' is the result rounded to destination precision with an unbounded exponent range.

How the processor handles an underflow condition, depends on two related conditions:

- Creation of a tiny, non-zero result.
- Creation of an inexact result; that is, a result that cannot be represented exactly in the destination format.

Which of these events causes an underflow exception to be reported and how the processor responds to the exception condition depends on whether the underflow exception is masked:

- **Underflow exception masked —** The underflow exception is reported (the UE flag is set) only when the result is both tiny and inexact. The processor returns a correctly signed result whose magnitude is less than or equal to the smallest positive normal floating-point number to the destination operand, regardless of inexactness.
- **Underflow exception not masked —** The underflow exception is reported when the result is non-zero tiny, regardless of inexactness. The processor leaves the source and destination operands unaltered or stores a biased result in the destination operand (depending whether the underflow exception was generated during an Intel SSE/SSE2/SSE3/AVX floating-point operation or an x87 FPU operation) and invokes a software exception handler.

See the following sections for information regarding the numeric underflow exception when detected while executing x87 FPU instructions or while executing Intel SSE/SSE2/SSE3/SSE4.1/AVX instructions:

- X87 FPU; Section 8.5.5, "Numeric Underflow Exception (#U)."
- SIMD floating-point exceptions; Section 11.5.2.5, "Numeric Underflow Exception (#U)."
- Section 12.8.4, "IEEE 754 Compliance of Intel® SSE4.1 Floating-Point Instructions."
- Section 14.10, "SIMD Floating-Point Exceptions."

4.9.1.6 Inexact-Result (Precision) Exception (#P)

The inexact-result exception (also called the precision exception) occurs if the result of an operation is not exactly representable in the destination format. For example, the fraction 1/3 cannot be precisely represented in binary floating-point form. This exception occurs frequently and indicates that some (normally acceptable) accuracy will be lost due to rounding. The exception is supported for applications that need to perform exact arithmetic only. Because the rounded result is generally satisfactory for most applications, this exception is commonly masked.

If the inexact-result exception is masked when an inexact-result condition occurs and a numeric overflow or underflow condition has not occurred, the processor sets the PE flag and stores the rounded result in the destination operand. The current rounding mode determines the method used to round the result. See Section 4.8.4, "Rounding."

If the inexact-result exception is not masked when an inexact result occurs and numeric overflow or underflow has not occurred, the PE flag is set, the rounded result is stored in the destination operand, and a software exception handler is invoked.

If an inexact result occurs in conjunction with numeric overflow or underflow, one of the following operations is carried out:

- If an inexact result occurs along with masked overflow or underflow, the OE flag or UE flag and the PE flag are set and the result is stored as described for the overflow or underflow exceptions; see Section 4.9.1.4, "Numeric Overflow Exception (#O)," or Section 4.9.1.5, "Numeric Underflow Exception (#U)." If the inexact result exception is unmasked, the processor also invokes a software exception handler.
- If an inexact result occurs along with unmasked overflow or underflow and the destination operand is a register, the OE or UE flag and the PE flag are set, the result is stored as described for the overflow or underflow exceptions, and a software exception handler is invoked.

If an unmasked numeric overflow or underflow exception occurs and the destination operand is a memory location (which can happen only for a floating-point store), the inexact-result condition is not reported and the C1 flag is cleared.

See the following sections for information regarding the inexact-result exception when detected while executing x87 FPU or Intel SSE/SSE2/SSE3/SSE4.1/AVX instructions:

- X87 FPU; Section 8.5.6, "Inexact-Result (Precision) Exception (#P)."
- SIMD floating-point exceptions; Section 11.5.2.3, "Divide-By-Zero Exception (#Z)."
- Section 12.8.4, "IEEE 754 Compliance of Intel® SSE4.1 Floating-Point Instructions."
- Section 14.10, "SIMD Floating-Point Exceptions."

4.9.2 Floating-Point Exception Priority

The processor handles exceptions according to a predetermined precedence. When an instruction generates two or more exception conditions, the exception precedence sometimes results in the higher-priority exception being handled and the lower-priority exceptions being ignored. For example, dividing an SNaN by zero can potentially signal an invalid-operation exception (due to the SNaN operand) and a divide-by-zero exception. Here, if both exceptions are masked, the processor handles the higher-priority exception only (the invalid-operation exception), returning a QNaN to the destination. Alternately, a denormal-operand or inexact-result exception can accompany a numeric underflow or overflow exception with both exceptions being handled.

The precedence for floating-point exceptions is as follows:

1. Invalid-operation exception, subdivided as follows:
   a. Stack underflow (occurs with x87 FPU only).
   b. Stack overflow (occurs with x87 FPU only).
   c. Operand of unsupported format (occurs with x87 FPU only when using the double extended precision floating-point format).
   d. SNaN operand.
2. QNaN operand. Though this is not an exception, the handling of a QNaN operand has precedence over lower-priority exceptions. For example, a QNaN divided by zero results in a QNaN, not a zero-divide exception.
3. Any other invalid-operation exception not mentioned above or a divide-by-zero exception.
4. Denormal-operand exception. If masked, then instruction execution continues, and a lower-priority exception can occur as well.
5. Numeric overflow and underflow exceptions; possibly in conjunction with the inexact-result exception.
6. Inexact-result exception.

Invalid operation, zero divide, and denormal operand exceptions are detected before a floating-point operation begins. Overflow, underflow, and precision exceptions are not detected until a true result has been computed. When an unmasked **pre-operation** exception is detected, the destination operand has not yet been updated, and appears as if the offending instruction has not been executed. When an unmasked **post-operation** exception is detected, the destination operand may be updated with a result, depending on the nature of the exception (except for Intel SSE/SSE2/SSE3/AVX instructions, which do not update their destination operands in such cases).

4.9.3 Typical Actions of a Floating-Point Exception Handler

After the floating-point exception handler is invoked, the processor handles the exception in the same manner that it handles non-floating-point exceptions. The floating-point exception handler is normally part of the operating system or executive software, and it usually invokes a user-registered floating-point exception handle.

A typical action of the exception handler is to store state information in memory. Other typical exception handler actions include:

- Examining the stored state information to determine the nature of the error.
- Taking actions to correct the condition that caused the error.
- Clearing the exception flags.
- Returning to the interrupted program and resuming normal execution.

In lieu of writing recovery procedures, the exception handler can do the following:

- Increment in software an exception counter for later display or printing.
- Print or display diagnostic information (such as the state information).
- Halt further program execution.

CHAPTER 5
INSTRUCTION SET SUMMARY

This chapter provides an abridged overview of Intel 64 and IA-32 instructions. Instructions are divided into the following groups:

- Section 5.1, “General-Purpose Instructions.”
- Section 5.2, “x87 FPU Instructions.”
- Section 5.3, “x87 FPU AND SIMD State Management Instructions.”
- Section 5.4, “MMX Instructions.”
- Section 5.5, “Intel® SSE Instructions.”
- Section 5.6, “Intel® SSE2 Instructions.”
- Section 5.7, “Intel® SSE3 Instructions.”
- Section 5.8, “Supplemental Streaming SIMD Extensions 3 (SSSE3) Instructions.”
- Section 5.9, “Intel® SSE4 Instructions.”
- Section 5.10, “Intel® SSE4.1 Instructions.”
- Section 5.11, “Intel® SSE4.2 Instruction Set.”
- Section 5.12, “Intel® AES-NI and PCLMULQDQ.”
- Section 5.13, “Intel® Advanced Vector Extensions (Intel® AVX).”
- Section 5.14, “16-bit Floating-Point Conversion.”
- Section 5.15, “Fused-Multiply-ADD (FMA).”
- Section 5.16, “Intel® Advanced Vector Extensions 2 (Intel® AVX2).”
- Section 5.17, “Intel® Transactional Synchronization Extensions (Intel® TSX).”
- Section 5.18, “Intel® SHA Extensions.”
- Section 5.19, “Intel® Advanced Vector Extensions 512 (Intel® AVX-512).”
- Section 5.20, “System Instructions.”
- Section 5.21, “64-Bit Mode Instructions.”
- Section 5.22, “Virtual-Machine Extensions.”
- Section 5.23, “Safer Mode Extensions.”
- Section 5.24, “Intel® Memory Protection Extensions.”
- Section 5.25, “Intel® Software Guard Extensions.”
- Section 5.26, “Shadow Stack Management Instructions.”
- Section 5.27, “Control Transfer Terminating Instructions.”
- Section 5.28, “Intel® AMX Instructions.”
- Section 5.29, “User Interrupt Instructions.”
- Section 5.30, “Enqueue Store Instructions.”
- Section 5.31, “Intel® Advanced Vector Extensions 10 Version 1 Instructions.”

Table 5-1 lists the groups and IA-32 processors that support each group. More recent instruction set extensions are listed in Table 5-2. Within these groups, most instructions are collected into functional subgroups.

Table 5-1. Instruction Groups in Intel® 64 and IA-32 Processors

Instruction Set Architecture	Intel 64 and IA-32 Processor Support
General Purpose	All Intel 64 and IA-32 processors.
X87 FPU	Intel486, Pentium, Pentium with MMX Technology, Celeron, Pentium Pro, Pentium II, Pentium II Xeon, Pentium III, Pentium III Xeon, Pentium 4, Intel Xeon processors, Pentium M, Intel Core Solo, Intel Core Duo, Intel Core 2 Duo processors, Intel Atom processors.
X87 FPU and SIMD State Management	Pentium II, Pentium II Xeon, Pentium III, Pentium III Xeon, Pentium 4, Intel Xeon processors, Pentium M, Intel Core Solo, Intel Core Duo, Intel Core 2 Duo processors, Intel Atom processors.
MMX Technology	Pentium with MMX Technology, Celeron, Pentium II, Pentium II Xeon, Pentium III, Pentium III Xeon, Pentium 4, Intel Xeon processors, Pentium M, Intel Core Solo, Intel Core Duo, Intel Core 2 Duo processors, Intel Atom processors.
SSE Extensions	Pentium III, Pentium III Xeon, Pentium 4, Intel Xeon processors, Pentium M, Intel Core Solo, Intel Core Duo, Intel Core 2 Duo processors, Intel Atom processors.
SSE2 Extensions	Pentium 4, Intel Xeon processors, Pentium M, Intel Core Solo, Intel Core Duo, Intel Core 2 Duo processors, Intel Atom processors.
SSE3 Extensions	Pentium 4 supporting HT Technology (built on 90 nm process technology), Intel Core Solo, Intel Core Duo, Intel Core 2 Duo processors, Intel Xeon processor 3xxxx, 5xxx, 7xxx Series, Intel Atom processors.
SSSE3 Extensions	Intel Xeon processor 3xxx, 5100, 5200, 5300, 5400, 5500, 5600, 7300, 7400, 7500 series, Intel Core 2 Extreme processors QX6000 series, Intel Core 2 Duo, Intel Core 2 Quad processors, Intel Pentium Dual-Core processors, Intel Atom processors.
IA-32e mode: 64-bit mode instructions	Intel 64 processors.
System Instructions	Intel 64 and IA-32 processors.
VMX Instructions	Intel 64 and IA-32 processors supporting Intel Virtualization Technology.
SMX Instructions	Intel Core 2 Duo processor E6x50, E8xxx; Intel Core 2 Quad processor Q9xxx.

Table 5-2. Instruction Set Extensions Introduction in Intel® 64 and IA-32 Processors

Instruction Set Architecture	Processor Generation Introduction
SSE4.1 Extensions	Intel® Xeon® processor 3100, 3300, 5200, 5400, 7400, 7500 series, Intel® Core™ 2 Extreme processors QX9000 series, Intel® Core™ 2 Quad processor Q9000 series, Intel® Core™ 2 Duo processors 8000 series and T9000 series, Intel Atom® processor based on Silvermont microarchitecture.
SSE4.2 Extensions, CRC32, POPCNT	Intel® Core™ i7 965 processor, Intel® Xeon® processors X3400, X3500, X5500, X6500, X7500 series, Intel Atom processor based on Silvermont microarchitecture.
Intel® AES-NI, PCLMULQDQ	Intel® Xeon® processor E7 series, Intel® Xeon® processors X3600 and X5600, Intel® Core™ i7 980X processor, Intel Atom processor based on Silvermont microarchitecture. Use CPUID to verify presence of Intel AES-NI and PCLMULQDQ across Intel® Core™ processor families.
Intel® AVX	Intel® Xeon® processor E3 and E5 families, 2nd Generation Intel® Core™ i7, i5, i3 processor 2xxx families.
F16C	3rd Generation Intel® Core™ processors, Intel® Xeon® processor E3-1200 v2 product family, Intel® Xeon® processor E5 v2 and E7 v2 families.
RDRAND	3rd Generation Intel Core processors, Intel Xeon processor E3-1200 v2 product family, Intel Xeon processor E5 v2 and E7 v2 families, Intel Atom processor based on Silvermont microarchitecture.
FS/GS base access	3rd Generation Intel Core processors, Intel Xeon processor E3-1200 v2 product family, Intel Xeon processor E5 v2 and E7 v2 families, Intel Atom® processor based on Goldmont microarchitecture.

Table 5-2. Instruction Set Extensions Introduction in Intel® 64 and IA-32 Processors (Contd.)

Instruction Set Architecture	Processor Generation Introduction
FMA, AVX2, BMI1, BMI2, INVPCID, LZCNT, Intel® TSX	Intel® Xeon® processor E3/E5/E7 v3 product families, 4th Generation Intel® Core™ processor family.
MOVBE	Intel Xeon processor E3/E5/E7 v3 product families, 4th Generation Intel Core processor family, Intel Atom processors.
PREFETCHW	Intel® Core™ M processor family; 5th Generation Intel® Core™ processor family, Intel Atom processor based on Silvermont microarchitecture.
ADX	Intel Core M processor family, 5th Generation Intel Core processor family.
RDSEED, CLAC, STAC	Intel Core M processor family, 5th Generation Intel Core processor family, Intel Atom processor based on Goldmont microarchitecture.
AVX512ER, AVX512PF, PREFETCHWT1	Intel® Xeon Phi™ Processor 3200, 5200, 7200 Series.
AVX512F, AVX512CD	Intel Xeon Phi Processor 3200, 5200, 7200 Series, Intel® Xeon® Scalable Processor Family, Intel® Core™ i3-8121U processor.
CLFLUSHOPT, XSAVEC, XSAVES, Intel® MPX	Intel Xeon Scalable Processor Family, 6th Generation Intel® Core™ processor family, Intel Atom processor based on Goldmont microarchitecture.
SGX1	6th Generation Intel Core processor family, Intel Atom® processor based on Goldmont Plus microarchitecture.
AVX512DQ, AVX512BW, AVX512VL	Intel Xeon Scalable Processor Family, Intel Core i3-8121U processor based on Cannon Lake microarchitecture.
CLWB	Intel Xeon Scalable Processor Family, Intel Atom® processor based on Tremont microarchitecture, 11th Generation Intel Core processor family based on Tiger Lake microarchitecture.
PKU	Intel Xeon Scalable Processor Family, 10th generation Intel® Core™ processors based on Comet Lake microarchitecture.
AVX512_IFMA, AVX512_VBMI	Intel Core i3-8121U processor based on Cannon Lake microarchitecture.
Intel® SHA Extensions	Intel Core i3-8121U processor based on Cannon Lake microarchitecture, Intel Atom processor based on Goldmont microarchitecture, 3rd Generation Intel® Xeon® Scalable Processor Family based on Ice Lake microarchitecture.
UMIP	Intel Core i3-8121U processor based on Cannon Lake microarchitecture, Intel Atom processor based on Goldmont Plus microarchitecture.
PTWRITE	Intel Atom processor based on Goldmont Plus microarchitecture, 12th generation Intel® Core™ processor supporting Alder Lake performance hybrid architecture, 4th generation Intel® Xeon® Scalable Processor Family based on Sapphire Rapids microarchitecture.
RDPID	10th Generation Intel® Core™ processor family based on Ice Lake microarchitecture, Intel Atom processor based on Goldmont Plus microarchitecture.
AVX512_4FMAPS, AVX512_4VNNIW	Intel® Xeon Phi™ Processor 7215, 7285, 7295 Series.
AVX512_VNNI	2nd Generation Intel® Xeon® Scalable Processor Family, 10th Generation Intel Core processor family based on Ice Lake microarchitecture.
AVX512_VPOPCNTDQ	Intel Xeon Phi Processor 7215, 7285, 7295 Series, 10th Generation Intel Core processor family based on Ice Lake microarchitecture.
Fast Short REP MOVSB	10th Generation Intel Core processor family based on Ice Lake microarchitecture.
GFNI (SSE)	10th Generation Intel Core processor family based on Ice Lake microarchitecture, Intel Atom processor based on Tremont microarchitecture.

Table 5-2. Instruction Set Extensions Introduction in Intel® 64 and IA-32 Processors (Contd.)

Instruction Set Architecture	Processor Generation Introduction
VAES, GFNI (AVX/AVX512), AVX512_VBMI2, VPCLMULQDQ, AVX512_BITALG	10th Generation Intel Core processor family based on Ice Lake microarchitecture.
Split Lock Detection	10th Generation Intel Core processor family based on Ice Lake microarchitecture, Intel Atom processor based on Tremont microarchitecture.
CLDEMOTE	Intel Atom processor based on Tremont microarchitecture, 4th generation Intel® Xeon® Scalable Processor Family based on Sapphire Rapids microarchitecture.
Direct stores: MOVDIRI, MOVDIR64B	Intel Atom processor based on Tremont microarchitecture, 11th Generation Intel Core processor family based on Tiger Lake microarchitecture, 4th generation Intel® Xeon® Scalable Processor Family based on Sapphire Rapids microarchitecture.
User wait: TPAUSE, UMONITOR, UMWAIT	Intel Atom processor based on Tremont microarchitecture, 12th generation Intel Core processor based on Alder Lake performance hybrid architecture, 4th generation Intel® Xeon® Scalable Processor Family based on Sapphire Rapids microarchitecture.
AVX512-BF16	3rd Generation Intel® Xeon® Scalable Processor Family based on Cooper Lake product, 4th generation Intel® Xeon® Scalable Processor Family based on Sapphire Rapids microarchitecture.
AVX512_VP2INTERSECT	11th Generation Intel Core processor family based on Tiger Lake microarchitecture. (Not currently supported in any other processors).
Key Locker[1]	11th Generation Intel Core processor family based on Tiger Lake microarchitecture, 12th generation Intel Core processor supporting Alder Lake performance hybrid architecture.
Control-flow Enforcement Technology (CET)	11th Generation Intel Core processor family based on Tiger Lake microarchitecture, 4th generation Intel® Xeon® Scalable Processor Family based on Sapphire Rapids microarchitecture, Intel® Xeon® 6 E-core processors based on Sierra Forest microarchitecture.
TME-MK[2], PCONFIG	3rd Generation Intel® Xeon® Scalable Processor Family based on Ice Lake microarchitecture.
WBNOINVD	3rd Generation Intel® Xeon® Scalable Processor Family based on Ice Lake microarchitecture.
LBRs (architectural)	12th generation Intel Core processor supporting Alder Lake performance hybrid architecture, 4th generation Intel® Xeon® Scalable Processor Family based on Sapphire Rapids microarchitecture, Intel® Xeon® 6 E-core processors based on Sierra Forest microarchitecture.
Intel® Virtualization Technology - Redirect Protection (Intel® VT-rp) and HLAT	12th generation Intel Core processor supporting Alder Lake performance hybrid architecture, 4th generation Intel® Xeon® Scalable Processor Family based on Sapphire Rapids microarchitecture, Intel® Xeon® 6 E-core processors based on Sierra Forest microarchitecture.
AVX-VNNI	12th generation Intel Core processor supporting Alder Lake performance hybrid architecture[3], 4th generation Intel® Xeon® Scalable Processor Family based on Sapphire Rapids microarchitecture, Intel® Xeon® 6 E-core processors based on Sierra Forest microarchitecture.
SERIALIZE	12th generation Intel Core processor supporting Alder Lake performance hybrid architecture, 4th generation Intel® Xeon® Scalable Processor Family based on Sapphire Rapids microarchitecture, Intel® Xeon® 6 E-core processors based on Sierra Forest microarchitecture.
Intel® Thread Director and HRESET	12th generation Intel Core processor supporting Alder Lake performance hybrid architecture.
Fast zero-length REP MOVSB, fast short REP STOSB	12th generation Intel Core processor supporting Alder Lake performance hybrid architecture, 4th generation Intel® Xeon® Scalable Processor Family based on Sapphire Rapids microarchitecture.
Fast Short REP CMPSB, fast short REP SCASB	4th generation Intel® Xeon® Scalable Processor Family based on Sapphire Rapids microarchitecture.
Supervisor Memory Protection Keys (PKS)	12th generation Intel Core processor supporting Alder Lake performance hybrid architecture, 4th generation Intel® Xeon® Scalable Processor Family based on Sapphire Rapids microarchitecture, Intel® Xeon® 6 E-core processors based on Sierra Forest microarchitecture.

Table 5-2. Instruction Set Extensions Introduction in Intel® 64 and IA-32 Processors (Contd.)

Instruction Set Architecture	Processor Generation Introduction
Attestation Services for Intel® SGX	3rd Generation Intel® Xeon® Scalable Processor Family based on Ice Lake microarchitecture.
Enqueue Stores: ENQCMD and ENQCMDS	4th generation Intel® Xeon® Scalable Processor Family based on Sapphire Rapids microarchitecture, Intel® Xeon® 6 E-core processors based on Sierra Forest microarchitecture.
Intel® TSX Suspend Load Address Tracking (TSXLDTRK)	4th generation Intel® Xeon® Scalable Processor Family based on Sapphire Rapids microarchitecture.
Intel® Advanced Matrix Extensions (Intel® AMX) Includes CPUID.1EH, "TMUL Information Main Leaf", and CPUID bits AMX_BF16, AMX_TILE, and AMX_INT8.	4th generation Intel® Xeon® Scalable Processor Family based on Sapphire Rapids microarchitecture.
User Interrupts (UINTR)	4th generation Intel® Xeon® Scalable Processor Family based on Sapphire Rapids microarchitecture, Intel® Xeon® 6 E-core processors based on Sierra Forest microarchitecture, Intel® Atom® P6900 Processor based on the Crestmont microarchitecture, Intel® Core™ Ultra 200H Series processors supporting Arrow Lake performance hybrid architecture.
IPI Virtualization	4th generation Intel® Xeon® Scalable Processor Family based on Sapphire Rapids microarchitecture, Intel® Xeon® 6 E-core processors based on Sierra Forest microarchitecture, Intel® Atom® P6900 Processor based on the Crestmont microarchitecture, Intel® Core™ Ultra 200H Series processors supporting Arrow Lake performance hybrid architecture.
AVX512-FP16 for the FP16 Data Type	4th generation Intel® Xeon® Scalable Processor Family based on Sapphire Rapids microarchitecture.
Virtualization of guest accesses to IA32_SPEC_CTRL	4th generation Intel® Xeon® Scalable Processor Family based on Sapphire Rapids microarchitecture, Intel® Xeon® 6 E-core processors based on Sierra Forest microarchitecture, Intel® Atom® P6900 Processor based on the Crestmont microarchitecture.
Linear Address Masking (LAM)	Intel® Xeon® 6 E-core processors based on Sierra Forest microarchitecture, Intel® Atom® P6900 Processor based on the Crestmont microarchitecture, Intel® Core™ Ultra 200H Series processors supporting Arrow Lake performance hybrid architecture.
Linear Address Space Separation (LASS)	Intel® Xeon® 6 E-core processors based on Sierra Forest microarchitecture, Intel® Atom® P6900 Processor based on the Crestmont microarchitecture, Intel® Core™ Ultra 200H Series processors supporting Arrow Lake performance hybrid architecture.
PREFETCHIT0/1	Intel® Xeon® 6 P-core processors based on Granite Rapids microarchitecture.
AMX_FP16	Intel® Xeon® 6 P-core processors based on Granite Rapids microarchitecture.
CMPCCXADD	Intel® Xeon® 6 E-core processors based on Sierra Forest microarchitecture, Intel® Atom® P6900 Processor based on the Crestmont microarchitecture, Intel® Core™ Ultra 200H Series processors supporting Arrow Lake performance hybrid architecture.
AVX-IFMA	Intel® Xeon® 6 E-core processors based on Sierra Forest microarchitecture, Intel® Atom® P6900 Processor based on the Crestmont microarchitecture, Intel® Core™ Ultra 200H Series processors supporting Arrow Lake performance hybrid architecture.
AVX-NE-CONVERT	Intel® Xeon® 6 E-core processors based on Sierra Forest microarchitecture, Intel® Atom® P6900 Processor based on the Crestmont microarchitecture, Intel® Core™ Ultra 200H Series processors supporting Arrow Lake performance hybrid architecture.
AVX-VNNI-INT8	Intel® Xeon® 6 E-core processors based on Sierra Forest microarchitecture, Intel® Atom® P6900 Processor based on the Crestmont microarchitecture, Intel® Core™ Ultra 200H Series processors supporting Arrow Lake performance hybrid architecture.
AVX-VNNI-INT16	Intel® Core™ Ultra 200S Series processors supporting Arrow Lake performance hybrid architecture.
SHA512	Intel® Core™ Ultra 200S Series processors supporting Arrow Lake performance hybrid architecture.

Table 5-2. Instruction Set Extensions Introduction in Intel® 64 and IA-32 Processors (Contd.)

Instruction Set Architecture	Processor Generation Introduction
SM3	Intel® Core™ Ultra 200S Series processors supporting Arrow Lake performance hybrid architecture.
SM4	Intel® Core™ Ultra 200S Series processors supporting Arrow Lake performance hybrid architecture.
RDMSRLIST, WRMSRLIST, and WRMSRNS	Intel® Xeon® 6 E-core processors based on Sierra Forest microarchitecture, Intel® Atom® P6900 Processor based on the Crestmont microarchitecture.
UC Lock Disable Causes #AC	Intel® Xeon® 6 E-core processors based on Sierra Forest microarchitecture, Intel® Atom® P6900 Processor based on the Crestmont microarchitecture.
LBR Event Logging	Intel® Xeon® 6 E-core processors based on Sierra Forest microarchitecture, Intel® Atom® P6900 Processor based on the Crestmont microarchitecture, Intel® Core™ Ultra 200S Series processors supporting Arrow Lake performance hybrid architecture.
UIRET flexibly updates UIF	Intel® Xeon® 6 E-core processors based on Sierra Forest microarchitecture, Intel® Atom® P6900 Processor based on the Crestmont microarchitecture, Intel® Core™ Ultra 200H Series processors supporting Arrow Lake performance hybrid architecture.
Intel® Advanced Vector Extensions 10 Version 1 (Intel® AVX10.1)	Intel® Xeon® 6 P-core processors based on Granite Rapids microarchitecture.

NOTES:

1. Details on Key Locker can be found in the Intel Key Locker Specification here:
   https://software.intel.com/content/www/us/en/develop/download/intel-key-locker-specification.html.
2. Further details on TME-MK usage can be found here:
   https://software.intel.com/sites/default/files/managed/a5/16/Multi-Key-Total-Memory-Encryption-Spec.pdf.
3. Alder Lake performance hybrid architecture does not support Intel® AVX-512. ISA features such as Intel® AVX, AVX-VNNI, Intel® AVX2, and UMONITOR/UMWAIT/TPAUSE are supported.

The following sections list instructions in each major group and subgroup. Given for each instruction is its mnemonic and descriptive names. When two or more mnemonics are given (for example, CMOVA/CMOVNBE), they represent different mnemonics for the same instruction opcode. Assemblers support redundant mnemonics for some instructions to make it easier to read code listings. For instance, CMOVA (Conditional move if above) and CMOVNBE (Conditional move if not below or equal) represent the same condition. For detailed information about specific instructions, see the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volumes 2A, 2B, 2C, & 2D.

5.1 GENERAL-PURPOSE INSTRUCTIONS

The general-purpose instructions perform basic data movement, arithmetic, logic, program flow, and string operations that programmers commonly use to write application and system software to run on Intel 64 and IA-32 processors. They operate on data contained in memory, in the general-purpose registers (EAX, EBX, ECX, EDX, EDI, ESI, EBP, and ESP) and in the EFLAGS register. They also operate on address information contained in memory, the general-purpose registers, and the segment registers (CS, DS, SS, ES, FS, and GS).

This group of instructions includes the data transfer, binary integer arithmetic, decimal arithmetic, logic operations, shift and rotate, bit and byte operations, program control, string, flag control, segment register operations, and miscellaneous subgroups. The sections that follow introduce each subgroup.

For more detailed information on general purpose-instructions, see Chapter 7, "Programming With General-Purpose Instructions."

5.1.1 Data Transfer Instructions

The data transfer instructions move data between memory and the general-purpose and segment registers. They also perform specific operations such as conditional moves, stack access, and data conversion.

MOV	Move data between general-purpose registers; move data between memory and general-purpose or segment registers; move immediates to general-purpose registers.
CMOVE/CMOVZ	Conditional move if equal/Conditional move if zero.
CMOVNE/CMOVNZ	Conditional move if not equal/Conditional move if not zero.
CMOVA/CMOVNBE	Conditional move if above/Conditional move if not below or equal.
CMOVAE/CMOVNB	Conditional move if above or equal/Conditional move if not below.
CMOVB/CMOVNAE	Conditional move if below/Conditional move if not above or equal.
CMOVBE/CMOVNA	Conditional move if below or equal/Conditional move if not above.
CMOVG/CMOVNLE	Conditional move if greater/Conditional move if not less or equal.
CMOVGE/CMOVNL	Conditional move if greater or equal/Conditional move if not less.
CMOVL/CMOVNGE	Conditional move if less/Conditional move if not greater or equal.
CMOVLE/CMOVNG	Conditional move if less or equal/Conditional move if not greater.
CMOVC	Conditional move if carry.
CMOVNC	Conditional move if not carry.
CMOVO	Conditional move if overflow.
CMOVNO	Conditional move if not overflow.
CMOVS	Conditional move if sign (negative).
CMOVNS	Conditional move if not sign (non-negative).
CMOVP/CMOVPE	Conditional move if parity/Conditional move if parity even.
CMOVNP/CMOVPO	Conditional move if not parity/Conditional move if parity odd.
XCHG	Exchange.
BSWAP	Byte swap.
XADD	Exchange and add.
CMPXCHG	Compare and exchange.
CMPXCHG8B	Compare and exchange 8 bytes.
PUSH	Push onto stack.
POP	Pop off of stack.
PUSHA/PUSHAD	Push general-purpose registers onto stack.
POPA/POPAD	Pop general-purpose registers from stack.
CWD/CDQ	Convert word to doubleword/Convert doubleword to quadword.
CBW/CWDE	Convert byte to word/Convert word to doubleword in EAX register.
MOVSX	Move and sign extend.
MOVZX	Move and zero extend.

5.1.2 Binary Arithmetic Instructions

The binary arithmetic instructions perform basic binary integer computations on byte, word, and doubleword integers located in memory and/or the general purpose registers.

ADCX	Unsigned integer add with carry.
ADOX	Unsigned integer add with overflow.
ADD	Integer add.
ADC	Add with carry.
SUB	Subtract.
SBB	Subtract with borrow.

IMUL	Signed multiply.
MUL	Unsigned multiply.
IDIV	Signed divide.
DIV	Unsigned divide.
INC	Increment.
DEC	Decrement.
NEG	Negate.
CMP	Compare.

5.1.3 Decimal Arithmetic Instructions

The decimal arithmetic instructions perform decimal arithmetic on binary coded decimal (BCD) data.

DAA	Decimal adjust after addition.
DAS	Decimal adjust after subtraction.
AAA	ASCII adjust after addition.
AAS	ASCII adjust after subtraction.
AAM	ASCII adjust after multiplication.
AAD	ASCII adjust before division.

5.1.4 Logical Instructions

The logical instructions perform basic AND, OR, XOR, and NOT logical operations on byte, word, and doubleword values.

AND	Perform bitwise logical AND.
OR	Perform bitwise logical OR.
XOR	Perform bitwise logical exclusive OR.
NOT	Perform bitwise logical NOT.

5.1.5 Shift and Rotate Instructions

The shift and rotate instructions shift and rotate the bits in word and doubleword operands.

SAR	Shift arithmetic right.
SHR	Shift logical right.
SAL/SHL	Shift arithmetic left/Shift logical left.
SHRD	Shift right double.
SHLD	Shift left double.
ROR	Rotate right.
ROL	Rotate left.
RCR	Rotate through carry right.
RCL	Rotate through carry left.

5.1.6 Bit and Byte Instructions

Bit instructions test and modify individual bits in word and doubleword operands. Byte instructions set the value of a byte operand to indicate the status of flags in the EFLAGS register.

BT	Bit test.
BTS	Bit test and set.

BTR	Bit test and reset.
BTC	Bit test and complement.
BSF	Bit scan forward.
BSR	Bit scan reverse.
SETE/SETZ	Set byte if equal/Set byte if zero.
SETNE/SETNZ	Set byte if not equal/Set byte if not zero.
SETA/SETNBE	Set byte if above/Set byte if not below or equal.
SETAE/SETNB/SETNC	Set byte if above or equal/Set byte if not below/Set byte if not carry.
SETB/SETNAE/SETC	Set byte if below/Set byte if not above or equal/Set byte if carry.
SETBE/SETNA	Set byte if below or equal/Set byte if not above.
SETG/SETNLE	Set byte if greater/Set byte if not less or equal.
SETGE/SETNL	Set byte if greater or equal/Set byte if not less.
SETL/SETNGE	Set byte if less/Set byte if not greater or equal.
SETLE/SETNG	Set byte if less or equal/Set byte if not greater.
SETS	Set byte if sign (negative).
SETNS	Set byte if not sign (non-negative).
SETO	Set byte if overflow.
SETNO	Set byte if not overflow.
SETPE/SETP	Set byte if parity even/Set byte if parity.
SETPO/SETNP	Set byte if parity odd/Set byte if not parity.
TEST	Logical compare.
CRC32[1]	Provides hardware acceleration to calculate cyclic redundancy checks for fast and efficient implementation of data integrity protocols.
POPCNT[2]	Calculates of number of bits set to 1 in the second operand (source) and returns the count in the first operand (a destination register).

5.1.7 Control Transfer Instructions

The control transfer instructions provide jump, conditional jump, loop, and call and return operations to control program flow.

JMP	Jump.
JE/JZ	Jump if equal/Jump if zero.
JNE/JNZ	Jump if not equal/Jump if not zero.
JA/JNBE	Jump if above/Jump if not below or equal.
JAE/JNB	Jump if above or equal/Jump if not below.
JB/JNAE	Jump if below/Jump if not above or equal.
JBE/JNA	Jump if below or equal/Jump if not above.
JG/JNLE	Jump if greater/Jump if not less or equal.
JGE/JNL	Jump if greater or equal/Jump if not less.
JL/JNGE	Jump if less/Jump if not greater or equal.
JLE/JNG	Jump if less or equal/Jump if not greater.
JC	Jump if carry.
JNC	Jump if not carry.
JO	Jump if overflow.

1. Processor support of CRC32 is enumerated by CPUID.01H:ECX.SSE4_2 = 1
2. Processor support of POPCNT is enumerated by CPUID.01H:ECX.POPCNT = 1

JNO	Jump if not overflow.
JS	Jump if sign (negative).
JNS	Jump if not sign (non-negative).
JPO/JNP	Jump if parity odd/Jump if not parity.
JPE/JP	Jump if parity even/Jump if parity.
JCXZ/JECXZ	Jump register CX zero/Jump register ECX zero.
LOOP	Loop with ECX counter.
LOOPZ/LOOPE	Loop with ECX and zero/Loop with ECX and equal.
LOOPNZ/LOOPNE	Loop with ECX and not zero/Loop with ECX and not equal.
CALL	Call procedure.
RET	Return.
IRET	Return from interrupt.
INT	Software interrupt.
INTO	Interrupt on overflow.
BOUND	Detect value out of range.
ENTER	High-level procedure entry.
LEAVE	High-level procedure exit.

5.1.8 String Instructions

The string instructions operate on strings of bytes, allowing them to be moved to and from memory.

MOVS/MOVSB	Move string/Move byte string.
MOVS/MOVSW	Move string/Move word string.
MOVS/MOVSD	Move string/Move doubleword string.
CMPS/CMPSB	Compare string/Compare byte string.
CMPS/CMPSW	Compare string/Compare word string.
CMPS/CMPSD	Compare string/Compare doubleword string.
SCAS/SCASB	Scan string/Scan byte string.
SCAS/SCASW	Scan string/Scan word string.
SCAS/SCASD	Scan string/Scan doubleword string.
LODS/LODSB	Load string/Load byte string.
LODS/LODSW	Load string/Load word string.
LODS/LODSD	Load string/Load doubleword string.
STOS/STOSB	Store string/Store byte string.
STOS/STOSW	Store string/Store word string.
STOS/STOSD	Store string/Store doubleword string.
REP	Repeat while ECX not zero.
REPE/REPZ	Repeat while equal/Repeat while zero.
REPNE/REPNZ	Repeat while not equal/Repeat while not zero.

5.1.9 I/O Instructions

These instructions move data between the processor's I/O ports and a register or memory.

IN	Read from a port.
OUT	Write to a port.
INS/INSB	Input string from port/Input byte string from port.
INS/INSW	Input string from port/Input word string from port.

INS/INSD	Input string from port/Input doubleword string from port.
OUTS/OUTSB	Output string to port/Output byte string to port.
OUTS/OUTSW	Output string to port/Output word string to port.
OUTS/OUTSD	Output string to port/Output doubleword string to port.

5.1.10 Enter and Leave Instructions

These instructions provide machine-language support for procedure calls in block-structured languages.

ENTER	High-level procedure entry.
LEAVE	High-level procedure exit.

5.1.11 Flag Control (EFLAG) Instructions

The flag control instructions operate on the flags in the EFLAGS register.

STC	Set carry flag.
CLC	Clear the carry flag.
CMC	Complement the carry flag.
CLD	Clear the direction flag.
STD	Set direction flag.
LAHF	Load flags into AH register.
SAHF	Store AH register into flags.
PUSHF/PUSHFD	Push EFLAGS onto stack.
POPF/POPFD	Pop EFLAGS from stack.
STI	Set interrupt flag.
CLI	Clear the interrupt flag.

5.1.12 Segment Register Instructions

The segment register instructions allow far pointers (segment addresses) to be loaded into the segment registers.

LDS	Load far pointer using DS.
LES	Load far pointer using ES.
LFS	Load far pointer using FS.
LGS	Load far pointer using GS.
LSS	Load far pointer using SS.

5.1.13 Miscellaneous Instructions

The miscellaneous instructions provide such functions as loading an effective address, executing a "no-operation," and retrieving processor identification information.

LEA	Load effective address.
NOP	No operation.
UD	Undefined instruction.
XLAT/XLATB	Table lookup translation.
CPUID	Processor identification.
MOVBE[1]	Move data after swapping data bytes.

1. Processor support of MOVBE is enumerated by CPUID.01H:ECX.MOVBE[22] = 1.

PREFETCHW	Prefetch data into cache in anticipation of write.
PREFETCHWT1	Prefetch hint T1 with intent to write.
CLFLUSH	Flushes and invalidates a memory operand and its associated cache line from all levels of the processor's cache hierarchy.
CLFLUSHOPT	Flushes and invalidates a memory operand and its associated cache line from all levels of the processor's cache hierarchy with optimized memory system throughput.

5.1.14 User Mode Extended State Save/Restore Instructions

XSAVE	Save processor extended states to memory.
XSAVEC	Save processor extended states with compaction to memory.
XSAVEOPT	Save processor extended states to memory, optimized.
XRSTOR	Restore processor extended states from memory.
XGETBV	Reads the state of an extended control register.

5.1.15 Random Number Generator Instructions

RDRAND	Retrieves a random number generated from hardware.
RDSEED	Retrieves a random number generated from hardware.

5.1.16 BMI1 and BMI2 Instructions

ANDN	Bitwise AND of first source with inverted second source operands.
BEXTR	Contiguous bitwise extract.
BLSI	Extract lowest set bit.
BLSMSK	Set all lower bits below first set bit to 1.
BLSR	Reset lowest set bit.
BZHI	Zero high bits starting from specified bit position.
LZCNT	Count the number of leading zero bits.
MULX	Unsigned multiply without affecting arithmetic flags.
PDEP	Parallel deposit of bits using a mask.
PEXT	Parallel extraction of bits using a mask.
RORX	Rotate right without affecting arithmetic flags.
SARX	Shift arithmetic right.
SHLX	Shift logic left.
SHRX	Shift logic right.
TZCNT	Count the number of trailing zero bits.

5.1.16.1 Detection of VEX-Encoded GPR Instructions, LZCNT, TZCNT, and PREFETCHW

VEX-encoded general-purpose instructions do not operate on any vector registers.

There are separate feature flags for the following subsets of instructions that operate on general purpose registers, and the detection requirements for hardware support are:

CPUID.07H.00H:EBX.BMI1[3]: if 1 indicates the processor supports the first group of advanced bit manipulation extensions (ANDN, BEXTR, BLSI, BLSMSK, BLSR, TZCNT);

CPUID.07H.00H:EBX.BMI2[8]: if 1 indicates the processor supports the second group of advanced bit manipulation extensions (BZHI, MULX, PDEP, PEXT, RORX, SARX, SHLX, SHRX);

CPUID.80000001H:ECX.LZCNT[5]: if 1 indicates the processor supports the LZCNT instruction.

CPUID.80000001H:ECX.PREFETCHW[8]: if 1 indicates the processor supports the PREFETCHW instruction.
CPUID.07H.00H:ECX.PREFETCHWT1[0]: if 1 indicates the processor supports the PREFETCHWT1 instruction.

5.2 X87 FPU INSTRUCTIONS

The x87 FPU instructions are executed by the processor's x87 FPU. These instructions operate on floating-point, integer, and binary-coded decimal (BCD) operands. For more detail on x87 FPU instructions, see Chapter 8, "Programming with the x87 FPU."

These instructions are divided into the following subgroups: data transfer, load constants, and FPU control instructions. The sections that follow introduce each subgroup.

5.2.1 X87 FPU Data Transfer Instructions

The data transfer instructions move floating-point, integer, and BCD values between memory and the x87 FPU registers. They also perform conditional move operations on floating-point operands.

FLD	Load floating-point value.
FST	Store floating-point value.
FSTP	Store floating-point value and pop.
FILD	Load integer.
FIST	Store integer.
FISTP[1]	Store integer and pop.
FBLD	Load BCD.
FBSTP	Store BCD and pop.
FXCH	Exchange registers.
FCMOVE	Floating-point conditional move if equal.
FCMOVNE	Floating-point conditional move if not equal.
FCMOVB	Floating-point conditional move if below.
FCMOVBE	Floating-point conditional move if below or equal.
FCMOVNB	Floating-point conditional move if not below.
FCMOVNBE	Floating-point conditional move if not below or equal.
FCMOVU	Floating-point conditional move if unordered.
FCMOVNU	Floating-point conditional move if not unordered.

5.2.2 X87 FPU Basic Arithmetic Instructions

The basic arithmetic instructions perform basic arithmetic operations on floating-point and integer operands.

FADD	Add floating-point.
FADDP	Add floating-point and pop.
FIADD	Add integer.
FSUB	Subtract floating-point.
FSUBP	Subtract floating-point and pop.
FISUB	Subtract integer.
FSUBR	Subtract floating-point reverse.
FSUBRP	Subtract floating-point reverse and pop.
FISUBR	Subtract integer reverse.

1. SSE3 provides an instruction FISTTP for integer conversion.

FMUL	Multiply floating-point.
FMULP	Multiply floating-point and pop.
FIMUL	Multiply integer.
FDIV	Divide floating-point.
FDIVP	Divide floating-point and pop.
FIDIV	Divide integer.
FDIVR	Divide floating-point reverse.
FDIVRP	Divide floating-point reverse and pop.
FIDIVR	Divide integer reverse.
FPREM	Partial remainder.
FPREM1	IEEE partial remainder.
FABS	Absolute value.
FCHS	Change sign.
FRNDINT	Round to integer.
FSCALE	Scale by power of two.
FSQRT	Square root.
FXTRACT	Extract exponent and significand.

5.2.3 X87 FPU Comparison Instructions

The compare instructions examine or compare floating-point or integer operands.

FCOM	Compare floating-point.
FCOMP	Compare floating-point and pop.
FCOMPP	Compare floating-point and pop twice.
FUCOM	Unordered compare floating-point.
FUCOMP	Unordered compare floating-point and pop.
FUCOMPP	Unordered compare floating-point and pop twice.
FICOM	Compare integer.
FICOMP	Compare integer and pop.
FCOMI	Compare floating-point and set EFLAGS.
FUCOMI	Unordered compare floating-point and set EFLAGS.
FCOMIP	Compare floating-point, set EFLAGS, and pop.
FUCOMIP	Unordered compare floating-point, set EFLAGS, and pop.
FTST	Test floating-point (compare with 0.0).
FXAM	Examine floating-point.

5.2.4 X87 FPU Transcendental Instructions

The transcendental instructions perform basic trigonometric and logarithmic operations on floating-point operands.

FSIN	Sine.
FCOS	Cosine.
FSINCOS	Sine and cosine.
FPTAN	Partial tangent.
FPATAN	Partial arctangent.
F2XM1	$2^x - 1$.
FYL2X	$y * \log_2 x$.
FYL2XP1	$y * \log_2 (x+1)$.

5.2.5 X87 FPU Load Constants Instructions

The load constants instructions load common constants, such as π, into the x87 floating-point registers.

FLD1	Load +1.0.
FLDZ	Load +0.0.
FLDPI	Load π.
FLDL2E	Load $\log_2 e$.
FLDLN2	Load $\log_e 2$.
FLDL2T	Load $\log_2 10$.
FLDLG2	Load $\log_{10} 2$.

5.2.6 X87 FPU Control Instructions

The x87 FPU control instructions operate on the x87 FPU register stack and save and restore the x87 FPU state.

FINCSTP	Increment FPU register stack pointer.
FDECSTP	Decrement FPU register stack pointer.
FFREE	Free floating-point register.
FINIT	Initialize FPU after checking error conditions.
FNINIT	Initialize FPU without checking error conditions.
FCLEX	Clear floating-point exception flags after checking for error conditions.
FNCLEX	Clear floating-point exception flags without checking for error conditions.
FSTCW	Store FPU control word after checking error conditions.
FNSTCW	Store FPU control word without checking error conditions.
FLDCW	Load FPU control word.
FSTENV	Store FPU environment after checking error conditions.
FNSTENV	Store FPU environment without checking error conditions.
FLDENV	Load FPU environment.
FSAVE	Save FPU state after checking error conditions.
FNSAVE	Save FPU state without checking error conditions.
FRSTOR	Restore FPU state.
FSTSW	Store FPU status word after checking error conditions.
FNSTSW	Store FPU status word without checking error conditions.
WAIT/FWAIT	Wait for FPU.
FNOP	FPU no operation.

5.3 X87 FPU AND SIMD STATE MANAGEMENT INSTRUCTIONS

Two state management instructions were introduced into the IA-32 architecture with the Pentium II processor family:

FXSAVE	Save x87 FPU and SIMD state.
FXRSTOR	Restore x87 FPU and SIMD state.

Initially, these instructions operated only on the x87 FPU (and MMX) registers to perform a fast save and restore, respectively, of the x87 FPU and MMX state. With the introduction of SSE extensions in the Pentium III processor family, these instructions were expanded to also save and restore the state of the XMM and MXCSR registers. Intel 64 architecture also supports these instructions.

See Section 10.5, “FXSAVE and FXRSTOR Instructions,” for more detail.

5.4 MMX INSTRUCTIONS

Four extensions have been introduced into the IA-32 architecture to permit IA-32 processors to perform single-instruction multiple-data (SIMD) operations. These extensions include the MMX technology, SSE extensions, SSE2 extensions, and SSE3 extensions. For a discussion that puts SIMD instructions in their historical context, see Section 2.2.7, "SIMD Instructions."

MMX instructions operate on packed byte, word, doubleword, or quadword integer operands contained in memory, in MMX registers, and/or in general-purpose registers. For more detail on these instructions, see Chapter 9, "Programming with Intel® MMX™ Technology."

MMX instructions can only be executed on Intel 64 and IA-32 processors that support the MMX technology. Support for these instructions can be detected with the CPUID instruction. See the description of the CPUID instruction in Chapter 3, "Instruction Set Reference, A-L," of the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 2A.

MMX instructions are divided into the following subgroups: data transfer, conversion, packed arithmetic, comparison, logical, shift and rotate, and state management instructions. The sections that follow introduce each subgroup.

5.4.1 MMX Data Transfer Instructions

The data transfer instructions move doubleword and quadword operands between MMX registers and between MMX registers and memory.

MOVD	Move doubleword.
MOVQ	Move quadword.

5.4.2 MMX Conversion Instructions

The conversion instructions pack and unpack bytes, words, and doublewords

PACKSSWB	Pack words into bytes with signed saturation.
PACKSSDW	Pack doublewords into words with signed saturation.
PACKUSWB	Pack words into bytes with unsigned saturation.
PUNPCKHBW	Unpack high-order bytes.
PUNPCKHWD	Unpack high-order words.
PUNPCKHDQ	Unpack high-order doublewords.
PUNPCKLBW	Unpack low-order bytes.
PUNPCKLWD	Unpack low-order words.
PUNPCKLDQ	Unpack low-order doublewords.

5.4.3 MMX Packed Arithmetic Instructions

The packed arithmetic instructions perform packed integer arithmetic on packed byte, word, and doubleword integers.

PADDB	Add packed byte integers.
PADDW	Add packed word integers.
PADDD	Add packed doubleword integers.
PADDSB	Add packed signed byte integers with signed saturation.
PADDSW	Add packed signed word integers with signed saturation.
PADDUSB	Add packed unsigned byte integers with unsigned saturation.
PADDUSW	Add packed unsigned word integers with unsigned saturation.
PSUBB	Subtract packed byte integers.
PSUBW	Subtract packed word integers.

PSUBD	Subtract packed doubleword integers.
PSUBSB	Subtract packed signed byte integers with signed saturation.
PSUBSW	Subtract packed signed word integers with signed saturation.
PSUBUSB	Subtract packed unsigned byte integers with unsigned saturation.
PSUBUSW	Subtract packed unsigned word integers with unsigned saturation.
PMULHW	Multiply packed signed word integers and store high result.
PMULLW	Multiply packed signed word integers and store low result.
PMADDWD	Multiply and add packed word integers.

5.4.4 MMX Comparison Instructions

The compare instructions compare packed bytes, words, or doublewords.

PCMPEQB	Compare packed bytes for equal.
PCMPEQW	Compare packed words for equal.
PCMPEQD	Compare packed doublewords for equal.
PCMPGTB	Compare packed signed byte integers for greater than.
PCMPGTW	Compare packed signed word integers for greater than.
PCMPGTD	Compare packed signed doubleword integers for greater than.

5.4.5 MMX Logical Instructions

The logical instructions perform AND, AND NOT, OR, and XOR operations on quadword operands.

PAND	Bitwise logical AND.
PANDN	Bitwise logical AND NOT.
POR	Bitwise logical OR.
PXOR	Bitwise logical exclusive OR.

5.4.6 MMX Shift and Rotate Instructions

The shift and rotate instructions shift and rotate packed bytes, words, or doublewords, or quadwords in 64-bit operands.

PSLLW	Shift packed words left logical.
PSLLD	Shift packed doublewords left logical.
PSLLQ	Shift packed quadword left logical.
PSRLW	Shift packed words right logical.
PSRLD	Shift packed doublewords right logical.
PSRLQ	Shift packed quadword right logical.
PSRAW	Shift packed words right arithmetic.
PSRAD	Shift packed doublewords right arithmetic.

5.4.7 MMX State Management Instructions

The EMMS instruction clears the MMX state from the MMX registers.

EMMS	Empty MMX state.

5.5 INTEL® SSE INSTRUCTIONS

Intel SSE instructions represent an extension of the SIMD execution model introduced with the MMX technology. For more detail on these instructions, see Chapter 10, "Programming with Intel® Streaming SIMD Extensions (Intel® SSE)."

Intel SSE instructions can only be executed on Intel 64 and IA-32 processors that support Intel SSE extensions. Support for these instructions can be detected with the CPUID instruction. See the description of the CPUID instruction in Chapter 3, "Instruction Set Reference, A-L," of the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 2A.

Intel SSE instructions are divided into four subgroups (note that the first subgroup has subordinate subgroups of its own):

- SIMD single precision floating-point instructions that operate on the XMM registers.
- MXCSR state management instructions.
- 64-bit SIMD integer instructions that operate on the MMX registers.
- Cacheability control, prefetch, and instruction ordering instructions.

The following sections provide an overview of these groups.

5.5.1 Intel® SSE SIMD Single Precision Floating-Point Instructions

These instructions operate on packed and scalar single precision floating-point values located in XMM registers and/or memory. This subgroup is further divided into the following subordinate subgroups: data transfer, packed arithmetic, comparison, logical, shuffle and unpack, and conversion instructions.

5.5.1.1 Intel® SSE Data Transfer Instructions

Intel SSE data transfer instructions move packed and scalar single precision floating-point operands between XMM registers and between XMM registers and memory.

MOVAPS	Move four aligned packed single precision floating-point values between XMM registers or between an XMM register and memory.
MOVUPS	Move four unaligned packed single precision floating-point values between XMM registers or between an XMM register and memory.
MOVHPS	Move two packed single precision floating-point values to and from the high quadword of an XMM register and memory.
MOVHLPS	Move two packed single precision floating-point values from the high quadword of an XMM register to the low quadword of another XMM register.
MOVLPS	Move two packed single precision floating-point values to and from the low quadword of an XMM register and memory.
MOVLHPS	Move two packed single precision floating-point values from the low quadword of an XMM register to the high quadword of another XMM register.
MOVMSKPS	Extract sign mask from four packed single precision floating-point values.
MOVSS	Move scalar single precision floating-point value between XMM registers or between an XMM register and memory.

5.5.1.2 Intel® SSE Packed Arithmetic Instructions

Intel SSE packed arithmetic instructions perform packed and scalar arithmetic operations on packed and scalar single precision floating-point operands.

ADDPS	Add packed single precision floating-point values.
ADDSS	Add scalar single precision floating-point values.
SUBPS	Subtract packed single precision floating-point values.
SUBSS	Subtract scalar single precision floating-point values.

MULPS	Multiply packed single precision floating-point values.
MULSS	Multiply scalar single precision floating-point values.
DIVPS	Divide packed single precision floating-point values.
DIVSS	Divide scalar single precision floating-point values.
RCPPS	Compute reciprocals of packed single precision floating-point values.
RCPSS	Compute reciprocal of scalar single precision floating-point values.
SQRTPS	Compute square roots of packed single precision floating-point values.
SQRTSS	Compute square root of scalar single precision floating-point values.
RSQRTPS	Compute reciprocals of square roots of packed single precision floating-point values.
RSQRTSS	Compute reciprocal of square root of scalar single precision floating-point values.
MAXPS	Return maximum packed single precision floating-point values.
MAXSS	Return maximum scalar single precision floating-point values.
MINPS	Return minimum packed single precision floating-point values.
MINSS	Return minimum scalar single precision floating-point values.

5.5.1.3 Intel® SSE Comparison Instructions

Intel SSE compare instructions compare packed and scalar single precision floating-point operands.

CMPPS	Compare packed single precision floating-point values.
CMPSS	Compare scalar single precision floating-point values.
COMISS	Perform ordered comparison of scalar single precision floating-point values and set flags in EFLAGS register.
UCOMISS	Perform unordered comparison of scalar single precision floating-point values and set flags in EFLAGS register.

5.5.1.4 Intel® SSE Logical Instructions

Intel SSE logical instructions perform bitwise AND, AND NOT, OR, and XOR operations on packed single precision floating-point operands.

ANDPS	Perform bitwise logical AND of packed single precision floating-point values.
ANDNPS	Perform bitwise logical AND NOT of packed single precision floating-point values.
ORPS	Perform bitwise logical OR of packed single precision floating-point values.
XORPS	Perform bitwise logical XOR of packed single precision floating-point values.

5.5.1.5 Intel® SSE Shuffle and Unpack Instructions

Intel SSE shuffle and unpack instructions shuffle or interleave single precision floating-point values in packed single precision floating-point operands.

SHUFPS	Shuffles values in packed single precision floating-point operands.
UNPCKHPS	Unpacks and interleaves the two high-order values from two single precision floating-point operands.
UNPCKLPS	Unpacks and interleaves the two low-order values from two single precision floating-point operands.

5.5.1.6 Intel® SSE Conversion Instructions

Intel SSE conversion instructions convert packed and individual doubleword integers into packed and scalar single precision floating-point values and vice versa.

CVTPI2PS	Convert packed doubleword integers to packed single precision floating-point values.
CVTSI2SS	Convert signed integer to scalar single precision floating-point value.

CVTPS2PI	Convert packed single precision floating-point values to packed doubleword integers.
CVTTPS2PI	Convert with truncation packed single precision floating-point values to packed double-word integers.
CVTSS2SI	Convert a scalar single precision floating-point value to a signed integer.
CVTTSS2SI	Convert with truncation a scalar single precision floating-point value to a scalar signed integer.

5.5.2 Intel® SSE MXCSR State Management Instructions

MXCSR state management instructions allow saving and restoring the state of the MXCSR control and status register.

LDMXCSR	Load MXCSR register.
STMXCSR	Save MXCSR register state.

5.5.3 Intel® SSE 64-Bit SIMD Integer Instructions

These Intel SSE 64-bit SIMD integer instructions perform additional operations on packed bytes, words, or double-words contained in MMX registers. They represent enhancements to the MMX instruction set described in Section 5.4, “MMX Instructions.”

PAVGB	Compute average of packed unsigned byte integers.
PAVGW	Compute average of packed unsigned word integers.
PEXTRW	Extract word.
PINSRW	Insert word.
PMAXUB	Maximum of packed unsigned byte integers.
PMAXSW	Maximum of packed signed word integers.
PMINUB	Minimum of packed unsigned byte integers.
PMINSW	Minimum of packed signed word integers.
PMOVMSKB	Move byte mask.
PMULHUW	Multiply packed unsigned integers and store high result.
PSADBW	Compute sum of absolute differences.
PSHUFW	Shuffle packed integer word in MMX register.

5.5.4 Intel® SSE Cacheability Control, Prefetch, and Instruction Ordering Instructions

The cacheability control instructions provide control over the caching of non-temporal data when storing data from the MMX and XMM registers to memory. The PREFETCH*h* allows data to be prefetched to a selected cache level. The SFENCE instruction controls instruction ordering on store operations.

MASKMOVQ	Non-temporal store of selected bytes from an MMX register into memory.
MOVNTQ	Non-temporal store of quadword from an MMX register into memory.
MOVNTPS	Non-temporal store of four packed single precision floating-point values from an XMM register into memory.
PREFETCH*h*	Load 32 or more of bytes from memory to a selected level of the processor’s cache hierarchy.
SFENCE	Serializes store operations.

5.6 INTEL® SSE2 INSTRUCTIONS

Intel SSE2 extensions represent an extension of the SIMD execution model introduced with MMX technology and the Intel SSE extensions. Intel SSE2 instructions operate on packed double precision floating-point operands and

on packed byte, word, doubleword, and quadword operands located in the XMM registers. For more detail on these instructions, see Chapter 11, "Programming with Intel® Streaming SIMD Extensions 2 (Intel® SSE2)."

Intel SSE2 instructions can only be executed on Intel 64 and IA-32 processors that support the Intel SSE2 extensions. Support for these instructions can be detected with the CPUID instruction. See the description of the CPUID instruction in Chapter 3, "Instruction Set Reference, A-L," of the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 2A.

These instructions are divided into four subgroups (note that the first subgroup is further divided into subordinate subgroups):

- Packed and scalar double precision floating-point instructions.
- Packed single precision floating-point conversion instructions.
- 128-bit SIMD integer instructions.
- Cacheability-control and instruction ordering instructions.

The following sections give an overview of each subgroup.

5.6.1 Intel® SSE2 Packed and Scalar Double Precision Floating-Point Instructions

Intel SSE2 packed and scalar double precision floating-point instructions are divided into the following subordinate subgroups: data movement, arithmetic, comparison, conversion, logical, and shuffle operations on double precision floating-point operands. These are introduced in the sections that follow.

5.6.1.1 Intel® SSE2 Data Movement Instructions

Intel SSE2 data movement instructions move double precision floating-point data between XMM registers and between XMM registers and memory.

MOVAPD	Move two aligned packed double precision floating-point values between XMM registers or between an XMM register and memory.
MOVUPD	Move two unaligned packed double precision floating-point values between XMM registers or between an XMM register and memory.
MOVHPD	Move high packed double precision floating-point value to and from the high quadword of an XMM register and memory.
MOVLPD	Move low packed single precision floating-point value to and from the low quadword of an XMM register and memory.
MOVMSKPD	Extract sign mask from two packed double precision floating-point values.
MOVSD	Move scalar double precision floating-point value between XMM registers or between an XMM register and memory.

5.6.1.2 Intel® SSE2 Packed Arithmetic Instructions

The arithmetic instructions perform addition, subtraction, multiply, divide, square root, and maximum/minimum operations on packed and scalar double precision floating-point operands.

ADDPD	Add packed double precision floating-point values.
ADDSD	Add scalar double precision floating-point values.
SUBPD	Subtract packed double precision floating-point values.
SUBSD	Subtract scalar double precision floating-point values.
MULPD	Multiply packed double precision floating-point values.
MULSD	Multiply scalar double precision floating-point values.
DIVPD	Divide packed double precision floating-point values.
DIVSD	Divide scalar double precision floating-point values.
SQRTPD	Compute packed square roots of packed double precision floating-point values.
SQRTSD	Compute scalar square root of scalar double precision floating-point values.

MAXPD	Return maximum packed double precision floating-point values.
MAXSD	Return maximum scalar double precision floating-point values.
MINPD	Return minimum packed double precision floating-point values.
MINSD	Return minimum scalar double precision floating-point values.

5.6.1.3 Intel® SSE2 Logical Instructions

Intel SSE2 logical instructions perform AND, AND NOT, OR, and XOR operations on packed double precision floating-point values.

ANDPD	Perform bitwise logical AND of packed double precision floating-point values.
ANDNPD	Perform bitwise logical AND NOT of packed double precision floating-point values.
ORPD	Perform bitwise logical OR of packed double precision floating-point values.
XORPD	Perform bitwise logical XOR of packed double precision floating-point values.

5.6.1.4 Intel® SSE2 Compare Instructions

Intel SSE2 compare instructions compare packed and scalar double precision floating-point values and return the results of the comparison either to the destination operand or to the EFLAGS register.

CMPPD	Compare packed double precision floating-point values.
CMPSD	Compare scalar double precision floating-point values.
COMISD	Perform ordered comparison of scalar double precision floating-point values and set flags in EFLAGS register.
UCOMISD	Perform unordered comparison of scalar double precision floating-point values and set flags in EFLAGS register.

5.6.1.5 Intel® SSE2 Shuffle and Unpack Instructions

Intel SSE2 shuffle and unpack instructions shuffle or interleave double precision floating-point values in packed double precision floating-point operands.

SHUFPD	Shuffles values in packed double precision floating-point operands.
UNPCKHPD	Unpacks and interleaves the high values from two packed double precision floating-point operands.
UNPCKLPD	Unpacks and interleaves the low values from two packed double precision floating-point operands.

5.6.1.6 Intel® SSE2 Conversion Instructions

Intel SSE2 conversion instructions convert packed and individual doubleword integers into packed and scalar double precision floating-point values and vice versa. They also convert between packed and scalar single precision and double precision floating-point values.

CVTPD2PI	Convert packed double precision floating-point values to packed doubleword integers.
CVTTPD2PI	Convert with truncation packed double precision floating-point values to packed doubleword integers.
CVTPI2PD	Convert packed doubleword integers to packed double precision floating-point values.
CVTPD2DQ	Convert packed double precision floating-point values to packed doubleword integers.
CVTTPD2DQ	Convert with truncation packed double precision floating-point values to packed doubleword integers.
CVTDQ2PD	Convert packed doubleword integers to packed double precision floating-point values.
CVTPS2PD	Convert packed single precision floating-point values to packed double precision floating-point values.

CVTPD2PS	Convert packed double precision floating-point values to packed single precision floating-point values.
CVTSS2SD	Convert scalar single precision floating-point values to scalar double precision floating-point values.
CVTSD2SS	Convert scalar double precision floating-point values to scalar single precision floating-point values.
CVTSD2SI	Convert scalar double precision floating-point values to a signed integer.
CVTTSD2SI	Convert with truncation scalar double precision floating-point values to a scalar signed integer.
CVTSI2SD	Convert signed integer to scalar double precision floating-point value.

5.6.2 Intel® SSE2 Packed Single Precision Floating-Point Instructions

Intel SSE2 packed single precision floating-point instructions perform conversion operations on single precision floating-point and integer operands. These instructions represent enhancements to the Intel SSE single precision floating-point instructions.

CVTDQ2PS	Convert packed doubleword integers to packed single precision floating-point values.
CVTPS2DQ	Convert packed single precision floating-point values to packed doubleword integers.
CVTTPS2DQ	Convert with truncation packed single precision floating-point values to packed doubleword integers.

5.6.3 Intel® SSE2 128-Bit SIMD Integer Instructions

Intel SSE2 SIMD integer instructions perform additional operations on packed words, doublewords, and quadwords contained in XMM and MMX registers.

MOVDQA	Move aligned double quadword.
MOVDQU	Move unaligned double quadword.
MOVQ2DQ	Move quadword integer from MMX to XMM registers.
MOVDQ2Q	Move quadword integer from XMM to MMX registers.
PMULUDQ	Multiply packed unsigned doubleword integers.
PADDQ	Add packed quadword integers.
PSUBQ	Subtract packed quadword integers.
PSHUFLW	Shuffle packed low words.
PSHUFHW	Shuffle packed high words.
PSHUFD	Shuffle packed doublewords.
PSLLDQ	Shift double quadword left logical.
PSRLDQ	Shift double quadword right logical.
PUNPCKHQDQ	Unpack high quadwords.
PUNPCKLQDQ	Unpack low quadwords.

5.6.4 Intel® SSE2 Cacheability Control and Ordering Instructions

Intel SSE2 cacheability control instructions provide additional operations for caching of non-temporal data when storing data from XMM registers to memory. LFENCE and MFENCE provide additional control of instruction ordering on store operations.

CLFLUSH	See Section 5.1.13.
LFENCE	Serializes load operations.
MFENCE	Serializes load and store operations.
PAUSE	Improves the performance of “spin-wait loops”.

MASKMOVDQU	Non-temporal store of selected bytes from an XMM register into memory.
MOVNTPD	Non-temporal store of two packed double precision floating-point values from an XMM register into memory.
MOVNTDQ	Non-temporal store of double quadword from an XMM register into memory.
MOVNTI	Non-temporal store of a doubleword from a general-purpose register into memory.

5.7 INTEL® SSE3 INSTRUCTIONS

The Intel SSE3 extensions offers 13 instructions that accelerate performance of Streaming SIMD Extensions technology, Streaming SIMD Extensions 2 technology, and x87-FP math capabilities. These instructions can be grouped into the following categories:

- One x87 FPU instruction used in integer conversion.
- One SIMD integer instruction that addresses unaligned data loads.
- Two SIMD floating-point packed ADD/SUB instructions.
- Four SIMD floating-point horizontal ADD/SUB instructions.
- Three SIMD floating-point LOAD/MOVE/DUPLICATE instructions.
- Two thread synchronization instructions.

Intel SSE3 instructions can only be executed on Intel 64 and IA-32 processors that support Intel SSE3 extensions. Support for these instructions can be detected with the CPUID instruction. See the description of the CPUID instruction in Chapter 3, "Instruction Set Reference, A-L," of the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 2A.

The sections that follow describe each subgroup.

5.7.1 Intel® SSE3 x87-FP Integer Conversion Instruction

FISTTP	Behaves like the FISTP instruction but uses truncation, irrespective of the rounding mode specified in the floating-point control word (FCW).

5.7.2 Intel® SSE3 Specialized 128-Bit Unaligned Data Load Instruction

LDDQU	Special 128-bit unaligned load designed to avoid cache line splits.

5.7.3 Intel® SSE3 SIMD Floating-Point Packed ADD/SUB Instructions

ADDSUBPS	Performs single precision addition on the second and fourth pairs of 32-bit data elements within the operands; single precision subtraction on the first and third pairs.
ADDSUBPD	Performs double precision addition on the second pair of quadwords, and double precision subtraction on the first pair.

5.7.4 Intel® SSE3 SIMD Floating-Point Horizontal ADD/SUB Instructions

HADDPS	Performs a single precision addition on contiguous data elements. The first data element of the result is obtained by adding the first and second elements of the first operand; the second element by adding the third and fourth elements of the first operand; the third by adding the first and second elements of the second operand; and the fourth by adding the third and fourth elements of the second operand.
HSUBPS	Performs a single precision subtraction on contiguous data elements. The first data element of the result is obtained by subtracting the second element of the first operand from the first element of the first operand; the second element by subtracting the fourth element of the first operand from the third element of the first operand; the third by

	subtracting the second element of the second operand from the first element of the second operand; and the fourth by subtracting the fourth element of the second operand from the third element of the second operand.
HADDPD	Performs a double precision addition on contiguous data elements. The first data element of the result is obtained by adding the first and second elements of the first operand; the second element by adding the first and second elements of the second operand.
HSUBPD	Performs a double precision subtraction on contiguous data elements. The first data element of the result is obtained by subtracting the second element of the first operand from the first element of the first operand; the second element by subtracting the second element of the second operand from the first element of the second operand.

5.7.5 Intel® SSE3 SIMD Floating-Point LOAD/MOVE/DUPLICATE Instructions

MOVSHDUP	Loads/moves 128 bits; duplicating the second and fourth 32-bit data elements.
MOVSLDUP	Loads/moves 128 bits; duplicating the first and third 32-bit data elements.
MOVDDUP	Loads/moves 64 bits (bits[63:0] if the source is a register) and returns the same 64 bits in both the lower and upper halves of the 128-bit result register; duplicates the 64 bits from the source.

5.7.6 Intel® SSE3 Agent Synchronization Instructions

MONITOR	Sets up an address range used to monitor write-back stores.
MWAIT	Enables a logical processor to enter into an optimized state while waiting for a write-back store to the address range set up by the MONITOR instruction.

5.8 SUPPLEMENTAL STREAMING SIMD EXTENSIONS 3 (SSSE3) INSTRUCTIONS

SSSE3 provide 32 instructions (represented by 14 mnemonics) to accelerate computations on packed integers. These include:

- Twelve instructions that perform horizontal addition or subtraction operations.
- Six instructions that evaluate absolute values.
- Two instructions that perform multiply and add operations and speed up the evaluation of dot products.
- Two instructions that accelerate packed-integer multiply operations and produce integer values with scaling.
- Two instructions that perform a byte-wise, in-place shuffle according to the second shuffle control operand.
- Six instructions that negate packed integers in the destination operand if the signs of the corresponding element in the source operand is less than zero.
- Two instructions that align data from the composite of two operands.

SSSE3 instructions can only be executed on Intel 64 and IA-32 processors that support SSSE3 extensions. Support for these instructions can be detected with the CPUID instruction. See the description of the CPUID instruction in Chapter 3, "Instruction Set Reference, A-L," of the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 2A.

The sections that follow describe each subgroup.

5.8.1 Horizontal Addition/Subtraction

PHADDW	Adds two adjacent, signed 16-bit integers horizontally from the source and destination operands and packs the signed 16-bit results to the destination operand.
PHADDSW	Adds two adjacent, signed 16-bit integers horizontally from the source and destination operands and packs the signed, saturated 16-bit results to the destination operand.

PHADDD	Adds two adjacent, signed 32-bit integers horizontally from the source and destination operands and packs the signed 32-bit results to the destination operand.
PHSUBW	Performs horizontal subtraction on each adjacent pair of 16-bit signed integers by subtracting the most significant word from the least significant word of each pair in the source and destination operands. The signed 16-bit results are packed and written to the destination operand.
PHSUBSW	Performs horizontal subtraction on each adjacent pair of 16-bit signed integers by subtracting the most significant word from the least significant word of each pair in the source and destination operands. The signed, saturated 16-bit results are packed and written to the destination operand.
PHSUBD	Performs horizontal subtraction on each adjacent pair of 32-bit signed integers by subtracting the most significant doubleword from the least significant double word of each pair in the source and destination operands. The signed 32-bit results are packed and written to the destination operand.

5.8.2 Packed Absolute Values

PABSB	Computes the absolute value of each signed byte data element.
PABSW	Computes the absolute value of each signed 16-bit data element.
PABSD	Computes the absolute value of each signed 32-bit data element.

5.8.3 Multiply and Add Packed Signed and Unsigned Bytes

PMADDUBSW	Multiplies each unsigned byte value with the corresponding signed byte value to produce an intermediate, 16-bit signed integer. Each adjacent pair of 16-bit signed values are added horizontally. The signed, saturated 16-bit results are packed to the destination operand.

5.8.4 Packed Multiply High with Round and Scale

PMULHRSW	Multiplies vertically each signed 16-bit integer from the destination operand with the corresponding signed 16-bit integer of the source operand, producing intermediate, signed 32-bit integers. Each intermediate 32-bit integer is truncated to the 18 most significant bits. Rounding is always performed by adding 1 to the least significant bit of the 18-bit intermediate result. The final result is obtained by selecting the 16 bits immediately to the right of the most significant bit of each 18-bit intermediate result and packed to the destination operand.

5.8.5 Packed Shuffle Bytes

PSHUFB	Permutes each byte in place, according to a shuffle control mask. The least significant three or four bits of each shuffle control byte of the control mask form the shuffle index. The shuffle mask is unaffected. If the most significant bit (bit 7) of a shuffle control byte is set, the constant zero is written in the result byte.

5.8.6 Packed Sign

PSIGNB/W/D	Negates each signed integer element of the destination operand if the sign of the corresponding data element in the source operand is less than zero.

5.8.7 Packed Align Right

PALIGNR	Source operand is appended after the destination operand forming an intermediate value of twice the width of an operand. The result is extracted from the intermediate value into the destination operand by selecting the 128-bit or 64-bit value that are right-aligned to the byte offset specified by the immediate value.

5.9 INTEL® SSE4 INSTRUCTIONS

Intel Streaming SIMD Extensions 4 (Intel SSE4) introduces 54 new instructions. 47 of the Intel SSE4 instructions are referred to as Intel SSE4.1 in this document, and 7 new Intel SSE4 instructions are referred to as Intel SSE4.2.

Intel SSE4.1 is targeted to improve the performance of media, imaging, and 3D workloads. Intel SSE4.1 adds instructions that improve compiler vectorization and significantly increase support for packed dword computation. The technology also provides a hint that can improve memory throughput when reading from uncacheable WC memory type.

The 47 Intel SSE4.1 instructions include:

- Two instructions perform packed dword multiplies.
- Two instructions perform floating-point dot products with input/output selects.
- One instruction performs a load with a streaming hint.
- Six instructions simplify packed blending.
- Eight instructions expand support for packed integer MIN/MAX.
- Four instructions support floating-point round with selectable rounding mode and precision exception override.
- Seven instructions improve data insertion and extractions from XMM registers
- Twelve instructions improve packed integer format conversions (sign and zero extensions).
- One instruction improves SAD (sum absolute difference) generation for small block sizes.
- One instruction aids horizontal searching operations.
- One instruction improves masked comparisons.
- One instruction adds qword packed equality comparisons.
- One instruction adds dword packing with unsigned saturation.

The Intel SSE4.2 instructions operating on XMM registers include:

- String and text processing that can take advantage of single-instruction multiple-data programming techniques.
- A SIMD integer instruction that enhances the capability of the 128-bit integer SIMD capability in SSE4.1.

5.10 INTEL® SSE4.1 INSTRUCTIONS

Intel SSE4.1 instructions can use an XMM register as a source or destination. Programming Intel SSE4.1 is similar to programming 128-bit Integer SIMD and floating-point SIMD instructions in Intel SSE/SSE2/SSE3/SSSE3. Intel SSE4.1 does not provide any 64-bit integer SIMD instructions operating on MMX registers. The sections that follow describe each subgroup.

5.10.1 Dword Multiply Instructions

PMULLD	Returns four lower 32-bits of the 64-bit results of signed 32-bit integer multiplies.
PMULDQ	Returns two 64-bit signed result of signed 32-bit integer multiplies.

5.10.2 Floating-Point Dot Product Instructions

DPPD	Perform double precision dot product for up to 2 elements and broadcast.
DPPS	Perform single precision dot products for up to 4 elements and broadcast.

5.10.3 Streaming Load Hint Instruction

MOVNTDQA	Provides a non-temporal hint that can cause adjacent 16-byte items within an aligned 64-byte region (a streaming line) to be fetched and held in a small set of temporary buffers ("streaming load buffers"). Subsequent streaming loads to other aligned 16-byte items in the same streaming line may be supplied from the streaming load buffer and can improve throughput.

5.10.4 Packed Blending Instructions

BLENDPD	Conditionally copies specified double precision floating-point data elements in the source operand to the corresponding data elements in the destination, using an immediate byte control.
BLENDPS	Conditionally copies specified single precision floating-point data elements in the source operand to the corresponding data elements in the destination, using an immediate byte control.
BLENDVPD	Conditionally copies specified double precision floating-point data elements in the source operand to the corresponding data elements in the destination, using an implied mask.
BLENDVPS	Conditionally copies specified single precision floating-point data elements in the source operand to the corresponding data elements in the destination, using an implied mask.
PBLENDVB	Conditionally copies specified byte elements in the source operand to the corresponding elements in the destination, using an implied mask.
PBLENDW	Conditionally copies specified word elements in the source operand to the corresponding elements in the destination, using an immediate byte control.

5.10.5 Packed Integer MIN/MAX Instructions

PMINUW	Compare packed unsigned word integers.
PMINUD	Compare packed unsigned dword integers.
PMINSB	Compare packed signed byte integers.
PMINSD	Compare packed signed dword integers.
PMAXUW	Compare packed unsigned word integers.
PMAXUD	Compare packed unsigned dword integers.
PMAXSB	Compare packed signed byte integers.
PMAXSD	Compare packed signed dword integers.

5.10.6 Floating-Point Round Instructions With Selectable Rounding Mode

ROUNDPS	Round packed single precision floating-point values into integer values and return rounded floating-point values.
ROUNDPD	Round packed double precision floating-point values into integer values and return rounded floating-point values.
ROUNDSS	Round the low packed single precision floating-point value into an integer value and return a rounded floating-point value.
ROUNDSD	Round the low packed double precision floating-point value into an integer value and return a rounded floating-point value.

5.10.7 Insertion and Extractions from XMM Registers

EXTRACTPS	Extracts a single precision floating-point value from a specified offset in an XMM register and stores the result to memory or a general-purpose register.
INSERTPS	Inserts a single precision floating-point value from either a 32-bit memory location or selected from a specified offset in an XMM register to a specified offset in the destination XMM register. In addition, INSERTPS allows zeroing out selected data elements in the destination, using a mask.
PINSRB	Insert a byte value from a register or memory into an XMM register.
PINSRD	Insert a dword value from 32-bit register or memory into an XMM register.
PINSRQ	Insert a qword value from 64-bit register or memory into an XMM register.
PEXTRB	Extract a byte from an XMM register and insert the value into a general-purpose register or memory.
PEXTRW	Extract a word from an XMM register and insert the value into a general-purpose register or memory.
PEXTRD	Extract a dword from an XMM register and insert the value into a general-purpose register or memory.
PEXTRQ	Extract a qword from an XMM register and insert the value into a general-purpose register or memory.

5.10.8 Packed Integer Format Conversions

PMOVSXBW	Sign extend the lower 8-bit integer of each packed word element into packed signed word integers.
PMOVZXBW	Zero extend the lower 8-bit integer of each packed word element into packed signed word integers.
PMOVSXBD	Sign extend the lower 8-bit integer of each packed dword element into packed signed dword integers.
PMOVZXBD	Zero extend the lower 8-bit integer of each packed dword element into packed signed dword integers.
PMOVSXWD	Sign extend the lower 16-bit integer of each packed dword element into packed signed dword integers.
PMOVZXWD	Zero extend the lower 16-bit integer of each packed dword element into packed signed dword integers.
PMOVSXBQ	Sign extend the lower 8-bit integer of each packed qword element into packed signed qword integers.
PMOVZXBQ	Zero extend the lower 8-bit integer of each packed qword element into packed signed qword integers.
PMOVSXWQ	Sign extend the lower 16-bit integer of each packed qword element into packed signed qword integers.
PMOVZXWQ	Zero extend the lower 16-bit integer of each packed qword element into packed signed qword integers.
PMOVSXDQ	Sign extend the lower 32-bit integer of each packed qword element into packed signed qword integers.
PMOVZXDQ	Zero extend the lower 32-bit integer of each packed qword element into packed signed qword integers.

5.10.9 Improved Sums of Absolute Differences (SAD) for 4-Byte Blocks

MPSADBW	Performs eight 4-byte wide Sum of Absolute Differences operations to produce eight word integers.

5.10.10 Horizontal Search

PHMINPOSUW	Finds the value and location of the minimum unsigned word from one of 8 horizontally packed unsigned words. The resulting value and location (offset within the source) are packed into the low dword of the destination XMM register.

5.10.11 Packed Test

PTEST	Performs a logical AND between the destination with this mask and sets the ZF flag if the result is zero. The CF flag (zero for TEST) is set if the inverted mask AND'd with the destination is all zeroes.

5.10.12 Packed Qword Equality Comparisons

PCMPEQQ	128-bit packed qword equality test.

5.10.13 Dword Packing With Unsigned Saturation

PACKUSDW	Packs dword to word with unsigned saturation.

5.11 INTEL® SSE4.2 INSTRUCTION SET

Five of the Intel SSE4.2 instructions operate on XMM register as a source or destination. These include four text/string processing instructions and one packed quadword compare SIMD instruction. Programming these five Intel SSE4.2 instructions is similar to programming 128-bit Integer SIMD in Intel SSE2/SSSE3. Intel SSE4.2 does not provide any 64-bit integer SIMD instructions.

CRC32 operates on general-purpose registers and is summarized in Section 5.1.6. The sections that follow summarize each subgroup.

5.11.1 String and Text Processing Instructions

PCMPESTRI	Packed compare explicit-length strings, return index in ECX/RCX.
PCMPESTRM	Packed compare explicit-length strings, return mask in XMM0.
PCMPISTRI	Packed compare implicit-length strings, return index in ECX/RCX.
PCMPISTRM	Packed compare implicit-length strings, return mask in XMM0.

5.11.2 Packed Comparison SIMD Integer Instruction

PCMPGTQ	Performs logical compare of greater-than on packed integer quadwords.

5.12 INTEL® AES-NI AND PCLMULQDQ

Six Intel® AES-NI instructions operate on XMM registers to provide accelerated primitives for block encryption/decryption using Advanced Encryption Standard (FIPS-197). The PCLMULQDQ instruction performs carry-less multiplication for two binary numbers up to 64-bit wide.

AESDEC	Perform an AES decryption round using an 128-bit state and a round key.
AESDECLAST	Perform the last AES decryption round using an 128-bit state and a round key.
AESENC	Perform an AES encryption round using an 128-bit state and a round key.
AESENCLAST	Perform the last AES encryption round using an 128-bit state and a round key.
AESIMC	Perform an inverse mix column transformation primitive.

AESKEYGENASSIST	Assist the creation of round keys with a key expansion schedule.
PCLMULQDQ	Perform carryless multiplication of two 64-bit numbers.

5.13 INTEL® ADVANCED VECTOR EXTENSIONS (INTEL® AVX)

Intel® Advanced Vector Extensions (AVX) promote legacy 128-bit SIMD instruction sets that operate on the XMM register set to use a "vector extension" (VEX) prefix and operates on 256-bit vector registers (YMM). Almost all prior generations of 128-bit SIMD instructions that operate on XMM (but not on MMX registers) are promoted to support three-operand syntax with VEX-128 encoding.

VEX-prefix encoded Intel AVX instructions support 256-bit and 128-bit floating-point operations by extending the legacy 128-bit SIMD floating-point instructions to support three-operand syntax.

Additional functional enhancements are also provided with VEX-encoded Intel AVX instructions.

The list of Intel AVX instructions is included in the following tables:

- Table 14-2 lists 256-bit and 128-bit floating-point arithmetic instructions promoted from legacy 128-bit SIMD instruction sets.
- Table 14-3 lists 256-bit and 128-bit data movement and processing instructions promoted from legacy 128-bit SIMD instruction sets.
- Table 14-4 lists functional enhancements of 256-bit Intel AVX instructions not available from legacy 128-bit SIMD instruction sets.
- Table 14-5 lists 128-bit integer and floating-point instructions promoted from legacy 128-bit SIMD instruction sets.
- Table 14-6 lists functional enhancements of 128-bit Intel AVX instructions not available from legacy 128-bit SIMD instruction sets.
- Table 14-7 lists 128-bit data movement and processing instructions promoted from legacy instruction sets.

5.14 16-BIT FLOATING-POINT CONVERSION

Conversions between single precision floating-point (32-bit) and half precision floating-point (16-bit) data are provided by the VCVTPS2PH and VCVTPH2PS instructions, introduced beginning with the third generation of Intel Core processors based on Ivy Bridge microarchitecture:

VCVTPH2PS	Convert eight/four data elements containing 16-bit floating-point data into eight/four single precision floating-point data.
VCVTPS2PH	Convert eight/four data elements containing single precision floating-point data into eight/four 16-bit floating-point data.

Starting with the 4th generation Intel Xeon Scalable Processor Family based on Sapphire Rapids microarchitecture, Intel® AVX-512 instruction set architecture for FP16 was added, supporting a wide range of general-purpose numeric operations for 16-bit half precision floating-point values (binary16 in IEEE Standard 754-2019 for Floating-Point Arithmetic, aka half precision or FP16). Section 5.19 includes a list of these instructions.

5.15 FUSED-MULTIPLY-ADD (FMA)

FMA extensions enhances Intel AVX with high-throughput, arithmetic capabilities covering fused multiply-add, fused multiply-subtract, fused multiply add/subtract interleave, signed-reversed multiply on fused multiply-add and multiply-subtract. FMA extensions provide 36 256-bit floating-point instructions to perform computation on 256-bit vectors and additional 128-bit and scalar FMA instructions.

- Table 14-15 lists FMA instruction sets.

5.16 INTEL® ADVANCED VECTOR EXTENSIONS 2 (INTEL® AVX2)

Intel® AVX2 extends Intel AVX by promoting most of the 128-bit SIMD integer instructions with 256-bit numeric processing capabilities. Intel AVX2 instructions follow the same programming model as AVX instructions.

In addition, AVX2 provide enhanced functionalities for broadcast/permute operations on data elements, vector shift instructions with variable-shift count per data element, and instructions to fetch non-contiguous data elements from memory.

- Table 14-18 lists promoted vector integer instructions in AVX2.
- Table 14-19 lists new instructions in AVX2 that complements AVX.

5.17 INTEL® TRANSACTIONAL SYNCHRONIZATION EXTENSIONS (INTEL® TSX)

XABORT	Abort an RTM transaction execution.
XACQUIRE	Prefix hint to the beginning of an HLE transaction region.
XRELEASE	Prefix hint to the end of an HLE transaction region.
XBEGIN	Transaction begin of an RTM transaction region.
XEND	Transaction end of an RTM transaction region.
XTEST	Test if executing in a transactional region.
XRESLDTRK	Resume tracking load addresses.
XSUSLDTRK	Suspend tracking load addresses.

5.18 INTEL® SHA EXTENSIONS

Intel® SHA extensions provide a set of instructions that target the acceleration of the Secure Hash Algorithm (SHA), specifically the SHA-1 and SHA-256 variants.

SHA1MSG1	Perform an intermediate calculation for the next four SHA1 message dwords from the previous message dwords.
SHA1MSG2	Perform the final calculation for the next four SHA1 message dwords from the intermediate message dwords.
SHA1NEXTE	Calculate SHA1 state E after four rounds.
SHA1RNDS4	Perform four rounds of SHA1 operations.
SHA256MSG1	Perform an intermediate calculation for the next four SHA256 message dwords.
SHA256MSG2	Perform the final calculation for the next four SHA256 message dwords.
SHA256RNDS2	Perform two rounds of SHA256 operations.

5.19 INTEL® ADVANCED VECTOR EXTENSIONS 512 (INTEL® AVX-512)

The Intel® AVX-512 family comprises a collection of 512-bit SIMD instruction sets to accelerate a diverse range of applications. Intel AVX-512 instructions provide a wide range of functionality that support programming in 512-bit, 256 and 128-bit vector register, plus support for opmask registers and instructions operating on opmask registers.

The collection of 512-bit SIMD instruction sets in Intel AVX-512 include new functionality not available in Intel AVX and Intel AVX2, and promoted instructions similar to equivalent ones in Intel AVX/Intel AVX2 but with enhancement provided by opmask registers not available to VEX-encoded Intel AVX/Intel AVX2. Some instruction mnemonics in Intel AVX/Intel AVX2 that are promoted into Intel AVX-512 can be replaced by new instruction mnemonics that are available only with EVEX encoding, e.g., VBROADCASTF128 into VBROADCASTF32X4. Details of EVEX instruction encoding are discussed in Section 2.7, “Intel® AVX-512 Encoding,” of the Intel® 64 and IA-32 Architectures Software Developer’s Manual, Volume 2A. Starting with the 4th generation Intel Xeon Scalable Processor Family, an Intel AVX-512 instruction set architecture for FP16 was added, supporting a wide range of

general-purpose numeric operations for 16-bit half precision floating-point values, which complements the existing 32-bit and 64-bit floating-point instructions already available in the Intel Xeon processor-based products.

512-bit instruction mnemonics in AVX-512F instructions that are not Intel AVX or AVX2 promotions include:

VALIGND/Q	Perform dword/qword alignment of two concatenated source vectors.
VBLENDMPD/PS	Replace the VBLENDVPD/PS instructions (using opmask as select control).
VCOMPRESSPD/PS	Compress packed DP or SP elements of a vector.
VCVT(T)PD2UDQ	Convert packed DP FP elements of a vector to packed unsigned 32-bit integers.
VCVT(T)PS2UDQ	Convert packed SP FP elements of a vector to packed unsigned 32-bit integers.
VCVTQQ2PD/PS	Convert packed signed 64-bit integers to packed DP/SP FP elements.
VCVT(T)SD2USI	Convert the low DP FP element of a vector to an unsigned integer.
VCVT(T)SS2USI	Convert the low SP FP element of a vector to an unsigned integer.
VCVTUDQ2PD/PS	Convert packed unsigned 32-bit integers to packed DP/SP FP elements.
VCVTUSI2USD/S	Convert an unsigned integer to the low DP/SP FP element and merge to a vector.
VEXPANDPD/PS	Expand packed DP or SP elements of a vector.
VEXTRACTF32X4/64X4	Extract a vector from a full-length vector with 32/64-bit granular update.
VEXTRACTI32X4/64X4	Extract a vector from a full-length vector with 32/64-bit granular update.
VFIXUPIMMPD/PS	Perform fix-up to special values in DP/SP FP vectors.
VFIXUPIMMSD/SS	Perform fix-up to special values of the low DP/SP FP element.
VGETEXPPD/PS	Convert the exponent of DP/SP FP elements of a vector into FP values.
VGETEXPSD/SS	Convert the exponent of the low DP/SP FP element in a vector into FP value.
VGETMANTPD/PS	Convert the mantissa of DP/SP FP elements of a vector into FP values.
VGETMANTSD/SS	Convert the mantissa of the low DP/SP FP element of a vector into FP value.
VINSERTF32X4/64X4	Insert a 128/256-bit vector into a full-length vector with 32/64-bit granular update.
VMOVDQA32/64	VMOVDQA with 32/64-bit granular conditional update.
VMOVDQU32/64	VMOVDQU with 32/64-bit granular conditional update.
VPBLENDMD/Q	Blend dword/qword elements using opmask as select control.
VPBROADCASTD/Q	Broadcast from general-purpose register to vector register.
VPCMPD/UD	Compare packed signed/unsigned dwords using specified primitive.
VPCMPQ/UQ	Compare packed signed/unsigned quadwords using specified primitive.
VPCOMPRESSQ/D	Compress packed 64/32-bit elements of a vector.
VPERMI2D/Q	Full permute of two tables of dword/qword elements overwriting the index vector.
VPERMI2PD/PS	Full permute of two tables of DP/SP elements overwriting the index vector.
VPERMT2D/Q	Full permute of two tables of dword/qword elements overwriting one source table.
VPERMT2PD/PS	Full permute of two tables of DP/SP elements overwriting one source table.
VPEXPANDD/Q	Expand packed dword/qword elements of a vector.
VPMAXSQ	Compute maximum of packed signed 64-bit integer elements.
VPMAXUD/UQ	Compute maximum of packed unsigned 32/64-bit integer elements.
VPMINSQ	Compute minimum of packed signed 64-bit integer elements.
VPMINUD/UQ	Compute minimum of packed unsigned 32/64-bit integer elements.
VPMOV(S|US)QB	Down convert qword elements in a vector to byte elements using truncation (saturation | unsigned saturation).
VPMOV(S|US)QW	Down convert qword elements in a vector to word elements using truncation (saturation | unsigned saturation).
VPMOV(S|US)QD	Down convert qword elements in a vector to dword elements using truncation (saturation | unsigned saturation).
VPMOV(S|US)DB	Down convert dword elements in a vector to byte elements using truncation (saturation | unsigned saturation).

VPMOV(S|US)DW	Down convert dword elements in a vector to word elements using truncation (saturation | unsigned saturation).
VPROLD/Q	Rotate dword/qword element left by a constant shift count with conditional update.
VPROLVD/Q	Rotate dword/qword element left by shift counts specified in a vector with conditional update.
VPRORD/Q	Rotate dword/qword element right by a constant shift count with conditional update.
VPRORRD/Q	Rotate dword/qword element right by shift counts specified in a vector with conditional update.
VPSCATTERDD/DQ	Scatter dword/qword elements in a vector to memory using dword indices.
VPSCATTERQD/QQ	Scatter dword/qword elements in a vector to memory using qword indices.
VPSRAQ	Shift qwords right by a constant shift count and shifting in sign bits.
VPSRAVQ	Shift qwords right by shift counts in a vector and shifting in sign bits.
VPTESTNMD/Q	Perform bitwise NAND of dword/qword elements of two vectors and write results to opmask.
VPTERLOGD/Q	Perform bitwise ternary logic operation of three vectors with 32/64 bit granular conditional update.
VPTESTMD/Q	Perform bitwise AND of dword/qword elements of two vectors and write results to opmask.
VRCP14PD/PS	Compute approximate reciprocals of packed DP/SP FP elements of a vector.
VRCP14SD/SS	Compute the approximate reciprocal of the low DP/SP FP element of a vector.
VRNDSCALEPD/PS	Round packed DP/SP FP elements of a vector to specified number of fraction bits.
VRNDSCALESD/SS	Round the low DP/SP FP element of a vector to specified number of fraction bits.
VRSQRT14PD/PS	Compute approximate reciprocals of square roots of packed DP/SP FP elements of a vector.
VRSQRT14SD/SS	Compute the approximate reciprocal of square root of the low DP/SP FP element of a vector.
VSCALEPD/PS	Multiply packed DP/SP FP elements of a vector by powers of two with exponents specified in a second vector.
VSCALESD/SS	Multiply the low DP/SP FP element of a vector by powers of two with exponent specified in the corresponding element of a second vector.
VSCATTERDD/DQ	Scatter SP/DP FP elements in a vector to memory using dword indices.
VSCATTERQD/QQ	Scatter SP/DP FP elements in a vector to memory using qword indices.
VSHUFF32X4/64X2	Shuffle 128-bit lanes of a vector with 32/64 bit granular conditional update.
VSHUFI32X4/64X2	Shuffle 128-bit lanes of a vector with 32/64 bit granular conditional update.

512-bit instruction mnemonics in AVX-512DQ that are not Intel AVX or AVX2 promotions include:

VCVT(T)PD2QQ	Convert packed DP FP elements of a vector to packed signed 64-bit integers.
VCVT(T)PD2UQQ	Convert packed DP FP elements of a vector to packed unsigned 64-bit integers.
VCVT(T)PS2QQ	Convert packed SP FP elements of a vector to packed signed 64-bit integers.
VCVT(T)PS2UQQ	Convert packed SP FP elements of a vector to packed unsigned 64-bit integers.
VCVTUQQ2PD/PS	Convert packed unsigned 64-bit integers to packed DP/SP FP elements.
VEXTRACTF64X2	Extract a vector from a full-length vector with 64-bit granular update.
VEXTRACTI64X2	Extract a vector from a full-length vector with 64-bit granular update.
VFPCLASSPD/PS	Test packed DP/SP FP elements in a vector by numeric/special-value category.
VFPCLASSSD/SS	Test the low DP/SP FP element by numeric/special-value category.
VINSERTF64X2	Insert a 128-bit vector into a full-length vector with 64-bit granular update.
VINSERTI64X2	Insert a 128-bit vector into a full-length vector with 64-bit granular update.
VPMOVM2D/Q	Convert opmask register to vector register in 32/64-bit granularity.
VPMOVB2D/Q2M	Convert a vector register in 32/64-bit granularity to an opmask register.

VPMULLQ	Multiply packed signed 64-bit integer elements of two vectors and store low 64-bit signed result.
VRANGEPD/PS	Perform RANGE operation on each pair of DP/SP FP elements of two vectors using specified range primitive in imm8.
VRANGESD/SS	Perform RANGE operation on the pair of low DP/SP FP element of two vectors using specified range primitive in imm8.
VREDUCEPD/PS	Perform Reduction operation on packed DP/SP FP elements of a vector using specified reduction primitive in imm8.
VREDUCESD/SS	Perform Reduction operation on the low DP/SP FP element of a vector using specified reduction primitive in imm8.

512-bit instruction mnemonics in AVX-512BW that are not Intel AVX or AVX2 promotions include:

VDBPSADBW	Double block packed Sum-Absolute-Differences on unsigned bytes.
VMOVDQU8/16	VMOVDQU with 8/16-bit granular conditional update.
VPBLENDMB	Replaces the VPBLENDVB instruction (using opmask as select control).
VPBLENDMW	Blend word elements using opmask as select control.
VPBROADCASTB/W	Broadcast from general-purpose register to vector register.
VPCMPB/UB	Compare packed signed/unsigned bytes using specified primitive.
VPCMPW/UW	Compare packed signed/unsigned words using specified primitive.
VPERMW	Permute packed word elements.
VPERMI2W	Full permute from two tables of word elements overwriting the index vector.
VPMOVM2B/W	Convert opmask register to vector register in 8/16-bit granularity.
VPMOVB2M/W2M	Convert a vector register in 8/16-bit granularity to an opmask register.
VPMOV(S|US)WB	Down convert word elements in a vector to byte elements using truncation (saturation | unsigned saturation).
VPSLLVW	Shift word elements in a vector left by shift counts in a vector.
VPSRAVW	Shift words right by shift counts in a vector and shifting in sign bits.
VPSRLVW	Shift word elements in a vector right by shift counts in a vector.
VPTESTNMB/W	Perform bitwise NAND of byte/word elements of two vectors and write results to opmask.
VPTESTMB/W	Perform bitwise AND of byte/word elements of two vectors and write results to opmask.

512-bit instruction mnemonics in AVX-512CD that are not Intel AVX or AVX2 promotions include:

VPBROADCASTM	Broadcast from opmask register to vector register.
VPCONFLICTD/Q	Detect conflicts within a vector of packed 32/64-bit integers.
VPLZCNTD/Q	Count the number of leading zero bits of packed dword/qword elements.

Opmask instructions include:

KADDB/W/D/Q	Add two 8/16/32/64-bit opmasks.
KANDB/W/D/Q	Logical AND two 8/16/32/64-bit opmasks.
KANDNB/W/D/Q	Logical AND NOT two 8/16/32/64-bit opmasks.
KMOVB/W/D/Q	Move from or move to opmask register of 8/16/32/64-bit data.
KNOTB/W/D/Q	Bitwise NOT of two 8/16/32/64-bit opmasks.
KORB/W/D/Q	Logical OR two 8/16/32/64-bit opmasks.
KORTESTB/W/D/Q	Update EFLAGS according to the result of bitwise OR of two 8/16/32/64-bit opmasks.
KSHIFTLB/W/D/Q	Shift left 8/16/32/64-bit opmask by specified count.
KSHIFTRB/W/D/Q	Shift right 8/16/32/64-bit opmask by specified count.
KTESTB/W/D/Q	Update EFLAGS according to the result of bitwise TEST of two 8/16/32/64-bit opmasks.

KUNPCKBW/WD/DQ	Unpack and interleave two 8/16/32-bit opmasks into 16/32/64-bit mask.
KXNORB/W/D/Q	Bitwise logical XNOR of two 8/16/32/64-bit opmasks.
KXORB/W/D/Q	Logical XOR of two 8/16/32/64-bit opmasks.

512-bit instruction mnemonics in AVX-512ER include:

VEXP2PD/PS	Compute approximate base-2 exponential of packed DP/SP FP elements of a vector.
VEXP2SD/SS	Compute approximate base-2 exponential of the low DP/SP FP element of a vector.
VRCP28PD/PS	Compute approximate reciprocals to 28 bits of packed DP/SP FP elements of a vector.
VRCP28SD/SS	Compute the approximate reciprocal to 28 bits of the low DP/SP FP element of a vector.
VRSQRT28PD/PS	Compute approximate reciprocals of square roots to 28 bits of packed DP/SP FP elements of a vector.
VRSQRT28SD/SS	Compute the approximate reciprocal of square root to 28 bits of the low DP/SP FP element of a vector.

512-bit instruction mnemonics in AVX-512PF include:

VGATHERPF0DPD/PS	Sparse prefetch of packed DP/SP FP vector with T0 hint using dword indices.
VGATHERPF0QPD/PS	Sparse prefetch of packed DP/SP FP vector with T0 hint using qword indices.
VGATHERPF1DPD/PS	Sparse prefetch of packed DP/SP FP vector with T1 hint using dword indices.
VGATHERPF1QPD/PS	Sparse prefetch of packed DP/SP FP vector with T1 hint using qword indices.
VSCATTERPF0DPD/PS	Sparse prefetch of packed DP/SP FP vector with T0 hint to write using dword indices.
VSCATTERPF0QPD/PS	Sparse prefetch of packed DP/SP FP vector with T0 hint to write using qword indices.
VSCATTERPF1DPD/PS	Sparse prefetch of packed DP/SP FP vector with T1 hint to write using dword indices.
VSCATTERPF1QPD/PS	Sparse prefetch of packed DP/SP FP vector with T1 hint to write using qword indices.

512-bit instruction mnemonics in AVX512-FP16 include:

VADDPH/SH	Add packed/scalar FP16 values.
VCMPPH/SH	Compare packed/scalar FP16 values.
VCOMISH	Compare scalar ordered FP16 values and set EFLAGS.
VCVTDQ2PH	Convert packed signed doubleword integers to packed FP16 values.
VCVTPD2PH	Convert packed double precision FP values to packed FP16 values.
VCVTPH2DQ/QQ	Convert packed FP16 values to signed doubleword/quadword integers.
VCVTPH2PD	Convert packed FP16 values to FP64 values.
VCVTPH2PS[X]	Convert packed FP16 values to single precision floating-point values.
VCVTPH2QQ	Convert packed FP16 values to signed quadword integer values.
VCVTPH2UDQ/QQ	Convert packed FP16 values to unsigned doubleword/quadword integers.
VCVTPH2UW/W	Convert packed FP16 values to unsigned/signed word integers.
VCVTPS2PH[X]	Convert packed single precision floating-point values to packed FP16 values.
VCVTQQ2PH	Convert packed signed quadword integers to packed FP16 values.
VCVTSD2SH	Convert low FP64 value to an FP16 value.
VCVTSH2SD/SS	Convert low FP16 value to an FP64/FP32 value.
VCVTSH2SI/USI	Convert low FP16 value to signed/unsigned integer.
VCVTSI2SH	Convert a signed doubleword/quadword integer to an FP16 value.
VCVTSS2SH	Convert low FP32 value to an FP16 value.
VCVTTPH2DQ/QQ	Convert with truncation packed FP16 values to signed doubleword/quadword integers.
VCVTTPH2UDQ/QQ	Convert with truncation packed FP16 values to unsigned doubleword/quadword integers.
VCVTTPH2UW/W	Convert packed FP16 values to unsigned/signed word integers.

VCVTTSH2SI/USI	Convert with truncation low FP16 value to a signed/unsigned integer.
VCVTUDQ2PH	Convert packed unsigned doubleword integers to packed FP16 values.
VCVTUQQ2PH	Convert packed unsigned quadword integers to packed FP16 values.
VCVTUSI2SH	Convert unsigned doubleword integer to an FP16 value.
VCVTUW2PH	Convert packed unsigned word integers to FP16 values.
VCVTW2PH	Convert packed signed word integers to FP16 values.
VDIVPH/SH	Divide packed/scalar FP16 values.
VF[C]MADDCPH	Complex multiply and accumulate FP16 values.
VF[C]MADDCSH	Complex multiply and accumulate scalar FP16 values.
VF[C]MULCPH	Complex multiply FP16 values.
VF[C]MULCSH	Complex multiply scalar FP16 values.
VF[,N]MADD[132,213,231]PH	Fused multiply-add of packed FP16 values.
VF[,N]MADD[132,213,231]SH	Fused multiply-add of scalar FP16 values.
VFMADDSUB[132,213,231]PH	Fused multiply-alternating add/subtract of packed FP16 values.
VFMSUBADD[132,213,231]PH	Fused multiply-alternating subtract/add of packed FP16 values.
VF[,N]MSUB[132,213,231]PH	Fused multiply-subtract of packed FP16 values.
VF[,N]MSUB[132,213,231]SH	Fused multiply-subtract of scalar FP16 values.
VFPCLASSPH/SH	Test types of packed/scalar FP16 values.
VGETEXPPH/SH	Convert exponents of packed/scalar FP16 values to FP16 values.
VGETMANTPH/SH	Extract FP16 vector of normalized mantissas from FP16 vector/scalar.
VMAXPH/PS	Return maximum of packed/scalar FP16 values.
VMINPH/PS	Return minimum of packed/scalar FP16 values.
VMOVSH	Move scalar FP16 value.
VMOVW	Move word.
VMULPH/SH	Multiply packed/scalar FP16 values.
VRCPPH/SH	Compute reciprocals of packed/scalar FP16 values.
VREDUCEPH/SH	Perform reduction transformation on packed/scalar FP16 values.
VRNDSCALEPH/SH	Round packed/scalar FP16 values to include a given number of fraction bits.
VRSQRTPH/SH	Compute reciprocals of square roots of packed/scalar FP16 values.
VSCALEPH/SH	Scale packed/scalar FP16 values with FP16 values.
VSQRTPH/SH	Compute square root of packed/scalar FP16 values.
VSUBPH/SH	Subtract packed/scalar FP16 values.
VUCOMISH	Unordered compare scalar FP16 values and set EFLAGS.

5.20 SYSTEM INSTRUCTIONS

The following system instructions are used to control those functions of the processor that are provided to support for operating systems and executives.

CLAC	Clear AC Flag in EFLAGS register.
STAC	Set AC Flag in EFLAGS register.
LGDT	Load global descriptor table (GDT) register.
SGDT	Store global descriptor table (GDT) register.
LLDT	Load local descriptor table (LDT) register.
SLDT	Store local descriptor table (LDT) register.
LTR	Load task register.
STR	Store task register.

LIDT	Load interrupt descriptor table (IDT) register.
SIDT	Store interrupt descriptor table (IDT) register.
MOV	Load and store control registers.
LMSW	Load machine status word.
SMSW	Store machine status word.
CLTS	Clear the task-switched flag.
ARPL	Adjust requested privilege level.
LAR	Load access rights.
LSL	Load segment limit.
VERR	Verify segment for reading
VERW	Verify segment for writing.
MOV	Load and store debug registers.
INVD	Invalidate cache, no writeback.
WBINVD	Invalidate cache, with writeback.
INVLPG	Invalidate TLB Entry.
INVPCID	Invalidate Process-Context Identifier.
LOCK (prefix)	Perform atomic access to memory (can be applied to a number of general purpose instructions that provide memory source/destination access).
HLT	Halt processor.
RSM	Return from system management mode (SMM).
RDMSR	Read model-specific register.
WRMSR	Write model-specific register.
RDPMC	Read performance monitoring counters.
RDTSC	Read time stamp counter.
RDTSCP	Read time stamp counter and processor ID.
SYSENTER	Fast System Call, transfers to a flat protected mode kernel at CPL = 0.
SYSEXIT	Fast System Call, transfers to a flat protected mode kernel at CPL = 3.
XSAVE	Save processor extended states to memory.
XSAVEC	Save processor extended states with compaction to memory.
XSAVEOPT	Save processor extended states to memory, optimized.
XSAVES	Save processor supervisor-mode extended states to memory.
XRSTOR	Restore processor extended states from memory.
XRSTORS	Restore processor supervisor-mode extended states from memory.
XGETBV	Reads the state of an extended control register.
XSETBV	Writes the state of an extended control register.
RDFSBASE	Reads from FS base address at any privilege level.
RDGSBASE	Reads from GS base address at any privilege level.
WRFSBASE	Writes to FS base address at any privilege level.
WRGSBASE	Writes to GS base address at any privilege level.

5.21 64-BIT MODE INSTRUCTIONS

The following instructions are introduced in 64-bit mode. This mode is a sub-mode of IA-32e mode.

CDQE	Convert doubleword to quadword.
CMPSQ	Compare string operands.
CMPXCHG16B	Compare RDX:RAX with m128.

LODSQ	Load qword at address (R)SI into RAX.
MOVSQ	Move qword from address (R)SI to (R)DI.
MOVZX (64-bits)	Move bytes/words to doublewords/quadwords, zero-extension.
STOSQ	Store RAX at address RDI.
SWAPGS	Exchanges current GS base register value with value in MSR address C0000102H.
SYSCALL	Fast call to privilege level 0 system procedures.
SYSRET	Return from fast system call.

5.22 VIRTUAL-MACHINE EXTENSIONS

The behavior of the VMCS-maintenance instructions is summarized below:

VMPTRLD	Takes a single 64-bit source operand in memory. It makes the referenced VMCS active and current.
VMPTRST	Takes a single 64-bit destination operand that is in memory. Current-VMCS pointer is stored into the destination operand.
VMCLEAR	Takes a single 64-bit operand in memory. The instruction sets the launch state of the VMCS referenced by the operand to "clear", renders that VMCS inactive, and ensures that data for the VMCS have been written to the VMCS-data area in the referenced VMCS region.
VMREAD	Reads a component from the VMCS (the encoding of that field is given in a register operand) and stores it into a destination operand.
VMWRITE	Writes a component to the VMCS (the encoding of that field is given in a register operand) from a source operand.

The behavior of the VMX management instructions is summarized below:

VMLAUNCH	Launches a virtual machine managed by the VMCS. A VM entry occurs, transferring control to the VM.
VMRESUME	Resumes a virtual machine managed by the VMCS. A VM entry occurs, transferring control to the VM.
VMXOFF	Causes the processor to leave VMX operation.
VMXON	Takes a single 64-bit source operand in memory. It causes a logical processor to enter VMX root operation and to use the memory referenced by the operand to support VMX operation.

The behavior of the VMX-specific TLB-management instructions is summarized below:

INVEPT	Invalidate cached **Extended Page Table** (EPT) mappings in the processor to synchronize address translation in virtual machines with memory-resident EPT pages.
INVVPID	Invalidate cached mappings of address translation based on the **Virtual Processor ID** (VPID).

None of the instructions above can be executed in compatibility mode; they generate invalid-opcode exceptions if executed in compatibility mode.

The behavior of the guest-available instructions is summarized below:

VMCALL	Allows a guest in VMX non-root operation to call the VMM for service. A VM exit occurs, transferring control to the VMM.
VMFUNC	Allows software in VMX non-root operation to invoke a VM function, which is processor functionality enabled and configured by software in VMX root operation. No VM exit occurs.

5.23 SAFER MODE EXTENSIONS

The behavior of the GETSEC instruction leaves of the Safer Mode Extensions (SMX) are summarized below:

GETSEC[CAPABILITIES] Returns the available leaf functions of the GETSEC instruction.
GETSEC[ENTERACCS] Loads an authenticated code chipset module and enters authenticated code execution mode.
GETSEC[EXITAC] Exits authenticated code execution mode.
GETSEC[SENTER] Establishes a Measured Launched Environment (MLE) which has its dynamic root of trust anchored to a chipset supporting Intel Trusted Execution Technology.
GETSEC[SEXIT] Exits the MLE.
GETSEC[PARAMETERS] Returns SMX related parameter information.
GETSEC[SMCRTL] SMX mode control.
GETSEC[WAKEUP] Wakes up sleeping logical processors inside an MLE.

5.24 INTEL® MEMORY PROTECTION EXTENSIONS

Intel Memory Protection Extensions (Intel MPX) provides a set of instructions to enable software to add robust bounds checking capability to memory references. Details of Intel MPX are described in Appendix E, "Intel® Memory Protection Extensions."

BNDMK Create a LowerBound and an UpperBound in a register.
BNDCL Check the address of a memory reference against a LowerBound.
BNDCU Check the address of a memory reference against an UpperBound in 1's complement form.
BNDCN Check the address of a memory reference against an UpperBound not in 1's complement form.
BNDMOV Copy or load from memory of the LowerBound and UpperBound to a register.
BNDMOV Store to memory of the LowerBound and UpperBound from a register.
BNDLDX Load bounds using address translation.
BNDSTX Store bounds using address translation.

5.25 INTEL® SOFTWARE GUARD EXTENSIONS

Intel Software Guard Extensions (Intel SGX) provide two sets of instruction leaf functions to enable application software to instantiate a protected container, referred to as an enclave. The enclave instructions are organized as leaf functions under two instruction mnemonics: ENCLS (ring 0) and ENCLU (ring 3). Details of Intel SGX are described in Chapter 37 through Chapter 43 of the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3D.

The first implementation of Intel SGX is also referred to as SGX1, it is introduced with the 6th Generation Intel Core Processors. The leaf functions supported in SGX1 are shown in Table 5-3.

Table 5-3. Supervisor and User Mode Enclave Instruction Leaf Functions in Long-Form of SGX1

Supervisor Instruction	Description	User Instruction	Description
ENCLS[EADD]	Add a page	ENCLU[EENTER]	Enter an Enclave
ENCLS[EBLOCK]	Block an EPC page	ENCLU[EEXIT]	Exit an Enclave
ENCLS[ECREATE]	Create an enclave	ENCLU[EGETKEY]	Create a cryptographic key
ENCLS[EDBGRD]	Read data by debugger	ENCLU[EREPORT]	Create a cryptographic report
ENCLS[EDBGWR]	Write data by debugger	ENCLU[ERESUME]	Re-enter an Enclave
ENCLS[EEXTEND]	Extend EPC page measurement	ENCLU [EVERIFYREPORT2]	Verify a cryptographic report of TDX Trust Domain (TD)
ENCLS[EINIT]	Initialize an enclave		
ENCLS[ELDB]	Load an EPC page as blocked		

Table 5-3. Supervisor and User Mode Enclave Instruction Leaf Functions in Long-Form of SGX1

Supervisor Instruction	Description	User Instruction	Description
ENCLS[ELDU]	Load an EPC page as unblocked		
ENCLS[EPA]	Add version array		
ENCLS[EREMOVE]	Remove a page from EPC		
ENCLS[ETRACK]	Activate EBLOCK checks		
ENCLS[EWB]	Write back/invalidate an EPC page		
ENCLS[EUPDATESVN]	Update CR_CPUSVN		

5.26 SHADOW STACK MANAGEMENT INSTRUCTIONS

Shadow stack management instructions allow the program and run-time to perform operations like recovering from control protection faults, shadow stack switching, etc. The following instructions are provided.

CLRSSBSY Clear busy bit in a supervisor shadow stack token.
INCSSP Increment the shadow stack pointer (SSP).
RDSSP Read shadow stack point (SSP).
RSTORSSP Restore a shadow stack pointer (SSP).
SAVEPREVSSP Save previous shadow stack pointer (SSP).
SETSSBSY Set busy bit in a supervisor shadow stack token.
WRSS Write to a shadow stack.
WRUSS Write to a user mode shadow stack.

5.27 CONTROL TRANSFER TERMINATING INSTRUCTIONS

ENDBR32 Terminate an Indirect Branch in 32-bit and Compatibility Mode.
ENDBR64 Terminate an Indirect Branch in 64-bit Mode.

5.28 INTEL® AMX INSTRUCTIONS

LDTILECFG Load tile configuration.
STTILECFG Store tile configuration.
TDPBF16PS Dot product of BF16 tiles accumulated into packed single precision tile.
TDPBSSD Dot product of signed bytes with dword accumulation.
TDPBSUD Dot product of signed/unsigned bytes with dword accumulation.
TDPBUSD Dot product of unsigned/signed bytes with dword accumulation.
TDPBUUD Dot product of unsigned bytes with dword accumulation.
TILELOADD Load data into tile.
TILELOADDT1 Load data into tile with hint to optimize data caching.
TILERELEASE Release tile.
TILESTORED Store tile.
TILEZERO Zero tile.

5.29 USER INTERRUPT INSTRUCTIONS

CLUI Clear user interrupt flag.

SENDUIPI	Send user interprocessor interrupt.
STUI	Set user interrupt flag.
TESTUI	Determine user interrupt flag.
UIRET	User-interrupt return.

5.30 ENQUEUE STORE INSTRUCTIONS

ENQCMD	Enqueue command.
ENQCMDS	Enqueue command supervisor.

5.31 INTEL® ADVANCED VECTOR EXTENSIONS 10 VERSION 1 INSTRUCTIONS

Intel® Advanced Vector Extensions 10 Version 1 (Intel® AVX10.1) is based on the Intel AVX-512 ISA feature set and includes all Intel AVX-512 instructions introduced with the Intel® Xeon® 6 P-core processor based on Granite Rapids microarchitecture. Intel AVX10.1 supports all instruction vector lengths (128, 256, and 512), as well as scalar and opmask instructions.

For a list of Intel AVX-512 instructions, see Section 5.19, “Intel® Advanced Vector Extensions 512 (Intel® AVX-512).” Additionally, note that some Intel AVX and Intel AVX2 instructions were promoted to Intel AVX512 and are also supported. See Section 5.13, “Intel® Advanced Vector Extensions (Intel® AVX),” Section 5.16, “Intel® Advanced Vector Extensions 2 (Intel® AVX2),” and Chapter 16, “Programming with Intel® AVX10,” for further details.

CHAPTER 6
PROCEDURE CALLS, INTERRUPTS, AND EXCEPTIONS

This chapter describes the facilities in the Intel 64 and IA-32 architectures for executing calls to procedures or subroutines. It also describes how interrupts and exceptions are handled from the perspective of an application programmer.

6.1 PROCEDURE CALL TYPES

The processor supports procedure calls in the following two different ways:

- CALL and RET instructions.
- ENTER and LEAVE instructions, in conjunction with the CALL and RET instructions.

Both of these procedure call mechanisms use the procedure stack, commonly referred to simply as "the stack," to save the state of the calling procedure, pass parameters to the called procedure, and store local variables for the currently executing procedure.

The processor's facilities for handling interrupts and exceptions are similar to those used by the CALL and RET instructions.

Processors that support Control-Flow Enforcement Technology (CET) support an additional stack referred to as "the shadow stack". The CALL instruction, when shadow stacks are enabled, additionally saves the state of the calling procedure on the shadow stack; and the RET instruction restores the state of the calling procedure if the state on the stack and the shadow stack match.

6.2 STACKS

The stack (see Figure 6-1) is a contiguous array of memory locations. It is contained in a segment and identified by the segment selector in the SS register. When using the flat memory model, the stack can be located anywhere in the linear address space for the program. A stack can be up to 4 GBytes long, the maximum size of a segment.

Items are placed on the stack using the PUSH instruction and removed from the stack using the POP instruction. When an item is pushed onto the stack, the processor decrements the ESP register, then writes the item at the new top of stack. When an item is popped off the stack, the processor reads the item from the top of stack, then increments the ESP register. In this manner, the stack grows **down** in memory (towards lesser addresses) when items are pushed on the stack and shrinks **up** (towards greater addresses) when the items are popped from the stack.

A program or operating system/executive can set up many stacks. For example, in multitasking systems, each task can be given its own stack. The number of stacks in a system is limited by the maximum number of segments and the available physical memory.

When a system sets up many stacks, only one stack—the **current stack**—is available at a time. The current stack is the one contained in the segment referenced by the SS register.

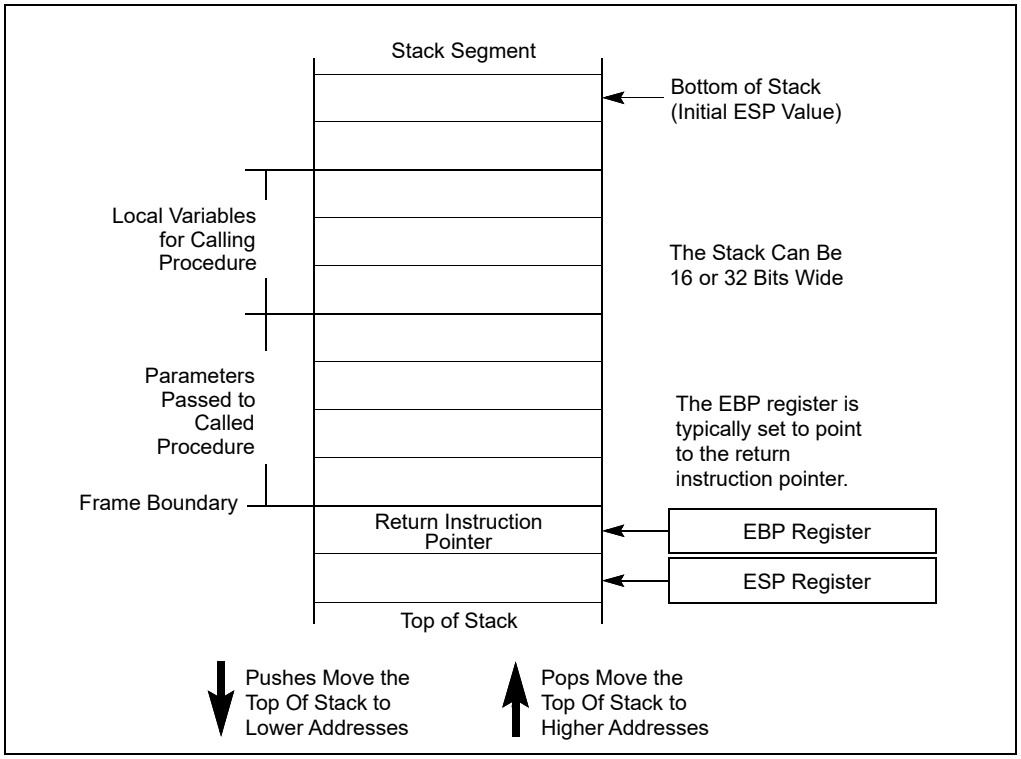


Figure 6-1. Stack Structure

The processor references the SS register automatically for all stack operations. For example, when the ESP register is used as a memory address, it automatically points to an address in the current stack. Also, the CALL, RET, PUSH, POP, ENTER, and LEAVE instructions all perform operations on the current stack.

6.2.1 Setting Up a Stack

To set a stack and establish it as the current stack, the program or operating system/executive must do the following:

1. Establish a stack segment.
2. Load the segment selector for the stack segment into the SS register using a MOV, POP, or LSS instruction.
3. Load the stack pointer for the stack into the ESP register using a MOV, POP, or LSS instruction. The LSS instruction can be used to load the SS and ESP registers in one operation.

See "Segment Descriptors" in Chapter 3, "Protected-Mode Memory Management," of the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3A, for information on how to set up a segment descriptor and segment limits for a stack segment.

6.2.2 Stack Alignment

The stack pointer for a stack segment should be aligned on 16-bit (word) or 32-bit (double-word) boundaries, depending on the width of the stack segment. The D flag in the segment descriptor for the current code segment sets the stack-segment width (see "Segment Descriptors" in Chapter 3, "Protected-Mode Memory Management," of the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3A). The PUSH and POP instructions use the D flag to determine how much to decrement or increment the stack pointer on a push or pop operation, respectively. When the stack width is 16 bits, the stack pointer is incremented or decremented in 16-bit increments; when the width is 32 bits, the stack pointer is incremented or decremented in 32-bit increments. Pushing a 16-bit value onto a 32-bit wide stack can result in stack misaligned (that is, the stack pointer is not aligned on a double-

word boundary). One exception to this rule is when the contents of a segment register (a 16-bit segment selector) are pushed onto a 32-bit wide stack. Here, the processor automatically aligns the stack pointer to the next 32-bit boundary.

The processor does not check stack pointer alignment. It is the responsibility of the programs, tasks, and system procedures running on the processor to maintain proper alignment of stack pointers. Misaligning a stack pointer can cause serious performance degradation and in some instances program failures.

6.2.3 Address-Size Attributes for Stack Accesses

Instructions that use the stack implicitly (such as the PUSH and POP instructions) have two address-size attributes each of either 16 or 32 bits. This is because they always have the implicit address of the top of the stack, and they may also have an explicit memory address (for example, PUSH Array1[EBX]). The attribute of the explicit address is determined by the D flag of the current code segment and the presence or absence of the 67H address-size prefix.

The address-size attribute of the top of the stack determines whether SP or ESP is used for the stack access. Stack operations with an address-size attribute of 16 use the 16-bit SP stack pointer register and can use a maximum stack address of FFFFH; stack operations with an address-size attribute of 32 bits use the 32-bit ESP register and can use a maximum address of FFFFFFFFH. The default address-size attribute for data segments used as stacks is controlled by the B flag of the segment's descriptor. When this flag is clear, the default address-size attribute is 16; when the flag is set, the address-size attribute is 32.

6.2.4 Procedure Linking Information

The processor provides two pointers for linking of procedures: the stack-frame base pointer and the return instruction pointer. When used in conjunction with a standard software procedure-call technique, these pointers permit reliable and coherent linking of procedures.

6.2.4.1 Stack-Frame Base Pointer

The stack is typically divided into frames. Each stack frame can then contain local variables, parameters to be passed to another procedure, and procedure linking information. The stack-frame base pointer (contained in the EBP register) identifies a fixed reference point within the stack frame for the called procedure. To use the stack-frame base pointer, the called procedure typically copies the contents of the ESP register into the EBP register prior to pushing any local variables on the stack. The stack-frame base pointer then permits easy access to data structures passed on the stack, to the return instruction pointer, and to local variables added to the stack by the called procedure.

Like the ESP register, the EBP register automatically points to an address in the current stack segment (that is, the segment specified by the current contents of the SS register).

6.2.4.2 Return Instruction Pointer

Prior to branching to the first instruction of the called procedure, the CALL instruction pushes the address in the EIP register onto the current stack. This address is then called the return-instruction pointer and it points to the instruction where execution of the calling procedure should resume following a return from the called procedure. Upon returning from a called procedure, the RET instruction pops the return-instruction pointer from the stack back into the EIP register. Execution of the calling procedure then resumes.

The processor does not keep track of the location of the return-instruction pointer. It is thus up to the programmer to ensure that stack pointer is pointing to the return-instruction pointer on the stack, prior to issuing a RET instruction. A common way to reset the stack pointer to the point to the return-instruction pointer is to move the contents of the EBP register into the ESP register. If the EBP register is loaded with the stack pointer immediately following a procedure call, it should point to the return instruction pointer on the stack.

The processor does not require that the return instruction pointer point back to the calling procedure. Prior to executing the RET instruction, the return instruction pointer can be manipulated in software to point to any address

in the current code segment (near return) or another code segment (far return). Performing such an operation, however, should be undertaken very cautiously, using only well defined code entry points.

6.2.5 Stack Behavior in 64-Bit Mode

In 64-bit mode, address calculations that reference SS segments are treated as if the segment base is zero. Fields (base, limit, and attribute) in segment descriptor registers are ignored. SS DPL is modified such that it is always equal to CPL. This will be true even if it is the only field in the SS descriptor that is modified.

Registers E(SP), E(IP) and E(BP) are promoted to 64-bits and are re-named RSP, RIP, and RBP respectively. Some forms of segment load instructions are invalid (for example, LDS, POP ES).

PUSH/POP instructions increment/decrement the stack using a 64-bit width. When the contents of a segment register is pushed onto 64-bit stack, the pointer is automatically aligned to 64 bits (as with a stack that has a 32-bit width).

6.3 SHADOW STACKS

A shadow stack is a second stack used exclusively for control transfer operations. This stack is separate from the procedure stack. The shadow stack is not used to store data, hence is not explicitly writeable by software. Writes to the shadow stack are restricted to control transfer instructions and shadow stack management instructions. Shadow stacks can be enabled separately for privilege level 3 (user mode) or privilege levels less than 3 (supervisor mode).

Shadow stacks are active only in protected mode with paging enabled. Shadow stacks cannot be enabled for a program executing in virtual 8086 mode.

Processors that support shadow stacks have an architectural register called the shadow stack pointer (SSP) that points to the current top of the shadow stack. The SSP cannot be directly encoded as a source, destination, or memory operand in instructions. The width of the shadow stack is 32-bit in 32-bit/compatibility mode, and is 64-bit in 64-bit mode. The address-size attribute of the shadow stack is likewise 32-bit in 32-bit/compatibility mode, and 64-bit in 64-bit mode.

The size of the shadow stack pushes and pops for far CALL and call to interrupt/exception handlers is fixed at 64 bits, and the processor uses 8-byte, zero padded stores for these pushes in 32-bit/compatibility modes.

6.4 CALLING PROCEDURES USING CALL AND RET

The CALL instruction allows control transfers to procedures within the current code segment (**near call**) and in a different code segment (**far call**). Near calls usually provide access to local procedures within the currently running program or task. Far calls are usually used to access operating system procedures or procedures in a different task. See "CALL—Call Procedure" in Chapter 3, "Instruction Set Reference, A-L," of the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 2A, for a detailed description of the CALL instruction.

The RET instruction also allows near and far returns to match the near and far versions of the CALL instruction. In addition, the RET instruction allows a program to increment the stack pointer on a return to release parameters from the stack. The number of bytes released from the stack is determined by an optional argument (*n*) to the RET instruction. See "RET—Return from Procedure" in Chapter 4, "Instruction Set Reference, M-U," of the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 2B, for a detailed description of the RET instruction.

6.4.1 Near CALL and RET Operation

When executing a near call, the processor does the following (see Figure 6-2):

1. Pushes the current value of the EIP register on the stack.

   If shadow stack is enabled and the displacement value is not 0, pushes the current value of the EIP register on the shadow stack.

2. Loads the offset of the called procedure in the EIP register.
3. Begins execution of the called procedure.

When executing a near return, the processor performs these actions:

1. Pops the top-of-stack value (the return instruction pointer) into the EIP register.

   If shadow stack is enabled, pops the top-of-stack (the return instruction pointer) value from the shadow stack and if it's not the same as the return instruction pointer popped from the stack, then the processor causes a control protection exception with error code NEAR-RET (#CP(NEAR-RET)).
2. If the RET instruction has an optional *n* argument, increments the stack pointer by the number of bytes specified with the *n* operand to release parameters from the stack.
3. Resumes execution of the calling procedure.

6.4.2 Far CALL and RET Operation

When executing a far call, the processor performs these actions (see Figure 6-2):

1. Pushes the current value of the CS register on the stack.

   If shadow stack is enabled:

   a. Temporarily saves the current value of the SSP register internally and aligns the SSP to the next 8 byte boundary.

   b. Pushes the current value of the CS register on the shadow stack.

   c. Pushes the current value of LIP (CS.base + EIP) on the shadow stack.

   d. Pushes the internally saved value of the SSP register on the shadow stack.
2. Pushes the current value of the EIP register on the stack.
3. Loads the segment selector of the segment that contains the called procedure in the CS register.
4. Loads the offset of the called procedure in the EIP register.
5. Begins execution of the called procedure.

When executing a far return, the processor does the following:

1. Pops the top-of-stack value (the return instruction pointer) into the EIP register.
2. Pops the top-of-stack value (the segment selector for the code segment being returned to) into the CS register.

   If shadow stack is enabled:

   a. Causes a control protection exception (#CP(FAR-RET/IRET)) if the SSP is not aligned to 8 bytes.

   b. Compares the values on the shadow stack at address SSP+8 (the LIP) and SSP+16 (the CS) to the CS and (CS.base + EIP) popped from the stack, and causes a control protection exception (#CP(FAR-RET/IRET)) if they do not match.

   c. Pops the top-of-stack value (the SSP of the procedure being returned to) from shadow stack into the SSP register.
3. If the RET instruction has an optional *n* argument, increments the stack pointer by the number of bytes specified with the *n* operand to release parameters from the stack.
4. Resumes execution of the calling procedure.

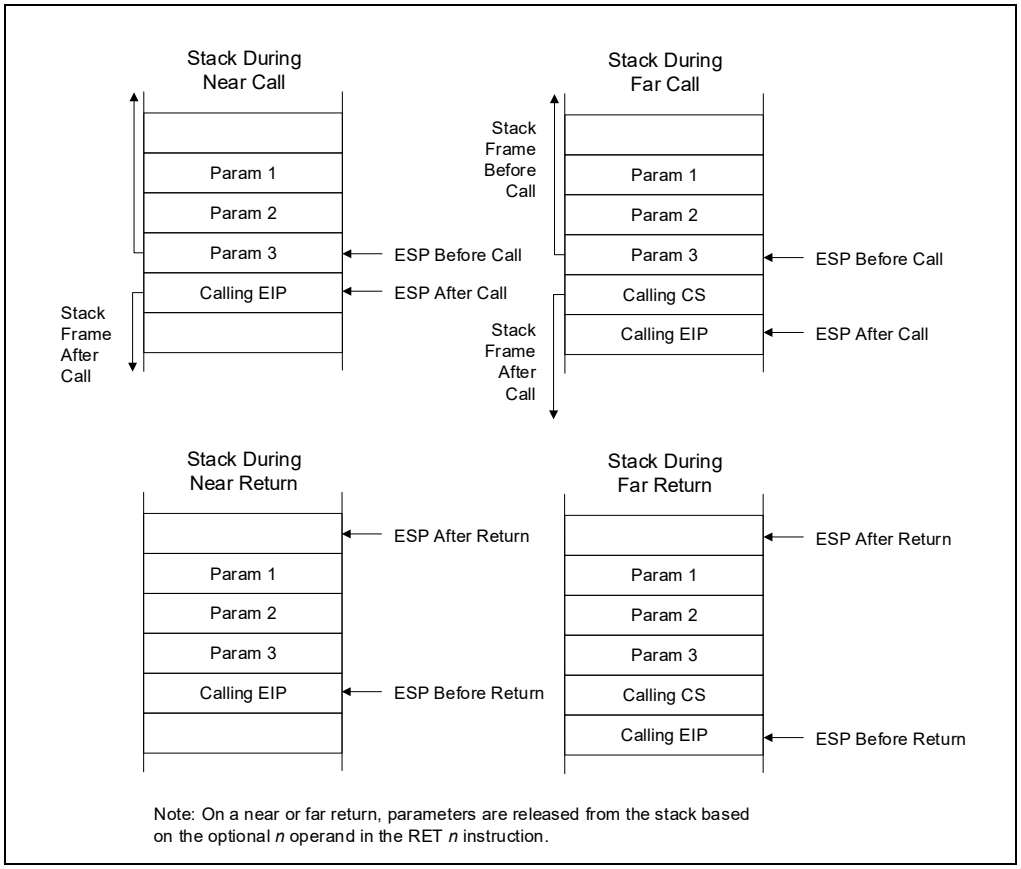


Figure 6-2. Stack on Near and Far Calls

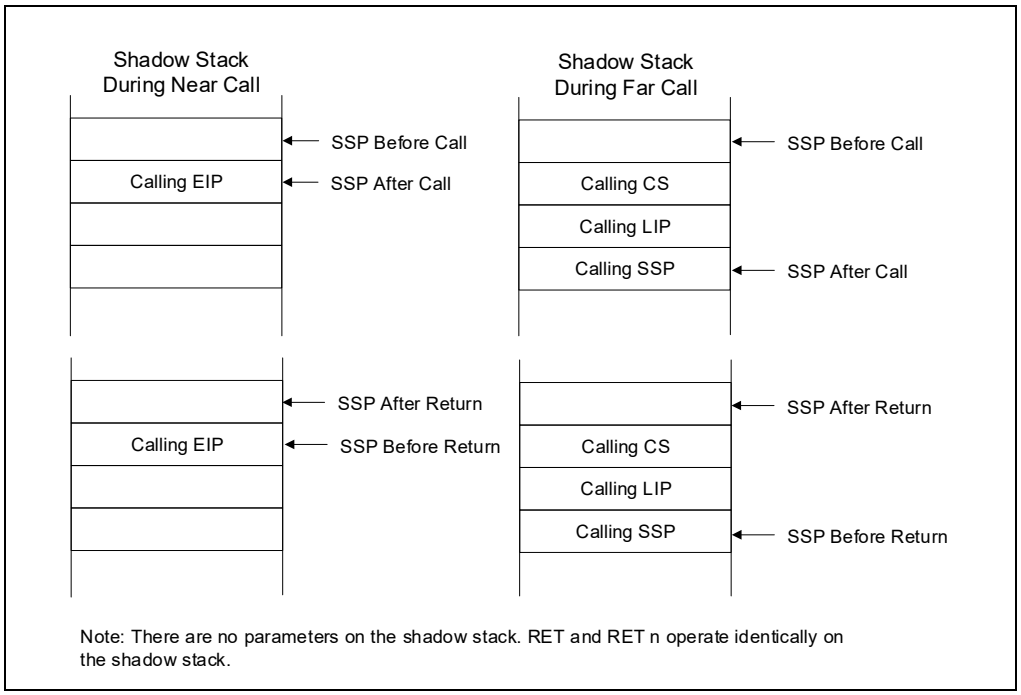


Figure 6-3. Shadow Stack on Near and Far Calls

6.4.3 Parameter Passing

Parameters can be passed between procedures in any of three ways: through general-purpose registers, in an argument list, or on the stack.

6.4.3.1 Passing Parameters Through the General-Purpose Registers

The processor does not save the state of the general-purpose registers on procedure calls. A calling procedure can thus pass up to six parameters to the called procedure by copying the parameters into any of these registers (except the ESP and EBP registers) prior to executing the CALL instruction. The called procedure can likewise pass parameters back to the calling procedure through general-purpose registers.

6.4.3.2 Passing Parameters on the Stack

To pass a large number of parameters to the called procedure, the parameters can be placed on the stack, in the stack frame for the calling procedure. Here, it is useful to use the stack-frame base pointer (in the EBP register) to make a frame boundary for easy access to the parameters.

The stack can also be used to pass parameters back from the called procedure to the calling procedure.

6.4.3.3 Passing Parameters in an Argument List

An alternate method of passing a larger number of parameters (or a data structure) to the called procedure is to place the parameters in an argument list in one of the data segments in memory. A pointer to the argument list can then be passed to the called procedure through a general-purpose register or the stack. Parameters can also be passed back to the calling procedure in this same manner.

6.4.4 Saving Procedure State Information

The processor does not save the contents of the general-purpose registers, segment registers, or the EFLAGS register on a procedure call. A calling procedure should explicitly save the values in any of the general-purpose registers that it will need when it resumes execution after a return. These values can be saved on the stack or in memory in one of the data segments.

The PUSHA and POPA instructions facilitate saving and restoring the contents of the general-purpose registers. PUSHA pushes the values in all the general-purpose registers on the stack in the following order: EAX, ECX, EDX, EBX, ESP (the value prior to executing the PUSHA instruction), EBP, ESI, and EDI. The POPA instruction pops all the register values saved with a PUSHA instruction (except the ESP value) from the stack to their respective registers.

If a called procedure changes the state of any of the segment registers explicitly, it should restore them to their former values before executing a return to the calling procedure.

If a calling procedure needs to maintain the state of the EFLAGS register, it can save and restore all or part of the register using the PUSHF/PUSHFD and POPF/POPFD instructions. The PUSHF instruction pushes the lower word of the EFLAGS register on the stack, while the PUSHFD instruction pushes the entire register. The POPF instruction pops a word from the stack into the lower word of the EFLAGS register, while the POPFD instruction pops a double word from the stack into the register.

6.4.5 Calls to Other Privilege Levels

The IA-32 architecture’s protection mechanism recognizes four privilege levels, numbered from 0 to 3, where a greater number mean less privilege. The reason to use privilege levels is to improve the reliability of operating systems. For example, Figure 6-4 shows how privilege levels can be interpreted as rings of protection.

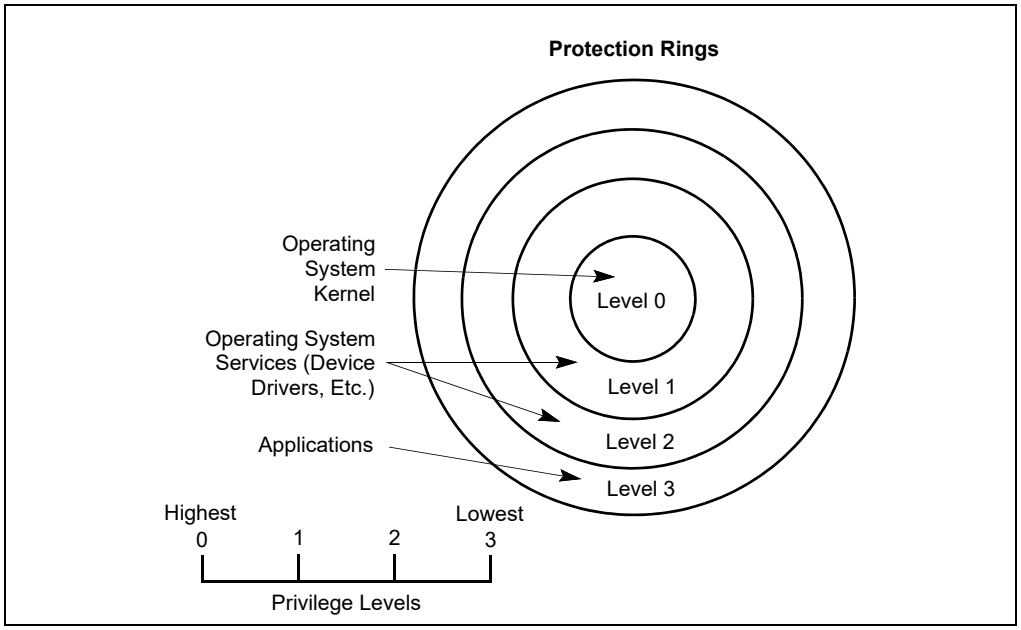


Figure 6-4. Protection Rings

In this example, the highest privilege level 0 (at the center of the diagram) is used for segments that contain the most critical code modules in the system, usually the kernel of an operating system. The outer rings (with progressively lower privileges) are used for segments that contain code modules for less critical software.

Code modules in lower privilege segments can only access modules operating at higher privilege segments by means of a tightly controlled and protected interface called a **gate**. Attempts to access higher privilege segments without going through a protection gate and without having sufficient access rights causes a general-protection exception (#GP) to be generated.

If an operating system or executive uses this multilevel protection mechanism, a call to a procedure that is in a more privileged protection level than the calling procedure is handled in a similar manner as a far call (see Section 6.4.2, "Far CALL and RET Operation"). The differences are as follows:

- The segment selector provided in the CALL instruction references a special data structure called a **call gate descriptor**. Among other things, the call gate descriptor provides the following:
  - — Access rights information.
  - — The segment selector for the code segment of the called procedure.
  - — An offset into the code segment (that is, the instruction pointer for the called procedure).
- The processor switches to a new stack to execute the called procedure. Each privilege level has its own stack. The segment selector and stack pointer for the privilege level 3 stack are stored in the SS and ESP registers, respectively, and are automatically saved when a call to a more privileged level occurs. The segment selectors and stack pointers for the privilege level 2, 1, and 0 stacks are stored in a system segment called the task state segment (TSS).

The use of a call gate and the TSS during a stack switch are transparent to the calling procedure, except when a general-protection exception is raised.

Flexible return and event delivery (FRED) defines new mechanisms for changing privilege levels. See Chapter 8, "Flexible Return and Event Delivery (FRED)," in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3. When FRED transitions are enabled, call gates cannot be used to change privilege level. An execution of CALL causes a general-protection exception (#GP) if it accesses a call gate. Similarly, an execution of RET causes a #GP if it would change privilege level. In any of these cases, the fault identifies the relevant selector in its error code.

6.4.6 CALL and RET Operation Between Privilege Levels

When making a call to a more privileged protection level, the processor does the following (see Figure 6-5):

1. Performs an access rights check (privilege check).
2. Temporarily saves (internally) the current contents of the SS, ESP, CS, and EIP registers.

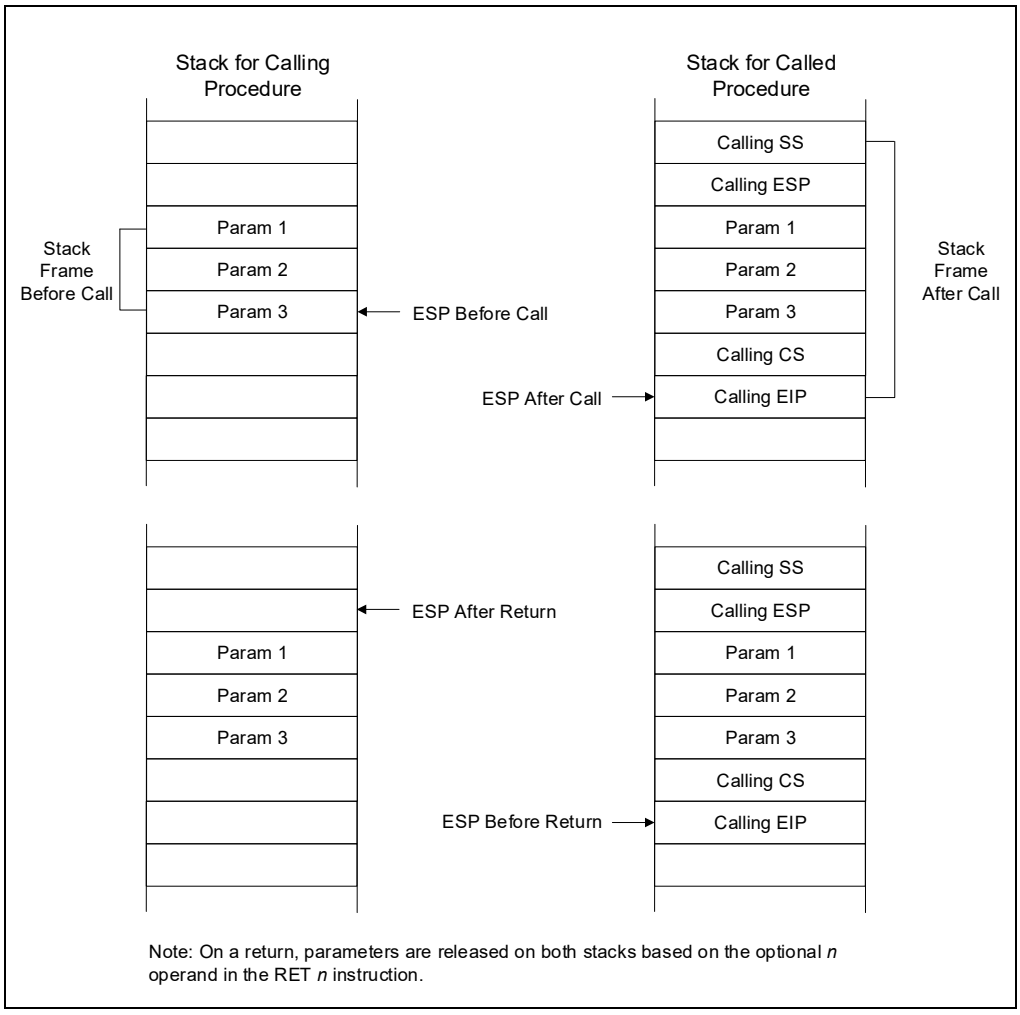


Figure 6-5. Stack Switch on a Call to a Different Privilege Level

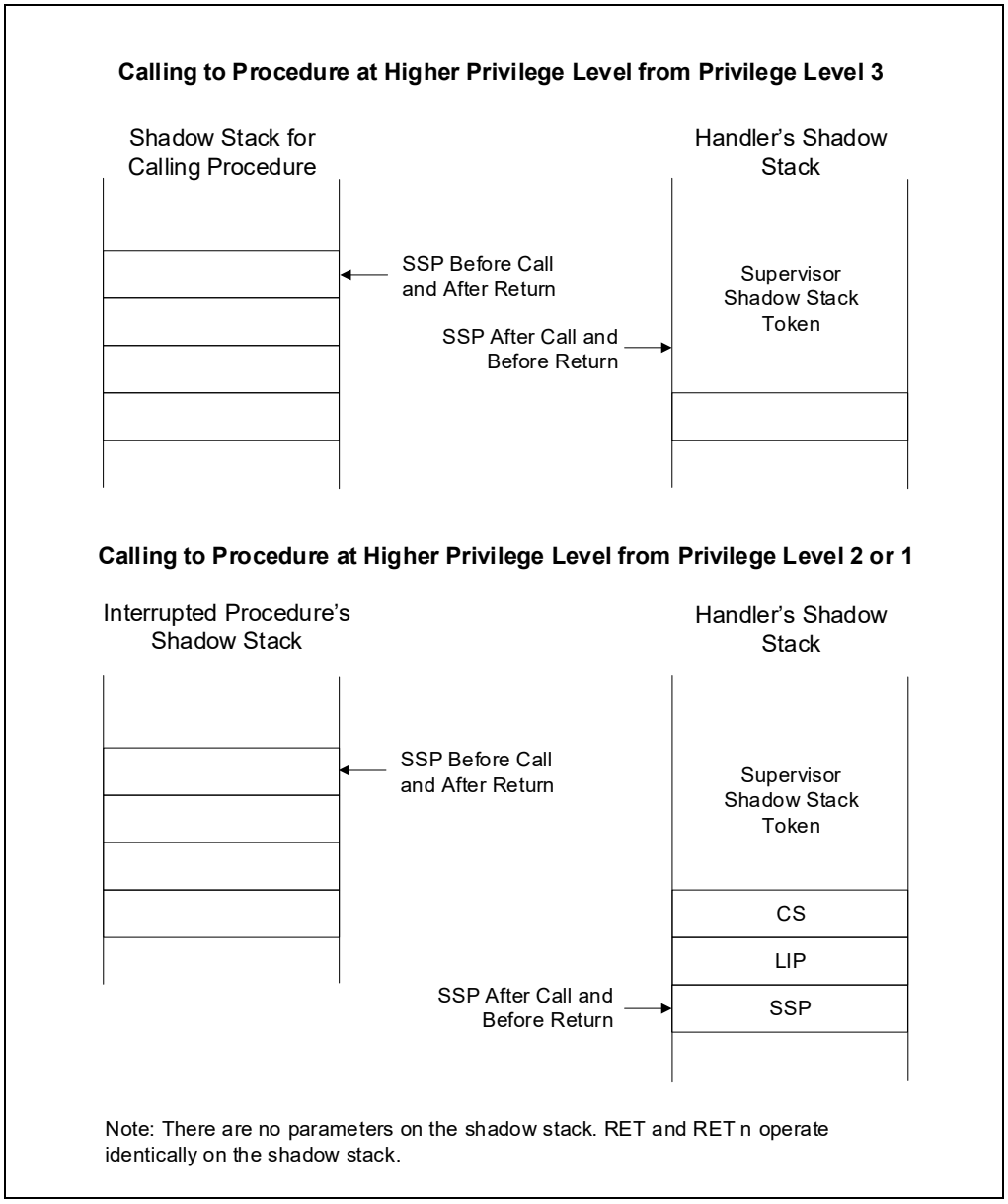


Figure 6-6. Shadow Stack Switch on a Call to a Different Privilege Level

3. Loads the segment selector and stack pointer for the new stack (that is, the stack for the privilege level being called) from the TSS into the SS and ESP registers and switches to the new stack.
4. Pushes the temporarily saved SS and ESP values for the calling procedure's stack onto the new stack.
5. Copies the parameters from the calling procedure's stack to the new stack. A value in the call gate descriptor determines how many parameters to copy to the new stack.
6. Pushes the temporarily saved CS and EIP values for the calling procedure to the new stack.

   If shadow stack is enabled at the privilege level of the calling procedure, then the processor temporarily saves the SSP of the calling procedure internally. If the calling procedure is at privilege level 3, the SSP of the calling procedure is also saved into the IA32_PL3_SSP MSR.

If shadow stack is enabled at the privilege level of the called procedure, then the SSP for the called procedure is obtained from one of the MSRs listed below, depending on the target privilege level. The SSP obtained is then verified to ensure it points to a valid supervisor shadow stack that is not currently active by verifying a supervisor shadow stack token at the address pointed to by the SSP. The operations performed to verify and acquire the supervisor shadow stack token by making it busy are as described in Section 18.2.3 of the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 1.

— IA32_PL2_SSP if transitioning to ring 2.

— IA32_PL1_SSP if transitioning to ring 1.

— IA32_PL0_SSP if transitioning to ring 0.

If shadow stack is enabled at the privilege level of the called procedure and the calling procedure was not at privilege level 3, then the processor pushes the temporarily saved CS, LIP (CS.base + EIP), and SSP of the calling procedure to the new shadow stack.[1]

7. Loads the segment selector for the new code segment and the new instruction pointer from the call gate into the CS and EIP registers, respectively.
8. Begins execution of the called procedure at the new privilege level.

When executing a return from the privileged procedure, the processor performs these actions:

1. Performs a privilege check.
2. Restores the CS and EIP registers to their values prior to the call.

   If shadow stack is enabled at the current privilege level:

   — Causes a control protection exception (#CP(FAR-RET/IRET)) if SSP is not aligned to 8 bytes.

   — If the privilege level of the procedure being returned to is less than 3 (returning to supervisor mode):

   • Compares the values on shadow stack at address SSP+8 (the LIP) and SSP+16 (the CS) to the CS and (CS.base + EIP) popped from the stack and causes a control protection exception (#CP(FAR-RET/IRET)) if they do not match.

   • Temporarily saves the top-of-stack value (the SSP of the procedure being returned to) internally.

   — If a busy supervisor shadow stack token is present at address SSP+24, then marks the token free using operations described in Section 18.2.3 of the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 1.

   — If the privilege level of the procedure being returned to is less than 3 (returning to supervisor mode), restores the SSP register from the internally saved value.

   — If the privilege level of the procedure being returned to is 3 (returning to user mode) and shadow stack is enabled at privilege level 3, then restores the SSP register with value of IA32_PL3_SSP MSR.
3. If the RET instruction has an optional *n* argument, increments the stack pointer by the number of bytes specified with the *n* operand to release parameters from the stack. If the call gate descriptor specifies that one or more parameters be copied from one stack to the other, a RET *n* instruction must be used to release the parameters from both stacks. Here, the *n* operand specifies the number of bytes occupied on each stack by the parameters. On a return, the processor increments ESP by *n* for each stack to step over (effectively remove) these parameters from the stacks.
4. Restores the SS and ESP registers to their values prior to the call, which causes a switch back to the stack of the calling procedure.
5. If the RET instruction has an optional *n* argument, increments the stack pointer by the number of bytes specified with the *n* operand to release parameters from the stack (see explanation in step 3).
6. Resumes execution of the calling procedure.

See Chapter 6, "Protection," in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3A, for detailed information on calls to privileged levels and the call gate descriptor.

1. If any of these pushes leads to an exception or a VM exit, the supervisor shadow-stack token remains busy.

6.4.7 Branch Functions in 64-Bit Mode

The 64-bit extensions expand branching mechanisms to accommodate branches in 64-bit linear-address space. These are:

- Near-branch semantics are redefined in 64-bit mode.
- In 64-bit mode and compatibility mode, 64-bit call-gate descriptors for far calls are available.

In 64-bit mode, the operand size for all near branches (CALL, RET, JCC, JCXZ, JMP, and LOOP) is forced to 64 bits. These instructions update the 64-bit RIP without the need for a REX operand-size prefix.

The following aspects of near branches are controlled by the effective operand size:

- Truncation of the size of the instruction pointer.
- Size of a stack pop or push, due to a CALL or RET.
- Size of a stack-pointer increment or decrement, due to a CALL or RET.
- Indirect-branch operand size.

In 64-bit mode, all of the above actions are forced to 64 bits regardless of operand size prefixes (operand size prefixes are silently ignored). However, the displacement field for relative branches is still limited to 32 bits and the address size for near branches is not forced in 64-bit mode.

Address sizes affect the size of RCX used for JCXZ and LOOP; they also impact the address calculation for memory indirect branches. Such addresses are 64 bits by default; but they can be overridden to 32 bits by an address size prefix.

Software typically uses far branches to change privilege levels. The legacy IA-32 architecture provides the call-gate mechanism to allow software to branch from one privilege level to another, although call gates can also be used for branches that do not change privilege levels. When call gates are used, the selector portion of the direct or indirect pointer references a gate descriptor (the offset in the instruction is ignored). The offset to the destination's code segment is taken from the call-gate descriptor.

64-bit mode redefines the type value of a 32-bit call-gate descriptor type to a 64-bit call gate descriptor and expands the size of the 64-bit descriptor to hold a 64-bit offset. The 64-bit mode call-gate descriptor allows far branches that reference any location in the supported linear-address space. These call gates also hold the target code selector (CS), allowing changes to privilege level and default size as a result of the gate transition.

Because immediates are generally specified up to 32 bits, the only way to specify a full 64-bit absolute RIP in 64-bit mode is with an indirect branch. For this reason, direct far branches are eliminated from the instruction set in 64-bit mode.

64-bit mode also expands the semantics of the SYSENTER and SYSEXIT instructions so that the instructions operate within a 64-bit memory space. The mode also introduces two new instructions: SYSCALL and SYSRET (which are valid only in 64-bit mode). For details, see "SYSENTER—Fast System Call," "SYSEXIT—Fast Return from Fast System Call," "SYSCALL—Fast System Call," and "SYSRET—Return From Fast System Call" in Chapter 4, "Instruction Set Reference, M-U," of the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 2B.

6.5 INTERRUPTS AND EXCEPTIONS

The processor provides two mechanisms for interrupting program execution, interrupts, and exceptions:

- An **interrupt** is an asynchronous event that is typically triggered by an I/O device.
- An **exception** is a synchronous event that is generated when the processor detects one or more predefined conditions while executing an instruction. The IA-32 architecture specifies three classes of exceptions: faults, traps, and aborts.

The processor responds to interrupts and exceptions in essentially the same way. When an interrupt or exception is signaled, the processor halts execution of the current program or task and switches to a handler procedure that has been written specifically to handle the interrupt or exception condition. The processor accesses the handler procedure through an entry in the interrupt descriptor table (IDT) or through FRED event delivery. When the handler has completed handling the interrupt or exception, program control is returned to the interrupted program or task.

The operating system, executive, and/or device drivers normally handle interrupts and exceptions independently from application programs or tasks. Application programs can, however, access the interrupt and exception handlers incorporated in an operating system or executive through assembly-language calls. The remainder of this section gives a brief overview of the processor's interrupt and exception handling mechanism. See Chapter 7, "Interrupt and Exception Handling," in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3A, for a description of this mechanism.

The IA-32 Architecture defines 18 predefined interrupts and exceptions and 224 user defined interrupts, which are associated with entries in the IDT. Each interrupt and exception in the IDT is identified with a number, called a **vector**. Table 6-1 lists the interrupts and exceptions with entries in the IDT and their respective vectors. Vectors 0 through 8, 10 through 14, and 16 through 19 are the predefined interrupts and exceptions; vectors 32 through 255 are for software-defined interrupts, which are for either **software interrupts** or **maskable hardware interrupts**.

When FRED transitions are enabled, the processor does not use the IDT. Instead, interrupts and exceptions are delivered using **FRED event delivery**. FRED event delivery saves an event's vector on the stack, along with the event type (e.g., interrupt or exception). The event type allows software to distinguish interrupts and exceptions with the same vector.

Note that the processor defines several additional interrupts that do not point to entries in the IDT and are not delivered using FRED event delivery; the most notable of these interrupts is the SMI interrupt. See Chapter 7, "Interrupt and Exception Handling," in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3A, for more information about the interrupts and exceptions.

When the processor detects an interrupt or exception, it does one of the following things:

- Executes an implicit call to a handler procedure.
- Executes an implicit call to a handler task.

6.5.1 Call and Return Operation for Interrupt or Exception Handling Procedures

This section describes **IDT event delivery**, a method for calling OS handlers for interrupts and exceptions. When FRED transitions are enabled, FRED event delivery is used instead. FRED event delivery is detailed in Section 8.3, "FRED Event Delivery" in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3.

A call to an interrupt or exception handler procedure is similar to a procedure call to another protection level (see Section 6.4.6, "CALL and RET Operation Between Privilege Levels"). Here, the vector references one of two kinds of gates in the IDT: an **interrupt gate** or a **trap gate**. Interrupt and trap gates are similar to call gates in that they provide the following information:

- Access rights information
- The segment selector for the code segment that contains the handler procedure
- An offset into the code segment to the first instruction of the handler procedure

The difference between an interrupt gate and a trap gate is as follows. If an interrupt or exception handler is called through an interrupt gate, the processor clears the interrupt enable (IF) flag in the EFLAGS register to prevent subsequent interrupts from interfering with the execution of the handler. When a handler is called through a trap gate, the state of the IF flag is not changed.

Table 6-1. Exceptions and Interrupts

Vector	Mnemonic	Description	Source
0	#DE	Divide Error	DIV and IDIV instructions.
1	#DB	Debug	Any code or data reference.
2		NMI Interrupt	Non-maskable external interrupt.
3	#BP	Breakpoint	INT3 instruction.
4	#OF	Overflow	INTO instruction.
5	#BR	BOUND Range Exceeded	BOUND instruction.
6	#UD	Invalid Opcode (Undefined Opcode)	UD instruction or reserved opcode.

Table 6-1. Exceptions and Interrupts (Contd.)

Vector	Mnemonic	Description	Source
7	#NM	Device Not Available (No Math Coprocessor)	Floating-point or WAIT/FWAIT instruction.
8	#DF	Double Fault	Any instruction that can generate an exception, an NMI, or an INTR.
9	#MF	CoProcessor Segment Overrun (reserved)	Floating-point instruction.[1]
10	#TS	Invalid TSS	Task switch or TSS access.
11	#NP	Segment Not Present	Loading segment registers or accessing system segments.
12	#SS	Stack Segment Fault	Stack operations and SS register loads.
13	#GP	General Protection	Any memory reference and other protection checks.
14	#PF	Page Fault	Any memory reference.
15		Reserved	
16	#MF	Floating-Point Error (Math Fault)	Floating-point or WAIT/FWAIT instruction.
17	#AC	Alignment Check	Any data reference in memory.[2]
18	#MC	Machine Check	Error codes (if any) and source are model dependent.[3]
19	#XM	SIMD Floating-Point Exception	SIMD Floating-Point Instruction[4]
20	#VE	Virtualization Exception	EPT violations[5]
21	#CP	Control Protection Exception	The RET, IRET, RSTORSSP, and SETSSBSY instructions can generate this exception. When CET indirect branch tracking is enabled, this exception can be generated due to a missing ENDBRANCH instruction at the target of an indirect call or jump.
22-31		Reserved	
32-255		Maskable Interrupts	External interrupt from INTR pin or INT *n* instruction.

NOTES:

1. IA-32 processors after the Intel386 processor do not generate this exception.
2. This exception was introduced in the Intel486 processor.
3. This exception was introduced in the Pentium processor and enhanced in the P6 family processors.
4. This exception was introduced in the Pentium III processor.
5. This exception can occur only on processors that support the 1-setting of the "EPT-violation #VE" VM-execution control.

If the code segment for the handler procedure has the same privilege level as the currently executing program or task, the handler procedure uses the current stack; if the handler executes at a more privileged level, the processor switches to the stack for the handler's privilege level.

If no stack switch occurs, the processor does the following when calling an interrupt or exception handler (see Figure 6-7):

1. Pushes the current contents of the EFLAGS, CS, and EIP registers (in that order) on the stack.

   If shadow stack is enabled:

   a. Temporarily saves the current value of the SSP register internally.

   b. Pushes the current value of the CS register on the shadow stack.

   c. Pushes the current value of LIP (CS.base + EIP) on the shadow stack.

   d. Pushes the temporarily saved SSP value on the shadow stack.
2. Pushes an error code (if appropriate) on the stack.
3. Loads the segment selector for the new code segment and the new instruction pointer (from the interrupt gate or trap gate) into the CS and EIP registers, respectively.
4. If the call is through an interrupt gate, clears the IF flag in the EFLAGS register.

5. Begins execution of the handler procedure.

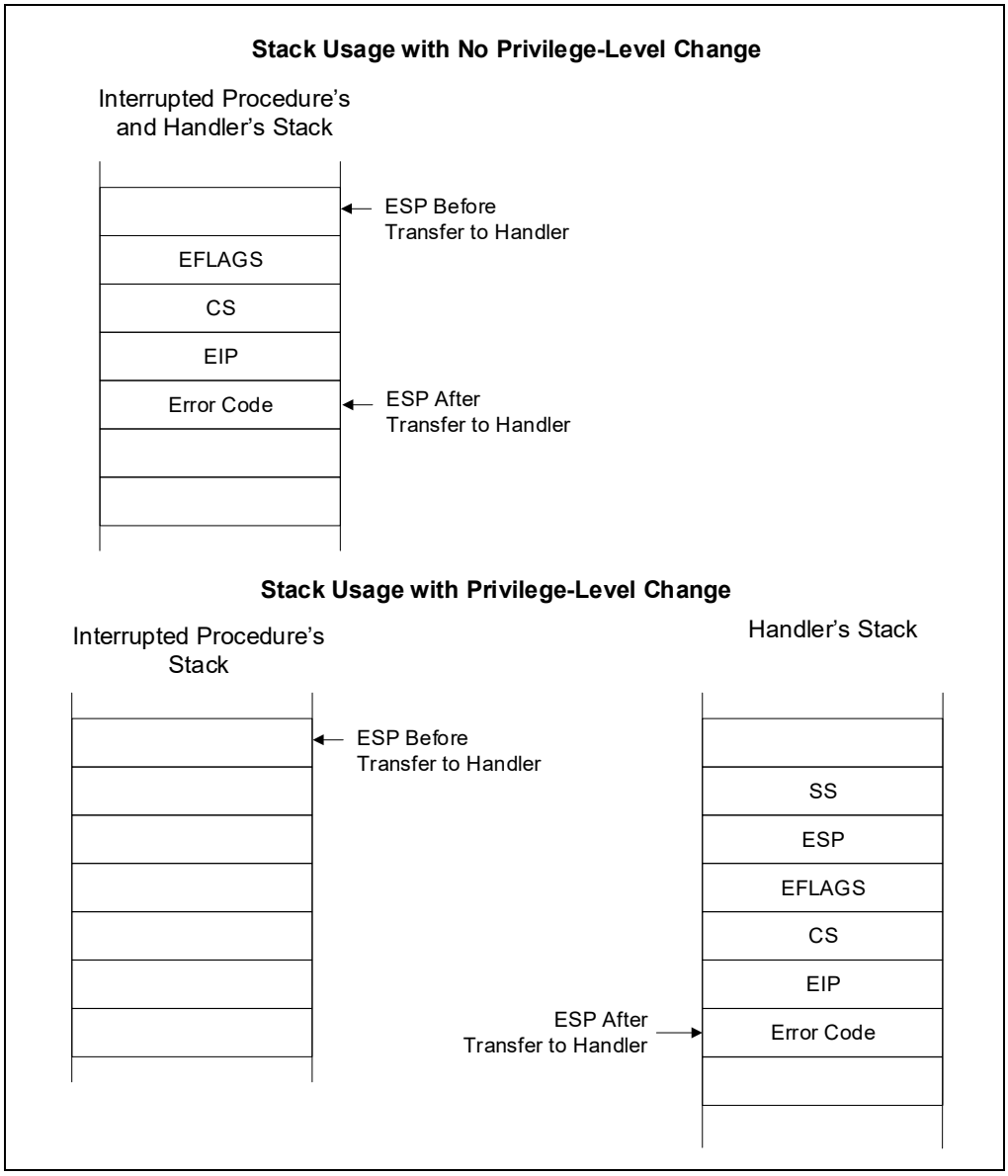


Figure 6-7. Stack Usage on Transfers to Interrupt and Exception Handling Routines

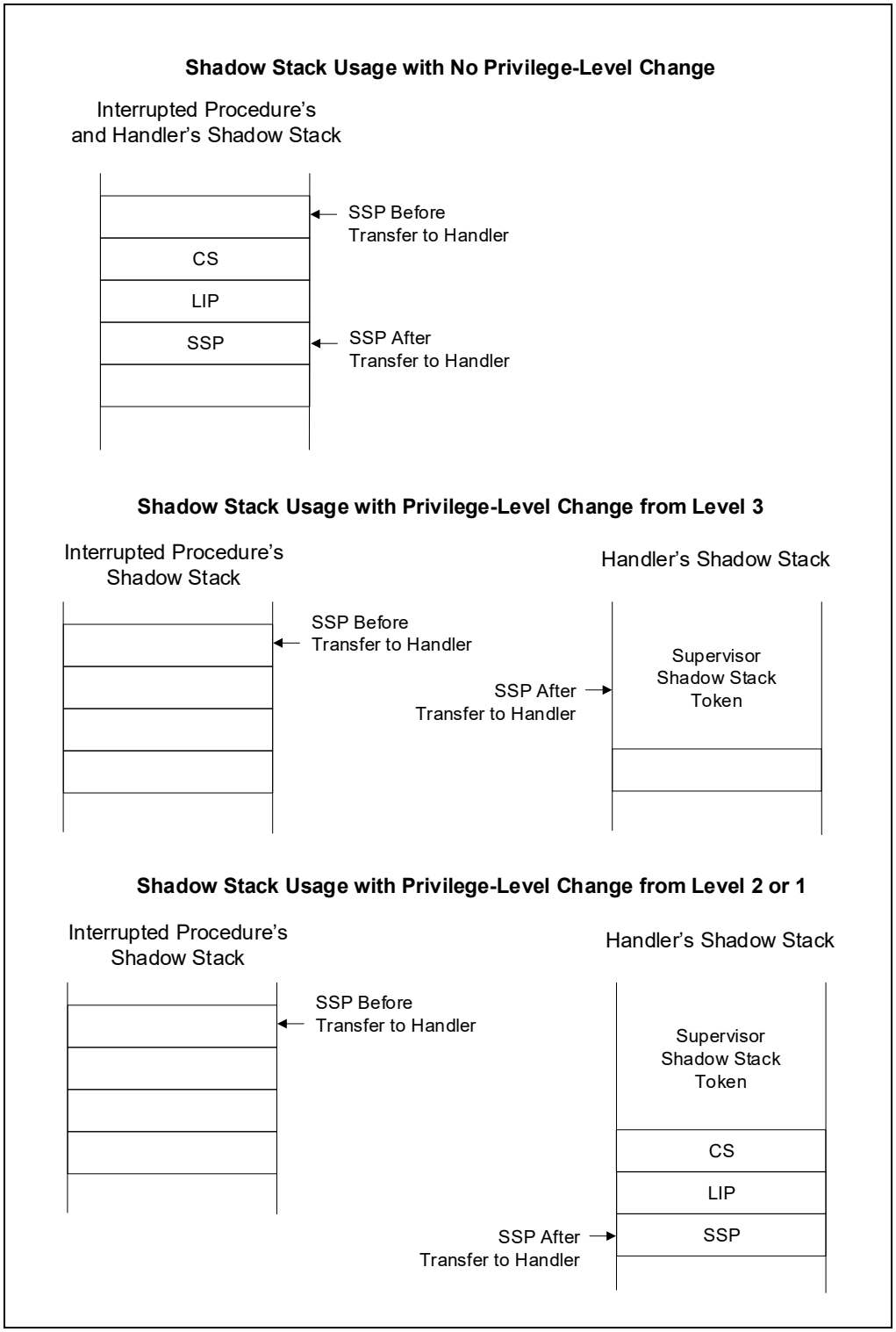


Figure 6-8. Shadow Stack Usage on Transfers to Interrupt and Exception Handling Routines

If a stack switch does occur, the processor does the following:

1. Temporarily saves (internally) the current contents of the SS, ESP, EFLAGS, CS, and EIP registers.
2. Loads the segment selector and stack pointer for the new stack (that is, the stack for the privilege level being called) from the TSS into the SS and ESP registers and switches to the new stack.
3. Pushes the temporarily saved SS, ESP, EFLAGS, CS, and EIP values for the interrupted procedure's stack onto the new stack.

   If shadow stack is enabled at the privilege level of the interrupted procedure, then the processor temporarily saves the SSP of the interrupted procedure internally. If the interrupted procedure is at privilege level 3, the SSP of the interrupted procedure is also saved into the IA32_PL3_SSP MSR.

   If shadow stack is enabled at the privilege level being called, then the SSP for the called privilege level is obtained from one of the MSRs listed below, depending on the target privilege level. The SSP obtained is then verified to ensure it points to a valid supervisor shadow stack that is not currently active by verifying a supervisor shadow stack token at the address pointed to by the SSP. The operations performed to verify and acquire the supervisor shadow stack token by making it busy are as described in Section 18.2.3 of the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 1.

   — IA32_PL2_SSP if transitioning to ring 2.

   — IA32_PL1_SSP if transitioning to ring 1.

   — IA32_PL0_SSP if transitioning to ring 0.

   If shadow stack is enabled at the privilege level being called and the interrupted procedure was not at privilege level 3, then the processor pushes the temporarily saved CS, LIP (CS.base + EIP), and SSP of the interrupted procedure to the new shadow stack.[1]
4. Pushes an error code on the new stack (if appropriate).
5. Loads the segment selector for the new code segment and the new instruction pointer (from the interrupt gate or trap gate) into the CS and EIP registers, respectively.
6. If the call is through an interrupt gate, clears the IF flag in the EFLAGS register.
7. Begins execution of the handler procedure at the new privilege level.

A return from an interrupt or exception handler is initiated with the IRET instruction. The IRET instruction is similar to the far RET instruction, except that it also restores the contents of the EFLAGS register for the interrupted procedure. When executing a return from an interrupt or exception handler from the same privilege level as the interrupted procedure, the processor performs these actions:

1. Restores the CS and EIP registers to their values prior to the interrupt or exception.

   If shadow stack is enabled:

   a. Compares the values on the shadow stack at address SSP+8 (the LIP) and SSP+16 (the CS) to the CS and (CS.base + EIP) popped from the stack, and causes a control protection exception (#CP(FAR-RET/IRET)) if they do not match.

   b. Pops the top-of-stack value (the SSP prior to the interrupt or exception) from the shadow stack into the SSP register.
2. Restores the EFLAGS register.
3. Increments the stack pointer appropriately.
4. Resumes execution of the interrupted procedure.

When executing a return from an interrupt or exception handler from a different privilege level than the interrupted procedure, the processor performs these actions:

1. Performs a privilege check.
2. Restores the CS and EIP registers to their values prior to the interrupt or exception.
3. Restores the EFLAGS register.

---

1. If any of these pushes leads to an exception or a VM exit, the supervisor shadow-stack token remains busy.

If shadow stack is enabled at the current privilege level:

— If SSP is not aligned to 8 bytes, then causes a control protection exception (#CP(FAR-RET/IRET)).

— If the privilege level of the procedure being returned to is less than 3 (returning to supervisor mode):

- Compares the values on the shadow stack at address SSP+8 (the LIP) and SSP+16 (the CS) to the CS and (CS.base + EIP) popped from the stack, and causes a control protection exception (#CP(FAR-RET/IRET)) if they do not match.
- Temporarily saves the top-of-stack value (the SSP of the procedure being returned to) internally.

— If a busy supervisor shadow stack token is present at address SSP+24, then marks the token free using operations described in Section 18.2.3 of the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 1.

— If the privilege level of the procedure being returned to is less than 3 (returning to supervisor mode), restores the SSP register from the internally saved value.

— If the privilege level of the procedure being returned to is 3 (returning to user mode) and shadow stack is enabled at privilege level 3, then restores the SSP register with the value of the IA32_PL3_SSP MSR.

4. Restores the SS and ESP registers to their values prior to the interrupt or exception, resulting in a stack switch back to the stack of the interrupted procedure.
5. Resumes execution of the interrupted procedure.

6.5.2 Calls to Interrupt or Exception Handler Tasks

Interrupt and exception handler routines can also be executed in a separate task. Here, an interrupt or exception causes a task switch to a handler task. The handler task is given its own address space and (optionally) can execute at a higher protection level than application programs or tasks.

The switch to the handler task is accomplished with an implicit task call that references a **task gate descriptor**. The task gate provides access to the address space for the handler task. As part of the task switch, the processor saves complete state information for the interrupted program or task. Upon returning from the handler task, the state of the interrupted program or task is restored and execution continues. See Chapter 7, "Interrupt and Exception Handling," in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3A, for more information on handling interrupts and exceptions through handler tasks.

6.5.3 Interrupt and Exception Handling in Real-Address Mode

When operating in real-address mode, the processor responds to an interrupt or exception with an implicit far call to an interrupt or exception handler. The processor uses the interrupt or exception vector as an index into an interrupt table. The interrupt table contains instruction pointers to the interrupt and exception handler procedures.

The processor saves the state of the EFLAGS register, the EIP register, the CS register, and an optional error code on the stack before switching to the handler procedure.

A return from the interrupt or exception handler is carried out with the IRET instruction.

See Chapter 23, "8086 Emulation," in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3B, for more information on handling interrupts and exceptions in real-address mode.

6.5.4 INT *n*, INTO, INT3, INT1, and BOUND Instructions

The INT *n*, INTO, INT3, and BOUND instructions allow a program or task to explicitly call an interrupt or exception handler. The INT *n* instruction (opcode CD) uses a vector as an argument, which allows a program to call any interrupt handler.

The INTO instruction (opcode CE) explicitly calls the overflow exception (#OF) handler if the overflow flag (OF) in the EFLAGS register is set. The OF flag indicates overflow on arithmetic instructions, but it does not automatically raise an overflow exception. An overflow exception can only be raised explicitly in either of the following ways:

- Execute the INTO instruction.
- Test the OF flag and execute the INT *n* instruction with an argument of 4 (the vector of the overflow exception) if the flag is set.

Both the methods of dealing with overflow conditions allow a program to test for overflow at specific places in the instruction stream.

The INT3 instruction (opcode CC) explicitly calls the breakpoint exception (#BP) handler. Similarly, the INT1 instruction (opcode F1) explicitly calls the debug exception (#DB) handler.[1]

The BOUND instruction explicitly calls the BOUND-range exceeded exception (#BR) handler if an operand is found to be not within predefined boundaries in memory. This instruction is provided for checking references to arrays and other data structures. Like the overflow exception, the BOUND-range exceeded exception can only be raised explicitly with the BOUND instruction or the INT *n* instruction with an argument of 5 (the vector of the bounds-check exception). The processor does not implicitly perform bounds checks and raise the BOUND-range exceeded exception.

6.5.5 Handling Floating-Point Exceptions

When operating on individual or packed floating-point values, the IA-32 architecture supports a set of six floating-point exceptions. These exceptions can be generated during operations performed by the x87 FPU instructions or by SSE/SSE2/SSE3 instructions. When an x87 FPU instruction (including the FISTTP instruction in SSE3) generates one or more of these exceptions, it in turn generates floating-point error exception (#MF); when an SSE/SSE2/SSE3 instruction generates a floating-point exception, it in turn generates SIMD floating-point exception (#XM).

See the following sections for further descriptions of the floating-point exceptions, how they are generated, and how they are handled:

- Section 4.9.1, “Floating-Point Exception Conditions,” and Section 4.9.3, “Typical Actions of a Floating-Point Exception Handler.”
- Section 8.4, “x87 FPU Floating-Point Exception Handling,” and Section 8.5, “x87 FPU Floating-Point Exception Conditions.”
- Section 11.5.1, “SIMD Floating-Point Exceptions.”
- Interrupt Behavior.

6.5.6 Interrupt and Exception Behavior in 64-Bit Mode

64-bit extensions expand the legacy IA-32 interrupt-processing and exception-processing mechanism to allow support for 64-bit operating systems and applications. Changes include:

- All interrupt handlers pointed to by the IDT are 64-bit code (does not apply to the SMI handler).
- The size of interrupt-stack pushes is fixed at 64 bits. The processor uses 8-byte, zero extended stores.
- The stack pointer (SS:RSP) is pushed unconditionally on interrupts. In legacy environments, this push is conditional and based on a change in current privilege level (CPL).
- The new SS is set to NULL if there is a change in CPL.
- IRET behavior changes.
- There is a new interrupt stack-switch mechanism and a new interrupt shadow stack-switch mechanism.
- The alignment of interrupt stack frame is different.

The above items apply to IDT event delivery. When FRED transitions are enabled, FRED event delivery is used instead. See Section 8.3, “FRED Event Delivery” in the Intel® 64 and IA-32 Architectures Software Developer’s Manual, Volume 3.

1. Hardware vendors may use the INT1 instruction for hardware debug. For that reason, Intel recommends software vendors instead use the INT3 instruction for software breakpoints.

6.6 PROCEDURE CALLS FOR BLOCK-STRUCTURED LANGUAGES

The IA-32 architecture supports an alternate method of performing procedure calls with the ENTER (enter procedure) and LEAVE (leave procedure) instructions. These instructions automatically create and release, respectively, stack frames for called procedures. The stack frames have predefined spaces for local variables and the necessary pointers to allow coherent returns from called procedures. They also allow scope rules to be implemented so that procedures can access their own local variables and some number of other variables located in other stack frames.

ENTER and LEAVE offer two benefits:

- They provide machine-language support for implementing block-structured languages, such as C and Pascal.
- They simplify procedure entry and exit in compiler-generated code.

6.6.1 ENTER Instruction

The ENTER instruction creates a stack frame compatible with the scope rules typically used in block-structured languages. In block-structured languages, the scope of a procedure is the set of variables to which it has access. The rules for scope vary among languages. They may be based on the nesting of procedures, the division of the program into separately compiled files, or some other modularization scheme.

ENTER has two operands. The first specifies the number of bytes to be reserved on the stack for dynamic storage for the procedure being called. Dynamic storage is the memory allocated for variables created when the procedure is called, also known as automatic variables. The second parameter is the lexical nesting level (from 0 to 31) of the procedure. The nesting level is the depth of a procedure in a hierarchy of procedure calls. The lexical level is unrelated to either the protection privilege level or to the I/O privilege level of the currently running program or task.

ENTER, in the following example, allocates 2 Kbytes of dynamic storage on the stack and sets up pointers to two previous stack frames in the stack frame for this procedure:

```
ENTER 2048,3
```

The lexical nesting level determines the number of stack frame pointers to copy into the new stack frame from the preceding frame. A stack frame pointer is a doubleword used to access the variables of a procedure. The set of stack frame pointers used by a procedure to access the variables of other procedures is called the display. The first doubleword in the display is a pointer to the previous stack frame. This pointer is used by a LEAVE instruction to undo the effect of an ENTER instruction by discarding the current stack frame.

After the ENTER instruction creates the display for a procedure, it allocates the dynamic local variables for the procedure by decrementing the contents of the ESP register by the number of bytes specified in the first parameter. This new value in the ESP register serves as the initial top-of-stack for all PUSH and POP operations within the procedure.

To allow a procedure to address its display, the ENTER instruction leaves the EBP register pointing to the first doubleword in the display. Because stacks grow down, this is actually the doubleword with the highest address in the display. Data manipulation instructions that specify the EBP register as a base register automatically address locations within the stack segment instead of the data segment.

The ENTER instruction can be used in two ways: nested and non-nested. If the lexical level is 0, the non-nested form is used. The non-nested form pushes the contents of the EBP register on the stack, copies the contents of the ESP register into the EBP register, and subtracts the first operand from the contents of the ESP register to allocate dynamic storage. The non-nested form differs from the nested form in that no stack frame pointers are copied. The nested form of the ENTER instruction occurs when the second parameter (lexical level) is not zero.

The following pseudo code shows the formal definition of the ENTER instruction. STORAGE is the number of bytes of dynamic storage to allocate for local variables, and LEVEL is the lexical nesting level.

```
PUSH EBP;
FRAME_PTR := ESP;
IF LEVEL > 0
    THEN
        DO (LEVEL – 1) times
            EBP := EBP – 4;
            PUSH Pointer(EBP); (* doubleword pointed to by EBP *)
        OD;
    PUSH FRAME_PTR;
FI;
EBP := FRAME_PTR;
ESP := ESP – STORAGE;
```

The main procedure (in which all other procedures are nested) operates at the highest lexical level, level 1. The first procedure it calls operates at the next deeper lexical level, level 2. A level 2 procedure can access the variables of the main program, which are at fixed locations specified by the compiler. In the case of level 1, the ENTER instruction allocates only the requested dynamic storage on the stack because there is no previous display to copy.

A procedure that calls another procedure at a lower lexical level gives the called procedure access to the variables of the caller. The ENTER instruction provides this access by placing a pointer to the calling procedure's stack frame in the display.

A procedure that calls another procedure at the same lexical level should not give access to its variables. In this case, the ENTER instruction copies only that part of the display from the calling procedure which refers to previously nested procedures operating at higher lexical levels. The new stack frame does not include the pointer for addressing the calling procedure’s stack frame.

The ENTER instruction treats a re-entrant procedure as a call to a procedure at the same lexical level. In this case, each succeeding iteration of the re-entrant procedure can address only its own variables and the variables of the procedures within which it is nested. A re-entrant procedure always can address its own variables; it does not require pointers to the stack frames of previous iterations.

By copying only the stack frame pointers of procedures at higher lexical levels, the ENTER instruction makes certain that procedures access only those variables of higher lexical levels, not those at parallel lexical levels (see Figure 6-9).

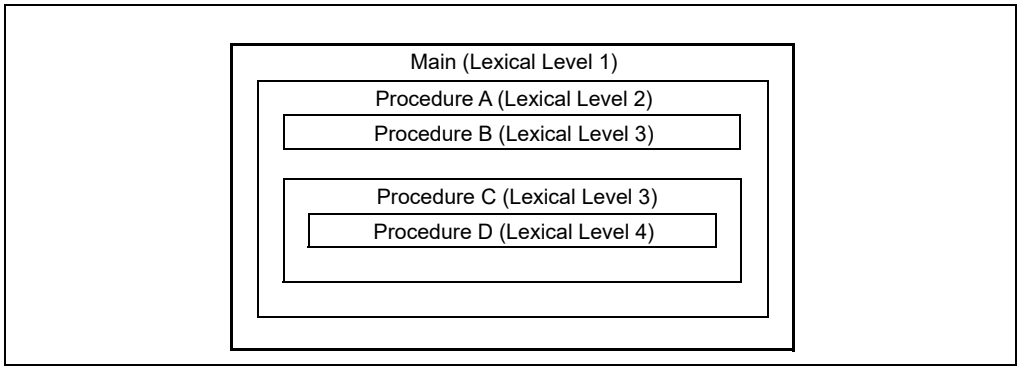


Figure 6-9. Nested Procedures

Block-structured languages can use the lexical levels defined by ENTER to control access to the variables of nested procedures. In Figure 6-9, for example, if procedure A calls procedure B which, in turn, calls procedure C, then procedure C will have access to the variables of the MAIN procedure and procedure A, but not those of procedure B because they are at the same lexical level. The following definition describes the access to variables for the nested procedures in Figure 6-9.

1. MAIN has variables at fixed locations.
2. Procedure A can access only the variables of MAIN.

3. Procedure B can access only the variables of procedure A and MAIN. Procedure B cannot access the variables of procedure C or procedure D.
4. Procedure C can access only the variables of procedure A and MAIN. Procedure C cannot access the variables of procedure B or procedure D.
5. Procedure D can access the variables of procedure C, procedure A, and MAIN. Procedure D cannot access the variables of procedure B.

In Figure 6-10, an ENTER instruction at the beginning of the MAIN procedure creates three doublewords of dynamic storage for MAIN, but copies no pointers from other stack frames. The first doubleword in the display holds a copy of the last value in the EBP register before the ENTER instruction was executed. The second doubleword holds a copy of the contents of the EBP register following the ENTER instruction. After the instruction is executed, the EBP register points to the first doubleword pushed on the stack, and the ESP register points to the last doubleword in the stack frame.

When MAIN calls procedure A, the ENTER instruction creates a new display (see Figure 6-11). The first doubleword is the last value held in MAIN's EBP register. The second doubleword is a pointer to MAIN's stack frame which is copied from the second doubleword in MAIN's display. This happens to be another copy of the last value held in MAIN’s EBP register. Procedure A can access variables in MAIN because MAIN is at level 1.

Therefore the base address for the dynamic storage used in MAIN is the current address in the EBP register, plus four bytes to account for the saved contents of MAIN’s EBP register. All dynamic variables for MAIN are at fixed, positive offsets from this value.

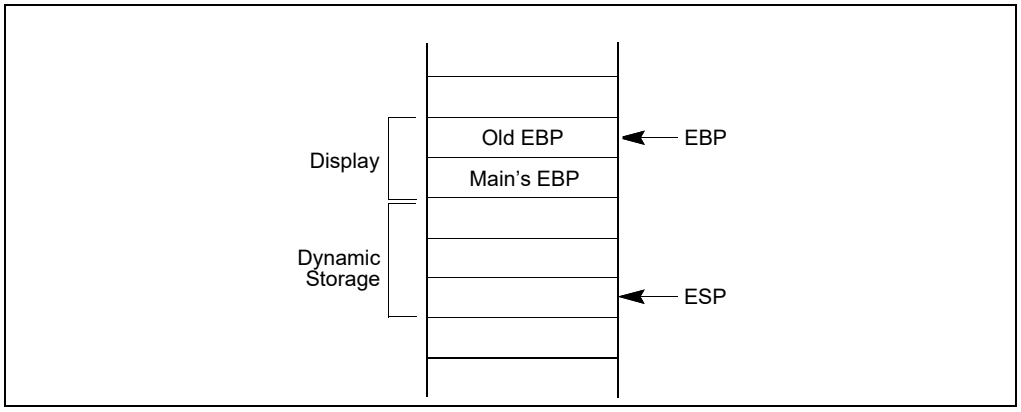


Figure 6-10. Stack Frame After Entering the MAIN Procedure

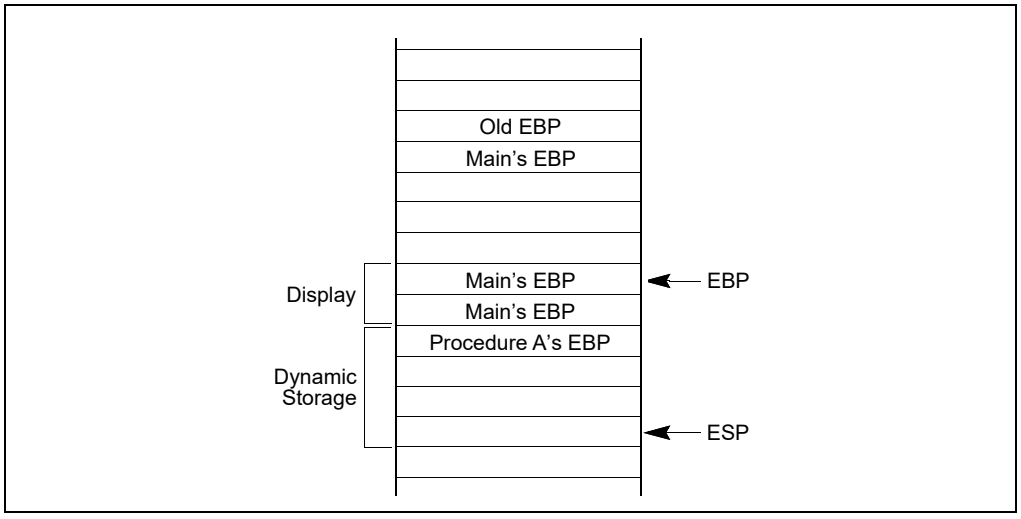


Figure 6-11. Stack Frame After Entering Procedure A

When procedure A calls procedure B, the ENTER instruction creates a new display (see Figure 6-12). The first doubleword holds a copy of the last value in procedure A's EBP register. The second and third doublewords are copies of the two stack frame pointers in procedure A's display. Procedure B can access variables in procedure A and MAIN by using the stack frame pointers in its display.

When procedure B calls procedure C, the ENTER instruction creates a new display for procedure C (see Figure 6-13). The first doubleword holds a copy of the last value in procedure B's EBP register. This is used by the LEAVE instruction to restore procedure B's stack frame. The second and third doublewords are copies of the two stack frame pointers in procedure A's display. If procedure C were at the next deeper lexical level from procedure B, a fourth doubleword would be copied, which would be the stack frame pointer to procedure B's local variables.

Note that procedure B and procedure C are at the same level, so procedure C is not intended to access procedure B's variables. This does not mean that procedure C is completely isolated from procedure B; procedure C is called by procedure B, so the pointer to the returning stack frame is a pointer to procedure B's stack frame. In addition, procedure B can pass parameters to procedure C either on the stack or through variables global to both procedures (that is, variables in the scope of both procedures).

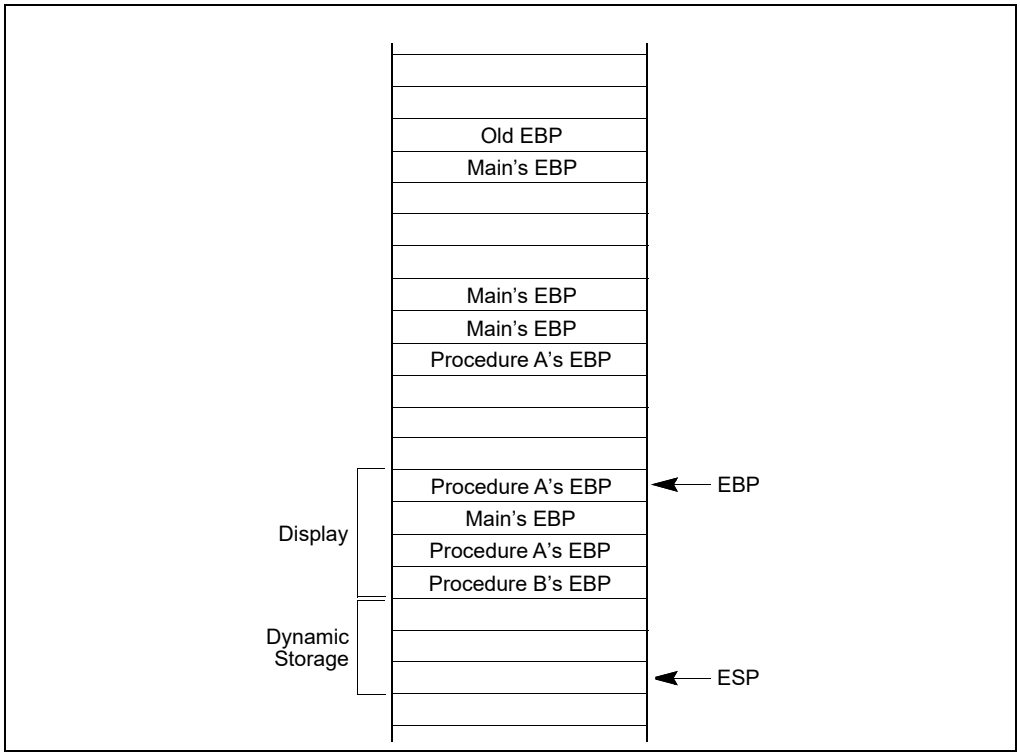


Figure 6-12. Stack Frame After Entering Procedure B

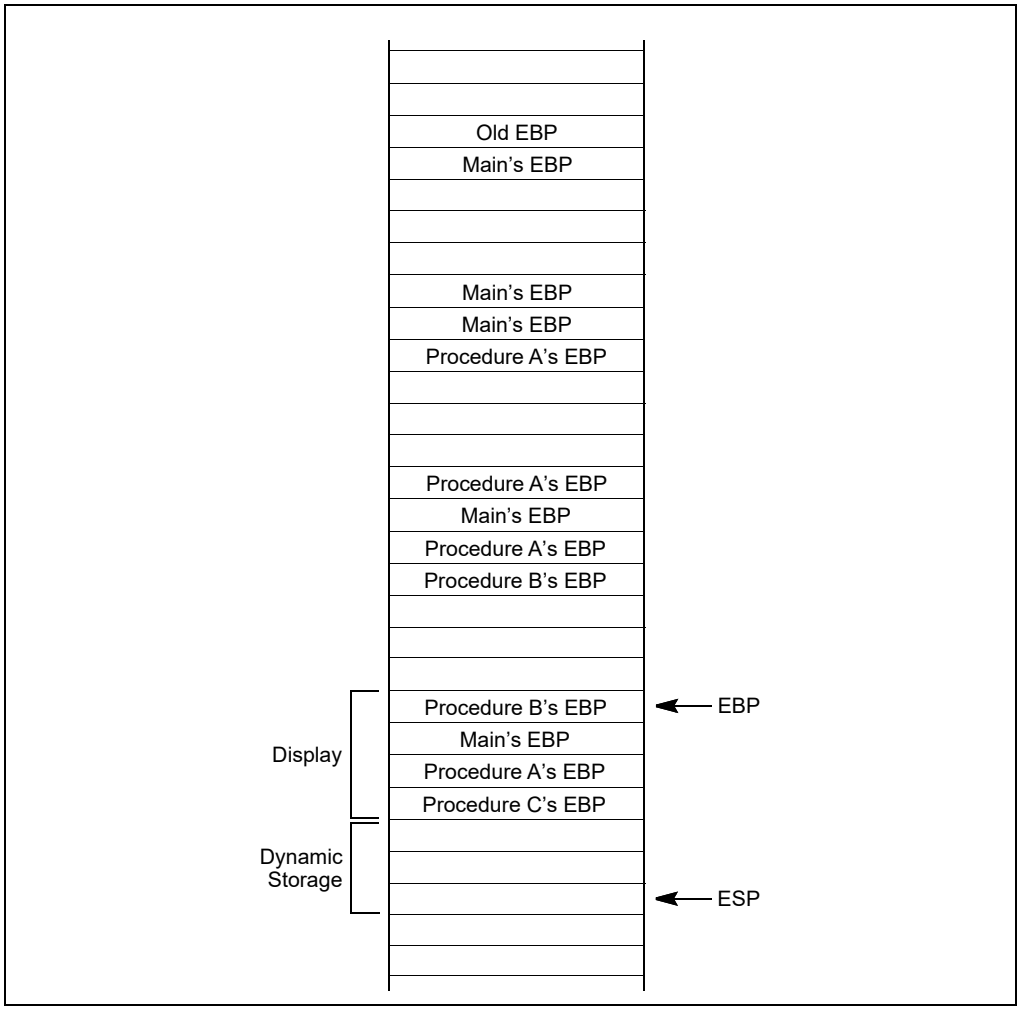


Figure 6-13. Stack Frame After Entering Procedure C

6.6.2 LEAVE Instruction

The LEAVE instruction, which does not have any operands, reverses the action of the previous ENTER instruction. The LEAVE instruction copies the contents of the EBP register into the ESP register to release all stack space allocated to the procedure. Then it restores the old value of the EBP register from the stack. This simultaneously restores the ESP register to its original value. A subsequent RET instruction then can remove any arguments and the return address pushed on the stack by the calling program for use by the procedure.

CHAPTER 7
PROGRAMMING WITH GENERAL-PURPOSE INSTRUCTIONS

General-purpose (GP) instructions are a subset of the IA-32 instructions that represent the fundamental instruction set for the Intel IA-32 processors. These instructions were introduced into the IA-32 architecture with the first IA-32 processors (the Intel 8086 and 8088). Additional instructions were added to the general-purpose instruction set in subsequent families of IA-32 processors (the Intel 286, Intel386, Intel486, Pentium, Pentium Pro, and Pentium II processors).

Intel 64 architecture further extends the capability of most general-purpose instructions so that they are able to handle 64-bit data in 64-bit mode. A small number of general-purpose instructions (still supported in non-64-bit modes) are not supported in 64-bit mode.

General-purpose instructions perform basic data movement, memory addressing, arithmetic and logical, program flow control, input/output, and string operations on a set of integer, pointer, and BCD data types. This chapter provides an overview of the general-purpose instructions. See the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volumes 2A, 2B, 2C, & 2D, for detailed descriptions of individual instructions.

7.1 PROGRAMMING ENVIRONMENT FOR GP INSTRUCTIONS

The programming environment for the general-purpose instructions consists of the set of registers and address space. The environment includes the following items:

- **General-purpose registers —** Eight 32-bit general-purpose registers (see Section 3.4.1, "General-Purpose Registers") are used in non-64-bit modes to address operands in memory. These registers are referenced by the names EAX, EBX, ECX, EDX, EBP, ESI EDI, and ESP.
- **Segment registers —** The six 16-bit segment registers contain segment pointers for use in accessing memory (see Section 3.4.2, "Segment Registers"). These registers are referenced by the names CS, DS, SS, ES, FS, and GS.
- **EFLAGS register —** This 32-bit register (see Section 3.4.3, "EFLAGS Register") is used to provide status and control for basic arithmetic, compare, and system operations.
- **EIP register —** This 32-bit register contains the current instruction pointer (see Section 3.5, "Instruction Pointer").

General-purpose instructions operate on the following data types. The width of valid data types is dependent on processor mode (see Chapter 4):

- Bytes, words, doublewords.
- Signed and unsigned byte, word, doubleword integers.
- Near and far pointers.
- Bit fields.
- BCD integers.

7.2 PROGRAMMING ENVIRONMENT FOR GP INSTRUCTIONS IN 64-BIT MODE

The programming environment for the general-purpose instructions in 64-bit mode is similar to that described in Section 7.1.

- **General-purpose registers —** In 64-bit mode, sixteen general-purpose registers available. These include the eight GPRs described in Section 7.1 and eight new GPRs (R8D-R15D). R8D-R15D are available by using a REX prefix. All sixteen GPRs can be promoted to 64 bits. The 64-bit registers are referenced as RAX, RBX, RCX, RDX, RBP, RSI, RDI, RSP, and R8-R15 (see Section 3.4.1.1, "General-Purpose Registers in 64-Bit Mode"). Promotion to 64-bit operand requires REX prefix encodings.

- **Segment registers —** In 64-bit mode, segmentation is available but it is set up uniquely (see Section 3.4.2.1, "Segment Registers in 64-Bit Mode").
- **Flags and Status register —** When the processor is running in 64-bit mode, EFLAGS becomes the 64-bit RFLAGS register (see Section 3.4.3, "EFLAGS Register").
- **Instruction Pointer register —** In 64-bit mode, the EIP register becomes the 64-bit RIP register (see Section 3.5.1, "Instruction Pointer in 64-Bit Mode").

General-purpose instructions operate on the following data types in 64-bit mode. The width of valid data types is dependent on default operand size, address size, or a prefix that overrides the default size:

- Bytes, words, doublewords, quadwords.
- Signed and unsigned byte, word, doubleword, quadword integers.
- Near and far pointers.
- Bit fields.

See also:

- Chapter 3, "Basic Execution Environment," for more information about IA-32e modes.
- Chapter 2, "Instruction Format," in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 2A, for more detailed information about REX prefixes.
- Intel® 64 and IA-32 Architectures Software Developer's Manual, Volumes 2A, 2B, 2C, & 2D, for a complete listing of all instructions. This information documents the behavior of individual instructions in the 64-bit mode context.

7.3 SUMMARY OF GP INSTRUCTIONS

General purpose instructions are divided into the following subgroups:

- Data transfer.
- Binary arithmetic.
- Decimal arithmetic.
- Logical.
- Shift and rotate.
- Bit and byte.
- Control transfer.
- String.
- I/O.
- Enter and Leave.
- Flag control.
- Segment register.
- Miscellaneous.

Each sub-group of general-purpose instructions is discussed in the context of non-64-bit mode operation first. Changes in 64-bit mode beyond those affected by the use of the REX prefixes are discussed in separate sub-sections within each subgroup. For a simple list of general-purpose instructions by subgroup, see Chapter 5.

7.3.1 Data Transfer Instructions

The data transfer instructions move bytes, words, doublewords, or quadwords both between memory and the processor's registers and between registers. For the purpose of this discussion, these instructions are divided into subordinate subgroups that provide for:

- General data movement.

- Exchange.
- Stack manipulation.
- Type conversion.

7.3.1.1 General Data Movement Instructions

Move instructions — The MOV (move) and CMOVcc (conditional move) instructions transfer data between memory and registers or between registers.

The MOV instruction performs basic load data and store data operations between memory and the processor's registers and data movement operations between registers. It handles data transfers along the paths listed in Table 7-1. (See "MOV—Move to/from Control Registers" and "MOV—Move to/from Debug Registers" in Chapter 4, "Instruction Set Reference, M-U," of the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 2A, for information on moving data to and from the control and debug registers.)

The MOV instruction cannot move data from one memory location to another or from one segment register to another segment register. Memory-to-memory moves are performed with the MOVS (string move) instruction (see Section 7.3.9, "String Operations").

Conditional move instructions — The CMOVcc instructions are a group of instructions that check the state of the status flags in the EFLAGS register and perform a move operation if the flags are in a specified state. These instructions can be used to move a 16-bit or 32-bit value from memory to a general-purpose register or from one general-purpose register to another. The flag state being tested is specified with a condition code (cc) associated with the instruction. If the condition is not satisfied, a move is not performed and execution continues with the instruction following the CMOVcc instruction.

Table 7-1. Move Instruction Operations

Type of Data Movement	Source → Destination
From memory to a register	Memory location → General-purpose register Memory location → Segment register
From a register to memory	General-purpose register → Memory location Segment register → Memory location
Between registers	General-purpose register → General-purpose register General-purpose register → Segment register Segment register → General-purpose register General-purpose register → Control register Control register → General-purpose register General-purpose register → Debug register Debug register → General-purpose register
Immediate data to a register	Immediate → General-purpose register
Immediate data to memory	Immediate → Memory location

Table 7-2 shows mnemonics for CMOVcc instructions and the conditions being tested for each instruction. The condition code mnemonics are appended to the letters "CMOV" to form the mnemonics for CMOVcc instructions. The instructions listed in Table 7-2 as pairs (for example, CMOVA/CMOVNBE) are alternate names for the same instruction. The assembler provides these alternate names to make it easier to read program listings.

CMOVcc instructions are useful for optimizing small IF constructions. They also help eliminate branching overhead for IF statements and the possibility of branch mispredictions by the processor.

These conditional move instructions are supported in the P6 family, Pentium 4, and Intel Xeon processors. Software can check if CMOVcc instructions are supported by checking the processor's feature information with the CPUID instruction.

7.3.1.2 Exchange Instructions

The exchange instructions swap the contents of one or more operands and, in some cases, perform additional operations such as asserting the LOCK signal or modifying flags in the EFLAGS register.

The XCHG (exchange) instruction swaps the contents of two operands. This instruction takes the place of three MOV instructions and does not require a temporary location to save the contents of one operand location while the other is being loaded. When a memory operand is used with the XCHG instruction, the processor's LOCK signal is automatically asserted. This instruction is thus useful for implementing semaphores or similar data structures for process synchronization. See "Bus Locking" in Chapter 11, "Multiple-Processor Management,"of the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3A, for more information on bus locking.

The BSWAP (byte swap) instruction reverses the byte order in a 32-bit register operand. Bit positions 0 through 7 are exchanged with 24 through 31, and bit positions 8 through 15 are exchanged with 16 through 23. Executing this instruction twice in a row leaves the register with the same value as before. The BSWAP instruction is useful for converting between "big-endian" and "little-endian" data formats. This instruction also speeds execution of decimal arithmetic. (The XCHG instruction can be used to swap the bytes in a word.)

Table 7-2. Conditional Move Instructions

Instruction Mnemonic	Status Flag States	Condition Description
Unsigned Conditional Moves		
CMOVA/CMOVNBE	(CF or ZF) = 0	Above/not below or equal
CMOVAE/CMOVNB	CF = 0	Above or equal/not below
CMOVNC	CF = 0	Not carry
CMOVB/CMOVNAE	CF = 1	Below/not above or equal
CMOVC	CF = 1	Carry
CMOVBE/CMOVNA	(CF or ZF) = 1	Below or equal/not above
CMOVE/CMOVZ	ZF = 1	Equal/zero
CMOVNE/CMOVNZ	ZF = 0	Not equal/not zero
CMOVP/CMOVPE	PF = 1	Parity/parity even
CMOVNP/CMOVPO	PF = 0	Not parity/parity odd
Signed Conditional Moves		
CMOVGE/CMOVNL	(SF xor OF) = 0	Greater or equal/not less
CMOVL/CMOVNGE	(SF xor OF) = 1	Less/not greater or equal
CMOVLE/CMOVNG	((SF xor OF) or ZF) = 1	Less or equal/not greater
CMOVO	OF = 1	Overflow
CMOVNO	OF = 0	Not overflow
CMOVS	SF = 1	Sign (negative)
CMOVNS	SF = 0	Not sign (non-negative)

The XADD (exchange and add) instruction swaps two operands and then stores the sum of the two operands in the destination operand. The status flags in the EFLAGS register indicate the result of the addition. This instruction can be combined with the LOCK prefix (see "LOCK—Assert LOCK# Signal Prefix" in Chapter 3, "Instruction Set Reference, A-L," of the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 2A) in a multiprocessing system to allow multiple processors to execute one DO loop.

The CMPXCHG (compare and exchange) and CMPXCHG8B (compare and exchange 8 bytes) instructions are used to synchronize operations in systems that use multiple processors. The CMPXCHG instruction requires three operands: a source operand in a register, another source operand in the EAX register, and a destination operand. If the values contained in the destination operand and the EAX register are equal, the destination operand is replaced with the value of the other source operand (the value not in the EAX register). Otherwise, the original

value of the destination operand is loaded in the EAX register. The status flags in the EFLAGS register reflect the result that would have been obtained by subtracting the destination operand from the value in the EAX register.

The CMPXCHG instruction is commonly used for testing and modifying semaphores. It checks to see if a semaphore is free. If the semaphore is free, it is marked allocated; otherwise it gets the ID of the current owner. This is all done in one uninterruptible operation. In a single-processor system, the CMPXCHG instruction eliminates the need to switch to protection level 0 (to disable interrupts) before executing multiple instructions to test and modify a semaphore.

For multiple processor systems, CMPXCHG can be combined with the LOCK prefix to perform the compare and exchange operation atomically. (See "Locked Atomic Operations" in Chapter 11, "Multiple-Processor Management," of the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3A, for more information on atomic operations.)

The CMPXCHG8B instruction also requires three operands: a 64-bit value in EDX:EAX, a 64-bit value in ECX:EBX, and a destination operand in memory. The instruction compares the 64-bit value in the EDX:EAX registers with the destination operand. If they are equal, the 64-bit value in the ECX:EBX registers is stored in the destination operand. If the EDX:EAX registers and the destination are not equal, the destination is loaded in the EDX:EAX registers. The CMPXCHG8B instruction can be combined with the LOCK prefix to perform the operation atomically.

7.3.1.3 Exchange Instructions in 64-Bit Mode

The CMPXCHG16B instruction is available in 64-bit mode only. It is an extension of the functionality provided by CMPXCHG8B that operates on 128-bits of data.

7.3.1.4 Stack Manipulation Instructions

The PUSH, POP, PUSHA (push all registers), and POPA (pop all registers) instructions move data to and from the stack. The PUSH instruction decrements the stack pointer (contained in the ESP register), then copies the source operand to the top of stack (see Figure 7-1). It operates on memory operands, immediate operands, and register operands (including segment registers). The PUSH instruction is commonly used to place parameters on the stack before calling a procedure. It can also be used to reserve space on the stack for temporary variables.

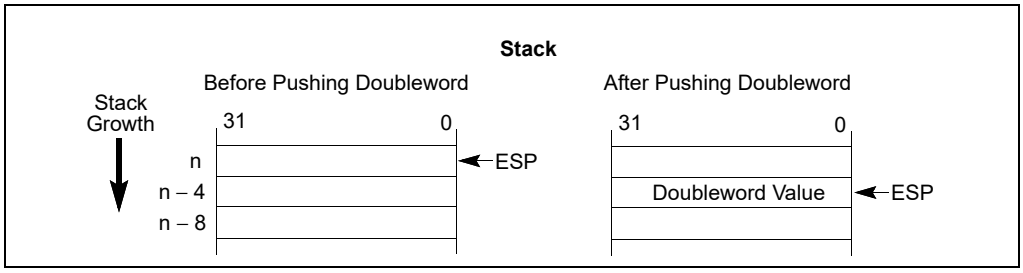


Figure 7-1. Operation of the PUSH Instruction

The PUSHA instruction saves the contents of the eight general-purpose registers on the stack (see Figure 7-2). This instruction simplifies procedure calls by reducing the number of instructions required to save the contents of the general-purpose registers. The registers are pushed on the stack in the following order: EAX, ECX, EDX, EBX, the initial value of ESP before EAX was pushed, EBP, ESI, and EDI.

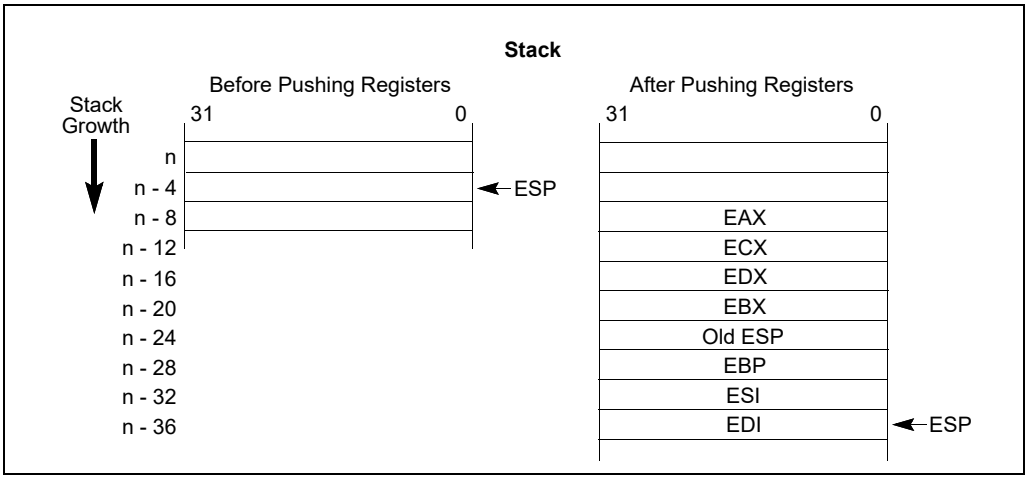


Figure 7-2. Operation of the PUSHA Instruction

The POP instruction copies the word or doubleword at the current top of stack (indicated by the ESP register) to the location specified with the destination operand. It then increments the ESP register to point to the new top of stack (see Figure 7-3). The destination operand may specify a general-purpose register, a segment register, or a memory location.

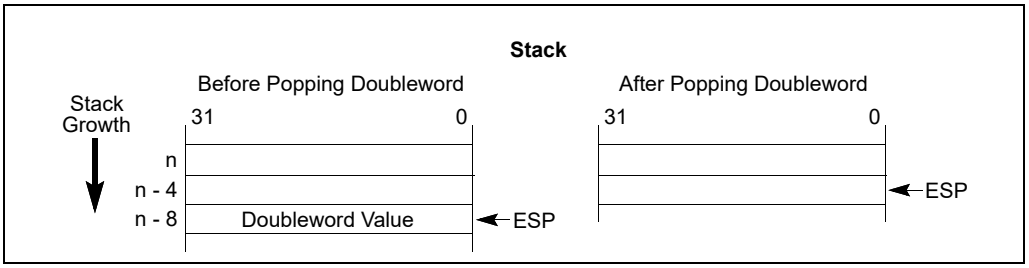


Figure 7-3. Operation of the POP Instruction

The POPA instruction reverses the effect of the PUSHA instruction. It pops the top eight words or doublewords from the top of the stack into the general-purpose registers, except for the ESP register (see Figure 7-4). If the operand-size attribute is 32, the doublewords on the stack are transferred to the registers in the following order: EDI, ESI, EBP, ignore doubleword, EBX, EDX, ECX, and EAX. The ESP register is restored by the action of popping the stack. If the operand-size attribute is 16, the words on the stack are transferred to the registers in the following order: DI, SI, BP, ignore word, BX, DX, CX, and AX.

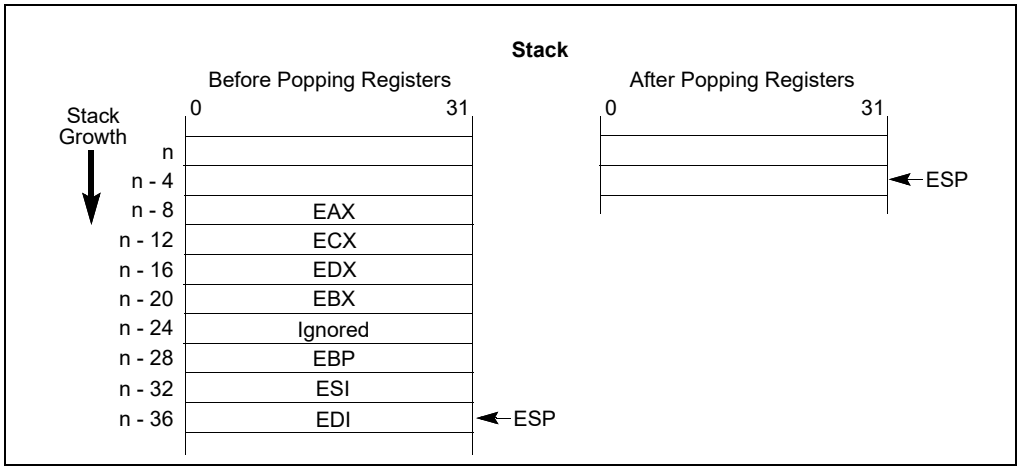


Figure 7-4. Operation of the POPA Instruction

7.3.1.5 Stack Manipulation Instructions in 64-Bit Mode

In 64-bit mode, the stack pointer size is 64 bits and cannot be overridden by an instruction prefix. In implicit stack references, address-size overrides are ignored. Pushes and pops of 32-bit values on the stack are not possible in 64-bit mode. 16-bit pushes and pops are supported by using the 66H operand-size prefix. PUSHA, PUSHAD, POPA, and POPAD are not supported.

7.3.1.6 Type Conversion Instructions

The type conversion instructions convert bytes into words, words into doublewords, and doublewords into quadwords. These instructions are especially useful for converting integers to larger integer formats, because they perform sign extension (see Figure 7-5).

Two kinds of type conversion instructions are provided: simple conversion and move and convert.

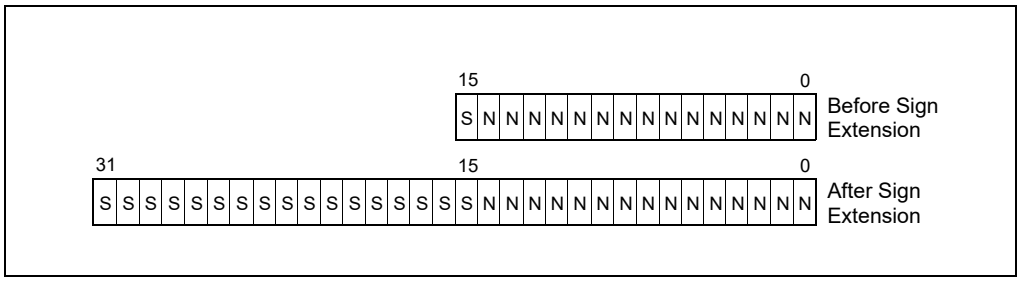


Figure 7-5. Sign Extension

Simple conversion — The CBW (convert byte to word), CWDE (convert word to doubleword extended), CWD (convert word to doubleword), and CDQ (convert doubleword to quadword) instructions perform sign extension to double the size of the source operand.

The CBW instruction copies the sign (bit 7) of the byte in the AL register into every bit position of the upper byte of the AX register. The CWDE instruction copies the sign (bit 15) of the word in the AX register into every bit position of the high word of the EAX register.

The CWD instruction copies the sign (bit 15) of the word in the AX register into every bit position in the DX register. The CDQ instruction copies the sign (bit 31) of the doubleword in the EAX register into every bit position in the EDX register. The CWD instruction can be used to produce a doubleword dividend from a word before a word division, and the CDQ instruction can be used to produce a quadword dividend from a doubleword before doubleword division.

Move with sign or zero extension — The MOVSX (move with sign extension) and MOVZX (move with zero extension) instructions move the source operand into a register then perform the sign extension.

The MOVSX instruction extends an 8-bit value to a 16-bit value or an 8-bit or 16-bit value to a 32-bit value by sign extending the source operand, as shown in Figure 7-5. The MOVZX instruction extends an 8-bit value to a 16-bit value or an 8-bit or 16-bit value to a 32-bit value by zero extending the source operand.

7.3.1.7 Type Conversion Instructions in 64-Bit Mode

The MOVSXD instruction operates on 64-bit data. It sign-extends a 32-bit value to 64 bits. This instruction is not encodable in non-64-bit modes.

7.3.2 Binary Arithmetic Instructions

Binary arithmetic instructions operate on 8-, 16-, and 32-bit numeric data encoded as signed or unsigned binary integers. The binary arithmetic instructions may also be used in algorithms that operate on decimal (BCD) values.

For the purpose of this discussion, these instructions are divided into subordinate subgroups of instructions that:

- Add and subtract.
- Increment and decrement.
- Compare and change signs.
- Multiply and divide.

7.3.2.1 Addition and Subtraction Instructions

The ADD (add integers), ADC (add integers with carry), SUB (subtract integers), and SBB (subtract integers with borrow) instructions perform addition and subtraction operations on signed or unsigned integer operands.

The ADD instruction computes the sum of two integer operands.

The ADC instruction computes the sum of two integer operands, plus 1 if the CF flag is set. This instruction is used to propagate a carry when adding numbers in stages.

The SUB instruction computes the difference of two integer operands.

The SBB instruction computes the difference of two integer operands, minus 1 if the CF flag is set. This instruction is used to propagate a borrow when subtracting numbers in stages.

7.3.2.2 Increment and Decrement Instructions

The INC (increment) and DEC (decrement) instructions add 1 to or subtract 1 from an unsigned integer operand, respectively. A primary use of these instructions is for implementing counters.

7.3.2.3 Increment and Decrement Instructions in 64-Bit Mode

The INC and DEC instructions are supported in 64-bit mode. However, some forms of INC and DEC (the register operand being encoded using register extension field in the MOD R/M byte) are not encodable in 64-bit mode because the opcodes are treated as REX prefixes.

7.3.2.4 Comparison and Sign Change Instructions

The CMP (compare) instruction computes the difference between two integer operands and updates the OF, SF, ZF, AF, PF, and CF flags according to the result. The source operands are not modified, nor is the result saved. The CMP instruction is commonly used in conjunction with a J*cc* (jump) or SET*cc* (byte set on condition) instruction, with the latter instructions performing an action based on the result of a CMP instruction.

The NEG (negate) instruction subtracts a signed integer operand from zero. The effect of the NEG instruction is to change the sign of a two's complement operand while keeping its magnitude.

7.3.2.5 Multiplication and Division Instructions

The processor provides two multiply instructions, MUL (unsigned multiply) and IMUL (signed multiply), and two divide instructions, DIV (unsigned divide) and IDIV (signed divide).

The MUL instruction multiplies two unsigned integer operands. The result is computed to twice the size of the source operands (for example, if word operands are being multiplied, the result is a doubleword).

The IMUL instruction multiplies two signed integer operands. The result is computed to twice the size of the source operands; however, in some cases the result is truncated to the size of the source operands (see "IMUL—Signed Multiply" in Chapter 3, "Instruction Set Reference, A-L," of the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 2A).

The DIV instruction divides one unsigned operand by another unsigned operand and returns a quotient and a remainder.

The IDIV instruction is identical to the DIV instruction, except that IDIV performs a signed division.

7.3.3 Decimal Arithmetic Instructions

Decimal arithmetic can be performed by combining the binary arithmetic instructions ADD, SUB, MUL, and DIV (discussed in Section 7.3.2, "Binary Arithmetic Instructions") with the decimal arithmetic instructions. The decimal arithmetic instructions are provided to carry out the following operations:

- To adjust the results of a previous binary arithmetic operation to produce a valid BCD result.
- To adjust the operands of a subsequent binary arithmetic operation so that the operation will produce a valid BCD result.

These instructions operate on both packed and unpacked BCD values. For the purpose of this discussion, the decimal arithmetic instructions are divided into subordinate subgroups of instructions that provide:

- Packed BCD adjustments.
- Unpacked BCD adjustments.

7.3.3.1 Packed BCD Adjustment Instructions

The DAA (decimal adjust after addition) and DAS (decimal adjust after subtraction) instructions adjust the results of operations performed on packed BCD integers (see Section 4.7, "BCD and Packed BCD Integers"). Adding two packed BCD values requires two instructions: an ADD instruction followed by a DAA instruction. The ADD instruction adds (binary addition) the two values and stores the result in the AL register. The DAA instruction then adjusts the value in the AL register to obtain a valid, 2-digit, packed BCD value and sets the CF flag if a decimal carry occurred as the result of the addition.

Likewise, subtracting one packed BCD value from another requires a SUB instruction followed by a DAS instruction. The SUB instruction subtracts (binary subtraction) one BCD value from another and stores the result in the AL register. The DAS instruction then adjusts the value in the AL register to obtain a valid, 2-digit, packed BCD value and sets the CF flag if a decimal borrow occurred as the result of the subtraction.

7.3.3.2 Unpacked BCD Adjustment Instructions

The AAA (ASCII adjust after addition), AAS (ASCII adjust after subtraction), AAM (ASCII adjust after multiplication), and AAD (ASCII adjust before division) instructions adjust the results of arithmetic operations performed on unpacked BCD values (see Section 4.7, "BCD and Packed BCD Integers"). All these instructions assume that the value to be adjusted is stored in the AL register or, in one instance, the AL and AH registers.

The AAA instruction adjusts the contents of the AL register following the addition of two unpacked BCD values. It converts the binary value in the AL register into a decimal value and stores the result in the AL register in unpacked BCD format (the decimal number is stored in the lower 4 bits of the register and the upper 4 bits are cleared). If a decimal carry occurred as a result of the addition, the CF flag is set and the contents of the AH register are incremented by 1.

The AAS instruction adjusts the contents of the AL register following the subtraction of two unpacked BCD values. Here again, a binary value is converted into an unpacked BCD value. If a borrow was required to complete the decimal subtract, the CF flag is set and the contents of the AH register are decremented by 1.

The AAM instruction adjusts the contents of the AL register following a multiplication of two unpacked BCD values. It converts the binary value in the AL register into a decimal value and stores the least significant digit of the result in the AL register (in unpacked BCD format) and the most significant digit, if there is one, in the AH register (also in unpacked BCD format).

The AAD instruction adjusts a two-digit BCD value so that when the value is divided with the DIV instruction, a valid unpacked BCD result is obtained. The instruction converts the BCD value in registers AH (most significant digit) and AL (least significant digit) into a binary value and stores the result in register AL. When the value in AL is divided by an unpacked BCD value, the quotient and remainder will be automatically encoded in unpacked BCD format.

7.3.4 Decimal Arithmetic Instructions in 64-Bit Mode

Decimal arithmetic instructions are not supported in 64-bit mode, they are either invalid or not encodable.

7.3.5 Logical Instructions

The logical instructions AND, OR, XOR (exclusive or), and NOT perform the standard Boolean operations for which they are named. The AND, OR, and XOR instructions require two operands; the NOT instruction operates on a single operand.

7.3.6 Shift and Rotate Instructions

The shift and rotate instructions rearrange the bits within an operand. For the purpose of this discussion, these instructions are further divided into subordinate subgroups of instructions that:

- Shift bits
- Double-shift bits (move them between operands)
- Rotate bits

7.3.6.1 Shift Instructions

The SAL (shift arithmetic left), SHL (shift logical left), SAR (shift arithmetic right), SHR (shift logical right) instructions perform an arithmetic or logical shift of the bits in a byte, word, or doubleword.

The SAL and SHL instructions perform the same operation (see Figure 7-6). They shift the source operand left by from 1 to 31 bit positions. Empty bit positions are cleared. The CF flag is loaded with the last bit shifted out of the operand.

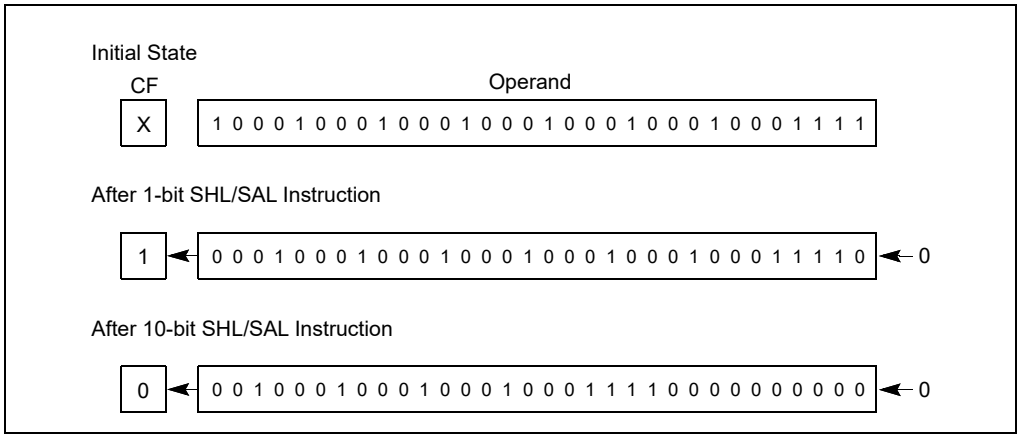


Figure 7-6. SHL/SAL Instruction Operation

The SHR instruction shifts the source operand right by from 1 to 31 bit positions (see Figure 7-7). As with the SHL/SAL instruction, the empty bit positions are cleared and the CF flag is loaded with the last bit shifted out of the operand.

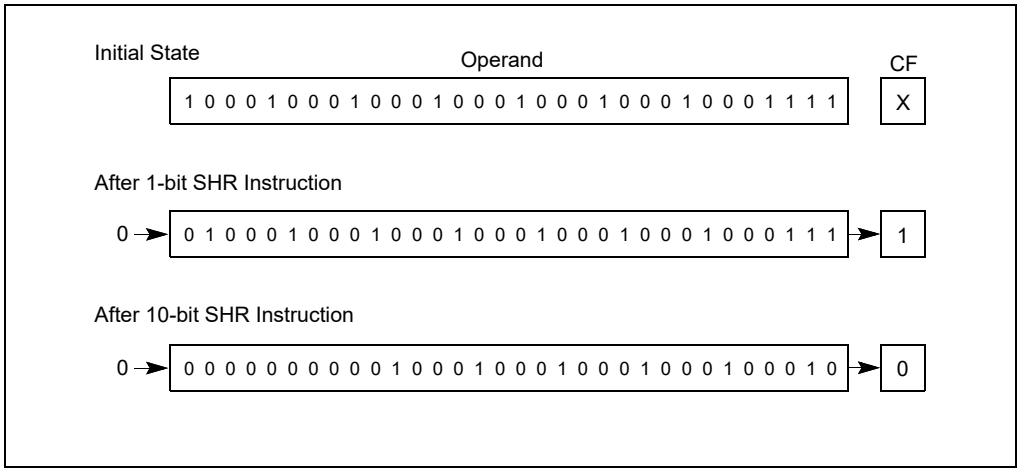


Figure 7-7. SHR Instruction Operation

The SAR instruction shifts the source operand right by from 1 to 31 bit positions (see Figure 7-8). This instruction differs from the SHR instruction in that it preserves the sign of the source operand by clearing empty bit positions if the operand is positive or setting the empty bits if the operand is negative. Again, the CF flag is loaded with the last bit shifted out of the operand.

The SAR and SHR instructions can also be used to perform division by powers of 2 (see "SAL/SAR/SHL/SHR—Shift Instructions" in Chapter 4, "Instruction Set Reference, M-U," of the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 2B).

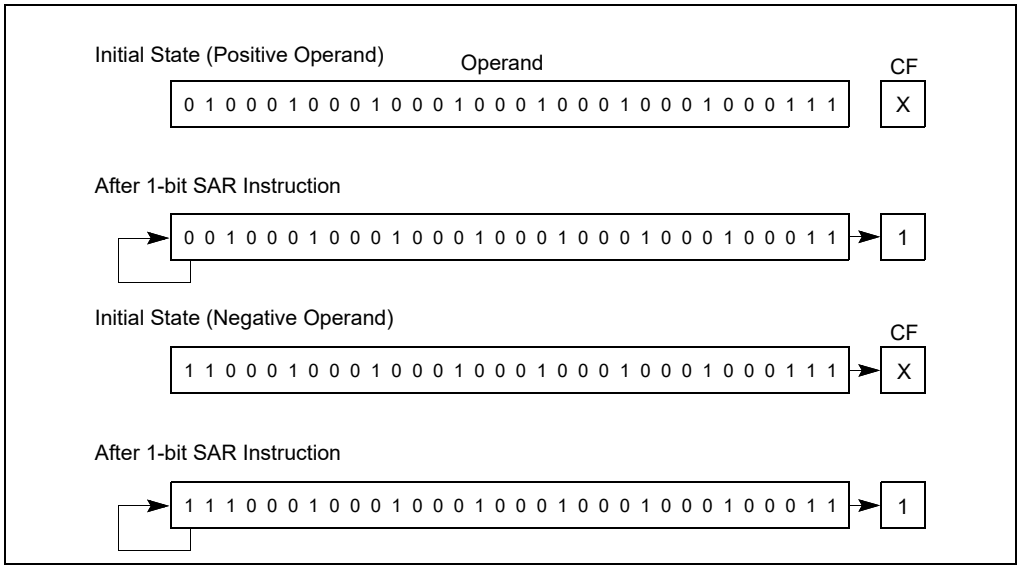


Figure 7-8. SAR Instruction Operation

7.3.6.2 Double-Shift Instructions

The SHLD (shift left double) and SHRD (shift right double) instructions shift a specified number of bits from one operand to another (see Figure 7-9). They are provided to facilitate operations on unaligned bit strings. They can also be used to implement a variety of bit string move operations.

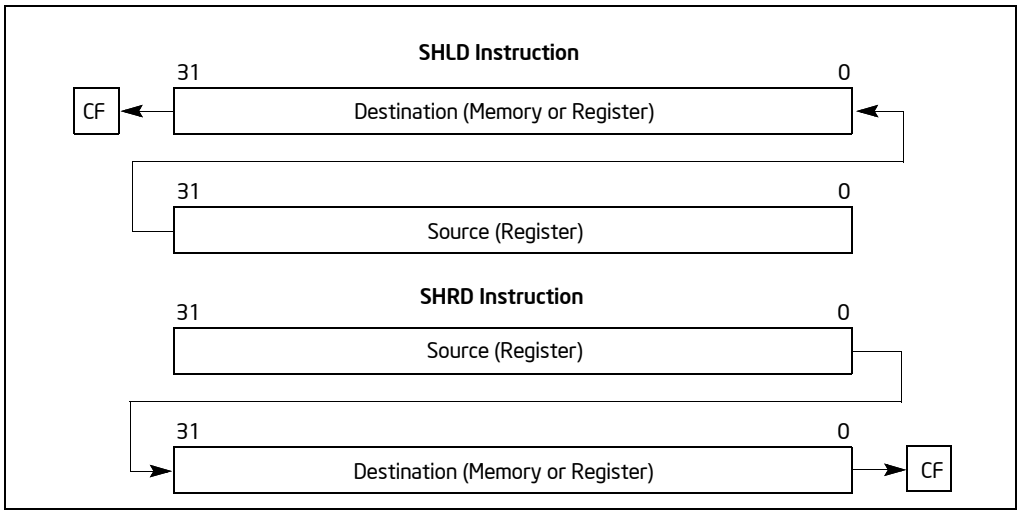


Figure 7-9. SHLD and SHRD Instruction Operations

The SHLD instruction shifts the bits in the destination operand to the left and fills the empty bit positions (in the destination operand) with bits shifted out of the source operand. The destination and source operands must be the same length (either words or doublewords). The shift count can range from 0 to 31 bits. The result of this shift operation is stored in the destination operand, and the source operand is not modified. The CF flag is loaded with the last bit shifted out of the destination operand.

The SHRD instruction operates the same as the SHLD instruction except bits are shifted to the right in the destination operand, with the empty bit positions filled with bits shifted out of the source operand.

7.3.6.3 Rotate Instructions

The ROL (rotate left), ROR (rotate right), RCL (rotate through carry left) and RCR (rotate through carry right) instructions rotate the bits in the destination operand out of one end and back through the other end (see Figure 7-10). Unlike a shift, no bits are lost during a rotation. The rotate count can range from 0 to 31.

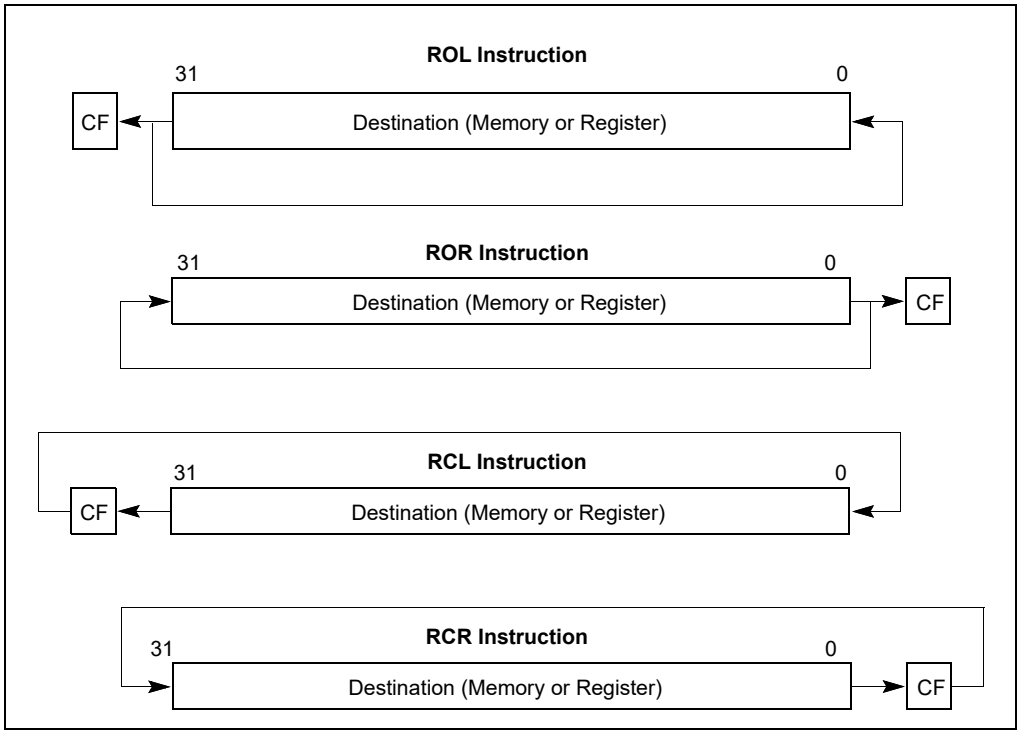


Figure 7-10. ROL, ROR, RCL, and RCR Instruction Operations

The ROL instruction rotates the bits in the operand to the left (toward more significant bit locations). The ROR instruction rotates the operand right (toward less significant bit locations).

The RCL instruction rotates the bits in the operand to the left, through the CF flag. This instruction treats the CF flag as a one-bit extension on the upper end of the operand. Each bit that exits from the most significant bit location of the operand moves into the CF flag. At the same time, the bit in the CF flag enters the least significant bit location of the operand.

The RCR instruction rotates the bits in the operand to the right through the CF flag.

For all the rotate instructions, the CF flag always contains the value of the last bit rotated out of the operand, even if the instruction does not use the CF flag as an extension of the operand. The value of this flag can then be tested by a conditional jump instruction (JC or JNC).

7.3.7 Bit and Byte Instructions

These instructions operate on bit or byte strings. For the purpose of this discussion, they are further divided into subordinate subgroups that:

- Test and modify a single bit.
- Scan a bit string.
- Set a byte given conditions.
- Test operands and report results.

7.3.7.1 Bit Test and Modify Instructions

The bit test and modify instructions (see Table 7-3) operate on a single bit, which can be in an operand. The location of the bit is specified as an offset from the least significant bit of the operand. When the processor identifies the bit to be tested and modified, it first loads the CF flag with the current value of the bit. Then it assigns a new value to the selected bit, as determined by the modify operation for the instruction.

Table 7-3. Bit Test and Modify Instructions

Instruction	Effect on CF Flag	Effect on Selected Bit
BT (Bit Test)	CF flag ← Selected Bit	No effect
BTS (Bit Test and Set)	CF flag ← Selected Bit	Selected Bit ← 1
BTR (Bit Test and Reset)	CF flag ← Selected Bit	Selected Bit ← 0
BTC (Bit Test and Complement)	CF flag ← Selected Bit	Selected Bit ← NOT (Selected Bit)

7.3.7.2 Bit Scan Instructions

The BSF (bit scan forward) and BSR (bit scan reverse) instructions scan a bit string in a source operand for a set bit and store the bit index of the first set bit found in a destination register. The bit index is the offset from the least significant bit (bit 0) in the bit string to the first set bit. The BSF instruction scans the source operand low-to-high (from bit 0 of the source operand toward the most significant bit); the BSR instruction scans high-to-low (from the most significant bit toward the least significant bit).

7.3.7.3 Byte Set on Condition Instructions

The SET*cc* (set byte on condition) instructions set a destination-operand byte to 0 or 1, depending on the state of selected status flags (CF, OF, SF, ZF, and PF) in the EFLAGS register. The suffix (*cc*) added to the SET mnemonic determines the condition being tested for.

For example, the SETO instruction tests for overflow. If the OF flag is set, the destination byte is set to 1; if OF is clear, the destination byte is cleared to 0. Appendix B, "EFLAGS Condition Codes," lists the conditions it is possible to test for with this instruction.

7.3.7.4 Test Instruction

The TEST instruction performs a logical AND of two operands and sets the SF, ZF, and PF flags according to the results. The flags can then be tested by the conditional jump or loop instructions or the SET*cc* instructions. The TEST instruction differs from the AND instruction in that it does not alter either of the operands.

7.3.8 Control Transfer Instructions

The processor provides both conditional and unconditional control transfer instructions to direct the flow of program execution. Conditional transfers are taken only for specified states of the status flags in the EFLAGS register. Unconditional control transfers are always executed.

For the purpose of this discussion, these instructions are further divided into subordinate subgroups that process:

- Unconditional transfers.
- Conditional transfers.
- Software interrupts.

7.3.8.1 Unconditional Transfer Instructions

The JMP, CALL, RET, INT, and IRET instructions transfer program control to another location (destination address) in the instruction stream. The destination can be within the same code segment (near transfer) or in a different code segment (far transfer).

Jump instruction — The JMP (jump) instruction unconditionally transfers program control to a destination instruction. The transfer is one-way; that is, a return address is not saved. A destination operand specifies the address (the instruction pointer) of the destination instruction. The address can be a **relative address** or an **absolute address**.

A **relative address** is a displacement (offset) with respect to the address in the EIP register. The destination address (a near pointer) is formed by adding the displacement to the address in the EIP register. The displacement is specified with a signed integer, allowing jumps either forward or backward in the instruction stream.

An **absolute address** is a offset from address 0 of a segment. It can be specified in either of the following ways:

- **An address in a general-purpose register —** This address is treated as a near pointer, which is copied into the EIP register. Program execution then continues at the new address within the current code segment.
- **An address specified using the standard addressing modes of the processor —** Here, the address can be a near pointer or a far pointer. If the address is for a near pointer, the address is translated into an offset and copied into the EIP register. If the address is for a far pointer, the address is translated into a segment selector (which is copied into the CS register) and an offset (which is copied into the EIP register).

In protected mode, the JMP instruction also allows jumps to a call gate, a task gate, and a task-state segment.

Call and return instructions — The CALL (call procedure) and RET (return from procedure) instructions allow a jump from one procedure (or subroutine) to another and a subsequent jump back (return) to the calling procedure.

The CALL instruction transfers program control from the current (or calling) procedure to another procedure (the called procedure). To allow a subsequent return to the calling procedure, the CALL instruction saves the current contents of the EIP register on the stack before jumping to the called procedure. The EIP register (prior to transferring program control) contains the address of the instruction following the CALL instruction. When this address is pushed on the stack, it is referred to as the **return instruction pointer** or **return address**.

The address of the called procedure (the address of the first instruction in the procedure being jumped to) is specified in a CALL instruction the same way as it is in a JMP instruction (see "Jump instruction" on page 7-15). The address can be specified as a relative address or an absolute address. If an absolute address is specified, it can be either a near or a far pointer.

The RET instruction transfers program control from the procedure currently being executed (the called procedure) back to the procedure that called it (the calling procedure). Transfer of control is accomplished by copying the return instruction pointer from the stack into the EIP register. Program execution then continues with the instruction pointed to by the EIP register.

The RET instruction has an optional operand, the value of which is added to the contents of the ESP register as part of the return operation. This operand allows the stack pointer to be incremented to remove parameters from the stack that were pushed on the stack by the calling procedure.

See Section 6.4, "Calling Procedures Using CALL and RET," for more information on the mechanics of making procedure calls with the CALL and RET instructions.

Return from interrupt instruction — When the processor services an interrupt, it performs an implicit call to an interrupt-handling procedure. The IRET (return from interrupt) instruction returns program control from an interrupt handler to the interrupted procedure (that is, the procedure that was executing when the interrupt occurred). The IRET instruction performs a similar operation to the RET instruction (see "Call and return instructions" on page 7-15) except that it also restores the EFLAGS register from the stack. The contents of the EFLAGS register are automatically stored on the stack along with the return instruction pointer when the processor services an interrupt.

7.3.8.2 Conditional Transfer Instructions

The conditional transfer instructions execute jumps or loops that transfer program control to another instruction in the instruction stream if specified conditions are met. The conditions for control transfer are specified with a set of condition codes that define various states of the status flags (CF, ZF, OF, PF, and SF) in the EFLAGS register.

Conditional jump instructions — The J*cc* (conditional) jump instructions transfer program control to a destination instruction if the conditions specified with the condition code (*cc*) associated with the instruction are satisfied (see Table 7-4). If the condition is not satisfied, execution continues with the instruction following the J*cc* instruction. As with the JMP instruction, the transfer is one-way; that is, a return address is not saved.

Table 7-4. Conditional Jump Instructions

Instruction Mnemonic	Condition (Flag States)	Description
Unsigned Conditional Jumps		
JA/JNBE	(CF or ZF) = 0	Above/not below or equal
JAE/JNB	CF = 0	Above or equal/not below
JB/JNAE	CF = 1	Below/not above or equal
JBE/JNA	(CF or ZF) = 1	Below or equal/not above
JC	CF = 1	Carry
JE/JZ	ZF = 1	Equal/zero
JNC	CF = 0	Not carry
JNE/JNZ	ZF = 0	Not equal/not zero
JNP/JPO	PF = 0	Not parity/parity odd
JP/JPE	PF = 1	Parity/parity even
JCXZ	CX = 0	Register CX is zero
JECXZ	ECX = 0	Register ECX is zero
Signed Conditional Jumps		
JG/JNLE	((SF xor OF) or ZF) = 0	Greater/not less or equal
JGE/JNL	(SF xor OF) = 0	Greater or equal/not less
JL/JNGE	(SF xor OF) = 1	Less/not greater or equal
JLE/JNG	((SF xor OF) or ZF) = 1	Less or equal/not greater
JNO	OF = 0	Not overflow
JNS	SF = 0	Not sign (non-negative)
JO	OF = 1	Overflow
JS	SF = 1	Sign (negative)

The destination operand specifies a relative address (a signed offset with respect to the address in the EIP register) that points to an instruction in the current code segment. The *Jcc* instructions do not support far transfers; however, far transfers can be accomplished with a combination of a *Jcc* and a JMP instruction (see "*Jcc*—Jump if Condition Is Met" in Chapter 3, "Instruction Set Reference, A-L," of the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 2A).

Table 7-4 shows the mnemonics for the *Jcc* instructions and the conditions being tested for each instruction. The condition code mnemonics are appended to the letter "J" to form the mnemonic for a *Jcc* instruction. The instructions are divided into two groups: unsigned and signed conditional jumps. These groups correspond to the results of operations performed on unsigned and signed integers respectively. Those instructions listed as pairs (for example, JA/JNBE) are alternate names for the same instruction. Assemblers provide alternate names to make it easier to read program listings.

The JCXZ and JECXZ instructions test the CX and ECX registers, respectively, instead of one or more status flags. See "Jump if zero instructions" on page 7-17 for more information about these instructions.

Loop instructions — The LOOP, LOOPE (loop while equal), LOOPZ (loop while zero), LOOPNE (loop while not equal), and LOOPNZ (loop while not zero) instructions are conditional jump instructions that use the value of the ECX register as a count for the number of times to execute a loop. All the loop instructions decrement the count in the ECX register each time they are executed and terminate a loop when zero is reached. The LOOPE, LOOPZ, LOOPNE, and LOOPNZ instructions also accept the ZF flag as a condition for terminating the loop before the count reaches zero.

The LOOP instruction decrements the contents of the ECX register (or the CX register, if the address-size attribute is 16), then tests the register for the loop-termination condition. If the count in the ECX register is non-zero, program control is transferred to the instruction address specified by the destination operand. The destination

operand is a relative address (that is, an offset relative to the contents of the EIP register), and it generally points to the first instruction in the block of code that is to be executed in the loop. When the count in the ECX register reaches zero, program control is transferred to the instruction immediately following the LOOP instruction, which terminates the loop. If the count in the ECX register is zero when the LOOP instruction is first executed, the register is pre-decremented to FFFFFFFFH, causing the loop to be executed 2^{32} times.

The LOOPE and LOOPZ instructions perform the same operation (they are mnemonics for the same instruction). These instructions operate the same as the LOOP instruction, except that they also test the ZF flag.

If the count in the ECX register is not zero and the ZF flag is set, program control is transferred to the destination operand. When the count reaches zero or the ZF flag is clear, the loop is terminated by transferring program control to the instruction immediately following the LOOPE/LOOPZ instruction.

The LOOPNE and LOOPNZ instructions (mnemonics for the same instruction) operate the same as the LOOPE/LOOPZ instructions, except that they terminate the loop if the ZF flag is set.

Jump if zero instructions — The JECXZ (jump if ECX zero) instruction jumps to the location specified in the destination operand if the ECX register contains the value zero. This instruction can be used in combination with a loop instruction (LOOP, LOOPE, LOOPZ, LOOPNE, or LOOPNZ) to test the ECX register prior to beginning a loop. As described in "Loop instructions" on page 7-16, the loop instructions decrement the contents of the ECX register before testing for zero. If the value in the ECX register is zero initially, it will be decremented to FFFFFFFFH on the first loop instruction, causing the loop to be executed 2^{32} times. To prevent this problem, a JECXZ instruction can be inserted at the beginning of the code block for the loop, causing a jump out of the loop if the ECX register count is initially zero. When used with repeated string scan and compare instructions, the JECXZ instruction can determine whether the loop terminated because the count reached zero or because the scan or compare conditions were satisfied.

The JCXZ (jump if CX is zero) instruction operates the same as the JECXZ instruction when the 16-bit address-size attribute is used. Here, the CX register is tested for zero.

7.3.8.3 Control Transfer Instructions in 64-Bit Mode

In 64-bit mode, the operand size for all near branches (CALL, RET, JCC, JCXZ, JMP, and LOOP) is forced to 64 bits. The listed instructions update the 64-bit RIP without need for a REX operand-size prefix.

Near branches in the following operations are forced to 64-bits (regardless of operand size prefixes):

- Truncation of the size of the instruction pointer.
- Size of a stack pop or push, due to CALL or RET.
- Size of a stack-pointer increment or decrement, due to CALL or RET.
- Indirect-branch operand size.

Note that the displacement field for relative branches is still limited to 32 bits and the address size for near branches is not forced.

Address size determines the register size (CX/ECX/RCX) used for JCXZ and LOOP. It also impacts the address calculation for memory indirect branches. Addresses size is 64 bits by default, although it can be over-ridden to 32 bits (using a prefix).

7.3.8.4 Software Interrupt Instructions

The INT *n* (software interrupt), INTO (interrupt on overflow), and BOUND (detect value out of range) instructions allow a program to explicitly raise a specified interrupt or exception, which in turn causes the handler routine for the interrupt or exception to be called.

The INT *n* instruction can raise any of the processor's interrupts or exceptions by encoding the vector of the interrupt or exception in the instruction. This instruction can be used to support software generated interrupts or to test the operation of interrupt and exception handlers.

The IRET (return from interrupt) instruction returns program control from an interrupt handler to the interrupted procedure. The IRET instruction performs a similar operation to the RET instruction.

The CALL (call procedure) and RET (return from procedure) instructions allow a jump from one procedure to another and a subsequent return to the calling procedure. EFLAGS register contents are automatically stored on the stack along with the return instruction pointer when the processor services an interrupt.

The INTO instruction raises the overflow exception if the OF flag is set. If the flag is clear, execution continues without raising the exception. This instruction allows software to access the overflow exception handler explicitly to check for overflow conditions.

The BOUND instruction compares a signed value against upper and lower bounds, and raises the "BOUND range exceeded" exception if the value is less than the lower bound or greater than the upper bound. This instruction is useful for operations such as checking an array index to make sure it falls within the range defined for the array.

7.3.8.5 Software Interrupt Instructions in 64-Bit Mode and Compatibility Mode

In 64-bit mode, the stack size is 8 bytes wide. IRET must pop 8-byte items off the stack. SS:RSP pops unconditionally. BOUND is not supported.

In compatibility mode, SS:RSP is popped only if the CPL changes.

7.3.9 String Operations

The GP instructions includes a set of **string instructions** that are designed to access large data structures; these are introduced in Section 7.3.9.1. Section 7.3.9.2 describes how REP prefixes can be used with these instructions to perform more complex **repeated string operations**. Certain processors optimize repeated string operations with **fast-string operation**, as described in Section 7.3.9.3. Section 7.3.9.4 explains how string operations can be used in 64-bit mode.

7.3.9.1 String Instructions

The MOVS (Move String), CMPS (Compare string), SCAS (Scan string), LODS (Load string), and STOS (Store string) instructions permit large data structures, such as alphanumeric character strings, to be moved and examined in memory. These instructions operate on individual elements in a string, which can be a byte, word, or doubleword. The string elements to be operated on are identified with the ESI (source string element) and EDI (destination string element) registers. Both of these registers contain absolute addresses (offsets into a segment) that point to a string element.

By default, the ESI register addresses the segment identified with the DS segment register. A segment-override prefix allows the ESI register to be associated with the CS, SS, ES, FS, or GS segment register. The EDI register addresses the segment identified with the ES segment register; no segment override is allowed for the EDI register. The use of two different segment registers in the string instructions permits operations to be performed on strings located in different segments. Or by associating the ESI register with the ES segment register, both the source and destination strings can be located in the same segment. (This latter condition can also be achieved by loading the DS and ES segment registers with the same segment selector and allowing the ESI register to default to the DS register.)

The MOVS instruction moves the string element addressed by the ESI register to the location addressed by the EDI register. The assembler recognizes three "short forms" of this instruction, which specify the size of the string to be moved: MOVSB (move byte string), MOVSW (move word string), and MOVSD (move doubleword string).

The CMPS instruction subtracts the destination string element from the source string element and updates the status flags (CF, ZF, OF, SF, PF, and AF) in the EFLAGS register according to the results. Neither string element is written back to memory. The assembler recognizes three "short forms" of the CMPS instruction: CMPSB (compare byte strings), CMPSW (compare word strings), and CMPSD (compare doubleword strings).

The SCAS instruction subtracts the destination string element from the contents of the EAX, AX, or AL register (depending on operand length) and updates the status flags according to the results. The string element and register contents are not modified. The following "short forms" of the SCAS instruction specify the operand length: SCASB (scan byte string), SCASW (scan word string), and SCASD (scan doubleword string).

The LODS instruction loads the source string element identified by the ESI register into the EAX register (for a doubleword string), the AX register (for a word string), or the AL register (for a byte string). The "short forms" for

this instruction are LODSB (load byte string), LODSW (load word string), and LODSD (load doubleword string). This instruction is usually used in a loop, where other instructions process each element of the string after they are loaded into the target register.

The STOS instruction stores the source string element from the EAX (doubleword string), AX (word string), or AL (byte string) register into the memory location identified with the EDI register. The "short forms" for this instruction are STOSB (store byte string), STOSW (store word string), and STOSD (store doubleword string). This instruction is also normally used in a loop. Here a string is commonly loaded into the register with a LODS instruction, operated on by other instructions, and then stored again in memory with a STOS instruction.

The I/O instructions (see Section 7.3.10, "I/O Instructions") also perform operations on strings in memory.

7.3.9.2 Repeated String Operations

Each of the string instructions described in Section 7.3.9.1 perform one iteration of a string operation. To operate on strings longer than a doubleword, the string instructions can be combined with a repeat prefix (REP) to create a repeating instruction or be placed in a loop.

When used in string instructions, the ESI and EDI registers are automatically incremented or decremented after each iteration of an instruction to point to the next element (byte, word, or doubleword) in the string. String operations can thus begin at higher addresses and work toward lower ones, or they can begin at lower addresses and work toward higher ones. The DF flag in the EFLAGS register controls whether the registers are incremented (DF = 0) or decremented (DF = 1). The STD and CLD instructions set and clear this flag, respectively.

The following repeat prefixes can be used in conjunction with a count in the ECX register to cause a string instruction to repeat:

- **REP** — Repeat while the ECX register not zero.
- **REPE/REPZ** — Repeat while the ECX register not zero and the ZF flag is set.
- **REPNE/REPNZ** — Repeat while the ECX register not zero and the ZF flag is clear.

When a string instruction has a repeat prefix, the operation executes until one of the termination conditions specified by the prefix is satisfied. The REPE/REPZ and REPNE/REPNZ prefixes are used only with the CMPS and SCAS instructions. Also, note that a REP STOS instruction is the fastest way to initialize a large block of memory.

7.3.9.3 Fast-String Operation

To improve performance, some processors support modifications to the processor's operation during the string store operations initiated with the MOVS, MOVSB, STOS, and STOSB instructions. This optimized operation, called **fast-string operation**, is used when the execution of one of those instructions meets certain initial conditions (see below). Instructions using fast-string operation effectively operate on the string in groups that may include multiple elements of the native data size (byte, word, doubleword, or quadword). With fast-string operation, the processor recognizes interrupts and data breakpoints only on boundaries between these groups. Fast-string operation is used only if the source and destination addresses both use either the WB or WC memory types.

The initial conditions for fast-string operation are implementation-specific and may vary with the native string size. Examples of parameters that may impact the use of fast-string operation include the following:

- The alignment indicated in the EDI and ESI alignment registers.
- The address order of the string operation.
- The value of the initial operation counter (ECX).
- The difference between the source and destination addresses.
- The value of the direction flag (RFLAGS.DF).

NOTE

Initial conditions for fast-string operation in future Intel 64 or IA-32 processor families may differ from above. The Intel® 64 and IA-32 Architectures Optimization Reference Manual may contain model-specific information.

Software can disable fast-string operation by clearing the fast-string-enable bit (bit 0) of IA32_MISC_ENABLE MSR. However, Intel recommends that system software always enable fast-string operation.

When fast-string operation is enabled (because IA32_MISC_ENABLE[0] = 1), some processors may further enhance the operation of the REP MOVSB and REP STOSB instructions. A processor supports these enhancements if CPUID.07H.00H:EBX[9] is 1. The Intel® 64 and IA-32 Architectures Optimization Reference Manual may include model-specific recommendations for use of these enhancements.

The stores produced by fast-string operation may appear to execute out of order or be repeated. Software dependent upon sequential store ordering should not use string operations for the entire data structure to be stored. Data and semaphores should be separated. Order-dependent code should write to a discrete semaphore variable after any string operations to allow correctly ordered data to be seen by all processors. Atomicity of load and store operations is guaranteed only for native data elements of the string with native data size, and only if they are included in a single cache line. See Section 11.2.4, "Fast-String Operation and Out-of-Order Stores," of the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3A.

7.3.9.4 String Operations in 64-Bit Mode

The behavior of MOVS (Move String), CMPS (Compare string), SCAS (Scan string), LODS (Load string), and STOS (Store string) instructions in 64-bit mode is similar to their behavior in non-64-bit modes, with the following differences:

- The source operand is specified by RSI or DS:ESI, depending on the address size attribute of the operation.
- The destination operand is specified by RDI or DS:EDI, depending on the address size attribute of the operation.
- Operation on 64-bit data is supported by using the REX.W prefix.

When using REP prefixes for string operations in 64-bit mode, the repeat count is specified by RCX or ECX (depending on the address size attribute of the operation). The default address size is 64 bits.

7.3.10 I/O Instructions

The IN (input from port to register), INS (input from port to string), OUT (output from register to port), and OUTS (output string to port) instructions move data between the processor's I/O ports and either a register or memory.

The register I/O instructions (IN and OUT) move data between an I/O port and the EAX register (32-bit I/O), the AX register (16-bit I/O), or the AL (8-bit I/O) register. The I/O port being read or written to is specified with an immediate operand or an address in the DX register.

The block I/O instructions (INS and OUTS) instructions move blocks of data (strings) between an I/O port and memory. These instructions operate similar to the string instructions (see Section 7.3.9, "String Operations"). The ESI and EDI registers are used to specify string elements in memory and the repeat prefix (REP) is used to repeat the instructions to implement block moves. The assembler recognizes the following alternate mnemonics for these instructions: INSB (input byte), INSW (input word), and INSD (input doubleword), and OUTSB (output byte), OUTSW (output word), and OUTSD (output doubleword).

The INS and OUTS instructions use an address in the DX register to specify the I/O port to be read or written to.

7.3.11 I/O Instructions in 64-Bit Mode

For I/O instructions to and from memory, the differences in 64-bit mode are:

- The source operand is specified by RSI or DS:ESI, depending on the address size attribute of the operation.
- The destination operand is specified by RDI or DS:EDI, depending on the address size attribute of the operation.
- Operation on 64-bit data is not encodable and REX prefixes are silently ignored.

7.3.12 Enter and Leave Instructions

The ENTER and LEAVE instructions provide machine-language support for procedure calls in block-structured languages, such as C and Pascal. These instructions and the call and return mechanism that they support are described in detail in Section 6.6, "Procedure Calls for Block-Structured Languages."

7.3.13 Flag Control (EFLAG) Instructions

The Flag Control (EFLAG) instructions allow the state of selected flags in the EFLAGS register to be read or modified. For the purpose of this discussion, these instructions are further divided into subordinate subgroups of instructions that manipulate:

- Carry and direction flags.
- The EFLAGS register.
- Interrupt flags.

7.3.13.1 Carry and Direction Flag Instructions

The STC (set carry flag), CLC (clear carry flag), and CMC (complement carry flag) instructions allow the CF flag in the EFLAGS register to be modified directly. They are typically used to initialize the CF flag to a known state before an instruction that uses the flag in an operation is executed. They are also used in conjunction with the rotate-with-carry instructions (RCL and RCR).

The STD (set direction flag) and CLD (clear direction flag) instructions allow the DF flag in the EFLAGS register to be modified directly. The DF flag determines the direction in which index registers ESI and EDI are stepped when executing string processing instructions. If the DF flag is clear, the index registers are incremented after each iteration of a string instruction; if the DF flag is set, the registers are decremented.

7.3.13.2 EFLAGS Transfer Instructions

The EFLAGS transfer instructions allow groups of flags in the EFLAGS register to be copied to a register or memory or be loaded from a register or memory.

The LAHF (load AH from flags) and SAHF (store AH into flags) instructions operate on five of the EFLAGS status flags (SF, ZF, AF, PF, and CF). The LAHF instruction copies the status flags to bits 7, 6, 4, 2, and 0 of the AH register, respectively. The contents of the remaining bits in the register (bits 5, 3, and 1) are unaffected, and the contents of the EFLAGS register remain unchanged. The SAHF instruction copies bits 7, 6, 4, 2, and 0 from the AH register into the SF, ZF, AF, PF, and CF flags, respectively in the EFLAGS register.

The PUSHF (push flags), PUSHFD (push flags double), POPF (pop flags), and POPFD (pop flags double) instructions copy the flags in the EFLAGS register to and from the stack. The PUSHF instruction pushes the lower word of the EFLAGS register onto the stack (see Figure 7-11). The PUSHFD instruction pushes the entire EFLAGS register onto the stack (with the RF and VM flags read as clear).

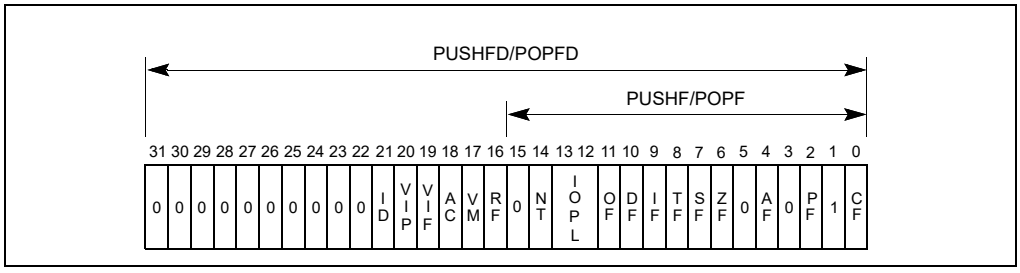


Figure 7-11. Flags Affected by the PUSHF, POPF, PUSHFD, and POPFD Instructions

The POPF instruction pops a word from the stack into the EFLAGS register. Only bits 11, 10, 8, 7, 6, 4, 2, and 0 of the EFLAGS register are affected with all uses of this instruction. If the current privilege level (CPL) of the current

code segment is 0 (most privileged), the IOPL bits (bits 13 and 12) also are affected. If the I/O privilege level (IOPL) is greater than or equal to the CPL, numerically, the IF flag (bit 9) also is affected.

The POPFD instruction pops a doubleword into the EFLAGS register. This instruction can change the state of the AC bit (bit 18) and the ID bit (bit 21), as well as the bits affected by a POPF instruction. The restrictions for changing the IOPL bits and the IF flag that were given for the POPF instruction also apply to the POPFD instruction.

7.3.13.3 Interrupt Flag Instructions

The STI (set interrupt flag) and CLI (clear interrupt flag) instructions allow the interrupt IF flag in the EFLAGS register to be modified directly. The IF flag controls the servicing of hardware-generated interrupts (those received at the processor's INTR pin). If the IF flag is set, the processor services hardware interrupts; if the IF flag is clear, hardware interrupts are masked.

The ability to execute these instructions depends on the operating mode of the processor and the current privilege level (CPL) of the program or task attempting to execute these instructions.

7.3.14 Flag Control (RFLAG) Instructions in 64-Bit Mode

In 64-bit mode, the LAHF and SAHF instructions are supported if CPUID.80000001H:ECX.LAHF_SAHF_64[0] = 1.

PUSHF and POPF behave the same in 64-bit mode as in non-64-bit mode. PUSHFD always pushes 64-bit RFLAGS onto the stack (with the RF and VM flags read as clear). POPFD always pops a 64-bit value from the top of the stack and loads the lower 32 bits into RFLAGS. It then zero extends the upper bits of RFLAGS.

7.3.15 Segment Register Instructions

The processor provides a variety of instructions that address the segment registers of the processor directly. These instructions are only used when an operating system or executive is using the segmented or the real-address mode memory model.

For the purpose of this discussion, these instructions are divided into subordinate subgroups of instructions that allow:

- Segment-register load and store.
- Far control transfers.
- Software interrupt calls.
- Handling of far pointers.

7.3.15.1 Segment-Register Load and Store Instructions

The MOV instruction (introduced in Section 7.3.1.1, "General Data Movement Instructions") and the PUSH and POP instructions (introduced in Section 7.3.1.4, "Stack Manipulation Instructions") can transfer 16-bit segment selectors to and from segment registers (DS, ES, FS, GS, and SS). The transfers are always made to or from a segment register and a general-purpose register or memory. Transfers between segment registers are not supported.

The POP and MOV instructions cannot place a value in the CS register. Only the far control-transfer versions of the JMP, CALL, and RET instructions (see Section 7.3.15.2, "Far Control Transfer Instructions") affect the CS register directly.

7.3.15.2 Far Control Transfer Instructions

The JMP and CALL instructions (see Section 7.3.8, "Control Transfer Instructions") both accept a far pointer as a destination to transfer program control to a segment other than the segment currently being pointed to by the CS register. When a far call is made with the CALL instruction, the current values of the EIP and CS registers are both pushed on the stack.

The RET instruction (see "Call and return instructions" on page 7-15) can be used to execute a far return. Here, program control is transferred from a code segment that contains a called procedure back to the code segment that

contained the calling procedure. The RET instruction restores the values of the CS and EIP registers for the calling procedure from the stack.

7.3.15.3 Software Interrupt Instructions

The software interrupt instructions INT, INTO, and IRET (see Section 7.3.8.4, "Software Interrupt Instructions") can also call and return from interrupt and exception handler procedures that are located in a code segment other than the current code segment. With these instructions, however, the switching of code segments is handled transparently from the application program.

7.3.15.4 Load Far Pointer Instructions

The load far pointer instructions LDS (load far pointer using DS), LES (load far pointer using ES), LFS (load far pointer using FS), LGS (load far pointer using GS), and LSS (load far pointer using SS) load a far pointer from memory into a segment register and a general-purpose general register. The segment selector part of the far pointer is loaded into the selected segment register and the offset is loaded into the selected general-purpose register.

7.3.16 Miscellaneous Instructions

The following instructions perform operations that are of interest to applications programmers. For the purpose of this discussion, these instructions are further divided into subordinate subgroups of instructions that provide for:

- Address computations.
- Table lookup.
- Processor identification.
- NOP and undefined instruction entry.

7.3.16.1 Address Computation Instruction

The LEA (load effective address) instruction computes the effective address in memory (offset within a segment) of a source operand and places it in a general-purpose register. This instruction can interpret any of the processor's addressing modes and can perform any indexing or scaling that may be needed. It is especially useful for initializing the ESI or EDI registers before the execution of string instructions or for initializing the EBX register before an XLAT instruction.

7.3.16.2 Table Lookup Instructions

The XLAT and XLATB (table lookup) instructions replace the contents of the AL register with a byte read from a translation table in memory. The initial value in the AL register is interpreted as an unsigned index into the translation table. This index is added to the contents of the EBX register (which contains the base address of the table) to calculate the address of the table entry. These instructions are used for applications such as converting character codes from one alphabet into another (for example, an ASCII code could be used to look up its EBCDIC equivalent in a table).

7.3.16.3 Processor Identification Instruction

The CPUID (processor identification) instruction returns information about the processor on which the instruction is executed.

7.3.16.4 No-Operation and Undefined Instructions

The NOP (no operation) instruction increments the EIP register to point at the next instruction, but affects nothing else.

The UD (undefined) instruction generates an invalid opcode exception. Intel reserves the opcode for this instruction for this function. The instruction is provided to allow software to test an invalid opcode exception handler.

7.3.17 Random Number Generator Instructions

The instructions for generating random numbers to comply with NIST SP800-90A, SP800-90B, and SP800-90C standards are described in this section.

7.3.17.1 RDRAND

The RDRAND instruction returns a random number. All Intel processors that support the RDRAND instruction indicate the availability of the RDRAND instruction via reporting CPUID.01H:ECX.RDRAND[30] = 1.

RDRAND returns random numbers that are supplied by a cryptographically secure, deterministic random bit generator DRBG. The DRBG is designed to meet the NIST SP 800-90A standard. The DRBG is re-seeded frequently from an on-chip non-deterministic entropy source to guarantee data returned by RDRAND is statistically uniform, non-periodic and non-deterministic.

In order for the hardware design to meet its security goals, the random number generator continuously tests itself and the random data it is generating. Runtime failures in the random number generator circuitry or statistically anomalous data occurring by chance will be detected by the self test hardware and flag the resulting data as being bad. In such extremely rare cases, the RDRAND instruction will return no data instead of bad data.

Under heavy load, with multiple cores executing RDRAND in parallel, it is possible, though unlikely, for the demand of random numbers by software processes/threads to exceed the rate at which the random number generator hardware can supply them. This will lead to the RDRAND instruction returning no data transitorily. The RDRAND instruction indicates the occurrence of this rare situation by clearing the CF flag.

The RDRAND instruction returns with the carry flag set (CF = 1) to indicate valid data is returned. It is recommended that software using the RDRAND instruction to get random numbers retry for a limited number of iterations while RDRAND returns CF = 0 and complete when valid data is returned, indicated with CF = 1. This will deal with transitory underflows. A retry limit should be employed to prevent a hard failure in the RNG (expected to be extremely rare) leading to a busy loop in software.

The intrinsic primitive for RDRAND is defined to address software's need for the common cases (CF = 1) and the rare situations (CF = 0). The intrinsic primitive returns a value that reflects the value of the carry flag returned by the underlying RDRAND instruction. The example below illustrates the recommended usage of an RDRAND intrinsic in a utility function, a loop to fetch a 64 bit random value with a retry count limit of 10. A C implementation might be written as follows:

```
-------------------------------------------------------------------------------
#define SUCCESS 1
#define RETRY_LIMIT_EXCEEDED 0
#define RETRY_LIMIT 10

int get_random_64(unsigned __int 64 * arand)
{int i ;
    for (i = 0; i < RETRY_LIMIT; i ++) {
        if(_rdrand64_step(arand) ) return SUCCESS;
    }
    return RETRY_LIMIT_EXCEEDED;
}
-------------------------------------------------------------------------------
```

7.3.17.2 RDSEED

The RDSEED instruction returns a random number. All Intel processors that support the RDSEED instruction indicate the availability of the RDSEED instruction via reporting CPUID.07H.00H:EBX.RDSEED[18] = 1.

RDSEED returns random numbers that are supplied by a cryptographically secure, enhanced non-deterministic random bit generator (Enhanced NRBG). The NRBG is designed to meet the NIST SP 800-90B and NIST SP800-90C standards.

In order for the hardware design to meet its security goals, the random number generator continuously tests itself and the random data it is generating. Runtime failures in the random number generator circuitry or statistically anomalous data occurring by chance will be detected by the self test hardware and flag the resulting data as being bad. In such extremely rare cases, the RDSEED instruction will return no data instead of bad data.

Under heavy load, with multiple cores executing RDSEED in parallel, it is possible for the demand of random numbers by software processes/threads to exceed the rate at which the random number generator hardware can supply them. This will lead to the RDSEED instruction returning no data transitorily. The RDSEED instruction indicates the occurrence of this situation by clearing the CF flag.

The RDSEED instruction returns with the carry flag set (CF = 1) to indicate valid data is returned. It is recommended that software using the RDSEED instruction to get random numbers retry for a limited number of iterations while RDSEED returns CF = 0 and complete when valid data is returned, indicated with CF = 1. This will deal with transitory underflows. A retry limit should be employed to prevent a hard failure in the NRBG (expected to be extremely rare) leading to a busy loop in software.

The intrinsic primitive for RDSEED is defined to address software’s need for the common cases (CF = 1) and the rare situations (CF = 0). The intrinsic primitive returns a value that reflects the value of the carry flag returned by the underlying RDSEED instruction.

CHAPTER 8
PROGRAMMING WITH THE X87 FPU

The x87 Floating-Point Unit (FPU) provides high-performance floating-point processing capabilities for use in graphics processing, scientific, engineering, and business applications. It supports the floating-point, integer, and packed BCD integer data types and the floating-point processing algorithms and exception handling architecture defined in the IEEE Standard 754 for Floating-Point Arithmetic.

This chapter describes the x87 FPU execution environment and instruction set. It also provides exception handling information that is specific to the x87 FPU. Refer to the following chapters or sections of chapters for additional information about x87 FPU instructions and floating-point operations:

- The Intel® 64 and IA-32 Architectures Software Developer's Manual, Volumes 2A, 2B, 2C, & 2D, provides detailed descriptions of x87 FPU instructions.
- Section 4.2.2, "Floating-Point Data Types," Section 4.2.1.2, "Signed Integers," and Section 4.7, "BCD and Packed BCD Integers," describe the floating-point, integer, and BCD data types.
- Section 4.9, "Overview of Floating-Point Exceptions," Section 4.9.1, "Floating-Point Exception Conditions," and Section 4.9.2, "Floating-Point Exception Priority," give an overview of the floating-point exceptions that the x87 FPU can detect and report.

8.1 X87 FPU EXECUTION ENVIRONMENT

The x87 FPU represents a separate execution environment within the IA-32 architecture (see Figure 8-1). This execution environment consists of eight data registers (called the x87 FPU data registers) and the following special-purpose registers:

- Status register.
- Control register.
- Tag word register.
- Last instruction pointer register.
- Last data (operand) pointer register.
- Opcode register.

These registers are described in the following sections.

The x87 FPU executes instructions from the processor's normal instruction stream. The state of the x87 FPU is independent from the state of the basic execution environment and from the state of SSE/SSE2/SSE3 extensions.

However, the x87 FPU and Intel MMX technology share state because the MMX registers are aliased to the x87 FPU data registers. Therefore, when writing code that uses x87 FPU and MMX instructions, the programmer must explicitly manage the x87 FPU and MMX state (see Section 9.5, "Compatibility with x87 FPU Architecture").

8.1.1 x87 FPU in 64-Bit Mode and Compatibility Mode

In compatibility mode and 64-bit mode, x87 FPU instructions function like they do in protected mode. Memory operands are specified using the ModR/M, SIB encoding that is described in Section 3.7.5, "Specifying an Offset."

8.1.2 x87 FPU Data Registers

The x87 FPU data registers (shown in Figure 8-1) consist of eight 80-bit registers. Values are stored in these registers in the double extended precision floating-point format shown in Figure 4-3. When floating-point, integer, or packed BCD integer values are loaded from memory into any of the x87 FPU data registers, the values are automatically converted into double extended precision floating-point format (if they are not already in that format). When computation results are subsequently transferred back into memory from any of the x87 FPU registers, the

results can be left in the double extended precision floating-point format or converted back into a shorter floating-point format, an integer format, or the packed BCD integer format. (See Section 8.2, "x87 FPU Data Types," for a description of the data types operated on by the x87 FPU.)

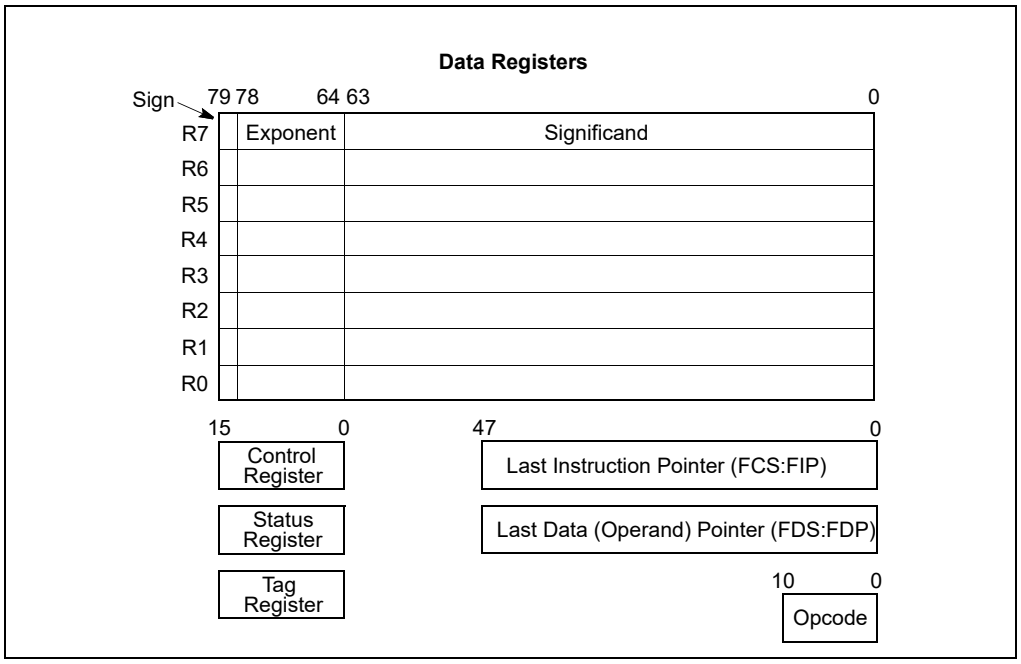


Figure 8-1. x87 FPU Execution Environment

The x87 FPU instructions treat the eight x87 FPU data registers as a register stack (see Figure 8-2). All addressing of the data registers is relative to the register on the top of the stack. The register number of the current top-of-stack register is stored in the TOP (stack TOP) field in the x87 FPU status word. Load operations decrement TOP by one and load a value into the new top-of-stack register, and store operations store the value from the current TOP register in memory and then increment TOP by one. (For the x87 FPU, a load operation is equivalent to a push and a store operation is equivalent to a pop.) Note that load and store operations are also available that do not push and pop the stack.

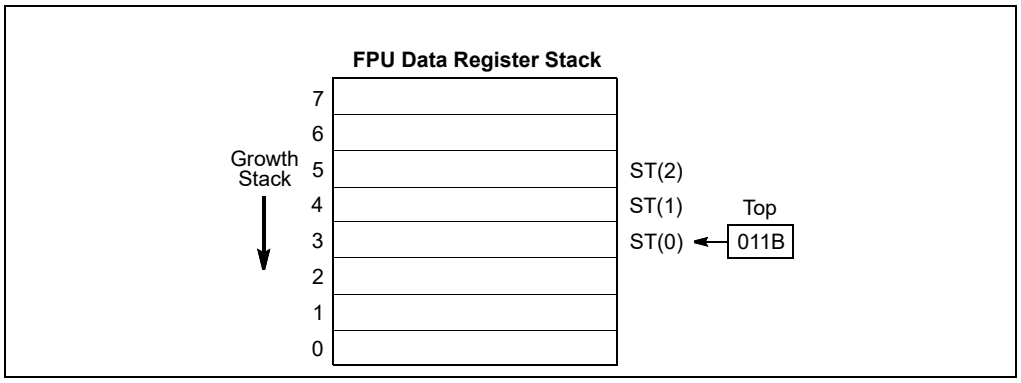


Figure 8-2. x87 FPU Data Register Stack

If a load operation is performed when TOP is at 0, register wraparound occurs and the new value of TOP is set to 7. The floating-point stack-overflow exception indicates when wraparound might cause an unsaved value to be overwritten (see Section 8.5.1.1, "Stack Overflow or Underflow Exception (#IS)").

Many floating-point instructions have several addressing modes that permit the programmer to implicitly operate on the top of the stack, or to explicitly operate on specific registers relative to the TOP. Assemblers support these

register addressing modes, using the expression ST(0), or simply ST, to represent the current stack top and ST(i) to specify the *i*th register from TOP in the stack ($0 \le i \le 7$). For example, if TOP contains 011B (register 3 is the top of the stack), the following instruction would add the contents of two registers in the stack (registers 3 and 5):

```
FADD ST, ST(2);
```

Figure 8-3 shows an example of how the stack structure of the x87 FPU registers and instructions are typically used to perform a series of computations. Here, a two-dimensional dot product is computed, as follows:

1. The first instruction (FLD value1) decrements the stack register pointer (TOP) and loads the value 5.6 from memory into ST(0). The result of this operation is shown in snap-shot (a).
2. The second instruction multiplies the value in ST(0) by the value 2.4 from memory and stores the result in ST(0), shown in snap-shot (b).
3. The third instruction decrements TOP and loads the value 3.8 in ST(0).
4. The fourth instruction multiplies the value in ST(0) by the value 10.3 from memory and stores the result in ST(0), shown in snap-shot (c).
5. The fifth instruction adds the value and the value in ST(1) and stores the result in ST(0), shown in snap-shot (d).

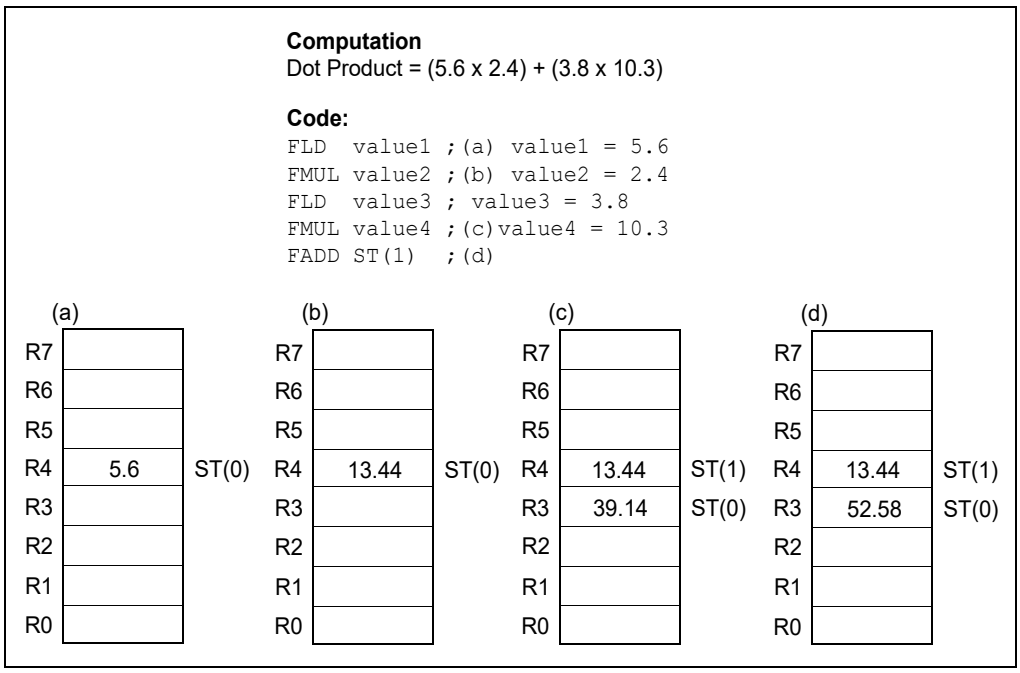


Figure 8-3. Example x87 FPU Dot Product Computation

The style of programming demonstrated in this example is supported by the floating-point instruction set. In cases where the stack structure causes computation bottlenecks, the FXCH (exchange x87 FPU register contents) instruction can be used to streamline a computation.

8.1.2.1 Parameter Passing With the x87 FPU Register Stack

Like the general-purpose registers, the contents of the x87 FPU data registers are unaffected by procedure calls, or in other words, the values are maintained across procedure boundaries. A calling procedure can thus use the x87 FPU data registers (as well as the procedure stack) for passing parameter between procedures. The called procedure can reference parameters passed through the register stack using the current stack register pointer (TOP) and the ST(0) and ST(i) nomenclature. It is also common practice for a called procedure to leave a return value or result in register ST(0) when returning execution to the calling procedure or program.

When mixing MMX and x87 FPU instructions in the procedures or code sequences, the programmer is responsible for maintaining the integrity of parameters being passed in the x87 FPU data registers. If an MMX instruction is executed before the parameters in the x87 FPU data registers have been passed to another procedure, the parameters may be lost (see Section 9.5, "Compatibility with x87 FPU Architecture").

8.1.3 x87 FPU Status Register

The 16-bit x87 FPU status register (see Figure 8-4) indicates the current state of the x87 FPU. The flags in the x87 FPU status register include the FPU busy flag, top-of-stack (TOP) pointer, condition code flags, exception summary status flag, stack fault flag, and exception flags. The x87 FPU sets the flags in this register to show the results of operations.

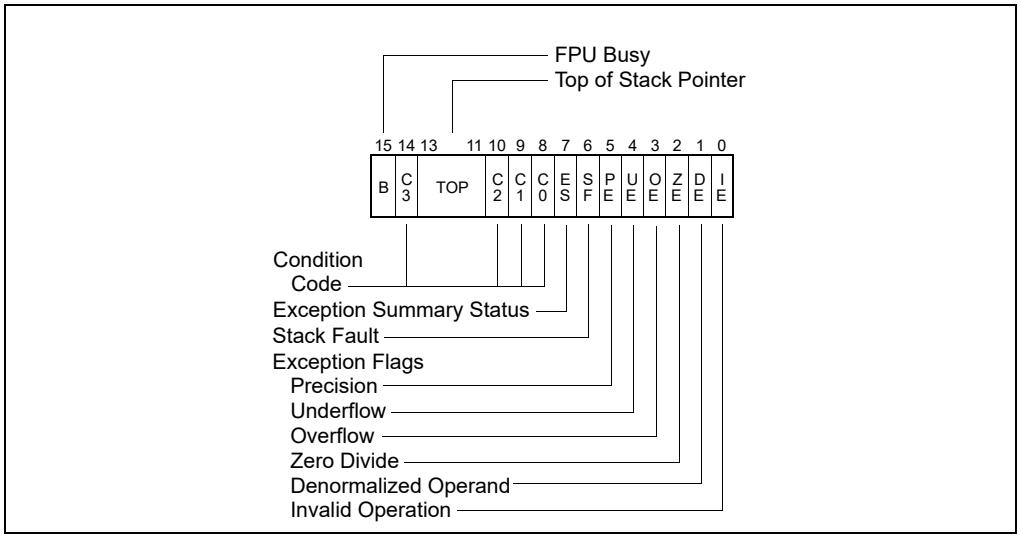


Figure 8-4. x87 FPU Status Word

The contents of the x87 FPU status register (referred to as the x87 FPU status word) can be stored in memory using the FSTSW/FNSTSW, FSTENV/FNSTENV, FSAVE/FNSAVE, and FXSAVE instructions. It can also be stored in the AX register of the integer unit, using the FSTSW/FNSTSW instructions.

8.1.3.1 Top of Stack (TOP) Pointer

A pointer to the x87 FPU data register that is currently at the top of the x87 FPU register stack is contained in bits 11 through 13 of the x87 FPU status word. This pointer, which is commonly referred to as TOP (for top-of-stack), is a binary value from 0 to 7. See Section 8.1.2, "x87 FPU Data Registers," for more information about the TOP pointer.

8.1.3.2 Condition Code Flags

The four condition code flags (C0 through C3) indicate the results of floating-point comparison and arithmetic operations. Table 8-1 summarizes the manner in which the floating-point instructions set the condition code flags. These condition code bits are used principally for conditional branching and for storage of information used in exception handling (see Section 8.1.4, "Branching and Conditional Moves on Condition Codes").

As shown in Table 8-1, the C1 condition code flag is used for a variety of functions. When both the IE and SF flags in the x87 FPU status word are set, indicating a stack overflow or underflow exception (#IS), the C1 flag distinguishes between overflow (C1 = 1) and underflow (C1 = 0). When the PE flag in the status word is set, indicating an inexact (rounded) result, the C1 flag is set to 1 if the last rounding by the instruction was upward. The FXAM instruction sets C1 to the sign of the value being examined.

The C2 condition code flag is used by the FPREM and FPREM1 instructions to indicate an incomplete reduction (or partial remainder). When a successful reduction has been completed, the C0, C3, and C1 condition code flags are set to the three least-significant bits of the quotient (Q2, Q1, and Q0, respectively). See "FPREM1—Partial Remainder" in Chapter 3, "Instruction Set Reference, A-L," of the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 2A, for more information on how these instructions use the condition code flags.

The FPTAN, FSIN, FCOS, and FSINCOS instructions set the C2 flag to 1 to indicate that the source operand is beyond the allowable range of $\pm 2^{63}$ and clear the C2 flag if the source operand is within the allowable range.

Where the state of the condition code flags are listed as undefined in Table 8-1, do not rely on any specific value in these flags.

8.1.3.3 x87 FPU Floating-Point Exception Flags

The six x87 FPU floating-point exception flags (bits 0 through 5) of the x87 FPU status word indicate that one or more floating-point exceptions have been detected since the bits were last cleared. The individual exception flags (IE, DE, ZE, OE, UE, and PE) are described in detail in Section 8.4, "x87 FPU Floating-Point Exception Handling." Each of the exception flags can be masked by an exception mask bit in the x87 FPU control word (see Section 8.1.5, "x87 FPU Control Word"). The exception summary status flag (ES, bit 7) is set when any of the unmasked exception flags are set. When the ES flag is set, the x87 FPU exception handler is invoked, using one of the techniques described in Section 8.7, "Handling x87 FPU Exceptions in Software." (Note that if an exception flag is masked, the x87 FPU will still set the appropriate flag if the associated exception occurs, but it will not set the ES flag.)

The exception flags are "sticky" bits (once set, they remain set until explicitly cleared). They can be cleared by executing the FCLEX/FNCLEX (clear exceptions) instructions, by reinitializing the x87 FPU with the FINIT/FNINIT or FSAVE/FNSAVE instructions, or by overwriting the flags with an FRSTOR or FLDENV instruction.

The B-bit (bit 15) is included for 8087 compatibility only. It reflects the contents of the ES flag.

Table 8-1. Condition Code Interpretation

Instruction	C0	C3	C2	C1
FCOM, FCOMP, FCOMPP, FICOM, FICOMP, FTST, FUCOM, FUCOMP, FUCOMPP	Result of Comparison		Operands are not Comparable	0 or #IS
FCOMI, FCOMIP, FUCOMI, FUCOMIP	Undefined. (These instructions set the status flags in the EFLAGS register.)			#IS
FXAM	Operand class			Sign
FPREM, FPREM1	Q2	Q1	0 = reduction complete 1 = reduction incomplete	Q0 or #IS
F2XM1, FADD, FADDP, FBSTP, FCMOVcc, FIADD, FDIV, FDIVP, FDIVR, FDIVRP, FIDIV, FIDIVR, FIMUL, FIST, FISTP, FISUB, FISUBR, FMUL, FMULP, FPATAN, FRNDINT, FSCALE, FST, FSTP, FSUB, FSUBP, FSUBR, FSUBRP, FSQRT, FYL2X, FYL2XP1	Undefined			Roundup or #IS
FCOS, FSIN, FSINCOS, FPTAN	Undefined		0 = source operand within range 1 = source operand out of range	Roundup or #IS (Undefined if C2 = 1)
FABS, FBLD, FCHS, FDECSTP, FILD, FINCSTP, FLD, Load Constants, FSTP (ext. prec.), FXCH, FXTRACT	Undefined			0 or #IS

Table 8-1. Condition Code Interpretation (Contd.)

FLDENV, FRSTOR	Each bit loaded from memory			
FFREE, FLDCW, FCLEX/FNCLEX, FNOP, FSTCW/FNSTCW, FSTENV/FNSTENV, FSTSW/FNSTSW,	Undefined			
FINIT/FNINIT, FSAVE/FNSAVE	0	0	0	0

8.1.3.4 Stack Fault Flag

The stack fault flag (bit 6 of the x87 FPU status word) indicates that stack overflow or stack underflow has occurred with data in the x87 FPU data register stack. The x87 FPU explicitly sets the SF flag when it detects a stack overflow or underflow condition, but it does not explicitly clear the flag when it detects an invalid-arithmetic-operand condition.

When this flag is set, the condition code flag C1 indicates the nature of the fault: overflow (C1 = 1) and underflow (C1 = 0). The SF flag is a "sticky" flag, meaning that after it is set, the processor does not clear it until it is explicitly instructed to do so (for example, by an FINIT/FNINIT, FCLEX/FNCLEX, or FSAVE/FNSAVE instruction).

See Section 8.1.7, "x87 FPU Tag Word," for more information on x87 FPU stack faults.

8.1.4 Branching and Conditional Moves on Condition Codes

The x87 FPU (beginning with the P6 family processors) supports two mechanisms for branching and performing conditional moves according to comparisons of two floating-point values. These mechanism are referred to here as the "old mechanism" and the "new mechanism."

The old mechanism is available in the x87 FPU prior to the P6 family processors and in P6 family processors. This mechanism uses the floating-point compare instructions (FCOM, FCOMP, FCOMPP, FTST, FUCOMPP, FICOM, and FICOMP) to compare two floating-point values and set the condition code flags (C0 through C3) according to the results. The contents of the condition code flags are then copied into the status flags of the EFLAGS register using a two step process (see Figure 8-5):

1. The FSTSW AX instruction moves the x87 FPU status word into the AX register.
2. The SAHF instruction copies the upper 8 bits of the AX register, which includes the condition code flags, into the lower 8 bits of the EFLAGS register.

When the condition code flags have been loaded into the EFLAGS register, conditional jumps or conditional moves can be performed based on the new settings of the status flags in the EFLAGS register.

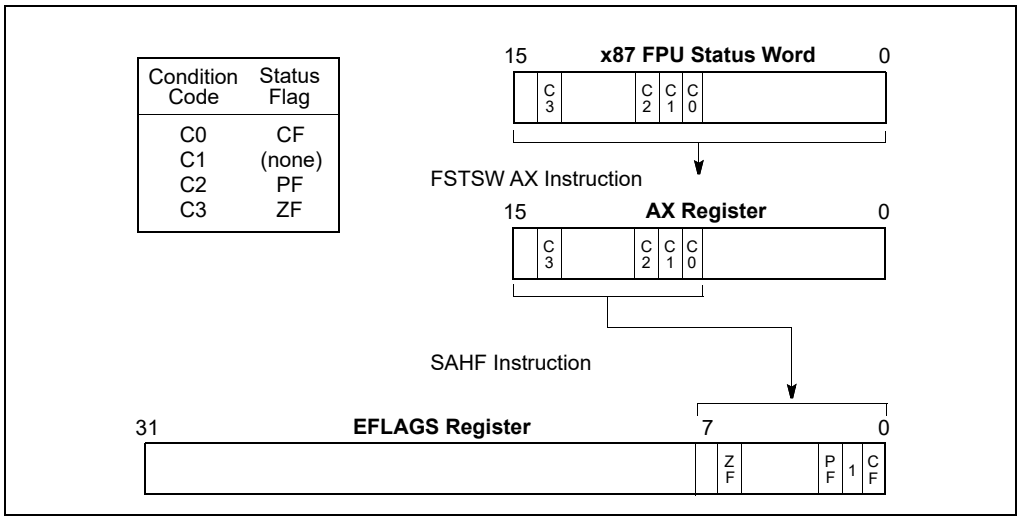


Figure 8-5. Moving the Condition Codes to the EFLAGS Register

The new mechanism is available beginning with the P6 family processors. Using this mechanism, the new floating-point compare and set EFLAGS instructions (FCOMI, FCOMIP, FUCOMI, and FUCOMIP) compare two floating-point values and set the ZF, PF, and CF flags in the EFLAGS register directly. A single instruction thus replaces the three instructions required by the old mechanism.

Note also that the FCMOV*cc* instructions (also new in the P6 family processors) allow conditional moves of floating-point values (values in the x87 FPU data registers) based on the setting of the status flags (ZF, PF, and CF) in the EFLAGS register. These instructions eliminate the need for an IF statement to perform conditional moves of floating-point values.

8.1.5 x87 FPU Control Word

The 16-bit x87 FPU control word (see Figure 8-6) controls the precision of the x87 FPU and rounding method used. It also contains the x87 FPU floating-point exception mask bits. The control word is cached in the x87 FPU control register. The contents of this register can be loaded with the FLDCW instruction and stored in memory with the FSTCW/FNSTCW instructions.

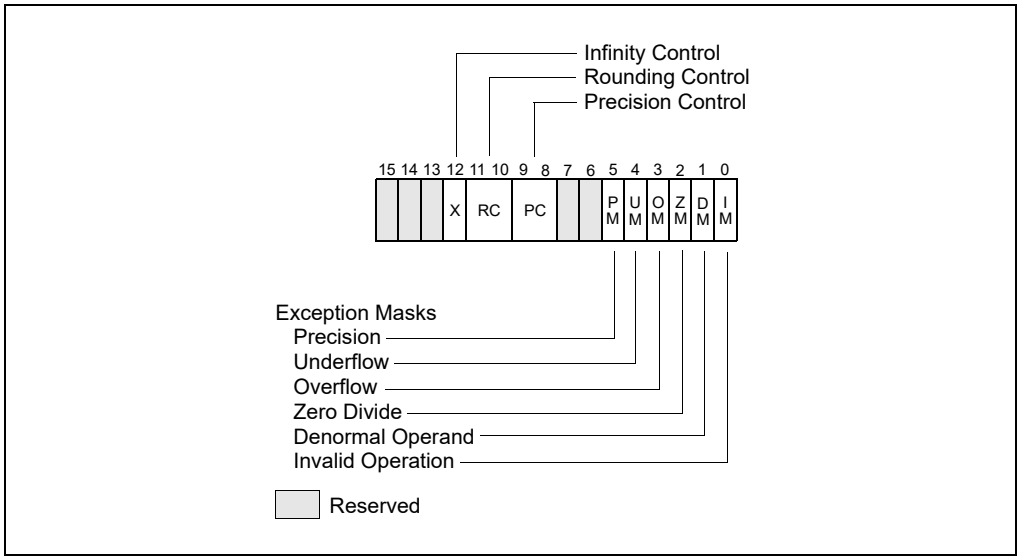


Figure 8-6. x87 FPU Control Word

When the x87 FPU is initialized with either an FINIT/FNINIT or FSAVE/FNSAVE instruction, the x87 FPU control word is set to 037FH, which masks all floating-point exceptions, sets rounding to nearest, and sets the x87 FPU precision to 64 bits.

8.1.5.1 x87 FPU Floating-Point Exception Mask Bits

The exception-flag mask bits (bits 0 through 5 of the x87 FPU control word) mask the 6 floating-point exception flags in the x87 FPU status word. When one of these mask bits is set, its corresponding x87 FPU floating-point exception is blocked from being generated.

8.1.5.2 Precision Control Field

The precision-control (PC) field (bits 8 and 9 of the x87 FPU control word) determines the precision (64, 53, or 24 bits) of floating-point calculations made by the x87 FPU (see Table 8-2). The default precision is double extended precision, which uses the full 64-bit significand available with the double extended precision floating-point format of the x87 FPU data registers. This setting is best suited for most applications, because it allows applications to take full advantage of the maximum precision available with the x87 FPU data registers.

Table 8-2. Precision Control Field (PC)

Precision	PC Field
Single Precision (24 bits)	00B
Reserved	01B
Double Precision (53 bits)	10B
Double Extended Precision (64 bits)	11B

The double precision and single precision settings reduce the size of the significand to 53 bits and 24 bits, respectively. These settings are provided to support IEEE Standard 754 and to provide compatibility with the specifications of certain existing programming languages. Using these settings nullifies the advantages of the double extended precision floating-point format's 64-bit significand length. When reduced precision is specified, the rounding of the significand value clears the unused bits on the right to zeros.

The precision-control bits only affect the results of the following floating-point instructions: FADD, FADDP, FIADD, FSUB, FSUBP, FISUB, FSUBR, FSUBRP, FISUBR, FMUL, FMULP, FIMUL, FDIV, FDIVP, FIDIV, FDIVR, FDIVRP, FIDIVR, and FSQRT.

8.1.5.3 Rounding Control Field

The rounding-control (RC) field of the x87 FPU control register (bits 10 and 11) controls how the results of x87 FPU floating-point instructions are rounded. See Section 4.8.4, "Rounding," for a discussion of rounding of floating-point values; See Section 4.8.4.1, "Rounding Control (RC) Fields," for the encodings of the RC field.

8.1.6 Infinity Control Flag

The infinity control flag (bit 12 of the x87 FPU control word) is provided for compatibility with the Intel 287 Math Coprocessor; it is not meaningful for later version x87 FPU coprocessors or IA-32 processors. See Section 4.8.3.3, "Signed Infinities," for information on how the x87 FPUs handle infinity values.

8.1.7 x87 FPU Tag Word

The 16-bit tag word (see Figure 8-7) indicates the contents of each the 8 registers in the x87 FPU data-register stack (one 2-bit tag per register). The tag codes indicate whether a register contains a valid number, zero, or a special floating-point number (NaN, infinity, denormal, or unsupported format), or whether it is empty. The x87 FPU tag word is cached in the x87 FPU in the x87 FPU tag word register. When the x87 FPU is initialized with either an FINIT/FNINIT or FSAVE/FNSAVE instruction, the x87 FPU tag word is set to FFFFH, which marks all the x87 FPU data registers as empty.

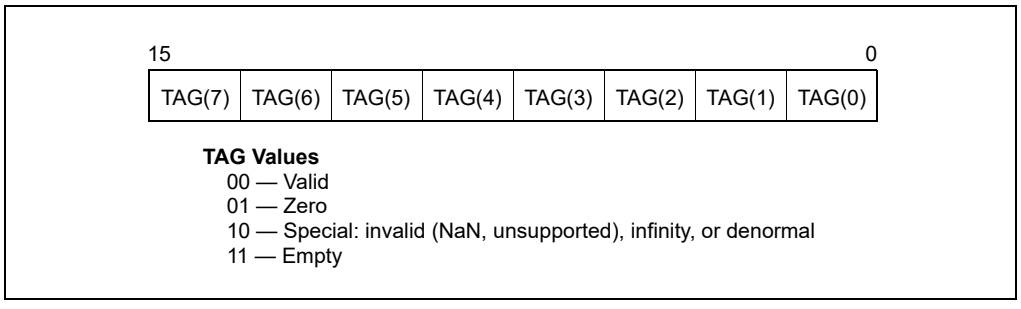


Figure 8-7. x87 FPU Tag Word

Each tag in the x87 FPU tag word corresponds to a physical register (numbers 0 through 7). The current top-of-stack (TOP) pointer stored in the x87 FPU status word can be used to associate tags with registers relative to ST(0).

The x87 FPU uses the tag values to detect stack overflow and underflow conditions (see Section 8.5.1.1, “Stack Overflow or Underflow Exception (#IS)”).

Application programs and exception handlers can use this tag information to check the contents of an x87 FPU data register without performing complex decoding of the actual data in the register. To read the tag register, it must be stored in memory using either the FSTENV/FNSTENV or FSAVE/FNSAVE instructions. The location of the tag word in memory after being saved with one of these instructions is shown in Figures 8-9 through 8-12.

Software cannot directly load or modify the tags in the tag register. The FLDENV and FRSTOR instructions load an image of the tag register into the x87 FPU; however, the x87 FPU uses those tag values only to determine if the data registers are empty (11B) or non-empty (00B, 01B, or 10B).

If the tag register image indicates that a data register is empty, the tag in the tag register for that data register is marked empty (11B); if the tag register image indicates that the data register is non-empty, the x87 FPU reads the actual value in the data register and sets the tag for the register accordingly. This action prevents a program from setting the values in the tag register to incorrectly represent the actual contents of non-empty data registers.

8.1.8 x87 FPU Instruction and Data (Operand) Pointers

The x87 FPU stores pointers to the instruction and data (operand) for the last non-control instruction executed. These are the x87 FPU instruction pointer and x87 FPU data (operand) pointers; software can save these pointers to provide state information for exception handlers. The pointers are illustrated in Figure 8-1 (the figure illustrates the pointers as used outside 64-bit mode; see below).

Note that the value in the x87 FPU data pointer is always a pointer to a memory operand. If the last non-control instruction that was executed did not have a memory operand, the value in the data pointer is undefined (reserved). If CPUID.07H.00H:EBX[6] = 1, the data pointer is updated only for x87 non-control instructions that incur unmasked x87 exceptions.

The contents of the x87 FPU instruction and data pointers remain unchanged when any of the following instructions are executed: FCLEX/FNCLEX, FLDCW, FSTCW/FNSTCW, FSTSW/FNSTSW, FSTENV/FNSTENV, FLDENV, and WAIT/FWAIT.

For all the x87 FPUs and Numeric Processor Extensions (NPXs) except the 8087, the x87 FPU instruction pointer points to any prefixes that preceded the instruction. For the 8087, the x87 FPU instruction pointer points only to the actual opcode.

The x87 FPU instruction and data pointers each consists of an offset and a segment selector:

- The x87 FPU Instruction Pointer Offset (FIP) comprises 64 bits on processors that support IA-32e mode; on other processors, it offset comprises 32 bits.
- The x87 FPU Instruction Pointer Selector (FCS) comprises 16 bits.
- The x87 FPU Data Pointer Offset (FDP) comprises 64 bits on processors that support IA-32e mode; on other processors, it offset comprises 32 bits.
- The x87 FPU Data Pointer Selector (FDS) comprises 16 bits.

The pointers are accessed by the FINIT/FNINIT, FLDENV, FRSTOR, FSAVE/FNSAVE, FSTENV/FNSTENV, FXRSTOR, FXSAVE, XRSTOR, XSAVE, and XSAVEOPT instructions as follows:

- FINIT/FNINIT. Each instruction clears FIP, FCS, FDP, and FDS.
- FLDENV, FRSTOR. These instructions use the memory formats given in Figures 8-9 through 8-12:
  - For each of FIP and FDP, each instruction loads the lower 32 bits from memory and clears the upper 32 bits.
  - If CR0.PE = 1, each instruction loads FCS and FDS from memory; otherwise, it clears them.
- FSAVE/FNSAVE, FSTENV/FNSTENV. These instructions use the memory formats given in Figures 8-9 through 8-12.
  - Each instruction saves the lower 32 bits of each FIP and FDP into memory. the upper 32 bits are not saved.
  - If CR0.PE = 1, each instruction saves FCS and FDS into memory. If CPUID.07H.00H:EBX[13] = 1, the processor deprecates FCS and FDS; it saves each as 0000H.
  - After saving these data into memory, FSAVE/FNSAVE clears FIP, FCS, FDP, and FDS.

- FXRSTOR, XRSTOR. These instructions load data from a memory image whose format depend on operating mode and the REX prefix. The memory formats are given in Tables 3-45, 3-48, and 3-49 in Chapter 3, "Instruction Set Reference, A-L," of the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 2A.
  - Outside of 64-bit mode or if REX.W = 0, the instructions operate as follows:
    - For each of FIP and FDP, each instruction loads the lower 32 bits from memory and clears the upper 32 bits.
    - Each instruction loads FCS and FDS from memory.
  - In 64-bit mode with REX.W = 1, the instructions operate as follows:
    - Each instruction loads FIP and FDP from memory.
    - Each instruction clears FCS and FDS.
- FXSAVE, XSAVE, and XSAVEOPT. These instructions store data into a memory image whose format depend on operating mode and the REX prefix. The memory formats are given in Tables 3-45, 3-48, and 3-49 in Chapter 3, "Instruction Set Reference, A-L," of the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 2A.
  - Outside of 64-bit mode or if REX.W = 0, the instructions operate as follows:
    - Each instruction saves the lower 32 bits of each of FIP and FDP into memory. The upper 32 bits are not saved.
    - Each instruction saves FCS and FDS into memory. If CPUID.07H.00H:EBX[13] = 1, the processor deprecates FCS and FDS; it saves each as 0000H.
  - In 64-bit mode with REX.W = 1, each instruction saves FIP and FDP into memory. FCS and FDS are not saved.

8.1.9 Last Instruction Opcode

The x87 FPU stores in the 11-bit x87 FPU opcode register (FOP) the opcode of the last x87 non-control instruction executed that incurred an unmasked x87 exception. (This information provides state information for exception handlers.) Only the first and second opcode bytes (after all prefixes) are stored in the x87 FPU opcode register. Figure 8-8 shows the encoding of these two bytes. Since the upper 5 bits of the first opcode byte are the same for all floating-point opcodes (11011B), only the lower 3 bits of this byte are stored in the opcode register.

8.1.9.1 Fopcode Compatibility Sub-mode

Some Pentium 4 and Intel Xeon processors provide program control over the value stored into FOP. Here, bit 2 of the IA32_MISC_ENABLE MSR enables (set) or disables (clear) the fopcode compatibility mode.

If fopcode compatibility mode is enabled, FOP is defined as it had been in previous IA-32 implementations, as the opcode of the last x87 non-control instruction executed (even if that instruction did not incur an unmasked x87 exception).

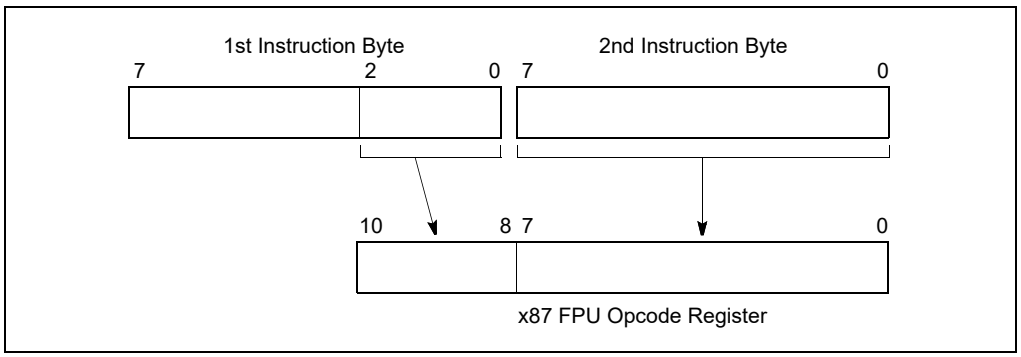


Figure 8-8. Contents of x87 FPU Opcode Registers

The fopcode compatibility mode should be enabled only when x87 FPU floating-point exception handlers are designed to use the fopcode to analyze program performance or restart a program after an exception has been handled.

More recent Intel 64 processors do not support fopcode compatibility mode and do not allow software to set bit 2 of the IA32_MISC_ENABLE MSR.

8.1.10 Saving the x87 FPU State with FSTENV/FNSTENV and FSAVE/FNSAVE

The FSTENV/FNSTENV and FSAVE/FNSAVE instructions store x87 FPU state information in memory for use by exception handlers and other system and application software. The FSTENV/FNSTENV instruction saves the contents of the status, control, tag, x87 FPU instruction pointer, x87 FPU data pointer, and opcode registers. The FSAVE/FNSAVE instruction stores that information plus the contents of the x87 FPU data registers. Note that the FSAVE/FNSAVE instruction also initializes the x87 FPU to default values (just as the FINIT/FNINIT instruction does) after it has saved the original state of the x87 FPU.

The manner in which this information is stored in memory depends on the operating mode of the processor (protected mode or real-address mode) and on the operand-size attribute in effect (32-bit or 16-bit). See Figures 8-9 through 8-12. In virtual-8086 mode or SMM, the real-address mode formats shown in Figure 8-12 is used. See Chapter 34, "System Management Mode," of the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3C, for information on using the x87 FPU while in SMM.

The FLDENV and FRSTOR instructions allow x87 FPU state information to be loaded from memory into the x87 FPU. Here, the FLDENV instruction loads only the status, control, tag, x87 FPU instruction pointer, x87 FPU data pointer, and opcode registers, and the FRSTOR instruction loads all the x87 FPU registers, including the x87 FPU stack registers.

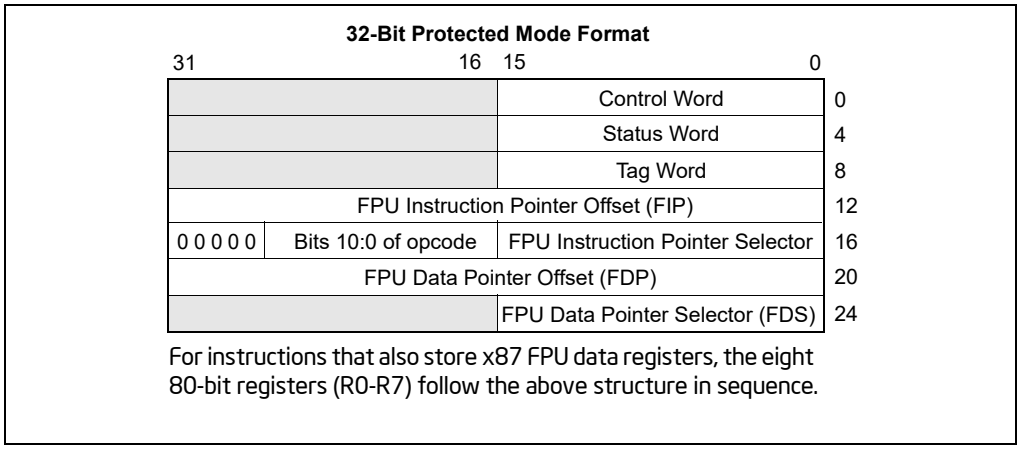


Figure 8-9. Protected Mode x87 FPU State Image in Memory, 32-Bit Format

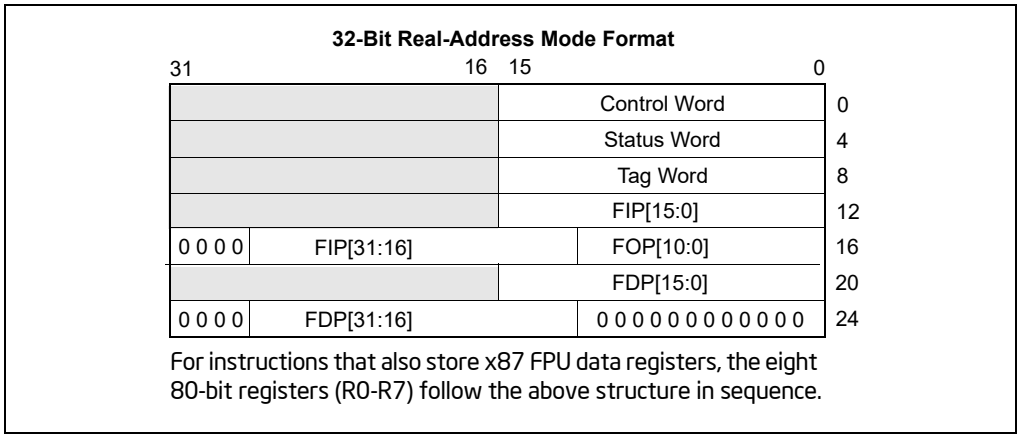


Figure 8-10. Real Mode x87 FPU State Image in Memory, 32-Bit Format

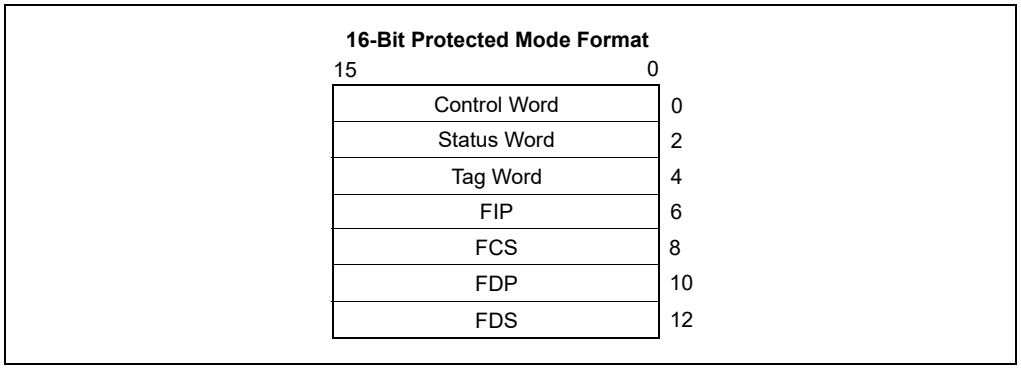


Figure 8-11. Protected Mode x87 FPU State Image in Memory, 16-Bit Format

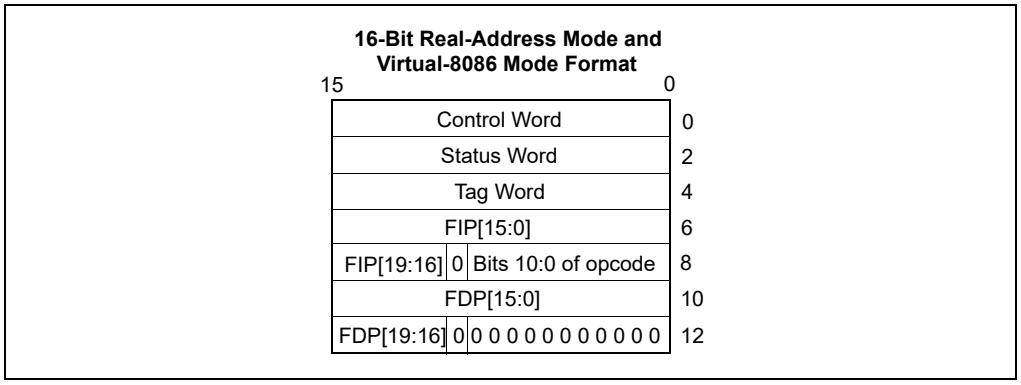


Figure 8-12. Real Mode x87 FPU State Image in Memory, 16-Bit Format

8.1.11 Saving the x87 FPU State with FXSAVE

The FXSAVE and FXRSTOR instructions save and restore, respectively, the x87 FPU state along with the state of the XMM registers and the MXCSR register. Using the FXSAVE instruction to save the x87 FPU state has two benefits: (1) FXSAVE executes faster than FSAVE, and (2) FXSAVE saves the entire x87 FPU, MMX, and XMM state in one operation. See Section 10.5, "FXSAVE and FXRSTOR Instructions," for additional information about these instructions.

8.2 X87 FPU DATA TYPES

The x87 FPU recognizes and operates on the following seven data types (see Figures 8-13): single precision floating-point, double precision floating-point, double extended precision floating-point, signed word integer, signed doubleword integer, signed quadword integer, and packed BCD decimal integers.

For detailed information about these data types, see Section 4.2.2, "Floating-Point Data Types," Section 4.2.1.2, "Signed Integers," and Section 4.7, "BCD and Packed BCD Integers."

With the exception of the 80-bit double extended precision floating-point format, all of these data types exist in memory only. When they are loaded into x87 FPU data registers, they are converted into double extended precision floating-point format and operated on in that format.

Denormal values are also supported in each of the floating-point types, as required by IEEE Standard 754. When a denormal number in single precision or double precision floating-point format is used as a source operand and the denormal exception is masked, the x87 FPU automatically **normalizes** the number when it is converted to double extended precision format.

When stored in memory, the least significant byte of an x87 FPU data-type value is stored at the initial address specified for the value. Successive bytes from the value are then stored in successively higher addresses in memory. The floating-point instructions load and store memory operands using only the initial address of the operand.

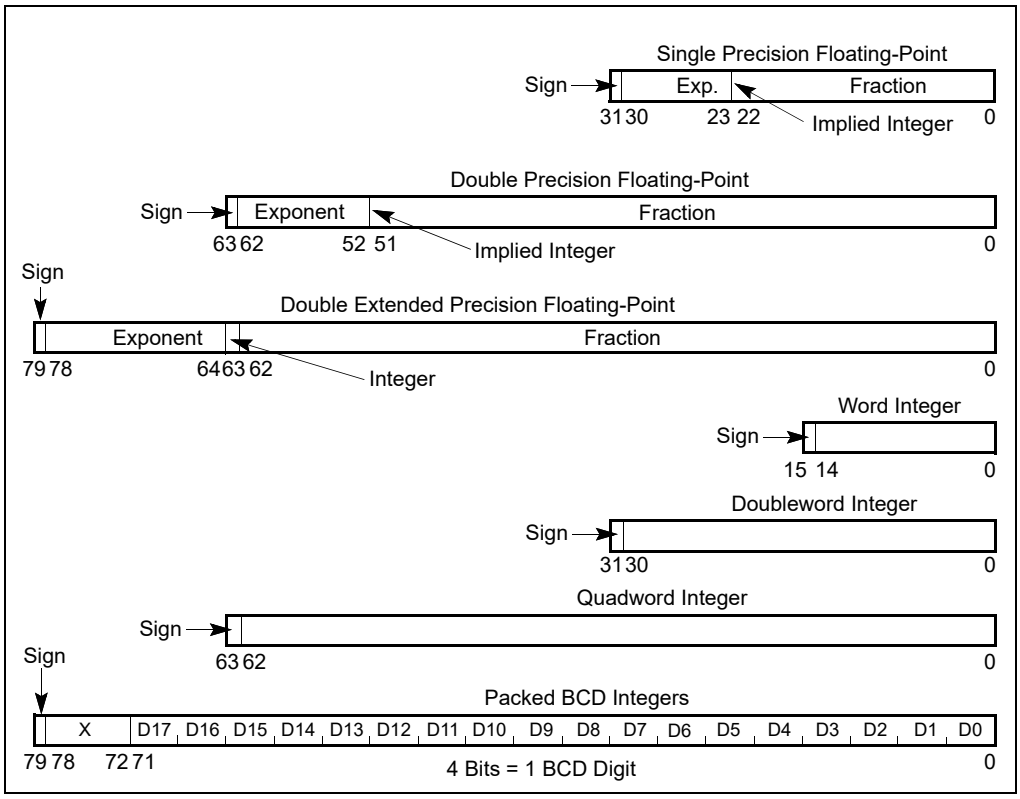


Figure 8-13. x87 FPU Data Type Formats

As a general rule, values should be stored in memory in double precision format. This format provides sufficient range and precision to return correct results with a minimum of programmer attention. The single precision format is useful for debugging algorithms, because rounding problems will manifest themselves more quickly in this format. The double extended precision format is normally reserved for holding intermediate results in the x87 FPU registers and constants. Its extra length is designed to shield final results from the effects of rounding and overflow/underflow in intermediate calculations. However, when an application requires the maximum range and precision of the x87 FPU (for data storage, computations, and results), values can be stored in memory in double extended precision format.

8.2.1 Indefinites

For each x87 FPU data type, one unique encoding is reserved for representing the special value **indefinite**. The x87 FPU produces indefinite values as responses to some masked floating-point invalid-operation exceptions. See Tables 4-1, 4-3, and 4-5 for the encoding of the integer indefinite, QNaN floating-point indefinite, and packed BCD integer indefinite, respectively.

The binary integer encoding 100..00B represents either of two things, depending on the circumstances of its use:

- The largest negative number supported by the format (-2^{15}, -2^{31}, or -2^{63}).
- The **integer indefinite** value.

If this encoding is used as a source operand (as in an integer load or integer arithmetic instruction), the x87 FPU interprets it as the largest negative number representable in the format being used. If the x87 FPU detects an invalid operation when storing an integer value in memory with an FIST/FISTP instruction and the invalid-operation exception is masked, the x87 FPU stores the integer indefinite encoding in the destination operand as a masked response to the exception. In situations where the origin of a value with this encoding may be ambiguous, the invalid-operation exception flag can be examined to see if the value was produced as a response to an exception.

8.2.2 Unsupported Double Extended Precision Floating-Point Encodings and Pseudo-Denormals

The double extended precision floating-point format permits many encodings that do not fall into any of the categories shown in Table 4-3. Table 8-3 shows these unsupported encodings. Some of these encodings were supported by the Intel 287 math coprocessor; however, most of them are not supported by the Intel 387 math coprocessor and later IA-32 processors. These encodings are no longer supported due to changes made in the final version of IEEE Standard 754 that eliminated these encodings.

Specifically, the categories of encodings formerly known as pseudo-NaNs, pseudo-infinities, and un-normal numbers are not supported and should not be used as operand values. The Intel 387 math coprocessor and later IA-32 processors generate an invalid-operation exception when these encodings are encountered as operands.

Beginning with the Intel 387 math coprocessor, the encodings formerly known as pseudo-denormal numbers are not generated by IA-32 processors. When encountered as operands, however, they are handled correctly, considering the biased exponent as 1 (and the unbiased exponent as -16382); that is, they are treated as denormals and a denormal exception is generated. Pseudo-denormal numbers should not be used as operand values. They are supported by current IA-32 processors (as described here) to support legacy code.

Table 8-3. Unsupported Double Extended Precision Floating-Point Encodings and Pseudo-Denormals

Class		Sign	Biased Exponent	Significand	
				Integer	Fraction
Positive Pseudo-NaNs	Quiet	0 . 0	11..11 . 11..11	0	11..11 . 10..00
	Signaling	0 . 0	11..11 . 11..11	0	01..11 . 00..01
Positive Floating-Point	Pseudo-infinity	0	11..11	0	00..00
	Unnormals	0 . 0	11..10 . 00..01	0	11..11 . 00..00
	Pseudo-denormals	0 . 0	00..00 . 00..00	1	11..11 . 00..00

Table 8-3. Unsupported Double Extended Precision Floating-Point Encodings and Pseudo-Denormals (Contd.)

Negative Floating-Point	Pseudo-denormals	1 . 1	00..00 . 00..00	1	11..11 . 00..00
	Unnormals	1 . 1	11..10 . 00..01	0	11..01 . 00..00
	Pseudo-infinity	1	11..11	0	00..00
Negative Pseudo-NaNs	Signaling	1 . 1	11..11 . 11..11	0	01..11 . 00..01
	Quiet	1 . 1	11..11 . 11..11	0	11..11 . 10..00
			← 15 bits →		← 63 bits →

8.3 X87 FPU INSTRUCTION SET

The floating-point instructions that the x87 FPU supports can be grouped into six functional categories:

- Data transfer instructions.
- Basic arithmetic instructions.
- Comparison instructions.
- Transcendental instructions.
- Load constant instructions.
- x87 FPU control instructions.

See Section 5.2, "x87 FPU Instructions," for a list of the floating-point instructions by category.

The following section briefly describes the instructions in each category. Detailed descriptions of the floating-point instructions are given in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volumes 2A, 2B, 2C, & 2D.

8.3.1 Escape (ESC) Instructions

All of the instructions in the x87 FPU instruction set fall into a class of instructions known as escape (ESC) instructions. All of these instructions have a common opcode format, where the first byte of the opcode is one of the numbers from D8H through DFH.

8.3.2 x87 FPU Instruction Operands

Most floating-point instructions require one or two operands, located on the x87 FPU data-register stack or in memory. (None of the floating-point instructions accept immediate operands.)

When an operand is located in a data register, it is referenced relative to the ST(0) register (the register at the top of the register stack), rather than by a physical register number. Often the ST(0) register is an implied operand.

Operands in memory can be referenced using the same operand addressing methods described in Section 3.7, "Operand Addressing."

8.3.3 Data Transfer Instructions

The data transfer instructions (see Table 8-4) perform the following operations:

- Load a floating-point, integer, or packed BCD operand from memory into the ST(0) register.
- Store the value in an ST(0) register to memory in floating-point, integer, or packed BCD format.
- Move values between registers in the x87 FPU register stack.

The FLD (load floating-point) instruction pushes a floating-point operand from memory onto the top of the x87 FPU data-register stack. If the operand is in single precision or double precision floating-point format, it is automatically converted to double extended precision floating-point format. This instruction can also be used to push the value in a selected x87 FPU data register onto the top of the register stack.

The FILD (load integer) instruction converts an integer operand in memory into double extended precision floating-point format and pushes the value onto the top of the register stack. The FBLD (load packed decimal) instruction performs the same load operation for a packed BCD operand in memory.

Table 8-4. Data Transfer Instructions

Floating-Point		Integer		Packed Decimal	
FLD	Load Floating-Point	FILD	Load Integer	FBLD	Load Packed Decimal
FST	Store Floating-Point	FIST	Store Integer		
FSTP	Store Floating-Point and Pop	FISTP	Store Integer and Pop	FBSTP	Store Packed Decimal and Pop
FXCH	Exchange Register Contents				
FCMOV*cc*	Conditional Move				

The FST (store floating-point) and FIST (store integer) instructions store the value in register ST(0) in memory in the destination format (floating-point or integer, respectively). Again, the format conversion is carried out automatically.

The FSTP (store floating-point and pop), FISTP (store integer and pop), and FBSTP (store packed decimal and pop) instructions store the value in the ST(0) registers into memory in the destination format (floating-point, integer, or packed BCD), then performs a **pop** operation on the register stack. A pop operation causes the ST(0) register to be marked empty and the stack pointer (TOP) in the x87 FPU control work to be incremented by 1. The FSTP instruction can also be used to copy the value in the ST(0) register to another x87 FPU register [ST(i)].

The FXCH (exchange register contents) instruction exchanges the value in a selected register in the stack [ST(i)] with the value in ST(0).

The FCMOV*cc* (conditional move) instructions move the value in a selected register in the stack [ST(i)] to register ST(0) if a condition specified with a condition code (*cc*) is satisfied (see Table 8-5). The condition being tested for is represented by the status flags in the EFLAGS register. The condition code mnemonics are appended to the letters "FCMOV" to form the mnemonic for a FCMOV*cc* instruction.

Table 8-5. Floating-Point Conditional Move Instructions

Instruction Mnemonic	Status Flag States	Condition Description
FCMOVB	CF=1	Below
FCMOVNB	CF=0	Not below
FCMOVE	ZF=1	Equal
FCMOVNE	ZF=0	Not equal

Table 8-5. Floating-Point Conditional Move Instructions (Contd.)

Instruction Mnemonic	Status Flag States	Condition Description
FCMOVBE	CF=1 or ZF=1	Below or equal
FCMOVNBE	CF=0 or ZF=0	Not below nor equal
FCMOVU	PF=1	Unordered
FCMOVNU	PF=0	Not unordered

Like the CMOV*cc* instructions, the FCMOV*cc* instructions are useful for optimizing small IF constructions. They also help eliminate branching overhead for IF operations and the possibility of branch mispredictions by the processor.

Software can check if the FCMOV*cc* instructions are supported by checking the processor's feature information with the CPUID instruction.

8.3.4 Load Constant Instructions

The following instructions push commonly used constants onto the top [ST(0)] of the x87 FPU register stack:

FLDZ	Load +0.0.
FLD1	Load +1.0.
FLDPI	Load π.
FLDL2T	Load $\log_2 10$.
FLDL2E	Load $\log_2 e$.
FLDLG2	Load $\log_{10} 2$.
FLDLN2	Load $\log_e 2$.

The constant values have full double extended precision floating-point precision (64 bits) and are accurate to approximately 19 decimal digits. They are stored internally in a format more precise than double extended precision floating-point. When loading the constant, the x87 FPU rounds the more precise internal constant according to the RC (rounding control) field of the x87 FPU control word. The inexact-result exception (#P) is not generated as a result of this rounding, nor is the C1 flag set in the x87 FPU status word if the value is rounded up. See Section 8.3.8, "Approximation of Pi," for information on the π constant.

8.3.5 Basic Arithmetic Instructions

The following floating-point instructions perform basic arithmetic operations on floating-point numbers. Where applicable, these instructions match IEEE Standard 754:

FADD/FADDP	Add floating-point.
FIADD	Add integer to floating-point.
FSUB/FSUBP	Subtract floating-point.
FISUB	Subtract integer from floating-point.
FSUBR/FSUBRP	Reverse subtract floating-point.
FISUBR	Reverse subtract floating-point from integer.
FMUL/FMULP	Multiply floating-point.
FIMUL	Multiply integer by floating-point.
FDIV/FDIVP	Divide floating-point.
FIDIV	Divide floating-point by integer.
FDIVR/FDIVRP	Reverse divide.
FIDIVR	Reverse divide integer by floating-point.
FABS	Absolute value.
FCHS	Change sign.

FSQRT	Square root.
FPREM	Partial remainder.
FPREM1	IEEE partial remainder.
FRNDINT	Round to integral value.
FXTRACT	Extract exponent and significand.

The add, subtract, multiply, and divide instructions operate on the following types of operands:

- Two x87 FPU data registers.
- An x87 FPU data register and a floating-point or integer value in memory.

See Section 8.1.2, "x87 FPU Data Registers," for a description of how operands are referenced on the data register stack.

Operands in memory can be in single precision floating-point, double precision floating-point, word-integer, or doubleword-integer format. They are converted to double extended precision floating-point format automatically.

Reverse versions of the subtract (FSUBR) and divide (FDIVR) instructions enable efficient coding. For example, the following options are available with the FSUB and FSUBR instructions for operating on values in a specified x87 FPU data register ST(*i*) and the ST(0) register:

FSUB:

ST(0) := ST(0) – ST(*i*)
ST(*i*) := ST(*i*) – ST(0)

FSUBR:

ST(0) := ST(*i*) – ST(0)
ST(*i*) := ST(0) – ST(*i*)

These instructions eliminate the need to exchange values between the ST(0) register and another x87 FPU register to perform a subtraction or division.

The pop versions of the add, subtract, multiply, and divide instructions offer the option of popping the x87 FPU register stack following the arithmetic operation. These instructions operate on values in the ST(*i*) and ST(0) registers, store the result in the ST(*i*) register, and pop the ST(0) register.

The FPREM instruction computes the remainder from the division of two operands in the manner used by the Intel 8087 and Intel 287 math coprocessors; the FPREM1 instruction computes the remainder in the manner specified in IEEE Standard 754.

The FSQRT instruction computes the square root of the source operand.

The FRNDINT instruction returns a floating-point value that is the integral value closest to the source value in the direction of the rounding mode specified in the RC field of the x87 FPU control word.

The FABS, FCHS, and FXTRACT instructions perform convenient arithmetic operations. The FABS instruction produces the absolute value of the source operand. The FCHS instruction changes the sign of the source operand. The FXTRACT instruction separates the source operand into its exponent and fraction and stores each value in a register in floating-point format.

8.3.6 Comparison and Classification Instructions

The following instructions compare or classify floating-point values:

FCOM/FCOMP/FCOMPP	Compare floating-point and set x87 FPU condition code flags.
FUCOM/FUCOMP/FUCOMPP	Unordered compare floating-point and set x87 FPU condition code flags.
FICOM/FICOMP	Compare integer and set x87 FPU condition code flags.
FCOMI/FCOMIP	Compare floating-point and set EFLAGS status flags.
FUCOMI/FUCOMIP	Unordered compare floating-point and set EFLAGS status flags.
FTST	Test (compare floating-point with 0.0).

FXAM Examine.

Comparison of floating-point values differ from comparison of integers because floating-point values have four (rather than three) mutually exclusive relationships: less than, equal, greater than, and unordered.

The unordered relationship is true when at least one of the two values being compared is a NaN or in an unsupported format. This additional relationship is required because, by definition, NaNs are not numbers, so they cannot have less than, equal, or greater than relationships with other floating-point values.

The FCOM, FCOMP, and FCOMPP instructions compare the value in register ST(0) with a floating-point source operand and set the condition code flags (C0, C2, and C3) in the x87 FPU status word according to the results (see Table 8-6).

If an unordered condition is detected (one or both of the values are NaNs or in an undefined format), a floating-point invalid-operation exception is generated.

The pop versions of the instruction pop the x87 FPU register stack once or twice after the comparison operation is complete.

The FUCOM, FUCOMP, and FUCOMPP instructions operate the same as the FCOM, FCOMP, and FCOMPP instructions. The only difference is that with the FUCOM, FUCOMP, and FUCOMPP instructions, if an unordered condition is detected because one or both of the operands are QNaNs, the floating-point invalid-operation exception is not generated.

Table 8-6. Setting of x87 FPU Condition Code Flags for Floating-Point Number Comparisons

Condition	C3	C2	C0
ST(0) > Source Operand	0	0	0
ST(0) < Source Operand	0	0	1
ST(0) = Source Operand	1	0	0
Unordered	1	1	1

The FICOM and FICOMP instructions also operate the same as the FCOM and FCOMP instructions, except that the source operand is an integer value in memory. The integer value is automatically converted into an double extended precision floating-point value prior to making the comparison. The FICOMP instruction pops the x87 FPU register stack following the comparison operation.

The FTST instruction performs the same operation as the FCOM instruction, except that the value in register ST(0) is always compared with the value 0.0.

The FCOMI and FCOMIP instructions were introduced into the IA-32 architecture in the P6 family processors. They perform the same comparison as the FCOM and FCOMP instructions, except that they set the status flags (ZF, PF, and CF) in the EFLAGS register to indicate the results of the comparison (see Table 8-7) instead of the x87 FPU condition code flags. The FCOMI and FCOMIP instructions allow condition branch instructions (J*cc*) to be executed directly from the results of their comparison.

Table 8-7. Setting of EFLAGS Status Flags for Floating-Point Number Comparisons

Comparison Results	ZF	PF	CF
ST0 > ST(*i*)	0	0	0
ST0 < ST(*i*)	0	0	1
ST0 = ST(*i*)	1	0	0
Unordered	1	1	1

Software can check if the FCOMI and FCOMIP instructions are supported by checking the processor's feature information with the CPUID instruction.

The FUCOMI and FUCOMIP instructions operate the same as the FCOMI and FCOMIP instructions, except that they do not generate a floating-point invalid-operation exception if the unordered condition is the result of one or both of the operands being a QNaN. The FCOMIP and FUCOMIP instructions pop the x87 FPU register stack following the comparison operation.

The FXAM instruction determines the classification of the floating-point value in the ST(0) register (that is, whether the value is zero, a denormal number, a normal finite number, ∞, a NaN, or an unsupported format) or that the register is empty. It sets the x87 FPU condition code flags to indicate the classification (see "FXAM—Examine" in Chapter 3, "Instruction Set Reference, A-L," of the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 2A). It also sets the C1 flag to indicate the sign of the value.

8.3.6.1 Branching on the x87 FPU Condition Codes

The processor does not offer any control-flow instructions that branch on the setting of the condition code flags (C0, C2, and C3) in the x87 FPU status word. To branch on the state of these flags, the x87 FPU status word must first be moved to the AX register in the integer unit. The FSTSW AX (store status word) instruction can be used for this purpose. When these flags are in the AX register, the TEST instruction can be used to control conditional branching as follows:

1. Check for an unordered result. Use the TEST instruction to compare the contents of the AX register with the constant 0400H (see Table 8-8). This operation will clear the ZF flag in the EFLAGS register if the condition code flags indicate an unordered result; otherwise, the ZF flag will be set. The JNZ instruction can then be used to transfer control (if necessary) to a procedure for handling unordered operands.

Table 8-8. TEST Instruction Constants for Conditional Branching

Order	Constant	Branch
ST(0) > Source Operand	4500H	JZ
ST(0) < Source Operand	0100H	JNZ
ST(0) = Source Operand	4000H	JNZ
Unordered	0400H	JNZ

2. Check ordered comparison result. Use the constants given in Table 8-8 in the TEST instruction to test for a less than, equal to, or greater than result, then use the corresponding conditional branch instruction to transfer program control to the appropriate procedure or section of code.

If a program or procedure has been thoroughly tested and it incorporates periodic checks for QNaN results, then it is not necessary to check for the unordered result every time a comparison is made.

See Section 8.1.4, "Branching and Conditional Moves on Condition Codes," for another technique for branching on x87 FPU condition codes.

Some non-comparison x87 FPU instructions update the condition code flags in the x87 FPU status word. To ensure that the status word is not altered inadvertently, store it immediately following a comparison operation.

8.3.7 Trigonometric Instructions

The following instructions perform four common trigonometric functions:

FSIN	Sine.
FCOS	Cosine.
FSINCOS	Sine and cosine.
FPTAN	Tangent.
FPATAN	Arctangent.

These instructions operate on the top one or two registers of the x87 FPU register stack and they return their results to the stack. The source operands for the FSIN, FCOS, FSINCOS, and FPTAN instructions must be given in radians; the source operand for the FPATAN instruction is given in rectangular coordinate units.

The FSINCOS instruction returns both the sine and the cosine of a source operand value. It operates faster than executing the FSIN and FCOS instructions in succession.

The FPATAN instruction computes the arctangent of ST(1) divided by ST(0), returning a result in radians. It is useful for converting rectangular coordinates to polar coordinates.

See Section 8.3.8, "Approximation of Pi," and Section 8.3.10, "Transcendental Instruction Accuracy," for information regarding the accuracy of these instructions.

8.3.8 Approximation of Pi

When the argument (source operand) of a trigonometric function is within the domain of the function, the argument is automatically reduced by the appropriate multiple of 2π through the same reduction mechanism used by the FPREM and FPREM1 instructions. The internal value of π (3.1415926…) that the x87 FPU uses for argument reduction and other computations, denoted as Pi in the expression below. The numerical value of Pi can be written as:

$$Pi = 0.f * 2^2$$

where the fraction f is expressed in binary form as:

f = C90FDAA2 2168C234 C

(The spaces in the fraction above indicate 32-bit boundaries.)

The internal approximation Pi of the value π has a 66 significant bits. Since the exact value of π represented in binary has the next 3 bits equal to 0, it means that Pi is the value of π rounded to nearest-even to 68 bits, and also the value of π rounded toward zero (truncated) to 69 bits.

However, accuracy problems may arise because this relatively short finite approximation Pi of the number π is used for calculating the reduced argument of the trigonometric function approximations in the implementations of FSIN, FCOS, FSINCOS, and FPTAN. Alternately, this means that FSIN (x), FCOS (x), and FPTAN (x) are really approximating the mathematical functions sin (x * π /Pi), cos (x * π / Pi), and tan (x * π / Pi), and not exactly sin (x), cos (x), and tan (x). (Note that FSINCOS is the equivalent of FSIN and FCOS combined together). The period of sin (x * π /Pi) for example is 2* Pi, and not 2π.

See also Section 8.3.10, "Transcendental Instruction Accuracy," for more information on the accuracy of these functions.

8.3.9 Logarithmic, Exponential, and Scale

The following instructions provide two different logarithmic functions, an exponential function and a scale function:

FYL2X	Logarithm.
FYL2XP1	Logarithm epsilon.
F2XM1	Exponential.
FSCALE	Scale.

The FYL2X and FYL2XP1 instructions perform two different base 2 logarithmic operations. The FYL2X instruction computes ($y * \log_2 x$). This operation permits the calculation of the log of any base using the following equation:

$$\log_b x = (1/\log_2 b) * \log_2 x$$

The FYL2XP1 instruction computes ($y * \log_2(x + 1)$). This operation provides optimum accuracy for values of x that are close to 0.

The F2XM1 instruction computes ($2^x - 1$). This instruction only operates on source values in the range –1.0 to +1.0.

The FSCALE instruction multiplies the source operand by a power of 2.

8.3.10 Transcendental Instruction Accuracy

New transcendental instruction algorithms were incorporated into the IA-32 architecture beginning with the Pentium processors. These new algorithms (used in transcendental instructions FSIN, FCOS, FSINCOS, FPTAN, FPATAN, F2XM1, FYL2X, and FYL2XP1) allow a higher level of accuracy than was possible in earlier IA-32 proces-

sors and x87 math coprocessors. The accuracy of these instructions is measured in terms of **units in the last place** (**ulp**). For a given argument *x*, let *f*(*x*) and *F*(*x*) be the correct and computed (approximate) function values, respectively. The error in ulps is defined to be:

$$error = \left| \frac{f(x) - F(x)}{2^{k-63}} \right|$$

where *k* is an integer such that:

$$1 \le 2^{-k} f(x) < 2.$$

With the Pentium processor and later IA-32 processors, the worst case error on transcendental functions is less than 1 ulp when rounding to the nearest (even) and less than 1.5 ulps when rounding in other modes. The functions are guaranteed to be monotonic, with respect to the input operands, throughout the domain supported by the instruction.

However, for FSIN, FCOS, FSINCOS, and FPTAN which approximate periodic trigonometric functions, the previous statement about maximum ulp errors is true only when these instructions are applied to reduced argument (see Section 8.3.8, "Approximation of Pi"). This is due to the fact that only 66 significant bits are retained in the finite approximation Pi of the number π (3.1415926…), used internally for calculating the reduced argument in FSIN, FCOS, FSINCOS, and FPTAN. This approximation of π is not always sufficiently accurate for good argument reduction.

For single precision, the argument of FSIN, FCOS, FSINCOS, and FPTAN must exceed 200,000 radians in order for the error of the result to exceed 1 ulp when rounding to the nearest (even), or 1.5 ulps when rounding in other (directed) rounding modes.

For double and double-extended precision, the ulp errors will grow above these thresholds for arguments much smaller in magnitude. The ulp errors increase significantly when the argument approaches the value of π (or Pi) for FSIN, and when it approaches π/2(or Pi/2) for FCOS, FSINCOS, and FPTAN.

For all three IEEE precisions supported (32-bit single precision, 64-bit double precision, and 80-bit double-extended precision), applying FSIN, FCOS, FSINCOS, or FPTAN to arguments larger than a certain value can lead to reduced arguments (calculated internally) that are inaccurate or even very inaccurate in some cases. This leads to equally inaccurate approximations of the corresponding mathematical functions. In particular, arguments that are close to certain values will lose significance when reduced, leading to increased relative (and ulp) errors in the results of FSIN, FCOS, FSINCOS, and FPTAN. These values are:

- Any non-zero multiple of π for FSIN.
- Any multiple of π, plus π/2 for FCOS.
- Any non-zero multiple of π/2 for FSINCOS and FPTAN.

If the arguments passed to FSIN, FCOS, FSINCOS, and FPTAN are not close to these values then even the finite approximation Pi of π used internally for argument reduction will allow for results that have good accuracy.

Therefore, in order to avoid such errors it is recommended to perform accurate argument reduction in software, and to apply FSIN, FCOS, FSINCOS, and FPTAN to reduced arguments only. Regardless of the target precision (single, double, or double-extended), it is safe to reduce the argument to a value smaller in absolute value than about 3π/4 for FSIN, and smaller than about 3π/8 for FCOS, FSINCOS, and FPTAN.

The thresholds shown above are not exact. For example, accuracy measurements show that the double-extended precision result of FSIN will not have errors larger than 0.72 ulp for |x| < 2.82 (so |x| < 3π/4 will ensure good accuracy, as 3π/4 < 2.82). On the same interval, double precision results from FSIN will have errors at most slightly larger than 0.5 ulp, and single precision results will be correctly rounded in the vast majority of cases.

Likewise, the double-extended precision result of FCOS will not have errors larger than 0.82 ulp for |x| < 1.31 (so |x| < 3π/8 will ensure good accuracy, as 3π/8 < 1.31). On the same interval, double precision results from FCOS will have errors at most slightly larger than 0.5 ulp, and single precision results will be correctly rounded in the vast majority of cases.

FSINCOS behaves similarly to FSIN and FCOS, combined as a pair.

Finally, the double-extended precision result of FPTAN will not have errors larger than 0.78 ulp for |x| < 1.25 (so |x| < 3π/8 will ensure good accuracy, as 3π/8 < 1.25). On the same interval, double precision results from FPTAN will have errors at most slightly larger than 0.5 ulp, and single precision results will be correctly rounded in the vast majority of cases.

A recommended alternative in order to avoid the accuracy issues that might be caused by FSIN, FCOS, FSINCOS, and FPTAN, is to use good quality mathematical library implementations of the sin, cos, sincos, and tan functions, for example those from the Intel® Math Library available in the Intel® Compiler.

The instructions FYL2X and FYL2XP1 are two operand instructions and are guaranteed to be within 1 ulp only when y equals 1. When y is not equal to 1, the maximum ulp error is always within 1.35 ulps in round to nearest mode. (For the two operand functions, monotonicity was proved by holding one of the operands constant.)

8.3.11 x87 FPU Control Instructions

The following instructions control the state and modes of operation of the x87 FPU. They also allow the status of the x87 FPU to be examined:

FINIT/FNINIT	Initialize x87 FPU.
FLDCW	Load x87 FPU control word.
FSTCW/FNSTCW	Store x87 FPU control word.
FSTSW/FNSTSW	Store x87 FPU status word.
FCLEX/FNCLEX	Clear x87 FPU exception flags.
FLDENV	Load x87 FPU environment.
FSTENV/FNSTENV	Store x87 FPU environment.
FRSTOR	Restore x87 FPU state.
FSAVE/FNSAVE	Save x87 FPU state.
FINCSTP	Increment x87 FPU register stack pointer.
FDECSTP	Decrement x87 FPU register stack pointer.
FFREE	Free x87 FPU register.
FNOP	No operation.
WAIT/FWAIT	Check for and handle pending unmasked x87 FPU exceptions.

The FINIT/FNINIT instructions initialize the x87 FPU and its internal registers to default values.

The FLDCW instructions loads the x87 FPU control word register with a value from memory. The FSTCW/FNSTCW and FSTSW/FNSTSW instructions store the x87 FPU control and status words, respectively, in memory (or for an FSTSW/FNSTSW instruction in a general-purpose register).

The FSTENV/FNSTENV and FSAVE/FNSAVE instructions save the x87 FPU environment and state, respectively, in memory. The x87 FPU environment includes all the x87 FPU's control and status registers; the x87 FPU state includes the x87 FPU environment and the data registers in the x87 FPU register stack. (The FSAVE/FNSAVE instruction also initializes the x87 FPU to default values, like the FINIT/FNINIT instruction, after it saves the original state of the x87 FPU.)

The FLDENV and FRSTOR instructions load the x87 FPU environment and state, respectively, from memory into the x87 FPU. These instructions are commonly used when switching tasks or contexts.

The WAIT/FWAIT instructions are synchronization instructions. (They are actually mnemonics for the same opcode.) These instructions check the x87 FPU status word for pending unmasked x87 FPU exceptions. If any pending unmasked x87 FPU exceptions are found, they are handled before the processor resumes execution of the instructions (integer, floating-point, or system instruction) in the instruction stream. The WAIT/FWAIT instructions are provided to allow synchronization of instruction execution between the x87 FPU and the processor's integer unit. See Section 8.6, "x87 FPU Exception Synchronization," for more information on the use of the WAIT/FWAIT instructions.

8.3.12 Waiting vs. Non-waiting Instructions

All of the x87 FPU instructions except a few special control instructions perform a wait operation (similar to the WAIT/FWAIT instructions), to check for and handle pending unmasked x87 FPU floating-point exceptions, before they perform their primary operation (such as adding two floating-point numbers). These instructions are called **waiting** instructions. Some of the x87 FPU control instructions, such as FSTSW/FNSTSW, have both a waiting and a non-waiting version. The waiting version (with the "F" prefix) executes a wait operation before it performs its primary operation; whereas, the non-waiting version (with the "FN" prefix) ignores pending unmasked exceptions.

Non-waiting instructions allow software to save the current x87 FPU state without first handling pending exceptions or to reset or reinitialize the x87 FPU without regard for pending exceptions.

NOTES

When operating a Pentium or Intel486 processor in MS-DOS compatibility mode, it is possible (under unusual circumstances) for a non-waiting instruction to be interrupted prior to being executed to handle a pending x87 FPU exception.

When operating a P6 family, Pentium 4, or Intel Xeon processor in MS-DOS compatibility mode, non-waiting instructions cannot be interrupted in this way.

8.3.13 Unsupported x87 FPU Instructions

The Intel 8087 instructions FENI and FDISI and the Intel 287 math coprocessor instruction FSETPM perform no function in the Intel 387 math coprocessor and later IA-32 processors. If these opcodes are detected in the instruction stream, the x87 FPU performs no specific operation and no internal x87 FPU states are affected.

8.4 X87 FPU FLOATING-POINT EXCEPTION HANDLING

The x87 FPU detects the six classes of exception conditions described in Section 4.9, "Overview of Floating-Point Exceptions":

- Invalid operation (#I), with two subclasses:
  - Stack overflow or underflow (#IS).
  - Invalid arithmetic operation (#IA).
- Denormalized operand (#D).
- Divide-by-zero (#Z).
- Numeric overflow (#O).
- Numeric underflow (#U).
- Inexact result (precision) (#P).

Each of the six exception classes has a corresponding flag bit in the x87 FPU status word and a mask bit in the x87 FPU control word (see Section 8.1.3, "x87 FPU Status Register," and Section 8.1.5, "x87 FPU Control Word," respectively). In addition, the exception summary (ES) flag in the status word indicates when one or more unmasked exceptions has been detected. The stack fault (SF) flag (also in the status word) distinguishes between the two types of invalid-operation exceptions.

The mask bits can be set with FLDCW, FRSTOR, or FXRSTOR; they can be read with either FSTCW/FNSTCW, FSAVE/FNSAVE, or FXSAVE. The flag bits can be read with the FSTSW/FNSTSW, FSAVE/FNSAVE, or FXSAVE instruction.

NOTE

Section 4.9.1, "Floating-Point Exception Conditions," provides a general overview of how the IA-32 processor detects and handles the various classes of floating-point exceptions. This information pertains to the x87 FPU as well as the Intel SSE, SSE2, and SSE3 instructions.

The following sections give specific information about how the x87 FPU handles floating-point exceptions that are unique to the x87 FPU.

8.4.1 Arithmetic vs. Non-arithmetic Instructions

When dealing with floating-point exceptions, it is useful to distinguish between **arithmetic instructions** and **non-arithmetic instructions**. Non-arithmetic instructions have no operands or do not make substantial changes to their operands. Arithmetic instructions do make significant changes to their operands; in particular, they make changes that could result in floating-point exceptions being signaled. Table 8-9 lists the non-arithmetic and arithmetic instructions. It should be noted that some non-arithmetic instructions can signal a floating-point stack (fault) exception, but this exception is not the result of an operation on an operand.

Table 8-9. Arithmetic and Non-arithmetic Instructions

Non-arithmetic Instructions	Arithmetic Instructions
FABS	F2XM1
FCHS	FADD/FADDP
FCLEX	FBLD
FDECSTP	FBSTP
FFREE	FCOM/FCOMP/FCOMPP
FINCSTP	FCOS
FINIT/FNINIT	FDIV/FDIVP/FDIVR/FDIVRP
FLD (register-to-register)	FIADD
FLD (extended format from memory)	FICOM/FICOMP
FLD constant	FIDIV/FIDIVR
FLDCW	FILD
FLDENV	FIMUL
FNOP	FIST/FISTP[1]
FRSTOR	FISUB/FISUBR
FSAVE/FNSAVE	FLD (single and double)
FST/FSTP (register-to-register)	FMUL/FMULP
FSTP (extended format to memory)	FPATAN
FSTCW/FNSTCW	FPREM/FPREM1
FSTENV/FNSTENV	FPTAN
FSTSW/FNSTSW	FRNDINT
WAIT/FWAIT	FSCALE
FXAM	FSIN
FXCH	FSINCOS
	FSQRT
	FST/FSTP (single and double)
	FSUB/FSUBP/FSUBR/FSUBRP
	FTST
	FUCOM/FUCOMP/FUCOMPP
	FXTRACT
	FYL2X/FYL2XP1

Table 8-9. Arithmetic and Non-arithmetic Instructions (Contd.)

Non-arithmetic Instructions	Arithmetic Instructions
NOTE: 1. The FISTTP instruction in SSE3 is an arithmetic x87 FPU instruction.	

8.5 X87 FPU FLOATING-POINT EXCEPTION CONDITIONS

The following sections describe the various conditions that cause a floating-point exception to be generated by the x87 FPU and the masked response of the x87 FPU when these conditions are detected. The Intel® 64 and IA-32 Architectures Software Developer's Manual, Volumes 2A, 2B, 2C, & 2D, lists the floating-point exceptions that can be signaled for each floating-point instruction.

See Section 4.9.2, "Floating-Point Exception Priority," for a description of the rules for exception precedence when more than one floating-point exception condition is detected for an instruction.

8.5.1 Invalid Operation Exception

The floating-point invalid-operation exception occurs in response to two sub-classes of operations:

- Stack overflow or underflow (#IS).
- Invalid arithmetic operand (#IA).

The flag for this exception (IE) is bit 0 of the x87 FPU status word, and the mask bit (IM) is bit 0 of the x87 FPU control word. The stack fault flag (SF) of the x87 FPU status word indicates the type of operation that caused the exception. When the SF flag is set to 1, a stack operation has resulted in stack overflow or underflow; when the flag is cleared to 0, an arithmetic instruction has encountered an invalid operand. Note that the x87 FPU explicitly sets the SF flag when it detects a stack overflow or underflow condition, but it does not explicitly clear the flag when it detects an invalid-arithmetic-operand condition. As a result, the state of the SF flag can be 1 following an invalid-arithmetic-operation exception, if it was not cleared from the last time a stack overflow or underflow condition occurred. See Section 8.1.3.4, "Stack Fault Flag," for more information about the SF flag.

8.5.1.1 Stack Overflow or Underflow Exception (#IS)

The x87 FPU tag word keeps track of the contents of the registers in the x87 FPU register stack (see Section 8.1.7, "x87 FPU Tag Word"). It then uses this information to detect two different types of stack faults:

- **Stack overflow** — An instruction attempts to load a non-empty x87 FPU register from memory. A non-empty register is defined as a register containing a zero (tag value of 01), a valid value (tag value of 00), or a special value (tag value of 10).
- **Stack underflow** — An instruction references an empty x87 FPU register as a source operand, including attempting to write the contents of an empty register to memory. An empty register has a tag value of 11.

NOTES

The term stack overflow originates from the situation where the program has loaded (pushed) eight values from memory onto the x87 FPU register stack and the next value pushed on the stack causes a stack wraparound to a register that already contains a value.

The term stack underflow originates from the opposite situation. Here, a program has stored (popped) eight values from the x87 FPU register stack to memory and the next value popped from the stack causes stack wraparound to an empty register.

When the x87 FPU detects stack overflow or underflow, it sets the IE flag (bit 0) and the SF flag (bit 6) in the x87 FPU status word to 1. It then sets condition-code flag C1 (bit 9) in the x87 FPU status word to 1 if stack overflow occurred or to 0 if stack underflow occurred.

If the invalid-operation exception is masked, the x87 FPU returns the floating-point, integer, or packed decimal integer indefinite value to the destination operand, depending on the instruction being executed. This value overwrites the destination register or memory location specified by the instruction.

If the invalid-operation exception is not masked, a software exception handler is invoked (see Section 8.7, "Handling x87 FPU Exceptions in Software") and the top-of-stack pointer (TOP) and source operands remain unchanged.

8.5.1.2 Invalid Arithmetic Operand Exception (#IA)

The x87 FPU is able to detect a variety of invalid arithmetic operations that can be coded in a program. These operations are listed in Table 8-10. (This list includes the invalid operations defined in IEEE Standard 754.)

When the x87 FPU detects an invalid arithmetic operand, it sets the IE flag (bit 0) in the x87 FPU status word to 1. If the invalid-operation exception is masked, the x87 FPU then returns an indefinite value or QNaN to the destination operand and/or sets the floating-point condition codes as shown in Table 8-10. If the invalid-operation exception is not masked, a software exception handler is invoked (see Section 8.7, "Handling x87 FPU Exceptions in Software") and the top-of-stack pointer (TOP) and source operands remain unchanged.

Table 8-10. Invalid Arithmetic Operations and the Masked Responses to Them

Condition	Masked Response
Any arithmetic operation on an operand that is in an unsupported format.	Return the QNaN floating-point indefinite value to the destination operand.
Any arithmetic operation on a SNaN.	Return a QNaN to the destination operand (see Table 4-8).
Ordered compare and test operations: one or both operands are NaNs.	Set the condition code flags (C0, C2, and C3) in the x87 FPU status word or the CF, PF, and ZF flags in the EFLAGS register to 111B (not comparable).
Addition: operands are opposite-signed infinities. Subtraction: operands are like-signed infinities.	Return the QNaN floating-point indefinite value to the destination operand.
Multiplication: ∞ by 0; 0 by ∞.	Return the QNaN floating-point indefinite value to the destination operand.
Division: ∞ by ∞; 0 by 0.	Return the QNaN floating-point indefinite value to the destination operand.
Remainder instructions FPREM, FPREM1: modulus (divisor) is 0 or dividend is ∞.	Return the QNaN floating-point indefinite; clear condition code flag C2 to 0.
Trigonometric instructions FCOS, FPTAN, FSIN, FSINCOS: source operand is ∞.	Return the QNaN floating-point indefinite; clear condition code flag C2 to 0.
FSQRT: negative operand (except FSQRT (–0) = –0); FYL2X: negative operand (except FYL2X (–0) = –∞); FYL2XP1: operand more negative than –1.	Return the QNaN floating-point indefinite value to the destination operand.
FBSTP: Converted value cannot be represented in 18 decimal digits, or source value is an SNaN, QNaN, $\pm\infty$, or in an unsupported format.	Store packed BCD integer indefinite value in the destination operand.
FIST/FISTP: Converted value exceeds representable integer range of the destination operand, or source value is an SNaN, QNaN, $\pm\infty$, or in an unsupported format.	Store integer indefinite value in the destination operand.
FXCH: one or both registers are tagged empty.	Load empty registers with the QNaN floating-point indefinite value, then perform the exchange.

Normally, when one or both of the source operands is a QNaN (and neither is an SNaN or in an unsupported format), an invalid-operand exception is not generated. An exception to this rule is most of the compare instructions (such as the FCOM and FCOMI instructions) and the floating-point to integer conversion instructions

(FIST/FISTP and FBSTP). With these instructions, a QNaN source operand will generate an invalid-operand exception.

8.5.2 Denormal Operand Exception (#D)

The x87 FPU signals the denormal-operand exception under the following conditions:

- If an arithmetic instruction attempts to operate on a denormal operand (see Section 4.8.3.2, "Normalized and Denormalized Finite Numbers").
- If an attempt is made to load a denormal single precision or double precision floating-point value into an x87 FPU register. (If the denormal value being loaded is a double extended precision floating-point value, the denormal-operand exception is not reported.)

The flag (DE) for this exception is bit 1 of the x87 FPU status word, and the mask bit (DM) is bit 1 of the x87 FPU control word.

When a denormal-operand exception occurs and the exception is masked, the x87 FPU sets the DE flag, then proceeds with the instruction. The denormal operand in single- or double precision floating-point format is automatically normalized when converted to the double extended precision floating-point format. Subsequent operations will benefit from the additional precision of the internal double extended precision floating-point format.

When a denormal-operand exception occurs and the exception is not masked, the DE flag is set and a software exception handler is invoked (see Section 8.7, "Handling x87 FPU Exceptions in Software"). The top-of-stack pointer (TOP) and source operands remain unchanged.

For additional information about the denormal-operation exception, see Section 4.9.1.2, "Denormal Operand Exception (#D)."

8.5.3 Divide-By-Zero Exception (#Z)

The x87 FPU reports a floating-point divide-by-zero exception whenever an instruction attempts to divide a finite non-zero operand by 0. The flag (ZE) for this exception is bit 2 of the x87 FPU status word, and the mask bit (ZM) is bit 2 of the x87 FPU control word. The FDIV, FDIVP, FDIVR, FDIVRP, FIDIV, and FIDIVR instructions and the other instructions that perform division internally (FYL2X and FXTRACT) can report the divide-by-zero exception.

When a divide-by-zero exception occurs and the exception is masked, the x87 FPU sets the ZE flag and returns the values shown in Table 8-11. If the divide-by-zero exception is not masked, the ZE flag is set, a software exception handler is invoked (see Section 8.7, "Handling x87 FPU Exceptions in Software"), and the top-of-stack pointer (TOP) and source operands remain unchanged.

Table 8-11. Divide-By-Zero Conditions and the Masked Responses to Them

Condition	Masked Response
Divide or reverse divide operation with a 0 divisor.	Returns an ∞ signed with the exclusive OR of the sign of the two operands to the destination operand.
FYL2X instruction.	Returns an ∞ signed with the opposite sign of the non-zero operand to the destination operand.
FXTRACT instruction.	ST(1) is set to –∞; ST(0) is set to 0 with the same sign as the source operand.

8.5.4 Numeric Overflow Exception (#O)

The x87 FPU reports a floating-point numeric overflow exception (#O) whenever the rounded result of an arithmetic instruction exceeds the largest allowable finite value that will fit into the floating-point format of the destination operand. (See Section 4.9.1.4, "Numeric Overflow Exception (#O)," for additional information about the numeric overflow exception.)

When using the x87 FPU, numeric overflow can occur on arithmetic operations where the result is stored in an x87 FPU data register. It can also occur on store floating-point operations (using the FST and FSTP instructions), where a within-range value in a data register is stored in memory in a single precision or double precision floating-point

format. The numeric overflow exception cannot occur when storing values in an integer or BCD integer format. Instead, the invalid-arithmetic-operand exception is signaled.

The flag (OE) for the numeric-overflow exception is bit 3 of the x87 FPU status word, and the mask bit (OM) is bit 3 of the x87 FPU control word.

When a numeric-overflow exception occurs and the exception is masked, the x87 FPU sets the OE flag and returns one of the values shown in Table 4-11. The value returned depends on the current rounding mode of the x87 FPU (see Section 8.1.5.3, "Rounding Control Field").

The action that the x87 FPU takes when numeric overflow occurs and the numeric-overflow exception is not masked, depends on whether the instruction is supposed to store the result in memory or on the register stack.

- **Destination is a memory location —** The OE flag is set and a software exception handler is invoked (see Section 8.7, "Handling x87 FPU Exceptions in Software"). The top-of-stack pointer (TOP) and source and destination operands remain unchanged. Because the data in the stack is in double extended precision format, the exception handler has the option either of re-executing the store instruction after proper adjustment of the operand or of rounding the significand on the stack to the destination's precision as the standard requires. The exception handler should ultimately store a value into the destination location in memory if the program is to continue.
- **Destination is the register stack —** The significand of the result is rounded according to current settings of the precision and rounding control bits in the x87 FPU control word and the exponent of the result is adjusted by dividing it by 2^{24576}. (For instructions not affected by the precision field, the significand is rounded to double-extended precision.) The resulting value is stored in the destination operand. Condition code bit C1 in the x87 FPU status word (called in this situation the "round-up bit") is set if the significand was rounded upward and cleared if the result was rounded toward 0. After the result is stored, the OE flag is set and a software exception handler is invoked. The scaling bias value 24,576 is equal to $3 * 2^{13}$. Biasing the exponent by 24,576 normally translates the number as nearly as possible to the middle of the double extended precision floating-point exponent range so that, if desired, it can be used in subsequent scaled operations with less risk of causing further exceptions.

  When using the FSCALE instruction, massive overflow can occur, where the result is too large to be represented, even with a bias-adjusted exponent. Here, if overflow occurs again, after the result has been biased, a properly signed ∞ is stored in the destination operand.

8.5.5 Numeric Underflow Exception (#U)

The x87 FPU detects a potential floating-point numeric underflow condition whenever the result of an arithmetic instruction is non-zero and tiny; that is, the magnitude of the rounded result with unbounded exponent is non-zero and less than the smallest possible normalized, finite value that will fit into the floating-point format of the destination operand. See Section 4.9.1.5, "Numeric Underflow Exception (#U)," for additional information about the numeric underflow exception.

Like numeric overflow, numeric underflow can occur on arithmetic operations where the result is stored in an x87 FPU data register. It can also occur on store floating-point operations (with the FST and FSTP instructions), where a within-range value in a data register is stored in memory in the smaller single precision or double precision floating-point formats. A numeric underflow exception cannot occur when storing values in an integer or BCD integer format, because a value with magnitude less than 1 is always rounded to an integral value of 0 or 1, depending on the rounding mode in effect.

The flag (UE) for the numeric-underflow exception is bit 4 of the x87 FPU status word, and the mask bit (UM) is bit 4 of the x87 FPU control word.

When a numeric-underflow condition occurs and the exception is masked, the x87 FPU performs the operation described in Section 4.9.1.5, "Numeric Underflow Exception (#U)."

When the exception is not masked, the action of the x87 FPU depends on whether the instruction is supposed to store the result in a memory location or on the x87 FPU register stack.

- **Destination is a memory location —** (Can occur only with a store instruction.) The UE flag is set and a software exception handler is invoked; see Section 8.2, "x87 FPU Data Types." The top-of-stack pointer (TOP) and source and destination operands remain unchanged, and no result is stored in memory.
  Because the data in the stack is in double extended precision format, the exception handler has the option either of re-executing the store instruction after proper adjustment of the operand or of rounding the significand on the stack to the destination's precision as the standard requires. The exception handler should ultimately store a value into the destination location in memory if the program is to continue.
- **Destination is the register stack —** The significand of the result is rounded according to current settings of the precision and rounding control bits in the x87 FPU control word and the exponent of the result is adjusted by multiplying it by 2^{24576}. (For instructions not affected by the precision field, the significand is rounded to double extended precision.) The resulting value is stored in the destination operand. Condition code bit C1 in the x87 FPU status register (acting here as a "round-up bit") is set if the significand was rounded upward and cleared if the result was rounded toward 0. After the result is stored, the UE flag is set and a software exception handler is invoked. The scaling bias value 24,576 is the same as is used for the overflow exception and has the same effect, which is to translate the result as nearly as possible to the middle of the double extended precision floating-point exponent range.

  When using the FSCALE instruction, massive underflow can occur, where the magnitude of the result is too small to be represented, even with a bias-adjusted exponent. Here, if underflow occurs again after the result has been biased, a properly signed 0 is stored in the destination operand.

8.5.6 Inexact-Result (Precision) Exception (#P)

The inexact-result exception (also called the precision exception) occurs if the result of an operation is not exactly representable in the destination format. (See Section 4.9.1.6, "Inexact-Result (Precision) Exception (#P)," for additional information about the numeric overflow exception.) Note that the transcendental instructions (FSIN, FCOS, FSINCOS, FPTAN, FPATAN, F2XM1, FYL2X, and FYL2XP1) by nature produce inexact results.

The inexact-result exception flag (PE) is bit 5 of the x87 FPU status word, and the mask bit (PM) is bit 5 of the x87 FPU control word.

If the inexact-result exception is masked when an inexact-result condition occurs and a numeric overflow or underflow condition has not occurred, the x87 FPU handles the exception as described in Section 4.9.1.6, "Inexact-Result (Precision) Exception (#P)," with one additional action. The C1 (round-up) bit in the x87 FPU status word is set to indicate whether the inexact result was rounded up (C1 is set) or "not rounded up" (C1 is cleared). In the "not rounded up" case, the least-significant bits of the inexact result are truncated so that the result fits in the destination format.

If the inexact-result exception is not masked when an inexact result occurs and numeric overflow or underflow has not occurred, the x87 FPU handles the exception as described in the previous paragraph and, in addition, invokes a software exception handler.

If an inexact result occurs in conjunction with numeric overflow or underflow, the x87 FPU carries out one of the following operations:

- If an inexact result occurs in conjunction with masked overflow or underflow, the OE or UE flag and the PE flag are set and the result is stored as described for the overflow or underflow exceptions (see Section 8.5.4, "Numeric Overflow Exception (#O)," or Section 8.5.5, "Numeric Underflow Exception (#U)"). If the inexact result exception is unmasked, the x87 FPU also invokes a software exception handler.
- If an inexact result occurs in conjunction with unmasked overflow or underflow and the destination operand is a register, the OE or UE flag and the PE flag are set, the result is stored as described for the overflow or underflow exceptions (see Section 8.5.4, "Numeric Overflow Exception (#O)," or Section 8.5.5, "Numeric Underflow Exception (#U)") and a software exception handler is invoked.

If an unmasked numeric overflow or underflow exception occurs and the destination operand is a memory location (which can happen only for a floating-point store), the inexact-result condition is not reported and the C1 flag is cleared.

8.6 X87 FPU EXCEPTION SYNCHRONIZATION

Because the integer unit and x87 FPU are separate execution units, it is possible for the processor to execute floating-point, integer, and system instructions concurrently. No special programming techniques are required to gain the advantages of concurrent execution. (Floating-point instructions are placed in the instruction stream along with the integer and system instructions.) However, concurrent execution can cause problems for floating-point exception handlers.

This problem is related to the way the x87 FPU signals the existence of unmasked floating-point exceptions. (Special exception synchronization is not required for masked floating-point exceptions, because the x87 FPU always returns a masked result to the destination operand.)

When a floating-point exception is unmasked and the exception condition occurs, the x87 FPU stops further execution of the floating-point instruction and signals the exception event. On the next occurrence of a floating-point instruction or a WAIT/FWAIT instruction in the instruction stream, the processor checks the ES flag in the x87 FPU status word for pending floating-point exceptions. If floating-point exceptions are pending, the x87 FPU makes an implicit call (traps) to the floating-point software exception handler. The exception handler can then execute recovery procedures for selected or all floating-point exceptions.

Synchronization problems occur in the time between the moment when the exception is signaled and when it is actually handled. Because of concurrent execution, integer or system instructions can be executed during this time. It is thus possible for the source or destination operands for a floating-point instruction that faulted to be overwritten in memory, making it impossible for the exception handler to analyze or recover from the exception.

To solve this problem, an exception synchronizing instruction (either a floating-point instruction or a WAIT/FWAIT instruction) can be placed immediately after any floating-point instruction that might present a situation where state information pertaining to a floating-point exception might be lost or corrupted. Floating-point instructions that store data in memory are prime candidates for synchronization. For example, the following three lines of code have the potential for exception synchronization problems:

```
FILD COUNT          ;Floating-point instruction
INC COUNT           ;Integer instruction
FSQRT               ;Subsequent floating-point instruction
```

In this example, the INC instruction modifies the source operand of the floating-point instruction, FILD. If an exception is signaled during the execution of the FILD instruction, the INC instruction would be allowed to overwrite the value stored in the COUNT memory location before the floating-point exception handler is called. With the COUNT variable modified, the floating-point exception handler would not be able to recover from the error.

Rearranging the instructions, as follows, so that the FSQRT instruction follows the FILD instruction, synchronizes floating-point exception handling and eliminates the possibility of the COUNT variable being overwritten before the floating-point exception handler is invoked.

```
FILD COUNT          ;Floating-point instruction
FSQRT               ;Subsequent floating-point instruction synchronizes
                    ;any exceptions generated by the FILD instruction.
INC COUNT           ;Integer instruction
```

The FSQRT instruction does not require any synchronization, because the results of this instruction are stored in the x87 FPU data registers and will remain there, undisturbed, until the next floating-point or WAIT/FWAIT instruction is executed. To absolutely ensure that any exceptions emanating from the FSQRT instruction are handled (for example, prior to a procedure call), a WAIT instruction can be placed directly after the FSQRT instruction.

Note that some floating-point instructions (non-waiting instructions) do not check for pending unmasked exceptions (see Section 8.3.11, “x87 FPU Control Instructions”). They include the FNINIT, FNSTENV, FNSAVE, FNSTSW, FNSTCW, and FNCLEX instructions. When an FNINIT, FNSTENV, FNSAVE, or FNCLEX instruction is executed, all pending exceptions are essentially lost (either the x87 FPU status register is cleared or all exceptions are masked). The FNSTSW and FNSTCW instructions do not check for pending interrupts, but they do not modify the x87 FPU status and control registers. A subsequent “waiting” floating-point instruction can then handle any pending exceptions.

8.7 HANDLING X87 FPU EXCEPTIONS IN SOFTWARE

The x87 FPU in Pentium and later IA-32 processors provides two different modes of operation for invoking a software exception handler for floating-point exceptions: native mode and MS-DOS compatibility mode. The mode of operation is selected by CR0.NE[bit 5]. See Chapter 2, "System Architecture Overview," in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3A, for more information about the NE flag.

8.7.1 Native Mode

The native mode for handling floating-point exceptions is selected by setting CR0.NE[bit 5] to 1. In this mode, if the x87 FPU detects an exception condition while executing a floating-point instruction and the exception is unmasked (the mask bit for the exception is cleared), the x87 FPU sets the flag for the exception and the ES flag in the x87 FPU status word. It then invokes the software exception handler through the floating-point-error exception (#MF, exception vector 16), immediately before execution of any of the following instructions in the processor's instruction stream:

- The next floating-point instruction, unless it is one of the non-waiting instructions (FNINIT, FNCLEX, FNSTSW, FNSTCW, FNSTENV, and FNSAVE).
- The next WAIT/FWAIT instruction.
- The next MMX instruction.

If the next floating-point instruction in the instruction stream is a non-waiting instruction, the x87 FPU executes the instruction without invoking the software exception handler.

8.7.2 MS-DOS* Compatibility Sub-mode

If CR0.NE[bit 5] is 0, the MS-DOS compatibility mode for handling floating-point exceptions is selected. In this mode, the software exception handler for floating-point exceptions is invoked externally using the processor's FERR#, INTR, and IGNNE# pins. This method of reporting floating-point errors and invoking an exception handler is provided to support the floating-point exception handling mechanism used in PC systems that are running the MS-DOS or Windows* 95 operating system.

Using FERR# and IGNNE# to handle floating-point exception is deprecated by modern operating systems, this approach also limits newer processors to operate with one logical processor active.

The MS-DOS compatibility mode is typically used as follows to invoke the floating-point exception handler:

1. If the x87 FPU detects an unmasked floating-point exception, it sets the flag for the exception and the ES flag in the x87 FPU status word.
2. If the IGNNE# pin is deasserted, the x87 FPU then asserts the FERR# pin either immediately, or else delayed (deferred) until just before the execution of the next waiting floating-point instruction or MMX instruction. Whether the FERR# pin is asserted immediately or delayed depends on the type of processor, the instruction, and the type of exception.
3. If a preceding floating-point instruction has set the exception flag for an unmasked x87 FPU exception, the processor freezes just before executing the **next** WAIT instruction, waiting floating-point instruction, or MMX instruction. Whether the FERR# pin was asserted at the preceding floating-point instruction or is just now being asserted, the freezing of the processor assures that the x87 FPU exception handler will be invoked before the new floating-point (or MMX) instruction gets executed.
4. The FERR# pin is connected through external hardware to IRQ13 of a cascaded, programmable interrupt controller (PIC). When the FERR# pin is asserted, the PIC is programmed to generate an interrupt 75H.
5. The PIC asserts the INTR pin on the processor to signal the interrupt 75H.
6. The BIOS for the PC system handles the interrupt 75H by branching to the interrupt 02H (NMI) interrupt handler.
7. The interrupt 02H handler determines if the interrupt is the result of an NMI interrupt or a floating-point exception.

8. If a floating-point exception is detected, the interrupt 02H handler branches to the floating-point exception handler.

If the IGNNE# pin is asserted, the processor ignores floating-point error conditions. This pin is provided to inhibit floating-point exceptions from being generated while the floating-point exception handler is servicing a previously signaled floating-point exception.

Appendix D, "Guidelines for Writing SIMD Floating-Point Exception Handlers," describes the MS-DOS compatibility mode in much greater detail. This mode is somewhat more complicated in the Intel486 and Pentium processor implementations, as described in Appendix D.

8.7.3 Handling x87 FPU Exceptions in Software

Section 4.9.3, "Typical Actions of a Floating-Point Exception Handler," shows actions that may be carried out by a floating-point exception handler. The state of the x87 FPU can be saved with the FSTENV/FNSTENV or FSAVE/FNSAVE instructions; see Section 8.1.10, "Saving the x87 FPU State with FSTENV/FNSTENV and FSAVE/FNSAVE."

If the faulting floating-point instruction is followed by one or more non-floating-point instructions, it may not be useful to re-execute the faulting instruction. See Section 8.6, "x87 FPU Exception Synchronization," for more information on synchronizing floating-point exceptions.

In cases where the handler needs to restart program execution with the faulting instruction, the IRET instruction cannot be used directly. The reason for this is that because the exception is not generated until the next floating-point or WAIT/FWAIT instruction following the faulting floating-point instruction, the return instruction pointer on the stack may not point to the faulting instruction. To restart program execution at the faulting instruction, the exception handler must obtain a pointer to the instruction from the saved x87 FPU state information, load it into the return instruction pointer location on the stack, and then execute the IRET instruction.

CHAPTER 9
PROGRAMMING WITH INTEL® MMX™ TECHNOLOGY

The Intel MMX technology was introduced into the IA-32 architecture in the Pentium II processor family and Pentium processor with MMX technology. The extensions introduced in MMX technology support a single-instruction, multiple-data (SIMD) execution model that is designed to accelerate the performance of advanced media and communications applications.

This chapter describes MMX technology.

9.1 OVERVIEW OF MMX TECHNOLOGY

MMX technology defines a simple and flexible SIMD execution model to handle 64-bit packed integer data. This model adds the following features to the IA-32 architecture, while maintaining backwards compatibility with all IA-32 applications and operating-system code:

- Eight new 64-bit data registers, called MMX registers.
- Three new packed data types:
  - 64-bit packed byte integers (signed and unsigned).
  - 64-bit packed word integers (signed and unsigned).
  - 64-bit packed doubleword integers (signed and unsigned).
- Instructions that support the new data types and to handle MMX state management.
- Extensions to the CPUID instruction.

MMX technology is accessible from all the IA32-architecture execution modes (protected mode, real address mode, and virtual 8086 mode). It does not add any new modes to the architecture.

The following sections of this chapter describe MMX technology's programming environment, including MMX register set, data types, and instruction set. Additional instructions that operate on MMX registers have been added to the IA-32 architecture by the SSE/SSE2 extensions.

For more information, see:

- Section 10.4.4, "Intel® SSE 64-Bit SIMD Integer Instructions," describes MMX instructions added to the IA-32 architecture with the SSE extensions.
- Section 11.4.2, "Intel® SSE2 64-Bit and 128-Bit SIMD Integer Instructions," describes MMX instructions added to the IA-32 architecture with SSE2 extensions.
- The Intel® 64 and IA-32 Architectures Software Developer's Manual, Volumes 2A, 2B, 2C, & 2D, gives detailed descriptions of MMX instructions.
- Chapter 15, "Intel® MMX™ Technology System Programming," in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3B, describes the manner in which MMX technology is integrated into the IA-32 system programming model.

9.2 THE MMX TECHNOLOGY PROGRAMMING ENVIRONMENT

Figure 9-1 shows the execution environment for MMX technology. All MMX instructions operate on MMX registers, the general-purpose registers, and/or memory as follows:

- **MMX registers —** These eight registers (see Figure 9-1) are used to perform operations on 64-bit packed integer data. They are named MM0 through MM7.

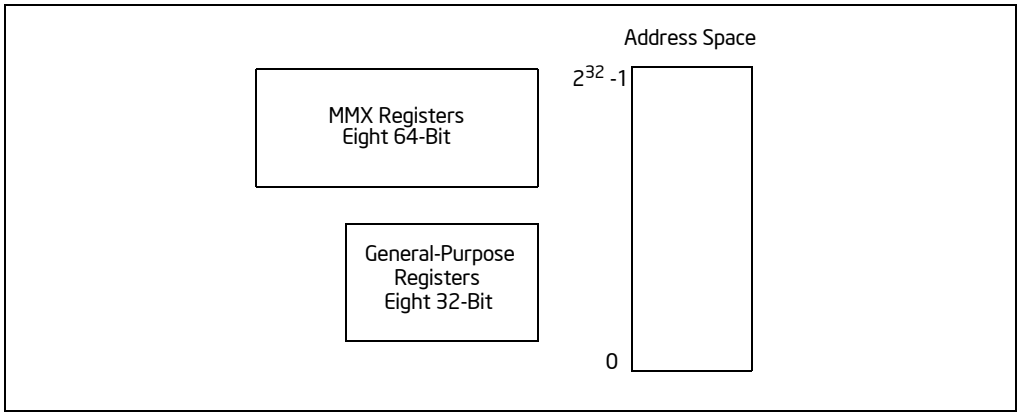


Figure 9-1. MMX Technology Execution Environment

- **General-purpose registers —** The eight general-purpose registers (see Figure 3-5) are used with existing IA-32 addressing modes to address operands in memory. (MMX registers cannot be used to address memory). General-purpose registers are also used to hold operands for some MMX technology operations. They are EAX, EBX, ECX, EDX, EBP, ESI, EDI, and ESP.

9.2.1 MMX Technology in 64-Bit Mode and Compatibility Mode

In compatibility mode and 64-bit mode, MMX instructions function like they do in protected mode. Memory operands are specified using the ModR/M, SIB encoding described in Section 3.7.5.

9.2.2 MMX Registers

The MMX register set consists of eight 64-bit registers (see Figure 9-2), that are used to perform calculations on the MMX packed integer data types. Values in MMX registers have the same format as a 64-bit quantity in memory.

The MMX registers have two data access modes: 64-bit access mode and 32-bit access mode. The 64-bit access mode is used for:

- 64-bit memory accesses.
- 64-bit transfers between MMX registers.
- All pack, logical, and arithmetic instructions.
- Some unpack instructions.

The 32-bit access mode is used for:

- 32-bit memory accesses.
- 32-bit transfer between general-purpose registers and MMX registers.
- Some unpack instructions.

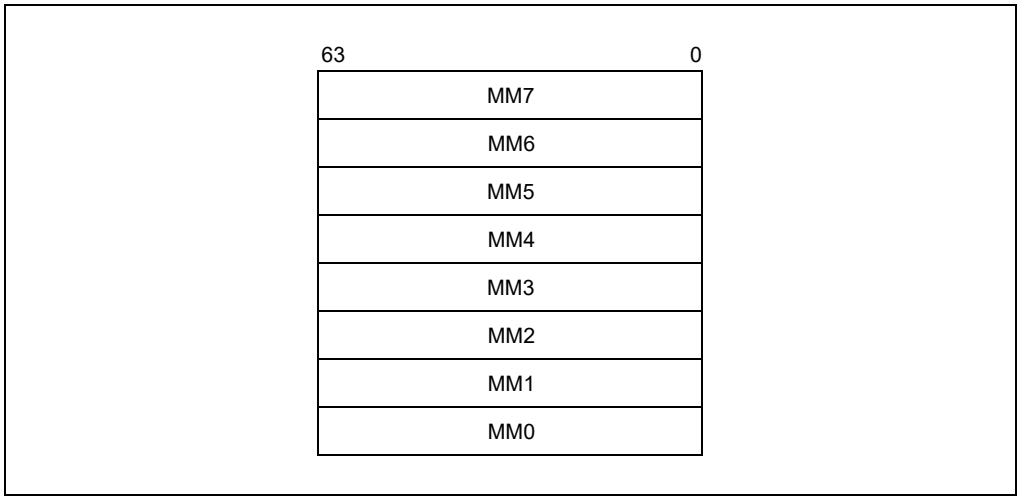


Figure 9-2. MMX Register Set

Although MMX registers are defined in the IA-32 architecture as separate registers, they are aliased to the registers in the FPU data register stack (R0 through R7).

See also Section 9.5, "Compatibility with x87 FPU Architecture."

9.2.3 MMX Data Types

MMX technology introduced the following 64-bit data types to the IA-32 architecture (see Figure 9-3):

- 64-bit packed byte integers — eight packed bytes.
- 64-bit packed word integers — four packed words.
- 64-bit packed doubleword integers — two packed doublewords.

MMX instructions move 64-bit packed data types (packed bytes, packed words, or packed doublewords) and the quadword data type between MMX registers and memory or between MMX registers in 64-bit blocks. However, when performing arithmetic or logical operations on the packed data types, MMX instructions operate in parallel on the individual bytes, words, or doublewords contained in MMX registers; see Section 9.2.5, "Single Instruction, Multiple Data (SIMD) Execution Model."

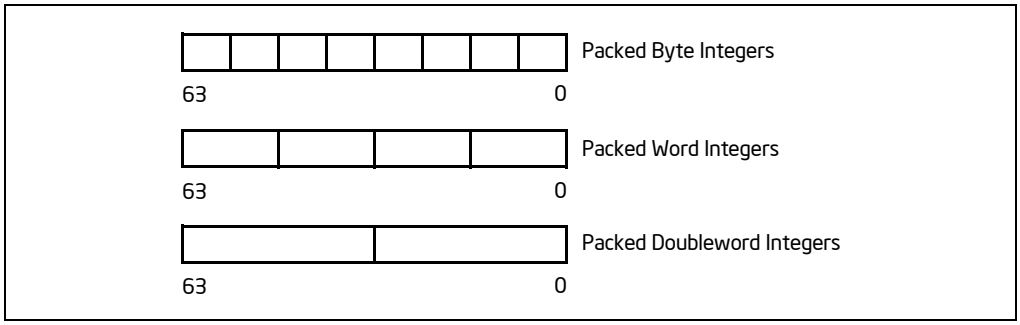


Figure 9-3. Data Types Introduced with the MMX Technology

9.2.4 Memory Data Formats

When stored in memory: bytes, words, and doublewords in the packed data types are stored in consecutive addresses. The least significant byte, word, or doubleword is stored at the lowest address and the most significant byte, word, or doubleword is stored at the high address. The ordering of bytes, words, or doublewords in memory is always little endian. That is, the bytes with the low addresses are less significant than the bytes with high addresses.

9.2.5 Single Instruction, Multiple Data (SIMD) Execution Model

MMX technology uses the single instruction, multiple data (SIMD) technique for performing arithmetic and logical operations on bytes, words, or doublewords packed into MMX registers (see Figure 9-4). For example, the PADDSW instruction adds 4 signed word integers from one source operand to 4 signed word integers in a second source operand and stores 4 word integer results in a destination operand. This SIMD technique speeds up software performance by allowing the same operation to be carried out on multiple data elements in parallel. MMX technology supports parallel operations on byte, word, and doubleword data elements when contained in MMX registers.

The SIMD execution model supported in the MMX technology directly addresses the needs of modern media, communications, and graphics applications, which often use sophisticated algorithms that perform the same operations on a large number of small data types (bytes, words, and doublewords). For example, most audio data is represented in 16-bit (word) quantities. The MMX instructions can operate on 4 words simultaneously with one instruction. Video and graphics information is commonly represented as palletized 8-bit (byte) quantities. In Figure 9-4, one MMX instruction operates on 8 bytes simultaneously.

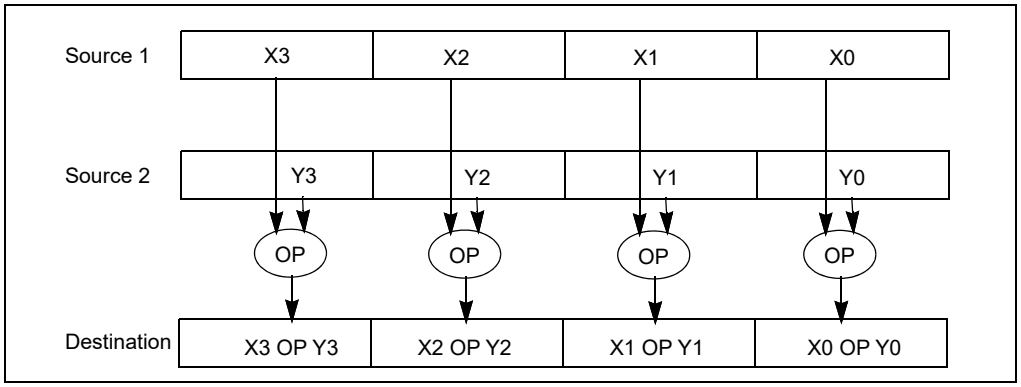


Figure 9-4. SIMD Execution Model

9.3 SATURATION AND WRAPAROUND MODES

When performing integer arithmetic, an operation may result in an out-of-range condition, where the true result cannot be represented in the destination format. For example, when performing arithmetic on signed word integers, positive overflow can occur when the true signed result is larger than 16 bits.

The MMX technology provides three ways of handling out-of-range conditions:

- **Wraparound arithmetic —** With wraparound arithmetic, a true out-of-range result is truncated (that is, the carry or overflow bit is ignored and only the least significant bits of the result are returned to the destination). Wraparound arithmetic is suitable for applications that control the range of operands to prevent out-of-range results. If the range of operands is not controlled, however, wraparound arithmetic can lead to large errors. For example, adding two large signed numbers can cause positive overflow and produce a negative result.
- **Signed saturation arithmetic —** With signed saturation arithmetic, out-of-range results are limited to the representable range of signed integers for the integer size being operated on (see Table 9-1). For example, if positive overflow occurs when operating on signed word integers, the result is "saturated" to 7FFFH, which is the largest positive integer that can be represented in 16 bits; if negative overflow occurs, the result is saturated to 8000H.
- **Unsigned saturation arithmetic —** With unsigned saturation arithmetic, out-of-range results are limited to the representable range of unsigned integers for the integer size. So, positive overflow when operating on unsigned byte integers results in FFH being returned and negative overflow results in 00H being returned.

Table 9-1. Data Range Limits for Saturation

Data Type	Lower Limit		Upper Limit	
	Hexadecimal	Decimal	Hexadecimal	Decimal
Signed Byte	80H	-128	7FH	127
Signed Word	8000H	-32,768	7FFFH	32,767
Unsigned Byte	00H	0	FFH	255
Unsigned Word	0000H	0	FFFFH	65,535

Saturation arithmetic provides an answer for many overflow situations. For example, in color calculations, saturation causes a color to remain pure black or pure white without allowing inversion. It also prevents wraparound artifacts from entering into computations when range checking of source operands it not used.

MMX instructions do not indicate overflow or underflow occurrence by generating exceptions or setting flags in the EFLAGS register.

9.4 MMX INSTRUCTIONS

The MMX instruction set consists of 47 instructions, grouped into the following categories:

- Data transfer
- Arithmetic
- Comparison
- Conversion
- Unpacking
- Logical
- Shift
- Empty MMX state instruction (EMMS)

Table 9-2 gives a summary of the instructions in the MMX instruction set. The following sections give a brief overview of the instructions within each group.

NOTES

The MMX instructions described in this chapter are those instructions that are available in an IA-32 processor when CPUID.01H:EDX.MMX[23] = 1.

Section 10.4.4, "Intel® SSE 64-Bit SIMD Integer Instructions," and Section 11.4.2, "Intel® SSE2 64-Bit and 128-Bit SIMD Integer Instructions," list additional instructions included with the Intel SSE/SSE2 extensions that operate on MMX registers but are not considered part of the MMX instruction set.

Table 9-2. MMX Instruction Set Summary

Category		Wraparound	Signed Saturation	Unsigned Saturation
Arithmetic	Addition	PADDB, PADDW, PADDD	PADDSB, PADDSW	PADDUSB, PADDUSW
	Subtraction	PSUBB, PSUBW, PSUBD	PSUBSB, PSUBSW	PSUBUSB, PSUBUSW
	Multiplication	PMULL, PMULH		
	Multiply and Add	PMADD		
Comparison	Compare for Equal	PCMPEQB, PCMPEQW, PCMPEQD		
	Compare for Greater Than	PCMPGTPB, PCMPGTPW, PCMPGTPD		
Conversion	Pack		PACKSSWB, PACKSSDW	PACKUSWB
Unpack	Unpack High	PUNPCKHBW, PUNPCKHWD, PUNPCKHDQ		
	Unpack Low	PUNPCKLBW, PUNPCKLWD, PUNPCKLDQ		
		Packed		**Full Quadword**
Logical	And			PAND
	And Not			PANDN
	Or			POR
	Exclusive OR			PXOR
Shift	Shift Left Logical	PSLLW, PSLLD		PSLLQ
	Shift Right Logical	PSRLW, PSRLD		PSRLQ
	Shift Right Arithmetic	PSRAW, PSRAD		
		Doubleword Transfers		**Quadword Transfers**
Data Transfer	Register to Register	MOVD		MOVQ
	Load from Memory	MOVD		MOVQ
	Store to Memory	MOVD		MOVQ
Empty MMX State		**EMMS**		

9.4.1 Data Transfer Instructions

The MOVD (Move 32 Bits) instruction transfers 32 bits of packed data from memory to an MMX register and vice versa; or from a general-purpose register to an MMX register and vice versa.

The MOVQ (Move 64 Bits) instruction transfers 64 bits of packed data from memory to an MMX register and vice versa; or transfers data between MMX registers.

9.4.2 Arithmetic Instructions

The arithmetic instructions perform addition, subtraction, multiplication, and multiply/add operations on packed data types.

The PADDB/PADDW/PADDD (add packed integers) instructions and the PSUBB/PSUBW/ PSUBD (subtract packed integers) instructions add or subtract the corresponding signed or unsigned data elements of the source and desti-

nation operands in wraparound mode. These instructions operate on packed byte, word, and doubleword data types.

The PADDSB/PADDSW (add packed signed integers with signed saturation) instructions and the PSUBSB/PSUBSW (subtract packed signed integers with signed saturation) instructions add or subtract the corresponding signed data elements of the source and destination operands and saturate the result to the limits of the signed data-type range. These instructions operate on packed byte and word data types.

The PADDUSB/PADDUSW (add packed unsigned integers with unsigned saturation) instructions and the PSUBUSB/PSUBUSW (subtract packed unsigned integers with unsigned saturation) instructions add or subtract the corresponding unsigned data elements of the source and destination operands and saturate the result to the limits of the unsigned data-type range. These instructions operate on packed byte and word data types.

The PMULHW (multiply packed signed integers and store high result) and PMULLW (multiply packed signed integers and store low result) instructions perform a signed multiply of the corresponding words of the source and destination operands and write the high-order or low-order 16 bits of each of the results, respectively, to the destination operand.

The PMADDWD (multiply and add packed integers) instruction computes the products of the corresponding signed words of the source and destination operands. The four intermediate 32-bit doubleword products are summed in pairs (high-order pair and low-order pair) to produce two 32-bit doubleword results.

9.4.3 Comparison Instructions

The PCMPEQB/PCMPEQW/PCMPEQD (compare packed data for equal) instructions and the PCMPGTB/PCMPGTW/PCMPGTD (compare packed signed integers for greater than) instructions compare the corresponding signed data elements (bytes, words, or doublewords) in the source and destination operands for equal to or greater than, respectively.

These instructions generate a mask of ones or zeros which are written to the destination operand. Logical operations can use the mask to select packed elements. This can be used to implement a packed conditional move operation without a branch or a set of branch instructions. No flags in the EFLAGS register are affected.

9.4.4 Conversion Instructions

The PACKSSWB (pack words into bytes with signed saturation) and PACKSSDW (pack doublewords into words with signed saturation) instructions convert signed words into signed bytes and signed doublewords into signed words, respectively, using signed saturation.

PACKUSWB (pack words into bytes with unsigned saturation) converts signed words into unsigned bytes, using unsigned saturation.

9.4.5 Unpack Instructions

The PUNPCKHBW/PUNPCKHWD/PUNPCKHDQ (unpack high-order data elements) instructions and the PUNPCKLBW/PUNPCKLWD/PUNPCKLDQ (unpack low-order data elements) instructions unpack bytes, words, or doublewords from the high- or low-order data elements of the source and destination operands and interleave them in the destination operand. By placing all 0s in the source operand, these instructions can be used to convert byte integers to word integers, word integers to doubleword integers, or doubleword integers to quadword integers.

9.4.6 Logical Instructions

PAND (bitwise logical AND), PANDN (bitwise logical AND NOT), POR (bitwise logical OR), and PXOR (bitwise logical exclusive OR) perform bitwise logical operations on the quadword source and destination operands.

9.4.7 Shift Instructions

The logical shift left, logical shift right and arithmetic shift right instructions shift each element by a specified number of bit positions.

The PSLLW/PSLLD/PSLLQ (shift packed data left logical) instructions and the PSRLW/PSRLD/PSRLQ (shift packed data right logical) instructions perform a logical left or right shift of the data elements and fill the empty high or low order bit positions with zeros. These instructions operate on packed words, doublewords, and quadwords.

The PSRAW/PSRAD (shift packed data right arithmetic) instructions perform an arithmetic right shift, copying the sign bit for each data element into empty bit positions on the upper end of each data element. This instruction operates on packed words and doublewords.

9.4.8 EMMS Instruction

The EMMS instruction empties the MMX state by setting the tags in x87 FPU tag word to 11B, indicating empty registers. This instruction must be executed at the end of an MMX routine before calling other routines that can execute floating-point instructions. See Section 9.6.3, "Using the EMMS Instruction," for more information on the use of this instruction.

9.5 COMPATIBILITY WITH X87 FPU ARCHITECTURE

The MMX state is aliased to the x87 FPU state. No new states or modes have been added to IA-32 architecture to support the MMX technology. The same floating-point instructions that save and restore the x87 FPU state also handle the MMX state (for example, during context switching).

MMX technology uses the same interface techniques between the x87 FPU and the operating system (primarily for task switching purposes). For more details, see Chapter 15, "Intel® MMX™ Technology System Programming," in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3A.

9.5.1 MMX Instructions and the x87 FPU Tag Word

After each MMX instruction, the entire x87 FPU tag word is set to valid (00B). The EMMS instruction (empty MMX state) sets the entire x87 FPU tag word to empty (11B).

Chapter 15, "Intel® MMX™ Technology System Programming," in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3A, provides additional information about the effects of x87 FPU and MMX instructions on the x87 FPU tag word. For a description of the tag word, see Section 8.1.7, "x87 FPU Tag Word."

9.6 WRITING APPLICATIONS WITH MMX CODE

The following sections give guidelines for writing application code that uses MMX technology.

9.6.1 Checking for MMX Technology Support

Before an application attempts to use the MMX technology, it should check that it is present on the processor. Check by following these steps:

1. Check that the processor supports the CPUID instruction by attempting to execute the CPUID instruction. If the processor does not support the CPUID instruction, this will generate an invalid-opcode exception (#UD).
2. Check that the processor supports the MMX technology (if CPUID.01H:EDX.MMX[23] = 1).
3. Check that emulation of the x87 FPU is disabled (if CR0.EM[bit 2] = 0).

If the processor attempts to execute an unsupported MMX instruction or attempts to execute an MMX instruction with CR0.EM[bit 2] set, this generates an invalid-opcode exception (#UD).

Example 9-1 illustrates how to use the CPUID instruction to detect the MMX technology. This example does not represent the entire CPUID sequence, but shows the portion used for detection of MMX technology.

Example 9-1. Partial Routine for Detecting MMX Technology with the CPUID Instruction

```
...                             ; identify existence of CPUID instruction
...                             ; identify Intel processor
mov     EAX, 1                  ; request for feature flags
CPUID                           ; 0FH, 0A2H CPUID instruction
test    EDX, 00800000H          ; Is IA MMX technology bit (Bit 23 of EDX) set?
jnz                         ; MMX_Technology_Found
```

9.6.2 Transitions Between x87 FPU and MMX Code

Applications can contain both x87 FPU floating-point and MMX instructions. However, because the MMX registers are aliased to the x87 FPU register stack, care must be taken when making transitions between x87 FPU instructions and MMX instructions to prevent incoherent or unexpected results.

When an MMX instruction (other than the EMMS instruction) is executed, the processor changes the x87 FPU state as follows:

- The TOS (top of stack) value of the x87 FPU status word is set to 0.
- The entire x87 FPU tag word is set to the valid state (00B in all tag fields).
- When an MMX instruction writes to an MMX register, it writes ones (11B) to the exponent part of the corresponding floating-point register (bits 64 through 79).

The net result of these actions is that any x87 FPU state prior to the execution of the MMX instruction is essentially lost.

When an x87 FPU instruction is executed, the processor assumes that the current state of the x87 FPU register stack and control registers is valid and executes the instruction without any preparatory modifications to the x87 FPU state.

If the application contains both x87 FPU floating-point and MMX instructions, the following guidelines are recommended:

- When transitioning between x87 FPU and MMX code, save the state of any x87 FPU data or control registers that need to be preserved for future use. The FSAVE and FXSAVE instructions save the entire x87 FPU state.
- When transitioning between MMX and x87 FPU code, do the following:
  - Save any data in the MMX registers that needs to be preserved for future use. FSAVE and FXSAVE also save the state of MMX registers.
  - Execute the EMMS instruction to clear the MMX state from the x87 data and control registers.

The following sections describe the use of the EMMS instruction and give additional guidelines for mixing x87 FPU and MMX code.

9.6.3 Using the EMMS Instruction

As described in Section 9.6.2, "Transitions Between x87 FPU and MMX Code," when an MMX instruction executes, the x87 FPU tag word is marked valid (00B). In this state, the execution of subsequent x87 FPU instructions may produce unexpected x87 FPU floating-point exceptions and/or incorrect results because the x87 FPU register stack appears to contain valid data. The EMMS instruction is provided to prevent this problem by marking the x87 FPU tag word as empty.

The EMMS instruction should be used in each of the following cases:

- When an application using the x87 FPU instructions calls an MMX technology library/DLL (use the EMMS instruction at the end of the MMX code).

- When an application using MMX instructions calls a x87 FPU floating-point library/DLL (use the EMMS instruction before calling the x87 FPU code).
- When a switch is made between MMX code in a task or thread and other tasks or threads in cooperative operating systems, unless it is certain that more MMX instructions will be executed before any x87 FPU code.

EMMS is not required when mixing MMX technology instructions with Intel SSE/SSE2/SSE3 instructions; see Section 11.6.7, "Interaction of Intel® SSE and SSE2 Instructions with x87 FPU and MMX Instructions."

9.6.4 Mixing MMX and x87 FPU Instructions

An application can contain both x87 FPU floating-point and MMX instructions. However, frequent transitions between MMX and x87 FPU instructions are not recommended, because they can degrade performance in some processor implementations. When mixing MMX code with x87 FPU code, follow these guidelines:

- Keep the code in separate modules, procedures, or routines.
- Do not rely on register contents across transitions between x87 FPU and MMX code modules.
- When transitioning between MMX code and x87 FPU code, save the MMX register state (if it will be needed in the future) and execute an EMMS instruction to empty the MMX state.
- When transitioning between x87 FPU code and MMX code, save the x87 FPU state if it will be needed in the future.

9.6.5 Interfacing with MMX Code

MMX technology enables direct access to all the MMX registers. This means that all existing interface conventions that apply to the use of the processor's general-purpose registers (EAX, EBX, etc.) also apply to the use of MMX registers.

An efficient interface to MMX routines might pass parameters and return values through the MMX registers or through a combination of memory locations (via the stack) and MMX registers. Do not use the EMMS instruction or mix MMX and x87 FPU code when using the MMX registers to pass parameters.

If a high-level language that does not support the MMX data types directly is used, the MMX data types can be defined as a 64-bit structure containing packed data types.

When implementing MMX instructions in high-level languages, other approaches can be taken, such as:

- Passing parameters to an MMX routine by passing a pointer to a structure via the stack.
- Returning a value from a function by returning a pointer to a structure.

9.6.6 Using MMX Code in a Multitasking Operating System Environment

An application needs to identify the nature of the multitasking operating system on which it runs. Each task retains its own state which must be saved when a task switch occurs. The processor state (context) consists of the general-purpose registers and the floating-point and MMX registers.

Operating systems can be classified into two types:

- Cooperative multitasking operating system.
- Preemptive multitasking operating system.

Cooperative multitasking operating systems do not save the FPU or MMX state when performing a context switch. Therefore, the application needs to save the relevant state before relinquishing direct or indirect control to the operating system.

Preemptive multitasking operating systems are responsible for saving and restoring the FPU and MMX state when performing a context switch. Therefore, the application does not have to save or restore the FPU and MMX state.

9.6.7 Exception Handling in MMX Code

MMX instructions generate the same type of memory-access exceptions as other IA-32 instructions (page fault, segment not present, and limit violations). Existing exception handlers do not have to be modified to handle these types of exceptions for MMX code.

Unless there is a pending floating-point exception, MMX instructions do not generate numeric exceptions. Therefore, there is no need to modify existing exception handlers or add new ones to handle numeric exceptions.

If a floating-point exception is pending, the subsequent MMX instruction generates a numeric error exception (interrupt 16 and/or assertion of the FERR# pin). The MMX instruction resumes execution upon return from the exception handler.

9.6.8 Register Mapping

MMX registers and their tags are mapped to physical locations of the floating-point registers and their tags. Register aliasing and mapping is described in more detail in Chapter 15, "Intel® MMX™ Technology System Programming," in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3A.

9.6.9 Effect of Instruction Prefixes on MMX Instructions

Table 9-3 describes the effect of instruction prefixes on MMX instructions. Unpredictable behavior can range from being treated as a reserved operation on one generation of IA-32 processors to generating an invalid opcode exception on another generation of processors.

Table 9-3. Effect of Prefixes on MMX Instructions

Prefix Type	Effect on MMX Instructions
Address Size Prefix (67H)	Affects instructions with a memory operand.
	Reserved for instructions without a memory operand and may result in unpredictable behavior.
Operand Size (66H)	Reserved and may result in unpredictable behavior.
Segment Override (2EH, 36H, 3EH, 26H, 64H, 65H)	Affects instructions with a memory operand.
	Reserved for instructions without a memory operand and may result in unpredictable behavior.
Repeat Prefix (F3H)	Reserved and may result in unpredictable behavior.
Repeat NE Prefix(F2H)	Reserved and may result in unpredictable behavior.
Lock Prefix (F0H)	Reserved; generates invalid opcode exception (#UD).

See "Instruction Prefixes" in Chapter 2, "Instruction Format," of the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 2A, for a description of the instruction prefixes.

CHAPTER 10
PROGRAMMING WITH INTEL® STREAMING SIMD EXTENSIONS (INTEL® SSE)

The Intel® Streaming SIMD Extensions (Intel® SSE) were introduced into the IA-32 architecture in the Pentium III processor family. These extensions enhance the performance of IA-32 processors for advanced 2-D and 3-D graphics, motion video, image processing, speech recognition, audio synthesis, telephony, and video conferencing.

This chapter describes SSE. Chapter 11, "Programming with Intel® Streaming SIMD Extensions 2 (Intel® SSE2)," provides information to assist in writing application programs that use Intel SSE2. Chapter 12, "Programming with Intel® SSE3, SSSE3, Intel® SSE4, and Intel® AES-NI," provides this information for Intel SSE3.

10.1 OVERVIEW OF INTEL® SSE

Intel MMX technology introduced single-instruction multiple-data (SIMD) capability into the IA-32 architecture, with the 64-bit MMX registers, 64-bit packed integer data types, and instructions that allowed SIMD operations to be performed on packed integers. Intel SSE expanded the SIMD execution model by adding facilities for handling packed and scalar single precision floating-point values contained in 128-bit registers.

If CPUID.01H:EDX.SSE[25] = 1, Intel SSE is available.

Intel SSE adds the following features to the IA-32 architecture, while maintaining backward compatibility with all existing IA-32 processors, applications, and operating systems:

- Eight 128-bit data registers (called XMM registers) in non-64-bit modes; 16 XMM registers are available in 64-bit mode.
- The 32-bit MXCSR register, which provides control and status bits for operations performed on XMM registers.
- The 128-bit packed single precision floating-point data type (four IEEE single precision floating-point values packed into a double quadword).
- Instructions that perform SIMD operations on single precision floating-point values and that extend SIMD operations that can be performed on integers:
  - 128-bit Packed and scalar single precision floating-point instructions that operate on data located in MMX registers.
  - 64-bit SIMD integer instructions that support additional operations on packed integer operands located in MMX registers.
- Instructions that save and restore the state of the MXCSR register.
- Instructions that support explicit prefetching of data, control of the cacheability of data, and control the ordering of store operations.
- Extensions to the CPUID instruction.

These features extend the IA-32 architecture's SIMD programming model in four important ways:

- The ability to perform SIMD operations on four packed single precision floating-point values enhances the performance of IA-32 processors for advanced media and communications applications that use computation-intensive algorithms to perform repetitive operations on large arrays of simple, native data elements.
- The ability to perform SIMD single precision floating-point operations in XMM registers and SIMD integer operations in MMX registers provides greater flexibility and throughput for executing applications that operate on large arrays of floating-point and integer data.
- Cache control instructions provide the ability to stream data in and out of XMM registers without polluting the caches and the ability to prefetch data to selected cache levels before it is actually used. Applications that require regular access to large amounts of data benefit from these prefetching and streaming store capabilities.
- The SFENCE (store fence) instruction provides greater control over the ordering of store operations when using weakly-ordered memory types.

Intel SSE is fully compatible with all software written for IA-32 processors. All existing software continues to run correctly, without modification, on processors that incorporate Intel SSE. Enhancements to CPUID permit detection of Intel SSE. Intel SSE is accessible from all IA-32 execution modes: protected mode, real address mode, and virtual-8086 mode.

The following sections of this chapter describe the programming environment for Intel SSE, including: XMM registers, the packed single precision floating-point data type, and Intel SSE instructions. For additional information, see:

- Section 11.6, "Writing Applications with Intel® SSE and SSE2."
- Section 11.5, "Intel® SSE, SSE2, and SSE3 Exceptions," describes the exceptions that can be generated with Intel SSE/SSE2/SSE3 instructions.
- The Intel® 64 and IA-32 Architectures Software Developer's Manual, Volumes 2A, 2B, 2C, & 2D, provides a detailed description of these instructions.
- Chapter 16, "System Programming for Instruction Set Extensions and Processor Extended States," in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3A, gives guidelines for integrating these extensions into an operating-system environment.

10.2 INTEL® SSE PROGRAMMING ENVIRONMENT

Figure 10-1 shows the execution environment for Intel SSE. All Intel SSE instructions operate on the XMM registers, MMX registers, and/or memory as follows:

- **XMM registers —** These eight registers (see Figure 10-2 and Section 10.2.2, "XMM Registers") are used to operate on packed or scalar single precision floating-point data. Scalar operations are operations performed on individual (unpacked) single precision floating-point values stored in the low doubleword of an XMM register. XMM registers are referenced by the names XMM0 through XMM7.

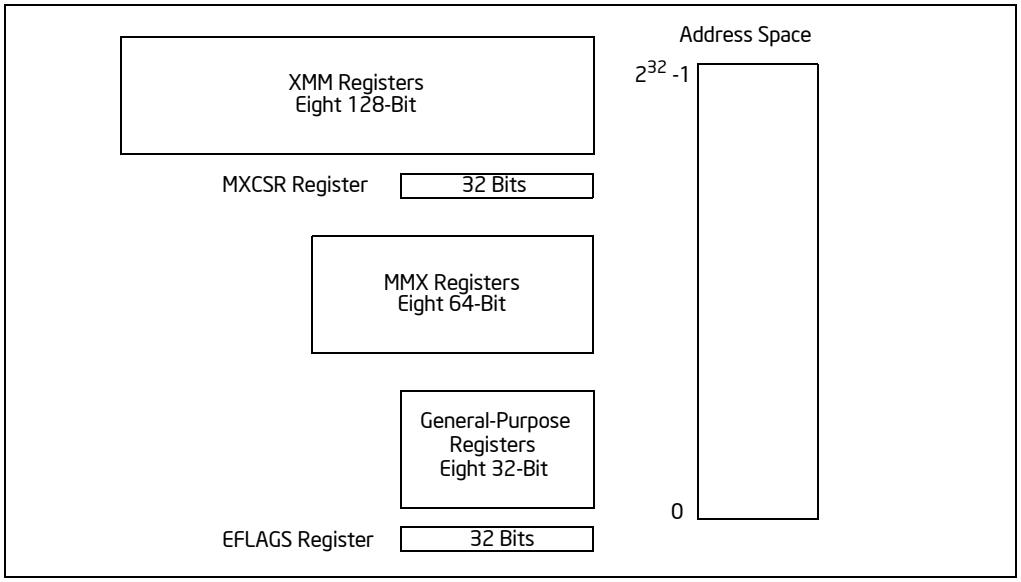


Figure 10-1. Intel® SSE Execution Environment

- **MXCSR register —** This 32-bit register (see Figure 10-3 and Section 10.2.3, "MXCSR Control and Status Register") provides status and control bits used in SIMD floating-point operations.
- **MMX registers —** These eight registers (see Figure 9-2) are used to perform operations on 64-bit packed integer data. They are also used to hold operands for some operations performed between the MMX and XMM registers. MMX registers are referenced by the names MM0 through MM7.
- **General-purpose registers —** The eight general-purpose registers (see Figure 3-5) are used along with the existing IA-32 addressing modes to address operands in memory. (MMX and XMM registers cannot be used to

address memory). The general-purpose registers are also used to hold operands for some SSE instructions and are referenced as EAX, EBX, ECX, EDX, EBP, ESI, EDI, and ESP.

- **EFLAGS register —** This 32-bit register (see Figure 3-8) is used to record result of some compare operations.

10.2.1 Intel® SSE in 64-Bit Mode and Compatibility Mode

In compatibility mode, Intel SSE functions like it does in protected mode. In 64-bit mode, eight additional XMM registers are accessible. Registers XMM8-XMM15 are accessed by using REX prefixes. Memory operands are specified using the ModR/M, SIB encoding described in Section 3.7.5.

Some Intel SSE instructions may be used to operate on general-purpose registers. Use the REX.W prefix to access 64-bit general-purpose registers. Note that if a REX prefix is used when it has no meaning, the prefix is ignored.

10.2.2 XMM Registers

Eight 128-bit XMM data registers were introduced into the IA-32 architecture with Intel SSE (see Figure 10-2). These registers can be accessed directly using the names XMM0 to XMM7; and they can be accessed independently from the x87 FPU and MMX registers and the general-purpose registers (that is, they are not aliased to any other of the processor's registers).

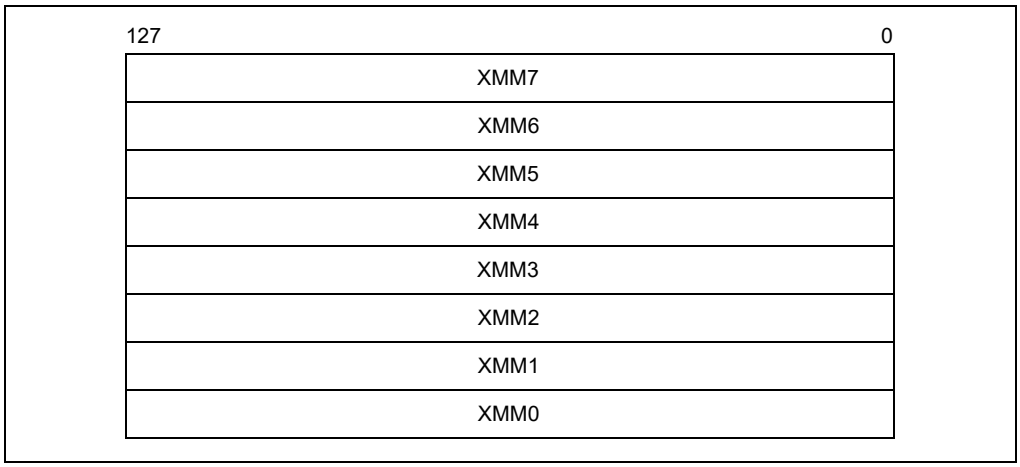


Figure 10-2. XMM Registers

Intel SSE instructions use the XMM registers only to operate on packed single precision floating-point operands. SSE2 extensions expand the functions of the XMM registers to operand on packed or scalar double precision floating-point operands and packed integer operands; see Section 11.2, "Intel® SSE2 Programming Environment," and Section 12.1, "Programming Environment and Data types."

XMM registers can only be used to perform calculations on data; they cannot be used to address memory. Addressing memory is accomplished by using the general-purpose registers.

Data can be loaded into XMM registers or written from the registers to memory in 32-bit, 64-bit, and 128-bit increments. When storing the entire contents of an XMM register in memory (128-bit store), the data is stored in 16 consecutive bytes, with the low-order byte of the register being stored in the first byte in memory.

10.2.3 MXCSR Control and Status Register

The 32-bit MXCSR register (see Figure 10-3) contains control and status information for Intel SSE, SSE2, and SSE3 SIMD floating-point operations. This register contains:

- Flag and mask bits for SIMD floating-point exceptions.
- Rounding control field for SIMD floating-point operations.

- Flush-to-zero flag that provides a means of controlling underflow conditions on SIMD floating-point operations.
- Denormals-are-zeros flag that controls how SIMD floating-point instructions handle denormal source operands.

The contents of this register can be loaded from memory with the LDMXCSR and FXRSTOR instructions and stored in memory with STMXCSR and FXSAVE.

Bits 16 through 31 of the MXCSR register are reserved and are cleared on a power-up or reset of the processor; attempting to write a non-zero value to these bits, using either the FXRSTOR or LDMXCSR instructions, will result in a general-protection exception (#GP) being generated.

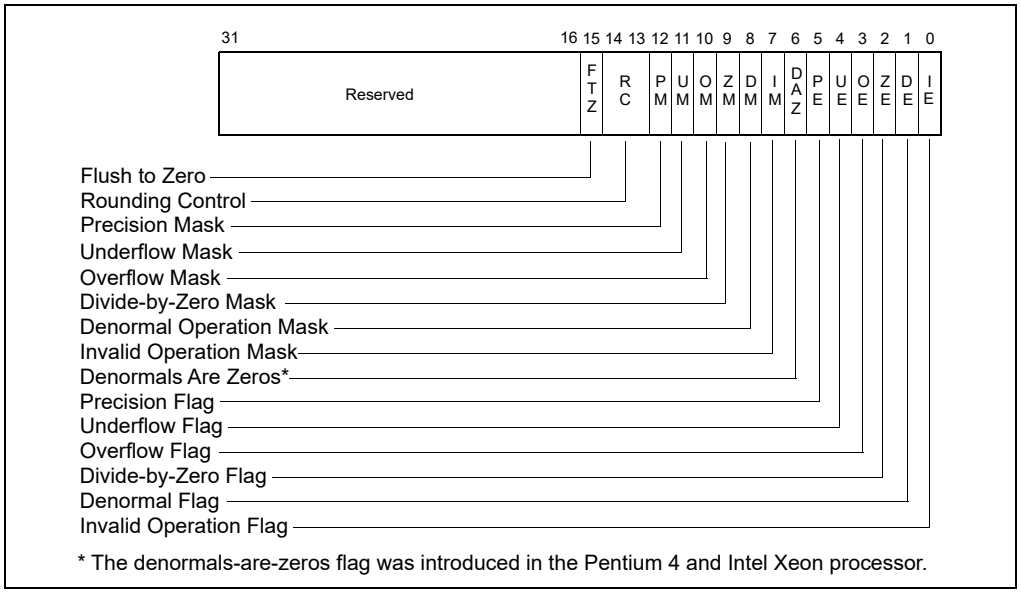


Figure 10-3. MXCSR Control/Status Register

10.2.3.1 SIMD Floating-Point Mask and Flag Bits

Bits 0 through 5 of the MXCSR register indicate whether a SIMD floating-point exception has been detected. They are "sticky" flags. That is, after a flag is set, it remains set until explicitly cleared. To clear these flags, use the LDMXCSR or the FXRSTOR instruction to write zeroes to them.

Bits 7 through 12 provide individual mask bits for the SIMD floating-point exceptions. An exception type is masked if the corresponding mask bit is set, and it is unmasked if the bit is clear. These mask bits are set upon a power-up or reset. This causes all SIMD floating-point exceptions to be initially masked.

If LDMXCSR or FXRSTOR clears a mask bit and sets the corresponding exception flag bit, a SIMD floating-point exception will not be generated as a result of this change. The unmasked exception will be generated only upon the execution of the next SSE/SSE2/SSE3 instruction that detects the unmasked exception condition.

For more information about the use of the SIMD floating-point exception mask and flag bits, see Section 11.5, "Intel® SSE, SSE2, and SSE3 Exceptions," and Section 12.8, "Intel® SSE3, SSSE3, And Intel® SSE4 Exceptions."

10.2.3.2 SIMD Floating-Point Rounding Control Field

Bits 13 and 14 of the MXCSR register (the rounding control [RC] field) control how the results of SIMD floating-point instructions are rounded. See Section 4.8.4, "Rounding," for a description of the function and encoding of the rounding control bits.

10.2.3.3 Flush-To-Zero

Bit 15 (FTZ) of the MXCSR register enables the flush-to-zero mode, which controls the masked response to a SIMD floating-point underflow condition. When the underflow exception is masked and the flush-to-zero mode is enabled, the processor performs the following operations when it detects a floating-point underflow condition.

- Returns a zero result with the sign of the true result.
- Sets the precision and underflow exception flags.

If the underflow exception is not masked, the flush-to-zero bit is ignored.

The flush-to-zero mode is not compatible with IEEE Standard 754. The IEEE-mandated masked response to underflow is to deliver the denormalized result (see Section 4.8.3.2, "Normalized and Denormalized Finite Numbers"). The flush-to-zero mode is provided primarily for performance reasons. At the cost of a slight precision loss, faster execution can be achieved for applications where underflows are common and rounding the underflow result to zero can be tolerated.

The flush-to-zero bit is cleared upon a power-up or reset of the processor, disabling the flush-to-zero mode.

10.2.3.4 Denormals-Are-Zeros

Bit 6 (DAZ) of the MXCSR register enables the denormals-are-zeros mode, which controls the processor's response to a SIMD floating-point denormal operand condition. When the denormals-are-zeros flag is set, the processor converts all denormal source operands to a zero with the sign of the original operand before performing any computations on them. The processor does not set the denormal-operand exception flag (DE), regardless of the setting of the denormal-operand exception mask bit (DM); and it does not generate a denormal-operand exception if the exception is unmasked.

The denormals-are-zeros mode is not compatible with IEEE Standard 754 (see Section 4.8.3.2, "Normalized and Denormalized Finite Numbers"). The denormals-are-zeros mode is provided to improve processor performance for applications such as streaming media processing, where rounding a denormal operand to zero does not appreciably affect the quality of the processed data.

The denormals-are-zeros flag is cleared upon a power-up or reset of the processor, disabling the denormals-are-zeros mode.

The denormals-are-zeros mode was introduced in the Pentium 4 and Intel Xeon processor with the SSE2 extensions; however, it is fully compatible with the SSE SIMD floating-point instructions (that is, the denormals-are-zeros flag affects the operation of the SSE SIMD floating-point instructions). In earlier IA-32 processors and in some models of the Pentium 4 processor, this flag (bit 6) is reserved. See Section 11.6.3, "Checking for the DAZ Flag in the MXCSR Register," for instructions for detecting the availability of this feature.

Attempting to set bit 6 of the MXCSR register on processors that do not support the DAZ flag will cause a general-protection exception (#GP). See Section 11.6.6, "Guidelines for Writing to the MXCSR Register," for instructions for preventing such general-protection exceptions by using the MXCSR_MASK value returned by the FXSAVE instruction.

10.2.4 Compatibility of Intel® SSE with Intel® SSE2 and SSE3, MMX, and the x87 FPU

The state (XMM registers and MXCSR register) introduced into the IA-32 execution environment with Intel SSE is shared with Intel SSE2 and SSE3. Intel SSE, SSE2, and SSE3 instructions are fully compatible; they can be executed together in the same instruction stream with no need to save state when switching between instruction sets.

XMM registers are independent of the x87 FPU and MMX registers, so Intel SSE, SSE2, and SSE3 operations performed on the XMM registers can be performed in parallel with operations on the x87 FPU and MMX registers; see Section 11.6.7, "Interaction of Intel® SSE and SSE2 Instructions with x87 FPU and MMX Instructions."

The FXSAVE and FXRSTOR instructions save and restore the SSE/SSE2/SSE3 states along with the x87 FPU and MMX state.

10.3 INTEL® SSE DATA TYPES

Intel SSE introduced one data type, the 128-bit packed single precision floating-point data type, to the IA-32 architecture (see Figure 10-4). This data type consists of four IEEE 32-bit single precision floating-point values packed

into a double quadword. See Figure 4-3 for the layout of a single precision floating-point value; refer to Section 4.2.2, "Floating-Point Data Types," for a detailed description of the single precision floating-point format.

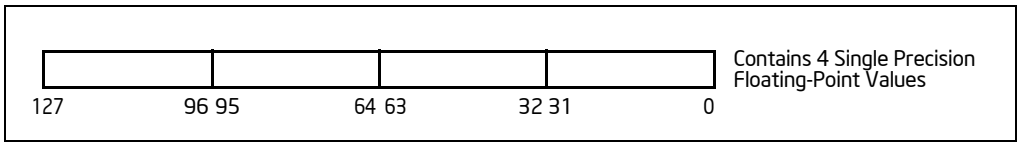


Figure 10-4. 128-Bit Packed Single Precision Floating-Point Data Type

This 128-bit packed single precision floating-point data type is operated on in the XMM registers or in memory. Conversion instructions are provided to convert two packed single precision floating-point values into two packed doubleword integers or a scalar single precision floating-point value into a doubleword integer (see Figure 11-8).

Intel SSE provides conversion instructions between XMM registers and MMX registers, and between XMM registers and general-purpose bit registers. See Figure 11-8.

The address of a 128-bit packed memory operand must be aligned on a 16-byte boundary, except in the following cases:

- The MOVUPS instruction supports unaligned accesses.
- Scalar instructions that use a 4-byte memory operand that is not subject to alignment requirements.

Figure 4-2 shows the byte order of 128-bit (double quadword) data types in memory.

10.4 INTEL® SSE INSTRUCTION SET

Intel SSE instructions are divided into four functional groups:

- Packed and scalar single precision floating-point instructions.
- 64-bit SIMD integer instructions.
- State management instructions.
- Cacheability control, prefetch, and memory ordering instructions.

The following sections give an overview of each of the instructions in these groups.

10.4.1 Intel® SSE Packed and Scalar Floating-Point Instructions

The packed and scalar single precision floating-point instructions are divided into the following subgroups:

- Data movement instructions.
- Arithmetic instructions.
- Logical instructions.
- Comparison instructions.
- Shuffle instructions.
- Conversion instructions.

The packed single precision floating-point instructions perform SIMD operations on packed single precision floating-point operands (see Figure 10-5). Each source operand contains four single precision floating-point values, and the destination operand contains the results of the operation (OP) performed in parallel on the corresponding values (X0 and Y0, X1 and Y1, X2 and Y2, and X3 and Y3) in each operand.

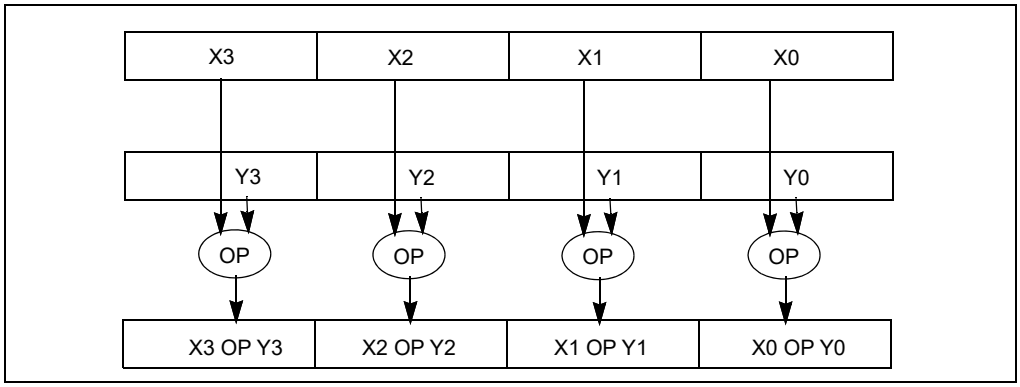


Figure 10-5. Packed Single Precision Floating-Point Operation

The scalar single precision floating-point instructions operate on the low (least significant) doublewords of the two source operands (X0 and Y0); see Figure 10-6. The three most significant doublewords (X1, X2, and X3) of the first source operand are passed through to the destination. The scalar operations are similar to the floating-point operations performed in the x87 FPU data registers with the precision control field in the x87 FPU control word set for single precision (24-bit significand), except that x87 stack operations use a 15-bit exponent range for the result, while SSE operations use an 8-bit exponent range.

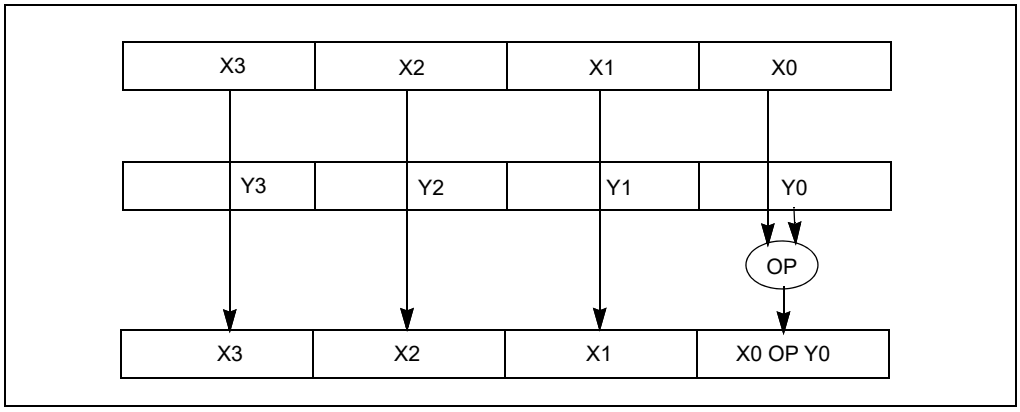


Figure 10-6. Scalar Single Precision Floating-Point Operation

10.4.1.1 Intel® SSE Data Movement Instructions

Intel SSE data movement instructions move single precision floating-point data between XMM registers and between an XMM register and memory.

The MOVAPS (move aligned packed single precision floating-point values) instruction transfers a double quadword operand containing four packed single precision floating-point values from memory to an XMM register and vice versa, or between XMM registers. The memory address must be aligned to a 16-byte boundary; otherwise, a general-protection exception (#GP) is generated.

The MOVUPS (move unaligned packed single precision, floating-point) instruction performs the same operations as the MOVAPS instruction, except that 16-byte alignment of a memory address is not required.

The MOVSS (move scalar single precision floating-point) instruction transfers a 32-bit single precision floating-point operand from memory to the low doubleword of an XMM register and vice versa, or between XMM registers.

The MOVLPS (move low packed single precision floating-point) instruction moves two packed single precision floating-point values from memory to the low quadword of an XMM register and vice versa. The high quadword of the register is left unchanged.

The MOVHPS (move high packed single precision floating-point) instruction moves two packed single precision floating-point values from memory to the high quadword of an XMM register and vice versa. The low quadword of the register is left unchanged.

The MOVLHPS (move packed single precision floating-point low to high) instruction moves two packed single precision floating-point values from the low quadword of the source XMM register into the high quadword of the destination XMM register. The low quadword of the destination register is left unchanged.

The MOVHLPS (move packed single precision floating-point high to low) instruction moves two packed single precision floating-point values from the high quadword of the source XMM register into the low quadword of the destination XMM register. The high quadword of the destination register is left unchanged.

The MOVMSKPS (move packed single precision floating-point mask) instruction transfers the most significant bit of each of the four packed single precision floating-point numbers in an XMM register to a general-purpose register. This 4-bit value can then be used as a condition to perform branching.

10.4.1.2 Intel® SSE Arithmetic Instructions

Intel SSE arithmetic instructions perform addition, subtraction, multiply, divide, reciprocal, square root, reciprocal of square root, and maximum/minimum operations on packed and scalar single precision floating-point values.

The ADDPS (add packed single precision floating-point values) and SUBPS (subtract packed single precision floating-point values) instructions add and subtract, respectively, two packed single precision floating-point operands.

The ADDSS (add scalar single precision floating-point values) and SUBSS (subtract scalar single precision floating-point values) instructions add and subtract, respectively, the low single precision floating-point values of two operands and store the result in the low doubleword of the destination operand.

The MULPS (multiply packed single precision floating-point values) instruction multiplies two packed single precision floating-point operands.

The MULSS (multiply scalar single precision floating-point values) instruction multiplies the low single precision floating-point values of two operands and stores the result in the low doubleword of the destination operand.

The DIVPS (divide packed, single precision floating-point values) instruction divides two packed single precision floating-point operands.

The DIVSS (divide scalar single precision floating-point values) instruction divides the low single precision floating-point values of two operands and stores the result in the low doubleword of the destination operand.

The RCPPS (compute reciprocals of packed single precision floating-point values) instruction computes the approximate reciprocals of values in a packed single precision floating-point operand.

The RCPSS (compute reciprocal of scalar single precision floating-point values) instruction computes the approximate reciprocal of the low single precision floating-point value in the source operand and stores the result in the low doubleword of the destination operand.

The SQRTPS (compute square roots of packed single precision floating-point values) instruction computes the square roots of the values in a packed single precision floating-point operand.

The SQRTSS (compute square root of scalar single precision floating-point values) instruction computes the square root of the low single precision floating-point value in the source operand and stores the result in the low doubleword of the destination operand.

The RSQRTPS (compute reciprocals of square roots of packed single precision floating-point values) instruction computes the approximate reciprocals of the square roots of the values in a packed single precision floating-point operand.

The RSQRTSS (reciprocal of square root of scalar single precision floating-point value) instruction computes the approximate reciprocal of the square root of the low single precision floating-point value in the source operand and stores the result in the low doubleword of the destination operand.

The MAXPS (return maximum of packed single precision floating-point values) instruction compares the corresponding values from two packed single precision floating-point operands and returns the numerically greater value from each comparison to the destination operand.

The MAXSS (return maximum of scalar single precision floating-point values) instruction compares the low values from two packed single precision floating-point operands and returns the numerically greater value from the comparison to the low doubleword of the destination operand.

The MINPS (return minimum of packed single precision floating-point values) instruction compares the corresponding values from two packed single precision floating-point operands and returns the numerically lesser value from each comparison to the destination operand.

The MINSS (return minimum of scalar single precision floating-point values) instruction compares the low values from two packed single precision floating-point operands and returns the numerically lesser value from the comparison to the low doubleword of the destination operand.

10.4.2 Intel® SSE Logical Instructions

Intel SSE logical instructions perform AND, AND NOT, OR, and XOR operations on packed single precision floating-point values.

The ANDPS (bitwise logical AND of packed single precision floating-point values) instruction returns the logical AND of two packed single precision floating-point operands.

The ANDNPS (bitwise logical AND NOT of packed single precision, floating-point values) instruction returns the logical AND NOT of two packed single precision floating-point operands.

The ORPS (bitwise logical OR of packed single precision, floating-point values) instruction returns the logical OR of two packed single precision floating-point operands.

The XORPS (bitwise logical XOR of packed single precision, floating-point values) instruction returns the logical XOR of two packed single precision floating-point operands.

10.4.2.1 Intel® SSE Comparison Instructions

The compare instructions compare packed and scalar single precision floating-point values and return the results of the comparison either to the destination operand or to the EFLAGS register.

The CMPPS (compare packed single precision floating-point values) instruction compares the corresponding values from two packed single precision floating-point operands, using an immediate operand as a predicate, and returns a 32-bit mask result of all 1s or all 0s for each comparison to the destination operand. The value of the immediate operand allows the selection of any of 8 compare conditions: equal, less than, less than equal, unordered, not equal, not less than, not less than or equal, or ordered.

The CMPSS (compare scalar single precision, floating-point values) instruction compares the low values from two packed single precision floating-point operands, using an immediate operand as a predicate, and returns a 32-bit mask result of all 1s or all 0s for the comparison to the low doubleword of the destination operand. The immediate operand selects the compare conditions as with the CMPPS instruction.

The COMISS (compare scalar single precision floating-point values and set EFLAGS) and UCOMISS (unordered compare scalar single precision floating-point values and set EFLAGS) instructions compare the low values of two packed single precision floating-point operands and set the ZF, PF, and CF flags in the EFLAGS register to show the result (greater than, less than, equal, or unordered). These two instructions differ as follows: the COMISS instruction signals a floating-point invalid-operation (#I) exception when a source operand is either a QNaN or an SNaN; the UCOMISS instruction only signals an invalid-operation exception when a source operand is an SNaN.

10.4.2.2 Intel® SSE Shuffle and Unpack Instructions

Intel SSE shuffle and unpack instructions shuffle or interleave the contents of two packed single precision floating-point values and store the results in the destination operand.

The SHUFPS (shuffle packed single precision floating-point values) instruction places any two of the four packed single precision floating-point values from the destination operand into the two low-order doublewords of the destination operand, and places any two of the four packed single precision floating-point values from the source operand in the two high-order doublewords of the destination operand (see Figure 10-7). By using the same register for the source and destination operands, the SHUFPS instruction can shuffle four single precision floating-point values into any order.

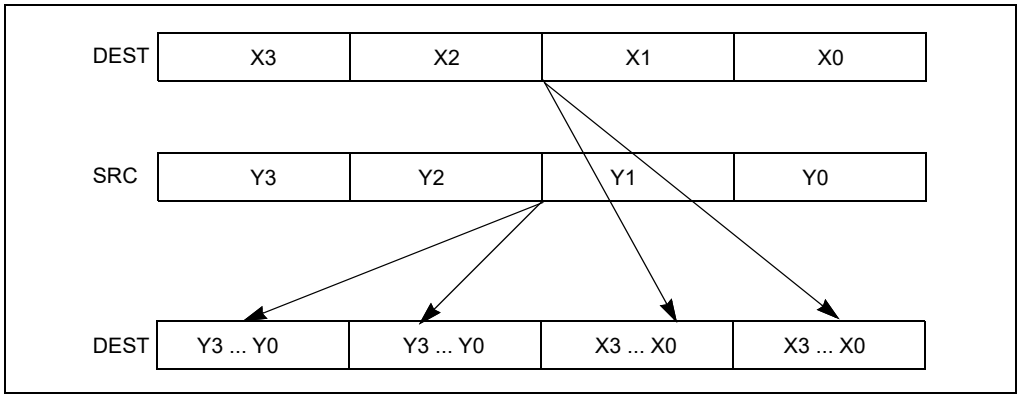


Figure 10-7. SHUFPS Instruction, Packed Shuffle Operation

The UNPCKHPS (unpack and interleave high packed single precision floating-point values) instruction performs an interleaved unpack of the high-order single precision floating-point values from the source and destination operands and stores the result in the destination operand (see Figure 10-8).

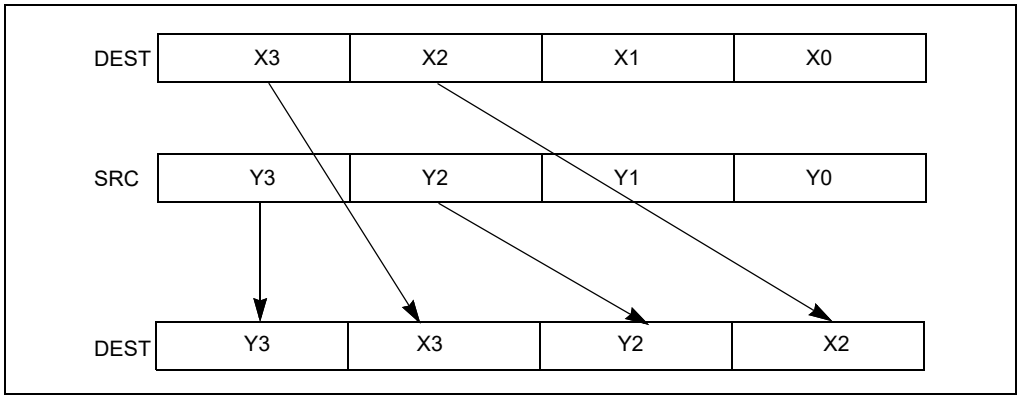


Figure 10-8. UNPCKHPS Instruction, High Unpack and Interleave Operation

The UNPCKLPS (unpack and interleave low packed single precision floating-point values) instruction performs an interleaved unpack of the low-order single precision floating-point values from the source and destination operands and stores the result in the destination operand (see Figure 10-9).

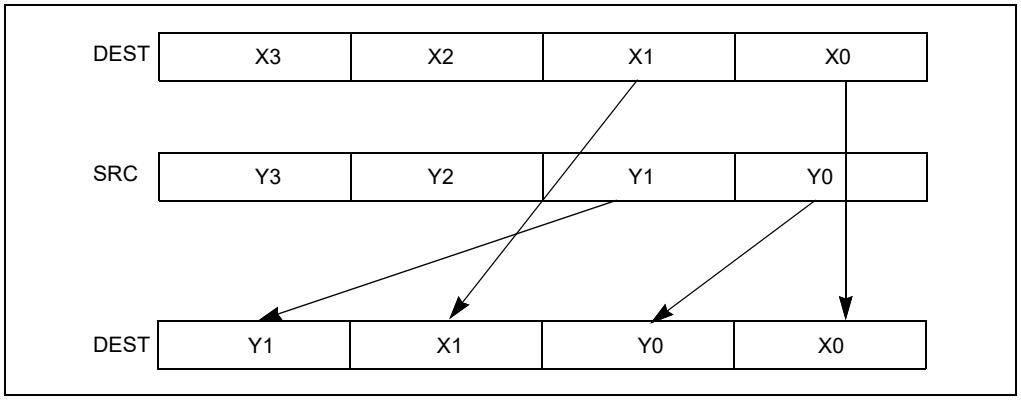


Figure 10-9. UNPCKLPS Instruction, Low Unpack and Interleave Operation

10.4.3 Intel® SSE Conversion Instructions

Intel SSE conversion instructions (see Figure 11-8) support packed and scalar conversions between single precision floating-point and doubleword integer formats.

The CVTPI2PS (convert packed doubleword integers to packed single precision floating-point values) instruction converts two packed signed doubleword integers into two packed single precision floating-point values. When the conversion is inexact, the result is rounded according to the rounding mode selected in the MXCSR register.

The CVTSI2SS (convert signed integer to scalar single precision floating-point value) instruction converts a signed doubleword or quadword integer into a single precision floating-point value. When the conversion is inexact, the result is rounded according to the rounding mode selected in the MXCSR register.

The CVTPS2PI (convert packed single precision floating-point values to packed doubleword integers) instruction converts two packed single precision floating-point values into two packed signed doubleword integers. When the conversion is inexact, the result is rounded according to the rounding mode selected in the MXCSR register. The CVTTPS2PI (convert with truncation packed single precision floating-point values to packed doubleword integers) instruction is similar to the CVTPS2PI instruction, except that truncation is used to round a source value to an integer value; see Section 4.8.4.2, "Truncation with Intel® SSE, SSE2, and AVX Conversion Instructions."

The CVTSS2SI (convert scalar single precision floating-point value to signed integer) instruction converts a single precision floating-point value into a signed integer. When the conversion is inexact, the result is rounded according to the rounding mode selected in the MXCSR register. The CVTTSS2SI (convert with truncation scalar single precision floating-point value to signed integer) instruction is similar to the CVTSS2SI instruction, except that truncation is used to round the source value to an integer value; see Section 4.8.4.2, "Truncation with Intel® SSE, SSE2, and AVX Conversion Instructions."

10.4.4 Intel® SSE 64-Bit SIMD Integer Instructions

Intel SSE adds the following 64-bit packed integer instructions to the IA-32 architecture. These instructions operate on data in MMX registers and 64-bit memory locations.

NOTE

When Intel SSE2 is present in an IA-32 processor, these instructions are extended to operate on 128-bit operands in XMM registers and 128-bit memory locations.

The PAVGB (compute average of packed unsigned byte integers) and PAVGW (compute average of packed unsigned word integers) instructions compute a SIMD average of two packed unsigned byte or word integer operands, respectively. For each corresponding pair of data elements in the packed source operands, the elements are added together, a 1 is added to the temporary sum, and that result is shifted right one bit position.

The PEXTRW (extract word) instruction copies a selected word from an MMX register into a general-purpose register.

The PINSRW (insert word) instruction copies a word from a general-purpose register or from memory into a selected word location in an MMX register.

The PMAXUB (maximum of packed unsigned byte integers) instruction compares the corresponding unsigned byte integers in two packed operands and returns the greater of each comparison to the destination operand.

The PMINUB (minimum of packed unsigned byte integers) instruction compares the corresponding unsigned byte integers in two packed operands and returns the lesser of each comparison to the destination operand.

The PMAXSW (maximum of packed signed word integers) instruction compares the corresponding signed word integers in two packed operands and returns the greater of each comparison to the destination operand.

The PMINSW (minimum of packed signed word integers) instruction compares the corresponding signed word integers in two packed operands and returns the lesser of each comparison to the destination operand.

The PMOVMSKB (move byte mask) instruction creates an 8-bit mask from the packed byte integers in an MMX register and stores the result in the low byte of a general-purpose register. The mask contains the most significant bit of each byte in the MMX register. (When operating on 128-bit operands, a 16-bit mask is created.)

The PMULHUW (multiply packed unsigned word integers and store high result) instruction performs a SIMD unsigned multiply of the words in the two source operands and returns the high word of each result to an MMX register.

The PSADBW (compute sum of absolute differences) instruction computes the SIMD absolute differences of the corresponding unsigned byte integers in two source operands, sums the differences, and stores the sum in the low word of the destination operand.

The PSHUFW (shuffle packed word integers) instruction shuffles the words in the source operand according to the order specified by an 8-bit immediate operand and returns the result to the destination operand.

10.4.5 MXCSR State Management Instructions

The MXCSR state management instructions (LDMXCSR and STMXCSR) load and save the state of the MXCSR register, respectively. The LDMXCSR instruction loads the MXCSR register from memory, while the STMXCSR instruction stores the contents of the register to memory.

10.4.6 Cacheability Control, Prefetch, and Memory Ordering Instructions

Intel SSE introduced several new instructions to give programs more control over the caching of data. They also introduces the PREFETCH*h* instructions, which provide the ability to prefetch data to a specified cache level, and the SFENCE instruction, which enforces program ordering on stores. These instructions are described in the following sections.

10.4.6.1 Cacheability Control Instructions

The following three instructions enable data from the MMX and XMM registers to be stored to memory using a non-temporal hint. The non-temporal hint directs the processor to store the data to memory without writing the data into the cache hierarchy. See Section 10.4.6.2, "Caching of Temporal vs. Non-Temporal Data," for information about non-temporal stores and hints.

The MOVNTQ (store quadword using non-temporal hint) instruction stores packed integer data from an MMX register to memory, using a non-temporal hint.

The MOVNTPS (store packed single precision floating-point values using non-temporal hint) instruction stores packed floating-point data from an XMM register to memory, using a non-temporal hint.

The MASKMOVQ (store selected bytes of quadword) instruction stores selected byte integers from an MMX register to memory, using a byte mask to selectively write the individual bytes. This instruction also uses a non-temporal hint.

10.4.6.2 Caching of Temporal vs. Non-Temporal Data

Data referenced by a program can be temporal (data will be used again) or non-temporal (data will be referenced once and not reused in the immediate future). For example, program code is generally temporal, whereas, multimedia data, such as the display list in a 3-D graphics application, is often non-temporal. To make efficient use of the processor's caches, it is generally desirable to cache temporal data and not cache non-temporal data. Overloading the processor's caches with non-temporal data is sometimes referred to as "polluting the caches." The Intel SSE and SSE2 cacheability control instructions enable a program to write non-temporal data to memory in a manner that minimizes pollution of caches.

These Intel SSE and SSE2 non-temporal store instructions minimize cache pollutions by treating the memory being accessed as the write combining (WC) type. If a program specifies a non-temporal store with one of these instructions and the memory type of the destination region is write back (WB), write through (WT), or write combining (WC), the processor will do the following:

- If the memory location being written to is present in the cache hierarchy, the data in the caches is evicted.[1]

1. Some older CPU implementations (e.g., Pentium M) allowed addresses being written with a non-temporal store instruction to be updated in-place if the memory type was not WC and line was already in the cache.

- The non-temporal data is written to memory with WC semantics.

See also: Chapter 14, "Memory Cache Control," in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3A.

Using the WC semantics, the store transaction will be weakly ordered, meaning that the data may not be written to memory in program order, and the store will not write allocate (that is, the processor will not fetch the corresponding cache line into the cache hierarchy, prior to performing the store). Also, different processor implementations may choose to collapse and combine these stores.

The memory type of the region being written to can override the non-temporal hint, if the memory address specified for the non-temporal store is in uncacheable memory. Uncacheable as referred to here means that the region being written to has been mapped with either an uncacheable (UC) or write protected (WP) memory type.

In general, WC semantics require software to ensure coherence, with respect to other processors and other system agents (such as graphics cards). Appropriate use of synchronization and fencing must be performed for producer-consumer usage models. Fencing ensures that all system agents have global visibility of the stored data; for instance, failure to fence may result in a written cache line staying within a processor and not being visible to other agents.

The memory type visible on the bus in the presence of memory type aliasing is implementation specific. As one possible example, the memory type written to the bus may reflect the memory type for the first store to this line, as seen in program order; other alternatives are possible. This behavior should be considered reserved, and dependence on the behavior of any particular implementation risks future incompatibility.

NOTE

Some older CPU implementations (e.g., Pentium M) may implement non-temporal stores by updating in place data that already reside in the cache hierarchy. For such processors, the destination region should also be mapped as WC. If mapped as WB or WT, there is the potential for speculative processor reads to bring the data into the caches; in this case, non-temporal stores would then update in place, and data would not be flushed from the processor by a subsequent fencing operation.

10.4.6.3 PREFETCH*h* Instructions

The PREFETCH*h* instructions permit programs to load data into the processor at a suggested cache level, so that the data is closer to the processor's load and store unit when it is needed. These instructions fetch 32 aligned bytes (or more, depending on the implementation) containing the addressed byte to a location in the cache hierarchy specified by the temporal locality hint (see Table 10-1). In this table, the first-level cache is closest to the processor and second-level cache is farther away from the processor than the first-level cache. The hints specify a prefetch of either temporal or non-temporal data (see Section 10.4.6.2, "Caching of Temporal vs. Non-Temporal Data"). Subsequent accesses to temporal data are treated like normal accesses, while those to non-temporal data will continue to minimize cache pollution. If the data is already present at a level of the cache hierarchy that is closer to the processor, the PREFETCH*h* instruction will not result in any data movement. The PREFETCH*h* instructions do not affect functional behavior of the program.

See Section 11.6.13, "Cacheability Hint Instructions," for additional information about the PREFETCH*h* instructions.

Table 10-1. PREFETCH*h* Instructions Caching Hints

PREFETCH*h* Instruction Mnemonic	Actions
PREFETCHT0	Temporal data—fetch data into all levels of cache hierarchy: ▪ Pentium III processor—1st-level cache or 2nd-level cache ▪ Pentium 4 and Intel Xeon processor—2nd-level cache
PREFETCHT1	Temporal data—fetch data into level 2 cache and higher ▪ Pentium III processor—2nd-level cache ▪ Pentium 4 and Intel Xeon processor—2nd-level cache

Table 10-1. PREFETCH*h* Instructions Caching Hints (Contd.)

PREFETCH*h* Instruction Mnemonic	Actions
PREFETCHT2	Temporal data—fetch data into level 2 cache and higher ▪ Pentium III processor—2nd-level cache ▪ Pentium 4 and Intel Xeon processor—2nd-level cache
PREFETCHNTA	Non-temporal data—fetch data into location close to the processor, minimizing cache pollution ▪ Pentium III processor—1st-level cache ▪ Pentium 4 and Intel Xeon processor—2nd-level cache

10.4.6.4 SFENCE Instruction

The SFENCE (Store Fence) instruction controls write ordering by creating a fence for memory store operations. This instruction guarantees that the result of every store instruction that precedes the store fence in program order is globally visible before any store instruction that follows the fence. The SFENCE instruction provides an efficient way of ensuring ordering between procedures that produce weakly-ordered data and procedures that consume that data.

10.5 FXSAVE AND FXRSTOR INSTRUCTIONS

The FXSAVE and FXRSTOR instructions were introduced into the IA-32 architecture in the Pentium II processor family (prior to the introduction of the SSE extensions). The original versions of these instructions performed a fast save and restore, respectively, of the x87 execution environment (**x87 state**). (By saving the state of the x87 FPU data registers, the FXSAVE and FXRSTOR instructions implicitly save and restore the state of the MMX registers.)

The SSE extensions expanded the scope of these instructions to save and restore the states of the XMM registers and the MXCSR register (**SSE state**), along with x87 state.

The FXSAVE and FXRSTOR instructions can be used in place of the FSAVE/FNSAVE and FRSTOR instructions; however, the operation of the FXSAVE and FXRSTOR instructions are not identical to the operation of FSAVE/FNSAVE and FRSTOR.

NOTE

The FXSAVE and FXRSTOR instructions are not considered part of the SSE instruction group. They have a separate CPUID feature bit to indicate whether they are present (if CPUID.01H:EDX.FXSR[24] = 1). The CPUID feature bit for SSE extensions does not indicate the presence of FXSAVE and FXRSTOR.

The FXSAVE and FXRSTOR instructions organize x87 state and SSE state in a region of memory called the **FXSAVE area**. Section 10.5.1 provides details of the FXSAVE area and its format. Section 10.5.2 describes operation of FXSAVE, and Section 10.5.3 describes the operation of FXRSTOR.

10.5.1 FXSAVE Area

The FXSAVE and FXRSTOR instructions organize x87 state and SSE state in a region of memory called the **FXSAVE area**. Each of the instructions takes a memory operand that specifies the 16-byte aligned base address of the FXSAVE area on which it operates.

Every FXSAVE area comprises the 512 bytes starting at the area's base address. Table 10-2 illustrates the format of the first 416 bytes of the legacy region of an FXSAVE area.

Table 10-2. Format of an FXSAVE Area

15 14	13 12	11 10	9 8	7 6	5	4	3 2	1 0	
Reserved	CS or FPU IP bits 63:32	FPU IP bits 31:0		FOP	Rsvd.	FTW	FSW	FCW	**0**
MXCSR_MASK		MXCSR		Reserved	DS or FPU DP bits 63:32		FPU DP bits 31:0		**16**
Reserved		ST0/MM0							**32**
Reserved		ST1/MM1							**48**
Reserved		ST2/MM2							**64**
Reserved		ST3/MM3							**80**
Reserved		ST4/MM4							**96**
Reserved		ST5/MM5							**112**
Reserved		ST6/MM6							**128**
Reserved		ST7/MM7							**144**
XMM0									**160**
XMM1									**176**
XMM2									**192**
XMM3									**208**
XMM4									**224**
XMM5									**240**
XMM6									**256**
XMM7									**272**
XMM8									**288**
XMM9									**304**
XMM10									**320**
XMM11									**336**
XMM12									**352**
XMM13									**368**
XMM14									**384**
XMM15									**400**

The x87 state component comprises bytes 23:0 and bytes 159:32. The SSE state component comprises bytes 31:24 and bytes 415:160. FXSAVE and FXRSTOR do not use bytes 511:416; bytes 463:416 are reserved.

Section 10.5.2 and Section 10.5.3 provide details of how FXSAVE and FXRSTOR use an FXSAVE area.

10.5.1.1 x87 State

Table 10-2 illustrates how FXSAVE and FXRSTOR organize x87 state and SSE state; the x87 state is listed below, along with details of its interactions with FXSAVE and FXRSTOR:

- Bytes 1:0, 3:2, and 7:6 are used for x87 FPU Control Word (FCW), x87 FPU Status Word (FSW), and x87 FPU Opcode (FOP), respectively.

- Byte 4 is used for an abridged version of the x87 FPU Tag Word (FTW). The following items describe its usage:
  - For each j, $0 \le j \le 7$, FXSAVE saves a 0 into bit j of byte 4 if x87 FPU data register STj has a empty tag; otherwise, FXSAVE saves a 1 into bit j of byte 4.
  - For each j, $0 \le j \le 7$, FXRSTOR establishes the tag value for x87 FPU data register STj as follows. If bit j of byte 4 is 0, the tag for STj in the tag register for that data register is marked empty (11B); otherwise, the x87 FPU sets the tag for STj based on the value being loaded into that register (see below).
- Bytes 15:8 are used as follows:
  - If the instruction has no REX prefix, or if REX.W = 0:
    - Bytes 11:8 are used for bits 31:0 of the x87 FPU Instruction Pointer Offset (FIP).
    - If CPUID.07H.00H:EBX[13] = 0, bytes 13:12 are used for x87 FPU Instruction Pointer Selector (FPU CS). Otherwise, the processor deprecates the FPU CS value: FXSAVE saves it as 0000H.
    - Bytes 15:14 are not used.
  - If the instruction has a REX prefix with REX.W = 1, bytes 15:8 are used for the full 64 bits of FIP.
- Bytes 23:16 are used as follows:
  - If the instruction has no REX prefix, or if REX.W = 0:
    - Bytes 19:16 are used for bits 31:0 of the x87 FPU Data Pointer Offset (FDP).
    - If CPUID.07H.00H:EBX[13] = 0, bytes 21:20 are used for x87 FPU Data Pointer Selector (FPU DS). Otherwise, the processor deprecates the FPU DS value: FXSAVE saves it as 0000H.
    - Bytes 23:22 are not used.
  - If the instruction has a REX prefix with REX.W = 1, bytes 23:16 are used for the full 64 bits of FDP.
- Bytes 31:24 are used for SSE state (see Section 10.5.1.2).
- Bytes 159:32 are used for the registers ST0–ST7 (MM0–MM7). Each of the 8 registers is allocated a 128-bit region, with the low 80 bits used for the register and the upper 48 bits unused.

10.5.1.2 SSE State

Table 10-2 illustrates how FXSAVE and FXRSTOR organize x87 state and SSE state; the SSE state is listed below, along with details of its interactions with FXSAVE and FXRSTOR:

- Bytes 23:0 are used for x87 state (see Section 10.5.1.1).
- Bytes 27:24 are used for the MXCSR register. FXRSTOR generates a general-protection fault (#GP) in response to an attempt to set any of the reserved bits in the MXCSR register.
- Bytes 31:28 are used for the MXCSR_MASK value. FXRSTOR ignores this field.
- Bytes 159:32 are used for x87 state.
- Bytes 287:160 are used for the registers XMM0–XMM7.
- Bytes 415:288 are used for the registers XMM8–XMM15. These fields are used only in 64-bit mode. Executions of FXSAVE outside 64-bit mode do not write to these bytes; executions of FXRSTOR outside 64-bit mode do not read these bytes and do not update XMM8–XMM15.

If CR4.OSFXSR = 0, FXSAVE and FXRSTOR may or may not operate on SSE state; this behavior is implementation dependent. Moreover, SSE instructions cannot be used unless CR4.OSFXSR = 1.

10.5.2 Operation of FXSAVE

The FXSAVE instruction takes a single memory operand, which is an FXSAVE area. The instruction stores x87 state and SSE state to the FXSAVE area. See Section 10.5.1.1 and Section 10.5.1.2 for details regarding mode-specific operation and operation determined by instruction prefixes.

10.5.3 Operation of FXRSTOR

The FXRSTOR instruction takes a single memory operand, which is an FXSAVE area. If the value at bytes 27:24 of the FXSAVE area is not a legal value for the MXCSR register (e.g., the value sets reserved bits), execution of FXRSTOR results in a general-protection fault (#GP). Otherwise, the instruction loads x87 state and SSE state from the FXSAVE area. See Section 10.5.1.1 and Section 10.5.1.2 for details regarding mode-specific operation and operation determined by instruction prefixes.

10.6 HANDLING INTEL® SSE INSTRUCTION EXCEPTIONS

See Section 11.5, "Intel® SSE, SSE2, and SSE3 Exceptions," for a detailed discussion of the general and SIMD floating-point exceptions that can be generated with the Intel SSE instructions and for guidelines for handling these exceptions when they occur.

10.7 WRITING APPLICATIONS WITH INTEL® SSE

See Section 11.6, "Writing Applications with Intel® SSE and SSE2," for additional information about writing applications and operating-system code using Intel SSE.

CHAPTER 11
PROGRAMMING WITH INTEL® STREAMING SIMD EXTENSIONS 2 (INTEL® SSE2)

The streaming SIMD extensions 2 (SSE2) were introduced into the IA-32 architecture in the Pentium 4 and Intel Xeon processors. These extensions enhance the performance of IA-32 processors for advanced 3-D graphics, video decoding/encoding, speech recognition, E-commerce, Internet, scientific, and engineering applications.

This chapter describes the SSE2 extensions and provides information to assist in writing application programs that use these and the SSE extensions.

11.1 OVERVIEW OF INTEL® SSE2

Intel SSE2 uses the single instruction multiple data (SIMD) execution model that is used with MMX technology and Intel SSE. They extend this model with support for packed double precision floating-point values and for 128-bit packed integers.

If CPUID.01H:EDX.SSE2[26] = 1, Intel SSE2 is present.

Intel SSE2 adds the following features to the IA-32 architecture, while maintaining backward compatibility with all existing IA-32 processors, applications, and operating systems.

- Six data types:
  - 128-bit packed double precision floating-point (two IEEE Standard 754 double precision floating-point values packed into a double quadword).
  - 128-bit packed byte integers.
  - 128-bit packed word integers.
  - 128-bit packed doubleword integers.
  - 128-bit packed quadword integers.
- Instructions to support the additional data types and extend existing SIMD integer operations:
  - Packed and scalar double precision floating-point instructions.
  - Additional 64-bit and 128-bit SIMD integer instructions.
  - 128-bit versions of SIMD integer instructions introduced with the MMX technology and Intel SSE.
  - Additional cacheability-control and instruction-ordering instructions.
- Modifications to existing IA-32 instructions to support Intel SSE2 features:
  - Extensions and modifications to the CPUID instruction.
  - Modifications to the RDPMC instruction.

These new features extend the IA-32 architecture's SIMD programming model in three important ways:

- They provide the ability to perform SIMD operations on pairs of packed double precision floating-point values. This permits higher precision computations to be carried out in XMM registers, which enhances processor performance in scientific and engineering applications and in applications that use advanced 3-D geometry techniques (such as ray tracing). Additional flexibility is provided with instructions that operate on single (scalar) double precision floating-point values located in the low quadword of an XMM register.
- They provide the ability to operate on 128-bit packed integers (bytes, words, doublewords, and quadwords) in XMM registers. This provides greater flexibility and greater throughput when performing SIMD operations on packed integers. The capability is particularly useful for applications such as RSA authentication and RC5 encryption. Using the full set of SIMD registers, data types, and instructions provided with the MMX technology and Intel SSE/SSE2, programmers can develop algorithms that finely mix packed single- and double precision floating-point data and 64- and 128-bit packed integer data.
- Intel SSE2 enhances the support introduced with Intel SSE for controlling the cacheability of SIMD data. Intel SSE2 cache control instructions provide the ability to stream data in and out of the XMM registers without polluting the caches and the ability to prefetch data before it is actually used.

Intel SSE2 is fully compatible with all software written for IA-32 processors. All existing software continues to run correctly, without modification, on processors that incorporate Intel SSE2, as well as in the presence of applications that incorporate these extensions. Enhancements to the CPUID instruction permit detection of Intel SSE2. Also, because Intel SSE2 uses the same registers as Intel SSE, no new operating-system support is required for saving and restoring program state during a context switch beyond that provided for Intel SSE.

Intel SSE2 is accessible from all IA-32 execution modes: protected mode, real address mode, and virtual 8086 mode.

The following sections in this chapter describe the programming environment for Intel SSE2, including: the 128-bit XMM floating-point register set, data types, and Intel SSE2 instructions. The chapter also describes exceptions that can be generated with the Intel SSE and SSE2 instructions and gives guidelines for writing applications with Intel SSE and SSE2.

For additional information about Intel SSE2, see:

- The Intel® 64 and IA-32 Architectures Software Developer's Manual, Volumes 2A, 2B, 2C, & 2D, provides a detailed description of individual Intel SSE2 instructions.
- Chapter 16, "System Programming for Instruction Set Extensions and Processor Extended States," in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3A, gives guidelines for integrating Intel SSE and SSE2 into an operating-system environment.

11.2 INTEL® SSE2 PROGRAMMING ENVIRONMENT

Figure 11-1 shows the programming environment for Intel SSE2. No new registers or other instruction execution state are defined with Intel SSE2. Intel SSE2 instructions use XMM registers, MMX registers, and/or IA-32 general-purpose registers, as follows:

- **XMM registers —** These eight registers (see Figure 10-2) are used to operate on packed or scalar double precision floating-point data. Scalar operations are operations performed on individual (unpacked) double precision floating-point values stored in the low quadword of an XMM register. XMM registers are also used to perform operations on 128-bit packed integer data. They are referenced by the names XMM0 through XMM7.

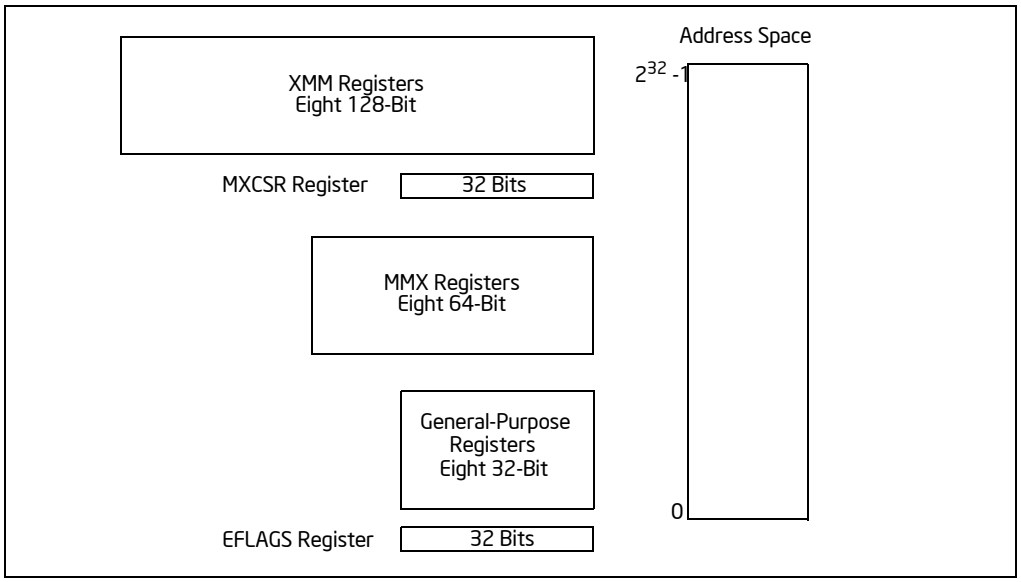


Figure 11-1. Intel® Streaming SIMD Extensions 2 Execution Environment

- **MXCSR register —** This 32-bit register (see Figure 10-3) provides status and control bits used in floating-point operations. The denormals-are-zeros and flush-to-zero flags in this register provide a higher performance alternative for the handling of denormal source operands and denormal (underflow) results. For more

information on the functions of these flags see Section 10.2.3.4, "Denormals-Are-Zeros," and Section 10.2.3.3, "Flush-To-Zero."

- **MMX registers —** These eight registers (see Figure 9-2) are used to perform operations on 64-bit packed integer data. They are also used to hold operands for some operations performed between MMX and XMM registers. MMX registers are referenced by the names MM0 through MM7.
- **General-purpose registers —** The eight general-purpose registers (see Figure 3-5) are used along with the existing IA-32 addressing modes to address operands in memory. MMX and XMM registers cannot be used to address memory. The general-purpose registers are also used to hold operands for some SSE2 instructions. These registers are referenced by the names EAX, EBX, ECX, EDX, EBP, ESI, EDI, and ESP.
- **EFLAGS register —** This 32-bit register (see Figure 3-8) is used to record the results of some compare operations.

11.2.1 Intel® SSE2 in 64-Bit Mode and Compatibility Mode

In compatibility mode, Intel SSE2 functions like it does in protected mode. In 64-bit mode, eight additional XMM registers are accessible. Registers XMM8-XMM15 are accessed by using REX prefixes.

Memory operands are specified using the ModR/M, SIB encoding described in Section 3.7.5.

Some Intel SSE2 instructions may be used to operate on general-purpose registers. Use the REX.W prefix to access 64-bit general-purpose registers. Note that if a REX prefix is used when it has no meaning, the prefix is ignored.

11.2.2 Compatibility of Intel® SSE2 with Intel® SSE, MMX Technology, and x87 FPU Programming Environment

Intel SSE2 does not introduce any new state to the IA-32 execution environment beyond that of Intel SSE. Intel SSE2 represents an enhancement of Intel SSE; they are fully compatible and share the same state information. Intel SSE and SSE2 instructions can be executed together in the same instruction stream without the need to save state when switching between instruction sets.

XMM registers are independent of the x87 FPU and MMX registers; so Intel SSE and SSE2 operations performed on XMM registers can be performed in parallel with x87 FPU or MMX technology operations; see Section 11.6.7, "Interaction of Intel® SSE and SSE2 Instructions with x87 FPU and MMX Instructions."

The FXSAVE and FXRSTOR instructions save and restore the SSE and SSE2 states along with the x87 FPU and MMX states.

11.2.3 Denormals-Are-Zeros Flag

The denormals-are-zeros flag (bit 6 in the MXCSR register) was introduced into the IA-32 architecture with Intel SSE2. See Section 10.2.3.4, "Denormals-Are-Zeros," for a description of this flag.

11.3 INTEL® SSE2 DATA TYPES

Intel SSE2 introduced one 128-bit packed floating-point data type and four 128-bit SIMD integer data types to the IA-32 architecture (see Figure 11-2).

- **Packed double precision floating-point —** This 128-bit data type consists of two IEEE 64-bit double precision floating-point values packed into a double quadword. See Figure 4-3 for the layout of a 64-bit double precision floating-point value; refer to Section 4.2.2, "Floating-Point Data Types," for a detailed description of double precision floating-point values.
- **128-bit packed integers —** The four 128-bit packed integer data types can contain 16 byte integers, 8 word integers, 4 doubleword integers, or 2 quadword integers. Refer to Section 4.6.2, "128-Bit Packed SIMD Data Types," for a detailed description of the 128-bit packed integers.

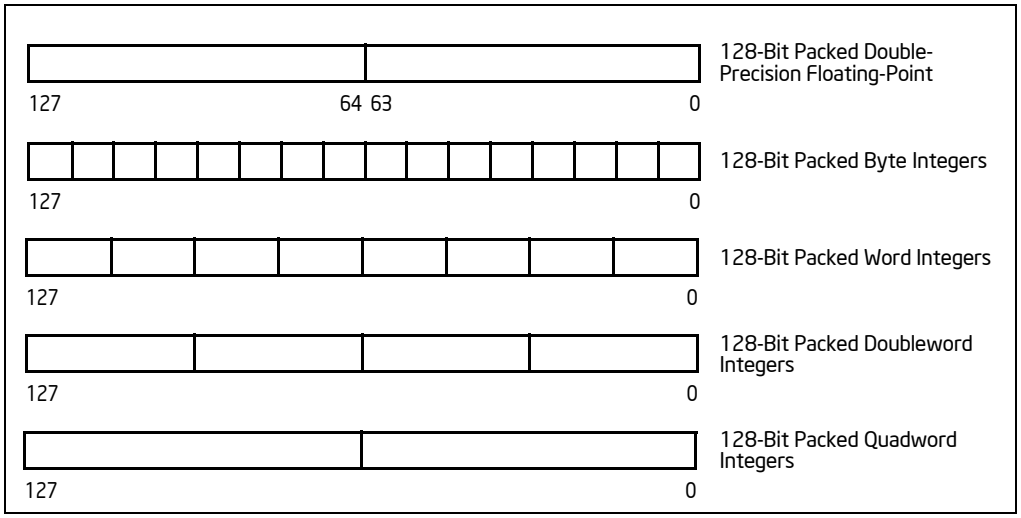


Figure 11-2. Data Types Introduced with Intel® SSE2

All of these data types are operated on in XMM registers or memory. Instructions are provided to convert between these 128-bit data types and the 64-bit and 32-bit data types.

The address of a 128-bit packed memory operand must be aligned on a 16-byte boundary, except in the following cases:

- A MOVUPD instruction that supports unaligned accesses.
- Scalar instructions that use an 8-byte memory operand that is not subject to alignment requirements.

Figure 4-2 shows the byte order of 128-bit (double quadword) and 64-bit (quadword) data types in memory.

11.4 INTEL® SSE2 INSTRUCTIONS

The Intel SSE2 instructions are divided into four functional groups:

- Packed and scalar double precision floating-point instructions.
- 64-bit and 128-bit SIMD integer instructions.
- 128-bit extensions of SIMD integer instructions introduced with the MMX technology and Intel SSE.
- Cacheability-control and instruction-ordering instructions.

The following sections provide more information about each group.

11.4.1 Packed and Scalar Double Precision Floating-Point Instructions

The packed and scalar double precision floating-point instructions are divided into the following sub-groups:

- Data movement instructions.
- Arithmetic instructions.
- Comparison instructions.
- Conversion instructions.
- Logical instructions.
- Shuffle instructions.

The packed double precision floating-point instructions perform SIMD operations similarly to the packed single precision floating-point instructions (see Figure 11-3). Each source operand contains two double precision floating-

point values, and the destination operand contains the results of the operation (OP) performed in parallel on the corresponding values (X0 and Y0, and X1 and Y1) in each operand.

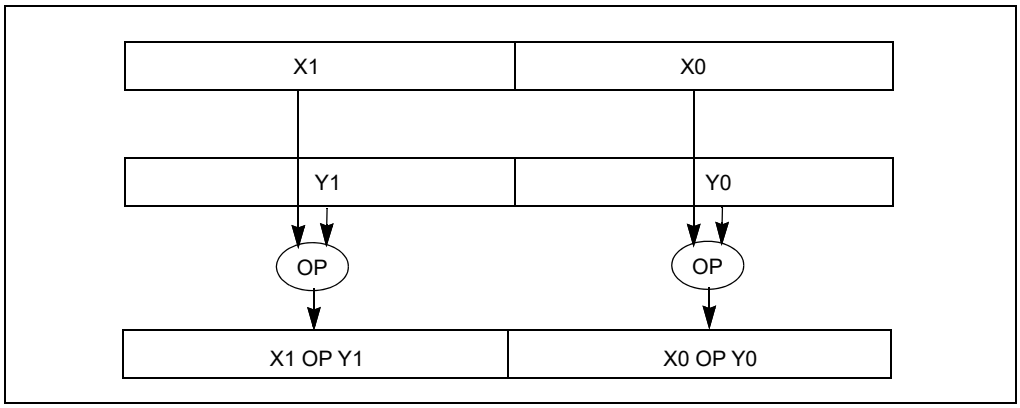


Figure 11-3. Packed Double Precision Floating-Point Operations

The scalar double precision floating-point instructions operate on the low (least significant) quadwords of two source operands (X0 and Y0), as shown in Figure 11-4. The high quadword (X1) of the first source operand is passed through to the destination. The scalar operations are similar to the floating-point operations performed in x87 FPU data registers with the precision control field in the x87 FPU control word set for double precision (53-bit significand), except that x87 stack operations use a 15-bit exponent range for the result while Intel SSE2 operations use an 11-bit exponent range.

See Section 11.6.8, "Compatibility of SIMD and x87 FPU Floating-Point Data Types," for more information about obtaining compatible results when performing both scalar double precision floating-point operations in XMM registers and in x87 FPU data registers.

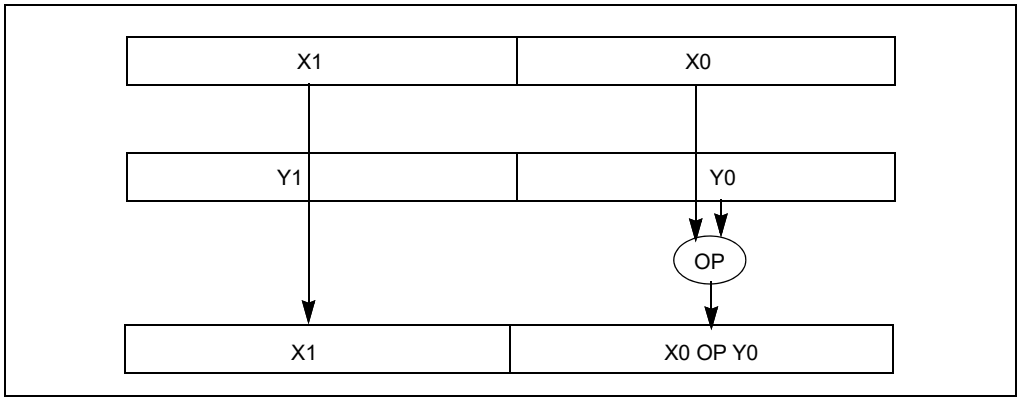


Figure 11-4. Scalar Double Precision Floating-Point Operations

11.4.1.1 Data Movement Instructions

Data movement instructions move double precision floating-point data between XMM registers and between XMM registers and memory.

The MOVAPD (move aligned packed double precision floating-point) instruction transfers a 128-bit packed double precision floating-point operand from memory to an XMM register or vice versa, or between XMM registers. The memory address must be aligned to a 16-byte boundary; if not, a general-protection exception (GP#) is generated.

The MOVUPD (move unaligned packed double precision floating-point) instruction transfers a 128-bit packed double precision floating-point operand from memory to an XMM register or vice versa, or between XMM registers. Alignment of the memory address is not required.

The MOVSD (move scalar double precision floating-point) instruction transfers a 64-bit double precision floating-point operand from memory to the low quadword of an XMM register or vice versa, or between XMM registers. Alignment of the memory address is not required, unless alignment checking is enabled.

The MOVHPD (move high packed double precision floating-point) instruction transfers a 64-bit double precision floating-point operand from memory to the high quadword of an XMM register or vice versa. The low quadword of the register is left unchanged. Alignment of the memory address is not required, unless alignment checking is enabled.

The MOVLPD (move low packed double precision floating-point) instruction transfers a 64-bit double precision floating-point operand from memory to the low quadword of an XMM register or vice versa. The high quadword of the register is left unchanged. Alignment of the memory address is not required, unless alignment checking is enabled.

The MOVMSKPD (move packed double precision floating-point mask) instruction extracts the sign bit of each of the two packed double precision floating-point numbers in an XMM register and saves them in a general-purpose register. This 2-bit value can then be used as a condition to perform branching.

11.4.1.2 Intel® SSE2 Arithmetic Instructions

Intel SSE2 arithmetic instructions perform addition, subtraction, multiply, divide, square root, and maximum/minimum operations on packed and scalar double precision floating-point values.

The ADDPD (add packed double precision floating-point values) and SUBPD (subtract packed double precision floating-point values) instructions add and subtract, respectively, two packed double precision floating-point operands.

The ADDSD (add scalar double precision floating-point values) and SUBSD (subtract scalar double precision floating-point values) instructions add and subtract, respectively, the low double precision floating-point values of two operands and stores the result in the low quadword of the destination operand.

The MULPD (multiply packed double precision floating-point values) instruction multiplies two packed double precision floating-point operands.

The MULSD (multiply scalar double precision floating-point values) instruction multiplies the low double precision floating-point values of two operands and stores the result in the low quadword of the destination operand.

The DIVPD (divide packed double precision floating-point values) instruction divides two packed double precision floating-point operands.

The DIVSD (divide scalar double precision floating-point values) instruction divides the low double precision floating-point values of two operands and stores the result in the low quadword of the destination operand.

The SQRTPD (compute square roots of packed double precision floating-point values) instruction computes the square roots of the values in a packed double precision floating-point operand.

The SQRTSD (compute square root of scalar double precision floating-point values) instruction computes the square root of the low double precision floating-point value in the source operand and stores the result in the low quadword of the destination operand.

The MAXPD (return maximum of packed double precision floating-point values) instruction compares the corresponding values in two packed double precision floating-point operands and returns the numerically greater value from each comparison to the destination operand.

The MAXSD (return maximum of scalar double precision floating-point values) instruction compares the low double precision floating-point values from two packed double precision floating-point operands and returns the numerically higher value from the comparison to the low quadword of the destination operand.

The MINPD (return minimum of packed double precision floating-point values) instruction compares the corresponding values from two packed double precision floating-point operands and returns the numerically lesser value from each comparison to the destination operand.

The MINSD (return minimum of scalar double precision floating-point values) instruction compares the low values from two packed double precision floating-point operands and returns the numerically lesser value from the comparison to the low quadword of the destination operand.

11.4.1.3 Intel® SSE2 Logical Instructions

intel SSE2 logical instructions perform AND, AND NOT, OR, and XOR operations on packed double precision floating-point values.

The ANDPD (bitwise logical AND of packed double precision floating-point values) instruction returns the logical AND of two packed double precision floating-point operands.

The ANDNPD (bitwise logical AND NOT of packed double precision floating-point values) instruction returns the logical AND NOT of two packed double precision floating-point operands.

The ORPD (bitwise logical OR of packed double precision floating-point values) instruction returns the logical OR of two packed double precision floating-point operands.

The XORPD (bitwise logical XOR of packed double precision floating-point values) instruction returns the logical XOR of two packed double precision floating-point operands.

11.4.1.4 Intel® SSE2 Comparison Instructions

Intel SSE2 compare instructions compare packed and scalar double precision floating-point values and return the results of the comparison either to the destination operand or to the EFLAGS register.

The CMPPD (compare packed double precision floating-point values) instruction compares the corresponding values from two packed double precision floating-point operands, using an immediate operand as a predicate, and returns a 64-bit mask result of all 1s or all 0s for each comparison to the destination operand. The value of the immediate operand allows the selection of any of eight compare conditions: equal, less than, less than equal, unordered, not equal, not less than, not less than or equal, or ordered.

The CMPSD (compare scalar double precision floating-point values) instruction compares the low values from two packed double precision floating-point operands, using an immediate operand as a predicate, and returns a 64-bit mask result of all 1s or all 0s for the comparison to the low quadword of the destination operand. The immediate operand selects the compare condition as with the CMPPD instruction.

The COMISD (compare scalar double precision floating-point values and set EFLAGS) and UCOMISD (unordered compare scalar double precision floating-point values and set EFLAGS) instructions compare the low values of two packed double precision floating-point operands and set the ZF, PF, and CF flags in the EFLAGS register to show the result (greater than, less than, equal, or unordered). These two instructions differ as follows: the COMISD instruction signals a floating-point invalid-operation (#I) exception when a source operand is either a QNaN or an SNaN; the UCOMISD instruction only signals an invalid-operation exception when a source operand is an SNaN.

11.4.1.5 Intel® SSE2 Shuffle and Unpack Instructions

Intel SSE2 shuffle instructions shuffle the contents of two packed double precision floating-point values and store the results in the destination operand.

The SHUFPD (shuffle packed double precision floating-point values) instruction places either of the two packed double precision floating-point values from the destination operand in the low quadword of the destination operand, and places either of the two packed double precision floating-point values from source operand in the high quadword of the destination operand (see Figure 11-5). By using the same register for the source and destination operands, the SHUFPD instruction can swap two packed double precision floating-point values.

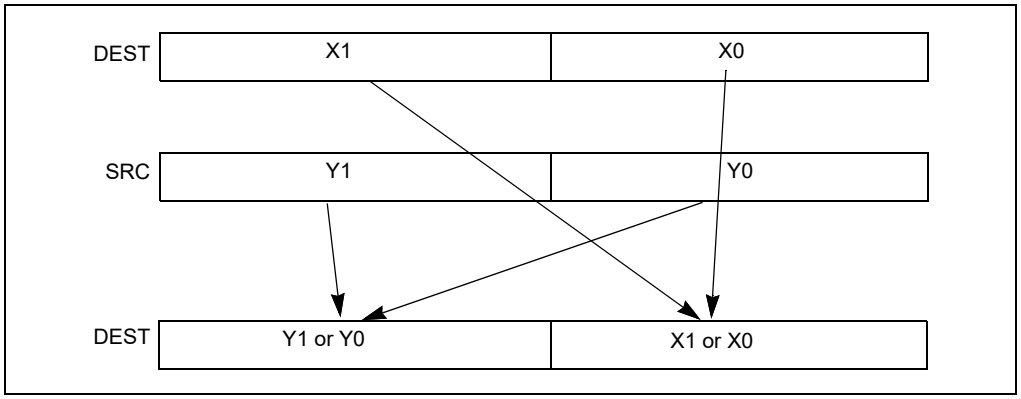


Figure 11-5. SHUFPD Instruction, Packed Shuffle Operation

The UNPCKHPD (unpack and interleave high packed double precision floating-point values) instruction performs an interleaved unpack of the high values from the source and destination operands and stores the result in the destination operand (see Figure 11-6).

The UNPCKLPD (unpack and interleave low packed double precision floating-point values) instruction performs an interleaved unpack of the low values from the source and destination operands and stores the result in the destination operand (see Figure 11-7).

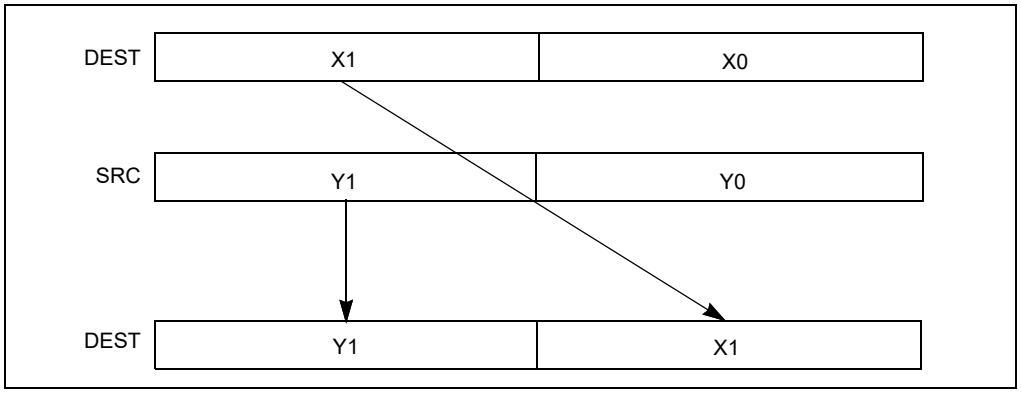


Figure 11-6. UNPCKHPD Instruction, High Unpack, and Interleave Operation

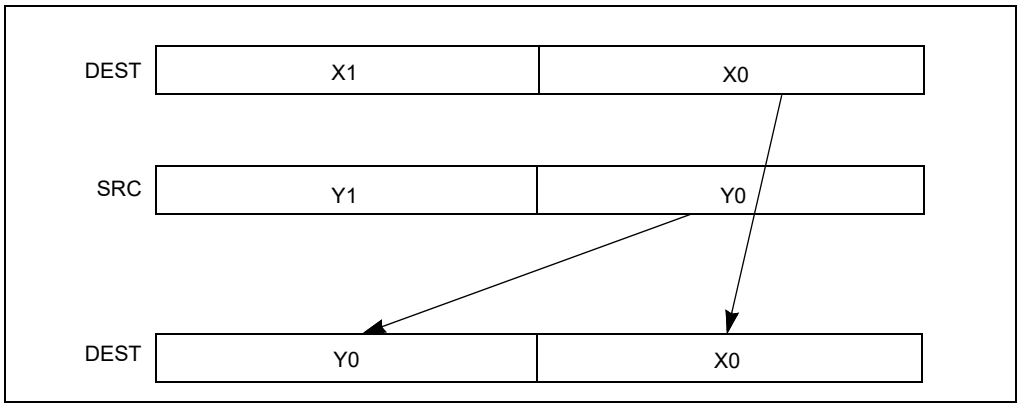


Figure 11-7. UNPCKLPD Instruction, Low Unpack, and Interleave Operation

11.4.1.6 Intel® SSE2 Conversion Instructions

Intel SSE2 conversion instructions (see Figure 11-8) support packed and scalar conversions between:

- Double precision and single precision floating-point formats.
- Double precision floating-point and doubleword integer formats.
- Single precision floating-point and doubleword integer formats.

Conversion between double precision and single precision floating-points values — The following instructions convert operands between double precision and single precision floating-point formats. The operands being operated on are contained in XMM registers or memory (at most, one operand can reside in memory; the destination is always an MMX register).

The CVTPS2PD (convert packed single precision floating-point values to packed double precision floating-point values) instruction converts two packed single precision floating-point values to two double precision floating-point values.

The CVTPD2PS (convert packed double precision floating-point values to packed single precision floating-point values) instruction converts two packed double precision floating-point values to two single precision floating-point values. When a conversion is inexact, the result is rounded according to the rounding mode selected in the MXCSR register.

The CVTSS2SD (convert scalar single precision floating-point value to scalar double precision floating-point value) instruction converts a single precision floating-point value to a double precision floating-point value.

The CVTSD2SS (convert scalar double precision floating-point value to scalar single precision floating-point value) instruction converts a double precision floating-point value to a single precision floating-point value. When the conversion is inexact, the result is rounded according to the rounding mode selected in the MXCSR register.

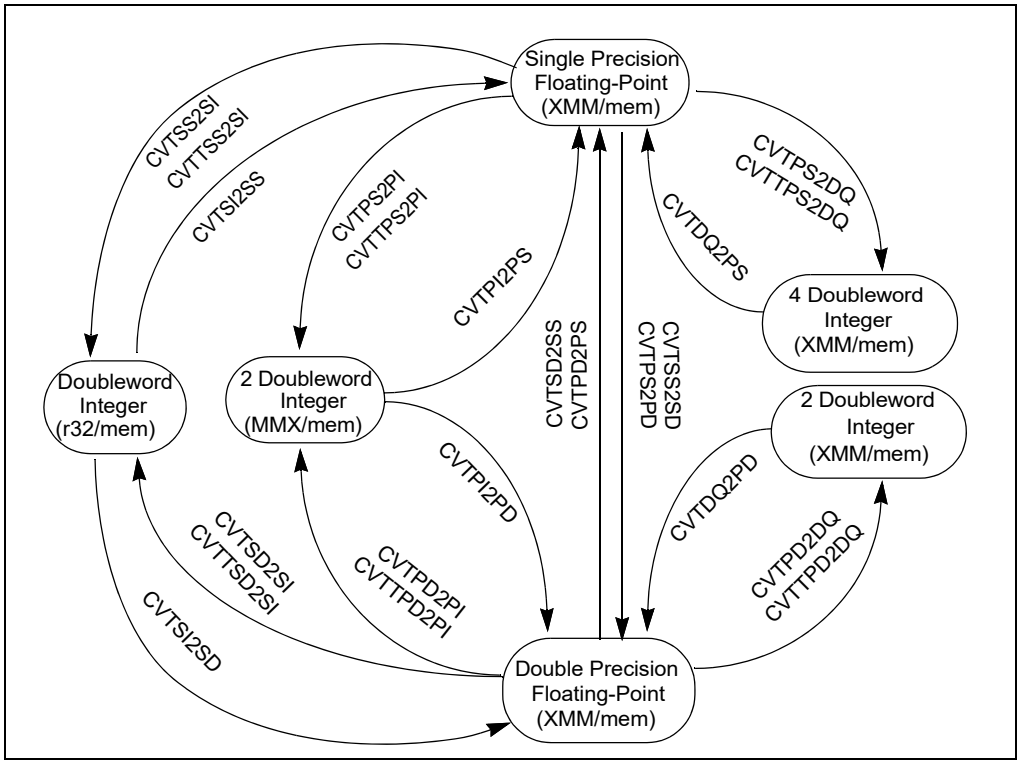


Figure 11-8. Intel® SSE and SSE2 Conversion Instructions

Conversion between double precision floating-point values and doubleword integers — The following instructions convert operands between double precision floating-point and doubleword integer formats. Operands

are housed in XMM registers, MMX registers, general registers or memory (at most one operand can reside in memory; the destination is always an XMM, MMX, or general register).

The CVTPD2PI (convert packed double precision floating-point values to packed doubleword integers) instruction converts two packed double precision floating-point numbers to two packed signed doubleword integers, with the result stored in an MMX register. When rounding to an integer value, the source value is rounded according to the rounding mode in the MXCSR register. The CVTTPD2PI (convert with truncation packed double precision floating-point values to packed doubleword integers) instruction is similar to the CVTPD2PI instruction except that truncation is used to round a source value to an integer value; see Section 4.8.4.2, "Truncation with Intel® SSE, SSE2, and AVX Conversion Instructions."

The CVTPI2PD (convert packed doubleword integers to packed double precision floating-point values) instruction converts two packed signed doubleword integers to two double precision floating-point values.

The CVTPD2DQ (convert packed double precision floating-point values to packed doubleword integers) instruction converts two packed double precision floating-point numbers to two packed signed doubleword integers, with the result stored in the low quadword of an XMM register. When rounding an integer value, the source value is rounded according to the rounding mode selected in the MXCSR register. The CVTTPD2DQ (convert with truncation packed double precision floating-point values to packed doubleword integers) instruction is similar to the CVTPD2DQ instruction except that truncation is used to round a source value to an integer value; see Section 4.8.4.2, "Truncation with Intel® SSE, SSE2, and AVX Conversion Instructions."

The CVTDQ2PD (convert packed doubleword integers to packed double precision floating-point values) instruction converts two packed signed doubleword integers located in the low-order doublewords of an XMM register to two double precision floating-point values.

The CVTSD2SI (convert scalar double precision floating-point value to signed integer) instruction converts a double precision floating-point value to a signed integer, and stores the result in a general-purpose register. When rounding an integer value, the source value is rounded according to the rounding mode selected in the MXCSR register. The CVTTSD2SI (convert with truncation scalar double precision floating-point value to signed integer) instruction is similar to the CVTSD2SI instruction except that truncation is used to round the source value to an integer value; see Section 4.8.4.2, "Truncation with Intel® SSE, SSE2, and AVX Conversion Instructions."

The CVTSI2SD (convert signed integer to scalar double precision floating-point value) instruction converts a signed doubleword or quadword integer in a general-purpose register to a double precision floating-point number, and stores the result in an XMM register.

Conversion between single precision floating-point and doubleword integer formats — These instructions convert between packed single precision floating-point and packed doubleword integer formats. Operands are housed in XMM registers, MMX registers, general registers, or memory (the latter for at most one source operand). The destination is always an XMM, MMX, or general register. These SSE2 instructions supplement conversion instructions (CVTPI2PS, CVTPS2PI, CVTTPS2PI, CVTSI2SS, CVTSS2SI, and CVTTSS2SI) introduced with Intel SSE extensions.

The CVTPS2DQ (convert packed single precision floating-point values to packed doubleword integers) instruction converts four packed single precision floating-point values to four packed signed doubleword integers, with the source and destination operands in XMM registers or memory (the latter for at most one source operand). When the conversion is inexact, the rounded value according to the rounding mode selected in the MXCSR register is returned. The CVTTPS2DQ (convert with truncation packed single precision floating-point values to packed doubleword integers) instruction is similar to the CVTPS2DQ instruction except that truncation is used to round a source value to an integer value; see Section 4.8.4.2, "Truncation with Intel® SSE, SSE2, and AVX Conversion Instructions."

The CVTDQ2PS (convert packed doubleword integers to packed single precision floating-point values) instruction converts four packed signed doubleword integers to four packed single precision floating-point numbers, with the source and destination operands in XMM registers or memory (the latter for at most one source operand). When the conversion is inexact, the rounded value according to the rounding mode selected in the MXCSR register is returned.

11.4.2 Intel® SSE2 64-Bit and 128-Bit SIMD Integer Instructions

Intel SSE2 adds several 128-bit packed integer instructions to the IA-32 architecture. Where appropriate, a 64-bit version of each of these instructions is also provided. The 128-bit versions of instructions operate on data in XMM registers; 64-bit versions operate on data in MMX registers. The instructions follow.

The MOVDQA (move aligned double quadword) instruction transfers a double quadword operand from memory to an XMM register or vice versa; or between XMM registers. The memory address must be aligned to a 16-byte boundary; otherwise, a general-protection exception (#GP) is generated.

The MOVDQU (move unaligned double quadword) instruction performs the same operations as the MOVDQA instruction, except that 16-byte alignment of a memory address is not required.

The PADDQ (packed quadword add) instruction adds two packed quadword integer operands or two single quadword integer operands, and stores the results in an XMM or MMX register, respectively. This instruction can operate on either unsigned or signed (two's complement notation) integer operands.

The PSUBQ (packed quadword subtract) instruction subtracts two packed quadword integer operands or two single quadword integer operands, and stores the results in an XMM or MMX register, respectively. Like the PADDQ instruction, PSUBQ can operate on either unsigned or signed (two's complement notation) integer operands.

The PMULUDQ (multiply packed unsigned doubleword integers) instruction performs an unsigned multiply of unsigned doubleword integers and returns a quadword result. Both 64-bit and 128-bit versions of this instruction are available. The 64-bit version operates on two doubleword integers stored in the low doubleword of each source operand, and the quadword result is returned to an MMX register. The 128-bit version performs a packed multiply of two pairs of doubleword integers. Here, the doublewords are packed in the first and third doublewords of the source operands, and the quadword results are stored in the low and high quadwords of an XMM register.

The PSHUFLW (shuffle packed low words) instruction shuffles the word integers packed into the low quadword of the source operand and stores the shuffled result in the low quadword of the destination operand. An 8-bit immediate operand specifies the shuffle order.

The PSHUFHW (shuffle packed high words) instruction shuffles the word integers packed into the high quadword of the source operand and stores the shuffled result in the high quadword of the destination operand. An 8-bit immediate operand specifies the shuffle order.

The PSHUFD (shuffle packed doubleword integers) instruction shuffles the doubleword integers packed into the source operand and stores the shuffled result in the destination operand. An 8-bit immediate operand specifies the shuffle order.

The PSLLDQ (shift double quadword left logical) instruction shifts the contents of the source operand to the left by the amount of bytes specified by an immediate operand. The empty low-order bytes are cleared (set to 0).

The PSRLDQ (shift double quadword right logical) instruction shifts the contents of the source operand to the right by the amount of bytes specified by an immediate operand. The empty high-order bytes are cleared (set to 0).

The PUNPCKHQDQ (Unpack high quadwords) instruction interleaves the high quadword of the source operand and the high quadword of the destination operand and writes them to the destination register.

The PUNPCKLQDQ (Unpack low quadwords) instruction interleaves the low quadwords of the source operand and the low quadwords of the destination operand and writes them to the destination register.

Two additional SSE instructions enable data movement from the MMX registers to the XMM registers.

The MOVQ2DQ (move quadword integer from MMX to XMM registers) instruction moves the quadword integer from an MMX source register to an XMM destination register.

The MOVDQ2Q (move quadword integer from XMM to MMX registers) instruction moves the low quadword integer from an XMM source register to an MMX destination register.

11.4.3 128-Bit SIMD Integer Instruction Extensions

All of 64-bit SIMD integer instructions introduced with MMX technology and Intel SSE (with the exception of the PSHUFW instruction) have been extended by Intel SSE2 to operate on 128-bit packed integer operands located in XMM registers. The 128-bit versions of these instructions follow the same SIMD conventions regarding packed

operands as the 64-bit versions. For example, where the 64-bit version of the PADDB instruction operates on 8 packed bytes, the 128-bit version operates on 16 packed bytes.

11.4.4 Cacheability Control and Memory Ordering Instructions

Intel SSE2 instructions that give programs more control over the caching, loading, and storing of data. are described below.

11.4.4.1 FLUSH Cache Line

The CLFLUSH (flush cache line) instruction writes and invalidates the cache line associated with a specified linear address. The invalidation is for all levels of the processor's cache hierarchy, and it is broadcast throughout the cache coherency domain.

NOTE

CLFLUSH was introduced with Intel SSE2. However, the instruction can be implemented in IA-32 processors that do not implement Intel SSE2. Detect CLFLUSH using the feature bit (if CPUID.01H:EDX.CLFLUSH[19] = 1).

11.4.4.2 Cacheability Control Instructions

The following four instructions enable data from XMM and general-purpose registers to be stored to memory using a non-temporal hint. The non-temporal hint directs the processor to store data to memory without writing the data into the cache hierarchy. See Section 10.4.6.2, "Caching of Temporal vs. Non-Temporal Data," for more information about non-temporal stores and hints.

The MOVNTDQ (store double quadword using non-temporal hint) instruction stores packed integer data from an XMM register to memory, using a non-temporal hint.

The MOVNTPD (store packed double precision floating-point values using non-temporal hint) instruction stores packed double precision floating-point data from an XMM register to memory, using a non-temporal hint.

The MOVNTI (store doubleword using non-temporal hint) instruction stores integer data from a general-purpose register to memory, using a non-temporal hint.

The MASKMOVDQU (store selected bytes of double quadword) instruction stores selected byte integers from an XMM register to memory, using a byte mask to selectively write the individual bytes. The memory location does not need to be aligned on a natural boundary. This instruction also uses a non-temporal hint.

11.4.4.3 Memory Ordering Instructions

Intel SSE2 introduced two fence instructions (LFENCE and MFENCE) as companions to the SFENCE instruction introduced with Intel SSE.

The LFENCE instruction establishes a memory fence for loads. It guarantees ordering between two loads and prevents speculative loads from passing the load fence (that is, no speculative loads are allowed until all loads specified before the load fence have been carried out).

The MFENCE instruction establishes a memory fence for both loads and stores. The processor ensures that no load or store after MFENCE will become globally visible until all loads and stores before MFENCE are globally visible.[1] Note that the sequences LFENCE;SFENCE and SFENCE;LFENCE are not equivalent to MFENCE because neither ensures that older stores are globally observed prior to younger loads.

11.4.4.4 Pause

The PAUSE instruction is provided to improve the performance of "spin-wait loops" executed on a Pentium 4 or Intel Xeon processor. On a Pentium 4 processor, it also provides the added benefit of reducing processor power

1. A load is considered to become globally visible when the value to be loaded is determined.

consumption while executing a spin-wait loop. It is recommended that a PAUSE instruction always be included in the code sequence for a spin-wait loop.

11.4.5 Branch Hints

Intel SSE2 designates two instruction prefixes (2EH and 3EH) to provide branch hints to the processor (see "Instruction Prefixes" in Chapter 2 of the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 2A). These prefixes can only be used with the Jcc instruction and only at the machine code level (that is, there are no mnemonics for the branch hints).

11.5 INTEL® SSE, SSE2, AND SSE3 EXCEPTIONS

Intel SSE, SSE2, and SSE3 instructions generate two general types of exceptions:

- Non-numeric exceptions.
- SIMD floating-point exceptions.[1]

Intel SSE, SSE2, and SSE3 instructions can generate the same type of memory-access and non-numeric exceptions as other IA-32 architecture instructions. Existing exception handlers can generally handle these exceptions without any code modification. See "Providing Non-Numeric Exception Handlers for Exceptions Generated by the SSE, SSE2, and SSE3 Instructions" in Chapter 16 of the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3A, for a list of the non-numeric exceptions that can be generated by the Intel SSE, SSE2, SSE3 instructions and for guidelines for handling these exceptions.

Intel SSE, SSE2, and SSE3 instructions do not generate numeric exceptions on packed integer operations; however, they can generate numeric (SIMD floating-point) exceptions on packed single precision and double precision floating-point operations. These SIMD floating-point exceptions are defined in the IEEE Standard 754 for Floating-Point Arithmetic and are the same exceptions that are generated for x87 FPU instructions. See Section 11.5.1, "SIMD Floating-Point Exceptions," for a description of these exceptions.

11.5.1 SIMD Floating-Point Exceptions

SIMD floating-point exceptions are those exceptions that can be generated by Intel SSE, SSE2, and SSE3 instructions that operate on packed or scalar floating-point operands.

Six classes of SIMD floating-point exceptions can be generated:

- Invalid operation (#I).
- Divide-by-zero (#Z).
- Denormal operand (#D).
- Numeric overflow (#O).
- Numeric underflow (#U).
- Inexact result (Precision) (#P).

All of these exceptions (except the denormal operand exception) are defined in IEEE Standard 754, and they are the same exceptions that are generated with the x87 floating-point instructions. Section 4.9, "Overview of Floating-Point Exceptions," gives a detailed description of these exceptions and of how and when they are generated. The following sections discuss the implementation of these exceptions in the Intel SSE/SSE2/SSE3 extensions.

All SIMD floating-point exceptions are precise and occur as soon as the instruction completes execution.

1. The FISTTP instruction in Intel SSE3 does not generate SIMD floating-point exceptions, but it can generate x87 FPU floating-point exceptions.

Each of the six exception conditions has a corresponding flag (IE, DE, ZE, OE, UE, and PE) and mask bit (IM, DM, ZM, OM, UM, and PM) in the MXCSR register (see Figure 10-3). The mask bits can be set with the LDMXCSR or FXRSTOR instruction; the mask and flag bits can be read with the STMXCSR or FXSAVE instruction.

The OSXMMEXCEPT flag (bit 10) of control register CR4 provides additional control over generation of SIMD floating-point exceptions by allowing the operating system to indicate whether or not it supports software exception handlers for SIMD floating-point exceptions. If an unmasked SIMD floating-point exception is generated and the OSXMMEXCEPT flag is set, the processor invokes a software exception handler by generating a SIMD floating-point exception (#XM). If the OSXMMEXCEPT bit is clear, the processor generates an invalid-opcode exception (#UD) on the first Intel SSE or SSE2 instruction that detects a SIMD floating-point exception condition. See Section 11.6.2, "Checking for Intel® SSE and SSE2 Support."

11.5.2 SIMD Floating-Point Exception Conditions

The following sections describe the conditions that cause a SIMD floating-point exception to be generated and the masked response of the processor when these conditions are detected.

See Section 4.9.2, "Floating-Point Exception Priority," for a description of the rules for exception precedence when more than one floating-point exception condition is detected for an instruction.

11.5.2.1 Invalid Operation Exception (#I)

The floating-point invalid-operation exception (#I) occurs in response to an invalid arithmetic operand. The flag (IE) and mask (IM) bits for the invalid operation exception are bits 0 and 7, respectively, in the MXCSR register.

If the invalid-operation exception is masked, the processor returns a QNaN, QNaN floating-point indefinite, integer indefinite, one of the source operands to the destination operand, or it sets the EFLAGS, depending on the operation being performed. When a value is returned to the destination operand, it overwrites the destination register specified by the instruction. Table 11-1 lists the invalid-arithmetic operations that the processor detects for instructions and the masked responses to these operations.

Table 11-1. Masked Responses of Intel® SSE, SSE2, and SSE3 Instructions to Invalid Arithmetic Operations

Condition	Masked Response
ADDPS, ADDSS, ADDPD, ADDSD, SUBPS, SUBSS, SUBPD, SUBSD, MULPS, MULSS, MULPD, MULSD, DIVPS, DIVSS, DIVPD, DIVSD, ADDSUBPD, ADDSUBPD, HADDPD, HADDPS, HSUBPD or HSUBPS instruction with an SNaN operand	Return the SNaN converted to a QNaN; Refer to Table 4-8 for more details.
SQRTPS, SQRTSS, SQRTPD, or SQRTSD with SNaN operands	Return the SNaN converted to a QNaN.
SQRTPS, SQRTSS, SQRTPD, or SQRTSD with negative operands (except zero)	Return the QNaN floating-point Indefinite.
MAXPS, MAXSS, MAXPD, MAXSD, MINPS, MINSS, MINPD, or MINSD instruction with QNaN or SNaN operands	Return the source 2 operand value.
CMPPS, CMPSS, CMPPD or CMPSD instruction with QNaN or SNaN operands	Return a mask of all 0s (except for the predicates "not-equal," "unordered," "not-less-than," or "not-less-than-or-equal," which returns a mask of all 1s).
CVTPD2PS, CVTSD2SS, CVTPS2PD, CVTSS2SD with SNaN operands	Return the SNaN converted to a QNaN.
COMISS or COMISD with QNaN or SNaN operand(s)	Set EFLAGS values to "not comparable."
Addition of opposite signed infinities or subtraction of like-signed infinities	Return the QNaN floating-point Indefinite.
Multiplication of infinity by zero	Return the QNaN floating-point Indefinite.
Divide of (0/0) or (∞ / ∞)	Return the QNaN floating-point Indefinite.

Table 11-1. Masked Responses of Intel® SSE, SSE2, and SSE3 Instructions to Invalid Arithmetic Operations (Contd.)

Condition	Masked Response
Conversion to integer when the value in the source register is a NaN, ∞, or exceeds the representable range for CVTPS2PI, CVTTPS2PI, CVTSS2SI, CVTTSS2SI, CVTPD2PI, CVTSD2SI, CVTPD2DQ, CVTTPD2PI, CVTTSD2SI, CVTTPD2DQ, CVTPS2DQ, or CVTTPS2DQ	Return the integer Indefinite.

If the invalid operation exception is not masked, a software exception handler is invoked and the operands remain unchanged. See Section 11.5.4, "Handling SIMD Floating-Point Exceptions in Software."

Normally, when one or more of the source operands are QNaNs (and neither is an SNaN or in an unsupported format), an invalid-operation exception is not generated. The following instructions are exceptions to this rule: the COMISS and COMISD instructions; and the CMPPS, CMPSS, CMPPD, and CMPSD instructions (when the predicate is less than, less-than or equal, not less-than, or not less-than or equal). With these instructions, a QNaN source operand will generate an invalid-operation exception.

The invalid-operation exception is not affected by the flush-to-zero mode or by the denormals-are-zeros mode.

11.5.2.2 Denormal-Operand Exception (#D)

The processor signals the denormal-operand exception if an arithmetic instruction attempts to operate on a denormal operand. The flag (DE) and mask (DM) bits for the denormal-operand exception are bits 1 and 8, respectively, in the MXCSR register.

The CVTPI2PD, CVTPD2PI, CVTTPD2PI, CVTDQ2PD, CVTPD2DQ, CVTTPD2DQ, CVTSI2SD, CVTSD2SI, CVTTSD2SI, CVTPI2PS, CVTPS2PI, CVTTPS2PI, CVTSS2SI, CVTTSS2SI, CVTSI2SS, CVTDQ2PS, CVTPS2DQ, and CVTTPS2DQ conversion instructions do not signal denormal exceptions. The RCPSS, RCPPS, RSQRTSS, and RSQRTPS instructions do not signal any kind of floating-point exception.

The denormals-are-zero flag (bit 6) of the MXCSR register provides an additional option for handling denormal-operand exceptions. When this flag is set, denormal source operands are automatically converted to zeros with the sign of the source operand (see Section 10.2.3.4, "Denormals-Are-Zeros"). The denormal operand exception is not affected by the flush-to-zero mode.

See Section 4.9.1.2, "Denormal Operand Exception (#D)," for more information about the denormal exception. See Section 11.5.4, "Handling SIMD Floating-Point Exceptions in Software," for information on handling unmasked exceptions.

11.5.2.3 Divide-By-Zero Exception (#Z)

The processor reports a divide-by-zero exception when a DIVPS, DIVSS, DIVPD or DIVSD instruction attempts to divide a finite non-zero operand by 0. The flag (ZE) and mask (ZM) bits for the divide-by-zero exception are bits 2 and 9, respectively, in the MXCSR register.

See Section 4.9.1.3, "Divide-By-Zero Exception (#Z)," for more information about the divide-by-zero exception. See Section 11.5.4, "Handling SIMD Floating-Point Exceptions in Software," for information on handling unmasked exceptions.

The divide-by-zero exception is not affected by the flush-to-zero mode at a single-instruction boundary.

While DAZ does not affect the rules for signaling IEEE exceptions, operations on denormal inputs might have different results when DAZ=1. As a consequence, DAZ can have an effect on the floating-point exceptions - including the divide-by-zero exception - when observed for a given operation involving denormal inputs.

11.5.2.4 Numeric Overflow Exception (#O)

The processor reports a numeric overflow exception whenever the rounded result of an arithmetic instruction exceeds the largest allowable finite value that fits in the destination operand. This exception can be generated with the ADDPS, ADDSS, ADDPD, ADDSD, SUBPS, SUBSS, SUBPD, SUBSD, MULPS, MULSS, MULPD, MULSD, DIVPS, DIVSS, DIVPD, DIVSD, CVTPD2PS, CVTSD2SS, ADDSUBPD, ADDSUBPS, HADDPD, HADDPS, HSUBPD, and

HSUBPS instructions. The flag (OE) and mask (OM) bits for the numeric overflow exception are bits 3 and 10, respectively, in the MXCSR register.

See Section 4.9.1.4, "Numeric Overflow Exception (#O)," for more information about the numeric-overflow exception. See Section 11.5.4, "Handling SIMD Floating-Point Exceptions in Software," for information on handling unmasked exceptions.

The numeric overflow exception is not affected by the flush-to-zero mode or by the denormals-are-zeros mode.

11.5.2.5 Numeric Underflow Exception (#U)

The processor reports a numeric underflow exception whenever the magnitude of the rounded result of an arithmetic instruction, with unbounded exponent, is less than the smallest possible normalized, finite value that will fit in the destination operand and the numeric-underflow exception is not masked. If the numeric underflow exception is masked, both underflow and the inexact-result condition must be detected before numeric underflow is reported. This exception can be generated with the ADDPS, ADDSS, ADDPD, ADDSD, SUBPS, SUBSS, SUBPD, SUBSD, MULPS, MULSS, MULPD, MULSD, DIVPS, DIVSS, DIVPD, DIVSD, CVTPD2PS, CVTSD2SS, ADDSUBPD, ADDSUBPS, HADDPD, HADDPS, HSUBPD, and HSUBPS instructions. The flag (UE) and mask (UM) bits for the numeric underflow exception are bits 4 and 11, respectively, in the MXCSR register.

The flush-to-zero flag (bit 15) of the MXCSR register provides an additional option for handling numeric underflow exceptions. When this flag is set and the numeric underflow exception is masked, tiny results are returned as a zero with the sign of the true result; see Section 10.2.3.3, "Flush-To-Zero."

Underflow will occur when a tiny non-zero result is detected (the result has to be also inexact if underflow exceptions are masked), as described in the IEEE Standard 754-2008. While DAZ does not affect the rules for signaling IEEE exceptions, operations on denormal inputs might have different results when DAZ=1. As a consequence, DAZ can have an effect on the floating-point exceptions - including the underflow exception - when observed for a given operation involving denormal inputs.

See Section 4.9.1.5, "Numeric Underflow Exception (#U)," for more information about the numeric underflow exception. See Section 11.5.4, "Handling SIMD Floating-Point Exceptions in Software," for information on handling unmasked exceptions.

11.5.2.6 Inexact-Result (Precision) Exception (#P)

The inexact-result exception (also called the precision exception) occurs if the result of an operation is not exactly representable in the destination format. For example, the fraction 1/3 cannot be precisely represented in binary form. This exception occurs frequently and indicates that some (normally acceptable) accuracy has been lost. The exception is supported for applications that need to perform exact arithmetic only. Because the rounded result is generally satisfactory for most applications, this exception is commonly masked.

The flag (PE) and mask (PM) bits for the inexact-result exception are bits 5 and 12, respectively, in the MXCSR register.

See Section 4.9.1.6, "Inexact-Result (Precision) Exception (#P)," for more information about the inexact-result exception. See Section 11.5.4, "Handling SIMD Floating-Point Exceptions in Software," for information on handling unmasked exceptions.

In flush-to-zero mode, the inexact result exception is reported.

11.5.3 Generating SIMD Floating-Point Exceptions

When the processor executes a packed or scalar floating-point instruction, it looks for and reports on SIMD floating-point exception conditions using two sequential steps:

1. Looks for, reports on, and handles pre-computation exception conditions (invalid-operand, divide-by-zero, and denormal operand)
2. Looks for, reports on, and handles post-computation exception conditions (numeric overflow, numeric underflow, and inexact result)

If both pre- and post-computational exceptions are unmasked, it is possible for the processor to generate a SIMD floating-point exception (#XM) twice during the execution of an SSE, SSE2 or SSE3 instruction: once when it detects and handles a pre-computational exception and when it detects a post-computational exception.

11.5.3.1 Handling Masked Exceptions

If all exceptions are masked, the processor handles the exceptions it detects by placing the masked result (or results for packed operands) in a destination operand and continuing program execution. The masked result may be a rounded normalized value, signed infinity, a denormal finite number, zero, a QNaN floating-point indefinite, or a QNaN depending on the exception condition detected. In most cases, the corresponding exception flag bit in MXCSR is also set. The one situation where an exception flag is not set is when an underflow condition is detected and it is not accompanied by an inexact result.

When operating on packed floating-point operands, the processor returns a masked result for each of the sub-operand computations and sets a separate set of internal exception flags for each computation. It then performs a logical-OR on the internal exception flag settings and sets the exception flags in the MXCSR register according to the results of OR operations.

For example, Figure 11-9 shows the results of an MULPS instruction. In the example, all SIMD floating-point exceptions are masked. Assume that a denormal exception condition is detected prior to the multiplication of sub-operands X0 and Y0, no exception condition is detected for the multiplication of X1 and Y1, a numeric overflow exception condition is detected for the multiplication of X2 and Y2, and another denormal exception is detected prior to the multiplication of sub-operands X3 and Y3. Because denormal exceptions are masked, the processor uses the denormal source values in the multiplications of (X0 and Y0) and of (X3 and Y3) passing the results of the multiplications through to the destination operand. With the denormal operand, the result of the X0 and Y0 computation is a normalized finite value, with no exceptions detected. However, the X3 and Y3 computation produces a tiny and inexact result. This causes the corresponding internal numeric underflow and inexact-result exception flags to be set.

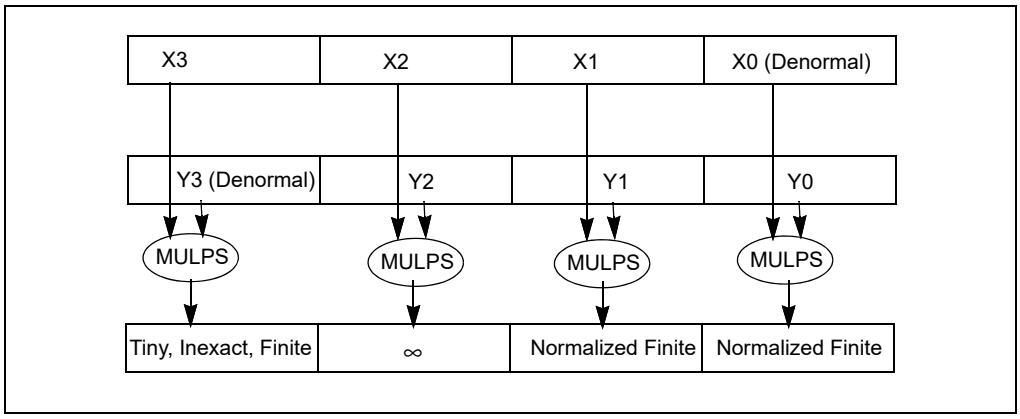


Figure 11-9. Example Masked Response for Packed Operations

For the multiplication of X2 and Y2, the processor stores the floating-point ∞ in the destination operand, and sets the corresponding internal sub-operand numeric overflow flag. The result of the X1 and Y1 multiplication is passed through to the destination operand, with no internal sub-operand exception flags being set. Following the computations, the individual sub-operand exceptions flags for denormal operand, numeric underflow, inexact result, and numeric overflow are OR'd and the corresponding flags are set in the MXCSR register.

The net result of this computation is that:

- Multiplication of X0 and Y0 produces a normalized finite result
- Multiplication of X1 and Y1 produces a normalized finite result
- Multiplication of X2 and Y2 produces a floating-point ∞ result
- Multiplication of X3 and Y3 produces a tiny, inexact, finite result

- Denormal operand, numeric underflow, numeric underflow, and inexact result flags are set in the MXCSR register

11.5.3.2 Handling Unmasked Exceptions

If all exceptions are unmasked, the processor:

1. First detects any pre-computation exceptions: it ORs those exceptions, sets the appropriate exception flags, leaves the source and destination operands unaltered, and goes to step 2. If it does not detect any pre-computation exceptions, it goes to step 5.
2. Checks CR4.OSXMMEXCPT[bit 10]. If this flag is set, the processor goes to step 3; if the flag is clear, it generates an invalid-opcode exception (#UD) and makes an implicit call to the invalid-opcode exception handler.
3. Generates a SIMD floating-point exception (#XM) and makes an implicit call to the SIMD floating-point exception handler.
4. If the exception handler is able to fix the source operands that generated the pre-computation exceptions or mask the condition in such a way as to allow the processor to continue executing the instruction, the processor resumes instruction execution as described in step 5.
5. Upon returning from the exception handler (or if no pre-computation exceptions were detected), the processor checks for post-computation exceptions. If the processor detects any post-computation exceptions: it ORs those exceptions, sets the appropriate exception flags, leaves the source and destination operands unaltered, and repeats steps 2, 3, and 4.
6. Upon returning from the exceptions handler in step 4 (or if no post-computation exceptions were detected), the processor completes the execution of the instruction.

The implication of this procedure is that for unmasked exceptions, the processor can generate a SIMD floating-point exception (#XM) twice: once if it detects pre-computation exception conditions and a second time if it detects post-computation exception conditions. For example, if SIMD floating-point exceptions are unmasked for the computation shown in Figure 11-9, the processor would generate one SIMD floating-point exception for denormal operand conditions and a second SIMD floating-point exception for overflow and underflow (no inexact result exception would be generated because the multiplications of X0 and Y0 and of X1 and Y1 are exact).

11.5.3.3 Handling Combinations of Masked and Unmasked Exceptions

In situations where both masked and unmasked exceptions are detected, the processor will set exception flags for the masked and the unmasked exceptions. However, it will not return masked results until after the processor has detected and handled unmasked post-computation exceptions and returned from the exception handler (as in step 6 above) to finish executing the instruction.

11.5.4 Handling SIMD Floating-Point Exceptions in Software

Section 4.9.3, "Typical Actions of a Floating-Point Exception Handler," shows actions that may be carried out by a SIMD floating-point exception handler. The SSE/SSE2/SSE3 state is saved with the FXSAVE instruction; see Section 11.6.5, "Saving and Restoring the SSE/SSE2 State."

11.5.5 Interaction of SIMD and x87 FPU Floating-Point Exceptions

SIMD floating-point exceptions are generated independently from x87 FPU floating-point exceptions. SIMD floating-point exceptions do not cause assertion of the FERR# pin (independent of the value of CR0.NE[bit 5]). They ignore the assertion and deassertion of the IGNNE# pin.

If applications use Intel SSE/SSE2/SSE3 instructions along with x87 FPU instructions (in the same task or program), consider the following:

- SIMD floating-point exceptions are reported independently from the x87 FPU floating-point exceptions. SIMD and x87 FPU floating-point exceptions can be unmasked independently. Separate x87 FPU and SIMD floating-

point exception handlers must be provided if the same exception is unmasked for x87 FPU and for Intel SSE/SSE2/SSE3 operations.

- The rounding mode specified in the MXCSR register does not affect x87 FPU instructions. Likewise, the rounding mode specified in the x87 FPU control word does not affect the Intel SSE/SSE2/SSE3 instructions. To use the same rounding mode, the rounding control bits in the MXCSR register and in the x87 FPU control word must be set explicitly to the same value.
- The flush-to-zero mode set in the MXCSR register for Intel SSE/SSE2/SSE3 instructions has no counterpart in the x87 FPU. For compatibility with the x87 FPU, set the flush-to-zero bit to 0.
- The denormals-are-zeros mode set in the MXCSR register for Intel SSE/SSE2/SSE3 instructions has no counterpart in the x87 FPU. For compatibility with the x87 FPU, set the denormals-are-zeros bit to 0.
- An application that expects to detect x87 FPU exceptions that occur during the execution of x87 FPU instructions will not be notified if exceptions occurs during the execution of corresponding Intel SSE/SSE2/SSE3[1] instructions, unless the exception masks that are enabled in the x87 FPU control word have also been enabled in the MXCSR register and the application is capable of handling SIMD floating-point exceptions (#XM).
  - — Masked exceptions that occur during an SSE/SSE2/SSE3 library call cannot be detected by unmasking the exceptions after the call (in an attempt to generate the fault based on the fact that an exception flag is set). A SIMD floating-point exception flag that is set when the corresponding exception is unmasked will not generate a fault; only the next occurrence of that unmasked exception will generate a fault.
  - — An application which checks the x87 FPU status word to determine if any masked exception flags were set during an x87 FPU library call will also need to check the MXCSR register to detect a similar occurrence of a masked exception flag being set during an SSE/SSE2/SSE3 library call.

11.6 WRITING APPLICATIONS WITH INTEL® SSE AND SSE2

The following sections give some guidelines for writing application programs and operating-system code that uses Intel SSE and SSE2. Because Intel SSE and SSE2 share the same state and perform companion operations, these guidelines apply to both sets of extensions.

Chapter 16 in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3A, discusses the interface to the processor for context switching as well as other operating system considerations when writing code that uses Intel SSE, SSE2, and SSE3.

11.6.1 General Guidelines for Using Intel® SSE and SSE2

The following guidelines describe how to take full advantage of the performance gains available with Intel SSE and SSE2:

- Ensure that the processor supports Intel SSE and SSE2.
- Ensure that your operating system supports Intel SSE and SSE2. (Operating system support for Intel SSE implies support for Intel SSE2, and vice versa.)
- Use stack and data alignment techniques to keep data properly aligned for efficient memory use.
- Use the non-temporal store instructions offered with Intel SSE and SSE2.
- Employ the optimization and scheduling techniques described in the Intel® 64 and IA-32 Architectures Optimization Reference Manual; see Section 1.4, "Related Literature," for the order number for this manual.

11.6.2 Checking for Intel® SSE and SSE2 Support

Before an application attempts to use Intel SSE and/or Intel SSE2, it should check that they are present on the processor:

1. Intel SSE3 refers to ADDSUBPD, ADDSUBPS, HADDPD, HADDPS, HSUBPD, and HSUBPS. The only other Intel SSE3 instruction that can raise floating-point exceptions is FISTTP; it can generate x87 FPU invalid operation and inexact result exceptions.

1. Check that the processor supports the CPUID instruction. Bit 21 of the EFLAGS register can be used to check processor's support the CPUID instruction.
2. Check that the processor supports Intel SSE and/or SSE2 (true if CPUID.01H:EDX.SSE[25] = 1 and/or CPUID.01H:EDX.SSE2[26] = 1).

The operating system must provide system level support for handling SSE state, exceptions before an application can use Intel SSE and/or Intel SSE2; see Chapter 16 in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3A.

If the processor attempts to execute an unsupported Intel SSE or SSE2 instruction, the processor will generate an invalid-opcode exception (#UD). If an operating system did not provide adequate system level support for Intel SSE, executing an Intel SSE or SSE2 instructions can also generate #UD.

11.6.3 Checking for the DAZ Flag in the MXCSR Register

The denormals-are-zero flag in the MXCSR register is available in most of the Pentium 4 processors and in the Intel Xeon processor, with the exception of some early steppings. To check for the presence of the DAZ flag in the MXCSR register, do the following:

1. Establish a 512-byte FXSAVE area in memory.
2. Clear the FXSAVE area to all 0s.
3. Execute the FXSAVE instruction, using the address of the first byte of the cleared FXSAVE area as a source operand. See "FXSAVE—Save x87 FPU, MMX, SSE, and SSE2 State" in Chapter 3 of the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 2A, for a description of the FXSAVE instruction and the layout of the FXSAVE image.
4. Check the value in the MXCSR_MASK field in the FXSAVE image (bytes 28 through 31).
   - If the value of the MXCSR_MASK field is 00000000H, the DAZ flag and denormals-are-zero mode are not supported.
   - If the value of the MXCSR_MASK field is non-zero and bit 6 is set, the DAZ flag and denormals-are-zero mode are supported.

If the DAZ flag is not supported, then it is a reserved bit and attempting to write a 1 to it will cause a general-protection exception (#GP). See Section 11.6.6, "Guidelines for Writing to the MXCSR Register," for general guidelines for preventing general-protection exceptions when writing to the MXCSR register.

11.6.4 Initialization of Intel® SSE and SSE2

The SSE and SSE2 state is contained in the XMM and MXCSR registers. Upon a hardware reset of the processor, this state is initialized as follows (see Table 11-2):

- All SIMD floating-point exceptions are masked (bits 7 through 12 of the MXCSR register is set to 1).
- All SIMD floating-point exception flags are cleared (bits 0 through 5 of the MXCSR register is set to 0).
- The rounding control is set to round-nearest (bits 13 and 14 of the MXCSR register are set to 00B).
- The flush-to-zero mode is disabled (bit 15 of the MXCSR register is set to 0).
- The denormals-are-zeros mode is disabled (bit 6 of the MXCSR register is set to 0). If the denormals-are-zeros mode is not supported, this bit is reserved and will be set to 0 on initialization.
- Each of the XMM registers is cleared (set to all zeros).

Table 11-2. SSE and SSE2 State Following a Power-up/Reset or INIT

Registers	Power-Up or Reset	INIT
XMM0 through XMM7	+0.0	Unchanged
MXCSR	1F80H	Unchanged

If the processor is reset by asserting the INIT# pin, the SSE and SSE2 state is not changed.

11.6.5 Saving and Restoring the SSE/SSE2 State

The FXSAVE instruction saves the x87 FPU, MMX, SSE, and SSE2 states (which includes the contents of eight XMM registers and the MXCSR registers) in a 512-byte block of memory. The FXRSTOR instruction restores the saved SSE and SSE2 state from memory. See the FXSAVE instruction in Chapter 3 of the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 2A, for the layout of the 512-byte state block.

In addition to saving and restoring the SSE and SSE2 state, FXSAVE and FXRSTOR also save and restore the x87 FPU state (because MMX registers are aliased to the x87 FPU data registers this includes saving and restoring the MMX state). For greater code efficiency, it is suggested that FXSAVE and FXRSTOR be substituted for the FSAVE, FNSAVE, and FRSTOR instructions in the following situations:

- When a context switch is being made in a multitasking environment
- During calls and returns from interrupt and exception handlers

In situations where the code is switching between x87 FPU and MMX technology computations (without a context switch or a call to an interrupt or exception), the FSAVE/FNSAVE and FRSTOR instructions are more efficient than the FXSAVE and FXRSTOR instructions.

11.6.6 Guidelines for Writing to the MXCSR Register

The MXCSR has several reserved bits, and attempting to write a 1 to any of these bits will cause a general-protection exception (#GP) to be generated. To allow software to identify these reserved bits, the MXCSR_MASK value is provided. Software can determine this mask value as follows:

1. Establish a 512-byte FXSAVE area in memory.
2. Clear the FXSAVE area to all 0s.
3. Execute the FXSAVE instruction, using the address of the first byte of the cleared FXSAVE area as a source operand. See "FXSAVE—Save x87 FPU, MMX, SSE, and SSE2 State" in Chapter 3 of the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 2A, for a description of FXSAVE and the layout of the FXSAVE image.
4. Check the value in the MXCSR_MASK field in the FXSAVE image (bytes 28 through 31).
   - If the value of the MXCSR_MASK field is 00000000H, then the MXCSR_MASK value is the default value of 0000FFBFH. Note that this value indicates that bit 6 of the MXCSR register is reserved; this setting indicates that the denormals-are-zero mode is not supported on the processor.
   - If the value of the MXCSR_MASK field is non-zero, the MXCSR_MASK value should be used as the MXCSR_MASK.

All bits set to 0 in the MXCSR_MASK value indicate reserved bits in the MXCSR register. Thus, if the MXCSR_MASK value is AND'd with a value to be written into the MXCSR register, the resulting value will be assured of having all its reserved bits set to 0, preventing the possibility of a general-protection exception being generated when the value is written to the MXCSR register.

For example, the default MXCSR_MASK value when 00000000H is returned in the FXSAVE image is 0000FFBFH. If software AND's a value to be written to MXCSR register with 0000FFBFH, bit 6 of the result (the DAZ flag) will be ensured of being set to 0, which is the required setting to prevent general-protection exceptions on processors that do not support the denormals-are-zero mode.

To prevent general-protection exceptions, the MXCSR_MASK value should be AND'd with the value to be written into the MXCSR register in the following situations:

- Operating system routines that receive a parameter from an application program and then write that value to the MXCSR register (either with an FXRSTOR or LDMXCSR instruction)
- Any application program that writes to the MXCSR register and that needs to run robustly on several different IA-32 processors

Note that all bits in the MXCSR_MASK value that are set to 1 indicate features that are supported by the MXCSR register; they can be treated as feature flags for identifying processor capabilities.

11.6.7 Interaction of Intel® SSE and SSE2 Instructions with x87 FPU and MMX Instructions

The XMM registers and the x87 FPU and MMX registers represent separate execution environments, which has certain ramifications when executing Intel SSE, SSE2, MMX, and x87 FPU instructions in the same code module or when mixing code modules that contain these instructions:

- Those Intel SSE and SSE2 instructions that operate only on XMM registers (such as the packed and scalar floating-point instructions and the 128-bit SIMD integer instructions) in the same instruction stream with 64-bit SIMD integer or x87 FPU instructions without any restrictions. For example, an application can perform the majority of its floating-point computations in the XMM registers, using the packed and scalar floating-point instructions, and at the same time use the x87 FPU to perform trigonometric and other transcendental computations. Likewise, an application can perform packed 64-bit and 128-bit SIMD integer operations together without restrictions.
- Those Intel SSE and SSE2 instructions that operate on MMX registers (such as the CVTPS2PI, CVTTPS2PI, CVTPI2PS, CVTPD2PI, CVTTPD2PI, CVTPI2PD, MOVDQ2Q, MOVQ2DQ, PADDQ, and PSUBQ instructions) can also be executed in the same instruction stream as 64-bit SIMD integer or x87 FPU instructions, however, here they are subject to the restrictions on the simultaneous use of MMX technology and x87 FPU instructions, which include:
  - Transition from x87 FPU to MMX technology instructions or to Intel SSE or SSE2 instructions that operate on MMX registers should be preceded by saving the state of the x87 FPU.
  - Transition from MMX technology instructions or from Intel SSE or SSE2 instructions that operate on MMX registers to x87 FPU instructions should be preceded by execution of the EMMS instruction.

11.6.8 Compatibility of SIMD and x87 FPU Floating-Point Data Types

Intel SSE and SSE2 instructions operate on the same single precision and double precision floating-point data types that the x87 FPU operates on. However, when operating on these data types, Intel SSE and SSE2 operate on them in their native format (single precision or double precision), in contrast to the x87 FPU which extends them to double extended precision floating-point format to perform computations and then rounds the result back to a single precision or double precision format before writing results to memory. Because the x87 FPU operates on a higher precision format and then rounds the result to a lower precision format, it may return a slightly different result when performing the same operation on the same single precision or double precision floating-point values than is returned by Intel SSE and SSE2. The difference occurs only in the least-significant bits of the significand.

11.6.9 Mixing Packed and Scalar Floating-Point and 128-Bit SIMD Integer Instructions and Data

Intel SSE and SSE2 define typed operations on packed and scalar floating-point data types and on 128-bit SIMD integer data types, but IA-32 processors do not enforce this typing at the architectural level. They only enforce it at the microarchitectural level. Therefore, when a Pentium 4 or Intel Xeon processor loads a packed or scalar floating-point operand or a 128-bit packed integer operand from memory into an XMM register, it does not check that the actual data being loaded matches the data type specified in the instruction. Likewise, when the processor performs an arithmetic operation on the data in an XMM register, it does not check that the data being operated on matches the data type specified in the instruction.

As a general rule, because data typing of SIMD floating-point and integer data types is not enforced at the architectural level, it is the responsibility of the programmer, assembler, or compiler to ensure that code enforces data typing. Failure to enforce correct data typing can lead to computations that return unexpected results.

For example, in the following code sample, two packed single precision floating-point operands are moved from memory into XMM registers (using MOVAPS instructions); then a double precision packed add operation (using the ADDPD instruction) is performed on the operands:

```
movaps      xmm0, [eax]   ; EAX register contains pointer to packed
                          ; single precision floating-point operand
movaps      xmm1, [ebx]
addpd       xmm0, xmm1
```

Pentium 4 and Intel Xeon processors execute these instructions without generating an invalid-operand exception (#UD) and will produce the expected results in register XMM0 (that is, the high and low 64-bits of each register will be treated as a double precision floating-point value and the processor will operate on them accordingly). Because the data types operated on and the data type expected by the ADDPD instruction were inconsistent, the instruction may result in a SIMD floating-point exception (such as numeric overflow [#O] or invalid operation [#I]) being generated, but the actual source of the problem (inconsistent data types) is not detected.

The ability to operate on an operand that contains a data type that is inconsistent with the typing of the instruction being executed, permits some valid operations to be performed. For example, the following instructions load a packed double precision floating-point operand from memory to register XMM0, and a mask to register XMM1; then they use XORPD to toggle the sign bits of the two packed values in register XMM0.

```
movapd      xmm0, [eax]   ; EAX register contains pointer to packed
                          ; double precision floating-point operand
movaps      xmm1, [ebx]    ; EBX register contains pointer to packed
                           ; double precision floating-point mask
xorpd       xmm0, xmm1    ; XOR operation toggles sign bits using
                          ; the mask in xmm1
```

In this example: XORPS or PXOR can be used in place of XORPD and yield the same correct result. However, because of the type mismatch between the operand data type and the instruction data type, a latency penalty will be incurred due to implementations of the instructions at the microarchitecture level.

Latency penalties can also be incurred by using move instructions of the wrong type. For example, MOVAPS and MOVAPD can both be used to move a packed single precision operand from memory to an XMM register. However, if MOVAPD is used, a latency penalty will be incurred when a correctly typed instruction attempts to use the data in the register.

Note that these latency penalties are not incurred when moving data from XMM registers to memory.

11.6.10 Interfacing with Intel® SSE and SSE2 Procedures and Functions

Intel SSE and SSE2 allow direct access to XMM registers. This means that all existing interface conventions between procedures and functions that apply to the use of the general-purpose registers (EAX, EBX, etc.) also apply to XMM register usage.

11.6.10.1 Passing Parameters in XMM Registers

The state of XMM registers is preserved across procedure (or function) boundaries. Parameters can be passed from one procedure to another using XMM registers.

11.6.10.2 Saving XMM Register State on a Procedure or Function Call

The state of XMM registers can be saved in two ways: using an FXSAVE instruction or a move instruction. FXSAVE saves the state of all XMM registers (along with the state of MXCSR and the x87 FPU registers). This instruction is typically used for major changes in the context of the execution environment, such as a task switch. FXRSTOR restores the XMM, MXCSR, and x87 FPU registers stored with FXSAVE.

In cases where only XMM registers must be saved, or where selected XMM registers need to be saved, move instructions (MOVAPS, MOVUPS, MOVSS, MOVAPD, MOVUPD, MOVSD, MOVDQA, and MOVDQU) can be used. These instructions can also be used to restore the contents of XMM registers. To avoid performance degradation when saving XMM registers to memory or when loading XMM registers from memory, be sure to use the appropriately typed move instructions.

The move instructions can also be used to save the contents of XMM registers on the stack. Here, the stack pointer (in the ESP register) can be used as the memory address to the next available byte in the stack. Note that the stack pointer is not automatically incremented when using a move instruction (as it is with PUSH).

A move-instruction procedure that saves the contents of an XMM register to the stack is responsible for decrementing the value in the ESP register by 16. Likewise, a move-instruction procedure that loads an XMM register from the stack needs also to increment the ESP register by 16. To avoid performance degradation when moving the contents of XMM registers, use the appropriately typed move instructions.

Use the LDMXCSR and STMXCSR instructions to save and restore, respectively, the contents of the MXCSR register on a procedure call and return.

11.6.10.3 Caller-Save Recommendation for Procedure and Function Calls

When making procedure (or function) calls from SSE or SSE2 code, a caller-save convention is recommended for saving the state of the calling procedure. Using this convention, any register whose content must survive intact across a procedure call must be stored in memory by the calling procedure prior to executing the call.

The primary reason for using the caller-save convention is to prevent performance degradation. XMM registers can contain packed or scalar double precision floating-point, packed single precision floating-point, and 128-bit packed integer data types. The called procedure has no way of knowing the data types in XMM registers following a call; so it is unlikely to use the correctly typed move instruction to store the contents of XMM registers in memory or to restore the contents of XMM registers from memory.

As described in Section 11.6.9, "Mixing Packed and Scalar Floating-Point and 128-Bit SIMD Integer Instructions and Data," executing a move instruction that does not match the type for the data being moved to/from XMM registers will be carried out correctly, but can lead to a greater instruction latency.

11.6.11 Updating Existing MMX Technology Routines Using 128-Bit SIMD Integer Instructions

Intel SSE2 extends all 64-bit MMX SIMD integer instructions to operate on 128-bit SIMD integers using XMM registers. The extended 128-bit SIMD integer instructions operate like the 64-bit SIMD integer instructions; this simplifies the porting of MMX technology applications. However, there are considerations:

- To take advantage of wider 128-bit SIMD integer instructions, MMX technology code must be recompiled to reference the XMM registers instead of MMX registers.
- Computation instructions that reference memory operands that are not aligned on 16-byte boundaries should be replaced with an unaligned 128-bit load (MOVUDQ instruction) followed by a version of the same computation operation that uses register instead of memory operands. Use of 128-bit packed integer computation instructions with memory operands that are not 16-byte aligned results in a general protection exception (#GP).
- Extension of the PSHUFW instruction (shuffle word across 64-bit integer operand) across a full 128-bit operand is emulated by a combination of the following instructions: PSHUFHW, PSHUFLW, and PSHUFD.
- Use of the 64-bit shift by bit instructions (PSRLQ, PSLLQ) can be extended to 128 bits in either of two ways:
  - Use of PSRLQ and PSLLQ, along with masking logic operations.
  - Rewriting the code sequence to use PSRLDQ and PSLLDQ (shift double quadword operand by bytes)
- Loop counters need to be updated, since each 128-bit SIMD integer instruction operates on twice the amount of data as its 64-bit SIMD integer counterpart.

11.6.12 Branching on Arithmetic Operations

There are no condition codes in SSE or SSE2 states. A packed-data comparison instruction generates a mask which can then be transferred to an integer register. The following code sequence provides an example of how to perform a conditional branch, based on the result of an Intel SSE2 arithmetic operation.

```
cmppd       XMM0, XMM1          ; generates a mask in XMM0
movmskpd    EAX, XMM0           ; moves a 2 bit mask to eax
test        EAX, 0              ; compare with desired result
jne         BRANCH TARGET
```

The COMISD and UCOMISD instructions update the EFLAGS as the result of a scalar comparison. A conditional branch can then be scheduled immediately following COMISD/UCOMISD.

11.6.13 Cacheability Hint Instructions

Intel SSE and SSE2 cacheability control instructions enable the programmer to control prefetching, caching, loading, and storing of data. When correctly used, these instructions improve application performance.

To make efficient use of the processor's super-scalar microarchitecture, a program needs to provide a steady stream of data to the executing program to avoid stalling the processor. PREFETCH*h* instructions minimize the latency of data accesses in performance-critical sections of application code by allowing data to be fetched into the processor cache hierarchy in advance of actual usage.

PREFETCH*h* instructions do not change the user-visible semantics of a program, although they may affect performance. The operation of these instructions is implementation-dependent. Programmers may need to tune code for each IA-32 processor implementation. Excessive usage of PREFETCH*h* instructions may waste memory bandwidth and reduce performance. For more detailed information on the use of prefetch hints, refer to Chapter 7, "Optimizing Cache Usage," in the Intel® 64 and IA-32 Architectures Optimization Reference Manual.

The non-temporal store instructions (MOVNTI, MOVNTPD, MOVNTPS, MOVNTDQ, MOVNTQ, MASKMOVQ, and MASKMOVDQU) minimize cache pollution when writing non-temporal data to memory (see Section 10.4.6.1, "Cacheability Control Instructions," and Section 10.4.6.2, "Caching of Temporal vs. Non-Temporal Data"). They prevent non-temporal data from being written into processor caches on a store operation.

Besides reducing cache pollution, the use of weakly-ordered memory types can be important under certain data sharing relationships, such as a producer-consumer relationship. The use of weakly ordered memory can make the assembling of data more efficient; but care must be taken to ensure that the consumer obtains the data that the producer intended. Some common usage models that may be affected in this way by weakly-ordered stores are:

- Library functions that use weakly ordered memory to write results.
- Compiler-generated code that writes weakly-ordered results.
- Hand-crafted code.

The degree to which a consumer of data knows that the data is weakly ordered can vary for these cases. As a result, the SFENCE or MFENCE instruction should be used to ensure ordering between routines that produce weakly-ordered data and routines that consume the data. SFENCE and MFENCE provide a performance-efficient way to ensure ordering by guaranteeing that every store instruction that precedes SFENCE/MFENCE in program order is globally visible before a store instruction that follows the fence.

11.6.14 Effect of Instruction Prefixes on Intel® SSE and SSE2 Instructions

Table 11-3 describes the effects of instruction prefixes on Intel SSE and SSE2 instructions. (Table 11-3 also applies to SIMD integer and SIMD floating-point instructions in Intel SSE3.) Unpredictable behavior can range from prefixes being treated as a reserved operation on one generation of IA-32 processors to generating an invalid opcode exception on another generation of processors.

See also "Instruction Prefixes" in Chapter 2 of the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 2A, for complete description of instruction prefixes.

NOTE

Some Intel SSE, SSE2, and SSE3 instructions have two-byte opcodes that are either 2 bytes or 3 bytes in length. Two-byte opcodes that are 3 bytes in length consist of: a mandatory prefix (F2H, F3H, or 66H), 0FH, and an opcode byte. See Table 11-3.

Table 11-3. Effect of Prefixes on the Intel® SSE, SSE2, and SSE3 Instructions

Prefix Type	Effect on the Intel® SSE, SSE2, and SSE3 Instructions
Address Size Prefix (67H)	Affects instructions with a memory operand.
	Reserved for instructions without a memory operand and may result in unpredictable behavior.
Operand Size (66H)	Reserved and may result in unpredictable behavior.
Segment Override (2EH, 36H, 3EH, 26H, 64H, 65H)	Affects instructions with a memory operand.
	Reserved for instructions without a memory operand and may result in unpredictable behavior.
Repeat Prefixes (F2H and F3H)	Reserved and may result in unpredictable behavior.
Lock Prefix (F0H)	Reserved; generates invalid opcode exception (#UD).

CHAPTER 12
PROGRAMMING WITH INTEL® SSE3, SSSE3, INTEL® SSE4, AND INTEL® AES-NI

This chapter describes the Intel SSE3, SSSE3, and Intel SSE4 instructions, and provides information to assist in writing application programs that use these extensions.

Intel AES-NI and PCLMLQDQ are instruction extensions targeted to accelerate high-speed block encryption and cryptographic processing. Section 12.13 covers these instructions and their relationship to the Advanced Encryption Standard (AES).

12.1 PROGRAMMING ENVIRONMENT AND DATA TYPES

The programming environment for using Intel SSE3, SSSE3, and Intel SSE4 is unchanged from those shown in Figure 3-1 and Figure 3-2. These extensions do not introduce new data types. XMM registers are used to operate on packed integer data, single precision floating-point data, or double precision floating-point data.

One Intel SSE3 instruction uses the x87 FPU for x87-style programming. There are two Intel SSE3 instructions that use the general registers for thread synchronization. The MXCSR register governs SIMD floating-point operations. Note, however, that the x87 FPU control word does not affect the Intel SSE3 instruction that is executed by the x87 FPU (FISTTP), other than by unmasking an invalid operand or inexact result exception.

Intel SSE4 instructions do not use MMX registers. The majority of Intel SSE4.2[1] and SSE4.1 instructions operate on XMM registers.

12.1.1 Intel® SSE3, SSSE3, and Intel® SSE4 in 64-Bit Mode and Compatibility Mode

In compatibility mode, Intel SSE3, SSSE3, and Intel SSE4 function like they do in protected mode. In 64-bit mode, eight additional XMM registers are accessible. Registers XMM8-XMM15 are accessed by using REX prefixes.

Memory operands are specified using the ModR/M, SIB encoding described in Section 3.7.5.

Some Intel SSE3, SSSE3, and Intel SSE4 instructions may be used to operate on general-purpose registers. Use the REX.W prefix to access 64-bit general-purpose registers. Note that if a REX prefix is used when it has no meaning, the prefix is ignored.

12.1.2 Compatibility of Intel® SSE3 and SSSE3 with MMX Technology, the x87 FPU Environment, and Intel® SSE and SSE2

Intel SSE3, SSSE3, and Intel SSE4 do not introduce any new state to the Intel 64 and IA-32 execution environments.

For SIMD and x87 programming, the FXSAVE and FXRSTOR instructions save and restore the architectural states of XMM, MXCSR, x87 FPU, and MMX registers. The MONITOR and MWAIT instructions use general purpose registers on input, they do not modify the content of those registers.

12.1.3 Horizontal and Asymmetric Processing

Many of the Intel SSE/SSE2/SSE3 and SSSE3 instructions accelerate SIMD data processing using a model referred to as vertical computation. Using this model, data flow is vertical between the data elements of the inputs and the output.

1. Although the presence of CRC32 support is enumerated by CPUID.01H:ECX.SSE4_2 = 1, CRC32 operates on general purpose registers.

Figure 12-1 illustrates the asymmetric processing of the Intel SSE3 instruction ADDSUBPD. Figure 12-2 illustrates the horizontal data movement of the Intel SSE3 instruction HADDPD.

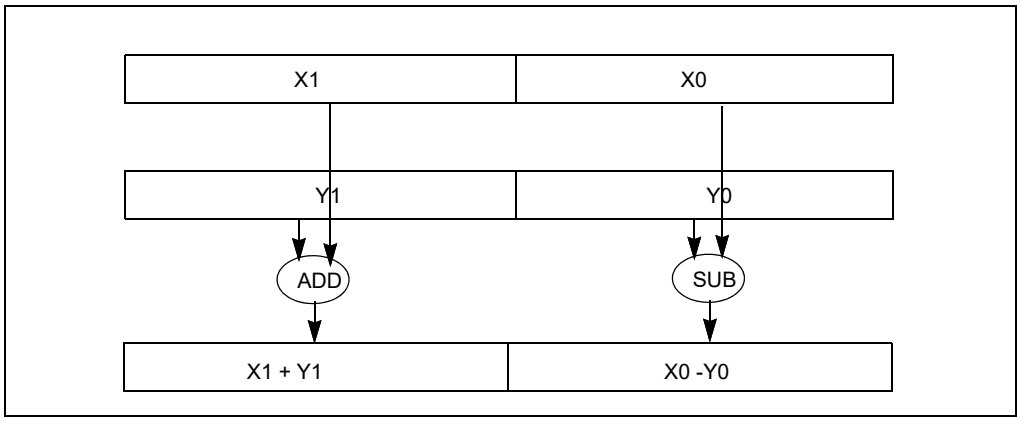


Figure 12-1. Asymmetric Processing in ADDSUBPD

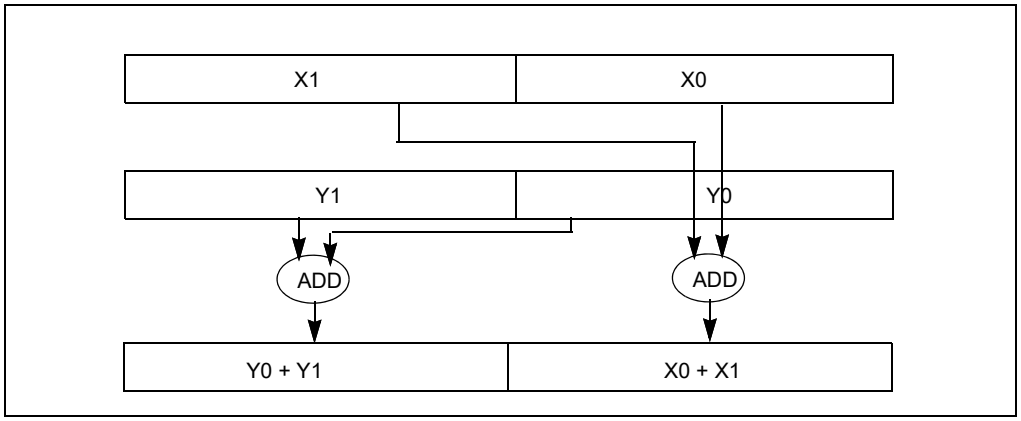


Figure 12-2. Horizontal Data Movement in HADDPD

12.2 OVERVIEW OF INTEL® SSE3 INSTRUCTIONS

Intel SSE3 extensions include 13 instructions. See:

- Section 12.3, "Intel® SSE3 Instructions," provides an introduction to individual Intel SSE3 instructions.
- The Intel® 64 and IA-32 Architectures Software Developer's Manual, Volumes 2A, 2B, 2C, & 2D, provides detailed information on individual instructions.
- Chapter 16, "System Programming for Instruction Set Extensions and Processor Extended States," in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3A, gives guidelines for integrating Intel SSE/SSE2/SSE3 extensions into an operating-system environment.

12.3 INTEL® SSE3 INSTRUCTIONS

Intel SSE3 instructions are grouped as follows:

- x87 FPU instruction:
  - — One instruction that improves x87 FPU floating-point to integer conversion.

- SIMD integer instruction:
  - One instruction that provides a specialized 128-bit unaligned data load.
- SIMD floating-point instructions:
  - Three instructions that enhance LOAD/MOVE/DUPLICATE performance.
  - Two instructions that provide packed addition/subtraction.
  - Four instructions that provide horizontal addition/subtraction.
- Thread synchronization instructions:
  - Two instructions that improve synchronization between multi-threaded agents.

The instructions are discussed in more detail in the following paragraphs.

12.3.1 x87 FPU Instruction for Integer Conversion

The FISTTP instruction (x87 FPU Store Integer and Pop with Truncation) behaves like FISTP, but uses truncation regardless of what rounding mode is specified in the x87 FPU control word. The instruction converts the top of stack (ST0) to integer with rounding to and pops the stack.

The FISTTP instruction is available in three precisions: short integer (word or 16-bit), integer (double word or 32-bit), and long integer (64-bit). With FISTTP, applications no longer need to change the FCW when truncation is required.

12.3.2 SIMD Integer Instruction for Specialized 128-Bit Unaligned Data Load

The LDDQU instruction is a special 128-bit unaligned load designed to avoid cache line splits. If the address of a 16-byte load is on a 16-byte boundary, LDQQU loads the bytes requested. If the address of the load is not aligned on a 16-byte boundary, LDDQU loads a 32-byte block starting at the 16-byte aligned address immediately below the load request. It then extracts the requested 16 bytes.

The instruction provides significant performance improvement on 128-bit unaligned memory accesses at the cost of some usage model restrictions.

12.3.3 SIMD Floating-Point Instructions That Enhance LOAD/MOVE/DUPLICATE Performance

The MOVSHDUP instruction loads/moves 128-bits, duplicating the second and fourth 32-bit data elements.

- MOVSHDUP OperandA, OperandB
  - OperandA (128 bits, four data elements): 3_a, 2_a, 1_a, 0_a
  - OperandB (128 bits, four data elements): 3_b, 2_b, 1_b, 0_b
  - Result (stored in OperandA): 3_b, 3_b, 1_b, 1_b

The MOVSLDUP instruction loads/moves 128-bits, duplicating the first and third 32-bit data elements.

- MOVSLDUP OperandA, OperandB
  - OperandA (128 bits, four data elements): 3_a, 2_a, 1_a, 0_a
  - OperandB (128 bits, four data elements): 3_b, 2_b, 1_b, 0_b
  - Result (stored in OperandA): 2_b, 2_b, 0_b, 0_b

The MOVDDUP instruction loads/moves 64-bits; duplicating the 64 bits from the source.

- MOVDDUP OperandA, OperandB
  - OperandA (128 bits, two data elements): 1_a, 0_a
  - OperandB (64 bits, one data element): 0_b
  - Result (stored in OperandA): 0_b, 0_b

12.3.4 SIMD Floating-Point Instructions Provide Packed Addition/Subtraction

The ADDSUBPS instruction has two 128-bit operands. The instruction performs single precision addition on the second and fourth pairs of 32-bit data elements within the operands; and single precision subtraction on the first and third pairs.

- ADDSUBPS OperandA, OperandB
  - — OperandA (128 bits, four data elements): 3_a, 2_a, 1_a, 0_a
  - — OperandB (128 bits, four data elements): 3_b, 2_b, 1_b, 0_b
  - — Result (stored in OperandA): 3_a+3_b, 2_a-2_b, 1_a+1_b, 0_a-0_b

The ADDSUBPD instruction has two 128-bit operands. The instruction performs double precision addition on the second pair of quadwords, and double precision subtraction on the first pair.

- ADDSUBPD OperandA, OperandB
  - — OperandA (128 bits, two data elements): 1_a, 0_a
  - — OperandB (128 bits, two data elements): 1_b, 0_b
  - — Result (stored in OperandA): 1_a+1_b, 0_a-0_b

12.3.5 SIMD Floating-Point Instructions Provide Horizontal Addition/Subtraction

Most SIMD instructions operate vertically. This means that the result in position i is a function of the elements in position i of both operands. Horizontal addition/subtraction operates horizontally. This means that contiguous data elements in the same source operand are used to produce a result.

The HADDPS instruction performs a single precision addition on contiguous data elements. The first data element of the result is obtained by adding the first and second elements of the first operand; the second element by adding the third and fourth elements of the first operand; the third by adding the first and second elements of the second operand; and the fourth by adding the third and fourth elements of the second operand.

- HADDPS OperandA, OperandB
  - — OperandA (128 bits, four data elements): 3_a, 2_a, 1_a, 0_a
  - — OperandB (128 bits, four data elements): 3_b, 2_b, 1_b, 0_b
  - — Result (Stored in OperandA): 3_b+2_b, 1_b+0_b, 3_a+2_a, 1_a+0_a

The HSUBPS instruction performs a single precision subtraction on contiguous data elements. The first data element of the result is obtained by subtracting the second element of the first operand from the first element of the first operand; the second element by subtracting the fourth element of the first operand from the third element of the first operand; the third by subtracting the second element of the second operand from the first element of the second operand; and the fourth by subtracting the fourth element of the second operand from the third element of the second operand.

- HSUBPS OperandA, OperandB
  - — OperandA (128 bits, four data elements): 3_a, 2_a, 1_a, 0_a
  - — OperandB (128 bits, four data elements): 3_b, 2_b, 1_b, 0_b
  - — Result (Stored in OperandA): 2_b-3_b, 0_b-1_b, 2_a-3_a, 0_a-1_a

The HADDPD instruction performs a double precision addition on contiguous data elements. The first data element of the result is obtained by adding the first and second elements of the first operand; the second element by adding the first and second elements of the second operand.

- HADDPD OperandA, OperandB
  - — OperandA (128 bits, two data elements): 1_a, 0_a
  - — OperandB (128 bits, two data elements): 1_b, 0_b
  - — Result (Stored in OperandA): 1_b+0_b, 1_a+0_a

The HSUBPD instruction performs a double precision subtraction on contiguous data elements. The first data element of the result is obtained by subtracting the second element of the first operand from the first element of the first operand; the second element by subtracting the second element of the second operand from the first element of the second operand.

- HSUBPD OperandA OperandB
  - OperandA (128 bits, two data elements): 1_a, 0_a
  - OperandB (128 bits, two data elements): 1_b, 0_b
  - Result (Stored in OperandA): 0_b-1_b, 0_a-1_a

12.3.6 Two Thread Synchronization Instructions

The MONITOR instruction sets up an address range that is used to monitor write-back-stores.

MWAIT enables a logical processor to enter into an optimized state while waiting for a write-back-store to the address range set up by MONITOR. MONITOR and MWAIT require the use of general purpose registers for its input. The registers used by MONITOR and MWAIT must be initialized properly; register content is not modified by these instructions.

12.4 WRITING APPLICATIONS WITH INTEL® SSE3

The following sections give guidelines for writing application programs and operating-system code that use Intel SSE3 instructions.

12.4.1 Guidelines for Using Intel® SSE3

The following guidelines describe how to maximize the benefits of using Intel SSE3:

- Check that the processor supports Intel SSE3.
  - Applications may need to ensure that the target operating system supports Intel SSE3. (Operating system support for the Intel SSE implies sufficient support for Intel SSE2 and SSE3.)
- Ensure your operating system supports MONITOR and MWAIT.
- Employ the optimization and scheduling techniques described in the Intel® 64 and IA-32 Architectures Optimization Reference Manual (see Section 1.4, “Related Literature”).

12.4.2 Checking for Intel® SSE3 Support

Before an application attempts to use the SIMD subset of Intel SSE3 instructions, the application should follow the steps illustrated in Section 11.6.2, “Checking for Intel® SSE and SSE2 Support.” Next, use the additional step provided below:

- Check that the processor supports the SIMD and x87 Intel SSE3 extensions (if CPUID.01H:ECX.SSE3[0] = 1).

An operating system that provides application support for Intel SSE and SSE2 also provides sufficient application support for Intel SSE3. To use FISTTP, software only needs to check support for Intel SSE3.

In the initial implementation of MONITOR and MWAIT, these two instructions are available to ring 0 and conditionally available at ring level greater than 0. Before an application attempts to use the MONITOR and MWAIT instructions, the application should use the following steps:

1. Check that the processor supports MONITOR and MWAIT. If CPUID.01H:ECX.MONITOR[3] = 1, MONITOR and MWAIT are available at ring 0.
2. Query the smallest and largest line size that MONITOR uses. Use CPUID.05H:EAX.SMALLEST[15:0];EBX.LARGEST[15:0]. Values are returned in bytes in EAX and EBX.
3. Ensure the memory address range(s) that will be supplied to MONITOR meets memory type requirements.

MONITOR and MWAIT are targeted for system software that supports efficient thread synchronization, see Chapter 16 in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3A for details.

12.4.3 Enable FTZ and DAZ for SIMD Floating-Point Computation

Enabling the FTZ and DAZ flags in the MXCSR register is likely to accelerate SIMD floating-point computation where strict compliance to the IEEE standard 754-1985 is not required. The FTZ flag is available to Intel 64 and IA-32 processors that support Intel SSE; DAZ is available to Intel 64 processors and to most IA-32 processors that support Intel SSE, SSE2, and SSE3.

Software can detect the presence of DAZ, modify the MXCSR register, and save and restore state information by following the techniques discussed in Section 11.6.3 through Section 11.6.6.

12.4.4 Programming Intel® SSE3 with Intel® SSE and SSE2

SIMD instructions in Intel SSE3 are intended to complement the use of Intel SSE and SSE2 in programming SIMD applications. Application software that intends to use Intel SSE3 instructions should also check for the availability of Intel SSE and SSE2 instructions.

The FISTTP instruction in Intel SSE3 is intended to accelerate x87 style programming where performance is limited by frequent floating-point conversion to integers; this happens when the x87 FPU control word is modified frequently. Use of the FISTTP instruction can eliminate the need to access the x87 FPU control word.

12.5 OVERVIEW OF SSSE3 INSTRUCTIONS

SSSE3 provides 32 instructions to accelerate a variety of multimedia and signal processing applications employing SIMD integer data. See:

- Section 12.6, "SSSE3 Instructions," provides an introduction to individual SSSE3 instructions.
- The Intel® 64 and IA-32 Architectures Software Developer's Manual, Volumes 2A, 2B, 2C, & 2D, provides detailed information on individual instructions.
- Chapter 16, "System Programming for Instruction Set Extensions and Processor Extended States," in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3A, gives guidelines for integrating SSSE3 and Intel SSE, SSE2, and SSE3 into an operating-system environment.

12.6 SSSE3 INSTRUCTIONS

SSSE3 instructions include:

- Twelve instructions that perform horizontal addition or subtraction operations.
- Six instructions that evaluate the absolute values.
- Two instructions that perform multiply and add operations and speed up the evaluation of dot products.
- Two instructions that accelerate packed-integer multiply operations and produce integer values with scaling.
- Two instructions that perform a byte-wise, in-place shuffle according to the second shuffle control operand.
- Six instructions that negate packed integers in the destination operand if the signs of the corresponding element in the source operand is less than zero.
- Two instructions that align data from the composite of two operands.

The operands of these instructions are packed integers of byte, word, or double word sizes. The operands are stored as 64 or 128 bit data in MMX registers, XMM registers, or memory.

The instructions are discussed in more detail in the following paragraphs.

12.6.1 Horizontal Addition/Subtraction

In analogy to the packed, floating-point horizontal add and subtract instructions in Intel SSE3, SSSE3 offers similar capabilities on packed integer data. Data elements of signed words, doublewords are supported. Saturated version for horizontal add and subtract on signed words are also supported. The horizontal data movement of PHADD is shown in Figure 12-3.

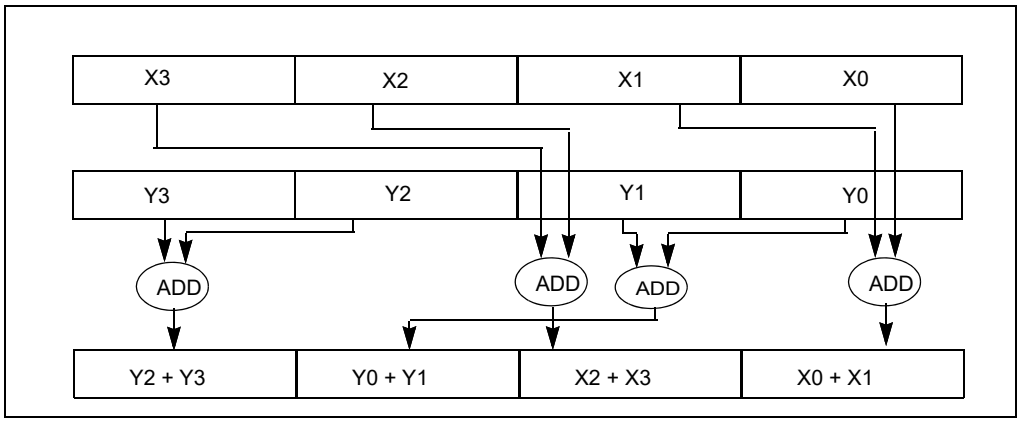


Figure 12-3. Horizontal Data Movement in PHADDD

There are six horizontal add instructions (represented by three mnemonics); three operate on 128-bit operands and three operate on 64-bit operands. The width of each data element is either 16 bits or 32 bits. The mnemonics are listed below.

- PHADDW adds two adjacent, signed 16-bit integers horizontally from the source and destination operands and packs the signed 16-bit results to the destination operand.
- PHADDSW adds two adjacent, signed 16-bit integers horizontally from the source and destination operands and packs the signed, saturated 16-bit results to the destination operand.
- PHADDD adds two adjacent, signed 32-bit integers horizontally from the source and destination operands and packs the signed 32-bit results to the destination operand.

There are six horizontal subtract instructions (represented by three mnemonics); three operate on 128-bit operands and three operate on 64-bit operands. The width of each data element is either 16 bits or 32 bits. These are listed below.

- PHSUBW performs horizontal subtraction on each adjacent pair of 16-bit signed integers by subtracting the most significant word from the least significant word of each pair in the source and destination operands. The signed 16-bit results are packed and written to the destination operand.
- PHSUBSW performs horizontal subtraction on each adjacent pair of 16-bit signed integers by subtracting the most significant word from the least significant word of each pair in the source and destination operands. The signed, saturated 16-bit results are packed and written to the destination operand.
- PHSUBD performs horizontal subtraction on each adjacent pair of 32-bit signed integers by subtracting the most significant doubleword from the least significant double word of each pair in the source and destination operands. The signed 32-bit results are packed and written to the destination operand.

12.6.2 Packed Absolute Values

There are six packed-absolute-value instructions (represented by three mnemonics). Three operate on 128-bit operands and three operate on 64-bit operands. The widths of data elements are 8 bits, 16 bits or 32 bits. The absolute value of each data element of the source operand is stored as an UNSIGNED result in the destination operand.

- PABSB computes the absolute value of each signed byte data element.

- PABSW computes the absolute value of each signed 16-bit data element.
- PABSD computes the absolute value of each signed 32-bit data element.

12.6.3 Multiply and Add Packed Signed and Unsigned Bytes

There are two multiply-and-add-packed-signed-unsigned-byte instructions (represented by one mnemonic). One operates on 128-bit operands and the other operates on 64-bit operands. Multiplications are performed on each vertical pair of data elements. The data elements in the source operand are signed byte values, the input data elements of the destination operand are unsigned byte values.

- PMADDUBSW multiplies each unsigned byte value with the corresponding signed byte value to produce an intermediate, 16-bit signed integer. Each adjacent pair of 16-bit signed values are added horizontally. The signed, saturated 16-bit results are packed to the destination operand.

12.6.4 Packed Multiply High with Round and Scale

There are two packed-multiply-high-with-round-and-scale instructions (represented by one mnemonic). One operates on 128-bit operands and the other operates on 64-bit operands.

- PMULHRSW multiplies vertically each signed 16-bit integer from the destination operand with the corresponding signed 16-bit integer of the source operand, producing intermediate, signed 32-bit integers. Each intermediate 32-bit integer is truncated to the 18 most significant bits. Rounding is always performed by adding 1 to the least significant bit of the 18-bit intermediate result. The final result is obtained by selecting the 16 bits immediately to the right of the most significant bit of each 18-bit intermediate result and packed to the destination operand.

12.6.5 Packed Shuffle Bytes

There are two packed-shuffle-bytes instructions (represented by one mnemonic). One operates on 128-bit operands and the other operates on 64-bit operands. The shuffle operations are performed bytewise on the destination operand using the source operand as a control mask.

- PSHUFB permutes each byte in place, according to a shuffle control mask. The least significant three or four bits of each shuffle control byte of the control mask form the shuffle index. The shuffle mask is unaffected. If the most significant bit (bit 7) of a shuffle control byte is set, the constant zero is written in the result byte.

12.6.6 Packed Sign

There are six packed-sign instructions (represented by three mnemonics). Three operate on 128-bit operands and three operate on 64-bit operands. The widths of each data element for these instructions are 8 bit, 16 bit or 32 bit signed integers.

- PSIGNB/W/D negates each signed integer element of the destination operand if the sign of the corresponding data element in the source operand is less than zero.

12.6.7 Packed Align Right

There are two packed-align-right instructions (represented by one mnemonic). One operates on 128-bit operands and the other operates on 64-bit operands. These instructions concatenate the destination and source operand into a composite, and extract the result from the composite according to an immediate constant.

- PALIGNR's source operand is appended after the destination operand forming an intermediate value of twice the width of an operand. The result is extracted from the intermediate value into the destination operand by selecting the 128-bit or 64-bit value that are right-aligned to the byte offset specified by the immediate value.

12.7 WRITING APPLICATIONS WITH SSSE3 EXTENSIONS

The following sections give guidelines for writing application programs and operating-system code that use SSSE3 instructions.

12.7.1 Guidelines for Using SSSE3

The following guidelines describe how to maximize the benefits of using SSSE3:

- Check that the processor supports SSSE3.
- Ensure that your operating system supports SSSE3 and Intel SSE, SSE2, and SSE3. (Operating system support for Intel SSE implies sufficient support for SSSE3 and Intel SSE2 and SSE3.)
- Employ the optimization and scheduling techniques described in the Intel® 64 and IA-32 Architectures Optimization Reference Manual (see Section 1.4, "Related Literature").

12.7.2 Checking for SSSE3 Support

Before an application attempts to use SSSE3, the application should follow the steps illustrated in Section 11.6.2, "Checking for Intel® SSE and SSE2 Support." Next, use the additional step provided below:

- Check that the processor supports SSSE3 (if CPUID.01H:ECX.SSSE3[9] = 1).

12.8 INTEL® SSE3, SSSE3, AND INTEL® SSE4 EXCEPTIONS

Intel SSE3, SSSE3, and Intel SSE4 instructions can generate the same type of memory-access and non-numeric exceptions as other Intel 64 or IA-32 instructions. Existing exception handlers generally handle these exceptions without code modification.

FISTTP can generate floating-point exceptions. Some Intel SSE3 instructions can also generate SIMD floating-point exceptions.

Intel SSE3 additions and changes are noted in the following sections. See also: Section 11.5, "Intel® SSE, SSE2, and SSE3 Exceptions".

12.8.1 Device Not Available (DNA) Exceptions

Intel SSE3, SSSE3, and Intel SSE4 will cause a DNA Exception (#NM) if the processor attempts to execute an Intel SSE3 instruction while CR0.TS[bit 3] = 1. If CPUID.01H:ECX.SSE3[0] = 0, execution of an Intel SSE3 instruction will cause an invalid opcode fault regardless of the state of CR0.TS[bit 3].

Similarly, an attempt to execute an SSSE3 instruction on a processor that reports CPUID.01H:ECX.SSSE3[9] = 0 will cause an invalid opcode fault regardless of the state of CR0.TS[bit 3]. An attempt to execute an Intel SSE4.1 instruction on a processor that reports CPUID.01H:ECX.SSE4_1[19] = 0 will cause an invalid opcode fault regardless of the state of CR0.TS[bit 3].

An attempt to execute PCMPGTQ or any one of the four string processing instructions in Intel SSE4.2 on a processor that reports CPUID.01H:ECX.SSE4_2[20] = 0 will cause an invalid opcode fault regardless of the state of CR0.TS[bit 3]. CRC32 and POPCNT do not cause #NM.

12.8.2 Numeric Error Flag and IGNNE#

Most Intel SSE3 instructions ignore CR0.NE[bit 5] (treats it as if it were always set) and the IGNNE# pin. With one exception, all use the exception 19 (#XM) software exception for error reporting. The exception is FISTTP; it behaves like other x87-FP instructions.

SSSE3 instructions ignore CR0.NE[bit 5] (treats it as if it were always set) and the IGNNE# pin.

SSSE3 instructions do not cause floating-point errors. Floating-point numeric errors for Intel SSE4.1 are described in Section 12.8.4. Intel SSE4.2 instructions do not cause floating-point errors.

12.8.3 Emulation

CR0.EM is used by some software to emulate x87 floating-point instructions. CR0.EM[bit 2] cannot be used for emulation of SSSE3 and Intel SSE, SSE2, SSE3, and SSE4. If an Intel SSE3, SSSE3, or Intel SSE4 instruction execute with CR0.EM[bit 2] set, an invalid opcode exception (INT 6) is generated instead of a device not available exception (INT 7).

12.8.4 IEEE 754 Compliance of Intel® SSE4.1 Floating-Point Instructions

The six Intel SSE4.1 instructions that perform floating-point arithmetic are:

- DPPS
- DPPD
- ROUNDPS
- ROUNDPD
- ROUNDSS
- ROUNDSD

Dot Product operations are not specified in IEEE-754. When neither FTZ nor DAZ are enabled, the dot product instructions resemble sequences of IEEE-754 multiplies and adds (with rounding at each stage), except that the treatment of input NaN's is implementation specific (there will be at least one NaN in the output). The input select fields (bits imm8[4:7]) force input elements to +0.0f prior to the first multiply and will suppress input exceptions that would otherwise have been be generated.

As a convenience to the exception handler, any exceptions signaled from DPPS or DPPD leave the destination unmodified.

Round operations signal invalid and precision only.

Table 12-1. SIMD Numeric Exceptions Signaled by SSE4.1

	DPPS	DPPD	ROUNDPS ROUNDSS	ROUNDPD ROUNDSD
Overflow	X	X		
Underflow	X	X		
Invalid	X	X	X (1)	X (1)
Inexact Precision	X	X	X (2)	X (2)
Denormal	X	X		

NOTE:

1. Invalid is signaled only if Src = SNaN.
2. Precision is ignored (regardless of the MXCSR precision mask) if if imm8[3] = '1'.

The other Intel SSE4.1 instructions with floating-point arguments (BLENDPS, BLENDPD, BLENDVPS, BLENDVPD, INSERTPS, EXTRACTPS) do not signal any SIMD numeric exceptions.

12.9 INTEL® SSE4 OVERVIEW

Intel SSE4 comprises two sets of extensions: Intel SSE4.1 and SSE4.2. Intel SSE4.1 is targeted to improve the performance of media, imaging, and 3D workloads. Intel SSE4.1 adds instructions that improve compiler vectoriza-

tion and significantly increase support for packed dword computation. The technology also provides a hint that can improve memory throughput when reading from uncacheable WC memory type.

The 47 Intel SSE4.1 instructions include:

- Two instructions perform packed dword multiplies.
- Two instructions perform floating-point dot products with input/output selects.
- One instruction performs a load with a streaming hint.
- Six instructions simplify packed blending.
- Eight instructions expand support for packed integer MIN/MAX.
- Four instructions support floating-point round with selectable rounding mode and precision exception override.
- Seven instructions improve data insertion and extractions from XMM registers
- Twelve instructions improve packed integer format conversions (sign and zero extensions).
- One instruction improves SAD (sum absolute difference) generation for small block sizes.
- One instruction aids horizontal searching operations.
- One instruction improves masked comparisons.
- One instruction adds qword packed equality comparisons.
- One instruction adds dword packing with unsigned saturation.

The Intel SSE4.2 instructions operating on XMM registers improve performance in the following areas:

- String and text processing that can take advantage of single-instruction multiple-data programming techniques.
- A SIMD integer instruction that enhances the capability of the 128-bit integer SIMD capability in Intel SSE4.1.

12.10 INTEL® SSE4.1 INSTRUCTION SET

12.10.1 Dword Multiply Instructions

Intel SSE4.1 adds two dword multiply instructions that aid vectorization. They allow four simultaneous 32 bit by 32 bit multiplies. PMULLD returns a low 32-bits of the result and PMULDQ returns a 64-bit signed result. These represent the most common integer multiply operation. See Table 12-2.

Table 12-2. Enhanced 32-Bit SIMD Multiply Supported by Intel® SSE4.1

		32-Bit Integer Operation	
		Unsigned x Unsigned	**Signed x Signed**
Result	**Low 32-bit**	(not available)	PMULLD
	High 32-bit	(not available)	(not available)
	64-bit	PMULUDQ*	PMULDQ

NOTE:

* Available prior to Intel SSE4.1.

12.10.2 Floating-Point Dot Product Instructions

Intel SSE4.1 adds two instructions for double precision (for up to 2 elements; DPPD) and single precision dot products (for up to 4 elements; DPPS).

These dot-product instructions include source select and destination broadcast which generally improves the flexibility. For example, a single DPPS instruction can be used for a 2, 3, or 4 element dot product.

12.10.3 Streaming Load Hint Instruction

Historically, CPU read accesses of WC memory type regions have significantly lower throughput than accesses to cacheable memory.

The streaming load instruction in SSE4.1, MOVNTDQA, provides a non-temporal hint that can cause adjacent 16-byte items within an aligned 64-byte region of WC memory type (a streaming line) to be fetched and held in a small set of temporary buffers ("streaming load buffers"). Subsequent streaming loads to other aligned 16-byte items in the same streaming line may be satisfied from the streaming load buffer and can improve throughput.

Programmers are advised to use the following practices to improve the efficiency of MOVNTDQA streaming loads from WC memory:

- Streaming loads must be 16-byte aligned.
- Temporally group streaming loads of the same streaming cache line for effective use of the small number of streaming load buffers. If loads to the same streaming line are excessively spaced apart, it may cause the streaming line to be re-fetched from memory.
- Temporally group streaming loads from at most a few streaming lines together. The number of streaming load buffers is small; grouping a modest number of streams will avoid running out of streaming load buffers and the resultant re-fetching of streaming lines from memory.
- Avoid writing to a streaming line until all 16-byte-aligned reads from the streaming line have occurred. Reading a 16-byte item from a streaming line that has been written, may cause the streaming line to be re-fetched.
- Avoid reading a given 16-byte item within a streaming line more than once; repeated loads of a particular 16-byte item are likely to cause the streaming line to be re-fetched.
- The streaming load buffers, reflecting the WC memory type characteristics, are not required to be snooped by operations from other agents. Software should not rely upon such coherency actions to provide any data coherency with respect to other logical processors or bus agents. Rather, software must ensure the consistency of WC memory accesses between producers and consumers.
- Streaming loads may be weakly ordered and may appear to software to execute out of order with respect to other memory operations. Software must explicitly use MFENCE if it needs to preserve order among streaming loads or between streaming loads and other memory operations.
- Streaming loads must not be used to reference memory addresses that are mapped to I/O devices having side effects or when reads to these devices are destructive. This is because MOVNTDQA is speculative in nature.

Example 12-1 provides a sketch of the basic assembly sequences that illustrate the principles of using MOVNTDQA in a situation with a producer-consumer accessing a WC memory region.

Example 12-1. Sketch of MOVNTDQA Usage of a Consumer and a PCI Producer

```
// P0: producer is a PCI device writing into the WC space
# the PCI device updates status through a UC flag, "u_dev_status" .
# the protocol for "u_dev_status" : 0: produce; 1: consume; 2: all done

    mov eax, $0
    mov [u_dev_status], eax
producerStart:
    mov eax, [u_dev_status]    # poll status flag to see if consumer is requestion data
    cmp eax, $0                #
    jne done                   # I no longer need to produce
    commence PCI writes to WC region..

    mov eax, $1  # producer ready to notify the consumer via status flag
    mov  [u_dev_status], eax
# now wait for consumer to signal its status
spinloop:
    cmp [u_dev_status], $1     # did I get a signal from the consumer ?
    jne producerStart           # yes I did
    jmp spinloop               # check again
done:
// producer is finished at this point
```

```
// P1: consumer check PCI status flag to consume WC data
    mov eax, $0  # request to the producer
    mov [u_dev_status], eax
consumerStart:
    mov; eax, [u_dev_status]  # reads the value of the PCI status
    cmp eax, $1                    # has producer written
    jne consumerStart               # tight loop; make it more efficient with pause, etc.
    mfence   # producer finished device writes to WC, ensure WC region is coherent
ntread:
    movntdqa xmm0, [addr]
    movntdqa xmm1, [addr + 16]
    movntdqa xmm2, [addr + 32]
    movntdqa xmm3, [addr + 48]
    ...  # do any more NT reads as needed
    mfence  # ensure PCI device reads the correct value of [u_dev_status]
# now decide whether we are done or we need the producer to produce more data
# if we are done write a 2 into the variable, otherwise write a 0 into the variable
    mov eax, $0/$2          # end or continue producing
    mov [u_dev_status], eax
# if I want to consume again I will jump back to consumerStart after storing a 0 into eax
# otherwise I am done
```

12.10.4 Packed Blending Instructions

Intel SSE4.1 adds 6 instructions used for blending (BLENDPS, BLENDPD, BLENDVPS, BLENDVPD, PBLENDVB, PBLENDW).

Blending conditionally copies a data element in a source operand to the same element in the destination. Intel SSE4.1 instructions improve blending operations for most field sizes. A single new Intel SSE4.1 instruction can generally replace a sequence of 2 to 4 operations using previous architectures.

The variable blend instructions (BLENDVPS, BLENDVPD, PBLENDW) introduce the use of control bits stored in an implicit XMM register (XMM0). The most significant bit in each field (the sign bit, for 2's complement integer or floating-point) is used as a selector. See Table 12-3.

Table 12-3. Blend Field Size and Control Modes Supported by Intel® SSE4.1

Instructions	Packed Double FP	Packed Single FP	Packed QWord	Packed DWord	Packed Word	Packed Byte	Blend Control
BLENDPS		X					Imm8
BLENDPD	X						Imm8
BLENDVPS		X		X(1)			XMM0
BLENDVPD	X		X(1)				XMM0
PBLENDVB			(2)	(2)	(2)	X	XMM0
PBLENDW			X	X	X		Imm8

NOTE:

1. Use of floating-point SIMD instructions on integer data types may incur performance penalties.
2. Byte variable blend can be used for larger sized fields by reformatting (or shuffling) the blend control.

12.10.5 Packed Integer MIN/MAX Instructions

Intel SSE4.1 adds 8 packed integer MIN and MAX instructions: PMINUW, PMINUD, PMINSB, PMINSD; PMAXUW, PMAXUD, PMAXSB, and PMAXSD.

Four 32-bit integer packed MIN and MAX instructions operate on unsigned and signed dwords. Two instructions operate on signed bytes. Two instructions operate on unsigned words. See Table 12-4.

Table 12-4. Enhanced SIMD Integer MIN/MAX Instructions Supported by Intel® SSE4.1

		Integer Width		
		Byte	Word	DWord
Integer Format	Unsigned	PMINUB* PMAXUB*	PMINUW PMAXUW	PMINUD PMAXUD
	Signed	PMINSB PMAXSB	PMINSW* PMAXSW*	PMINSD PMAXSD

NOTE:

* Available prior to Intel SSE4.1.

12.10.6 Floating-Point Round Instructions with Selectable Rounding Mode

High level languages and libraries often expose rounding operations having a variety of numeric rounding and exception behaviors. Using Intel SSE, SSE2, and SSE3 instructions to mitigate the rounding-mode-related problem is sometimes not straight forward.

Intel SSE4.1 introduces four rounding instructions (ROUNDPS, ROUNDPD, ROUNDSS, and ROUNDSD) that cover scalar and packed single- and double precision floating-point operands. The rounding mode can be selected using an immediate from one of the IEEE-754 modes (Nearest, -Inf, +Inf, and Truncate) without changing the current

rounding mode; or the instruction can be forced to use the current rounding mode. Another bit in the immediate is used to suppress inexact precision exceptions.

Rounding instructions in Intel SSE4.1 generally permit single-instruction solutions to C99 functions ceil(), floor(), trunc(), rint(), nearbyint(). These instructions simplify the implementations of half-way-away-from-zero rounding modes as used by C99 round() and F90's nint().

12.10.7 Insertion and Extractions from XMM Registers

Intel SSE4.1 adds 7 instructions (corresponding to 9 assembly instruction mnemonics) that simplify data insertion and extraction between general-purpose register (GPR) and XMM registers: EXTRACTPS, INSERTPS, PINSRB, PINSRD, PINSRQ, PEXTRB, PEXTRW, PEXTRD, and PEXTRQ. When accessing memory, no alignment is required for any of these instructions (unless alignment checking is enabled).

EXTRACTPS extracts a single precision floating-point value from any dword offset in an XMM register and stores the result to memory or a general-purpose register. INSERTPS inserts a single floating-point value from either a 32-bit memory location or from specified element in an XMM register to a selected element in the destination XMM register. In addition, INSERTPS allows the insertion of +0.0f into any destination elements using a mask.

PINSRB, PINSRD, and PINSRQ insert byte, dword, or qword integer values from a register or memory into an XMM register. Insertion of integer word values were already supported by Intel SSE2 (PINSRW).

PEXTRB, PEXTRW, PEXTRD, and PEXTRQ extract byte, word, dword, and qword from an XMM register and insert the values into a general-purpose register or memory.

12.10.8 Packed Integer Format Conversions

A common type of operation on packed integers is the conversion by zero- or sign-extension of packed integers into wider data types. Intel SSE4.1 adds 12 instructions that convert from a smaller packed integer type to a larger integer type: PMOVSXBW, PMOVZXBW, PMOVSXBD, PMOVZXBD, PMOVSXWD, PMOVZXWD, PMOVSXBQ, PMOVZXBQ, PMOVSXWQ, PMOVZXWQ, PMOVSXDQ, and PMOVZXDQ.

The source operand is from either an XMM register or memory; the destination is an XMM register. See Table 12-5.

When accessing memory, no alignment is required for any of the instructions unless alignment checking is enabled. In which case, all conversions must be aligned to the width of the memory reference. The number of elements converted (and width of memory reference) is illustrated in Table 12-6. The alignment requirement is shown in parenthesis.

Table 12-5. New SIMD Integer Conversions Supported by Intel® SSE4.1

		Source Type		
		Byte	**Word**	**Dword**
Destination Type	Signed Word Unsigned Word	PMOVSXBW PMOVZXBW		
	Signed Dword Unsigned Dword	PMOVSXBD PMOVZXBD	PMOVSXWD PMOVZXWD	
	Signed Qword Unsigned Qword	PMOVSXBQ PMOVZXBQ	PMOVSXWQ PMOVZXWQ	PMOVSXDQ PMOVZXDQ

Table 12-6. New SIMD Integer Conversions Supported by Intel® SSE4.1

		Source Type		
		Byte	**Word**	**Dword**
Destination Type	Word	8 (64 bits)		
	Dword	4 (32 bits)	4 (64 bits)	
	Qword	2 (16 bits)	2 (32 bits)	2 (64 bits)

12.10.9 Improved Sums of Absolute Differences (SAD) for 4-Byte Blocks

Intel SSE4.1 adds an instruction (MPSADBW) that performs eight 4-byte wide SAD operations per instruction to produce eight results. Compared to PSADBW, MPSADBW operates on smaller chunks (4-byte instead of 8-byte chunks); this makes the instruction better suited to video coding standards such as VC.1 and H.264. MPSADBW performs four times the number of absolute difference operations than that of PSADBW (per instruction). This can improve performance for dense motion searches.

MPSADBW uses a 4-byte wide field from a source operand; the offset of the 4-byte field within the 128-bit source operand is specified by two immediate control bits. MPSADBW produces eight 16-bit SAD results. Each 16-bit SAD result is formed from overlapping pairs of 4 bytes in the destination with the 4-byte field from the source operand. MPSADBW uses eleven consecutive bytes in the destination operand, its offset is specified by a control bit in the immediate byte (i.e., the offset can be from byte 0 or from byte 4). Figure 12-4 illustrates the operation of MPSADBW. MPSADBW can simplify coding of dense motion estimation by providing source and destination offset control, higher throughput of SAD operations, and the smaller chunk size.

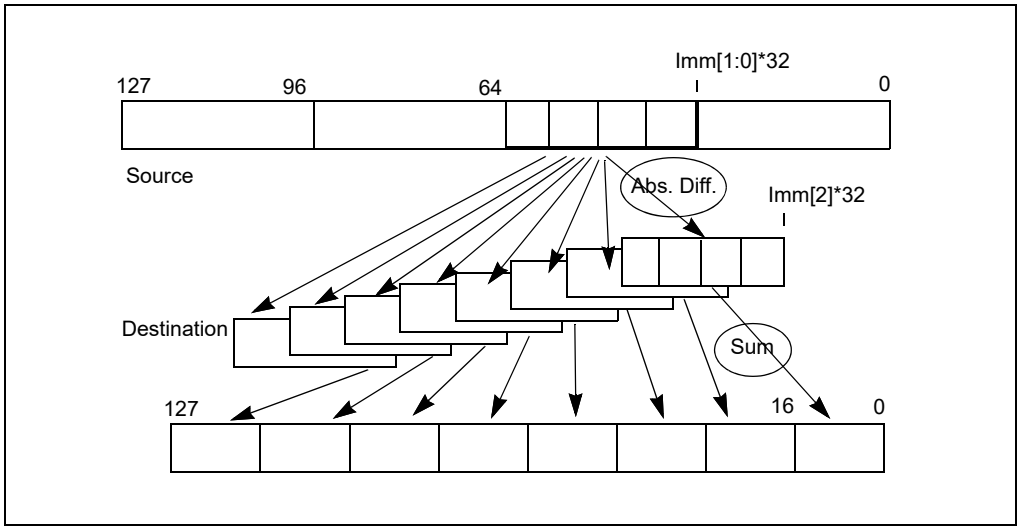


Figure 12-4. MPSADBW Operation

12.10.10 Horizontal Search

Intel SSE4.1 adds a search instruction (PHMINPOSUW) that finds the value and location of the minimum unsigned word from one of 8 horizontally packed unsigned words. The resulting value and location (offset within the source) are packed into the low dword of the destination XMM register.

Rapid search is often a significant component of motion estimation. MPSADBW and PHMINPOSUW can be used together to improve video encode.

12.10.11 Packed Test

The packed test instruction PTEST is similar to a 128-bit equivalent to the legacy instruction TEST. With PTEST, the source argument is typically used like a bit mask.

PTEST performs a logical AND between the destination with this mask and sets the ZF flag if the result is zero. The CF flag (zero for TEST) is set if the inverted mask AND'd with the destination is all zero. Because the destination is not modified, PTEST simplifies branching operations (such as branching on signs of packed floating-point numbers, or branching on zero fields).

12.10.12 Packed Qword Equality Comparisons

Intel SSE4.1 adds a 128-bit packed qword equality test. The new instruction (PCMPEQQ) is identical to PCMPEQD, but has qword granularity.

12.10.13 Dword Packing With Unsigned Saturation

Intel SSE4.1 adds a new instruction PACKUSDW to complete the set of small integer pack instructions in the family of SIMD instruction extensions. PACKUSDW packs dword to word with unsigned saturation. See Table 12-7 for the complete set of packing instructions for small integers.

Table 12-7. Enhanced SIMD Pack Support by Intel® SSE4.1

		Pack Type	
		DWord -> Word	**Word -> Byte**
Saturation Type	**Unsigned**	PACKUSDW (new!)	PACKUSWB
	Signed	PACKSSDW	PACKSSWB

12.11 INTEL® SSE4.2 INSTRUCTION SET

Five of the seven Intel SSE4.2 instructions can use an XMM register as a source or destination. These include four text/string processing instructions and one packed quadword compare SIMD instruction. Programming these five Intel SSE4.2 instructions is similar to programming 128-bit Integer SIMD in Intel SSE2 or SSSE3. Intel SSE4.2 does not provide any 64-bit integer SIMD instructions.

12.11.1 String and Text Processing Instructions

String and text processing instructions in Intel SSE4.2 allocates four opcodes to provide a rich set of string and text processing capabilities that traditionally required many more opcodes. These four instructions use XMM registers to process string or text elements of up to 128-bits (16 bytes or 8 words). Each instruction uses an immediate byte to support a rich set of programmable controls. A string-processing Intel SSE4.2 instruction returns the result of processing each pair of string elements using either an index or a mask.

The capabilities of the string/text processing instructions include:

- Handling string/text fragments consisting of bytes or words, either signed or unsigned.
- Support for partial string or fragments less than 16 bytes in length, using either explicit length or implicit null-termination.
- Four types of string compare operations on word/byte elements.
- Up to 256 compare operations performed in a single instruction on all string/text element pairs.
- Built-in aggregation of intermediate results from comparisons.

- Programmable control of processing on intermediate results.
- Programmable control of output formats in terms of an index or mask.
- Bi-directional support for the index format.
- Support for two mask formats: bit or natural element width.
- Not requiring 16-byte alignment for memory operand.

The four Intel SSE4.2 instructions that process text/string fragments are:

- PCMPESTRI — Packed compare explicit-length strings, return index in ECX/RCX.
- PCMPESTRM — Packed compare explicit-length strings, return mask in XMM0.
- PCMPISTRI — Packed compare implicit-length strings, return index in ECX/RCX.
- PCMPISTRM — Packed compare implicit-length strings, return mask in XMM0.

All four of these instructions require the use of an immediate byte to control operation. The two source operands can be XMM registers or a combination of XMM register and memory address. The immediate byte provides programmable control with the following attributes:

- Input data format.
- Compare operation mode.
- Intermediate result processing.
- Output selection.

Depending on the output format associated with the instruction, the text/string processing instructions implicitly uses either a general-purpose register (ECX/RCX) or an XMM register (XMM0) to return the final result.

Two of the four text-string processing instructions specify string length explicitly. They use two general-purpose registers (EDX, EAX) to specify the number of valid data elements (either word or byte) in the source operands. The other two instructions specify valid string elements using null termination. A data element is considered valid only if it has a lower index than the least significant null data element.

12.11.1.1 Memory Operand Alignment

The text and string processing instructions in Intel SSE4.2 do not perform alignment checking on memory operands. This is different from most other 128-bit SIMD instructions accessing the XMM registers. The absence of an alignment check for these four instructions does not imply any modification to the existing definitions of other instructions.

12.11.2 Packed Comparison SIMD Integer Instruction

Intel SSE4.2 also provides a 128-bit integer SIMD instruction PCMPGTQ that performs logical compare of greater-than on packed integer quadwords.

12.12 WRITING APPLICATIONS WITH INTEL® SSE4 EXTENSIONS

12.12.1 Guidelines for Using Intel® SSE4 Extensions

The following guidelines describe how to maximize the benefits of using Intel SSE4 extensions:

- Check that the processor supports Intel SSE4 extensions.
- Ensure that the operating system supports SSSE3 and Intel SSE, SSE2, and SSE3. (Operating system support for Intel SSE implies sufficient support for SSSE3 and Intel SSE2, SSE3, and SSE4.)
- Employ the optimization and scheduling techniques described in the Intel® 64 and IA-32 Architectures Optimization Reference Manual (see Section 1.4, “Related Literature”).

12.12.2 Checking for Intel® SSE4.1 Support

Before an application attempts to use Intel SSE4.1 instructions, the application should follow the steps illustrated in Section 11.6.2, "Checking for Intel® SSE and SSE2 Support." Next, use the additional step provided below:

Check that the processor supports Intel SSE4.1 (if CPUID.01H:ECX.SSE4_1[19] = 1), Intel SSE3 (if CPUID.01H:ECX.SSE3[0] = 1), and SSSE3 (if CPUID.01H:ECX.SSSE3[9] = 1).

12.12.3 Checking for Intel® SSE4.2 Support

Before an application attempts to use the following Intel SSE4.2 instructions: PCMPESTRI/PCMPESTRM/PCMPISTRI/PCMPISTRM, PCMPGTQ; the application should follow the steps illustrated in Section 11.6.2, "Checking for Intel® SSE and SSE2 Support." Next, use the additional steps provided below:

- Check that the processor supports Intel SSE4.2 (if CPUID.01H:ECX.SSE4_2[20] = 1), Intel SSE4.1 (if CPUID.01H:ECX.SSE4_1[19] = 1), and SSSE3 (if CPUID.01H:ECX.SSSE3[9] = 1).
- Before an application attempts to use the CRC32 instruction, it must check that the processor supports Intel SSE4.2 (if CPUID.01H:ECX.SSE4_2[20] = 1).
- Before an application attempts to use the POPCNT instruction, it must check that the processor supports Intel SSE4.2 (if CPUID.01H:ECX.SSE4_2[20] = 1) and POPCNT (if CPUID.01H:ECX.POPCNT[23] = 1).

12.13 INTEL® AES-NI OVERVIEW

Intel AES-NI provides six instructions to accelerate symmetric block encryption/decryption of 128-bit data blocks using the Advanced Encryption Standard (AES) specified by the NIST publication FIPS 197. Specifically, two instructions (AESENC and AESENCLAST) target the AES encryption rounds; and two instructions (AESDEC and AESDECLAST) target AES decryption rounds using the Equivalent Inverse Cipher. One instruction (AESIMC) targets the Inverse MixColumn transformation primitive, and one instruction (AESKEYGEN) targets generation of round keys from the cipher key for the AES encryption/decryption rounds.

AES supports encryption/decryption using cipher key lengths of 128, 192, and 256 bits by processing the data block in 10, 12, and 14 rounds of predefined transformations. Figure 12-5 depicts the cryptographic processing of a block of 128-bit plain text into cipher text.

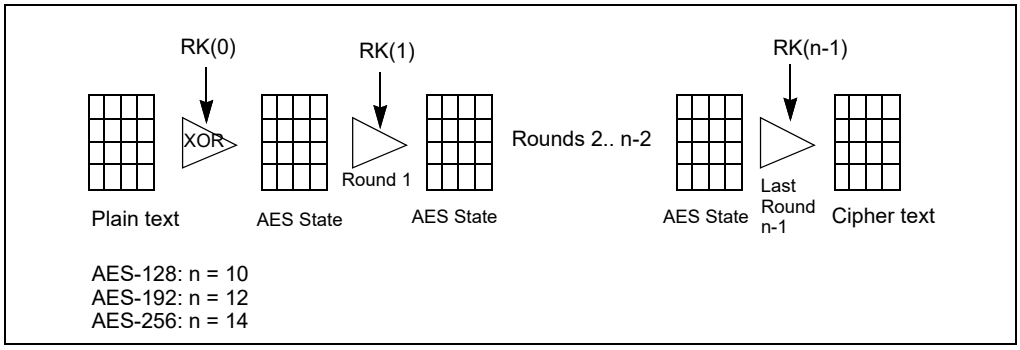


Figure 12-5. AES State Flow

The predefined AES transformation primitives are described in the next few sections, they are also referenced in the operation flow of instruction reference page of these instructions.

12.13.1 Little-Endian Architecture and Big-Endian Specification (FIPS 197)

FIPS 197 document defines the Advanced Encryption Standard (AES) and includes a set of test vectors for testing all of the steps in the algorithm, and can be used for testing and debugging.

The following observation is important for using the AES instructions offered in Intel 64 Architecture: FIPS 197 text convention is to write hex strings with the low-memory byte on the left and the high-memory byte on the right. Intel's convention is the reverse. It is similar to the difference between Big Endian and Little Endian notations.

In other words, a 128 bits vector in the FIPS document, when read from left to right, is encoded as [7:0, 15:8, 23:16, 31:24, …127:120]. Note that inside the byte, the encoding is [7:0], so the first bit from the left is the most significant bit. In practice, the test vectors are written in hexadecimal notation, where pairs of hexadecimal digits define the different bytes. To translate the FIPS 197 notation to an Intel 64 architecture compatible ("Little Endian") format, each test vector needs to be byte-reflected to [127:120,… 31:24, 23:16, 15:8, 7:0].

Example A:

FIPS Test vector: 000102030405060708090a0b0c0d0e0fH

Intel AES Hardware: 0f0e0d0c0b0a09080706050403020100H

It should be pointed out that the only thing at issue is a textual convention, and programmers do not need to perform byte-reversal in their code, when using the AES instructions.

12.13.1.1 AES Data Structure in Intel® 64 Architecture

The AES instructions that are defined in this document operate on one or on two 128 bits source operands: State and Round Key. From the architectural point of view, the state is input in an xmm register and the Round key is input either in an xmm register or a 128-bit memory location.

In AES algorithm, the state (128 bits) can be viewed as four 32-bit doublewords ("Words" in AES terminology): X3, X2, X1, and X0.

The state may also be viewed as a set of 16 bytes. The 16 bytes can also be viewed as a 4x4 matrix of bytes where S(i, j) with i, j = 0, 1, 2, 3 compose the 32-bit "words" as follows:

X0 = S (3, 0) S (2, 0) S (1, 0) S (0, 0)

X1 = S (3, 1) S (2, 1) S (1, 1) S (0, 1)

X2 = S (3, 2) S (2, 2) S (1, 2) S (0, 2)

X3 = S (3, 3) S (2, 3) S (1, 3) S (0, 3)

The following tables, Table 12-8 through Table 12-11, illustrate various representations of a 128-bit state.

Table 12-8. Byte and 32-Bit Word Representation of a 128-Bit State

Byte #	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit Position	**127-120**	**119-112**	**111-103**	**103-96**	**95-88**	**87-80**	**79-72**	**71-64**	**63-56**	**55-48**	**47-40**	**39-32**	**31-24**	**23-16**	**15-8**	**7-0**
	127 - 96				95 - 64				64 - 32				31 - 0			
State Word	X3				X2				X1				X0			
State Byte	P	O	N	M	L	K	J	I	H	G	F	E	D	C	B	A

Table 12-9. Matrix Representation of a 128-Bit State

A	E	I	M	S(0, 0)	S(0, 1)	S(0, 2)	S(0, 3)
B	F	J	N	S(1, 0)	S(1, 1)	S(1, 2)	S(1, 3)
C	G	K	O	S(2, 0)	S(2, 1)	S(2, 2)	S(2, 3)
D	H	L	P	S(3, 0)	S(3, 1)	S(3, 2)	S(3, 3)

Example:

FIPS vector: d4 bf 5d 30 e0 b4 52 ae b8 41 11 f1 1e 27 98 e5

This vector has the "least significant" byte d4 and the significant byte e5 (written in Big Endian format in the FIPS document). When it is translated to IA notations, the encoding is:

Table 12-10. Little Endian Representation of a 128-Bit State

Byte #	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
State Byte	P	O	N	M	L	K	J	I	H	G	F	E	D	C	B	A
State Value	e5	98	27	1e	f1	11	41	b8	ae	52	b4	e0	30	5d	bf	d4

Table 12-11. Little Endian Representation of a 4x4 Byte Matrix

A	E	I	M		d4	e0	b8	1e
B	F	J	N		bf	b4	41	27
C	G	K	O		5d	52	11	98
D	H	L	P		30	ae	f1	e5

12.13.2 AES Transformations and Functions

The following functions and transformations are used in the algorithmic descriptions of AES instruction extensions AESDEC, AESDECLAST, AESENC, AESENCLAST, AESIMC, and AESKEYGENASSIST.

Note that these transformations are expressed here in a Little Endian format (and not as in the FIPS 197 document).

- MixColumns(): A byte-oriented 4x4 matrix transformation on the matrix representation of a 128-bit AES state. A FIPS-197 defined 4x4 matrix is multiplied to each 4x1 column vector of the AES state. The columns are considered polynomials with coefficients in the Finite Field that is used in the definition of FIPS 197, the operations ("multiplication" and "addition") are in that Finite Field, and the polynomials are reduced modulo x^4+1.

  The MixColumns() transformation defines the relationship between each byte of the result state, represented as S'(i, j) of a 4x4 matrix (see Section 12.13.1), as a function of input state bytes, S(i, j), as follows

  S'(0, j) := FF_MUL(02H, S(0, j)) XOR FF_MUL(03H, S(1, j)) XOR S(2, j) XOR S(3, j)

  S'(1, j) := S(0, j) XOR FF_MUL(02H, S(1, j)) XOR FF_MUL(03H, S(2, j)) XOR S(3, j)

  S'(2, j) := S(0, j) XOR S(1, j) XOR FF_MUL(02H, S(2, j)) XOR FF_MUL(03H, S(3, j))

  S'(3, j) := FF_MUL(03H, S(0, j)) XOR S(1, j) XOR S(2, j) XOR FF_MUL(02H, S(3, j))

  where j = 0, 1, 2, 3. FF_MUL(Byte1, Byte2) denotes the result of multiplying two elements (represented by Byte1 and byte2) in the Finite Field representation that defines AES. The result of produced bye FF_MUL(Byte1, Byte2) is an element in the Finite Field (represented as a byte). A Finite Field is a field with a finite number of elements, and when this number can be represented as a power of 2 (2n), its elements can be represented as the set of 2n binary strings of length n. AES uses a finite field with n=8 (having 256 elements). With this representation, "addition" of two elements in that field is a bit-wise XOR of their binary-string representation, producing another element in the field. Multiplication of two elements in that field is defined using an irreducible polynomial (for AES, this polynomial is $m(x) = x^8 + x^4 + x^3 + x + 1$). In this Finite Field representation, the bit value of bit position k of a byte represents the coefficient of a polynomial of order k, e.g., 1010_1101B (ADH) is represented by the polynomial $(x^7 + x^5 + x^3 + x^2 + 1)$. The byte value result of multiplication of two elements is obtained by a carry-less multiplication of the two corresponding polynomials, followed by reduction modulo the polynomial, where the remainder is calculated using operations defined in the field. For example, FF_MUL(57H, 83H) = C1H, because the carry-less polynomial multiplication of the polynomials represented by 57H and 83H produces $(x^{13} + x^{11} + x^9 + x^8 + x^6 + x^5 + x^4 + x^3 + 1)$, and the remainder modulo m(x) is $(x^7 + x^6 + 1)$.
- RotWord(): performs a byte-wise cyclic permutation (rotate right in little-endian byte order) on a 32-bit AES word.

The output word X'[j] of RotWord(X[j]) where X[j] represent the four bytes of column j, S(i, j), in descending order X[j] = (S(3, j), S(2, j), S(1, j), S(0, j)); X'[j] = (S'(3, j), S'(2, j), S'(1, j), S'(0, j)) := (S(0, j), S(3, j), S(2, j), S(1, j))

- ShiftRows(): A byte-oriented matrix transformation that processes the matrix representation of a 16-byte AES state by cyclically shifting the last three rows of the state by different offset to the left, see Table 12-12.

Table 12-12. The ShiftRows Transformation

Matrix Representation of Input State				Output of ShiftRows			
A	E	I	M	A	E	I	M
B	F	J	N	F	J	N	B
C	G	K	O	K	O	C	G
D	H	L	P	P	D	H	L

- SubBytes(): A byte-oriented transformation that processes the 128-bit AES state by applying a non-linear substitution table (S-BOX) on each byte of the state.

  The SubBytes() function defines the relationship between each byte of the result state S'(i, j) as a function of input state byte S(i, j), by

  S'(i, j) := S-Box (S(i, j)[7:4], S(i, j)[3:0])

  where S-BOX (S[7:4], S[3:0]) represents a look-up operation on a 16x16 table to return a byte value, see Table 12-13.

Table 12-13. Look-up Table Associated with S-Box Transformation

		S[3:0]															
		0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
S[7:4]	0	**63**	**7c**	**77**	**7b**	**f2**	**6b**	**6f**	**c5**	**30**	**01**	**67**	**2b**	**fe**	**d7**	**ab**	**76**
	1	**ca**	**82**	**c9**	**7d**	**fa**	**59**	**47**	**f0**	**ad**	**d4**	**a2**	**af**	**9c**	**a4**	**72**	**c0**
	2	**b7**	**fd**	**93**	**26**	**36**	**3f**	**f7**	**cc**	**34**	**a5**	**e5**	**f1**	**71**	**d8**	**31**	**15**
	3	**04**	**c7**	**23**	**c3**	**18**	**96**	**05**	**9a**	**07**	**12**	**80**	**e2**	**eb**	**27**	**b2**	**75**
	4	**09**	**83**	**2c**	**1a**	**1b**	**6e**	**5a**	**a0**	**52**	**3b**	**d6**	**b3**	**29**	**e3**	**2f**	**84**
	5	**53**	**d1**	**00**	**ed**	**20**	**fc**	**b1**	**5b**	**6a**	**cb**	**be**	**39**	**4a**	**4c**	**58**	**cf**
	6	**d0**	**ef**	**aa**	**fb**	**43**	**4d**	**33**	**85**	**45**	**f9**	**02**	**7f**	**50**	**3c**	**9f**	**a8**
	7	**51**	**a3**	**40**	**8f**	**92**	**9d**	**38**	**f5**	**bc**	**b6**	**da**	**21**	**10**	**ff**	**f3**	**d2**
	8	**cd**	**0c**	**13**	**ec**	**5f**	**97**	**44**	**17**	**c4**	**a7**	**7e**	**3d**	**64**	**5d**	**19**	**73**
	9	**60**	**81**	**4f**	**dc**	**22**	**2a**	**90**	**88**	**46**	**ee**	**b8**	**14**	**de**	**5e**	**0b**	**db**
	a	**e0**	**32**	**3a**	**0a**	**49**	**06**	**24**	**5c**	**c2**	**d3**	**ac**	**62**	**91**	**95**	**e4**	**79**
	b	**e7**	**c8**	**37**	**6d**	**8d**	**d5**	**4e**	**a9**	**6c**	**56**	**f4**	**ea**	**65**	**7a**	**ae**	**08**
	c	**ba**	**78**	**25**	**2e**	**1c**	**a6**	**b4**	**c6**	**e8**	**dd**	**74**	**1f**	**4b**	**bd**	**8b**	**8a**
	d	**70**	**3e**	**b5**	**66**	**48**	**03**	**f6**	**0e**	**61**	**35**	**57**	**b9**	**86**	**c1**	**1d**	**9e**
	e	**e1**	**f8**	**98**	**11**	**69**	**d9**	**8e**	**94**	**9b**	**1e**	**87**	**e9**	**ce**	**55**	**28**	**df**
	f	**8c**	**a1**	**89**	**0d**	**bf**	**e6**	**42**	**68**	**41**	**99**	**2d**	**0f**	**b0**	**54**	**bb**	**16**

- SubWord(): produces an output AES word (four bytes) from the four bytes of an input word using a non-linear substitution table (S-BOX).

X'[j] = (S'(3, j), S'(2, j), S'(1, j), S'(0, j)) := (S-Box (S(3, j)), S-Box(S(2, j)), S-Box(S(1, j)), S-Box(S(0, j)))

- InvMixColumns(): The inverse transformation of MixColumns().

  The InvMixColumns() transformation defines the relationship between each byte of the result state S'(i, j) as a function of input state bytes, S(i, j), by

  S'(0, j) := FF_MUL(0eH, S(0, j)) XOR FF_MUL(0bH, S(1, j)) XOR FF_MUL(0dH, S(2, j)) XOR FF_MUL(09H, S(3, j))

  S'(1, j) := FF_MUL(09H, S(0, j)) XOR FF_MUL(0eH, S(1, j)) XOR FF_MUL(0bH, S(2, j)) XOR FF_MUL(0dH, S(3, j))

  S'(2, j) := FF_MUL(0dH, S(0, j)) XOR FF_MUL(09H, S(1, j)) XOR FF_MUL(0eH, S(2, j)) XOR FF_MUL(0bH, S(3, j))

  S'(3, j) := FF_MUL(0bH, S(0, j)) XOR FF_MUL(0dH, S(1, j)) XOR FF_MUL(09H, S(2, j)) XOR FF_MUL(0eH, S(3, j)), where j = 0, 1, 2, 3.

- InvShiftRows(): The inverse transformation of InvShiftRows(). The InvShiftRows() transforms the matrix representation of a 16-byte AES state by cyclically shifting the last three rows of the state by different offset to the right, see Table 12-14.

Table 12-14. The InvShiftRows Transformation

Matrix Representation of Input State				Output of ShiftRows			
A	E	I	M	A	E	I	M
B	F	J	N	N	B	F	J
C	G	K	O	K	O	C	G
D	H	L	P	H	L	P	D

- InvSubBytes(): The inverse transformation of SubBytes().

  The InvSubBytes() transformation defines the relationship between each byte of the result state S'(i, j) as a function of input state byte S(i, j), by

  S'(i, j) := InvS-Box (S(i, j)[7:4], S(i, j)[3:0])

  where InvS-BOX (S[7:4], S[3:0]) represents a look-up operation on a 16x16 table to return a byte value, see Table 12-15.

Table 12-15. Look-up Table Associated with InvS-Box Transformation

		S[3:0]														
		0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
S[7:4]	0	52	09	6a	d5	30	36	a5	38	bf	40	a3	9e	81	f3	d7	fb
	1	7c	e3	39	82	9b	2f	ff	87	34	8e	43	44	c4	de	e9	cb
	2	54	7b	94	32	a6	c2	23	3d	ee	4c	95	0b	42	fa	c3	4e
	3	08	2e	a1	66	28	d9	24	b2	76	5b	a2	49	6d	8b	d1	25
	4	72	f8	f6	64	86	68	98	16	d4	a4	5c	cc	5d	65	b6	92
	5	6c	70	48	50	fd	ed	b9	da	5e	15	46	57	a7	8d	9d	84
	6	90	d8	ab	00	8c	bc	d3	0a	f7	e4	58	05	b8	b3	45	06
	7	d0	2c	1e	8f	ca	3f	0f	02	c1	af	bd	03	01	13	8a	6b
	8	3a	91	11	41	4f	67	dc	ea	97	f2	cf	ce	f0	b4	e6	73
	9	96	ac	74	22	e7	ad	35	85	e2	f9	37	e8	1c	75	df	6e
	a	47	f1	1a	71	1d	29	c5	89	6f	b7	62	0e	aa	18	be	1b
	b	fc	56	3e	4b	c6	d2	79	20	9a	db	c0	fe	78	cd	5a	f4
	c	1f	dd	a8	33	88	07	c7	31	b1	12	10	59	27	80	ec	5f
	d	60	51	7f	a9	19	b5	4a	0d	2d	e5	7a	9f	93	c9	9c	ef
	e	a0	e0	3b	4d	ae	2a	f5	b0	c8	eb	bb	3c	83	53	99	61
	f	17	2b	04	7e	ba	77	d6	26	e1	69	14	63	55	21	0c	7d

12.13.3 PCLMULQDQ

The PCLMULQDQ instruction performs carry-less multiplication of two 64-bit data into a 128-bit result. Carry-less multiplication of two 128-bit data into a 256-bit result can use PCLMULQDQ as building blocks.

Carry-less multiplication is a component of many cryptographic systems. It is an important piece of implementing Galois Counter Mode (GCM) operation of block ciphers. GCM operation can be used in conjunction with AES algorithms to add authentication capability. GCM usage models also include IPsec, storage standard, and security protocols over fiber channel. Additionally, PCLMULQDQ can be used in calculations of hash functions and CRC using arbitrary polynomials.

12.13.4 Checking for Intel® AES-NI Support

Before an application attempts to use AESNI instructions or PCLMULQDQ, the application should follow the steps illustrated in Section 11.6.2, "Checking for Intel® SSE and SSE2 Support." Next, use the additional step provided below:

Check that the processor supports Intel AES-NI (if CPUID.01H:ECX.AESNI[25] = 1); check that the processor supports PCLMULQDQ (if CPUID.01H:ECX.PCLMULQDQ[1] = 1).

CHAPTER 13
MANAGING STATE USING THE XSAVE FEATURE SET

The XSAVE feature set extends the functionality of the FXSAVE and FXRSTOR instructions (see Section 10.5, "FXSAVE and FXRSTOR Instructions") by supporting the saving and restoring of processor state in addition to the x87 execution environment (**x87 state**) and the registers used by the streaming SIMD extensions (**SSE state**).

The **XSAVE feature set** comprises eight instructions. XGETBV and XSETBV allow software to read and write the extended control register XCR0, which controls the operation of the XSAVE feature set. XSAVE, XSAVEOPT, XSAVEC, and XSAVES are four instructions that save processor state to memory; XRSTOR and XRSTORS are corresponding instructions that load processor state from memory. XGETBV, XSAVE, XSAVEOPT, XSAVEC, and XRSTOR can be executed at any privilege level; XSETBV, XSAVES, and XRSTORS can be executed only if CPL = 0. In addition to XCR0, the XSAVES and XRSTORS instructions are controlled also by the IA32_XSS MSR (index DA0H).

The XSAVE feature set organizes the state that manages into **state components**. Operation of the instructions is based on **state-component bitmaps** that have the same format as XCR0 and as the IA32_XSS MSR: each bit corresponds to a state component. Section 13.1 discusses these state components and bitmaps in more detail.

Section 13.2 describes how the processor enumerates support for the XSAVE feature set and for **XSAVE-enabled features** (those features that require the use of the XSAVE feature set for their enabling). Section 13.3 explains how software can enable the XSAVE feature set and XSAVE-enabled features.

The XSAVE feature set allows saving and loading processor state from a region of memory called an **XSAVE area**. Section 13.4 presents details of the XSAVE area and its organization. Each XSAVE-managed state component is associated with a section of the XSAVE area. Section 13.5 describes in detail each of the XSAVE-managed state components.

Section 13.7 through Section 13.12 describe the operation of XSAVE, XRSTOR, XSAVEOPT, XSAVEC, XSAVES, and XRSTORS, respectively.

Section 13.13 provides some details about memory accesses performed by instructions in the XSAVE feature set, and Section 13.14 describes a facility called **extended feature disable** (XFD).

13.1 XSAVE-SUPPORTED FEATURES AND STATE-COMPONENT BITMAPS

The XSAVE feature set supports the saving and restoring of **state components**, each of which is a discrete set of processor registers (or parts of registers). In general, each such state component corresponds to a particular CPU feature. Such a feature is **XSAVE-supported**. Some XSAVE-supported features use registers in multiple XSAVE-managed state components.

The XSAVE feature set organizes the state components of the XSAVE-supported features using **state-component bitmaps**. A state-component bitmap comprises 64 bits; each bit in such a bitmap corresponds to a single state component. The following bits are defined in state-component bitmaps (details on individual state components are provided in subsections of Section 13.5):

- Bit 0 corresponds to the state component used for the x87 FPU execution environment (**x87 state**).
- Bit 1 corresponds to the state component used for registers used by the streaming SIMD extensions (**SSE state**).
- Bit 2 corresponds to the state component used for the additional register state used by the Intel® Advanced Vector Extensions (**AVX state**).
- Bits 4:3 correspond to the two state components used for the additional register state used by Intel® Memory Protection Extensions (**MPX state**):
  - State component 3 is used for the 4 128-bit bounds registers BND0–BND3 (**BNDREGS state**).
  - State component 4 is used for the 64-bit user-mode MPX configuration register BNDCFGU and the 64-bit MPX status register BNDSTATUS (**BNDCSR state**).
- Bits 7:5 correspond to the three state components used for the additional register state used by Intel® Advanced Vector Extensions 512 (**AVX-512 state**):

  - State component 5 is used for the 8 64-bit opmask registers k0–k7 (**opmask state**).
  - State component 6 is used for the upper 256 bits of the registers ZMM0–ZMM15. These 16 256-bit values are denoted ZMM0_H–ZMM15_H (**ZMM_Hi256 state**).
  - State component 7 is used for the 16 512-bit registers ZMM16–ZMM31 (**Hi16_ZMM state**).
- Bit 8 corresponds to the state component used for the Intel Processor Trace MSRs (**PT state**).
- Bit 9 corresponds to the state component used for the protection-key feature's register PKRU (**PKRU state**).
- Bit 10 corresponds to the state component used for the IA32_PASID MSR used by the ENQCMD instruction for a process address space identifiers (**PASID state**).
- Bits 12:11 correspond to the two state components used for the additional register state used by Control-Flow Enforcement Technology (**CET state**):
  - State component 11 is used for the 2 MSRs controlling user-mode functionality for CET (**CET_U state**).
  - State component 12 is used for the 3 MSRs containing shadow-stack pointers for privilege levels 0–2 (**CET_S state**).
- Bit 13 corresponds to the state component used for an MSR used to control hardware duty cycling (**HDC state**).
- Bit 14 corresponds to the state component used for user interrupts (**UINTR state**).
- Bit 15 corresponds to the state component used for last-branch record configuration (**LBR state**).
- Bit 16 corresponds to the state component used for an MSR used to control hardware P-states (**HWP state**).
- Bits 18:17 correspond to the two state components used for the additional register state used by Intel® Advanced Matrix Extensions (**AMX state**):
  - State component 17 is used for the 64-byte TILECFG register (**TILECFG state**).
  - State component 18 is used for the 8192 bytes of tile data (**TILEDATA state**).

Bits in the range 62:19 are not currently defined in state-component bitmaps and are reserved for future expansion. As individual state components are defined using those bits, additional sub-sections will be updated within Section 13.5 over time. Bit 63 is used for special functionality in some bitmaps and does not correspond to any state component.

The state component corresponding to bit *i* of state-component bitmaps is called **state component *i***. Thus, x87 state is state component 0; SSE state is state component 1; AVX state is state component 2; MPX state comprises state components 3–4; AVX-512 state comprises state components 5–7; PT state is state component 8; PKRU state is state component 9; PASID state is state component 10; CET state comprises state components 11–12; HDC state is state component 13; UINTR state is state component 14; LBR state is state component 15; HWP state is state component 16; AMX state comprises state components 17–18.

The XSAVE feature set uses state-component bitmaps in multiple ways. Most of the instructions use an implicit operand (in EDX:EAX), called the **instruction mask**, which is the state-component bitmap that specifies the state components on which the instruction operates.

Some state components are **user state components**, and they can be managed by the entire XSAVE feature set. Other state components are **supervisor state components**, and they can be managed only by XSAVES and XRSTORS. The state components corresponding to bit 9, to bits 18:17, and to bits in the range 7:0 are user state components; those corresponding to bit 8, to bits in the range 13:10, and to bits 16:14 are supervisor state components.

Extended control register XCR0 contains a state-component bitmap that specifies the user state components that software has enabled the XSAVE feature set to manage. If the bit corresponding to a state component is clear in XCR0, instructions in the XSAVE feature set will not operate on that state component, regardless of the value of the instruction mask.

The IA32_XSS MSR (index DA0H) contains a state-component bitmap that specifies the supervisor state components that software has enabled XSAVES and XRSTORS to manage (XSAVE, XSAVEC, XSAVEOPT, and XRSTOR cannot manage supervisor state components). If the bit corresponding to a state component is clear in the IA32_XSS MSR, XSAVES and XRSTORS will not operate on that state component, regardless of the value of the instruction mask.

Some XSAVE-supported features can be used only if XCR0 has been configured so that the features' state components can be managed by the XSAVE feature set. (This applies only to features with user state components.) Such state components and features are **XSAVE-enabled**. In general, the processor will not modify (or allow modification of) the registers of a state component of an XSAVE-enabled feature if the bit corresponding to that state component is clear in XCR0. (If software clears such a bit in XCR0, the processor preserves the corresponding state component.) If an XSAVE-enabled feature has not been fully enabled in XCR0, execution of any instruction defined for that feature causes an invalid-opcode exception (#UD).

As will be explained in Section 13.3, the XSAVE feature set is enabled only if CR4.OSXSAVE[bit 18] = 1. If CR4.OSXSAVE = 0, the processor treats XSAVE-enabled state features and their state components as if all bits in XCR0 were clear; the state components cannot be modified and the features' instructions cannot be executed.

The state components for x87 state, for SSE state, for PT state, for PKRU state, for PASID state, for CET state, for HDC state, for UINTR state, for LBR state, and for HWP state are XSAVE-managed but the corresponding features are not XSAVE-enabled. Processors allow modification of this state, as well as execution of x87 FPU instructions and SSE instructions and use of Intel Processor Trace, protection keys, the ENQCMD instruction and the IA32_PASID MSR, CET, hardware duty cycling, user interrupts, LBRs, and hardware P-states, regardless of the value of CR4.OSXSAVE and XCR0.

13.2 ENUMERATION OF CPU SUPPORT FOR XSAVE INSTRUCTIONS AND XSAVE-SUPPORTED FEATURES

A processor enumerates support for the XSAVE feature set and for features supported by that feature set using the CPUID instruction. The following items provide specific details:

- CPUID.01H:ECX.XSAVE[26] enumerates general support for the XSAVE feature set:
  - — If this bit is 0, the processor does not support any of the following instructions: XGETBV, XRSTOR, XRSTORS, XSAVE, XSAVEC, XSAVEOPT, XSAVES, and XSETBV; the processor provides no further enumeration through CPUID.0DH (see below).
  - — If this bit is 1, the processor supports the following instructions: XGETBV, XRSTOR, XSAVE, and XSETBV.[1] Further enumeration is provided through CPUID.0DH.

  CR4.OSXSAVE can be set to 1 if and only if CPUID.01H:ECX.XSAVE[26] is enumerated as 1.
- CPUID.0DH enumerates details of CPU support through a set of sub-leaves. Software selects a specific sub-leaf by the value placed in the ECX register. The following items provide specific details:
  - — CPUID.0DH.00H.
    - EDX:EAX is a bitmap of all the user state components that can be managed using the XSAVE feature set. A bit can be set in XCR0 if and only if the corresponding bit is set in this bitmap. Every processor that supports the XSAVE feature set will set EAX[0] (x87 state) and EAX[1] (SSE state).

      If EAX[*i*] = 1 (for $1 < i < 32$) or EDX[*i*–32] = 1 (for $32 \le i < 63$), sub-leaf *i* enumerates details for state component *i* (see below).
    - ECX enumerates the size (in bytes) required by the XSAVE instruction for an XSAVE area containing all the user state components supported by this processor.
    - EBX enumerates the size (in bytes) required by the XSAVE instruction for an XSAVE area containing all the user state components corresponding to bits currently set in XCR0.
  - — CPUID.0DH.01H.
    - EAX[0] enumerates support for the XSAVEOPT instruction. The instruction is supported if and only if this bit is 1. If EAX[0] = 0, execution of XSAVEOPT causes an invalid-opcode exception (#UD).
    - EAX[1] enumerates support for **compaction extensions** to the XSAVE feature set. The following are supported if this bit is 1:

1. If CPUID.01H:ECX.XSAVE[26] = 1, XGETBV and XSETBV may be executed with ECX = 0 (to read and write XCR0). Any support for execution of these instructions with other values of ECX is enumerated separately.

  - The compacted format of the extended region of XSAVE areas (see Section 13.4.3).
  - The XSAVEC instruction. If EAX[1] = 0, execution of XSAVEC causes a #UD.
  - Execution of the compacted form of XRSTOR (see Section 13.8).
- EAX[2] enumerates support for execution of XGETBV with ECX = 1. This allows software to determine the state of the init optimization. See Section 13.6.
- EAX[3] enumerates support for XSAVES, XRSTORS, and the IA32_XSS MSR. If EAX[3] = 0, execution of XSAVES or XRSTORS causes a #UD; an attempt to access the IA32_XSS MSR using RDMSR or WRMSR causes a general-protection exception (#GP). Every processor that supports a supervisor state component sets EAX[3]. Every processor that sets EAX[3] (XSAVES, XRSTORS, IA32_XSS) will also set EAX[1] (the compaction extensions).
- EAX[4] enumerates general support for extended feature disable (XFD). See Section 13.14 for details.
- EAX[31:5] are reserved.
- EBX enumerates the size (in bytes) defined as follows:
  - If EAX[3] is enumerated as 1, EBX enumerates the size required by the XSAVES instruction for an XSAVE area containing all the state components corresponding to bits currently set in XCR0 | IA32_XSS.
  - If EAX[3] is enumerated as 0 and EAX[1] is enumerated as 1, EBX enumerates the size required by the XSAVEC instruction for an XSAVE area containing all the state components corresponding to bits currently set in XCR0.
  - If EAX[1] and EAX[3] are both enumerated as 0, EBX enumerates zero.
- EDX:ECX is a bitmap of all the supervisor state components that can be managed by XSAVES and XRSTORS. A bit can be set in the IA32_XSS MSR if and only if the corresponding bit is set in this bitmap.

NOTE

In summary, the XSAVE feature set supports state component*i* ($0 \le i < 63$) if one of the following is true: (1) $i < 32$ and CPUID.0DH.00H:EAX[i] = 1; (2) $i \ge 32$ and CPUID.0DH.00H:EDX[$i-32$] = 1; (3) $i < 32$ and CPUID.0DH.01H:ECX[i] = 1; or (4) $i \ge 32$ and CPUID.0DH.01H:EDX[$i-32$] = 1. The XSAVE feature set supports user state component *i* if (1) or (2) holds; if (3) or (4) holds, state component*i* is a supervisor state component and support is limited to XSAVES and XRSTORS.

— CPUID.0DH.*i* ($i > 1$). This sub-leaf enumerates details for state component *i*. If the XSAVE feature set supports state component *i* (see note above), the following items provide specific details:

- EAX enumerates the size (in bytes) required for state component *i*.
- If state component *i* is a user state component, EBX enumerates the offset (in bytes, from the base of the XSAVE area) of the section used for state component *i*. (This offset applies only when the standard format for the extended region of the XSAVE area is being used; see Section 13.4.3.)
- If state component *i* is a supervisor state component, EBX returns 0.
- If state component *i* is a user state component, ECX[0] return 0; if state component *i* is a supervisor state component, ECX[0] returns 1.
- The value returned by ECX[1] indicates the alignment of state component *i* when the compacted format of the extended region of an XSAVE area is used (see Section 13.4.3). If ECX[1] returns 0, state component *i* is located immediately following the preceding state component; if ECX[1] returns 1, state component *i* is located on the next 64-byte boundary following the preceding state component.
- If the processor supports XFD for state component *i*, ECX[2] returns 1; otherwise, ECX[2] returns 0.
- ECX[31:3] and EDX return 0.

If the XSAVE feature set does not support state component *i*, sub-leaf *i* returns 0 in EAX, EBX, ECX, and EDX.

13.3 ENABLING THE XSAVE FEATURE SET AND XSAVE-ENABLED FEATURES

Software enables the XSAVE feature set by setting CR4.OSXSAVE[bit 18] to 1 (e.g., with the MOV to CR4 instruction). If this bit is 0, execution of any of XGETBV, XRSTOR, XRSTORS, XSAVE, XSAVEC, XSAVEOPT, XSAVES, and XSETBV causes an invalid-opcode exception (#UD).

When CR4.OSXSAVE = 1 and CPL = 0, executing the XSETBV instruction with ECX = 0 writes the 64-bit value in EDX:EAX to XCR0 (EAX is written to XCR0[31:0] and EDX to XCR0[63:32]). (Execution of the XSETBV instruction causes a general-protection fault — #GP — if CPL > 0.) The following items provide details regarding individual bits in XCR0:

- XCR0[0] is associated with x87 state (see Section 13.5.1). XCR0[0] is always 1. It has that value coming out of RESET. Executing the XSETBV instruction causes a general-protection fault (#GP) if ECX = 0 and EAX[0] is 0.
- XCR0[1] is associated with SSE state (see Section 13.5.2). Software can use the XSAVE feature set to manage SSE state only if XCR0[1] = 1. The value of XCR0[1] in no way determines whether software can execute SSE instructions (these instructions can be executed even if XCR0[1] = 0).

  XCR0[1] is 0 coming out of RESET. As noted in Section 13.2, every processor that supports the XSAVE feature set allows software to set XCR0[1].
- XCR0[2] is associated with AVX state (see Section 13.5.3). Software can use the XSAVE feature set to manage AVX state only if XCR0[2] = 1. In addition, software can execute Intel AVX instructions only if CR4.OSXSAVE = XCR0[2] = 1. Otherwise, any execution of an Intel AVX instruction causes an invalid-opcode exception (#UD).

  XCR0[2] is 0 coming out of RESET. As noted in Section 13.2, a processor allows software to set XCR0[2] if and only if CPUID.0DH.00H:EAX[2] = 1. In addition, executing the XSETBV instruction causes a general-protection fault (#GP) if ECX = 0 and EAX[2:1] has the value 10b; that is, software cannot enable the XSAVE feature set for AVX state but not for SSE state.

  As noted in Section 13.1, the processor will preserve AVX state unmodified if software clears XCR0[2]. However, clearing XCR0[2] while AVX state is not in its initial configuration may cause SSE instructions to incur a power and performance penalty. See Section 16.5.3, "Enable the Use Of XSAVE Feature Set And XSAVE State Components," of the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3A, for how system software can avoid this penalty.
- XCR0[4:3] are associated with MPX state (see Section 13.5.4). Software can use the XSAVE feature set to manage MPX state only if XCR0[4:3] = 11b. In addition, MPX instructions operate as defined only if CR4.OSXSAVE = 1 and XCR0[4:3] = 11b. Otherwise, execution of an MPX instruction causes no operation (as a NOP instruction); in addition, executions of CALL, RET, JMP, and Jcc do not initialize the bounds registers, and they ignore any F2H (BND) prefix.[1]

  XCR0[4:3] have value 00b coming out of RESET. As noted in Section 13.2, a processor allows software to set XCR0[4:3] to 11b if and only if CPUID.0DH.00H:EAX[4:3] = 11b. In addition, executing the XSETBV instruction causes a general-protection fault (#GP) if ECX = 0, EAX[4:3] is neither 00b nor 11b; that is, software can enable the XSAVE feature set for MPX state only if it does so for both state components.

  As noted in Section 13.1, the processor will preserve MPX state unmodified if software clears XCR0[4:3].
- XCR0[7:5] are associated with AVX-512 state (see Section 13.5.5). Software can use the XSAVE feature set to manage AVX-512 state only if XCR0[7:5] = 111b. In addition, software can execute Intel AVX-512 instructions only if CR4.OSXSAVE = 1 and XCR0[7:5] = 111b. Otherwise, any execution of an Intel AVX-512 instruction causes an invalid-opcode exception (#UD).

  XCR0[7:5] have value 000b coming out of RESET. As noted in Section 13.2, a processor allows software to set XCR0[7:5] to 111b if and only if CPUID.0DH.00H:EAX[7:5] = 111b. In addition, executing the XSETBV instruction causes a general-protection fault (#GP) if ECX = 0, EAX[7:5] is not 000b, and any bit is clear in EAX[2:1] or EAX[7:5]; that is, software can enable the XSAVE feature set for AVX-512 state only if it does so for all three state components, and only if it also does so for AVX state and SSE state. This implies that the value of XCR0[7:5] is always either 000b or 111b.

  As noted in Section 13.1, the processor will preserve AVX-512 state unmodified if software clears XCR0[7:5]. However, clearing XCR0[7:5] while AVX-512 state is not in its initial configuration may cause SSE and Intel AVX instructions to incur a power and performance penalty. See Section 16.5.3, "Enable the Use Of XSAVE Feature

1. Prior to the introduction of MPX, the opcodes defining MPX instructions operated as NOP, and the CALL, RET, JMP, and Jcc instructions ignored any F2H prefix.

Set And XSAVE State Components," of the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3A, for how system software can avoid this penalty.

- XCR0[9] is associated with PKRU state (see Section 13.5.7). Software can use the XSAVE feature set to manage PKRU state only if XCR0[9] = 1. The value of XCR0[9] in no way determines whether software can use protection keys or execute other instructions that access PKRU state (these instructions can be executed even if XCR0[9] = 0).

  XCR0[9] is 0 coming out of RESET. As noted in Section 13.2, a processor allows software to set XCR0[9] if and only if CPUID.0DH.00H:EAX[9] = 1.

- XCR0[18:17] are associated with AMX state (see Section 13.5.14). Software can use the XSAVE feature set to manage AMX state only if XCR0[18:17] = 11b. In addition, software can execute Intel AMX instructions only if CR4.OSXSAVE = 1 and XCR0[18:17] = 11b. Otherwise, any execution of an Intel AMX instruction causes an invalid-opcode exception (#UD).

  XCR0[18:17] have value 00b coming out of RESET. As noted in Section 13.2, a processor allows software to set XCR0[18:17] to 11b if and only if CPUID.0DH.00H:EAX[18:17] = 11b. In addition, executing the XSETBV instruction causes a general-protection fault (#GP) if ECX = 0 and EAX[17] ≠ EAX[18] (TILECFG and TILEDATA must be enabled together). This implies that the value of XCR0[18:17] is always either 00b or 11b.

  While Intel AMX instructions can be executed only in 64-bit mode, instructions of the XSAVE feature set can operate on TILECFG and TILEDATA in any mode. It is recommended that only 64-bit operating systems enable Intel AMX by setting XCR0[18:17].

- XCR0[63:19], XCR0[16:10], and XCR0[8] are reserved.[1] Executing the XSETBV instruction causes a general-protection fault (#GP) if ECX = 0 and any corresponding bit in EDX:EAX is not 0. These bits in XCR0 are all 0 coming out of RESET.

Software operating with CPL > 0 may need to determine whether the XSAVE feature set and certain XSAVE-enabled features have been enabled. If CPL > 0, execution of the MOV from CR4 instruction causes a general-protection fault (#GP). The following alternative mechanisms allow software to discover the enabling of the XSAVE feature set regardless of CPL:

- The value of CR4.OSXSAVE is returned in CPUID.01H:ECX.OSXSAVE[27]. If software determines that CPUID.01H:ECX.OSXSAVE = 1, the processor supports the XSAVE feature set and the feature set has been enabled in CR4.
- Executing the XGETBV instruction with ECX = 0 returns the value of XCR0 in EDX:EAX. XGETBV can be executed if CR4.OSXSAVE = 1 (if CPUID.01H:ECX.OSXSAVE = 1), regardless of CPL.

Thus, software can use the following algorithm to determine the support and enabling for the XSAVE feature set:

1. Use CPUID to discover the value of CPUID.01H:ECX.OSXSAVE.
   - — If the bit is 0, either the XSAVE feature set is not supported by the processor or has not been enabled by software. Either way, the XSAVE feature set is not available, nor are XSAVE-enabled features such as AVX.
   - — If the bit is 1, the processor supports the XSAVE feature set — including the XGETBV instruction — and it has been enabled by software. The XSAVE feature set can be used to manage x87 state (because XCR0[0] is always 1). Software requiring more detailed information can go on to the next step.
2. Execute XGETBV with ECX = 0 to discover the value of XCR0. If XCR0[1] = 1, the XSAVE feature set can be used to manage SSE state. If XCR0[2] = 1, the XSAVE feature set can be used to manage AVX state and software can execute Intel AVX instructions. If XCR0[4:3] is 11b, the XSAVE feature set can be used to manage MPX state and software can execute Intel MPX instructions. If XCR0[7:5] is 111b, the XSAVE feature set can be used to manage AVX-512 state and software can execute Intel AVX-512 instructions. If XCR0[9] = 1, the XSAVE feature set can be used to manage PKRU state.

The IA32_XSS MSR (with MSR index DA0H) is zero coming out of RESET. If CR4.OSXSAVE = 1, CPUID.0DH.01H:EAX[3] = 1, and CPL= 0, executing the WRMSR instruction with ECX = DA0H writes the 64-bit value in EDX:EAX to the IA32_XSS MSR (EAX is written to IA32_XSS[31:0] and EDX to IA32_XSS[63:32]). The following items provide details regarding individual bits in the IA32_XSS MSR:

1. Bit 8 and bits 16:10 correspond to supervisor state components. Since bits can be set in XCR0 only for user state components, those bits of XCR0 must be 0.

- IA32_XSS[8] is associated with PT state (see Section 13.5.6). Software can use XSAVES and XRSTORS to manage PT state only if IA32_XSS[8] = 1. The value of IA32_XSS[8] does not determine whether software can use Intel Processor Trace (the feature can be used even if IA32_XSS[8] = 0).
- IA32_XSS[10] is associated with PASID state (see Section 13.5.8). Software can use the XSAVES and XRSTORS to manage PASID state only if IA32_XSS[10] = 1. The value of IA32_XSS[10] does not determine whether software can use the ENQCMD instruction, which uses the IA32_PASID MSR. (ENQCMD can be used even if IA32_XSS[10] is 0.)
- IA32_XSS[12:11] are associated with CET state (see Section 13.5.9), IA32_XSS[11] with CET_U state and IA32_XSS[12] with CET_S state. Software can use the XSAVES and XRSTORS to manage CET_U state (respectively, CET_S state) only if IA32_XSS[11] = 1 (respectively, IA32_XSS[12] = 1). The value of IA32_XSS[12:11] does not determine whether software can use CET (the feature can be used even if either of IA32_XSS[12:11] is 0).
- IA32_XSS[13] is associated with HDC state (see Section 13.5.10). Software can use XSAVES and XRSTORS to manage HDC state only if IA32_XSS[13] = 1. The value of IA32_XSS[13] does not determine whether software can use hardware duty cycling (the feature can be used even if IA32_XSS[13] = 0).
- IA32_XSS[14] is associated with UINTR state (see Section 13.5.11). Software can use XSAVES and XRSTORS to manage UINTR state only if IA32_XSS[14] = 1. The value of IA32_XSS[14] does not determine whether software can use user interrupts (the feature can be used even if IA32_XSS[14] = 0).
- IA32_XSS[15] is associated with LBR state (see Section 13.5.12). Software can use XSAVES and XRSTORS to manage LBR state only if IA32_XSS[15] = 1. The value of IA32_XSS[15] does not determine whether software can use LBRs (the feature can be used even if IA32_XSS[15] = 0).
- IA32_XSS[16] is associated with HWP state (see Section 13.5.13). Software can use XSAVES and XRSTORS to manage HWP state only if IA32_XSS[16] = 1. The value of IA32_XSS[16] does not determine whether software can use hardware P-states (the feature can be used even if IA32_XSS[16] = 0).
- IA32_XSS[63:17], IA32_XSS[9] and IA32_XSS[7:0] are reserved.[1] Executing the WRMSR instruction causes a general-protection fault (#GP) if ECX = DA0H and any corresponding bit in EDX:EAX is not 0. These bits in XCR0 are all 0 coming out of RESET.

The IA32_XSS MSR is 0 coming out of RESET.

There is no mechanism by which software operating with CPL > 0 can discover the value of the IA32_XSS MSR.

13.4 XSAVE AREA

The XSAVE feature set includes instructions that save and restore the XSAVE-managed state components to and from memory: XSAVE, XSAVEOPT, XSAVEC, and XSAVES (for saving); and XRSTOR and XRSTORS (for restoring). The processor organizes the state components in a region of memory called an **XSAVE area**. Each of the save and restore instructions takes a memory operand that specifies the 64-byte aligned base address of the XSAVE area on which it operates.

Every XSAVE area has the following format:

- The **legacy region**. The legacy region of an XSAVE area comprises the 512 bytes starting at the area's base address. It is used to manage the state components for x87 state and SSE state. The legacy region is described in more detail in Section 13.4.1.
- The **XSAVE header**. The XSAVE header of an XSAVE area comprises the 64 bytes starting at an offset of 512 bytes from the area's base address. The XSAVE header is described in more detail in Section 13.4.2.
- The **extended region**. The extended region of an XSAVE area starts at an offset of 576 bytes from the area's base address. It is used to manage the state components other than those for x87 state and SSE state. The extended region is described in more detail in Section 13.4.3. The size of the extended region is determined by which state components the processor supports and which bits have been set in XCR0 and IA32_XSS (see Section 13.3).

1. Bit 9 and bits 7:0 correspond to user state components. Since bits can be set in the IA32_XSS MSR only for supervisor state components, those bits of the MSR must be 0.

13.4.1 Legacy Region of an XSAVE Area

The legacy region of an XSAVE area comprises the 512 bytes starting at the area's base address. It has the same format as the FXSAVE area (see Section 10.5.1). The XSAVE feature set uses the legacy area for x87 state (state component 0) and SSE state (state component 1). Table 13-1 illustrates the format of the first 416 bytes of the legacy region of an XSAVE area.

Table 13-1. Format of the Legacy Region of an XSAVE Area

15 14	13 12	11 10	9 8	7 6	5	4	3 2	1 0	
FIP[63:48] or reserved	FCS or FIP[47:32]	FIP[31:0]		FOP	Rsvd.	FTW	FSW	FCW	**0**
MXCSR_MASK		MXCSR		FDP[63:48] or reserved	FDS or FDP[47:32]		FDP[31:0]		**16**
Reserved			ST0/MM0						**32**
Reserved			ST1/MM1						**48**
Reserved			ST2/MM2						**64**
Reserved			ST3/MM3						**80**
Reserved			ST4/MM4						**96**
Reserved			ST5/MM5						**112**
Reserved			ST6/MM6						**128**
Reserved			ST7/MM7						**144**
XMM0									**160**
XMM1									**176**
XMM2									**192**
XMM3									**208**
XMM4									**224**
XMM5									**240**
XMM6									**256**
XMM7									**272**
XMM8									**288**
XMM9									**304**
XMM10									**320**
XMM11									**336**
XMM12									**352**
XMM13									**368**
XMM14									**384**
XMM15									**400**

The x87 state component comprises bytes 23:0 and bytes 159:32. The SSE state component comprises bytes 31:24 and bytes 415:160. The XSAVE feature set does not use bytes 511:416; bytes 463:416 are reserved.

Section 13.7 through Section 13.9 provide details of how instructions in the XSAVE feature set use the legacy region of an XSAVE area.

13.4.2 XSAVE Header

The XSAVE header of an XSAVE area comprises the 64 bytes starting at offset 512 from the area's base address:

- Bytes 7:0 of the XSAVE header is a state-component bitmap (see Section 13.1) called **XSTATE_BV**. It identifies the state components in the XSAVE area.
- Bytes 15:8 of the XSAVE header is a state-component bitmap called **XCOMP_BV**. It is used as follows:
  - XCOMP_BV[63] indicates the format of the extended region of the XSAVE area (see Section 13.4.3). If it is clear, the standard format is used. If it is set, the compacted format is used; XCOMP_BV[62:0] provide format specifics as specified in Section 13.4.3.
  - XCOMP_BV[63] determines which form of the XRSTOR instruction is used. If the bit is set, the compacted form is used; otherwise, the standard form is used. See Section 13.8.
  - All bits in XCOMP_BV should be 0 if the processor does not support the compaction extensions to the XSAVE feature set.
- Bytes 63:16 of the XSAVE header are reserved.

Section 13.7 through Section 13.9 provide details of how instructions in the XSAVE feature set use the XSAVE header of an XSAVE area.

13.4.3 Extended Region of an XSAVE Area

The extended region of an XSAVE area starts at byte offset 576 from the area's base address. The size of the extended region is determined by which state components the processor supports and which bits have been set in XCR0 | IA32_XSS (see Section 13.3). The XSAVE feature set uses the extended area for each state component *i*, where $i \geq 2$.

The extended region of the an XSAVE area may have one of two formats. The **standard format** is supported by all processors that support the XSAVE feature set; the **compacted format** is supported by those processors that support the compaction extensions to the XSAVE feature set (see Section 13.2). Bit 63 of the XCOMP_BV field in the XSAVE header (see Section 13.4.2) indicates which format is used.

The following items describe the two possible formats of the extended region:

- **Standard format.** Each state component *i* ($i \geq 2$) is located at the byte offset from the base address of the XSAVE area enumerated in CPUID.0DH.*i*:EBX. (CPUID.0DH.*i*:EAX enumerates the number of bytes required for state component *i*.
- **Compacted format.** Each state component *i* ($i \geq 2$) is located at a byte offset from the base address of the XSAVE area based on the XCOMP_BV field in the XSAVE header:
  - If XCOMP_BV[*i*] = 0, state component *i* is not in the XSAVE area.
  - If XCOMP_BV[*i*] = 1, state component *i* is located at a byte offset $location_I$ from the base address of the XSAVE area, where $location_I$ is determined by the following items:
    - If XCOMP_BV[*j*] = 0 for every *j*, $2 \leq j < i$, $location_I$ is 576. (This item applies if *i* is the first bit set in bits 62:2 of the XCOMP_BV; it implies that state component *i* is located at the beginning of the extended region.)
    - Otherwise, let *j*, $2 \leq j < i$, be the greatest value such that XCOMP_BV[*j*] = 1. Then $location_I$ is determined by the following values: $location_J$; $size_J$, as enumerated in CPUID.0DH.*j*:EAX; and the value of $align_I$, as enumerated in CPUID.0DH.*i*:ECX[1]:
      - If $align_I$ = 0, $location_I = location_J + size_J$. (This item implies that state component *i* is located immediately following the preceding state component whose bit is set in XCOMP_BV.)
      - If $align_I$ = 1, $location_I$ = ceiling($location_J + size_J$, 64). (This item implies that state component *i* is located on the next 64-byte boundary following the preceding state component whose bit is set in XCOMP_BV.)

13.5 XSAVE-MANAGED STATE

The section provides details regarding how the XSAVE feature set interacts with the various XSAVE-managed state components.

Unless otherwise state, the state pertaining to a particular state component is saved beginning at byte 0 of the section of the XSAVE are corresponding to that state component.

13.5.1 x87 State

Instructions in the XSAVE feature set can manage the same state of the x87 FPU execution environment (**x87 state**) that can be managed using the FXSAVE and FXRSTOR instructions. They organize all x87 state as a user state component in the legacy region of the XSAVE area (see Section 13.4.1). This region is illustrated in Table 13-1; the x87 state is listed below, along with details of its interactions with the XSAVE feature set:

- Bytes 1:0, 3:2, 7:6. These are used for the x87 FPU Control Word (FCW), the x87 FPU Status Word (FSW), and the x87 FPU Opcode (FOP), respectively.
- Byte 4 is used for an abridged version of the x87 FPU Tag Word (FTW). The following items describe its usage:
  - For each j, $0 \le j \le 7$, XSAVE, XSAVEOPT, XSAVEC, and XSAVES save a 0 into bit j of byte 4 if x87 FPU data register STj has a empty tag; otherwise, XSAVE, XSAVEOPT, XSAVEC, and XSAVES save a 1 into bit j of byte 4.
  - For each j, $0 \le j \le 7$, XRSTOR and XRSTORS establish the tag value for x87 FPU data register STj as follows. If bit j of byte 4 is 0, the tag for STj in the tag register for that data register is marked empty (11B); otherwise, the x87 FPU sets the tag for STj based on the value being loaded into that register (see below).
- Bytes 15:8 are used as follows:
  - If the instruction has no REX prefix, or if REX.W = 0:
    - Bytes 11:8 are used for bits 31:0 of the x87 FPU Instruction Pointer Offset (FIP).
    - If CPUID.07H.00H:EBX[13] = 0, bytes 13:12 are used for x87 FPU Instruction Pointer Selector (FCS). Otherwise, XSAVE, XSAVEOPT, XSAVEC, and XSAVES save these bytes as 0000H, and XRSTOR and XRSTORS ignore them.
    - Bytes 15:14 are not used.
  - If the instruction has a REX prefix with REX.W = 1, bytes 15:8 are used for the full 64 bits of FIP.
- Bytes 23:16 are used as follows:
  - If the instruction has no REX prefix, or if REX.W = 0:
    - Bytes 19:16 are used for bits 31:0 of the x87 FPU Data Pointer Offset (FDP).
    - If CPUID.07H.00H:EBX[13] = 0, bytes 21:20 are used for x87 FPU Data Pointer Selector (FDS). Otherwise, XSAVE, XSAVEOPT, XSAVEC, and XSAVES save these bytes as 0000H; and XRSTOR and XRSTORS ignore them.
    - Bytes 23:22 are not used.
  - If the instruction has a REX prefix with REX.W = 1, bytes 23:16 are used for the full 64 bits of FDP.
- Bytes 31:24 are used for SSE state (see Section 13.5.2).
- Bytes 159:32 are used for the registers ST0–ST7 (MM0–MM7). Each of the 8 register is allocated a 128-bit region, with the low 80 bits used for the register and the upper 48 bits unused.

x87 state is XSAVE-managed but the x87 FPU feature is not XSAVE-enabled. The XSAVE feature set can operate on x87 state only if the feature set is enabled (CR4.OSXSAVE = 1).[1] Software can otherwise use x87 state even if the XSAVE feature set is not enabled.

1. The processor ensures that XCR0[0] is always 1.

13.5.2 SSE State

Instructions in the XSAVE feature set can manage the registers used by the streaming SIMD extensions (**SSE state**) just as the FXSAVE and FXRSTOR instructions do. They organize all SSE state as a user state component in the legacy region of the XSAVE area (see Section 13.4.1). This region is illustrated in Table 13-1; the SSE state is listed below, along with details of its interactions with the XSAVE feature set:

- Bytes 23:0 are used for x87 state (see Section 13.5.1).
- Bytes 27:24 are used for the MXCSR register. XRSTOR and XRSTORS generate general-protection faults (#GP) in response to attempts to set any of the reserved bits of the MXCSR register.[1]
- Bytes 31:28 are used for the MXCSR_MASK value. XRSTOR and XRSTORS ignore this field.
- Bytes 159:32 are used for x87 state.
- Bytes 287:160 are used for the registers XMM0–XMM7.
- Bytes 415:288 are used for the registers XMM8–XMM15. These fields are used only in 64-bit mode. Executions of XSAVE, XSAVEOPT, XSAVEC, and XSAVES outside 64-bit mode do not modify these bytes; executions of XRSTOR and XRSTORS outside 64-bit mode do not update XMM8–XMM15. See Section 13.13.

SSE state is XSAVE-managed but the SSE feature is not XSAVE-enabled. The XSAVE feature set can operate on SSE state only if the feature set is enabled (CR4.OSXSAVE = 1) and has been configured to manage SSE state (XCR0[1] = 1). Software can otherwise use SSE state even if the XSAVE feature set is not enabled or has not been configured to manage SSE state.

13.5.3 AVX State

The register state used by the Intel® Advanced Vector Extensions (Intel AVX) comprises the MXCSR register and 16 256-bit vector registers called YMM0–YMM15. The low 128 bits of each register YMM*i* is identical to the SSE register XMM*i*. Thus, the new state register state added by Intel AVX comprises the upper 128 bits of the registers YMM0–YMM15. These 16 128-bit values are denoted YMM0_H–YMM15_H and are collectively called **AVX state**.

As noted in Section 13.1, the XSAVE feature set manages AVX state as user state component 2. Thus, AVX state is located in the extended region of the XSAVE area (see Section 13.4.3).

As noted in Section 13.2, CPUID.0DH.02H:EBX enumerates the offset (in bytes, from the base of the XSAVE area) of the section of the extended region of the XSAVE area used for AVX state (when the standard format of the extended region is used). CPUID.0DH.02H:EAX enumerates the size (in bytes) required for AVX state.

The XSAVE feature set partitions YMM0_H–YMM15_H in a manner similar to that used for the XMM registers (see Section 13.5.2). Bytes 127:0 of the AVX-state section are used for YMM0_H–YMM7_H. Bytes 255:128 are used for YMM8_H–YMM15_H, but they are used only in 64-bit mode. Executions of XSAVE, XSAVEOPT, XSAVEC, and XSAVES outside 64-bit mode do not modify bytes 255:128; executions of XRSTOR and XRSTORS outside 64-bit mode do not update YMM8_H–YMM15_H. See Section 13.13. In general, bytes 16*i*+15:16*i* are used for YMM*i*_H (for $0 \leq i \leq 15$).

AVX state is XSAVE-managed and the Intel AVX feature is XSAVE-enabled. The XSAVE feature set can operate on AVX state only if the feature set is enabled (CR4.OSXSAVE = 1) and has been configured to manage AVX state (XCR0[2] = 1). Intel AVX instructions cannot be used unless the XSAVE feature set is enabled and has been configured to manage AVX state.

13.5.4 MPX State

The register state used by the Intel® Memory Protection Extensions (MPX) comprises the 4 128-bit bounds registers BND0–BND3 (**BNDREGS state**); and the 64-bit user-mode configuration register BNDCFGU and the 64-bit MPX status register BNDSTATUS (collectively, **BNDCSR state**). Together, these two user state components compose **MPX state**.

1. While MXCSR and MXCSR_MASK are part of SSE state, their treatment by the XSAVE feature set is not the same as that of the XMM registers. See Section 13.7 through Section 13.11 for details.

As noted in Section 13.1, the XSAVE feature set manages MPX state as state components 3–4. Thus, MPX state is located in the extended region of the XSAVE area (see Section 13.4.3). The following items detail how these state components are organized in this region:

- **BNDREGS state.** As noted in Section 13.2, CPUID.0DH.03H:EBX enumerates the offset (in bytes, from the base of the XSAVE area) of the section of the extended region of the XSAVE area used for BNDREGS state (when the standard format of the extended region is used). CPUID.0DH.03H:EAX enumerates the size (in bytes) required for BNDREGS state. The BNDREGS section is used for the 4 128-bit bound registers BND0–BND3, with bytes 16*i*+15:16*i* being used for BND*i*.
- **BNDCSR state.** As noted in Section 13.2, CPUID.0DH.04H:EBX enumerates the offset of the section of the extended region of the XSAVE area used for BNDCSR state (when the standard format of the extended region is used). CPUID.0DH.04H:EAX enumerates the size (in bytes) required for BNDCSR state. In the BNDSCR section, bytes 7:0 are used for BNDCFGU and bytes 15:8 are used for BNDSTATUS.

Both components of MPX state are XSAVE-managed and the Intel MPX feature is XSAVE-enabled. The XSAVE feature set can operate on MPX state only if the feature set is enabled (CR4.OSXSAVE = 1) and has been configured to manage MPX state (XCR0[4:3] = 11b). Intel MPX instructions cannot be used unless the XSAVE feature set is enabled and has been configured to manage MPX state.

13.5.5 AVX-512 State

The register state used by the Intel® Advanced Vector Extensions 512 (Intel AVX-512) comprises the MXCSR register, the 8 64-bit opmask registers k0–k7, and 32 512-bit vector registers called ZMM0–ZMM31. For each *i*, 0 ≤ *i* ≤ 15, the low 256 bits of register ZMM*i* is identical to the Intel AVX register YMM*i*. Thus, the new state register state added by Intel AVX-512 comprises the following user state components:

- The opmask registers, collectively called **opmask state**.
- The upper 256 bits of the registers ZMM0–ZMM15. These 16 256-bit values are denoted ZMM0_H–ZMM15_H and are collectively called **ZMM_Hi256 state**.
- The 16 512-bit registers ZMM16–ZMM31, collectively called **Hi16_ZMM state**.

Together, these three state components compose **AVX-512 state**.

As noted in Section 13.1, the XSAVE feature set manages AVX-512 state as state components 5–7. Thus, AVX-512 state is located in the extended region of the XSAVE area (see Section 13.4.3). The following items detail how these state components are organized in this region:

- **Opmask state.** As noted in Section 13.2, CPUID.0DH.05H:EBX enumerates the offset (in bytes, from the base of the XSAVE area) of the section of the extended region of the XSAVE area used for opmask state (when the standard format of the extended region is used). CPUID.0DH.05H:EAX enumerates the size (in bytes) required for opmask state. The opmask section is used for the 8 64-bit opmask registers k0–k7, with bytes 8*i*+7:8*i* being used for k*i*.
- **ZMM_Hi256 state.** As noted in Section 13.2, CPUID.0DH.06H:EBX enumerates the offset of the section of the extended region of the XSAVE area used for ZMM_Hi256 state (when the standard format of the extended region is used). CPUID.0DH.06H:EAX enumerates the size (in bytes) required for ZMM_Hi256 state.

  The XSAVE feature set partitions ZMM0_H–ZMM15_H in a manner similar to that used for the XMM registers (see Section 13.5.2). Bytes 255:0 of the ZMM_Hi256-state section are used for ZMM0_H–ZMM7_H. Bytes 511:256 are used for ZMM8_H–ZMM15_H, but they are used only in 64-bit mode. Executions of XSAVE, XSAVEOPT, XSAVEC, and XSAVES outside 64-bit mode do not modify bytes 511:256; executions of XRSTOR and XRSTORS outside 64-bit mode do not update ZMM8_H–ZMM15_H. See Section 13.13. In general, bytes 32*i*+31:32*i* are used for ZMM*i*_H (for 0 ≤ *i* ≤ 15).
- **Hi16_ZMM state.** As noted in Section 13.2, CPUID.0DH.07H:EBX enumerates the offset of the section of the extended region of the XSAVE area used for Hi16_ZMM state (when the standard format of the extended region is used). CPUID.0DH.07H:EAX enumerates the size (in bytes) required for Hi16_ZMM state.

  The XSAVE feature set accesses Hi16_ZMM state only in 64-bit mode. Executions of XSAVE, XSAVEOPT, XSAVEC, and XSAVES outside 64-bit mode do not modify the Hi16_ZMM section; executions of XRSTOR and XRSTORS outside 64-bit mode do not update ZMM16–ZMM31. See Section 13.13. In general, bytes 64(*i*-16)+63:64(*i*-16) are used for ZMM*i* (for 16 ≤ *i* ≤ 31).

All three components of AVX-512 state are XSAVE-managed and the Intel AVX-512 feature is XSAVE-enabled. The XSAVE feature set can operate on AVX-512 state only if the feature set is enabled (CR4.OSXSAVE = 1) and has been configured to manage AVX-512 state (XCR0[7:5] = 111b). Intel AVX-512 instructions cannot be used unless the XSAVE feature set is enabled and has been configured to manage AVX-512 state.

13.5.6 PT State

The register state used by Intel Processor Trace (**PT state**) comprises the following 9 MSRs: IA32_RTIT_CTL, IA32_RTIT_OUTPUT_BASE, IA32_RTIT_OUTPUT_MASK_PTRS, IA32_RTIT_STATUS, IA32_RTIT_CR3_MATCH, IA32_RTIT_ADDR0_A, IA32_RTIT_ADDR0_B, IA32_RTIT_ADDR1_A, and IA32_RTIT_ADDR1_B.[1]

As noted in Section 13.1, the XSAVE feature set manages PT state as supervisor state component 8. Thus, PT state is located in the extended region of the XSAVE area (see Section 13.4.3). As noted in Section 13.2, CPUID.0DH.08H:EAX enumerates the size (in bytes) required for PT state. The MSRs are each allocated 8 bytes in the state component in the order given above. Thus, IA32_RTIT_CTL is at byte offset 0, IA32_RTIT_OUTPUT_BASE at byte offset 8, etc. Any locations in the state component at or beyond byte offset 72 are reserved.

PT state is XSAVE-managed but Intel Processor Trace is not XSAVE-enabled. The XSAVE feature set can operate on PT state only if the feature set is enabled (CR4.OSXSAVE = 1) and has been configured to manage PT state (IA32_XSS[8] = 1). Software can otherwise use Intel Processor Trace and access its MSRs (using RDMSR and WRMSR) even if the XSAVE feature set is not enabled or has not been configured to manage PT state.

The following items describe special treatment of PT state by the XSAVES and XRSTORS instructions:

- If XSAVES saves PT state, the instruction clears IA32_RTIT_CTL.TraceEn (bit 0) after saving the value of the IA32_RTIT_CTL MSR and before saving any other PT state. If XSAVES causes a fault or a VM exit, it restores IA32_RTIT_CTL.TraceEn to its original value.
- If XSAVES saves PT state, the instruction saves zeroes in the reserved portions of the state component.
- If XRSTORS would restore (or initialize) PT state and IA32_RTIT_CTL.TraceEn = 1, the instruction causes a general-protection exception (#GP) before modifying PT state.
- If XRSTORS causes an exception or a VM exit, it does so before any modification to IA32_RTIT_CTL.TraceEn (even if it has loaded other PT state).

13.5.7 PKRU State

The register state used by the protection-key feature (**PKRU state**) is the 32-bit PKRU register. As noted in Section 13.1, the XSAVE feature set manages PKRU state as user state component 9. Thus, PKRU state is located in the extended region of the XSAVE area (see Section 13.4.3).

As noted in Section 13.2, CPUID.0DH.09H:EBX enumerates the offset (in bytes, from the base of the XSAVE area) of the section of the extended region of the XSAVE area used for PKRU state (when the standard format of the extended region is used). CPUID.0DH.09H:EAX enumerates the size (in bytes) required for PKRU state. The XSAVE feature set uses bytes 3:0 of the PK-state section for the PKRU register.

PKRU state is XSAVE-managed but the protection-key feature is not XSAVE-enabled. The XSAVE feature set can operate on PKRU state only if the feature set is enabled (CR4.OSXSAVE = 1) and has been configured to manage PKRU state (XCR0[9] = 1). Software can otherwise use protection keys and access PKRU state even if the XSAVE feature set is not enabled or has not been configured to manage PKRU state.

The value of the PKRU register determines the access rights for user-mode linear addresses. (See Section 5.6, "Access Rights," of Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3A.) The access rights that pertain to an execution of the XRSTOR and XRSTORS instructions are determined by the value of the register before the execution and not by any value that the execution might load into the PKRU register.

1. These MSRs might not be supported by every processor that supports Intel Processor Trace. Software can use the CPUID instruction to discover which are supported; see Section 36.3.1, "Detection of Intel Processor Trace and Capability Enumeration," of Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3C.

13.5.8 PASID State

The register state used by the ENQCMD instruction and process address space identifiers (**PASID state**) comprises the IA32_PASID MSR.

As noted in Section 13.1, the XSAVE feature set manages PASID state as supervisor state component 10. Thus, PASID state is located in the extended region of the XSAVE area (see Section 13.4.3). As noted in Section 13.2, CPUID.0DH.0AH:EAX enumerates the size (in bytes) required for PASID state. The IA32_PASID MSR is allocated 8 bytes at byte offset 0 in the state component.

PASID state is XSAVE-managed but the ENQCMD instruction and process address space identifiers are not XSAVE-enabled. The XSAVE feature set can operate on PASID state only if the feature set is enabled (CR4.OSXSAVE = 1) and has been configured to manage PASID state (IA32_XSS[10] = 1). Software can otherwise use the ENQCMD instruction and process address space identifiers, and access the IA32_PASID MSR (using RDMSR and WRMSR) even if the XSAVE feature set is not enabled or has not been configured to manage PASID state.

13.5.9 CET State

The register state used by Control-Flow Enforcement Technology (CET) comprises the two 64-bit MSRs (IA32_U_CET and IA32_PL3_SSP) that manage CET when CPL = 3 (**CET_U state**); and the three 64-bit MSRs (IA32_PL0_SSP–IA32_PL2_SSP) that manage CET when CPL < 3 (**CET_S state**). Together, these two supervisor state components compose **CET state**.[1]

As noted in Section 13.1, the XSAVE feature set manages CET state as supervisor state components 11–12. Thus, CET state is located in the extended region of the XSAVE area (see Section 13.4.3). The following items detail how these state components are organized in this region:

- **CET_U state.** As noted in Section 13.2, CPUID.0DH.0BH:EAX enumerates the size (in bytes) required for CET_U state. The CET_U section is used for the 64-bit MSRs IA32_U_CET and IA32_PL3_SSP, with bytes 7:0 being used for IA32_U_CET and bytes 15:8 being used for IA32_PL3_SSP.
- **CET_S state.** As noted in Section 13.2, CPUID.0DH.0CH:EAX enumerates the size (in bytes) required for CET_S state. The CET_S section is used for the three 64-bit MSRs IA32_PL0_SSP–IA32_PL2_SSP, with bytes 8*i*+7:8*i* being used for IA32_PL*i*_SSP.

The two components of CET state are XSAVE-managed and CET is not XSAVE-enabled. The XSAVE feature set can operate on CET_U state (respectively, CET_S state) only if the feature set is enabled (CR4.OSXSAVE = 1) and has been configured to manage CET_U state (respectively, CET_S state) by setting IA32_XSS[11] (respectively, IA32_XSS[12]). Software can otherwise use CET and access the CET MSRs (using RDMSR and WRMSR) even if the XSAVE feature set is not enabled or has not been configured to manage CET state.

13.5.10 HDC State

The register state used by hardware duty cycling (**HDC state**) comprises the IA32_PM_CTL1 MSR.

As noted in Section 13.1, the XSAVE feature set manages HDC state as supervisor state component 13. Thus, HDC state is located in the extended region of the XSAVE area (see Section 13.4.3). As noted in Section 13.2, CPUID.0DH.0DH:EAX enumerates the size (in bytes) required for HDC state. The IA32_PM_CTL1 MSR is allocated 8 bytes at byte offset 0 in the state component.

HDC state is XSAVE-managed but hardware duty cycling is not XSAVE-enabled. The XSAVE feature set can operate on HDC state only if the feature set is enabled (CR4.OSXSAVE = 1) and has been configured to manage HDC state (IA32_XSS[13] = 1). Software can otherwise use hardware duty cycling and access the IA32_PM_CTL1 MSR (using RDMSR and WRMSR) even if the XSAVE feature set is not enabled or has not been configured to manage HDC state.

13.5.11 UINTR State

The register state used by user interrupts (**UINTR state**) comprises 48 bytes in memory with the following layout:

1. The IA32_S_CET and IA32_INTERRUPT_SSP_TABLE_ADDR MSRs also control CET when CPL < 3. However, they are not managed by the XSAVE feature set and are thus not considered in this chapter.

- Bytes 7:0 are for the IA32_UINTR_HANDLER MSR.
- Bytes 15:8 are for the IA32_UINTR_STACKADJUST MSR.
- Bytes 23:16 are for the IA32_UINTR_MISC MSR with exception of the last bit (bit 7 of byte 23), which is used for UIF. (Because UIF is not part of the IA32_UINTR_MISC MSR, software that reads a value from bytes 23:16 should clear bit 63 of that 64-bit value before attempting to write it to the IA32_UINTR_MISC MSR.).
- Bytes 31:24 are for the IA32_UINTR_PD MSR.
- Bytes 39:32 are for the IA32_UINTR_RR MSR.
- Bytes 47:40 are for the IA32_UINTR_TT MSR.

As noted in Section 13.1, the XSAVE feature set manages UINTR state as supervisor state component 14. Thus, UINTR state is located in the extended region of the XSAVE area (see Section 13.4.3). As noted in Section 13.2, CPUID.0DH.0EH:EAX enumerates the size (in bytes) required for UINTR state.

UINTR state is XSAVE-managed but user interrupts are not XSAVE-enabled. The XSAVE feature set can operate on UINTR state only if the feature set is enabled (CR4.OSXSAVE = 1) and has been configured to manage UINTR state (IA32_XSS[14] = 1). Software can otherwise use user interrupts and access the MSRs (using RDMSR and WRMSR) even if the XSAVE feature set is not enabled or has not been configured to manage UINTR state.

The management of the UINTR state component by XSAVES follows the architecture of the XSAVE feature set. The following items identify points that are specific to saving the UINTR state component:

- XSAVES writes the user-interrupt registers to the user-interrupt state component using the format specified above.
- XSAVES stores zeros to bits and bytes identified above as reserved.
- The values saved for the IA32_UINTR_HANDLER, IA32_UINTR_STACKADJUST, IA32_UINTR_PD, and IA32_UINTR_TT MSRs are always canonical relative to the maximum linear-address width enumerated by CPUID[1].
- After saving the user-interrupt state component, XSAVES clears UINV. (UINV is IA32_UINTR_MISC[39:32]; XSAVES does not modify the remainder of that MSR.)

The management of the user-interrupt state component by XRSTORS follows the architecture of the XSAVE feature set. The following items identify points that are specific to restoring the user-interrupt state component:

- Before restoring the user-interrupt state component, XRSTORS verifies that UINV is 0. If it is not, XRSTORS causes a general-protection fault (#GP) before loading any part of the user-interrupt state component. (UINV is IA32_UINTR_MISC[39:32]; XRSTORS does not check the contents of the remainder of that MSR.)
- If the instruction mask and XSAVE area used by XRSTORS indicates that the user-interrupt state component should be loaded from the XSAVE area, XRSTORS reads the user-interrupt registers from the XSAVE area using the format identified above. The values read cause a general-protection fault (#GP) in any of the following cases:
  - If the value to be loaded into any one of the IA32_UINTR_HANDLER, IA32_UINTR_STACKADJUST, IA32_UINTR_PD, or IA32_UINTR_TT MSRs is not canonical relative to the maximum linear-address width enumerated by CPUID.
  - If the value to be loaded into the IA32_UINTR_MISC MSR sets any of bits 62:40. These bits are reserved in the MSR. (Bit 63 is also reserved in the MSR, but the XSAVE feature set uses bit 63 of this value for UIF.)
  - If the value to be loaded into the IA32_UINTR_ PD MSR sets any of bits 5:0. These bits are reserved in the MSR.
  - If the value to be loaded into the IA32_UINTR_ TT MSR sets any of bits 3:1. These bits are reserved in the MSR.
- If XRSTORS causes a fault or a VM exit after loading any part of the user-interrupt state component, XRSTORS clears UINV before delivering the fault or VM exit. (Other elements of user-interrupt state, including other parts of the IA32_UINTR_MISC MSR, may retain the values that were loaded by XRSTORS.)

1. They might not be canonical relative to the current paging mode if it supports only smaller linear addresses.

- After an execution of XRSTORS that loads the user-interrupt state component, the logical processor recognizes a pending user interrupt if and only if some bit is set in the IA32_UINTR_RR MSR (see Section 9.4.1 in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3A).

13.5.12 LBR State

The register state used by last-branch records (**LBR state**) comprises 101 MSRs organized as follows: IA32_LBR_CTL; IA32_LBR_DEPTH; IA32_LER_FROM_IP; IA32_LER_TO_IP; IA32_LER_INFO; and 32 triples of MSRs, IA32_LBR_*i*_FROM_IP, IA32_LBR_*i*_TO_IP, IA32_LBR_*i*_INFO, for each value of *i*, $0 \le i \le 31$.

As noted in Section 13.1, the XSAVE feature set manages LBR state as supervisor state component 15. Thus, LBR state is located in the extended region of the XSAVE area (see Section 13.4.3). As noted in Section 13.2, CPUID.0DH.0FH:EAX enumerates the size (in bytes) required for LBR state. The IA32_LBR_CTL MSR is allocated 8 bytes at byte offset 0 in the state component. The remaining MSRs are each allocated 8 bytes in the state component in the order given above. Thus, IA32_LBR_DEPTH is at byte offset 8, … , IA32_LBR_0_FROM_IP at byte offset 40, IA32_LBR_0_TO_IP at byte offset 48, IA32_LBR_0_INFO at byte offset 56, IA32_LBR_1_FROM_IP at byte offset 64, …, and IA32_LBR_31_INFO at byte offset 800. Any locations in the state component at or beyond byte offset 808 are reserved.

LBR state is XSAVE-managed but LBRs are not XSAVE-enabled. The XSAVE feature set can operate on LBR state only if the feature set is enabled (CR4.OSXSAVE = 1) and has been configured to manage LBR state (IA32_XSS[15] = 1). Software can otherwise use LBRs and access the MSRs (using RDMSR and WRMSR) even if the XSAVE feature set is not enabled or has not been configured to manage LBR state.

The following items describe special treatment of LBR state by the XSAVES and XRSTORS instructions:

- If XSAVES would save LBR state and that state is not in its initial configuration (see Section 13.6), the instruction always saves IA32_LBR_CTL, IA32_LBR_DEPTH, IA32_LER_FROM_IP, IA32_LER_TO_IP, and IA32_LER_INFO. It saves the triples IA32_LBR_*i*_FROM_IP, IA32_LBR_*i*_TO_IP, IA32_LBR_*i*_INFO, for each value of *i*, $0 \le i < D$, where D is the value of IA32_LBR_DEPTH. It will not save the values of the remaining triples, although it may access the corresponding fields in the XSAVE area.
- If XSAVES would save LBR state and that state is in its initial configuration, the instruction does not save any LBR state and will not access that component of the XSAVE area.
- If XRSTORS would initialize LBR state, IA32_LBR_DEPTH is not modified and zero is written to the other MSRs that compose LBR state.
- If XRSTORS would restore LBR state, behavior depends on the current value of IA32_LBR_DEPTH and the value of corresponding field in the XSAVE area:
  - If the current value of IA32_LBR_DEPTH equals the value of corresponding field in the XSAVE area, the instruction restores IA32_LBR_CTL, IA32_LER_FROM_IP, IA32_LER_TO_IP, IA32_LER_INFO, and the triples IA32_LBR_*i*_FROM_IP, IA32_LBR_*i*_TO_IP, IA32_LBR_*i*_INFO, for each value of *i*, $0 \le i < D$, where D is the value of IA32_LBR_DEPTH. It will not restore the values of the remaining triples, although it may access the corresponding fields in the XSAVE area.
  - If the IA32_LBR_DEPTH field in the XSAVE area sets any reserved bits, the instruction causes a general-protection exception (#GP).
  - If neither of the previous items apply, the instruction restores IA32_LBR_CTL, IA32_LER_FROM_IP, IA32_LER_TO_IP, and IA32_LER_INFO, but it writes zero to the triples IA32_LBR_*i*_FROM_IP, IA32_LBR_*i*_TO_IP, IA32_LBR_*i*_INFO, for each value of *i*, $0 \le i \le 31$. Such an execution does not modify XINUSE[15] (see Section 13.6 and Section 13.12).

13.5.13 HWP State

The register state used by hardware P-states (**HWP state**) comprises the IA32_HWP_REQUEST MSR.

As noted in Section 13.1, the XSAVE feature set manages HWP state as supervisor state component 16. Thus, HWP state is located in the extended region of the XSAVE area (see Section 13.4.3). As noted in Section 13.2, CPUID.0DH.10H:EAX enumerates the size (in bytes) required for HWP state. The IA32_HWP_REQUEST MSR is allocated 8 bytes at byte offset 0 in the state component.

HWP state is XSAVE-managed but the hardware P-states feature is not XSAVE-enabled. The XSAVE feature set can operate on HWP state only if the feature set is enabled (CR4.OSXSAVE = 1) and has been configured to manage HWP state (IA32_XSS[16] = 1). Software can otherwise use hardware P-states and access the IA32_HWP_REQUEST MSR (using RDMSR and WRMSR) even if the XSAVE feature set is not enabled or has not been configured to manage HWP state.

13.5.14 AMX State

The register state used by the Intel® Advanced Matrix Extensions (Intel AMX) comprises two state components, TILECFG and TILEDATA. Together, these two state components compose **AMX state**.

As noted in Section 13.1, the XSAVE feature set manages AMX state as state components 17–18. Thus, AMX state is located in the extended region of the XSAVE area (see Section 13.4.3). The following items detail how these state components are organized in this region:

- **TILECFG state.** As noted in Section 13.1, the XSAVE feature set manages TILECFG state as user state component 17. Thus, TILECFG state is located in the extended region of the XSAVE area (see Section 13.4.3). As noted in Section 13.2, CPUID.0DH.11H:EAX enumerates the size (in bytes) required for TILECFG state.
- **TILEDATA state.** As noted in Section 13.1, the XSAVE feature set manages TILEDATA state as user state component 18. Thus, TILEDATA state is located in the extended region of the XSAVE area (see Section 13.4.3). As noted in Section 13.2, CPUID.0DH.12H:EAX enumerates the size (in bytes) required for TILEDATA state.

Both components of AMX state are XSAVE-managed, and the AMX feature is XSAVE-enabled. The XSAVE feature set can operate on AMX state only if the feature set is enabled (CR4.OSXSAVE = 1) and has been configured to manage AMX state (XCR0[18:17] = 11b). Intel AMX instructions cannot be used unless the XSAVE feature set is enabled and has been configured to manage AMX state.

The following items describe special treatment of TILECFG and TILEDATA by the XSAVE feature set:

- Loading of TILECFG and TILEDATA by XRSTOR and XRSTORS:
  - While the LDTILECFG instruction generates a general-protection fault (#GP) if it would load the TILECFG register with an unsupported value executions of XRSTOR and XRSTORS do not do so. Instead, they initialize the register (resulting in TILES_CONFIGURED = 0).

    While executions of LDTILECFG initialize TILEDATA, executions of XRSTOR and XRSTORS do not modify TILEDATA unless loading it from memory.

    While the value of the TILECFG register can limit how Intel AMX instructions access TILEDATA, such limitations do not apply to XRSTOR and XRSTORS. An execution of either of those instructions loads all 8 KBytes of TILEDATA regardless of the value in the TILECFG register (or the value that the instruction may be loading into that register).
- Saving of TILEDATA by XSAVE, XSAVEC, XSAVEOPT, and XSAVES:
  - While the value of the TILECFG register can limit how Intel AMX instructions access TILEDATA, such limitations do not apply to XSAVE, XSAVEC, XSAVEOPT, and XSAVES. An execution of any of those instructions saves all 8 KBytes of TILEDATA regardless of the value in the TILECFG register.

13.6 PROCESSOR TRACKING OF XSAVE-MANAGED STATE

The XSAVEOPT, XSAVEC, and XSAVES instructions use two optimizations to reduce the amount of data that they write to memory. They avoid writing data for any state component known to be in its initial configuration (the **init optimization**). In addition, if either XSAVEOPT or XSAVES is using the same XSAVE area as that used by the most recent execution of XRSTOR or XRSTORS, it may avoid writing data for any state component whose configuration is known not to have been modified since then (the **modified optimization**). (XSAVE does not use these optimizations, and XSAVEC does not use the modified optimization.) The operation of XSAVEOPT, XSAVEC, and XSAVES are described in more detail in Section 13.9 through Section 13.11.

A processor can support the init and modified optimizations with special hardware that tracks the state components that might benefit from those optimizations. Other implementations might not include such hardware; such a

processor would always consider each such state component as not in its initial configuration and as modified since the last execution of XRSTOR or XRSTORS.

The following notation describes the state of the init and modified optimizations:

- XINUSE denotes the state-component bitmap corresponding to the init optimization. If XINUSE[*i*] = 0, state component *i* is known to be in its initial configuration; otherwise XINUSE[*i*] = 1. It is possible for XINUSE[*i*] to be 1 even when state component *i* is in its initial configuration. On a processor that does not support the init optimization, XINUSE[*i*] is always 1 for every value of *i*.

  Executing XGETBV with ECX = 1 returns in EDX:EAX the logical-AND of XCR0 and the current value of the XINUSE state-component bitmap. Such an execution of XGETBV always sets EAX[1] to 1 if XCR0[1] = 1 and MXCSR does not have its RESET value of 1F80H. Section 13.2 explains how software can determine whether a processor supports this use of XGETBV.
- XMODIFIED denotes the state-component bitmap corresponding to the modified optimization. If XMODIFIED[*i*] = 0, state component *i* is known not to have been modified since the most recent execution of XRSTOR or XRSTORS; otherwise XMODIFIED[*i*] = 1. It is possible for XMODIFIED[*i*] to be 1 even when state component *i* has not been modified since the most recent execution of XRSTOR or XRSTORS. On a processor that does not support the modified optimization, XMODIFIED[*i*] is always 1 for every value of *i*.

A processor that implements the modified optimization saves information about the most recent execution of XRSTOR or XRSTORS in a quantity called **XRSTOR_INFO**, a 4-tuple containing the following: (1) the CPL; (2) whether the logical processor was in VMX non-root operation; (3) the linear address of the XSAVE area; and (4) the XCOMP_BV field in the XSAVE area. An execution of XSAVEOPT or XSAVES uses the modified optimization only if that execution corresponds to XRSTOR_INFO on these four parameters.

This mechanism implies that, depending on details of the operating system, the processor might determine that an execution of XSAVEOPT by one user application corresponds to an earlier execution of XRSTOR by a different application. For this reason, Intel recommends the application software not use the XSAVEOPT instruction.

The following items specify the initial configuration each state component (for the purposes of defining the XINUSE bitmap):

- **x87 state.** x87 state is in its initial configuration if the following all hold: FCW is 037FH; FSW is 0000H; FTW is FFFFH; FCS and FDS are each 0000H; FIP and FDP are each 00000000_00000000H; each of ST0–ST7 is 0000_00000000_00000000H.
- **SSE state.** In 64-bit mode, SSE state is in its initial configuration if each of XMM0–XMM15 is 0. Outside 64-bit mode, SSE state is in its initial configuration if each of XMM0–XMM7 is 0. XINUSE[1] pertains only to the state of the XMM registers and not to MXCSR. An execution of XRSTOR or XRSTORS outside 64-bit mode does not update XMM8–XMM15. (See Section 13.13.)
- **AVX state.** In 64-bit mode, AVX state is in its initial configuration if each of YMM0_H–YMM15_H is 0. Outside 64-bit mode, AVX state is in its initial configuration if each of YMM0_H–YMM7_H is 0. An execution of XRSTOR or XRSTORS outside 64-bit mode does not update YMM8_H–YMM15_H. (See Section 13.13.)
- **BNDREGS state.** BNDREGS state is in its initial configuration if the value of each of BND0–BND3 is 0.
- **BNDCSR state.** BNDCSR state is in its initial configuration if BNDCFGU and BNDCSR each has value 0.
- **Opmask state.** Opmask state is in its initial configuration if each of the opmask registers k0–k7 is 0.
- **ZMM_Hi256 state.** In 64-bit mode, ZMM_Hi256 state is in its initial configuration if each of ZMM0_H–ZMM15_H is 0. Outside 64-bit mode, ZMM_Hi256 state is in its initial configuration if each of ZMM0_H–ZMM7_H is 0. An execution of XRSTOR or XRSTORS outside 64-bit mode does not update ZMM8_H–ZMM15_H. (See Section 13.13.)
- **Hi16_ZMM state.** In 64-bit mode, Hi16_ZMM state is in its initial configuration if each of ZMM16–ZMM31 is 0. Outside 64-bit mode, Hi16_ZMM state is always in its initial configuration. An execution of XRSTOR or XRSTORS outside 64-bit mode does not update ZMM31–ZMM31. (See Section 13.13.)
- **PT state.** PT state is in its initial configuration if each of the 9 MSRs is 0.
- **PKRU state.** PKRU state is in its initial configuration if the value of the PKRU is 0.
- **PASID state.** PASID state is in its initial configuration if the value of the IA32_PASID MSR is 0.
- **CET_U state.** CET_U state is in its initial configuration if both of the MSRs are 0.
- **CET_S state.** CET_S state is in its initial configuration if each of the three MSRs is 0.

- **HDC state.** HDC state is in its initial configuration if the value of the IA32_PM_CTL1 MSR is 1.
- **UINTR state.** UINTR state is in its initial configuration if all user-interrupt registers (including UIF) are zero.
- **LBR state.** LBR state is in its initial configuration if the value of each of the MSRs is 0, with the exception of IA32_LBR_DEPTH. XINUSE[15] does not pertain to IA32_LBR_DEPTH.
- **HWP state.** HWP state is in its initial configuration if the value of the IA32_HWP_REQUEST MSR is 8000FF01H.
- **AMX state.** AMX state is in its initial configuration if the TILECFG register is zero and all tile data are zero.

13.7 OPERATION OF XSAVE

The XSAVE instruction takes a single memory operand, which is an XSAVE area. In addition, the register pair EDX:EAX is an implicit operand used as a state-component bitmap (see Section 13.1) called the **instruction mask**. The logical-AND of XCR0 and the instruction mask is the **requested-feature bitmap** (**RFBM**) of the user state components to be saved.

The following conditions cause execution of the XSAVE instruction to generate a fault:

- If the XSAVE feature set is not enabled (CR4.OSXSAVE = 0), an invalid-opcode exception (#UD) occurs.
- If CR0.TS[bit 3] is 1, a device-not-available exception (#NM) occurs.
- If the address of the XSAVE area is not 64-byte aligned, a general-protection exception (#GP) occurs.[1]

If none of these conditions cause a fault, execution of XSAVE reads the XSTATE_BV field of the XSAVE header (see Section 13.4.2) and writes it back to memory, setting XSTATE_BV[*i*] ($0 \le i \le 63$) as follows:

- If RFBM[*i*] = 0, XSTATE_BV[*i*] is not changed.
- If RFBM[*i*] = 1, XSTATE_BV[*i*] is set to the value of XINUSE[*i*]. Section 13.6 defines XINUSE to describe the processor init optimization and specifies the initial configuration of each state component. The nature of that optimization implies the following:
  - If state component *i* is in its initial configuration, XINUSE[*i*] may be either 0 or 1, and XSTATE_BV[*i*] may be written with either 0 or 1.

    XINUSE[1] pertains only to the state of the XMM registers and not to MXCSR. Thus, XSTATE_BV[1] may be written with 0 even if MXCSR does not have its RESET value of 1F80H.
  - If state component *i* is not in its initial configuration, XINUSE[*i*] = 1 and XSTATE_BV[*i*] is written with 1.

  (As explained in Section 13.6, the initial configurations of some state components may depend on whether the processor is in 64-bit mode.)

The XSAVE instruction does not write any part of the XSAVE header other than the XSTATE_BV field; in particular, it does **not** write to the XCOMP_BV field.

Execution of XSAVE saves into the XSAVE area those state components corresponding to bits that are set in RFBM. State components 0 and 1 are located in the legacy region of the XSAVE area (see Section 13.4.1). Each state component *i*, $2 \le i \le 62$, is located in the extended region; the XSAVE instruction always uses the standard format for the extended region (see Section 13.4.3).

The MXCSR register and MXCSR_MASK are part of SSE state (see Section 13.5.2) and are thus associated with RFBM[1]. However, the XSAVE instruction also saves these values when RFBM[2] = 1 (even if RFBM[1] = 0).

See Section 13.5 for specifics for each state component and for details regarding mode-specific operation and operation determined by instruction prefixes. See Section 13.13 for details regarding faults caused by memory accesses.

13.8 OPERATION OF XRSTOR

The XRSTOR instruction takes a single memory operand, which is an XSAVE area. In addition, the register pair EDX:EAX is an implicit operand used as a state-component bitmap (see Section 13.1) called the **instruction**

1. If CR0.AM = 1, CPL = 3, and EFLAGS.AC =1, an alignment-check exception (#AC) may occur instead of #GP.

mask. The logical-AND of XCR0 and the instruction mask is the **requested-feature bitmap** (**RFBM**) of the user state components to be restored.

The following conditions cause execution of the XRSTOR instruction to generate a fault:

- If the XSAVE feature set is not enabled (CR4.OSXSAVE = 0), an invalid-opcode exception (#UD) occurs.
- If CR0.TS[bit 3] is 1, a device-not-available exception (#NM) occurs.
- If the address of the XSAVE area is not 64-byte aligned, a general-protection exception (#GP) occurs.[1]

After checking for these faults, the XRSTOR instruction reads the XCOMP_BV field in the XSAVE area's XSAVE header (see Section 13.4.2). If XCOMP_BV[63] = 0, the **standard form of XRSTOR** is executed (see Section 13.8.1); otherwise, the **compacted form of XRSTOR** is executed (see Section 13.8.2).[2]

See Section 13.2 for details of how to determine whether the compacted form of XRSTOR is supported.

13.8.1 Standard Form of XRSTOR

The standard from of XRSTOR performs additional fault checking. Either of the following conditions causes a general-protection exception (#GP):

- The XSTATE_BV field of the XSAVE header sets a bit that is not set in XCR0.
- Bytes 23:8 of the XSAVE header are not all 0 (this implies that all bits in XCOMP_BV are 0).[3]

If none of these conditions cause a fault, the processor updates each state component *i* for which RFBM[*i*] = 1. XRSTOR updates state component *i* based on the value of bit *i* in the XSTATE_BV field of the XSAVE header:

- If XSTATE_BV[*i*] = 0, the state component is set to its initial configuration. Section 13.6 specifies the initial configuration of each state component.

  The initial configuration of state component 1 pertains only to the XMM registers and not to MXCSR. See below for the treatment of MXCSR
- If XSTATE_BV[*i*] = 1, the state component is loaded with data from the XSAVE area. See Section 13.5 for specifics for each state component and for details regarding mode-specific operation and operation determined by instruction prefixes. See Section 13.13 for details regarding faults caused by memory accesses.

  State components 0 and 1 are located in the legacy region of the XSAVE area (see Section 13.4.1). Each state component *i*, $2 \le i \le 62$, is located in the extended region; the standard form of XRSTOR uses the standard format for the extended region (see Section 13.4.3).

The MXCSR register is part of state component 1, SSE state (see Section 13.5.2). However, the standard form of XRSTOR loads the MXCSR register from memory whenever the RFBM[1] (SSE) or RFBM[2] (AVX) is set, regardless of the values of XSTATE_BV[1] and XSTATE_BV[2]. The standard form of XRSTOR causes a general-protection exception (#GP) if it would load MXCSR with an illegal value.

13.8.2 Compacted Form of XRSTOR

The compacted from of XRSTOR performs additional fault checking. Any of the following conditions causes a #GP:

- The XCOMP_BV field of the XSAVE header sets a bit in the range 62:0 that is not set in XCR0.
- The XSTATE_BV field of the XSAVE header sets a bit (including bit 63) that is not set in XCOMP_BV.
- Bytes 63:16 of the XSAVE header are not all 0.

If none of these conditions cause a fault, the processor updates each state component *i* for which RFBM[*i*] = 1. XRSTOR updates state component *i* based on the value of bit *i* in the XSTATE_BV field of the XSAVE header:

---

1. If CR0.AM = 1, CPL = 3, and EFLAGS.AC =1, an alignment-check exception (#AC) may occur instead of #GP.
2. If the processor does not support the compacted form of XRSTOR, it may execute the standard form of XRSTOR without first reading the XCOMP_BV field. A processor supports the compacted form of XRSTOR only if it enumerates CPUID.0DH.01H:EAX[1] as 1.
3. Bytes 63:24 of the XSAVE header are also reserved. Software should ensure that bytes 63:16 of the XSAVE header are all 0 in any XSAVE area. (Bytes 15:8 should also be 0 if the XSAVE area is to be used on a processor that does not support the compaction extensions to the XSAVE feature set.)

- If XSTATE_BV[*i*] = 0, the state component is set to its initial configuration. Section 13.6 specifies the initial configuration of each state component.

  If XSTATE_BV[1] = 0, the compacted form XRSTOR initializes MXCSR to 1F80H. (This differs from the standard from of XRSTOR, which loads MXCSR from the XSAVE area whenever either RFBM[1] or RFBM[2] is set.)

  State component *i* is set to its initial configuration as indicated above if RFBM[*i*] = 1 and XSTATE_BV[*i*] = 0 — **even if XCOMP_BV[*i*] = 0**. This is true for all values of *i*, including 0 (x87 state) and 1 (SSE state).
- If XSTATE_BV[*i*] = 1, the state component is loaded with data from the XSAVE area.[1] See Section 13.5 for specifics for each state component and for details regarding mode-specific operation and operation determined by instruction prefixes. See Section 13.13 for details regarding faults caused by memory accesses.

  State components 0 and 1 are located in the legacy region of the XSAVE area (see Section 13.4.1). Each state component *i*, $2 \le i \le 62$, is located in the extended region; the compacted form of the XRSTOR instruction uses the compacted format for the extended region (see Section 13.4.3).

The MXCSR register is part of SSE state (see Section 13.5.2) and is thus loaded from memory if RFBM[1] = XSTATE_BV[*i*] = 1. The compacted form of XRSTOR does not consider RFBM[2] (AVX) when determining whether to update MXCSR. (This is a difference from the standard form of XRSTOR.) The compacted form of XRSTOR causes a general-protection exception (#GP) if it would load MXCSR with an illegal value.

13.8.3 XRSTOR and the Init and Modified Optimizations

Execution of the XRSTOR instruction causes the processor to update its tracking for the init and modified optimizations (see Section 13.6). The following items provide details:

- The processor updates its tracking for the init optimization as follows:
  - If RFBM[*i*] = 0, XINUSE[*i*] is not changed.
  - If RFBM[*i*] = 1 and XSTATE_BV[*i*] = 0, state component *i* may be tracked as init; XINUSE[*i*] may be set to 0 or 1. (As noted in Section 13.6, a processor need not implement the init optimization for state component *i*; a processor that does not do so implicitly maintains XINUSE[*i*] = 1 at all times.)
  - If RFBM[*i*] = 1 and XSTATE_BV[*i*] = 1, state component *i* is tracked as not init; XINUSE[*i*] is set to 1.
- The processor updates its tracking for the modified optimization and records information about the XRSTOR execution for future interaction with the XSAVEOPT and XSAVES instructions (see Section 13.9 and Section 13.11) as follows:
  - If RFBM[*i*] = 0, state component *i* is tracked as modified; XMODIFIED[*i*] is set to 1.
  - If RFBM[*i*] = 1, state component *i* may be tracked as unmodified; XMODIFIED[*i*] may be set to 0 or 1. (As noted in Section 13.6, a processor need not implement the modified optimization for state component *i*; a processor that does not do so implicitly maintains XMODIFIED[*i*] = 1 at all times.)
  - XRSTOR_INFO is set to the 4-tuple $\langle w,x,y,z \rangle$, where w is the CPL (0); x is 1 if the logical processor is in VMX non-root operation and 0 otherwise; *y* is the linear address of the XSAVE area; and *z* is XCOMP_BV. In particular, the standard form of XRSTOR always sets *z* to all zeroes, while the compacted form of XRSTORS never does so (because it sets at least bit 63 to 1).

Note that, if RFBM is entirely zero (e.g., because the instruction mask in EDX:EAX is zero), no state components are modified, the XINUSE bitmap is not modified, and all bits are set in the XMODIFIED bitmap. Thus, if EDX:EAX was zero for the most recent execution of XRSTOR, an execution of XSAVEOPT or XSAVES will identify all state components as modified and will thus not use the modified optimization.

13.9 OPERATION OF XSAVEOPT

The operation of XSAVEOPT is similar to that of XSAVE. Unlike XSAVE, XSAVEOPT uses the init optimization (by which it may omit saving state components that are in their initial configuration) and the modified optimization (by

1. Earlier fault checking ensured that, if the instruction has reached this point in execution and XSTATE_BV[*i*] is 1, then XCOMP_BV[*i*] is also 1.

which it may omit saving state components that have not been modified since the last execution of XRSTOR); see Section 13.6. See Section 13.2 for details of how to determine whether XSAVEOPT is supported.

The XSAVEOPT instruction takes a single memory operand, which is an XSAVE area. In addition, the register pair EDX:EAX is an implicit operand used as a state-component bitmap (see Section 13.1) called the **instruction mask**. The logical (bitwise) AND of XCR0 and the instruction mask is the **requested-feature bitmap** (**RFBM**) of the user state components to be saved.

The following conditions cause execution of the XSAVEOPT instruction to generate a fault:

- If the XSAVE feature set is not enabled (CR4.OSXSAVE = 0), an invalid-opcode exception (#UD) occurs.
- If CR0.TS[bit 3] is 1, a device-not-available exception (#NM) occurs.
- If the address of the XSAVE area is not 64-byte aligned, a general-protection exception (#GP) occurs.[1]

If none of these conditions cause a fault, execution of XSAVEOPT reads the XSTATE_BV field of the XSAVE header (see Section 13.4.2) and writes it back to memory, setting XSTATE_BV[*i*] ($0 \le i \le 63$) as follows:

- If RFBM[*i*] = 0, XSTATE_BV[*i*] is not changed.
- If RFBM[*i*] = 1, XSTATE_BV[*i*] is set to the value of XINUSE[*i*]. Section 13.6 defines XINUSE to describe the processor init optimization and specifies the initial configuration of each state component. The nature of that optimization implies the following:
  - If the state component is in its initial configuration, XINUSE[*i*] may be either 0 or 1, and XSTATE_BV[*i*] may be written with either 0 or 1.

    XINUSE[1] pertains only to the state of the XMM registers and not to MXCSR. Thus, XSTATE_BV[1] may be written with 0 even if MXCSR does not have its RESET value of 1F80H.
  - If the state component is not in its initial configuration, XSTATE_BV[*i*] is written with 1.

  (As explained in Section 13.6, the initial configurations of some state components may depend on whether the processor is in 64-bit mode.)

The XSAVEOPT instruction does not write any part of the XSAVE header other than the XSTATE_BV field; in particular, it does not write to the XCOMP_BV field.

Execution of XSAVEOPT saves into the XSAVE area those state components corresponding to bits that are set in RFBM (subject to the optimizations described below). State components 0 and 1 are located in the legacy region of the XSAVE area (see Section 13.4.1). Each state component *i*, $2 \le i \le 62$, is located in the extended region; the XSAVEOPT instruction always uses the standard format for the extended region (see Section 13.4.3).

See Section 13.5 for specifics for each state component and for details regarding mode-specific operation and operation determined by instruction prefixes. See Section 13.13 for details regarding faults caused by memory accesses.

Execution of XSAVEOPT performs two optimizations that reduce the amount of data written to memory:

- **Init optimization.**
  If XINUSE[*i*] = 0, state component *i* is not saved to the XSAVE area (even if RFBM[*i*] = 1). (See below for exceptions made for MXCSR.)
- **Modified optimization.**
  Each execution of XRSTOR and XRSTORS establishes XRSTOR_INFO as a 4-tuple $\langle w,x,y,z \rangle$ (see Section 13.8.3 and Section 13.12). Execution of XSAVEOPT uses the modified optimization only if the following all hold for the current value of XRSTOR_INFO:
  - *w* = CPL;
  - *x* = 1 if and only if the logical processor is in VMX non-root operation;
  - *y* is the linear address of the XSAVE area being used by XSAVEOPT; and
  - *z* is 00000000_00000000H. (This last item implies that XSAVEOPT does not use the modified optimization if the last execution of XRSTOR used the compacted form, or if an execution of XRSTORS followed the last execution of XRSTOR.)

1. If CR0.AM = 1, CPL = 3, and EFLAGS.AC =1, an alignment-check exception (#AC) may occur instead of #GP.

If XSAVEOPT uses the modified optimization and XMODIFIED[*i*] = 0 (see Section 13.6), state component *i* is not saved to the XSAVE area.

(In practice, the benefit of the modified optimization for state component *i* depends on how the processor is tracking state component *i*; see Section 13.6. Limitations on the tracking ability may result in state component *i* being saved even though is in the same configuration that was loaded by the previous execution of XRSTOR.)

Depending on details of the operating system, an execution of XSAVEOPT by a user application might use the modified optimization when the most recent execution of XRSTOR was by a different application. Because of this, Intel recommends the application software not use the XSAVEOPT instruction.

The MXCSR register and MXCSR_MASK are part of SSE state (see Section 13.5.2) and are thus associated with bit 1 of RFBM. However, the XSAVEOPT instruction also saves these values when RFBM[2] = 1 (even if RFBM[1] = 0). The init and modified optimizations do not apply to the MXCSR register and MXCSR_MASK.

13.10 OPERATION OF XSAVEC

The operation of XSAVEC is similar to that of XSAVE. Two main differences are (1) XSAVEC uses the compacted format for the extended region of the XSAVE area; and (2) XSAVEC uses the init optimization (see Section 13.6). Unlike XSAVEOPT, XSAVEC does not use the modified optimization. See Section 13.2 for details of how to determine whether XSAVEC is supported.

The XSAVEC instruction takes a single memory operand, which is an XSAVE area. In addition, the register pair EDX:EAX is an implicit operand used as a state-component bitmap (see Section 13.1) called the **instruction mask**. The logical (bitwise) AND of XCR0 and the instruction mask is the **requested-feature bitmap** (**RFBM**) of the user state components to be saved.

The following conditions cause execution of the XSAVEC instruction to generate a fault:

- If the XSAVE feature set is not enabled (CR4.OSXSAVE = 0), an invalid-opcode exception (#UD) occurs.
- If CR0.TS[bit 3] is 1, a device-not-available exception (#NM) occurs.
- If the address of the XSAVE area is not 64-byte aligned, a general-protection exception (#GP) occurs.[1]

If none of these conditions cause a fault, execution of XSAVEC writes the XSTATE_BV field of the XSAVE header (see Section 13.4.2), setting XSTATE_BV[*i*] ($0 \le i \le 63$) as follows:[2]

- If RFBM[*i*] = 0, XSTATE_BV[*i*] is written as 0.
- If RFBM[*i*] = 1, XSTATE_BV[*i*] is set to the value of XINUSE[*i*] (see below for an exception made for XSTATE_BV[1]). Section 13.6 defines XINUSE to describe the processor init optimization and specifies the initial configuration of each state component. The nature of that optimization implies the following:
  - If state component *i* is in its initial configuration, XSTATE_BV[*i*] may be written with either 0 or 1.
  - If state component *i* is not in its initial configuration, XSTATE_BV[*i*] is written with 1.

  XINUSE[1] pertains only to the state of the XMM registers and not to MXCSR. However, if RFBM[1] = 1 and MXCSR does not have the value 1F80H, XSAVEC writes XSTATE_BV[1] as 1 even if XINUSE[1] = 0.

  (As explained in Section 13.6, the initial configurations of some state components may depend on whether the processor is in 64-bit mode.)

The XSAVEC instructions sets bit 63 of the XCOMP_BV field of the XSAVE header while writing RFBM[62:0] to XCOMP_BV[62:0]. The XSAVEC instruction does not write any part of the XSAVE header other than the XSTATE_BV and XCOMP_BV fields.

Execution of XSAVEC saves into the XSAVE area those state components corresponding to bits that are set in RFBM (subject to the init optimization described below). State components 0 and 1 are located in the legacy region of the XSAVE area (see Section 13.4.1). Each state component *i*, $2 \le i \le 62$, is located in the extended region; the XSAVEC instruction always uses the compacted format for the extended region (see Section 13.4.3).

1. If CR0.AM = 1, CPL = 3, and EFLAGS.AC =1, an alignment-check exception (#AC) may occur instead of #GP.
2. Unlike the XSAVE and XSAVEOPT instructions, the XSAVEC instruction does **not** read the XSTATE_BV field of the XSAVE header.

See Section 13.5 for specifics for each state component and for details regarding mode-specific operation and operation determined by instruction prefixes. See Section 13.13 for details regarding faults caused by memory accesses.

Execution of XSAVEC performs the init optimization to reduce the amount of data written to memory. If XINUSE[*i*] = 0, state component *i* is not saved to the XSAVE area (even if RFBM[*i*] = 1). However, if RFBM[1] = 1 and MXCSR does not have the value 1F80H, XSAVEC saves all of state component 1 (SSE — including the XMM registers) even if XINUSE[1] = 0. Unlike the XSAVE instruction, RFBM[2] does not determine whether XSAVEC saves MXCSR and MXCSR_MASK.

13.11 OPERATION OF XSAVES

The operation of XSAVES is similar to that of XSAVEC. The main differences are (1) XSAVES can be executed only if CPL = 0; (2) XSAVES can operate on the state components whose bits are set in XCR0 | IA32_XSS and can thus operate on supervisor state components; and (3) XSAVES uses the modified optimization (see Section 13.6). See Section 13.2 for details of how to determine whether XSAVES is supported.

The XSAVES instruction takes a single memory operand, which is an XSAVE area. In addition, the register pair EDX:EAX is an implicit operand used as a state-component bitmap (see Section 13.1) called the **instruction mask**. EDX:EAX & (XCR0 | IA32_XSS) (the logical AND the instruction mask with the logical OR of XCR0 and IA32_XSS) is the **requested-feature bitmap** (**RFBM**) of the state components to be saved.

The following conditions cause execution of the XSAVES instruction to generate a fault:

- If the XSAVE feature set is not enabled (CR4.OSXSAVE = 0), an invalid-opcode exception (#UD) occurs.
- If CR0.TS[bit 3] is 1, a device-not-available exception (#NM) occurs.
- If CPL > 0 or if the address of the XSAVE area is not 64-byte aligned, a general-protection exception (#GP) occurs.

If none of these conditions cause a fault, execution of XSAVES writes the XSTATE_BV field of the XSAVE header (see Section 13.4.2), setting XSTATE_BV[*i*] ($0 \le i \le 63$) as follows:

- If RFBM[*i*] = 0, XSTATE_BV[*i*] is written as 0.
- If RFBM[*i*] = 1, XSTATE_BV[*i*] is set to the value of XINUSE[*i*] (see below for an exception made for XSTATE_BV[1]). Section 13.6 defines XINUSE to describe the processor init optimization and specifies the initial configuration of each state component. The nature of that optimization implies the following:
  - If state component *i* is in its initial configuration, XSTATE_BV[*i*] may be written with either 0 or 1.
  - If state component *i* is not in its initial configuration, XSTATE_BV[*i*] is written with 1.

  XINUSE[1] pertains only to the state of the XMM registers and not to MXCSR. However, if RFBM[1] = 1 and MXCSR does not have the value 1F80H, XSAVES writes XSTATE_BV[1] as 1 even if XINUSE[1] = 0.

  (As explained in Section 13.6, the initial configurations of some state components may depend on whether the processor is in 64-bit mode.)

The XSAVES instructions sets bit 63 of the XCOMP_BV field of the XSAVE header while writing RFBM[62:0] to XCOMP_BV[62:0]. The XSAVES instruction does not write any part of the XSAVE header other than the XSTATE_BV and XCOMP_BV fields.

Execution of XSAVES saves into the XSAVE area those state components corresponding to bits that are set in RFBM (subject to the optimizations described below). State components 0 and 1 are located in the legacy region of the XSAVE area (see Section 13.4.1). Each state component *i*, $2 \le i \le 62$, is located in the extended region; the XSAVES instruction always uses the compacted format for the extended region (see Section 13.4.3).

See Section 13.5 for specifics for each state component and for details regarding mode-specific operation and operation determined by instruction prefixes; in particular, see Section 13.5.6, Section 13.5.11, Section 13.5.12, and Section 13.5.14 for special treatment by XSAVES of PT state, UINTR state, LBR state, and AMX state, respectively. See Section 13.13 for details regarding faults caused by memory accesses.

Execution of XSAVES performs the init optimization to reduce the amount of data written to memory. If XINUSE[*i*] = 0, state component *i* is not saved to the XSAVE area (even if RFBM[*i*] = 1). However, if RFBM[1] = 1

and MXCSR does not have the value 1F80H, XSAVES saves all of state component 1 (SSE — including the XMM registers) even if XINUSE[1] = 0.

Like XSAVEOPT, XSAVES may perform the modified optimization. Each execution of XRSTOR and XRSTORS establishes XRSTOR_INFO as a 4-tuple ⟨*w*,*x*,*y*,*z*⟩ (see Section 13.8.3 and Section 13.12). Execution of XSAVES uses the modified optimization only if the following all hold:

- *w* = CPL;
- *x* = 1 if and only if the logical processor is in VMX non-root operation;
- *y* is the linear address of the XSAVE area being used by XSAVEOPT; and
- *z*[63] is 1 and *z*[62:0] = RFBM[62:0]. (This last item implies that XSAVES does not use the modified optimization if the last execution of XRSTOR used the standard form and followed the last execution of XRSTORS.)

If XSAVES uses the modified optimization and XMODIFIED[*i*] = 0 (see Section 13.6), state component *i* is not saved to the XSAVE area.

13.12 OPERATION OF XRSTORS

The operation of XRSTORS is similar to that of XRSTOR. Three main differences are (1) XRSTORS can be executed only if CPL = 0; (2) XRSTORS can operate on the state components whose bits are set in XCR0 | IA32_XSS and can thus operate on supervisor state components; and (3) XRSTORS has only a compacted form (no standard form; see Section 13.8). See Section 13.2 for details of how to determine whether XRSTORS is supported.

The XRSTORS instruction takes a single memory operand, which is an XSAVE area. In addition, the register pair EDX:EAX is an implicit operand used as a state-component bitmap (see Section 13.1) called the **instruction mask**. EDX:EAX & (XCR0 | IA32_XSS) (the logical AND the instruction mask with the logical OR of XCR0 and IA32_XSS) is the **requested-feature bitmap** (**RFBM**) of the state components to be restored.

The following conditions cause execution of the XRSTOR instruction to generate a fault:

- If the XSAVE feature set is not enabled (CR4.OSXSAVE = 0), an invalid-opcode exception (#UD) occurs.
- If CR0.TS[bit 3] is 1, a device-not-available exception (#NM) occurs.
- If CPL > 0 or if the address of the XSAVE area is not 64-byte aligned, a general-protection exception (#GP) occurs.

After checking for these faults, the XRSTORS instruction reads the first 64 bytes of the XSAVE header, including the XSTATE_BV and XCOMP_BV fields (see Section 13.4.2). A #GP occurs if any of the following conditions hold for the values read:

- XCOMP_BV[63] = 0.
- XCOMP_BV sets a bit in the range 62:0 that is not set in XCR0 | IA32_XSS.
- XSTATE_BV sets a bit (including bit 63) that is not set in XCOMP_BV.
- Bytes 63:16 of the XSAVE header are not all 0.

If none of these conditions cause a fault, the processor updates each state component *i* for which RFBM[*i*] = 1. XRSTORS updates state component *i* based on the value of bit *i* in the XSTATE_BV field of the XSAVE header:

- If XSTATE_BV[*i*] = 0, the state component is set to its initial configuration. Section 13.6 specifies the initial configuration of each state component. If XSTATE_BV[1] = 0, XRSTORS initializes MXCSR to 1F80H.

  State component *i* is set to its initial configuration as indicated above if RFBM[*i*] = 1 and XSTATE_BV[*i*] = 0 — **even if XCOMP_BV[*i*] = 0**. This is true for all values of *i*, including 0 (x87 state) and 1 (SSE state).
- If XSTATE_BV[*i*] = 1, the state component is loaded with data from the XSAVE area.[1] See Section 13.5 for specifics for each state component and for details regarding mode-specific operation and operation determined by instruction prefixes; in particular, see Section 13.5.6 and Section 13.5.12 for special treatment by XRSTORS of PT state and LBR state, respectively. See Section 13.13 for details regarding faults caused by memory accesses.

1. Earlier fault checking ensured that, if the instruction has reached this point in execution and XSTATE_BV[*i*] is 1, then XCOMP_BV[*i*] is also 1.

If XRSTORS is restoring a supervisor state component, the instruction causes a general-protection exception (#GP) if it would load any element of that component with an unsupported value (e.g., by setting a reserved bit in an MSR) or if a bit is set in any reserved portion of the state component in the XSAVE area.

State components 0 and 1 are located in the legacy region of the XSAVE area (see Section 13.4.1). Each state component *i*, $2 \le i \le 62$, is located in the extended region; XRSTORS uses the compacted format for the extended region (see Section 13.4.3).

The MXCSR register is part of SSE state (see Section 13.5.2) and is thus loaded from memory if RFBM[1] = XSTATE_BV[*i*] = 1. XRSTORS causes a general-protection exception (#GP) if it would load MXCSR with an illegal value.

If an execution of XRSTORS causes an exception or a VM exit during or after restoring a supervisor state component, each element of that state component may have the value it held before the XRSTORS execution, the value loaded from the XSAVE area, or the element's initial value (as defined in Section 13.6). See Section 13.5.6 for some special treatment of PT state for the case in which XRSTORS causes an exception or a VM exit.

Like XRSTOR, execution of XRSTORS causes the processor to update is tracking for the init and modified optimizations (see Section 13.6 and Section 13.8.3). The following items provide details:

- The processor updates its tracking for the init optimization as follows:
  - If RFBM[*i*] = 0, XINUSE[*i*] is not changed.
  - If RFBM[*i*] = 1 and XSTATE_BV[*i*] = 0, state component *i* may be tracked as init; XINUSE[*i*] may be set to 0 or 1.
  - If RFBM[*i*] = 1 and XSTATE_BV[*i*] = 1, state component *i* is tracked as not init; XINUSE[*i*] is set to 1.[1]
- The processor updates its tracking for the modified optimization and records information about the XRSTORS execution for future interaction with the XSAVEOPT and XSAVES instructions as follows:
  - If RFBM[*i*] = 0, state component *i* is tracked as modified; XMODIFIED[*i*] is set to 1.
  - If RFBM[*i*] = 1, state component *i* may be tracked as unmodified; XMODIFIED[*i*] may be set to 0 or 1.
  - XRSTOR_INFO is set to the 4-tuple $\langle w,x,y,z \rangle$, where *w* is the CPL; *x* is 1 if the logical processor is in VMX non-root operation and 0 otherwise; *y* is the linear address of the XSAVE area; and *z* is XCOMP_BV (this implies that *z*[63] = 1).

Note that, if RFBM is entirely zero (e.g., because the instruction mask in EDX:EAX is zero), no state components are modified, the XINUSE bitmap is not modified, and all bits are set in the XMODIFIED bitmap. Thus, if EDX:EAX was zero for the most recent execution of XRSTORS, an execution of XSAVEOPT or XSAVES will identify all state components as modified and will thus not use the modified optimization.

13.13 MEMORY ACCESSES BY THE XSAVE FEATURE SET

Each instruction in the XSAVE feature set operates on a set of XSAVE-managed state components. The specific set of components on which an instruction operates is determined by the values of XCR0, the IA32_XSS MSR, EDX:EAX, and (for XRSTOR and XRSTORS) the XSAVE header.

Section 13.4 provides the details necessary to determine the location of each state component for any execution of an instruction in the XSAVE feature set. An execution of an instruction in the XSAVE feature set may access any byte of any state component on which that execution operates even when saving a state component is omitted because it is in its initial configuration; when restoring a state component to its initial configuration; or when XFD is enabled for the state components (see Section 13.14).

Section 13.5 provides details of the different XSAVE-managed state components. Some portions of some of these components are accessible only in 64-bit mode. Executions of XRSTOR and XRSTORS outside 64-bit mode will not update those portions; executions of XSAVE, XSAVEC, XSAVEOPT, and XSAVES will not modify the corresponding locations in memory.

1. For LBR state (state component 15), XRSTORS may leave XINUSE[15] unmodified in certain situations even if RFBM[15] = 1 = XSTATE_BV[15] = 1. See Section 13.5.12.

Despite this fact, any execution of these instructions outside 64-bit mode may access any byte in any state component on which that execution operates — even those at addresses corresponding to registers that are accessible only in 64-bit mode. As a result, such an execution may incur a fault due to an attempt to access such an address.

For example, an execution of XSAVE outside 64-bit mode may incur a page fault if paging does not map as read/write the section of the XSAVE area containing state component 7 (Hi16_ZMM state) — despite the fact that state component 7 can be accessed only in 64-bit mode.

13.14 EXTENDED FEATURE DISABLE (XFD)

Extended feature disable (**XFD**) is an extension to the XSAVE feature set that allows an operating system to enable a feature while preventing specific user threads from using the feature. This section describes XFD.

As noted in Section 13.2, a processor that supports XFD enumerates CPUID.0DH.01H:EAX[4] as 1. Such a processor supports two new MSRs: IA32_XFD (MSR address 1C4H) and IA32_XFD_ERR (MSR address 1C5H). Each of these MSRs contains a state-component bitmap. Bit *i* of either MSR can be set to 1 only if CPUID.0DH.*i*:ECX[2] is enumerated as 1 (see Section 13.2). An execution of WRMSR that attempts to set an unsupported bit in either MSR causes a general-protection fault (#GP). The reset values of both of these MSRs are zero.

XFD is enabled for state component *i* if XCR0[*i*] = IA32_XFD[*i*] = 1. (IA32_XFD[*i*] does not affect processor operations if XCR0[*i*] = 0.) In compacted format, the IA32_XFD MSR does not impact the computation of XCOMP_BV by the XSAVEC or XSAVES instructions and thus does not impact the format of the extended region of the XSAVE area. When XFD is enabled for a state component, any instruction that would access that state component does not execute and instead generates a device-not-available exception (#NM).

Exceptions are made for certain instructions (including those that initialize the state component). The following items provide details:

- LDTILECFG and TILERELEASE initialize the TILEDATA state component. An execution of either of these instructions does not generate #NM when XCR0[18] = IA32_XFD[18] = 1; instead, it initializes TILEDATA normally. (Note that STTILECFG does not use the TILEDATA state component. Thus, an execution of this instruction does not generate #NM when XCR0[18] = IA32_XFD[18] = 1.)
- If XRSTOR or XRSTORS is loading state component *i* and bit *i* of the XSTATE_BV field of the XSAVE header is 0, the instruction does not generate #NM when XCR0[*i*] = IA32_XFD[*i*] = 1; instead, it initializes the state component normally. (If bit *i* of the XSTATE_BV field of the XSAVE header is 1, the instruction does generate #NM.)
- If XSAVE, XSAVEC, XSAVEOPT, or XSAVES is saving the state component *i*, the instruction does not generate #NM when XCR0[*i*] = IA32_XFD[*i*] = 1; instead, it operates as if XINUSE[i] = 0 (and the state component was in its initial state): it saves bit *i* of XSTATE_BV field of the XSAVE header as 0; in addition, XSAVE saves the initial configuration of the state component (the other instructions do not save state component *i*).
- Enclave entry instructions (ENCLU[EENTER] and ENCLU[ERESUME]) generate #NM if XCR0[*i*] = IA32_XFD[*i*] = 1 and bit *i* is set in the XFRM field in the attributes of the enclave being entered.

When XFD causes an instruction to generate #NM, the processor loads the IA32_XFD_ERR MSR to identify the disabled state component(s). Specifically, the MSR is loaded with the logical AND of the IA32_XFD MSR and the bitmap corresponding to the state component(s) required by the faulting instruction.

Device-not-available exceptions that are not due to XFD — those resulting from setting CR0.TS to 1 — do not modify the IA32_XFD_ERR MSR.

CHAPTER 14
PROGRAMMING WITH INTEL® AVX, FMA, AND INTEL® AVX2

Intel® Advanced Vector Extensions (Intel® AVX) introduces 256-bit vector processing capability. The Intel AVX instruction set extends 128-bit SIMD instruction sets by employing a new instruction encoding scheme via a vector extension prefix (VEX). Intel AVX also offers several enhanced features beyond those available in prior generations of 128-bit SIMD extensions.

FMA (Fused Multiply Add) extensions enhances Intel AVX further in floating-point numeric computations. FMA provides high-throughput, arithmetic operations cover fused multiply-add, fused multiply-subtract, fused multiply add/subtract interleave, signed-reversed multiply on fused multiply-add and multiply-subtract.

Intel® Advanced Vector Extensions 2 (Intel® AVX2) provides 256-bit integer SIMD extensions that accelerate computation across integer and floating-point domains using 256-bit vector registers.

This chapter summarizes the key features of Intel AVX, FMA, and Intel AVX2.

14.1 INTEL® AVX OVERVIEW

Intel AVX introduces the following architectural enhancements:

- Support for 256-bit wide vectors with the YMM vector register set.
- 256-bit floating-point instruction set enhancement with up to 2X performance gain relative to 128-bit Streaming SIMD extensions.
- Enhancement of legacy 128-bit SIMD instruction extensions to support three-operand syntax and to simplify compiler vectorization of high-level language expressions.
- VEX prefix-encoded instruction syntax support for generalized three-operand syntax to improve instruction programming flexibility and efficient encoding of new instruction extensions.
- Most VEX-encoded 128-bit and 256-bit AVX instructions (with both load and computational operation semantics) are not restricted to 16-byte or 32-byte memory alignment.
- Support flexible deployment of 256-bit AVX code, 128-bit AVX code, legacy 128-bit code and scalar code.

With the exception of SIMD instructions operating on MMX registers, almost all legacy 128-bit SIMD instructions have AVX equivalents that support three operand syntax. 256-bit AVX instructions employ three-operand syntax and some with 4-operand syntax.

14.1.1 256-Bit Wide SIMD Register Support

Intel AVX introduces support for 256-bit wide SIMD registers (YMM0-YMM7 in operating modes that are 32-bit or less, YMM0-YMM15 in 64-bit mode). The lower 128-bits of the YMM registers are aliased to the respective 128-bit XMM registers.

Legacy SSE instructions (i.e., SIMD instructions operating on XMM state but not using the VEX prefix, also referred to non-VEX encoded SIMD instructions) will not access the upper bits beyond bit 128 of the YMM registers. AVX instructions with a VEX prefix and vector length of 128-bits zeroes the upper bits (above bit 128) of the YMM register.

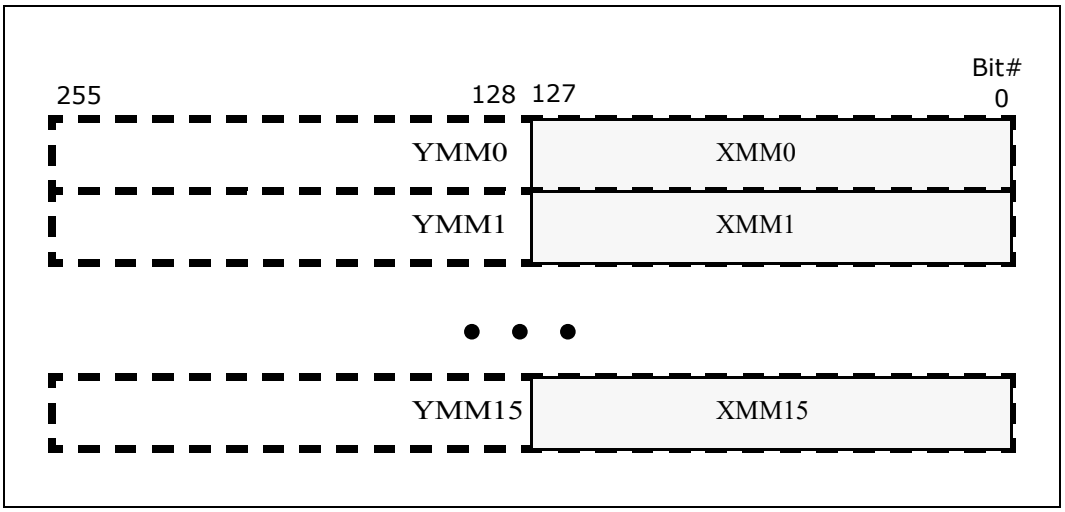


Figure 14-1. 256-Bit Wide SIMD Register

14.1.2 Instruction Syntax Enhancements

Intel AVX employs an instruction encoding scheme using a new prefix (known as "VEX" prefix). Instruction encoding using the VEX prefix can directly encode a register operand within the VEX prefix. This support two new instruction syntax in Intel 64 architecture:

- A non-destructive operand (in a three-operand instruction syntax): The non-destructive source reduces the number of registers, register-register copies and explicit load operations required in typical SSE loops, reduces code size, and improves micro-fusion opportunities.
- A third source operand (in a four-operand instruction syntax) via the upper 4 bits in an 8-bit immediate field. Support for the third source operand is defined for selected instructions (e.g., VBLENDVPD, VBLENDVPS, PBLENDVB).

Two-operand instruction syntax previously expressed in legacy SSE instruction as

```
ADDPS xmm1, xmm2/m128
```

128-bit AVX equivalent can be expressed in three-operand syntax as

```
VADDPS xmm1, xmm2, xmm3/m128
```

In four-operand syntax, the extra register operand is encoded in the immediate byte.

Note SIMD instructions supporting three-operand syntax but processing only 128-bits of data are considered part of the 256-bit SIMD instruction set extensions of AVX, because bits 255:128 of the destination register are zeroed by the processor.

14.1.3 VEX Prefix Instruction Encoding Support

Intel AVX introduces a new prefix, referred to as VEX, in the Intel 64 and IA-32 instruction encoding format. Instruction encoding using the VEX prefix provides the following capabilities:

- Direct encoding of a register operand within VEX. This provides instruction syntax support for non-destructive source operand.
- Efficient encoding of instruction syntax operating on 128-bit and 256-bit register sets.

- Compaction of REX prefix functionality: The equivalent functionality of the REX prefix is encoded within VEX.
- Compaction of SIMD prefix functionality and escape byte encoding: The functionality of SIMD prefix (66H, F2H, F3H) on opcode is equivalent to an opcode extension field to introduce new processing primitives. This functionality is replaced by a more compact representation of opcode extension within the VEX prefix. Similarly, the functionality of the escape opcode byte (0FH) and two-byte escape (0F38H, 0F3AH) are also compacted within the VEX prefix encoding.
- Most VEX-encoded SIMD numeric and data processing instruction semantics with memory operand have relaxed memory alignment requirements than instructions encoded using SIMD prefixes (see Section 14.9).

VEX prefix encoding applies to SIMD instructions operating on YMM registers, XMM registers, and in some cases with a general-purpose register as one of the operand. VEX prefix is not supported for instructions operating on MMX or x87 registers. Details of VEX prefix and instruction encoding are discussed in Chapter 2, "Instruction Format," of Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 2A.

14.2 FUNCTIONAL OVERVIEW

Intel AVX provides comprehensive functional improvements over previous generations of SIMD instruction extensions. The functional improvements include:

- 256-bit floating-point arithmetic primitives: Intel AVX enhances existing 128-bit floating-point arithmetic instructions with 256-bit capabilities for floating-point processing. Table 14-1 lists SIMD instructions promoted to Intel AVX.
- Enhancements for flexible SIMD data movements: Intel AVX provides a number of new data movement primitives to enable efficient SIMD programming in relation to loading non-unit-strided data into SIMD registers, intra-register SIMD data manipulation, conditional expression and branch handling, etc. Enhancements for SIMD data movement primitives cover 256-bit and 128-bit vector floating-point data, and across 128-bit integer SIMD data processing using VEX-encoded instructions.

Table 14-1. Promoted SSSE3 and Intel® SSE, SSE2, SSE3, and SSE4 Instructions

VEX.256 Encoding	VEX.128 Encoding	Group	Instruction	If No, Reason?
yes	yes	YY 0F 1X	MOVUPS	
no	yes		MOVSS	scalar
yes	yes		MOVUPD	
no	yes		MOVSD	scalar
no	yes		MOVLPS	Note 1
no	yes		MOVLPD	Note 1
no	yes		MOVLHPS	Redundant with VPERMILPS
yes	yes		MOVDDUP	
yes	yes		MOVSLDUP	
yes	yes		UNPCKLPS	
yes	yes		UNPCKLPD	
yes	yes		UNPCKHPS	
yes	yes		UNPCKHPD	
no	yes		MOVHPS	Note 1
no	yes		MOVHPD	Note 1
no	yes		MOVHLPS	Redundant with VPERMILPS
yes	yes		MOVAPS	
yes	yes		MOVSHDUP	
yes	yes		MOVAPD	

VEX.256 Encoding	VEX.128 Encoding	Group	Instruction	If No, Reason?
no	no		CVTPI2PS	MMX
no	yes		CVTSI2SS	scalar
no	no		CVTPI2PD	MMX
no	yes		CVTSI2SD	scalar
no	yes		MOVNTPS	
no	yes		MOVNTPD	
no	no		CVTTPS2PI	MMX
no	yes		CVTTSS2SI	scalar
no	no		CVTTPD2PI	MMX
no	yes		CVTTSD2SI	scalar
no	no		CVTPS2PI	MMX
no	yes		CVTSS2SI	scalar
no	no		CVTPD2PI	MMX
no	yes		CVTSD2SI	scalar
no	yes		UCOMISS	scalar
no	yes		UCOMISD	scalar
no	yes		COMISS	scalar
no	yes		COMISD	scalar
yes	yes	YY 0F 5X	MOVMSKPS	
yes	yes		MOVMSKPD	
yes	yes		SQRTPS	
no	yes		SQRTSS	scalar
yes	yes		SQRTPD	
no	yes		SQRTSD	scalar
yes	yes		RSQRTPS	
no	yes		RSQRTSS	scalar
yes	yes		RCPPS	
no	yes		RCPSS	scalar
yes	yes		ANDPS	
yes	yes		ANDPD	
yes	yes		ANDNPS	
yes	yes		ANDNPD	
yes	yes		ORPS	
yes	yes		ORPD	
yes	yes		XORPS	
yes	yes		XORPD	
yes	yes		ADDPS	
no	yes		ADDSS	scalar
yes	yes		ADDPD	
no	yes		ADDSD	scalar
yes	yes		MULPS	
no	yes		MULSS	scalar
yes	yes		MULPD	
no	yes		MULSD	scalar

VEX.256 Encoding	VEX.128 Encoding	Group	Instruction	If No, Reason?
yes	yes		CVTPS2PD	
no	yes		CVTSS2SD	scalar
yes	yes		CVTPD2PS	
no	yes		CVTSD2SS	scalar
yes	yes		CVTDQ2PS	
yes	yes		CVTPS2DQ	
yes	yes		CVTTPS2DQ	
yes	yes		SUBPS	
no	yes		SUBSS	scalar
yes	yes		SUBPD	
no	yes		SUBSD	scalar
yes	yes		MINPS	
no	yes		MINSS	scalar
yes	yes		MINPD	
no	yes		MINSD	scalar
yes	yes		DIVPS	
no	yes		DIVSS	scalar
yes	yes		DIVPD	
no	yes		DIVSD	scalar
yes	yes		MAXPS	
no	yes		MAXSS	scalar
yes	yes		MAXPD	
no	yes		MAXSD	scalar
no	yes	YY 0F 6X	PUNPCKLBW	VI
no	yes		PUNPCKLWD	VI
no	yes		PUNPCKLDQ	VI
no	yes		PACKSSWB	VI
no	yes		PCMPGTB	VI
no	yes		PCMPGTW	VI
no	yes		PCMPGTD	VI
no	yes		PACKUSWB	VI
no	yes		PUNPCKHBW	VI
no	yes		PUNPCKHWD	VI
no	yes		PUNPCKHDQ	VI
no	yes		PACKSSDW	VI
no	yes		PUNPCKLQDQ	VI
no	yes		PUNPCKHQDQ	VI
no	yes		MOVD	scalar
no	yes		MOVQ	scalar
yes	yes		MOVDQA	
yes	yes		MOVDQU	
no	yes	YY 0F 7X	PSHUFD	VI
no	yes		PSHUFHW	VI
no	yes		PSHUFLW	VI

VEX.256 Encoding	VEX.128 Encoding	Group	Instruction	If No, Reason?
no	yes		PCMPEQB	VI
no	yes		PCMPEQW	VI
no	yes		PCMPEQD	VI
yes	yes		HADDPD	
yes	yes		HADDPS	
yes	yes		HSUBPD	
yes	yes		HSUBPS	
no	yes		MOVD	VI
no	yes		MOVQ	VI
yes	yes		MOVDQA	
yes	yes		MOVDQU	
no	yes	YY 0F AX	LDMXCSR	
no	yes		STMXCSR	
yes	yes	YY 0F CX	CMPPS	
no	yes		CMPSS	scalar
yes	yes		CMPPD	
no	yes		CMPSD	scalar
no	yes		PINSRW	VI
no	yes		PEXTRW	VI
yes	yes		SHUFPS	
yes	yes		SHUFPD	
yes	yes	YY 0F DX	ADDSUBPD	
yes	yes		ADDSUBPS	
no	yes		PSRLW	VI
no	yes		PSRLD	VI
no	yes		PSRLQ	VI
no	yes		PADDQ	VI
no	yes		PMULLW	VI
no	no		MOVQ2DQ	MMX
no	no		MOVDQ2Q	MMX
no	yes		PMOVMSKB	VI
no	yes		PSUBUSB	VI
no	yes		PSUBUSW	VI
no	yes		PMINUB	VI
no	yes		PAND	VI
no	yes		PADDUSB	VI
no	yes		PADDUSW	VI
no	yes		PMAXUB	VI
no	yes		PANDN	VI
no	yes	YY 0F EX	PAVGB	VI
no	yes		PSRAW	VI
no	yes		PSRAD	VI
no	yes		PAVGW	VI
no	yes		PMULHUW	VI

VEX.256 Encoding	VEX.128 Encoding	Group	Instruction	If No, Reason?
no	yes		PMULHW	VI
yes	yes		CVTPD2DQ	
yes	yes		CVTTPD2DQ	
yes	yes		CVTDQ2PD	
no	yes		MOVNTDQ	VI
no	yes		PSUBSB	VI
no	yes		PSUBSW	VI
no	yes		PMINSW	VI
no	yes		POR	VI
no	yes		PADDSB	VI
no	yes		PADDSW	VI
no	yes		PMAXSW	VI
no	yes		PXOR	VI
yes	yes	YY 0F FX	LDDQU	VI
no	yes		PSLLW	VI
no	yes		PSLLD	VI
no	yes		PSLLQ	VI
no	yes		PMULUDQ	VI
no	yes		PMADDWD	VI
no	yes		PSADBW	VI
no	yes		MASKMOVDQU	
no	yes		PSUBB	VI
no	yes		PSUBW	VI
no	yes		PSUBD	VI
no	yes		PSUBQ	VI
no	yes		PADDB	VI
no	yes		PADDW	VI
no	yes		PADDD	VI
no	yes	SSSE3	PHADDW	VI
no	yes		PHADDSW	VI
no	yes		PHADDD	VI
no	yes		PHSUBW	VI
no	yes		PHSUBSW	VI
no	yes		PHSUBD	VI
no	yes		PMADDUBSW	VI
no	yes		PALIGNR	VI
no	yes		PSHUFB	VI
no	yes		PMULHRSW	VI
no	yes		PSIGNB	VI
no	yes		PSIGNW	VI
no	yes		PSIGND	VI
no	yes		PABSB	VI
no	yes		PABSW	VI
no	yes		PABSD	VI

VEX.256 Encoding	VEX.128 Encoding	Group	Instruction	If No, Reason?
yes	yes	SSE4.1	BLENDPS	
yes	yes		BLENDPD	
yes	yes		BLENDVPS	Note 2
yes	yes		BLENDVPD	Note 2
no	yes		DPPD	
yes	yes		DPPS	
no	yes		EXTRACTPS	Note 3
no	yes		INSERTPS	Note 3
no	yes		MOVNTDQA	
no	yes		MPSADBW	VI
no	yes		PACKUSDW	VI
no	yes		PBLENDVB	VI
no	yes		PBLENDW	VI
no	yes		PCMPEQQ	VI
no	yes		PEXTRD	VI
no	yes		PEXTRQ	VI
no	yes		PEXTRB	VI
no	yes		PEXTRW	VI
no	yes		PHMINPOSUW	VI
no	yes		PINSRB	VI
no	yes		PINSRD	VI
no	yes		PINSRQ	VI
no	yes		PMAXSB	VI
no	yes		PMAXSD	VI
no	yes		PMAXUD	VI
no	yes		PMAXUW	VI
no	yes		PMINSB	VI
no	yes		PMINSD	VI
no	yes		PMINUD	VI
no	yes		PMINUW	VI
no	yes		PMOVSXxx	VI
no	yes		PMOVZXxx	VI
no	yes		PMULDQ	VI
no	yes		PMULLD	VI
yes	yes		PTEST	
yes	yes		ROUNDPD	
yes	yes		ROUNDPS	
no	yes		ROUNDSD	scalar
no	yes		ROUNDSS	scalar
no	yes	SSE4.2	PCMPGTQ	VI
no	no	SSE4.2	CRC32c	integer
no	yes		PCMPESTRI	VI
no	yes		PCMPESTRM	VI

VEX.256 Encoding	VEX.128 Encoding	Group	Instruction	If No, Reason?
no	yes		PCMPISTRI	VI
no	yes		PCMPISTRM	VI
no	no	SSE4.2	POPCNT	integer

14.2.1 256-Bit Floating-Point Arithmetic Processing Enhancements

Intel AVX provides 35 256-bit floating-point arithmetic instructions, see Table 14-2. The arithmetic operations cover add, subtract, multiply, divide, square-root, compare, max, min, round, etc., on single precision and double precision floating-point data.

The enhancement in AVX on floating-point compare operation provides 32 conditional predicates to improve programming flexibility in evaluating conditional expressions.

Table 14-2. Promoted 256-Bit and 128-Bit Arithmetic Intel® AVX Instructions

VEX.256 Encoding	VEX.128 Encoding	Legacy Instruction Mnemonic
yes	yes	SQRTPS, SQRTPD, RSQRTPS, RCPPS
yes	yes	ADDPS, ADDPD, SUBPS, SUBPD
yes	yes	MULPS, MULPD, DIVPS, DIVPD
yes	yes	CVTPS2PD, CVTPD2PS
yes	yes	CVTDQ2PS, CVTPS2DQ
yes	yes	CVTTPS2DQ, CVTTPD2DQ
yes	yes	CVTPD2DQ, CVTDQ2PD
yes	yes	MINPS, MINPD, MAXPS, MAXPD
yes	yes	HADDPD, HADDPS, HSUBPD, HSUBPS
yes	yes	CMPPS, CMPPD
yes	yes	ADDSUBPD, ADDSUBPS, DPPS
yes	yes	ROUNDPD, ROUNDPS

14.2.2 256-Bit Non-Arithmetic Instruction Enhancements

Intel AVX provides primitives for handling data movement within 256-bit floating-point vectors and promotes many 128-bit floating data processing instructions to handle 256-bit floating-point vectors.

Intel AVX includes 39 256-bit data movement and processing instructions that are promoted from previous generations of SIMD instruction extensions, ranging from logical, blend, convert, test, unpacking, shuffling, load, and stores (see Table 14-3).

Table 14-3. Promoted 256-Bit and 128-Bit Data Movement Intel® AVX Instructions

VEX.256 Encoding	VEX.128 Encoding	Legacy Instruction Mnemonic
yes	yes	MOVAPS, MOVAPD, MOVDQA
yes	yes	MOVUPS, MOVUPD, MOVDQU
yes	yes	MOVMSKPS, MOVMSKPD
yes	yes	LDDQU, MOVNTPS, MOVNTPD, MOVNTDQ, MOVNTDQA
yes	yes	MOVSHDUP, MOVSLDUP, MOVDDUP

Table 14-3. Promoted 256-Bit and 128-Bit Data Movement Intel® AVX Instructions (Contd.)

VEX.256 Encoding	VEX.128 Encoding	Legacy Instruction Mnemonic
yes	yes	UNPCKHPD, UNPCKHPS, UNPCKLPD
yes	yes	BLENDPS, BLENDPD
yes	yes	SHUFPD, SHUFPS, UNPCKLPS
yes	yes	BLENDVPS, BLENDVPD
yes	yes	PTEST, MOVMSKPD, MOVMSKPS
yes	yes	XORPS, XORPD, ORPS, ORPD
yes	yes	ANDNPD, ANDNPS, ANDPD, ANDPS

Intel AVX introduces 18 data processing instructions that operate on 256-bit vectors, Table 14-4. These new primitives cover the following operations:

- Non-unit-strided fetching of SIMD data. Intel AVX provides several flexible SIMD floating-point data fetching primitives:
  - Broadcast of single or multiple data elements into a 256-bit destination.
  - Masked move primitives to load or store SIMD data elements conditionally.
- Intra-register manipulation of SIMD data elements. Intel AVX provides several flexible SIMD floating-point data manipulation primitives:
  - Insert/extract multiple SIMD floating-point data elements to/from 256-bit SIMD registers.
  - Permute primitives to facilitate efficient manipulation of floating-point data elements in 256-bit SIMD registers.
- Branch handling. Intel AVX provides several primitives to enable handling of branches in SIMD programming:
  - Variable blend instructions supports four-operand syntax with non-destructive source syntax. This is more flexible than the equivalent Intel SSE4 instruction syntax which uses the XMM0 register as the implied mask for blend selection.
  - Packed TEST instructions for floating-point data.

Table 14-4. 256-Bit Intel® AVX Instruction Enhancements

Instruction	Description
VBROADCASTF128 ymm1, m128	Broadcast 128-bit floating-point values in mem to low and high 128-bits in ymm1.
VBROADCASTSD ymm1, m64	Broadcast double precision floating-point element in mem to four locations in ymm1.
VBROADCASTSS ymm1, m32	Broadcast single precision floating-point element in mem to eight locations in ymm1.
VEXTRACTF128 xmm1/m128, ymm2, imm8	Extracts 128-bits of packed floating-point values from ymm2 and store results in xmm1/mem.
VINSERTF128 ymm1, ymm2, xmm3/m128, imm8	Insert 128-bits of packed floating-point values from xmm3/mem and the remaining values from ymm2 into ymm1.
VMASKMOVPS ymm1, ymm2, m256	Load packed single precision values from mem using mask in ymm2 and store in ymm1.
VMASKMOVPD ymm1, ymm2, m256	Load packed double precision values from mem using mask in ymm2 and store in ymm1.
VMASKMOVPS m256, ymm1, ymm2	Store packed single precision values from ymm2 mask in ymm1.
VMASKMOVPD m256, ymm1, ymm2	Store packed double precision values from ymm2 using mask in ymm1.
VPERMILPD ymm1, ymm2, ymm3/m256	Permute double precision floating-point values in ymm2 using controls from xmm3/mem and store result in ymm1.

Table 14-4. 256-Bit Intel® AVX Instruction Enhancements (Contd.)

Instruction	Description
VPERMILPD ymm1, ymm2/m256 imm8	Permute double precision floating-point values in ymm2/mem using controls from imm8 and store result in ymm1.
VPERMILPS ymm1, ymm2, ymm/m256	Permute single precision floating-point values in ymm2 using controls from ymm3/mem and store result in ymm1.
VPERMILPS ymm1, ymm2/m256, imm8	Permute single precision floating-point values in ymm2/mem using controls from imm8 and store result in ymm1.
VPERM2F128 ymm1, ymm2, ymm3/m256, imm8	Permute 128-bit floating-point fields in ymm2 and ymm3/mem using controls from imm8 and store result in ymm1.
VTESTPS ymm1, ymm2/m256	Set ZF if ymm2/mem AND ymm1 result is all 0s in packed single precision sign bits. Set CF if ymm2/mem AND NOT ymm1 result is all 0s in packed single precision sign bits.
VTESTPD ymm1, ymm2/m256	Set ZF if ymm2/mem AND ymm1 result is all 0s in packed double precision sign bits. Set CF if ymm2/mem AND NOT ymm1 result is all 0s in packed double precision sign bits.
VZEROALL	Zero all YMM registers.
VZEROUPPER	Zero upper 128 bits of all YMM registers.

14.2.3 Arithmetic Primitives for 128-Bit Vector and Scalar processing

Intel AVX provides a full complement of 128-bit numeric processing instructions that employ VEX-prefix encoding. These VEX-encoded instructions generally provide the same functionality over instructions operating on XMM register that are encoded using SIMD prefixes. The 128-bit numeric processing instructions in AVX cover floating-point and integer data processing; across 128-bit vector and scalar processing. Table 14-5 lists the state of promotion of legacy SIMD arithmetic ISA to VEX-128 encoding. Legacy SIMD floating-point arithmetic ISA promoted to VEX-256 encoding also support VEX-128 encoding (see Table 14-2).

The enhancement in Intel AVX on 128-bit floating-point compare operation provides 32 conditional predicates to improve programming flexibility in evaluating conditional expressions. This contrasts with floating-point SIMD compare instructions in Intel SSE and SSE2 supporting only eight conditional predicates.

Table 14-5. Promotion of Legacy SIMD ISA to 128-Bit Arithmetic Intel® AVX Instructions

VEX.256 Encoding	VEX.128 Encoding	Instruction	Reason Not Promoted
no	no	CVTPI2PS, CVTPI2PD, CVTPD2PI	MMX
no	no	CVTTPS2PI, CVTTPD2PI, CVTPS2PI	MMX
no	yes	CVTSI2SS, CVTSI2SD, CVTSD2SI	Scalar
no	yes	CVTTSS2SI, CVTTSD2SI, CVTSS2SI	Scalar
no	yes	COMISD, RSQRTSS, RCPSS	Scalar
no	yes	UCOMISS, UCOMISD, COMISS,	Scalar
no	yes	ADDSS, ADDSD, SUBSS, SUBSD	Scalar
no	yes	MULSS, MULSD, DIVSS, DIVSD	Scalar
no	yes	SQRTSS, SQRTSD	Scalar
no	yes	CVTSS2SD, CVTSD2SS	Scalar
no	yes	MINSS, MINSD, MAXSS, MAXSD	Scalar
no	yes	PAND, PANDN, POR, PXOR	VI
no	yes	PCMPGTB, PCMPGTW, PCMPGTD	VI

Table 14-5. Promotion of Legacy SIMD ISA to 128-Bit Arithmetic Intel® AVX Instructions (Contd.)

VEX.256 Encoding	VEX.128 Encoding	Instruction	Reason Not Promoted
no	yes	PMADDWD, PMADDUBSW	VI
no	yes	PAVGB, PAVGW, PMULUDQ	VI
no	yes	PCMPEQB, PCMPEQW, PCMPEQD	VI
no	yes	PMULLW, PMULHUW, PMULHW	VI
no	yes	PSUBSW, PADDSW, PSADBW	VI
no	yes	PADDUSB, PADDUSW, PADDSB	VI
no	yes	PSUBUSB, PSUBUSW, PSUBSB	VI
no	yes	PMINUB, PMINSW	VI
no	yes	PMAXUB, PMAXSW	VI
no	yes	PADDB, PADDW, PADDD, PADDQ	VI
no	yes	PSUBB, PSUBW, PSUBD, PSUBQ	VI
no	yes	PSLLW, PSLLD, PSLLQ, PSRAW	VI
no	yes	PSRLW, PSRLD, PSRLQ, PSRAD	VI
CPUID.01H:ECX.SSSE3[9]			
no	yes	PHSUBW, PHSUBD, PHSUBSW	VI
no	yes	PHADDW, PHADDD, PHADDSW	VI
no	yes	PMULHRSW	VI
no	yes	PSIGNB, PSIGNW, PSIGND	VI
no	yes	PABSB, PABSW, PABSD	VI
CPUID.01H:ECX.SSE4_1[19]			
no	yes	DPPD	
no	yes	PHMINPOSUW, MPSADBW	VI
no	yes	PMAXSB, PMAXSD, PMAXUD	VI
no	yes	PMINSB, PMINSD, PMINUD	VI
no	yes	PMAXUW, PMINUW	VI
no	yes	PMOVSXxx, PMOVZXxx	VI
no	yes	PMULDQ, PMULLD	VI
no	yes	ROUNDSD, ROUNDSS	Scalar
CPUID.01H:ECX.POPCNT[23]			
no	yes	POPCNT	Integer
CPUID.01H:ECX.SSE4_2[20]			
no	yes	PCMPGTQ	VI
no	no	CRC32	Integer
no	yes	PCMPESTRI, PCMPESTRM	VI
no	yes	PCMPISTRI, PCMPISTRM	VI
CPUID.01H:ECX.PCLMULQDQ[1]			
no	yes	PCLMULQDQ	VI
CPUID.01H:ECX.AESNI[25]			

Table 14-5. Promotion of Legacy SIMD ISA to 128-Bit Arithmetic Intel® AVX Instructions (Contd.)

VEX.256 Encoding	VEX.128 Encoding	Instruction	Reason Not Promoted
no	yes	AESDEC, AESDECLAST	VI
no	yes	AESENC, AESENCLAST	VI
no	yes	AESIMX, AESKEYGENASSIST	VI

Description of Column "Reason not promoted":

- **MMX**: Instructions referencing MMX registers do not support VEX.
- **Scalar:** Scalar instructions are not promoted to 256-bit.
- **Integer:** Integer instructions are not promoted.
- **VI:** "Vector Integer" instructions are not promoted to 256-bit.

14.2.4 Non-Arithmetic Primitives for 128-Bit Vector and Scalar Processing

Intel AVX provides a full complement of data processing instructions that employ VEX-prefix encoding. These VEX-encoded instructions generally provide the same functionality over instructions operating on XMM register that are encoded using SIMD prefixes.

A subset of new functionalities listed in Table 14-4 is also extended via VEX.128 encoding. These enhancements in AVX on 128-bit data processing primitives include 11 new instructions (see Table 14-6) with the following capabilities:

- Non-unit-strided fetching of SIMD data. AVX provides several flexible SIMD floating-point data fetching primitives:
  - broadcast of single data element into a 128-bit destination,
  - masked move primitives to load or store SIMD data elements conditionally,
- Intra-register manipulation of SIMD data elements. AVX provides several flexible SIMD floating-point data manipulation primitives:
  - permute primitives to facilitate efficient manipulation of floating-point data elements in 128-bit SIMD registers
- Branch handling. AVX provides several primitives to enable handling of branches in SIMD programming:
  - new variable blend instructions supports four-operand syntax with non-destructive source syntax. Branching conditions dependent on floating-point data or integer data can benefit from Intel AVX. This is more flexible than non-VEX encoded instruction syntax that uses the XMM0 register as implied mask for blend selection. While variable blend with implied XMM0 syntax is supported in SSE4 using SIMD prefix encoding, VEX-encoded 128-bit variable blend instructions only support the more flexible four-operand syntax.
  - Packed TEST instructions for floating-point data.

Table 14-6. 128-Bit Intel® AVX Instruction Enhancement

Instruction	Description
VBROADCASTSS xmm1, m32	Broadcast single precision floating-point element in mem to four locations in xmm1.
VMASKMOVPS xmm1, xmm2, m128	Load packed single precision values from mem using mask in xmm2 and store in xmm1.
VMASKMOVPD xmm1, xmm2, m128	Load packed double precision values from mem using mask in xmm2 and store in xmm1.
VMASKMOVPS m128, xmm1, xmm2	Store packed single precision values from xmm2 using mask in xmm1.
VMASKMOVPD m128, xmm1, xmm2	Store packed double precision values from xmm2 using mask in xmm1.

Table 14-6. 128-Bit Intel® AVX Instruction Enhancement (Contd.)

Instruction	Description
VPERMILPD xmm1, xmm2, xmm3/m128	Permute double precision floating-point values in xmm2 using controls from xmm3/mem and store result in xmm1.
VPERMILPD xmm1, xmm2/m128, imm8	Permute double precision floating-point values in xmm2/mem using controls from imm8 and store result in xmm1.
VPERMILPS xmm1, xmm2, xmm3/m128	Permute single precision floating-point values in xmm2 using controls from xmm3/mem and store result in xmm1.
VPERMILPS xmm1, xmm2/m128, imm8	Permute single precision floating-point values in xmm2/mem using controls from imm8 and store result in xmm1.
VTESTPS xmm1, xmm2/m128	Set ZF if xmm2/mem AND xmm1 result is all 0s in packed single precision sign bits. Set CF if xmm2/mem AND NOT xmm1 result is all 0s in packed single precision sign bits.
VTESTPD xmm1, xmm2/m128	Set ZF if xmm2/mem AND xmm1 result is all 0s in packed single precision sign bits. Set CF if xmm2/mem AND NOT xmm1 result is all 0s in packed double precision sign bits.

The 128-bit data processing instructions in AVX cover floating-point and integer data movement primitives. Legacy SIMD non-arithmetic ISA promoted to VEX-256 encoding also support VEX-128 encoding (see Table 14-3). Table 14-7 lists the state of promotion of the remaining legacy SIMD non-arithmetic ISA to VEX-128 encoding.

Table 14-7. Promotion of Legacy SIMD ISA to 128-Bit Non-Arithmetic Intel® AVX instruction

VEX.256 Encoding	VEX.128 Encoding	Instruction	Reason Not Promoted
no	no	MOVQ2DQ, MOVDQ2Q	MMX
no	yes	LDMXCSR, STMXCSR	
no	yes	MOVSS, MOVSD, CMPSS, CMPSD	Scalar
no	yes	MOVHPS, MOVHPD	Note 1
no	yes	MOVLPS, MOVLPD	Note 1
no	yes	MOVLHPS, MOVHLPS	Redundant with VPERMILPS
no	yes	MOVQ, MOVD	Scalar
no	yes	PACKUSWB, PACKSSDW, PACKSSWB	VI
no	yes	PUNPCKHBW, PUNPCKHWD	VI
no	yes	PUNPCKLBW, PUNPCKLWD	VI
no	yes	PUNPCKHDQ, PUNPCKLDQ	VI
no	yes	PUNPCKLQDQ, PUNPCKHQDQ	VI
no	yes	PSHUFHW, PSHUFLW, PSHUFD	VI
no	yes	PMOVMSKB, MASKMOVDQU	VI
no	yes	PAND, PANDN, POR, PXOR	VI
no	yes	PINSRW, PEXTRW,	VI
CPUID.01H:ECX.SSSE3[9]			
no	yes	PALIGNR, PSHUFB	VI
CPUID.01H:ECX.SSE4_1[19]			
no	yes	EXTRACTPS, INSERTPS	Note 3
no	yes	PACKUSDW, PCMPEQQ	VI

Table 14-7. Promotion of Legacy SIMD ISA to 128-Bit Non-Arithmetic Intel® AVX instruction (Contd.)

VEX.256 Encoding	VEX.128 Encoding	Instruction	Reason Not Promoted
no	yes	PBLENDVB, PBLENDW	VI
no	yes	PEXTRW, PEXTRB, PEXTRD, PEXTRQ	VI
no	yes	PINSRB, PINSRD, PINSRQ	VI

Description of column "Reason not promoted":

- **MMX**: Instructions referencing MMX registers do not support VEX.
- **Scalar:** Scalar instructions are not promoted to 256-bit.
- **VI:** "Vector Integer" instructions are not promoted to 256-bit.
- **Note 1**: MOVLPD/PS and MOVHPD/PS are not promoted to 256-bit. The equivalent functionality are provided by VINSERTF128 and VEXTRACTF128 instructions as the existing instructions have no natural 256b extension
- **Note 3**: It is expected that using 128-bit INSERTPS followed by a VINSERTF128 would be better than promoting INSERTPS to 256-bit (for example).

14.3 DETECTION OF INTEL® AVX INSTRUCTIONS

Intel AVX instructions operate on the 256-bit YMM register state. Application detection of new instruction extensions operating on the YMM state follows the general procedural flow in Figure 14-2.

Prior to using Intel AVX, the application must identify that the operating system supports the XGETBV instruction, the YMM register state, in addition to processor's support for YMM state management using XSAVE/XRSTOR and AVX instructions. The following simplified sequence accomplishes both and is strongly recommended.

1) Detect CPUID.01H:ECX.OSXSAVE[27] = 1 (XGETBV enabled for application use[1]).

2) Issue XGETBV and verify that XCR0[2:1] = '11b' (XMM state and YMM state are enabled by OS).

3) detect CPUID.01H:ECX.AVX[28] = 1 (AVX instructions supported).

(Step 3 can be done in any order relative to 1 and 2.)

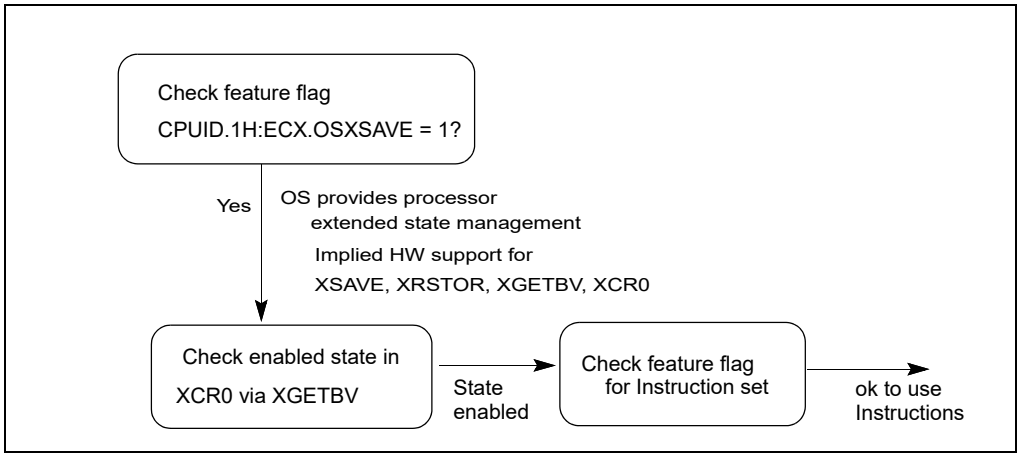


Figure 14-2. General Procedural Flow of Application Detection of Intel® AVX

1. If CPUID.01H:ECX.OSXSAVE reports 1, it also indirectly implies the processor supports XSAVE, XRSTOR, XGETBV, processor extended state bit vector XCR0. Thus an application may streamline the checking of CPUID feature flags for XSAVE and OSXSAVE. XSETBV is a privileged instruction.

The following pseudocode illustrates this recommended application Intel AVX detection process:

Example 14-1. Detection of Intel® AVX Instruction

```
INT supports_AVX()
{    mov       eax, 1
     cpuid
     and       ecx, 018000000H
     cmp       ecx, 018000000H; check both OSXSAVE and AVX feature flags
      jne      not_supported
     ; processor supports AVX instructions and XGETBV is enabled by OS
     mov       ecx, 0; specify 0 for XCR0 register
     XGETBV              ; result in EDX:EAX
     and       eax, 06H
     cmp       eax, 06H; check OS has enabled both XMM and YMM state support
     jne       not_supported
     mov       eax, 1
     jmp       done
NOT_SUPPORTED:
     mov       eax, 0
     done:
}
```

NOTE

It is unwise for an application to rely exclusively on CPUID.01H:ECX.AVX[28] or at all on CPUID.01H:ECX.XSAVE[26]: These indicate hardware support but not operating system support. If YMM state management is not enabled by an operating systems, Intel AVX instructions will #UD regardless of CPUID.01H:ECX.AVX[28]. "CPUID.01H:ECX.XSAVE[26] = 1" does not guarantee the OS actually uses the XSAVE process for state management.

These steps above also apply to enhanced 128-bit SIMD floating-pointing instructions in Intel AVX (using VEX prefix-encoding) that operate on the YMM states.

14.3.1 Detection of VEX-Encoded AES and VPCLMULQDQ

The VAESDEC/VAESDECLAST/VAESENC/VAESENCLAST/VAESIMC/VAESKEYGENASSIST instructions operate on YMM states. The detection sequence must combine checking for CPUID.01H:ECX.AES[25] = 1 and the sequence for detection application support for Intel AVX.

Example 14-2. Detection of VEX-Encoded Intel® AES-NI Instructions

```
INT supports_VAESNI()
{    mov       eax, 1
     cpuid
     and       ecx, 01A000000H
     cmp       ecx, 01A000000H; check OSXSAVE AVX and AESNI feature flags
      jne      not_supported
     ; processor supports AVX and VEX-encoded AESNI and XGETBV is enabled by OS
     mov       ecx, 0; specify 0 for XCR0 register
     XGETBV              ; result in EDX:EAX
     and       eax, 06H
     cmp       eax, 06H; check OS has enabled both XMM and YMM state support
     jne       not_supported
     mov       eax, 1
     jmp       done
NOT_SUPPORTED:
     mov       eax, 0
done:
```

Similarly, the detection sequence for VPCLMULQDQ must combine checking for CPUID.01H:ECX.PCLMULQDQ[1] = 1 and the sequence for detection application support for Intel AVX.

This is shown in the pseudocode provided in Example 14-3.

Example 14-3. Detection of VEX-Encoded Intel® AES-NI Instructions

```
INT supports_VPCLMULQDQ)
{    mov       eax, 1
     cpuid
     and       ecx, 018000002H
     cmp       ecx, 018000002H; check OSXSAVE AVX and PCLMULQDQ feature flags
      jne      not_supported
     ; processor supports AVX and VEX-encoded PCLMULQDQ and XGETBV is enabled by OS
     mov       ecx, 0; specify 0 for XCR0 register
     XGETBV              ; result in EDX:EAX
     and       eax, 06H
     cmp       eax, 06H; check OS has enabled both XMM and YMM state support
     jne       not_supported


     mov       eax, 1
     jmp       done
NOT_SUPPORTED:
     mov       eax, 0
done:
```

14.4 HALF PRECISION FLOATING-POINT CONVERSION

VCVTPH2PS and VCVTPS2PH are two instructions supporting half precision floating-point data type conversion to and from single precision floating-point data types.

Half precision floating-point values are not used by the processor directly for arithmetic operations. But the conversion operation are subject to SIMD floating-point exceptions.

Additionally, the conversion operations of VCVTPS2PH allow programmer to specify rounding control using control fields in an immediate byte. The effects of the immediate byte are listed in Table 14-8.

Rounding control can use Imm[2] to select an override RC field specified in Imm[1:0] or use MXCSR setting.

Table 14-8. Immediate Byte Encoding for 16-Bit Floating-Point Conversion Instructions

Bits	Field Name/value	Description	Comment
Imm[1:0]	RC=00B	Round to nearest even	If Imm[2] = 0
	RC=01B	Round down	
	RC=10B	Round up	
	RC=11B	Truncate	
Imm[2]	MS1=0	Use imm[1:0] for rounding	Ignore MXCSR.RC
	MS1=1	Use MXCSR.RC for rounding	
Imm[7:3]	Ignored	Ignored by processor	

Specific SIMD floating-point exceptions that can occur in conversion operations are shown in Table 14-9 and Table 14-10.

Table 14-9. Non-Numerical Behavior for VCVTPH2PS and VCVTPS2PH

Source Operands	Masked Result	Unmasked Result
QNaN	QNaN1[1]	QNaN1[1] (not an exception)
SNaN	QNaN1[2]	None

NOTES:

1. The half precision output QNaN1 is created from the single precision input QNaN as follows: the sign bit is preserved, the 8-bit exponent FFH is replaced by the 5-bit exponent 1FH, and the 24-bit significand is truncated to an 11-bit significand by removing its 14 least significant bits.
2. The half precision output QNaN1 is created from the single precision input SNaN as follows: the sign bit is preserved, the 8-bit exponent FFH is replaced by the 5-bit exponent 1FH, and the 24-bit significand is truncated to an 11-bit significand by removing its 14 least significant bits. The second most significant bit of the significand is changed from 0 to 1 to convert the signaling NaN into a quiet NaN.

Table 14-10. Invalid Operation for VCVTPH2PS and VCVTPS2PH

Instruction	Condition	Masked Result	Unmasked Result
VCVTPH2PS	SRC = NaN	See Table 14-9	#I=1
VCVTPS2PH	SRC = NaN	See Table 14-9	#I=1

The VCVTPS2PH instruction can cause denormal exceptions if the value of the source operand is denormal relative to the numerical range represented by the source format (see Table 14-11).

Table 14-11. Denormal Condition Summary

Instruction	Condition	Masked Result	Unmasked Result
VCVTPH2PS	SRC is denormal relative to input format	res = Result rounded to the destination precision and using the bounded exponent, but only if no unmasked post-computation exception occurs. #DE unchanged	Same as masked result.
VCVTPS2PH	SRC is denormal relative to input format	res = Result rounded to the destination precision and using the bounded exponent, but only if no unmasked post-computation exception occurs. #DE=1	#DE=1

The VCVTPS2PH instruction can cause an underflow exception if the result of the conversion is less than the underflow threshold for half precision floating-point data type, i.e., $| x | < 1.0 * 2^{-14}$.

Table 14-12. Underflow Condition for VCVTPS2PH

Instruction	Condition	Masked Result[1]	Unmasked Result
VCVTPS2PH	Result < smallest destination precision final normal value[2]	Result = +0 or -0, denormal, normal. #UE =1. #PE = 1 if the result is inexact.	#UE=1, #PE = 1 if the result is inexact.

NOTES:

1. Masked and unmasked results are shown in Table 14-11.
2. MXCSR.FTZ is ignored, the processor behaves as if MXCSR.FTZ = 0.

The VCVTPS2PH instruction can cause an overflow exception if the result of the conversion is greater than the maximum representable value for half precision floating-point data type, i.e., $| x | \geq 1.0 * 2^{16}$.

Table 14-13. Overflow Condition for VCVTPS2PH

Instruction	Condition	Masked Result	Unmasked Result
VCVTPS2PH	Result ≥ largest destination precision finial normal value[1]	Result = +Inf or -Inf. #OE=1.	#OE=1.

The VCVTPS2PH instruction can cause an inexact exception if the result of the conversion is not exactly representable in the destination format.

Table 14-14. Inexact Condition for VCVTPS2PH

Instruction	Condition	Masked Result[1]	Unmasked Result
VCVTPS2PH	The result is not representable in the destination format	res = Result rounded to the destination precision and using the bounded exponent, but only if no unmasked underflow or overflow conditions occur (this exception can occur in the presence of a masked underflow or overflow). #PE=1.	Only if no underflow/overflow condition occurred, or if the corresponding exceptions are masked: ▪ Set #OE if masked overflow and set result as described above for masked overflow. ▪ Set #UE if masked underflow and set result as described above for masked underflow. If neither underflow nor overflow, result equals the result rounded to the destination precision and using the bounded exponent set #PE = 1.

NOTES:

1. If a source is denormal relative to input format with DM masked and at least one of PM or UM unmasked, then an exception will be raised with DE, UE, and PE set.

14.4.1 Detection of F16C Instructions

Applications using float 16 instruction must follow a detection sequence similar to Intel AVX to ensure:

- The OS has enabled YMM state management support.
- The processor supports Intel AVX as indicated by the CPUID feature flag, i.e., CPUID.01H:ECX.AVX[28] = 1.
- The processor supports 16-bit floating-point conversion instructions via a CPUID feature flag (CPUID.01H:ECX.F16C[29] = 1).

Application detection of Float-16 conversion instructions follow the general procedural flow in Figure 14-3.

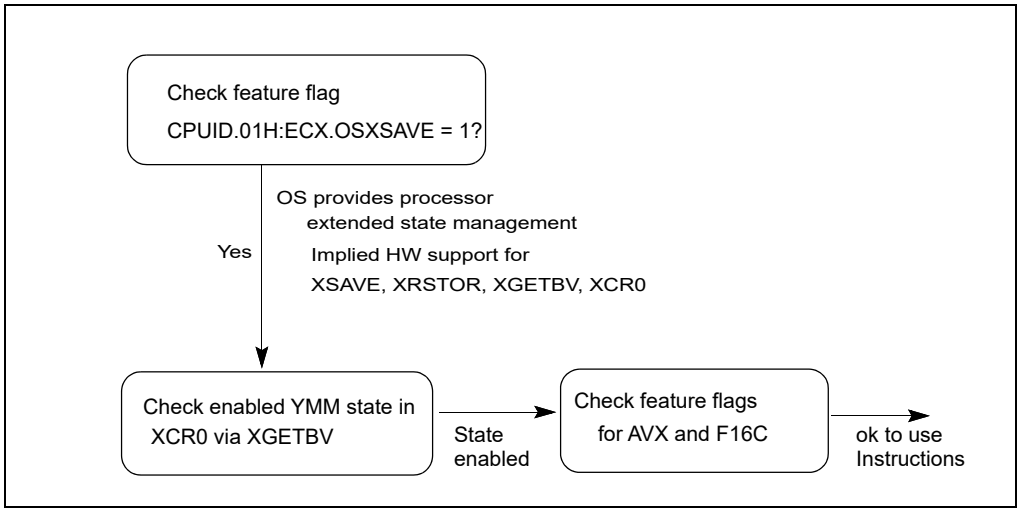


Figure 14-3. General Procedural Flow of Application Detection of Float-16

```
---------------------------------------------------------------------------------
INT supports_f16c()
{       ; result in eax
        mov eax, 1
        cpuid
        and ecx, 038000000H
        cmp ecx, 038000000H; check OSXSAVE, AVX, F16C feature flags
        jne not_supported
        ; processor supports AVX,F16C instructions and XGETBV is enabled by OS
        mov ecx, 0; specify 0 for XCR0 register
        XGETBV; result in EDX:EAX
        and eax, 06H
        cmp eax, 06H; check OS has enabled both XMM and YMM state support
        jne not_supported
        mov eax, 1
        jmp done
        NOT_SUPPORTED:
        mov eax, 0
        done:
}
---------------------------------------------------------------------------
```

14.5 FUSED-MULTIPLY-ADD (FMA) EXTENSIONS

FMA extensions enhances Intel AVX with high-throughput, arithmetic capabilities covering fused multiply-add, fused multiply-subtract, fused multiply add/subtract interleave, signed-reversed multiply on fused multiply-add and multiply-subtract. FMA extensions provide 36 256-bit floating-point instructions to perform computation on 256-bit vectors and additional 128-bit and scalar FMA instructions.

FMA extensions also provide 60 128-bit floating-point instructions to process 128-bit vector and scalar data. The arithmetic operations cover fused multiply-add, fused multiply-subtract, signed-reversed multiply on fused multiply-add and multiply-subtract.

Table 14-15. FMA Instructions

Instruction	Description
VFMADD132PD/VFMADD213PD/VFMADD231PD xmm0, xmm1, xmm2/m128; ymm0, ymm1, ymm2/m256	Fused Multiply-Add of Packed Double Precision Floating-Point Values
VFMADD132PS/VFMADD213PS/VFMADD231PS xmm0, xmm1, xmm2/m128; ymm0, ymm1, ymm2/m256	Fused Multiply-Add of Packed Single Precision Floating-Point Values
VFMADD132SD/VFMADD213SD/VFMADD231SD xmm0, xmm1, xmm2/m64	Fused Multiply-Add of Scalar Double Precision Floating-Point Values
VFMADD132SS/VFMADD213SS/VFMADD231SS xmm0, xmm1, xmm2/m32	Fused Multiply-Add of Scalar Single Precision Floating-Point Values
VFMADDSUB132PD/VFMADDSUB213PD/VFMADDSUB231PD xmm0, xmm1, xmm2/m128; ymm0, ymm1, ymm2/m256	Fused Multiply-Alternating Add/Subtract of Packed Double Precision Floating-Point Values
VFMADDSUB132PS/VFMADDSUB213PS/VFMADDSUB231PS xmm0, xmm1, xmm2/m128; ymm0, ymm1, ymm2/m256	Fused Multiply-Alternating Add/Subtract of Packed Single Precision Floating-Point Values
VFMSUBADD132PD/VFMSUBADD213PD/VFMSUBADD231PD xmm0, xmm1, xmm2/m128; ymm0, ymm1, ymm2/m256	Fused Multiply-Alternating Subtract/Add of Packed Double Precision Floating-Point Values
VFMSUBADD132PS/VFMSUBADD213PS/VFMSUBADD231PS xmm0, xmm1, xmm2/m128; ymm0, ymm1, ymm2/m256	Fused Multiply-Alternating Subtract/Add of Packed Single Precision Floating-Point Values
VFMSUB132PD/VFMSUB213PD/VFMSUB231PD xmm0, xmm1, xmm2/m128; ymm0, ymm1, ymm2/m256	Fused Multiply-Subtract of Packed Double Precision Floating-Point Values
VFMSUB132PS/VFMSUB213PS/VFMSUB231PS xmm0, xmm1, xmm2/m128; ymm0, ymm1, ymm2/m256	Fused Multiply-Subtract of Packed Single Precision Floating-Point Values
VFMSUB132SD/VFMSUB213SD/VFMSUB231SD xmm0, xmm1, xmm2/m64	Fused Multiply-Subtract of Scalar Double Precision Floating-Point Values
VFMSUB132SS/VFMSUB213SS/VFMSUB231SS xmm0, xmm1, xmm2/m32	Fused Multiply-Subtract of Scalar Single Precision Floating-Point Values
VFNMADD132PD/VFNMADD213PD/VFNMADD231PD xmm0, xmm1, xmm2/m128; ymm0, ymm1, ymm2/m256	Fused Negative Multiply-Add of Packed Double Precision Floating-Point Values
VFNMADD132PS/VFNMADD213PS/VFNMADD231PS xmm0, xmm1, xmm2/m128; ymm0, ymm1, ymm2/m256	Fused Negative Multiply-Add of Packed Single Precision Floating-Point Values
VFNMADD132SD/VFNMADD213SD/VFNMADD231SD xmm0, xmm1, xmm2/m64	Fused Negative Multiply-Add of Scalar Double Precision Floating-Point Values
VFNMADD132SS/VFNMADD213SS/VFNMADD231SS xmm0, xmm1, xmm2/m32	Fused Negative Multiply-Add of Scalar Single Precision Floating-Point Values
VFNMSUB132PD/VFNMSUB213PD/VFNMSUB231PD xmm0, xmm1, xmm2/m128; ymm0, ymm1, ymm2/m256	Fused Negative Multiply-Subtract of Packed Double Precision Floating-Point Values
VFNMSUB132PS/VFNMSUB213PS/VFNMSUB231PS xmm0, xmm1, xmm2/m128; ymm0, ymm1, ymm2/m256	Fused Negative Multiply-Subtract of Packed Single Precision Floating-Point Values

Table 14-15. FMA Instructions (Contd.)

Instruction	Description
VFNMSUB132SD/VFNMSUB213SD/VFNMSUB231SD xmm0, xmm1, xmm2/m64	Fused Negative Multiply-Subtract of Scalar Double Precision Floating-Point Values
VFNMSUB132SS/VFNMSUB213SS/VFNMSUB231SS xmm0, xmm1, xmm2/m32	Fused Negative Multiply-Subtract of Scalar Single Precision Floating-Point Values

14.5.1 FMA Instruction Operand Order and Arithmetic Behavior

FMA instruction mnemonics are defined explicitly with an ordered three digits, e.g., VFMADD132PD. The value of each digit refers to the ordering of the three source operand as defined by instruction encoding specification:

- '1': The first source operand (also the destination operand) in the syntactical order listed in this specification.
- '2': The second source operand in the syntactical order. This is a YMM/XMM register, encoded using VEX prefix.
- '3': The third source operand in the syntactical order. The first and third operand are encoded following ModR/M encoding rules.

The ordering of each digit within the mnemonic refers to the floating-point data listed on the right-hand side of the arithmetic equation of each FMA operation (see Table 14-17):

- The first position in the three digits of a FMA mnemonic refers to the operand position of the first FP data expressed in the arithmetic equation of FMA operation, the multiplicand.
- The second position in the three digits of a FMA mnemonic refers to the operand position of the second FP data expressed in the arithmetic equation of FMA operation, the multiplier.
- The third position in the three digits of a FMA mnemonic refers to the operand position of the FP data being added/subtracted to the multiplication result.

Note the non-numerical result of an FMA operation does not resemble the mathematically-defined commutative property between the multiplicand and the multiplier values (see Table 14-17). Consequently, software tools (such as an assembler) may support a complementary set of FMA mnemonics for each FMA instruction for ease of programming to take advantage of the mathematical property of commutative multiplications. For example, an assembler may optionally support the complementary mnemonic "VFMADD312PD" in addition to the true mnemonic "VFMADD132PD". The assembler will generate the same instruction opcode sequence corresponding to VFMADD132PD. The processor executes VFMADD132PD and report any NAN conditions based on the definition of VFMADD132PD. Similarly, if the complementary mnemonic VFMADD123PD is supported by an assembler at source level, it must generate the opcode sequence corresponding to VFMADD213PD; the complementary mnemonic VFMADD321PD must produce the opcode sequence defined by VFMADD231PD. In the absence of FMA operations reporting a NAN result, the numerical results of using either mnemonic with an assembler supporting both mnemonics will match the behavior defined in Table 14-17. Support for the complementary FMA mnemonics by software tools is optional.

14.5.2 Fused-Multiply-ADD (FMA) Numeric Behavior

FMA instructions can perform fused-multiply-add operations (including fused-multiply-subtract, and other varieties) on packed and scalar data elements in the instruction operands. Separate FMA instructions are provided to handle different types of arithmetic operations on the three source operands.

FMA instruction syntax is defined using three source operands and the first source operand is updated based on the result of the arithmetic operations of the data elements of 128-bit or 256-bit operands, i.e., The first source operand is also the destination operand.

The arithmetic FMA operation performed in an FMA instruction takes one of several forms, r=(x*y)+z, r=(x*y)-z, r=-(x*y)+z, or r=-(x*y)-z. Packed FMA instructions can perform eight single precision FMA operations or four double precision FMA operations with 256-bit vectors.

Scalar FMA instructions only perform one arithmetic operation on the low order data element. The content of the rest of the data elements in the lower 128-bits of the destination operand is preserved. the upper 128bits of the destination operand are filled with zero.

An arithmetic FMA operation of the form, r=(x*y)+z, takes two IEEE-754-2008 single (double) precision values and multiplies them to form an infinite precision intermediate value. This intermediate value is added to a third single (double) precision value (also at infinite precision) and rounded to produce a single (double) precision result.

Table 14-17 describes the numerical behavior of the FMA operation, r=(x*y)+z, r=(x*y)-z, r=-(x*y)+z, r=-(x*y)-z for various input values. The input values can be 0, finite non-zero (F in Table 14-17), infinity of either sign (INF in Table 14-17), positive infinity (+INF in Table 14-17), negative infinity (-INF in Table 14-17), or NaN (including QNaN or SNaN). If any one of the input values is a NAN, the result of FMA operation, r, may be a quietized NAN. The result can be either Q(x), Q(y), or Q(z), see Table 14-17. If x is a NaN, then:

- Q(x) = x if x is QNaN, or
- Q(x) = the quietized NaN obtained from x if x is SNaN.

The notation for the output value in Table 14-17 are:

- "+INF": positive infinity, "-INF": negative infinity. When the result depends on a conditional expression, both values are listed in the result column and the condition is described in the comment column.
- QNaNIndefinite represents the QNaN which has the sign bit equal to 1, the most significand field equal to 1, and the remaining significand field bits equal to 0.
- The summation or subtraction of 0s or identical values in FMA operation can lead to the following situations shown in Table 14-16.
- If the FMA computation represents an invalid operation (e.g., when adding two INF with opposite signs)), the invalid exception is signaled, and the MXCSR.IE flag is set.

Table 14-16. Rounding Behavior of Zero Result in FMA Operation

x*y	z	(x*y) + z	(x*y) - z	- (x*y) + z	- (x*y) - z
(+0)	(+0)	+0 in all rounding modes	- 0 when rounding down, and +0 otherwise	- 0 when rounding down, and +0 otherwise	- 0 in all rounding modes
(+0)	(-0)	- 0 when rounding down, and +0 otherwise	+0 in all rounding modes	- 0 in all rounding modes	- 0 when rounding down, and +0 otherwise
(-0)	(+0)	- 0 when rounding down, and +0 otherwise	- 0 in all rounding modes	+ 0 in all rounding modes	- 0 when rounding down, and +0 otherwise
(-0)	(-0)	- 0 in all rounding modes	- 0 when rounding down, and +0 otherwise	- 0 when rounding down, and +0 otherwise	+ 0 in all rounding modes
F	-F	- 0 when rounding down, and +0 otherwise	2*F	-2*F	- 0 when rounding down, and +0 otherwise
F	F	2*F	- 0 when rounding down, and +0 otherwise	- 0 when rounding down, and +0 otherwise	-2*F

Table 14-17. FMA Numeric Behavior

x (multiplicand)	y (multiplier)	z	r=(x*y) +z	r=(x*y) -z	r = -(x*y)+z	r= -(x*y)-z	Comment
NaN	0, F, INF, NaN	0, F, INF, NaN	Q(x)	Q(x)	Q(x)	Q(x)	Signal invalid exception if x or y or z is SNaN
0, F, INF	NaN	0, F, INF, NaN	Q(y)	Q(y)	Q(y)	Q(y)	Signal invalid exception if y or z is SNaN
0, F, INF	0, F, INF	NaN	Q(z)	Q(z)	Q(z)	Q(z)	Signal invalid exception if z is SNaN
INF	F, INF	+INF	+INF	QNaNIn-definite	QNaNIn-definite	-INF	if x*y and z have the same sign
			QNaNIn-definite	-INF	+INF	QNaNIn-definite	if x*y and z have opposite signs

x (multiplicand)	y (multiplier)	z	r=(x*y) +z	r=(x*y) -z	r = -(x*y)+z	r= -(x*y)-z	Comment
INF	F, INF	-INF	-INF	QNaNIn-definite	QNaNIn-definite	+INF	if x*y and z have the same sign
			QNaNIn-definite	+INF	-INF	QNaNIn-definite	if x*y and z have opposite signs
INF	F, INF	0, F	+INF	+INF	-INF	-INF	if x and y have the same sign
			-INF	-INF	+INF	+INF	if x and y have opposite signs
INF	0	0, F, INF	QNaNIn-definite	QNaNIn-definite	QNaNIn-definite	QNaNIn-definite	Signal invalid exception
0	INF	0, F, INF	QNaNIn-definite	QNaNIn-definite	QNaNIn-definite	QNaNIn-definite	Signal invalid exception
F	INF	+INF	+INF	QNaNIn-definite	QNaNIn-definite	-INF	if x*y and z have the same sign
			QNaNIn-definite	-INF	+INF	QNaNIn-definite	if x*y and z have opposite signs
F	INF	-INF	-INF	QNaNIn-definite	QNaNIn-definite	+INF	if x*y and z have the same sign
			QNaNIn-definite	+INF	-INF	QNaNIn-definite	if x*y and z have opposite signs
F	INF	0,F	+INF	+INF	-INF	-INF	if x * y > 0
			-INF	-INF	+INF	+INF	if x * y < 0
0,F	0,F	INF	+INF	-INF	+INF	-INF	if z > 0
			-INF	+INF	-INF	+INF	if z < 0
0	0	0	0	0	0	0	The sign of the result depends on the sign of the operands and on the rounding mode. The product x*y is +0 or -0, depending on the signs of x and y. The summation/subtraction of the zero representing (x*y) and the zero representing z can lead to one of the four cases shown in Table 14-16.
0	F	0	0	0	0	0	
F	0	0	0	0	0	0	
0	0	F	z	-z	z	-z	
0	F	F	z	-z	z	-z	
F	0	F	z	-z	z	-z	
F	F	0	x*y	x*y	-x*y	-x*y	Rounded to the destination precision, with bounded exponent
F	F	F	(x*y)+z	(x*y)-z	-(x*y)+z	-(x*y)-z	Rounded to the destination precision, with bounded exponent; however, if the exact values of x*y and z are equal in magnitude with signs resulting in the FMA operation producing 0, the rounding behavior described in Table 14-16.

If unmasked floating-point exceptions are signaled (invalid operation, denormal operand, overflow, underflow, or inexact result) the result register is left unchanged and a floating-point exception handler is invoked.

14.5.3 Detection of FMA

Hardware support for FMA is indicated by CPUID.01H:ECX.FMA[12]=1.

Application Software must identify that hardware supports AVX, after that it must also detect support for FMA by CPUID.01H:ECX.FMA[12]. The recommended pseudocode sequence for detection of FMA is:

```
INT supports_fma()
{       ; result in eax
        mov eax, 1
        cpuid
        and ecx, 018001000H
        cmp ecx, 018001000H; check OSXSAVE, AVX, FMA feature flags
         jne not_supported
        ; processor supports AVX,FMA instructions and XGETBV is enabled by OS
        mov ecx, 0; specify 0 for XCR0 register
        XGETBV; result in EDX:EAX
        and eax, 06H
        cmp eax, 06H; check OS has enabled both XMM and YMM state support
        jne not_supported
        mov eax, 1
        jmp done
        NOT_SUPPORTED:
        mov eax, 0
        done:
}
```

Note that FMA comprises 256-bit and 128-bit SIMD instructions operating on YMM states.

14.6 OVERVIEW OF INTEL® ADVANCED VECTOR EXTENSIONS 2 (INTEL® AVX2)

Intel® AVX2 extends Intel AVX by promoting most of the 128-bit SIMD integer instructions with 256-bit numeric processing capabilities. Intel AVX2 instructions follow the same programming model as AVX instructions.

In addition, Intel AVX2 provide enhanced functionalities for broadcast/permute operations on data elements, vector shift instructions with variable-shift count per data element, and instructions to fetch non-contiguous data elements from memory.

14.6.1 Intel® AVX2 and 256-Bit Vector Integer Processing

Intel AVX2 promotes the vast majority of 128-bit integer SIMD instruction sets to operate with 256-bit wide YMM registers. Intel AVX2 instructions are encoded using the VEX prefix and require the same operating system support as Intel AVX. Generally, most of the promoted 256-bit vector integer instructions follow the 128-bit lane operation, similar to the promoted 256-bit floating-point SIMD instructions in Intel AVX.

Newer functionalities in Intel AVX2 generally fall into the following categories:

- Fetching non-contiguous data elements from memory using vector-index memory addressing. These “gather” instructions introduce a new memory-addressing form, consisting of a base register and multiple indices specified by a vector register (either XMM or YMM). Data elements sizes of 32 and 64-bits are supported, and data types for floating-point and integer elements are also supported.
- Cross-lane functionalities are provided with several new instructions for broadcast and permute operations. Some of the 256-bit vector integer instructions promoted from legacy SSE instruction sets also exhibit cross-lane behavior, e.g., VPMOVZ/VPMOVS family.
- Intel AVX2 complements the Intel AVX instructions that are typed for floating-point operation with a full complement of equivalent set for operating with 32/64-bit integer data elements.
- Vector shift instructions with per-element shift count. Data elements sizes of 32 and 64 bits are supported.

14.7 PROMOTED VECTOR INTEGER INSTRUCTIONS IN INTEL® AVX2

In Intel AVX2, most SSSE3 and Intel SSE, SSE2, SSE3, and SSE4 vector integer instructions have been promoted to support VEX.256 encodings. Table 14-18 summarizes the promotion status for existing instructions. The column "VEX.128" indicates whether the instruction using VEX.128 prefix encoding is supported.

The column "VEX.256" indicates whether 256-bit vector form of the instruction using the VEX.256 prefix encoding is supported, and under which feature flag.

Table 14-18. Promoted Vector Integer SIMD Instructions in Intel® AVX2

VEX.256 Encoding	VEX.128 Encoding	Group	Instruction
AVX2	AVX	YY 0F 6X	PUNPCKLBW
AVX2	AVX		PUNPCKLWD
AVX2	AVX		PUNPCKLDQ
AVX2	AVX		PACKSSWB
AVX2	AVX		PCMPGTB
AVX2	AVX		PCMPGTW
AVX2	AVX		PCMPGTD
AVX2	AVX		PACKUSWB
AVX2	AVX		PUNPCKHBW
AVX2	AVX		PUNPCKHWD
AVX2	AVX		PUNPCKHDQ
AVX2	AVX		PACKSSDW
AVX2	AVX		PUNPCKLQDQ
AVX2	AVX		PUNPCKHQDQ
no	AVX		MOVD
no	AVX		MOVQ
AVX	AVX		MOVDQA
AVX	AVX		MOVDQU
AVX2	AVX	YY 0F 7X	PSHUFD
AVX2	AVX		PSHUFHW
AVX2	AVX		PSHUFLW
AVX2	AVX		PCMPEQB
AVX2	AVX		PCMPEQW
AVX2	AVX		PCMPEQD
AVX	AVX		MOVDQA
AVX	AVX		MOVDQU
no	AVX		PINSRW
no	AVX		PEXTRW
AVX2	AVX		PSRLW
AVX2	AVX		PSRLD
AVX2	AVX		PSRLQ
AVX2	AVX		PADDQ
AVX2	AVX		PMULLW

Table 14-18. Promoted Vector Integer SIMD Instructions in Intel® AVX2 (Contd.)

VEX.256 Encoding	VEX.128 Encoding	Group	Instruction
AVX2	AVX		PMOVMSKB
AVX2	AVX		PSUBUSB
AVX2	AVX		PSUBUSW
AVX2	AVX		PMINUB
AVX2	AVX		PAND
AVX2	AVX		PADDUSB
AVX2	AVX		PADDUSW
AVX2	AVX		PMAXUB
AVX2	AVX		PANDN
AVX2	AVX	YY 0F EX	PAVGB
AVX2	AVX		PSRAW
AVX2	AVX		PSRAD
AVX2	AVX		PAVGW
AVX2	AVX		PMULHUW
AVX2	AVX		PMULHW
AVX	AVX		MOVNTDQ
AVX2	AVX		PSUBSB
AVX2	AVX		PSUBSW
AVX2	AVX		PMINSW
AVX2	AVX		POR
AVX2	AVX		PADDSB
AVX2	AVX		PADDSW
AVX2	AVX		PMAXSW
AVX2	AVX		PXOR
AVX	AVX	YY 0F FX	LDDQU
AVX2	AVX		PSLLW
AVX2	AVX		PSLLD
AVX2	AVX		PSLLQ
AVX2	AVX		PMULUDQ
AVX2	AVX		PMADDWD
AVX2	AVX		PSADBW
AVX2	AVX		PSUBB
AVX2	AVX		PSUBW
AVX2	AVX		PSUBD
AVX2	AVX		PSUBQ
AVX2	AVX		PADDB
AVX2	AVX		PADDW
AVX2	AVX		PADDD

Table 14-18. Promoted Vector Integer SIMD Instructions in Intel® AVX2 (Contd.)

VEX.256 Encoding	VEX.128 Encoding	Group	Instruction
AVX2	AVX	SSSE3	PHADDW
AVX2	AVX		PHADDSW
AVX2	AVX		PHADDD
AVX2	AVX		PHSUBW
AVX2	AVX		PHSUBSW
AVX2	AVX		PHSUBD
AVX2	AVX		PMADDUBSW
AVX2	AVX		PALIGNR
AVX2	AVX		PSHUFB
AVX2	AVX		PMULHRSW
AVX2	AVX		PSIGNB
AVX2	AVX		PSIGNW
AVX2	AVX		PSIGND
AVX2	AVX		PABSB
AVX2	AVX		PABSW
AVX2	AVX		PABSD
AVX2	AVX		MOVNTDQA
AVX2	AVX		MPSADBW
AVX2	AVX		PACKUSDW
AVX2	AVX		PBLENDVB
AVX2	AVX		PBLENDW
AVX2	AVX		PCMPEQQ
no	AVX		PEXTRD
no	AVX		PEXTRQ
no	AVX		PEXTRB
no	AVX		PEXTRW
no	AVX		PHMINPOSUW
no	AVX		PINSRB
no	AVX		PINSRD
no	AVX		PINSRQ
AVX2	AVX		PMAXSB
AVX2	AVX		PMAXSD
AVX2	AVX		PMAXUD
AVX2	AVX		PMAXUW
AVX2	AVX		PMINSB
AVX2	AVX		PMINSD
AVX2	AVX		PMINUD
AVX2	AVX		PMINUW

Table 14-18. Promoted Vector Integer SIMD Instructions in Intel® AVX2 (Contd.)

VEX.256 Encoding	VEX.128 Encoding	Group	Instruction
AVX2	AVX		PMOVSXxx
AVX2	AVX		PMOVZXxx
AVX2	AVX		PMULDQ
AVX2	AVX		PMULLD
AVX	AVX		PTEST
AVX2	AVX	SSE4.2	PCMPGTQ
no	AVX		PCMPESTRI
no	AVX		PCMPESTRM
no	AVX		PCMPISTRI
no	AVX		PCMPISTRM
no	AVX	AESNI	AESDEC
no	AVX		AESDECLAST
no	AVX		AESENC
no	AVX		AESECNLAST
no	AVX		AESIMC
no	AVX		AESKEYGENASSIST
no	AVX	CLMUL	PCLMULQDQ

Table 14-19 compares complementary SIMD functionalities introduced in Intel AVX and AVX2. instructions.

Table 14-19. VEX-Only SIMD Instructions in Intel® AVX and AVX2

Intel® AVX2	Intel® AVX	Comment
VBROADCASTI128	VBROADCASTF128	256-bit only
VBROADCASTSD ymm1, xmm	VBROADCASTSD ymm1, m64	256-bit only
VBROADCASTSS (from xmm)	VBROADCASTSS (from m32)	
VEXTRACTI128	VEXTRACTF128	256-bit only
VINSERTI128	VINSERTF128	256-bit only
VPMASKMOVD	VMASKMOVPS	
VPMASKMOVQ!	VMASKMOVPD	
	VPERMILPD	in-lane
	VPERMILPS	in-lane
VPERM2I128	VPERM2F128	256-bit only
VPERMD		cross-lane
VPERMPS		cross-lane
VPERMQ		cross-lane
VPERMPD		cross-lane
	VTESTPD	
	VTESTPS	

Table 14-19. VEX-Only SIMD Instructions in Intel® AVX and AVX2 (Contd.)

Intel® AVX2	Intel® AVX	Comment
VPBLENDD		
VPSLLVD/Q		
VPSRAVD		
VPSRLVD/Q		
VGATHERDPD/QPD		
VGATHERDPS/QPS		
VPGATHERDD/QD		
VPGATHERDQ/QQ		

Table 14-20. New Primitive in Intel® AVX2 Instructions

Instruction	Description
VPERMD ymm1, ymm2, ymm3/m256	Permute doublewords in ymm3/m256 using indexes in ymm2 and store the result in ymm1.
VPERMPD ymm1, ymm2/m256, imm8	Permute double precision FP elements in ymm2/m256 using indexes in imm8 and store the result in ymm1.
VPERMPS ymm1, ymm2, ymm3/m256	Permute single precision FP elements in ymm3/m256 using indexes in ymm2 and store the result in ymm1.
VPERMQ ymm1, ymm2/m256, imm8	Permute quadwords in ymm2/m256 using indexes in imm8 and store the result in ymm1.
VPSLLVD xmm1, xmm2, xmm3/m128	Shift doublewords in xmm2 left by amount specified in the corresponding element of xmm3/m128 while shifting in 0s.
VPSLLVQ xmm1, xmm2, xmm3/m128	Shift quadwords in xmm2 left by amount specified in the corresponding element of xmm3/m128 while shifting in 0s.
VPSLLVD ymm1, ymm2, ymm3/m256	Shift doublewords in ymm2 left by amount specified in the corresponding element of ymm3/m256 while shifting in 0s.
VPSLLVQ ymm1, ymm2, ymm3/m256	Shift quadwords in ymm2 left by amount specified in the corresponding element of ymm3/m256 while shifting in 0s.
VPSRAVD xmm1, xmm2, xmm3/m128	Shift doublewords in xmm2 right by amount specified in the corresponding element of xmm3/m128 while shifting in the sign bits.
VPSRLVD xmm1, xmm2, xmm3/m128	Shift doublewords in xmm2 right by amount specified in the corresponding element of xmm3/m128 while shifting in 0s.
VPSRLVQ xmm1, xmm2, xmm3/m128	Shift quadwords in xmm2 right by amount specified in the corresponding element of xmm3/m128 while shifting in 0s.
VPSRLVD ymm1, ymm2, ymm3/m256	Shift doublewords in ymm2 right by amount specified in the corresponding element of ymm3/m256 while shifting in 0s.
VPSRLVQ ymm1, ymm2, ymm3/m256	Shift quadwords in ymm2 right by amount specified in the corresponding element of ymm3/m256 while shifting in 0s.
VGATHERDD xmm1, vm32x, xmm2	Using dword indices specified in vm32x, gather dword values from memory conditioned on mask specified by xmm2. Conditionally gathered elements are merged into xmm1.
VGATHERQD xmm1, vm64x, xmm2	Using qword indices specified in vm64x, gather dword values from memory conditioned on mask specified by xmm2. Conditionally gathered elements are merged into xmm1.
VGATHERDD ymm1, vm32y, ymm2	Using dword indices specified in vm32y, gather dword values from memory conditioned on mask specified by ymm2. Conditionally gathered elements are merged into ymm1.
VGATHERQD ymm1, vm64y, ymm2	Using qword indices specified in vm64y, gather dword values from memory conditioned on mask specified by ymm2. Conditionally gathered elements are merged into ymm1.

Instruction	Description
VGATHERDPD xmm1, vm32x, xmm2	Using dword indices specified in vm32x, gather double precision FP values from memory conditioned on mask specified by xmm2. Conditionally gathered elements are merged into xmm1.
VGATHERQPD xmm1, vm64x, xmm2	Using qword indices specified in vm64x, gather double precision FP values from memory conditioned on mask specified by xmm2. Conditionally gathered elements are merged into xmm1.
VGATHERDPD ymm1, vm32x, ymm2	Using dword indices specified in vm32x, gather double precision FP values from memory conditioned on mask specified by ymm2. Conditionally gathered elements are merged into ymm1.
VGATHERQPD ymm1, vm64y ymm2	Using qword indices specified in vm64y, gather double precision FP values from memory conditioned on mask specified by ymm2. Conditionally gathered elements are merged into ymm1.
VGATHERDPS xmm1, vm32x, xmm2	Using dword indices specified in vm32x, gather single precision FP values from memory conditioned on mask specified by xmm2. Conditionally gathered elements are merged into xmm1.
VGATHERQPS xmm1, vm64x, xmm2	Using qword indices specified in vm64x, gather single precision FP values from memory conditioned on mask specified by xmm2. Conditionally gathered elements are merged into xmm1.
VGATHERDPS ymm1, vm32y, ymm2	Using dword indices specified in vm32y, gather single precision FP values from memory conditioned on mask specified by ymm2. Conditionally gathered elements are merged into ymm1.
VGATHERQPS xmm1, vm64y, xmm2	Using qword indices specified in vm64y, gather single precision FP values from memory conditioned on mask specified by xmm2. Conditionally gathered elements are merged into xmm1.
VGATHERDQ xmm1, vm32x, xmm2	Using dword indices specified in vm32x, gather qword values from memory conditioned on mask specified by xmm2. Conditionally gathered elements are merged into xmm1.
VGATHERQQ xmm1, vm64x, xmm2	Using qword indices specified in vm64x, gather qword values from memory conditioned on mask specified by xmm2. Conditionally gathered elements are merged into xmm1.
VGATHERDQ ymm1, vm32x, ymm2	Using dword indices specified in vm32x, gather qword values from memory conditioned on mask specified by ymm2. Conditionally gathered elements are merged into ymm1.
VGATHERQQ ymm1, vm64y, ymm2	Using qword indices specified in vm64y, gather qword values from memory conditioned on mask specified by ymm2. Conditionally gathered elements are merged into ymm1.

14.7.1 Detection of Intel® AVX2

Hardware support for Intel AVX2 is indicated by CPUID.07H.00H:EBX.AVX2[5] =1.

Application Software must identify that hardware supports Intel AVX, after that it must also detect support for Intel AVX2 by checking CPUID.07H.00H:EBX.AVX2[5]. The recommended pseudocode sequence for detection of Intel AVX2 is:

```
---------------------------------------------------------------------------------
INT supports_avx2()
{       ; result in eax
        mov eax, 1
        cpuid
        and ecx, 018000000H
        cmp ecx, 018000000H; check both OSXSAVE and AVX feature flags
        jne not_supported
        ; processor supports AVX instructions and XGETBV is enabled by OS
        mov eax, 7
```

```
        mov ecx, 0
        cpuid
        and ebx, 20H
        cmp ebx, 20H; check AVX2 feature flags
        jne not_supported
        mov ecx, 0; specify 0 for XCR0 register
        XGETBV; result in EDX:EAX
        and eax, 06H
        cmp eax, 06H; check OS has enabled both XMM and YMM state support
        jne not_supported
        mov eax, 1
        jmp done
        NOT_SUPPORTED:
        mov eax, 0
        done:
}
--------------------------------------------------------------------------------
```

14.8 ACCESSING YMM REGISTERS

The lower 128 bits of a YMM register is aliased to the corresponding XMM register. Legacy SSE instructions (i.e., SIMD instructions operating on XMM state but not using the VEX prefix, also referred to non-VEX encoded SIMD instructions) will not access the upper bits (255:128) of the YMM registers. AVX and FMA instructions with a VEX prefix and vector length of 128-bits zeroes the upper 128 bits of the YMM register.

Upper bits of YMM registers (255:128) can be read and written by many instructions with a VEX.256 prefix.

XSAVE and XRSTOR may be used to save and restore the upper bits of the YMM registers.

14.9 MEMORY ALIGNMENT

Memory alignment requirements on VEX-encoded instruction differs from non-VEX-encoded instructions. Memory alignment applies to non-VEX-encoded SIMD instructions in three categories:

- Explicitly-aligned SIMD load and store instructions accessing 16 bytes of memory (e.g., MOVAPD, MOVAPS, MOVDQA, etc.). These instructions always require memory address to be aligned on 16-byte boundary.
- Explicitly-unaligned SIMD load and store instructions accessing 16 bytes or less of data from memory (e.g., MOVUPD, MOVUPS, MOVDQU, MOVQ, MOVD, etc.). These instructions do not require memory address to be aligned on 16-byte boundary.
- The vast majority of arithmetic and data processing instructions in legacy SSE instructions (non-VEX-encoded SIMD instructions) support memory access semantics. When these instructions access 16 bytes of data from memory, the memory address must be aligned on 16-byte boundary.

Most arithmetic and data processing instructions encoded using the VEX prefix and performing memory accesses have more flexible memory alignment requirements than instructions that are encoded without the VEX prefix. Specifically,

- With the exception of explicitly aligned 16 or 32 byte SIMD load/store instructions, most VEX-encoded, arithmetic and data processing instructions operate in a flexible environment regarding memory address alignment, i.e., VEX-encoded instruction with 32-byte or 16-byte load semantics will support unaligned load operation by default. Memory arguments for most instructions with VEX prefix operate normally without

causing #GP(0) on any byte-granularity alignment (unlike Legacy SSE instructions). The instructions that require explicit memory alignment requirements are listed in Table 14-22.

Software may see performance penalties when unaligned accesses cross cacheline boundaries, so reasonable attempts to align commonly used data sets should continue to be pursued.

Atomic memory operation in Intel 64 and IA-32 architecture is guaranteed only for a subset of memory operand sizes and alignment scenarios. The list of guaranteed atomic operations are described in Section 11.1.1 of the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3A. Intel AVX and FMA instructions do not introduce any new guaranteed atomic memory operations.

Intel AVX instructions can generate an #AC(0) fault on misaligned 4 or 8-byte memory references in Ring-3 when CR0.AM= 1. 16 and 32-byte memory references will not generate #AC(0) fault. See Table 14-21 for details.

Certain Intel AVX instructions always require 16- or 32-byte alignment (see the complete list of such instructions in Table 14-22). These instructions will #GP(0) if not aligned to 16-byte boundaries (for 16-byte granularity loads and stores) or 32-byte boundaries (for 32-byte loads and stores).

Table 14-21. Alignment Faulting Conditions when Memory Access is Not Aligned

EFLAGS.AC==1 && Ring-3 && CR0.AM == 1			0	1
Instruction Type	AVX, FMA,	16- or 32-byte "explicitly unaligned" loads and stores (see Table 14-23)	no fault	no fault
		VEX op YMM, m256	no fault	no fault
		VEX op XMM, m128	no fault	no fault
		"explicitly aligned" loads and stores (see Table 14-22)	#GP(0)	#GP(0)
		2, 4, or 8-byte loads and stores	no fault	#AC(0)
	SSE	16 byte "explicitly unaligned" loads and stores (see Table 14-23)	no fault	no fault
		op XMM, m128	#GP(0)	#GP(0)
		"explicitly aligned" loads and stores (see Table 14-22)	#GP(0)	#GP(0)
		2, 4, or 8-byte loads and stores	no fault	#AC(0)

Table 14-22. Instructions Requiring Explicitly Aligned Memory

Require 16-byte alignment	Require 32-byte alignment
(V)MOVDQA xmm, m128	VMOVDQA ymm, m256
(V)MOVDQA m128, xmm	VMOVDQA m256, ymm
(V)MOVAPS xmm, m128	VMOVAPS ymm, m256
(V)MOVAPS m128, xmm	VMOVAPS m256, ymm
(V)MOVAPD xmm, m128	VMOVAPD ymm, m256
(V)MOVAPD m128, xmm	VMOVAPD m256, ymm
(V)MOVNTPS m128, xmm	VMOVNTPS m256, ymm
(V)MOVNTPD m128, xmm	VMOVNTPD m256, ymm
(V)MOVNTDQ m128, xmm	VMOVNTDQ m256, ymm
(V)MOVNTDQA xmm, m128	VMOVNTDQA ymm, m256

Table 14-23. Instructions Not Requiring Explicit Memory Alignment

(V)MOVDQU xmm, m128
(V)MOVDQU m128, m128
(V)MOVUPS xmm, m128
(V)MOVUPS m128, xmm
(V)MOVUPD xmm, m128
(V)MOVUPD m128, xmm
VMOVDQU ymm, m256
VMOVDQU m256, ymm
VMOVUPS ymm, m256
VMOVUPS m256, ymm
VMOVUPD ymm, m256
VMOVUPD m256, ymm

14.10 SIMD FLOATING-POINT EXCEPTIONS

Intel AVX instructions can generate SIMD floating-point exceptions (#XM) and respond to exception masks in the same way as Legacy SSE instructions. When CR4.OSXMMEXCPT=0 any unmasked FP exceptions generate an Undefined Opcode exception (#UD).

Intel AVX FP exceptions are created in a similar fashion (differing only in number of elements) to Legacy SSE and SSE2 instructions capable of generating SIMD floating-point exceptions.

AVX introduces no new arithmetic operations (AVX floating-point are analogues of existing Legacy SSE instructions).

F16C, FMA instructions can generate SIMD floating-point exceptions (#XM). The requirements that apply to Intel AVX also apply to F16C and FMA.

The subset of Intel AVX2 instructions that operate on floating-point data do not generate #XM.

The detailed exception conditions for Intel AVX instructions and legacy SIMD instructions (excluding instructions that operates on MMX registers) are described in a number of exception class types, depending on the operand syntax and memory operation characteristics. The complete list of SIMD instruction exception class types are defined in Chapter 2, "Instruction Format," of the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 2A.

14.11 EMULATION

Setting the CR0.EMbit to 1 provides a technique to emulate Legacy SSE floating-point instruction sets in software. This technique is not supported with AVX instructions.

If an operating system wishes to emulate AVX instructions, set XCR0[2:1] to zero. This will cause AVX instructions to #UD. Emulation of F16C, AVX2, and FMA by operating system can be done similarly as with emulating AVX instructions.

14.12 WRITING INTEL® AVX FLOATING-POINT EXCEPTION HANDLERS

Intel AVX and FMA floating-point exceptions are handled in an entirely analogous way to Legacy SSE floating-point exceptions. To handle unmasked SIMD floating-point exceptions, the operating system or executive must provide an exception handler. The section titled "SSE and SSE2 SIMD Floating-Point Exceptions" in Chapter 11, "Program-

ming with Streaming SIMD Extensions 2 (SSE2),” describes the SIMD floating-point exception classes and gives suggestions for writing an exception handler to handle them.

To indicate that the operating system provides a handler for SIMD floating-point exceptions (#XM), the CR4.OSXMMEXCPT flag (bit 10) must be set.

The guidelines for writing Intel AVX floating-point exception handlers also apply to F16C and FMA.

14.13 GENERAL PURPOSE INSTRUCTION SET ENHANCEMENTS

Enhancements in the general-purpose instruction set consist of several categories:

- A rich collection of instructions to manipulate integer data at bit-granularity. Most of the bit-manipulation instructions employ VEX-prefix encoding to support three-operand syntax with non-destructive source operands. Two of the bit-manipulating instructions (LZCNT, TZCNT) are not encoded using VEX. The VEX-encoded bit-manipulation instructions include: ANDN, BEXTR, BLSI, BLSMSK, BLSR, BZHI, PEXT, PDEP, SARX, SHLX, SHRX, and RORX.
- Enhanced integer multiply instruction (MULX) in conjunctions with some of the bit-manipulation instructions allow software to accelerate calculation of large integer numerics (wider than 128-bits).
- INVPCID instruction targets system software that manages processor context IDs.

CHAPTER 15
PROGRAMMING WITH INTEL® AVX-512

15.1 OVERVIEW

The Intel AVX-512 family comprises a collection of instruction set extensions, including AVX-512 Foundation, AVX-512 Exponential and Reciprocal instructions, AVX-512 Conflict, AVX-512 Prefetch, and additional 512-bit SIMD instruction extensions, including AVX512-FP16. Intel AVX-512 instructions are natural extensions to Intel AVX and Intel AVX2. Intel AVX-512 introduces the following architectural enhancements:

- Support for 512-bit wide vectors and SIMD register set. 512-bit register state is managed by the operating system using XSAVE/XRSTOR instructions introduced in 45 nm Intel 64 processors (see the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 2B, and the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3A).
- Support for 16 new, 512-bit SIMD registers (for a total of 32 SIMD registers, ZMM0 through ZMM31) in 64-bit mode. The extra 16 registers state is managed by the operating system using XSAVE/XRSTOR/XSAVEOPT.
- Support for 8 new opmask registers (k0 through k7) used for conditional execution and efficient merging of destination operands. The opmask register state is managed by the operating system using the XSAVE/XRSTOR/XSAVEOPT instructions.
- A new encoding prefix (referred to as EVEX) to support additional vector length encoding up to 512 bits. The EVEX prefix builds upon the foundations of the VEX prefix to provide compact, efficient encoding for functionality available to VEX encoding plus the following enhanced vector capabilities:
  - — Opmasks.
  - — Embedded broadcast.
  - — Instruction prefix-embedded rounding control.
  - — Compressed address displacements.

15.1.1 512-Bit Wide SIMD Register Support

Intel AVX-512 instructions support 512-bit wide SIMD registers (ZMM0-ZMM31). The lower 256-bits of the ZMM registers are aliased to the respective 256-bit YMM registers and the lower 128-bit are aliased to the respective 128-bit XMM registers.

15.1.2 32 SIMD Register Support

Intel AVX-512 instructions also support 32 SIMD registers in 64-bit mode (XMM0-XMM31, YMM0-YMM31 and ZMM0-ZMM31). The number of available vector registers in 32-bit mode is still 8.

15.1.3 Eight Opmask Register Support

Intel AVX-512 instructions support 8 opmask registers (k0-k7). The width of each opmask register is architecturally defined as size MAX_KL (64 bits). Seven of the eight opmask registers (k1-k7) can be used in conjunction with EVEX-encoded AVX-512 Foundation instructions to provide conditional execution and efficient merging of data elements in the destination operand. The encoding of opmask register k0 is typically used when all data elements (unconditional processing) are desired. Additionally, the opmask registers are also used as vector flags/element-level vector sources to introduce novel SIMD functionality as seen in new instructions such as VCOMPRESSPS.

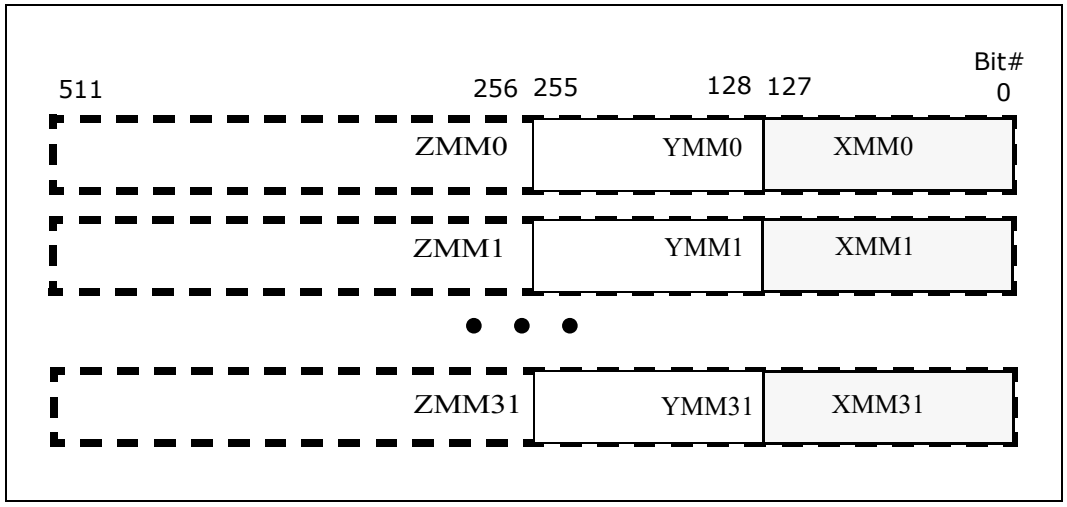


Figure 15-1. 512-Bit Wide Vectors and SIMD Register Set

15.1.4 Instruction Syntax Enhancement

The architecture of EVEX encoding enhances the vector instruction encoding scheme in the following way:

- 512-bit vector-length, up to 32 ZMM registers, and enhanced vector programming environment are supported using the enhanced VEX (EVEX).

The EVEX prefix provides more encodable bit fields than the VEX prefix. In addition to encoding 32 ZMM registers in 64-bit mode, instruction encoding using the EVEX prefix can directly encode 7 (out of 8) opmask register operands to provide conditional processing in vector instruction programming. The enhanced vector programming environment can be explicitly expressed in the instruction syntax to include the following elements:

- An opmask operand: the opmask registers are expressed using the notation "k1" through "k7". An EVEX-encoded instruction supporting conditional vector operation using the opmask register k1 is expressed by attaching the notation {k1} next to the destination operand. The use of this feature is optional for most instructions. There are two types of masking (merging and zeroing) differentiated using the EVEX.z bit ({z} in instruction signature).
- Embedded broadcast may be supported for some instructions on the source operand that can be encoded as a memory vector. Data elements of a memory vector may be conditionally fetched or written to.
- For instruction syntax that operates only on floating-point data in SIMD registers with rounding semantics, the EVEX encoding can provide explicit rounding control within the EVEX bit fields at either scalar or 512-bit vector length.

In AVX-512 instructions, vector addition of all elements of the source operands can be expressed in the same syntax as AVX instruction:

```
VADDPS zmm1, zmm2, zmm3
```

Additionally, the EVEX encoding scheme of AVX-512 Foundation can express conditional vector addition as:

```
VADDPS zmm1 {k1}{z}, zmm2, zmm3
```

where:

- Conditional processing and updates to destination are expressed with an opmask register.
- Zeroing behavior of the opmask selected destination element is expressed by the {z} modifier (with merging as the default if no modifier is specified).

Note that some SIMD instructions supporting three-operand syntax but processing only less than or equal to 128-bits of data are considered part of the 512-bit SIMD instruction set extensions, because bits MAXVL-1:128 of the destination register are zeroed by the processor. The same rule applies to instructions operating on 256-bits of data where bits MAXVL-1:256 of the destination register are zeroed.

15.1.5 EVEX Instruction Encoding Support

Intel AVX-512 instructions employ a new encoding prefix, referred to as EVEX, in the Intel 64 and IA-32 instruction encoding format. Instruction encoding using the EVEX prefix provides the following capabilities:

- Direct encoding of a SIMD register operand within EVEX (similar to VEX). This provides instruction syntax support for three source operands.
- Compaction of REX prefix functionality and extended SIMD register encoding: the equivalent REX-prefix compaction functionality offered by the VEX prefix is provided within EVEX. Furthermore, EVEX extends the operand encoding capability to allow direct addressing of up to 32 ZMM registers in 64-bit mode.
- Compaction of SIMD prefix functionality and escape byte encoding: the functionality of a SIMD prefix (66H, F2H, F3H) on opcode is equivalent to an opcode extension field to introduce new processing primitives. This functionality is provided in the VEX prefix encoding scheme and employed within the EVEX prefix. Similarly, the functionality of the escape opcode byte (0FH) and two-byte escape (0F38H, 0F3AH) are also compacted within the EVEX prefix encoding.
- Most EVEX-encoded SIMD numeric and data processing instruction semantics with memory operands have more relaxed memory alignment requirements than instructions encoded using SIMD prefixes (see Section 15.7, “Memory Alignment”).
- Direct encoding of an opmask operand within the EVEX prefix. This provides instruction syntax support for conditional vector-element operation and merging of destination operand using an opmask register (k1-k7).
- Direct encoding of a broadcast attribute for instructions with a memory operand source. This provides instruction syntax support for elements broadcasting the second operand before being used in the actual operation.
- Compressed memory address displacements for a more compact instruction encoding byte sequence.

EVEX encoding applies to SIMD instructions operating on XMM, YMM, and ZMM registers. EVEX is not supported for instructions operating on MMX or x87 registers. Details of EVEX instruction encoding are discussed in Section 2.7, “Intel® AVX-512 Encoding,” of the Intel® 64 and IA-32 Architectures Software Developer’s Manual, Volume 2A.

15.2 DETECTION OF AVX-512 FOUNDATION INSTRUCTIONS

The majority of AVX-512 Foundation instructions are encoded using the EVEX encoding scheme. EVEX-encoded instructions can operate on the 512-bit ZMM register state plus 8 opmask registers. The opmask instructions in AVX-512 Foundation instructions operate only on opmask registers or with a general purpose register. System software requirements to support the ZMM state and opmask instructions are described in Section 15.5, “Accessing XMM, YMM, AND ZMM Registers.”

Processor support of AVX-512 Foundation instructions is indicated by CPUID.07H.00H:EBX.AVX512F[16] = 1. Detection of AVX-512 Foundation instructions operating on ZMM states and opmask registers needs to follow the general procedural flow in Figure 15-2.

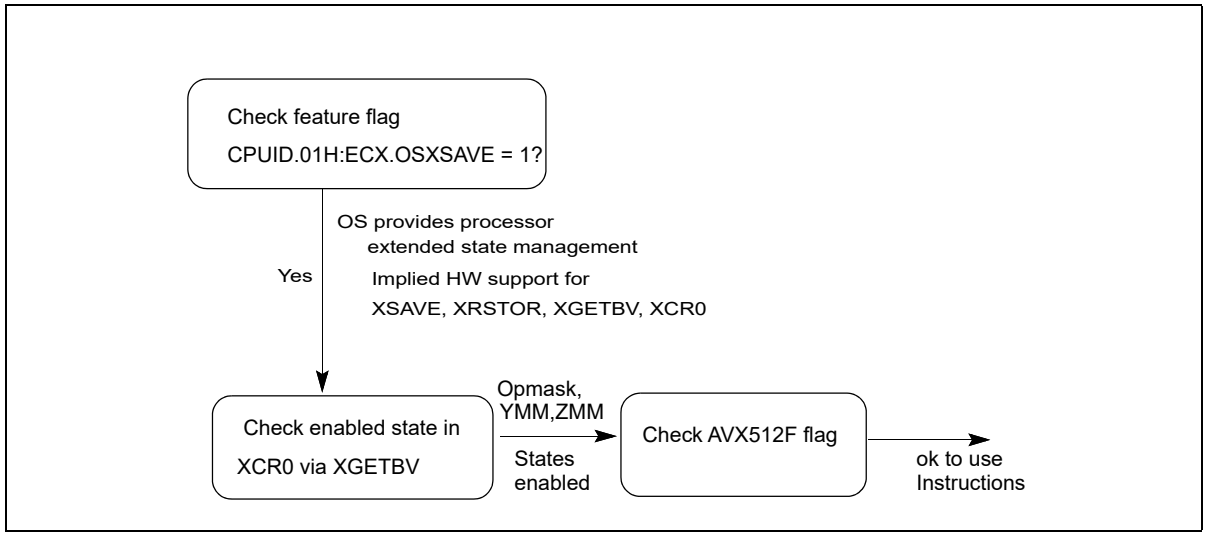


Figure 15-2. Procedural Flow for Application Detection of AVX-512 Foundation Instructions

Prior to using AVX-512 Foundation instructions, the application must identify that the operating system supports the XGETBV instruction and the ZMM register state, in addition to confirming the processor's support for ZMM state management using XSAVE/XRSTOR and AVX-512 Foundation instructions. The following simplified sequence accomplishes both and is strongly recommended.

1. Detect CPUID.01H:ECX.OSXSAVE[27] =1 (XGETBV enabled for application use[1]).
2. Execute XGETBV and verify that XCR0[7:5] = '111b' (OPMASK state, upper 256-bit of ZMM0-ZMM15 and ZMM16-ZMM31 state are enabled by OS) and that XCR0[2:1] = '11b' (XMM state and YMM state are enabled by OS).
3. Detect CPUID.07H.00H:EBX.AVX512F[16] = 1.

15.2.1 Additional 512-Bit Instruction Extensions of the Intel® AVX-512 Family

Processor support of the Intel AVX-512 Exponential and Reciprocal instructions are indicated by querying the feature flag:

- If CPUID.07H.00H:EBX.AVX512_ER[27] = 1, the collection of VEXP2PD/VEXP2PS/VRCP28xx/VRSQRT28xx instructions are supported.

Processor support of the Intel AVX-512 Prefetch instructions are indicated by querying the feature flag:

- If CPUID.07H.00H:EBX.AVX512_PF[26] = 1, a collection of VGATHERPF0xxx/VGATHERPF1xxx/VSCATTER-PF0xxx/VSCATTERPF1xxx instructions are supported.

Detection of 512-bit instructions operating on ZMM states and opmask registers, outside of AVX-512 Foundation, needs to follow the general procedural flow in Figure 15-3.

1. If CPUID.01H:ECX.OSXSAVE reports 1, it also indirectly implies the processor supports XSAVE, XRSTOR, XGETBV, processor extended state bit vector XCR0 register. Thus an application may streamline the checking of CPUID feature flags for XSAVE and OSXSAVE. XSETBV is a privileged instruction.

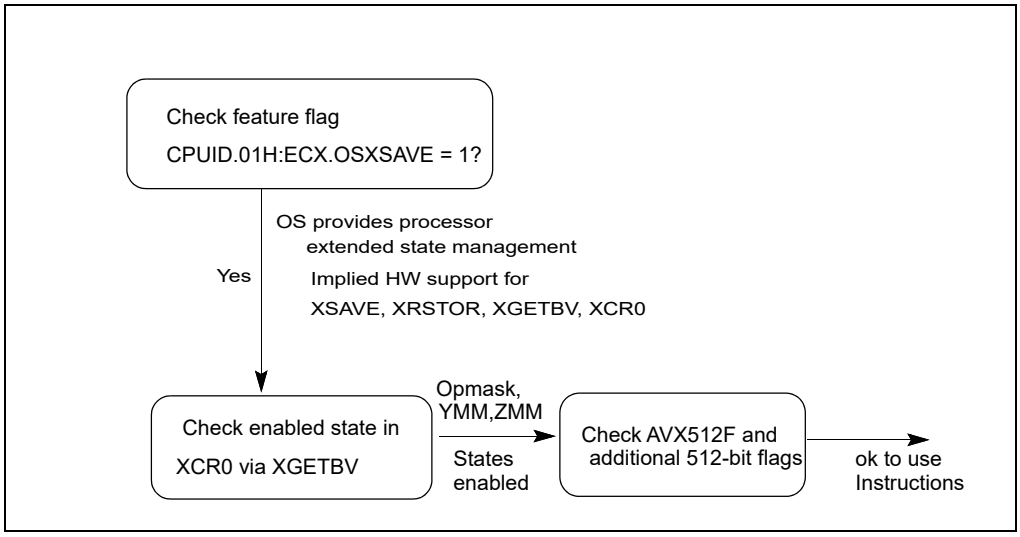


Figure 15-3. Procedural Flow for Application Detection of 512-Bit Instructions

PREFETCHT1W does not require OS support for XMM/YMM/ZMM/k-reg, SIMD FP exception support.

Procedural Flow of Application Detection of other 512-bit extensions:

Prior to using the Intel AVX-512 Exponential and Reciprocal instructions, the application must identify that the operating system supports the XGETBV instruction and the ZMM register state, in addition to confirming the processor's support for ZMM state management using XSAVE/XRSTOR and AVX-512 Foundation instructions. The following simplified sequence accomplishes both and is strongly recommended.

1. Detect CPUID.01H:ECX.OSXSAVE[27] = 1 (XGETBV enabled for application use).
2. Execute XGETBV and verify that XCR0[7:5] = '111b' (OPMASK state, upper 256-bit of ZMM0-ZMM15 and ZMM16-ZMM31 state are enabled by OS) and that XCR0[2:1] = '11b' (XMM state and YMM state are enabled by OS).
3. Verify both CPUID.07H.00H:EBX.AVX512F[16] = 1, and CPUID.07H.00H:EBX.AVX512_ER[27] = 1.

Prior to using the Intel AVX-512 Prefetch instructions, the application must identify that the operating system supports the XGETBV instruction and the ZMM register state, in addition to confirming the processor's support for ZMM state management using XSAVE/XRSTOR and AVX-512 Foundation instructions. The following simplified sequence accomplishes both and is strongly recommended.

1. Detect CPUID.01H:ECX.OSXSAVE[27] = 1 (XGETBV enabled for application use).
2. Execute XGETBV and verify that XCR0[7:5] = '111b' (OPMASK state, upper 256-bit of ZMM0-ZMM15 and ZMM16-ZMM31 state are enabled by OS) and that XCR0[2:1] = '11b' (XMM state and YMM state are enabled by OS).
3. Verify both CPUID.07H.00H:EBX.AVX512F[16] = 1, and CPUID.07H.00H:EBX.AVX512_PF[26] = 1.

15.2.2 Detection of AVX512-FP16 Instructions

The AVX512-FP16 ISA extensions require that the AVX512BW feature be implemented since the instructions for manipulating 32b masks are associated with AVX512BW.

15.3 DETECTION OF 512-BIT INSTRUCTION GROUPS OF THE INTEL® AVX-512 FAMILY

In addition to the Intel AVX-512 Foundation instructions, the Intel AVX-512 family provides several groups of instruction extensions that can operate in vector lengths of 512/256/128 bits. Each group is enumerated by a CPUID.07H feature flag and can be encoded via the EVEX.L'L field to support operation at vector lengths smaller than 512 bits. These instruction groups are listed in Table 15-1.

Table 15-1. 512-Bit Instruction Groups in the Intel® AVX-512 Family

CPUID.07H Feature Flag Bit	Feature Flag Abbreviation of 512-Bit Instruction Group	SW Detection Flow
CPUID.07H.00H:EBX[16]	AVX512F (AVX-512 Foundation)	Figure 15-2
CPUID.07H.00H:EBX[28]	AVX512CD	Figure 15-4
CPUID.07H.00H:EBX[17]	AVX512DQ	Figure 15-4
CPUID.07H.00H:EBX[30]	AVX512BW	Figure 15-4

Software must follow the detection procedure for the 512-bit AVX-512 Foundation instructions as described in Section 15.2.

Detection of other 512-bit sibling instruction groups listed in Table 15-1 (excluding AVX512F) follows the procedure described in Figure 15-4.

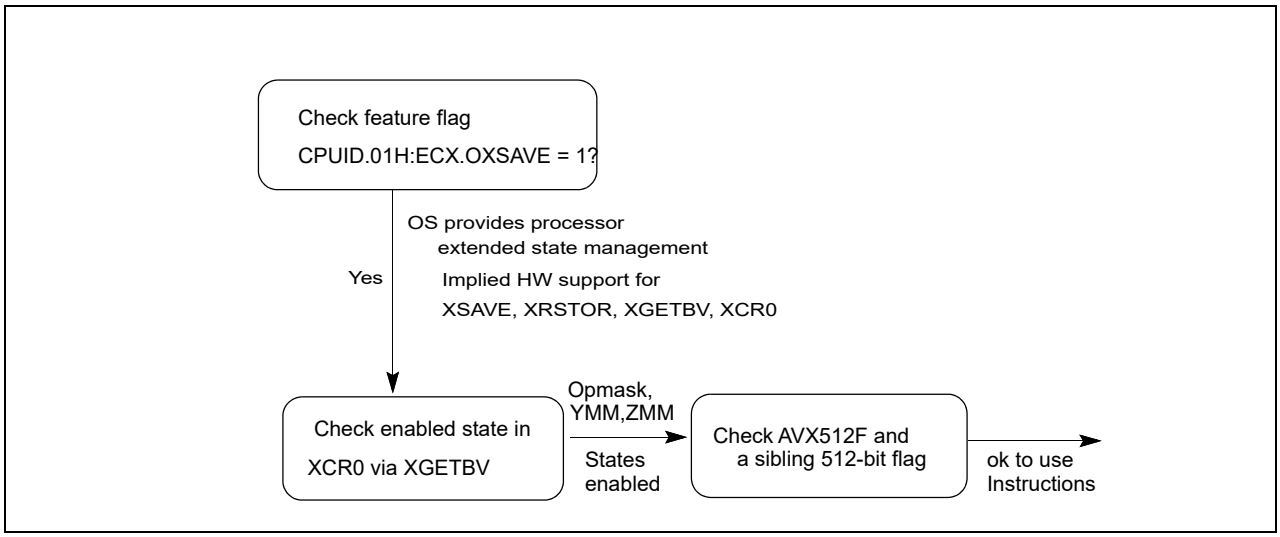


Figure 15-4. Procedural Flow for Application Detection of 512-Bit Instruction Groups

To detect 512-bit instructions enumerated by AVX512CD, the following sequence is strongly recommended.

1. Detect CPUID.01H:ECX.OSXSAVE[27] = 1 (XGETBV enabled for application use).
2. Execute XGETBV and verify that XCR0[7:5] = '111b' (OPMASK state, upper 256-bit of ZMM0-ZMM15 and ZMM16-ZMM31 state are enabled by OS) and that XCR0[2:1] = '11b' (XMM state and YMM state are enabled by OS).
3. Verify both CPUID.07H.00H:EBX.AVX512F[16] = 1, CPUID.07H.00H:EBX.AVX512CD[28] = 1.

Similarly, the detection procedure for enumerating 512-bit instructions reported by AVX512DW follows the same flow.

15.4 DETECTION OF INTEL® AVX-512 INSTRUCTION GROUPS OPERATING AT 256 AND 128-BIT VECTOR LENGTHS

For each of the 512-bit instruction groups in the Intel AVX-512 family listed in Table 15-1, the EVEX encoding scheme may support a vast majority of these instructions operating at 256-bit or 128-bit (if applicable) vector lengths. Encoding support for vector lengths smaller than 512-bits is indicated by CPUID.07H.00H:EBX[31], abbreviated as AVX512VL.

The AVX512VL flag alone is never sufficient to determine a given Intel AVX-512 instruction may be encoded at vector lengths smaller than 512 bits. Software must use the procedure described in Figure 15-5 and Table 15-2.

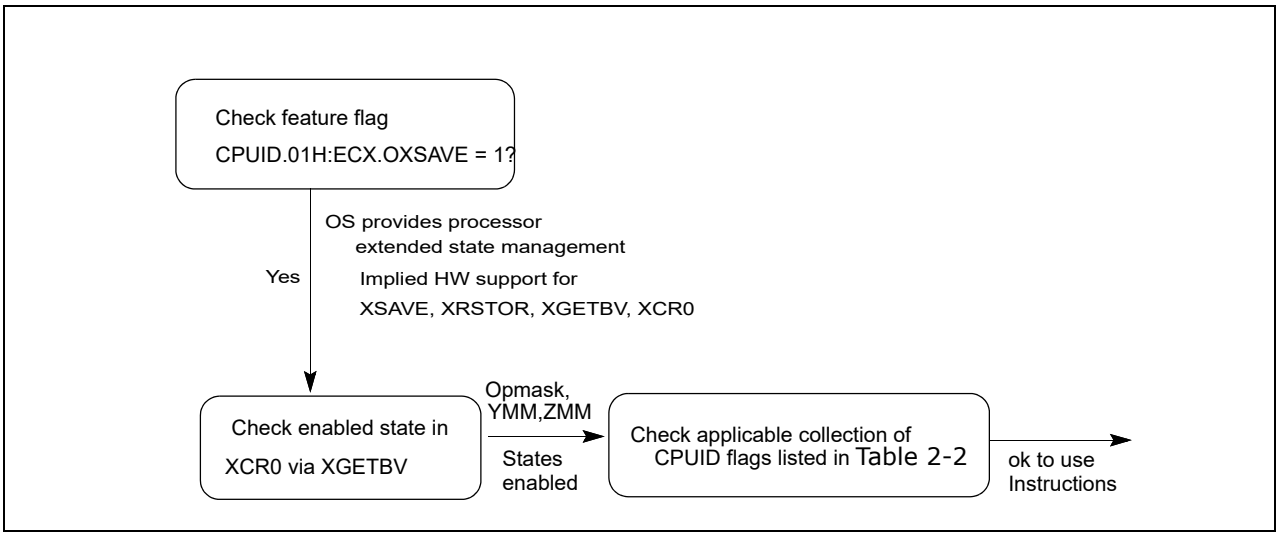


Figure 15-5. Procedural Flow for Detection of Intel® AVX-512 Instructions Operating at Vector Lengths < 512

To illustrate the procedure described in Figure 15-5 and Table 15-2 for software to use EVEX.256 encoded VPCONFLICT, the following sequence is provided. It is strongly recommended this sequence is followed.

1) Detect CPUID.01H:ECX.OSXSAVE[27] = 1 (XGETBV enabled for application use).

2) Execute XGETBV and verify that XCR0[7:5] = ‘111b’ (OPMASK state, upper 256-bit of ZMM0-ZMM15 and ZMM16-ZMM31 state are enabled by OS) and that XCR0[2:1] = ‘11b’ (XMM state and YMM state are enabled by OS).

3) Verify CPUID.07H.00H:EBX.AVX512F[16] = 1, CPUID.07H.00H:EBX.AVX512CD[28] = 1, and CPUID.07H.00H:EBX.AVX512VL[31] = 1.

Table 15-2. Feature Flag Collection Required of 256/128 Bit Vector Lengths for Each Instruction Group

Usage of 256/128 Vector Lengths	Feature Flag Collection to Verify
AVX512F	AVX512F & AVX512VL
AVX512CD	AVX512F & AVX512CD & AVX512VL
AVX512DQ	AVX512F & AVX512DQ & AVX512VL
AVX512BW	AVX512F & AVX512BW & AVX512VL

In some specific cases, AVX512VL may only support EVEX.256 encoding but not EVEX.128. These cases are listed in Table 15-3.

Table 15-3. Instruction Mnemonics That Do Not Support EVEX.128 Encoding

Instruction Group	Instruction Mnemonics Supporting EVEX.256 Only Using AVX512VL
AVX512F	VBROADCASTSD, VBROADCASTF32X4, VEXTRACTI32X4, VINSERTF32X4, VINSERTI32X4, VPERMD, VPERMPD, VPERMPS, VPERMQ, VSHUFF32X4, VSHUFF64X2, VSHUFI32X4, VSHUFI64X2
AVX512CD	
AVX512DQ	VBROADCASTF32X2, VBROADCASTF64X2, VBROADCASTI32X4, VBROADCASTI64X2, VEXTRACTI64X2, VINSERTF64X2, VINSERTI64X2,
AVX512BW	

15.5 ACCESSING XMM, YMM, AND ZMM REGISTERS

The lower 128 bits of a YMM register is aliased to the corresponding XMM register. Legacy SSE instructions (i.e., SIMD instructions operating on XMM state but not using the VEX prefix, also referred to non-VEX encoded SIMD instructions) will not access the upper bits (MAXVL-1:128) of the YMM registers. AVX and FMA instructions with a VEX prefix and vector length of 128-bits zeroes the upper 128 bits of the YMM register.

Upper bits of YMM registers (255:128) can be read and written to by many instructions with a VEX.256 prefix.

XSAVE and XRSTOR may be used to save and restore the upper bits of the YMM registers.

The lower 256 bits of a ZMM register are aliased to the corresponding YMM register. Legacy SSE instructions (i.e., SIMD instructions operating on XMM state but not using the VEX prefix, also referred to non-VEX encoded SIMD instructions) will not access the upper bits (MAXVL-1:128) of the ZMM registers, where MAXVL is maximum vector length (currently 512 bits). AVX and FMA instructions with a VEX prefix and vector length of 128-bits zero the upper 384 bits of the ZMM register, while the VEX prefix and vector length of 256-bits zeroes the upper 256 bits of the ZMM register.

Upper bits of ZMM registers (511:256) can be read and written to by instructions with an EVEX.512 prefix.

15.6 ENHANCED VECTOR PROGRAMMING ENVIRONMENT USING EVEX ENCODING

EVEX-encoded AVX-512 instructions support an enhanced vector programming environment. The enhanced vector programming environment uses the combination of EVEX bit-field encodings and a set of eight opmask registers to provide the following capabilities:

- Conditional vector processing of an EVEX-encoded instruction. Opmask registers k1 through k7 can be used to conditionally govern the per-data-element computational operation and the per-element updates to the destination operand of an AVX-512 Foundation instruction. Each bit of the opmask register governs one vector element operation (a vector element can be 8 bits, 16 bits, 32 bits or 64 bits).
- In addition to providing predication control on vector instructions via EVEX bit-field encoding, the opmask registers can also be used similarly on general-purpose registers as source/destination operands using modR/M encoding for non-mask-related instructions. In this case, an opmask register k0 through k7 can be selected.
- In 64-bit mode, 32 vector registers can be encoded using the EVEX prefix.
- Broadcast may be supported for some instructions on the operand that can be encoded as a memory vector. The data elements of a memory vector may be conditionally fetched or written to, and the vector size is dependent on the data transformation function.
- Flexible rounding control for the register-to-register flavor of EVEX encoded 512-bit and scalar instructions. Four rounding modes are supported by direct encoding within the EVEX prefix, overriding MXCSR settings.
- Broadcast of one element to the rest of the destination vector register.
- Compressed 8-bit displacement encoding scheme to increase the instruction encoding density for instructions that normally require disp32 syntax.

15.6.1 OPMASK Register to Predicate Vector Data Processing

AVX-512 instructions using EVEX encode a predicate operand to conditionally control per-element computational operation and updating of the result to the destination operand. The predicate operand is known as the opmask register. The opmask is a set of eight architectural registers of size MAX_KL (64-bit). Note that from this set of eight architectural registers, only k1 through k7 can be addressed as a predicate operand. k0 can be used as a regular source or destination but cannot be encoded as a predicate operand. Note also that a predicate operand can be used to enable memory fault-suppression for some instructions with a memory operand (source or destination).

As a predicate operand, the opmask registers contain one bit to govern the operation/update to each data element of a vector register. In general, opmask registers can support instructions with all element sizes: byte (int8), word (int16), single precision floating-point (float32), integer doubleword (int32), double precision floating-point (float64), integer quadword (int64). Therefore, a ZMM vector register can hold 8, 16, 32, or 64 elements in principle. The length of an opmask register, MAX_KL, is sufficient to handle up to 64 elements with one bit per element, i.e., 64 bits. Masking is supported in most of the AVX-512 instructions. For a given vector length, each instruction accesses only the number of least significant mask bits that are needed based on its data type. For example, AVX-512 Foundation instructions operating on 64-bit data elements with a 512-bit vector length, only use the 8 least significant bits of the opmask register.

An opmask register affects an AVX-512 instruction at per-element granularity. Any numeric or non-numeric operation of each data element and per-element updates of intermediate results to the destination operand are predicated on the corresponding bit of the opmask register.

An opmask serving as a predicate operand in AVX-512 obeys the following properties:

- The instruction's operation is not performed for an element if the corresponding opmask bit is not set. This implies that no exception or violation can be caused by an operation on a masked-off element. Consequently, no MXCSR exception flag is updated as a result of a masked-off operation.
- A destination element is not updated with the result of the operation if the corresponding writemask bit is not set. Instead, the destination element value must be preserved (merging-masking) or it must be zeroed out (zeroing-masking).
- For some instructions with a memory operand, memory faults are suppressed for elements with a mask bit of 0.

Note that this feature provides a versatile construct to implement control-flow predication as the mask in effect provides a merging behavior for AVX-512 vector register destinations. As an alternative the masking can be used for zeroing instead of merging, so that the masked out elements are updated with 0 instead of preserving the old value. The zeroing behavior is provided to remove the implicit dependency on the old value when it is not needed.

Most instructions with masking enabled accept both forms of masking. Instructions that must have EVEX.aaa bits different than 0 (gather and scatter) and instructions that write to memory only accept merging-masking.

It's important to note that the per-element destination update rule also applies when the destination operand is a memory location. Vectors are written on a per element basis, based on the opmask register used as a predicate operand.

The value of an opmask register can be:

- Generated as a result of a vector instruction (e.g., CMP, FPCLASS, etc.).
- Loaded from memory.
- Loaded from a GPR register.
- Modified by mask-to-mask operations.

Opmask registers can be used for purposes outside of predication. For example, they can be used to manipulate sparse sets of elements from a vector, or used to set the EFLAGS based on the 0/0xFFFFFFFFFFFFFFFF/other status of the OR of two opmask registers.

15.6.1.1 Opmask Register K0

The only exception to the opmask rules described above is that opmask k0 cannot be used as a predicate operand. Opmask k0 cannot be encoded as a predicate operand for a vector operation; the encoding value that would select opmask k0 will instead select an implicit opmask value of 0xFFFFFFFFFFFFFFFF, thereby effectively disabling

masking. Opmask register k0 can still be used for any instruction that takes opmask register(s) as operand(s) (either source or destination).

Note that certain instructions implicitly use the opmask as an extra destination operand. In such cases, trying to use the "no mask" feature will translate into a #UD fault being raised.

15.6.1.2 Example of Opmask Usages

The example below illustrates the predicated vector add operation and predicated updates of added results into the destination operand. The initial state of vector registers zmm0, zmm1, and zmm2 and k3 are:

```
MSB.....................................LSB

zmm0 =
[ 0x00000003 0x00000002 0x00000001 0x00000000 ] (bytes 15 through 0)
[ 0x00000007 0x00000006 0x00000005 0x00000004 ] (bytes 31 through 16)
[ 0x0000000B 0x0000000A 0x00000009 0x00000008 ] (bytes 47 through 32)
[ 0x0000000F 0x0000000E 0x0000000D 0x0000000C ] (bytes 63 through 48)


zmm1 =
[ 0x0000000F 0x0000000F 0x0000000F 0x0000000F ] (bytes 15 through 0)
[ 0x0000000F 0x0000000F 0x0000000F 0x0000000F ] (bytes 31 through 16)
[ 0x0000000F 0x0000000F 0x0000000F 0x0000000F ] (bytes 47 through 32)
[ 0x0000000F 0x0000000F 0x0000000F 0x0000000F ] (bytes 63 through 48)


zmm2 =
[ 0xAAAAAAAA 0xAAAAAAAA 0xAAAAAAAA 0xAAAAAAAA ] (bytes 15 through 0)
[ 0xBBBBBBBB 0xBBBBBBBB 0xBBBBBBBB 0xBBBBBBBB ] (bytes 31 through 16)
[ 0xCCCCCCCC 0xCCCCCCCC 0xCCCCCCCC 0xCCCCCCCC ] (bytes 47 through 32)
[ 0xDDDDDDDD 0xDDDDDDDD 0xDDDDDDDD 0xDDDDDDDD ] (bytes 63 through 48)


k3 = 0x8F03 (1000 1111 0000 0011)
```

An opmask register serving as a predicate operand is expressed as a curly-braces-enclosed decorator following the first operand in the Intel assembly syntax. Given this state, we will execute the following instruction:

```
vpaddd zmm2 {k3}, zmm0, zmm1
```

The vpaddd instruction performs 32-bit integer additions on each data element conditionally based on the corresponding bit value in the predicate operand k3. Since per-element operations are not operated if the corresponding bit of the predicate mask is not set, the intermediate result is:

```
[ ********** ********** 0x00000010 0x0000000F ] (bytes 15 through 0)
[ ********** ********** ********** ********** ] (bytes 31 through 16)
[ 0x0000001A 0x00000019 0x00000018 0x00000017 ] (bytes 47 through 32)
[ 0x0000001E ********** ********** ********** ] (bytes 63 through 48)
```

where "**********" indicates that no operation is performed.

This intermediate result is then written into the destination vector register, zmm2, using the opmask register k3 as the writemask, producing the following final result:

```
zmm2 =
[ 0xAAAAAAAA 0xAAAAAAAA 0x00000010 0x0000000F ] (bytes 15 through 0)
[ 0xBBBBBBBB 0xBBBBBBBB 0xBBBBBBBB 0xBBBBBBBB ] (bytes 31 through 16)
[ 0x0000001A 0x00000019 0x00000018 0x00000017 ] (bytes 47 through 32)
[ 0x0000001E 0xDDDDDDDD 0xDDDDDDDD 0xDDDDDDDD ] (bytes 63 through 48)
```

Note that for a 64-bit instruction (for example, vaddpd), only the 8 LSB of mask k3 (0x03) would be used to identify the predicate operation on each one of the 8 elements of the source/destination vectors.

15.6.2 OpMask Instructions

AVX-512 Foundation instructions provide a collection of opmask instructions that allow programmers to set, copy, or operate on the contents of a given opmask register. There are three types of opmask instructions:

- **Mask read/write instructions**: These instructions move data between a general-purpose integer register or memory and an opmask mask register, or between two opmask registers. For example:
  - kmovw k1, ebx; move lower 16 bits of ebx to k1.
- **Flag instructions**: This category consists of instructions that modify EFLAGS based on the content of opmask registers.
  - kortestw k1, k2; OR registers k1 and k2 and updated EFLAGS accordingly.
- **Mask logical instructions**: These instructions perform standard bitwise logical operations between opmask registers.
  - kandw k1, k2, k3; AND lowest 16 bits of registers k2 and k3, leaving the result in k1.

15.6.3 Broadcast

EVEX encoding provides a bit-field to encode data broadcast for some load-op instructions, i.e., instructions that load data from memory and perform some computational or data movement operation. A source element from memory can be broadcasted (repeated) across all the elements of the effective source operand (up to 16 times for a 32-bit data element, up to 8 times for a 64-bit data element). This is useful when reusing the same scalar operand for all the operations in a vector instruction. Note that some processors may perform multiple loads of the source element and thus software should not rely on atomicity of the data being broadcast (e.g., when the source element is simultaneously modified by another logical processor).

Broadcast is only enabled on instructions with an element size of 32 bits or 64 bits. Byte and word instructions do not support embedded broadcast.

The functionality of data broadcast is expressed as a curly-braces-enclosed decorator following the last register/memory operand in the Intel assembly syntax.

For instance:

```
vmulps zmm1, zmm2, [rax] {1to16}
```

The {1to16} primitive loads one float32 (single precision) element from memory, replicates it 16 times to form a vector of 16 32-bit floating-point elements, multiplies the 16 float32 elements with the corresponding elements in the first source operand vector, and puts each of the 16 results into the destination operand.

AVX-512 instructions with store semantics and pure load instructions do not support broadcast primitives.

```
vmovaps [rax] {k3}, zmm19
```

In contrast, the k3 opmask register is used as the predicate operand in the above example. Only the store operation on data elements corresponding to the non-zero bits in k3 will be performed.

15.6.4 Static Rounding Mode and Suppress All Exceptions

In previous SIMD instruction extensions (up to AVX and AVX2), rounding control is generally specified in MXCSR, with a handful of instructions providing per-instruction rounding override via encoding fields within the imm8 operand. AVX-512 offers a more flexible encoding attribute to override MXCSR-based rounding control for floating-pointing instructions with rounding semantics. This rounding attribute embedded in the EVEX prefix is called Static (per instruction) Rounding Mode or Rounding Mode override. This attribute allows programmers to statically apply a specific arithmetic rounding mode irrespective of the value of RM bits in MXCSR. It is available only to register-to-register flavors of EVEX-encoded floating-point instructions with rounding semantic. The differences between these three rounding control interfaces are summarized in Table 15-4.

Table 15-4. Characteristics of Three Rounding Control Interfaces

Rounding Interface	Static Rounding Override	Imm8 Embedded Rounding Override	MXCSR Rounding Control
Semantic Requirement	FP rounding	FP rounding	FP rounding
Prefix Requirement	EVEX.B = 1	NA	NA
Rounding Control	EVEX.L'L	IMM8[1:0] or MXCSR.RC (depending on IMM8[2])	MXCSR.RC
Suppress All Exceptions (SAE)	Implied	no	no
SIMD FP Exception #XM	All suppressed	Can raise #I, #P (unless SPE is set)	MXCSR masking controls
MXCSR flag update	No	yes (except PE if SPE is set)	Yes
Precedence	Above MXCSR.RC	Above EVEX.L'L	Default
Scope	512-bit, reg-reg, Scalar reg-reg	ROUNDPx, ROUNDSx, VCVTPS2PH, VRNDSCALExx	All SIMD operands, vector lengths

The static rounding-mode override in Intel AVX-512 also implies the "suppress-all-exceptions" (SAE) attribute. The SAE effect is as if all the MXCSR mask bits are set, and none of the MXCSR flags will be updated. Using static rounding-mode via EVEX without SAE is not supported.

Static Rounding Mode and SAE control can be enabled in the encoding of the instruction by setting the EVEX.b bit to 1 in a register-register vector instruction. In such a case, vector length is assumed to be MAXVL (512-bit in case of AVX-512 packed vector instructions) or 128-bit for scalar instructions. Table 15-5 summarizes the possible static rounding-mode assignments in AVX-512 instructions.

Note that some instructions already allow specifying the rounding mode statically via immediate bits. In such cases, the immediate bits take precedence over the embedded rounding mode (in the same vein that they take precedence over whatever MXCSR.RM says).

Table 15-5. Static Rounding Mode

Function	Description
{rn-sae}	Round to nearest (even) + SAE
{rd-sae}	Round down (toward -inf) + SAE
{ru-sae}	Round up (toward +inf) + SAE
{rz-sae}	Round toward zero (Truncate) + SAE

An example of use would be as follows:

```
vaddps zmm7 {k6}, zmm2, zmm4, {rd-sae}
```

This would perform the single precision floating-point addition of vectors zmm2 and zmm4 with round-towards-minus-infinity, leaving the result in vector zmm7 using k6 as conditional writemask.

Note that MXCSR.RM bits are ignored and unaffected by the outcome of this instruction.

Examples of instruction instances where the static rounding-mode is not allowed are shown below:

```
; rounding-mode already specified in the instruction immediate
vrndscaleps zmm7 {k6}, zmm2, 0x00

; instructions with memory operands
vmulps zmm7 {k6}, zmm2,[rax], {rd-sae}

; instructions with vector length different than MAXVL (512-bit)
vaddps ymm7 {k6}, ymm2, ymm4,{rd-sae}
```

15.6.5 Compressed Disp8*N Encoding

EVEX encoding supports a new displacement representation that allows for a more compact encoding of memory addressing commonly used in unrolled code, where an 8-bit displacement can address a range exceeding the dynamic range of an 8-bit value. This compressed displacement encoding is referred to as disp8*N, where N is a constant implied by the memory operation characteristic of each instruction.

The compressed displacement is based on the assumption that the effective displacement (of a memory operand occurring in a loop) is a multiple of the granularity of the memory access of each iteration. Since the base register in memory addressing already provides byte-granular resolution, the lower bits of the traditional disp8 operand become redundant, and can be implied from the memory operation characteristic.

The memory operation characteristics depend on the following:

- The destination operand is updated as a full vector, a single element, or multi-element tuples.
- The memory source operand (or vector source operand if the destination operand is memory) is fetched (or treated) as a full vector, a single element, or multi-element tuples.

For example:

```
vaddps zmm7, zmm2, disp8[membase + index*8]
```

The destination zmm7 is updated as a full 512-bit vector, and 64-bytes of data are fetched from memory as a full vector; the next unrolled iteration may fetch from memory in 64-byte granularity per iteration. There are 6 bits of lowest address that can be compressed, hence N = 2^6 = 64. The contribution of “disp8” to effective address calculation is 64*disp8.

```
vbroadcastf32x4 zmm7, disp8[membase + index*8]
```

In VBROADCASTF32x4, memory is fetched as a 4tuple of 4 32-bit entities. Hence the common lowest address bits that can be compressed are 4, corresponding to the 4tuple width of 2^4 = 16 bytes (4x32 bits). Therefore, N = 2^4.

For EVEX encoded instructions that update only one element in the destination, or the source element is fetched individually, the number of lowest address bits that can be compressed is generally the width in bytes of the data element, hence N = 2^(width).

15.7 MEMORY ALIGNMENT

Memory alignment requirements on EVEX-encoded SIMD instructions are similar to VEX-encoded SIMD instructions. Memory alignment applies to EVEX-encoded SIMD instructions in three categories:

- Explicitly-aligned SIMD load and store instructions accessing 64 bytes of memory with EVEX prefix encoded vector length of 512 bits (e.g., VMOVAPD, VMOVAPS, VMOVDQA, etc.). These instructions always require the memory address to be aligned on a 64-byte boundary.

- Explicitly-unaligned SIMD load and store instructions accessing 64 bytes or less of data from memory (e.g., VMOVUPD, VMOVUPS, VMOVDQU, VMOVQ, VMOVD, etc.). These instructions do not require the memory address to be aligned on a natural vector-length byte boundary.
- Most arithmetic and data processing instructions encoded using EVEX support memory access semantics. When these instructions access from memory, there are no alignment restrictions.

Software may see performance penalties when unaligned accesses cross cacheline boundaries or vector-length naturally-aligned boundaries, so reasonable attempts to align commonly used data sets should continue to be pursued.

Atomic memory operation in Intel 64 and IA-32 architecture is guaranteed only for a subset of memory operand sizes and alignment scenarios. The guaranteed atomic operations are described in Section 11.1.1, "Guaranteed Atomic Operations," of the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3A. Intel AVX and FMA instructions do not introduce any new guaranteed atomic memory operations.

Intel AVX-512 instructions may generate an #AC(0) fault on misaligned 4 or 8-byte memory references in Ring-3 when CR0.AM=1. 16, 32, and 64-byte memory references will not generate an #AC(0) fault. See Table 15-7 for details.

Certain AVX-512 Foundation instructions always require 64-byte alignment (see the complete list of VEX and EVEX encoded instructions in Table 15-6). These instructions will #GP(0) if not aligned to 64-byte boundaries.

Table 15-6. SIMD Instructions Requiring Explicitly Aligned Memory

Require 16-byte alignment	Require 32-byte alignment	Require 64-byte alignment*
(V)MOVDQA xmm, m128	VMOVDQA ymm, m256	VMOVDQA zmm, m512
(V)MOVDQA m128, xmm	VMOVDQA m256, ymm	VMOVDQA m512, zmm
(V)MOVAPS xmm, m128	VMOVAPS ymm, m256	VMOVAPS zmm, m512
(V)MOVAPS m128, xmm	VMOVAPS m256, ymm	VMOVAPS m512, zmm
(V)MOVAPD xmm, m128	VMOVAPD ymm, m256	VMOVAPD zmm, m512
(V)MOVAPD m128, xmm	VMOVAPD m256, ymm	VMOVAPD m512, zmm
(V)MOVNTDQA xmm, m128	VMOVNTPS m256, ymm	VMOVNTPS m512, zmm
(V)MOVNTPS m128, xmm	VMOVNTPD m256, ymm	VMOVNTPD m512, zmm
(V)MOVNTPD m128, xmm	VMOVNTDQ m256, ymm	VMOVNTDQ m512, zmm
(V)MOVNTDQ m128, xmm	VMOVNTDQA ymm, m256	VMOVNTDQA zmm, m512

Table 15-7. Instructions Not Requiring Explicit Memory Alignment

(V)MOVDQU xmm, m128	VMOVDQU ymm, m256	VMOVDQU zmm, m512
(V)MOVDQU m128, m128	VMOVDQU m256, ymm	VMOVDQU m512, zmm
(V)MOVUPS xmm, m128	VMOVUPS ymm, m256	VMOVUPS zmm, m512
(V)MOVUPS m128, xmm	VMOVUPS m256, ymm	VMOVUPS m512, zmm
(V)MOVUPD xmm, m128	VMOVUPD ymm, m256	VMOVUPD zmm, m512
(V)MOVUPD m128, xmm	VMOVUPD m256, ymm	VMOVUPD m512, zmm

15.8 SIMD FLOATING-POINT EXCEPTIONS

AVX-512 instructions can generate SIMD floating-point exceptions (#XM) if embedded "suppress all exceptions" (SAE) in EVEX is not set. When SAE is not set, these instructions will respond to exception masks of MXCSR in the same way as VEX-encoded AVX instructions. When CR4.OSXMMEXCPT=0, any unmasked FP exceptions generate an Undefined Opcode exception (#UD).

15.9 INSTRUCTION EXCEPTION SPECIFICATION

Exception behavior of VEX-encoded Intel AVX and Intel AVX2 instructions are described in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 2A. Exception behavior of Intel AVX-512 Foundation instructions and additional 512-bit extensions are described in Section 2.8, "Exception Classifications of EVEX-Encoded instructions," and Section 2.9, "Exception Classifications of Opmask instructions, Type K20 and Type K21."

15.10 EMULATION

Setting the CR0.EM bit to 1 provides a technique to emulate legacy Intel SSE floating-point instruction sets in software. This technique is not supported with Intel AVX instructions, nor FMA instructions.

If an operating system wishes to emulate Intel AVX instructions, set XCR0[2:1] to zero. This will cause Intel AVX instructions to #UD. Emulation of FMA by the operating system can be done similarly as with emulating Intel AVX instructions.

15.11 WRITING FLOATING-POINT EXCEPTION HANDLERS

Intel AVX-512, Intel AVX, and FMA floating-point exceptions are handled in an entirely analogous way to legacy SSE floating-point exceptions. To handle unmasked SIMD floating-point exceptions, the operating system or executive must provide an exception handler. Section 11.5.1, "SIMD Floating-Point Exceptions," describes the SIMD floating-point exception classes and gives suggestions for writing an exception handler to handle them.

To indicate that the operating system provides a handler for SIMD floating-point exceptions (#XM), the CR4.OSXMMEXCPT flag (bit 10) must be set.

CHAPTER 16
PROGRAMMING WITH INTEL® AVX10

16.1 INTRODUCTION

Intel® Advanced Vector Extensions 10 (Intel® AVX10) represents an enhancement to Intel® Advanced Vector Extensions 512 (Intel® AVX-512). Intel AVX10 establishes a common, converged vector instruction set across all Intel architectures, incorporating the modern vectorization aspects of Intel AVX-512.

Intel AVX10 is based on Intel AVX-512 and includes all Intel AVX-512 instructions. It supports all instruction vector lengths (128, 256, and 512), as well as scalar and opmask instructions.

16.2 FEATURE VERSIONING AND ENUMERATION

Most Intel AVX10 instructions and features will be organized in collections called **versions**. In some situations, a processor may introduce AVX10 instructions that are not part of that processor's AVX10 version. Such instructions will be enumerated discretely (see below).

AVX10 versions support enumeration that is monotonically increasing and inclusive. This can simplify application development by ensuring that all Intel processors support the same features and instructions at a given Intel AVX10 version number, as well as reduce the number of CPUID feature flags required to be checked by an application to determine feature support. In this enumeration paradigm, the application developer only need to check a CPUID feature flag indicating that the Intel AVX10 ISA is supported and a version number to ensure that the supported version is greater than or equal to the desired version.

The AVX10 feature flag indicates processor support for Intel AVX10 and the presence of a "Converged Vector ISA" leaf containing a field for the version number. AVX10 features or instructions that are not part of an AVX10 version when they are introduced will be enumerated with a discrete feature flag in that CPUID leaf. See Table 16-1 for details.

Table 16-1. CPUID Enumeration of Intel® AVX10

CPUID Bit	Description	Type
CPUID.07H.01H:EDX[19]	If 1, Intel® AVX10 is supported.	Bit (0/1)
CPUID.24H.00H:EAX[31:0]	Reports the maximum supported sub-leaf.	Integer
CPUID.24H.00H:EBX[7:0]	Reports the Intel AVX10 version.	Integer (≥ 1)
CPUID.24H.00H:EBX[15:8]	Reserved.	N/A
CPUID.24H.00H:EBX[18:16]	Reserved.	Always 111B[1]
CPUID.24H.00H:EBX[31:19]	Reserved.	N/A
CPUID.24H.00H:ECX[31:0]	Reserved.	N/A
CPUID.24H.00H:EDX[31:0]	Reserved.	N/A
CPUID.24H.01H:EAX[31:0]	Reserved for discrete feature bits.	N/A
CPUID.24H.01H:EBX[31:0]	Reserved for discrete feature bits.	N/A
CPUID.24H.01H:ECX[31:0]	Reserved for discrete feature bits.	N/A
CPUID.24H.01H:EDX[31:0]	Reserved for discrete feature bits.	N/A

NOTES:

1. Earlier versions of this specification documented these bits as enumerating support for different vector lengths. Processors enumerating Intel® AVX10 support all vector widths.

Several important principles of Intel AVX10 enumeration are the following:

- Versions will be inclusive such that version N+1 is a superset of version N. Once an instruction is introduced in Intel AVX10.x, it is expected to be carried forward in all subsequent Intel AVX10 versions, allowing a developer to check only for a version greater than or equal to the desired version.
- Any processor that enumerates support for Intel AVX10 will also enumerate support for Intel AVX, Intel AVX2, and Intel AVX-512 (see Table 16-2).

The first version of Intel AVX10 (Version 1, or Intel® AVX10.1) supports the Intel AVX-512 instruction families shown in Table 16-2.

Table 16-2. Intel® AVX-512 CPUID Feature Flags Included in Intel® AVX10

Feature Introduction	Intel® AVX-512 CPUID Feature Flags Included in Intel® AVX10
Intel® Xeon® Scalable Processor Family based on Skylake microarchitecture	AVX512F, AVX512CD, AVX512BW, AVX512DQ
Intel® Core™ processors based on Cannon Lake microarchitecture	AVX512-VBMI, AVX512-IFMA
2nd generation Intel® Xeon® Scalable Processor Family based on Cascade Lake product	AVX512-VNNI
3rd generation Intel® Xeon® Scalable Processor Family based on Cooper Lake product	AVX512-BF16
3rd generation Intel® Xeon® Scalable Processor Family based on Ice Lake microarchitecture	AVX512-VPOPCNTDQ, AVX512-VBMI2, VAES, GFNI, VPCLMULQDQ, AVX512-BITALG
4th generation Intel® Xeon® Scalable Processor Family based on Sapphire Rapids microarchitecture	AVX512-FP16

NOTE

VAES, VPCLMULQDQ, and GFNI EVEX instructions will be supported on Intel AVX10.1 machines but will continue to be enumerated by their existing discrete CPUID feature flags. This requires the developer to check for both the feature and Intel AVX10, e.g., {AVX10.1 AND VAES}.

New vector ISA features will only be added to the Intel AVX10 ISA moving forward.

CHAPTER 17
PROGRAMMING WITH INTEL® TRANSACTIONAL SYNCHRONIZATION EXTENSIONS

17.1 OVERVIEW

This chapter describes the software programming interface to the Intel® Transactional Synchronization Extensions of the Intel 64 architecture.

Multi-threaded applications take advantage of increasing number of cores to achieve high performance. However, writing multi-threaded applications requires programmers to reason about data sharing among multiple threads. Access to shared data typically requires synchronization mechanisms. These mechanisms ensure multiple threads update shared data by serializing operations on the shared data, often through the use of a critical section protected by a lock. Since serialization limits concurrency, programmers try to limit synchronization overheads. They do this either through minimizing the use of synchronization or through the use of fine-grain locks; where multiple locks each protect different shared data. Unfortunately, this process is difficult and error prone; a missed or incorrect synchronization can cause an application to fail. Conservatively adding synchronization and using coarser granularity locks, where a few locks each protect many items of shared data, helps avoid correctness problems but limits performance due to excessive serialization. While programmers must use static information to determine when to serialize, the determination as to whether actually to serialize is best done dynamically.

Intel® Transactional Synchronization Extensions aim to improve the performance of lock-protected critical sections while maintaining the lock-based programming model.

17.2 INTEL® TRANSACTIONAL SYNCHRONIZATION EXTENSIONS

Intel® Transactional Synchronization Extensions (Intel® TSX) allow the processor to determine dynamically whether threads need to serialize through lock-protected critical sections, and to perform serialization only when required. This lets the hardware expose and exploit concurrency hidden in an application due to dynamically unnecessary synchronization through a technique known as lock elision.

With lock elision, the hardware executes the programmer-specified critical sections (also referred to as transactional regions) transactionally. In such an execution, the lock variable is only read within the transactional region; it is not written to (and therefore not acquired) with the expectation that the lock variable remains unchanged after the transactional region, thus exposing concurrency.

If the transactional execution completes successfully, then the hardware ensures that all memory operations performed within the transactional region will appear to have occurred instantaneously when viewed from other logical processors, a process referred to as an **atomic commit**. Any updates performed within the transactional region are made visible to other processors only on an atomic commit.

Since a successful transactional execution ensures an atomic commit, the processor can execute the programmer-specified code section optimistically without synchronization. If synchronization was unnecessary for that specific execution, execution can commit without any cross-thread serialization.

If the transactional execution is unsuccessful, the processor cannot commit the updates atomically. When this happens, the processor will roll back the execution, a process referred to as a **transactional abort**. On a transactional abort, the processor will discard all updates performed in the region, restore architectural state to appear as if the optimistic execution never occurred, and resume execution non-transactionally. Depending on the policy in place, lock elision may be retried or the lock may be explicitly acquired to ensure forward progress.

Intel TSX provides two software interfaces for programmers.

- Hardware Lock Elision (HLE) is a legacy compatible instruction set extension comprising the XACQUIRE and XRELEASE prefixes.
- Restricted Transactional Memory (RTM) is an instruction set interface comprising the XBEGIN and XEND instructions.

Programmers who would like to run Intel TSX-enabled software on legacy hardware would use the HLE interface to implement lock elision. On the other hand, programmers who do not have legacy hardware requirements and who deal with more complex locking primitives would use the RTM software interface of Intel TSX to implement lock elision. In the latter case when using new instructions, the programmer must always provide a non-transactional path (which would have code to eventually acquire the lock being elided) to execute following a transactional abort and must not rely on the transactional execution alone.

Intel TSX provides the XTEST instruction to test whether a logical processor is executing transactionally, and the XABORT instruction to abort a transactional region.

A processor can perform a transactional abort for numerous reasons. A primary cause is due to conflicting accesses between the transactionally executing logical processor and another logical processor. Such conflicting accesses may prevent a successful transactional execution. Memory addresses read from within a transactional region constitute the **read-set** of the transactional region and addresses written to within the transactional region constitute the **write-set** of the transactional region. Intel TSX maintains the read- and write-sets at the granularity of a cache line.

A conflicting data access occurs if another logical processor either reads a location that is part of the transactional region's write-set or writes a location that is a part of either the read- or write-set of the transactional region. We refer to this as a **data conflict**. Since Intel TSX detects data conflicts at the granularity of a cache line, unrelated data locations placed in the same cache line will be detected as conflicts. Transactional aborts may also occur due to limited transactional resources. For example, the amount of data accessed in the region may exceed an implementation-specific capacity. Additionally, some instructions and system events may cause transactional aborts.

Additionally, Intel TSX provides the XSUSLDTRK and XRESLDTRK instructions to suspend and resume load address tracking.

17.2.1 HLE Software Interface

HLE provides two instruction prefix hints: XACQUIRE and XRELEASE.

The programmer uses the XACQUIRE prefix in front of the instruction that is used to acquire the lock that is protecting the critical section. The processor treats the indication as a hint to elide the write associated with the lock acquire operation. Even though the lock acquire has an associated write operation to the lock, the processor does not add the address of the lock to the transactional region's write-set nor does it issue any write requests to the lock. Instead, the address of the lock is added to the read-set. The logical processor enters transactional execution. If the lock was available before the XACQUIRE prefixed instruction, all other processors will continue to see it as available afterwards. Since the transactionally executing logical processor neither added the address of the lock to its write-set nor performed externally visible write operations to it, other logical processors can read the lock without causing a data conflict. This allows other logical processors to also enter and concurrently execute the critical section protected by the lock. The processor automatically detects any data conflicts that occur during the transactional execution and will perform a transactional abort if necessary.

Even though the eliding processor did not perform any external write operations to the lock, the hardware ensures program order of operations on the lock. If the eliding processor itself reads the value of the lock in the critical section, it will appear as if the processor had acquired the lock, i.e., the read will return the non-elided value. This behavior makes an HLE execution functionally equivalent to an execution without the HLE prefixes.

The programmer uses the XRELEASE prefix in front of the instruction that is used to release the lock protecting the critical section. This involves a write to the lock. If the instruction is restoring the value of the lock to the value it had prior to the XACQUIRE prefixed lock acquire operation on the same lock, then the processor elides the external write request associated with the release of the lock and does not add the address of the lock to the write-set. The processor then attempts to commit the transactional execution.

With HLE, if multiple threads execute critical sections protected by the same lock but they do not perform any conflicting operations on each other's data, then the threads can execute concurrently and without serialization. Even though the software uses lock acquisition operations on a common lock, the hardware recognizes this, elides the lock, and executes the critical sections on the two threads without requiring any communication through the lock — if such communication was dynamically unnecessary.

If the processor is unable to execute the region transactionally, it will execute the region non-transactionally and without elision. HLE enabled software has the same forward progress guarantees as the underlying non-HLE lock-based execution. For successful HLE execution, the lock and the critical section code must follow certain guidelines

(discussed in Section 17.3.3 and Section 17.3.9). These guidelines only affect performance; not following these guidelines will not cause a functional failure.

Hardware without HLE support will ignore the XACQUIRE and XRELEASE prefix hints and will not perform any elision since these prefixes correspond to the REPNE/REPE IA-32 prefixes which are ignored on the instructions where XACQUIRE and XRELEASE are valid. Importantly, HLE is compatible with the existing lock-based programming model. Improper use of hints will not cause functional bugs though it may expose latent bugs already in the code.

17.2.2 RTM Software Interface

RTM provides three instructions: XBEGIN, XEND, and XABORT.

Software uses the XBEGIN instruction to specify the start of the transactional region and the XEND instruction to specify the end of the transactional region. The XBEGIN instruction takes an operand that provides a relative offset to the **fallback instruction address** if the transactional region could not be successfully executed transactionally. Software using these instructions to implement lock elision must test the lock within the transactional region, and only if free should try to commit. Further, the software may also define a policy to retry if the lock is not free.

A processor may abort transactional execution for many reasons. The hardware automatically detects transactional abort conditions and restarts execution from the fallback instruction address with the architectural state corresponding to that at the start of the XBEGIN instruction and the EAX register updated to describe the abort status.

The XABORT instruction allows programmers to abort the execution of a transactional region explicitly. The XABORT instruction takes an 8 bit immediate argument that is loaded into the EAX register and will thus be available to software following a transactional abort.

Hardware provides no guarantees as to whether a transactional execution will ever successfully commit. Programmers must always provide an alternative code sequence in the fallback path to guarantee forward progress. When using the instructions for lock elision, this may be as simple as acquiring a lock and executing the specified code region non-transactionally. Further, a transactional region that always aborts on a given implementation may complete transactionally on a future implementation. Therefore, programmers must ensure the code paths for the transactional region and the alternative code sequence are functionally tested.

If the RTM software interface is used for anything other than lock elision, the programmer must similarly ensure that the fallback path is inter-operable with the transactionally executing path.

17.3 INTEL® TSX APPLICATION PROGRAMMING MODEL

17.3.1 Detection of Transactional Synchronization Support

17.3.1.1 Detection of HLE Support

A processor supports HLE execution if CPUID.07H.00H:EBX.HLE[4] = 1. However, an application can use the HLE prefixes (XACQUIRE and XRELEASE) without checking whether the processor supports HLE. Processors without HLE support ignore these prefixes and will execute the code without entering transactional execution.

17.3.1.2 Detection of RTM Support

A processor supports RTM execution if CPUID.07H.00H:EBX.RTM[11] = 1. An application must check if the processor supports RTM before it uses the RTM instructions (XBEGIN, XEND, and XABORT). These instructions will generate a #UD exception when used on a processor that does not support RTM.

17.3.1.3 Detection of XTEST Instruction

A processor supports the XTEST instruction if it supports either HLE or RTM. An application must check either of these feature flags before using the XTEST instruction. This instruction will generate a #UD exception when used on a processor that does not support either HLE or RTM.

17.3.1.4 Detection of Intel® TSX Suspend Load Address Tracking

A processor supports Intel TSX suspend/resume of load address tracking if CPUID.07H.00H:EDX.TSXLDTRK[16] = 1. An application must check if the processor supports Intel TSX suspend/resume of load address tracking before it uses the Intel TSX suspend/resume load address tracking instructions (XSUSLDTRK and XRESLDTRK). These instructions will generate a #UD exception when used on a processor that does not support Intel TSX suspend/resume load address tracking.

17.3.2 Querying Transactional Execution Status

The XTEST instruction can be used to determine the transactional status of a transactional region specified by HLE or RTM. Note, while the HLE prefixes are ignored on processors that do not support HLE, the XTEST instruction will generate a #UD exception when used on processors that do not support either HLE or RTM.

17.3.3 Requirements for HLE Locks

For HLE execution to successfully commit transactionally, the lock must satisfy certain properties and access to the lock must follow certain guidelines.

- An XRELEASE prefixed instruction must restore the value of the elided lock to the value it had before the lock acquisition. This allows hardware to safely elide locks by not adding them to the write-set. The data size and data address of the lock release (XRELEASE prefixed) instruction must match that of the lock acquire (XACQUIRE prefixed) and the lock must not cross a cache line boundary.
- Software should not write to the elided lock inside a transactional HLE region with any instruction other than an XRELEASE prefixed instruction, otherwise it may cause a transactional abort. In addition, recursive locks (where a thread acquires the same lock multiple times without first releasing the lock) may also cause a transactional abort. Note that software can observe the result of the elided lock acquire inside the critical section. Such a read operation will return the value of the write to the lock.

The processor automatically detects violations to these guidelines, and safely transitions to a non-transactional execution without elision. Since Intel TSX detects conflicts at the granularity of a cache line, writes to data collocated on the same cache line as the elided lock may be detected as data conflicts by other logical processors eliding the same lock.

17.3.4 Transactional Nesting

Both HLE- and RTM-based transactional executions support nested transactional regions. However, a transactional abort restores state to the operation that started transactional execution: either the outermost XACQUIRE prefixed HLE eligible instruction or the outermost XBEGIN instruction. The processor treats all nested transactional regions as one monolithic transactional region.

17.3.4.1 HLE Nesting and Elision

Programmers can nest HLE regions up to an implementation specific depth of MAX_HLE_NEST_COUNT. Each logical processor tracks the nesting count internally but this count is not available to software. An XACQUIRE prefixed HLE-eligible instruction increments the nesting count, and an XRELEASE prefixed HLE-eligible instruction decrements it. The logical processor enters transactional execution when the nesting count goes from zero to one. The logical processor attempts to commit only when the nesting count becomes zero. A transactional abort may occur if the nesting count exceeds MAX_HLE_NEST_COUNT.

In addition to supporting nested HLE regions, the processor can also elide multiple nested locks. The processor tracks a lock for elision beginning with the XACQUIRE prefixed HLE eligible instruction for that lock and ending with

the XRELEASE prefixed HLE eligible instruction for that same lock. The processor can, at any one time, track up to a MAX_HLE_ELIDED_LOCKS number of locks. For example, if the implementation supports a MAX_HLE_ELIDED_LOCKS value of two and if the programmer nests three HLE identified critical sections (by performing XACQUIRE prefixed HLE eligible instructions on three distinct locks without performing an intervening XRELEASE prefixed HLE eligible instruction on any one of the locks), then the first two locks will be elided, but the third won't be elided (but will be added to the transaction's write-set). However, the execution will still continue transactionally. Once an XRELEASE for one of the two elided locks is encountered, a subsequent lock acquired through the XACQUIRE prefixed HLE eligible instruction will be elided.

The processor attempts to commit the HLE execution when all elided XACQUIRE and XRELEASE pairs have been matched, the nesting count goes to zero, and the locks have satisfied the requirements described earlier. If execution cannot commit atomically, then execution transitions to a non-transactional execution without elision as if the first instruction did not have an XACQUIRE prefix.

17.3.4.2 RTM Nesting

Programmers can nest RTM-based transactional regions up to an implementation specific MAX_RTM_NEST_COUNT. The logical processor tracks the nesting count internally but this count is not available to software. An XBEGIN instruction increments the nesting count, and an XEND instruction decrements it. The logical processor attempts to commit only if the nesting count becomes zero. A transactional abort occurs if the nesting count exceeds MAX_RTM_NEST_COUNT.

17.3.4.3 Nesting HLE and RTM

HLE and RTM provide two alternative software interfaces to a common transactional execution capability. The behavior when HLE and RTM are nested together—HLE inside RTM or RTM inside HLE—is implementation specific. However, in all cases, the implementation will maintain HLE and RTM semantics. An implementation may choose to ignore HLE hints when used inside RTM regions, and may cause a transactional abort when RTM instructions are used inside HLE regions. In the latter case, the transition from transactional to non-transactional execution occurs seamlessly since the processor will re-execute the HLE region without actually doing elision, and then execute the RTM instructions.

17.3.5 RTM Abort Status Definition

RTM uses the EAX register to communicate abort status to software. Following an RTM abort the EAX register has the following definition.

Table 17-1. RTM Abort Status Definition

EAX Register Bit Position	Meaning
0	Set if abort caused by XABORT instruction.
1	If set, the transactional execution may succeed on a retry. This bit is always clear if bit 0 is set.
2	Set if another logical processor conflicted with a memory address that was part of the transactional execution that aborted.
3	Set if an internal buffer to track transactional state overflowed.
4	Set if a debug exception (#DB) or breakpoint exception (#BP) was hit.
5	Set if an abort occurred during execution of a nested transactional execution.
23:6	Reserved.
31:24	XABORT argument (only valid if bit 0 set, otherwise reserved).

The EAX abort status for RTM only provides causes for aborts. It does not by itself encode whether an abort or commit occurred for the RTM region. The value of EAX can be 0 following an RTM abort. For example, a CPUID

instruction when used inside an RTM region causes a transactional abort and may not satisfy the requirements for setting any of the EAX bits. This may result in an EAX value of 0.

17.3.6 RTM Memory Ordering

A successful RTM commit causes all memory operations in the RTM region to appear to execute atomically. A successfully committed RTM region consisting of an XBEGIN followed by an XEND, even with no memory operations in the RTM region, has the same ordering semantics as a LOCK prefixed instruction.

The XBEGIN instruction does not have fencing semantics. However, if an RTM execution aborts, all memory updates from within the RTM region are discarded and never made visible to any other logical processor.

17.3.7 RTM-Enabled Debugger Support

Any debug exception (#DB) or breakpoint exception (#BP) inside an RTM region causes a transactional abort and, by default, redirects control flow to the fallback instruction address with architectural state recovered and bit 4 in EAX set. However, to allow software debuggers to intercept execution on debug or breakpoint exceptions, the RTM architecture provides additional capability called **advanced debugging of RTM transactional regions**.

Advanced debugging of RTM transactional regions is enabled if bit 11 of DR7 and bit 15 of the IA32_DEBUGCTL MSR are both 1. In this case, any RTM transactional abort due to a #DB or #BP causes execution to roll back to just before the XBEGIN instruction (EAX is restored to the value it had before XBEGIN) and then delivers a #DB. (A #DB is delivered even if the transactional abort was caused by a #BP.) DR6[16] is cleared to indicate that the exception resulted from a debug or breakpoint exception inside an RTM region. See also Section 20.3.3, "Debug Exceptions, Breakpoint Exceptions, and Restricted Transactional Memory (RTM)," of the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3B.

17.3.8 Intel® TSX Suspend/Resume Load Address Tracking Support

Programmers can use Intel TSX suspend/resume of load address tracking to choose which memory accesses do not need to be tracked in the Intel TSX read set. A programmer who uses the suspend/resume load address tracking feature must ensure that there are no atomicity requirements related to the addresses they choose to exclude from the read set as hardware will not detect read-write conflicts for those addresses.

To prevent load addresses from being entered into the read set, the programmer should use the XSUSLDTRK and XRESLDTRK instructions. The XSUSLDTRK instruction suspends loads tracking and thus specifies the start of a **suspend region**; addresses of subsequent loads will not be added to the transaction read set. The XRESLDTRK instruction resumes load tracking and thus specifies the end of a suspend region; addresses of subsequent loads will be added to the transaction read set.

The execution of a suspend region is similar to transaction execution with the following exceptions:

- The addresses of loads in a suspend region are not tracked for read-write conflicts if the addresses are accessed inside the suspend region only (they are not added to the transaction read set). The addresses are still tracked if they are accessed outside of the suspend region inside the transaction.
- Transaction start/end inside a suspend region is not supported; any execution of XBEGIN or XEND inside a suspend region will cause the transaction to abort.
- There is no support for nesting of suspend regions; execution of XSUSLDTRK in a suspend region will cause a transaction to abort.

17.3.9 Programming Considerations

Typical programmer-identified regions are expected to execute transactionally and to commit successfully. However, Intel TSX does not provide any such guarantee. A transactional execution may abort for many reasons. To take full advantage of the transactional capabilities, programmers should follow certain guidelines to increase the probability of their transactional execution committing successfully.

This section discusses various events that may cause transactional aborts. The architecture ensures that updates performed within a transactional region that subsequently aborts execution will never become visible. Only a

committed transactional execution updates architectural state. Transactional aborts never cause functional failures and only affect performance.

17.3.9.1 Instruction Based Considerations

Programmers can use any instruction safely inside a transactional region. Further, programmers can use the Intel TSX instructions and prefixes at any privilege level. However, some instructions will always abort the transactional execution and cause execution to seamlessly and safely transition to a non-transactional path.

Intel TSX allows for most common instructions to be used inside transactional regions without causing aborts. The following operations inside a transactional region do not typically cause an abort.

- Operations on the instruction pointer register, general purpose registers (GPRs) and the status flags (CF, OF, SF, PF, AF, and ZF).
- Operations on XMM and YMM registers and the MXCSR register

However, programmers must be careful when intermixing SSE and AVX operations inside a transactional region. Intermixing SSE instructions accessing XMM registers and AVX instructions accessing YMM registers may cause transactional regions to abort.

CLD and STD instructions when used inside transactional regions may cause aborts if they change the value of the DF flag. However, if DF is 1, the STD instruction will not cause an abort. Similarly, if DF is 0, the CLD instruction will not cause an abort.

Instructions not enumerated here as causing abort when used inside a transactional region will typically not cause the execution to abort (examples include but are not limited to MFENCE, LFENCE, SFENCE, RDTSC, RDTSCP, etc.).

The following instructions will abort transactional execution on any implementation:

- XABORT
- CPUID
- PAUSE
- ENCLS
- ENCLU

In addition, in some implementations, the following instructions may always cause transactional aborts. These instructions are not expected to be commonly used inside typical transactional regions. However, programmers must not rely on these instructions to force a transactional abort, since whether they cause transactional aborts is implementation dependent.

- Operations on X87 and MMX architecture state. This includes all MMX and X87 instructions, including the FXRSTOR and FXSAVE instructions.
- Update to non-status portion of EFLAGS or to UIF: CLI, CLUI, STI, STUI, POPFD, POPFQ, CLAC, and STAC.
- Instructions that update segment registers, debug registers and/or control registers: MOV to DS/ES/FS/GS/SS, POP DS/ES/FS/GS/SS, LDS, LES, LFS, LGS, LSS, SWAPGS, WRFSBASE, WRGSBASE, LGDT, SGDT, LIDT, SIDT, LLDT, SLDT, LTR, STR, Far CALL, Far JMP, Far RET, IRET, MOV to DRx, MOV to CR0/CR2/CR3/CR4/CR8, CLTS, and LMSW.
- Ring transitions: SYSENTER, SYSCALL, SYSEXIT, and SYSRET.
- TLB and Cacheability control: CLFLUSH, CLFLUSHOPT, CLWB, INVD, WBINVD, INVLPG, INVPCID, and memory instructions with a non-temporal hint (V/MOVNTDQA, V/MOVNTDQ, V/MOVNTI, V/MOVNTPD, V/MOVNTPS, V/MOVNTQ, V/MASKMOVQ, and V/MASKMOVDQU).
- Extended state management: XRSTOR, XRSTORS, XSAVE, XSAVEC, XSAVEOPT, XSAVES, and XSETBV.
- Interrupts: INT *n*, INTO, INT3, and INT1.
- I/O: IN, INS, REP INS, OUT, OUTS, REP OUTS and their variants.
- VMX: VMPTRLD, VMPTRST, VMCLEAR, VMREAD, VMWRITE, VMCALL, VMLAUNCH, VMRESUME, VMXOFF, VMXON, INVEPT, INVVPID, and VMFUNC.
- SMX: GETSEC.
- UD0, UD1, UD2, UDB, RSM, RDMSR, WRMSR, WRPKRU, HLT, MONITOR, MWAIT, and VZEROUPPER.

17.3.9.2 Runtime Considerations

In addition to the instruction-based considerations, runtime events may cause transactional execution to abort. These may be due to data access patterns or micro-architectural implementation causes. Keep in mind that the following list is not a comprehensive discussion of all abort causes.

Any fault or trap in a transactional region that must be exposed to software will be suppressed. Transactional execution will abort and execution will transition to a non-transactional execution, as if the fault or trap had never occurred. If any exception is not masked, that will result in a transactional abort and it will be as if the exception had never occurred.

When executed in VMX non-root operation, certain instructions may result in a VM exit. When such instructions are executed inside a transactional region, then instead of causing a VM exit, they will cause a transactional abort and the execution will appear as if instruction that would have caused a VM exit never executed.

Synchronous exception events (#DE, #OF, #NP, #SS, #GP, #BR, #UD, #AC, #XM, #PF, #NM, #TS, #MF, #DB, #BP/INT3) that occur during transactional execution may cause an execution not to commit transactionally, and require a non-transactional execution. These events are suppressed as if they had never occurred. With HLE, since the non-transactional code path is identical to the transactional code path, these events will typically re-appear when the instruction that caused the exception is re-executed non-transactionally, causing the associated synchronous events to be delivered appropriately in the non-transactional execution. The same behavior also applies to synchronous events (EPT violations, EPT misconfigurations, and accesses to the APIC-access page) that occur in VMX non-root operation.

Asynchronous events (NMI, SMI, INTR, IPI, PMI, etc.) occurring during transactional execution may cause the transactional execution to abort and transition to a non-transactional execution. The asynchronous events will be pended and handled after the transactional abort is processed. The same behavior also applies to asynchronous events (VMX-preemption timer expiry, virtual-interrupt delivery, and interrupt-window exiting) that occur in VMX non-root operation.

Transactional execution only supports write-back cacheable memory type operations. A transactional region may always abort if it includes operations on any other memory type. This includes instruction fetches to UC memory type.

Memory accesses within a transactional region may require the processor to set the Accessed and Dirty flags of the referenced page table entry. The behavior of how the processor handles this is implementation specific. Some implementations may allow the updates to these flags to become externally visible even if the transactional region subsequently aborts. Some Intel TSX implementations may choose to abort the transactional execution if these flags need to be updated. Further, a processor's page-table walk may generate accesses to its own transactionally written but uncommitted state. Some Intel TSX implementations may choose to abort the execution of a transactional region in such situations. Regardless, the architecture ensures that, if the transactional region aborts, then the transactionally written state will not be made architecturally visible through the behavior of structures such as TLBs.

Executing self-modifying code transactionally may also cause transactional aborts. Programmers must continue to follow the Intel recommended guidelines for writing self-modifying and cross-modifying code even when employing Intel TSX.

While an Intel TSX implementation will typically provide sufficient resources for executing common transactional regions, implementation constraints and excessive sizes for transactional regions may cause a transactional execution to abort and transition to a non-transactional execution. The architecture provides no guarantee of the amount of resources available to do transactional execution and does not guarantee that a transactional execution will ever succeed.

Conflicting requests to a cache line accessed within a transactional region may prevent the transactional region from executing successfully. For example, if logical processor P0 reads line A in a transactional region and another logical processor P1 writes A (either inside or outside a transactional region) then logical processor P0 may abort if logical processor P1’s write interferes with processor P0's ability to execute transactionally. Similarly, if P0 writes line A in a transactional region and P1 reads or writes A (either inside or outside a transactional region), then P0 may abort if P1's access to A interferes with P0's ability to execute transactionally. In addition, other coherence traffic may at times appear as conflicting requests and may cause aborts. While these false conflicts may happen, they are expected to be uncommon. The conflict resolution policy to determine whether P0 or P1 aborts in the above scenarios is implementation specific.

CHAPTER 18
CONTROL-FLOW ENFORCEMENT TECHNOLOGY (CET)

18.1 INTRODUCTION

Return-oriented programming (ROP), and similarly CALL/JMP-oriented programming (COP/JOP), have been the prevalent attack methodologies for stealth exploit writers targeting vulnerabilities in programs. These attack methodologies have the common elements:

- A code module with execution privilege and contain small snippets of code sequence with the characteristic: at least one instruction in the sequence being a control transfer instruction that depends on data either in the return stack or in a register for the target address.
- Diverting the control flow instruction (e.g., RET, CALL, JMP) from its original target address to a new target (via modification in the data stack or in the register).

Control-Flow Enforcement Technology (CET) provides the following capabilities to defend against ROP/COP/JOP style control-flow subversion attacks:

- Shadow stack: Return address protection to defend against ROP.
- Indirect branch tracking: Free branch protection to defend against COP/JOP.

Both capabilities introduce new instruction set extensions, and are described in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volumes 2A, 2B, 2C, & 2D.

Control-Flow Enforcement Technology introduces a new exception (#CP) with interrupt vector 21.

18.1.1 Shadow Stack

A shadow stack is a second stack for the program that is used exclusively for control transfer operations. This stack is separate from the data stack and can be enabled for operation individually in user mode or supervisor mode. When shadow stacks are enabled, the CALL instruction pushes the return address on both the data and shadow stack. The RET instruction pops the return address from both stacks and compares them. If the return addresses from the two stacks do not match, the processor signals a control protection exception (#CP). Note that the shadow stack only holds the return addresses and not parameters passed to the call instruction.

The shadow stack is protected from tamper through the page table protections such that regular store instructions cannot modify the contents of the shadow stack. To provide this protection the page table protections are extended to support an additional attribute for pages to mark them as "Shadow Stack" pages. When shadow stacks are enabled, control transfer instructions/flows like near call, far call, call to interrupt/exception handlers, etc. store return addresses to the shadow stack and the access will fault if the underlying page is not marked as a "Shadow Stack" page. However stores from instructions like MOV, XSAVE, etc. will not be allowed. Likewise control transfer instructions like near RET, far RET, IRET, etc. when they attempt to read from the shadow stack the access will fault if the underlying page is not marked as a "Shadow Stack" page. This paging protection detects and prevents conditions that cause an overflow or underflow of the shadow stack when the shadow stack is delimited by non-shadow stack guard pages, or any malicious attempts to redirect the processor to consume data from addresses that are not shadow stack addresses.

18.1.2 Indirect Branch Tracking

The ENDBRANCH instruction is a new instruction that is used to mark valid jump target addresses of indirect calls and jumps in the program. This instruction opcode is selected to be one that is a NOP on legacy machines such that programs compiled with ENDBRANCH new instruction continue to function on old machines without the CET enforcement. On processors that support CET the ENDBRANCH is still a NOP and is primarily used as a marker instruction by the processor pipeline to detect control flow violations. The CPU implements a state machine that tracks indirect JMP and CALL instructions. When one of these instructions is executed, the state machine moves from IDLE to WAIT_FOR_ENDBRANCH state. In WAIT_FOR_ENDBRANCH state the next instruction in the program

stream must be an ENDBRANCH. If the next instruction is not an ENDBRANCH, the processor causes a control protection exception (#CP); otherwise, the state machine moves back to IDLE state.

18.1.3 Speculative Behavior when CET is Enabled

Speculative execution of near indirect JMP/CALL/RET indirect branches may be able to create an active side channel vulnerability that reveals the contents of data.

There are two basic methods that an attacker may be able to use to control indirect branch speculation in order to speculatively execute code that causes a side channel:

1. Attacker controlled prediction.
2. Attacker controlled jump redirection.

With attacker controlled prediction, the attacker trains indirect branch predictors such that the desired victim indirect branch goes to the attacker desired location. Examples include Branch Target Injection (also called "Variant 2" and "Spectre") and RSB wrap on underflow (also called "ret2spec").

With attacker controlled jump redirection, the attacker controls a speculative-only value used as input to the indirect branch so that the branch mispredicts to the attacker desired location. Examples of this include Bound Check Bypass Store (where a speculative store containing an attacker controlled value may overwrite the indirect branch target before the load of the target) and Speculative Store Bypass (where a load of the indirect branch target may bypass the most recent store of the target value and thus speculatively read an older attacker controlled value at the same memory location).

In addition to the existing mitigation features like IBRS, STIBP, and IBPB, processors supporting CET will have a variety of additional features to constrain control flow speculation in order to mitigate such attacks. For details on these features, see Section 18.2.6, "Constraining Execution at Targets of RET," and Section 18.3.8, "Constraining Speculation after Missing ENDBRANCH."

18.2 SHADOW STACKS

A shadow stack is a second expand down stack used exclusively for control transfer operations. This stack is separate from the data stack. The shadow stack is not used to store data and hence is not explicitly writeable by software. Writes to the shadow stack are restricted to control transfer instructions and shadow stack management instructions. The shadow stack feature can be enabled separately in user mode (CPL == 3) or supervisor mode (CPL < 3).

Shadow stacks operate only in protected mode. Shadow stacks cannot be enabled in virtual 8086 mode.

It is recommended to not configure the shadow stack in the linear address range 0 to 64 KB or adjacent to the canonical address boundary.

18.2.1 Shadow Stack Pointer and its Operand and Address Size Attributes

When CET is enabled the processor supports a new architectural register, shadow stack pointer (SSP), when the processor supports the shadow stack feature. The SSP cannot be directly encoded as a source, destination or memory operand in instructions. The SSP points to the current top of the shadow stack.

The width of the shadow stack is 32-bit in 32-bit/compatibility mode and is 64-bit in 64-bit mode. The address-size attribute of the shadow stack is likewise 32-bit in 32-bit/compatibility mode and 64-bit in 64-bit mode.

18.2.2 Terminology

When shadow stacks are enabled, certain control transfer instructions/flows and shadow stack management instructions do loads and stores from and to the shadow stack. Such loads and stores from control transfer instructions and shadow stack management instructions are termed as **shadow-stack loads** and **shadow-stack stores** to distinguish them from a loads and stores performed by other instructions like MOV, XSAVES, etc.

The pseudocode for the instruction operations use the notation ShadowStackEnabled(CPL) as a test of whether shadow stacks are enabled at the CPL. This term returns a TRUE or FALSE indication as follows.

```
ShadowStackEnabled(CPL):
    IF CR4.CET = 1 AND CR0.PE = 1 AND EFLAGS.VM = 0
        IF CPL = 3
            THEN
                (* Obtain the shadow stack enable from IA32_U_CET MSR (MSR address 6A0H) used to enable
                 feature for CPL = 3 *)
                SHADOW_STACK_ENABLED = IA32_U_CET.SH_STK_EN;
            ELSE
                (* Obtain the shadow stack enable from IA32_S_CET MSR (MSR address 6A2H) used to enable
                 feature for CPL < 3 *)
                SHADOW_STACK_ENABLED = IA32_S_CET.SH_STK_EN;
        FI;
        IF SHADOW_STACK_ENABLED = 1
            THEN
                return TRUE;
            ELSE
                return FALSE;
        FI;
    ELSE
        (* Shadow stacks not enabled in real mode and virtual-8086 mode or if the master CET feature
        enable in CR4 is disabled *)
    return FALSE;
ENDIF
```

Additionally, the following terms are used.

- ShadowStackPush4B: Decrements the shadow stack pointer (SSP) by 4 bytes and copies the 4 byte source operand to the top of the shadow stack.
- ShadowStackPush8B : Decrements the shadow stack pointer (SSP) by 8 bytes and copies the 8 byte source operand to the top of the shadow stack.
- ShadowStackPop4B: Copies 4 bytes at the current top of stack (indicated by the SSP register) to the location specified with the destination operand. It then increments the SSP register by 4 bytes to point to the new top of stack.
- ShadowStackPop8B: Copies 8 bytes at the current top of stack (indicated by the SSP register) to the location specified with the destination operand. It then increments the SSP register by 8 bytes to point to the new top of stack.
- shadow_stack_lock_cmpxchg8B(address, new_value, expected_value): this function executes atomically and compares the expected_value to the 8 byte read from memory specified by the address operand using a locked shadow-stack load. If the two values are equal, the new_value is written to the address using an unlocking shadow-stack store. If the two values are not equal, then the value read by the shadow-stack load is written back, also using an unlocking shadow-stack store. The function returns the value read from the memory specified by the address operand.

18.2.3 Supervisor Shadow Stack Token

This section describes shadow-stack support that is provided for call gates, IDT event delivery, and IRET. This behavior does not apply when FRED transitions are enabled.

On an inter-privilege far CALL or when calling an interrupt/exception handler at a higher privilege level, a stack switch occurs; if shadow stacks are enabled at the new privilege level, then a shadow stack switch occurs. Shadow stacks that can be switched to by hardware as part of a privilege change are required to have a supervisor shadow stack token set up by the supervisor to provide the address of the new SSP register. The supervisor shadow stack tokens also serve the purpose of enforcing that a shadow stack can be made active on only one logical processor

when switched to by the processor. The supervisor shadow stack token must be set up only on shadow stacks intended to be used on these transfers. The address of the supervisor shadow stack token is programmed into the IA32_PLx_SSP MSR (where $0 \leq x \leq 2$). The WRMSR and XRSTORS instructions require the address specified in the IA32_PLx_SSP MSR (where $0 \leq x \leq 2$) to be 4 byte aligned; otherwise, the instruction causes a general protection exception (#GP(0)).

The supervisor shadow stack token is a 64-bit value formulated as follows.

- Bit 63:3: Bits 63:3 of the linear address of the supervisor shadow stack token.
- Bit 2: Reserved. Must be zero.
- Bit 1: Reserved. Must be zero.
- Bit 0: Busy bit. If 0, indicates this shadow stack is not active on any logical processor. If 1, indicates this shadow stack is currently active on one of the logical processors.

The following figure illustrates a supervisor shadow stack with a supervisor shadow stack token located at its base.

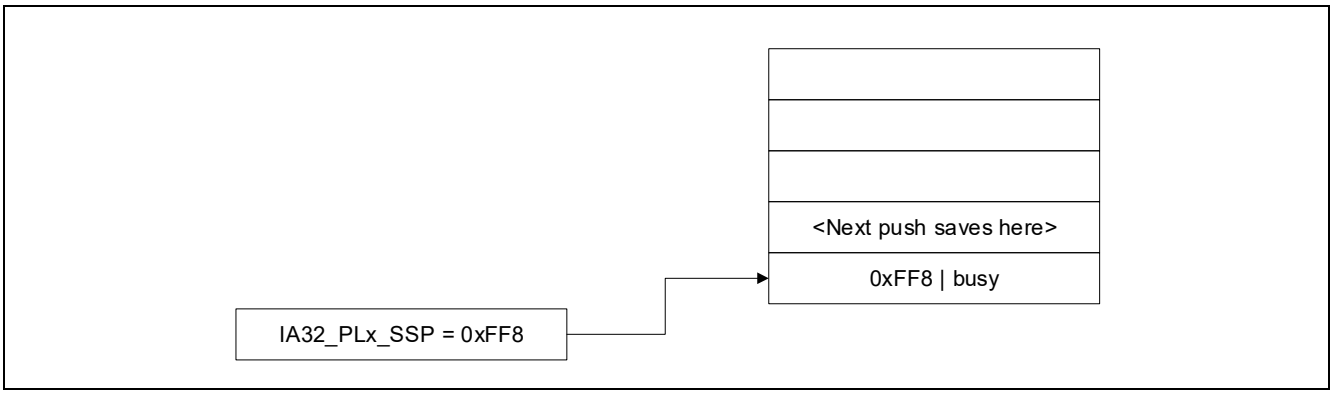


Figure 18-1. Supervisor Shadow Stack with a Supervisor Shadow Stack Token

If the far CALL or event delivery will push a 24-byte stack frame after the token is acquired, the 8-byte supervisor shadow stack token and the stack frame must be fully contained within a 32-byte region that is aligned to 32-bytes on the shadow stack. If they are not, a general-protection exception (#GP(0)) occurs.

The processor does the following checks prior to switching to a supervisor shadow stack programmed into the IA32_PLx_SSP MSR. These steps are performed atomically.

1. Load the supervisor shadow stack token from the address specified in the IA32_PLx_SSP MSR using a locked shadow-stack store.
2. Check if the busy bit in the token is 0; reserved bits must be 0.
3. Check if the address programmed in the MSR matches the address in the supervisor shadow stack token; reserved bits must be 0.
4. If checks 2 and 3 are successful, then set the busy bit in the token using an unlocking shadow-stack store and switching the SSP to the value specified in the IA32_PLx_SSP MSR.
5. If checks 2 or 3 fail, write back the value read at step 1 using an unlocking shadow-stack store (the busy bit is not set) and raise a #GP(0) exception.

If the far CALL or event delivery pushes a stack frame after the token is acquired and any of the pushes causes a fault or VM exit, the processor will revert to the old shadow stack and the busy bit in the new shadow stack's token remains set. The new shadow stack is said to be **prematurely busy**. Software should enable supervisor shadow stacks only if it is certain that this situation cannot occur. If CPUID.07H.01H:EDX[18] is enumerated as 1, it is sufficient for an operating system to ensure that none of the pushes can cause a page fault.

On a far RET to a lesser privilege level or on an IRET that switches shadow stack, the instruction clears the busy bit in the shadow stack token as follows. These steps are also performed atomically.

1. Load the supervisor shadow stack token from the SSP using a locked shadow-stack load.

2. Check if the busy bit in the token is 1; reserved bits must be 0.
3. Check if the address programmed in supervisor shadow stack token matches SSP; reserved bits must be 0.
4. If checks 2 and 3 are successful, then write back the token with an unlocking shadow-stack store, clearing the busy bit; otherwise, write back the value read at step 1 using an unlocking shadow-stack store and continue without modifying the contents of the shadow stack pointed to by SSP.

18.2.4 Shadow Stack Usage on Task Switch

A task switch (see Chapter 10, "Task Management," in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3A) may be invoked by:

- JMP or CALL instruction to a TSS descriptor in the GDT.
- JMP or CALL instruction to a task-gate descriptor in the GDT or the current LDT.
- An interrupt or exception vector points to a task-gate descriptor in the IDT.

When CR4.CET = 1, the new task must be associated with a 32-bit TSS and must not indicate a transition to virtual-8086 mode. The 32-bit SSP for the new task is located at offset 104 in the 32-bit TSS. Thus the TSS of the new task must be at least 108 bytes. This SSP is required to be 8 byte aligned, and required to point to a "supervisor shadow stack" token (though the task may be at CPL3).

On a task switch initiated by a CALL instruction, an interrupt, or exception, the SSP of the old task is pushed onto the shadow stack of the new task along with the CS and LIP of the old task. This is true even for a nested task switch initiated by a CALL instruction. Likewise, on a task switch initiated by IRET, the SSP of the new task is restored from the shadow stack of old task. The CS and LIP on the shadow stack of the old task are matched against the return address determined by the CS and EIP of the new task. If the match fails, a #CP(FAR-RET/IRET) exception is reported.

18.2.5 Switching Shadow Stacks

The architecture provides a mechanism to switch shadow stacks using a pair of instructions; RSTORSSP and SAVEPREVSSP. The RSTORSSP instruction verifies a shadow-stack-restore token located at the top of the new shadow stack and referenced by the memory operand of this instruction. After RSTORSSP determines the validity of the restore point on the new shadow stack, it switches the SSP to point to the token. The shadow-stack-restore token is a 64-bit value formatted as follows.

- Bit 63:2: Value of shadow stack pointer when this restore point was created.
- Bit 1: Reserved. Must be zero.
- Bit 0: Mode bit. If 0, the token is a compatibility/legacy mode shadow-stack-restore token. If 1, then this shadow stack restore token can be used with a RSTORSSP instruction in 64-bit mode.

The shadow-stack-restore token is created by the SAVEPREVSSP instruction. The operating system may also create a restore point on a shadow stack by creating a shadow-stack-restore token.

Once the shadow stack has been switched to a new shadow stack by the RSTORSSP instruction, software can create a restore point on the old shadow stack by executing the SAVEPREVSSP instruction. In order to allow the SAVEPREVSSP instruction to determine the address where to save the shadow-stack-restore token, the RSTORSSP instruction replaces the shadow-stack-restore token with a previous-ssp token that holds the value of the SSP at the time the RSTORSSP instruction was invoked. The previous-ssp token is formatted as follows.

- Bit 63:2: Shadow stack pointer when the RSTORSSP instruction was invoked, i.e., the SSP of the old shadow stack.
- Bit 1: Set to 1.
- Bit 0: Mode bit. If 0, then this previous-ssp token can be used with a SAVEPREVSSP instruction in compatibility/legacy mode. If 1, then this previous-ssp token can be used with a SAVEPREVSSP instruction in 64-bit mode.

The following figure illustrates the RSTORSSP instruction operation during a shadow stack switching sequence.

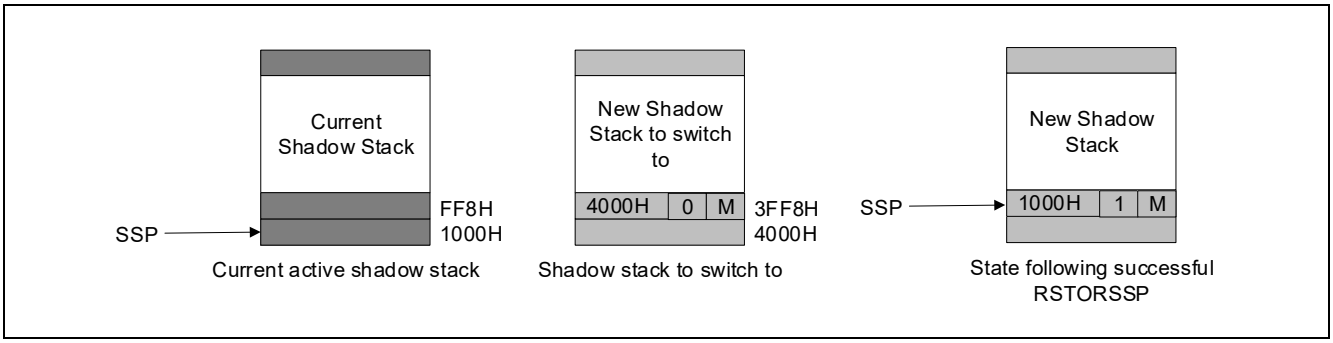


Figure 18-2. RSTORSSP to Switch to New Shadow Stack

In this example, the initial SSP is 1000H and the shadow-stack-restore token is on a new shadow stack at address 3FF8H. The token at address 3FF8H holds the SSP when this restore point was created; in this example it is 4000H.

In order to switch to the new shadow stack, the RSTORSSP instruction is invoked with the memory operand pointing set to 3FF8H. When the RSTORSSP instruction completes, the SSP is set to 3FF8H and the shadow-stack-restore token at 3FF8H is replaced by a previous-ssp token that holds the address 1000H, i.e., the old SSP.

The following figure illustrates the SAVEPREVSSP instruction operation during a shadow stack switching sequence.

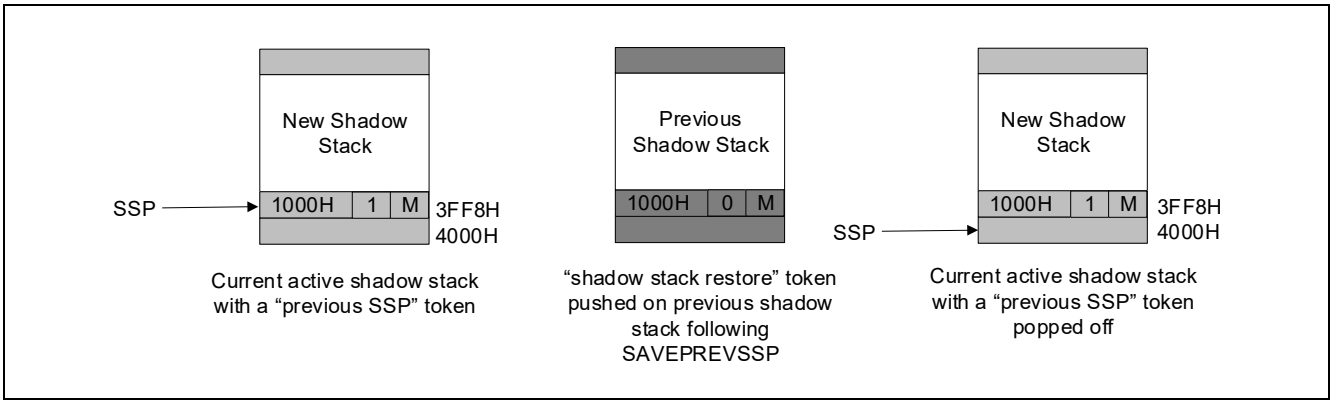


Figure 18-3. SAVEPREVSSP to Save a Restore Point

To allow switching back to this old shadow stack, a SAVEPREVSSP instruction is now invoked. The SAVEPREVSSP instruction does not take any memory operand and expects to find a previous-ssp token at the top of the shadow stack, i.e., at address 3FF8H. The SAVEPREVSSP instruction then saves a shadow-stack-restore token on the old shadow stack at address FF8H, and the token itself holds the address 1000H which is the address recorded in the previous-ssp token. The SAVEPREVSSP instruction also pops the previous-ssp token off the current shadow stack and thus the SSP following SAVEPREVSSP is 4000H.

Subsequently to switch back to the old shadow stack, a RSTORSSP instruction may be invoked with memory operand set to FF8H.

If, following a switch to a new shadow stack, it is not required to create a restore point on the old shadow stack, then the previous-ssp token created by the RSTORSSP instruction can be popped off the shadow stack by using the INCSSP instruction.

See the SAVEPREVSSP and RSTORSSP instruction operations for the detailed algorithm.

18.2.6 Constraining Execution at Targets of RET

Instructions at the target of a RET instruction will not execute, even speculatively, if the RET addresses (either from normal stack or shadow stack) are speculative-only or do not match, unless the target of the RET is also predicted (e.g., by a Return Stack Buffer prediction), when CET shadow stack is enabled. A RET address would be speculative-only if it was modified by an older speculative-only store, or was an older value than the most recent value stored to that address on the logical processor.

18.3 INDIRECT BRANCH TRACKING

When the indirect branch tracking feature is active, the indirect JMP/CALL instruction behavior changes as follows.

- JMP: If the next instruction retired after an indirect JMP is not an ENDBR32 instruction in legacy and compatibility mode, or ENDBR64 instruction in 64-bit mode, then a #CP fault is generated. Below JMP instructions are tracked to enforce an ENDBRANCH. Note that Jcc, RIP relative, and far direct JMP are not included as these have an offset encoded into the instruction and are not exploitable to create unintended control transfers.
  - JMP r/m16, JMP r/m32, JMP r/m64
  - JMP m16:16, JMP m16:32, JMP m16:64
- CALL: If the next instruction retired after an indirect CALL is not an ENDBR32 instruction in legacy and compatibility mode, or ENDBR64 in 64-bit mode, then a #CP fault is generated. Below CALL instructions are tracked to enforce an ENDBRANCH. Note that relative and zero displacement forms of CALL instructions are not included as these have an offset encoded into the instruction and are not exploitable to create unintended control transfers.
  - CALL r/m16, CALL r/m32, CALL r/m64
  - CALL m16:16, CALL m16:32, CALL m16:64

The ENDBR32 and ENDBR64 instructions will have the same effect as the NOP instruction on Intel 64 processors that do not support CET. On processors supporting CET, these instructions do not change register or flag state. This allows CET instrumented programs to execute on processors that do not support CET. Even when CET is supported and enabled, these NOP-like instructions do not affect the execution state of the program, do not cause any additional register pressure, and are minimally intrusive from power and performance perspectives.

The processor implements two dual-state machines to track indirect CALL/JMP for terminations. One state machine is maintained for user mode and one for supervisor mode. At reset the user and supervisor mode state machines are in IDLE state.

On instructions other than indirect CALL/JMP, the state machine stays in the IDLE state.

On an indirect CALL or JMP instruction, the state machine transitions to the WAIT_FOR_ENDBRANCH state.

In the WAIT_FOR_ENDBRANCH state, the indirect branch tracking state machine verifies the next instruction is an ENDBR32 instruction in legacy and compatibility mode, or ENDBR64 instruction in 64-bit mode, and either:

- Causes a #CP fault, or
- Allows the next instruction if legacy compatibility configuration allows (see Section 18.3.6).

The priority of the #CP(ENDBRANCH) exception relative to other events is as follows.

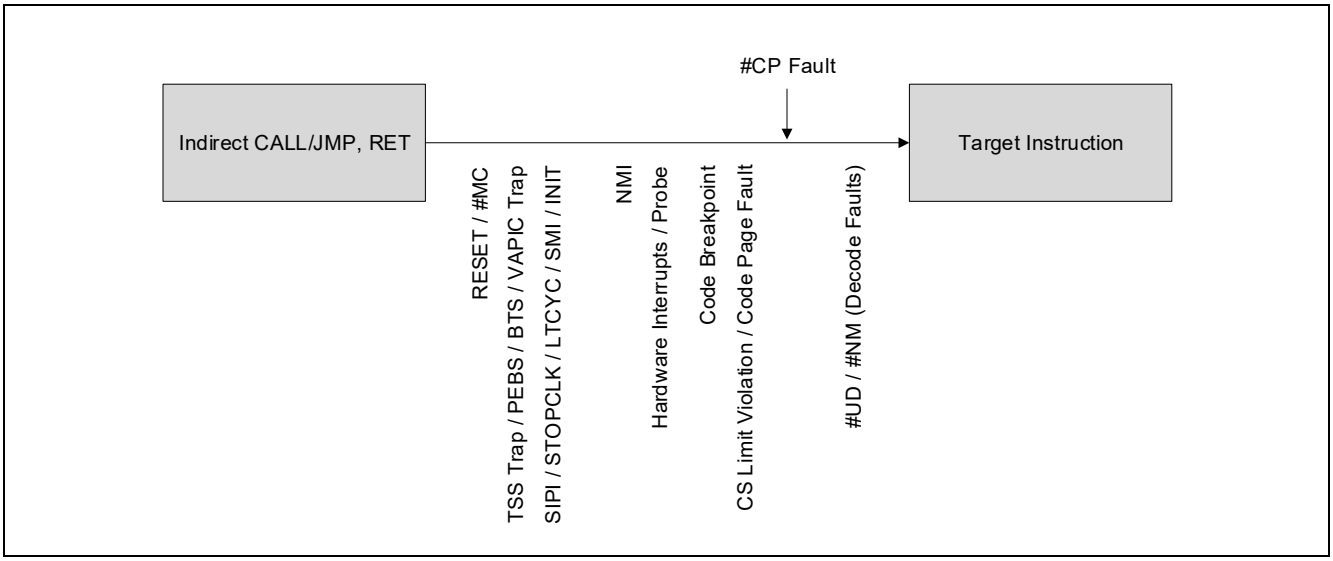


Figure 18-4. Priority of Control Protection Exception on Missing ENDBRANCH

Higher priority faults/traps/events that occur at the end of an indirect CALL/JMP are delivered ahead of any #CP(ENDBRANCH) fault. The CET state machine at the privilege level where the higher priority fault/trap/event occurred retains its state when the control transfers to the fault/trap/event handler. The instruction pointer pushed on the stack for a #CP(ENDBRANCH) fault is the address of the instruction at the target of the indirect CALL/JMP that caused the fault.

18.3.1 No-track Prefix for Near Indirect CALL/JMP

CET allows software to designate certain indirect CALL and JMP instructions as "non-tracked indirect control transfer instructions". Software (e.g., compiler generated code for switch statements, jump tables, etc.) should use the no-track prefix only if they have generated code to validate the possible targets of this CALL/JMP to be legal targets. Software (e.g., compilers), when using the no-track prefix with CALL/JMP where an absolute offset is specified indirectly in a memory location, should ensure that such memory locations cannot be tampered. When enabled by setting the NO_TRACK_EN control in the IA32_U_CET/IA32_S_CET MSR, near indirect CALL and JMP instructions when prefixed with 3EH do not modify the CET indirect branch tracker. Far CALL and JMP instructions are always tracked and ignore the 3EH prefix. When this control is 0, near indirect CALL and JMP instructions are always tracked irrespective of the presence of the 3EH prefix.

In 64-bit mode, the 3EH prefix on an indirect CALL or JMP is recognized as a no-track prefix if there isn't a 64H/65H prefix on the instruction.

In legacy/compatibility mode, the 3EH prefix on an indirect CALL or JMP is recognized as a no-track prefix when it is the last group 2 prefix on the instruction.

18.3.2 Terminology

The pseudocode for the instruction operations use a notation EndbranchEnabled(CPL) as a test of whether indirect branch tracking is enabled at the CPL. This term returns a TRUE or FALSE indication as follows.

```
EndbranchEnabled(CPL):
    IF CR4.CET = 1 AND CR0.PE = 1 AND EFLAGS.VM = 0
        IF CPL = 3
            THEN
                (* Obtain the ENDBRANCH enable from MSR used to enable feature for CPL = 3 *)
                ENDBR_ENABLED = IA32_U_CET.ENDBR_EN;
            ELSE
                (* Obtain the ENDBRANCH enable from MSR used to enable feature for CPL < 3 *)
                ENDBR_ENABLED = IA32_S_CET.ENDBR_EN;
        FI;
        IF ENDBR_ENABLED = 1
            THEN
                return TRUE;
            ELSE
                return FALSE;
        FI;
    ELSE
        (* Indirect branch tracking is not enabled in real mode and virtual-8086 mode or if the master CET feature
         enable in CR4 is disabled *)
        return FALSE;
    ENDIF
```

Likewise the notation EndbranchEnabledAndNotSuppressed is defined as follows:

```
EndbranchEnabledAndNotSuppressed(CPL):
    IF CR4.CET = 1 AND CR0.PE = 1 AND EFLAGS.VM = 0
        IF CPL = 3
            THEN
                (* Obtain the ENDBRANCH enable from MSR used to enable feature for CPL = 3 *)
                ENDBR_ENABLED = IA32_U_CET.ENDBR_EN;
                SUPPRESSED = IA32_U_CET.SUPPRESS;
            ELSE
                (* Obtain the ENDBRANCH enable from MSR used to enable feature for CPL < 3 *)
                ENDBR_ENABLED = IA32_S_CET.ENDBR_EN;
                SUPPRESSED = IA32_S_CET.SUPPRESS;
        FI;
        IF ENDBR_ENABLED = 1 AND SUPPRESSED = 0
            THEN
                return TRUE;
            ELSE
                return FALSE;
        FI;
    ELSE
        (* Indirect branch tracking is not enabled in real mode and virtual-8086 mode or if the master CET feature
         enable in CR4 is disabled *)
        return FALSE;
    ENDIF
```

18.3.3 Indirect Branch Tracking

The hardware implements two CET indirect branch tracker state machines, one for user mode (CPL == 3) and one for supervisor mode (CPL < 3). At any time, which of the CET indirect branch trackers is in the active state depends on the CPL of the machine. When a user space program is executing, the CPL 3 CET indirect branch tracker is active. When supervisor mode software is executing, the CPL < 3 tracker is active. This section describes the various control transfer conditions and the tracker state on those transfers.

18.3.3.1 Control Transfers between CPL 3 and CPL < 3

Some events and instructions can cause control transfer to occur from CPL 3 to CPL < 3, and vice versa. As part of the CPL change the hardware also switches the active CET indirect branch tracker. For example, when an interrupt occurs during execution of a user mode (CPL == 3) program and it causes the CPL to switch to supervisor mode (CPL < 3) then, as part of the CPL change, the user mode CET indirect branch tracker becomes inactive and the supervisor mode CET indirect branch tracker becomes active. A subsequent IRET is used by the interrupt handler to return to the interrupted user mode program. This IRET causes the processor to switch the CPL to user mode (CPL ==3) and, as part of the CPL change, the supervisor mode CET indirect branch tracker becomes inactive and the user mode CET indirect branch tracker becomes active.

The CPL where the event or instruction that caused the control transfer occurs is termed the source CPL, and the CET indirect branch tracker state at that CPL is referred here as the source CET indirect branch tracker state. The CPL reached at the end of the control transfer is termed the destination CPL, and the CET indirect branch tracker state at that CPL is referred to as the destination CET indirect branch tracker state.

This section describes various cases of control transfers that occur between user mode (CPL 3) and supervisor mode (CPL < 3).

In all these cases the source CET indirect branch tracker state becomes not active and retains its state (IDLE, WAIT_FOR_ENDBRANCH), and the target CET indirect branch tracker state becomes active if there was no fault during the transfer.

- Case 1: Far CALL/JMP, SYSCALL/SYSENTER
  - If indirect branch tracking is enabled, the target indirect branch tracker state becomes active and is unsuppressed and goes to WAIT_FOR_ENDBRANCH. This enforces that the subroutine invoked by a far CALL/JMP must begin with an ENDBRANCH.
- Case 2: Hardware interrupt/trap/exception/NMI/Software interrupt/Machine Checks
  - If indirect branch tracking is enabled, the target indirect branch tracker state becomes active and is unsuppressed and goes to WAIT_FOR_ENDBRANCH.
- Case 3: IRET/Far RET
  - If indirect branch tracking enabled, the target indirect branch tracker becomes active and keeps its state. If the user mode was interrupted by a higher priority event, like an interrupt at the end of the indirect CALL/JMP, then when an IRET or Far RET is used to return to the interrupted user mode program, the user mode indirect branch tracker retains its state and a #CP fault will occur if the next instruction decoded is not an ENDBR32/64 according to mode of machine.

18.3.3.2 Control Transfers within CPL 3 or CPL < 3

Some events and instructions can cause control transfer to occur within CPL 3 or CPL < 3. For such transfers since the CPL class does not change, the same indirect branch tracker is used at the beginning and end of the control transfer.

- Case 1: Far CALL/JMP, Near indirect CALL/JMPCALL/JMP
  - Far CALL/JMP: If indirect branch tracking is enabled, active indirect branch tracker is unsuppressed and goes to WAIT_FOR_ENDBRANCH.
  - Near indirect CALL/JMPCALL/JMP: If indirect branch tracking is enabled and not suppressed, active indirect branch tracker goes to WAIT_FOR_ENDBRANCH.
- Case 2: Hardware interrupt/trap/exception/NMI/Software interrupt/Machine Checks

  - If indirect branch tracking is enabled, the active indirect branch tracker is unsuppressed and goes to WAIT_FOR_ENDBRANCH.
- Case 3: IRET
  - If indirect branch tracking is enabled, the active indirect branch tracker keeps its state.

18.3.4 Indirect Branch Tracking State Machine

The state machine is described by Table 18-1.

Table 18-1. Indirect Branch Tracking State Machine

Current State	Trigger	Next State
TRACKER=IDLE, SUPPRESS=0, ENDBR_EN=1	Instructions other than indirect CALL/JMP or 3EH prefixed near indirect CALL/JMP and NO_TRACK_EN=1	TRACKER=IDLE, SUPPRESS=0, ENDBR_EN=1
	Indirect CALL/JMP without 3EH prefix Indirect CALL/JMP with 3EH prefix and NO_TRACK_EN=0 Far CALL/JMP	TRACKER=WAIT_FOR_ENDBRANCH, SUPPRESS=0, ENDBR_EN=1
TRACKER= WAIT_FOR_ENDBRANCH, SUPPRESS=0, ENDBR_EN=1	INT3/INT1	TRACKER= WAIT_FOR_ENDBRANCH, SUPPRESS=0, ENDBR_EN=1
	ENDBRANCH instruction	TRACKER=IDLE, SUPPRESS=0, ENDBR_EN=1
	Successful ENCLU[ERESUME]	TRACKER=IDLE, SUPPRESS=0, ENDBR_EN=1
	Instructions other than ENDBRANCH, successful ENCLU[ERESUME] or INT3 or INT1	If legacy compatibility treatment is not enabled or if not allowed by legacy code page bitmap: ▪ No state change and deliver #CP (ENDBRANCH) If legacy compatibility treatment is enabled and transfer allowed by legacy code page bitmap: ▪ TRACKER=IDLE, SUPRESS=!SUPPRESS_DIS, ENDBR_EN=1
TRACKER=x, SUPPRESS=x, ENDBR_EN=0	All instructions	TRACKER=x, SUPPRESS=x, ENDBR_EN=0
TRACKER=IDLE, SUPPRESS=1, ENDBR_EN=1	Far CALL/JMP, INTn/INT3/INTO	TRACKER=WAIT_FOR_ENDBRANCH, SUPPRESS=0, ENDBR_EN=1
	ENDBRANCH instruction Successful ENCLU[ERESUME]	TRACKER=IDLE, SUPPRESS=0, ENDBR_EN=1
	All other instructions including indirect CALL/JMP	TRACKER=IDLE, SUPPRESS=1, ENDBR_EN=1
TRACKER=1, SUPPRESS=1, ENDBR_EN=1 (This state cannot be reached by hardware and is disallowed as a valid state by WRMSR/XRSTORS/VM entry/VM exit)	NA	NA

18.3.5 INT3 Treatment

INT3 are treated special in the WAIT_FOR_ENDBRANCH state. Occurrence of INT3 do not move the tracker to IDLE but instead the #BP trap from the INT3 instructions respectively is delivered as a higher priority event than the #CP exception due to missing ENDBRANCH.

Inside an enclave, INT3 delivers a fault-class exception and thus does not require the CPL to be less than DPL in the IDT gate 3. Following opt-out entry, the instruction delivers #UD. Following opt-in entry, INT3 delivers #BP. The special treatment of INT3 in WAIT_FOR_ENDBRANCH state does not apply in enclave mode following opt-out entry.

18.3.6 Legacy Compatibility Treatment

ENDBRANCH legacy compatibility treatment allows a CET enabled program to be used with legacy software that was not compiled / instrumented with ENDBRANCH. A CET enabled program enters legacy compatibility treatment when all of the below conditions are met.

1. Legacy compatibility configuration is enabled in this CPL class by setting the LEG_IW_EN bit in IA32_U_CET/IA32_S_CET.
2. Control transfer is performed using an indirect CALL/JMP without no-track prefix to an instruction other than ENDBRANCH.
3. The legacy code page bitmap is setup to indicate that the target of the control transfer is a legacy code page.

The legacy code page bitmap is a data structure in program memory that is used by the hardware to determine if the code page to which a legacy transfer is being performed is allowed. The access rights for accessing the legacy code page bitmap is determined by the current privilege level (CPL). The legacy code page bitmap is expected to be setup as a read-only data structure.

When a matching ENDBRANCH instruction is not decoded at the target of an indirect CALL/JMP when required, the processor performs the below actions.

CET indirect branch tracking state machine violation event handler:

```
If LEG_IW_EN == 1
    LA = LIP;
    IF ENCLAVE_MODE == 1
        LA = LA – SECS.BASEADDR;
    ENDIF
    (* Load byte from bitmap. Address-size attribute for this load is 64 bits if IA32_EFER.LMA is 1 and is 32 bits when IA32_EFER.LMA
is 0 *)
    IF (IA32_EFER.LMA & CS.L) == 0
        BITMAP_BYTE = load 1 byte from address (BITMAP_BASE + LA[31:15])
    ELSE IF (CR4.LA57 == 0)
        BITMAP_BYTE = load 1 byte from address (BITMAP_BASE + LA[47:15])
    ELSE
        BITMAP_BYTE = load 1 byte from address (BITMAP_BASE + LA[56:15])
    FI;
    IF BITMAP_BYTE & (1 << LA[14:12]) == 0 then Deliver #CP(ENDBRANCH) fault
        IF CPL = 3
            IA32_U_CET.TRACKER = IDLE
            IA32_U_CET.SUPPRESS = IA32_U_CET.SUPPRESS_DIS == 0 ? 1 : 0
        ELSE
            IA32_S_CET.TRACKER = IDLE
            IA32_S_CET.SUPPRESS = IA32_S_CET.SUPPRESS_DIS == 0 ? 1 : 0
    ENDIF
    Restart the instruction (handle all arch. consistency around MOV SS state machines, STI etc.) without
    opening up interrupt/trap window.
ELSE
    Deliver #CP(ENDBRANCH) Fault
```

```
ENDIF
```

Faults/traps in pseudocode are delivered normally (e.g., #PF, EPT violation). On a fault, the active tracker holds the last value (WAIT_FOR_ENDBRANCH) and the address saved on the stack is the current IP (instruction that wasn't the ENDBRANCH).

The CET indirect branch tracking state machine is suppressed in legacy compatibility mode if the SUPPRESS_DIS control bit is 0.

Once the CET indirect branch tracking state machine has been suppressed, subsequent indirect CALL/JMP are not tracked for termination instruction.

Once CET indirect branch tracking has been suppressed, subsequent execution of ENDBRANCH instructions will do the following (see the ENDBR32 and ENDBR64 instructions in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 2A for details).

```
IF EndbranchEnabled(CPL) == 0
    NOP
ELSE
    SUPPRESS = 0
    TRACKER = IDLE
ENDIF
```

18.3.6.1 Legacy Code Page Bitmap Format

The legacy code page bitmap is a flat bitmap whose linear address is pointed to by the EB_LEG_BITMAP_BASE. Each bit in the bitmap represents a 4K page in linear memory. If the bit is 1 it indicates that the corresponding code page is a legacy code page; else it is a CET-enabled code page.

The processor uses the linear address of the instruction to which legacy transfer was attempted to lookup the bitmap. Bits of the linear address used as index in the bitmap are as follows.

- In legacy and compatibility mode: Bits 31:12.
- In 64-bit mode (EFER.LMA=1 and CS.L=1): Bits 47:12.

18.3.7 Other Considerations

18.3.7.1 Intel® Transactional Synchronization Extensions (Intel® TSX) Interactions

The XBEGIN instruction encodes the relative offset to the abort handler and hence the fallback to the abort handler can be considered as a "direct" branch and the abort handler does not need to have an ENDBRANCH.

CET continues to enforce indirect CALL/JMP tracking within a transaction. Legacy compatibility treatment inside a transaction functions normally. If a transaction abort occurs then the processor sets the state of the indirect branch tracker to IDLE and not-suppressed.

18.3.7.2 #CP(ENDBRANCH) Priority w.r.t #NM and #UD

#NM, #UD and #CP(ENDBRANCH) are opcode based faults. However, #CP(ENDBRANCH) is in a higher priority class than #NM and #UD as CET architecturally requires an ENDBRANCH at target of indirect CALL/JMP.

18.3.7.3 #CP(ENDBRANCH) Priority w.r.t #BP and #DB

Debug Exceptions priority is as follows.

- Traps delivered before any #CP(ENDBRANCH) fault: Data breakpoint trap, IO breakpoint trap single step trap, task switch trap.
- Code Breakpoint fault detected before instruction decode and delivered before #CP(ENDBRANCH).
- General-detect (GD) exception condition fault: Lower priority than #CP(ENDBRANCH).

- On IRET back from #DB/#BP, the source indirect branch tracker becomes active if enabled and not suppressed.

INT3 does not cause #CP(ENDBRANCH) to support debugger usage of replacing bytes of ENDBRANCH with INT3 to set breakpoints. INT3 at target of a CALL-JMP(indirect) cause #BP(INT3) instead of #CP(ENDBRANCH), #CP(ENDBRANCH) fault is delayed. #BP caused by INT3 treated like other events that are higher priority than CET fault. On IRET back from #BP the source indirect tracker becomes active if enabled and not suppressed.

18.3.8 Constraining Speculation after Missing ENDBRANCH

When the CET tracker is in the WAIT_FOR_ENDBRANCH state, instruction execution will be limited or blocked, even speculatively, if the next instruction is not an ENDBRANCH.

This means that when indirect branch tracking is enabled and not suppressed, the instructions at the target of a near indirect JMP/CALL without the no-track prefix will only speculatively execute if there is an ENDBRANCH at the target. This can constrain both attacker controlled prediction as well as attacker controlled jump redirection attacks on near indirect JMPs/CALLs by reducing the gadgets available to an attacker using these techniques. Early implementations of CET may limit the speculative execution to a small number of instructions (less than 8, with no more than 5 loads) past a missing ENDBRANCH, while later implementations will completely block the speculative execution of instructions after a missing ENDBRANCH.

This mechanism also limits or blocks speculation of the next sequential instructions after an indirect JMP or CALL, presuming the JMP/CALL puts the CET tracker into the WAIT_FOR_ENDBRANCH state and the next sequential instruction is not an ENDBRANCH.

18.4 INTEL® TRUSTED EXECUTION TECHNOLOGY (INTEL® TXT) INTERACTIONS

GETSEC[ENTERACCS] and GETSEC[SENTER] clear CR4.CET, and it is not restored when these instructions complete.

GETSEC[EXITAC] will cause #GP(0) fault if CR4.CET is set.

CHAPTER 19
PROGRAMMING WITH INTEL® ADVANCED MATRIX EXTENSIONS

19.1 INTRODUCTION

Intel® Advanced Matrix Extensions (Intel® AMX) is a new 64-bit programming paradigm consisting of two components: a set of 2-dimensional registers (tiles) representing sub-arrays from a larger 2-dimensional memory image, and an accelerator able to operate on tiles, the first implementation is called TMUL (tile matrix multiply unit).

An Intel AMX implementation enumerates to the programmer how the tiles can be programmed by providing a palette of options. Two palettes are supported; palette 0 represents the initialized state, and palette 1 consists of 8 KB of storage spread across 8 tile registers named TMM0..TMM7. Each tile has a maximum size of 16 rows x 64 bytes, (1 KB), however the programmer can configure each tile to smaller dimensions appropriate to their algorithm. The tile dimensions supplied by the programmer (rows and bytes_per_row, i.e., **colsb**) are metadata that drives the execution of tile and accelerator instructions. In this way, a single instruction can launch autonomous multi-cycle execution in the tile and accelerator hardware. The palette value (**palette_id**) and metadata are held internally in a tile related control register (TILECFG). The TILECFG contents will be commensurate with that reported in the palette_table (see "CPUID—CPU Identification" in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 2A for a description of the available parameters).

Intel AMX is an extensible architecture. New accelerators can be added, or the TMUL accelerator may be enhanced to provide higher performance. In these cases, the state (TILEDATA) provided by tiles may need to be made larger, either in one of the metadata dimensions (more rows or colsb) and/or by supporting more tile registers (names). The extensibility is carried out by adding new palette entries describing the additional state. Since execution is driven through metadata, an existing Intel AMX binary could take advantage of larger storage sizes and higher performance TMUL units by selecting the most powerful palette indicated by CPUID and adjusting loop and pointer updates accordingly.

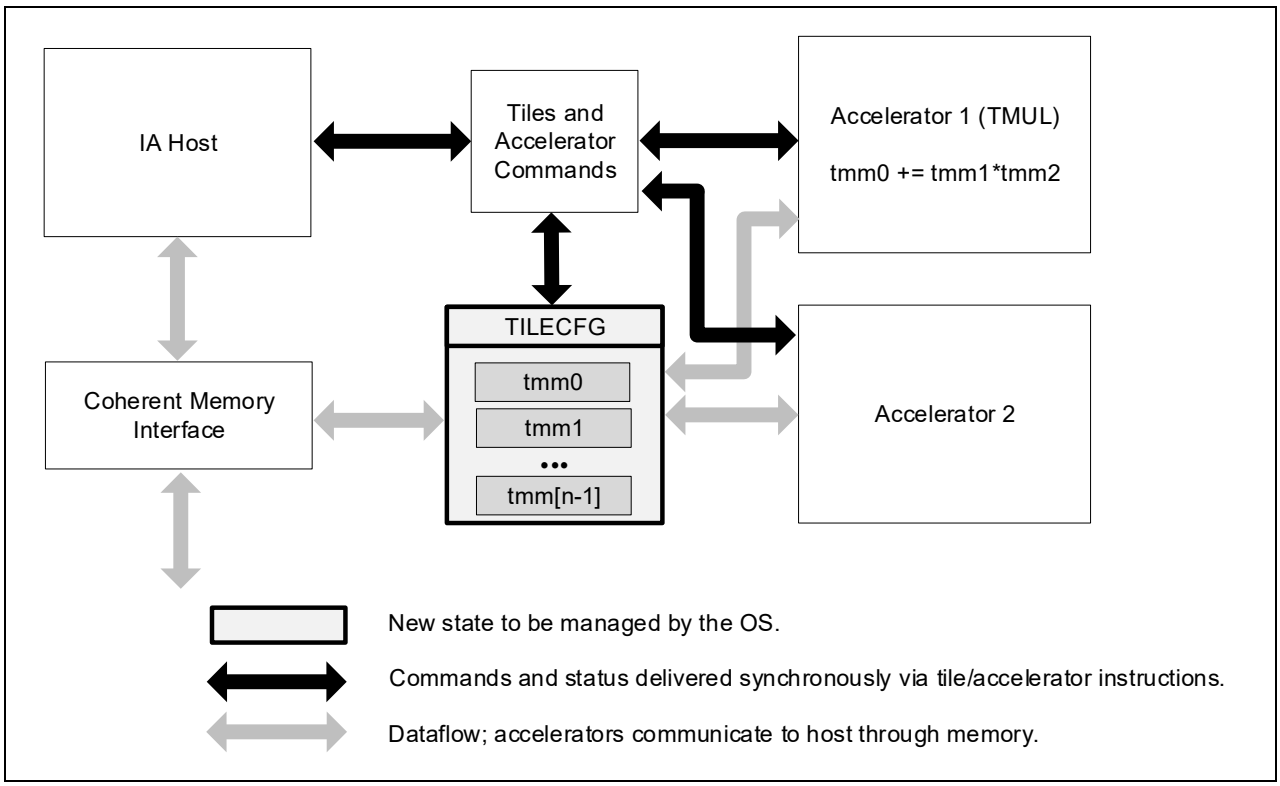


Figure 19-1. Intel® AMX Architecture

Figure 19-1 shows a conceptual diagram of the Intel AMX architecture. An Intel architecture host drives the algorithm, the memory blocking, loop indices and pointer arithmetic. Tile loads and stores and accelerator commands are sent to multi-cycle execution units. Status, if required, is reported back. Intel AMX instructions are synchronous in the Intel architecture instruction stream and the memory loaded and stored by the tile instructions is coherent with respect to the host's memory accesses. There are no restrictions on interleaving of Intel architecture and Intel AMX code or restrictions on the resources the host can use in parallel with Intel AMX (e.g., Intel AVX-512). There is also no architectural requirement on the Intel architecture compute capability of the Intel architecture host other than it supports 64-bit mode.

Intel AMX instructions use new registers and inherit basic behavior from Intel architecture in the same manner that Intel SSE and Intel AVX did. Tile instructions include loads and stores using the traditional Intel architecture register set as pointers. The TMUL instruction set (defined to be CPUID bits AMX_BF16 and AMX_INT8) only supports reg-reg operations.

TILECFG is programmed using the LDTILECFG instruction. The selected palette defines the available storage and general configuration while the rest of the memory data specifies the number of rows and column bytes for each tile. Consistency checks are performed to ensure the TILECFG matches the restrictions of the palette. A General Protection fault (#GP) is reported if the LDTILECFG fails consistency checks. A successful load of TILECFG with a palette_id other than 0 is represented in this document with TILES_CONFIGURED = 1. When the TILECFG is initialized (palette_id = 0), it is represented in the document as TILES_CONFIGURED = 0. Nearly all Intel AMX instructions will generate a #UD exception if TILES_CONFIGURED is not equal to 1; the exceptions are those that do TILECFG maintenance: LDTILECFG, STTILECFG, and TILERELEASE.

If a tile is configured to contain M rows by N column bytes, LDTILECFG will ensure that the metadata values are appropriate to the palette (e.g., that $M \leq 16$ and $N \leq 64$ for palette 1). The four M and N values can all be different as long as they adhere to the restrictions of the palette. Further dynamic checks are done in the tile and the TMUL instruction set to deal with cases where a legally configured tile may be inappropriate for the instruction operation. Tile registers can be set to 'invalid' by configuring the rows and colsb to '0'.

Tile loads and stores are strided accesses from the application memory to packed rows of data. Algorithms are expressed assuming row major data layout. Column major users should translate the terms according to their orientation.

TILELOAD* and TILESTORE* instructions are restartable and can handle (up to) 2*rows page faults per instruction. Restartability is provided by a **start_row** parameter in the TILECFG register.

The TMUL unit is conceptually a grid of fused multiply-add units able to read and write tiles. The dimensions of the TMUL unit (tmul_maxk and tmul_maxn) are enumerated similar to the maximum dimensions of the tiles (see "CPUID—CPU Identification" in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 2A for details).

The matrix multiplications in the TMUL instruction set compute C[M][N] += A[M][K] * B[K][N]. The M, N, and K values will cause the TMUL instruction set to generate a #UD exception if the dimensions do not match for matrix multiply or do not match the palette.

In Figure 19-2, the number of rows in tile B matches the K dimension in the matrix multiplication pseudocode. K dimensions smaller than that enumerated in the TMUL grid are also possible and any additional computation the TMUL unit can support will not affect the result.

The number of elements specified by colsb of the B matrix is also less than or equal to tmul_maxn. Any remaining values beyond that specified by the metadata will be set to zero.

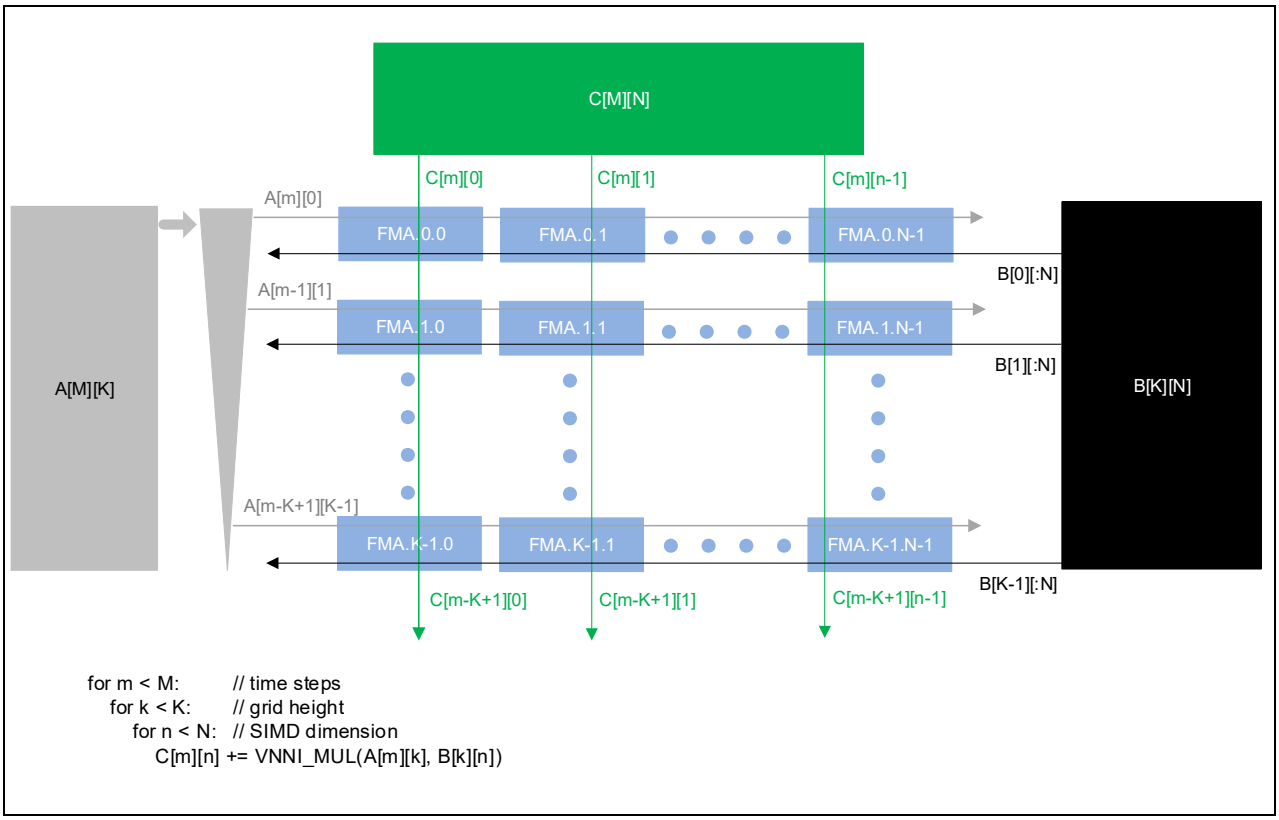


Figure 19-2. The TMUL Unit

The XSAVE feature set supports context management of the new state defined for Intel AMX. This support is described in Section 19.2.

19.1.1 Tile Architecture Details

The supported parameters for the tile architecture are reported via CPUID; this includes information about how the number of tile registers (max_names) can be configured (the palette). Configuring the tile architecture is intended to be done once when entering a region of tile code using the LDTILECFG instruction specifying the selected palette and describing in detail the configuration for each tile. Incorrect assignments will result in a General Protection fault (#GP). Successful LDTILECFG initializes (zeroes) TILEDATA.

Exiting a tile region is done with the TILERELEASE instruction. It takes no parameters and invalidates all tiles (indicating that the data no longer needs any saving or restoring). Essentially, it is an optimization of LDTILECFG with an implicit palette of 0.

For applications that execute consecutive Intel AMX regions with differing configurations, TILERELEASE is not required between them since the second LDTILECFG will clear all the data while loading the new configuration. There is no instruction set support for automatic nesting of tile regions, though with sufficient effort software can accomplish this by saving and restoring TILEDATA and TILECFG either through the XSAVE architecture or the Intel AMX instructions.

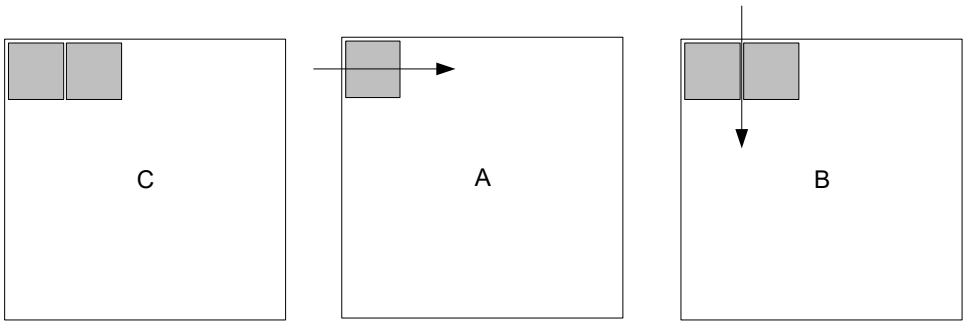
The tile architecture boots in its INIT state, with TILECFG and TILEDATA set to zero. A successfully executing LDTILECFG instruction to a non-zero palette sets the TILES_CONFIGURED=1, indicating the TILECFG is not in the INIT state. The TILERELEASE instruction sets TILES_CONFIGURED = 0 and initializes (zeroes) TILEDATA.

To facilitate handling of tile configuration data, there is a STTILECFG instruction. If the tile configuration is in the INIT state (TILES_CONFIGURED == 0), then STTILECFG will write 64 bytes of zeros. Otherwise STTILECFG will store the TILECFG to memory in the format used by LDTILECFG.

19.1.2 TMUL Architecture Details

The supported parameters for the TMUL architecture are reported via CPUID; see "CPUID—CPU Identification" in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 2A, for details. These parameters include a maximum height (**tmul_maxk**) and a maximum SIMD dimension (**tmul_maxn**). The metadata that accompanies the srcdest, src1, and src2 tiles to the TMUL unit will be dynamically checked to see that they match the TMUL unit support for the data type and match the requirements of a meaningful matrix multiplication.

Figure 19-3 shows an example of the inner loop of an algorithm of using the TMUL architecture to compute a matrix multiplication. In this example, we use two result tiles, tmm0 and tmm1, from matrix C to accumulate the intermediate results. One tile from the A matrix (tmm2) is re-used twice as we multiply it by two tiles from the B matrix. The algorithm then advances pointers to load a new A tile and two new B tiles from the directions indicated by the arrows. An outer loop, not shown, adjusts the pointers for the C tiles.




```
   LDTILECFG [rax]
// assume some outer loops driving the cache tiling (not shown)
{
   TILELOADD tmm0, [rsi+rdi]     // srcdst, RSI points to C, RDI is strided value
   TILELOADD tmm1, [rsi+rdi+N]   // second tile of C, unrolling in SIMD dimension N
   MOV r14, 0
LOOP:
   TILELOADD tmm2, [r8+r9]       // src2 is strided load of A, reused for 2 TMUL instr.
   TILELOADD tmm3, [r10+r11]     // src1 is strided load of B
   TDPBUSD   tmm0, tmm2, tmm3  // update left tile of C
   TILELOADD tmm3, [r10+r11+N]   // src1 loaded with B from next rightmost tile
   TDPBUSD   tmm1, tmm2, tmm3  // update right tile of C
   ADD r8, K                 // update pointers by constants known outside of loop
   ADD r10, K*r11
   ADD r14, K
   CMP r14, LIMIT
   JNE LOOP

   TILESTORED [rsi+rdi],   tmm0  // update the C matrix in memory
   TILESTORED [rsi+rdi+M], tmm1
} // end of outer loop

   TILERELEASE         // return tiles to INIT state
```

Figure 19-3. Matrix Multiply C+= A*B

19.1.3 Handling of Tile Row and Column Limits

Intel AMX operations will zero any rows and any columns beyond the dimensions specified by TILECFG. Tile operations will zero the data beyond the configured number of column bytes as each row is written. For example, with 64-byte rows and a tile configured with 10 rows and 48 columns, an operation writing dword elements would write

each of the first 10 rows with 48 bytes of output/result data and zero the remaining 16 bytes in each row. Tile operations also fully zero any rows after the first 10 configured rows. When using a 1 KByte tile with 64-byte rows, there would be 16 rows, so in this example, the last 6 rows would also be zeroed.

Intel AMX instructions will always obey the metadata on reads and the zeroing rules on writes, and so a subsequent XSAVE would see zeros in the appropriate locations. Tiles that are not written by Intel AMX instructions between XRSTOR and XSAVE will write back with the same image they were loaded with regardless of the value of TILECFG.

19.1.4 Exceptions and Interrupts

Tile instructions are restartable so that operations that access strided memory can restart after page faults. To support restarting instructions after these events, the instructions store information in the **TILECFG.start_row** register. TILECFG.start_row indicates the row that should be used for restart; i.e., it indicates **next row after** the rows that have already been successfully loaded (on a TILELOAD) or written to memory (on a TILESTORE) and prevents repeating work that was successfully done.

The TMUL instruction set is not sensitive to the TILECFG.start_row value; this is due to there not being TMUL instructions with memory operands or any restartable faults.

19.2 RECOMMENDATIONS FOR SYSTEM SOFTWARE

Intel AMX is an XSAVE-enabled feature, meaning that it requires use of the XSAVE feature set for their enabling. Specifically, Intel AMX instructions and state are available only if system software has set CR4.OSXSAVE and also set XCR0[18:17] to 11B. In addition, use of Intel AMX instructions is disabled if system software has used extended feature disable (XFD) and set either IA32_XFD[17] or IA32_XFD[18] to 1. See Chapter 13, “Managing State Using the XSAVE Feature Set,” for more details.

NOTE

The first processors implementing Intel AMX will support setting IA32_XFD[18] but not IA32_XFD[17].

Once Intel AMX has been enabled, system software can disable it by clearing XCR0[18:17], by clearing CR4.OSXSAVE, or by setting either IA32_XFD[17] or IA32_XFD[18]. Before doing so, system software should first initialize AMX state (e.g., by executing TILERELEASE); maintaining AMX state in a non-initialized state may have negative power and performance implications and will prevent the execution of In-Field Scan tests. In addition, software should not rely on the state of the tile data after setting IA32_XFD[17] or IA32_XFD[18]; software should always reload or reinitialize the tile data after clearing IA32_XFD[17] and IA32_XFD[18].

System software should not use XFD to implement a “lazy restore” approach to management of the TILEDATA state component. This approach will **not** operate correctly for a variety of reasons. One is that the LDTILECFG and TILERELEASE instructions initialize TILEDATA and do not cause an #NM exception. Another is that an execution of XSAVE, XSAVEC, XSAVEOPT, or XSAVES by a user thread will save TILEDATA as initialized instead of the data expected by the user thread.

19.3 IMPLEMENTATION PARAMETERS

The parameters are reported via CPUID leaf 1DH. Index 0 reports all zeros for all fields.

```
define palette_table[id]:
    uint16_t total_tile_bytes
    uint16_t bytes_per_tile
    uint16_t bytes_per_row
    uint16_t max_names
    uint16_t max_rows
```

The tile parameters are set by LDTILECFG or XRSTOR* of TILECFG:

```
define tile[tid]:
    byte rows
    word colsb // bytes_per_row
    bool valid
```

19.4 HELPER FUNCTIONS

The helper functions used in Intel AMX instructions are defined below.

```
define write_row_and_zero(treg, r, data, nbytes):
   for j in 0 ...nbytes-1:
       treg.row[r].byte[j] := data.byte[j]

   // zero the rest of the row
   for j in nbytes ... palette_table[tilecfg.palette_id].bytes_per_row-1:
      treg.row[r].byte[j] := 0


define zero_upper_rows(treg, r):
   for i in r ... palette_table[tilecfg.palette_id].max_rows-1:
      for j in 0 ... palette_table[tilecfg.palette_id].bytes_per_row-1:
            treg.row[i].byte[j] := 0


define zero_tilecfg_start():
   tilecfg.start_row :=0


define zero_all_tile_data():
  if XCR0[TILEDATA]:
      b := CPUID.0DH.TILEDATA:EAX // size of feature
      for j in 0 ...b:
            TILEDATA.byte[j] := 0
```

```
define xcr0_supports_palette(palette_id):
  if palette_id == 0:
     return 1
  elif palette_id == 1:
     if XCR0[TILECFG] and XCR0[TILEDATA]:
        return 1
   return 0
```

CHAPTER 20
INPUT/OUTPUT

In addition to transferring data to and from external memory, IA-32 processors can also transfer data to and from input/output ports (I/O ports). I/O ports are created in system hardware by circuity that decodes the control, data, and address pins on the processor. These I/O ports are then configured to communicate with peripheral devices. An I/O port can be an input port, an output port, or a bidirectional port. Some I/O ports are used for transmitting data, such as to and from the transmit and receive registers, respectively, of a serial interface device. Other I/O ports are used to control peripheral devices, such as the control registers of a disk controller.

This chapter describes the processor's I/O architecture. The topics discussed include:

- I/O port addressing.
- I/O instructions.
- I/O protection mechanism.

20.1 I/O PORT ADDRESSING

The processor permits applications to access I/O ports in either of two ways:

- Through a separate I/O address space.
- Through memory-mapped I/O.

Accessing I/O ports through the I/O address space is handled through a set of I/O instructions and a special I/O protection mechanism. Accessing I/O ports through memory-mapped I/O is handled with the processor's general-purpose move and string instructions, with protection provided through segmentation or paging. I/O ports can be mapped so that they appear in the I/O address space or the physical-memory address space (memory mapped I/O) or both.

One benefit of using the I/O address space is that writes to I/O ports are guaranteed to be completed before the next instruction in the instruction stream is executed. Thus, I/O writes to control system hardware cause the hardware to be set to its new state before any other instructions are executed. See Section 20.6, "Ordering I/O," for more information on serializing of I/O operations.

20.2 I/O PORT HARDWARE

From a hardware point of view, I/O addressing is handled through the processor's address lines. For the P6 family, Pentium 4, and Intel Xeon processors, the request command lines signal whether the address lines are being driven with a memory address or an I/O address; for Pentium processors and earlier IA-32 processors, the M/IO# pin indicates a memory address (1) or an I/O address (0). When the separate I/O address space is selected, it is the responsibility of the hardware to decode the memory-I/O bus transaction to select I/O ports rather than memory. Data is transmitted between the processor and an I/O device through the data lines.

20.3 I/O ADDRESS SPACE

The processor's I/O address space is separate and distinct from the physical-memory address space. The I/O address space consists of 2^{16} (64K) individually addressable 8-bit I/O ports, numbered 0 through FFFFH. I/O port addresses 0F8H through 0FFH are reserved. Do not assign I/O ports to these addresses. The result of an attempt to address beyond the I/O address space limit of FFFFH is implementation-specific; see the Developer's Manuals for specific processors for more details.

Any two consecutive 8-bit ports can be treated as a 16-bit port, and any four consecutive ports can be a 32-bit port. In this manner, the processor can transfer 8, 16, or 32 bits to or from a device in the I/O address space. Like words in memory, 16-bit ports should be aligned to even addresses (0, 2, 4, ...) so that all 16 bits can be transferred in a

single bus cycle. Likewise, 32-bit ports should be aligned to addresses that are multiples of four (0, 4, 8, ...). The processor supports data transfers to unaligned ports, but there is a performance penalty because one or more extra bus cycle must be used.

The exact order of bus cycles used to access unaligned ports is undefined and is not guaranteed to remain the same in future IA-32 processors. If hardware or software requires that I/O ports be written to in a particular order, that order must be specified explicitly. For example, to load a word-length I/O port at address 2H and then another word port at 4H, two word-length writes must be used, rather than a single doubleword write at 2H.

Note that the processor does not mask parity errors for bus cycles to the I/O address space. Accessing I/O ports through the I/O address space is thus a possible source of parity errors.

20.3.1 Memory-Mapped I/O

I/O devices that respond like memory components can be accessed through the processor's physical-memory address space (see Figure 20-1). When using memory-mapped I/O, any of the processor's instructions that reference memory can be used to access an I/O port located at a physical-memory address. For example, the MOV instruction can transfer data between any register and a memory-mapped I/O port. The AND, OR, and TEST instructions may be used to manipulate bits in the control and status registers of a memory-mapped peripheral device.

Certain instructions may take an exception or VM exit after completing a memory access (either a read or a write) to a memory-mapped I/O address. This exception or VM exit could be due to the instruction performing multiple memory accesses (e.g., MOVS, PUSH mem, POP mem, PUSHAD, etc.) or could be due to the ordering of exceptions or VM exits within the instruction (e.g., a DIV mem that takes a #DE or a CALL that causes a task switch VM exit). If software later re-executes that instruction (e.g., after an IRET or VMRESUME), the MMIO (memory-mapped I/O) access may occur again. If the memory-mapped I/O access has a side-effect, that side-effect may be executed each time the memory-mapped I/O access occurs. If that is problematic, software must ensure that exceptions or VM exits do not occur after accessing the MMIO.

When using memory-mapped I/O, caching of the address space mapped for I/O operations must be prevented. With the Pentium 4, Intel Xeon, and P6 family processors, caching of I/O accesses can be prevented by using memory type range registers (MTRRs) to map the address space used for the memory-mapped I/O as uncacheable (UC). See Chapter 14, "Memory Cache Control," in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3A, for a complete discussion of the MTRRs.

The Pentium and Intel486 processors do not support MTRRs. Instead, they provide the KEN# pin, which when held inactive (high) prevents caching of all addresses sent out on the system bus. To use this pin, external address decoding logic is required to block caching in specific address spaces.

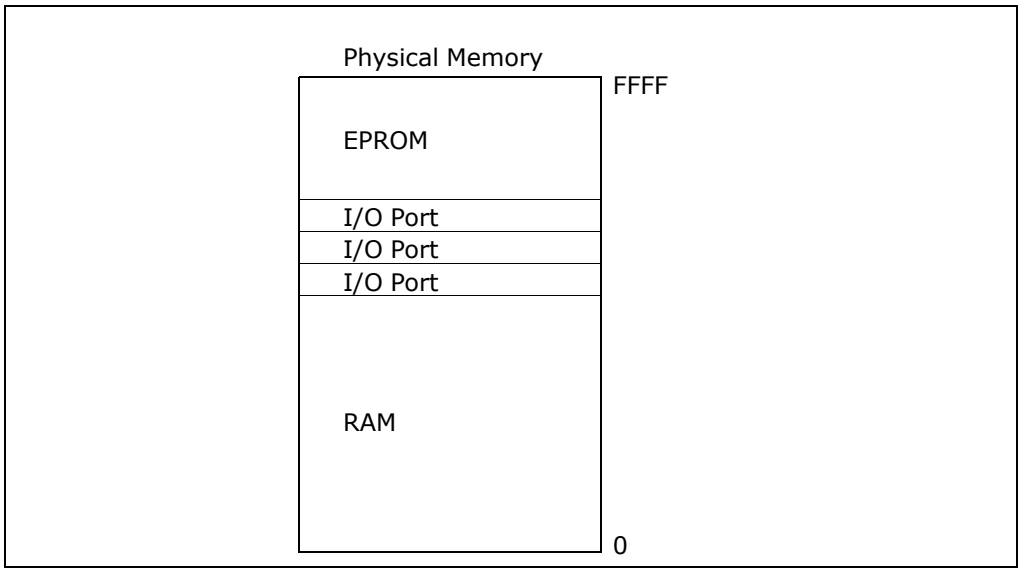


Figure 20-1. Memory-Mapped I/O

All the IA-32 processors that have on-chip caches also provide the PCD (page-level cache disable) flag in page table and page directory entries. This flag allows caching to be disabled on a page-by-page basis. See "Page-Directory and Page-Table Entries" in Chapter 4 of the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3A.

20.4 I/O INSTRUCTIONS

The processor's I/O instructions provide access to I/O ports through the I/O address space. (These instructions cannot be used to access memory-mapped I/O ports.) There are two groups of I/O instructions:

- Those that transfer a single item (byte, word, or doubleword) between an I/O port and a general-purpose register.
- Those that transfer strings of items (strings of bytes, words, or doublewords) between an I/O port and memory.

The register I/O instructions IN (input from I/O port) and OUT (output to I/O port) move data between I/O ports and the EAX register (32-bit I/O), the AX register (16-bit I/O), or the AL (8-bit I/O) register. The address of the I/O port can be given with an immediate value or a value in the DX register.

The string I/O instructions INS (input string from I/O port) and OUTS (output string to I/O port) move data between an I/O port and a memory location. The address of the I/O port being accessed is given in the DX register; the source or destination memory address is given in the DS:ESI or ES:EDI register, respectively.

When used with the repeat prefix REP, the INS and OUTS instructions perform string (or block) input or output operations. The repeat prefix REP modifies the INS and OUTS instructions to transfer blocks of data between an I/O port and memory. Here, the ESI or EDI register is incremented or decremented (according to the setting of the DF flag in the EFLAGS register) after each byte, word, or doubleword is transferred between the selected I/O port and memory.

See the references for IN, INS, OUT, and OUTS in Chapter 3 and Chapter 4 of the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volumes 2A, 2B, 2C, & 2D, for more information on these instructions.

20.5 PROTECTED-MODE I/O

When the processor is running in protected mode, the following protection mechanisms regulate access to I/O ports:

- When accessing I/O ports through the I/O address space, two protection devices control access:
  - The I/O privilege level (IOPL) field in the EFLAGS register.
  - The I/O permission bit map of a task state segment (TSS).
- When accessing memory-mapped I/O ports, the normal segmentation and paging protection and the MTRRs (in processors that support them) also affect access to I/O ports. See Chapter 6, "Protection," and Chapter 14, "Memory Cache Control," in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3A, for a complete discussion of memory protection.

The following sections describe the protection mechanisms available when accessing I/O ports in the I/O address space with the I/O instructions.

20.5.1 I/O Privilege Level

In systems where I/O protection is used, the IOPL field in the EFLAGS register controls access to the I/O address space by restricting use of selected instructions. This protection mechanism permits the operating system or executive to set the privilege level needed to perform I/O. In a typical protection ring model, access to the I/O address space is restricted to privilege levels 0 and 1. Here, the kernel and the device drivers are allowed to perform I/O, while less privileged device drivers and application programs are denied access to the I/O address space. Application programs must then make calls to the operating system to perform I/O.

The following instructions can be executed only if the current privilege level (CPL) of the program or task currently executing is less than or equal to the IOPL: IN, INS, OUT, OUTS, CLI (clear interrupt-enable flag), and STI (set interrupt-enable flag). These instructions are called **I/O sensitive** instructions, because they are sensitive to the IOPL field. Any attempt by a less privileged program or task to use an I/O sensitive instruction results in a general-protection exception (#GP) being signaled. Because each task has its own copy of the EFLAGS register, each task can have a different IOPL.

The I/O permission bit map in the TSS can be used to modify the effect of the IOPL on I/O sensitive instructions, allowing access to some I/O ports by less privileged programs or tasks (see Section 20.5.2, "I/O Permission Bit Map").

A program or task can change its IOPL only with the POPF and IRET instructions; however, such changes are privileged. No procedure may change the current IOPL unless it is running at privilege level 0. An attempt by a less privileged procedure to change the IOPL does not result in an exception; the IOPL simply remains unchanged.

The POPF instruction also may be used to change the state of the IF flag (as can the CLI and STI instructions); however, the POPF instruction in this case is also I/O sensitive. A procedure may use the POPF instruction to change the setting of the IF flag only if the CPL is less than or equal to the current IOPL. An attempt by a less privileged procedure to change the IF flag does not result in an exception; the IF flag simply remains unchanged.

20.5.2 I/O Permission Bit Map

The I/O permission bit map is a device for permitting limited access to I/O ports by less privileged programs or tasks and for tasks operating in virtual-8086 mode. The I/O permission bit map is located in the TSS (see Figure 20-2) for the currently running task or program. The address of the first byte of the I/O permission bit map is given in the I/O map base address field of the TSS. The size of the I/O permission bit map and its location in the TSS are variable.

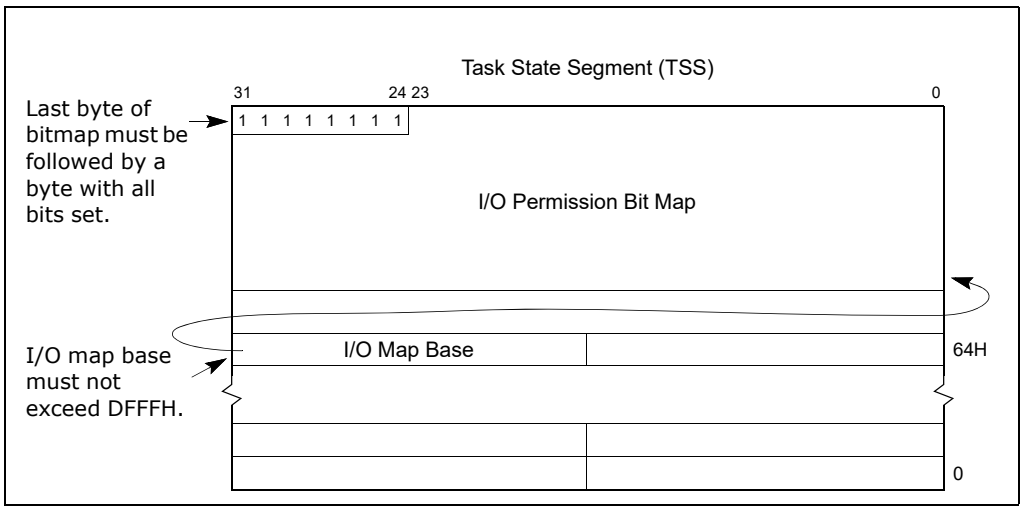


Figure 20-2. I/O Permission Bit Map

Because each task has its own TSS, each task has its own I/O permission bit map. Access to individual I/O ports can thus be granted to individual tasks.

If in protected mode and the CPL is less than or equal to the current IOPL, the processor allows all I/O operations to proceed. If the CPL is greater than the IOPL or if the processor is operating in virtual-8086 mode, the processor checks the I/O permission bit map to determine if access to a particular I/O port is allowed. Each bit in the map corresponds to an I/O port byte address. For example, the control bit for I/O port address 29H in the I/O address space is found at bit position 1 of the sixth byte in the bit map. Before granting I/O access, the processor tests all the bits corresponding to the I/O port being addressed. For a doubleword access, for example, the processors tests the four bits corresponding to the four adjacent 8-bit port addresses. If any tested bit is set, a general-protection exception (#GP) is signaled. If all tested bits are clear, the I/O operation is allowed to proceed.

Because I/O port addresses are not necessarily aligned to word and doubleword boundaries, the processor reads two bytes from the I/O permission bit map for every access to an I/O port. To prevent exceptions from being generated when the ports with the highest addresses are accessed, an extra byte needs to be included in the TSS immediately after the table. This byte must have all of its bits set, and it must be within the segment limit.

It is not necessary for the I/O permission bit map to represent all the I/O addresses. I/O addresses not spanned by the map are treated as if they had set bits in the map. For example, if the TSS segment limit is 10 bytes past the bit-map base address, the map has 11 bytes and the first 80 I/O ports are mapped. Higher addresses in the I/O address space generate exceptions.

If the I/O bit map base address is greater than or equal to the TSS segment limit, there is no I/O permission map, and all I/O instructions generate exceptions when the CPL is greater than the current IOPL.

20.6 ORDERING I/O

When controlling I/O devices it is often important that memory and I/O operations be carried out in precisely the order programmed. For example, a program may write a command to an I/O port, then read the status of the I/O device from another I/O port. It is important that the status returned be the status of the device **after** it receives the command, not **before**.

When using memory-mapped I/O, caution should be taken to avoid situations in which the programmed order is not preserved by the processor. To optimize performance, the processor allows cacheable memory reads to be reordered ahead of buffered writes in most situations. Internally, processor reads (cache hits) can be reordered around buffered writes. When using memory-mapped I/O, therefore, it is possible that an I/O read might be performed before the memory write of a previous instruction. The recommended method of enforcing program ordering of memory-mapped I/O accesses with the Pentium 4, Intel Xeon, and P6 family processors is to use the MTRRs to make the memory mapped I/O address space uncacheable; for the Pentium and Intel486 processors, either the KEN# pin or the PCD flags can be used for this purpose (see Section 20.3.1, “Memory-Mapped I/O”).

When the target of a read or write is in an uncacheable region of memory, memory reordering does not occur externally at the processor’s pins (that is, reads and writes appear in-order). Designating a memory mapped I/O region of the address space as uncacheable ensures that reads and writes of I/O devices are carried out in program order. See Chapter 14, “Memory Cache Control,” in the Intel® 64 and IA-32 Architectures Software Developer’s Manual, Volume 3A, for more information on using MTRRs.

Another method of enforcing program order is to insert one of the serializing instructions, such as the CPUID instruction, between operations. See Chapter 11, “Multiple-Processor Management,” in the Intel® 64 and IA-32 Architectures Software Developer’s Manual, Volume 3A, for more information on serialization of instructions.

It should be noted that the chipset being used to support the processor (bus controller, memory controller, and/or I/O controller) may post writes to uncacheable memory which can lead to out-of-order execution of memory accesses. In situations where out-of-order processing of memory accesses by the chipset can potentially cause faulty memory-mapped I/O processing, code must be written to force synchronization and ordering of I/O operations. Serializing instructions can often be used for this purpose.

When the I/O address space is used instead of memory-mapped I/O, the situation is different in two respects:

- The processor never buffers I/O writes. Therefore, strict ordering of I/O operations is enforced by the processor. (As with memory-mapped I/O, it is possible for a chipset to post writes in certain I/O ranges.)
- The processor synchronizes I/O instruction execution with external bus activity (see Table 20-1).

Table 20-1. I/O Instruction Serialization

Instruction Being Executed	Processor Delays Execution of …		Until Completion of …	
	Current Instruction?	Next Instruction?	Pending Stores?	Current Store?
IN	Yes		Yes	
INS	Yes		Yes	
REP INS	Yes		Yes	
OUT		Yes	Yes	Yes
OUTS		Yes	Yes	Yes
REP OUTS		Yes	Yes	Yes

CHAPTER 21
PROCESSOR IDENTIFICATION AND FEATURE DETERMINATION

When writing software intended to run on Intel processors, it is necessary to identify the type of processor present in a system and the processor features that are available to an application.

The CPUID instruction, known as CPU Identification, was introduced with the Intel® Pentium processor to query the processor's information name space for its identity and supported features. Logically, the CPUID name space comprises a series of nodes indexed by leaf (using the input value of EAX) and in some cases further indexed by sub-leaf (using the input value of ECX). The value of a queried node is returned in EAX, EBX, ECX, and EDX. Note that not all leaves have sub-leaf indexing and the input ECX value will be ignored in those cases. The full description of CPUID can be found in Chapter 3 of the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 2A.

All references to "MAX_LEAF" throughout this chapter are used as an abbreviation of "CPUID.00H:EAX.MAX_LEAF".

21.1 IMPORTANT CONSIDERATIONS WHEN USING THE CPUID INSTRUCTION

This section outlines additional factors to consider when using the CPUID instruction.

21.1.1 Guidelines for Using the CPUID Instruction

Use the CPUID instruction for processor identification in the Pentium M processor family, Pentium 4 processor family, Intel Xeon processor family, P6 family, Pentium processor, and later Intel486 processors. This instruction returns the family, model, and (for some processors) a brand string for the processor that executes the instruction. It also indicates the features that are present in the processor and gives information about the processor's caches and TLB.

The ID flag (bit 21) in the EFLAGS register indicates support for the CPUID instruction. If a software procedure can set and clear this flag, the processor executing the procedure supports the CPUID instruction. The CPUID instruction will cause the invalid opcode exception (#UD) if executed on a processor that does not support it.

To obtain processor identification information, a source operand value is placed in the EAX register to select the type of information to be returned. When the CPUID instruction is executed, selected information is returned in the EAX, EBX, ECX, and EDX registers.

The following guidelines are among the most important and should always be followed when using the CPUID instruction to determine available features:

- Always begin by testing for the "GenuineIntel," message in the EBX, EDX, and ECX registers when the CPUID instruction is executed with EAX equal to 0. If the processor is not genuine Intel, the feature identification flags may have different meanings than are described in Intel documentation.
- Test feature identification flags individually and do not make assumptions about undefined bits.

21.1.2 Identification of Earlier Processors

The CPUID instruction is not available in earlier Intel processors up through the earlier Intel 486 processors. For these processors, several other architectural features can be exploited to identify the processor.

The settings of bits 12 and 13 (IOPL), 14 (NT), and 15 (reserved) in the EFLAGS register are different for Intel's 32-bit processors than for the Intel 8086 and Intel 286 processors. By examining the settings of these bits (with the PUSHF/PUSHFD and POPF/POPFD instructions), an application program can determine whether the processor is an 8086, Intel 286, or one of the Intel 32-bit processors:

- 8086 processor — Bits 12 through 15 of the EFLAGS register are always set.
- Intel 286 processor — Bits 12 through 15 are always clear in real-address mode.

- 32-bit processors — In real-address mode, bit 15 is always clear and bits 12 through 14 have the last value loaded into them. In protected mode, bit 15 is always clear, bit 14 has the last value loaded into it, and the IOPL bits depend on the current privilege level (CPL). The IOPL field can be changed only if the CPL is 0.

Other EFLAGS register bits that can be used to differentiate between the 32-bit processors:

- Bit 18 (AC) — Implemented only on the Pentium 4, Intel Xeon, P6 family, Pentium, and Intel486 processors. The inability to set or clear this bit distinguishes an Intel386 processor from the later IA-32 processors.
- Bit 21 (ID) — Determines if the processor is able to execute the CPUID instruction. The ability to set and clear this bit indicates that it is a Pentium 4, Intel Xeon, P6 family, Pentium, or later-version Intel486 processor.

To determine whether an x87 FPU or Numeric Processor Extension (NPX) is present in a system, applications can write to the x87 FPU status and control registers using the FNINIT instruction and then verify that the correct values are read back using the FNSTENV instruction.

After determining that an x87 FPU or NPX is present, its type can then be determined. In most cases, the processor type will determine the type of FPU or NPX; however, an Intel386 processor is compatible with either an Intel 287 or Intel 387 math coprocessor.

The method the coprocessor uses to represent ∞ (after the execution of the FINIT, FNINIT, or RESET instruction) indicates which coprocessor is present. The Intel 287 math coprocessor uses the same bit representation for +∞ and –∞; whereas, the Intel 387 math coprocessor uses different representations for +∞ and –∞.

21.1.3 CPUID Basic and Extended Range

The CPUID basic range starts at CPUID.00H and ends at the maximum leaf enumerated in CPUID.00H:EAX.MAX_LEAF[31:0].

The legacy set of CPUID leaves are defined as leaves 00H, 01H, and 02H, which represent the architecture up to and including Pentium II. Processors provided legacy compatibility by limiting the exposed number of leaves to just these legacy leaves by setting IA32_MISC_ENABLE[22] (Limit CPUID Maxval). This is no longer supported on processors that report CPUID.07H.01H:EBX.CPUIDMAXVAL_LIM_RMV[3] as 1; for such processors, IA32_MISC_ENABLE[22] cannot be set to 1 to limit the value returned by CPUID.00H:EAX.MAX_LEAF.

The extended CPUID range starts at leaf 80000000H and ends at the maximum leaf enumerated in CPUID.80000000H:EAX.MAX_EXTENDED_LEAF[31:0].

Older processors before the Pentium 4 do not support the extended CPUID range and treat bit 31 of CPUID's input EAX value as zero.

If a value entered for CPUID.EAX is higher than the maximum input value for basic or extended function for that processor then the data for the highest basic information leaf is returned. Software should not rely on the values returned by the processor outside of the above ranges.

The range CPUID.40000000H to CPUID.4FFFFFFFH do not return feature information for the processor. These are allocated for emulation by software.

21.1.4 CPUID Domains

The fields of each CPUID node are classified into one of several CPUID domains. The fields may be classified separately within a specific node or in aggregate for all the nodes in a leaf or sub-leaf. On a properly configured platform, all logical processors within a CPUID domain return a consistent output value for fields belonging to that domain. As an example, the initial X2APIC ID value returned in CPUID.1FH.00H:EDX[31:0] is classified as being in the Logical Processor Domain because the value is unique for each logical processor in the platform. Whereas, the CLFLUSH Line Size returned in CPUID.00H:EBX[15:8] is classified as Platform Domain because it must be consistent for all logical processor within the entire platform.

- **Platform Domain**—A properly configured platform would provide consistent values for these CPUID fields for each logical processor in the platform.
- **Package Domain**—A properly configured platform provides consistent values for these CPUID fields for each logical processor within the same processor package. These values however can be different when comparing the values of logical processors on different packages.

- **Logical Processor Domain**—A properly configured platform can provide different values for these CPUID fields for each logical processor in the platform. The values contained within these may have their own scope as per a specific shared resource (i.e., cache, hybrid, etc.); in that case, each logical processor may need to be queried to obtain the full platform view of given features.

21.1.5 CPUID Runtime Mutable Fields

A CPUID field is said to be mutable if it can change during runtime. Such fields are affected by supervisor-mode operations that can affect processor mode, status bits, or privileged registers. Mutable fields that are expected to change dynamically as part of normal operation are shown in the table Table 21-1. Mutable fields that should remain consistent are shown in the table Table 21-2. Note all of the listed controls may not be available on all Intel processors.

Table 21-1. Runtime Mutable CPUID Fields Expected to Change During Normal Operation

Leaf	Sub-Leaf	Register	Field Name	Description and Mutability Control
01H	Ignored	ECX[27]	OSXSAVE	If 1, the OS has set CR4.OSXSAVE[bit 18] to enable XSETBV/XGETBV instructions to access XCR0 and to support processor extended state management using XSAVE/XRSTOR.
07H	00H	ECX[4]	OSPKE	If 1, OS has set CR4.PKE to enable protection keys (and the RDPKRU/WRPKRU instructions).
0DH	00H	EBX[31:0]	XSAVE_BYTES_ ENABLED_FEATURE	The size of the XSAVE/XRSTOR area required for the state bits enabled in XCR0.
0DH	01H	EBX[31:0]	XSAVE_BYTES_ ENABLED_FEATURE	The size of the XSAVES/XRSTORS area required for the state bits enabled in XCR0 and IA32_XSS.
19H	00H	EBX[0]	AESKLE	If 1, if the AES Key Locker instructions have been activated by system firmware and the OS has set CR4.KL[bit 19] = 1.
80000001H	Ignored	EDX[20]	SYSCALL_SYSRET_64	Intel processors support SYSCALL and SYSRET only in 64-bit mode. This feature flag is always enumerated as 0 outside 64-bit mode.

Table 21-2. Runtime Mutable CPUID Fields That Should Remain Consistent

Leaf	Sub-Leaf	Register	Field Name	Description and Mutability Control
00H	Ignored	EAX[31:0]	MAX_LEAF	Support for legacy software by limiting CPUID number of leaves reporting to a maximum of 2. This is set by using IA32_MISC_ENABLE[22] Limit CPUID Maxval.
01H	Ignored	ECX[3]	MONITOR	This feature flag reflects the setting in IA32_MISC_ENABLE[18] Enable Monitor FSM.
01H	Ignored	ECX[7]	EIST	This feature flag reflects the setting in IA32_MISC_ENABLE[16] Enhanced Intel SpeedStep Technology Enable.
01H	Ignored	EDX[9]	APIC	This feature flag reflects IA32_APIC_BASE[11], APIC Global Enable.
05H	Ignored	ECX[0]	MONITOR_MWAIT_ EXTENSIONS	This field not available when CPUID.01H:ECX.MONITOR[3] = 0.

Table 21-2. Runtime Mutable CPUID Fields That Should Remain Consistent (Contd.)

Leaf	Sub-Leaf	Register	Field Name	Description and Mutability Control
05H	Ignored	ECX[1]	INTERRUPT_AS_BREAK_ EVENT	This field not available when CPUID.01H:ECX.MONITOR[3] = 0.
05H	Ignored	EDX[15:0]	LARGEST_MONITOR_ LINE_SIZE	This field not available when CPUID.01H:ECX.MONITOR[3] = 0.
05H	Ignored	EAX[15:0]	SMALLEST_MONITOR_ LINE_SIZE	This field not available when CPUID.01H:ECX.MONITOR[3] = 0.

21.1.6 CPUID Reserved Fields

Software must ignore and not rely upon the values returned by reserved fields of a CPUID leaf or sub-leaf because they may have meaning on future processors. Once a previously-reserved field becomes defined, this specification will be updated to reflect that.

21.1.7 CPUID Instruction for Serialization

Although the CPUID instruction provides serialization, it is not the preferred method on newer processors that support the SERIALIZE instruction, which is enumerated via CPUID.07H.00H:EDX[14]=1. If backward compatibility is required with older processors, use leaf 00H [CPUID.00H] for serialization because it has the lowest latency when executed. See "Serializing Instructions" in Chapter 11 of the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3A for more details.

21.1.8 IA32_BIOS_SIGN_ID Returns Microcode Update Signature

For processors that support the microcode update facility, the IA32_BIOS_SIGN_ID MSR is loaded with the update signature whenever CPUID executes. The signature is returned in the upper DWORD. For details, see Chapter 11 in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3A.

21.2 METHODS FOR RETURNING BRANDING INFORMATION USING CPUID

Use the following techniques to access branding information:

1. Processor brand string method.
2. Processor brand index; this method uses a software supplied brand string table.

These two methods are discussed in the following sections. For methods that are available in early processors, see Section 21.1.2, "Identification of Earlier Processors," of the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 1.

21.2.1 The Processor Brand String Method

Figure 21-1 describes the algorithm used for detection of the brand string. Processor brand identification software should execute this algorithm on all Intel 64 and IA-32 processors.

This method (introduced with Pentium 4 processors) returns an ASCII brand identification string and the Processor Base frequency of the processor to the EAX, EBX, ECX, and EDX registers.

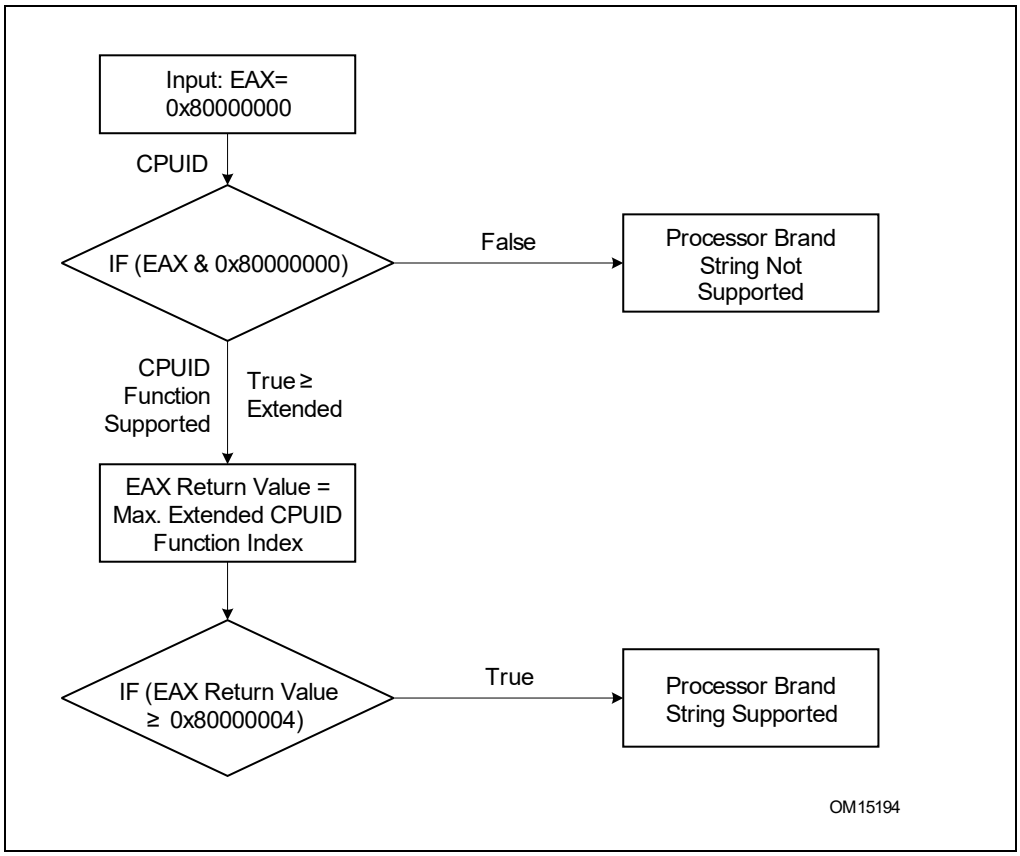


Figure 21-1. Determination of Support for the Processor Brand String

21.2.2 The Processor Brand Index Method

The brand index method (introduced with Pentium® III Xeon® processors) provides an entry point into a brand identification table that is maintained in memory by software. In this table, each brand index is associated with an ASCII brand identification string that identifies the official Intel family and model number of a processor.

When CPUID executes with EAX set to 1, the processor returns a brand index to the low byte in EBX. Software can then use this index to locate the brand identification string for the processor in the brand identification table. The first entry (brand index 0) in this table is reserved, allowing for backward compatibility with processors that do not support the brand identification feature. Starting with processor signature family ID = 0FH, model = 03H, brand index method is no longer supported. Use brand string method instead.

Table 21-3 shows brand indices that have identification strings associated with them.

Table 21-3. Mapping of Brand Indices; and Intel 64 and IA-32 Processor Brand Strings

Brand Index	Brand String
00H	This processor does not support the brand identification feature
01H	Intel® Celeron® processor[1]
02H	Intel® Pentium® III processor[1]
03H	Intel® Pentium® III Xeon® processor; If processor signature = 000006B1h, then Intel® Celeron® processor
04H	Intel® Pentium® III processor
06H	Mobile Intel® Pentium® III processor-M
07H	Mobile Intel® Celeron® processor[1]

Table 21-3. Mapping of Brand Indices; and Intel 64 and IA-32 Processor Brand Strings

08H	Intel® Pentium® 4 processor
09H	Intel® Pentium® 4 processor
0AH	Intel® Celeron® processor[1]
0BH	Intel® Xeon® processor; If processor signature = 00000F13h, then Intel® Xeon® processor MP
0CH	Intel® Xeon® processor MP
0EH	Mobile Intel® Pentium® 4 processor-M; If processor signature = 00000F13h, then Intel® Xeon® processor
0FH	Mobile Intel® Celeron® processor[1]
11H	Mobile Genuine Intel® processor
12H	Intel® Celeron® M processor
13H	Mobile Intel® Celeron® processor[1]
14H	Intel® Celeron® processor
15H	Mobile Genuine Intel® processor
16H	Intel® Pentium® M processor
17H	Mobile Intel® Celeron® processor[1]
18H – 0FFH	RESERVED

NOTES:

1. Indicates versions of these processors that were introduced after the Pentium III.

21.3 CPUID LEAVES

The remainder of this chapter provides CPUID enumeration information for Intel® 64 and IA-32 architectures.

CPUID.00H -- Maximum Input for Basic CPUID and Vendor ID

CPUID.00H returns the highest value the CPUID recognizes for returning basic processor information. The value is returned in the EAX register and is processor specific.

- This leaf is always valid.
- This leaf does not contain sub-leaves and provides the same information regardless of the value of ECX.

Table 21-4. Leaf 00H Maximum Input for Basic CPUID and Vendor ID

Register	Field Name	Description	Domain
EAX[31:0]	MAX_LEAF	Maximum input value for basic CPUID Information.	Platform
EBX[31:0]	VENDOR_ID_1	"Genu"	Platform
ECX[31:0]	VENDOR_ID_2	"ntel"	Platform
EDX[31:0]	VENDOR_ID_3	"inel"	Platform

A vendor identification string is also returned in EBX, EDX, and ECX. For Intel processors, the string is "Genu-ineIntel" and is expressed:

EBX := 756e6547h (* "Genu", with G in the low eight bits of BL *)
EDX := 49656e69h (* "ineI", with i in the low eight bits of DL *)
ECX := 6c65746eh (* "ntel", with n in the low eight bits of CL *)

CPUID.01H -- Version and Features

CPUID.01H returns type, family, model, stepping, and feature information.
- This leaf is valid if MAX_LEAF ≥ 01H.
- This leaf does not contain sub-leaves and provides the same information regardless of the value of ECX.

Table 21-5. Leaf 01H Output Registers

CPUID Output Registers	Description
EAX[31:0]	Version information: Type, Family, Model, and Stepping ID (see "CPUID.01H:EAX—Version Information: Type, Family, Model and Stepping ID").
EBX[31:0]	Feature information (see "CPUID.01H:EBX—Feature Information").
ECX[31:0]	Feature information (see "CPUID.01H:ECX—Feature Information").
EDX[31:0]	Feature information (see "CPUID.01H:EDX—Feature Information").

CPUID.01H:EAX Version Information: Type, Family, Model and Stepping ID

The EAX register of CPUID.01H returns the information shown below.

Table 21-6. Leaf 01H Version and Features Returned in EAX

Register	Field Name	Description	Domain
EAX[3:0]	STEPPING_ID	Identifies a revision of the specific processor family and model. The stepping information is specified as a per-package basis for legacy processors. More recent processors do not allow mixing steppings.	Package
EAX[7:4]	MODEL_ID	Identifies a set of processors within a family. Certain models of Pentium® 4 processors allowed mixed Model IDs and would have this identified as a Package Domain.	Platform
EAX[11:8]	FAMILY_ID	Identifies a set of processors that have a general architectural similarity.	Platform
EAX[13:12]	PROCESSOR_TYPE	Identifies specific type of processor.	Platform
EAX[15:14]	Reserved	Reserved.	
EAX[19:16]	EXTENDED_MODEL_ID	When the Family ID is 06H or 0FH, this field is prepended to the Model ID to provide an 8-bit model identification.	Platform
EAX[27:20]	EXTENDED_FAMILY_ID	When the Family ID is 0FH, this field is added to the Family ID to provide an 8-bit family identification.	Platform
EAX[31:28]	Reserved	Reserved.	

CPUID.01H returns version information in EAX. For example: model, family, and processor type for the Intel Xeon processor 5100 series is as follows:
- Model — 1111B
- Family — 0101B
- Processor Type — 00B

See table below for available processor type values. Stepping IDs are provided as needed.

Table 21-7. Processor Type Field

Type	Encoding
Original OEM Processor	00B
Intel OverDrive® Processor	01B
Dual processor (not applicable to Intel486 processors)	10B
Intel reserved	11B

NOTE

See Section 21.1.2, "Identification of Earlier Processors," for information on identifying earlier IA-32 processors. The Extended Family ID needs to be examined only when the Family ID is 0FH. Integrate the fields into a display using the following rule:

```
IF Family_ID ≠ 0FH THEN
    DisplayFamily = Family_ID;
ELSE
    DisplayFamily = Extended_Family_ID + Family_ID;
FI;
```

The Extended Model ID needs to be examined only when the Family ID is 06H or 0FH. Integrate the field into a display using the following rule:

```
IF (Family_ID = 06H or Family_ID = 0FH) THEN
    DisplayModel = (Extended_Model_ID « 4) + Model_ID;
ELSE
    DisplayModel = Model_ID;
FI;
```

CPUID.01H:EBX Feature information.

The EBX register of CPUID.01H returns the information shown below.

Table 21-8. Leaf 01H Version and Features Returned in EBX

Register	Field Name	Description	Domain
EBX[7:0]	BRAND_INDEX	This number provides an entry into a brand string table that contains brand strings for IA-32 processors. More information about this field is provided in Section 21.2.2, "The Processor Brand Index Method."	Platform
EBX[15:8]	CLFLUSH_LINE_SIZE	Value * 8 = cache line size in bytes. This number indicates the size of the cache line flushed by the CLFLUSH and CLFLUSHOPT instructions in 8-byte increments. This field was introduced in the Pentium 4 processor.	Platform
EBX[23:16]	APIC_ID_SPACE	Maximum number of addressable IDs for logical processors in this physical package. The nearest power-of-2 integer that is not smaller than EBX[23:16] is the number of unique initial APIC IDs reserved for addressing different logical processors in a physical package. This field is only valid if CPUID.01H.EDX.HTT[28]= 1. See further details below on the usage of this field.	Platform

EBX[31:24]	INITIAL_APIC_ID	This number is the 8-bit ID that is assigned to the local APIC on the processor during power up. This field was introduced in the Penium 4 processor. The 8-bit initial APIC ID in EBX[31:24] is replaced by the 32-bit x2APIC ID, available in Leaf 0BH and Leaf 1FH.	Logical Processor

The Maximum Addressable IDs for logical processors in this package should not be used on platforms that support CPUID leaf 0BH or CPUID leaf 1FH as it can be saturated and incorrect. Modern platforms can have many more processors than can be enumerated or have topology domains with discontinuous APIC ID reservations. To correctly enumerate APIC ID information on modern platforms, use CPUID.0BH or CPUID.1FH.

CPUID.01H:ECX Feature Information

The ECX register of CPUID.01H returns the information shown below. For all feature flags, a 1 indicates that the feature is supported. Software should identify Intel as the vendor to properly interpret feature flags. Software must confirm that a processor feature is present using feature flags returned by CPUID prior to using the feature. Software should not depend on future offerings retaining all features.

Table 21-9. Leaf 01H Version and Features Returned in ECX

Register	Field Name	Description	Domain
ECX[0]	SSE3	If 1, supports Streaming SIMD Extensions 3.	Platform
ECX[1]	PCLMULQDQ	If 1, supports the PCLMULQDQ instruction.	Platform
ECX[2]	DTES64	64-bit DS Area. If 1, supports DS area using 64-bit layout.	Platform
ECX[3]	MONITOR	If 1, supports the MONITOR/MWAIT and CPUID.05H.	Platform
ECX[4]	DS_CPL	If 1, supports the extensions to the Debug Store feature to allow for branch message storage qualified by CPL.	Platform
ECX[5]	VMX	If 1, supports the Virtual Machine Extensions.	Platform
ECX[6]	SMX	If 1, supports Safer Mode Extensions. See Chapter 7, "Safer Mode Extensions Reference."	Platform
ECX[7]	EIST	If 1, supports Enhanced Intel SpeedStep® technology.	Platform
ECX[8]	TM2	If 1, supports Thermal Monitor 2.	Platform
ECX[9]	SSSE3	If 1, supports Supplemental Streaming SIMD Extensions 3.	Platform
ECX[10]	L1_CONTEXT_ID	If 1, the L1 data cache mode can be set to either adaptive mode or shared mode. See definition of the IA32_MISC_ENABLE MSR Bit 24 (L1 Data Cache Context Mode) for details.	Platform
ECX[11]	DEBUG_INTERFACE	If 1, supports IA32_DEBUG_INTERFACE MSR for silicon debug.	Platform
ECX[12]	FMA	If 1, supports FMA extensions using YMM state.	Platform

ECX[13]	CMPXCHG16B	If 1, supports this instruction. See the "CMPXCHG8B/CMPXCHG16B—Compare and Exchange Bytes" section in this chapter for a description.	Platform
ECX[14]	XTPR_UPDATE_CONTROL	If 1, supports changing IA32_MISC_ENABLE[bit 23].	Platform
ECX[15]	PERF_CAPABILITIES	If 1, supports the performance and debug feature indication MSR IA32_PERF_CAPABILITIES.	Platform
ECX[16]	Reserved	Reserved.	
ECX[17]	PCID	If 1, supports Process-context identifiers and software setting CR4.PCIDE to 1.Process-context identifiers.	Platform
ECX[18]	DCA	If 1, supports the ability to prefetch data from a memory mapped device. See CPUID.09H.	Platform
ECX[19]	SSE4_1	If 1, supports SSE4.1.	Platform
ECX[20]	SSE4_2	If 1, supports SSE4.2.	Platform
ECX[21]	X2APIC	If 1, supports x2APIC feature.	Platform
ECX[22]	MOVBE	If 1, supports MOVBE instruction.	Platform
ECX[23]	POPCNT	If 1, supports the POPCNT instruction.	Platform
ECX[24]	TSC_DEADLINE	If 1, the processor's local APIC timer supports one-shot operation using a TSC deadline value.	Platform
ECX[25]	AESNI	If 1, supports the AESNI instruction extensions.	Platform
ECX[26]	XSAVE	If 1, supports the XSAVE/XRSTOR processor extended states feature, the XSETBV/XGETBV instructions, and XCR0.	Platform
ECX[27]	OSXSAVE	If 1, the OS has set CR4.OSXSAVE[bit 18] to enable XSETBV/XGETBV instructions to access XCR0 and to support processor extended state management using XSAVE/XRSTOR.	Logical Processor
ECX[28]	AVX	If 1, supports the AVX instruction extensions.	Platform
ECX[29]	F16C	If 1, supports 16-bit floating-point conversion instructions.	Platform
ECX[30]	RDRAND	If 1, supports RDRAND instruction.	Platform
ECX[31]	Not Used	Intel processors always return 0. Allocated for use by software emulation.	Platform

CPUID.01H:EDX Feature Information

The EDX register of CPUID.01H returns the information shown below. For all feature flags, a 1 indicates that the feature is supported. Software should identify Intel as the vendor to properly interpret feature flags. Software must confirm that a processor feature is present using feature flags returned by CPUID prior to using the feature. Software should not depend on future offerings retaining all features.

Table 21-10. Leaf 01H Version and Features Returned in EDX

Register	Field Name	Description	Domain
EDX[0]	FPU	Floating Point Unit On-Chip. The processor contains an x87 FPU.	Platform

EDX[1]	VME	If 1, supports Virtual 8086 mode enhancements, including CR4.VME for controlling the feature; CR4.PVI for protected mode virtual interrupts; software interrupt indirection; expansion of the TSS with the software indirection bitmap; and EFLAGS.VIF and EFLAGS.VIP flags.	Platform
EDX[2]	DE	If 1, supports I/O breakpoints debugging extensions, including CR4.DE for controlling the feature, and optional trapping of accesses to DR4 and DR5.	Platform
EDX[3]	PSE	P If 1, supports page size extensions for large pages of size 4 MByte, including: CR4.PSE for controlling the feature; the defined dirty bit in PDE (Page Directory Entries); optional reserved bit trapping in CR3; PDEs; and PTEs.	Platform
EDX[4]	TSC	If 1, supports the Time Stamp Counter, RDTSC instruction, including CR4.TSD for controlling privilege.	Platform
EDX[5]	MSR	If 1, supports the Model Specific Registers RDMSR and WRMSR Instructions. Some of the MSRs are implementation dependent.	Platform
EDX[6]	PAE	If 1, supports the Physical Address Extension which is for physical addresses greater than 32 bits, including: extended page table entry formats; an extra level in the page translation tables; and 2-MByte pages rather than 4 Mbyte pages.	Platform
EDX[7]	MCE	If 1, supports exception 18 for Machine Checks, including CR4.MCE for controlling the feature. This feature does not define the modelspecific implementations of machine-check error logging, reporting, and processor shutdowns. Machine Check exception handlers may have to depend on processor version to do model specific processing of the exception, or test for the presence of the Machine Check feature.	Platform
EDX[8]	CMPXCHG8B	If 1, supports the CMPXCHG8B (64 bits) Instruction, implicitly locked and atomic.	Platform
EDX[9]	APIC	If 1, the processor contains an Advanced Programmable Interrupt Controller (APIC), responding to memory mapped commands in the physical address range FEE00000H to FEE00FFFH (by default - some processors permit the APIC to be relocated).	Platform
EDX[10]	Reserved	Reserved.	
EDX[11]	SEP	If 1 supports the SYSENTER and SYSEXIT Instructions and associated MSRs.	Platform

EDX[12]	MTRR	If 1, supports the Memory Type Range Registers. (The MTRRcap MSR contains feature bits that describe what memory types are supported, how many variable MTRRs are supported, and whether fixed MTRRs are supported.)	Platform
EDX[13]	PGE	If 1, supports the global bit in paging-structure entries that map a page, indicating TLB entries that are common to different processes and need not be flushed. The CR4.PGE bit controls this feature.	Platform
EDX[14]	MCA	If 1, supports the Machine Check Architecture feature. The MCG_CAP MSR contains feature bits describing how many banks of error reporting MSRs are supported.	Platform
EDX[15]	CMOV	If 1, supports the Conditional Move Instructions. If CPUID.01H:EDX.FPU[0] (x87 FPU present) is 1 also, supports the FCOMI and FCMOV instructions.	Platform
EDX[16]	PAT	If 1, supports the Page Attribute Table feature. (This feature augments the Memory Type Range Registers (MTRRs), allowing an operating system to specify attributes of memory accessed through a linear address on a 4KB granularity.)	Platform
EDX[17]	PSE_36	If 1, supports the 36-Bit Page Size Extension which enables 4-MByte pages addressing physical memory beyond 4 GBytes with 32-bit paging. This feature indicates that upper bits of the physical address of a 4-MByte page are encoded in bits 20:13 of the page directory entry. Such physical addresses are limited by MAXPHYADDR and may be up to 40 bits in size.	Platform
EDX[18]	PSN	If 1, supports the 96-bit Processor Serial Number identification number feature, and the feature is enabled. Available only in Pentium III, see CPUID.03H.	Platform
EDX[19]	CLFLUSH	If 1, supports the CLFLUSH instruction.	Platform
EDX[20]	Reserved	Reserved.	
EDX[21]	DS	If 1, supports the Debug Store feature which provides the ability to write debug information into a memory resident buffer. This feature is used by the branch trace store (BTS) and processor event-based sampling (PEBS) facilities (see Chapter 19, "Debug, Branch Profile, TSC, and Intel® Resource Director Technology (Intel® RDT) Features," in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3B).	Platform

EDX[22]	ACPI	If 1, supports the Thermal Monitor and Software Controlled Clock Facilities. These are internal MSRs that allow processor temperature to be monitored and processor performance to be modulated in predefined duty cycles under software control.	Platform
EDX[23]	MMX	If 1, supports the Intel MMX Technology.	Platform
EDX[24]	FXSR	If 1, supports the FXSAVE and FXRSTOR Instructions, which are fast save and restore of the floating-point context, and the availability of CR4.OSFXSR for an operating system to indicate support of same.	Platform
EDX[25]	SSE	If 1, supports SSE.	Platform
EDX[26]	SSE2	If 1, supports SSE2.	Platform
EDX[27]	SELF_SNOOP	If 1, supports Self Snoop which is the management of conflicting memory types by performing a snoop of its own cache structure for transactions issued to the bus.	Platform
EDX[28]	HTT	If 1, the value in CPUID.1.EBX[23:16] (the Maximum number of addressable IDs for logical processors in this package) is valid for the package. If 0, there is only a single logical processor in the package and software should assume only a single APIC ID is reserved.	Platform
EDX[29]	TM	If 1, supports the Thermal Monitor feature in which the processor implements the thermal monitor automatic thermal control circuitry (TCC).Thermal Monitor.	Platform
EDX[30]	Reserved	Reserved.	
EDX[31]	PBE	If 1, supports the Pending Break Enable feature, which is the use of the FERR#/PBE# pin when the processor is in the stop-clock state (STPCLK# is asserted) to signal the processor that an interrupt is pending and that the processor should return to normal operation to handle the interrupt.	Platform

CPUID.02H -- TLB/Cache/Prefetch Information

CPUID.02H returns TLB, cache, and prefetch information.This leaf has been superseded by CPUID.04H for cache enumeration and CPUID.18H for TLB enumeration. These processors will also report new descriptor values of types 0FEh or 0FFh to refer enumerations to CPUID.04H and CPUID.18H.

- This leaf is valid if MAX_LEAF ≥ 02H.
- This leaf does not contain sub-leaves and provides the same information regardless of the value of ECX.

Table 21-11. Leaf 02H TLB/Cache/Prefetch Information

Register	Field Name	Description	Domain
EAX[7:0]	Reserved	Reserved with a value of 1	
EAX[15:8]	DESCRIPTOR_1	See Table "Encoding of CPUID Leaf 2 Descriptors" below this table.	Logical Processor
EAX[23:16]	DESCRIPTOR_2	See Table "Encoding of CPUID Leaf 2 Descriptors" below this table.	Logical Processor
EAX[31:24]	DESCRIPTOR_3	See Table "Encoding of CPUID Leaf 2 Descriptors" below this table.	Logical Processor
EBX[7:0]	DESCRIPTOR_4	See Table "Encoding of CPUID Leaf 2 Descriptors" below this table.	Logical Processor
EBX[15:8]	DESCRIPTOR_5	See Table "Encoding of CPUID Leaf 2 Descriptors" below this table.	Logical Processor
EBX[23:16]	DESCRIPTOR_6	See Table "Encoding of CPUID Leaf 2 Descriptors" below this table.	Logical Processor
EBX[31:24]	DESCRIPTOR_7	See Table "Encoding of CPUID Leaf 2 Descriptors" below this table.	Logical Processor
ECX[7:0]	DESCRIPTOR_8	See Table "Encoding of CPUID Leaf 2 Descriptors" below this table.	Logical Processor
ECX[15:8]	DESCRIPTOR_9	See Table "Encoding of CPUID Leaf 2 Descriptors" below this table.	Logical Processor
ECX[23:16]	DESCRIPTOR_10	See Table "Encoding of CPUID Leaf 2 Descriptors" below this table.	Logical Processor
ECX[31:24]	DESCRIPTOR_11	See Table "Encoding of CPUID Leaf 2 Descriptors" below this table.	Logical Processor
EDX[7:0]	DESCRIPTOR_12	See Table "Encoding of CPUID Leaf 2 Descriptors" below this table.	Logical Processor
EDX[15:8]	DESCRIPTOR_13	See Table "Encoding of CPUID Leaf 2 Descriptors" below this table.	Logical Processor
EDX[23:16]	DESCRIPTOR_14	See Table "Encoding of CPUID Leaf 2 Descriptors" below this table.	Logical Processor
EDX[31:24]	DESCRIPTOR_15	See Table "Encoding of CPUID Leaf 2 Descriptors" below this table.	Logical Processor

CPUID.02H returns information about the processor's internal TLBs, cache, and prefetch hardware in the EAX, EBX, ECX, and EDX registers. The information is reported in encoded form and fall into the following categories:

- The least-significant byte in register EAX (register AL) will always return 01H. Software should ignore this value and not interpret it as an informational descriptor.
- The most significant bit (bit 31) of each register indicates whether the register contains valid information (set to 0) or is reserved (set to 1).
- If a register contains valid information, the information is contained in 1-byte descriptors. There are four types of encoding values for the byte descriptor, the encoding of these descriptors and encoding type are listed in the table

below.
Note that the order of descriptors in the EAX, EBX, ECX, and EDX registers is not defined; that is, specific bytes are not designated to contain descriptors for specific cache, prefetch, or TLB types. The descriptors may appear in any order. Note also a processor may report a general descriptor type FFH and FEH and not report any byte descriptor of "cache type" or "*TLB type" via CPUID.02H.

Table 21-12. Encoding of CPUID Leaf 2 Descriptors

Descriptor Value	Type	Cache or TLB Description
00H	General	Null descriptor, this byte contains no information.
01H	TLB	Instruction TLB: 4 KByte pages, 4-way set associative, 32 entries.
02H	TLB	Instruction TLB: 4 MByte pages, fully associative, 2 entries.
03H	TLB	Data TLB: 4 KByte pages, 4-way set associative, 64 entries.
04H	TLB	Data TLB: 4 MByte pages, 4-way set associative, 8 entries.
05H	TLB	Data TLB1: 4 MByte pages, 4-way set associative, 32 entries.
06H	Cache	1st-level instruction cache: 8 KBytes, 4-way set associative, 32 byte line size.
08H	Cache	1st-level instruction cache: 16 KBytes, 4-way set associative, 32 byte line size.
09H	Cache	1st-level instruction cache: 32KBytes, 4-way set associative, 64 byte line size.
0AH	Cache	1st-level data cache: 8 KBytes, 2-way set associative, 32 byte line size.
0BH	TLB	Instruction TLB: 4 MByte pages, 4-way set associative, 4 entries.
0CH	Cache	1st-level data cache: 16 KBytes, 4-way set associative, 32 byte line size.
0DH	Cache	1st-level data cache: 16 KBytes, 4-way set associative, 64 byte line size.
0EH	Cache	1st-level data cache: 24 KBytes, 6-way set associative, 64 byte line size.
1DH	Cache	2nd-level cache: 128 KBytes, 2-way set associative, 64 byte line size.
21H	Cache	2nd-level cache: 256 KBytes, 8-way set associative, 64 byte line size.
22H	Cache	3rd-level cache: 512 KBytes, 4-way set associative, 64 byte line size, 2 lines per sector.
23H	Cache	3rd-level cache: 1 MBytes, 8-way set associative, 64 byte line size, 2 lines per sector.
24H	Cache	2nd-level cache: 1 MBytes, 16-way set associative, 64 byte line size.
25H	Cache	3rd-level cache: 2 MBytes, 8-way set associative, 64 byte line size, 2 lines per sector.
29H	Cache	3rd-level cache: 4 MBytes, 8-way set associative, 64 byte line size, 2 lines per sector.
2CH	Cache	1st-level data cache: 32 KBytes, 8-way set associative, 64 byte line size.
30H	Cache	1st-level instruction cache: 32 KBytes, 8-way set associative, 64 byte line size.
40H	Cache	No 2nd-level cache or, if processor contains a valid 2nd-level cache, no 3rd-level cache.
41H	Cache	2nd-level cache: 128 KBytes, 4-way set associative, 32 byte line size.
42H	Cache	2nd-level cache: 256 KBytes, 4-way set associative, 32 byte line size.
43H	Cache	2nd-level cache: 512 KBytes, 4-way set associative, 32 byte line size.
44H	Cache	2nd-level cache: 1 MByte, 4-way set associative, 32 byte line size.
45H	Cache	2nd-level cache: 2 MByte, 4-way set associative, 32 byte line size.
46H	Cache	3rd-level cache: 4 MByte, 4-way set associative, 64 byte line size.
47H	Cache	3rd-level cache: 8 MByte, 8-way set associative, 64 byte line size.
48H	Cache	2nd-level cache: 3MByte, 12-way set associative, 64 byte line size.
49H	Cache	3rd-level cache: 4MB, 16-way set associative, 64-byte line size (Intel Xeon processor MP, Family 0FH, Model 06H) 2nd-level cache: 4 MByte, 16-way set associative, 64 byte line size.

4AH	Cache	3rd-level cache: 6MByte, 12-way set associative, 64 byte line size.
4BH	Cache	3rd-level cache: 8MByte, 16-way set associative, 64 byte line size.
4CH	Cache	3rd-level cache: 12MByte, 12-way set associative, 64 byte line size.
4DH	Cache	3rd-level cache: 16MByte, 16-way set associative, 64 byte line size.
4EH	Cache	2nd-level cache: 6MByte, 24-way set associative, 64 byte line size.
4FH	TLB	Instruction TLB: 4 KByte pages, 32 entries.
50H	TLB	Instruction TLB: 4 KByte and 2-MByte or 4-MByte pages, 64 entries.
51H	TLB	Instruction TLB: 4 KByte and 2-MByte or 4-MByte pages, 128 entries.
52H	TLB	Instruction TLB: 4 KByte and 2-MByte or 4-MByte pages, 256 entries.
55H	TLB	Instruction TLB: 2-MByte or 4-MByte pages, fully associative, 7 entries.
56H	TLB	Data TLB0: 4 MByte pages, 4-way set associative, 16 entries.
57H	TLB	Data TLB0: 4 KByte pages, 4-way associative, 16 entries.
59H	TLB	Data TLB0: 4 KByte pages, fully associative, 16 entries.
5AH	TLB	Data TLB0: 2 MByte or 4 MByte pages, 4-way set associative, 32 entries.
5BH	TLB	Data TLB: 4 KByte and 4 MByte pages, 64 entries.
5CH	TLB	Data TLB: 4 KByte and 4 MByte pages,128 entries.
5DH	TLB	Data TLB: 4 KByte and 4 MByte pages,256 entries.
60H	Cache	1st-level data cache: 16 KByte, 8-way set associative, 64 byte line size.
61H	TLB	Instruction TLB: 4 KByte pages, fully associative, 48 entries.
63H	TLB	Data TLB: 2 MByte or 4 MByte pages, 4-way set associative, 32 entries and a separate array with 1 GByte pages, 4-way set associative, 4 entries.
64H	TLB	Data TLB: 4 KByte pages, 4-way set associative, 512 entries.
66H	Cache	1st-level data cache: 8 KByte, 4-way set associative, 64 byte line size.
67H	Cache	1st-level data cache: 16 KByte, 4-way set associative, 64 byte line size.
68H	Cache	1st-level data cache: 32 KByte, 4-way set associative, 64 byte line size.
6AH	Cache	uTLB: 4 KByte pages, 8-way set associative, 64 entries.
6BH	Cache	DTLB: 4 KByte pages, 8-way set associative, 256 entries.
6CH	Cache	DTLB: 2M/4M pages, 8-way set associative, 128 entries.
6DH	Cache	DTLB: 1 GByte pages, fully associative, 16 entries.
70H	Cache	Trace cache: 12 K-?op, 8-way set associative.
71H	Cache	Trace cache: 16 K-?op, 8-way set associative.
72H	Cache	Trace cache: 32 K-?op, 8-way set associative.
76H	TLB	Instruction TLB: 2M/4M pages, fully associative, 8 entries.
78H	Cache	2nd-level cache: 1 MByte, 4-way set associative, 64byte line size.
79H	Cache	2nd-level cache: 128 KByte, 8-way set associative, 64 byte line size, 2 lines per sector.
7AH	Cache	2nd-level cache: 256 KByte, 8-way set associative, 64 byte line size, 2 lines per sector.
7BH	Cache	2nd-level cache: 512 KByte, 8-way set associative, 64 byte line size, 2 lines per sector.
7CH	Cache	2nd-level cache: 1 MByte, 8-way set associative, 64 byte line size, 2 lines per sector.
7DH	Cache	2nd-level cache: 2 MByte, 8-way set associative, 64byte line size.
7FH	Cache	2nd-level cache: 512 KByte, 2-way set associative, 64-byte line size.

80H	Cache	2nd-level cache: 512 KByte, 8-way set associative, 64-byte line size.
82H	Cache	2nd-level cache: 256 KByte, 8-way set associative, 32 byte line size.
83H	Cache	2nd-level cache: 512 KByte, 8-way set associative, 32 byte line size.
84H	Cache	2nd-level cache: 1 MByte, 8-way set associative, 32 byte line size.
85H	Cache	2nd-level cache: 2 MByte, 8-way set associative, 32 byte line size.
86H	Cache	2nd-level cache: 512 KByte, 4-way set associative, 64 byte line size.
87H	Cache	2nd-level cache: 1 MByte, 8-way set associative, 64 byte line size.
A0H	DTLB	DTLB: 4k pages, fully associative, 32 entries.
B0H	TLB	Instruction TLB: 4 KByte pages, 4-way set associative, 128 entries.
B1H	TLB	Instruction TLB: 2M pages, 4-way, 8 entries or 4M pages, 4-way, 4 entries.
B2H	TLB	Instruction TLB: 4KByte pages, 4-way set associative, 64 entries.
B3H	TLB	Data TLB: 4 KByte pages, 4-way set associative, 128 entries.
B4H	TLB	Data TLB1: 4 KByte pages, 4-way associative, 256 entries.
B5H	TLB	Instruction TLB: 4KByte pages, 8-way set associative, 64 entries.
B6H	TLB	Instruction TLB: 4KByte pages, 8-way set associative, 128 entries.
BAH	TLB	Data TLB1: 4 KByte pages, 4-way associative, 64 entries.
C0H	TLB	Data TLB: 4 KByte and 4 MByte pages, 4-way associative, 8 entries.
C1H	STLB	Shared 2nd-Level TLB: 4 KByte/2MByte pages, 8-way associative, 1024 entries.
C2H	DTLB	DTLB: 2 MByte/4 MByte pages, 4-way associative, 16 entries.
C3H	STLB	Shared 2nd-Level TLB: 4 KByte /2 MByte pages, 6-way associative, 1536 entries. Also 1GBbyte pages, 4-way, 16 entries.
C4H	DTLB	DTLB: 2 MByte/4 MByte pages, 4-way associative, 32 entries.
CAH	STLB	Shared 2nd-Level TLB: 4 KByte pages, 4-way associative, 512 entries.
D0H	Cache	3rd-level cache: 512 KByte, 4-way set associative, 64 byte line size.
D1H	Cache	3rd-level cache: 1 MByte, 4-way set associative, 64 byte line size.
D2H	Cache	3rd-level cache: 2 MByte, 4-way set associative, 64 byte line size.
D6H	Cache	3rd-level cache: 1 MByte, 8-way set associative, 64 byte line size.
D7H	Cache	3rd-level cache: 2 MByte, 8-way set associative, 64 byte line size.
D8H	Cache	3rd-level cache: 4 MByte, 8-way set associative, 64 byte line size.
DCH	Cache	3rd-level cache: 1.5 MByte, 12-way set associative, 64 byte line size.
DDH	Cache	3rd-level cache: 3 MByte, 12-way set associative, 64 byte line size.
DEH	Cache	3rd-level cache: 6 MByte, 12-way set associative, 64 byte line size.
E2H	Cache	3rd-level cache: 2 MByte, 16-way set associative, 64 byte line size.
E3H	Cache	3rd-level cache: 4 MByte, 16-way set associative, 64 byte line size.
E4H	Cache	3rd-level cache: 8 MByte, 16-way set associative, 64 byte line size.
EAH	Cache	3rd-level cache: 12MByte, 24-way set associative, 64 byte line size.
EBH	Cache	3rd-level cache: 18MByte, 24-way set associative, 64 byte line size.
ECH	Cache	3rd-level cache: 24MByte, 24-way set associative, 64 byte line size.
F0H	Prefetch	64-Byte prefetching.
F1H	Prefetch	128-Byte prefetching.

FEH	General	CPUID leaf 2 does not report TLB descriptor information; use CPUID leaf 18H to query TLB and other address translation parameters.
FFH	General	CPUID leaf 2 does not report cache descriptor information, use CPUID leaf 4 to query cache parameters.

Example 21-1. Example of Cache and TLB Interpretation

The first member of the family of Pentium 4 processors returns the following information about caches and TLBs when the CPUID executes with an input value of 2:
EAX 66 5B 50 01H
EBX 0H
ECX 0H
EDX 00 7A 70 00H
Which means:
• The least-significant byte (byte 0) of register EAX is set to 01H. This value should be ignored.
• The most-significant bit of all four registers (EAX, EBX, ECX, and EDX) is set to 0, indicating that each register contains valid 1-byte descriptors.
• Bytes 1, 2, and 3 of register EAX indicate that the processor has:
  — 50H - a 64-entry instruction TLB, for mapping 4-KByte and 2-MByte or 4-MByte pages.
  — 5BH - a 64-entry data TLB, for mapping 4-KByte and 4-MByte pages.
  — 66H - an 8-KByte 1st level data cache, 4-way set associative, with a 64-Byte cache line size.
• The descriptors in registers EBX and ECX are valid, but contain NULL descriptors.
• Bytes 0, 1, 2, and 3 of register EDX indicate that the processor has:
  — 00H - NULL descriptor.
  — 70H - Trace cache: 12 K-µop, 8-way set associative.
  — 7AH - a 256-KByte 2nd level cache, 8-way set associative, with a sectored, 64-byte cache line size.
  — 00H - NULL descriptor.

CPUID.03H -- Processor Serial Number

CPUID.03H returns the processor serial number, if available. Processor serial number (PSN) is not supported in the Pentium 4 processor or later.

- This leaf is valid if MAX_LEAF ≥ 03H.
- This leaf does not contain sub-leaves and provides the same information regardless of the value of ECX.

Table 21-13. Leaf 03H Processor Serial Number

Register	Field Name	Description	Domain
EAX[31:0]	Reserved	Reserved.	
EBX[31:0]	Reserved	Reserved.	
ECX[31:0]	PSN_31_0	Bits 00-31 of 96-bit processor serial number. (Available in Pentium III processor only; otherwise, the value in this register is reserved.)	Package
EDX[31:0]	PSN_63_32	Bits 32-63 of 96-bit processor serial number. (Available in Pentium III processor only; otherwise, the value in this register is reserved.)	Package

CPUID.04H -- Deterministic Cache Parameters

CPUID.04H returns the deterministic cache parameters for each cache level.
• This leaf is valid if CPUID.04H.00H:EAX[4:0] <> 0 and MAX_LEAF ≥ 04H.
• The sub-leaves are enumerated until sub-leaf n returns 0 in EAX[4:0].
• If ECX contains an invalid sub-leaf index, EAX/EBX/ECX/EDX return 0. Sub-leaf index n+1 is invalid if sub-leaf n returns EAX[4:0] as 0.

Table 21-14. Leaf 04H Deterministic Cache Parameters

Register	Field Name	Description	Domain
EAX[4:0]	CACHE_TYPE	0 = Null, no more caches. 1 = Data Cache. 2 = Instruction Cache. 3 = Unified Cache. 4-31 = Reserved.	Logical Processor
EAX[7:5]	CACHE_LEVEL	Cache level (starts at 1).	Logical Processor
EAX[8]	SELF_INITIALIZING_CACHE	Self initializing cache level (does not need software initialization).	Logical Processor
EAX[9]	FULLY_ASSOC	Fully associative cache.	Logical Processor
EAX[13:10]	Reserved	Reserved.	
EAX[25:14]	MAX_LP_ADDRESSABLE_IDS	Maximum number of addressable IDs for logical processors sharing this cache. Add one to the return value to get the result. The nearest power-of-2 integer that is not smaller than (1 + EAX[25:14]) is the number of unique initial APIC IDs reserved for addressing different logical processors sharing this cache.	Logical Processor
EAX[31:26]	MAX_CORES_ADDRESSABLE_IDS_PKG	Maximum number of addressable IDs for processor cores in the physical package. Add one to the return value to get the result. The nearest power-of-2 integer that is not smaller than (1 + EAX[31:26]) is the number of unique Core_IDs reserved for addressing different processor cores in a physical package. Core ID is a subset of bits of the initial APIC ID. The returned value is constant for valid initial values in ECX. Valid ECX values start from 0. The maximum number of addressable IDs for processor cores in the physical package field may contain a saturated value and will not correctly identify the addressable ID reservations for cores in this package on processors where CPUID.0BH and/or CPUID.1FH exist. Processors which enumerate topology information in either CPUID.0BH or CPUID.1FH need to use those leaves to obtain the correct topology details.	Platform
EBX[11:0]	LINE_SIZE	System Coherency Line Size. Add one to the return value to get the result.	Platform
EBX[21:12]	PHYS_LINE_PARTITIONS	Physical line partitions. Add one to the return value to get the result.	Logical Processor
EBX[31:22]	NUM_WAYS	Ways of associativity. Add one to the return value to get the result.	Logical Processor

ECX[31:0]	NUM_SETS	Number of sets. Add one to the return value to get the result.	Logical Processor
EDX[0]	NOT_LWR_CACHE_FLUSH	0 = WBINVD/INVD from threads sharing this cache acts upon lower level caches for threads sharing this cache. 1 = WBINVD/INVD is not guaranteed to act upon lower level caches of non-originating threads sharing this cache.	Logical Processor
EDX[1]	INCLUSIVE_CACHE	0 = Cache is not inclusive of lower cache levels. 1 = Cache is inclusive of lower cache levels.	Logical Processor
EDX[2]	COMPLEX_CACHE_INDEXING	0 = Direct mapped cache. 1 = A complex function is used to index the cache, potentially using all address bits.	Logical Processor
EDX[31:3]	Reserved	Reserved.	

When CPUID executes with EAX set to 04H and ECX contains an index value, the processor returns encoded data that describe a set of deterministic cache parameters (for the cache level associated with the input in ECX). Valid index values start from 0. Software can enumerate the deterministic cache parameters for each level of the cache hierarchy starting with an index value of 0, until the parameters report the value associated with the cache type field is 0.
This Cache Size in Bytes
= (Ways + 1) * (Partitions + 1) * (Line_Size + 1) * (Sets + 1)
= (EBX[31:22] + 1) * (EBX[21:12] + 1) * (EBX[11:0] + 1) * (ECX + 1)
The CPUID.04H also reports data that can be used to derive the topology of processor cores in a physical package on legacy processors. This information is constant for all valid index values. Software can query the raw data reported by executing CPUID with EAX=04H and ECX=0 and use it as part of the topology enumeration algorithm on processors that do not enumerate either CPUID.0BH or CPUID.1FH as described in Chapter 10, "Multiple-Processor Management," in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3A.

CPUID.05H -- MONITOR and MWAIT Features

CPUID.05H returns the MONITOR and MWAIT feature information.
- This leaf is valid if MAX_LEAF ≥ 05H.
- This leaf does not contain sub-leaves and provides the same information regardless of the value of ECX.

Table 21-15. Leaf 05H MONITOR and MWAIT Features

Register	Field Name	Description	Domain
EAX[15:0]	SMALLEST_MONITOR_LINE_SIZE	Smallest monitor-line size in bytes (default is processor's monitor granularity).	Platform
EAX[31:16]	Reserved	Reserved.	
EBX[15:0]	LARGEST_MONITOR_LINE_SIZE	Largest monitor-line size in bytes (default is processor's monitor granularity).	Platform
EBX[31:16]	Reserved	Reserved.	
ECX[0]	MONITOR_MWAIT_EXTENSIONS	If 1, supports enumeration of MONITOR/MWAIT extensions (beyond EAX and EBX registers).	Platform
ECX[1]	INTERRUPT_AS_BREAK_EVENT	If 1, supports treating interrupts as break-event for MWAIT, even when interrupts are disabled.	Platform
ECX[2]	Reserved	Reserved.	
ECX[3]	MONITORLESS_MWAIT	If 1, supports monitorless MWAIT.	Platform
ECX[31:4]	Reserved	Reserved.	
EDX[3:0]	C0_SUB_STATES	Number of C0* sub C-states supported using MWAIT.	Platform
EDX[7:4]	C1_SUB_STATES	Number of C1* sub C-states supported using MWAIT.	Platform
EDX[11:8]	C2_SUB_STATES	Number of C2* sub C-states supported using MWAIT.	Platform
EDX[15:12]	C3_SUB_STATES	Number of C3* sub C-states supported using MWAIT.	Platform
EDX[19:16]	C4_SUB_STATES	Number of C4* sub C-states supported using MWAIT.	Platform
EDX[23:20]	C5_SUB_STATES	Number of C5* sub C-states supported using MWAIT.	Platform
EDX[27:24]	C6_SUB_STATES	Number of C6* sub C-states supported using MWAIT.	Platform
EDX[31:28]	C7_SUB_STATES	Number of C7* sub C-states supported using MWAIT.	Platform

NOTE

The definition of C0 through C7 states for MWAIT extensions are processor-specific C-states, not ACPI C-state.

CPUID.05H returns information about features available to MONITOR/MWAIT instructions. The MONITOR instruction is used for address-range monitoring in conjunction with MWAIT instruction. The MWAIT instruction optionally provides additional extensions for advanced power management.

CPUID.06H -- Thermal and Power Management Features

CPUID.06H returns information about thermal and power management features.
- This leaf is valid if MAX_LEAF ≥ 06H.
- This leaf does not contain sub-leaves and provides the same information regardless of the value of ECX.

Table 21-16. Leaf 06H Thermal and Power Management Features

Register	Field Name	Description	Domain
EAX[0]	DIGITAL_TEMP_SENSOR	If 1, supports Digital Temperature Sensor.	Platform
EAX[1]	TURBO_BOOST	If 1, supports Intel Turbo Boost Technology. (see description of IA32_MISC_ENABLE[38]).	Platform
EAX[2]	ALWAYS_RUNNING_APIC_TIMER	If 1, supports APIC-Timer-always-running feature.	Platform
EAX[3]	Reserved	Reserved.	
EAX[4]	POWER_LIMIT_NOTIFY	If 1, supports power limit notification controls.	Platform
EAX[5]	EXT_CLOCK_MOD	If 1, supports clock modulation duty cycle extension.	Platform
EAX[6]	PKG_THERM_MGMT	If 1, supports package thermal management.	Platform
EAX[7]	HWP	If 1, supports HWP base registers (IA32_PM_ENABLE[bit 0], IA32_HWP_CAPABILITIES, IA32_HWP_REQUEST, IA32_HWP_STATUS).	Platform
EAX[8]	HWP_INTERRUPT	If 1, supports IA32_HWP_INTERRUPT MSR.	Platform
EAX[9]	HWP_ACTIVITY_WINDOW	If 1, supports IA32_HWP_REQUEST[bits 41:32].	Platform
EAX[10]	HWP_EPP	If 1, supports IA32_HWP_REQUEST[bits 31:24].	Platform
EAX[11]	HWP_REQUEST_PKG	If 1, supports IA32_HWP_REQUEST_PKG MSR.	Platform
EAX[12]	Reserved	Reserved.	
EAX[13]	HDC	If 1, supports HDC base registers IA32_PKG_HDC_CTL, IA32_PM_CTL1, and IA32_THREAD_STALL MSRs.	Platform
EAX[14]	TURBO_BOOST_MAX	If 1, supports Intel® Turbo Boost Max Technology 3.0.	Platform
EAX[15]	HWP_CAP	If 1, supports Highest Performance change capability.	Platform
EAX[16]	HWP_PECI_OVERRIDE	If 1, supports HWP PECI override.	Platform
EAX[17]	FLEXIBLE_HWP	If 1, supports Flexible HWP.	Platform
EAX[18]	HWP_REQUEST_FAST_ACCESS	If 1, supports Fast access mode for the IA32_HWP_REQUEST MSR.	Platform
EAX[19]	HW_FEEDBACK	If 1, supports IA32_HW_FEEDBACK_PTR MSR, IA32_HW_FEEDBACK_CONFIG MSR, IA32_PACKAGE_THERM_STATUS MSR bit 26, and IA32_PACKAGE_THERM_INTERRUPT MSR bit 25.	Platform
EAX[20]	HWP_REQUEST_IGNORE_IDLE	If 1, supports Ignoring Idle Logical Processor HWP request.	Platform
EAX[21]	Reserved	Reserved.	
EAX[22]	HWP_CTL	If 1, supports IA32_HWP_CTL MSR.	Platform

EAX[23]	THREAD_DIRECTOR	If 1, supports Intel® Thread Director. IA32_HW_FEEDBACK_CHAR and IA32_HW_FEEDBACK_THREAD_CONFIG MSRs are supported if set.	Platform
EAX[31:24]	Reserved	Reserved.	
EBX[3:0]	DTS_NUM_INT_THRESHOLDS	Number of Interrupt Thresholds in Digital Thermal Sensor.	Platform
EBX[31:4]	Reserved	Reserved.	
ECX[0]	HW_FEEDBACK_CAP	If 1, supports IA32_MPERF and IA32_APERF which provide a measure of delivered processor performance (since last reset of the counters), as a percentage of the expected processor performance when running at the TSC frequency.	Platform
ECX[2:1]	Reserved	Reserved.	
ECX[3]	ENERGY_PERF_BIAS	If 1, supports performance-energy bias preference and a new architectural MSR called IA32_ENERGY_PERF_BIAS (1B0H).	Platform
ECX[7:4]	Reserved	Reserved.	
ECX[15:8]	HW_FEEDBACK_NUM_CLASSES	Number of Intel® Thread Director classes supported by the processor. Information for that many classes is written into the Intel Thread Director Table by the hardware.	Platform
ECX[31:16]	Reserved	Reserved.	
EDX[7:0]	HW_FEEDBACK_CAPS	Bitmap of supported hardware feedback interface capabilities. 0 = If 1, supports performance capability reporting. 1 = If 1, supports energy efficiency capability reporting. 2-7 = Reserved. Bits 0 and 1 will always be set together.	Package
EDX[11:8]	HW_FEEDBACK_TABLE_SIZE	Enumerates the size of the hardware feedback interface structure in number of 4 KB pages. Add one to the return value to get the result.	Package
EDX[15:12]	Reserved	Reserved.	
EDX[31:16]	HW_FEEDBACK_TABLE_INDEX	Index (starting at 0) of this logical processor's row in the hardware feedback interface structure. Note that on some parts the index may be same for multiple logical processors. On some parts the indices may not be contiguous, i.e., there may be unused rows in the hardware feedback interface structure.	Logical Processor

Details around these features are described in Chapter 16, "Power and Thermal Management," in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3B.

CPUID.07H -- Structured Extended Feature Flags

CPUID.07H returns structured extended feature flags enumeration information. The sub-sections of Section provide leaf 07H information.

- This leaf is valid if MAX_LEAF ≥ 07H.
- The maximum sub-leaf value for ECX is specified in CPUID.07H.00H.EAX[31:0] MAX_SUBLEAF.
- If ECX contains an invalid Sub-leaf index, EAX/EBX/ECX/EDX return 0. Sub-leaf index n is invalid if n exceeds the value that sub-leaf 0 returns in EAX.

CPUID.07H.00H -- Structured Extended Feature Flags Main Sub-Leaf

CPUID.07H.00H returns the maximum input value of the highest leaf 07H sub-leaf; and EBX, ECX, and EDX contain information of extended feature flags.

Table 21-17. Leaf 07H Sub-Leaf (ECX=0) Output Registers

CPUID Output Registers	Field Name	Description	CPUID Domain
EAX[31:0]	MAX_SUBLEAF	Reports the maximum input value for supported leaf 07H subleaves.	Platform
EBX[31:0]		Extended Feature Flags Information in EBX (see "CPUID.07H.00H:EBX—Extended Feature Flags Information")	
ECX[31:0]		Extended Feature Flags Information in ECX (see "CPUID.07H.00H:ECX—Extended Feature Flags Information")	
EDX[31:0]		Extended Feature Flags Information in EDX (see "CPUID.07H.00H:EDX—Extended Feature Flags Information")	

Table 21-18. Leaf 07H.00H Structured Extended Feature Flags Returned in EAX

Register	Field Name	Description	Domain
EAX[31:0]	MAX_SUBLEAF	Reports the maximum input value for supported leaf 07H sub-leaves.	Platform

CPUID.07H.00H:EBX Extended Feature Flags Information

The EBX register of CPUID.07H.00H returns the information shown below.

Table 21-19. Leaf 07H.00H Structured Extended Feature Flags Returned in EBX

Register	Field Name	Description	Domain
EBX[0]	FSGSBASE	If 1, supports RDFSBASE/RDGSBASE/WRFS-BASE/WRGSBASE.	Platform
EBX[1]	TSC_ADJUST	If 1, the IA32_TSC_ADJUST MSR is supported.	Platform
EBX[2]	SGX	If 1, supports Intel® Software Guard Extensions (Intel® SGX Extensions).	Platform
EBX[3]	BMI1	If 1, supports the BMI1 instructions.	Platform
EBX[4]	HLE	If 1, supports the Hardware Lock Elision instruction set.	Platform
EBX[5]	AVX2	If 1, supports Intel® Advanced Vector Extensions 2 (Intel® AVX2).	Platform

EBX[6]	FDP_EXCPTN_ONLY	If 1, the x87 FPU Data Pointer is updated only on x87 exceptions.	Platform
EBX[7]	SMEP	If 1, supports Supervisor-Mode Execution Prevention.	Platform
EBX[8]	BMI2	If 1, supports the BMI2 instructions.	Platform
EBX[9]	ENH_REP_MOVSB_STOSB	If 1, supports Enhanced REP MOVSB/STOSB.	Platform
EBX[10]	INVPCID	If 1, supports INVPCID instruction for system software that manages process-context identifiers.	Platform
EBX[11]	RTM	If 1, supports the Restricted Transactional Memory instruction set.	Platform
EBX[12]	RDT_M	If 1, supports Intel® Resource Director Technology (Intel® RDT) Monitoring capability.	Platform
EBX[13]	FCS_FDS_DEPRECATION	If 1, deprecates FPU CS and FPU DS values.	Platform
EBX[14]	MPX	If 1, supports Intel® Memory Protection Extensions.	Platform
EBX[15]	RDT_A	If 1, supports Intel® Resource Director Technology (Intel® RDT) Allocation capability.	Platform
EBX[16]	AVX512F	If 1, supports the AVX512F instructions.	Platform
EBX[17]	AVX512DQ	If 1, supports the AVX512DQ instructions.	Platform
EBX[18]	RDSEED	If 1, supports the RDSEED instruction.	Platform
EBX[19]	ADX	If 1, supports the ADX instructions.	Platform
EBX[20]	SMAP	If 1, supports Supervisor-Mode Access Prevention and the CLAC/STAC instructions.	Platform
EBX[21]	AVX512_IFMA	If 1, supports the AVX512_IFMA instructions.	Platform
EBX[22]	Reserved	Reserved.	
EBX[23]	CLFLUSHOPT	If 1, supports the CLFLUSHOPT instruction.	Platform
EBX[24]	CLWB	If 1, supports the CLWB instruction.	Platform
EBX[25]	INTEL_PROC_TRACE	If 1, supports Intel® Processor Trace.	Platform
EBX[26]	AVX512PF	If 1, supports the AVX512PF instructions. (Intel® Xeon Phi™ only.)	Platform
EBX[27]	AVX512ER	If 1, supports the AVX512ER instructions. (Intel® Xeon Phi™ only.)	Platform
EBX[28]	AVX512CD	If 1, supports the AVX512CD instructions.	Platform
EBX[29]	SHA	If 1, supports Intel® Secure Hash Algorithm Extensions (Intel® SHA Extensions).	Platform
EBX[30]	AVX512BW	If 1, supports the AVX512BW instructions.	Platform
EBX[31]	AVX512VL	If 1, supports the AVX512VL instructions.	Platform

CPUID.07H.00H:ECX Extended Feature Flags Information

The ECX register of CPUID.07H.00H returns the information shown below.

Table 21-20. Leaf 07H.00H Structured Extended Feature Flags Returned in ECX

Register	Field Name	Description	Domain

ECX[0]	PREFETCHWT1	If 1, supports the PREFETCHWT1 instruction. (Intel® Xeon Phi™ only.)	Platform
ECX[1]	AVX512_VBMI	If 1, supports the AVX512_VBMI instructions.	Platform
ECX[2]	UMIP	If 1, supports user-mode instruction prevention.	Platform
ECX[3]	PKU	If 1, supports protection keys for user-mode pages.	Platform
ECX[4]	OSPKE	If 1, the OS has set CR4.PKE to enable protection keys and the RDPKRU/WRPKRU instructions.	Logical Processor
ECX[5]	WAITPKG	If 1, supports the TPAUSE, UMONITOR, and UMWAIT instructions.	Platform
ECX[6]	AVX512_VBMI2	If 1, supports the AVX512_VBMI2 instructions.	Platform
ECX[7]	CET_SS	If 1, supports CET shadow stack features. Processors that set this bit define bits 1:0 of the IA32_U_CET and IA32_S_CET MSRs. Enumerates support for the following MSRs: IA32_INTERRUPT_SPP_TABLE_ADDR, IA32_PL3_SSP, IA32_PL2_SSP, IA32_PL1_SSP, and IA32_PL0_SSP.	Platform
ECX[8]	GFNI	If 1, supports the GFNI instruction set.	Platform
ECX[9]	VAES	If 1 and Intel AVX supported, supports the VEX-encoded AES instruction set.	Platform
ECX[10]	VPCLMULQDQ	If 1 and Intel AVX supported, supports the VPCLMULQDQ instruction.	Platform
ECX[11]	AVX512_VNNI	If 1, supports the AVX512_VNNI instructions.	Platform
ECX[12]	AVX512_BITALG	If 1, supports the AVX512_BITALG instructions.	Platform
ECX[13]	TME_EN	If 1, the following MSRs are supported: IA32_TME_CAPABILITY, IA32_TME_ACTIVATE, IA32_TME_EXCLUDE_MASK, and IA32_TME_EXCLUDE_BASE.	Platform
ECX[14]	AVX512_VPOPCNTDQ	If 1, supports the AVX512_VPOPCNTDQ instructions.	Platform
ECX[15]	Reserved	Reserved.	
ECX[16]	LA57	If 1, supports 57-bit linear addresses and five-level paging.	Platform
ECX[21:17]	MPX_MAWAU	The value of MAWAU used by the BNDLDX and BNDSTX instructions in 64-bit mode.	Platform
ECX[22]	RDPID	If 1, RDPID and the IA32_TSC_AUX MSR are available.	Platform
ECX[23]	KEY_LOCKER	If 1, supports Key Locker.	Platform
ECX[24]	BUS_LOCK_DETECT	If 1, indicates support for OS bus-lock detection.	Platform
ECX[25]	CLDEMOTE	If 1, supports cache line demote.	Platform
ECX[26]	Reserved	Reserved.	
ECX[27]	MOVDIRI	If 1, supports the MOVDIRI instruction.	Platform
ECX[28]	MOVDIR64B	If 1, supports the MOVDIR64B instruction.	Platform
ECX[29]	ENQCMD	If 1, supports Enqueue Stores.	Platform

ECX[30]	SGX_LC	If 1, supports SGX Launch Configuration.	Platform
ECX[31]	PKS	If 1, supports protection keys for supervisor-mode pages.	Platform

CPUID.07H.00H:EDX Extended Feature Flags Information

The EDX register of CPUID.07H.00H returns the information shown below.

Table 21-21. Leaf 07H.00H Structured Extended Feature Flags Returned in EDX

Register	Field Name	Description	Domain
EDX[0]	Reserved	Reserved.	
EDX[1]	SGX_KEYS	If 1, supports Attestation Services for Intel® SGX.	Platform
EDX[2]	AVX512_4VNNIW	If 1, supports the AVX512_4VNNIW instructions. (Intel® Xeon Phi™ only.)	Platform
EDX[3]	AVX512_4FMAPS	If 1, supports the AVX512_4FMAPS instructions. (Intel® Xeon Phi™ only.)	Platform
EDX[4]	FAST_SHORT_REP_MOVSB	If 1, supports Fast Short REP MOVSB.	Platform
EDX[5]	UINTR	If 1, supports user interrupts.	Platform
EDX[7:6]	Reserved	Reserved.	
EDX[8]	AVX512_VP2INTERSECT	If 1, supports the AVX512_VP2INTERSECT instruction.	Platform
EDX[9]	MCU_OPT_CTRL	If 1, supports both the IA32_MCU_OPT_CTRL MSR and its bit 0 (RNGDS_MITG_DIS).	Platform
EDX[10]	MD_CLEAR	If 1, supports MD_CLEAR.	Platform
EDX[11]	RTM_ALWAYS_ABORT	If 1, any execution of XBEGIN immediately aborts and transitions to the specified fallback address.	Platform
EDX[12]	Reserved	Reserved.	
EDX[13]	RTM_FORCE_ABORT	If 1, supports RTM_FORCE_ABORT and the IA32_TSX_FORCE_ABORT MSR. These allow software to set IA32_TSX_FORCE_ABORT[0] (RTM_FORCE_ABORT).	Platform
EDX[14]	SERIALIZE	If 1, supports SERIALIZE instruction.	Platform
EDX[15]	HYBRID	If 1, the processor is identified as a hybrid part. If CPUID.00H.MAXLEAF ≥ 1AH and CPUID.1AH:EAX <> 0, then the Native Model ID Enumeration Leaf 1AH exists.	Platform
EDX[16]	TSXLDTRK	If 1, supports Intel TSX suspend/resume of load address tracking.	Platform
EDX[17]	Reserved	Reserved.	
EDX[18]	PCONFIG	If 1, supports the PCONFIG instruction.	Platform
EDX[19]	ARCH_LBRS	If 1, supports architectural LBRs.	Platform
EDX[20]	CET_IBT	If 1, supports CET indirect branch tracking features. Processors that set this bit define bits 5:2 and bits 63:10 of the IA32_U_CET and IA32_S_CET MSRs.	Platform

EDX[21]	Reserved	Reserved.	
EDX[22]	AMX_BF16	If 1, supports tile computational operations on bfloat16 numbers.	Platform
EDX[23]	AVX512_FP16	If 1, supports the FP16 data type with AVX512 instructions.	Platform
EDX[24]	AMX_TILE	If 1, supports tile architecture.	Platform
EDX[25]	AMX_INT8	If 1, supports tile computational operations on 8-bit integers.	Platform
EDX[26]	IBRS_IBPB	If 1, supports indirect branch restricted speculation (IBRS) and the indirect branch predictor barrier (IBPB). Processors that set this bit support the IA32_SPEC_CTRL MSR and the IA32_PRED_CMD MSR. They allow software to set IA32_SPEC_CTRL[0] (IBRS) and IA32_PRED_CMD[0] (IBPB).	Platform
EDX[27]	SPEC_CTRL_ST_PREDICTORS	If 1, supports single thread indirect branch predictors (STIBP). Processors that set this bit support the IA32_SPEC_CTRL MSR. They allow software to set IA32_SPEC_CTRL[1] (STIBP).	Platform
EDX[28]	L1D_FLUSH_INTERFACE	If 1, supports L1D_FLUSH. Processors that set this bit support the IA32_FLUSH_CMD MSR. They allow software to set IA32_FLUSH_CMD[0] (L1D_- FLUSH).	Platform
EDX[29]	ARCH_CAPABILITIES	If 1, supports the IA32_ARCH_CAPABILITIES MSR.	Platform
EDX[30]	CORE_CAPABILITIES	If 1, supports the IA32_CORE_CAPABILITIES MSR. IA32_CORE_CAPABILITIES is an architectural MSR that enumerates model-specific features. A bit being set in this MSR indicates that a model specific feature is supported; software must still consult CPUID family/model/stepping to determine the behavior of the enumerated feature as features enumerated in IA32_CORE_CAPABILITIES may have different behavior on different processor models. Some of these features may have behavior that is consistent across processor models (and for which consultation of CPUID family/model/stepping is not necessary); such features are identified explicitly where they are documented in this manual.	Platform
EDX[31]	SPEC_CTRL_SSBD	If 1, supports Speculative Store Bypass Disable (SSBD). Processors that set this bit support the IA32_SPEC_CTRL MSR. They allow software to set IA32_SPEC_CTRL[2] (SSBD).	Platform

CPUID.07H.01H -- Structured Extended Feature Sub-Leaf 1

CPUID.07H.01H:EAX Extended Feature Information

The EAX register of CPUID.07H.01H returns the information shown below.

Table 21-22. Leaf 07H.01H Structured Extended Feature Flags Returned in EAX

Register	Field Name	Description	Domain
EAX[0]	SHA512	If 1, supports the SHA512 instructions.	Platform
EAX[1]	SM3	If 1, supports the SM3 instructions.	Platform
EAX[2]	SM4	If 1, supports the SM4 instructions.	Platform
EAX[3]	Reserved	Reserved.	
EAX[4]	AVX_VNNI	If 1, supports the VEX-encoded versions of the Vector Neural Network Instructions.	Platform
EAX[5]	AVX512_BF16	If 1, supports the Vector Neural Network Instructions supporting BFLOAT16 inputs and conversion instructions from IEEE single precision.	Platform
EAX[6]	LASS	If 1, supports Linear Address Space Separation.	Platform
EAX[7]	CMPCCXADD	If 1, supports the CMPccXADD instruction.	Platform
EAX[8]	ARCH_PERFMON_EXT	If 1, supports ArchPerfmonExt. When set, indicates that the Architectural Performance Monitoring Extended Leaf (EAX=23H) is valid.	Platform
EAX[9]	Reserved	Reserved.	
EAX[10]	FAST_REP_MOVSB	If 1, supports fast zero-length REP MOVSB.	Platform
EAX[11]	FAST_REP_STOSB	If 1, supports fast short REP STOSB.	Platform
EAX[12]	FAST_REP_CMPSB_SCASB	If 1, supports fast short REP CMPSB, REP SCASB.	Platform
EAX[16:13]	Reserved	Reserved.	
EAX[17]	FRED	If 1, supports Flexible Return and Event Delivery and the architectural state (MSRs) defined by FRED. Any Intel processor that enumerates support for FRED transitions will also enumerate support for LKGS.	Platform
EAX[18]	LKGS	If 1, supports the LKGS (load into IA32_KERNEL_GS_BASE) instruction.	Platform
EAX[19]	WRMSRNS	If 1, supports the WRMSRNS instruction.	Platform
EAX[20]	NMI_SRC	If 1, supports NMI-source reporting.	Platform
EAX[21]	AMX_FP16	If 1, supports tile computational operations on FP16 numbers.	Platform
EAX[22]	HRESET	If 1, supports history reset via the HRESET instruction and the IA32_HRESET_ENABLE MSR. When set, indicates that the Processor History Reset Leaf (EAX = 20H) is valid.	Platform
EAX[23]	AVX_IFMA	If 1, supports the AVX-IFMA instructions.	Platform
EAX[25:24]	Reserved	Reserved.	
EAX[26]	LAM	If 1, supports Linear Address Masking.	Platform
EAX[27]	MSRLIST	If 1, supports the RDMSRLIST and WRMSRLIST instructions and the IA32_BARRIER MSR.	Platform
EAX[29:28]	Reserved	Reserved.	
EAX[30]	INVD_DISABLE_POST_BIOS_DONE	If 1, supports INVD execution prevention after BIOS Done.	Platform

EAX[31]	Reserved	Reserved.	

CPUID.07H.01H:EBX Extended Feature Information

The EBX register of CPUID.07H.01H returns the information shown below.

Table 21-23. Leaf 07H.01H Structured Extended Feature Flags Returned in EBX

Register	Field Name	Description	Domain
EBX[0]	PPIN	If 1, supports the IA32_PPIN and IA32_PPIN_CTL MSRs.	Platform
EBX[1]	PBNDKB	If 1, supports the PBNDKB instruction and enumerates the existence of the IA32_TSE_CAPABILITY MSR.	Platform
EBX[2]	Reserved	Reserved.	
EBX[3]	CPUIDMAXVAL_LIM_RMV	If 1, IA32_MISC_ENABLE[bit 22] cannot be set to 1 to limit the value returned by CPUID.00H:EAX[7:0].	Platform
EBX[31:4]	Reserved	Reserved.	

CPUID.07H.01H:ECX Extended Feature Information

The ECX register of CPUID.07H.01H returns the information shown below.

Table 21-24. Leaf 07H.01H Structured Extended Feature Flags Returned in ECX

Register	Field Name	Description	Domain
ECX[0]	RDT_M_ASYM	If 1, at least one logical processor on this platform supports Asymmetrical Intel® RDT Monitoring capability.	
ECX[1]	RDT_A_ASYM	If 1, at least one logical processor on this platform supports Asymmetrical Intel® RDT Allocation capability.	
ECX[4:2]	Reserved	Reserved.	
ECX[5]	MSR_IMM	If 1, supports the immediate forms of the RDMSR and WRMSRNS instructions.	Platform
ECX[31:6]	Reserved	Reserved.	

CPUID.07H.01H:EDX Extended Feature Information

The EDX register of CPUID.07H.01H returns the information shown below.

Table 21-25. Leaf 07H.01H Structured Extended Feature Flags Returned in EDX

Register	Field Name	Description	Domain
EDX[3:0]	Reserved	Reserved.	
EDX[4]	AVX_VNNI_INT8	If 1, supports the AVX-VNNI-INT8 instructions.	Platform
EDX[5]	AVX_NE_CONVERT	If 1, supports the AVX-NE-CONVERT instructions.	Platform
EDX[7:6]	Reserved	Reserved.	

EDX[8]	AMX_COMPLEX	If 1, supports the AMX_COMPLEX instructions.	
EDX[9]	Reserved	Reserved.	
EDX[10]	AVX_VNNI_INT16	If 1, supports the AVX-VNNI-INT16 instructions.	Platform
EDX[13:11]	Reserved	Reserved.	
EDX[14]	PREFETCHI	If 1, supports the PREFETCHIT0/1 instructions.	Platform
EDX[15]	USER_MSR	If 1, supports the URDMSR and UWRMSR instructions.	Platform
EDX[16]	Reserved	Reserved.	
EDX[17]	UIRET_UIF	If 1, UIRET sets UIF to the value of bit 1 of the RFLAGS image loaded from the stack.	Platform
EDX[18]	CET_SSS	If 1, indicates that an operating system can enable supervisor shadow stacks as long as it ensures that a supervisor shadow stack cannot become prematurely busy due to page faults (see Section 17.2.3 of the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 1). When emulating the CPUID instruction, a virtual-machine monitor (VMM) should return this bit as 1 only if it ensures that VM exits cannot cause a guest supervisor shadow stack to appear to be prematurely busy. Such a VMM could set the "prematurely busy shadow stack" VM-exit control and use the additional information that it provides.	Platform
EDX[19]	AVX10	If 1, supports the Intel® AVX10 instructions and indicates the presence of CPUID.24H, which enumerates the version number.	Platform
EDX[21:20]	Reserved	Reserved.	
EDX[22]	SEC-TEE-ATTESTATION	N/A	Platform
EDX[23]	MWAIT	If 1, MWAIT is supported (even if CPUID.01H:ECX.MONITOR[3] is enumerated as 0).	Platform
EDX[24]	SLSM	Static LSM is supported on this platform. If set, IA32_INTEGRITY_STATUS (0x2DC) is available for software use.	Platform
EDX[31:25]	Reserved	Reserved.	

CPUID.07H.02H -- Structured Extended Feature Sub-Leaf 2

CPUID.07H.02H returns the structured extended feature information contained in the sub-sections of this section.

Table 21-26. Leaf 07H Sub-Leaf (ECX=2) Output Registers

CPUID Output Registers	Description
EAX[31:0]	Reserved
EBX[31:0]	Reserved
ECX[31:0]	Reserved
EDX[31:0]	Extended Feature Information (see "CPUID.07H.02H:EDX—Extended Feature Information")

CPUID.07H.02H:EDX Extended Feature Information

The EDX register of CPUID.07H.02H returns the information shown below.

Table 21-27. CPUID.07H.02H Extended Feature Information Provided in EDX[1]

Register	Field Name	Description	Domain
EDX[0]	PSFD	If 1, supports bit 7 of the IA32_SPEC_CTRL MSR. Bit 7 of this MSR disables Fast Store Forwarding Predictor without disabling Speculative Store Bypass.	Platform
EDX[1]	IPRED_CTRL	If 1, supports bits 3 and 4 of the IA32_SPEC_CTRL MSR. Bit 3 of this MSR enables IPRED_DIS control for CPL3. Bit 4 of this MSR enables IPRED_DIS control for CPL0/1/2.	Platform
EDX[2]	RRSBA_CTRL	If 1, supports bits 5 and 6 of the IA32_SPEC_CTRL MSR. Bit 5 of this MSR disables RRSBA behavior for CPL3. Bit 6 of this MSR disables RRSBA behavior for CPL0/1/2.	Platform
EDX[3]	DDPD_U	If 1, supports bit 8 of the IA32_SPEC_CTRL MSR. Bit 8 of this MSR disables Data Dependent Prefetcher.	Platform
EDX[4]	BHI_CTRL	If 1, supports bit 10 of the IA32_SPEC_CTRL MSR. Bit 10 of this MSR enables BHI_DIS_S behavior.	Platform
EDX[5]	MCDT_NO	If 1, the processor does not exhibit MXCSR Configuration Dependent Timing (MCDT) behavior and does not need to be mitigated to avoid data- dependent behavior for certain instructions.	Platform
EDX[6]	UC_LOCK_DISABLE	If 1, supports the UC-lock disable feature and it causes #AC.	Platform
EDX[7]	MONITOR_MITG_NO	If 1, the MONITOR/UMONITOR instructions are not affected by performance or power issues due to MONITOR/UMONITOR instructions exceeding the capacity of an internal monitor tracking table. If 0, then the product may be affected by this issue.	Platform
EDX[31:8]	Reserved	Reserved.	

NOTE

1. Leaf 07H output depends on the initial value in ECX. If ECX contains an invalid sub-leaf index, EDX returns 0.

CPUID.08H -- Reserved

This leaf is reserved.

Table 21-28. Leaf 08H Reserved

Register	Field Name	Description	Domain
EAX[31:0]	Reserved	Reserved.	
EBX[31:0]	Reserved	Reserved.	
ECX[31:0]	Reserved	Reserved.	
EDX[31:0]	Reserved	Reserved.	

CPUID.09H -- Direct Cache Access Information

CPUID.09H returns information about Direct Cache Access capabilities.
- This leaf is valid if CPUID.01H:ECX.DCA[18] = 1 and MAX_LEAF ≥ 09H.
- This leaf does not contain sub-leaves and provides the same information regardless of the value of ECX.

Table 21-29. Leaf 09H Direct Cache Access Information

Register	Field Name	Description	Domain
EAX[31:0]	PLATFORM_DCA_CAP	Value of bits [31:0] of IA32_PLATFORM_DCA_CAP MSR (address 1F8H).	Platform
EBX[31:0]	Reserved	Reserved.	
ECX[31:0]	Reserved	Reserved.	
EDX[31:0]	Reserved	Reserved.	

The IA32_PLATFORM_DCA_CAP MSR is valid when CPUID.01H:ECX.DCA[18] = 1.

CPUID.0AH -- Architectural Performance Monitoring

CPUID.0AH returns information about support for architectural performance monitoring capabilities.
- This leaf is valid if CPUID.0AH:EAX[7:0] (Version ID) > 0 and MAX_LEAF ≥ 0AH.
- This leaf does not contain sub-leaves and provides the same information regardless of the value of ECX.

Table 21-30. Leaf 0AH Architectural Performance Monitoring

Register	Field Name	Description	Domain
EAX[7:0]	VERSION	Version ID of architectural performance monitoring.	Platform
EAX[15:8]	NUM_GP_CTRS	Number of general-purpose performance monitoring counter(s) per logical processor.	Platform
EAX[23:16]	GP_CTR_WIDTH	Bit width of general-purpose, performance monitoring counter.	Platform
EAX[31:24]	EVENT_ENUM_LENGTH	Length of EBX bit vector to enumerate architectural performance monitoring events. Architectural event x is supported if EBX[x]=0 && EAX[31:24]>x.	Platform
EBX[0]	CORE_CYC_NA	Core cycle event not available if 1 or if EAX[31:24]<1.	Platform
EBX[1]	INTR_RET_NA	Instruction retired event not available if 1 or if EAX[31:24]<2.	Platform
EBX[2]	REF_CYC_NA	Reference cycles event not available if 1 or if EAX[31:24]<3.	Platform
EBX[3]	LLC_CYC_NA	Last-level cache reference event not available if 1 or if EAX[31:24]<4.	Platform
EBX[4]	LLC_MISSES_NA	Last-level cache misses event not available if 1 or if EAX[31:24]<5.	Platform
EBX[5]	BR_INSTR_RET_NA	Branch instruction retired event not available if 1 or if EAX[31:24]<6.	Platform
EBX[6]	BR_MISPRED_RET_NA	Branch mispredict retired event not available if 1 or if EAX[31:24]<7.	Platform
EBX[7]	SLOTS_NA	Top-down slots event not available if 1 or if EAX[31:24]<8.	Platform
EBX[8]	BACKEND_NA	Topdown backend bound not available if 1 or if EAX[31:24] < 9.	Platform
EBX[9]	BADSPEC_NA	Topdown bad speculation not available if 1 or if EAX[31:24] < 10.	Platform
EBX[10]	FRONTEND_NA	Topdown frontend bound not available if 1 or if EAX[31:24] < 11.	Platform
EBX[11]	RETIRING_NA	Topdown retiring not available if 1 or if EAX[31:24] < 12.	Platform
EBX[12]	LBR_INSERTS_NA	LBR inserts not available if 1 or if EAX[31:24] < 13.	Platform
EBX[31:13]	Reserved	Reserved.	

ECX[31:0]	FIXED_CTR_MASK	Supported fixed counters bit mask. Fixed-function performance counter 'i' is supported if bit 'i' is 1 (first counter index starts at zero). It is recommended to use the following logic to determine if a Fixed Counter is supported: FxCtr[i]_is_supported := ECX[i] || (EDX[4:0] > i);	Platform
EDX[4:0]	NUM_FIXED_CTR	Number of contiguous fixed-function performance counters starting from 0 (if Version ID > 1).	Platform
EDX[12:5]	FIXED_CTR_WIDTH	Bit width of fixed-function performance counters (if Version ID > 1).	Platform
EDX[14:13]	Reserved	Reserved.	
EDX[15]	ANYTHREAD_DEPRECATION	Starting with Architectural Performance Monitoring Version 5, this field indicates that a processor supports AnyThread mode deprecation. If this field is set, software can choose to ignore guidelines in "AnyThread Counting and Software Evolution" of Chapter 21, "Performance Monitoring," in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3B	Platform
EDX[19:16]	SLOTS_PER_CYC	If this field is non-zero, it represents the number of Top-down Microarchitecture Analysis (TMA) slots per cycle. This number can be multiplied by the number of cycles (from CPU_CLK_UNHALTED.THREAD / CPU_CLK_UNHALTED.CORE or IA32_FIXED_CTR1) to determine the total number of slots. If this field is zero, IA32_FIXED_CTR3 should be used to determine the total number of slots.	Platform
EDX[31:20]	Reserved	Reserved.	

For each version of architectural performance monitoring capability, software must enumerate this leaf to discover the programming facilities and the architectural performance events available in the processor. The details are described in Chapter 21, "Performance Monitoring," in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3C.

CPUID.0BH -- Extended Topology

CPUID.0BH returns information about Extended Topology. CPUID.1FH is a preferred superset to leaf 0BH. Intel recommends first checking for the existence of leaf 1FH before using leaf 0BH.

- This leaf is valid if CPUID.0BH.00H:EBX[15:0] <> 0 and MAX_LEAF ≥ 0BH.
  - When the leaf is invalid, CPUID.0BH.00H:ECX.DOMAIN_TYPE[15:8] will report the Domain Type ID as Invalid (0).
- The sub-leaves are enumerated until sub-leaf n returns 0 in EBX[15:0].
- If ECX contains an invalid sub-leaf index, EAX/EBX return 0. Sub-leaf index n+1 is invalid if sub-leaf n returns EBX[15:0] as 0.

CPUID.0BH -- ECX >= 0

Table 21-31. Leaf 0BH Extended Topology

Register	Field Name	Description	Domain
EAX[4:0]	SHIFT_COUNT	The number of bits that the x2APIC ID must be shifted to the right to address instances of the next higher-scoped domain. When logical processor is not supported by the processor, the value of this field at the Logical Processor domain sub-leaf may be returned as either 0 (no allocated bits in the x2APIC ID) or 1 (one allocated bit in the x2APIC ID); software should plan accordingly.	Platform
EAX[31:5]	Reserved	Reserved.	
EBX[15:0]	NEXT_LEVEL_NUM_LP	The number of logical processors across all instances of this domain within the next higher-scoped domain. (For example, in a processor socket/package comprising "M" cores of "N" logical processors each, the "core" domain sub-leaf value of this field would be M*N.) This number reflects configuration as shipped by Intel. This field may also contain asymmetric values across different logical processors, as an example of a mix of cores that support more than one logical processor with cores that support only one logical processor. Note, software must not use this field to enumerate processor topology. Software must not use the value of EBX[15:0] to enumerate processor topology of the system. The value is only intended for display and diagnostic purposes. The actual number of logical processors available to BIOS/OS/Applications may be different from the value of EBX[15:0], depending on software and platform hardware configurations.	Logical Processor
EBX[31:16]	Reserved	Reserved.	
ECX[7:0]	LEVEL_NUM	The input ECX sub-leaf index.	Platform

ECX[15:8]	DOMAIN_TYPE	This field provides an identification value which indicates the domain shown in the table. Although domains are ordered, their assigned identification values are not and software should not depend on it. Note that enumeration values of 0 and 3-255 are reserved.	Platform
ECX[31:16]	Reserved	Reserved.	
EDX[31:0]	X2APIC_ID	The X2APIC ID of this logical processor.	Logical Processor

The sub-leaves of CPUID.0BH describe an ordered hierarchy of logical processors starting from the smallest-scoped domain of a Logical Processor (sub-leaf index 0) to the Core domain (sub-leaf index 1) to the largest-scoped domain (the last valid sub-leaf index) that is implicitly subordinate to the unenumerated highest-scoped domain of the processor package (socket).
The details of each valid domain is enumerated by a corresponding sub-leaf. Details for a domain include its type and how all instances of that domain determine the number of logical processors and x2 APIC ID partitioning at the next higher-scoped domain. The ordering of domains within the hierarchy is fixed architecturally as shown below. For a given processor, not all domains may be relevant or enumerated; however, the logical processor and core domains are always enumerated.
For two valid sub-leaves N and N+1, sub-leaf N+1 represents the next immediate higher-scoped domain with respect to the domain of sub-leaf N for the given processor.
If sub-leaf index "N" returns an invalid domain type in ECX[15:08] (00H), then all sub-leaves with an index greater than "N" also return an invalid domain type. A sub-leaf returning an invalid domain always returns 0 in EAX and EBX.

Table 21-32. Hierarchy of Valid Domain Enumerations in CPUID.0BH:ECX[15:8]

Hierarchy	Domain	Domain Type ID Value
Invalid	Invalid	0
Lowest	Logical Processor	1
...	Core	2
Highest	Package/Socket	(Implied)
Reserved	Reserved	3-255

CPUID.0CH -- Reserved

Table 21-33. Leaf 0CH Reserved

Register	Field Name	Description	Domain
EAX[31:0]	Reserved	Reserved.	
EBX[31:0]	Reserved	Reserved.	
ECX[31:0]	Reserved	Reserved.	
EDX[31:0]	Reserved	Reserved.	

CPUID.0DH -- Processor Extended State

CPUID.0DH returns a bit-vector representation of all processor state extensions that are supported in the processor and storage size requirements of the XSAVE/XRSTOR area.

- This leaf is valid if CPUID.01H:ECX.XSAVE[26] = 1 and MAX_LEAF ≥ 0DH.
- Sub-leafs 0 and 1 are always valid; consult them to determine which other sub-leafs are present as described in "CPUID.0DH.n, n>01H—State Sub-Leaves".

CPUID.0DH.00H -- Processor Extended State Main Sub-Leaf

CPUID.0DH.00H returns the processor extended state information.

Table 21-34. Leaf 0DH.00H Processor Extended State

Register	Field Name	Description	Domain
EAX[0]	X87	x87 state.	Platform
EAX[1]	SSE	SSE state.	Platform
EAX[2]	AVX	AVX state.	Platform
EAX[3]	MPX_BNDREGS	MPX state.	Platform
EAX[4]	MPX_BNDCSR	MPX state.	Platform
EAX[5]	AVX512_OPMASK	AVX-512 Opmask state.	Platform
EAX[6]	AVX512_ZMM_HI256	AVX-512 ZMM upper 256 data state.	Platform
EAX[7]	AVX512_HI16_ZMM	AVX-512 upper 16 ZMM registers state.	Platform
EAX[8]	N/A	Always returns 0 (Allocated for IA32_XSS).	Platform
EAX[9]	PKRU	PKRU state.	Platform
EAX[16:10]	N/A	Always returns 0 (Allocated for IA32_XSS).	Platform
EAX[17]	AMX_TILECFG	TILECFG state.	Platform
EAX[18]	AMX_TILEDATA	TILEDATA state.	Platform
EAX[31:19]	Reserved	Reserved.	
EBX[31:0]	XSAVE_BYTES_ENABLED_FEATURES	Maximum size (bytes, from the beginning of the XSAVE/XRSTOR save area) required by enabled features in XCR0. May be different than ECX if some features at the end of the XSAVE save area are not enabled.	Logical Processor
ECX[31:0]	XSAVE_BYTES_SUPPORTED_FEATURES	Maximum size (bytes, from the beginning of the XSAVE/XRSTOR save area) of the XSAVE/XRSTOR save area required by all supported features in the processors, i.e. all the valid bit fields in XCR0.	Platform
EDX[31:0]	VALID_XCR0_UPPER_32	Reports the supported bits of the upper 32 bits of XCR0. XCR0[n+32] can be set to 1 only if EDX[n] is 1.	Platform

The EAX register of CPUID.0DH.00H reports the supported bits of the lower 32 bits of XCR0. XCR0[n] can be set to 1 only if EAX[n] is 1. The details are described in Chapter 13.2, "Enumeration of CPUID Support for XSAVE Instructions and XSAVE-Supported Features," in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 1.

CPUID.0DH.01H -- Feature and Supervisor State Sub-Leaf

CPUID.0DH.01H returns feature and supervisor state information.

Table 21-35. Leaf 0DH.01H Processor Extended State

Register	Field Name	Description	Domain
EAX[0]	XSAVEOPT	If 1, supports XSAVEOPT.	Platform
EAX[1]	XSAVEC	If 1, supports XSAVEC and the compacted form of XRSTOR.	Platform
EAX[2]	XGETBV1	If 1, supports XGETBV with ECX = 1.	Platform
EAX[3]	XSAVES	If 1, supports XSAVES/XRSTORS and IA32_XSS.	Platform
EAX[4]	XFD	If 1, supports extended feature disable (XFD).	Platform
EAX[31:5]	Reserved	Reserved.	
EBX[31:0]	XSAVES_BYTES_ENABLED_FEATURES	The size in bytes of the XSAVE area containing all states enabled by XCR0 | IA32_XSS. If EAX[3] is enumerated as 0 and EAX[1] is enumerated as 1, EBX enumerates the size of the XSAVE area containing all states enabled by XCR0. If EAX[1] and EAX[3] are both enumerated as 0, EBX enumerates zero.	Logical Processor
ECX[7:0]	N/A	Always returns 0 (Allocated for XCR0).	Platform
ECX[8]	PT	PT state.	Platform
ECX[9]	Reserved	Always returns 0 (Allocated for XCR0).	
ECX[10]	PASID	PASID state.	Platform
ECX[11]	CET_U	CET user state.	Platform
ECX[12]	CET_S	CET supervisor state.	Platform
ECX[13]	HDC	HDC state.	Platform
ECX[14]	UINTR	UINTR state.	Platform
ECX[15]	LBR	LBR state (only for the architectural LBR feature).	Platform
ECX[16]	HWP	HWP state.	Platform
ECX[18:17]	N/A	Always returns 0 (Allocated for XCR0).	Platform
ECX[31:19]	Reserved	Reserved.	
EDX[31:0]	Reserved	Reserved	

NOTE

ECX reports the supported bits of the lower 32 bits of the IA32_XSS MSR. IA32_XSS[n] can be set to 1 only if ECX[n] is 1. EDX reports the supported bits of the upper 32 bits of the IA32_XSS MSR. IA32_XSS[n+32] can be set to 1 only if EDX[n] is 1. The details are described in Chapter 13.2, "Enumeration of CPUID Support for XSAVE Instructions and XSAVE-Supported Features," in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 1.

CPUID.0DH.SUB-LEAVES -- Sub-leaves

CPUID.0DH.n,where n>1, returns information about the size and offset of each processor extended state save area within the XSAVE/XRSTOR area. Software can use the forward-extendable technique depicted below to query in bulk the valid sub-leaves and obtain size and offset information for each processor extended state save area that is supported:

```
//* For each supported feature indicated by sub-leaf 0 and 1, read the size and offset sub-leaf *//
For j= 2 to 62
   If (CPUID.0DH.00H:<EDX:EAX>[j] == 1 or // Use 64-bit value of EDX:EAX
       CPUID.0DH.01H:<EDX:ECX>[j] == 1) // Use 64-bit value of EDX:ECX
        Read(CPUID.0DH.j) // Examine the size and offset.
   END IF
END FOR
```

Table 21-36. Leaf 0DH.SUB-LEAVES Processor Extended State

Register	Field Name	Description	Domain
EAX[31:0]	COMP_SIZE	The size in bytes (from the offset specified in EBX) of the save area for an extended state feature associated with a valid sub-leaf index n.	Platform
EBX[31:0]	COMP_OFFSET	The offset in bytes of this extended state component's save area from the beginning of the XSAVE/XRSTOR area. This field reports 0 if the sub-leaf index, n, does not map to a valid bit in the XCR0 register. If ECX contains an invalid sub-leaf index, EAX/EBX/ECX/EDX return 0. Sub-leaf n (0 ≤ n ≤ 31) is invalid if sub-leaf 0 returns 0 in EAX[n] and sub-leaf 1 returns 0 in ECX[n]. Sub-leaf n (32 ≤ n ≤ 63) is invalid if sub-leaf 0 returns 0 in EDX[n-32] and sub-leaf 1 returns 0 in EDX[n-32]	Platform
ECX[0]	COMP_SUP	This bit is set if the bit n (corresponding to the sub-leaf index) is supported in the IA32_XSS MSR; it is clear if bit n is instead supported in XCR0.	Platform
ECX[1]	COMP_64B_ALIGNED	This bit is set if, when the compacted format of an XSAVE area is used, this extended state component located on the next 64-byte boundary following the preceding state component (otherwise, it is located immediately following the preceding state component)	Platform
ECX[2]	COMP_XFD	This bit is set to indicate support for XFD faulting.	Platform
ECX[31:3]	Reserved	Reserved.	
EDX[31:0]	Reserved	Reserved.	

CPUID.0EH -- Reserved

This leaf is reserved.

Table 21-37. Leaf 0EH Reserved

Register	Field Name	Description	Domain
EAX[31:0]	Reserved	Reserved.	
EBX[31:0]	Reserved	Reserved.	
ECX[31:0]	Reserved	Reserved.	
EDX[31:0]	Reserved	Reserved.	

CPUID.0FH -- Intel® Resource Director Technology (Intel® RDT) Monitoring

CPUID.0FH returns information for the Intel Resource Director Technology Monitoring capabilities.
As described below, software uses the bit vector returned in EDX by sub-leaf 00H to determine the available resource types (ResID) that can be monitored. This information is necessary for software to program the IA32_PQR_ASSOC and IA32_QM_EVTSEL MSRs such that Quality-of-Service data can be read afterwards from the IA32_QM_CTR MSR.

- This leaf is valid if CPUID.07H.00H:EBX.RDT_M[12] = 1 and MAX_LEAF ≥ 0FH.
- If the leaf is valid, sub-leaf 00H is always valid. Sub-leaf n (n ≥ 1) is only valid when (CPUID.0FH.00H:EDX[n] == 1).

CPUID.0FH.00H -- Intel® RDT Monitoring Main Sub-Leaf

CPUID.0FH.00H returns information about Intel RDT Monitoring.

Table 21-38. Leaf 0FH.00H Intel® Resource Director Technology (Intel® RDT) Monitoring

Register	Field Name	Description	Domain
EAX[31:0]	Reserved	Reserved.	
EBX[31:0]	MAX_RMID	Maximum range (zero-based) of RMID within this physical processor of all types.	Platform
ECX[31:0]	Reserved	Reserved.	
EDX[0]	Reserved	Reserved.	
EDX[1]	L3_MON	If 1, supports L3 Cache Intel RDT Monitoring. Sub-leaf index 0 reports valid resource type starting at bit position 1 of EDX.	Platform
EDX[31:2]	Reserved	Reserved.	

CPUID.0FH.00H returns information about the bit-vector representation of QoS monitoring resource types that are supported in the processor and maximum range of RMID values the processor can use to monitor of any supported resource types. Each bit, starting from bit 1, corresponds to a specific resource type if the bit is set. The bit position corresponds to the sub-leaf index (or ResID) that software must use to query QoS monitoring capability available for that type.

CPUID.0FH.01H -- L3 Cache Intel® Resource Director Technology Monitoring

CPUID.0FH.01H returns information about L3 Cache Intel RDT monitoring.

Table 21-39. Leaf 0FH.01H Intel® Resource Director Technology (Intel® RDT) Monitoring

Register	Field Name	Description	Domain
EAX[7:0]	CTR_WIDTH	The counter width is encoded as an offset from 24b. A value of zero in this field indicates that 24-bit counters are supported. A value of 8 in this field indicates that 32-bit counters are supported.	Platform
EAX[8]	RDT_M_OVF	If 1, supports an overflow bit in the IA32_QM_CTR MSR (bit 61).	Platform
EAX[9]	IO_RDT_CMT	If 1, indicates the presence of non-CPU agent supporting Intel RDT CMT.	Platform
EAX[10]	IO_RDT_MBM	If 1, indicates the presence of non-CPU agent supporting Intel RDT MBM support.	Platform

EAX[31:11]	Reserved	Reserved.	
EBX[31:0]	CONV_FACTOR	Factor used to convert from reported IA32_QM_CTR value to derived occupancy metric (bytes) and Memory Bandwidth Monitoring (MBM) metrics.	Platform
ECX[31:0]	MAX_RMID_L3	Maximum range (zero-based) of RMID of this resource type.	Platform
EDX[0]	CMT_L3_OCCUP	If 1, supports L3 occupancy monitoring.	Platform
EDX[1]	MBM_L3_TOTAL	If 1, supports L3 total bandwidth monitoring.	Platform
EDX[2]	MBM_L3_LOCAL	If 1, supports L3 local bandwidth monitoring.	Platform
EDX[31:3]	Reserved	Reserved.	

CPUID.10H -- Intel® Resource Director Technology (Intel® RDT) Allocation

CPUID.10H returns information for Intel Resource Director Technology Allocation. This leaf is valid when CPUID.07H.00H:EBX.RDT_A[15] = 1. As described below, software uses the bit vector returned in EBX by subleaf 00H to determine the available QoS Enforcement (allocation) resource types that are supported in the processor. This information is necessary for software to configure each class of services using capability bit masks in the QoS Mask registers, IA32_resourceType_Mask_n.

- This leaf is valid if CPUID.07H.00H:EBX.RDT_A[15] = 1 and MAX_LEAF ≥ 10H.
- If the leaf is valid, sub-leaf 00H is always valid. Sub-leaf n (n ≥ 1) is only valid when (CPUID.10H.00H:EBX[n] == 1).

CPUID.10H.00H -- Intel® RDT Allocation Main Sub-Leaf

CPUID.10H.00H returns information about Intel RDT Allocation.

Table 21-40. Leaf 10H.00H Intel® Resource Director Technology (Intel® RDT) Allocation

Register	Field Name	Description	Domain
EAX[31:0]	Reserved	Reserved.	
EBX[0]	Reserved	Reserved.	
EBX[1]	CAT_L3	If 1, supports L3 Cache Allocation Technology. Sub-leaf index 0 reports valid resource identification (ResID) starting at bit position 1 of EBX.	Platform
EBX[2]	CAT_L2	If 1, supports L2 Cache Allocation Technology.	Platform
EBX[3]	MBA	If 1, supports Memory Bandwidth Allocation.	Platform
EBX[4]	Reserved	Reserved.	
EBX[5]	CBA	If 1, supports Cache Bandwidth Allocation.	Platform
EBX[6]	RESOURCE_PRIORITY	If 1, supports Resource Priority.	Platform
EBX[31:7]	Reserved	Reserved.	
ECX[31:0]	Reserved	Reserved.	
EDX[31:0]	Reserved	Reserved.	

CPUID.10H.00H returns information about the bit-vector representation of QoS Enforcement resource types that are supported in the processor. Each bit, starting from bit 1, corresponds to a specific resource type if the bit is set. The bit position corresponds to the sub-leaf index (or ResID) that software must use to query QoS enforcement capability available for that type.

CPUID.10H.01H -- L3 Cache Allocation Technology

CPUID.10H.ResID=1 returns information about L3 Cache Allocation Technology.

Table 21-41. Leaf 10H.01H Intel® Resource Director Technology (Intel® RDT) Allocation

Register	Field Name	Description	Domain
EAX[4:0]	CAT_L3_BITMASK_LENGTH	Length of the capacity bit mask for the corresponding ResID. Add one to the return value to get the result.	Platform
EAX[31:5]	Reserved	Reserved.	
EBX[31:0]	CAT_L3_CONTENTION	Bit-granular map of isolation/contention of allocation units.	Platform

ECX[0]	Reserved	Reserved.	
ECX[1]	CAT_L3_NONCPU	If 1, supports L3 CAT for non-CPU agents.	Platform
ECX[2]	CAT_L3_CDP	If 1, supports L3 Code and Data Prioritization Technology.	Platform
ECX[3]	CAT_L3_NONCONTIG	If 1, supports non-contiguous capacity bitmasks. The bits that are set in the various IA32_L3_MASK_n registers do not have to be contiguous.	Platform
ECX[31:4]	Reserved	Reserved.	
EDX[15:0]	CAT_L3_MAX_CLOS	Highest Class of Service (COS) number supported for this ResID.	Platform
EDX[31:16]	Reserved	Reserved.	

CPUID.10H.02H -- L2 Cache Allocation Technology

CPUID.10H.ResID=2 returns information about L2 Cache Allocation Technology.

Table 21-42. Leaf 10H.02H Intel® Resource Director Technology (Intel® RDT) Allocation

Register	Field Name	Description	Domain
EAX[4:0]	CAT_L2_BITMASK_LENGTH	Length of the capacity bit mask for the corresponding ResID. Add one to the return value to get the result.	Platform
EAX[31:5]	Reserved	Reserved.	
EBX[31:0]	CAT_L2_CONTENTION	Bit-granular map of isolation/contention of allocation units.	Platform
ECX[1:0]	Reserved	Reserved.	
ECX[2]	CAT_L2_CDP	If 1, supports L2 Code and Data Prioritization Technology.	Platform
ECX[3]	CAT_L2_NONCONTIG	If 1, supports non-contiguous capacity bitmasks. The bits that are set in the various IA32_L2_MASK_n registers do not have to be contiguous.	Platform
ECX[31:4]	Reserved	Reserved.	
EDX[15:0]	CAT_L2_MAX_CLOS	Highest Class of Service (COS) number supported for this ResID.	Platform
EDX[31:16]	Reserved	Reserved.	

CPUID.10H.03H -- Memory Bandwidth Allocation

CPUID.10H.ResID=3 returns information about Memory Bandwidth Allocation.

Table 21-43. Leaf 10H.03H Intel® Resource Director Technology (Intel® RDT) Allocation

Register	Field Name	Description	Domain
EAX[11:0]	MBA_MAX	Reports the maximum MBA throttling value supported for the corresponding ResID. Add one to the return value to get the result.	Platform
EAX[31:12]	Reserved	Reserved.	

EBX[31:0]	Reserved	Reserved.	
ECX[0]	PER_THREAD_MBA	Per-thread MBA controls are supported.	Platform
ECX[1]	Reserved	Reserved.	
ECX[2]	MBA_LINEAR	If 1, the response of the delay values is linear.	Platform
ECX[31:3]	Reserved	Reserved.	
EDX[15:0]	MBA_MAX_CLOS	Highest Class of Service (COS) number supported for this ResID.	Platform
EDX[31:16]	Reserved	Reserved.	

CPUID.10H.05H -- Cache Bandwidth Allocation

Table 21-44. Leaf 10H.05H Intel® Resource Director Technology (Intel® RDT) Allocation

Register	Field Name	Description	Domain
EAX[7:0]	CBA_MAX_LEVELS	Reports the maximum core throttling level supported for the corresponding ResID. Add one to the return value to get the number of throttling levels supported.	Platform
EAX[11:8]	BW_SCOPE	If 1, indicates the logical processor scope of the IA32_QoS_Core_BW_Thrtl_n MSRs. Other values are reserved.	Platform
EAX[31:12]	Reserved	Reserved.	
EBX[31:0]	Reserved	Reserved.	
ECX[2:0]	Reserved	Reserved.	
ECX[3]	CBA_LINEAR	If 1, the response of the bandwidth control is approximately linear. If 0, the response of the bandwidth control is non-linear.	Platform
ECX[31:4]	Reserved	Reserved.	
EDX[15:0]	CBA_MAX_CLOS	Highest Class of Service (COS) number supported for this ResID.	Platform
EDX[31:16]	Reserved	Reserved.	

CPUID.10H.06H -- Resource Priority Control

Table 21-45. Leaf 10H.06H Intel® Resource Director Technology (Intel® RDT) Allocation

Register	Field Name	Description	Domain
EAX[0]	THREAD_ENABLE	If 1, supports per-thread enable of RP through the IA32_RESOURCE_PRIORITY MSR.	Platform
EAX[1]	PACKAGE_ENABLE	If 1, supports physical processor package enable of RP through the IA32_RESOURCE PRIORITY_PKG MSR.	Platform
EAX[31:2]	Reserved	Reserved.	
EBX[31:0]	Reserved	Reserved.	
ECX[31:0]	Reserved	Reserved.	
EDX[31:0]	Reserved	Reserved.	

CPUID.11H -- Reserved

This leaf is reserved.

Table 21-46. Leaf 11H Reserved

Register	Field Name	Description	Domain
EAX[31:0]	Reserved	Reserved.	
EBX[31:0]	Reserved	Reserved.	
ECX[31:0]	Reserved	Reserved.	
EDX[31:0]	Reserved	Reserved.	

CPUID.12H -- Intel® Software Guard Extensions (Intel® SGX) Capability

CPUID.12H returns information about Intel® SGX capabilities. More details can be found in Chapter 35, "Introduction to Intel® Software Guard Extensions," and Chapter 36, "Enclave Access Control and Data Structures," of the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3D.

- This leaf is valid when CPUID.07H.00H:EBX.SGX[2] = 1 and MAX_LEAF ≥ 12H.
- If the leaf is valid, sub-leaf 00H and 01H are always valid. Sub-leaf n (n ≥ 2) is only valid when CPUID.12H.n:EAX[3:0] != 0.

CPUID.12H.00H -- Intel® SGX Main Sub-Leaf

CPUID.12H.00H returns information about Intel® SGX capabilities. It is only valid when CPUID.07H.00H:EBX.SGX = 1.

Table 21-47. Leaf 12H.00H Intel® Software Guard Extensions (Intel® SGX) Capability

Register	Field Name	Description	Domain
EAX[0]	SGX1	If 1, supports the collection of SGX1 Leaf functions.	Platform
EAX[1]	SGX2	If 1, supports the collection of SGX2 Leaf functions.	Platform
EAX[6:2]	Reserved	Reserved.	
EAX[7]	EVERIFYREPORT2	If 1, supports the ENCLU instruction Leaf EVERIFYREPORT2.	Platform
EAX[9:8]	Reserved	Reserved.	
EAX[10]	EUPDATESVN	If 1, supports the ENCLS instruction Leaf EUPDATESVN.	Platform
EAX[11]	EDECCSSA	If 1, supports the ENCLU instruction Leaf EDECCSSA.	Platform
EAX[31:12]	Reserved	Reserved.	
EBX[31:0]	MISCSELECT	Bit vector of supported extended SGX features. The definition of MISCSELECT can be found in Section 36.7.2, "SECS.MISCSELECT Field," of the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3D.	Platform
ECX[31:0]	Reserved	Reserved.	
EDX[7:0]	MAX_ENCLAVE_SIZE_NOT_64	The maximum supported enclave size in non-64-bit mode is 2^(EDX[7:0]).	Platform
EDX[15:8]	MAX_ENCLAVE_SIZE_64	The maximum supported enclave size in 64-bit mode is 2^(EDX[15:8]).	Platform
EDX[31:16]	Reserved	Reserved.	

CPUID.12H.01H -- Intel® SGX Attributes

CPUID.12H.01H returns information about Intel® SGX Attributes. It is only valid when CPUID.07H.00H:EBX.SGX = 1.

Table 21-48. Leaf 12H.01H Intel® Software Guard Extensions (Intel® SGX) Capability

Register	Field Name	Description	Domain

EAX[31:0]	ECREATE_SECS_ATTRIBUTES_31_0	Reports the valid bits of SECS.ATTRIBUTES[31:0] that software can set with ECREATE.	Platform
EBX[31:0]	ECREATE_SECS_ATTRIBUTES_63_32	Reports the valid bits of SECS.ATTRIBUTES[63:32] that software can set with ECREATE.	Platform
ECX[31:0]	ECREATE_SECS_ATTRIBUTES_95_64	Reports the valid bits of SECS.ATTRIBUTES[95:64] that software can set with ECREATE.	Platform
EDX[31:0]	ECREATE_SECS_ATTRIBUTES_127_96	Reports the valid bits of SECS.ATTRIBUTES[127:96] that software can set with ECREATE.	Platform

The definition of the attributes can be found in Section 36.7.1, "ATTRIBUTES," of the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3D

CPUID.12H -- n≥2 - Intel® SGX Enclave Page Cache

CPUID.12H with ECX≥2 returns information about Intel® SGX Enclave Page Cache and is supported if CPUID.07H.00H:EBX.SGX = 1.
For sub-leaves where ECX≥2, the definition of EAX[31:4], EBX, ECX, and EDX depends on the sub-leaf type listed below.

Sub-Leaf Encoding Type EAX[3:0] = 0000b (Invalid)

This sub-leaf is invalid. EDX:ECX:EBX:EAX return 0.

CPUID.12H -- Sub-Leaf Encoding Type EAX[3:0] = 0001b

This sub-leaf enumerates an EPC section with EDX:ECX, EBX:EAX defined as follows.

Table 21-49. Leaf 12H.SUB-LEAF ENCODING TYPE EAX[3:0] = 0001B Intel® Software Guard Extensions (Intel® SGX) Capability

Register	Field Name	Description	Domain
EAX[3:0]	SUB_LEAF_TYPE	Value is 0001b.	Platform
EAX[11:4]	Reserved	Reserved.	
EAX[31:12]	EPC_SECTION_ADDR_31_12	Bits 31:12 of the physical address of the base of the EPC section.	Platform
EBX[19:0]	EPC_SECTION_ADDR_51_32	Bits 51:32 of the physical address of the base of the EPC section.	Platform
EBX[31:20]	Reserved	Reserved.	

ECX[3:0]	EPC_SECTION_PROPERTY	EPC Section Property Encoding Definitions, as follows: 0000b – All bits in EDX:ECX are enumerated as 0. 0001b – This section has confidentiality, integrity, and replay protection. 0010b – This section has confidentiality protection only. 0011b - This section has confidentiality and integrity protection. All other encodings are reserved.	Platform
ECX[11:4]	Reserved	Reserved.	
ECX[31:12]	EPC_SECTION_SIZE_31_12	Bits 31:12 of the size of the corresponding EPC section within the Processor Reserved Memory.	Platform
EDX[19:0]	EPC_SECTION_SIZE_51_32	Bits 51:32 of the size of the corresponding EPC section within the Processor Reserved Memory.	Platform
EDX[31:20]	Reserved	Reserved.	

CPUID.13H -- Reserved

This leaf is reserved.

Table 21-50. Leaf 13H Reserved

Register	Field Name	Description	Domain
EAX[31:0]	Reserved	Reserved.	
EBX[31:0]	Reserved	Reserved.	
ECX[31:0]	Reserved	Reserved.	
EDX[31:0]	Reserved	Reserved.	

CPUID.14H -- Intel® Processor Trace (Intel® PT)

CPUID.14H returns information about Intel® Processor Trace (PT).
CPUID.14H.00H returns information about Intel Processor Trace extensions.
CPUID.14H.n (n > 0 and less than the number of non-zero bits in CPUID.14H.00H:EAX) returns information about packet generation in Intel Processor Trace.
For more details on Intel PT, see Chapter 34, "Intel® Processor Trace," in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3D.
• This leaf is valid when CPUID.07H.00H:EBX.INTEL_PROC_TRACE[25] = 1 and MAX_LEAF ≥ 14H.
• The maximum sub-leaf value for ECX is specified in CPUID.14H.00H.EAX[31:0] MAX_SUBLEAF.
• If ECX contains an invalid sub-leaf index, EAX/EBX/ECX/EDX return 0. Sub-leaf index n is invalid if n exceeds the value that sub-leaf 0 returns in EAX.

CPUID.14H.00H -- Intel® PT Main Sub-Leaf

CPUID.14H.00H returns information about Intel Processor Trace extensions.

Table 21-51. Leaf 14H.00H Intel® Processor Trace (Intel® PT)

Register	Field Name	Description	Domain
EAX[31:0]	MAX_SUBLEAF	Reports the maximum sub-leaf supported in leaf 14H.	Platform
EBX[0]	CR3_FILTER	If 1, supports that IA32_RTIT_CTL.CR3Filter can be set to 1, and that IA32_RTIT_CR3_MATCH MSR can be accessed.	Platform
EBX[1]	CYC_ACC	If 1, supports Configurable PSB and Cycle-Accurate Mode.	Platform
EBX[2]	IP_FILTER	If 1, supports IP Filtering, TraceStop filtering, and preservation of Intel PT MSRs across warm reset.	Platform
EBX[3]	MTC	If 1, supports MTC timing packet and suppression of COFI-based packets.	Platform
EBX[4]	PTWRITE	If 1, supports PTWRITE. Writes can set IA32_RTIT_CTL[12] (PTWEn) and IA32_RTIT_CTL[5] (FUPonPTW), and PTWRITE can generate packets.	Platform
EBX[5]	PWR_EVT_TRACE	If 1, supports Power Event Trace. Writes can set IA32_RTIT_CTL[4] (PwrEvtEn), enabling Power Event Trace packet generation.	Platform
EBX[6]	PMI_PRESERVE	If 1, supports the PSB and PMI preservation. Writes can set IA32_RTIT_CTL[56] (InjectPsbPmiOnEn- able), enabling the processor to set IA32_RTIT_STATUS[7] (PendTopaPMI) and/or IA32_RTIT_STATUS[6] (PendPSB) in order to preserve ToPA PMIs and/or PSBs otherwise lost due to Intel PT disable. Writes can also set PendToPAPMI and PendPSB.	Platform
EBX[7]	EVENT_TRACE	If 1, supports that writes can set IA32_RTIT_CTL[31] (EventEn), enabling Event Trace packet generation.	Platform
EBX[8]	TNT_DIS	If 1, supports that writes can set IA32_RTIT_CTL[55] (DisTNT), disabling TNT packet generation.	Platform

EBX[9]	PTTT	If 1, Processor Trace Trigger Tracing (PTTT) is supported.	Platform
EBX[31:10]	Reserved	Reserved.	
ECX[0]	TOPAOUT	If 1, supports that tracing can be enabled with IA32_RTIT_CTL.ToPA = 1, hence utilizing the ToPA output scheme; IA32_RTIT_OUTPUT_BASE and IA32_RTIT_OUPUT_MASK_PTRS MSRs can be accessed.	Platform
ECX[1]	MENTRY	If 1, supports that ToPA tables can hold any number of output entries, up to the maximum allowed by the MaskOrTableOffset field of IA32_RTIT_OUTPUT_MASK_PTRS.	Platform
ECX[2]	SNGL_RNG_OUT	If 1, supports the Single-Range Output scheme.	Platform
ECX[3]	TRACE_TRANSPORT_SUBSYSTEM	If 1, supports the output to Trace Transport subsystem.	Platform
ECX[30:4]	Reserved	Reserved.	
ECX[31]	LIP	If 1, the generated packets which contain IP payloads contain LIP. If 0, the generated packets which contain IP payloads contain Effective IP. Trace segments using a flat memory model will generate the same information regardless of how a logical processor reports this value since LIP=EIP.	Logical Processor
EDX[31:0]	Reserved	Reserved.	

CPUID.14H.01H -- Feature Information Sub-Leaf

CPUID.14H.01H returns information about packet generation in Intel Processor Trace.

Table 21-52. Leaf 14H.01H Intel® Processor Trace (Intel® PT)

Register	Field Name	Description	Domain
EAX[2:0]	RANGECNT	Number of configurable Address Ranges for filtering.	Platform
EAX[7:3]	Reserved	Reserved.	
EAX[10:8]	TRIGGER_CFG_CNT	Number of IA32_RTIT_TRIGGERx_CFG MSRs. The number of triggers supported is 4x this value.	Platform
EAX[15:11]	Reserved	Reserved.	
EAX[31:16]	MTC_RATE	Bitmap of supported MTC period encodings.	Platform
EBX[15:0]	CYC_THRESHOLDS	Bitmap of supported Cycle Threshold value encodings.	Platform
EBX[31:16]	PSB_RATE	Bitmap of supported Configurable PSB frequency encodings.	Platform
ECX[0]	ICNT	If 1, the trigger action EN_ICNT is supported.	Platform
ECX[1]	TRIGGER_PAUSE	If 1, the trigger actions TRACE_PAUSE and TRACE_RESUME are supported.	Platform
ECX[14:2]	Reserved	Reserved.	

ECX[15]	TRIGGER_DR_MATCH	If 1, trigger input DR match is supported.	Platform
ECX[31:16]	Reserved	Reserved.	
EDX[31:0]	Reserved	Reserved.	

CPUID.15H -- Time Stamp Counter and Nominal Core Crystal Clock

CPUID.15H returns information about the Time Stamp Counter and the Nominal Core Crystal Clock.
- This leaf is valid if MAX_LEAF ≥ 15H.
- This leaf does not contain sub-leaves and provides the same information regardless of the value of ECX.

Table 21-53. Leaf 15H Time Stamp Counter and Nominal Core Crystal Clock

Register	Field Name	Description	Domain
EAX[31:0]	DENOMINATOR	An unsigned integer which is the denominator of the TSC/"core crystal clock" ratio.	Platform
EBX[31:0]	NUMERATOR	An unsigned integer which is the numerator of the TSC/"core crystal clock" ratio. If 0, the TSC/"core crystal clock" ratio is not enumerated.	Platform
ECX[31:0]	NOMINAL_ART_FREQUENCY	An unsigned integer which is the nominal frequency of the core crystal clock in Hz. If 0, the nominal core crystal clock frequency is not enumerated. Note, the core crystal clock may differ from the reference clock, bus clock or core clock frequencies.	Platform
EDX[31:0]	Reserved	Reserved.	

Dividing EBX[31:0] by EAX[31:0] provides the ratio of the TSC frequency to the core crystal clock frequency. The Timestamp Counter frequency is computed by multiplying that ratio by ECX[31:0], such as TSC_frequency = ECX * EBX/EAX.

CPUID.16H -- Processor Frequency Information

CPUID.16H returns information about processor frequency information.
Data is returned from this interface in accordance with the processor's specification and does not reflect actual values. Suitable use of this data includes the display of processor information in like manner to the processor brand string and for determining the appropriate range to use when displaying processor information e.g. frequency history graphs. The returned information should not be used for any other purpose as the returned information does not accurately correlate to information / counters returned by other processor interfaces.
While a processor may support the Processor Frequency Information leaf, fields that return a value of zero are not supported.

- This leaf is valid if MAX_LEAF ≥ 16H.
- This leaf does not contain sub-leaves and provides the same information regardless of the value of ECX.

Table 21-54. Leaf 16H Processor Frequency Information

Register	Field Name	Description	Domain
EAX[15:0]	PROCESSOR_BASE_FREQUENCY	Processor Base Frequency (in MHz).	Logical Processor
EAX[31:16]	Reserved	Reserved.	
EBX[15:0]	MAXIMUM_FREQUENCY	Maximum Frequency (in MHz).	Logical Processor
EBX[31:16]	Reserved	Reserved.	
ECX[15:0]	BUS_FREQUENCY	Bus (Reference) Frequency (in MHz).	Logical Processor
ECX[31:16]	Reserved	Reserved.	
EDX[31:0]	Reserved	Reserved.	

CPUID.17H -- System-on-Chip Vendor Attribute

CPUID.17H returns System-on-Chip vendor attribute information.
• This leaf is valid if CPUID.17H.00H:EAX[31:0] (MaxSOCID_Index) ≥ 3 and MAX_LEAF ≥ 17H.
• The maximum sub-leaf value for ECX is specified in CPUID.17H.00H.EAX[31:0] MaxSOCID_Index.
• If ECX contains an invalid sub-leaf index, EAX/EBX/ECX/EDX return 0. Sub-leaf index n is invalid if n exceeds the value that sub-leaf 0 returns in EAX.

CPUID.17H.00H -- Main Sub-Leaf

CPUID.17H.00H returns System-on-Chip vendor attribute information.

Table 21-55. Leaf 17H.00H System-on-Chip Vendor Attribute

Register	Field Name	Description	Domain
EAX[31:0]	MAX_SOCID_INDEX	Reports the maximum input value of supported Sub-leaf in Leaf 17H.	Platform
EBX[15:0]	SOC_VENDOR_ID	SOC Vendor ID.	Platform
EBX[16]	IS_VENDOR_SCHEME	If 1, the SOC Vendor ID field is assigned via an industry standard enumeration scheme. Otherwise, the SOC Vendor ID field is assigned by Intel.	Platform
EBX[31:17]	Reserved	Reserved.	
ECX[31:0]	PROJECT_ID	A unique number an SOC vendor assigns to its SOC projects.	Platform
EDX[31:0]	STEPPING_ID	A unique number within an SOC project that an SOC vendor assigns.	Package

CPUID.17H.01H -- Vendor Brand String Sub-Leaf (Bytes 0 to 15)

CPUID.17H with ECX=1, 2, or 3 returns information for System-on-Chip vendor branding string.
SOC Vendor Brand String is a UTF-8 encoded string padded with trailing bytes of 00H. The complete SOC Vendor Brand String is constructed by concatenating in ascending order the output of each sub-leaf 1 to 3. For the output of each sub-leaf, byte are ordered in ascending order of EAX:EBX:ECX:EDX.
Leaf 17H sub-leaves 4 and above are reserved.

Table 21-56. Leaf 17H.01H System-on-Chip Vendor Attribute

Register	Field Name	Description	Domain
EAX[31:0]	VENDOR_BRAND_STRING_BYTES_0_to_3	SOC Vendor Brand String. UTF-8 encoded string.	Platform
EBX[31:0]	VENDOR_BRAND_STRING_BYTES_4_to_7	SOC Vendor Brand String. UTF-8 encoded string.	Platform
ECX[31:0]	VENDOR_BRAND_STRING_BYTES_8_to_11	SOC Vendor Brand String. UTF-8 encoded string.	Platform
EDX[31:0]	VENDOR_BRAND_STRING_BYTES_12_to_15	SOC Vendor Brand String. UTF-8 encoded string.	Platform

CPUID.17H.02H -- Vendor Brand String Sub-Leaf (Bytes 16 to 31)

Table 21-57. Leaf 17H.02H System-on-Chip Vendor Attribute

Register	Field Name	Description	Domain
EAX[31:0]	VENDOR_BRAND_STRING_BYTES_16_to_19	SOC Vendor Brand String. UTF-8 encoded string.	Platform
EBX[31:0]	VENDOR_BRAND_STRING_BYTES_20_to_23	SOC Vendor Brand String. UTF-8 encoded string.	Platform

ECX[31:0]	VENDOR_BRAND_STRING_BYTES_24_to_27	SOC Vendor Brand String. UTF-8 encoded string.	Platform
EDX[31:0]	VENDOR_BRAND_STRING_BYTES_28_to_31	SOC Vendor Brand String. UTF-8 encoded string.	Platform

CPUID.17H.03H -- Vendor Brand String Sub-Leaf (Bytes 32 to 47)

Table 21-58. Leaf 17H.03H System-on-Chip Vendor Attribute

Register	Field Name	Description	Domain
EAX[31:0]	VENDOR_BRAND_STRING_BYTES_32_to_35	SOC Vendor Brand String. UTF-8 encoded string.	Platform
EBX[31:0]	VENDOR_BRAND_STRING_BYTES_36_to_39	SOC Vendor Brand String. UTF-8 encoded string.	Platform
ECX[31:0]	VENDOR_BRAND_STRING_BYTES_40_to_43	SOC Vendor Brand String. UTF-8 encoded string.	Platform
EDX[31:0]	VENDOR_BRAND_STRING_BYTES_44_to_47	SOC Vendor Brand String. UTF-8 encoded string.	Platform

CPUID.17H.M>MAXSOCID_INDEX—RESERVED SUB-LEAVES -- m>MaxSOCID_Index—Reserved Sub-Leaves

CPUID.17H with ECX > MaxSOCID_Index is reserved and returns all zeroes.

Table 21-59. Leaf 17H.M>MAXSOCID_INDEX—RESERVED SUB-LEAVES System-on-Chip Vendor Attribute

Register	Field Name	Description	Domain
EAX[31:0]	Reserved	Reserved	
EBX[31:0]	Reserved	Reserved	
ECX[31:0]	Reserved	Reserved	
EDX[31:0]	Reserved	Reserved	

CPUID.18H -- Deterministic Address Translation Parameters

CPUID.18H returns information about the Deterministic Address Translation Parameters. Each sub-leaf enumerates a different address translation structure.

- This leaf is valid if CPUID.18H.00H:EAX[31:0] <> 0 and MAX_LEAF ≥ 18H.
- The maximum sub-leaf value for ECX is specified in CPUID.18H.00H.EAX[31:0] MAX_SUBLEAF.
- If ECX contains an invalid sub-leaf index, EAX/EBX/ECX/EDX return 0. Sub-leaf index n is invalid if n exceeds the value that sub-leaf 0 returns in EAX. A sub-leaf index is also invalid if EDX[4:0] returns 0.
- Valid sub-leaves do not need to be contiguous or in any particular order. A valid sub-leaf may be in a higher input ECX value than an invalid sub-leaf or than a valid sub-leaf of a higher or lower-level structure.

CPUID.18H.00H -- Main Sub-Leaf

Deterministic Address Translation Parameters Main Leaf

Table 21-60. Leaf 18H.00H Deterministic Address Translation Parameters

Register	Field Name	Description	Domain
EAX[31:0]	MAX_SUBLEAF	Reports the maximum input value of supported sub-leaf in leaf 18H.	Logical Processor
EBX[31:0]	Reserved	Reserved.	
ECX[31:0]	Reserved	Reserved.	
EDX[4:0]	TYPE	Will always return 0.	Logical Processor
EDX[31:5]	Reserved	Reserved.	

CPUID.18H.ECX >= 1 -- Sub-Leaves

CPUID.18H with ECX≥1 returns Deterministic Address Translation Parameters information.

Table 21-61. Leaf 18H.ECX >= 1 Deterministic Address Translation Parameters

Register	Field Name	Description	Domain
EAX[31:0]	Reserved	Reserved.	
EBX[0]	4KB_ENTRIES	If 1, supports 4K page size entries in this structure.	Logical Processor
EBX[1]	2MB_ENTRIES	If 1, supports 2MB page size entries in this structure.	Logical Processor
EBX[2]	4MB_ENTRIES	If 1, supports 4MB page size entries in this structure.	Logical Processor
EBX[3]	1GB_ENTRIES	If 1, supports 1 GB page size entries in this structure.	Logical Processor
EBX[7:4]	Reserved	Reserved.	
EBX[10:8]	PARTITIONING	Partitioning (0:Soft partitioning between the logical processors sharing this structure).	Logical Processor
EBX[15:11]	Reserved	Reserved.	
EBX[31:16]	NUM_WAYS	W = ways of associativity.	Logical Processor
ECX[31:0]	NUM_SETS	S = number of sets.	Logical Processor

EDX[4:0]	TYPE	00000b: Null (indicates this Sub-leaf is not valid). 00001b: Data TLB. 00010b: Instruction TLB. 00011b: Unified TLB.1 00100b: Load Only TLB. Hit on loads; fills on both loads and stores. 00101b: Store Only TLB. Hit on stores; fill on stores. All other encodings are reserved. Some unified TLBs will allow a single TLB entry to satisfy data read/write and instruction fetches. Others will require separate entries (e.g., one loaded on data read/write and another loaded on an instruction fetch). See the Intel® 64 and IA-32 Architectures Optimization Reference Manual for details of a particular product.	Logical Processor
EDX[7:5]	LEVEL_NUM	Translation cache level (starts at 1).	Logical Processor
EDX[8]	FULLY_ASSOC	Fully associative structure.	Logical Processor
EDX[13:9]	Reserved	Reserved.	
EDX[25:14]	MAX_LP_ADDRESSABLE_IDS	Maximum number of addressable IDs for logical processors sharing this translation cache. Add one to the return value to get the result.	Logical Processor
EDX[31:26]	Reserved	Reserved.	

CPUID.19H -- Key Locker

CPUID.19H returns Key Locker information.

- This leaf is valid if CPUID.07H.00H:ECX.KEY_LOCKER[23] = 1 and MAX_LEAF ≥ 19H.
- This leaf does not contain sub-leaves and provides the same information regardless of the value of ECX.

Table 21-62. Leaf 19H Key Locker

Register	Field Name	Description	Domain
EAX[0]	CPL0_RESTRICT	If 1, supports the Key Locker restriction of CPL0-only.[1]	Platform
EAX[1]	NO_ENCRYPT_RESTRICT	If 1, supports the Key Locker restriction of no-encrypt.[1]	Platform
EAX[2]	NO_DECRYPT_RESTRICT	If 1, supports the Key Locker restriction of no-decrypt.[1]	Platform
EAX[31:3]	Reserved	Reserved.	
EBX[0]	AESKLE	If 1, the AES Key Locker instructions are fully enabled. CPUID.19H:EBX.AESKLE[0] is enumerated as 1 if the AES Key Locker instructions have been activated by system firmware and CR4.KL[bit 19] = 1. Software can check this bit after setting CR4.KL to determine whether AES Key Locker instructions have been enabled. Note that some processors may allow enabling of those instructions without activation by system firmware. Some processors may not support the use of AES Key Locker instructions in system-management-mode (SMM). Those processors enumerate CPUID.19H:EBX.AESKLE[0] as 0 in SMM regardless of the setting of CR4.KL.	Logical Processor
EBX[1]	Reserved	Reserved.	
EBX[2]	AES_WIDE	If 1, supports the AES wide Key Locker instructions.[1]	Platform
EBX[3]	Reserved	Reserved.	
EBX[4]	IWKEYBACKUP	If 1, supports the Key Locker MSRs (IA32_COPY_LOCAL_TO_PLATFORM, IA23_COPY_PLATFORM_TO_LOCAL, IA32_COPY_STATUS, and IA32_IWKEYBACKUP_STATUS) and backing up the internal wrapping key.[1]	Platform
EBX[31:5]	Reserved	Reserved.	
ECX[0]	NOBACKUP	If 1, supports the NoBackup parameter to LOADIWKEY.[1]	Platform
ECX[1]	RAND_IWKEY	If 1, supports KeySource encoding of 1 (randomization of the internal wrapping key).[1]	Platform
ECX[31:2]	Reserved	Reserved.	
EDX[31:0]	Reserved	Reserved.	

NOTE

1. This field is valid only if CPUID.19H:EBX.AESKLE[0] = 1.

CPUID.1AH -- Native Model ID Enumeration

CPUID.1AH returns Native Model ID information.
This leaf exists on all logical processors in a hybrid package, it may also be present in other processor configurations.

- This leaf is valid if CPUID.1AH.00H:EAX[31:0] <> 0 and MAX_LEAF ≥ 1AH.
- The only valid sub-leaf is 0 and ECX must be set to 0.

Table 21-63. Leaf 1AH Native Model ID Enumeration

Register	Field Name	Description	Domain
EAX[23:0]	CORE_NATIVE_MODEL_ID	The core-type and native model ID can be used to uniquely identify the microarchitecture of the core. This native model ID is not unique across core types, and not related to the model ID reported in CPUID.01H, and does not identify the SOC.	Logical Processor
EAX[31:24]	CORE_TYPE	10H: Reserved 20H: Intel® Atom® 30H: Reserved 40H: Intel® Core The core type may only be used as an identification of the microarchitecture for this logical processor and its numeric value has no significance, neither large nor small. This field neither implies nor expresses any other attribute to this logical processor and software should not assume any.	Logical Processor
EBX[31:0]	Reserved	Reserved.	
ECX[31:0]	Reserved	Reserved.	
EDX[31:0]	Reserved	Reserved.	

CPUID.1BH -- PCONFIG Information

CPUID.1BH -- Output Registers Format for All Sub-Leaves

CPUID.1BH returns information for PCONFIG capabilities. This information is enumerated in sub-leaves selected by the value of ECX (starting with 0).

- This leaf is valid if CPUID.07H.00H:EDX.PCONFIG[18] = 1 and MAX_LEAF ≥ 1BH.
- Sub-leaves are enumerated until sub-leaf n, where EAX[11:0] returns 0.

Table 21-64. Leaf 1BH PCONFIG Information

Register	Field Name	Description	Domain
EAX[11:0]	SUB_LEAF_TYPE	0 (Invalid)	Platform
EAX[31:12]	Reserved	Reserved.	
EBX[31:0]	Reserved	Reserved.	
ECX[31:0]	Reserved	Reserved.	
EDX[31:0]	Reserved	Reserved.	

Each sub-leaf of CPUID.1BH enumerates its sub-leaf type in EAX. If a sub-leaf type is 0, the sub-leaf is invalid and zero is returned in EBX, ECX, and EDX. In this case, all subsequent sub-leaves (selected by larger input values of ECX) are also invalid.

The only valid sub-leaf type currently defined is 1, indicating that the sub-leaf enumerates target identifiers for the PCONFIG instruction. Any non-zero value returned in EBX, ECX, or EDX indicates a valid target identifier of the PCONFIG instruction (any value of zero should be ignored). The only target identifier currently defined is 1, indicating TME-MK. See the "PCONFIG—Platform Configuration" instruction in Chapter 4 of the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 2B, for more information.

CPUID.1BH.OUTPUT REGISTERS FOR SUB-LEAVE TYPE TARGET IDENTIFIER (1) -- Output Registers for Sub-Leave Type Target Identifier (1)

Table 21-65. Leaf 1BH.OUTPUT REGISTERS FOR SUB-LEAVE TYPE TARGET IDENTIFIER (1) PCONFIG Information

Register	Field Name	Description	Domain
EAX[11:0]	SUB_LEAF_TYPE	1 (Target Identifier)	Platform
EAX[31:12]	Reserved	Reserved.	
EBX[31:0]	TARGET_IDENTIFIER_1	Target identifier	Platform
ECX[31:0]	TARGET_IDENTIFIER_2	Target identifier	Platform
EDX[31:0]	TARGET_IDENTIFIER_3	Target identifier	Platform

The only current valid target identifier is 1 for MK-TME.

CPUID.1CH -- Last Branch Records (LBR) Information

CPUID.1CH returns information about architectural Last Branch Records (LBR). For details on LBR, see Chapter 20, "Last Branch Records," in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3B.

- This leaf is valid if CPUID.07H.00H:EDX.ARCH_LBRS[19] = 1 and MAX_LEAF ≥ 1CH.
- This leaf does not contain sub-leaves and provides the same information regardless of the value of ECX.

Table 21-66. Leaf 1CH Last Branch Records (LBR) Information

Register	Field Name	Description	Domain
EAX[7:0]	LBR_DEPTH_VALUES	For each bit n set in this field, the IA32_LBR_DEPTH.DEPTH value 8*(n+1) is supported.	Platform
EAX[29:8]	Reserved	Reserved.	
EAX[30]	DEEP_C_STATE_RESET	If 1, supports that LBRs may be cleared on an MWAIT that requests a C-state numerically greater than C1.	Platform
EAX[31]	IP_VALUES_CONTAIN_LIP	If 1, the LBR IP values contain LIP. If 0, IP values contain Effective IP. Trace segments using a flat memory model will generate the same information regardless of how a logical processor reports this value since LIP = EIP	Logical Processor
EBX[0]	CPL_FILTERING	If 1, supports setting IA32_LBR_CTL[2:1] to a non-zero value.	Platform
EBX[1]	BRANCH_FILTERING	If 1, supports setting IA32_LBR_CTL[22:16] to a non-zero value.	Platform
EBX[2]	CALL_STACK_MODE	If 1, supports setting IA32_LBR_CTL[3] to 1.	Platform
EBX[31:3]	Reserved	Reserved.	
ECX[0]	MISPREDICT_BIT	If 1, IA32_LBR_x_INFO[63] holds indication of branch misprediction (MISPRED).	Platform
ECX[1]	TIMED_LBRS	If 1, IA32_LBR_x_INFO[15:0] holds CPU cycles since last LBR entry (CYC_CNT), and IA32_LBR_x_INFO[60] holds an indication of whether the value held there is valid (CYC_CNT_VALID).	Platform
ECX[2]	BRANCH_TYPE_FIELD_SUPPORTED	If 1, IA32_LBR_INFO_x[59:56] holds indication of the recorded operation's branch type (BR_TYPE).	Platform
ECX[15:3]	Reserved	Reserved.	
ECX[19:16]	EVENT_LOGGING_BITMAP	The event logging bitmap, wherein each set bit corresponds to a programmable performance monitoring counter that supports LBR event logging.	Platform
ECX[31:20]	Reserved	Reserved.	
EDX[31:0]	Reserved	Reserved.	

CPUID.1DH -- Tile Information

CPUID.1DH returns information about tile architecture and tile palette 1 (see Chapter 19, "Programming with Intel® Advanced Matrix Extensions," in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 1).

- This leaf is valid if CPUID.07H.00H:EDX.AMX_TILE[24] = 1 and MAX_LEAF ≥ 1DH.
- The maximum sub-leaf value for ECX is specified in CPUID.1DH.00H.EAX[31:0] max_palette.
- If ECX contains an invalid sub-leaf index, EAX/EBX/ECX/EDX return 0. Sub-leaf index n is invalid if n exceeds the value that sub-leaf 0 returns in EAX.

CPUID.1DH.00H -- Tile Information Main Sub-Leaf

CPUID.1DH.00H returns the tile architecture information.

Table 21-67. Leaf 1DH.00H Tile Information

Register	Field Name	Description	Domain
EAX[31:0]	MAX_PALETTE	Highest numbered palette sub-leaf. Value = 1.	Platform
EBX[31:0]	Reserved	Reserved.	
ECX[31:0]	Reserved	Reserved.	
EDX[31:0]	Reserved	Reserved.	

CPUID.1DH.01H -- Tile Palette 1

CPUID.1DH.01H returns tile palette information.

Table 21-68. Leaf 1DH.01H Tile Information

Register	Field Name	Description	Domain
EAX[15:0]	TOTAL_TILE_BYTES	Palette 1 total_tile_bytes. Value = 8192.	Platform
EAX[31:16]	BYTES_PER_TILE	Palette 1 bytes_per_tile. Value = 1024.	Platform
EBX[15:0]	BYTES_PER_ROW	Palette 1 bytes_per_row. Value = 64.	Platform
EBX[31:16]	MAX_NAMES	Palette 1 max_names (number of tile registers). Value = 8.	Platform
ECX[15:0]	MAX_ROWS	Palette 1 max_rows. Value = 16.	Platform
ECX[31:16]	Reserved	Reserved.	
EDX[31:0]	Reserved	Reserved.	

CPUID.1EH -- TMUL Information

CPUID.1EH returns information about TMUL capabilities (see Chapter 19, "Programming with Intel® Advanced Matrix Extensions," in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 1).

- This leaf is valid if CPUID.07H.00H:EDX.AMX_TILE[24] = 1 and MAX_LEAF ≥ 1EH.
- The only valid sub-leaf is 0 and ECX must be set to 0.

CPUID.1EH.00H -- TMUL Information Main Leaf

TMUL Main Leaf Information

Table 21-69. Leaf 1EH.00H TMUL Information

Register	Field Name	Description	Domain
EAX[31:0]	Reserved	Reserved.	
EBX[7:0]	TMUL_MAXK	tmul_maxk (rows or columns). Value = 16.	Platform
EBX[23:8]	TMUL_MAXN	tmul_maxn (column bytes). Value = 64.	Platform
EBX[31:24]	Reserved	Reserved.	
ECX[31:0]	Reserved	Reserved.	
EDX[31:0]	Reserved	Reserved.	

CPUID.1FH -- V2 Extended Topology

CPUID.1FH returns information about V2 Extended Topology.
CPUID.1FH is a preferred superset to leaf 0BH. Intel recommends using leaf 1FH when available rather than leaf 0BH and ensuring that any leaf 0BH algorithms are updated to support leaf 1FH.

- This leaf is valid if CPUID.1FH.00H:EBX[15:0] <> 0 and MAX_LEAF ≥ 1FH.
  - When the leaf is invalid, CPUID.1FH.00H:ECX.DOMAIN_TYPE[15:8] will report the Domain Type ID as Invalid (0).
- The sub-leaves are enumerated until sub-leaf n returns 0 in EBX[15:0].
- If ECX contains an invalid sub-leaf index, EAX/EBX return 0. Sub-leaf index n+1 is invalid if sub-leaf n returns EBX[15:0] as 0.

CPUID.1FH -- ECX >= 0

Table 21-70. Leaf 1FH V2 Extended Topology

Register	Field Name	Description	Domain
EAX[4:0]	SHIFT_COUNT	The number of bits that the x2APIC ID must be shifted to the right to address instances of the next higher-scoped domain. When logical processor is not supported by the processor, the value of this field at the Logical Processor domain sub-leaf may be returned as either 0 (no allocated bits in the x2APIC ID) or 1 (one allocated bit in the x2APIC ID); software should plan accordingly.	Platform
EAX[31:5]	Reserved	Reserved.	
EBX[15:0]	NEXT_LEVEL_NUM_LP	The number of logical processors across all instances of this domain within the next higher-scoped domain relative to this current logical processor. (For example, in a processor socket/package comprising "M" dies of "N" cores each, where each core has "L" logical processors, the "die" domain sub-leaf value of this field would be M*N*L. In an asymmetric topology this would be the summation of the value across the lower domain level instances to create each upper domain level instance.) This number reflects configuration as shipped by Intel. Note that the number of logical processors can be asymmetric in which case "L" may be different on different logical processors, as an example a core with 2 logical processors on the same platform as a core with 1 logical processor. Note, software must not use this field to enumerate processor topology. Software must not use the value of EBX[15:0] to enumerate processor topology of the system. The value is only intended for display and diagnostic purposes. The actual number of logical processors available to BIOS/OS/Applications may be different from the value of EBX[15:0], depending on software and platform hardware configurations.	Logical Processor
EBX[31:16]	Reserved	Reserved.	

ECX[7:0]	LEVEL_NUM	The input ECX sub-leaf index.	Platform
ECX[15:8]	DOMAIN_TYPE	This field provides an identification value which indicates the domain as shown in the table. Although domains are ordered, their assigned identification values are not and software should not depend on it. (For example, if a new domain between core and module is specified, it will have an identification value higher than 5.) See the table below for the current list of valid enumerations. Note that enumeration values of 0 and 7-255 are reserved.	Platform
ECX[31:16]	Reserved	Reserved.	
EDX[31:0]	X2APIC_ID	The x2APIC ID of the current logical processor is always valid and does not vary with the sub-leaf index in ECX.	Logical Processor

The sub-leaves of CPUID.1FH describe an ordered hierarchy of logical processors starting from the smallest scoped domain of a Logical Processor (sub-leaf index 0) to the Core domain (sub-leaf index 1) to the largest scoped domain (the last valid sub-leaf index) that is implicitly subordinate to the unenumerated highest-scoped domain of the processor package (socket).
The details of each valid domain is enumerated by a corresponding sub-leaf. Details for a domain include its type and how all instances of that domain determine the number of logical processors and x2 APIC ID partitioning at the next higher-scoped domain. The ordering of domains within the hierarchy is fixed architecturally as shown below. For a given processor, not all domains may be relevant or enumerated; however, the logical processor and core domains are always enumerated. As an example, a processor may report an ordered hierarchy consisting only of "Logical Processor," "Core," and "Die."
For two valid sub-leaves N and N+1, sub-leaf N+1 represents the next immediate higher-scoped domain with respect to the domain of sub-leaf N for the given processor.
If sub-leaf index "N" returns an invalid domain type in ECX[15:08] (00H), then all sub-leaves with an index greater than "N" also return an invalid domain type. A sub-leaf returning an invalid domain always returns 0 in EAX and EBX.

Table 21-71. Hierarchy of Valid Domain Enumerations in CPUID.1FH:ECX[15:8]

Hierarchy	Domain	Domain Type ID Value
Invalid	Invalid	0
Lowest	Logical Processor	1
...	Core	2
...	Module	3
...	Tile	4
...	Die	5
...	DieGrp	6
Highest	Package/Socket	(Implied)
Reserved	Reserved	7-255

CPUID.20H -- Processor History Reset Information

CPUID.20H returns information about processor history reset when CPUID.07H.01H:EAX.HRESET[22] = 1.
• This leaf is valid if CPUID.07H.01H:EAX.HRESET[22] = 1 and MAX_LEAF ≥ 20H.
• The maximum sub-leaf value for ECX is specified in CPUID.20H.00H.EAX[31:0] MAX_SUBLEAF.
• If ECX contains an invalid sub-leaf index, EAX/EBX/ECX/EDX return 0. Sub-leaf index n is invalid if n exceeds the value that sub-leaf 0 returns in EAX.

CPUID.20H.00H -- Processor History Reset Sub-leaf

Table 21-72. Leaf 20H.00H Processor History Reset Information

Register	Field Name	Description	Domain
EAX[31:0]	MAX_SUBLEAF	Reports the maximum number of sub-leaves that are supported in leaf 20H.	Platform
EBX[0]	THREAD_DIRECTOR_HRESET	Indicates support for both HRESET's EAX[0] parameter, and IA32_HRESET_ENABLE[0] set by the OS to enable reset of Intel® Thread Director history.	Platform
EBX[31:1]	Reserved	Reserved.	
ECX[31:0]	Reserved	Reserved.	
EDX[31:0]	Reserved	Reserved.	

CPUID.21H -- Unimplemented

Does not return feature information for the processor. Allocated for use by TDX modules; see Intel® Trust Domain Extensions (Intel® TDX) Module Base Architecture Specification. Software emulating CPUID should not change the information returned for this leaf.

CPUID.22H -- Reserved

This leaf is reserved.

Table 21-73. Leaf 22H Reserved

Register	Field Name	Description	Domain
EAX[31:0]	Reserved	Reserved.	
EBX[31:0]	Reserved	Reserved.	
ECX[31:0]	Reserved	Reserved.	
EDX[31:0]	Reserved	Reserved.	

CPUID.23H -- Architectural Performance Monitoring Extended

CPUID.23H returns architectural performance monitoring extended information.
• This leaf is valid if CPUID.07H.01H:EAX.ARCH_PERFMON_EXT[8] = 1 and MAX_LEAF ≥ 23H.
• The sub-leaves of this leaf are enumerated by a bitmask specified in CPUID.23H.00H.EAX[31:0] SUBLEAF_MASK. The bit numbers of set bits in the bitmask represent valid sub-leaf indexes.
• If ECX contains an invalid sub-leaf index, EAX/EBX/ECX/EDX return 0. Sub-leaf index is invalid if the index as a bit number is clear in the Available Sub-Leaf Mask or is greater than 31.

CPUID.23H.00H -- Main Sub-Leaf

Table 21-74. Leaf 23H.00H Architectural Performance Monitoring Extended

Register	Field Name	Description	Domain
EAX[31:0]	SUBLEAF_MASK	If bit n is set, sub-leaf n is supported. (For unsupported sub-leaves, 0 is returned in the registers EAX, EBX, ECX, and EDX.)	Logical Processor
EBX[0]	UNITMASK2	If 1, supports the UnitMask2 field in the IA32_PERFEVTSELx MSRs.	Logical Processor
EBX[1]	EQ	If 1, supports the equal flag in the IA32_PERFEVTSELx MSRS.	Logical Processor
EBX[31:2]	Reserved	Reserved.	
ECX[7:0]	SLOTS_PER_CYC	If this field is non-zero, it represents the number of Top-down Microarchitecture Analysis (TMA) slots per cycle. This number can be multiplied by the number of cycles (from CPU_CLK_UNHALTED.THREAD / CPU_CLK_UNHALTED.CORE or IA32_FIXED_CTR1) to determine the total number of slots. If this field is zero, IA32_FIXED_CTR3 should be used to determine the total number of slots.	Logical Processor
ECX[31:8]	Reserved	Reserved.	
EDX[31:0]	Reserved	Reserved.	

CPUID.23H.01H -- Counter Information Sub-Leaf

Table 21-75. Leaf 23H.01H Architectural Performance Monitoring Extended

Register	Field Name	Description	Domain
EAX[31:0]	GP_COUNTERS	For each bit n set in this field, the processor supports general-purpose performance monitoring counter n.	Logical Processor
EBX[31:0]	FIXED_COUNTERS	For each bit m set in this field, the processor supports fixed-function performance monitoring counter m. The valid range of fixed-function counters is 0 through 15.	Logical Processor
ECX[31:0]	Reserved	Reserved.	
EDX[31:0]	Reserved	Reserved.	

CPUID.23H.02H -- Bitmap of Auto Counter Reload Sub-Leaf

Table 21-76. Leaf 23H.02H Architectural Performance Monitoring Extended

Register	Field Name	Description	Domain
EAX[31:0]	ACR_GP_RELOAD	General counters that can be reloaded. For each bit n set in this field, the processor supports ACR for general-purpose performance monitoring counter n.	Logical Processor
EBX[31:0]	ACR_FIXED_RELOAD	Fixed counters that can be reloaded. For each bit m set in this field, the processor supports ACR for fixed-function performance monitoring counter m.	Logical Processor
ECX[31:0]	ACR_GP_TRIGGER	General counters that can cause reloads. For each bit y set in this field, the processor allows general-purpose performance monitoring counter y to reload all existing general-purpose performance monitoring counters capable of being reloaded.	Logical Processor
EDX[31:0]	ACR_FIXED_TRIGGER	Fixed counters that can cause reloads. For each bit x set in this field, the processor allows fixed-function performance monitoring counter x to reload all existing fixed-function performance monitoring counters capable of being reloaded.	Logical Processor

CPUID.23H.03H -- Architectural Performance Monitoring Events Bitmap Sub-Leaf

For each bit n set in this field, the processor supports Architectural Performance Monitoring Event of index n.

Table 21-77. Leaf 23H.03H Architectural Performance Monitoring Extended

Register	Field Name	Description	Domain
EAX[0]	CORE_CYC	If 1, supports architectural index 0.	Logical Processor
EAX[1]	INSTR_RET	If 1, supports architectural index 1.	Logical Processor
EAX[2]	REF_CYC	If 1, supports architectural index 2.	Logical Processor
EAX[3]	LLC_REF	If 1, supports architectural index 3.	Logical Processor
EAX[4]	LLC_MISSES	If 1, supports architectural index 4.	Logical Processor
EAX[5]	BR_INSTR_RET	If 1, supports architectural index 5	Logical Processor
EAX[6]	BR_MISPRED_RET	If 1, supports architectural index 6	Logical Processor
EAX[7]	SLOTS	If 1, supports architectural index 7	Logical Processor
EAX[8]	BACKEND	If 1, supports architectural index 8	Logical Processor
EAX[9]	BADSPEC	If 1, supports architectural index 9	Logical Processor

EAX[10]	FRONTEND	If 1, supports architectural index 10	Logical Processor
EAX[11]	RETIRING	If 1, supports architectural index 11	Logical Processor
EAX[12]	LBR_INSERTS	If 1, supports architectural index 12	Logical Processor
EAX[31:13]	Reserved	Reserved.	
EBX[31:0]	Reserved	Reserved.	
ECX[31:0]	Reserved	Reserved.	
EDX[31:0]	Reserved	Reserved.	

CPUID.23H.04H -- PEBS Capabilities

Table 21-78. Leaf 23H.04H Architectural Performance Monitoring Extended

Register	Field Name	Description	Domain
EAX[31:0]	Reserved	Reserved.	
EBX[2:0]	Reserved	Reserved.	
EBX[3]	ALLOW_IN_RECORD	If 1, indicates that the ALLOW_IN_RECORD bit is available in the IA32_PMC_GPn_CFG_C and IA32_PMC_FXm_CFG_C MSRs	Logical Processor
EBX[7:4]	CNTR	Counters group sub-groups general-purpose counters, fixed-function counters, and performance metrics are available.	Logical Processor
EBX[9:8]	LBR	LBR group and both bits [41:40] are available.	Logical Processor
EBX[15:10]	Reserved	Reserved.	
EBX[23:16]	XER	XER group bits [50:49] and bits [55:53] are available. See Section 11.4.4, "XSAVEEnabled Registers Group," for XER fields.	Logical Processor
EBX[28:24]	Reserved	Reserved.	
EBX[29]	GPR	If 1, the GPR group is available.	Logical Processor
EBX[30]	AUX	If 1, the AUX group is available.	Logical Processor
EBX[31]	Reserved	Reserved.	
ECX[31:0]	Reserved	Reserved.	
EDX[31:0]	Reserved	Reserved.	

CPUID.23H.05H -- Arch PEBS GP and Fixed Counters supported

Table 21-79. Leaf 23H.05H Architectural Performance Monitoring Extended

Register	Field Name	Description	Domain

EAX[31:0]	GP_PEBS	Bit vector of general-purpose counters for which the Architectural PEBS mechanism is available (bit n == GP counter #n). If EAX[n] == 1, then the IA32_PMC_GPn_CFG_C MSR is available, and PEBS is supported on that counter; the PEBS_EN[63] field can be set; and the RELOAD[31:0] field can be set. Note that CPUID.23H.04H:EBX governs which adaptive group bits can be set.	Logical Processor
EBX[31:0]	GP_PDIST	General-purpose counters for which PEBS supports PDIST.	Logical Processor
ECX[31:0]	FIXED_PEBS	Bit vector of fixed-function counters for which the Architectural PEBS mechanism is available. If ECX[x] == 1, then the IA32_PMC_FXm_CFG_C MSR is available, and PEBS is supported; the PEBS_EN[63] field can be set; and the RELOAD[31:0] field can be set. Note that CPUID.23H.04H:EBX governs which adaptive group bits can be set.	Logical Processor
EDX[31:0]	FIXED_PDIST	Fixed-function counters for which PEBS supports PDIST.	Logical Processor

CPUID.24H -- Converged Vector ISA

When CPUID.24H, the processor returns Intel AVX10 converged vector ISA information.
This leaf is supported when CPUID.07H.01H:EDX.AVX10[19] = 1.
• This leaf is valid if CPUID.07H.01H:EDX.AVX10[19] = 1 and MAX_LEAF ≥ 24H.
• The maximum sub-leaf value for ECX is specified in CPUID.24H.00H.EAX[31:0] MAX_SUBLEAF.
• If ECX contains an invalid sub-leaf index, EAX/EBX/ECX/EDX return 0. Sub-leaf index n is invalid if n exceeds the value that sub-leaf 0 returns in EAX.

CPUID.24H.00H -- Converged Vector ISA Main Sub-Leaf

Table 21-80. Leaf 24H.00H Converged Vector ISA

Register	Field Name	Description	Domain
EAX[31:0]	MAX_SUBLEAF	Reports the maximum number of sub-leaves that are supported in leaf 24H.	Platform
EBX[7:0]	VECTOR_ISA_VERSION	Reports the Intel® AVX10 Converged Vector ISA version.	Platform
EBX[15:8]	Reserved	Reserved.	
EBX[18:16]	Reserved at 111	Always 111b. Earlier versions of this specification documented these bits as enumerating support for different vector lengths. Processors enumerating Intel® AVX10 support all vector lengths.	Platform
EBX[31:19]	Reserved	Reserved.	
ECX[31:0]	Reserved	Reserved.	
EDX[31:0]	Reserved	Reserved.	

CPUID.27H -- Intel® Resource Director Technology (Intel® RDT) Asymmetric Monitoring

CPUID.27H returns information for the Intel Resource Director Technology Monitoring capabilities with asymmetric topology.

As described below, software uses the bit vector returned in EDX by sub-leaf 00H to determine the available resource types (ResID) that can be monitored. This information is necessary for software to program the IA32_PQR_ASSOC and IA32_QM_EVTSEL MSRs such that Quality-of-Service data can be read afterwards from the IA32_QM_CTR MSR.

- This leaf is valid if CPUID.07H.01H:ECX.RDT_M_ASYM[0] = 1 and MAX_LEAF ≥ 27H.
- If the leaf is valid, sub-leaf 00H is always valid. Sub-leaf n (n ≥ 1) is only valid when (CPUID.27H.00H:EDX[n] == 1).
- This leaf must be read on each logical processor to determine the support on each processor.

CPUID.27H.00H -- Intel® RDT Asymmetric Monitoring Main Sub-Leaf

CPUID.27H.00H returns information about Intel RDT Monitoring Asymmetric.

Table 21-81. Leaf 27H.00H Intel® Resource Director Technology (Intel® RDT) Asymmetric Monitoring

Register	Field Name	Description	Domain
EAX[31:0]	Reserved	Reserved.	
EBX[31:0]	MAX_RMID	Maximum range (zero-based) of RMID within this physical processor of all types.	Logical Processor
ECX[31:0]	Reserved	Reserved.	
EDX[0]	Reserved	Reserved.	
EDX[1]	L3_MON	If 1, supports L3 Cache Intel RDT Monitoring. Sub-leaf index 0 reports valid resource type starting at bit position 1 of EDX.	Logical Processor
EDX[31:2]	Reserved	Reserved.	

CPUID.27H.00H returns information about the bit-vector representation of QoS monitoring resource types that are supported in the processor and maximum range of RMID values the processor can use to monitor of any supported resource types. Each bit, starting from bit 1, corresponds to a specific resource type if the bit is set. The bit position corresponds to the sub-leaf index (or ResID) that software must use to query QoS monitoring capability available for that type.

CPUID.27H.01H -- L3 Cache Intel® Resource Director Technology Asymmetric Monitoring

CPUID.27H.01H returns information about L3 Cache Intel RDT monitoring asymmetric.

Table 21-82. Leaf 27H.01H Intel® Resource Director Technology (Intel® RDT) Asymmetric Monitoring

Register	Field Name	Description	Domain
EAX[7:0]	CTR_WIDTH	The counter width is encoded as an offset from 24b. A value of zero in this field indicates that 24-bit counters are supported. A value of 8 in this field indicates that 32-bit counters are supported.	Logical Processor
EAX[8]	RDT_M_OVF	If 1, supports an overflow bit in the IA32_QM_CTR MSR (bit 61).	Logical Processor

EAX[9]	IO_RDT_CMT	If 1, indicates the presence of non-CPU agent supporting Intel RDT CMT.	Logical Processor
EAX[10]	IO_RDT_MBM	If 1, indicates the presence of non-CPU agent supporting Intel RDT MBM support.	Logical Processor
EAX[31:11]	Reserved	Reserved.	
EBX[31:0]	CONV_FACTOR	Factor used to convert from reported IA32_QM_CTR value to derived occupancy metric (bytes) and Memory Bandwidth Monitoring (MBM) metrics.	Logical Processor
ECX[31:0]	MAX_RMID_L3	Maximum range (zero-based) of RMID of this resource type.	Logical Processor
EDX[0]	CMT_L3_OCCUP	If 1, supports L3 occupancy monitoring.	Logical Processor
EDX[1]	MBM_L3_TOTAL	If 1, supports L3 total bandwidth monitoring.	Logical Processor
EDX[2]	MBM_L3_LOCAL	If 1, supports L3 local bandwidth monitoring.	Logical Processor
EDX[31:3]	Reserved	Reserved.	

CPUID.28H -- Intel® Resource Director Technology (Intel® RDT) Asymmetric Allocation

CPUID.28H returns information for Intel Resource Director Technology Allocation with asymmetric topology. This leaf is valid when CPUID.07H.01H:ECX.RDT_A_SYM[1] = 1. As described below, software uses the bit vector returned in EBX by subleaf 00H to determine the available QoS Enforcement (allocation) resource types that are supported in the
processor. This information is necessary for software to configure each class of services using capability bit masks in the QoS Mask registers, IA32_resourceType_Mask_n.

- This leaf is valid if CPUID.07H.01H:ECX.RDT_A_SYM[1] = 1 and MAX_LEAF ≥ 28H.
- If the leaf is valid, sub-leaf 00H is always valid. Sub-leaf n (n ≥ 1) is only valid when (CPUID.28H.00H:EBX[n] == 1).

CPUID.28H.00H -- Intel® RDT Asymmetric Allocation Main Sub-Leaf

CPUID.28H.00H returns information about Intel RDT Allocation Asymmetric.

Table 21-83. Leaf 28H.00H Intel® Resource Director Technology (Intel® RDT) Asymmetric Allocation

Register	Field Name	Description	Domain
EAX[31:0]	Reserved	Reserved.	
EBX[0]	Reserved	Reserved.	
EBX[1]	CAT_L3	Supports L3 Cache Allocation Technology if 1.	Logical Processor
EBX[2]	CAT_L2	Supports L2 Cache Allocation Technology if 1.	Logical Processor
EBX[3]	MBA	Supports Memory Bandwidth Allocation if 1.	Logical Processor
EBX[4]	Reserved	Reserved.	
EBX[4]	Reserved	Reserved.	
EBX[5]	CBA	If 1, supports Cache Bandwidth Allocation.	Logical Processor
EBX[6]	RESOURCE_PRIORITY	If 1, supports Resource Priority.	Platform
EBX[6:5]	Reserved	Reserved.	
EBX[31:7]	Reserved	Reserved.	
ECX[31:0]	Reserved	Reserved.	
EDX[31:0]	Reserved	Reserved.	

CPUID.28H.00H returns information about the bit-vector representation of QoS Enforcement resource types that are supported in the processor. Each bit, starting from bit 1, corresponds to a specific resource type if the bit is set. The bit position corresponds to the sub-leaf index (or ResID) that software must use to query QoS enforcement capability available for that type.

CPUID.28H.01H -- Asymmetric L3 Cache Allocation Technology

CPUID.28H.ResID=1 returns information about Asymmetric L3 Cache Allocation Technology.

Table 21-84. Leaf 28H.01H Intel® Resource Director Technology (Intel® RDT) Asymmetric Allocation

Register	Field Name	Description	Domain

EAX[4:0]	CAT_L3_BITMASK_LENGTH	Length of the capacity bit mask for the corresponding ResID. Add one to the return value to get the result.	Logical Processor
EAX[31:5]	Reserved	Reserved.	
EBX[31:0]	CAT_L3_CONTENTION	Bit-granular map of isolation/contention of allocation units.	Logical Processor
ECX[0]	Reserved	Reserved.	
ECX[1]	CAT_L3_NONCPU	N/A	Logical Processor
ECX[2]	CAT_L3_CDP	If 1, supports L3 Code and Data Prioritization Technology.	Logical Processor
ECX[3]	CAT_L3_NONCONTIG	If 1, supports non-contiguous capacity bitmasks. The bits that are set in the various IA32_L3_MASK_n registers do not have to be contiguous.	Logical Processor
ECX[31:4]	Reserved	Reserved.	
EDX[15:0]	CAT_L3_MAX_CLOS	Highest Class of Service (COS) number supported for this ResID.	Logical Processor
EDX[31:16]	Reserved	Reserved.	

CPUID.28H.02H -- Asymmetric L2 Cache Allocation Technology

CPUID.28H.ResID=2 returns information about Asymmetric L2 Cache Allocation Technology.

Table 21-85. Leaf 28H.02H Intel® Resource Director Technology (Intel® RDT) Asymmetric Allocation

Register	Field Name	Description	Domain
EAX[4:0]	CAT_L2_BITMASK_LENGTH	Length of the capacity bit mask for the corresponding ResID. Add one to the return value to get the result.	Logical Processor
EAX[31:5]	Reserved	Reserved.	
EBX[31:0]	CAT_L2_MAP	Bit-granular map of isolation/contention of allocation units.	Logical Processor
ECX[1:0]	Reserved	Reserved.	
ECX[2]	CAT_L2_CDP	If 1, supports L2 Code and Data Prioritization Technology.	Logical Processor
ECX[3]	CAT_L2_NONCONTIG	If 1, supports non-contiguous capacity bitmasks. The bits that are set in the various IA32_L2_MASK_n registers do not have to be contiguous.	Logical Processor
ECX[31:4]	Reserved	Reserved.	
EDX[15:0]	CAT_L2_MAX_CLOS	Highest Class of Service (COS) number supported for this ResID.	Logical Processor
EDX[31:16]	Reserved	Reserved.	

CPUID.28H.03H -- Asymmetric Memory Bandwidth Allocation

CPUID.28H.ResID=3 returns information about Asymmetric Memory Bandwidth Allocation.

Table 21-86. Leaf 28H.03H Intel® Resource Director Technology (Intel® RDT) Asymmetric Allocation

Register	Field Name	Description	Domain
EAX[11:0]	MBA_MAX	Reports the maximum MBA throttling value supported for the corresponding ResID. Add one to the return value to get the result.	Logical Processor
EAX[31:12]	Reserved	Reserved.	
EBX[31:0]	Reserved	Reserved.	
ECX[0]	PER_THREAD_MBA	Per-thread MBA controls are supported.	Logical Processor
ECX[1]	Reserved	Reserved.	
ECX[2]	MBA_LINEAR	If 1, the response of the delay values is linear.	Logical Processor
ECX[31:3]	Reserved	Reserved.	
EDX[15:0]	MBA_MAX_CLOS	Highest Class of Service (COS) number supported for this ResID.	Logical Processor
EDX[31:16]	Reserved	Reserved.	

CPUID.28H.05H -- Asymmetric Cache Bandwidth Allocation

Table 21-87. Leaf 28H.05H Intel® Resource Director Technology (Intel® RDT) Asymmetric Allocation

Register	Field Name	Description	Domain
EAX[7:0]	CBA_MAX_LEVELS	Reports the maximum core throttling level supported for the corresponding ResID. Add one to the return value to get the number of throttling levels supported.	Logical Processor
EAX[11:8]	BW_SCOPE	If 1, indicates the logical processor scope of the IA32_QoS_Core_BW_Thrtl_n MSRs. Other values are reserved.	Logical Processor
EAX[31:12]	Reserved	Reserved.	
EBX[31:0]	Reserved	Reserved.	
ECX[2:0]	Reserved	Reserved.	
ECX[3]	CBA_LINEAR	If 1, the response of the bandwidth control is approximately linear. If 0, the response of the bandwidth control is non-linear.	Logical Processor
ECX[31:4]	Reserved	Reserved.	
EDX[15:0]	CBA_MAX_CLOS	Highest Class of Service (COS) number supported for this ResID.	Logical Processor
EDX[31:16]	Reserved	Reserved.	

CPUID.28H.06H -- Resource Priority Control

Table 21-88. Leaf 28H.06H Intel® Resource Director Technology (Intel® RDT) Asymmetric Allocation

Register	Field Name	Description	Domain
EAX[0]	THREAD_ENABLE	If 1, supports per-thread enable of RP through the IA32_RESOURCE_PRIORITY MSR.	Platform

EAX[1]	PACKAGE_ENABLE	If 1, supports physical processor package enable of RP through the IA32_RESOURCE PRIORITY_PKG MSR.	Platform
EAX[31:2]	Reserved	Reserved.	
EBX[31:0]	Reserved	Reserved.	
ECX[31:0]	Reserved	Reserved.	
EDX[31:0]	Reserved	Reserved.	

CPUID.80000000H -- Maximum Input Value for Extended Function CPUID Information

CPUID.80000000H returns the highest value the processor recognizes for returning extended processor information. The value is returned in the EAX register and is processor specific.

- This leaf is supported starting with Pentium 4.
- Processors prior to Pentium 4 treat bit 31 as 0, and this leaf returns the values from CPUID.00H.
- This leaf does not contain sub-leaves and provides the same information regardless of the value of ECX.

Table 21-89. Leaf 80000000H Maximum Input Value for Extended Function CPUID Information

Register	Field Name	Description	Domain
EAX[31:0]	MAX_EXTENDED_LEAF	Maximum input value for Extended Function CPUID Information.	Platform
EBX[31:0]	Reserved	Reserved.	
ECX[31:0]	Reserved	Reserved.	
EDX[31:0]	Reserved	Reserved.	

CPUID.80000001H -- Extended Processor Signature and Feature Bits

CPUID.80000001H returns information about extended processor signature and features bits.

- This leaf is valid if MAX_EXTENDED_LEAF ≥ 80000001H.
- This leaf does not contain sub-leaves and provides the same information regardless of the value of ECX.

Table 21-90. Leaf 80000001H Extended Processor Signature and Feature Bits

Register	Field Name	Description	Domain
EAX[31:0]	Reserved	Reserved.	
EBX[31:0]	Reserved	Reserved.	
ECX[0]	LAHF_SAHF_64	If 1, supports the LAHF/SAHF instructions in 64-bit mode. LAHF and SAHF are always available in other modes, regardless of the enumeration of this feature flag.	Platform
ECX[4:1]	Reserved	Reserved.	
ECX[5]	LZCNT	If 1, supports the LZCNT instruction.	Platform
ECX[7:6]	Reserved	Reserved.	
ECX[8]	PREFETCHW	If 1, supports the PREFETCHW instruction.	Platform
ECX[31:9]	Reserved	Reserved.	
EDX[10:0]	Reserved	Reserved.	
EDX[11]	SYSCALL_SYSRET_64	If 1, supports SYSCALL/SYSRET. Intel processors support SYSCALL and SYSRET only in 64-bit mode. This feature flag is always enumerated as 0 outside 64-bit mode.	Platform
EDX[19:12]	Reserved	Reserved.	
EDX[20]	EXECUTE_DIS	If 1, supports Execute Disable Bit.	Platform
EDX[25:21]	Reserved	Reserved.	
EDX[26]	PAGE_1GB	If 1, supports 1-GByte pages.	Platform
EDX[27]	RDTSCP	If 1, supports RDTSCP and IA32_TSC_AUX.	Platform
EDX[28]	Reserved	Reserved.	
EDX[29]	INTEL64	If 1, supports Intel® 64 Architecture.	Platform
EDX[31:30]	Reserved	Reserved.	

CPUID.80000002H -- Processor Brand String (Bytes 0 to 15)

CPUID.80000002H returns information about the Processor Brand String. For additional details on Processor Brand String, see Section 21.2, "Methods for Returning Branding Information Using CPUID."

- This leaf is valid if MAX_EXTENDED_LEAF ≥ 80000002H.
- This leaf does not contain sub-leaves and provides the same information regardless of the value of ECX.

Table 21-91. Leaf 80000002H Processor Brand String (Bytes 0 to 15)

Register	Field Name	Description	Domain
EAX[31:0]	BRAND_NAME_0	Processor brand string.	Platform
EBX[31:0]	BRAND_NAME_1	Processor brand string continued.	Platform
ECX[31:0]	BRAND_NAME_2	Processor brand string continued.	Platform
EDX[31:0]	BRAND_NAME_3	Processor brand string continued.	Platform

CPUID.80000003H -- Processor brand string (Bytes 16 to 31)

CPUID.80000003H returns information about the Processor Brand String. For additional details on Processor Brand String, see Section 21.2, "Methods for Returning Branding Information Using CPUID."

- This leaf is valid if MAX_EXTENDED_LEAF ≥ 80000003H.
- This leaf does not contain sub-leaves and provides the same information regardless of the value of ECX.

Table 21-92. Leaf 80000003H Processor brand string (Bytes 16 to 31)

Register	Field Name	Description	Domain
EAX[31:0]	BRAND_NAME_4	Processor brand string continued.	Platform
EBX[31:0]	BRAND_NAME_5	Processor brand string continued.	Platform
ECX[31:0]	BRAND_NAME_6	Processor brand string continued.	Platform
EDX[31:0]	BRAND_NAME_7	Processor brand string continued.	Platform

CPUID.80000004H -- Processor brand string (Bytes 32 to 47)

CPUID.80000004H returns information about the Processor Brand String. For additional details on Processor Brand String, see Section 21.2, “Methods for Returning Branding Information Using CPUID.”

- This leaf is valid if MAX_EXTENDED_LEAF ≥ 80000004H.
- This leaf does not contain sub-leaves and provides the same information regardless of the value of ECX.

Table 21-93. Leaf 80000004H Processor brand string (Bytes 32 to 47)

Register	Field Name	Description	Domain
EAX[31:0]	BRAND_NAME_8	Processor brand string continued.	Platform
EBX[31:0]	BRAND_NAME_9	Processor brand string continued.	Platform
ECX[31:0]	BRAND_NAME_10	Processor brand string continued.	Platform
EDX[31:0]	BRAND_NAME_11	Processor brand string continued.	Platform

CPUID.80000005H -- Reserved

This leaf is reserved and returns all zeroes.

Table 21-94. Leaf 80000005H Reserved

Register	Field Name	Description	Domain
EAX[31:0]	Reserved	Reserved.	
EBX[31:0]	Reserved	Reserved.	
ECX[31:0]	Reserved	Reserved.	
EDX[31:0]	Reserved	Reserved.	

CPUID.80000006H -- Extended Function CPUID Information

CPUID.80000006H returns Extended Function CPUID information. The preferred method to enumerate caching information description is to use CPUID.04H—Deterministic Cache Parameters.

- This leaf is valid if MAX_EXTENDED_LEAF ≥ 80000006H.
- This leaf does not contain sub-leaves and provides the same information regardless of the value of ECX

Table 21-95. Leaf 80000006H Extended Function CPUID Information

Register	Field Name	Description	Domain
EAX[31:0]	Reserved	Reserved.	
EBX[31:0]	Reserved	Reserved.	
ECX[7:0]	L2_LINE_SIZE	Cache line size in bytes.	Logical Processor
ECX[11:8]	Reserved	Reserved.	
ECX[15:12]	L2_ASSOC	L2 associativity field. The L2 associativity field encodings are listed in the table below.	Logical Processor
ECX[31:16]	L2_SIZE	Cache size in 1K units.	Logical Processor
EDX[7:0]	Reserved	Reserved.	
EDX[31:8]	Reserved	Reserved.	

Table 21-96. L2 Associativity Field Encodings

Encoding Value	Description	Encoding Value	Description
00H	Disabled	08H	16 Ways
01H	1 Way (direct mapped)	09H	Reserved
02H	2 Ways	0AH	32 Ways
03H	Reserved	0BH	48 Ways
04H	4 Ways	0CH	64 Ways
05H	Reserved	0DH	96 Ways
06H	8 Ways	0EH	128 Ways
07H	See CPUID leaf 4 sub-leaf 2[1]	0FH	Fully Associative

CPUID.80000007H -- Extended Function CPUID Information 1

CPUID.80000007H returns Extended Function CPUID information.
- This leaf is valid if MAX_EXTENDED_LEAF ≥ 80000007H.
- This leaf does not contain sub-leaves and provides the same information regardless of the value of ECX.

Table 21-97. Leaf 80000007H Extended Function CPUID Information 1

Register	Field Name	Description	Domain
EAX[31:0]	Reserved	Reserved.	
EBX[31:0]	Reserved	Reserved.	
ECX[31:0]	Reserved	Reserved.	
EDX[7:0]	Reserved	Reserved.	
EDX[8]	TSC_INVARIANT	If 1, supports Invariant TSC.	Platform
EDX[31:9]	Reserved	Reserved.	

CPUID.80000008H -- Extended Function CPUID Information 2

CPUID.80000008H returns Extended Function CPUID information.
- This leaf is valid if MAX_EXTENDED_LEAF ≥ 80000008H.
- This leaf does not contain sub-leaves and provides the same information regardless of the value of ECX.

Table 21-98. Leaf 80000008H Extended Function CPUID Information 2

Register	Field Name	Description	Domain
EAX[7:0]	PHYS_ADDR_SIZE	Number of physical-address bits. If TME-MK is enabled, the number of bits that can be used to address memory may be reduced by IA32_TME_ACTIVATE[35:32].	Platform
EAX[15:8]	LIN_ADDR_SIZE	Number of linear-address bits.	Platform
EAX[23:16]	GUEST_PHYS_ADDR_SIZE	Number of guest-physical-address bits (for software operating in a virtual machine). If this field is zero, PHYS_ADDR_SIZE should be used. Intel processors return zero for this field. Software emulating CPUID may return a different value.	Platform
EAX[31:24]	Reserved	Reserved.	
EBX[8:0]	Reserved	Reserved.	
EBX[9]	WBNOINVD	If 1, supports the WBNOINVD instruction.	Platform
EBX[31:10]	Reserved	Reserved.	
ECX[31:0]	Reserved	Reserved.	
EDX[31:0]	Reserved	Reserved.	

APPENDIX A
EFLAGS CROSS-REFERENCE

A.1 EFLAGS AND INSTRUCTIONS

Table A-2 summarizes how the instructions affect the flags in the EFLAGS register. The following codes describe how the flags are affected.

Table A-1. Codes Describing Flags

T	Instruction tests flag.
M	Instruction modifies flag (either sets or resets depending on operands).
0	Instruction resets flag.
1	Instruction sets flag.
—	Instruction's effect on flag is undefined.
R	Instruction restores prior value of flag.
Blank	Instruction does not affect flag.

Table A-2. EFLAGS Cross-Reference

Instruction	OF	SF	ZF	AF	PF	CF	TF	IF	DF	NT	RF
AAA	—	—	—	TM	—	M					
AAD	—	M	M	—	M	—					
AAM	—	M	M	—	M	—					
AAS	—	—	—	TM	—	M					
ADC	M	M	M	M	M	TM					
ADD	M	M	M	M	M	M					
AND	0	M	M	—	M	0					
ARPL			M								
BOUND											
BSF/BSR	0	0	M	0	M	0					
BSWAP											
BT/BTS/BTR/BTC	—	—		—	—	M					
CALL											
CBW											
CLC						0					
CLD									0		
CLI								0			
CLTS											
CMC						M					
CMOV*cc*	T	T	T		T	T					
CMP	M	M	M	M	M	M					

Table A-2. EFLAGS Cross-Reference (Contd.)

Instruction	OF	SF	ZF	AF	PF	CF	TF	IF	DF	NT	RF
CMPS	M	M	M	M	M	M			T		
CMPXCHG	M	M	M	M	M	M					
CMPXCHG8B			M								
COMISD	0	0	M	0	M	M					
COMISS	0	0	M	0	M	M					
CPUID											
CWD											
DAA	—	M	M	TM	M	TM					
DAS	—	M	M	TM	M	TM					
DEC	M	M	M	M	M						
DIV	—	—	—	—	—	—					
ENTER											
ESC											
FCMOV*cc*			T		T	T					
FCOMI, FCOMIP, FUCOMI, FUCOMIP	0	0	M	0	M	M					
HLT											
IDIV	—	—	—	—	—	—					
IMUL	M	—	—	—	—	M					
IN											
INC	M	M	M	M	M						
INS									T		
INT							0			0	
INTO	T						0			0	
INVD											
INVLPG											
UCOMISD	0	0	M	0	M	M					
UCOMISS	0	0	M	0	M	M					
IRET	R	R	R	R	R	R	R	R	R	T	
J*cc*	T	T	T		T	T					
JCXZ											
JMP											
LAHF											
LAR			M								
LDS/LES/LSS/LFS/LGS											
LEA											
LEAVE											
LGDT/LIDT/LLDT/LMSW											
LOCK											

Table A-2. EFLAGS Cross-Reference (Contd.)

Instruction	OF	SF	ZF	AF	PF	CF	TF	IF	DF	NT	RF
LODS									T		
LOOP											
LOOPE/LOOPNE			T								
LSL			M								
LTR											
MONITOR											
MWAIT											
MOV											
MOV control, debug, test	—	—	—	—	—	—					
MOVS									T		
MOVSX/MOVZX											
MUL	M	—	—	—	—	M					
NEG	M	M	M	M	M	M					
NOP											
NOT											
OR	0	M	M	—	M	0					
OUT											
OUTS									T		
POP/POPA											
POPF	R	R	R	R	R	R	R	R	R	R	
PUSH/PUSHA/PUSHF											
RCL/RCR 1	M					TM					
RCL/RCR count	—					TM					
RDMSR											
RDPMC											
RDTSC											
REP/REPE/REPNE											
RET											
ROL/ROR 1	M					M					
ROL/ROR count	—					M					
RSM	M	M	M	M	M	M	M	M	M	M	M
SAHF		R	R	R	R	R					
SAL/SAR/SHL/SHR 1	M	M	M	—	M	M					
SAL/SAR/SHL/SHR count	—	M	M	—	M	M					
SBB	M	M	M	M	M	TM					
SCAS	M	M	M	M	M	M			T		
SET*cc*	T	T	T		T	T					
SGDT/SIDT/SLDT/SMSW											

Table A-2. EFLAGS Cross-Reference (Contd.)

Instruction	OF	SF	ZF	AF	PF	CF	TF	IF	DF	NT	RF
SHLD/SHRD	—	M	M	—	M	M					
STC						1					
STD									1		
STI								1			
STOS									T		
STR											
SUB	M	M	M	M	M	M					
TEST	0	M	M	—	M	0					
UD											
VERR/VERRW			M								
WAIT											
WBINVD											
WRMSR											
XADD	M	M	M	M	M	M					
XCHG											
XLAT											
XOR	0	M	M	—	M	0					

APPENDIX B
EFLAGS CONDITION CODES

B.1 CONDITION CODES

Table B-1 lists condition codes that can be queried using CMOV*cc*, FCMOV*cc*, J*cc,* and SET*cc*. Condition codes refer to the setting of one or more status flags (CF, OF, SF, ZF, and PF) in the EFLAGS register. In the table below:

- The "Mnemonic" column provides the suffix (cc) added to the instruction to specify a test condition.
- "Condition Tested For" describes the targeted condition.
- "Instruction Subcode" provides the opcode suffix added to the main opcode to specify the test condition.
- "Status Flags Setting" describes the flag setting.

Table B-1. EFLAGS Condition Codes

Mnemonic (*cc*)	Condition Tested For	Instruction Subcode	Status Flags Setting
O	Overflow	0000	OF = 1
NO	No overflow	0001	OF = 0
B C NAE	Below Carry Neither above nor equal	0010	CF = 1
NB NC AE	Not below Not carry Above or equal	0011	CF = 0
E Z	Equal Zero	0100	ZF = 1
NE NZ	Not equal Not zero	0101	ZF = 0
BE NA	Below or equal Not above	0110	(CF OR ZF) = 1
NBE A	Neither below nor equal Above	0111	(CF OR ZF) = 0
S	Sign	1000	SF = 1
NS	No sign	1001	SF = 0
P PE	Parity Parity even	1010	PF = 1
NP PO	No parity Parity odd	1011	PF = 0
L NGE	Less Neither greater nor equal	1100	(SF XOR OF) = 1
NL GE	Not less Greater or equal	1101	(SF XOR OF) = 0
LE NG	Less or equal Not greater	1110	((SF XOR OF) OR ZF) = 1
NLE G	Neither less nor equal Greater	1111	((SF XOR OF) OR ZF) = 0

Many of the test conditions are described in two different ways. For example, LE (less or equal) and NG (not greater) describe the same test condition. Alternate mnemonics are provided to make code more intelligible.

The terms “above” and “below” are associated with the CF flag and refer to the relation between two unsigned integer values. The terms “greater” and “less” are associated with the SF and OF flags and refer to the relation between two signed integer values.

APPENDIX C
FLOATING-POINT EXCEPTIONS SUMMARY

C.1 OVERVIEW

This appendix shows which of the floating-point exceptions can be generated for:

- x87 FPU instructions — see Table C-2.
- Intel SSE instructions — see Table C-3.
- Intel SSE2 instructions — see Table C-4.
- Intel SSE3 instructions — see Table C-5.
- Intel SSE4 instructions — see Table C-6.

Table C-1 lists types of floating-point exceptions that potentially can be generated by the x87 FPU and by Intel SSE, SSE2, and SSE3 instructions.

Table C-1. x87 FPU and SIMD Floating-Point Exceptions

Floating-point Exception	Description
#IS	Invalid-operation exception for stack underflow or stack overflow (can only be generated for x87 FPU instructions)*
#IA or #I	Invalid-operation exception for invalid arithmetic operands and unsupported formats*
#D	Denormal-operand exception
#Z	Divide-by-zero exception
#O	Numeric-overflow exception
#U	Numeric-underflow exception
#P	Inexact-result (precision) exception

NOTE:

* The x87 FPU instruction set generates two types of invalid-operation exceptions: #IS (stack underflow or stack overflow) and #IA (invalid arithmetic operation due to invalid arithmetic operands or unsupported formats). Intel SSE, SSE2, and SSE3 instructions potentially generate #I (invalid operation exceptions due to invalid arithmetic operands or unsupported formats).

The floating-point exceptions shown in Table C-1 (except for #D and #IS) are defined in IEEE Standard 754-1985 for Binary Floating-Point Arithmetic. See Section 4.9.1, "Floating-Point Exception Conditions," for a detailed discussion of floating-point exceptions.

C.2 X87 FPU INSTRUCTIONS

Table C-2 lists the x87 FPU instructions in alphabetical order. For each instruction, it summarizes the floating-point exceptions that the instruction can generate.

Table C-2. Exceptions Generated with x87 FPU Floating-Point Instructions

Mnemonic	Instruction	#IS	#IA	#D	#Z	#O	#U	#P
F2XM1	Exponential	Y	Y	Y			Y	Y
FABS	Absolute value	Y						
FADD(P)	Add floating-point	Y	Y	Y		Y	Y	Y
FBLD	BCD load	Y						

Table C-2. Exceptions Generated with x87 FPU Floating-Point Instructions (Contd.)

Mnemonic	Instruction	#IS	#IA	#D	#Z	#O	#U	#P
FBSTP	BCD store and pop	Y	Y					Y
FCHS	Change sign	Y						
FCLEX	Clear exceptions							
FCMOV*cc*	Floating-point conditional move	Y						
FCOM, FCOMP, FCOMPP	Compare floating-point	Y	Y	Y				
FCOMI, FCOMIP, FUCOMI, FUCOMIP	Compare floating-point and set EFLAGS	Y	Y	Y				
FCOS	Cosine	Y	Y	Y				Y
FDECSTP	Decrement stack pointer							
FDIV(R)(P)	Divide floating-point	Y	Y	Y	Y	Y	Y	Y
FFREE	Free register							
FIADD	Integer add	Y	Y	Y		Y	Y	Y
FICOM(P)	Integer compare	Y	Y	Y				
FIDIV	Integer divide	Y	Y	Y	Y		Y	Y
FIDIVR	Integer divide reversed	Y	Y	Y	Y	Y	Y	Y
FILD	Integer load	Y						
FIMUL	Integer multiply	Y	Y	Y		Y	Y	Y
FINCSTP	Increment stack pointer							
FINIT	Initialize processor							
FIST(P)	Integer store	Y	Y					Y
FISTTP	Truncate to integer (SSE3 instruction)	Y	Y					Y
FISUB(R)	Integer subtract	Y	Y	Y		Y	Y	Y
FLD extended or stack	Load floating-point	Y						
FLD single or double	Load floating-point	Y	Y	Y				
FLD1	Load + 1.0	Y						
FLDCW	Load Control word	Y	Y	Y	Y	Y	Y	Y
FLDENV	Load environment	Y	Y	Y	Y	Y	Y	Y
FLDL2E	Load $\log_2 e$	Y						
FLDL2T	Load $\log_2 10$	Y						
FLDLG2	Load $\log_{10} 2$	Y						
FLDLN2	Load $\log_e 2$	Y						
FLDPI	Load π	Y						
FLDZ	Load + 0.0	Y						
FMUL(P)	Multiply floating-point	Y	Y	Y		Y	Y	Y
FNOP	No operation							
FPATAN	Partial arctangent	Y	Y	Y			Y	Y
FPREM	Partial remainder	Y	Y	Y			Y	
FPREM1	IEEE partial remainder	Y	Y	Y			Y	

Table C-2. Exceptions Generated with x87 FPU Floating-Point Instructions (Contd.)

Mnemonic	Instruction	#IS	#IA	#D	#Z	#O	#U	#P
FPTAN	Partial tangent	Y	Y	Y			Y	Y
FRNDINT	Round to integer	Y	Y	Y				Y
FRSTOR	Restore state	Y	Y	Y	Y	Y	Y	Y
FSAVE	Save state							
FSCALE	Scale	Y	Y	Y		Y	Y	Y
FSIN	Sine	Y	Y	Y			Y	Y
FSINCOS	Sine and cosine	Y	Y	Y			Y	Y
FSQRT	Square root	Y	Y	Y				Y
FST(P) stack or extended	Store floating-point	Y						
FST(P) single or double	Store floating-point	Y	Y			Y	Y	Y
FSTCW	Store control word							
FSTENV	Store environment							
FSTSW (AX)	Store status word							
FSUB(R)(P)	Subtract floating-point	Y	Y	Y		Y	Y	Y
FTST	Test	Y	Y	Y				
FUCOM(P)(P)	Unordered compare floating-point	Y	Y	Y				
FWAIT	CPU Wait							
FXAM	Examine							
FXCH	Exchange registers	Y						
FXTRACT	Extract	Y	Y	Y	Y			
FYL2X	Logarithm	Y	Y	Y	Y	Y	Y	Y
FYL2XP1	Logarithm epsilon	Y	Y	Y		Y	Y	Y

C.3 INTEL® SSE INSTRUCTIONS

Table C-3 lists the Intel SSE instructions with at least one of the following characteristics:

- Has floating-point operands.
- Generates floating-point results.
- Reads or writes floating-point status and control information.

The table also summarizes the floating-point exceptions that each instruction can generate.

Table C-3. Exceptions Generated with Intel® SSE Instructions

Mnemonic	Instruction	#I	#D	#Z	#O	#U	#P
ADDPS	Packed add.	Y	Y		Y	Y	Y
ADDSS	Scalar add.	Y	Y		Y	Y	Y
ANDNPS	Packed logical INVERT and AND.						
ANDPS	Packed logical AND.						
CMPPS	Packed compare.	Y	Y				
CMPSS	Scalar compare.	Y	Y				

Table C-3. Exceptions Generated with Intel® SSE Instructions (Contd.)

Mnemonic	Instruction	#I	#D	#Z	#O	#U	#P
COMISS	Scalar ordered compare lower SP FP numbers and set the status flags.	Y	Y				
CVTPI2PS	Convert two 32-bit signed integers from MM2/Mem to two SP FP.						Y
CVTPS2PI	Convert lower two SP FP from XMM/Mem to two 32-bit signed integers in MM using rounding specified by MXCSR.	Y					Y
CVTSI2SS	Convert one 32-bit or 64-bit signed integer from Integer Reg/Mem to one SP FP.						Y
CVTSS2SI	Convert one SP FP from XMM/Mem to one 32-bit or 64-bit signed integer using rounding mode specified by MXCSR, and move the result to an integer register.	Y					Y
CVTTPS2PI	Convert two SP FP from XMM2/Mem to two 32-bit signed integers in MM1 using truncate.	Y					Y
CVTTSS2SI	Convert lowest SP FP from XMM/Mem to one 32-bit signed integer using truncate, and move the result to an integer register.	Y					Y
DIVPS	Packed divide.	Y	Y	Y	Y	Y	Y
DIVSS	Scalar divide.	Y	Y	Y	Y	Y	Y
LDMXCSR	Load control/status word.						
MAXPS	Packed maximum.	Y	Y				
MAXSS	Scalar maximum.	Y	Y				
MINPS	Packed minimum.	Y	Y				
MINSS	Scalar minimum.	Y	Y				
MOVAPS	Move four packed SP values.						
MOVHLPS	Move packed SP high to low.						
MOVHPS	Move two packed SP values between memory and the high half of an XMM register.						
MOVLHPS	Move packed SP low to high.						
MOVLPS	Move two packed SP values between memory and the low half of an XMM register.						
MOVMSKPS	Move sign mask to r32.						
MOVSS	Move scalar SP number between an XMM register and memory or a second XMM register.						
MOVUPS	Move unaligned packed data.						
MULPS	Packed multiply.	Y	Y		Y	Y	Y
MULSS	Scalar multiply.	Y	Y		Y	Y	Y
ORPS	Packed OR.						
RCPPS	Packed reciprocal.						
RCPSS	Scalar reciprocal.						
RSQRTPS	Packed reciprocal square root.						
RSQRTSS	Scalar reciprocal square root.						
SHUFPS	Shuffle.						
SQRTPS	Square Root of the packed SP FP numbers.	Y	Y				Y
SQRTSS	Scalar square root.	Y	Y				Y

Table C-3. Exceptions Generated with Intel® SSE Instructions (Contd.)

Mnemonic	Instruction	#I	#D	#Z	#O	#U	#P
STMXCSR	Store control/status word.						
SUBPS	Packed subtract.	Y	Y		Y	Y	Y
SUBSS	Scalar subtract.	Y	Y		Y	Y	Y
UCOMISS	Unordered compare lower SP FP numbers and set the status flags.	Y	Y				
UNPCKHPS	Interleave SP FP numbers.						
UNPCKLPS	Interleave SP FP numbers.						
XORPS	Packed XOR.						

C.4 INTEL® SSE2 INSTRUCTIONS

Table C-4 lists the Intel SSE2 instructions with at least one of the following characteristics:

- Floating-point operands.
- Floating-point results.

For each instruction, the table summarizes the floating-point exceptions that the instruction can generate.

Table C-4. Exceptions Generated with Intel® SSE2 Instructions

Instruction	Description	#I	#D	#Z	#O	#U	#P
ADDPD	Add two packed DP FP numbers from XMM2/Mem to XMM1.	Y	Y		Y	Y	Y
ADDSD	Add the lower DP FP number from XMM2/Mem to XMM1.	Y	Y		Y	Y	Y
ANDNPD	Invert the 128 bits in XMM1 and then AND the result with 128 bits from XMM2/Mem.						
ANDPD	Logical And of 128 bits from XMM2/Mem to XMM1 register.						
CMPPD	Compare packed DP FP numbers from XMM2/Mem to packed DP FP numbers in XMM1 register using imm8 as predicate.	Y	Y				
CMPSD	Compare lowest DP FP number from XMM2/Mem to lowest DP FP number in XMM1 register using imm8 as predicate.	Y	Y				
COMISD	Compare lower DP FP number in XMM1 register with lower DP FP number in XMM2/Mem and set the status flags accordingly	Y	Y				
CVTDQ2PS	Convert four 32-bit signed integers from XMM/Mem to four SP FP.						Y
CVTPS2DQ	Convert four SP FP from XMM/Mem to four 32-bit signed integers in XMM using rounding specified by MXCSR.	Y					Y
CVTTPS2DQ	Convert four SP FP from XMM/Mem to four 32-bit signed integers in XMM using truncate.	Y					Y
CVTDQ2PD	Convert two 32-bit signed integers in XMM2/Mem to 2 DP FP in xmm1 using rounding specified by MXCSR.						
CVTPD2DQ	Convert two DP FP from XMM2/Mem to two 32-bit signed integers in xmm1 using rounding specified by MXCSR.	Y					Y
CVTPD2PI	Convert lower two DP FP from XMM/Mem to two 32-bit signed integers in MM using rounding specified by MXCSR.	Y					Y
CVTPD2PS	Convert two DP FP to two SP FP.	Y	Y		Y	Y	Y
CVTPI2PD	Convert two 32-bit signed integers from MM2/Mem to two DP FP.						
CVTPS2PD	Convert two SP FP to two DP FP.	Y	Y				

Table C-4. Exceptions Generated with Intel® SSE2 Instructions (Contd.)

Instruction	Description	#I	#D	#Z	#O	#U	#P
CVTSD2SI	Convert one DP FP from XMM/Mem to one 32-bit or 64-bit signed integer using rounding mode specified by MXCSR, and move the result to an integer register.	Y					Y
CVTSD2SS	Convert scalar DP FP to scalar SP FP.	Y	Y		Y	Y	Y
CVTSI2SD	Convert one 32-bit or 64-bit signed integer from Integer Reg/Mem to one DP FP.						
CVTSS2SD	Convert scalar SP FP to scalar DP FP.	Y	Y				
CVTTPD2DQ	Convert two DP FP from XMM2/Mem to two 32-bit signed integers in XMM1 using truncate.	Y					Y
CVTTPD2PI	Convert two DP FP from XMM2/Mem to two 32-bit signed integers in MM1 using truncate.	Y					Y
CVTTSD2SI	Convert lowest DP FP from XMM/Mem to one 32 bit signed integer using truncate, and move the result to an integer register.	Y					Y
DIVPD	Divide packed DP FP numbers in XMM1 by XMM2/Mem	Y	Y	Y	Y	Y	Y
DIVSD	Divide lower DP FP numbers in XMM1 by XMM2/Mem	Y	Y	Y	Y	Y	Y
MAXPD	Return the maximum DP FP numbers between XMM2/Mem and XMM1.	Y	Y				
MAXSD	Return the maximum DP FP number between the lower DP FP numbers from XMM2/Mem and XMM1.	Y	Y				
MINPD	Return the minimum DP numbers between XMM2/Mem and XMM1.	Y	Y				
MINSD	Return the minimum DP FP number between the lowest DP FP numbers from XMM2/Mem and XMM1.	Y	Y				
MOVAPD	Move 128 bits representing 2 packed DP data from XMM2/Mem to XMM1 register. Or Move 128 bits representing 2 packed DP from XMM1 register to XMM2/Mem.						
MOVHPD	Move 64 bits representing one DP operand from Mem to upper field of XMM register. Or move 64 bits representing one DP operand from upper field of XMM register to Mem.						
MOVLPD	Move 64 bits representing one DP operand from Mem to lower field of XMM register. Or move 64 bits representing one DP operand from lower field of XMM register to Mem.						
MOVMSKPD	Move the sign mask to r32.						
MOVSD	Move 64 bits representing one scalar DP operand from XMM2/Mem to XMM1 register. Or move 64 bits representing one scalar DP operand from XMM1 register to XMM2/Mem.						
MOVUPD	Move 128 bits representing 2 DP data from XMM2/Mem to XMM1 register. Or move 128 bits representing 2 DP data from XMM1 register to XMM2/Mem.						
MULPD	Multiply packed DP FP numbers in XMM2/Mem to XMM1.	Y	Y		Y	Y	Y

Table C-4. Exceptions Generated with Intel® SSE2 Instructions (Contd.)

Instruction	Description	#I	#D	#Z	#O	#U	#P
MULSD	Multiply the lowest DP FP number in XMM2/Mem to XMM1.	Y	Y		Y	Y	Y
ORPD	OR 128 bits from XMM2/Mem to XMM1 register.						
SHUFPD	Shuffle Double.						
SQRTPD	Square Root Packed Double Precision	Y	Y				Y
SQRTSD	Square Root Scaler Double Precision	Y	Y				Y
SUBPD	Subtract Packed Double Precision.	Y	Y		Y	Y	Y
SUBSD	Subtract Scaler Double Precision.	Y	Y		Y	Y	Y
UCOMISD	Compare lower DP FP number in XMM1 register with lower DP FP number in XMM2/Mem and set the status flags accordingly.	Y	Y				
UNPCKHPD	Interleaves DP FP numbers from the high halves of XMM1 and XMM2/Mem into XMM1 register.						
UNPCKLPD	Interleaves DP FP numbers from the low halves of XMM1 and XMM2/Mem into XMM1 register.						
XORPD	XOR 128 bits from XMM2/Mem to XMM1 register.						

C.5 INTEL® SSE3 INSTRUCTIONS

Table C-5 lists the Intel SSE3 instructions that have at least one of the following characteristics:

- Has floating-point operands.
- Generates floating-point results.

For each instruction, the table summarizes the floating-point exceptions that the instruction can generate.

Table C-5. Exceptions Generated with Intel® SSE3 Instructions

Instruction	Description	#I	#D	#Z	#O	#U	#P
ADDSUBPD	Add /Sub packed DP FP numbers from XMM2/Mem to XMM1.	Y	Y		Y	Y	Y
ADDSUBPS	Add /Sub packed SP FP numbers from XMM2/Mem to XMM1.	Y	Y		Y	Y	Y
FISTTP	See Table C-2.	Y					Y
HADDPD	Add horizontally packed DP FP numbers XMM2/Mem to XMM1.	Y	Y		Y	Y	Y
HADDPS	Add horizontally packed SP FP numbers XMM2/Mem to XMM1	Y	Y		Y	Y	Y
HSUBPD	Sub horizontally packed DP FP numbers XMM2/Mem to XMM1	Y	Y		Y	Y	Y
HSUBPS	Sub horizontally packed SP FP numbers XMM2/Mem to XMM1	Y	Y		Y	Y	Y

Other Intel SSE3 instructions do not generate floating-point exceptions.

C.6 SSSE3 INSTRUCTIONS

SSSE3 instructions operate on integer data elements. They do not generate floating-point exceptions.

C.7 INTEL® SSE4 INSTRUCTIONS

Table C-6 lists the Intel SSE4.1 instructions that generate floating-point results.

For each instruction, the table summarizes the floating-point exceptions that the instruction can generate.

Table C-6. Exceptions Generated with Intel® SSE4 Instructions

Instruction	Description	#I	#D	#Z	#O	#U	#P
DPPD	DP FP dot product.	Y	Y		Y	Y	Y
DPPS	SP FP dot product.	Y	Y		Y	Y	Y
ROUNDPD	Round packed DP FP values to integer FP values.	Y					Y[1]
ROUNDPS	Round packed SP FP values to integer FP values.	Y					Y[1]
ROUNDSD	Round scalar DP FP value to integer FP value.	Y					Y[1]
ROUNDSS	Round scalar SP FP value to integer FP value.	Y					Y[1]

NOTES:

1. If bit 3 of immediate operand is 0.

Other Intel SSE4.1 and SSE4.2 instructions do not generate floating-point exceptions.

APPENDIX D
GUIDELINES FOR WRITING SIMD FLOATING-POINT EXCEPTION HANDLERS

See Section 11.5, "Intel® SSE, SSE2, and SSE3 Exceptions," for a detailed discussion of SIMD floating-point exceptions.

This appendix considers only Intel SSE, SSE2, and SSE3 instructions that can generate numeric (SIMD floating-point) exceptions, and gives an overview of the necessary support for handling such exceptions. This appendix does not address instructions that do not generate floating-point exceptions (such as RSQRTSS, RSQRTPS, RCPSS, or RCPPS), any x87 instructions, or any unlisted instruction.

For detailed information on which instructions generate numeric exceptions, and a listing of those exceptions, refer to Appendix C, "Floating-Point Exceptions Summary." Non-numeric exceptions are handled in a way similar to that for the standard IA-32 instructions.

D.1 TWO OPTIONS FOR HANDLING FLOATING-POINT EXCEPTIONS

Just as for x87 FPU floating-point exceptions, the processor takes one of two possible courses of action when an SSE/SSE2/SSE3 instruction raises a floating-point exception:

- If the exception being raised is masked (by setting the corresponding mask bit in the MXCSR to 1), then a default result is produced which is acceptable in most situations. No external indication of the exception is given, but the corresponding exception flags in the MXCSR are set and may be examined later. Note though that for packed operations, an exception flag that is set in the MXCSR will not tell which of the sub-operands caused the event to occur.
- If the exception being raised is not masked (by setting the corresponding mask bit in the MXCSR to 0), a software exception handler previously registered by the user with operating system support will be invoked through the SIMD floating-point exception (#XM, exception 19). This case is discussed below in Section D.2, "Software Exception Handling."

D.2 SOFTWARE EXCEPTION HANDLING

The #XM handler is usually part of the system software (the operating system kernel). Note that an interrupt descriptor table (IDT) entry must have been previously set up for exception 19 (refer to Chapter 7, "Interrupt and Exception Handling," in the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3A). Some compilers use specific run-time libraries to assist in floating-point exception handling. If any x87 FPU floating-point operations are going to be performed that might raise floating-point exceptions, then the exception handling routine must either disable all floating-point exceptions (for example, loading a local control word with FLDCW), or it must be implemented as re-entrant. If this is not the case, the routine has to clear the status flags for x87 FPU exceptions or to mask all x87 FPU floating-point exceptions. For SIMD floating-point exceptions though, the exception flags in MXCSR do not have to be cleared, even if they remain unmasked (but they may still be cleared). Exceptions are in this case precise and occur immediately, and a SIMD floating-point exception status flag that is set when the corresponding exception is unmasked will not generate an exception.

Typical actions performed by this low-level exception handling routine are:

- Incrementing an exception counter for later display or printing.
- Printing or displaying diagnostic information (e.g., the MXCSR and XMM registers).
- Aborting further execution, or using the exception pointers to build an instruction that will run without exception and executing it.
- Storing information about the exception in a data structure that will be passed to a higher level user exception handler.

In most cases (and this applies also to the Intel SSE, SSE2, and SSE3 instructions), there will be three main components of a low-level floating-point exception handler: a prologue, a body, and an epilogue.

The prologue performs functions that must be protected from possible interruption by higher-priority sources - typically saving registers and transferring diagnostic information from the processor to memory. When the critical processing has been completed, the prologue may re-enable interrupts to allow higher-priority interrupt handlers to preempt the exception handler (assuming that the interrupt handler was called through an interrupt gate, meaning that the processor cleared the interrupt enable (IF) flag in the EFLAGS register - refer to Section 6.5.1, "Call and Return Operation for Interrupt or Exception Handling Procedures").

The body of the exception handler examines the diagnostic information and makes a response that is application-dependent. It may range from halting execution, to displaying a message, to attempting to fix the problem and then proceeding with normal execution, to setting up a data structure, calling a higher-level user exception handler and continuing execution upon return from it. This latter case will be assumed in Section D.4, "SIMD Floating-Point Exceptions and the IEEE Standard 754," below.

Finally, the epilogue essentially reverses the actions of the prologue, restoring the processor state so that normal execution can be resumed.

The following example represents a typical exception handler. To link it with Example D-2 that will follow in Section D.4.3, "Example SIMD Floating-Point Emulation Implementation," assume that the body of the handler (not shown here in detail) passes the saved state to a routine that will examine in turn all the sub-operands of the excepting instruction, invoking a user floating-point exception handler if a particular set of sub-operands raises an unmasked (enabled) exception, or emulating the instruction otherwise.

Example D-1. SIMD Floating-Point Exception Handler

```
SIMD_FP_EXC_HANDLER PROC

;PROLOGUE
;SAVE REGISTERS THAT MIGHT BE USED BY THE EXCEPTION HANDLER
   PUSH EBP                          ;SAVE EBP
   PUSH EAX                          ;SAVE EAX
   ...
   MOV EBP, ESP                      ;SAVE ESP in EBP
   SUB ESP, 512                      ;ALLOCATE 512 BYTES
   AND ESP, 0fffffff0h               ;MAKE THE ADDRESS 16-BYTE ALIGNED
   FXSAVE [ESP]                      ;SAVE FP, MMX, AND SIMD FP STATE
   PUSH [EBP+EFLAGS_OFFSET]          ;COPY OLD EFLAGS TO STACK TOP
   POPFD                             ;RESTORE THE INTERRUPT ENABLE FLAG IF
                                     ;TO VALUE BEFORE SIMD FP EXCEPTION

;BODY
;APPLICATION-DEPENDENT EXCEPTION HANDLING CODE GOES HERE
   LDMXCSR LOCAL_MXCSR               ;LOAD LOCAL MXCSR VALUE IF NEEDED
   ...
   ...
;EPILOGUE
   FXRSTOR [ESP]                     ;RESTORE MODIFIED STATE IMAGE
   MOV ESP, EBP                      ;DE-ALLOCATE STACK SPACE
   ...
   POP EAX                           ;RESTORE EAX
   POP EBP                           ;RESTORE EBP
   IRET                              ;RETURN TO INTERRUPTED CALCULATION
SIMD_FP_EXC_HANDLER ENDP
```

D.3 EXCEPTION SYNCHRONIZATION

An SSE/SSE2/SSE3 instruction can execute in parallel with other similar instructions, with integer instructions, and with floating-point or MMX instructions. Unlike for x87 instructions, special precaution for exception synchronization is not necessary in this case. This is because floating-point exceptions for SSE/SSE2/SSE3 instructions occur immediately and are not delayed until a subsequent floating-point instruction is executed. However, floating-point emulation may be necessary when unmasked floating-point exceptions are generated.

D.4 SIMD FLOATING-POINT EXCEPTIONS AND THE IEEE STANDARD 754

SSE/SSE2/SSE3 extensions are 100% compatible with the IEEE Standard 754 for Floating-Point Arithmetic, satisfying all of its mandatory requirements (when the flush-to-zero or denormals-are-zeros modes are not enabled). But a programming environment that includes SSE/SSE2/SSE3 instructions will comply with both the obligatory and the strongly recommended requirements of the IEEE Standard 754 regarding floating-point exception handling, only as a combination of hardware and software (which is acceptable). The standard states that a user should be able to request a trap on any of the five floating-point exceptions (note that the denormal exception is an IA-32 addition), and it also specifies the values (operands or result) to be delivered to the exception handler.

The main issue is that for SSE/SSE2/SSE3 instructions that raise post-computation exceptions (traps: overflow, underflow, or inexact), unlike for x87 FPU instructions, the processor does not provide the result recommended by IEEE Standard 754 to the user handler. If a user program needs the result of an instruction that generated a post-computation exception, it is the responsibility of the software to produce this result by emulating the faulting SSE/SSE2/SSE3 instruction. Another issue is that the standard does not specify explicitly how to handle multiple floating-point exceptions that occur simultaneously. For packed operations, a logical OR of the flags that would be set by each sub-operation is used to set the exception flags in the MXCSR. The following subsections present one possible way to solve these problems.

D.4.1 Floating-Point Emulation

Every operating system must provide a kernel level floating-point exception handler (a template was presented in Section D.2, "Software Exception Handling," above). In the following discussion, assume that a user mode floating-point exception filter is supplied for SIMD floating-point exceptions (for example as part of a library of C functions), that a user program can invoke in order to handle unmasked exceptions. The user mode floating-point exception filter (not shown here) has to be able to emulate the subset of Intel SSE, SSE2, and SSE3 instructions that can generate numeric exceptions, and has to be able to invoke a user provided floating-point exception handler for floating-point exceptions. When a floating-point exception that is not masked is raised by an Intel SSE, SSE2, and SSE3 instruction, the low-level floating-point exception handler will be called. This low-level handler may in turn call the user mode floating-point exception filter. The filter function receives the original operands of the excepting instruction as no results are provided by the hardware, whether a pre-computation or a post-computation exception has occurred. The filter will unpack the operands into up to four sets of sub-operands, and will submit them one set at a time to an emulation function (See Example D-2 in Section D.4.3, "Example SIMD Floating-Point Emulation Implementation.") The emulation function will examine the sub-operands, and will possibly redo the necessary calculation.

Two cases are possible:

- If an unmasked (enabled) exception would occur in this process, the emulation function will return to its caller (the filter function) with the appropriate information. The filter will invoke a (previously registered) user floating-point exception handler for this set of sub-operands, and will record the result upon return from the user handler (provided the user handler allows continuation of the execution).
- If no unmasked (enabled) exception would occur, the emulation function will determine and will return to its caller the result of the operation for the current set of sub-operands (it has to be IEEE Standard 754 compliant). The filter function will record the result (plus any new flag settings).

The user level filter function will then call the emulation function for the next set of sub-operands (if any). When done with all the operand sets, the partial results will be packed (if the excepting instruction has a packed floating-point result, which is true for most SSE/SSE2/SSE3 numeric instructions) and the filter will return to the low-level exception handler, which in turn will return from the interruption, allowing execution to continue. Note that the

instruction pointer (EIP) has to be altered to point to the instruction following the excepting instruction, in order to continue execution correctly.

If a user mode floating-point exception filter is not provided, then all the work for decoding the excepting instruction, reading its operands, emulating the instruction for the components of the result that do not correspond to unmasked floating-point exceptions, and providing the compounded result will have to be performed by the user-provided floating-point exception handler.

Actual emulation might have to take place for one operand or pair of operands for scalar operations, and for all sub-operands or pairs of sub-operands for packed operations. The steps to perform are the following:

- The excepting instruction has to be decoded and the operands have to be read from the saved context.
- The instruction has to be emulated for each (pair of) sub-operand(s); if no floating-point exception occurs, the partial result has to be saved; if a masked floating-point exception occurs, the masked result has to be produced through emulation and saved, and the appropriate status flags have to be set; if an unmasked floating-point exception occurs, the result has to be generated by the user provided floating-point exception handler, and the appropriate status flags have to be set.
- The partial results have to be combined and written to the context that will be restored upon application program resumption.

A diagram of the control flow in handling an unmasked floating-point exception is presented below.

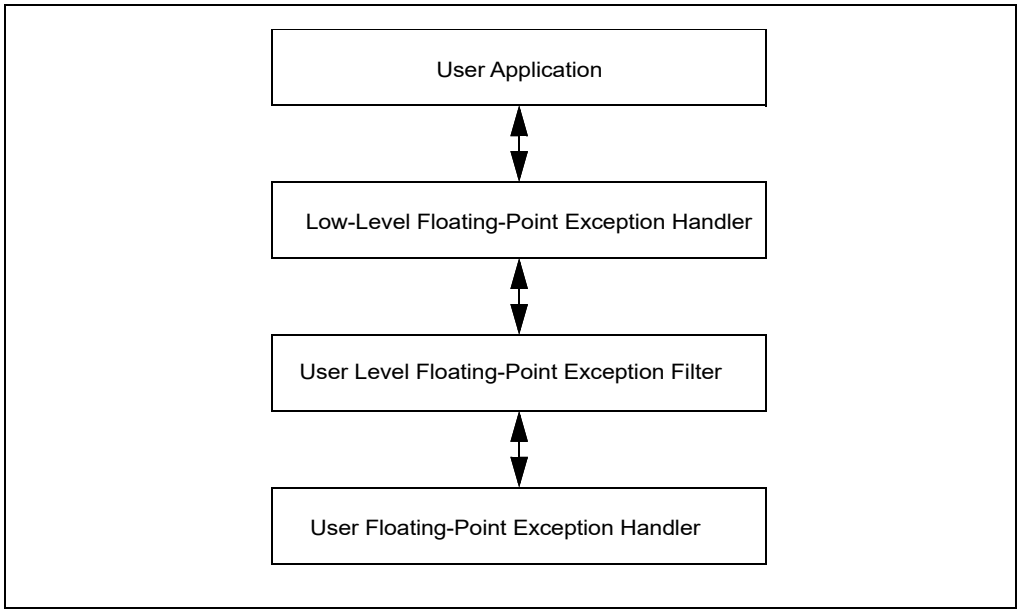


Figure D-1. Control Flow for Handling Unmasked Floating-Point Exceptions

From the user-level floating-point filter, Example D-2 in Section D.4.3, "Example SIMD Floating-Point Emulation Implementation," presents only the floating-point emulation part. In order to understand the actions involved, the expected response to exceptions has to be known for all Intel SSE, SSE2, and SSE3 numeric instructions in two situations: with exceptions enabled (unmasked result), and with exceptions disabled (masked result). The latter can be found in Section 6.5, "Interrupts and Exceptions." The response to NaN operands that do not raise an exception is specified in Section 4.8.3.4, "NaNs." Operations on NaNs are explained in the same source. This response is also discussed in more detail in the next subsection, along with the unmasked and masked responses to floating-point exceptions.

D.4.2 Intel® SSE, SSE2, and SSE3 Response To Floating-Point Exceptions

This subsection specifies the unmasked response expected from the Intel SSE, SSE2, and SSE3 instructions that raise floating-point exceptions. The masked response is given in parallel, as it is necessary in the emulation process

of the instructions that raise unmasked floating-point exceptions. The response to NaN operands is also included in more detail than in Section 4.8.3.4, "NaNs." For floating-point exception priority, refer to "Priority Among Simultaneous Exceptions and Interrupts" in Chapter 7, "Interrupt and Exception Handling," of the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3A.

D.4.2.1 Numeric Exceptions

There are six classes of numeric (floating-point) exception conditions that can occur: Invalid operation (#I), Divide-by-Zero (#Z), Denormal Operand (#D), Numeric Overflow (#O), Numeric Underflow (#U), and Inexact Result (precision) (#P). #I, #Z, #D are pre-computation exceptions (floating-point faults), detected before the arithmetic operation. #O, #U, #P are post-computation exceptions (floating-point traps).

Users can control how the Intel SSE, SSE2, and SSE3 floating-point exceptions are handled by setting the mask/unmask bits in MXCSR. Masked exceptions are handled by the processor, or by software if they are combined with unmasked exceptions occurring in the same instruction. Unmasked exceptions are usually handled by the low-level exception handler, in conjunction with user-level software.

D.4.2.2 Results of Operations with NaN Operands or a NaN Result for Intel® SSE, SSE2, and SSE3 Numeric Instructions

The tables below (E-1 through E-10) specify the response of Intel SSE, SSE2, and SSE3 instructions to NaN inputs, or to other inputs that lead to NaN results.

These results will be referenced by subsequent tables (e.g., E-10). Most operations do not raise an invalid exception for quiet NaN operands, but even so, they will have higher precedence over raising floating-point exceptions other than invalid operation.

Note that the single precision QNaN Indefinite value is FFC00000H, the double precision QNaN Indefinite value is FFF8000000000000H, and the Integer Indefinite value is 80000000H (not a floating-point number, but it can be the result of a conversion instruction from floating-point to integer).

For an unmasked exception, no result will be provided by the hardware to the user handler. If a user registered floating-point exception handler is invoked, it may provide a result for the excepting instruction, that will be used if execution of the application code is continued after returning from the interruption.

In Tables D-1 through Table D-12, the specified operands cause an invalid exception, unless the unmasked result is marked with "not an exception". In this latter case, the unmasked and masked results are the same.

Table D-1. ADDPS, ADDSS, SUBPS, SUBSS, MULPS, MULSS, DIVPS, DIVSS, ADDPD, ADDSD, SUBPD, SUBSD, MULPD, MULSD, DIVPD, DIVSD, ADDSUBPS, ADDSUBPD, HADDPS, HADDPD, HSUBPS, and HSUBPD

Source Operands	Masked Result	Unmasked Result
SNaN1 op[1] SNaN2	SNaN1 | 00400000H or SNaN1 | 0008000000000000H[2]	None
SNaN1 op QNaN2	SNaN1 | 00400000H or SNaN1 | 0008000000000000H[2]	None
QNaN1 op SNaN2	QNaN1	None
QNaN1 op QNaN2	QNaN1	QNaN1 (not an exception)
SNaN op real value	SNaN | 00400000H or SNaN1 | 0008000000000000H[2]	None
Real value op SNaN	SNaN | 00400000H or SNaN1 | 0008000000000000H[2]	None
QNaN op real value	QNaN	QNaN (not an exception)
Real value op QNaN	QNaN	QNaN (not an exception)

Table D-1. ADDPS, ADDSS, SUBPS, SUBSS, MULPS, MULSS, DIVPS, DIVSS, ADDPD, ADDSD, SUBPD, SUBSD, MULPD, MULSD, DIVPD, DIVSD, ADDSUBPS, ADDSUBPD, HADDPS, HADDPD, HSUBPS, and HSUBPD (Contd.)

Source Operands	Masked Result	Unmasked Result
Neither source operand is SNaN, but #I is signaled (e.g., for Inf - Inf, Inf * 0, Inf / Inf, 0/0)	Single precision or double precision QNaN Indefinite	None

NOTES:

1. For Tables E-1 to E-12: op denotes the operation to be performed.
2. SNaN | 00400000H is a quiet NaN in single precision format (if SNaN is in single precision) and SNaN | 0008000000000000H is a quiet NaN in double precision format (if SNaN is in double precision), obtained from the signaling NaN given as input.
3. Operations involving only quiet NaNs do not raise floating-point exceptions.

Table D-2. CMPPS.EQ, CMPSS.EQ, CMPPS.ORD, CMPSS.ORD, CMPPD.EQ, CMPSD.EQ, CMPPD.ORD, and CMPSD.ORD

Source Operands	Masked Result	Unmasked Result
NaN op Opd2 (any Opd2)	00000000H or 0000000000000000H[1]	00000000H or 0000000000000000H[1] (not an exception)
Opd1 op NaN (any Opd1)	00000000H or 0000000000000000H[1]	00000000H or 0000000000000000H[1] (not an exception)

NOTE:

1. 32-bit results are for single, and 64-bit results for double precision operations.

Table D-3. CMPPS.NEQ, CMPSS.NEQ, CMPPS.UNORD, CMPSS.UNORD, CMPPD.NEQ, CMPSD.NEQ, CMPPD.UNORD, and CMPSD.UNORD

Source Operands	Masked Result	Unmasked Result
NaN op Opd2 (any Opd2)	FFFFFFFFH or FFFFFFFFFFFFFFFFH[1]	FFFFFFFFH or FFFFFFFFFFFFFFFFH[1] (not an exception)
Opd1 op NaN (any Opd1)	FFFFFFFFH or FFFFFFFFFFFFFFFFH[1]	FFFFFFFFH or FFFFFFFFFFFFFFFFH[1] (not an exception)

NOTE:

1. 32-bit results are for single, and 64-bit results for double precision operations.

Table D-4. CMPPS.LT, CMPSS.LT, CMPPS.LE, CMPSS.LE, CMPPD.LT, CMPSD.LT, CMPPD.LE, and CMPSD.LE

Source Operands	Masked Result	Unmasked Result
NaN op Opd2 (any Opd2)	00000000H or 0000000000000000H[1]	None
Opd1 op NaN (any Opd1)	00000000H or 0000000000000000H[1]	None

NOTE:

1. 32-bit results are for single, and 64-bit results for double precision operations.

Table D-5. CMPPS.NLT, CMPSS.NLT, CMPPS.NLE, CMPSS.NLE, CMPPD.NLT, CMPSD.NLT, CMPPD.NLE, and CMPSD.NLE

Source Operands	Masked Result	Unmasked Result
NaN op Opd2 (any Opd2)	FFFFFFFFH or FFFFFFFFFFFFFFFFH[1]	None
Opd1 op NaN (any Opd1)	FFFFFFFFH or FFFFFFFFFFFFFFFFH[1]	None

NOTE:

1. 32-bit results are for single, and 64-bit results for double precision operations.

Table D-6. COMISS and COMISD

Source Operands	Masked Result	Unmasked Result
SNaN op Opd2 (any Opd2)	OF, SF, AF = 000 ZF, PF, CF = 111	None
Opd1 op SNaN (any Opd1)	OF, SF, AF = 000 ZF, PF, CF = 111	None
QNaN op Opd2 (any Opd2)	OF, SF, AF = 000 ZF, PF, CF = 111	None
Opd1 op QNaN (any Opd1)	OF, SF, AF = 000 ZF, PF, CF = 111	None

Table D-7. UCOMISS and UCOMISD

Source Operands	Masked Result	Unmasked Result
SNaN op Opd2 (any Opd2)	OF, SF, AF = 000 ZF, PF, CF = 111	None
Opd1 op SNaN (any Opd1)	OF, SF, AF = 000 ZF, PF, CF = 111	None
QNaN op Opd2 (any Opd2 ≠ SNaN)	OF, SF, AF = 000 ZF, PF, CF = 111	OF, SF, AF = 000 ZF, PF, CF = 111 (not an exception)
Opd1 op QNaN (any Opd1 ≠ SNaN)	OF, SF, AF = 000 ZF, PF, CF = 111	OF, SF, AF = 000 ZF, PF, CF = 111 (not an exception)

Table D-8. CVTPS2PI, CVTSS2SI, CVTTPS2PI, CVTTSS2SI, CVTPD2PI, CVTSD2SI, CVTTPD2PI, CVTTSD2SI, CVTPS2DQ, CVTTPS2DQ, CVTPD2DQ, and CVTTPD2DQ

Source Operand	Masked Result	Unmasked Result
SNaN	80000000H or 8000000000000000[1] (Integer Indefinite)	None
QNaN	80000000H or 8000000000000000[1] (Integer Indefinite)	None

NOTE:

1. 32-bit results are for single, and 64-bit results for double precision operations.

Table D-9. MAXPS, MAXSS, MINPS, MINSS, MAXPD, MAXSD, MINPD, and MINSD

Source Operands	Masked Result	Unmasked Result
Opd1 op NaN2 (any Opd1)	NaN2	None
NaN1 op Opd2 (any Opd2)	Opd2	None

NOTE:

1. SNaN and QNaN operands raise an Invalid Operation fault.

Table D-10. SQRTPS, SQRTSS, SQRTPD, and SQRTSD

Source Operand	Masked Result	Unmasked Result
QNaN	QNaN	QNaN (not an exception)
SNaN	SNaN | 00400000H or SNaN | 0008000000000000H[1]	None
Source operand is not SNaN; but #I is signaled (e.g., for sqrt (-1.0))	Single precision or double precision QNaN Indefinite	None

NOTE:

1. SNaN | 00400000H is a quiet NaN in single precision format (if SNaN is in single precision) and SNaN | 0008000000000000H is a quiet NaN in double precision format (if SNaN is in double precision), obtained from the signaling NaN given as input.

Table D-11. CVTPS2PD and CVTSS2SD

Source Operands	Masked Result	Unmasked Result
QNaN	QNaN1[1]	QNaN1[1] (not an exception)
SNaN	QNaN1[2]	None

NOTES:

1. The double precision output QNaN1 is created from the single precision input QNaN as follows: the sign bit is preserved, the 8-bit exponent FFH is replaced by the 11-bit exponent 7FFH, and the 24-bit significand is extended to a 53-bit significand by appending 29 bits equal to 0.
2. The double precision output QNaN1 is created from the single precision input SNaN as follows: the sign bit is preserved, the 8-bit exponent FFH is replaced by the 11-bit exponent 7FFH, and the 24-bit significand is extended to a 53-bit significand by pending 29 bits equal to 0. The second most significant bit of the significand is changed from 0 to 1 to convert the signaling NaN into a quiet NaN.

Table D-12. CVTPD2PS and CVTSD2SS

Source Operands	Masked Result	Unmasked Result
QNaN	QNaN1[1]	QNaN1[1] (not an exception)
SNaN	QNaN1[2]	None

NOTES:

1. The single precision output QNaN1 is created from the double precision input QNaN as follows: the sign bit is preserved, the 11-bit exponent 7FFH is replaced by the 8-bit exponent FFH, and the 53-bit significand is truncated to a 24-bit significand by removing its 29 least significant bits.
2. The single precision output QNaN1 is created from the double precision input SNaN as follows: the sign bit is preserved, the 11-bit exponent 7FFH is replaced by the 8-bit exponent FFH, and the 53-bit significand is truncated to a 24-bit significand by removing its 29 least significant bits. The second most significant bit of the significand is changed from 0 to 1 to convert the signaling NaN into a quiet NaN.

D.4.2.3 Condition Codes, Exception Flags, and Response for Masked and Unmasked Numeric Exceptions

In the following, the masked response is what the processor provides when a masked exception is raised by an Intel SSE, SSE2, or SSE3 numeric instruction. The same response is provided by the floating-point emulator for Intel SSE, SSE2, and SSE3 numeric instructions, when certain components of the quadruple input operands generate exceptions that are masked (the emulator also generates the correct answer, as specified by IEEE Standard 754 wherever applicable, in the case when no floating-point exception occurs). The unmasked response is what the emulator provides to the user handler for those components of the packed operands of Intel SSE, SSE2, and SSE3 instructions that raise unmasked exceptions. Note that for pre-computation exceptions (floating-point faults), no result is provided to the user handler. For post-computation exceptions (floating-point traps), a result is provided to the user handler, as specified below.

In the following tables, the result is denoted by 'res', with the understanding that for the actual instruction, the destination coincides with the first source operand (except for COMISS, UCOMISS, COMISD, and UCOMISD, whose destination is the EFLAGS register).

Table D-13. #I - Invalid Operations

Instruction	Condition	Masked Response	Unmasked Response and Exception Code
ADDPS ADDPD ADDSS ADDSD HADDPS HADDPD	src1 or src2[1] = SNaN	Refer to Table D-1 for NaN operands, #IA = 1	src1, src2 unchanged; #IA = 1
ADDSUBPS (the addition component) ADDSUBPD (the addition component)	src1 = +Inf, src2 = -Inf or src1 = -Inf, src2 = +Inf	res[1] = QNaN Indefinite, #IA = 1	
SUBPS SUBPD SUBSS SUBSD HSUBPS HSUBPD	src1 or src2 = SNaN	Refer to Table D-1 for NaN operands, #IA = 1	src1, src2 unchanged; #IA = 1
ADDSUBPS (the subtraction component) ADDSUBPD (the subtraction component)	src1 = +Inf, src2 = +Inf or src1 = -Inf, src2 = -Inf	res = QNaN Indefinite, #IA = 1	
MULPS MULPD	src1 or src2 = SNaN	Refer to Table D-1 for NaN operands, #IA = 1	src1, src2 unchanged; #IA = 1
MULSS MULSD	src1 = ±Inf, src2 = ±0 or src1 = ±0, src2 = ±Inf	res = QNaN Indefinite, #IA = 1	
DIVPS DIVPD	src1 or src2 = SNaN	Refer to Table D-1 for NaN operands, #IA = 1	src1, src2 unchanged; #IA = 1
DIVSS DIVSD	src1 = ±Inf, src2 = ±Inf or src1 = ±0, src2 = ±0	res = QNaN Indefinite, #IA = 1	
SQRTPS SQRTPD SQRTSS SQRTSD	src = SNaN	Refer to Table D-10 for NaN operands, #IA = 1	src unchanged, #IA = 1
	src < 0 (note that -0 < 0 is false)	res = QNaN Indefinite, #IA = 1	

Table D-13. #I - Invalid Operations (Contd.)

Instruction	Condition	Masked Response	Unmasked Response and Exception Code
MAXPS MAXSS MAXPD MAXSD	src1 = NaN or src2 = NaN	res = src2, #IA = 1	src1, src2 unchanged; #IA = 1
MINPS MINSS MINPD MINSD	src1 = NaN or src2 = NaN	res = src2, #IA = 1	src1, src2 unchanged; #IA = 1
CMPPS.LT CMPPS.LE CMPPS.NLT CMPPS.NLE CMPSS.LT CMPSS.LE CMPSS.NLT CMPSS.NLE CMPPD.LT CMPPD.LE CMPPD.NLT CMPPD.NLE CMPSD.LT CMPSD.LE CMPSD.NLT CMPSD.NLE	src1 = NaN or src2 = NaN	Refer to Table D-4 and Table D-5 for NaN operands; #IA = 1	src1, src2 unchanged; #IA = 1
COMISS COMISD	src1 = NaN or src2 = NaN	Refer to Table D-6 for NaN operands	src1, src2, EFLAGS unchanged; #IA = 1
UCOMISS UCOMISD	src1 = SNaN or src2 = SNaN	Refer to Table D-7 for NaN operands	src1, src2, EFLAGS unchanged; #IA = 1
CVTPS2PI CVTSS2SI CVTPD2PI CVTSD2SI CVTPS2DQ CVTPD2DQ	src = NaN, ±Inf, or $\|(src)_{rnd}\| > 7FFFFFFFH$ and $(src)_{rnd} \neq 80000000H$ See Note[2] for information on rnd.	res = Integer Indefinite, #IA = 1	src unchanged, #IA = 1
CVTTPS2PI CVTTSS2SI CVTTPD2PI CVTTSD2SI CVTTPS2DQ CVTTPD2DQ	src = NaN, ±Inf, or $\|(src)_{rz}\| > 7FFFFFFFH$ and $(src)_{rz} \neq 80000000H$ See Note[2] for information on rz.	res = Integer Indefinite, #IA = 1	src unchanged, #IA = 1

Table D-13. #I - Invalid Operations (Contd.)

Instruction	Condition	Masked Response	Unmasked Response and Exception Code
CVTPS2PD CVTSS2SD	src = SNAN	Refer to Table D-11 for NaN operands	src unchanged, #IA = 1
CVTPD2PS CVTSD2SS	src = SNAN	Refer to Table D-12 for NaN operands	src unchanged, #IA = 1

NOTES:

1. For Tables E-13 to E-18:
   - src denotes the single source operand of a unary operation.
   - src1, src2 denote the first and second source operand of a binary operation.
   - res denotes the numerical result of an operation.
2. rnd signifies the user rounding mode from MXCSR, and rz signifies the rounding mode toward zero. (truncate), when rounding a floating-point value to an integer. For more information, refer to Table 4-9.
3. For NAN encodings, see Table 4-3.

Table D-14. #Z - Divide-by-Zero

Instruction	Condition	Masked Response	Unmasked Response and Exception Code
DIVPS DIVSS DIVPD DIVPS	src1 = finite non-zero (normal, or denormal) src2 = ±0	res = ±Inf, #ZE = 1	src1, src2 unchanged; #ZE = 1

Table D-15. #D - Denormal Operand

Instruction	Condition	Masked Response	Unmasked Response and Exception Code
ADDPS ADDPD ADDSUBPS ADDSUBPD HADDPS HADDPD SUBPS SUBPD HSUBPS HSUBPD MULPS MULPD DIVPS DIVPD SQRTPS SQRTPD MAXPS MAXPD MINPS MINPD ADDSS ADDSD SUBSS SUBSD MULSS MULSD DIVSS DIVSD SQRTSS SQRTSD MAXSS MAXSD MINSS MINSD CVTPS2PD CVTSS2SD CVTPD2PS CVTSD2SS	src1 = denormal[1] or src2 = denormal (and the DAZ bit in MXCSR is 0)	res = Result rounded to the destination precision and using the bounded exponent, but only if no unmasked post-computation exception occurs; #DE = 1.	src1, src2 unchanged; #DE = 1 Note that SQRT, CVTPS2PD, CVTSS2SD, CVTPD2PS, CVTSD2SS have only 1 src.
CMPPS CMPPD CMPSS CMPSD	src1 = denormal[1] or src2 = denormal (and the DAZ bit in MXCSR is 0)	Comparison result, stored in the destination register; #DE = 1	src1, src2 unchanged; #DE = 1
COMISS COMISD UCOMISS UCOMISD	src1 = denormal[1] or src2 = denormal (and the DAZ bit in MXCSR is 0)	Comparison result, stored in the EFLAGS register; #DE = 1	src1, src2 unchanged; #DE = 1

NOTE:

1. For denormal encodings, see Section 4.8.3.2, “Normalized and Denormalized Finite Numbers.”

Table D-16. #O - Numeric Overflow

Instruction	Condition	Masked Response			Unmasked Response and Exception Code
ADDPS ADDSUBPS HADDPS SUBPS HSUBPS MULPS DIVPS ADDSS SUBSS MULSS DIVSS CVTPD2PS CVTSD2SS	Rounded result > largest single precision finite normal value	**Rounding**	**Sign**	**Result & Status Flags**	res = (result calculated with unbounded exponent and rounded to the destination precision) / 2^{192} #OE = 1 #PE = 1 if the result is inexact

		To nearest	+
			-
				#OE = 1, #PE = 1
				res = $+\infty$
				res = $-\infty$	

		Toward $-\infty$	+
			-
				#OE = 1, #PE = 1
				res = 1.11...1 * 2^{127}
				res = $-\infty$	

		Toward $+\infty$	+
			-
				#OE = 1, #PE = 1
				res = $+\infty$
				res = -1.11...1 * 2^{127}	

		Toward 0	+
			-
				#OE = 1, #PE = 1
				res = 1.11...1 * 2^{127}
				res = -1.11...1 * 2^{127}	

ADDPD ADDSUBPD HADDPD SUBPD HSUBPD MULPD DIVPD ADDSD SUBSD MULSD DIVSD	Rounded result > largest double precision finite normal value	**Rounding**	**Sign**	**Result & Status Flags**	res = (result calculated with unbounded exponent and rounded to the destination precision) / 2^{1536} ▪ #OE = 1 ▪ #PE = 1 if the result is inexact

		To nearest	+
			-
				#OE = 1, #PE = 1
				res = $+\infty$
				res = $-\infty$	

		Toward $-\infty$	+
			-
				#OE = 1, #PE = 1
				res = 1.11...1 * 2^{1023}
				res = $-\infty$	

		Toward $+\infty$	+
			-
				#OE = 1, #PE = 1
				res = $+\infty$
				res = -1.11...1 * 2^{1023}	

		Toward 0	+
			-
				#OE = 1, #PE = 1
				res = 1.11...1 * 2^{1023}
				res = -1.11...1 * 2^{1023}	

Table D-17. #U - Numeric Underflow

Instruction	Condition	Masked Response	Unmasked Response and Exception Code
ADDPS ADDSUBPS HADDPS SUBPS HSUBPS MULPS DIVPS ADDSS SUBSS MULSS DIVSS CVTPD2PS CVTSD2SS	Result calculated with unbounded exponent and rounded to the destination precision < smallest single precision finite normal value.	res = ±0, denormal, or normal #UE = 1 and #PE = 1, but only if the result is inexact	res = (result calculated with unbounded exponent and rounded to the destination precision) * 2^{192} ▪ #UE = 1 ▪ #PE = 1 if the result is inexact
ADDPD ADDSUBPD HADDPD SUBPD HSUBPD MULPD DIVPD ADDSD SUBSD MULSD DIVSD	Result calculated with unbounded exponent and rounded to the destination precision < smallest double precision finite normal value.	res = ±0, denormal or normal #UE = 1 and #PE = 1, but only if the result is inexact	res = (result calculated with unbounded exponent and rounded to the destination precision) * 2^{1536} ▪ #UE = 1 ▪ #PE = 1 if the result is inexact

Table D-18. #P - Inexact Result (Precision)

Instruction	Condition	Masked Response	Unmasked Response and Exception Code
ADDPS ADDPD ADDSUBPS ADDSUBPD HADDPS HADDPD SUBPS SUBPD HSUBPS HSUBPD MULPS MULPD DIVPS DIVPD SQRTPS SQRTPD CVTDQ2PS CVTPI2PS CVTPS2PI CVTPS2DQ CVTPD2PI CVTPD2DQ CVTPD2PS CVTTPS2PI CVTTPD2PI CVTTPD2DQ CVTTPS2DQ ADDSS ADDSD SUBSS SUBSD MULSS MULSD DIVSS DIVSD SQRTSS SQRTSD CVTSI2SS CVTSS2SI CVTSD2SI CVTSD2SS CVTTSS2SI CVTTSD2SI	The result is not exactly representable in the destination format.	res = Result rounded to the destination precision and using the bounded exponent, but only if no unmasked underflow or overflow conditions occur (this exception can occur in the presence of a masked underflow or overflow); #PE = 1.	Only if no underflow/overflow condition occurred, or if the corresponding exceptions are masked: ▪ Set #OE if masked overflow and set result as described above for masked overflow. ▪ Set #UE if masked underflow and set result as described above for masked underflow. If neither underflow nor overflow, res equals the result rounded to the destination precision and using the bounded exponent set #PE = 1.

D.4.3 Example SIMD Floating-Point Emulation Implementation

The sample code listed below may be considered as being part of a user-level floating-point exception filter for the intel SSE, SSE2, and SSE3 numeric instructions. It is assumed that the filter function is invoked by a low-level exception handler (invoked for exception 19 when an unmasked floating-point exception occurs), and that it operates as explained in Section D.4.1, "Floating-Point Emulation." The sample code does the emulation only for the SSE instructions for addition, subtraction, multiplication, and division. For this, it uses C code and x87 FPU operations. Operations corresponding to other Intel SSE, SSE2, and SSE3 numeric instructions can be emulated similarly. The example assumes that the emulation function receives a pointer to a data structure specifying a number of input parameters: the operation that caused the exception, a set of sub-operands (unpacked, of type float), the

rounding mode (the precision is always single), exception masks (having the same relative bit positions as in the MXCSR but starting from bit 0 in an unsigned integer), and flush-to-zero and denormals-are-zeros indicators.

The output parameters are a floating-point result (of type float), the cause of the exception (identified by constants not explicitly defined below), and the exception status flags. The corresponding C definition is:

```
typedef struct {
  unsigned int operation;              //SSE or SSE2 operation: ADDPS, ADDSS, ...
  unsigned int operand1_uint32; //first operand value
  unsigned int operand2_uint32; //second operand value (if any)
  float result_fval; // result value (if any)
  unsigned int rounding_mode; //rounding mode
  unsigned int exc_masks; //exception masks, in the order P,U,O,Z,D,I
  unsigned int exception_cause; //exception cause
  unsigned int status_flag_inexact; //inexact status flag
  unsigned int status_flag_underflow; //underflow status flag
  unsigned int status_flag_overflow; //overflow status flag
  unsigned int status_flag_divide_by_zero;
                                                //divide by zero status flag
  unsigned int status_flag_denormal_operand;
                                                //denormal operand status flag
  unsigned int status_flag_invalid_operation;
                                                //invalid operation status flag
  unsigned int ftz; // flush-to-zero flag
  unsigned int daz; // denormals-are-zeros flag
} EXC_ENV;
```

The arithmetic operations exemplified are emulated as follows:

1. If the denormals-are-zeros mode is enabled (the DAZ bit in MXCSR is set to 1), replace all the denormal inputs with zeroes of the same sign (the denormal flag is not affected by this change).
2. Perform the operation using x87 FPU instructions, with exceptions disabled, the original user rounding mode, and single precision. This reveals invalid, denormal, or divide-by-zero exceptions (if there are any) and stores the result in memory as a double precision value (whose exponent range is large enough to look like "unbounded" to the result of the single precision computation).
3. If no unmasked exceptions were detected, determine if the magnitude of the result is less than the smallest normal number that can be represented in single precision format, or greater than the largest normal number that can be represented in single precision format (huge). If an unmasked overflow or underflow occurs, calculate the scaled result that will be handed to the user exception handler, as specified by IEEE Standard 754.
4. If no exception was raised, calculate the result with a "bounded" exponent. If the result is tiny, it requires denormalization (shifting the significand right while incrementing the exponent to bring it into the admissible range of [-126,+127] for single precision floating-point numbers).

   The result obtained in step 2 cannot be used because it might incur a double rounding error (it was rounded to 24 bits in step 2, and might have to be rounded again in the denormalization process). To overcome this is, calculate the result as a double precision value, and store it to memory in single precision format.

   Rounding first to 53 bits in the significand, and then to 24 never causes a double rounding error (exact properties exist that state when double-rounding error occurs, but for the elementary arithmetic operations, the rule of thumb is that if an infinitely precise result is rounded to 2p+1 bits and then again to p bits, the result is the same as when rounding directly to p bits, which means that no double-rounding error occurs).
5. If the result is inexact and the inexact exceptions are unmasked, the calculated result will be delivered to the user floating-point exception handler.
6. The flush-to-zero case is dealt with if the result is tiny.

7. The emulation function returns RAISE_EXCEPTION to the filter function if an exception has to be raised (the exception_cause field indicates the cause). Otherwise, the emulation function returns DO_NOT_ RAISE_EXCEPTION. In the first case, the result is provided by the user exception handler called by the filter function. In the second case, it is provided by the emulation function. The filter function has to collect all the partial results, and to assemble the scalar or packed result that is used if execution is to continue.

Example D-2. SIMD Floating-Point Emulation

```
// masks for individual status word bits
#define PRECISION_MASK 20H
#define UNDERFLOW_MASK 10H
#define OVERFLOW_MASK 08H
#define ZERODIVIDE_MASK 04H
#define DENORMAL_MASK 02H
#define INVALID_MASK 01H

// 32-bit constants
static unsigned ZEROF_ARRAY[] = {00000000H};
#define  ZEROF *(float *) ZEROF_ARRAY
    // +0.0
static unsigned NZEROF_ARRAY[] = {80000000H};
#define  NZEROF *(float *) NZEROF_ARRAY
    // -0.0
static unsigned POSINFF_ARRAY[] = {7f800000H};
#define POSINFF *(float *)POSINFF_ARRAY
    // +Inf
static unsigned NEGINFF_ARRAY[] = {ff800000H};
#define NEGINFF *(float *)NEGINFF_ARRAY
    // -Inf

// 64-bit constants
static unsigned MIN_SINGLE_NORMAL_ARRAY [] = {00000000H, 38100000H};
#define MIN_SINGLE_NORMAL *(double *)MIN_SINGLE_NORMAL_ARRAY
    // +1.0 * 2^-126
static unsigned MAX_SINGLE_NORMAL_ARRAY [] = {70000000H, 47effffH};
#define MAX_SINGLE_NORMAL *(double *)MAX_SINGLE_NORMAL_ARRAY
    // +1.1...1*2^127
static unsigned TWO_TO_192_ARRAY[] = {00000000H, 4bf00000H};
#define TWO_TO_192 *(double *)TWO_TO_192_ARRAY
    // +1.0 * 2^192
static unsigned TWO_TO_M192_ARRAY[] = {00000000H, 33f00000H};
#define TWO_TO_M192 *(double *)TWO_TO_M192_ARRAY
    // +1.0 * 2^-192

// auxiliary functions
static int isnanf (unsigned int ); // returns 1 if f is a NaN, and 0 otherwise
static float quietf (unsigned int ); // converts a signaling NaN to a quiet
                          // NaN, and leaves a quiet NaN unchanged
static unsigned int check_for_daz (unsigned int ); // converts denormals
                          // to zeros of the same sign;
                          // does not affect any status flags

// emulation of SSE and SSE2 instructions using
// C code and x87 FPU instructions

unsigned int
simd_fp_emulate (EXC_ENV *exc_env)

{

  int uiopd1; // first operand of the add, subtract, multiply, or divide
  int uiopd2; // second operand of the add, subtract, multiply, or divide
  float res; // result of the add, subtract, multiply, or divide
  double dbl_res24; // result with 24-bit significand, but "unbounded" exponent
```

```
    // (needed to check tininess, to provide a scaled result to
    // an underflow/overflow trap handler, and in flush-to-zero mode)
double dbl_res;  // result in double precision format (needed to avoid a
   // double rounding error when denormalizing)
unsigned int result_tiny;
unsigned int result_huge;
unsigned short int sw; // 16 bits
unsigned short int cw; // 16 bits


// have to check first for faults (V, D, Z), and then for traps (O, U, I)

// initialize x87 FPU (floating-point exceptions are masked)
_asm {
  fninit;
}

result_tiny = 0;
result_huge = 0;

switch (exc_env->operation) {

  case ADDPS:
  case ADDSS:
  case SUBPS:
  case SUBSS:
  case MULPS:
  case MULSS:
  case DIVPS:
  case DIVSS:

    uiopd1 = exc_env->operand1_uint32; // copy as unsigned int
     // do not copy as float to avoid conversion
     // of SNaN to QNaN by compiled code
    uiopd2 = exc_env->operand2_uint32;
     // do not copy as float to avoid conversion of SNaN
     // to QNaN by compiled code
 uiopd1 = check_for_daz (uiopd1); // operand1 = +0.0 * operand1 if it is
    // denormal and DAZ=1
    uiopd2 = check_for_daz (uiopd2); // operand2 = +0.0 * operand2 if it is
     // denormal and DAZ=1

    // execute the operation and check whether the invalid, denormal, or
    // divide by zero flags are set and the respective exceptions enabled

    // set control word with rounding mode set to exc_env->rounding_mode,
    // single precision, and all exceptions disabled
    switch (exc_env->rounding_mode) {
      case ROUND_TO_NEAREST:
        cw = 003fH; // round to nearest, single precision, exceptions masked
        break;
      case ROUND_DOWN:
        cw = 043fH; // round down, single precision, exceptions masked
        break;
      case ROUND_UP:
        cw = 083fH; // round up, single precision, exceptions masked
        break;
      case ROUND_TO_ZERO:
        cw = 0c3fH; // round to zero, single precision, exceptions masked
        break;
      default:
        ;
    }
    __asm {
      fldcw WORD PTR cw;
```

```
}

// compute result and round to the destination precision, with
// "unbounded" exponent (first IEEE rounding)
switch (exc_env->operation) {

  case ADDPS:
  case ADDSS:
    // perform the addition
    __asm {
      fnclex;
      // load input operands
      fld DWORD PTR uiopd1; // may set denormal or invalid status flags
      fld DWORD PTR uiopd2; // may set denormal or invalid status flags
      faddp st(1), st(0); // may set inexact or invalid status flags
      // store result
      fstp  QWORD PTR dbl_res24; // exact
    }
    break;

  case SUBPS:
  case SUBSS:
    // perform the subtraction
    __asm {
      fnclex;
      // load input operands
      fld DWORD PTR uiopd1; // may set denormal or invalid status flags
      fld DWORD PTR uiopd2; // may set denormal or invalid status flags
      fsubp st(1), st(0); // may set the inexact or invalid status flags

     // store result
      fstp  QWORD PTR dbl_res24; // exact
    }
    break;

  case MULPS:
  case MULSS:
    // perform the multiplication
    __asm {
      fnclex;
      // load input operands
      fld DWORD PTR uiopd1; // may set denormal or invalid status flags
      fld DWORD PTR uiopd2; // may set denormal or invalid status flags
      fmulp st(1), st(0); // may set inexact or invalid status flags

     // store result
      fstp  QWORD PTR dbl_res24; // exact
    }
    break;

  case DIVPS:
  case DIVSS:
    // perform the division
    __asm {
      fnclex;
      // load input operands
      fld DWORD PTR uiopd1; // may set denormal or invalid status flags
      fld DWORD PTR uiopd2; // may set denormal or invalid status flags
      fdivp st(1), st(0); // may set the inexact, divide by zero, or
                          // invalid status flags
      // store result
      fstp  QWORD PTR dbl_res24; // exact
    }
    break;
```

```
        default:
          ; // will never occur

      }

      // read status word
      __asm {
        fstsw WORD PTR sw;
}

    if (sw & ZERODIVIDE_MASK)
    sw = sw & ~DENORMAL_MASK; // clear D flag for (denormal / 0)

      // if invalid flag is set, and invalid exceptions are enabled, take trap
      if (!(exc_env->exc_masks & INVALID_MASK) && (sw & INVALID_MASK)) {
        exc_env->status_flag_invalid_operation = 1;
        exc_env->exception_cause = INVALID_OPERATION;
        return (RAISE_EXCEPTION);
      }

    // checking for NaN operands has priority over denormal exceptions;
    // also fix for the SSE and SSE2
    // differences in treating two NaN inputs between the
    // instructions and other IA-32 instructions
    if (isnanf (uiopd1) || isnanf (uiopd2)) {

        if (isnanf (uiopd1) && isnanf (uiopd2))
            exc_env->result_fval = quietf (uiopd1);
        else
            exc_env->result_fval = (float)dbl_res24; // exact

        if (sw & INVALID_MASK) exc_env->status_flag_invalid_operation = 1;
        return (DO_NOT_RAISE_EXCEPTION);
      }

      // if denormal flag set, and denormal exceptions are enabled, take trap
      if (!(exc_env->exc_masks & DENORMAL_MASK) && (sw & DENORMAL_MASK)) {
        exc_env->status_flag_denormal_operand = 1;
        exc_env->exception_cause = DENORMAL_OPERAND;
        return (RAISE_EXCEPTION);
      }

      // if divide by zero flag set, and divide by zero exceptions are
      // enabled, take trap (for divide only)
      if (!(exc_env->exc_masks & ZERODIVIDE_MASK) && (sw & ZERODIVIDE_MASK)) {
        exc_env->status_flag_divide_by_zero = 1;
        exc_env->exception_cause = DIVIDE_BY_ZERO;
        return (RAISE_EXCEPTION);
      }

      // done if the result is a NaN (QNaN Indefinite)
      res = (float)dbl_res24;
      if (isnanf (*(unsigned int *)&res)) {
        exc_env->result_fval = res; // exact
        exc_env->status_flag_invalid_operation = 1;
        return (DO_NOT_RAISE_EXCEPTION);
      }

      // dbl_res24 is not a NaN at this point

      if (sw & DENORMAL_MASK) exc_env->status_flag_denormal_operand = 1;

      // Note: (dbl_res24 == 0.0 && sw & PRECISION_MASK) cannot occur
      if (-MIN_SINGLE_NORMAL < dbl_res24 && dbl_res24 < 0.0 ||
            0.0 < dbl_res24 && dbl_res24 < MIN_SINGLE_NORMAL) {
```

```
  result_tiny = 1;
}

// check if the result is huge
if (NEGINFF < dbl_res24 && dbl_res24 < -MAX_SINGLE_NORMAL ||
    MAX_SINGLE_NORMAL < dbl_res24 && dbl_res24 < POSINFF) {
  result_huge = 1;
}

// at this point, there are no enabled I,D, or Z exceptions
// to take; the instr.
// might lead to an enabled underflow, enabled underflow and inexact,
// enabled overflow, enabled overflow and inexact, enabled inexact, or
// none of these; if there are no U or O enabled exceptions, re-execute
// the instruction using IA-32 double precision format, and the
// user's rounding mode; exceptions must have
// been disabled before calling
// this function; an inexact exception may be reported on the 53-bit
// fsubp, fmulp, or on both the 53-bit and 24-bit conversions, while an
// overflow or underflow (with traps disabled) may be reported on the
// conversion from dbl_res to res

// check whether there is an underflow, overflow,
// or inexact trap to be taken
// if the underflow traps are enabled and the result is
// tiny, take underflow trap

if (!(exc_env->exc_masks & UNDERFLOW_MASK) && result_tiny) {
  dbl_res24 = TWO_TO_192 * dbl_res24; // exact
  exc_env->status_flag_underflow = 1;
  exc_env->exception_cause = UNDERFLOW;
  exc_env->result_fval = (float)dbl_res24; // exact
  if (sw & PRECISION_MASK) exc_env->status_flag_inexact = 1;
  return (RAISE_EXCEPTION);
}

// if overflow traps are enabled and the result is huge, take
// overflow trap
if (!(exc_env->exc_masks & OVERFLOW_MASK) &&  result_huge) {
  dbl_res24 = TWO_TO_M192 * dbl_res24; // exact
  exc_env->status_flag_overflow = 1;
  exc_env->exception_cause = OVERFLOW;
  exc_env->result_fval = (float)dbl_res24; // exact
  if (sw & PRECISION_MASK) exc_env->status_flag_inexact = 1;
  return (RAISE_EXCEPTION);
}

// set control word with rounding mode set to exc_env->rounding_mode,
// double precision, and all exceptions disabled
cw = cw | 0200H; // set precision to double
__asm {
  fldcw WORD PTR cw;
}

switch (exc_env->operation) {

  case ADDPS:
  case ADDSS:
    // perform the addition
    __asm {
      // load input operands
      fld DWORD PTR uiopd1; // may set the denormal status flag
      fld DWORD PTR uiopd2; // may set the denormal status flag
      faddp st(1), st(0); // rounded to 53 bits, may set the inexact
                          // status flag
```

```
        // store result
        fstp  QWORD PTR dbl_res; // exact, will not set any flag
      }
      break;

    case SUBPS:
    case SUBSS:
      // perform the subtraction
      __asm {
        // load input operands
        fld DWORD PTR uiopd1; // may set the denormal status flag
        fld DWORD PTR uiopd2; // may set the denormal status flag
        fsubp st(1), st(0);   // rounded to 53 bits, may set the inexact
                              // status flag
        // store result
        fstp  QWORD PTR dbl_res; // exact, will not set any flag
      }
      break;

    case MULPS:
    case MULSS:
      // perform the multiplication
      __asm {
        // load input operands
        fld DWORD PTR uiopd1; // may set the denormal status flag
        fld DWORD PTR uiopd2; // may set the denormal status flag
        fmulp st(1), st(0);   // rounded to 53 bits, exact
       // store result
        fstp  QWORD PTR dbl_res; // exact, will not set any flag
      }
      break;

    case DIVPS:
    case DIVSS:
      // perform the division
      __asm {
        // load input operands
        fld DWORD PTR uiopd1; // may set the denormal status flag
        fld DWORD PTR uiopd2; // may set the denormal status flag
        fdivp st(1), st(0);   // rounded to 53 bits, may set the inexact
                              // status flag
        // store result
        fstp  QWORD PTR dbl_res; // exact, will not set any flag
      }
      break;

    default:
      ; // will never occur

  }

  // calculate result for the case an inexact trap has to be taken, or
  // when no trap occurs (second IEEE rounding)
  res = (float)dbl_res;
      // may set P, U or O; may also involve denormalizing the result

  // read status word
  __asm {
    fstsw WORD PTR sw;
  }

  // if inexact traps are enabled and result is inexact, take inexact trap
  if (!(exc_env->exc_masks & PRECISION_MASK) &&
      ((sw & PRECISION_MASK) || (exc_env->ftz && result_tiny))) {
    exc_env->status_flag_inexact = 1;
```

```
    exc_env->exception_cause = INEXACT;
    if (result_tiny) {
      exc_env->status_flag_underflow = 1;

      // if ftz = 1 and result is tiny, result = 0.0
      // (no need to check for underflow traps disabled: result tiny and
      // underflow traps enabled would have caused taking an underflow
      // trap above)
      if (exc_env->ftz) {
        if (res > 0.0)
          res = ZEROF;
        else if (res < 0.0)
          res = NZEROF;
        // else leave res unchanged
      }
    }
    if (result_huge) exc_env->status_flag_overflow = 1;
    exc_env->result_fval = res;
    return (RAISE_EXCEPTION);
  }

  // if it got here, then there is no trap to be taken; the following must
  // hold: ((the MXCSR U exceptions are disabled  or
  //
  // the MXCSR underflow exceptions are enabled and the underflow flag is
  // clear and (the inexact flag is set or the inexact flag is clear and
  // the 24-bit result with unbounded exponent is not tiny)))
  // and (the MXCSR overflow traps are disabled or the overflow flag is
  // clear) and (the MXCSR inexact traps are disabled or the inexact flag
  // is clear)
  //
  // in this case, the result has to be delivered (the status flags are
  // sticky, so they are all set correctly already)

  // read status word to see if result is inexact
  __asm {
    fstsw WORD PTR sw;
  }

  if (sw & UNDERFLOW_MASK) exc_env->status_flag_underflow = 1;
  if (sw & OVERFLOW_MASK) exc_env->status_flag_overflow = 1;
  if (sw & PRECISION_MASK) exc_env->status_flag_inexact = 1;

  // if ftz = 1, and result is tiny (underflow traps must be disabled),
  // result = 0.0
  if (exc_env->ftz && result_tiny) {
    if (res > 0.0)
      res = ZEROF;
    else if (res < 0.0)
      res = NZEROF;
    // else leave res unchanged

    exc_env->status_flag_inexact = 1;
    exc_env->status_flag_underflow = 1;
  }

  exc_env->result_fval = res;
  if (sw & ZERODIVIDE_MASK) exc_env->status_flag_divide_by_zero = 1;
  if (sw & DENORMAL_MASK) exc_env->status_flag_denormal= 1;
  if (sw & INVALID_MASK) exc_env->status_flag_invalid_operation = 1;
  return (DO_NOT_RAISE_EXCEPTION);

  break;

case CMPPS:
```

```
    case CMPSS:

      ...

      break;

    case COMISS:
    case UCOMISS:

      ...

      break;

    case CVTPI2PS:
    case CVTSI2SS:

      ...

      break;

    case CVTPS2PI:
    case CVTSS2SI:
    case CVTTPS2PI:
    case CVTTSS2SI:

      ...

      break;

    case MAXPS:
    case MAXSS:
    case MINPS:
    case MINSS:

      ...

      break;

    case SQRTPS:
    case SQRTSS:

      ...

      break;

...

case UNSPEC:

      ...

      break;

    default:
      ...

  }

}
```

APPENDIX E
INTEL® MEMORY PROTECTION EXTENSIONS

NOTE

Intel® MPX has been deprecated and will not be available on any future processors.

E.1 INTEL® MEMORY PROTECTION EXTENSIONS (INTEL® MPX)

Intel® Memory Protection Extensions (Intel® MPX) is a new capability introduced into Intel Architecture. Intel MPX can increase the robustness of software when it is used in conjunction with compiler changes to check memory references, for those references whose compile-time normal intentions are usurped at runtime due to buffer overflow or underflow. Two of the most important goals of Intel MPX are to provide this capability at low performance overhead for newly compiled code, and to provide compatibility mechanisms with legacy software components. A direct benefit Intel MPX provides is hardening software against malicious attacks designed to cause or exploit buffer overruns. This chapter describes the software visible interfaces of this extension.

E.2 INTRODUCTION

Intel MPX is designed to allow a system (i.e., the logical processor(s) and the OS software) to run both Intel MPX enabled software and legacy software (written for processors without Intel MPX). When executing software containing a mixture of Intel MPX-unaware code (legacy code) and Intel MPX-enabled code, the legacy code does not benefit from Intel MPX, but it also does not experience any change in functionality or reduction in performance. The performance of Intel MPX-enabled code running on processors that do not support Intel MPX may be similar to the use of embedding NOPs in the instruction stream.

Intel MPX is designed such that an Intel MPX enabled application can link with, call into, or be called from legacy software (libraries, etc.) while maintaining existing application binary interfaces (ABIs). And in most cases, the benefit of Intel MPX requires minimal changes to the source code at the application programming interfaces (APIs) to legacy library/applications. As described later, Intel MPX associates **bounds** with pointers in a novel manner, and the Intel MPX hardware uses **bounds** to check that the pointer based accesses are suitably constrained. Intel MPX enabled software is not required to uniformly or universally utilize the new hardware capabilities over all memory references. Specifically, programmers can selectively use Intel MPX to protect a subset of pointers.

The code enabled for Intel MPX benefits from memory protection against vulnerability such as buffer overrun. Therefore there is a heightened incentive for software vendors to adopt this technology. At the same time, the security benefit of Intel MPX-protection can be implemented according to the business priorities of software vendors. A software vendor can choose to adopt Intel MPX in some modules to realize partial benefit from Intel MPX quickly, and introduce Intel MPX in other modules in phases (e.g., some programmer intervention might be required at the interface to legacy calls). This adaptive property of Intel MPX is designed to give software vendors control on their schedule and modularity of adoption. It also allows a software vendor to secure defense for higher priority or more attack-prone software first; and allows the use of Intel MPX features in one phase of software engineering (e.g., testing) and not in another (e.g., general release) as dictated by business realities.

The initial goal of Intel MPX is twofold: (1) provide means to defend a system against attacks that originate external to some trust perimeter where the trust perimeter subsumes the system memory and integral data repositories, and (2) provide means to pinpoint accidental logic defects in pointer usage, by undergirding memory references with hardware based pointer validation.

As with any instruction set extensions, Intel MPX can be used by application developers beyond detecting buffer overflow, the processor does not limit the use of Intel MPX for buffer overflow detection.

E.3 INTEL MPX PROGRAMMING ENVIRONMENT

Intel MPX introduces new **bounds register**s and new instructions that operate on bounds registers. Intel MPX allows an OS to support user mode software (operating at CPL=3) and supervisor mode software (CPL < 3) to add memory protection capability against buffer overrun. It provides controls to enable Intel MPX extensions for user mode and supervisor mode independently. Intel MPX extensions are designed to allow software to associate bounds with pointers, and allow software to check memory references against the bounds associated with the pointer to prevent out of bound memory access (thus preventing buffer overflow).The bounds registers hold lower bound and upper bound that can be checked when referencing memory. An out-of-bounds memory reference then causes a #BR exception. Intel MPX also introduces configuration facilities that the OS must manage to support enabling of user-mode (and/or supervisor-mode) software operations using bounds registers.

E.3.1 Detection and Enumeration of Intel MPX Interfaces

Detection of hardware support for processor extended state component is provided by the main CPUID.0DH.00H. Specifically, the return value in EDX:EAX of CPUID.0DH.00H provides a 64-bit wide bit vector of hardware support of processor state components.

If CPUID.07H.00H:EBX.MPX[14] = 1 (the processor supports Intel MPX), CPUID.0DH.00H:EAX[4:3] will enumerate the XSAVE state components associated with Intel MPX. These two component states of Intel MPX are the following:

- BNDREGS: CPUID.0DH.00H:EAX[3] indicates XCR0.BNDREGS[bit 3] is supported. This bit indicates bound register component of Intel MPX state, comprised of four bounds registers, BND0-BND3 (see Appendix E.3.2).
- BNDCSR: CPUID.0DH.00H:EAX[4] indicates XCR0.BNDCSR[bit 4] is supported. This bit indicates bounds configuration and status component of Intel MPX comprised of BNDCFGU and BNDSTATUS. OS must enable both BNDCSR and BNDREGS bits in XCR0 to ensure full Intel MPX support to applications.
- The size of the processor state component, enabled by XCR0.BNDREGS, is enumerated by CPUID.0DH.03H:EAX[31:0] and the byte offset of this component relative to the beginning of the XSAVE/XRSTOR area is reported by CPUID.0DH.03H:EBX[31:0].
- The size of the processor state component, enabled by XCR0.BNDCSR, is enumerated by CPUID.0DH.04H:EAX[31:0] and the byte offset of this component relative to the beginning of the XSAVE/XRSTOR area is reported by CPUID.0DH.04H:EBX[31:0].

On processors that support Intel MPX, CPUID.0DH.00H:EAX[3] and CPUID.0DH.00H:EAX[4] will both be 1. On processors that do not support Intel MPX, CPUID.0DH.00H:EAX[3] and CPUID.0DH.00H:EAX[4] will both be 0.

The layout of XCR0 for extended processor state components defined in Intel Architecture is shown in Figure 2-8 of the Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3A.

Enabling Intel MPX requires an OS to manage bits [4:3] of XCR0; see Section 13.5.

The BNDLDX and BNDSTX instructions (Appendix E.4.3) each take an operand whose bits are used to traverse data structures in memory. In 64-bit mode, these instructions operate only on the lower bits in the supplied 64-bit addresses. The number of bits used is 48 plus a value called the **MPX address-width adjust** (**MAWA**). The MAWA value depends on CPL:

- If CPL < 3, the supervisor MAWA (**MAWAS**) is used. This value is 0.
- If CPL = 3, the user MAWA (**MAWAU**) is used. The value of MAWAU is enumerated in CPUID.07H.00H:ECX.MPX_MAWAU[21:17].

(Outside of 64-bit mode, BNDLDX, and BNDSTX use the entire 32 bits of the supplied linear-address operands.)

E.3.2 Bounds Registers

Intel MPX Architecture defines four new registers, BND0-BND3, which Intel MPX instructions operate on. Each bounds register stores a pair of 64-bit values which are the lower bound (LB) and upper bound (UB) of a buffer, see Figure E-1.

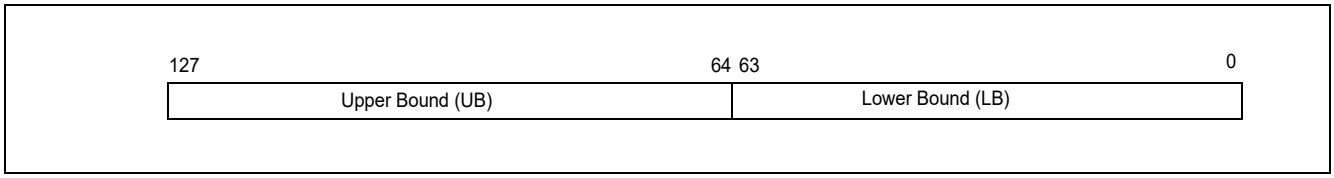


Figure E-1. Layout of the Bounds Registers BND0-BND3

The bounds are unsigned effective addresses, and are inclusive. The upper bounds are architecturally represented in 1/'s complement form. Lower bound = 0, and upper bound = 0 (1's complement of all 1s) will allow access to the entire address space. The bounds are considered as INIT when both lower and upper bounds are 0 (cover the entire address space). The two Intel MPX instructions which operate on the upper bound (BNDMK and BNDCU) account for the 1's complement representation of the upper bounds.

The instruction set does not impose any conventions on the use of bounds registers. Software has full flexibility associating pointers to bounds registers including sharing them for multiple pointers.

RESET or INIT# will initialize (write zero to) BND0–BND3.

E.3.3 Configuration and Status Registers

Intel MPX defines two configuration registers and one status register. The two configuration registers are defined for user mode (CPL = 3) and supervisor mode (CPL < 3). The user-mode configuration register BNDCFGU is accessible only with the XSAVE feature set instructions.

The supervisor mode configuration register is an MSR, referred to as IA32_BNDCFGS (MSR 0D90H). Because both configuration registers share a common layout (see Figure E-2), when describing the common behavior, these configuration registers are often denoted as BNDCFGx, where x can be U or S, for user and supervisor mode respectively.

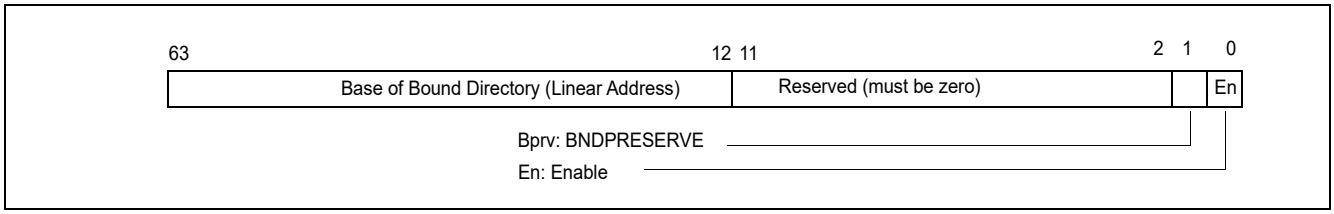


Figure E-2. Common Layout of the Bound Configuration Registers BNDCFGU and BNDCFGS

The Enable bit in BNDCFGU enables Intel MPX in user mode (CPL = 3), and the Enable bit in BNDCFGS enables Intel MPX in supervisor mode (CPL < 3). The BNDPRESERVE bit controls the initialization behavior of CALL/RET/JMP/Jcc instructions without the BND (F2H) prefix; see Appendix E.5.3.

WRMSR to BNDCFGS will #GP if any of the reserved bits of BNDCFGS is not zero or if the base address of the bound directory is not canonical. XRSTOR of BNDCFGU ignores the reserved bits and does not fault if any is non-zero; similarly, it ignores the upper bits of the base address of the bound directory and sign-extends the highest implemented bit of the linear address to guarantee the canonicality of this address.

Intel MPX also defines a status register (BNDSTATUS) primarily used to communicate status information for #BR exception. The layout of the status register is shown in Figure E-3.

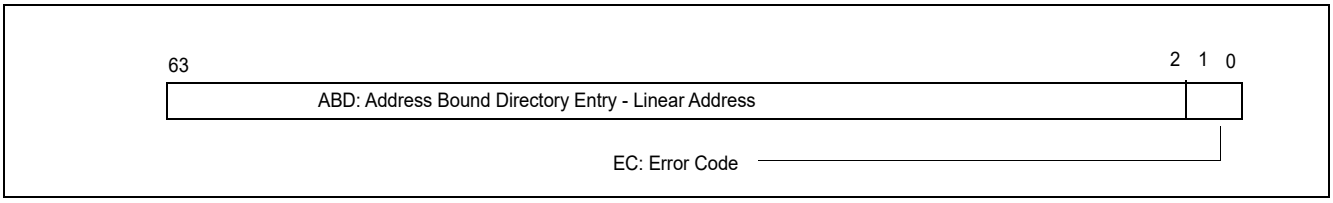


Figure E-3. Layout of the Bound Status Register BNDSTATUS

The BNDSTATUS register provides two fields to communicate the status of Intel MPX operations:

- EC (bits 1:0): The error code field communicates status information of a bound range exception #BR or operation involving bound directory.
- ABD: (bits 63:2):The address field of a bound directory entry can provide information when operation on the bound directory caused a #BR.

The valid error codes are defined in Table E-1.

Table E-1. Error Code Definition of BNDSTATUS

EC	Description	Meaning
00b[1]	No Intel MPX exception	No exception caused by Intel MPX operations.
01b	Bounds violation	#BR caused by BNDCL, BNDCU or BNDCN instructions; ABD is 0.
10b	Invalid BD entry	#BR caused by BNDLDX or BNDSTX instructions, ABD will be set to the linear address of the invalid bound-directory entry
11b	Reserved	Reserved

NOTES:

1. When legacy BOUND instruction cause a #BR with Intel MPX enabled (see Appendix E.5.4), EC is written with Zero.

RESET or INIT# will set BNDCFGx and BNDSTATUS registers to zero.

E.3.4 Read and Write of IA32_BNDCFGS

The RDMSR and WRMSR instructions can be used to read and write the IA32_BNDCFGS MSR. (The XSAVE state does not include IA32_BNDCFGS, and instructions in the XSAVE feature set do not access that register). Attempts to write to IA32_BNDCFGS check for canonicality of the addresses being loaded into IA32_BNDCFGS (regardless of mode at the time of execution) and will #GP if the address is not canonical or if reserved bits would be set.

Software can use RDMSR and WRMSR to read and write IA32_BNDCFGS as long as the processor implements Intel MPX, i.e., CPUID.07H.00H:EBX.MPX = 1. The states of CR4 and XCR0 have no impact on the ability to access IA32_BNDCFGS.

E.4 INTEL MPX INSTRUCTION SUMMARY

When Intel MPX is not enabled or not present, all Intel MPX instructions behave as NOP. There are eight Intel MPX instructions, Table E-2 provides a summary.

A C/C++ compiler can implement intrinsic support for Intel MPX instructions to facilitate pointer operation with capability of checking for valid bounds on pointers. Typically, Intel MPX intrinsics are implemented by compiler via inline code generation where bounds register allocations are handled by the compiler without requiring the

programmer to directly manipulate any bounds registers. Therefore no new data type for a bounds register is needed in the syntax of Intel MPX intrinsics.

Table E-2. Intel® MPX Instruction Summary

Intel MPX Instruction	Description
BNDMK b, m	Create LowerBound (LB) and UpperBound (UB) in the bounds register b
BNDCL b, r/m	Checks the address of a memory reference or address in r against the lower bound
BNDCU b, r/m	Checks the address of a memory reference or address in r against the upper bound in 1's complement form
BNDCN b, r/m	Checks the address of a memory reference or address in r against the upper bound not in 1's complement form
BNDMOV b, b/m	Copy/load LB and UB bounds from memory or a bounds register
BNDMOV b/m, b	Store LB and UB bounds in a bounds register to memory or another register
BNDLDX b, mib	Load bounds using address translation using an sib-addressing expression mib
BNDSTX mib, b	Store bounds using address translation using an sib-addressing expression mib

E.4.1 Instruction Encoding

All Intel MPX instructions are NOP on processors that report CPUID.07H.00H:EBX.MPX[14] = 0, or if Intel MPX is not enabled by the operating system (see Section 13.5). Applications can selectively opt-in to use Intel MPX instructions.

All Intel MPX opcodes encoded to operate on BND0-BND3 are valid Intel MPX instructions. All Intel MPX opcodes encoded to operate on bound registers beyond BND3 will #UD if Intel MPX is enabled.

BNDLDX/BNDSTX opcodes require 66H as a mandatory prefix with its operand size tied to the address size attribute of the supported operating modes. Attempt to override operand size attribute with 66H or with REX.W in 64-bit mode is ignored.

E.4.2 Usage and Examples

BNDMK is typically used after memory is allocated for a buffer, e.g., by functions such as malloc, calloc, or when the memory is allocated on the stack. However, many other usages are possible such as when accessing an array member of a structure.

Example E-1. BNDMK Example Usage in Application and Library Code

```
 //assume the array A is allocated on the stack at 'offset'
// from RBP.
int A[100];

// the instruction to store starting address of array will be:
LEA RAX, [RBP+offset]

// the instruction to create the bounds for array A will be:
BNDMK BND0, [RAX+399]

// Store RAX into BND0.LB, and ~(RAX+399) into BND0.UB.
```

```
// similarly, for a library implementation of dynamic allocated
// memory
int * k = malloc(100);

// assuming that malloc returns pointer k in RAX and holds
// (size - 1) in RCX, the malloc implementation will execute the
// following instruction before returning:
BNDMK BND0, [RAX+RCX]

// BND0.LB stores RAX, and BND0.UB stores ~(RAX+RCX)
```

BNDMOV is typically used to copy bounds from one bound register to another when a pointer is copied from one general purpose register to another, or to spill/fill bounds into memory corresponding to a spill/fill of a pointer.

Example E-2. BNDMOV Example

Spilling or caller save of bound register would use BNDMOV [RBP+ offset], BNDx. Assuming that the calling convention is that bound of first pointer is passed in BND0, and that bound happens to be in BND3 before the call, the software will add instruction BNDMOV BND0, BND3 prior to the call.

BNDCL/BNDCU/BNDCN are typically used before writing to a buffer but can be used in other instances as well. If there are no bounds violations as a result of bound check instruction, the processor will proceed to execute the next instruction. However, if the bound check fails, it will signal #BR exception (fault).

Typically, the pointer used to write to memory will be compared against lower bound. However, for upper bound check, the software must add the (operand size - 1) to the pointer before upper bound checking.

For example, the software intend to write 32-bit integer in 64-bit mode into a buffer at address specified in RAX, and the bounds are in register BND0, the instruction sequence will be:

```
BNDCL BND0, [RAX]
BNDCU BND0, [RAX+3] ; operand size is 4
MOV Dword ptr [RAX], RBX ; RBX has the data to be written to the buffer.
```

Software may move one of the two bound checks out of a loop if it can determine that memory is accessed strictly in ascending or descending order. For string instructions of the form REP MOVS, the software may choose to do check lower bound against first access and upper bound against last access to memory. However, if software wants to also check for wrap around conditions as part of address computation, it should check for both upper and lower bound for first and last instructions (total of four bound checks).

BNDSTX is used to store the bounds associated with a buffer and the "pointer value" of the pointer to that buffer onto a bound table entry via address translation using a two-level structure, see Appendix E.4.3.

For example, the software has a buffer with bounds stored in BND0, the pointer to the buffer is in ESI, the following sequence will store the "pointer value" (the buffer) and the bounds into a configured bound table entry using address translation from the linear address associated with the base of a SIB-addressing form consisting of a base register and a index register:

```
MOV ECX, Dword ptr [ESI] ; store the pointer value in the index register ECX
MOV EAX, ESI ; store the pointer in the base register EAX
BNDSTX Dword ptr [EAX+ECX], BND0 ; perform address translation from the linear address of the base EAX and store bounds and pointer value ECX onto a bound table entry.
```

Similarly to retrieve a buffer and its associated bounds from a bound table entry:

```
MOV EAX, dword ptr [EBX] ;
BNDLDX BND0, dword ptr [EBX+EAX]; perform address translation from the linear address of the base EBX, and loads bounds and pointer value from a bound table entry
```

E.4.3 Loading and Storing Bounds in Memory

Intel MPX defines two instructions to load and store of the linear address of a pointer to a buffer, along with the bounds of the buffer into a data structure of extended bounds. When storing these extended bounds, the processor parses the address of the pointer (where it is stored) to locate an entry in a **bound table** in which to store the extended bounds. Loading of an extended bounds performs the reverse sequence.

The memory representation of an extended bound is a 4-tuple consisting of lower bound, upper bound, pointer value and a reserved field (for use by future versions of Intel MPX; software must not use this field). Accesses to these extended bounds use 32-bit or 64-bit operands according to the current paging mode. Thus, a bound table entry is 4*64 bits (32 bytes) in 64-bit mode and 4*32 bits (16 bytes) outside 64-bit mode The linear address of a bound table is stored in a bound-directory entry (BDE). The linear address of the **bound directory** is derived from either BNDCFGU (CPL = 3) or BNDCFGS (CPL < 3).

The bound directory and bound tables are stored in application memory and are allocated by the application (in case of kernel use, the structures will be in kernel memory). The bound directory and each bound table are in contiguous linear memory.

Software should take care to allocate sufficient memory for the bound directory and the bound tables. The amount of memory required depends on the current operating mode and, in some cases, on CPL:

- In 64-bit mode:
  - Each bound table comprises 2^{17} 32-byte entries thus, the size of a bound table in 64-bit mode is 4 MBytes.
  - The size of the bound directory depends on the value of MAWA. Specifically, the bound directory comprises $2^{28+MAWA}$ 64-bit entries; thus, the size of a bound directory in 64-bit mode is 2^{1+MAWA} GBytes. The value of MAWA depends on CPL:
    - If CPL < 3, the supervisor MAWA (MAWAS) is used. This value is 0. Thus, when CPL < 3, a bound directory comprises 2^{28} 64-bit entries and the size of a bound directory is 2 GBytes.
    - If CPL = 3, the user MAWA (MAWAU) is used. The value of MAWAU is enumerated in CPUID.07H.00H:ECX.MPX_MAWAU[21:17]. When CPL = 3, a bound directory comprises $2^{28+MAWAU}$ 64-bit entries and the size of a bound directory is $2^{1+MAWAU}$GBytes.

NOTE

Software operating with CPL = 3 in 64-bit mode should use CPUID to determine the proper amount of memory to allocate for the bound directory.

- Outside 64-bit mode:
  - Each bound table comprises 2^{10} 16-byte entries; thus, the size of a bound table outside 64-bit mode is 16 KBytes.
  - The bound directory comprises 2^{20} 32-bit entries; thus, the size of a bound directory outside 64-bit mode is 4 MBytes. This size is independent of MAWA and CPL.

Bounds in memory are associated with the memory address where the pointer is stored, i.e., Ap. A linear address LAp is computed by adding the appropriate segment base to Ap. (Note: for these instructions, the segment override applies only to the computation.) Appendix E.4.3.1 and Appendix E.4.3.2 describe how BNDLDX and BNDSTX parse LAp to locate a bound-directory entry (BDE), which contains the address of a bound table, and then a bound-table entry (BTE), which contains the extended bounds for the pointer.

E.4.3.1 BNDLDX and BNDSTX in 64-Bit Mode

Figure E-4 shows the two-level structures for address translation of extended bounds in 64-bit mode.

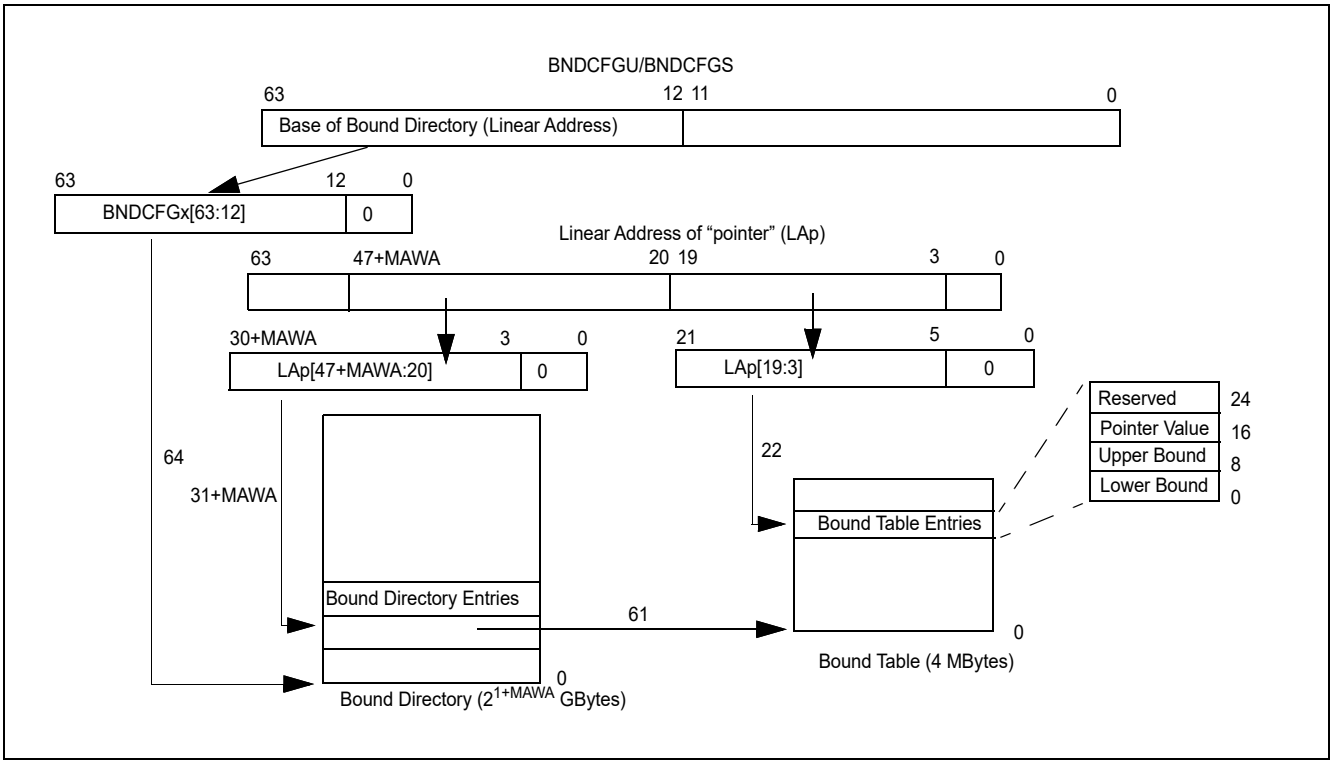


Figure E-4. Bound Paging Structure and Address Translation in 64-Bit Mode

As noted earlier, the linear address of the bound directory is derived from either BNDCFGU (CPL = 3) or BNDCFGS (CPL < 3). In 64-bit mode, each bound-directory entry (BDE) is 8 bytes. The number of entries in the bound directory is determined by the MPX address-width adjust (MAWA; see Appendix E.3.1). Specifically, the number of entries is $2^{28+MAWA}$.

In 64-bit mode, the processor uses the two-level structures to access extended bounds as follows:

- A bound directory is located at the 4-KByte aligned linear address specified in bits 63:12 of BNDCFGx (see Figure E-2). A bound directory comprises $2^{28+MAWA}$ 64-bit entries (BDEs); thus, the size of a bound directory in 64-bit mode is 2^{1+MAWA} GBytes. A BDE is selected using the LAp (linear address of pointer to a buffer) to construct a 64-bit offset as follows:
  - — bits 63:31+MAWA are 0;
  - — bits 30+MAWA:3 are LAp[47+MAWA:20]; and
  - — bits 2:0 are 0.

  The address of the BDE is the sum of the bound-directory base address (from BNDCFGx) plus this 64-bit offset.
- Bit 0 of a BDE is a valid bit. If this bit is 0, use of the BDE by BNDLDX or BNDSTX causes #BR, sets BNDSTATUS[1:0] to 10b (the error code), and loads BNDSTATUS[63:2] with bits 63:2 of the linear address of the BDE. Otherwise, the processor uses bits 63:3 of the BDE as the 8-byte aligned address of a bound table (BT); the processor ignores bits 2:1 of a BDE.

  A bound table comprises 2^{17} 32-byte entries (BTEs); thus, the size of a bound table in 64-bit mode is 4 MBytes. A BTE is selected using the LAp (linear address of pointer to a buffer) to construct an offset as follows:
  - — bits 21:5 are LAp[19:3]; and
  - — bits 4:0 are 0.

  The address of the BTE is the sum of the bound-table base address (from the BDE) plus this offset.
- Each BTE comprises the following:

— a 64-bit lower bound (LB) field;

— a 64-bit upper bound (UB) field;

— a 64-bit pointer value; and

— a 64-bit reserved field. This field is reserved for future Intel MPX; software must not use it.

E.4.3.2 BNDLDX and BNDSTX Outside 64-Bit Mode

Figure E-5 shows the two-level structures for address translation of extended bounds outside 64-bit mode.

As noted earlier, the linear address of the bound directory is derived from either BNDCFGU (CPL = 3) or BNDCFGS (CPL < 3). Outside 64-bit mode, each bound-directory entry (BDE) is 4 bytes. The number of entries in the bound directory is 2^{20}.

Outside 64-bit mode, the processor uses the two-level structures to access extended bounds as follows:

- A bound directory is located at the 4-KByte aligned linear address specified in bits 31:12 of BNDCFGx (see Figure E-2). A bound directory comprises 2^{20} 32-bit entries (BDEs); thus, the size of a bound directory outside 64-bit mode is 4 MBytes. A BDE is selected using the LAp (linear address of pointer to a buffer) to construct an offset as follows:

  — bits 21:2 are LAp[31:12]; and

  — bits 1:0 are 0.

  The address of the BDE is the sum of the bound-directory base address (from BNDCFGx) plus this offset.

- Bit 0 of a BDE is a valid bit. If this bit is 0, use of the BDE by BNDLDX or BNDSTX causes #BR, sets BNDSTATUS[1:0] to 10b (the error code), and loads BNDSTATUS[31:2] with bits 31:2 of the linear address of the BDE. Otherwise, the processor uses bits 31:2 of the BDE as the 4-byte aligned address of a bound table (BT); the processor ignores bit 1 of a BDE.

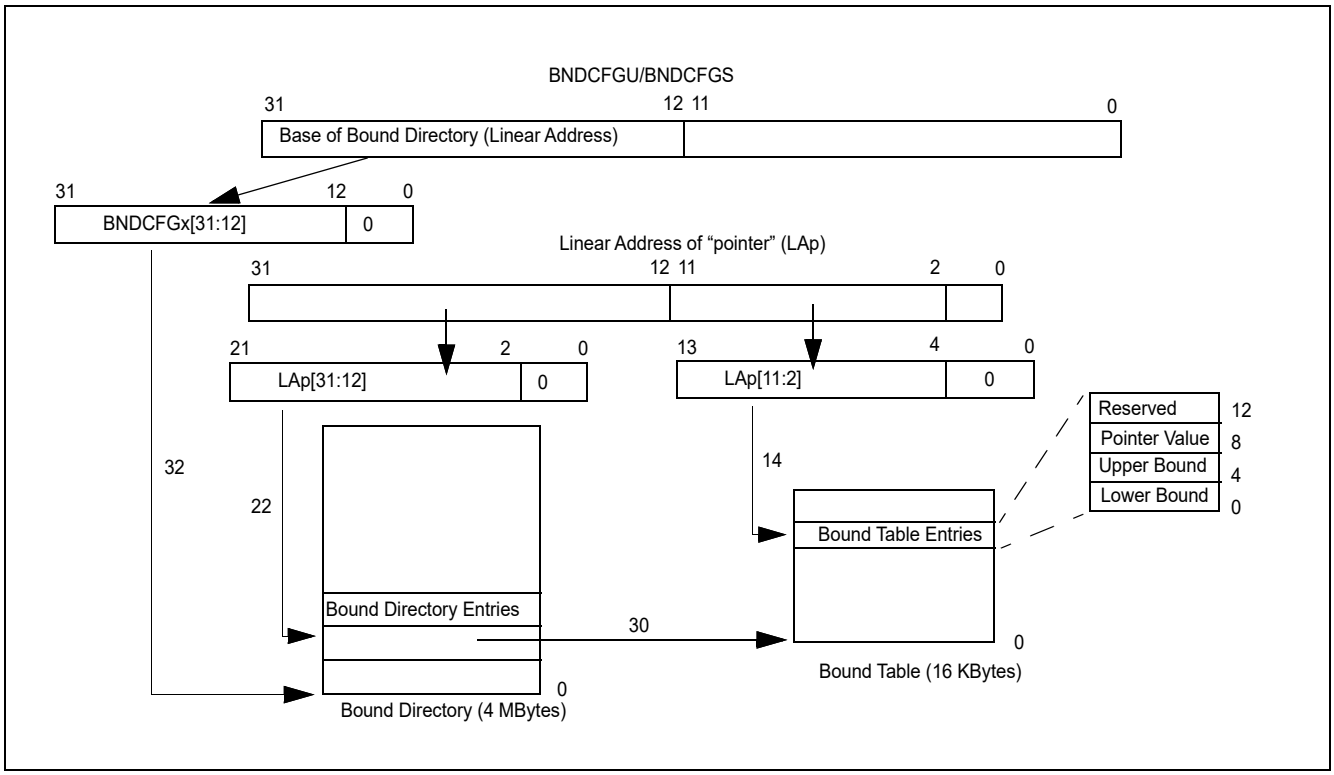


Figure E-5. Bound Paging Structure and Address Translation Outside 64-Bit Mode

A bound table comprises 2^{10} 16-byte entries (BTEs); thus, the size of a bound table outside 64-bit mode is 16 KBytes. A BTE is selected using the LAp (linear address of pointer to a buffer) to construct an offset as follows:

— bits 13:4 are LAp[11:2]; and

— bits 3:0 are 0.

The address of the BTE is the sum of the bound-table base address (from the BDE) plus this offset. This address is use as an offset into the DS segment to determine the linear address of the BTE.

- Each BTE comprises the following:

  — a 32-bit lower bound (LB) field;

  — a 32-bit upper bound (UB) field;

  — a 32-bit pointer value; and

  — a 32-bit reserved field. This field is reserved for future Intel MPX; software must not use it.

E.5 INTERACTIONS WITH INTEL MPX

E.5.1 Intel® MPX and Operating Modes

In 64-bit Mode, all Intel MPX instructions use 64-bit operands for bounds and 64 bit addressing, i.e., REX.W & 67H have no effect on data or address size.

XSAVE, XSAVEOPT, and XRSTOR load/store 64-bit values in all modes, as these state-management instructions are not Intel MPX instructions.

In compatibility and legacy modes (including 16-bit code segments, real and virtual 8086 modes) all Intel MPX instructions use 32-bit operands for bounds and 32 bit addressing. The upper 32-bits of destination bound register are cleared (consistent with behavior of integer registers)

In 32-bit and compatibility mode, the bounds are 32-bit, and are treated same as 32-bit integer registers. Therefore, when 32-bit bound is updated in a bound register, the upper 32-bits are undefined. When switching from 64-bit, the behavior of content of bounds register will be similar to that of general purpose registers.

Table E-3 describes the impact of 67H prefix on memory forms of Intel MPX instructions (register-only forms ignore 67H prefix) when Intel MPX is enabled:

Table E-3. Effective Address Size of Intel® MPX Instructions with 67H Prefix

Addressing Mode	67H Prefix	Effective Address Size used for Intel MPX instructions when Intel MPX is enabled
64-bit Mode	Y	64 bit addressing used
64-bit Mode	N	64 bit addressing used
32-bit Mode	Y	#UD
32-bit Mode	N	32 bit addressing used
16-bit Mode	Y	32 bit addressing used
16-bit Mode	N	#UD

E.5.2 Intel® MPX Support for Pointer Operations with Branching

Intel MPX provides flexibility in supporting pointer operation across control flow changes. Intel MPX allows

- compatibility with legacy code that may perform pointer operation across control flow changes and are unaware of Intel MPX, along with
- Intel MPX-aware code that adds bounds checking protection to pointer operation across control flow changes.

The interface to provide such flexibility consists of:

- Using a prefix, referred to as BND prefix, to relevant branch instructions: CALL, RET, JMP, and Jcc.
- BNDCFGU and BNDCFGS provides the bit field, BNDPRESERVE (bit 1).

The value of BNDPRESERVE in conjunction with the presence/absence the BND prefix with those branching instruction will determine whether the values in BND0-BND3 will be initialized or unchanged.

E.5.3 CALL, RET, JMP, and All Jcc

An application compiled to use Intel MPX will use the REPNE (F2H) prefix (denoted by BND) for all forms of near CALL, near RET, near JMP, short & near Jcc instructions (BND+CALL, BND+RET, BND+JMP, BND+Jcc). See Table E-4 for specific opcodes. All far CALL, RET, and JMP instructions plus short JMP (JMP rel 8, opcode EB) instructions will never cause bound registers to be initialized.

If BNDPRESERVE bit is one, above instructions will NOT INIT the bounds registers when BND prefix is not present for above instructions (legacy behavior). However, If BNDPRESERVE is zero, above instructions will INIT ALL bound registers (BND0-BND3) when BND prefix is not present for above instructions. If BND prefix is present for above instructions, the BND registers will NOT INIT any bound registers (BND0-BND3).

The legacy code will continue to use non-prefixed forms of these instructions, so if BNDPRESERVE is zero, all the bound registers will INIT by legacy code. This allows the legacy function to execute and return to callee with all bound registers initialized (legacy code by definition cannot make or load bounds in bound registers because it does not have Intel MPX instructions). This will eliminate compatibility concerns when legacy function might have changed the pointer in registers but did not update the value of the bounds registers associated with these pointers.

If BNDCFGx.BNDPRESERVE is clear then non-prefixed forms of these instructions will initialize all the bound registers. If this bit is set then non-prefixed and prefixed forms of these instructions will preserve the contents of bound registers as shown in Table E-4.

Table E-4. Bounds Register INIT Behavior Due to BND Prefix with Branch Instructions

Instruction	Branch Instruction Opcodes	BNDPRESERVE = 0	BNDPRESERVE = 1
CALL	E8, FF/2	Init BND0-BND3	BND0-BND3 unchanged
BND + CALL	F2 E8, F2 FF/2	BND0-BND3 unchanged	BND0-BND3 unchanged
RET	C2, C3	Init BND0-BND3	BND0-BND3 unchanged
BND + RET	F2 C2, F2 C3	BND0-BND3 unchanged	BND0-BND3 unchanged
JMP	E9, FF/4	Init BND0-BND3	BND0-BND3 unchanged
BND + JMP	F2 E9, F2 FF/4	BND0-BND3 unchanged	BND0-BND3 unchanged
Jcc	70 through 7F, 0F 80 through 0F 8F	Init BND0-BND3	BND0-BND3 unchanged
BND + Jcc	F2 70 through F2 7F, F2 0F 80 through F2 0F 8F	BND0-BND3 unchanged	BND0-BND3 unchanged

E.5.4 BOUND Instruction and Intel MPX

If Intel MPX in enabled (see Section 13.5) and a #BR was caused due to a BOUND instruction, then BOUND instruction will write zero to the BNDSTATUS register. In all other situations, BOUND instruction will not modify BNDSTATUS. Specifically, the operation of the BOUND instruction can be described as:

```
IF ( ( BOUND instruction caused #BR) AND ( CR4.OXXSAVE = 1 AND XCR0.BNDREGS = 1 AND XCR0.BNDCSR = 1) AND
    ( (CPL= 3 AND BNDCFGU.ENABLE = 1) OR (CPL < 3 AND BNDCFGS.ENABLE = 1) ) ) THEN
    BNDSTATUS := 0;
ELSE
    BNDSTATUS is not modified;
FI;
```

E.5.5 Programming Considerations

Intel MPX instruction set does not dictate any calling convention, but allows the calling convention extensions to be interoperable with legacy code by making use of the of the bound registers and the bound tables to convey arguments and return values.

E.5.6 Intel MPX and System Management Mode

Upon delivery of an SMI to a processor supporting Intel MPX, the contents of IA32_BNDCFGS is saved to SMM state save map (at offset 7ED0H) and the register is then cleared when entering into SMM. RSM restores IA32_BNDCFGS from the SMM state save map. The instruction forces the reserved bits (11:2) to 0 and sign-extends the highest implemented bit of the linear address to guarantee the canonicality of this address (regardless of what is in SMM state save map).

The content of IA32_BNDCFGS is cleared after entering into SMM. Thus, Intel MPX is disabled inside an SMM handler until SMM code enables it explicitly. This will prevent initialization of the bound registers by execution of CALL, RET, JMP, or Jcc in SMM code.

E.5.7 Support of Intel MPX in VMCS

A new guest-state field for IA32_BNDCFGS is added to the VMCS. In addition, two new controls are added:

- a VM-exit control called "clear BNDCFGS"
- a VM-entry control called "load BNDCFGS."

VM exits always save IA32_BNDCFGS into BNDCFGS field of VMCS; if "clear BNDCFGS" is 1, VM exits clear IA32_BNDCFGS. If "load BNDCFGS" is 1, VM entry loads IA32_BNDCFGS from VMCS. If loading IA32_BNDCFGS, VM entry should check the value of that register in the guest-state area of the VMCS and cause the VM entry to fail (late) if the value is one that would causes WRMSR to fault if executed in ring 0.

E.5.8 Support of Intel MPX in Intel TSX

For some processor implementations, the following Intel MPX instructions may always cause transactional aborts:

- An Intel TSX transaction abort will occur in case of legacy branch (that causes bounds registers INIT) when at least one bounds register was in a NON-INIT state.
- An Intel TSX transaction abort will occur in case of a BNDLDX & BNDSTX instruction on non-flat segment.

Intel MPX Instructions (including BND prefix + branch instructions) not enumerated above as causing transactional abort when used inside a transaction will typically not cause an Intel TSX transaction to abort.

INDEX

Numerics

128-bit
  packed byte integers data type, 4-10
  packed double precision floating-point
    data type, 4-10
  packed doubleword integers data type, 4-10
  packed quadword integers data type, 4-10
  packed SIMD data types, 4-9
  packed single precision floating-point
    data type, 4-10, 10-5
  packed word integers data type, 4-10
16-bit
  address size, 3-9
  operand size, 3-9
286 processor, 2-1
32-bit
  address size, 3-9
  operand size, 3-9
64-bit
  packed byte integers data type, 4-9, 9-3
  packed doubleword integers data type, 4-9
  packed doubleword integers data types, 9-3
  packed word integers data type, 4-9, 9-3
64-bit mode
  sub-mode of IA-32e, 3-1
  address calculation, 3-10
  address size, 3-19
  address space, 3-5
  BOUND instruction, 7-18
  branch behavior, 6-12
  byte register limitation, 3-13
  CALL instruction, 6-12, 7-17
  canonical address, 3-10
  CMPS instruction, 7-20
  CMPXCHG16B instruction, 7-5
  data types, 7-2
  DEC instruction, 7-8
  decimal arithmetic instructions, 7-10
  default operand and address sizes, 3-2
  exceptions, 6-19
  far pointer, 4-8
  feature list, 2-20
  GDTR register, 3-6
  IDTR register, 3-6
  INC instruction, 7-8
  instruction pointer, 3-10, 3-18
  instructions introduced, 5-38
  interrupts, 6-19
  introduction, 2-20, 3-1, 7-1
  IRET instruction, 7-18
  I/O instructions, 7-20
  JCC instruction, 6-12, 7-17
  JCXZ instruction, 6-12, 7-17
  JMP instruction, 6-12, 7-17
  LAHF instruction, 7-22
  LDTR register, 3-6
  legacy modes, 2-20
  LODS instruction, 7-20
  LOOP instruction, 6-12, 7-17
  memory models, 3-9
  memory operands, 3-21
  MMX technology, 9-2
  MOVS instruction, 7-20
  MOVSXD instruction, 7-8
  near pointer, 4-8
  operand addressing, 3-23
  operand size, 3-19
  operands, 3-20, 3-21
  POPF instruction, 7-22
  promoted instructions, 3-2
  PUSHA, PUSHAD, POPA, POPAD, 7-7
  PUSHF instruction, 7-22
  PUSHFD instruction, 7-22
  real address mode, 3-9
  register operands, 3-20
  REP prefix, 7-20
  RET instruction, 6-12, 7-17
  REX prefix, 3-2, 3-12, 3-19
  RFLAGS register, 7-22
  RIP register, 3-10
  RIP-relative addressing, 3-18, 3-24
  SAHF instruction, 7-22
  SCAS instruction, 7-20
  segment registers, 3-15
  segmentation, 3-9, 3-22
  SSE extensions, 10-3
  SSE2 extensions, 11-3
  SSE3 extensions, 12-1
  SSSE3 extensions, 12-1
  stack behavior, 6-4
  STOS instruction, 7-20
  TR register, 3-6
  x87 FPU, 8-1
  See also: IA-32e mode, compatibility mode
8086 processor, 2-1
8088 processor, 2-1

A

AAA instruction, 7-9
AAD instruction, 7-10
AAM instruction, 7-10
AAS instruction, 7-10
AC (alignment check) flag, EFLAGS register, 3-17
Access rights, segment descriptor, 6-8, 6-13
ADC instruction, 7-8
ADD instruction, 7-8
ADDPD instruction, 11-6
ADDPS instruction, 10-8
Address size attribute
  code segment, 3-18
  description of, 3-18
  of stack, 6-3
Address sizes, 3-9
Address space
  64-bit mode, 3-1, 3-5
  compatibility mode, 3-1
  overview of, 3-2
  physical, 3-6
Addressing modes
  assembler, 3-24
  base, 3-22, 3-23, 3-24

base plus displacement, 3-23
base plus index plus displacement, 3-23
base plus index time scale plus displacement, 3-23, 3-24
canonical address, 3-10
displacement, 3-22, 3-23
effective address, 3-22
immediate operands, 3-20
index, 3-22, 3-24
index times scale plus displacement, 3-23
memory operands, 3-21
register operands, 3-20
RIP-relative addressing, 3-18, 3-24
scale factor, 3-22, 3-24
specifying a segment selector, 3-21
specifying an offset, 3-22
specifying offsets in 64-bit mode, 3-23
ADDSD instruction, 11-6
ADDSS instruction, 10-8
ADDSUBPD instruction, 5-24, 12-4
ADDSUBPS instruction, 5-24, 12-4
Advanced media boost, 2-11
advanced smart cache, 2-10
AF (adjust) flag, EFLAGS register, 3-16, A-1
AH register, 3-12
AL register, 3-12
Alignment
words, doublewords, quadwords, 4-2
AND instruction, 7-10
ANDNPD instruction, 11-7
ANDNPS instruction, 10-9
ANDPD instruction, 11-7
ANDPS instruction, 10-9
Arctangent, x87 FPU operation, 8-20
Arithmetic instructions, x87 FPU, 8-25
Assembler, addressing modes, 3-24
Asymmetric processing model, 12-1
AX register, 3-12

B

B (default size) flag, segment descriptor, 3-18
Base (operand addressing), 3-22, 3-23, 3-24
Basic execution environment, 3-2
Basic programming environment, 7-1
B-bit, x87 FPU status word, 8-5
BCD integers
packed, 4-11
relationship to status flags, 3-17
unpacked, 4-10, 7-9
x87 FPU encoding, 4-11
BH register, 3-12
Bias value
numeric overflow, 8-29
numeric underflow, 8-30
Biased exponent, 4-14
Biasing constant, for floating-point numbers, 4-6
Binary numbers, 1-8
Binary-coded decimal (see BCD)
Bit field, 4-8
Bit order, 1-6
BL register, 3-12
BOUND instruction, 6-18, 7-18, 7-23
BOUND range exceeded exception (#BR), 6-19
BP register, 3-12
Branch
control transfer instructions, 7-14
hints, 11-13
on EFLAGS register status flags, 7-15, 8-6
on x87 FPU condition codes, 8-6, 8-20
prediction, 2-8
Brand information
processor brand index, 21-5
processor brand string, 21-4
BSF instruction, 7-14
BSR instruction, 7-14
BSWAP instruction, 7-4
BT instruction, 3-15, 3-16, 7-14
BTC instruction, 3-15, 3-16, 7-14
BTR instruction, 3-15, 3-16, 7-14
BTS instruction, 3-15, 3-16, 7-14
BX register, 3-12
Byte, 4-1
Byte order, 1-6

C

C1 flag, x87 FPU status word, 8-4, 8-26, 8-29, 8-30
C2 flag, x87 FPU status word, 8-5
cache, smart, 2-4
Call gate, 6-8
CALL instruction, 3-18, 6-3, 6-4, 6-8, 7-15, 7-22
Calls (see Procedure calls)
Canonical address, 3-10
CBW instruction, 7-7
CDQ instruction, 7-7
Celeron processor
description of, 2-2
CF (carry) flag, EFLAGS register, 3-16, A-1
CH register, 3-12
CL register, 3-12
CLC instruction, 3-16, 7-21
CLD instruction, 3-17, 7-21
CLFLUSH instruction, 11-12
CLI instruction, 20-4
CMC instruction, 3-16, 7-21
CMOVcc instructions, 7-3, 7-4
CMP instruction, 7-8
CMPPD instruction, 11-7
CMPPS instruction, 10-9
CMPS instruction, 3-17, 7-18
CMPSD instruction, 11-7
CMPSS instruction, 10-9
CMPXCHG instruction, 7-4
CMPXCHG16B instruction, 7-5
CMPXCHG8B instruction, 7-4
Code segment, 3-14
COMISD instruction, 11-7
COMISS instruction, 10-9
Compare
compare and exchange, 7-4
integers, 7-8
real numbers, x87 FPU, 8-19
strings, 7-18
Compatibility mode
address space, 3-1
branch functions, 6-12
call gate descriptors, 6-12
introduction, 2-20, 3-1
memory models, 3-9
MMX technology, 9-2

segmentation, 3-22
SSE extensions, 10-3
SSE2 extensions, 11-3
SSE3 extensions, 12-1
SSSE3 extensions, 12-1
x87 FPU, 8-1
See also: IA-32e mode, 64-bit mode
Compatibility, software, 1-7
Condition code flags, x87 FPU status word
branching on, 8-6
conditional moves on, 8-6
description of, 8-4
interpretation of, 8-5
use of, 8-19
Conditional moves, x87 FPU condition codes, 8-6
Constants (floating point), 8-17
Control registers
64-bit mode, 3-5
overview of, 3-4
Core microarchitecture, 2-10, 2-12, 2-13
core microarchitecture, 2-10, 2-12
Core Solo and Core Duo, 2-4
Cosine, x87 FPU operation, 8-20
CPUID instruction
CLFLUSH flag, 11-12
CMOVcc feature flag, 7-3
determine support for, 3-17
earlier processors, 21-1
FXSAVE-FXRSTOR flag, 10-14
MMX feature flag, 9-8
processor brand index, 21-4
processor brand string, 21-4
serializing use, 20-5
SSE feature flag, 10-1, 10-6
SSE2 feature flag, 11-1, 12-5
SSE3 feature flag, 12-5
SSSE2 feature flag, 12-9, 12-19, 12-24
summary of, 7-23
CS register, 3-13, 3-14
CTI instruction, 7-22
Current privilege level (see CPL)
Current stack, 6-1, 6-3
CVTDQ2PD instruction, 11-10
CVTDQ2PS instruction, 11-10
CVTPD2DQ instruction, 11-10
CVTPD2PI instruction, 11-10
CVTPD2PS instruction, 11-9
CVTPI2PD instruction, 11-10
CVTPI2PS instruction, 10-11
CVTPS2DQ instruction, 11-10
CVTPS2PD instruction, 11-9
CVTPS2PI instruction, 10-11
CVTSD2SI instruction, 11-10
CVTSD2SS instruction, 11-9
CVTSI2SD instruction, 11-10
CVTSI2SS instruction, 10-11
CVTSS2SD instruction, 11-9
CVTSS2SI instruction, 10-11
CVTTPD2DQ instruction, 11-10
CVTTPD2PI instruction, 11-10
CVTTPS2DQ instruction, 11-10
CVTTPS2PI instruction, 10-11
CVTTSD2SI instruction, 11-10
CVTTSS2SI instruction, 10-11
CWD instruction, 7-7
CWDE instruction, 7-7
CX register, 3-12

D

D (default size) flag, segment descriptor, 6-2, 6-3
DAA instruction, 7-9
DAS instruction, 7-9
Data movement instructions, 7-2
Data pointer, x87 FPU, 8-9
Data registers, x87 FPU, 8-1
Data segment, 3-14
Data types
128-bit packed SIMD, 4-9
64-bit mode, 7-2
64-bit packed SIMD, 4-9
alignment, 4-2
BCD integers, 4-10, 7-9
bit field, 4-8
byte, 4-1
doubleword, 4-1
floating-point, 4-4
fundamental, 4-1
integers, 4-3
numeric, 4-2
operated on by GP instructions, 7-1, 7-2
operated on by MMX technology, 9-3
operated on by SSE extensions, 10-5
operated on by SSE2 extensions, 11-3
operated on by x87 FPU, 8-13
operated on in 64-bit mode, 4-8
packed bytes, 9-3
packed doublewords, 9-3
packed SIMD, 4-9
packed words, 9-3
pointers, 4-7
quadword, 4-1, 9-3
signed integers, 4-4
strings, 4-9
unsigned integers, 4-3
word, 4-1
DAZ (denormals-are-zeros) flag
MXCSR register, 10-5
DE (denormal operand exception) flag
MXCSR register, 11-15
x87 FPU status word, 8-5, 8-28
Debug registers
64-bit mode, 3-5
legacy modes, 3-4
DEC instruction, 7-8
Decimal integers, x87 FPU, 4-11
Deeper sleep, 2-4
Denormal number (see Denormalized finite number)
Denormal operand exception (#D)
overview of, 4-21
SSE and SSE2 extensions, 11-15
x87 FPU, 8-27
Denormalization process, 4-16
Denormalized finite number, 4-5, 4-15
Denormals-are-zero
DAZ flag, MXCSR register, 10-5, 11-2, 11-3, 11-20
mode, 10-5, 11-20
DF (direction) flag, EFLAGS register, 3-17, A-1
DH register, 3-12
DI register, 3-12

Digital media boost, 2-4
Displacement (operand addressing), 3-22, 3-23, 3-24
DIV instruction, 7-9
Divide, 4-22
Divide by zero exception (#Z)
  SSE and SSE2 extensions, 11-15
  x87 FPU, 8-28
DIVPD instruction, 11-6
DIVPS instruction, 10-8
DIVSD instruction, 11-6
DIVSS instruction, 10-8
DL register, 3-12
DM (denormal operand exception) mask bit
  MXCSR register, 11-15
  x87 FPU, 8-28
  x87 FPU control word, 8-7
Double-extended-precision FP format, 4-4
Doubleword, 4-1
DS register, 3-13, 3-14
Dual-core technology
  introduction, 2-18
DX register, 3-12
Dynamic data flow analysis, 2-8
Dynamic execution, 2-7, 2-10, 2-12, 2-13

E

EAX register, 3-11, 3-12
EBP register, 3-11, 3-12, 6-3, 6-7
EBX register, 3-11, 3-12
ECX register, 3-11, 3-12
EDI register, 3-11, 3-12
EDX register, 3-11, 3-12
Effective address, 3-22
EFLAGS register
  64-bit mode, 7-2
  condition codes, B-1
  cross-reference with instructions, A-1
  description of, 3-15
  instructions that operate on, 7-21
  overview, 3-11
  part of basic programming environment, 7-1
  restoring from stack, 6-7
  saving on a procedure call, 6-7
  status flags, 8-6, 8-7, 8-19
  use with CMOVcc instructions, 7-3
EIP register
  description of, 3-18
  overview, 3-11
  part of basic programming environment, 7-1
  relationship to CS register, 3-14
EMMS instruction, 9-8, 9-9
Enhanced Intel Deeper Sleep, 2-4
ENTER instruction, 6-20, 7-21
GETSEC, 5-40
ES register, 3-13, 3-14
ES (exception summary) flag
  x87 FPU status word, 8-31
ESC instructions, x87 FPU, 8-15
ESI register, 3-11, 3-12
ESP register, 3-12
ESP register (stack pointer), 3-11, 6-3
Exception flags, x87 FPU status word, 8-5
Exception handlers
  overview of, 6-12
  SIMD floating-point exceptions, D-1
  SSE and SSE2 extensions, 11-18
  typical actions of a FP exception handler, 4-25
  x87 FPU, 8-32
Exception priority, floating-point exceptions, 4-24
Exception-flag masks, x87 FPU control word, 8-7
Exceptions
  64-bit mode, 6-19
  description of, 6-12
  handler, 6-12
  implicit call to handler, 6-1
  in real-address mode, 6-18
  notation, 1-9
Exponent, floating-point number, 4-12

F

F2XM1 instruction, 8-21
FABS instruction, 8-17
FADD instruction, 8-17
FADDP instruction, 8-17
Far call
  description of, 6-4
  operation, 6-5
Far pointer
  16-bit addressing, 3-9
  32-bit addressing, 3-9
  64-bit mode, 4-8
  description of, 3-7, 4-7
  legacy modes, 4-7
Far return operation, 6-5
FBLD instruction, 8-16
FBSTP instruction, 8-16
FCHS instruction, 8-17
FCLEX/FNCLEX instructions, 8-5
FCMOVcc instructions, 8-7, 8-16
FCOM instruction, 8-6, 8-18
FCOMI instruction, 8-7, 8-18
FCOMIP instruction, 8-7, 8-18
FCOMP instruction, 8-6, 8-18
FCOMPP instruction, 8-6, 8-18
FCOS instruction, 8-5, 8-20
FDIV instruction, 8-17
FDIVP instruction, 8-17
FDIVR instruction, 8-17
FDIVRP instruction, 8-17
Feature determination, of processor, 21-1
FIADD instruction, 8-17
FICOM instruction, 8-6, 8-18
FICOMP instruction, 8-6, 8-18
FIDIV instruction, 8-17
FIDIVR instruction, 8-17
FILD instruction, 8-16
FIMUL instruction, 8-17
FINIT/FNINIT instructions, 8-5, 8-7, 8-8, 8-23
FIST instruction, 8-16
FISTP instruction, 8-16
FISTTP instruction, 5-24, 12-3
FISUB instruction, 8-17
FISUBR instruction, 8-17
Flags
  cross-reference with instructions, A-1
Flat memory model, 3-7, 3-13
FLD instruction, 8-16
FLD1 instruction, 8-17

FLDCW instruction, 8-7, 8-23
FLDENV instruction, 8-5, 8-9, 8-11, 8-23
FLDL2E instruction, 8-17
FLDL2T instruction, 8-17
FLDLG2 instruction, 8-17
FLDLN2 instruction, 8-17
FLDPI instruction, 8-17
FLDSW instruction, 8-23
FLDZ instruction, 8-17
Floating-point data types
  biasing constant, 4-6
  denormalized finite number, 4-5
  description of, 4-4
  double extended precision format, 4-4, 4-5
  double precision format, 4-4, 4-5
  half precision format, 4-5
  infinites, 4-5
  normalized finite number, 4-5
  single precision format, 4-4, 4-5
  SSE extensions, 10-5
  SSE2 extensions, 11-3
  storing in memory, 4-6
  x87 FPU, 8-13
  zeros, 4-5
Floating-point exception handlers
  SSE and SSE2 extensions, 11-18
  typical actions, 4-25
  x87 FPU, 8-32
Floating-point exceptions
  denormal operand exception (#D), 4-21, 8-28, 11-15, C-1
  divide by zero exception (#Z), 4-22, 8-28, 11-15, C-1
  exception conditions, 4-21
  exception priority, 4-24
  inexact result (precision) exception (#P), 4-23, 8-30, 11-16, C-1
  invalid operation exception (#I), 4-21, 8-26, 11-14
  invalid-operation exception (#IA), C-1
  invalid-operation exception (#IS), C-1
  invalid-operation exception (#I), C-1
  numeric overflow exception (#O), 4-22, 8-28, 11-15, C-1
  numeric underflow exception (#U), 4-23, 8-29, 11-16, C-1
  summary of, 4-20, C-1
  typical handler actions, 4-25
Floating-point format
  biased exponent, 4-14
  description of, 8-13
  exponent, 4-12
  fraction, 4-12
  indefinite, 4-5
  QNaN floating-point indefinite, 4-18
  real number system, 4-12
  sign, 4-12
  significand, 4-12
Floating-point numbers
  defined, 4-12
  encoding, 4-5
Flush-to-zero
  FTZ flag, MXCSR register, 10-4, 11-2
  mode, 10-4
FMA operation, 14-22, 14-23
FMUL instruction, 8-17
FMULP instruction, 8-17
FNOP instruction, 8-23
Fopcode compatibility mode, 8-10
FPATAN instruction, 8-20, 8-21
FPREM instruction, 8-5, 8-18, 8-21
FPREM1 instruction, 8-5, 8-18, 8-21
FPTAN instruction, 8-5
Fraction, floating-point number, 4-12
FRNDINT instruction, 8-18
FRSTOR instruction, 8-5, 8-9, 8-11, 8-23
FS register, 3-13, 3-14
FSAVE/FNSAVE instructions, 8-4, 8-5, 8-9, 8-11, 8-23
FSCALE instruction, 8-21
FSIN instruction, 8-5, 8-20
FSINCOS instruction, 8-5, 8-20
FSQRT instruction, 8-18
FST instruction, 8-16
FSTCW/FNSTCW instructions, 8-7, 8-23
FSTENV/FNSTENV instructions, 8-4, 8-9, 8-11, 8-23
FSTP instruction, 8-16
FSTSW/FNSTSW instructions, 8-4, 8-23
FSUB instruction, 8-17
FSUBP instruction, 8-17
FSUBR instruction, 8-17
FSUBRP instruction, 8-17
FTST instruction, 8-6, 8-18
FUCOM instruction, 8-18
FUCOMI instruction, 8-7, 8-18
FUCOMIP instruction, 8-7, 8-18
FUCOMP instruction, 8-18
FUCOMPP instruction, 8-6, 8-18
FXAM instruction, 8-4, 8-19
FXCH instruction, 8-16
FXRSTOR instruction, 5-15, 8-12, 10-14, 11-23
FXSAVE instruction, 5-15, 8-12, 10-14, 11-23
FXTRACT instruction, 8-18
FYL2X instruction, 8-21
FYL2XP1 instruction, 8-21

G

GDTR register, 3-4, 3-6
General purpose registers
  64-bit mode, 3-5, 3-13
  description of, 3-11
  overview of, 3-2, 3-5
  parameter passing, 6-7
  part of basic programming environment, 7-1
  using REX prefix, 3-13
General-purpose instructions
  64-bit mode, 7-1
  basic programming environment, 7-1
  data types operated on, 7-1, 7-2
  description of, 7-1
  origin of, 7-1
  programming with, 7-1
  summary of, 5-6, 7-2
GS register, 3-13, 3-14

H

HADDPD instruction, 5-25, 12-4
HADDPS instruction, 5-24, 12-4
Hardware Lock Elision (HLE), 17-2
Hexadecimal numbers, 1-8
Horizontal processing model, 12-1
HSUBPD instruction, 5-25, 12-5
HSUBPS instruction, 5-24, 12-4
HT Technology
  first processor, 2-3

implementing, 2-17
introduction, 2-16

I

IA-32 architecture
history of, 2-1
introduction to, 2-1
IA-32e mode
introduction, 2-20
segmentation, 3-22
See also: 64-bit mode, compatibility mode
IA32_MISC_ENABLE MSR, 8-10
ID (identification) flag, EFLAGS register, 3-17
IDIV instruction, 7-9
IDTR register, 3-4, 3-6
IE (invalid operation exception) flag
MXCSR register, 11-14
x87 FPU status word, 8-5, 8-26, 8-27
IEEE Standard 754, 4-4, 4-12, 8-1
IF (interrupt enable) flag
EFLAGS register, 3-17, 6-13, 20-4, A-1
IM (invalid operation exception) mask bit
MXCSR register, 11-14
x87 FPU control word, 8-7
Immediate operands, 3-20
IMUL instruction, 7-9
IN instruction, 5-10, 7-20, 20-3
INC instruction, 7-8
Indefinite
description of, 4-18, 14-18
floating-point format, 4-5, 4-14
integer, 4-4, 8-14
packed BCD integer, 4-12
QNaN floating-point, 4-18
Index (operand addressing), 3-22, 3-23, 3-24
Inexact result (precision)
exception (#P), overview, 4-23
exception (#P), SSE-SSE2 extensions, 11-16
exception (#P), x87 FPU, 8-30
on floating-point operations, 4-19
Infinity control flag, x87 FPU control word, 8-8
Infinity, floating-point format, 4-5, 4-16
INIT pin, 3-15
Input/output (see I/O)
INS instruction, 5-10, 7-20, 20-3
Instruction operands, 1-7
Instruction pointer
64-bit mode, 7-2
EIP register, 3-11, 3-18
RIP register, 3-18
RIP, EIP, IP compared, 3-10
x87 FPU, 8-9
Instruction prefixes
effect on SSE and SSE2 instructions, 11-25
REX prefix, 3-2, 3-12
Instruction set
binary arithmetic instructions, 7-8
bit scan instructions, 7-14
bit test and modify instructions, 7-14
byte-set-on-condition instructions, 7-14
cacheability control instructions, 5-20, 5-23
comparison and sign change instruction, 7-8
control transfer instructions, 7-14
data movement instructions, 7-2
decimal arithmetic instructions, 7-9
EFLAGS cross-reference, A-1
EFLAGS instructions, 7-21
exchange instructions, 7-4
FXSAVE and FXRSTOR instructions, 5-15
general-purpose instructions, 5-6
grouped by processor, 5-2
increment and decrement instructions, 7-8
instruction ordering instructions, 5-20, 5-23
I/O instructions, 5-10, 7-20
logical instructions, 7-10
MMX instructions, 5-16, 9-5
multiply and divide instructions, 7-9
processor identification instruction, 7-23
repeating string operations, 7-19
rotate instructions, 7-13
segment register instructions, 7-22
shift instructions, 7-10
SIMD instructions, introduction to, 2-14
software interrupt instructions, 7-17
SSE instructions, 5-18
SSE2 instructions, 5-20
stack manipulation instructions, 7-5
string operation instructions, 7-18
summary, 5-1
system instructions, 5-32, 5-37
test instruction, 7-14
type conversion instructions, 7-7
x87 FPU and SIMD state management instructions, 5-15
x87 FPU instructions, 5-13
INT instruction, 6-18, 7-23
Integers
description of, 4-3
indefinite, 4-4, 8-14
signed integer encodings, 4-4
signed, description of, 4-4
unsigned integer encodings, 4-3
unsigned, description of, 4-3
Intel 64 architecture
64-bit mode, 3-1
64-bit mode instructions, 5-38
address space, 3-6
compatibility mode, 3-1
data types, 4-1
executing calls, 6-1
general purpose instructions, 7-1
generations, 2-20
history of, 2-1
IA32e mode, 3-1
introduction, 2-20
memory organization, 3-6, 3-8
See also: IA-32e mode
Intel Advanced Digital Media Boost, 2-4, 2-11
Intel Advanced Smart Cache, 2-10
Intel Advanced Thermal Manager, 2-4
Intel Core 2 Extreme processor family, 2-4, 2-18
Intel Core Duo processor, 2-4, 2-18
Intel Core microarchitecture, 2-4, 2-10, 2-12, 2-13, 2-18
Intel Core Solo processor, 2-4
Intel Dynamic Power Coordination, 2-4
Intel NetBurst microarchitecture, 1-3
description of, 2-8
introduction, 2-8
Intel Pentium D processor, 2-18
Intel Pentium processor Extreme Edition, 2-18

Intel Smart Cache, 2-4
Intel Smart Memory Access, 2-4, 2-11
Intel software network link, 1-10
Intel Transactional Synchronization, 15-3, 17-1
Intel VTune Performance Analyzer
  related information, 1-10
Intel Wide Dynamic Execution, 2-4, 2-10, 2-12, 2-13
Intel Xeon processor, 1-1
  description of, 2-3
Intel Xeon processor 5100 series, 2-4, 2-18
Intel386 processor, 2-1
Intel486 processor
  history of, 2-1
Inter-privilege level call
  description of, 6-7
  operation, 6-9
Inter-privilege level return
  description of, 6-7
  operation, 6-9
Interrupt gate, 6-13
Interrupt handler, 6-12
Interrupts
  64-bit mode, 6-19
  description of, 6-12
  handler, 6-12
  implicit call to an interrupt handler
    procedure, 6-13
  implicit call to an interrupt handler task, 6-18
  implicit call to interrupt handler procedure, 6-13
  implicit call to interrupt handler task, 6-18
  in real-address mode, 6-18
  maskable, 6-13
INTn instruction, 7-17
INTO instruction, 6-18, 7-18, 7-23
Invalid arithmetic operand exception (#IA)
  description of, 8-27
  masked response to, 8-27
Invalid operation exception (#I)
  overview, 4-21
  SSE and SSE2 extensions, 11-14
  x87 FPU, 8-26
IOPL (I/O privilege level) field
  EFLAGS register, 3-17, 20-3
IRET instruction, 3-18, 6-17, 6-18, 7-15, 7-23, 20-4
I/O
  address space, 20-1
  instruction serialization, 20-5
  instructions, 5-10, 7-20, 20-3
  I/O privilege level (see IOPL)
  map base, 20-4
  permission bit map, 20-4
  ports, 3-4, 20-1, 20-2, 20-3, 20-5
  sensitive instructions, 20-3

J

J-bit, 4-12
Jcc instructions, 3-17, 3-18, 7-15
JMP instruction, 3-18, 7-15, 7-22

L

L1 (level 1) cache, 2-7, 2-9
L2 (level 2) cache, 2-7, 2-9
LAHF instruction, 3-15, 7-21
Last instruction opcode, x87 FPU, 8-10
LDDQU instruction, 5-24, 12-3
LDMXCSR instruction, 10-12, 11-24
LDS instruction, 7-23
LDTR register, 3-4, 3-6
LEA instruction, 7-23
LEAVE instruction, 6-20, 6-24, 7-21
LES instruction, 7-23
LFENCE instruction, 11-12
LGS instruction, 7-23
Linear address, 3-7
Linear address space
  defined, 3-7
  maximum size, 3-7
LOCK signal, 7-4
LODS instruction, 3-17, 7-18
Log epsilon, x87 FPU operation, 8-21
Logical address, 3-7
LOOP instructions, 7-16
LOOPcc instructions, 3-17, 7-16
LSS instruction, 7-23

M

Machine check registers, 3-4
Maskable interrupts, 6-13
Masked responses
  denormal operand exception (#D), 4-21, 8-28
  divide by zero exception (#Z), 4-22, 8-28
  inexact result (precision) exception (#P), 4-24, 8-30
  invalid arithmetic operation (#IA), 8-27
  invalid operation exception (#I), 4-21
  numeric overflow exception (#O), 4-22, 8-29
  numeric underflow exception (#U), 4-23, 8-29
  stack overflow or underflow
    exception (#IS), 8-27
MASKMOVDQU instruction, 11-12, 11-25
MASKMOVQ instruction, 10-12, 11-25
Masks, exception-flags
  MXCSR register, 10-4
  x87 FPU control word, 8-7
MAXPD instruction, 11-6
MAXPS instruction, 10-8
MAXSD instruction, 11-6
MAXSS instruction, 10-9
Memory
  flat memory model, 3-7
  management registers, 3-4
  memory type range registers (MTRRs), 3-4
  modes of operation, 3-9
  organization, 3-6, 3-7
  physical, 3-6
  real address mode memory model, 3-7, 3-8
  segmented memory model, 3-7
  virtual-8086 mode memory model, 3-7, 3-8
Memory operands
  64-bit mode, 3-21
  legacy modes, 3-21
Memory-mapped I/O, 20-2
MFENCE instruction, 11-12, 11-25
Microarchitecture
  (see Intel NetBurst microarchitecture)
  (see P6 family microarchitecture)
MINPD instruction, 11-6
MINPS instruction, 10-9

MINSD instruction, 11-7
MINSS instruction, 10-9
MMX instruction set
  arithmetic instructions, 9-6
  comparison instructions, 9-7
  conversion instructions, 9-7
  data transfer instructions, 9-6
  EMMS instruction, 9-8
  logical instructions, 9-7
  overview, 9-5
  shift instructions, 9-8
MMX registers
  description of, 9-2
  overview of, 3-2
MMX technology
  64-bit mode, 9-2
  64-bit packed SIMD data types, 4-9
  compatibility mode, 9-2
  compatibility with FPU architecture, 9-8
  data types, 9-3
  detecting MMX technology with CPUID instruction, 9-8
  effect of instruction prefixes on MMX instructions, 9-11
  exception handling in MMX code, 9-11
  IA-32e mode, 9-2
  instruction set, 5-16, 9-5
  interfacing with MMX code, 9-10
  introduction to, 9-1
  memory data formats, 9-3
  mixing MMX and floating-point instructions, 9-10
  MMX registers, 9-2
  programming environment (overview), 9-1
  register mapping, 9-11
  saturation arithmetic, 9-4
  SIMD execution environment, 9-4
  transitions between x87 FPU - MMX code, 9-9
  updating MMX technology routines using 128-bit SIMD integer instructions, 11-24
  using MMX code in a multitasking operating system environment, 9-10
  using the EMMS instruction, 9-9
  wraparound mode, 9-4
Model-specific registers (see MSRs)
Modes of operation
  64-bit mode, 3-1
  compatibility mode, 3-1
  memory models used with, 3-9
  overview, 3-1, 3-5
  protected mode, 3-1
  real address mode, 3-1
  system management mode (SMM), 3-1
MONITOR instruction, 5-25, 12-5
Moore's law, 2-20
MOV instruction, 7-3, 7-22
MOVAPD instruction, 11-5, 11-23
MOVAPS instruction, 10-7, 11-23
MOVD instruction, 9-6
MOVDDUP instruction, 5-25, 12-3
MOVDQ2Q instruction, 11-11
MOVDQA instruction, 11-11, 11-23
MOVDQU instruction, 11-11, 11-23
MOVHLPS instruction, 10-8
MOVHPD instruction, 11-6
MOVHPS instruction, 10-8
MOVLHPS instruction, 10-8
MOVLPD instruction, 11-6
MOVLPS instruction, 10-7
MOVMSKPD instruction, 11-6
MOVMSKPS instruction, 10-8
MOVNTDQ instruction, 11-12, 11-25
MOVNTI instruction, 11-12, 11-25
MOVNTPD instruction, 11-12, 11-25
MOVNTPS instruction, 10-12, 11-25
MOVNTQ instruction, 10-12, 11-25
MOVQ instruction, 9-6
MOVQ2DQ instruction, 11-11
MOVS instruction, 3-17, 7-18
MOVSD instruction, 11-6, 11-23
MOVSHDUP instruction, 5-25, 12-3
MOVSLDUP instruction, 5-25, 12-3
MOVSS instruction, 10-7, 11-23
MOVSX instruction, 7-8
MOVSXD instruction, 7-8
MOVUPD instruction, 11-6, 11-23
MOVUPS instruction, 10-6, 10-7, 11-23
MOVZX instruction, 7-8
MS-DOS compatibility mode, 8-32
MSRs, 3-4
MTRRs, 3-4
MUL instruction, 7-9
MULPD instruction, 11-6
MULPS instruction, 10-8
MULSD instruction, 11-6
MULSS instruction, 10-8
Multi-core technology, 2-18
Multi-threading capability, 2-18
MWAIT instruction, 5-25, 12-5
MXCSR register, 11-16
  denormals-are-zero (DAZ) flag, 10-5, 11-2, 11-3
  description, 10-3
  flush-to-zero flag (FTZ), 10-4
  FXSAVE and FXRSTOR instructions, 11-23
  LDMXCSR instruction, 11-24
  load and store instructions, 10-12
  RC field, 4-19
  saving on a procedure or function call, 11-23
  SIMD floating-point mask and flag bits, 10-4
  SIMD floating-point rounding control field, 10-4
  state management instructions, 5-20, 10-12
  STMXCSR instruction, 11-24
  writing to while preventing general-protection exceptions (#GP), 11-21

N

NaNs
  description of, 4-14, 4-16
  encoding of, 4-5, 4-15
  SNaNs vs. QNaNs, 4-16
Near call
  description of, 6-4
  operation, 6-4
Near pointer
  64-bit mode, 4-8
  legacy modes, 4-7
Near return operation, 6-4
NEG instruction, 7-8
NetBurst microarchitecture (see Intel NetBurst microarchitecture)
Non-arithmetic instructions, x87 FPU, 8-25
Non-number encodings, floating-point format, 4-14
Non-temporal data

caching of, 10-12
description, 10-12
temporal vs. non-temporal data, 10-12
Non-waiting instructions, x87 FPU, 8-24, 8-32
NOP instruction, 7-23
Normalized finite number, 4-5, 4-14, 4-15
NOT instruction, 7-10
Notation
bit and byte order, 1-6
exceptions, 1-9
hexadecimal and binary numbers, 1-8
instruction operands, 1-7
notational conventions, 1-6
reserved bits, 1-7
segmented addressing, 1-8
NT (nested task) flag, EFLAGS register, 3-17, A-1
Numeric overflow exception (#O)
overview, 4-22
SSE and SSE2 extensions, 11-15
x87 FPU, 8-4, 8-28
Numeric underflow exception (#U)
overview, 4-23
SSE and SSE2 extensions, 11-16
x87 FPU, 8-4, 8-29

O

OE (numeric overflow exception) flag
MXCSR register, 11-16
x87 FPU status word, 8-5, 8-29
OF (overflow) flag
EFLAGS register, 3-16, 6-18
OF (overflow) flag, EFLAGS register, A-1
Offset (operand addressing, 64-bit mode), 3-23
Offset (operand addressing), 3-22
OM (numeric overflow exception) mask bit
MXCSR register, 11-16
x87 FPU control word, 8-7, 8-29
Operand
addressing, modes, 3-19
instruction, 1-7
size attribute, 3-18
sizes, 3-9, 3-19
x87 FPU instructions, 8-15
OR instruction, 7-10
Ordering I/O, 20-5
ORPD instruction, 11-7
ORPS instruction, 10-9
OSXMMEXCPT flag
control register CR4, 11-18
OUT instruction, 5-10, 7-20, 20-3
OUTS instruction, 5-10, 7-20, 20-3
Overflow exception (#OF), 6-18
Overflow, x87 FPU stack, 8-26

P

P6 family microarchitecture
description of, 2-7
history of, 2-2
P6 family processors
description of, 1-1
history of, 2-2
P6 family microarchitecture, 2-7
PABSB instruction, 5-26, 12-7
PABSD instruction, 12-8
PABSW instruction, 5-26, 12-8
Packed
BCD integer indefinite, 4-12
BCD integers, 4-11
bytes, 9-3
doublewords, 9-3
SIMD data types, 4-9
SIMD floating-point values, 4-9
SIMD integers, 4-9
words, 9-3
PACKSSWB instruction, 9-7
PACKUSWB instruction, 9-7
PADDB instruction, 9-6
PADDD instruction, 9-6
PADDQ instruction, 11-11
PADDSB instruction, 9-7
PADDSW instruction, 9-7
PADDUSB instruction, 9-7
PADDUSW instruction, 9-7
PADDW instruction, 9-6
PALIGNR instruction, 5-27, 12-8
PAND instruction, 9-7
PANDN instruction, 9-7
Parameter passing
argument list, 6-7
on stack, 6-7
on the stack, 6-7
through general-purpose registers, 6-7
x87 FPU register stack, 8-3
XMM registers, 11-23
PAUSE instruction, 11-12
PAVGB instruction, 10-11
PC (precision) field, x87 FPU control word, 8-7
PCMPEQB instruction, 9-7
PCMPEQD instruction, 9-7
PCMPEQW instruction, 9-7
PCMPGTB instruction, 9-7
PCMPGTD instruction, 9-7
PCMPGTW instruction, 9-7
PE (inexact result exception) flag, 11-16
MXCSR register, 4-19
x87 FPU status word, 4-19, 8-4, 8-5, 8-30
Pentium 4 processor, 1-1
description of, 2-3, 2-4
Pentium 4 processor supporting Hyper-Threading Technology
description of, 2-3, 2-4
Pentium II processor, 1-3
description of, 2-2
P6 family microarchitecture, 2-7
Pentium II Xeon processor
description of, 2-2
Pentium III processor, 1-3
description of, 2-2
P6 family microarchitecture, 2-7
Pentium III Xeon processor
description of, 2-3
Pentium M processor
description of, 2-3
instructions supported, 2-3
Pentium Pro processor, 1-3
description of, 2-2
P6 family microarchitecture, 2-7
Pentium processor, 1-1
history of, 2-2

Pentium processor Extreme Edition
introduction, 2-4
Pentium processor with MMX technology, 2-2
Performance monitoring counters, 3-4
PEXTRW instruction, 10-11
PF (parity) flag, EFLAGS register, 3-16, A-1
PHADDD instruction, 5-26, 12-7
PHADDSW instruction, 5-25, 12-7
PHADDW instruction, 5-25, 12-7
PHSUBD instruction, 5-26, 12-7
PHSUBSW instruction, 5-26, 12-7
PHSUBW instruction, 5-26, 12-7
Physical
address space, 3-6
memory, 3-6
PINSRW instruction, 10-11
Pi, x87 FPU constant, 8-21
PM (inexact result exception) mask bit
MXCSR register, 11-16
x87 FPU control word, 8-7, 8-30
PMADDUBSW instruction, 5-26, 12-8
PMADDWD instruction, 9-7
PMAXSW instruction, 10-11
PMAXUB instruction, 10-11
PMINSW instruction, 10-11
PMINUB instruction, 10-11
PMOVMSKB instruction, 10-11
PMULHRSW instruction, 5-26, 12-8
PMULHUW instruction, 10-12
PMULUDQ instruction, 11-11
Pointer data types, 4-7, 4-8
Pointers
64-bit mode, 4-8
far pointer, 4-7
near pointer, 4-7
POP instruction, 6-1, 6-2, 7-6, 7-22
POPA instruction, 6-7, 7-6
POPF instruction, 3-15, 6-7, 7-21, 20-4
POPFD instruction, 3-15, 6-7, 7-21
POR instruction, 9-7
Power coordination, 2-4
PREFETCHh instructions, 10-13, 11-25
Privilege levels
description of, 6-8
inter-privilege level calls, 6-7
protection rings, 6-8
stack switching, 6-14
Procedure calls
description of, 6-4
far call, 6-4
for block-structured languages, 6-20
inter-privilege level call, 6-9
linking, 6-3
near call, 6-4
overview, 6-1
return instruction pointer (EIP register), 6-3
saving procedure state information, 6-7
stack, 6-1
stack switching, 6-8
to exception handler procedure, 6-13
to exception task, 6-18
to interrupt handler procedure, 6-13
to interrupt task, 6-18
to other privilege levels, 6-7
types of, 6-1
Processor identification
earlier Intel architecture processors, 21-1
early processors, 21-1
using CPUID instruction, 21-1
Processor state information, saving, 6-7
Protected mode
I/O, 20-3
memory models used, 3-9
overview, 3-1
Protection rings, 6-8
PSADBW instruction, 10-12
PSHUFB instruction, 5-26, 12-8
PSHUFD instruction, 11-11
PSHUFHW instruction, 11-11
PSHUFLW instruction, 11-11
PSHUFW instruction, 10-12, 11-11
PSIGNB/W/D instruction, 5-26, 12-8
PSLLD instruction, 9-8
PSLLDQ instruction, 11-11
PSLLQ instruction, 9-8
PSLLW instruction, 9-8
PSRLDQ instruction, 11-11
PSUBB instruction, 9-6
PSUBD instruction, 9-6
PSUBQ instruction, 11-11
PSUBSB instruction, 9-7
PSUBSW instruction, 9-7
PSUBUSB instruction, 9-7
PSUBUSW instruction, 9-7
PSUBW instruction, 9-6
PUNPCKHBW instruction, 9-7
PUNPCKHDQ instruction, 9-7
PUNPCKHQDQ instruction, 11-11
PUNPCKHWD instruction, 9-7
PUNPCKLBW instruction, 9-7
PUNPCKLDQ instruction, 9-7
PUNPCKLQDQ instruction, 11-11
PUNPCKLWD instruction, 9-7
PUSH instruction, 6-1, 6-2, 7-5, 7-22
PUSHA instruction, 6-7, 7-5
PUSHF instruction, 3-15, 6-7, 7-21
PUSHFD instruction, 3-15, 6-7, 7-21
PXOR instruction, 9-7

Q

QNaN floating-point indefinite, 4-5, 4-18, 8-14
QNaNs
description of, 4-16
effect on COMISD and UCOMISD, 11-7
encodings, 4-5
operating on, 4-17
rules for generating, 4-17
using in applications, 4-17
Quadword, 4-1, 9-3
Quiet NaN (see QNaN)

R

R8D-R15D registers, 3-12
R8-R15 registers, 3-12
RAX register, 3-12
RBP register, 3-12, 6-4
RBX register, 3-12
RC (rounding control) field

MXCSR register, 4-19, 10-4
x87 FPU control word, 4-19, 8-8
RCL instruction, 7-13
RCPPS instruction, 10-8
RCPSS instruction, 10-8
RCR instruction, 7-13
RCX register, 3-12
RDI register, 3-12
RDRAND, 7-24
RDX register, 3-12
Real address mode
handling exceptions in, 6-18
handling interrupts in, 6-18
memory model, 3-7, 3-8
memory model used, 3-9
not in 64-bit mode, 3-9
overview, 3-1
Real numbers
continuum, 4-12
encoding, 4-14, 4-15
notation, 4-13, 14-18
system, 4-12
Register operands
64-bit mode, 3-20
legacy modes, 3-20
Register stack, x87 FPU, 8-1
Registers
64-bit mode, 3-12, 3-15
control registers, 3-4
CR in 64-bit mode, 3-5
debug registers, 3-4
EFLAGS register, 3-11, 3-15
EIP register, 3-11, 3-18
general purpose registers, 3-11
instruction pointer, 3-11
machine check registers, 3-4
memory management registers, 3-4
MMX registers, 3-2, 9-2
MSRs, 3-4
MTRRs, 3-4
MXCSR register, 10-4
performance monitoring counters, 3-4
REX prefix, 3-12
segment registers, 3-11, 3-13
x87 FPU registers, 8-1
XMM registers, 3-2, 10-3
Related literature, 1-9
REP/REPE/REPZ/REPNE/REPNZ
prefixes, 7-19, 20-3
Reserved bits, 1-7
RESET pin, 3-15
RET instruction, 3-18, 6-3, 6-4, 7-15, 7-22
Return instruction pointer, 6-3
Returns, from procedure calls
exception handler, return from, 6-13
far return, 6-5
inter-privilege level return, 6-9
interrupt handler, return from, 6-13
near return, 6-4
REX prefixes, 3-2, 3-12, 3-19
RF (resume) flag, EFLAGS register, 3-17, A-1
RFLAGS, 3-18
RFLAGS register, 7-22
See EFLAGS register
RIP register, 6-4
64-bit mode, 7-2
description of, 3-18
relation to EIP, 7-2
ROL instruction, 7-13
ROR instruction, 7-13
Rounding
modes, floating-point operations, 4-19
modes, x87 FPU, 8-8
toward zero (truncation), 4-19
Rounding control (RC) field
MXCSR register, 4-19, 10-4
x87 FPU control word, 4-19, 8-8
RSI register, 3-12
RSP register, 3-12, 6-4
RSQRTPS instruction, 10-8
RSQRTSS instruction, 10-8

S

SAHF instruction, 3-15, 7-21
SAL instruction, 7-10
SAR instruction, 7-11
Saturation arithmetic (MMX instructions), 9-4
SBB instruction, 7-8
Scalar operations
defined, 10-7, 11-5
scalar double precision FP operands, 11-5
scalar single precision FP operands, 10-7
Scale (operand addressing), 3-22, 3-23, 3-24
Scale, x87 FPU operation, 8-21
Scaling bias value, 8-29, 8-30
SCAS instruction, 3-17, 7-18
Segment
defined, 3-7
maximum number, 3-7
Segment override prefixes, 3-21
Segment registers
64-bit mode, 3-15, 3-22, 7-2
default usage rules, 3-21
description of, 3-11, 3-13
part of basic programming environment, 7-1
Segment selector
description of, 3-7, 3-13
segment override prefixes, 3-21
specifying, 3-21
Segmented memory model, 1-8, 3-7, 3-13
Serialization of I/O instructions, 20-5
Serializing instructions, 20-5
SETcc instructions, 3-17, 7-14
SF (sign) flag, EFLAGS register, 3-16, A-1
SF (stack fault) flag, x87 FPU status word, 8-6, 8-26
SFENCE instruction, 10-14, 11-12, 11-25
SHL instruction, 7-10
SHLD instruction, 7-12
SHR instruction, 7-11
SHRD instruction, 7-12
Shuffle instructions
SSE extensions, 10-9
SSE2 extensions, 11-7
SHUFPD instruction, 11-7
SI register, 3-12
Signaling NaN (see SNaN)
Signed
infinity, 4-16
integers, description of, 4-4

integers, encodings, 4-4
zero, 4-15
Significand, of floating-point number, 4-12
Sign, floating-point number, 4-12
SIMD floating-point exception (#XM), 11-18
SIMD floating-point exceptions
denormal operand exception (#D), 11-15
divide-by-zero (#Z), 11-15
exception conditions, 11-14
exception handlers, D-1
inexact result exception (#P), 11-16
invalid operation exception (#I), 11-14
list of, 11-13
numeric overflow exception (#O), 11-15
numeric underflow exception (#U), 11-16
precision exception (#P), 11-16
software handling, 11-18
summary of, C-1
writing exception handlers for, D-1
SIMD floating-point flag bits, 10-4
SIMD floating-point mask bits, 10-4
SIMD floating-point rounding control field, 10-4
SIMD (single instruction, multiple-data)
operations, on packed double precision floating-point operands , 11-4
SIMD (single-instruction, multiple-data)
execution model, 2-2, 9-4
instructions, 2-14, 5-20, 10-7
MMX instructions, 5-16
operations, on packed single precision floating-point operands, 10-6
packed data types, 4-9
SSE instructions, 5-18
SSE2 instructions, 11-4, 12-2, 12-6
Sine, x87 FPU operation, 8-20
Single precision floating-point format, 4-4
Sleep, 2-4
Smart cache, 2-4
Smart memory access, 2-11
smart memory access, 2-4
SMM
memory model used, 3-9
overview, 3-1
SNaNs
description of, 4-16
effect on COMISD and UCOMISD, 11-7
encodings, 4-5
operating on, 4-17
typical uses of, 4-16
using in applications, 4-17
Software compatibility, 1-7
SP register, 3-12
Speculative execution, 2-7, 2-9
Spin-wait loops
programming with PAUSE instruction, 11-12
SQRTPD instruction, 11-6
SQRTPS instruction, 10-8
SQRTSD instruction, 11-6
SQRTSS instruction, 10-8
SS register, 3-13, 3-14, 6-1
SSE extensions
128-bit packed single precision data type, 10-5
64-bit mode, 10-3
64-bit SIMD integer instructions, 10-11
branching on arithmetic operations, 11-24
cacheability control instructions, 10-12
cacheability hint instructions, 11-25
caller-save requirement for procedure and function calls, 11-24
checking for SSE and SSE2 support, 11-19
comparison instructions, 10-9
compatibility mode, 10-3
compatibility of SIMD and x87 FPU floating-point data types, 11-22
conversion instructions, 10-11
data movement instructions, 10-7
data types, 10-5, 12-1
denormal operand exception (#D), 11-15
denormals-are-zeros mode, 10-5
divide by zero exception (#Z), 11-15
exceptions, 11-13
floating-point format, 4-12
flush-to-zero mode, 10-4
generating SIMD FP exceptions, 11-16
handling combinations of masked and unmasked exceptions, 11-18
handling masked exceptions, 11-17
handling SIMD floating-point exceptions in software, 11-18
handling unmasked exceptions, 11-18
inexact result exception (#P), 11-16
instruction prefixes, effect on SSE and SSE2 instructions, 11-25
instruction set, 5-18, 10-6
interaction of SIMD and x87 FPU floating-point exceptions, 11-18
interaction of SSE and SSE2 instructions with x87 FPU and MMX instructions, 11-22
interfacing with SSE and SSE2 procedures and functions, 11-23
intermixing packed and scalar floating-point and 128-bit SIMD integer instructions and data, 11-22
introduction, 2-2
invalid operation exception (#I), 11-14
logical instructions, 10-9
masked responses to invalid arithmetic operations, 11-14
memory ordering instruction, 10-14
MMX technology compatibility, 10-5
MXCSR register, 10-3
MXCSR state management instructions, 10-12
non-temporal data, operating on, 10-12
numeric overflow exception (#O), 11-15
numeric underflow exception (#U), 11-16
packed 128-Bit SIMD data types, 4-9
packed and scalar floating-point instructions, 10-6
programming environment, 10-2
QNaN floating-point indefinite, 4-18
restoring SSE and SSE2 state, 11-21
REX prefixes, 10-3
saving SSE and SSE2 state, 11-21
saving XMM register state on a procedure or function call, 11-23
shuffle instructions, 10-9
SIMD floating-point exception conditions, 11-14
SIMD floating-point exception cross reference, C-3
SIMD floating-point exception (#XM), 11-18
SIMD floating-point exceptions, 11-13
SIMD floating-point mask and flag bits, 10-4
SIMD floating-point rounding control field, 10-4
SSE and SSE2 conversion instruction chart, 11-9
SSE feature flag, CPUID instruction, 11-20
SSE2 compatibility, 10-5
system programming, 13-23

unpack instructions, 10-9
updating MMX technology routines
using128-bit SIMD integer instructions, 11-24
x87 FPU compatibility, 10-5
XMM registers, 10-3
SSE feature flag, CPUID instruction, 11-20, 12-5
SSE instructions
descriptions of, 10-6
SIMD floating-point exception cross-reference, C-3
summary of, 5-18
SSE2 extensions
128-bit packed single precision
data type, 11-3
128-bit packed single precision data type, 12-1
128-bit SIMD integer instruction
extensions, 11-11
64-bit and 128-bit SIMD integer instructions, 11-11
64-bit mode, 11-3
arithmetic instructions, 11-6
branch hints, 11-13
branching on arithmetic operations, 11-24
cacheability control instructions, 11-12
cacheability hint instructions, 11-25
caller-save requirement for procedure and function calls, 11-24
checking for SSE and SSE2 support, 11-19
comparison instructions, 11-7
compatibility mode, 11-3
compatibility of SIMD and x87 FPU floating-point data types, 11-22
conversion instructions, 11-9
data movement instructions, 11-5
data types, 11-3, 12-1
denormal operand exception (#D), 11-15
denormals-are-zero mode, 11-3
divide by zero exception (#Z), 11-15
exceptions, 11-13
floating-point format, 4-12
generating SIMD floating-point exceptions, 11-16
handling combinations of masked and unmasked exceptions, 11-18
handling masked exceptions, 11-17
handling SIMD floating-point exceptions in software, 11-18
handling unmasked exceptions, 11-18
inexact result exception (#P), 11-16
instruction prefixes, effect on SSE and SSE2 instructions, 11-25
instruction set, 5-20
instructions, 11-4, 12-2, 12-6
interaction of SIMD and x87 FPU floating-point exceptions, 11-18
interaction of SSE and SSE2 instructions with x87 FPU and MMX instructions, 11-22
interfacing with SSE and SSE2 procedures and functions, 11-23
intermixing packed and scalar floating-point and 128-bit SIMD integer instructions and data, 11-22
invalid operation exception (#I), 11-14
logical instructions, 11-7
masked responses to invalid arithmetic operations, 11-14
memory ordering instructions, 11-12
MMX technology compatibility, 11-3
numeric overflow exception (#O), 11-15
numeric underflow exception (#U), 11-16
packed 128-Bit SIMD data types, 4-9
packed and scalar floating-point instructions, 11-4
programming environment, 11-2
QNaN floating-point indefinite, 4-18
restoring SSE and SSE2 state, 11-21
REX prefixes, 11-3
saving SSE and SSE2 state, 11-21
saving XMM register state on a procedure or function call, 11-23
shuffle instructions, 11-7
SIMD floating-point exception conditions, 11-14
SIMD floating-point exception cross reference, C-5
SIMD floating-point exception (#XM), 11-18
SIMD floating-point exceptions, 11-13
SSE and SSE2 conversion instruction chart, 11-9
SSE compatibility, 11-3
SSE2 feature flag, CPUID instruction, 11-20
system programming, 13-23
unpack instructions, 11-7
updating MMX technology routines using 128-bit SIMD integer instructions, 11-24
x87 FPU compatibility, 11-3
SSE2 feature flag, CPUID instruction, 11-20, 12-5
SSE2 instructions
descriptions of, 11-4, 12-2, 12-6
SIMD floating-point exception cross-reference, C-5
summary of, 5-20
SSE3 extensions
64-bit mode, 12-1
asymmetric processing, 12-1
compatibility mode, 12-1
DNA exceptions, 12-9
emulation, 12-10
enabling support in a system executive, 12-5, 12-18
exceptions, 12-9
guideline for packed addition/subtraction instructions, 12-6
horizontal addition/subtraction instructions, 12-4
horizontal processing, 12-1
instruction that addresses cache line splits, 5-24
instruction that improves X87-FP integer conversion, 5-24
instructions for horizontal addition/subtraction, 5-24
instructions for packed addition/subtraction, 5-24
instructions that enhance LOAD/MOVE/DUPLICATE, 5-25
instructions that improve synchronization between agents, 5-25
LOAD/MOVE/DUPLICATE enhancement instructions, 12-3
MMX technology compatibility, 12-1
numeric error flag and IGNNE#, 12-9
packed addition/subtraction instructions, 12-4
programming environment, 12-1
REX prefixes, 12-1
SIMD floating-point exception cross reference, C-7, C-8
specialized 120-bit load instruction, 12-3
SSE compatibility, 12-1
SSE2 compatibility, 12-1
system programming, 13-23
x87 FPU compatibility, 12-1
SSE3 instructions
descriptions of, 12-2
SIMD floating-point exception
cross-reference, C-7, C-8
summary of, 5-24
SSSE3 extensions
64-bit mode, 12-1
asymmetric processing, 12-1
checking for support, 12-9
compatibility mode, 12-1
data types, 12-1
DNA exceptions, 12-9

emulation, 12-10
enabling support in a system executive, 12-9
exceptions, 12-9
horizontal add/subtract instructions, 12-7
horizontal processing, 12-1
multiply and add packed instructions, 12-8
numeric error flag and IGNNE#, 12-9
packed absolute value instructions, 12-7
packed align instruction, 12-8
packed multiply high instructions, 12-8
packed shuffle instruction, 12-8
programming environment, 12-1
SSSE3 instructions
descriptions of, 12-6
summary of, 5-25
Stack
64-bit mode, 3-5, 6-4
64-bit mode behavior, 6-19
address-size attribute, 6-3
alignment, 6-2
alignment of stack pointer, 6-2
current stack, 6-1, 6-3
description of, 6-1
EIP register (return instruction pointer), 6-3
maximum size, 6-1
number allowed, 6-1
overview of, 3-4
passing parameters on, 6-7
popping values from, 6-1
procedure linking information, 6-3
pushing values on, 6-1
return instruction pointer, 6-3
SS register, 6-1
stack segment, 3-14, 6-1
stack-frame base pointer, EBP register, 6-3
switching
on calls to interrupt and exception handlers, 6-14
on inter-privilege level calls, 6-10, 6-17
privilege levels, 6-8
width, 6-2
Stack, x87 FPU
stack fault, 8-6
stack overflow and underflow exception (#IS), 8-4, 8-26
Status flags
EFLAGS register, 3-16, 8-6, 8-7, 8-19
STC instruction, 3-16, 7-21
STD instruction, 3-17, 7-21
STI instruction, 7-22, 20-4
Sticky bits, 8-5
STMXCSR instruction, 10-12, 11-24
STOS instruction, 3-17, 7-19
Streaming SIMD extensions 2 (see SSE2 extensions)
Streaming SIMD extensions (see SSE extensions)
String data type, 4-9
ST(0), top-of-stack register, 8-3
SUB instruction, 7-8
Superscalar microarchitecture
P6 family microarchitecture, 2-2
P6 family processors, 2-7
Pentium 4 processor, 2-9
Pentium Pro processor, 2-2
Pentium processor, 2-2
System management mode (see SMM)
System programming
SSE/SSE2/SSE3 extensions, 13-23

T

Tangent, x87 FPU operation, 8-20
Task gate, 6-18
Task register, 3-4
Task state segment (see TSS)
Tasks
exception handler, 6-18
interrupt handler, 6-18
Temporal data, 10-12
TEST instruction, 7-14
TF (trap) flag, EFLAGS register, 3-17, A-1
Thermal Monitor, 2-4
TOP (stack TOP) field
x87 FPU status word, 8-2, 9-9
TR register, 3-6
Trace cache, 2-9
Transcendental instruction accuracy, 8-21
Trap gate, 6-13
Truncation
description of, 4-19
with SSE-SSE2 conversion instructions, 4-19
TSS
I/O map base, 20-4
I/O permission bit map, 20-4
saving state of EFLAGS register, 3-15

U

UCOMISD instruction, 11-7
UCOMISS instruction, 10-9
UD2 instruction, 7-24
UE (numeric underflow exception) flag
MXCSR register, 11-16
x87 FPU status word, 8-5, 8-29
UM (numeric underflow exception) mask bit
MXCSR register, 11-16
x87 FPU control word, 8-7, 8-29
Underflow
FPU exception
(see Numeric underflow exception)
numeric, floating-point, 4-15
x87 FPU stack, 8-26
Underflow, x87 FPU stack, 8-26
Unpack instructions
SSE extensions, 10-9
SSE2 extensions, 11-7
UNPCKHPD instruction, 11-8
UNPCKHPS instruction, 10-10
UNPCKLPD instruction, 11-8
UNPCKLPS instruction, 10-10
Unsigned integers
description of, 4-3
range of, 4-3
types, 4-3
Unsupported, 8-14
floating-point formats, x87 FPU, 8-14
x87 FPU instructions, 8-24

V

VIF (virtual interrupt) flag, EFLAGS register, 3-17
VIP (virtual interrupt pending) flag
EFLAGS register, 3-17
Virtual 8086 mode
description of, 3-17

memory model, 3-7, 3-8
VM (virtual 8086 mode) flag, EFLAGS register, 3-17
VMCALL instruction, 5-39, 5-40
VMCLEAR instruction, 5-39, 5-40
VMLAUNCH instruction, 5-39, 5-40
VMPTRLD instruction, 5-39, 5-40
VMPTRST instruction, 5-39, 5-40
VMREAD instruction, 5-39, 5-40
VMRESUME instruction, 5-39, 5-40
VMWRITE instruction, 5-39, 5-40
VMX
instruction set, 5-39
introduction, 2-20
Virtual machine monitor (VMM), 2-20
virtualization, 2-20
VMXOFF instruction, 5-39
VMXON instruction, 5-39

W

Waiting instructions, x87 FPU, 8-24
WAIT/FWAIT instructions, 8-23, 8-31
WC memory type, 10-12
wide dynamic execution, 2-4
Word, 4-1
Wraparound mode (MMX instructions), 9-4

X

x87 FPU
64-bit mode, 8-1
compatibility mode, 8-1
control word, 8-7
data pointer, 8-9
data registers, 8-1
execution environment, 8-1
floating-point data types, 8-13
floating-point format, 4-12
fopcode compatibility mode, 8-10
FXSAVE and FXRSTOR instructions, 11-23
IEEE Standard 754, 8-1
instruction pointer, 8-9
instruction set, 8-15
last instruction opcode, 8-10
overview of registers, 3-2
programming, 8-1
QNaN floating-point indefinite, 4-18
register stack, 8-1
register stack, parameter passing, 8-3
registers, 8-1
save and restore state instructions, 5-15
saving registers, 11-23
state, 8-11
state, image, 8-11, 8-12
state, saving, 8-11, 8-12
status register, 8-4
tag word, 8-8
transcendental instruction accuracy, 8-21
x87 FPU control word
description of, 8-7
exception-flag mask bits, 8-7
infinity control flag, 8-8
precision control (PC) field, 8-7
rounding control (RC) field, 4-19, 8-8
x87 FPU exception handling
description of, 8-32
floating-point exception summary, C-1
MS-DOS compatibility mode, 8-32
native mode, 8-32
x87 FPU floating-point exceptions
denormal operand exception, 8-28
division-by-zero, 8-28
exception conditions, 8-26
exception summary, C-1
inexact-result (precision), 8-30
interaction of SIMD and x87 FPU floating-point exceptions, 11-18
invalid arithmetic operand, 8-26, 8-27
numeric overflow, 8-28
numeric underflow, 8-29
software handling, 8-32
stack overflow, 8-4, 8-26
stack underflow, 8-4, 8-26
summary of, 8-24
synchronization, 8-31
x87 FPU instructions
arithmetic vs. non-arithmetic instructions, 8-25
basic arithmetic, 8-17
comparison and classification, 8-18
control, 8-23
data transfer, 8-15
exponential, 8-21
instruction set, 8-15
load constant, 8-17
logarithmic, 8-21
operands, 8-15
overview, 8-15
save and restore state, 8-23
scale, 8-21
transcendental, 8-21
transitions between x87 FPU and MMX code, 9-9
trigonometric, 8-20
unsupported, 8-24
x87 FPU status word
condition code flags, 8-4
DE flag, 8-28
description of, 8-4
exception flags, 8-5
OE flag, 8-28
PE flag, 8-4
stack fault flag, 8-6
TOP field, 8-2
top of stack (TOP) pointer, 8-4
x87 FPU tag word, 8-8, 9-9
XADD instruction, 7-4
XCHG instruction, 7-4
XCR0, 14-15
XLAT/XLATB instruction, 7-23
XMM registers
64-bit mode, 3-5
description, 10-3
FXSAVE and FXRSTOR instructions, 11-23
overview of, 3-2
parameters passing in, 11-23
saving on a procedure or function call, 11-23
XOR instruction, 7-10
XORPD instruction, 11-7
XORPS instruction, 10-9
XRSTOR, 14-15, 15-1, 15-4
XSAVE, 14-15, 14-20, 14-25, 14-31, 14-32, 15-1, 15-4, 15-8

Z

ZE (divide by zero exception) flag
    x87 FPU status word, 8-5, 8-28
ZE (divide by zero exception) flag bit
    MXCSR register, 11-15
Zero, floating-point format, 4-5, 4-15
ZF (zero) flag, EFLAGS register, 3-16, A-1
ZM (divide by zero exception) mask bit
    MXCSR register, 11-15
    x87 FPU control word, 8-7, 8-28